

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME III OF EIGHT

Order and Access

DAYS 051 - 077

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

27 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME III · ORDER AND ACCESS

DAYS 51 TO 77

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume III of eight: **Order and Access**. Days 51 to 77.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
Part VII	Sorting	Days 51–59
Part VIII	Hashing: maps and sets	Days 60–67
Part IX	Stacks and queues	Days 68–77

PART X	Linked lists	Days 78–86
PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
PART XVI	Dynamic programming	Days 143–163
PART XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART VII · SORTING	1
051 Why sorting matters more than any single sorting algorithm	2
DSA Why sorting matters more than any single sorting algorithm	3
DESIGN Modelling a real domain	19
PRACTICPractice — Why sorting matters more than any single sorting algorithm	33
052 Bubble, selection and insertion sort, and what each one teaches	37
DSA Bubble, selection and insertion sort, and what each one teaches	38
DESIGN Common object-oriented interview questions	55
PRACTICPractice — Bubble, selection and insertion sort, and what each one teaches	71
053 Merge sort	75
DSA Merge sort	76
DESIGN Writing clean, testable classes	94
PRACTICPractice — Merge sort	109
054 Quicksort and partitioning	113
DSA Quicksort and partitioning	114
DESIGN Object-oriented design revision and interview questions	132
PRACTICPractice — Quicksort and partitioning	149
055 Quickselect: finding the Kth largest without sorting	154
DSA Quickselect: finding the Kth largest without sorting	155
DESIGN Single responsibility	172
PRACTICPractice — Quickselect: finding the Kth largest without sorting	188
056 Counting sort, radix sort, and bucket sort	193
DSA Counting sort, radix sort, and bucket sort	194
DESIGN Open for extension, closed for modification	214
PRACTICPractice — Counting sort, radix sort, and bucket sort	232
057 Stability, and what Python's sort actually does	237
DSA Stability, and what Python's sort actually does	238
DESIGN Liskov substitution	256
PRACTICPractice — Stability, and what Python's sort actually does	272
058 Custom comparators and sorting by keys	277
DSA Custom comparators and sorting by keys	278
DESIGN Interface segregation	295
PRACTICPractice — Custom comparators and sorting by keys	311
059 Sorting revision and mock round	316
DSA Sorting revision and mock round	317
DESIGN Dependency inversion	335

PRACTICPractice — Sorting revision and mock round	352
PART VIII · HASHING: MAPS AND SETS	357
060 Hash tables: how a dictionary finds anything instantly	358
DSA Hash tables: how a dictionary finds anything instantly	359
DESIGN DRY, KISS, and YAGNI	377
PRACTICPractice — Hash tables: how a dictionary finds anything instantly	393
061 Collisions, and why a hash map can turn slow	398
DSA Collisions, and why a hash map can turn slow	399
DESIGN Coupling, cohesion, and code smells	420
PRACTICPractice — Collisions, and why a hash map can turn slow	437
062 Sets: membership, deduplication, and the O(1) habit	443
DSA Sets: membership, deduplication, and the O(1) habit	444
DESIGN Design principles revision and interview questions	459
PRACTICPractice — Sets: membership, deduplication, and the O(1) habit	475
063 Counting with dictionaries	480
DSA Counting with dictionaries	481
DESIGN What a design pattern actually is	495
PRACTICPractice — Counting with dictionaries	507
064 Grouping: the key-design skill	512
DSA Grouping: the key-design skill	513
DESIGN Singleton	528
PRACTICPractice — Grouping: the key-design skill	541
065 Hashing your own objects	546
DSA Hashing your own objects	547
DESIGN Factory and abstract factory	563
PRACTICPractice — Hashing your own objects	577
066 When a hash map is the wrong answer	582
DSA When a hash map is the wrong answer	583
DESIGN Builder	597
PRACTICPractice — When a hash map is the wrong answer	611
067 Hashing revision and mock round	616
DSA Hashing revision and mock round	617
DESIGN Prototype, and cloning objects	633
PRACTICPractice — Hashing revision and mock round	646
PART IX · STACKS AND QUEUES	651
068 Stacks: last in, first out	652
DSA Stacks: last in, first out	653
DESIGN Adapter	668
PRACTICPractice — Stacks: last in, first out	681

069	Balanced brackets and the parsing family	686
DSA	Balanced brackets and the parsing family	687
DESIGN	Decorator	701
PRACTIC	Practice — Balanced brackets and the parsing family	714
070	Min stack, and stacks that carry extra state	719
DSA	Min stack, and stacks that carry extra state	720
DESIGN	Facade and proxy	734
PRACTIC	Practice — Min stack, and stacks that carry extra state	746
071	Monotonic stack: the next greater element	751
DSA	Monotonic stack: the next greater element	752
DESIGN	Strategy	766
PRACTIC	Practice — Monotonic stack: the next greater element	778
072	Largest rectangle in a histogram	783
DSA	Largest rectangle in a histogram	784
DESIGN	Observer	800
PRACTIC	Practice — Largest rectangle in a histogram	815
073	Queues: first in, first out	820
DSA	Queues: first in, first out	821
DESIGN	State	836
PRACTIC	Practice — Queues: first in, first out	851
074	Deque and the sliding-window maximum	856
DSA	Deque and the sliding-window maximum	857
DESIGN	Command and chain of responsibility	872
PRACTIC	Practice — Deque and the sliding-window maximum	887
075	A queue from two stacks, and a stack from queues	892
DSA	A queue from two stacks, and a stack from queues	893
DESIGN	Template method and iterator	909
PRACTIC	Practice — A queue from two stacks, and a stack from queues	925
076	LRU cache: the structure interviewers love	930
DSA	LRU cache: the structure interviewers love	931
DESIGN	Design patterns revision and interview questions	948
PRACTIC	Practice — LRU cache: the structure interviewers love	962
077	Stacks and queues revision and mock round	967
DSA	Stacks and queues revision and mock round	968
DESIGN	How to run a low-level design interview: the forty-minute script	983
PRACTIC	Practice — Stacks and queues revision and mock round	998
BACK MATTER		
A	Appendix A - The nine sections, and what each is for	1004
B	Appendix B - Recall cards	1006
C	Appendix C - The complete 180-day map	1036

D	Appendix D - Glossary	1038
E	Appendix E - References and further reading, by section	1055
F	Appendix F - Colophon: how this book was made	1057
G	Appendix G - Problem index	1058
H	Index	1065

List of figures

51.1	Data structures and algorithms - Why sorting matters more than any single sorting algorithm	2
51.2	System design - Modelling a real domain	2
52.1	System design - Common object-oriented interview questions	37
52.2	System design - Common object-oriented interview questions	37
53.1	System design - Writing clean, testable classes	75
54.1	System design - Object-oriented design revision and interview questions	113
54.2	System design - Object-oriented design revision and interview questions	113
55.1	System design - Single responsibility	154
55.2	System design - Single responsibility	154
56.1	System design - Open for extension, closed for modification	193
56.2	System design - Open for extension, closed for modification	193
57.1	System design - Liskov substitution	237
57.2	System design - Liskov substitution	237
58.1	System design - Interface segregation	277
58.2	System design - Interface segregation	277
59.1	Data structures and algorithms - Sorting revision and mock round	316
59.2	System design - Dependency inversion	316
60.1	System design - DRY, KISS, and YAGNI	358
62.1	System design - Design principles revision and interview questions	443
63.1	System design - What a design pattern actually is	480
64.1	System design - Singleton	512
65.1	System design - Factory and abstract factory	546
66.1	System design - Builder	582
67.1	Data structures and algorithms - Hashing revision and mock round	616
67.2	System design - Prototype, and cloning objects	616
68.1	System design - Adapter	652
69.1	System design - Decorator	686
70.1	System design - Facade and proxy	719
71.1	System design - Strategy	751
72.1	System design - Observer	783
72.2	System design - Observer	783
73.1	System design - State	820
73.2	System design - State	820

74.1	System design - Command and chain of responsibility	856
74.2	System design - Command and chain of responsibility	856
75.1	System design - Template method and iterator	892
75.2	System design - Template method and iterator	892
76.1	System design - Design patterns revision and interview questions	930
77.1	System design - How to run a low-level design interview: the forty-minute script	967
77.2	System design - How to run a low-level design interview: the forty-minute script	967

PART VII

Sorting

Days 51 to 59 · 9 chapters

Alongside, on the system design track:

Object-oriented design, SOLID and design principles

051 Why sorting matters more than any single sorting algorithm

052 Bubble, selection and insertion sort, and what each one teaches

053 Merge sort

054 Quicksort and partitioning

055 Quickselect: finding the Kth largest without sorting

056 Counting sort, radix sort, and bucket sort

057 Stability, and what Python's sort actually does

058 Custom comparators and sorting by keys

059 Sorting revision and mock round

Why sorting matters more than any single sorting algorithm

DATA STRUCTURES AND ALGORITHMS

Why sorting matters more than any single sorting algorithm

You reach for sort as a preprocessing step and can justify the extra $n \log n$.

SYSTEM DESIGN

Modelling a real domain

You can turn a page of product requirements into objects, fields and relationships.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Would sorting first make this problem easier?*
- *Here are the requirements. Model it.*

Why sorting matters more than any single sorting algorithm

1 What this is, and why they ask it

This opens the sorting phase, and it deliberately does not teach a sorting algorithm. Merge sort, quicksort and the rest arrive over the next eight days. Today is about the thing that actually shows up in interviews: **sorting as a first move**. You pay $O(n \log n)$ once, and in exchange the problem in front of you often collapses from quadratic to linear, because sorted data puts related things next to each other and lets you stop early.

They ask “would sorting first help?” because it is the highest-value habit in the whole subject and most candidates do not have it. A candidate without it writes a nested loop for “find the two closest values” and delivers $O(n^2)$. A candidate with it says “sort, then compare neighbours” and delivers $O(n \log n)$ in three lines. The habit is also what makes the intervals family, most of the greedy family, and half the two-pointer problems from [day 027](#) tractable at all. And the reason it is worth a whole day is the other half: knowing the four situations where sorting is the wrong move, and being able to say why.

2 The story

Sivadasan runs the collection point for mangoes at the edge of a village near Salem, and from the end of April he is there from five in the morning.

Fifteen or twenty growers come through in a day, each with two or three crates, and the fruit arrives exactly as it came off the trees — a big one, a small one, a middling one, whatever the picker’s hand found. Nobody grades on the farm.

The first thing Sivadasan does with every lot, before anything else at all, is put it over the grading table. It is a long sloped table with slats, and two boys work it, and at the end of it the whole lot is laid out along the bench in size order, smallest at the near end, largest at the far end.

It takes about eleven minutes for a hundred and fifty kilos, and it is eleven minutes he never argues about, because of what happens afterwards.

The price bands are cuts along the bench. Everything past that point is export grade, everything between these two marks is the middle rate, everything at the near end goes to the pulp factory. He does not have to look at any individual fruit to make those calls; the line has already made them.

Finding the biggest one for the display crate is walking to the far end.

The boxes for the city need six of nearly the same size in each. That used to mean picking one up, then hunting through the whole lot for a match. Now the matches are standing next to each other — he takes six in a row and they are as close in size as anything in that lot could be, because anything closer would have to be adjacent too.

And when a grower turns up late with one extra crate, the boys do not re-grade the whole bench. They take a fruit, hold it against the middle of the bench, and go left or right from there.

One thing he learnt the hard way, in his first season. Once the fruit is on the bench it is in size order and not in grower order, and if he does not know whose is whose he cannot pay anybody. So every crate goes over the table with a small numbered tag, and the tag stays with the fruit through the grading and comes off at the far end. The size order is what he wants; the grower's number is what he cannot afford to lose. He arranged for both rather than choosing.

3 The idea in plain English

Sivadasan's grading table is `sort()`. The eleven minutes are the $O(n \log n)$. The price cuts, the far end, the neighbouring matches and the late crate are the four things sorted data gives you. And the numbered tag is what you do when sorting would destroy something you need.

The four things sorting buys

Learn these as four separate payoffs, because problems announce which one they want.

One: equal things become adjacent. Duplicates, groups, and counts stop needing a search.

```
nums = [4, 1, 4, 9, 1, 4]
nums.sort()           # [1, 1, 4, 4, 4, 9]
# now "are there duplicates?" is one pass comparing each element with the one before it
```

Two: near things become adjacent. This is the one people miss, and it is the most powerful.

If the array is sorted, the two closest values are next to each other.

That single sentence turns a quadratic problem into a linear one. Finding the minimum difference between any two values needs $n \times (n-1) / 2$ comparisons on unsorted data — about 500 billion at a million elements. Sorted, it is one pass over neighbours.

Three: you can stop early, and you can binary search. Once sorted, everything from the whole of the [binary search phase](#) is available: $O(\log n)$ lookups, insertion points, counting values in a range. And a scan can quit the moment values get too large.

Four: order becomes a decision you can act on greedily. Intervals sorted by start time can be merged in one pass. Meetings sorted by end time can be packed greedily. Almost every problem in the greedy and intervals phase from [day 164](#) begins with a sort, and the sort is what makes the greedy correct rather than merely plausible.

The question to ask, every time

Would this be easier if the input were in order?

Ask it before you write anything. If the answer is yes, the follow-up is arithmetic:

does sorting first change the problem's class?

$O(n^2) \rightarrow O(n \log n)$	yes, sort.	500 billion ops $\rightarrow$ 20 million
$O(n^2) \rightarrow O(n)$ after		
an $O(n \log n)$ sort	yes, sort.	the sort dominates and it is cheap
$O(n) \rightarrow O(n \log n)$	NO. You made it worse.	
$O(n) \rightarrow O(n)$ after		
an $O(n \log n)$ sort	NO. Same class after, plus $n \log n$ before.	

The rule in one line: **sort when it lowers the class; never sort when the problem was already linear.**

The four times not to sort

This half is what separates a habit from a reflex.

One: a hash set or map already gives you $O(n)$. Two Sum on an unsorted array is $O(n)$ with a dictionary. Sorting first and using two pointers is $O(n \log n)$ — correct, slower, and it destroys the original indices, which the problem asks for. “Does a hash structure answer this in one pass?” is the question to ask before reaching for sort.

Two: you only need the top k , not the whole order. Sorting is $O(n \log n)$; a heap of size k is $O(n \log k)$, and quickselect is $O(n)$ on average.

```
n = 1,000,000, k = 10:
full sort      : 1,000,000 x 20 = 20,000,000 operations
heap of size k : 1,000,000 x 3.3 = 3,300,000
quickselect   : ~2,000,000 (average)
```

Heaps arrive on [day 113](#) and quickselect on [day 055](#); today, just know that “the top ten” is not “sort everything”.

Three: the original positions matter. Sorting throws away where things were. If the answer is an index, sorting the bare values loses it — and the fix is Sivasadan’s tag: sort pairs.

```
indexed = sorted((value, i) for i, value in enumerate(nums))
```

Now each value carries its original index through the sort, and both are available at the end.

Four: the data arrives as a stream, or it is enormous. Sorting needs all of it at once and $O(n)$ space if you cannot sort in place. A running median from a stream is two heaps, not a sort per element.

Using Python's sort properly

```
nums.sort()          # in place, returns None, O(1) extra space
ordered = sorted(nums) # a new list, leaves the original alone, O(n) space
```

`sort()` is a method on lists only. `sorted()` takes any iterable and always returns a list. `sort()` returns `None` — `x = nums.sort()` gives you `None`, and it is one of the most common beginner bugs in Python.

The `key` argument is the one that matters:

```
words.sort(key=len)          # by length
people.sort(key=lambda p: p.age) # by one field
people.sort(key=lambda p: (p.city, -p.age)) # by city, then by age descending
intervals.sort(key=lambda iv: iv[0]) # by start time -- the intervals opener
```

A **tuple key** sorts by the first element, then breaks ties with the second, and so on. The minus sign reverses a numeric field inside a tuple key, which `reverse=True` cannot do for one field only.

`key` is called **once per element**, not once per comparison — so an expensive key function costs $O(n)$ calls, not $O(n \log n)$. That is why `key=` is preferred over the old comparison-function style, and it is worth knowing.

What Python actually runs

Python's sort is **Timsort**: $O(n \log n)$ worst case, **stable**, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$. It is written in C, so its constant factor is far smaller than anything you write in Python.

Stable means equal elements keep their original relative order. That is what makes multi-pass sorting work:

```
people.sort(key=lambda p: p.name) # sort by name first
people.sort(key=lambda p: p.city) # then by city -- names stay ordered within each city
```

Stability gets a day of its own on [day 057](#). Today: know the word, and know Python has it.

In an interview you sort with `sorted()` and say $O(n \log n)$. Nobody asks you to implement a sort in order to use one. The next eight days teach how sorts work because interviewers ask *about* them — not because you should write one when you need ordered data.

4 The picture

The one sentence that does the most work:

```

unsorted    [ 41 , 7 , 63 , 12 , 44 , 10 , 40 ]
             the two closest values are 41 and 40 -- FIVE apart in the array.
             To find them you must compare every pair:  $7 \times 6 / 2 = 21$  comparisons.

sorted      [ 7 , 10 , 12 , 40 , 41 , 44 , 63 ]
             ^^^^^^^
             the two closest values are NEXT TO EACH OTHER.
             One pass over 6 neighbouring pairs.

why: if two values are closest, nothing lies between them -- so after sorting,
     nothing lies between them in the array either.

```

What to notice: the proof is one line and it is worth saying out loud in an interview. Closeness in value becomes closeness in position, and that is the whole reason sorting collapses so many problems.

The four payoffs, drawn on one array:

```

sorted [ 7 , 10 , 12 , 40 , 41 , 44 , 63 ]
        ^                               ^
        min                           max      ← 1. the ends, in  $O(1)$ 

[ 7 , 10 , 12 ] [ 40 , 41 , 44 , 63 ]
      ^
      a price cut / a binary search boundary ← 2. cuts, in  $O(\log n)$ 

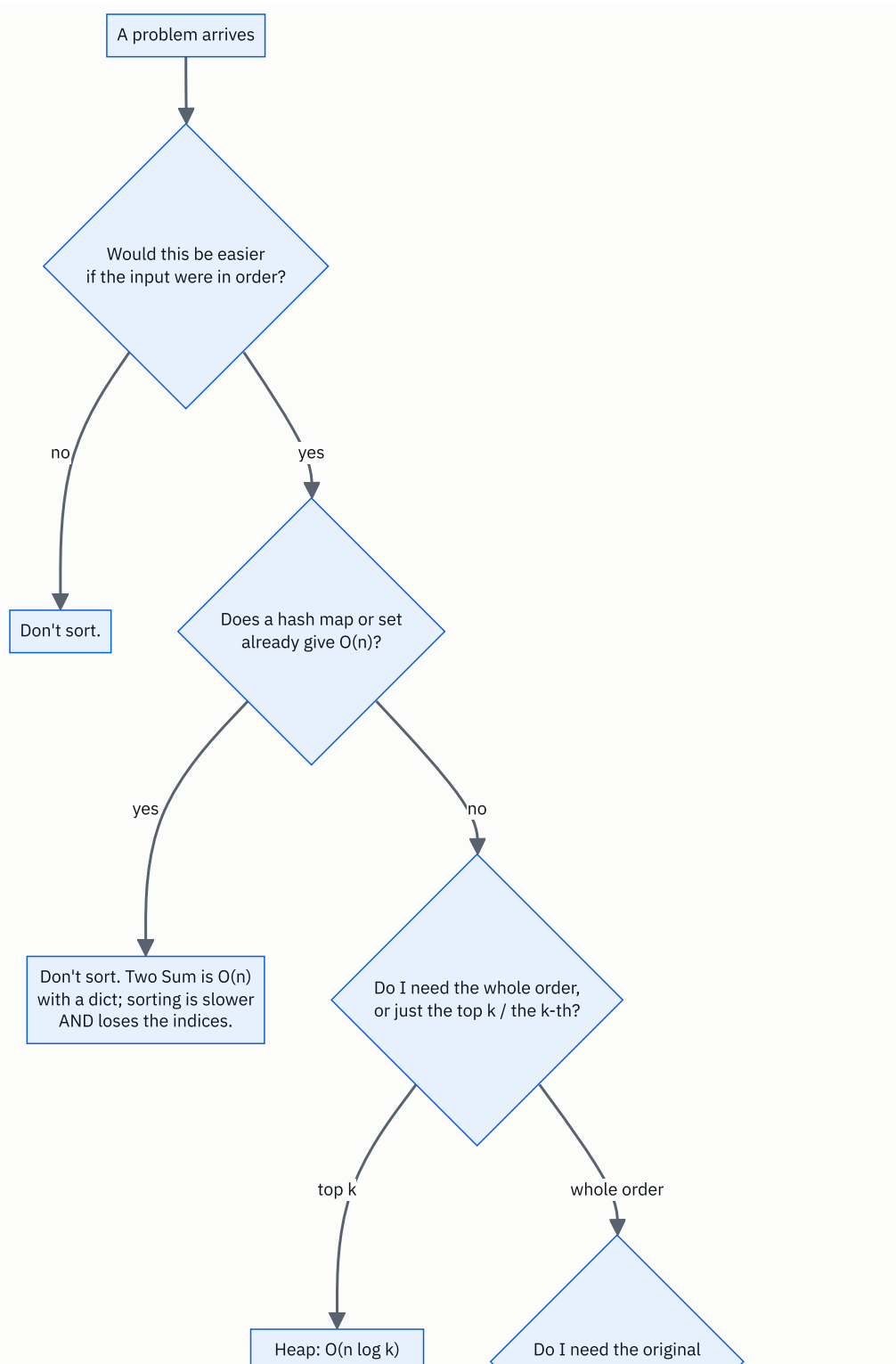
  10  12      40  41  44
  \___/      \___/  \___/
neighbouring pairs are the only candidates
for "closest", "duplicate", "same group" ← 3. adjacency, in  $O(n)$ 

[ ... ] → scan left to right, and once a value is too big, STOP
                                           ← 4. early exit

```

What to notice: all four come from one `sort()` call. That is why the `$O(n \log n)$` is usually worth it — you rarely buy just one of them.

The decision, as you should run it:



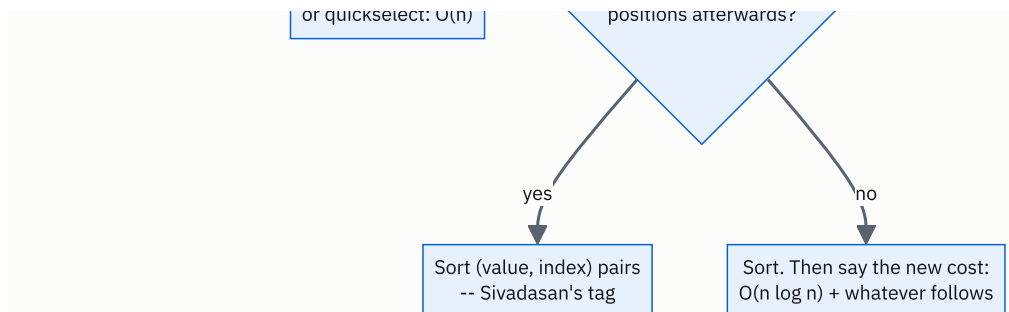


FIGURE 51.1

What to notice: three of the five leaves say “do not sort”. The habit is asking the question, not always answering yes.

5 The code, built step by step

The pattern that names the whole day

Closest pair of values — the problem that is quadratic without a sort and linear with one:

```
def min_difference(nums: list[int]) → int:
    nums = sorted(nums)                #  $O(n \log n)$ 
    return min(b - a for a, b in zip(nums, nums[1:])) #  $O(n)$  over neighbours
```

Two lines. `zip(nums, nums[1:])` pairs each element with the next one — the neighbour-walking idiom, and worth having in your hands. The unsorted version is a double loop and $O(n^2)$.

Duplicates, after a sort

```
def has_duplicate(nums: list[int]) → bool:
    nums = sorted(nums)
    return any(a == b for a, b in zip(nums, nums[1:]))
```

Correct, $O(n \log n)$, and the wrong answer in an interview — `len(set(nums)) != len(nums)` is $O(n)$. Written here on purpose: it is the clearest example of sorting being a reflex rather than a decision.

Merging intervals: sort, then one pass

```
def merge_intervals(intervals: list[list[int]]) → list[list[int]]:
    intervals.sort(key=lambda iv: iv[0]) # by start time -- the sort IS the algorithm
    merged: list[list[int]] = []
    for start, end in intervals:
        if merged and start <= merged[-1][1]: # overlaps the last merged one
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

The sort is the whole insight. Once intervals are in start order, an interval can only overlap the one immediately before it in the output — so one pass suffices, and no interval has to be compared with anything far away. Without the sort you are back to comparing every pair.

Carrying the index through the sort

```
def two_smallest_indices(nums: list[int]) → tuple[int, int]:
    """The original indices of the two smallest values."""
    pairs = sorted((value, i) for i, value in enumerate(nums))
    return pairs[0][1], pairs[1][1]
```

Sivadasan's tag. Tuples compare element by element, so `(value, i)` sorts by value and breaks ties by the original index — which also makes the result deterministic.

Sorting by several keys

```
people.sort(key=lambda p: (p.city, -p.age, p.name))
```

City ascending, then age descending, then name ascending. The minus works on numbers only; for a descending string field you need two passes and stability:

```
people.sort(key=lambda p: p.name, reverse=True)    # inner key first
people.sort(key=lambda p: p.city)                  # outer key second -- stability preserves the first
```

Sort by the least significant key first. That ordering is the part people get backwards.

The complete file

```
from bisect import bisect_left

def min_difference(nums: list[int]) → int:
    """Smallest difference between any two values.  $O(n \log n)$  instead of  $O(n^2)$ .

    Correct because closest-in-value implies adjacent-after-sorting: if nothing lies
    between two values, nothing lies between them in the sorted array either.
    """
    if len(nums) < 2:
        raise ValueError("need at least two values")
    ordered = sorted(nums)
    return min(b - a for a, b in zip(ordered, ordered[1:]))

def merge_intervals(intervals: list[list[int]]) → list[list[int]]:
    """LeetCode 56. Sort by start, then one pass. The sort is the algorithm."""
    ordered = sorted(intervals, key=lambda iv: iv[0])
    merged: list[list[int]] = []
    for start, end in ordered:
        if merged and start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged

def three_sum(nums: list[int]) → list[list[int]]:
    """LeetCode 15. Sort, then fix one value and two-point the rest.

     $O(n^2)$  instead of  $O(n^3)$  -- and the sort is also what makes skipping duplicates easy,
    because equal values are adjacent.
    """
    ordered = sorted(nums)
    out: list[list[int]] = []
    for i in range(len(ordered) - 2):
        if i > 0 and ordered[i] == ordered[i - 1]:
            continue # skip a repeated first value
        left, right = i + 1, len(ordered) - 1
        while left < right:
            total = ordered[i] + ordered[left] + ordered[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                out.append([ordered[i], ordered[left], ordered[right]])
                while left < right and ordered[left] == ordered[left + 1]:
                    left += 1 # skip repeats on the left
                while left < right and ordered[right] == ordered[right - 1]:
                    right -= 1
                left, right = left + 1, right - 1
    return out

def two_sum_hashed(nums: list[int], target: int) → list[int]:
    """LeetCode 1. NOT a sorting problem:  $O(n)$  with a dict, and it keeps the indices."""
    seen: dict[int, int] = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
    seen[x] = i
    return []

def count_in_range(nums: list[int], low: int, high: int) → int:
    """Sort once, then answer any number of range-count queries in  $O(\log n)$  each."""
    ordered = sorted(nums)
    return bisect_left(ordered, high + 1) - bisect_left(ordered, low)

if __name__ == "__main__":
    print(min_difference([41, 7, 63, 12, 44, 10, 40])) # 1 (41 and 40)
    print(min_difference([3, 3])) # 0

    print(merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]])) # [[1, 6], [8, 10], [15, 18]]
    print(merge_intervals([[1, 4], [4, 5]])) # [[1, 5]]
    print(merge_intervals([5, 6], [1, 3])) # [[1, 3], [5, 6]] ← sort matters
```



```

print(three_sum([-1, 0, 1, 2, -1, -4]))          # [[-1, -1, 2], [-1, 0, 1]]
print(three_sum([0, 0, 0, 0]))                  # [[0, 0, 0]]

print(two_sum_hashed([2, 7, 11, 15], 9))         # [0, 1]

nums = [41, 7, 63, 12, 44, 10, 40]
print(count_in_range(nums, 10, 44))             # 5

# the two things people get wrong about Python's sort
xs = [3, 1, 2]
print(xs.sort())                                # None ← sorts in place
print(xs)                                       # [1, 2, 3]
print(sorted([3, 1, 2]))                       # [1, 2, 3] ← returns a new list

```

Note `three_sum` : the sort does two jobs at once. It makes the two-pointer sweep possible, and it puts equal values next to each other so duplicates can be skipped with a comparison to the neighbour. One sort, two payoffs — which is the normal case, and the reason the $O(n \log n)$ is rarely the thing you regret.

6 What it costs

The sort itself

```

Timsort (Python's sort):   $O(n \log n)$  worst case,  $O(n)$  best case on nearly-sorted input
                           stable
                            $O(n)$  extra space worst case (it merges),  $O(1)$ -ish on runs

n =      1,000    → 1,000 x 10 = 10,000 comparisons
n = 1,000,000    → 1,000,000 x 20 = 20,000,000 comparisons (~0.4 s in pure Python terms,
                           but ~0.1 s because sorted() is C)

```

The trade, priced

The closest-pair problem, which is the honest advertisement for the whole idea:

```

n = 1,000,000

unsorted, every pair:  n(n-1)/2 = 499,999,500,000 comparisons  -- ~10 days in Python
sort + neighbour walk: 20,000,000 + 1,000,000 = 21,000,000    -- ~0.15 s

~24,000x fewer operations, bought with one sort.

```

When it is a loss

```

"are there duplicates?"
  sort + neighbour walk : 20,000,000 operations,  $O(n)$  space
  set(nums)              : 1,000,000 operations,  $O(n)$  space
                        → sorting is 20x slower for the SAME answer

"the ten largest of a million"
  full sort              : 20,000,000
  heap of size 10        : 3,300,000      ( $n \log k$ )
  quickselect            : ~2,000,000    (average)

```

The amortisation, which is the real argument at scale

```
q queries of "how many values are in [a, b]" on a fixed array of n = 100,000:

scan per query : q x 100,000
sort once + 2 binary searches per query : 1,700,000 + q x 34

break-even at about q = 17. After that the sort is free and getting freer.
```

Same shape as [day 037](#)'s prefix sums and [day 042](#)'s “many searches on fixed data”: **pay a preparation cost once, buy cheap answers forever**. That is the third time this course has made that argument, and interviewers like hearing it named as a recurring shape.

Space

```
nums.sort()      : 0(1) extra beyond Timsort's merge buffer -- mutates the caller's list
sorted(nums)     : 0(n) -- a new list, original untouched
```

Say which you are doing and why. Mutating an argument the caller still needs is a real bug and a cheap one to avoid.

7 The traps

The real error: `nums.sort()` returns `None`

```
nums = [3, 1, 2]
ordered = nums.sort()
print(ordered[0])
```

```
Traceback (most recent call last):
  File "day51.py", line 3, in <module>
    print(ordered[0])
          ~~~~~^
TypeError: 'NoneType' object is not subscriptable
```

`.sort()` sorts in place and returns nothing; `sorted()` returns a new list. The Python convention is that methods which mutate return `None`, which is also why `nums.reverse()` and `nums.append()` do. Learn the pair once: **`sorted()` when you want a value, `.sort()` when you want the side effect**.

The near-miss: sorting when a set would do

```
def has_duplicate_slow(nums):
    ordered = sorted(nums)
    return any(a == b for a, b in zip(ordered, ordered[1:]))
```

Correct, and twenty times slower than `len(set(nums)) != len(nums)` at a million elements. The interviewer will not tell you it is slow; they will ask “what’s the complexity?” and let you notice. **Ask whether a hash structure answers it in one pass before you sort.**

The near-miss: sorting away the answer

```
def two_sum_broken(nums, target):
    ordered = sorted(nums)          # ← indices are gone
    left, right = 0, len(ordered) - 1
    while left < right:
        total = ordered[left] + ordered[right]
        if total == target:
            return [left, right]    # indices into the SORTED list, not the original
    ...

print(two_sum_broken([3, 2, 4], 6))  # [0, 2] -- should be [1, 2]
```

```
[0, 2]
```

No error. The values `2` and `4` do sum to 6, and their positions in the sorted list are 0 and 2 — which are not their positions in the input. If the problem asks for indices, either use a dictionary (`0(n)`), and it is the right answer here) or sort `(value, index)` pairs. Sivadasan’s tag.

The near-miss: mutating the caller’s list

```
def process(nums):
    nums.sort()                # ← the caller's list is now reordered
    return nums[0]

original = [5, 2, 9]
print(process(original), original)
```

```
2 [2, 5, 9]
```

The caller passed a list and got it back rearranged. Sometimes that is wanted; more often it is a bug found three functions away. Use `sorted(nums)` inside a function unless you have a reason, and if you do mutate, say so in the docstring.

The trap: comparing things that cannot be compared

```
print(sorted([3, "1", 2]))
```

```
Traceback (most recent call last):
  File "day51.py", line 1, in <module>
    print(sorted([3, "1", 2]))
    ~~~~~^~~~~~
TypeError: '<' not supported between instances of 'str' and 'int'
```

Sorting needs a total order. Mixed types, `None` in a list of numbers, or your own class with no `__lt__` all raise this. For your own objects, either define `__lt__` or — better — pass `key=`, which keeps the ordering decision at the call site where the reader can see it.

The trap: sorting by the wrong key on intervals

```
intervals.sort(key=lambda iv: iv[1])          # by END time
```

For *merging* intervals you sort by **start**; for *packing the most non-overlapping meetings* you sort by **end**. Both are one-line sorts and they solve different problems, and using the wrong one gives a plausible wrong answer with no error. When the sort is the algorithm, the key is the algorithm — say out loud which field you are sorting on and why.

8 In the interview

How it gets asked

- “Would sorting first make this easier?” — asked directly, usually as a hint after you have written a nested loop.
- “Find the two closest numbers / the minimum absolute difference.” — the purest form. $O(n^2)$ is the trap answer.
- “Merge these intervals.” — LeetCode 56, where the sort is the entire insight and everything after is one pass.
- “You sorted. What did that cost you, and was it worth it?” — the follow-up, and the answer is arithmetic, not opinion.

What to say out loud, in the first ninety seconds

1. **Ask the question audibly.** “Before I write anything — would this be easier if the input were in order? Here I think yes, and let me say why.”
2. **Name which of the four payoffs you want.** “I want adjacency: if two values are closest, nothing lies between them, so after sorting they’re neighbours. That turns an all-pairs comparison into one pass.”
3. **Do the class arithmetic out loud.** “Unsorted it’s $O(n^2)$ — half a trillion comparisons at a million elements. Sorted it’s $O(n \log n)$ plus $O(n)$, so about twenty-one million. I’ll take the sort.”
4. **Check the three reasons not to.** “A hash set doesn’t answer this — it’s about closeness, not membership. I need the whole order, not the top k . And the problem wants a value, not an index, so I don’t need to carry positions through the sort.”
5. **Say which sort call and what it does to the input.** “`sorted(nums)`, so I don’t mutate the caller’s list. Python’s sort is Timsort — $O(n \log n)$, stable, and written in C.”

The follow-ups

“You sorted. Justify the extra $n \log n$.” With the arithmetic rather than a principle. The unsorted version compares every pair, which is $n(n-1)/2$ — at a million elements that is about five hundred billion comparisons, which is days in Python. Sorting is $n \log n$, so twenty million comparisons, and the pass over neighbours afterwards is another million: twenty-one million against five hundred billion, roughly twenty-four thousand times less work. So the sort

does not merely pay for itself, it changes the complexity class, and that is the test I apply. If sorting took an $O(n^2)$ problem to $O(n \log n)$, or made an $O(n^2)$ problem $O(n)$ after the sort, I sort. If the problem was already $O(n)$ — a membership question a set answers, or a single pass — then sorting makes it *worse*, and the honest thing is to notice that rather than reach for the habit. I would also say what the sort cost me besides time: `sorted` allocates an $O(n)$ copy, and if I used `.sort()` I would be mutating the caller's list, which is a decision rather than a detail.

“Can you do it without sorting?” It depends which payoff I was using, and I would answer by naming that first. If I sorted for *membership or duplicates*, then yes — a set does it in one pass, $O(n)$ time and $O(n)$ space, and it is strictly better. If I sorted for *the top k*, then yes — a heap of size k is $O(n \log k)$ and quickselect is $O(n)$ on average, so at a million elements and k of ten that is about three million or two million operations against twenty million for the full sort. If I sorted for *adjacency of near values* — closest pair, minimum difference — then no, not in the general case: knowing which values are closest genuinely requires knowing the order, and any comparison-based method that answers it can be used to sort, so it inherits the $n \log n$ lower bound. The exception is if the values are bounded integers, in which case counting sort or bucketing gives $O(n + \text{range})$, which is [day 056](#). So the answer is: for two of the four payoffs there is something better; for the ordering ones, the sort is the price of the information.

“The data arrives as a stream and you can’t hold it all. Now what?” Then sorting is off the table, because a sort needs every element present, and I would restructure around what the question actually needs. For the top k of a stream, keep a min-heap of size k : push each element, pop when the heap exceeds k , and at the end the heap holds the k largest — $O(n \log k)$ time and $O(k)$ space, and it never holds more than k items. For a running median, keep two heaps, a max heap of the lower half and a min heap of the upper half, rebalanced after each insert, which gives $O(\log n)$ per element. For approximate quantiles over a very large stream, the real-world answer is a sketch — t-digest or similar — which trades exactness for constant memory, and I would say plainly that it is approximate. And if the data is simply larger than memory but finite, the answer is an external merge sort: sort chunks that fit in memory, write them out, and merge the sorted runs — which is exactly what a database does for a large `ORDER BY`, and it is why [day 032](#)'s query plans show a sort spilling to disk.

A model answer

“Before I write anything, the question I ask on every array problem: would this be easier if the input were in order?”

Here it would, and I can say precisely why. The claim is that if two values are the closest pair, nothing lies between them in value — so after sorting, nothing lies between them in position either. They’re neighbours. That means I don’t have to compare every pair; I only have to compare each element with the one after it.

```
def min_difference(nums: list[int]) → int:
    ordered = sorted(nums)
    return min(b - a for a, b in zip(ordered, ordered[1:]))
```

The arithmetic that justifies the sort: unsorted, all pairs is $n(n-1)/2$, which at a million elements is about five hundred billion comparisons. Sorted, it’s twenty million for the sort plus a million for the walk — twenty-one million, about twenty-four thousand times less. The sort changed the complexity class, which is my test for whether to do it.

I’d also say what I checked before reaching for it. A hash set doesn’t help — this is about closeness, not membership, and a set knows nothing about order. I need the whole order, not the top k , so a heap wouldn’t be cheaper. And the problem asks for a difference rather than indices, so I don’t need to carry the original positions through — if it did, I’d sort `(value, index)` pairs so both survive.

I used `sorted` rather than `.sort()` deliberately, so the caller’s list isn’t mutated. That costs $O(n)$ space. Python’s sort is Timsort: $O(n \log n)$ worst case, stable, adaptive on nearly-sorted input, and implemented in C, so its constant factor beats anything I’d write.

The general habit I’d state: sort when it lowers the complexity class, and don’t when the problem was already linear. Duplicates is the counter-example I’d give — `len(set(nums))  $\neq$  len(nums)` is $O(n)$, and sorting to answer it is twenty times slower for the same answer.”

9 Recall card

- **Ask it on every array problem: “would this be easier in order?”** Sorting buys four things — equal values adjacent · **near values adjacent** · binary search and early exit · a correct greedy order.
- **The sentence that earns the $n \log n$:** *if two values are closest, nothing lies between them — so after sorting they are neighbours.* All-pairs 500 billion $\rightarrow$ 21 million at $n = 10^6$.
- **Sort only when it lowers the class.** Four times not to: a hash set already gives $O(n)$ (duplicates, Two Sum) · you want the top k (heap $O(n \log k)$, quickselect $O(n)$) · you need the original indices · the data is a stream.

- **Carry the tag when order would destroy something:** `sorted((value, i) for i, value in enumerate(nums))`. And `.sort()` returns `None` and mutates; `sorted()` returns a new list.
- `key=` **is called once per element** and tuple keys break ties (`(city, -age)`); multi-pass sorting needs **stability** and the *least* significant key first. Python runs **Timsort** — $O(n \log n)$, stable, adaptive, in C. In an interview you *call* a sort; the next eight days are about how they work because interviewers ask *about* them.

1 What this is, and why they ask it

Domain modelling is turning a description of how a business works into classes, fields and relationships that hold its rules. Not a database schema — that comes later and is a different question. A model: the things the business talks about, the invariants that must never be false, and where each rule lives.

They ask it because it is the first ten minutes of every low-level design round, and because it is the part that cannot be revised for. You have now met classes, encapsulation, inheritance, polymorphism, abstraction and composition — six days of tools. Today is the day you are handed a paragraph and asked to produce a model with them, under a clock, for a business you have never worked in. What separates candidates is not tool knowledge. It is three habits: using the words the business uses, finding the concept nobody named, and drawing the boundary around what must be consistent together.

2 The story

Kalyan finished his degree in June and started at his father's printing press in Sivakasi, and the first month was worse than either of them expected.

The press does wedding cards, calendars, shop labels — four machines, eleven people, and his father has run it since 1989.

The trouble was one word. Kalyan came in saying “order” for everything, because that is what a customer places, and for four weeks he was wrong in a way nobody could quite explain to him.

In the press there are three words and they are not the same word.

An **order** is what the customer asked for. Mrs Fernandez wants four hundred wedding cards, two hundred envelopes and a hundred small thank-you slips. One order.

A **job** is one thing set up on one machine. Those cards are one job on the big machine. The envelopes are a second job, and the slips a third, and they go on different machines on different days, because setting up a machine takes forty minutes and you do not do it twice for the same thing. One order, three jobs.

A **run** is one pass of a job through the machine. The card job is one run if everything goes well, and it is four runs if the paper jams twice and somebody notices the alignment is off.

For four weeks Kalyan answered the phone and said “yes ma’am, your order is done” because a job had come off the machine, and twice a customer arrived and only the envelopes were ready.

Then there was the thing nobody had a word for at all.

When a run goes wrong, they run it again. That second run is not a new order — the customer asked for nothing new. It is not really a new job either, because nothing was set up again. But it eats paper, it eats machine time, and it is the difference between a profitable month and a flat one. For twenty-five years it lived in his father’s head as a feeling about whether the month had gone well.

Kalyan gave it a name in September. He called it a spoil, because that is what the men on the floor had always called the wasted sheets, and he started counting spoils per job. His father looked at three months of it and moved the calendar work to the other machine, and that was worth about eleven thousand rupees a month.

The word already existed on the floor. It had just never made it as far as the office.

3 The idea in plain English

Kalyan’s four weeks are what happens when you model with your own words instead of the business’s. Order, job and run are the **entities**. The spoil is the concept the requirements never mentioned, and it is where the value was. And the fact that the men on the floor already had the word is the whole of what “use the domain’s language” means.

The procedure

Six steps, in order, and the order matters because each one depends on the last.

Step one: collect the nouns and verbs, in the business’s words. Underline them as they are written. If the requirements say “consignment”, your class is `Consignment`, not `Shipment`, even if `Shipment` is what you would have called it.

That habit has a name — the **ubiquitous language**, a single vocabulary shared by the code, the conversation and the business. It is not pedantry. When the code says `Shipment` and the operations team says “consignment”, every conversation between them costs a translation, and eventually somebody translates wrongly. Kalyan’s four weeks were exactly that cost.

Step two: separate entities from value objects.

- An **entity** has an identity that persists while everything about it changes. A `Job` is the same job after it is rescheduled, reprinted and renamed. Entities are usually mutable and compare by id.
- A **value object** is defined entirely by its values. `Money(400, "INR")` is interchangeable with any other four hundred rupees. Value objects should be **frozen**, compare by value, and are safe to share and hash.

The test, from [day 044](#): *would I care which one I got?* Two four-hundred-rupee amounts, no. Two jobs, yes.

Getting this wrong is expensive in a specific way: an entity treated as a value loses its history, and a value treated as an entity gets an id nobody needs and an equality that surprises people.

Step three: find the noun that is not in the paragraph. This is the step that produces good models, and it is the one candidates skip.

The requirements describe *things*. The interesting concepts are usually the *relationships between things*, and nobody writes those down because everyone in the business already knows them.

"members borrow books"	→ Loan	(nobody said the word)
"a customer books a seat for a show"	→ Booking	
"a vehicle parks in a spot"	→ Ticket	
"an order is printed"	→ Job, Run, and Kalyan's Spoil	

Every one of those is where the dates, the state and the money live. A model without `Loan` has to put the due date on `Copy`, which means a copy remembers only its current loan and history is gone. **When a rule has nowhere to live, you have found a missing class.**

Step four: write the invariants, as sentences. Two to four, in plain English:

An order is complete when all its jobs are complete. A run belongs to exactly one job. The paper used by a job is the sum of its runs, including spoils.

These do two things. They tell you where behaviour goes — the object that owns the data an invariant needs owns the method that enforces it. And they tell you where the boundaries go, which is step five.

Step five: draw the aggregate boundaries. An **aggregate** is a cluster of objects treated as one unit for consistency. One class in it is the **root**, and the rule is:

Everything outside the aggregate holds a reference to the root only. All changes go through the root, and the root enforces the invariants that span the cluster.

For the press, `Order` is a root and its `Job`s are inside it. Nothing outside reaches into a job to change its state; it asks the order. That is why “an order is complete when all its jobs are complete” can be guaranteed — there is one door, exactly as [day 045](#) argued.

Keep aggregates small. If your `Order` aggregate contains customers, invoices and the entire price list, then everything is one aggregate and the boundary means nothing.

Step six: put each verb on the class that owns the data it needs. The rule from [day 044](#), unchanged: `member.borrow()`, `loan.late_fee(returned_on)`, `order.is_complete()`. If a rule fits nowhere, go back to step three — it is usually a missing class rather than a service.

The same word meaning two things

One more idea, and it is the one that separates a good answer from a very good one.

In a large business the same word means different things in different parts of it. To sales, a “customer” is a lead with a pipeline stage. To support, a “customer” is an account with a ticket history. To billing, it is a payment method and an address. Forcing one `Customer` class to serve all three gives you a class with forty fields, most of them null most of the time.

The name for the boundary between those worlds is a **bounded context**: each context gets its own model of “customer”, and they are linked by an id rather than merged. In a forty-five-minute interview you will not build several contexts — but noticing out loud that *“‘customer’ means something different to billing than it does to support, and I’d keep those as separate models linked by id”* is a strong signal, and it takes one sentence.

What not to do

- **Do not design the database.** No foreign keys, no `BIGSERIAL`, no indexes. That is a different question and answering it here means you have misread the round — [day 043](#)’s wrong-round warning.
- **Do not model what was not asked for.** Loyalty points, refunds and audit logs are scope you chose to add.
- **Do not use your own words** where the business has its own. Say `Spoil` if they say spoil.
- **Do not produce fields with no behaviour.** That is the anaemic model, and six days of this phase have been about avoiding it.

4 The picture

The press, modelled — and the aggregate boundary drawn, which is the part most diagrams omit:

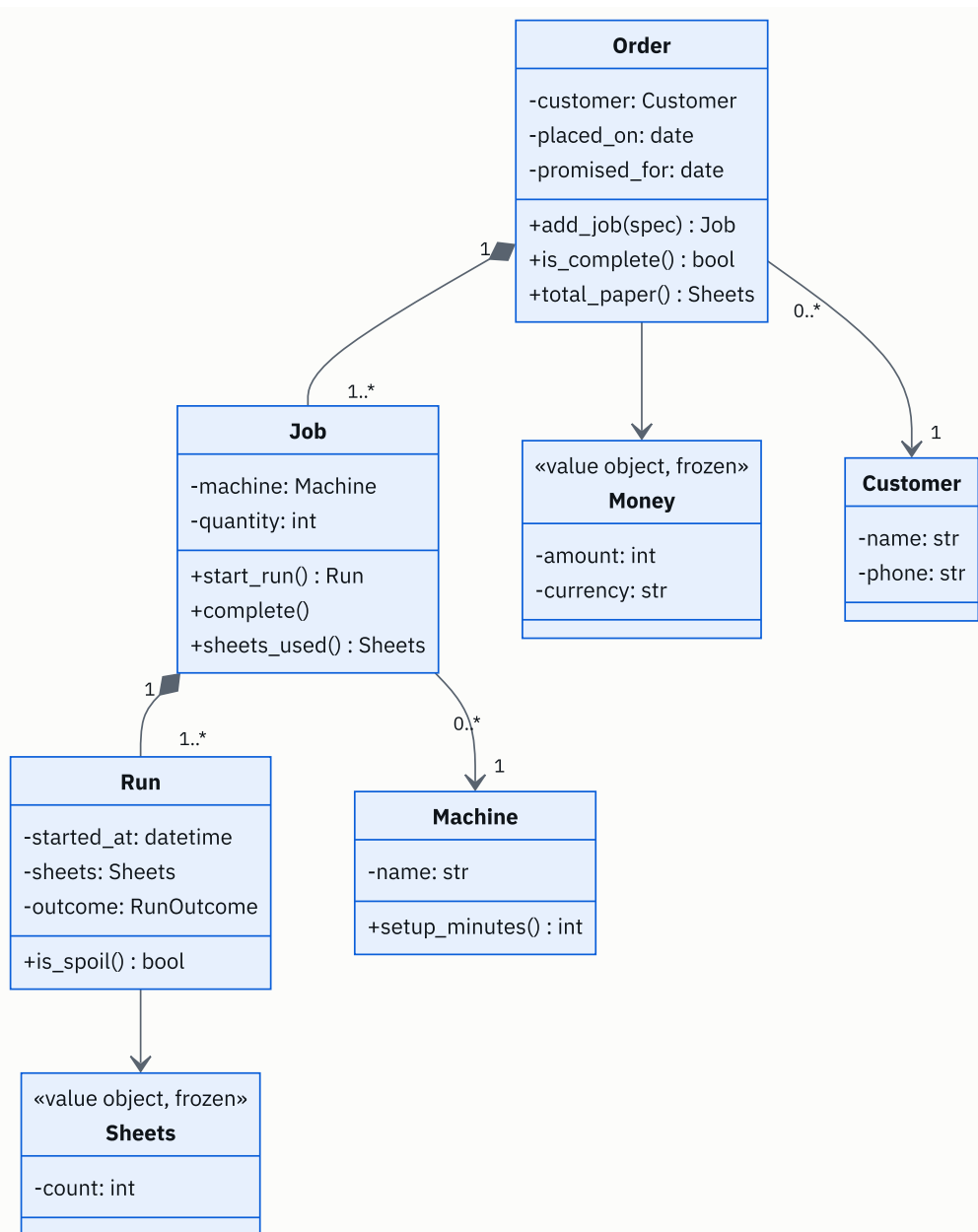
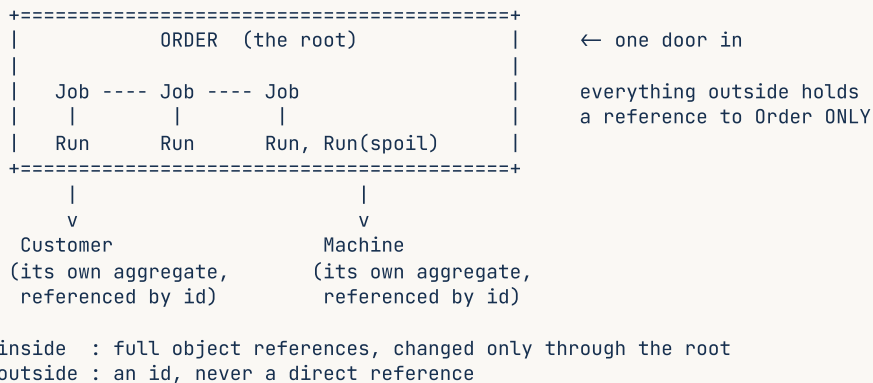


FIGURE 51.2

What to notice: `Run` is inside the `Order` aggregate, two levels down. Nothing outside the order ever holds a `Run` — if the shop floor wants to record a spoil, it goes `order.job(id).start_run()`. That is what makes “the paper used by an order is the sum of its runs” enforceable rather than hopeful.

The aggregate boundary, drawn plainly:



What to notice: `Customer` and `Machine` are outside. They have their own lifecycles — a customer exists before any order and after all of them — so they are separate aggregates referenced by id. Pulling them inside would make every order load a customer’s entire history.

5 How it actually works

A worked example, start to finish

Requirements, of the kind you will be handed:

A courier company collects parcels from senders and delivers them to recipients. A pickup can collect several parcels from one sender. Parcels travel between hubs and each movement is recorded. A parcel can be attempted for delivery up to three times; after that it is returned to the sender. Customers are quoted a price at booking, based on weight and distance.

Step one, the nouns in their words: parcel, sender, recipient, pickup, hub, movement, attempt, return, quote, weight, distance. Verbs: collect, deliver, travel, record, attempt, return, quote.

Step two, entities against values:

```

entities (identity persists):  Parcel, Pickup, Hub, Shipment, DeliveryAttempt
value objects (frozen):       Weight, Money, Address, TrackingNumber, Distance
not classes at all:           "three times" is a rule, not a thing

```

`Address` as a value object is the decision worth defending: two identical addresses are interchangeable, and freezing it means a parcel’s delivery address cannot be mutated from a distance.

Step three, the nouns that are not in the paragraph:

```

Movement / Leg    -- "parcels travel between hubs and each movement is recorded".
                   The movement is a THING: from hub, to hub, departed at, arrived at.
                   Without it, a parcel can only remember where it is now.

Quote              -- named, but easy to miss as a class. It has a price, the inputs that
                   produced it, and a time it was valid at. Without it, "why were we
                   charged this?" is unanswerable three months later.

Route              -- never mentioned. Whether you need it depends on whether the sequence
                   of movements is planned in advance or emerges. Ask.

```

Step four, the invariants:

A parcel is at exactly one hub, or in transit on exactly one movement, at any time. A parcel has at most three delivery attempts; the third failure returns it. A quote's price never changes after the booking accepts it.

That third one immediately tells you `Quote` is a frozen value snapshot, not a live calculation — which is [day 029's](#) snapshot rule arriving in object form.

Step five, the aggregates:

```

Parcel (root)    ---- Movement, DeliveryAttempt    one consistency boundary:
                                                         "at most three attempts" is enforced here

Pickup (root)    ---- references parcels by id        a pickup groups parcels; it does not own them

Hub (root)       ----                               its own lifecycle entirely

```

The key decision: a `Movement` belongs inside `Parcel` because the “exactly one location” invariant spans them. A `Pickup` references parcels **by id** rather than containing them, because a parcel outlives its pickup and belongs to the delivery process afterwards.

Step six, the behaviour:

```

class Parcel:
    def attempt_delivery(self, at: datetime, outcome: AttemptOutcome) → None:
        if self.is_returned:
            raise ValueError("parcel has already been returned to sender")
        self._attempts.append(DeliveryAttempt(at, outcome))
        if outcome is AttemptOutcome.FAILED and len(self._attempts) == 3:
            self._begin_return()          # the rule lives with the data it needs

```

The three-attempt rule sits on `Parcel` because `Parcel` is the only object that knows how many attempts there have been. Put it in a `DeliveryService` and the bulk-import path will forget it.

The names to know

These are the standard vocabulary and using them correctly is worth real credit:

- **Entity** — identity persists through change. Compare by id.
- **Value object** — defined by its values. Frozen, compare by value, safe to share.
- **Aggregate** and **aggregate root** — a consistency boundary and its single door.

- **Repository** — the thing that loads and saves an aggregate root: `OrderRepository.get(id)`, `.save(order)`. One repository per aggregate root, never one per class. This is also where [day 048](#)'s interface argument pays off — the repository is an interface, with a Postgres implementation and an in-memory fake.
- **Domain service** — for a rule that genuinely belongs to no single entity, such as pricing that needs a rate card and a distance table. It is the *last* resort, not the first, and the test is whether you tried step three properly.
- **Ubiquitous language** and **bounded context** — the vocabulary rule and its boundary.

These come from Eric Evans' *Domain-Driven Design*, and knowing where the words come from is worth a sentence if the interviewer uses them.

The order to speak in

```
minutes 0-2   read back the nouns and verbs IN THE BUSINESS'S WORDS, and ask 3-4 questions
minutes 2-4   entities and value objects, split with the "would I care which one I got" test
minutes 4-6   the missing noun -- name it, and say what has nowhere to live without it
minutes 6-8   the invariants, two to four, as sentences
minutes 8-11  the aggregates: which root, and what is referenced by id from outside
minutes 11-15 behaviour on the classes, one rule placed out loud with its reason
```

Fifteen minutes, and the remaining thirty are code and the extension question.

6 The numbers

Model size, and what it means when it is wrong

```
a 45-minute round:           6-10 classes
    fewer than 5             → under-modelled; the missing noun is usually why
    more than 15             → you modelled scope nobody asked for

per class:
    fields                   3-6             more than ~10 is two classes
    methods                  2-5             more than ~8 is two classes
    invariants               1-3

aggregates in a model of this size: 2-4 roots
    1 root → everything is one aggregate; the boundary means nothing
    6+ roots → probably one per class, which is also no boundary
```

The cost of the wrong word

Kalyan's four weeks, priced the way it actually shows up:

```
one mistranslated term ("order" for "job"):  
  ~2 wrong answers per week to customers x 4 weeks      = ~8 incidents  
  each one: a phone call, an apology, sometimes a redelivery  
  
in code, the same mistake:  
  every conversation between engineering and operations pays a translation  
  every new joiner learns two vocabularies  
  the bug where the translation is wrong ships, because it reads correctly
```

The fix costs nothing at design time and is nearly impossible later: renaming a core domain concept touches every file, every table, every dashboard and every conversation.

The missing noun, priced

```
no Loan class:  
  the due date goes on Copy      → a copy remembers only its CURRENT loan  
  loan history                  → gone  
  "how many times was this borrowed" → unanswerable  
  the late-fee rule             → lives in a service, at 3-4 call sites  
  
adding Loan later:  
  a data migration, plus every place that reads Copy.due_date    ~15-30 edits  
adding Loan at design time:  
  one class                                                       1 file
```

Aggregate size, and why it matters at run time

```
Order aggregate containing Customer, and Customer containing all their Orders:  
  loading one order → loads the customer → loads their 400 orders → ...  
  the classic accidental full-database load  
  
Order referencing Customer BY ID:  
  loading one order = 1 order + its jobs and runs ~ 20 objects
```

That is [day 041](#)'s N+1 arriving as a modelling decision rather than as a query decision — and it is decided here, months before anyone looks at a query log.

7 The trade-offs

A rich model against getting it built

Entities, value objects, aggregates and repositories are real structure and real time. *I would not build a full domain model for a service that reads a table and returns JSON* — a CRUD service with no invariants has nothing for a model to protect, and a dataclass plus a query is the honest answer. The trigger is invariants: two or three rules that must never be false are what a model is for.

The business's words against good names

Sometimes the business's word is genuinely bad — overloaded, or the same word for two things. *I would still use their word*, and if it truly names two things I would say so and propose splitting it with them rather than inventing a private vocabulary in the code. A model whose

names differ from the conversation costs a translation forever; a model that renames a concept *with the business's agreement* is a one-time cost.

Big aggregates against small ones

A large aggregate makes invariants easy — everything is inside one boundary, so consistency is free. It also makes every load enormous and every write contend with every other write on the same root. *I would keep aggregates as small as the invariants allow*, reference other aggregates by id, and accept that a rule spanning two aggregates becomes eventually consistent — which is a real cost and worth naming rather than hiding.

Modelling reality against modelling what the software needs

The world is infinitely detailed and your model should not be. *I would not model a parcel's physical dimensions if nothing prices or routes on them.* The test is not “is this true of the real thing?” but “does any rule in this system depend on it?” Kalyan's spoil earned its class because eleven thousand rupees a month depended on counting it.

The honest sentence

A domain model is worth building when the business has rules that must never be broken, and it is ceremony when it does not. What makes it good is not the number of classes — it is that the words match the ones people say out loud, that the concept nobody named has a home, and that every rule has exactly one place it can be enforced.

8 In the interview

How it gets asked

- “*Here are the requirements. Model it.*” — a paragraph, and fifteen minutes before code.
- “*What are the main entities?*” — the shorter version, often the opening of a bigger design round.
- “*Where does that rule live?*” — asked halfway through, about a specific requirement. This is the scored question.
- “*What would you call this?*” — a check on whether you are using their vocabulary or your own.

What to say out loud, in the first ninety seconds

1. **Read the nouns back in their words.** “*Let me play back the vocabulary: parcel, pickup, hub, movement, attempt, quote. I'll use those names exactly, even where I'd have picked different ones.*”
2. **Ask the three or four questions that change the model.** “*Is a pickup always from one sender? Is the route planned in advance or does it emerge? Does the quote's price ever change after booking? And does a parcel ever split across movements?*”

3. **Split entities from values, with the test.** “`Parcel` and `Hub` are entities — identity persists. `Weight`, `Money` and `Address` are value objects, frozen, compared by value. The test is whether I’d care which one I got: two identical addresses, no; two parcels, yes.”
4. **Name the missing noun and say what it rescues.** “`Movement` isn’t in the requirements as a thing, but ‘each movement is recorded’ means it is one — from hub, to hub, departed, arrived. Without it a parcel can only remember where it is now, and the tracking history has nowhere to live.”
5. **State two or three invariants as sentences**, then use them to draw the boundary.
6. **Place one rule out loud, with the reason.** “Three attempts then return goes on `Parcel`, because `Parcel` is the only object that knows how many attempts there have been.”

The follow-ups

“Where does that rule live, and why not in a service?” On the object that owns the data the rule reads and writes — that is the test I apply every time. The three-attempt rule needs the count of attempts and the parcel’s returned status, and only `Parcel` has both, so it goes on `Parcel.attempt_delivery`. If I put it in a `DeliveryService` instead, the rule lives at a call site, and then the courier app calls it, and the hub scanner calls it, and the bulk import script calls it — three copies of one rule, and the one somebody forgets is the one that ships, with no error at the moment it breaks. That is the anaemic model, and it is objects in name only. A domain service is legitimate, but only for a rule that genuinely belongs to no single entity — pricing that needs a rate card and a distance table, for instance, where neither the parcel nor the route owns the inputs. My rule is that a service is the last resort, and before reaching for one I go back and ask whether the rule is telling me about a class I have not created yet. Nine times out of ten it is.

“You’ve invented a class the requirements never mentioned. Justify it.” Because a rule and some data had nowhere to live, and that is the strongest signal in modelling. The requirements say “parcels travel between hubs and each movement is recorded”. If `Movement` is not a class, then recording a movement means writing to fields on `Parcel` — a `current_hub`, maybe a `departed_at` — and the moment I do that, the parcel remembers only where it is now. There is no history, so “where has this been” and “how long did the Coimbatore leg take” are unanswerable, and those are exactly the questions a courier company asks. So `Movement` is a class with a from-hub, a to-hub, a departure and an arrival, and the parcel holds a list of them. This is the same pattern as `Loan` in a library and `Ticket` in a car park — the interesting class is usually the *relationship* between two things, and nobody writes it down because everyone in the business already knows it is there. The check I use is: is there a rule, a date or an amount of money that has no home? If yes, that is a missing class, not a missing field.

“How do you decide what goes inside an aggregate and what is referenced by id?” By the invariants, and only by the invariants. An aggregate is a consistency boundary, so things go inside it when a rule spans them and must hold at every moment. “A parcel has at most three delivery attempts” spans the parcel and its attempts, so attempts are inside the parcel’s boundary, and nothing outside ever holds a reference to an attempt — changes go through the parcel,

which is the one door that can enforce the rule. `Customer` is different: no invariant spans a parcel and a customer, and a customer exists before any parcel and after all of them, so it is its own aggregate referenced by id. There are two costs to getting that wrong and they pull opposite ways. Make the aggregate too big — pull `Customer` inside, and their four hundred other parcels come with it — and loading one parcel loads the database, which is the $N + 1$ problem decided at design time rather than in a query log. Make it too small and a rule that must be atomic now spans two boundaries, which means it becomes eventually consistent and I have to say so out loud rather than pretend it is guaranteed. So: as small as the invariants allow, and reference everything else by id.

A model answer

“Let me play the vocabulary back first, because I want to use their words rather than mine.

Nouns: parcel, sender, recipient, pickup, hub, movement, attempt, quote. Verbs: collect, deliver, travel, record, attempt, return, quote. If the business says ‘consignment’ rather than ‘parcel’, I’d use consignment — a model whose names differ from the conversation costs a translation forever.

Four questions that would change the model: is a pickup always from one sender? Is the route planned in advance or does it emerge from the movements? Can a quote’s price change after booking? And does a parcel ever split?

Now the split. Entities, where identity persists through change: `Parcel`, `Hub`, `Pickup`. Value objects, frozen and compared by value: `Weight`, `Money`, `Address`, `TrackingNumber`. The test is whether I’d care which one I got — two identical addresses are interchangeable, two parcels are not.

Then the class that isn’t in the requirements, which is where the interesting part usually is. The line ‘each movement is recorded’ means `Movement` is a thing: from hub, to hub, departed at, arrived at. Without it, a parcel can only hold a `current_hub` field, so it remembers where it is and not where it has been — and ‘where has this parcel been’ is the single most common question a courier company gets asked. The same shape as `Loan` in a library or `Ticket` in a car park: the valuable class is the relationship, and nobody writes it down because everyone in the business already knows it.

Three invariants, as sentences. A parcel is at exactly one hub or on exactly one movement at any time. A parcel gets at most three delivery attempts, and the third failure returns it. A quote’s price never changes once the booking accepts it — which tells me `Quote` is a frozen snapshot rather than a live calculation.

Those invariants draw the boundaries. `Parcel` is an aggregate root, with `Movement` and `DeliveryAttempt` inside it, because both those rules span the parcel and its parts. `Hub` and `Customer` are separate roots referenced by id — no invariant spans them, and pulling a customer inside would mean loading one parcel drags in their entire history.

Behaviour goes with the data. `Parcel.attempt_delivery(at, outcome)` owns the three-attempt rule, because `Parcel` is the only object that knows the count. In a `DeliveryService` it would be duplicated across the courier app, the hub scanner and the import script, and the one somebody forgets is the one that ships.

One thing I’d flag before coding: ‘customer’ probably means something different to billing than to operations. I’d keep those as separate models linked by an id rather than one class with forty fields.”

9 Recall card

- **Six steps, in order:** nouns and verbs *in the business's words* → entities vs value objects → **the noun nobody wrote down** → invariants as sentences → aggregate boundaries → behaviour on the class that owns the data.
- **Use their vocabulary — the ubiquitous language.** `Consignment` if they say consignment, `Spoil` if the floor says spoil. A model whose names differ from the conversation costs a translation forever, and renaming later touches everything.
- **The missing noun is where the value is:** `Loan`, `Ticket`, `Movement`, `Booking`, `Quote`. The test — *is there a rule, a date or an amount of money with no home?* If yes, that is a class, not a field.
- **An aggregate is a consistency boundary with one door.** Inside: things a rule spans; changes go through the root. Outside: **referenced by id**. Too big → loading one order loads the database; too small → an atomic rule becomes eventually consistent, and you must say so.
- **Entity = identity persists (compare by id, mutable). Value object = defined by its values (frozen, compare by value).** Test: *would I care which one I got?* A **domain service** is the last resort — reach for it only after checking you have not missed a class. 6-10 classes, 2-4 roots.

PRACTICE

Practice — Why sorting matters more than any single sorting algorithm

DSA topic: Why sorting matters more than any single sorting algorithm **System design topic:** Modelling a real domain

Code these, in this order

One rule for the whole set: **before writing anything, say out loud whether sorting helps, which of the four payoffs you want, and what the cost becomes.** On one of these four the answer is no, and spotting which is the point.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Minimum Absolute Difference	LeetCode 1200 (Easy)	The purest form: closest-in-value becomes adjacent-after-sorting.
2	Merge Intervals	LeetCode 56 (Medium)	That the sort <i>is</i> the algorithm, and which field you sort on.
3	3Sum	LeetCode 15 (Medium)	One sort buying two things — the two-pointer sweep and easy duplicate skipping.
4	Two Sum	LeetCode 1 (Easy)	The trap: sorting is slower here <i>and</i> destroys the answer.

On problem 1, say the proof before the code

If two values are the closest pair, nothing lies between them in value — so after sorting, nothing lies between them in position either. Then two lines. Then state the cost both ways: $n(n-1)/2$ against $n \log n + n$, and the ratio at a million.

On problem 2, get the key right and prove it matters

Sort by **start**, then one pass. Then deliberately sort by `iv[1]` instead and run it on `[[1, 4], [0, 2], [3, 5]]` — you get a plausible wrong answer with no error. Say the difference: merge by start, pack the most non-overlapping meetings by end. When the sort is the algorithm, the key is the algorithm.

On problem 3, notice the sort is doing two jobs

Write it, then find both payoffs in your own code: the two-pointer sweep needs order, and the duplicate-skipping `while` loops need equal values to be adjacent. Say “one sort, two payoffs” out loud — that is why the $O(n \log n)$ is almost never the thing you regret.

On problem 4, write the wrong version first

Write the sort-plus-two-pointers version. Run it on `[3, 2, 4]` with target 6 and read the answer: `[0, 2]`, which are positions in the *sorted* list, not the input. No exception. Then write the dictionary version, say it is $O(n)$ rather than $O(n \log n)$, and say the second reason it is bet-

ter — it never had the indices to lose.

The four-payoffs drill

For each problem, name which of the four things sorting buys — equal-adjacent, near-adjacent, binary search / early exit, or a correct greedy order:

1. Are there any duplicates?
2. What is the smallest difference between any two values?
3. How many values lie between 10 and 44?
4. Merge overlapping intervals.
5. Group anagrams together.
6. What is the maximum number of non-overlapping meetings?

Two of these have a better non-sorting answer. Name them and say what it is.

The don't-sort drill

For each, say whether to sort — and if not, what instead, with the cost:

1. Find the ten largest of a million values.
2. Find whether any value appears twice.
3. Find the indices of two values summing to a target.
4. Find the median of a stream that will not fit in memory.
5. Answer a thousand “how many values in [a, b]” queries on a fixed array.
6. Find the two closest values.

The arithmetic drill

Answer with numbers, in under ten seconds each:

1. Comparisons to sort a million elements.
2. Comparisons to compare every pair of a million elements.
3. The ratio between those two.
4. Operations for the ten largest of a million: full sort, heap, quickselect.
5. The break-even query count for “sort once and binary search” against “scan every time” at $n = 100,000$.

The Python drill

Say what each does, and which is right for which situation:

1. `nums.sort()` versus `sorted(nums)` — return value, space, and what happens to the caller's list.
2. `key=lambda p: (p.city, -p.age)` — what order, and why the minus sign is needed rather than `reverse=True`.

3. Two-pass sorting by city then name — which pass goes first, and what property makes it work.
4. `sorted((value, i) for i, value in enumerate(nums))` — what problem does this solve.
5. `sorted([3, "1", 2])` — paste the exact error.
6. How many times is `key` called for a list of `n` elements?

The modelling drill

Set a timer for fifteen minutes and model this out loud, running the six steps in order:

A gym sells memberships. A member books classes; each class has a trainer, a room and a capacity. A member can cancel up to two hours before a class and the slot goes back on sale. Members on a monthly plan can book unlimited classes; members on a ten-class pack lose a credit per booking, and a cancellation returns it.

1. Nouns and verbs, in the gym's words. Then three questions that would change the model.
2. Entities against value objects, with the “would I care which one I got” test applied to at least two.
3. The noun that is not in the paragraph. Name it, and say exactly what has nowhere to live without it.
4. Three invariants, as sentences.
5. The aggregate roots, and what is referenced by id from outside — with the invariant that justifies each boundary.
6. Where the cancellation rule goes, and the one-sentence reason. Then where the credit rule goes, and whether it is the same object.

Then answer the extension you know is coming: “*now a class can be booked by a member on behalf of a guest.*” Say what changes and what does not.

The vocabulary drill

Define each in one sentence, then give one example from the gym model:

1. Entity.
2. Value object.
3. Aggregate root.
4. Repository — and how many you would have for the gym.
5. Domain service — and why it is the last resort.
6. Ubiquitous language.
7. Bounded context — and where one would appear in a gym chain.

The missing-noun drill

For each requirement, name the class that is not in the sentence, and say what has nowhere to live without it:

1. "Members borrow books."
 2. "A customer books a seat for a show."
 3. "A vehicle parks in a spot and pays on exit."
 4. "Parcels travel between hubs and each movement is recorded."
 5. "Employees claim expenses, which a manager approves."
 6. "A patient sees a doctor and is prescribed medicines."
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Would sorting first make this problem easier?* The question, the payoff you want, the class arithmetic with real numbers, and the three checks that would make the answer no.
 2. *Here are the requirements. Model it.* The six steps in order on the gym, ending on where one specific rule lives and why. If you reach for a service before step three, start again.
 3. *You sorted. Justify the extra $n \log n$.* Half a trillion against twenty-one million, the class-change test, and the counter-example where sorting makes it twenty times worse for the same answer.
-

Before you move on

- ☐ I said the sort decision out loud before coding on all four problems.
- ☐ I broke Merge Intervals by sorting on the wrong field and can say which key each problem wants.
- ☐ I wrote the sorted Two Sum, read the wrong indices, and can say both reasons the dict is better.
- ☐ I can give the four payoffs and the four times not to sort, unprompted.
- ☐ I can quote the closest-pair arithmetic and the ratio.
- ☐ I ran the gym modelling drill in fifteen minutes, including the missing noun and the aggregate boundaries.
- ☐ I can define entity, value object, aggregate root, repository and ubiquitous language in one sentence each.
- ☐ I answered all three questions above out loud.

Bubble, selection and insertion sort, and what each one teaches

DATA STRUCTURES AND ALGORITHMS

Bubble, selection and insertion sort, and what each one teaches

You can write all three and say the one case where insertion sort genuinely wins.

SYSTEM DESIGN

Common object-oriented interview questions

You can answer the standard OOP questions without reciting a textbook.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Write insertion sort. When would you actually use it?*
- *What is the difference between an abstract class and an interface?*

Bubble, selection and insertion sort, and what each one teaches

1 What this is, and why they ask it

Bubble sort, selection sort and insertion sort are the three simple sorting methods. All three take $O(n^2)$ time, all three fit in about eight lines, and all three sort in place — they rearrange the list you were given rather than building a new one. They are the sorts you can write from memory under pressure, and one of them is genuinely the fastest thing available in a real situation you will meet.

They ask about them for three reasons, and only one of them is “can you code”. The first is that insertion sort is a real answer: it is what every serious sorting library, Python’s included, drops into for small pieces of data, and knowing *why* is a proper answer to a proper question. The second is that these three differ in exactly the ways interviewers like to probe — number of comparisons against number of moves, best case against worst case, stable against unstable — so one small function opens four follow-ups. The third is that insertion sort is the first place you meet the idea of a **sorted region that grows**, which comes back on [day 053](#) as merging and on [day 114](#) as heapify. Write the three, and the rest of the phase has something to stand on.

2 The story

Meera has taught at a school in Trichy for nine years, and every December the annual day happens, and class six has to walk on stage in height order. Fifty children. It falls to whoever is free.

Her first year she did it the obvious way. She stood the children in a line as they came out, then walked from one end to the other looking at two children at a time. If the taller one was standing in front of the shorter one, she made the two of them change places, and then she looked at the next two. One walk down the line did not fix it, so she walked it again. And again. She kept walking until she got all the way down without having to move anybody, and then she knew it was right. It took her most of a lunch break, and she remembers being surprised how many times she had to send the same child backwards.

The next year Fathima did it, and Fathima did not walk up and down at all. She looked at the whole group, found the shortest child anywhere in it, and brought that child to the front. Then she looked at everybody who was left, found the shortest of those, and stood her second. Then third. Each child, once placed, never moved again. It took about the same amount of time, but the children liked it more, because nobody got shuffled back and forth.

The third year was different, because that year the children came out of the classroom one at a time instead of all at once. Anand took the first child and stood her by the wall. When the second came out, he held him against the first, and put him on the correct side of her. The third child walked along the little line that already existed, starting from the tall end, until he found the place where he was shorter than the one on his left and taller than the one on his right, and he stepped in there. Everybody after that shuffled one step along to make room.

And then it rained, and the children came out in the order they had been sitting in — which was more or less last year's height order already. Anand finished in under two minutes. Each child walked one or two places and stopped. The line was almost right before he started, and his way of doing it was the only one that noticed.

3 The idea in plain English

Three teachers, three methods, one job. Every one of them is a sorting method you can write in eight lines, and the differences between them are exactly what gets asked about.

Some vocabulary first, because all three share it.

A **comparison** is one question: *is this one bigger than that one?* A **swap** is exchanging two values so each sits where the other was. **In place** means you rearrange the list you were handed and use only a fixed amount of extra room — a couple of variables, not a second list. All three sorts today are in place. And the **sorted region** is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

Bubble sort — Meera's method

Walk the list from left to right. Look at each neighbouring pair. If the left one is bigger than the right one, swap them. When you get to the end, go back to the start and do it again. Stop when you manage a whole walk without swapping anything.

The name comes from what you see: on each walk, the biggest value still out of place gets carried all the way to the right end, like a bubble rising. After the first walk the largest value is definitely at the far right and never has to move again. After the second walk, the two largest are. So the sorted region grows at the **right** end, one value per walk.

The stopping rule matters and it is the part beginners drop. If you finish a walk having made zero swaps, the list is in order, and you can stop early. Without that check, bubble sort always does the full n walks even on already-sorted input.

Selection sort — Fathima's method

Look through the whole unsorted region and find the smallest value in it. Swap it into the first unsorted position. Now the sorted region at the **left** end is one longer. Repeat.

The distinguishing feature is the number of swaps. Selection sort does exactly $n - 1$ swaps, ever — one per position — no matter how jumbled the input is. It still does the same huge number of comparisons, but it barely moves anything. That matters when a “swap” is expensive: writing to flash memory, moving a large record, or anything where reading is cheap and writing is not.

Insertion sort — Anand’s method

Take the next unsorted value. Walk it backwards through the sorted region until you find where it belongs, shifting each larger value one place to the right to make room. Drop it in.

The sorted region grows at the **left** end, and — this is the difference that matters — the amount of work per value depends on how far back it has to walk. If it is already bigger than everything in the sorted region, it walks zero steps and stops immediately.

That is why Anand finished in two minutes when it rained. On input that is already nearly in order, every value walks one or two places and stops, so the total work is close to n rather than n^2 . A method whose running time gets better when the input is already partly ordered is called **adaptive**. Insertion sort is adaptive; bubble sort is adaptive only if you remember the early-stop check; selection sort is never adaptive, because it must scan the entire unsorted region to be sure it has found the smallest, and being nearly sorted does not shorten that scan by a single step.

Which one to actually use

Insertion sort. Not as a general sort — for anything large you call `sorted()` and say $O(n \log n)$, as on [day 051](#). But insertion sort is the right answer in three specific places, and being able to name them is what the question is really testing:

1. **Small inputs.** Under roughly 30 to 60 elements, insertion sort beats merge sort and quicksort in real time, because its per-element work is tiny and it does no allocation and no function calls. Python’s Timsort sorts short runs with a form of insertion sort for exactly this reason.
2. **Nearly-sorted input.** A list with a handful of elements out of place is sorted in close to $O(n)$.
3. **Data arriving one at a time.** If you must keep a list ordered while new values keep coming, the insertion step is the method. There is nothing to sort; there is one value to place.

Bubble sort is the one you should be able to write and then say honestly is never the right choice. It has selection sort’s comparison count and a far worse move count. It survives because it is easy to explain, not because anybody runs it.

4 The picture

Bubble sort, first walk over seven values. Watch the biggest value get carried right:

```

start  [ 5 , 1 , 4 , 2 , 8 , 0 , 2 ]
        ^^^^^
        [ 1 , 5 , 4 , 2 , 8 , 0 , 2 ]      5 > 1, swap
        ^^^^^
        [ 1 , 4 , 5 , 2 , 8 , 0 , 2 ]      5 > 4, swap
        ^^^^^
        [ 1 , 4 , 2 , 5 , 8 , 0 , 2 ]      5 > 2, swap
        ^^^^^
        [ 1 , 4 , 2 , 5 , 8 , 0 , 2 ]      5 < 8, leave it
        ^^^^^
        [ 1 , 4 , 2 , 5 , 0 , 8 , 2 ]      8 > 0, swap
        ^^^^^
        [ 1 , 4 , 2 , 5 , 0 , 8 , 2 ]      8 > 2, swap
        ^
after   [ 1 , 4 , 2 , 5 , 0 , 2 | 8 ]

```

LOCKED — the largest value is home after one walk

What to notice: one walk places exactly one value for certain. That is why you need $n-1$ walks, and it is where the n^2 comes from. Note also how far 5 travelled and how much shuffling that caused — bubble sort’s problem is not comparisons, it is moves.

Selection sort, same input. Watch the left end grow:

```

pass 0 [ 5 , 1 , 4 , 2 , 8 , 0 , 2 ]      smallest of all 7 is 0, at position 5
        swap positions 0 and 5
pass 1 [ 0 | 1 , 4 , 2 , 8 , 5 , 2 ]      smallest of the rest is 1, already in place
pass 2 [ 0 , 1 | 4 , 2 , 8 , 5 , 2 ]      smallest of the rest is 2, at position 3
pass 3 [ 0 , 1 , 2 | 8 , 5 , 4 , 2 ]      smallest of the rest is 2, at position 6
pass 4 [ 0 , 1 , 2 , 2 | 5 , 4 , 8 ]      smallest of the rest is 4
pass 5 [ 0 , 1 , 2 , 2 , 4 | 5 , 8 ]      smallest of the rest is 5, already in place
done   [ 0 , 1 , 2 , 2 , 4 , 5 , 8 ]
        sorted | unsorted

```

What to notice: six passes, six swaps. Total. Selection sort scans an enormous number of values and moves almost nothing — the exact opposite trade from bubble sort. Notice also that in pass 2 the 2 at position 3 jumped over the 2 at position 6, so the two equal values ended up in the opposite order from how they started. That is what “not stable” means, and it is the subject of [day 057](#).

Insertion sort, the same input. Watch each new value walk backwards:

```

sorted region | next value to place
step 1 [ 5 ] | 1 , 4 , 2 , 8 , 0 , 2      take 1, walk it past 5
step 2 [ 1 , 5 ] | 4 , 2 , 8 , 0 , 2      take 4, walk it past 5, stop at 1
step 3 [ 1 , 4 , 5 ] | 2 , 8 , 0 , 2      take 2, walk past 5 and 4, stop at 1
step 4 [ 1 , 2 , 4 , 5 ] | 8 , 0 , 2      take 8, ZERO steps -- already biggest
step 5 [ 1 , 2 , 4 , 5 , 8 ] | 0 , 2      take 0, walk all the way to the front
step 6 [ 0 , 1 , 2 , 4 , 5 , 8 ] | 2      take 2, walk past 8, 5, 4, stop after 2
done   [ 0 , 1 , 2 , 2 , 4 , 5 , 8 ]

```

What to notice: step 4 costs one comparison and no moves, and step 5 costs five of each. The work is *per value*, and it depends on the data. That single fact is the whole reason insertion sort is worth knowing — on nearly-sorted input every step looks like step 4.

The same three methods, side by side on the question that matters:

	comparisons	swaps / moves	best case	adaptive?
bubble sort	$\sim n^2 / 2$	up to $n^2 / 2$	$O(n) *$	only with the flag
selection sort	exactly $n^2 / 2$	exactly $n - 1$	$O(n^2)$	no, never
insertion sort	up to $n^2 / 2$	up to $n^2 / 2$	$O(n)$	yes, naturally

* with the early-stop check, and only on input that is already sorted

What to notice: three sorts with the same complexity class are not three of the same thing. The column that separates them in practice is the one on the right.

5 The code, built step by step

Bubble sort, one walk at a time

Start with the inner walk — one pass over neighbouring pairs:

```
def one_pass(nums: list[int]) → bool:
    """One left-to-right walk. Returns True if anything was swapped."""
    swapped = False
    for i in range(len(nums) - 1):
        if nums[i] > nums[i + 1]:
            nums[i], nums[i + 1] = nums[i + 1], nums[i]
            swapped = True
    return swapped
```

Note `len(nums) - 1`, not `len(nums)`. The loop reads `nums[i + 1]`, so the last valid `i` is one before the end. Getting that wrong is the single commonest bug in bubble sort and it raises a real error, shown in §7. Note also `nums[i], nums[i + 1] = nums[i + 1], nums[i]` — Python evaluates the right side fully before assigning, so this is a genuine swap with no temporary variable.

Now repeat the walk until nothing moves:

```
def bubble_sort(nums: list[int]) → None:
    """Sorts in place. Stops as soon as a full pass makes no swaps."""
    n = len(nums)
    for end in range(n - 1, 0, -1):
        # the last unsorted position, shrinking
        swapped = False
        for i in range(end):
            # never look past the sorted tail
            if nums[i] > nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]
                swapped = True
        if not swapped:
            # already in order: stop
            return
```

Two improvements over the naive version, and both get asked about. `range(end)` shrinks the walk each pass, because after pass one the last value is known to be correct and re-checking it is wasted work. The `swapped` flag is what makes bubble sort $O(n)$ on already-sorted input rather than $O(n^2)$ — if a whole walk changes nothing, nothing further can change.

Selection sort

```
def selection_sort(nums: list[int]) → None:
    """Sorts in place. Exactly len(nums) - 1 swaps, always."""
    n = len(nums)
    for start in range(n - 1):
        smallest = start
        for i in range(start + 1, n):
            if nums[i] < nums[smallest]:
                smallest = i
        nums[start], nums[smallest] = nums[smallest], nums[start]
```

The important line is `smallest = i`. You track the *position* of the smallest value, not the value itself, because you need to know where to swap from. Beginners write `smallest = nums[i]` and then cannot complete the swap.

`range(n - 1)` rather than `range(n)`: once the first `n-1` positions hold the right values, the last one has nothing left to compete with. One free pass saved.

Insertion sort

This is the one to learn properly. Build the inner walk first:

```
def place(nums: list[int], j: int) → None:
    """Walk nums[j] backwards into the sorted region nums[0:j]."""
    value = nums[j]
    i = j - 1
    while i ≥ 0 and nums[i] > value:
        nums[i + 1] = nums[i]
        i -= 1
    nums[i + 1] = value
```

Read the loop condition in the right order. `i ≥ 0` must come first, because Python's `and` stops evaluating as soon as the left side is false — so when `i` reaches `-1` the comparison `nums[i]` is never attempted. Swap the two conditions round and you get a silent wrong answer, because `nums[-1]` is the last element in Python rather than an error. That trap is in §7 with its output.

Notice there is no swapping here at all. Each larger value is copied one step right, and the value being placed is written once, at the end. That is half the moves of a swap-based version.

The full sort is that fragment in a loop:

```
def insertion_sort(nums: list[int]) → None:
    """Sorts in place. O(n) on nearly-sorted input, O(n^2) worst case."""
    for j in range(1, len(nums)):
        value = nums[j]
        i = j - 1
        while i ≥ 0 and nums[i] > value:
            nums[i + 1] = nums[i]
            i -= 1
        nums[i + 1] = value
```


`range(1, len(nums))` starts at 1, because a single element is already a sorted region of size one. That is the invariant, in the sense of [day 028](#): **before each step, `nums[0:j]` holds the first `j` values in sorted order**. Say that sentence out loud when you write this in an interview; it is the thing that proves you understand it rather than remember it.

The condition that keeps it stable

`nums[i] > value` uses a strict `>`. If you write `>=` instead, an incoming value walks *past* values equal to it, and two equal elements come out in the opposite order from how they went in. The sort still returns a correctly ordered list, so nothing fails — it silently stops being stable. One character.

Binary insertion sort, and why it does not help much

An obvious-looking improvement: instead of walking backwards to find the spot, binary search for it.

```
from bisect import bisect_right

def binary_insertion_sort(nums: list[int]) → None:
    for j in range(1, len(nums)):
        value = nums[j]
        pos = bisect_right(nums, value, 0, j) # where it belongs, in  $O(\log j)$ 
        nums[pos + 1 : j + 1] = nums[pos:j]   # shift the block right by one
        nums[pos] = value
```

This cuts comparisons from $O(n^2)$ to $O(n \log n)$ — and the running time stays $O(n^2)$, because you still have to *shift* every larger element to make room, and shifting is the expensive part. Worth knowing as a follow-up answer: it is a real improvement only when comparing two elements is far more expensive than moving them, such as sorting long strings by content.

The complete file

```
"""The three quadratic sorts, in place, with the properties that distinguish them."""
from bisect import bisect_right

def bubble_sort(nums: list[int]) → None:
    """Repeatedly swap out-of-order neighbours.  $O(n^2)$ ;  $O(n)$  if already sorted.

    Stable. In place. Never the right choice in production -- learn it to explain it.
    """
    for end in range(len(nums) - 1, 0, -1):
        swapped = False
        for i in range(end):
            if nums[i] > nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]
                swapped = True
        if not swapped:
            return

def selection_sort(nums: list[int]) → None:
    """Repeatedly move the smallest remaining value to the front. Always  $O(n^2)$ .

    NOT stable. Exactly  $n - 1$  swaps -- the fewest writes of any comparison sort,
    which is the one situation where it wins.
    """
    n = len(nums)
    for start in range(n - 1):
        smallest = start
        for i in range(start + 1, n):
            if nums[i] < nums[smallest]:
                smallest = i
        if smallest != start:
            # skip the pointless self-swap
            nums[start], nums[smallest] = nums[smallest], nums[start]

def insertion_sort(nums: list[int]) → None:
    """Grow a sorted prefix by walking each new value back into it.

    Stable (because the comparison is strict >). In place.  $O(n)$  on nearly-sorted
    input, which is why real libraries use it for small and almost-ordered runs.
    Invariant: before step  $j$ ,  $nums[0:j]$  holds the first  $j$  values in sorted order.
    """
    for j in range(1, len(nums)):
        value = nums[j]
        i = j - 1
        while i ≥ 0 and nums[i] > value:
            nums[i + 1] = nums[i]
            i -= 1
        nums[i + 1] = value

def binary_insertion_sort(nums: list[int]) → None:
    """Insertion sort with a binary search for the position.

     $O(n \log n)$  comparisons but still  $O(n^2)$  moves -- worth it only when a
    comparison costs far more than a move.
    """
    for j in range(1, len(nums)):
        value = nums[j]
        pos = bisect_right(nums, value, 0, j)
        nums[pos + 1 : j + 1] = nums[pos:j]
        nums[pos] = value

def insert_into_sorted(sorted_nums: list[int], value: int) → None:
```

```

"""The real use of the insertion idea: keep a list ordered as values arrive.

O(n) per value because of the shift, but there is no sort here at all --
one value, one placement.
"""
i = len(sorted_nums) - 1
sorted_nums.append(value)
while i ≥ 0 and sorted_nums[i] > value:
    sorted_nums[i + 1] = sorted_nums[i]
    i -= 1
sorted_nums[i + 1] = value

if __name__ == "__main__":
    from copy import deepcopy

    data = [5, 1, 4, 2, 8, 0, 2]

    for sort in (bubble_sort, selection_sort, insertion_sort, binary_insertion_sort):
        nums = deepcopy(data)
        sort(nums)
        print(f"{sort.__name__:24} {nums}")
    # bubble_sort           [0, 1, 2, 2, 4, 5, 8]
    # selection_sort        [0, 1, 2, 2, 4, 5, 8]
    # insertion_sort        [0, 1, 2, 2, 4, 5, 8]
    # binary_insertion_sort  [0, 1, 2, 2, 4, 5, 8]

    # the adaptive difference, made visible by counting shifts
    def count_shifts(nums: list[int]) → int:
        shifts = 0
        for j in range(1, len(nums)):
            value = nums[j]
            i = j - 1
            while i ≥ 0 and nums[i] > value:
                nums[i + 1] = nums[i]
                i -= 1
            shifts += 1
            nums[i + 1] = value
        return shifts

    print(count_shifts(list(range(1000))))           # 0      -- already sorted
    print(count_shifts(list(range(1000, 0, -1))))    # 499500 -- exactly reversed
    nearly = list(range(1000))
    nearly[100], nearly[103] = nearly[103], nearly[100]
    print(count_shifts(nearly))                      # 6      -- nearly sorted

    # keeping a list ordered as data arrives
    running: list[int] = []
    for x in [7, 3, 9, 1, 5]:
        insert_into_sorted(running, x)
        print(running)
    # [7] / [3, 7] / [3, 7, 9] / [1, 3, 7, 9] / [1, 3, 5, 7, 9]

```

6 What it costs

Counting the comparisons

Take selection sort, because it is the easiest to count exactly and the count is the same for all three in the worst case.

The outer loop runs with `start = 0, 1, 2, ..., n-2`. For `start = 0`, the inner loop runs from 1 to `n-1`, which is `n-1` comparisons. For `start = 1` it is `n-2`. And so on down to 1.

$$(n-1) + (n-2) + (n-3) + \dots + 2 + 1 = n(n-1)/2$$

$n = 10 \rightarrow 45$ comparisons
 $n = 100 \rightarrow 4,950$
 $n = 1,000 \rightarrow 499,500$
 $n = 10,000 \rightarrow 49,995,000$

That sum, $n(n-1)/2$, is $(n^2 - n)/2$. Drop the constant and the lower-order term and it is $O(n^2)$. Ten times the data is a hundred times the work, and that is the sentence to say out loud.

The three, priced separately

The complexity class is the same. The actual work is not.

	comparisons	element moves	best case
bubble sort	up to $n(n-1)/2$	up to $3 \times n(n-1)/2$	$O(n)$ with the flag
selection sort	exactly $n(n-1)/2$	exactly $3(n-1)$	$O(n^2)$ always
insertion sort	up to $n(n-1)/2$	up to $n(n-1)/2$	$O(n)$

* a swap is three writes: `temp = a; a = b; b = temp.`
 Insertion sort SHIFTS rather than swaps, so it writes once per displaced element.

At $n = 1,000$ on reversed input:

bubble sort	499,500 comparisons	+	1,498,500 writes
selection sort	499,500 comparisons	+	2,997 writes
insertion sort	499,500 comparisons	+	500,499 writes

Selection sort does five hundred times fewer writes than bubble sort for the same answer. If you are sorting records of two kilobytes each, or writing to storage where each write wears the hardware out, that column is the only one that matters. This is the honest case for selection sort, and it is the one thing it is good at.

The adaptive case, which is why insertion sort survives

$n = 1,000$			
already sorted	insertion sort:	999 comparisons, 0 moves	$\rightarrow O(n)$
	selection sort:	499,500 comparisons	$\rightarrow O(n^2)$
10 elements out of place, each by 3 positions			
	insertion sort:	~1,030 comparisons, ~30 moves	$\rightarrow O(n)$
	selection sort:	499,500 comparisons	$\rightarrow O(n^2)$

Nearly-sorted input makes insertion sort roughly **five hundred times faster** than selection sort at $n = 1,000$, and selection sort does not get one comparison cheaper. That gap is the answer to “when would you actually use it”.

Against the real sorts

n = 1,000	insertion sort	~ 500,000 operations	
	merge sort	~ 1,000 x 10 = 10,000 operations	→ 50x faster
n = 30	insertion sort	~ 30 x 29 / 2 = 435 operations, no allocation, no calls	
	merge sort	~ 30 x 5 = 150 "operations", plus 30 list allocations and ~60 function calls	
			→ insertion sort WINS in real time

The crossover in real implementations sits somewhere between 30 and 60 elements, depending on the language. Below it, n^2 with a tiny constant beats $n \log n$ with a large one. Above it, nothing saves you. Python's Timsort uses 64 as its threshold and sorts anything shorter with a binary insertion sort — the full story is on [day 057](#).

Space

All three are **O(1) extra space**. They mutate the list they are given and hold at most a couple of loop variables. That is worth saying explicitly in an interview, because merge sort tomorrow is not, and the contrast is one of the standard follow-ups.

7 The traps

The real error: walking one step too far

```
def bubble_broken(nums):
    for _ in range(len(nums)):
        for i in range(len(nums)):           # ← should be len(nums) - 1
            if nums[i] > nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]

bubble_broken([5, 1, 4])
```

```
Traceback (most recent call last):
  File "day52.py", line 8, in <module>
    bubble_broken([5, 1, 4])
    ~~~~~^~~~~~
  File "day52.py", line 4, in bubble_broken
    if nums[i] > nums[i + 1]:
        ~~~~~^~~~~~
IndexError: list index out of range
```

The rule, and it is worth saying as a rule: **if the body reads `nums[i + 1]`, the loop stops at `len(nums) - 1`**. Any time you compare an element with its neighbour, the range shrinks by one.

The near-miss that gives a wrong answer with no error

This is the one that costs interviews, because nothing crashes.

```
def insertion_broken(nums):
    for j in range(1, len(nums)):
        value = nums[j]
        i = j - 1
        while nums[i] > value and i ≥ 0:      # ← conditions in the wrong order
            nums[i + 1] = nums[i]
            i -= 1
        nums[i + 1] = value
    return nums

print(insertion_broken([3, 1, 2]))
```

```
[1, 2, 3]
```

It worked. Run it on the input that exposes it — one where the smallest value is not first:

```
print(insertion_broken([2, 3, 1]))
```

```
[3, 1, 2]
```

Wrong, and no exception. Here is why. When `i` reaches `-1`, Python evaluates `nums[-1] > value` first, and `nums[-1]` is the **last element of the list**, not an error. That comparison happens to be true, so the loop runs one more time, writes into `nums[0]`, and `i` becomes `-2`. Then `nums[-2]` is the second-to-last element. The walk wraps around the end of the list and corrupts it.

`i ≥ 0` must come first. Python's `and` short-circuits — it never evaluates the right side if the left side is false — and that is exactly what protects the lookup. **When one condition guards another, the guard goes first.**

The near-miss: bubble sort without the flag

```
def bubble_no_flag(nums):
    for end in range(len(nums) - 1, 0, -1):
        for i in range(end):
            if nums[i] > nums[i + 1]:
                nums[i], nums[i + 1] = nums[i + 1], nums[i]
```

Correct output, always. But hand it a sorted list of a thousand elements and it performs 499,500 comparisons to discover that there was nothing to do. With the flag it performs 999 and returns. The interviewer will ask “what is the best case?” and the honest answer for this version is $O(n^2)$, not $O(n)$. One boolean is the difference between the two answers.

The trap: tracking the value instead of the position

```
def selection_broken(nums):
    for start in range(len(nums) - 1):
        smallest = nums[start] # ← the value, not the position
        for i in range(start + 1, len(nums)):
            if nums[i] < smallest:
                smallest = nums[i]
        nums[start], nums[smallest] = nums[smallest], nums[start]
    return nums

print(selection_broken([5, 1, 4]))
```

[illegible]

You cannot swap using a value; you need to know where it lives. And when the values happen to be small non-negative integers this bug does *not* raise — it silently swaps the wrong pair. Track positions.

The trap: `>=` instead of `>` quietly loses stability

```
while i ≥ 0 and nums[i] ≥ value:      # ← was >
```

The list still comes out correctly ordered, so no test on the numbers catches it. But sort a list of `(name, score)` pairs by score and the names within each score come out reversed. Nothing fails until somebody sorts by two fields in two passes, as on [day 051](#), and gets an order they cannot explain. Strict `>` keeps equal elements in their original order. One character, and it is worth pointing at unprompted in an interview.

The trap: sorting in place and then using the return value

```
nums = [3, 1, 2]
result = insertion_sort(nums)
print(result[0])
```

```
Traceback (most recent call last):
  File "day52.py", line 3, in <module>
    print(result[0])
    ~~~~~^
TypeError: 'NoneType' object is not subscriptable
```

All three functions here sort in place and return `None`, which is the same convention `list.sort()` follows. Decide which you are writing, say so in the docstring, and be consistent. If the interviewer expects a returned list, `return nums` at the end and say you are doing it.

8 In the interview

How it gets asked

- “Write insertion sort.” — asked plainly, often as a warm-up before something harder, and the real test is whether you can finish it without an off-by-one and then say something intelligent about it.
- “When would you actually use insertion sort? It’s $O(n^2)$.” — the real question. Three answers: small inputs, nearly-sorted inputs, and data arriving one at a time.
- “You have three $O(n^2)$ sorts. Why does anyone distinguish them?” — comparisons against moves, best case, stability.
- “Your data is almost sorted — a few elements are out of place. What do you use?” — insertion sort, and the reason has to be the adaptive argument, not a preference.
- “Which of these is stable?” — insertion and bubble are; selection is not, and you should be able to give the two-element example that shows it.

What to say out loud, in the first ninety seconds

1. **Name the shape before you write.** “Insertion sort grows a sorted region at the front. For each new value I walk it backwards through that region, shifting larger values right, until it lands.”
2. **State the invariant.** “Before step j , the first j elements are the original first j values, in sorted order. That is what I am maintaining.”
3. **Flag the guard as you write it.** “The $i \geq 0$ has to come before the comparison, because `nums[-1]` is the last element in Python rather than an error — it would wrap round and corrupt the list.”
4. **Say shift, not swap.** “I’m shifting rather than swapping, so each displaced element is written once instead of three times, and the value being placed is written once at the end.”
5. **Give the cost with the case attached.** “ $O(n^2)$ worst case, $O(n)$ best case, $O(1)$ extra space, and it is stable because the comparison is strict.”

The follow-ups

“It’s $O(n^2)$. When would you actually use it?” Three situations, and the first is the one that matters most, because every real sorting library does it. On small inputs insertion sort genuinely beats merge sort and quicksort in wall-clock time — the constant factor is tiny, there is no allocation, no recursion and no function-call overhead, so somewhere between thirty and sixty elements the n^2 with a small constant wins against $n \log n$ with a large one. That is not a theoretical point: Python’s Timsort splits the list into runs and sorts anything under sixty-four elements with a binary insertion sort, and the same trick is in the C++ and Java standard libraries. The second situation is nearly-sorted input. Insertion sort is adaptive: each value walks backwards only as far as it needs to, so if ten elements out of a thousand are three places out of position, the whole sort is about a thousand comparisons and thirty moves — order n , not n^2 . That comes up more than people expect, because a log file that is almost in timestamp

order, or a list that was sorted and then had a few items updated, is a real thing. The third is when the data arrives one at a time and you must keep the collection ordered as it does. Then there is no sort at all — there is one value and one placement, and the insertion step is exactly the operation you need. What I would not claim is that it is a general-purpose sort. For a large unordered list I call `sorted()` and say $O(n \log n)$.

“Which of the three are stable, and why does it matter?” Insertion sort and bubble sort are stable; selection sort is not. Stable means two elements that compare equal come out in the order they went in. Insertion sort is stable because the walk stops the moment it meets a value that is not strictly greater — an incoming element never passes an equal one, so equal elements keep their relative order. Bubble sort is stable for the same reason: it only swaps when the left element is strictly greater. Selection sort is not, and the counter-example is small: take `[2a, 2b, 1]`, where the two twos are distinct records that compare equal. The first pass finds `1` as the smallest and swaps it with position zero, which throws `2a` to the back, and the result is `[1, 2b, 2a]` — the twos have swapped order. The long-distance swap is what breaks it, and no version of selection sort avoids that without giving up its one advantage, the $n-1$ swap count. It matters because of multi-pass sorting: if you sort by name and then by department, only a stable sort leaves the names ordered within each department. That is the standard way to sort by several keys, and it silently produces the wrong order with an unstable sort — no error, just an answer you cannot explain.

“Why would anyone use selection sort at all?” For exactly one property: it performs the fewest writes of any comparison sort. Exactly $n-1$ swaps, which is $3(n-1)$ element writes, regardless of the input. Bubble sort on reversed data of a thousand elements does about a million and a half writes; selection sort does two thousand nine hundred and ninety-seven. That matters whenever a write is much more expensive than a read: sorting large records in place where each move copies kilobytes, or writing to flash memory or EEPROM, where each cell tolerates a limited number of erase cycles and the hardware physically wears out. Outside of that, no — it is strictly worse than insertion sort, because it has the same comparison count, is not adaptive, and is not stable. I would name the write-count argument and then say plainly that on a normal list in memory I would not use it.

A model answer

“Insertion sort keeps a sorted region at the front of the list and grows it by one element at a time. For each new value I copy it out, then walk backwards through the sorted region shifting every larger element one place to the right, and when I find one that isn’t larger — or run off the front — I drop the value into the gap.

```
def insertion_sort(nums: list[int]) → None:
    for j in range(1, len(nums)):
        value = nums[j]
        i = j - 1
        while i ≥ 0 and nums[i] > value:
            nums[i + 1] = nums[i]
            i -= 1
        nums[i + 1] = value
```

Two things in there I’d point at. The `i ≥ 0` comes first deliberately — and short-circuits, and if the comparison ran first then at `i = -1` Python would read the *last* element of the list rather than raising, and the walk would wrap round and corrupt it. And the comparison is a strict `>`, which is what makes this stable: an incoming value never walks past a value equal to it, so equal elements keep their original order. With `≥` it still sorts correctly but silently stops being stable, and that breaks multi-pass sorting by two keys.

The invariant is: before step `j`, the first `j` elements are the original first `j` values in sorted order. Cost is $O(n^2)$ worst case — reversed input, where every value walks the whole way back, giving $n(n-1)/2$ comparisons, about half a million at a thousand elements. Best case is $O(n)$: on already sorted input each value fails the comparison immediately and does zero moves, so it’s $n-1$ comparisons. Space is $O(1)$ — it’s in place, unlike merge sort.

As for when I’d use it: not as a general sort, I’d call `sorted()` for that. But three real cases. Small inputs, under about fifty elements, where the tiny constant factor beats an $O(n \log n)$ sort with allocation and recursion — Python’s Timsort does exactly this, it insertion-sorts runs shorter than sixty-four. Nearly-sorted input, because it’s adaptive: ten elements a few places out of a thousand costs about a thousand comparisons, not half a million. And streaming data, where I need to keep a list ordered as values arrive — then the insertion step is the whole operation and there’s no sort to speak of.”

9 Recall card

- All three are $O(n^2)$, in place, $O(1)$ space — and they differ where it counts. Bubble: swap out-of-order neighbours, largest sinks right, needs the `swapped` flag to be $O(n)$ on sorted input. Selection: find the smallest, swap it forward — **exactly $n-1$ swaps**, never adaptive, **not stable**. Insertion: walk each value back into the sorted prefix, shifting as you go.

- **Insertion sort is the only one with a real answer to “when would you use it”:** small inputs (under ~ 50 ; Timsort’s threshold is 64), nearly-sorted input (**adaptive** — $O(n)$), and values arriving one at a time.
- **The invariant to say out loud:** *before step j , `nums[0:j]` holds the first j values in sorted order.* And it **shifts rather than swaps** — one write per displaced element instead of three.
- **Two one-character traps.** `while i  $\geq$  0 and nums[i] > value` — the guard must come first or `nums[-1]` wraps round and corrupts the list, silently. And `>` not `$\geq$` , or you lose **stability** with no visible failure.
- **The arithmetic:** $n(n-1)/2 = 499,500$ comparisons at $n = 1,000$. Writes on reversed input: bubble 1,498,500 · insertion 500,499 · **selection 2,997** — selection sort’s only virtue, and it matters for flash memory and large records.

1 What this is, and why they ask it

There is a fixed set of about a dozen object-oriented questions that get asked in almost every interview at almost every product company. What are the four pillars. Abstract class or interface. Overloading or overriding. Composition or inheritance. Encapsulation or abstraction. They are short questions with short answers, and they come in a rapid-fire block, usually in the first fifteen minutes of a design round or at the end of a coding round when there is time left over.

They ask them because they are cheap to ask and they separate people quickly. Everybody has read the same textbook answer, so the definition itself carries no information — the interviewer learns nothing from hearing “an interface is a contract”. What they are listening for is the second half of your answer: a situation where the choice actually mattered, and what went wrong when it was made badly. You have spent nine days building that second half. [Day 043](#) through [day 051](#) gave you encapsulation, inheritance, polymorphism, abstraction, composition, diagrams and domain modelling. Today is about turning that into answers that fit in forty-five seconds and do not sound memorised.

2 The story

Vignesh took his driving test in Coimbatore on a Tuesday morning in February, and he failed it, and the reason he failed is the thing he still tells people about.

He had studied properly. He had the little booklet they give you, and he had been through it so many times that he could recite whole lines of it. His cousin had tested him the night before and he got every single one right.

The examiner was a tired man called Balan who had been doing this for eighteen years. Before they even started the engine, sitting in the parked car with the door open and one foot on the ground, Balan asked him one question. “When do you use the handbrake?”

Vignesh said the sentence from the booklet, more or less word for word: “The handbrake is used to hold the vehicle stationary when it is parked.”

Balan nodded and wrote something down and did not say anything about it. They drove for about twenty minutes. Then, on the slope going up to the flyover near Gandhipuram, the light turned red and Vignesh stopped halfway up the rise. When it went green he lifted his foot off the clutch and the car rolled backwards, a foot, maybe a foot and a half, and the auto behind him leaned on his horn and kept leaning on it, and Vignesh’s hands went completely white on the wheel.

Balan did not say anything then either. He waited until they were back and parked and the engine was off. Then he said: “You know what the handbrake is. You don’t know what it’s for.”

Six weeks later Vignesh went back, and by chance he got Balan again, and Balan asked him the same question in the same parked car. This time Vignesh said: “On a hill, mostly. You hold the car with it, bring the clutch up to where it starts to bite, and then let it off — otherwise you roll back into whoever is behind you. That happened to me on the flyover in February and I have not forgotten it.”

Balan laughed, which Vignesh was not expecting at all, and made a mark on his sheet, and they drove. He passed. The words in the booklet had not changed. What had changed was that he had rolled backwards into a horn.

3 The idea in plain English

Every one of the standard object-oriented questions is a handbrake question. There is a booklet answer, which everybody has, and there is an answer from having rolled backwards, which almost nobody gives. The interviewer is listening for the second one.

So learn every answer in three parts, and say all three:

1. **The definition**, in one sentence. You still need it. It takes five seconds.
2. **The situation** where the difference actually decided something.
3. **The cost** — what goes wrong if you choose the other one, ideally with a number.

That is roughly forty-five seconds of speech. Any longer and you are lecturing; any shorter and you have only given the booklet.

Below are the questions, with all three parts. Each one leans on a day you have already done, so these are not new ideas — they are the same ideas compressed into interview shape.

“What are the four pillars of OOP?”

Definition. Encapsulation, abstraction, inheritance, polymorphism.

Say them, then immediately say which two do the work. **Encapsulation** is keeping the data and the rules that protect it in one place so the rule cannot be broken from outside (day 045). **Abstraction** is exposing what something can do and hiding how (day 048). **Inheritance** is declaring that every B is an A (day 046). **Polymorphism** is calling one method on several types and getting each type’s behaviour (day 047).

Situation. In real code you use encapsulation and polymorphism constantly and inheritance sparingly. Say that out loud — it is the sentence that shows you have written code rather than read about it.

“Encapsulation or abstraction — what is the difference?”

This is the one candidates muddle most, because the textbook definitions overlap.

Definition. Encapsulation is about **protecting state**: the balance cannot go negative because the only way to change it runs a check. Abstraction is about **hiding mechanism**: the caller says “charge this card” and does not learn which payment provider ran it.

The one-liner that separates them: *encapsulation hides data so a rule cannot be broken; abstraction hides implementation so it can be replaced.*

Cost. Break encapsulation and the same rule appears at nine call sites and the tenth one ships without it. Break abstraction and swapping payment providers touches sixty files instead of one.

“Abstract class or interface — which and why?”

Definition. An **abstract class** is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in. You cannot create one directly. An **interface** is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

The rule that decides it. An abstract class says “**is a kind of**”; an interface says “**is able to**”. A `SavingsAccount` *is a kind of* `Account` — abstract class. A `SavingsAccount` *is able to be* `Exportable` — interface.

Situation. Reach for an abstract class when there is genuine shared code that every subclass needs and you want to write it once: a base `Report` that handles the header, the footer and the timestamp, and leaves `body()` abstract. Reach for an interface when several unrelated types need to be usable in the same slot: `PaymentGateway` implemented by Razorpay, Stripe and an in-memory fake, which have nothing else in common.

Cost. A class can inherit from only one abstract class in Java and C#, and that slot is spent forever. It can implement any number of interfaces. So an abstract class is the expensive choice, and the honest default is: **inherit interfaces, compose implementations** (day 049).

In Python. There is no `interface` keyword. You get the same two things with `abc.ABC` plus `@abstractmethod` for the abstract-class case, and `typing.Protocol` for the interface case — and `Protocol` needs no inheritance at all, which is closer to what an interface really means. The mechanics are in §5.

“Overloading or overriding?”

Definition. Overriding is a subclass replacing a method it inherited — same name, same arguments, different behaviour, decided when the program runs. **Overloading** is several methods with the same name and different argument types in the same class, decided when the program is compiled.

Python has no overloading. A second `def` with the same name simply replaces the first. Say this plainly — interviewers ask it precisely to see whether you know your own language:

```
class Bill:
    def total(self, items): ...
    def total(self, items, discount): ...    # this REPLACES the one above
```

The Python answers are default arguments, `*args`, or `functools.singledispatch` when you genuinely need behaviour to vary by argument type.

Cost. Nothing warns you. The first `total` is gone, and every call to it fails at runtime with a `TypeError` about argument count.

“Composition or inheritance?”

Definition. Inheritance welds a variant into the type. Composition holds it as an object you can swap.

Situation and the number. This is the one where arithmetic wins the point. Three vehicle types $\times$ two fuel types $\times$ two trailer options $\times$ three ownership models is $3 \times 2 \times 2 \times 3 = 36$ classes with inheritance, and $3 + 2 + 2 + 3 = 10$ with composition. Inheritance multiplies axes; composition adds them. A new fuel type is six new classes against one.

When inheritance is still right. Exception trees, abstract base classes used purely as contracts, documented framework hooks, and genuine single-axis specialisation — all cases where the parent has almost no implementation to inherit.

“What is polymorphism? Give an example.”

Do not define it. **Show it.** Write the `if` chain, then delete it:

```
if vehicle.kind == "car":    fee = hours * 30
elif vehicle.kind == "bike": fee = hours * 10
elif vehicle.kind == "truck": fee = hours * 60
```

```
fee = vehicle.fee_for(hours)    # each type knows its own rate
```

Cost, in edits. Adding a van is one new file and one line of wiring, against four edits to code that already works and is already tested. And the type tag has to go, or somebody rebuilds the switch next quarter.

“`==` or `is`? And what about `__hash__`?”

Definition. `is` asks whether two names point at the same object in memory. `==` asks whether two objects are equal by whatever rule the class defines in `__eq__`. Default `__eq__` falls back to identity, so without one, `==` and `is` do the same thing.

The rule that gets tested. If you define `__eq__` you must define `__hash__`, and both must be built from fields that never change. Python quietly sets `__hash__` to `None` when you define `__eq__` without it, so the object stops being usable as a dictionary key at all.

Cost. Hash on a mutable field, put the object in a set, then change that field, and the object becomes unfindable in the set it is sitting in. No error — it is simply looked for in the wrong place.

“Static method, class method, or instance method?”

Definition. An **instance method** takes `self` and needs one particular object. A **class method** takes `cls`, belongs to the class, and is usually an alternative constructor. A **static method** takes neither and is a plain function that lives inside the class for tidiness.

```
class Money:
    def __init__(self, paise: int) → None:
        self.paise = paise

    def add(self, other: "Money") → "Money":          # instance: needs this object
        return Money(self.paise + other.paise)

    @classmethod
    def from_rupees(cls, rupees: float) → "Money": # class: another way to build one
        return cls(round(rupees * 100))

    @staticmethod
    def is_valid_currency(code: str) → bool:          # static: needs nothing
        return code in {"INR", "USD", "EUR"}
```

The tell. If a static method is doing real work with the object’s data, it is an instance method in disguise, and that is usually a symptom of the anaemic model ([day 044](#)).

“What is the diamond problem?”

Definition. Class D inherits from B and C, both of which inherit from A. If B and C both override a method, which one does D get? Java forbids multiple inheritance of classes to avoid the question entirely. Python allows it and answers it with a defined order.

The Python answer. The **method resolution order** — `D.__mro__` — is computed by a rule called C3 linearisation, and `super()` follows that order rather than jumping straight to a parent. Show the mechanics if asked (§5), then say the honest thing: **in production you avoid the situation**. Use one base class and compose the rest, or use mixins that touch disjoint methods.

4 The picture

The shape of a good answer, which is the same shape every time:

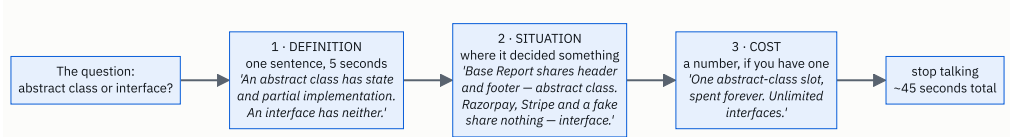


FIGURE 52.1

What to notice: almost every candidate delivers box one and stops. Boxes two and three are the whole difference, and they are short. The last box is a real instruction — over-answering a rapid-fire question is its own failure mode.

The two Python mechanisms that answer “abstract class or interface”, side by side:

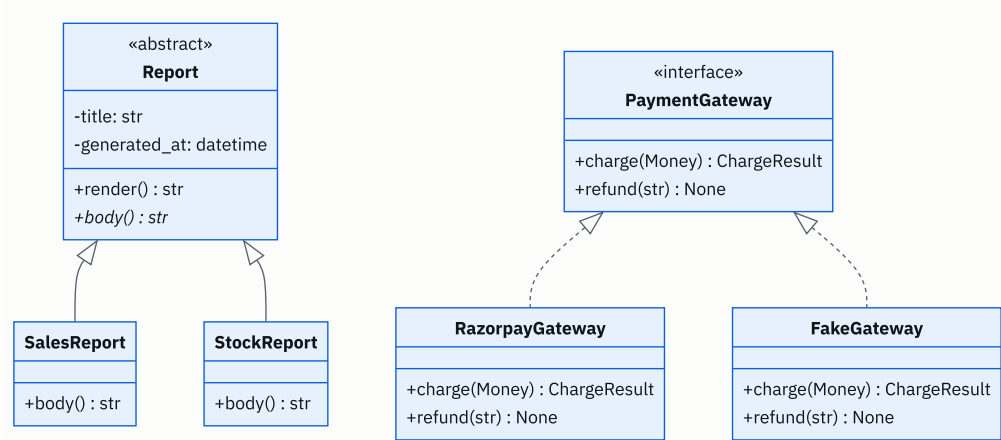


FIGURE 52.2

What to notice: the abstract class carries fields and a working `render()` — the solid triangle means the subclasses inherit real code. The interface carries nothing but signatures, and the dashed triangle means “implements”. `RazorpayGateway` and `FakeGateway` are not kinds of anything; they are merely both *able to* charge, and that is why the interface is right for them.

Where each question lives on the map you have already drawn:

STATE -----		BEHAVIOUR	
encapsulation "cannot break the rule"	abstraction "can replace the mechanism"	inheritance "is a kind of"	polymorphism "one call, many types"
private fields @property invariants	interfaces Protocol adapters	abstract classes super(), MRO the diamond	overriding duck typing singledispatch
day 045	day 048	day 046	day 047

What to notice: the twelve questions are four ideas asked four ways each. If you can place a question on this line, you already know which day’s argument answers it.

5 How it actually works

Abstract base classes in Python

```
from abc import ABC, abstractmethod

class Report(ABC):
    def __init__(self, title: str) → None:
        self.title = title                    # state: interfaces cannot do this

    def render(self) → str:
        return f"=== {self.title} ===\n{self.body()}\n--- end ---"                # shared implementation, written once

    @abstractmethod
    def body(self) → str:                     # subclasses must supply this
        ...
```

The enforcement is real and it happens early:

```
class BrokenReport(Report):
    pass
```

```
BrokenReport("Q3")
```

```
Traceback (most recent call last):
  File "day52.py", line 15, in <module>
    BrokenReport("Q3")
TypeError: Can't instantiate abstract class BrokenReport without an implementation
for abstract method 'body'
```

That failure happens at **construction**, not when `body()` is eventually called in production. It is the strongest argument for `ABC` over a bare base class that raises `NotImplementedError` in the body: the bad object never comes into existence.

This is exactly how Python's own `collections.abc` works. `MutableMapping` gives you `get`, `pop`, `setdefault`, `items`, `keys`, `update` and more for free, and demands five methods from you: `__getitem__`, `__setitem__`, `__delitem__`, `__iter__` and `__len__`. Django's `Model` and `BaseCommand` are the same idea at a larger scale — a partly-built class with named holes.

Interfaces in Python: `typing.Protocol`

```
from typing import Protocol

class PaymentGateway(Protocol):
    def charge(self, amount_paise: int, token: str) → str: ...
    def refund(self, charge_id: str) → None: ...
```

```
class RazorpayGateway:                    # note: inherits from NOTHING
    def charge(self, amount_paise: int, token: str) → str: ...
    def refund(self, charge_id: str) → None: ...

def checkout(gateway: PaymentGateway, amount_paise: int, token: str) → str:
    return gateway.charge(amount_paise, token)
```

`RazorpayGateway` satisfies `PaymentGateway` without importing it or inheriting from it. That is **structural typing** — the type checker matches on the shape of the methods rather than on a declared relationship. It is what people mean by **duck typing** (day 047) made checkable, and it is how Go's interfaces work too.

The practical difference:

<code>abc.ABC + @abstractmethod</code>	checked at construction time, at runtime the implementer must import and inherit can carry fields and working methods use when: "is a kind of", and there is real shared code
<code>typing.Protocol</code>	checked by mypy or pyright, before the code runs the implementer knows nothing about you signatures only use when: "is able to", and the implementers are unrelated

Method resolution order, and what `super()` really does

```
class A:
    def who(self) → str: return "A"
class B(A):
    def who(self) → str: return "B → " + super().who()
class C(A):
    def who(self) → str: return "C → " + super().who()
class D(B, C):
    pass

print(D().who())
print([c.__name__ for c in D.__mro__])
```

```
B → C → A
['D', 'B', 'C', 'A', 'object']
```

Read that output carefully, because it surprises people. `B.who` calls `super().who()`, and `super()` inside `B` does **not** mean “my parent `A`”. It means “the next class after `B` in `D`’s resolution order”, which is `C`. The order is a property of `D`, not of `B`.

That is the diamond problem, answered. Every class appears once, subclasses come before their parents, and the order you listed the bases in is preserved. Java sidesteps it by allowing only one parent class; C++ makes you say `virtual` and think about it.

The honest interview answer: know that `__mro__` exists and that `super()` follows it, then say you avoid deep multiple inheritance in production and use composition instead.

Shallow copy and deep copy

Asked constantly, and it is a two-line demonstration:

```
import copy

original = {"tags": ["new", "sale"], "price": 100}
shallow = copy.copy(original)
deep = copy.deepcopy(original)

original["tags"].append("clearance")
print(shallow["tags"])      # ['new', 'sale', 'clearance'] ← shared list
print(deep["tags"])         # ['new', 'sale']             ← its own list
```

`copy.copy` duplicates the outer container and keeps the same inner objects. `copy.deepcopy` walks the whole structure. The one to say out loud: **returning a shallow copy of your internal list is not encapsulation** — the caller still holds your inner objects and can mutate them.

`__eq__` and `__hash__` together

```
from dataclasses import dataclass

@dataclass(frozen=True)          # frozen=True gives __eq__ AND __hash__
class Money:
    paise: int
    currency: str

@dataclass(eq=True)              # eq without frozen: __hash__ becomes None
class Cart:
    items: list[str]

print(hash(Money(1000, "INR")))
print(hash(Cart(["a"])))
```

```
-1806852253442064055
Traceback (most recent call last):
  File "day52.py", line 14, in <module>
    print(hash(Cart(["a"])))
    ~~~~~^~~~~~
TypeError: unhashable type: 'Cart'
```

Python does this deliberately. An object whose equality can change must not be a dictionary key, because the dictionary would look for it in the wrong place. Value objects get `frozen=True`; entities compare on an unchanging id ([day 044](#)).

6 The numbers

The shape of the round

A 45-minute low-level design round at a product company:

0-15 min	rapid-fire OOP questions	8 to 12 questions
15-40 min	one design problem	
40-45 min	your questions	

12 questions in 15 minutes = 75 seconds per question, including the interviewer asking it and typing a note.

Your speaking budget: 40 to 50 seconds. About 120 words.

Time your answers against that. The commonest failure is not a wrong answer; it is a ninety-second answer to a fifteen-second question, which costs you three of the remaining questions and reads as not knowing what matters.

What a definition-only answer costs

12 questions, 3 marks each:

- 1 · the definition
- 1 · a situation where it mattered
- 1 · the cost of the other choice

booklet answers only	: 12 x 1 = 12 / 36	→ "knows the theory"
definition + situation	: 12 x 2 = 24 / 36	→ "has written some code"
all three	: 12 x 3 = 36 / 36	→ "has maintained code"

The gap between the first and the last is not knowledge. It is 25 extra words per answer.

The abstract-class slot, priced

Java / C#: 1 superclass per class, unlimited interfaces.

A class hierarchy 4 levels deep, and the team later needs a shared Auditable capability across 11 unrelated classes:

as an abstract class	: impossible for 9 of the 11 -- their one slot is spent. The fix is a refactor of the whole hierarchy.
as an interface	: 11 x 1 line = 11 lines, no hierarchy touched.

The `__hash__` mistake, priced

A set of 10,000 orders, keyed on a hash built from a mutable status field.

correct hash	: lookup touches ~1 bucket	→ 1 comparison, 0(1)
mutated field	: the object is in bucket 4,193 but now hashes to bucket 7,812	→ NOT FOUND, silently

"order in orders" returns False for an order that is standing in the set. Nothing raises. The bug is found weeks later, in the data.

The composition arithmetic, which wins the argument

```
Vehicle types 3 · fuel types 2 · trailer 2 · ownership 3  
  
inheritance : 3 x 2 x 2 x 3 = 36 classes.  A new fuel type: +18 classes.  
composition : 3 + 2 + 2 + 3 = 10 classes.  A new fuel type: +1 class.
```

Say the numbers rather than the principle. “Inheritance multiplies the axes, composition adds them” is a sentence; thirty-six against ten is an argument.

7 The trade-offs

The booklet answer is not always wrong

There is a real case for the short, textbook answer: an early telephone screen, run by a recruiter or a junior engineer working from a checklist, where the person opposite is matching your words against a list. Give the definition cleanly, add one sentence of situation, and stop. Reading the room is part of the skill. The three-part answer is for an engineer who will work with you.

Abstract class: what you give up

You give up the one inheritance slot, and you give up the freedom of implementers to be unrelated to you. You gain shared code written once and enforcement at construction.

I would not use an abstract class if the implementations have nothing in common except the method names, or if any of them is a type I do not own — a third-party class cannot be made to inherit from my base. Then it is an interface, and in Python a `Protocol`, which the implementer does not even have to know exists.

Interface: what you give up

You give up shared implementation. Every implementer writes its own version of everything, and if eight of them need the same helper, you have eight copies or an awkward mixin. You also give up runtime enforcement in the `Protocol` case — nothing stops a wrong object at construction; you find out when the type checker runs, or when the method is called.

I would not use an interface if there is genuinely one way to do most of the work and only one step varies. That is a base class with one abstract method, and pretending otherwise duplicates the common part.

Polymorphism: what you give up

Polymorphism makes new **types** cheap and new **operations** expensive. Adding a fifth vehicle type is one file. Adding a sixth method to the interface is an edit to every implementation, including the fakes in your tests.

I would not replace the `if` chain if the set of types is closed and will not grow — days of the week, suits in a deck, HTTP methods — or if you are branching on a *value* rather than a type. The trigger to refactor is the **second** place in the codebase switching on the same set of types.

Multiple inheritance: what you give up

Comprehension. A method call in a four-parent class is no longer answerable by reading the class; you have to run `__mro__` in your head. The rule that holds up in review: **at most one parent with implementation; anything else must be a mixin whose methods do not collide.**

I would not use multiple inheritance if two parents both define the same method with real behaviour. That is the diamond, and the fix is to hold one of them as a field instead of inheriting it.

The meta trade-off

Every one of these questions has a “correct” answer and a real answer, and the real answer usually begins with “it depends on...” followed by the thing it actually depends on. That is not hedging, as long as you then commit. **State the dependency, pick a side, and say what would change your mind.** A candidate who answers “interface, unless there is shared implementation worth writing once, in which case abstract class” has said more in eleven words than a paragraph of definition.

8 In the interview

How it gets asked

- “What’s the difference between an abstract class and an interface?” — the single most-asked OOP question at product companies. Expect a follow-up on which you would use for a specific case.
- “What are the four pillars of OOP?” — a warm-up. Do not spend two minutes on it. Name them, define two properly, and offer to go deeper.
- “Does Python support method overloading?” — a trap for people who learned OOP from Java notes. The answer is no, and then the three Python alternatives.
- “Composition or inheritance? Why does everyone say prefer composition?” — answer with the multiplication, not the principle.
- “What’s the difference between `=` and `is`?” — nearly always followed by `__eq__` and `__hash__`.
- “Give me an example of polymorphism from something you’ve built.” — the “from something you’ve built” is the whole question. Have one ready.

What to say out loud, in the first ninety seconds

For any question in this block, run the same three beats and then stop.

1. **The definition, in one sentence.** “An abstract class can hold state and working methods and declares some for subclasses to fill in. An interface is signatures only, no state.”
2. **The rule that decides between them.** “Abstract class is ‘is a kind of’. Interface is ‘is able to’.”
3. **A situation you can name.** “A base `Report` that owns the header and footer and leaves `body` abstract — that is an abstract class, because there is real shared code. `PaymentGateway` implemented by Razorpay, Stripe and an in-memory fake — that is an interface, because those three share nothing but the capability.”
4. **The cost.** “A class gets one abstract-class slot and unlimited interfaces, so the abstract class is the expensive choice. My default is inherit interfaces, compose implementations.”
5. **Then stop, and let them ask the follow-up.** Silence after a complete answer is fine. Filling it is how a good answer becomes a rambling one.

The follow-ups

“You said Python has no `interface` keyword. So how do you do it?” Two mechanisms, and they are for different situations. The first is `abc.ABC` with `@abstractmethod`, which is really the abstract-class case: the implementer inherits from my base, gets whatever shared code I put there, and Python refuses to construct a subclass that has not implemented every abstract method — the `TypeError` fires at construction rather than when the method is eventually called, which is the point of using it. The second is `typing.Protocol`, and that is the true interface. The implementing class inherits from nothing and does not import my module at all; a type checker matches on the shape of the methods. That is structural typing, which is the same thing Go’s interfaces do, and it is duck typing made checkable. I reach for `Protocol` when the implementers are unrelated or not mine — I cannot make a third-party class inherit from my base, but I can declare a `Protocol` it already satisfies. I reach for `ABC` when there is real shared implementation worth writing once, and when I want the failure at construction time rather than at type-check time. The one thing I would not do is write an abstract class with no implementation in it at all; that is an interface wearing the wrong costume, and it spends the subclass’s only inheritance slot for nothing.

“Does Python support method overloading? What happens if I define two methods with the same name?” No, and what happens is that the second definition silently replaces the first — the class body is just executed top to bottom, and the name ends up bound to whichever `def` ran last. There is no warning, and the failure shows up at the call site as a `TypeError` about the number of arguments, which is a confusing place to find it. The three things I would do instead depend on the case. If the variants differ by an optional value, default arguments — `def total(self, items, discount=0)`. If they differ by count, `*args`, though I would be suspicious of a method that genuinely wants two different shapes of input. And if the behaviour genuinely has to vary by argument type, then `functools singledispatch`, which registers separate implementations per type and dispatches on the first argument at runtime. There is also `typing.overload`, but that only declares signatures for the type checker — the implementation is still one function, so it docu-

ments rather than dispatches. Overriding is a different thing entirely and Python does support it fully: a subclass replaces an inherited method, resolved at runtime, and that is the mechanism polymorphism is built on.

“Give me a concrete example of when you chose composition over inheritance.” The clearest one is modelling vehicles for a parking system. The first shape that occurs to everyone is a subclass per kind of vehicle, and it survives exactly until a second axis appears. Once you have three vehicle types and two fuel types you need six classes, because `ElectricTruck` and `DieselTruck` are different subclasses. Add a trailer option and it is twelve, add three ownership models and it is thirty-six — the class count is the *product* of the axes, so every new axis multiplies. With composition it is a `Vehicle` that holds a size, a fuel system, a trailer and an ownership record: three plus two plus two plus three, which is ten classes, and a new fuel type is one new class rather than eighteen. The tell that made me change it was the class names — `ElectricTruck`, `DieselTruck` — adjective-noun names are almost always two axes wearing one class. There is a second payoff I would mention, which is testing: a composed object takes a `FakeFuelSystem` that reports zero range, so I can test the empty-tank path by handing in an object, instead of building a real vehicle and reaching past encapsulation to set up the case. Separately constructible means separately testable. I still use inheritance, but for the narrow cases — exception hierarchies, abstract base classes used as contracts, framework hooks that are documented as such.

A model answer

“An abstract class and an interface both let a caller depend on a capability rather than a concrete type, but they differ in what they can carry. An abstract class can hold fields, a constructor and fully working methods, and it declares some methods for subclasses to fill in. An interface is signatures only — no state, no implementation.

The rule I use to choose is that an abstract class says ‘is a kind of’ and an interface says ‘is able to’. A `SavingsAccount` is a kind of `Account`, so that is a base class. A `SavingsAccount` is able to be exported, and so is an `Invoice` and so is a `Customer`, and those three share nothing — so `Exportable` is an interface.

Concretely: I had a base `Report` that owned the header, the footer and the generated-at timestamp and left `body()` abstract. That is an abstract class, because there was real shared code and I wanted it written once. Against that, a `PaymentGateway` with Razorpay, Stripe and an in-memory fake behind it — those three have no shared implementation at all, only a shared capability, so that is an interface.

The cost that decides borderline cases: in Java or C# a class has exactly one superclass and unlimited interfaces. So the abstract class spends a slot that can never be reclaimed. If I later need eleven unrelated classes to share an `Auditable` capability, an interface is eleven one-line changes and an abstract class is impossible for most of them. My default is therefore inherit interfaces, compose implementations.

In Python there is no `interface` keyword, so it is `abc.ABC` with `@abstractmethod` for the abstract-class case — and that gives real enforcement, a `TypeError` at construction if a subclass has not implemented everything — or `typing.Protocol` for the interface case, where the implementing class inherits from nothing at all and the checker matches on the shape of the methods.”

9 Recall card

- Every standard OOP question is a handbrake question: booklet answer, then the one that matters. Answer in three beats and stop — **definition** (one sentence) · **situation** where it decided something · **cost** with a number. Budget 40-50 seconds, about 120 words.
- **Abstract class** = “is a kind of”; **interface** = “is able to”. Abstract class carries state and working methods and spends the one inheritance slot; interface carries signatures and you can have unlimited ones. Python: `abc.ABC` + `@abstractmethod` (fails at *construction*) versus `typing.Protocol` (structural, implementer inherits nothing).
- **Encapsulation hides data so a rule cannot be broken; abstraction hides mechanism so it can be replaced.** The pillars split the same way: encapsulation and abstraction are about *hiding*; inheritance and polymorphism are about *varying*.

- **Python has no overloading** — a second `def` silently replaces the first. Use defaults, `*args`, or `functools.singledispatch`. It has full **overriding**, resolved at runtime through `__mro__`, and `super()` means “next in the MRO”, not “my parent” — which is the diamond problem, answered.
- **Say numbers, not principles.** Composition against inheritance: $3 + 2 + 2 + 3 = 10$ classes against $3 \times 2 \times 2 \times 3 = 36$. Polymorphism against a switch: 1 new file + 1 line against 4 edits to working code. And define `__hash__` whenever you define `__eq__`, on fields that never change.

PRACTICE

Practice — Bubble, selection and insertion sort, and what each one teaches

DSA topic: Bubble, selection and insertion sort, and what each one teaches **System design topic:** Common object-oriented interview questions

Code these, in this order

One rule for the whole set: **write the sort from memory before you look at anything**. All three are under ten lines, and the only way to know you have them is to produce them cold.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sort an Array	LeetCode 912 (Medium)	Whether your hand-written sort survives 50,000 elements — insertion sort will time out, and noticing that is the point.
2	Insertion Sort List	LeetCode 147 (Medium)	The insertion idea without the ability to shift — you have to relink instead.
3	Sort Colors	LeetCode 75 (Medium)	That a selection-sort instinct is wrong here, and one pass with three regions is right.
4	Height Checker	LeetCode 1051 (Easy)	Counting how far the input is from sorted — the exact quantity insertion sort is sensitive to.

On problem 1, submit the slow one first

Write `insertion_sort` and submit it. Read the verdict:

```
Time Limit Exceeded
```

Then submit `sorted(nums)`. Say out loud what changed: n^2 at $n = 50,000$ is 1,250,000,000 comparisons; $n \log n$ is about 800,000. That is the difference between a timeout and forty milliseconds, and it is the reason you call a library sort for real data.

On problem 2, notice what you cannot do

A linked list has no shifting. You cannot move everything one place right to open a gap, because there is no “one place right” — you can only relink. So the insertion walk becomes: find the last node whose value is not greater, then point around. Say out loud which property of insertion sort survives that change (it still grows a sorted prefix) and which does not (it can no longer walk *backwards*, so you scan forward from the head each time).

On problem 3, resist selection sort

Sort Colors is three distinct values. The selection-sort instinct — find all the zeros, then all the ones — is two passes and $O(n)$ but it is not what the problem wants. Write the counting version first, then the one-pass Dutch-flag version with three regions, and say which of the two

you would offer in an interview and why.

On problem 4, connect it to the cost

Height Checker asks how many positions differ between the input and its sorted version. Solve it, then answer this: is that number the same as the number of shifts insertion sort would perform? Run the shift counter from §5 of the lesson on the same input and find out. It is not the same, and being able to say why — one counts *misplaced elements*, the other counts *inversions* — is a genuinely good answer to “how nearly sorted is nearly sorted”.

The write-from-memory drill

Set a timer. Two minutes each, no reference, and the code must run:

1. `bubble_sort`, with the early-stop flag and the shrinking inner range.
2. `selection_sort`, tracking the position of the smallest.
3. `insertion_sort`, with the guard condition in the right order.

Then, for each one you wrote, answer without looking: what is the best case, what is the worst case, how many swaps, and is it stable?

The break-it drill

Introduce each bug on purpose, run it, and read the output:

1. In bubble sort, change the inner range to `range(len(nums))`. Paste the exact error.
2. In insertion sort, swap the two conditions to `while nums[i] > value and i ≥ 0`. Run it on `[2, 3, 1]`. What comes out, and why is there no error?
3. In insertion sort, change `>` to `≥`. Sort `[("a", 2), ("b", 1), ("c", 2)]` by the number and say what changed about `a` and `c`.
4. In selection sort, write `smallest = nums[i]` instead of `smallest = i`. Run it on `[5, 1, 4]`, then on `[2, 0, 1]`, and explain why one raises and one does not.
5. Remove the `swapped` flag from bubble sort. What is the best case now?

The adaptive drill

Count shifts with the counter from the lesson, and say the number before you run it:

1. `list(range(1000))` — already sorted.
2. `list(range(1000, 0, -1))` — exactly reversed.
3. `list(range(1000))` with elements 100 and 103 swapped.
4. `list(range(1000))` with the last element moved to the front.
5. A list of 1,000 identical values.

Then say which of those five is the case that makes insertion sort worth using in production, and which one is the case people wrongly assume is fast.

The arithmetic drill

Answer with numbers, in under ten seconds each:

1. Comparisons for selection sort at $n = 1,000$.
2. Element writes for selection sort at $n = 1,000$.
3. Element writes for bubble sort on reversed input at $n = 1,000$.
4. Comparisons for insertion sort on already-sorted input at $n = 1,000$.
5. Roughly where the crossover with merge sort sits, and what Python's Timsort uses.

The rapid-fire drill

Twelve questions, forty-five seconds each, answered out loud in three beats — definition, situation, cost. Time yourself, and stop when the third beat is done:

1. What are the four pillars of OOP?
2. Abstract class or interface — which and why?
3. Encapsulation or abstraction — what is the difference?
4. Overloading or overriding? Does Python have both?
5. Composition or inheritance? Give the numbers.
6. What is polymorphism? Give an example from your own code.
7. `=` or `is`?
8. If I define `__eq__`, what else must I define, and why?
9. Static method, class method, or instance method?
10. What is the diamond problem, and how does Python answer it?
11. Shallow copy or deep copy?
12. Can you have an abstract class with no abstract methods? Should you?

Mark yourself out of three on each. Anything scoring one is a definition with nothing behind it.

The Python-mechanics drill

Run each, then explain the output before you read it:

1. Define a class with two methods of the same name and call the first one.
 2. Subclass an `ABC` without implementing an abstract method, then construct it. Paste the error.
 3. Write a class that satisfies a `Protocol` without importing it, and type-check it.
 4. Build the A/B/C/D diamond and print `D.__mro__`. Explain why `super()` inside `B` reaches `C`.
 5. `@dataclass(eq=True)` without `frozen=True`, then call `hash()` on an instance. Paste the error.
 6. `copy.copy` a dict containing a list, mutate the inner list, and print both.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Write insertion sort. When would you actually use it?* The code with no off-by-one, the invariant, the guard-order reason, and the three real cases — small inputs, nearly-sorted input, and values arriving one at a time.
 2. *What is the difference between an abstract class and an interface?* Definition, the “is a kind of” against “is able to” rule, one situation for each, and the one-slot cost. Then the Python mechanisms for both.
 3. *You have three $O(n^2)$ sorts. Why does anyone distinguish them?* Comparisons against writes with the numbers, best cases, adaptivity, and stability — ending on the one case where selection sort genuinely wins.
-

Before you move on

- [] I wrote all three sorts from memory in under two minutes each, and they ran.
- [] I introduced the guard-order bug, ran it on `[2, 3, 1]`, and can say why there is no error.
- [] I can quote the write counts at $n = 1,000$: bubble 1,498,500 · insertion 500,499 · selection 2,997.
- [] I can name the three situations where insertion sort is the right answer, unprompted.
- [] I submitted insertion sort to LeetCode 912, got the timeout, and can say the arithmetic behind it.
- [] I answered all twelve rapid-fire questions in three beats each, inside forty-five seconds.
- [] I can draw the abstract-class-against-interface diagram in any tool and say which triangle is which.
- [] I answered all three questions above out loud.

Merge sort

DATA STRUCTURES AND ALGORITHMS

Merge sort

You can write merge sort and explain the divide-and-conquer shape it teaches.

SYSTEM DESIGN

Writing clean, testable classes

You can inject dependencies so your class can be tested without a database.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Write merge sort. Why is it $O(n \log n)$?*
- *How would you unit test this class?*

1 What this is, and why they ask it

Merge sort is the first sorting method that is genuinely fast. It works by cutting the list in half, sorting each half, and then **merging** the two sorted halves back into one — and merging two already sorted lists is cheap, because you only ever have to look at the front of each. It runs in $O(n \log n)$ time on every input, with no bad cases at all, and it is stable.

They ask it for the same reason they ask about binary search: the pattern matters more than the function. **Divide and conquer** — split the problem, solve the pieces, combine the answers — is the shape behind quicksort tomorrow, behind most tree problems from [day 098](#), and behind half of the interview questions that sound nothing like sorting. And the merge step is a standalone skill: “merge two sorted lists” is LeetCode 88 and LeetCode 21, it is the core of “merge k sorted lists” ([day 117](#)), and it is how a database joins two indexed tables. Yesterday’s three sorts were $O(n^2)$. Today is the day the arithmetic changes, and being able to say *why* it changes — not just that it does — is the whole question.

2 The story

Sarita teaches at a school in Nagpur, and every March the half-yearly exams finish and two hundred and forty answer booklets land on the staff room table in one heap. They have to reach the head’s office in roll number order by the evening, and for her first two years she did that job alone.

She would pick up a booklet, find where it belonged in the little run she had built so far, and slot it in. Two hours, every time, and by the end of it her back hurt from standing.

The third year Mrs Kulkarni came in, looked at the heap, and split it into four roughly equal piles. One for Sarita, one for Girish, one for Anjali, one for herself. Everyone arranged their own quarter, sixty booklets each, and that part took about fifteen minutes because sixty is nothing.

Then there were four neat piles in order and the office wanted one.

Sarita and Girish sat at one end of the long table with their two piles in front of them, and this is the part she still thinks was clever. They never looked at more than two booklets. Whichever of the two top booklets had the smaller roll number came off and went face down on a new pile between them. Then they looked at the two new tops and did it again. Neither of them ever searched for anything, because the top of a pile that is already in order is the smallest one left in it. When Girish’s pile ran out, Sarita picked up everything she had left and put it on the new pile in one movement, still in order, without checking a single number.

Anjali and Mrs Kulkarni did the same thing at the other end of the table. Then the two of them combined the two big piles the same way. Three combining sessions in all, and every booklet passed through a hand exactly once in each session.

One thing they got wrong the first time and never again: they tried it on a small desk. You cannot combine two piles into one without somewhere to build the new pile. They needed the space of the whole table, empty, next to the piles they were working from.

The whole thing finished in twenty-five minutes. What Sarita noticed afterwards was that the combining sessions were not the slow part at all. Every session touched all two hundred and forty booklets, but there were only three sessions.

3 The idea in plain English

Sarita's four teachers are merge sort. Splitting the heap is the divide. Each teacher arranging her own quarter is conquering a smaller version of the same problem. The two-piles-into-one sessions are the **merge**, and the empty table is the extra memory merge sort needs.

There are three ideas here and they arrive in order.

One: merging two sorted lists is cheap

This is the engine, so learn it on its own before the sort.

You have two lists, each already in order. You want one list containing everything, in order. Look only at the front of each list. The smaller of those two is the smallest value remaining anywhere, so it goes next into the output. Move past it, and look again.

```
left  [ 2 , 5 , 9 ]      right [ 1 , 6 , 7 ]      out [ ]
      ^
  compare 2 and 1 → take 1
left  [ 2 , 5 , 9 ]      right [ 1 , 6 , 7 ]      out [ 1 ]
      ^
  compare 2 and 6 → take 2
```

You never search. Each step takes one comparison and moves one value into the output, so merging two lists of total length n costs n comparisons at worst. That is $O(n)$, and it is the fact everything else today rests on. When one list runs out, the rest of the other is copied straight across with no comparisons at all — Sarita picking up her remaining booklets in one movement.

Two: a function that calls itself

Merge sort is defined in terms of itself. To sort a list, you sort two smaller lists and merge them. That is **recursion**: a function whose body calls the same function on a smaller piece of the problem.

If you have not met it before, here is the whole of it. When Python runs a function call it pauses the current call, runs the new one, and then resumes exactly where it left off. There is no limit on this in principle — a function can call itself, and each call gets its own private copy of its variables. So `merge_sort` on eight elements pauses and calls `merge_sort` on four; that call pauses and calls `merge_sort` on two; that one pauses and calls `merge_sort` on one, which does nothing because a single element is already in order, and returns. Then everything resumes on the way back up, merging as it goes.

Two things must be true or it never stops:

- **A base case.** A rule that returns without calling itself again. Here it is “a list of length 0 or 1 is already sorted”.
- **Progress towards it.** Each call must work on something strictly smaller. Here each call gets half the list, so after a handful of steps the length is 1.

Recursion gets a phase of its own from [day 087](#). Today you need only those two rules, and a version that avoids recursion entirely is in §5 so you can see the same sort written as plain loops.

Three: the halving is where `log n` comes from

Halve 240 and you get 120, then 60, then 30, 15, 8, 4, 2, 1. That is eight steps. Halving any number down to 1 takes about `log2 n` steps — the same fact that made binary search `O(log n)` on [day 042](#).

So the work stacks up like this. Each **level** of splitting has some number of pieces, and merging all the pieces at that level touches every element exactly once — `n` work per level, no matter how many pieces there are. And there are `log2 n` levels. So the total is `n × log2 n`.

That is the whole complexity argument, and it is worth saying in exactly that order: **n work per level, log n levels**. Not “because it divides in half”, which is only half the reason. Binary search also divides in half and is `O(log n)`, because it *throws away* one half rather than sorting both. Merge sort visits both halves, and that is the difference.

What merge sort gives you that yesterday’s sorts do not

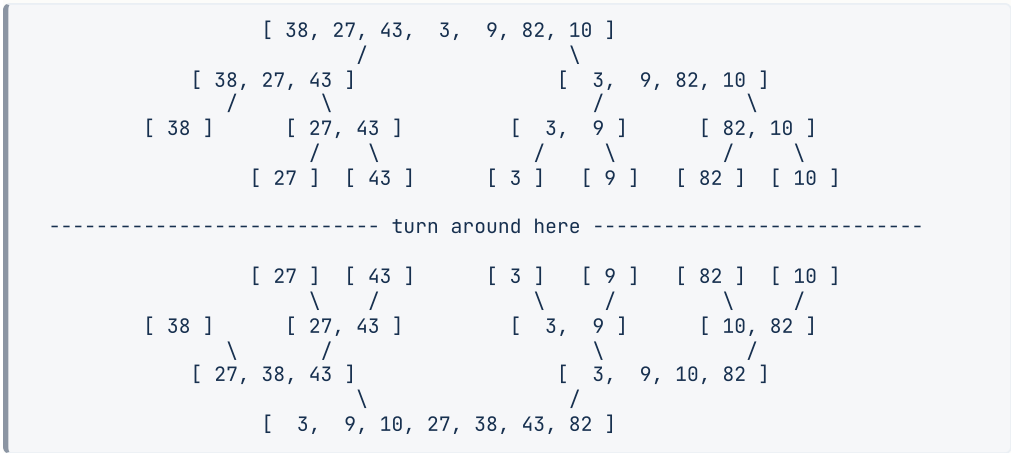
- `O(n log n)` **on every input**. Best case, average case and worst case are the same. There is no input that makes merge sort slow, which is not true of quicksort tomorrow.
- **Stable**. Equal elements keep their original order, provided the merge takes from the left list when the two fronts are equal. That is one `≤` and it is in §7.
- **It works when the data does not fit in memory**. Sort chunks that fit, write them out, then merge the sorted chunks. That is **external merge sort**, and it is what a database does for a large `ORDER BY` — the sort you saw spilling to disk in a query plan on [day 032](#).

What it costs you

$O(n)$ extra space. The merge cannot be done in place with any reasonable code — you need somewhere to build the output, which is Sarita’s empty table. That is the one real drawback and it is the first follow-up you will be asked.

4 The picture

The split, then the merge, on eight values. Read down, then back up:



What to notice: count the levels — four of them for seven elements, which is $\log_2 7$ rounded up plus one. Now look across any single level on the way back up: the total number of values being merged is always seven. That is the n per level, and multiplying the two gives $n \log n$. Notice also that nothing is sorted on the way *down*; all the work happens on the way back up.

The merge step itself, in detail. Two positions walking forward, and an output being filled:

left	index	0	1	2		right	index	0	1	2
	value	27	38	43			value	3	9	82
		+-----+						+-----+		
		^						^		
		i						j		
out	[]	compare left[0]=27 with right[0]=3				→	3 is smaller, take it, j++			
out	[3]	compare 27 with 9				→	9 is smaller, take it, j++			
out	[3, 9]	compare 27 with 82				→	27 is smaller, take it, i++			
out	[3, 9, 27]	compare 38 with 82				→	38 is smaller, take it, i++			
out	[3, 9, 27, 38]	compare 43 with 82				→	43 is smaller, take it, i++			
out	[3, 9, 27, 38, 43]									
		i has run off the end of left								
		→ copy the whole remaining right in one go: 82								
out	[3, 9, 27, 38, 43, 82]									

What to notice: six values, six steps, five comparisons. Every step consumes exactly one value. That is why the merge is $O(n)$ and not $O(n^2)$ — there is no searching anywhere in it. And notice the last line: when one side empties, the rest of the other is appended with zero com-

parisons.

Where the memory goes, which is the follow-up question drawn:

the input	[38, 27, 43, 3, 9, 82, 10]	n slots
during a merge	left [27, 38, 43] right [3, 9, 10, 82] out [3, 9, 10, 27, 38, 43, 82]	the two halves exist as separate lists plus the output
peak extra memory:	$O(n)$ -- one full extra copy of the data, at the top level	

What to notice: merge sort is not in place. Yesterday's three sorts used $O(1)$ extra space and this uses $O(n)$. Say that out loud before the interviewer asks.

5 The code, built step by step

The merge, on its own

Everything depends on this, so write it first and test it before you write the sort.

```
def merge(left: list[int], right: list[int]) → list[int]:
    out: list[int] = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:          # ≤ not < -- this is what keeps it stable
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])                # whichever side is left over, in one go
    out.extend(right[j:])              # exactly one of these two does anything
    return out
```

Three things to notice. The loop runs only while **both** lists have something left, which is why the two `extend` lines afterwards are not optional — without them you silently lose the tail. The `≤` rather than `<` is what makes the whole sort stable: when the two fronts are equal, taking from the left preserves the original order. And `left[i:]` on a spent list is `[]`, so both `extend` calls are always safe to run; exactly one of them ever has anything to add.

The sort, in four lines

```
def merge_sort(nums: list[int]) → list[int]:
    if len(nums) ≤ 1:                    # base case: nothing to do
        return nums
    middle = len(nums) // 2
    return merge(merge_sort(nums[:middle]), merge_sort(nums[middle:]))
```

That really is the whole sort. Read it as a sentence: *a list of one is already sorted; otherwise sort the left half, sort the right half, and merge them.*

`len(nums) // 2` uses integer division. Write `/` instead and you get a float, and slicing with a float raises — the exact message is in §7. And note the base case is `≤ 1`, not `= 1`: an empty list must return too, or a list of odd length eventually produces an empty half and the recursion never stops.

Why it always makes progress

`nums[:middle]` and `nums[middle:]` between them contain every element exactly once, and both are strictly shorter than `nums` whenever `len(nums) ≥ 2`. That guarantee is what makes the base case reachable. If you wrote `nums[:middle]` and `nums[middle - 1:]` by mistake, one branch would stop shrinking and Python would eventually raise `RecursionError`, which is also in §7.

The in-place version, which is what interviewers usually want

The version above allocates a new list at every level. The standard version sorts one list in place using a single scratch buffer, and it is what you should be able to produce if asked for the “proper” implementation:

```
def merge_sort_in_place(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Sorts nums[lo:hi] in place. Still O(n) extra space -- the buffer inside merge."""
    if hi is None:
        hi = len(nums)
    if hi - lo ≤ 1:
        return
    mid = (lo + hi) // 2
    merge_sort_in_place(nums, lo, mid)
    merge_sort_in_place(nums, mid, hi)
    _merge_range(nums, lo, mid, hi)
```

```
def _merge_range(nums: list[int], lo: int, mid: int, hi: int) → None:
    """Merge the two sorted runs nums[lo:mid] and nums[mid:hi] back into nums[lo:hi]."""
    buffer = nums[lo:mid]  # copy only the LEFT half out
    i, j, k = 0, mid, lo
    while i < len(buffer) and j < hi:
        if buffer[i] ≤ nums[j]:
            nums[k] = buffer[i]
            i += 1
        else:
            nums[k] = nums[j]
            j += 1
        k += 1
    while i < len(buffer):  # the right half is already in place
        nums[k] = buffer[i]
        i, k = i + 1, k + 1
```

Only the left half is copied out. The right half never needs copying, because anything still unread in it sits at or beyond the write position `k` — a value can never be overwritten before it is read. That is a genuinely subtle point and worth saying out loud if you write this version.

Bottom-up merge sort, with no recursion at all

Same algorithm, written as loops. Merge every neighbouring pair of runs of size 1, then of size 2, then 4, and so on:

```
def merge_sort_bottom_up(nums: list[int]) → list[int]:
    """No recursion. Merge runs of width 1, then 2, then 4 ... until width ≥ n."""
    out = list(nums)
    n = len(out)
    width = 1
    while width < n:
        for lo in range(0, n, width * 2):
            mid = min(lo + width, n)
            hi = min(lo + width * 2, n)
            out[lo:hi] = merge(out[lo:mid], out[mid:hi])
        width *= 2
    return out
```

The `min` calls handle the last group when `n` is not a power of two — the final piece is simply shorter, and merging a list with an empty list returns it unchanged. This version has the same `O(n log n)` cost and no recursion depth at all, which makes it the safe choice for very large inputs in Python.

The complete file

```
"""Merge sort: divide, conquer, and the merge that makes it  $O(n \log n)$ ."""

def merge(left: list[int], right: list[int]) → list[int]:
    """Combine two sorted lists into one sorted list.  $O(\text{len}(\text{left}) + \text{len}(\text{right}))$ .

    Stable: when the fronts are equal, the LEFT value is taken first.
    """
    out: list[int] = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out

def merge_sort(nums: list[int]) → list[int]:
    """ $O(n \log n)$  always -- best, average and worst.  $O(n)$  extra space. Stable.

    n work per level of merging,  $\log_2(n)$  levels.
    """
    if len(nums) ≤ 1:
        return nums
    middle = len(nums) // 2
    return merge(merge_sort(nums[:middle]), merge_sort(nums[middle:]))

def merge_sort_bottom_up(nums: list[int]) → list[int]:
    """The same sort with no recursion -- safe for inputs deeper than Python's limit."""
    out = list(nums)
    n = len(out)
    width = 1
    while width < n:
        for lo in range(0, n, width * 2):
            mid = min(lo + width, n)
            hi = min(lo + width * 2, n)
            out[lo:hi] = merge(out[lo:mid], out[mid:hi])
        width *= 2
    return out

def _merge_range(nums: list[int], lo: int, mid: int, hi: int) → None:
    buffer = nums[lo:mid]
    i, j, k = 0, mid, lo
    while i < len(buffer) and j < hi:
        if buffer[i] ≤ nums[j]:
            nums[k] = buffer[i]
            i += 1
        else:
            nums[k] = nums[j]
            j += 1
        k += 1
    while i < len(buffer):
        nums[k] = buffer[i]
        i, k = i + 1, k + 1

def merge_sort_in_place(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Sort nums[lo:hi] in place. Copies only the left half in each merge."""
    if hi is None:
        hi = len(nums)
    if hi - lo ≤ 1:
        return
```



```

mid = (lo + hi) // 2
merge_sort_in_place(nums, lo, mid)
merge_sort_in_place(nums, mid, hi)
_merge_range(nums, lo, mid, hi)

def merge_into(nums1: list[int], m: int, nums2: list[int], n: int) → None:
    """LeetCode 88. nums1 has m values then n empty slots; nums2 has n values.

    Merge BACKWARDS, from the largest, so nothing is overwritten before it is read.
    O(m + n) time, O(1) extra space -- the one merge that IS in place.
    """
    i, j, write = m - 1, n - 1, m + n - 1
    while j ≥ 0:
        if i ≥ 0 and nums1[i] > nums2[j]:
            nums1[write] = nums1[i]
            i -= 1
        else:
            nums1[write] = nums2[j]
            j -= 1
        write -= 1
    # nums2 must be exhausted

def count_inversions(nums: list[int]) → int:
    """How many pairs are out of order. The classic merge-sort follow-up.

    O(n log n) instead of the O(n^2) double loop: when a value is taken from the
    right half, every value still left in the left half forms an inversion with it.
    """

    def sort_and_count(values: list[int]) → tuple[list[int], int]:
        if len(values) ≤ 1:
            return values, 0
        middle = len(values) // 2
        left, a = sort_and_count(values[:middle])
        right, b = sort_and_count(values[middle:])
        out: list[int] = []
        i = j = 0
        crossing = 0
        while i < len(left) and j < len(right):
            if left[i] ≤ right[j]:
                out.append(left[i])
                i += 1
            else:
                out.append(right[j])
                j += 1
                crossing += len(left) - i
        out.extend(left[i:])
        out.extend(right[j:])
        return out, a + b + crossing

    return sort_and_count(nums)[1]

if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]

    print(merge([27, 38, 43], [3, 9, 82]))
    print(merge([1, 2], []))
    print(merge_sort(data))
    print(merge_sort([]))
    print(merge_sort([5]))
    print(merge_sort_bottom_up(data))

    in_place = list(data)
    merge_sort_in_place(in_place)
    print(in_place)

    a = [1, 2, 3, 0, 0, 0]
    merge_into(a, 3, [2, 5, 6], 3)
    print(a)

```

```

print(count_inversions([2, 4, 1, 3, 5]))    # 3
print(count_inversions([5, 4, 3, 2, 1]))    # 10 -- reversed: n(n-1)/2

# stability, demonstrated
pairs = [("b", 1), ("a", 2), ("c", 1)]
print(merge_sort_pairs := sorted(pairs, key=lambda p: p[1]))
# [('b', 1), ('c', 1), ('a', 2)] -- b before c, as in the input

```

6 What it costs

Time, counted level by level

This is the argument to say out loud, and it has exactly two parts.

Part one: each level costs n . At the top there is one merge of n values. At the next level down there are two merges of $n/2$ values each, which is n values in total. Below that, four merges of $n/4$. The number of pieces doubles and the size of each halves, so every level moves exactly n values.

```

level 0: 1 merge x n      values = n
level 1: 2 merges x n/2   values = n
level 2: 4 merges x n/4   values = n
level 3: 8 merges x n/8   values = n
      ---
      each level: n

```

Part two: there are $\log_2 n$ levels. Halving n down to 1 takes $\log_2 n$ steps.

```

n = 8          8 → 4 → 2 → 1          3 levels = log2(8)
n = 1,000      1000 → 500 → ... → 1    10 levels ~ log2(1000)
n = 1,000,000          20 levels ~ log2(10^6)

```

Multiply: n per level $\times \log_2 n$ levels = $O(n \log n)$.

```

n = 1,000 → 1,000 x 10 = 10,000 operations
n = 100,000 → 100,000 x 17 = 1,700,000
n = 1,000,000 → 1,000,000 x 20 = 20,000,000

```

Against yesterday

```

n = 100,000

insertion sort : n(n-1)/2 = 4,999,950,000 operations -- about 80 minutes in Python
merge sort     : n x log2(n) = 1,700,000 -- about 0.5 seconds

~2,900x fewer operations.

```

That ratio is the reason the whole phase exists. And note it *grows*: at a million elements the ratio is about 25,000.

The best case is the worst case

Merge sort always splits down to single elements and always merges every level, regardless of the input.

```
already sorted : n log n
reversed      : n log n
random        : n log n
all identical  : n log n
```

That predictability is a selling point. Quicksort tomorrow has a better constant factor but a possible $O(n^2)$; merge sort has no bad input at all. When an interviewer asks “which would you pick for sorting data an attacker can choose”, this is the answer.

Space

```
merge_sort (the simple version) : O(n) for the output lists,
                                plus O(log n) call frames for the recursion
merge_sort_in_place             : O(n/2) for the buffer at the top level, still O(n)
merge_into (LeetCode 88)       : O(1) -- but only because the room already exists
                                at the end of nums1, and you merge backwards
```

Say $O(n)$ and mean it. The recursion depth is only $\log_2 n$ — twenty frames at a million elements — so the call stack is never the problem; the data copies are.

The number of comparisons, exactly

The merge of two runs of total length m uses at most $m - 1$ comparisons and at least $m/2$. That gives a real range for the whole sort, and it is a good answer to “is it exactly $n \log n$ ”:

```
n = 1,024 (2^10 exactly)
at most : n log2(n) - n + 1 = 10,240 - 1,024 + 1 = 9,217 comparisons
at least : (n log2(n)) / 2 = 5,120 comparisons (when the halves interleave badly)
```

7 The traps

The real error: dividing with `/` instead of `//`

```
def merge_sort_broken(nums):
    if len(nums) <= 1:
        return nums
    middle = len(nums) / 2          # ← float division
    return merge(merge_sort_broken(nums[:middle]), merge_sort_broken(nums[middle:]))

print(merge_sort_broken([3, 1, 2]))
```

```

Traceback (most recent call last):
  File "day53.py", line 6, in <module>
    print(merge_sort_broken([3, 1, 2]))
    ~~~~~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^
  File "day53.py", line 5, in merge_sort_broken
    return merge(merge_sort_broken(nums[:middle]), merge_sort_broken(nums[middle:]))
    ~~~~~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^~^
TypeError: slice indices must be integers or None or have an __index__ method

```

In Python 3, `/` always produces a float, even for `4 / 2`. Positions into a list must be integers, so the rule is: **any division that produces a position uses `//`**. The same trap bit binary search on [day 043](#).

The real error: a base case that is never reached

```
def merge_sort_broken2(nums):
    if len(nums) == 1:          # ← = instead of ≤
        return nums
    middle = len(nums) // 2
    return merge(merge_sort_broken2(nums[:middle]), merge_sort_broken2(nums[middle:]))

print(merge_sort_broken2([1]))
```

```
Traceback (most recent call last):
  File "day53.py", line 6, in <module>
    print(merge_sort_broken2([]))
    ~~~~~^~~~~~
  File "day53.py", line 5, in merge_sort_broken2
    return merge(merge_sort_broken2(nums[:middle]), merge_sort_broken2(nums[middle:]))
  File "day53.py", line 5, in merge_sort_broken2
    return merge(merge_sort_broken2(nums[:middle]), merge_sort_broken2(nums[middle:]))
  File "day53.py", line 5, in merge_sort_broken2
    return merge(merge_sort_broken2(nums[:middle]), merge_sort_broken2(nums[middle:]))
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

An empty list has `middle = 0`, so `nums[:0]` is `[]` and `nums[0:]` is `[]` — the same size, forever. This is the general recursion failure and you will meet it again on [day 089](#): **the base case must cover every input that cannot shrink further, not just the one you had in mind.** `≤ 1` covers both 0 and 1.

Python's default recursion limit is 1,000 frames, which for merge sort would mean a list of about 2^{1000} elements — so if you see `RecursionError` from a merge sort, it is a bug, never a size problem.

The near-miss: forgetting the leftovers

```
def merge_broken(left, right):
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out # ← no extend

print(merge_broken([1, 2, 9], [3, 4]))
```

```
[1, 2, 3, 4]
```

The `9` is gone, and nothing raised. The loop condition needs **both** lists to be non-empty, so it exits the moment either one is spent — and whatever is left in the other is simply dropped. This is the commonest merge bug and it produces a shorter list that still looks plausible. **Check the length of your output against the sum of the inputs**; it is a one-line assertion that catches it instantly.

The near-miss: `<` instead of `<=` silently loses stability

```
if left[i] < right[j]: # ← was ≤
```

With `<`, an equal value on the right is taken first, so two equal elements swap order. The numbers still come out sorted. Sort a list of `(name, score)` by score, and the names inside each score come out in an order you cannot explain. As on [day 052](#), stability is one character and it never announces itself.

The trap: merging forwards when the output shares memory with the input

LeetCode 88 gives you `nums1 = [1, 2, 3, 0, 0, 0]` and asks you to merge `nums2 = [2, 5, 6]` into it in place. The obvious forward merge destroys the data:

```
def merge_into_broken(nums1, m, nums2, n):
    i = j = k = 0
    while j < n:
        if i < m and nums1[i] ≤ nums2[j]:
            nums1[k] = nums1[i]; i += 1
        else:
            nums1[k] = nums2[j]; j += 1
        k += 1

a = [1, 2, 3, 0, 0, 0]
merge_into_broken(a, 3, [2, 5, 6], 3)
print(a)
```

```
[1, 2, 2, 3, 5, 6]
```

That one happens to be right — which is exactly why it is dangerous. Try an input where a write lands on an unread value:

```
a = [4, 5, 6, 0, 0, 0]
merge_into_broken(a, 3, [1, 2, 3], 3)
print(a)
```

```
[1, 2, 3, 1, 2, 3]
```

Writing `1` into position 0 destroyed the `4` that had not been read yet. The fix is to merge **backwards** from the largest value, filling the empty space at the end first, so a write never lands anywhere still unread. That is `merge_into` in §5, and it is a favourite interview question precisely because the forward version passes the sample case.

The trap: claiming merge sort is in place

It is not, and the interviewer will check. The simple version allocates a new list at every level. The “in-place” version still copies half the range into a buffer at each merge. There is a genuine in-place merge algorithm, but it is $O(n \log^2 n)$ or needs block-swapping tricks that nobody writes from memory. **The honest answer is: $O(n)$ extra space, and if you need in-place with $O(n \log n)$ guaranteed, use heapsort.**

8 In the interview

How it gets asked

- “Write merge sort.” — asked directly, extremely often, usually as the first $O(n \log n)$ sort they check you can produce.
- “Why is it $O(n \log n)$?” — the real question, and the answer must be “n work per level, log n levels”, not “because it divides in half”.
- “Merge two sorted arrays.” — LeetCode 88 and 21. The merge step alone, and the in-place version needs the backwards trick.
- “Merge sort or quicksort — which would you use?” — expect this tomorrow too. The answer is about guaranteed worst case, stability and memory.
- “Count the inversions in this array.” — the standard follow-up that only merge sort answers in $n \log n$.
- “Sort a linked list.” — LeetCode 148. Merge sort is the right answer, because merging linked lists needs no extra space at all, and quicksort needs random access it does not have.

What to say out loud, in the first ninety seconds

1. **Say the sentence version before writing anything.** “A list of one is already sorted. Otherwise sort the left half, sort the right half, and merge the two sorted halves. The merge is the part that does the work.”

2. **Write the merge first, and say why it is cheap.** *“Merging two sorted lists is $O(n)$ because I only ever compare the two fronts — the smaller of them is the smallest value remaining anywhere, so there is no searching.”*
3. **Name the base case and the progress.** *“Base case is length 0 or 1. Each call gets a strictly smaller half, so it terminates.”*
4. **Give the complexity as two facts.** *“Each level of merging touches all n elements, and there are $\log_2 n$ levels of halving, so $n \log n$. And it is the same on every input — there is no bad case.”*
5. **Volunteer the cost.** *“It’s $O(n)$ extra space, unlike the quadratic sorts, and it is stable because the merge takes from the left when the fronts are equal.”*

The follow-ups

“Why is it $O(n \log n)$? Explain it without saying ‘because it divides in half.’ Because there are two separate quantities and the answer is their product. The first is the cost of one level of merging. At the top there is a single merge of all n elements. One level down there are two merges of $n/2$ each, which is still n elements moved in total. Below that, four merges of $n/4$ — again n . The number of pieces doubles at each level and the size of each piece halves, so those two cancel and every level costs exactly n . The second quantity is how many levels there are, and that is how many times you can halve n before reaching 1, which is $\log_2 n$ — twenty for a million elements. Multiply them: n per level times $\log n$ levels is $n \log n$. The reason “it divides in half” is not sufficient on its own is that binary search also divides in half and is $O(\log n)$, not $O(n \log n)$ — the difference is that binary search throws one half away and recurses into a single side, so it does constant work per level, whereas merge sort recurses into both halves and does linear work per level. The shape of the recursion, not the halving, is what decides it.

“It uses $O(n)$ extra space. Can you avoid that?” Not really, and I would rather say so than pretend. The merge fundamentally needs somewhere to write the combined result, because the two sorted runs sit next to each other in the array and writing the output over them would destroy values that have not been read yet. The best I can do in the standard implementation is copy only the *left* half into a buffer — the right half never needs copying, because anything still unread in it lies at or beyond the write position, so it cannot be clobbered. That halves the constant but it is still $O(n)$. There are genuine in-place merge algorithms using block rotations, but they are $O(n \log^2 n)$ or have constants bad enough that nobody uses them, and I would not attempt one from memory in an interview. If the requirement is truly $O(n \log n)$ time with $O(1)$ space, the right answer is heapsort. There are two important exceptions in the other direction. Sorting a linked list with merge sort needs no extra space at all beyond the recursion, because merging two linked lists is pure relinking — that is why merge sort is the standard answer to LeetCode 148. And merging into an array that already has room at the end, like LeetCode 88, is $O(1)$ extra space if you merge backwards from the largest value, so nothing is written over an unread slot.

“Merge sort or quicksort?” Quicksort by default for arrays in memory, merge sort when one of three things is true. Quicksort wins on the constant factor: it sorts in place, so it touches less memory and gets far better cache behaviour, and in practice it runs about two to three times faster than merge sort on the same array even though both are $n \log n$. What you give up is the guarantee — quicksort’s worst case is $O(n^2)$, and although a randomised pivot makes that astronomically unlikely, “unlikely” is not “impossible”, and with a deterministic pivot an adversary who can choose the input can force it. So I choose merge sort when the worst case actually matters — data that comes from outside and could be chosen against me, or a latency budget where a rare slow case is a broken promise rather than an average. Second, when I need stability: merge sort is stable and quicksort is not, and if I am sorting records by a second key after a first, that is not optional. Third, when the data does not fit in memory or is a linked list. External sorting is merge sort — sort chunks that fit in RAM, write them out, then merge the runs — and that is exactly what Postgres or MySQL does for a large ORDER BY that spills to disk. For linked lists, merge sort needs no random access and no extra space, and quicksort needs both.

A model answer

“Merge sort is divide and conquer. A list of zero or one elements is already sorted — that’s the base case. Otherwise I split the list in half, sort each half by the same method, and then merge the two sorted halves into one.

The merge is where all the work happens, so I’d write that first:

```
def merge(left: list[int], right: list[int]) → list[int]:
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out

def merge_sort(nums: list[int]) → list[int]:
    if len(nums) ≤ 1:
        return nums
    middle = len(nums) // 2
    return merge(merge_sort(nums[:middle]), merge_sort(nums[middle:]))
```

Two details in the merge I’d point at. Those two `extend` lines are not optional — the loop stops as soon as *either* list is empty, so whatever remains in the other has to be copied across, and forgetting that silently returns a shorter list. And the comparison is `≤` rather than `<`, which is what makes the sort stable: when the two fronts are equal I take from the left, so equal elements keep their original order.

On the complexity — it’s $O(n \log n)$ and the argument is two facts multiplied. Each level of merging touches every element exactly once: one merge of n at the top, two merges of $n/2$ below it, four of $n/4$ below that, so n per level regardless of how many pieces there are. And there are $\log_2 n$ levels, because that’s how many times you can halve n down to 1 — twenty levels at a million elements. n per level times $\log n$ levels is $n \log n$. That’s also why ‘it divides in half’ isn’t the full answer: binary search divides in half too and is $O(\log n)$, because it discards one side instead of recursing into both.

The cost is $O(n)$ extra space for the merge buffers, plus $O(\log n)$ stack frames, and that’s the main thing merge sort gives up against quicksort. What it buys is a guaranteed worst case — every input is $n \log n$, there’s no bad case — and stability. So I’d reach for it when the input could be chosen adversarially, when I need stability, or when the data doesn’t fit in memory, which is exactly the external merge sort a database runs for a large ORDER BY.”

9 Recall card

- **The sentence:** *a list of 0 or 1 is sorted; otherwise sort both halves and merge them.* The merge is the engine — compare only the two fronts, take the smaller, and when one side empties copy the rest across with no comparisons. Merging is **$O(n)$** because nothing is ever searched for.
- **Why $O(n \log n)$, in two facts multiplied:** every level of merging moves all **n** elements ($1 \times n$, then $2 \times n/2$, then $4 \times n/4 \dots$), and there are **$\log_2 n$** levels of halving. Not “because it halves” — binary search halves too and is $O(\log n)$, because it discards one side.
- **No bad input.** Best = average = worst = $n \log n$, and it is **stable** because the merge uses `≤` and takes from the left on a tie. Quicksort is faster in practice but can hit $O(n^2)$ and is not stable.
- **The cost is $O(n)$ extra space** — the merge cannot write over its own inputs. Copy only the *left* half into the buffer; the right half is always at or beyond the write position. Two exceptions: linked lists (pure relinking, $O(1)$) and LeetCode 88 (merge **backwards** into the free tail).
- **Three traps:** `//` not `/` for the middle (`TypeError: slice indices must be integers`) · base case `≤ 1` not `= 1` or an empty half recurses forever (`RecursionError`) · the two `extend` lines, or the tail is silently dropped. And the free follow-up: **count inversions** in $n \log n$ by adding `len(left) - i` whenever you take from the right.

1 What this is, and why they ask it

A class is **testable** when you can create it, run one of its methods, and check the result without starting anything else. No database, no network call, no waiting, no fixed clock. The technique that gets you there has one main idea: a class should be **given** the things it depends on rather than **fetching** them itself. That is called **dependency injection**, and it is three lines of code, not a framework.

They ask “how would you test this?” because it is the fastest way to find out whether you have ever maintained code. Untestable code is not a testing problem — it is a design problem that shows up as a testing problem, and the design flaw is always the same: the class reaches out and builds its own collaborators, so there is no way to stand between it and them. You met the fix already, in a different costume. [Day 048](#) said the strongest reason to define an interface is usually the in-memory fake for tests, and [day 049](#) said separately constructible means separately testable. Today is those two sentences turned into code you can write in an interview.

2 The story

The annual day at a school in Kochi is on a Saturday, and on the Wednesday before it, the rehearsal for the main play went badly for a reason that had nothing to do with the acting.

The play needs four things: a crown, a wooden sword, a large brass lamp, and a letter in a sealed envelope. All four live in the store room behind the stage, and the store room is locked, and the only key is with Joseph, who looks after the building and who had gone to Thrissur for a funeral.

The way rehearsals had always worked was that each boy fetched his own things. When it was nearly your scene you went round the back, found what you needed, and came out with it. So on Wednesday, with the store room shut, nobody could rehearse anything. Sunil stood at the side of the stage for forty minutes waiting to see if the key would turn up. It did not. Everyone went home.

On Thursday the teacher in charge, Ms Fernandes, changed one thing, and it was not about the store room. She put a boy called Riyas at the side of the stage with a table, and she said: from now on nobody goes to the back. Whatever your scene needs, Riyas hands it to you as you go on.

Thursday’s rehearsal ran completely. The crown was a circle of card with gold paper on it. The sword was a metre rule. The lamp was a steel tumbler turned upside down, and the sealed letter was an empty envelope with nothing written on it. Not one of the four was the real

thing, and it did not matter, because what the scene actually needed from the crown was something the boy could put on his head at the right moment, and a card circle does that perfectly well.

They rehearsed the whole play twice. They found two real problems — one boy's entrance was three seconds late every time, and two of them were standing in each other's light — and they fixed both before Joseph came back with the key.

On Friday the real crown and the real lamp came out of the store room. The play did not change. Rivas handed out the real things instead of the stand-ins, and everybody's part worked exactly as it had on Thursday, because nobody's part had ever depended on where the crown came from.

3 The idea in plain English

Ms Fernandes did one thing, and it is the whole lesson: she stopped the actors fetching and started handing things in. Everything else follows from that.

The problem: a class that fetches its own collaborators

Here is the untestable version. Read it and find the line that ruins it.

```
class OrderService:
    def __init__(self) → None:
        self.db = PostgresConnection("postgres://prod-db:5432/orders") # fetches its own
        self.mailer = Smtplib.SMTP("smtp.company.com") # and its own

    def place(self, customer_id: int, items: list[str]) → int:
        order_id = self.db.insert_order(customer_id, items)
        self.mailer.send(customer_id, f"Order {order_id} confirmed")
        return order_id
```

To test `place` you must have Postgres running, on that host, with that schema. You must have an SMTP server, or you send a real email to a real customer every time the test runs. And there is no way to check what happens when the database is down, because you cannot make it be down on demand.

This is the store room with the key gone. The dependency is **hidden inside the constructor**, so there is nowhere for a test to stand between `OrderService` and Postgres.

The fix: hand the collaborators in

```
class OrderService:
    def __init__(self, orders: OrderRepository, mailer: Mailer) → None:
        self.orders = orders # given, not fetched
        self.mailer = mailer

    def place(self, customer_id: int, items: list[str]) → int:
        order_id = self.orders.insert(customer_id, items)
        self.mailer.send(customer_id, f"Order {order_id} confirmed")
        return order_id
```

That is **constructor injection**, and it is the entire technique. The class now says what it needs and refuses to go and get it. Some vocabulary, each defined once:

- A **dependency** is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.
- **Dependency injection** is passing dependencies in from outside instead of constructing them inside. Constructor injection is the common form; method injection — passing it as an argument to one method — is the right choice when only one method needs it.
- A **seam** is a place where you can change behaviour without editing the class. Every injected dependency is a seam.
- A **test double** is a stand-in you pass in during a test. Riyas’s cardboard crown.
- The **composition root** is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module. It is the only file that mentions `PostgresOrderRepository`.

The four kinds of test double, named properly

Interviewers use these words precisely, so use them precisely.

NAME	WHAT IT DOES	USE IT WHEN
Dummy	Passed to satisfy a signature, never used.	The method under test does not touch it.
Stub	Returns canned answers. No logic.	You need the collaborator to <i>return</i> something.
Fake	A real working implementation, simplified. An in-memory dict instead of Postgres.	You need it to behave correctly across several calls.
Mock	Records the calls made to it so the test can assert on them.	The <i>effect</i> is the outcome — “an email was sent”.

The cardboard crown is a fake: it genuinely goes on a head. The empty envelope is a dummy: nobody opens it. If Ms Fernandes had stood at the side counting how many times the lamp was picked up, that would be a mock.

Prefer fakes to mocks. A mock asserts *how* your class did its job — which methods it called, in what order — so it breaks when you refactor the inside of a class that still behaves correctly. A fake asserts *what happened*, which is what you actually care about. Reach for a mock only when the outcome genuinely is a call to something outside, such as sending an email.

The three things that make code untestable, and their fixes

One: `new` inside a constructor. Fixed by injection, above.

Two: reading the clock, the random number generator, or the environment directly.

```
class Subscription:
    def is_expired(self) → bool:
        return datetime.now() > self.ends_at          # cannot test "expires tomorrow"
```

You cannot write a test for “what happens the day after it expires” without changing the computer’s clock. Inject the clock:

```
class Subscription:
    def is_expired(self, now: datetime) → bool:      # method injection: only this method needs it
        return now > self.ends_at
```

Now the test is one line and runs in a microsecond. The same applies to `random.random()`, `uuid.uuid4()` and `os.environ`. **Anything that returns a different answer on two identical runs must come in from outside.**

Three: global or class-level mutable state. A module-level `_connection` or a singleton means two tests running in the same process affect each other, and the second one fails only when the first one ran first. That is the worst class of bug to debug and it is entirely avoidable.

The rule that ties it together

A class should ask for what it needs and never go looking for it.

If a constructor contains the name of a concrete external thing — a driver, a URL, a hostname, a file path — that name belongs in the composition root instead.

4 The picture

The change Ms Fernandes made, in one diagram:

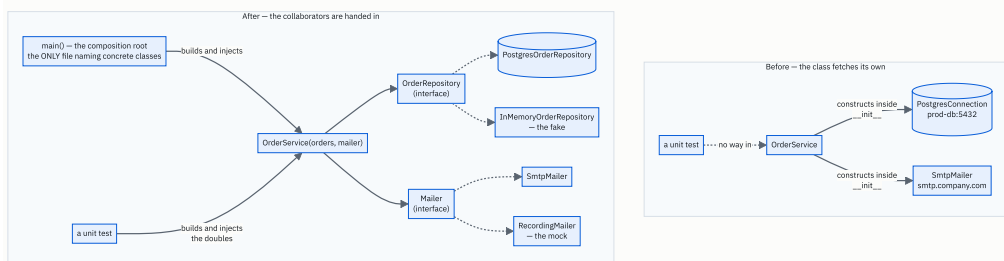


FIGURE 53.1

What to notice: the arrow from the test. In the top half there is no way for a test to get between `OrderService` and `Postgres`, so there is no unit test at all. In the bottom half the test does exactly what `main()` does — it constructs the object and hands in collaborators — and the only difference is which ones. `OrderService` cannot tell.

The seams, drawn on one class:

```

class Invoice:
    def __init__(self,
                    customer: Customer,      ← injected (seam)
                    rates: TaxRates,         ← injected (seam)
                    numbering: SequenceSource ← injected (seam)
    ) → None:

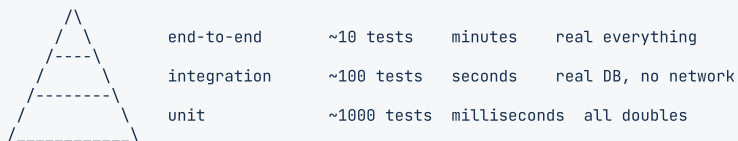
    def total(self, on: date) → Money:      ← the clock, injected per call (seam)

    def issue(self) → None:
        self.mailer = SmtplibMailer(...)   ← NOT a seam. Welded shut.
        now = datetime.now()                ← NOT a seam. Untestable.
        key = os.environ["API_KEY"]         ← NOT a seam. Environment-dependent.

```

What to notice: the top four lines can each be replaced from a test without touching this file. The bottom three cannot be replaced at all, and each one makes the whole method impossible to test in isolation. Count the seams in a class and you have measured its testability.

The test pyramid, which is why any of this matters:



The bottom layer only exists if your classes take their dependencies as arguments. A codebase that constructs its own collaborators has no bottom layer at all – every test is an integration test, and the suite takes twenty minutes.

What to notice: the shape is not an aesthetic preference. It is a consequence of injection. You cannot choose to have a thousand fast tests if your classes cannot be constructed without a database.

5 How it actually works

The whole thing, in runnable code

Start with the interface — a `Protocol`, as on [day 052](#), so implementations need not inherit anything:

```

from typing import Protocol

class OrderRepository(Protocol):
    def insert(self, customer_id: int, items: list[str]) → int: ...
    def get(self, order_id: int) → dict | None: ...

class Mailer(Protocol):
    def send(self, customer_id: int, body: str) → None: ...

```

The class under test depends only on those two names:

```

class OrderService:
    def __init__(self, orders: OrderRepository, mailer: Mailer) → None:
        self._orders = orders
        self._mailer = mailer

    def place(self, customer_id: int, items: list[str]) → int:
        if not items:
            raise ValueError("an order needs at least one item")
        order_id = self._orders.insert(customer_id, items)
        self._mailer.send(customer_id, f"Order {order_id} confirmed")
        return order_id

```

The fake is fifteen lines and it is a real implementation:

```

class InMemoryOrderRepository:
    """A working repository backed by a dict. Same behaviour, no Postgres."""

    def __init__(self) → None:
        self._rows: dict[int, dict] = {}
        self._next_id = 1

    def insert(self, customer_id: int, items: list[str]) → int:
        order_id = self._next_id
        self._rows[order_id] = {"customer_id": customer_id, "items": list(items)}
        self._next_id += 1
        return order_id

    def get(self, order_id: int) → dict | None:
        return self._rows.get(order_id)

class RecordingMailer:
    """A mock: it remembers what it was asked to send."""

    def __init__(self) → None:
        self.sent: list[tuple[int, str]] = []

    def send(self, customer_id: int, body: str) → None:
        self.sent.append((customer_id, body))

```

And the test, which needs nothing running:

```

def test_placing_an_order_stores_it_and_emails_the_customer() → None:
    orders = InMemoryOrderRepository()
    mailer = RecordingMailer()
    service = OrderService(orders, mailer)

    order_id = service.place(customer_id=7, items=["lamp", "shade"])

    assert orders.get(order_id) == {"customer_id": 7, "items": ["lamp", "shade"]}
    assert mailer.sent == [(7, f"Order {order_id} confirmed")]

```

Three lines of setup, one line of action, two assertions, and it runs in well under a millisecond. The fake checks the state that resulted; the mock checks the effect that had to leave the system.

Testing the failure path, which is the real payoff

The reason this matters is not that the happy path is faster to test. It is that the unhappy paths become testable at all:

```
class ExplodingRepository:
    def insert(self, customer_id: int, items: list[str]) → int:
        raise ConnectionError("could not connect to server: Connection refused")
    def get(self, order_id: int) → dict | None:
        raise ConnectionError("could not connect to server: Connection refused")

def test_no_email_is_sent_if_the_order_fails_to_save() → None:
    mailer = RecordingMailer()
    service = OrderService(ExplodingRepository(), mailer)

    with pytest.raises(ConnectionError):
        service.place(customer_id=7, items=["lamp"])

    assert mailer.sent == [] # the bug this test exists to prevent
```

You cannot write that test against a real database without unplugging it. This is the argument to make in an interview: **injection is not about speed, it is about reachability**. Roughly half the branches in production code are error paths, and without doubles most of them are never executed by any test.

Injecting the clock

```
from datetime import date
from typing import Protocol

class Clock(Protocol):
    def today(self) → date: ...

class SystemClock:
    def today(self) → date:
        return date.today()

class FixedClock:
    def __init__(self, day: date) → None:
        self._day = day
    def today(self) → date:
        return self._day
```

```
class Subscription:
    def __init__(self, ends_at: date, clock: Clock) → None:
        self._ends_at, self._clock = ends_at, clock

    def is_expired(self) → bool:
        return self._clock.today() > self._ends_at

def test_expires_the_day_after_it_ends() → None:
    sub = Subscription(date(2026, 3, 31), FixedClock(date(2026, 4, 1)))
    assert sub.is_expired()
```

The library alternative is `freezegun` or `pytest`'s `monkeypatch`, and both work. The injected clock is better in domain code because it makes the dependency visible in the signature rather than patching a global at test time — but say that you know both exist.

The composition root

Exactly one file names the real classes:

```
# main.py -- the only module that mentions Postgres or SMTP
def build_order_service() → OrderService:
    return OrderService(
        orders=PostgresOrderRepository(os.environ["DATABASE_URL"]),
        mailer=SmtpMailer(os.environ["SMTP_HOST"]),
    )
```

The check that this worked is a one-line command, and it is a good thing to say out loud:

```
grep -rn "psycopg" src/ --exclude-dir=adapters --exclude=main.py
```

If that returns nothing, the database driver is genuinely swappable.

What real frameworks do

- **pytest fixtures** are dependency injection for tests. A `@pytest.fixture` that returns an `InMemoryOrderRepository` is handed to every test that names it as an argument.
- **FastAPI's** `Depends` injects per-request collaborators into endpoint functions, and its `dependency_overrides` dict swaps them for fakes in tests — the composition root as a data structure.
- **Spring** in Java and **.NET's** built-in container do the same with annotations and a registry. They automate the wiring; they do not change the idea.
- `unittest.mock.patch` replaces a name in a module at test time. It works, and it is the escape hatch for code you cannot change — but it couples the test to the *import path* of the thing being patched, so moving a module breaks tests that never mentioned it. Prefer injection where you own the code.
- **Fakes shipped by real products:** `fakeredis` for Redis, SQLite in `:memory:` mode for a SQL database, `moto` for AWS, and `localstack` for a broader set. When you need the real thing, **testcontainers** starts a genuine Postgres in Docker per test session — the integration layer of the pyramid, not the unit layer.

6 The numbers

What the suite costs, with and without seams

```
A service with 800 tests.

every test hits a real Postgres:
  setup + teardown per test ~ 45 ms
  the test itself           ~ 15 ms
  800 x 60 ms               = 48 seconds ... if the DB is local and warm
  in CI, with container startup and migrations: 6-10 minutes

700 unit tests with fakes + 90 integration + 10 end-to-end:
  700 x 0.4 ms = 0.3 s
  90 x 60 ms   = 5.4 s
  10 x 4 s     = 40 s
  -----
  ~46 s total, and the 0.3 s part runs on every file save
```

The number that changes behaviour is the **0.3 seconds**. A suite you can run on every save gets run a hundred times a day. A suite that takes ten minutes gets run once, before you go home, and the feedback arrives after you have forgotten the change.

Reachability, which is the real argument

```
A typical service class: 40 branches.
~22 happy-path branches    reachable with real collaborators
~18 error branches         connection lost, timeout, duplicate key,
                           payment declined, rate limited, disk full

with real dependencies only : 22 / 40 reachable = 55% of branches testable
with injected doubles       : 40 / 40 reachable = 100%

Nearly half the code in a service is error handling, and it is exactly the half
that runs at 3 a.m.
```

Flakiness, priced

```
A suite of 800 tests where each has a 0.1% chance of a network flake:

P(at least one failure) = 1 - 0.999^800 = 55%

So more than half of all CI runs fail for no reason.
A team that sees that learns to re-run rather than to read, and a real
failure gets re-run too.

With 700 of the 800 using in-memory doubles, only 100 can flake:
1 - 0.999^100 = 9.5%
```

The cost of the change itself

Refactoring `OrderService` to take its dependencies as arguments:

```
the class           : 4 lines changed
the composition root : 1 new function, 6 lines
the fake repository : 15 lines, written once, reused by ~40 tests
call sites          : however many construct it -- usually 1, in main()
```

Total: about 25 lines, once. Against 800 tests that no longer need Postgres.

7 The trade-offs

What injection costs you

Wiring. Something must construct the objects and pass them in, and that something is code you did not have before. In a small program it is one function; in a large one it is a container, and a container is a thing to learn, debug and misconfigure.

Indirection. Reading `self._orders.insert(...)` no longer tells you what actually runs. You have to go to the composition root to find out. That is a genuine cost to a newcomer, and the mitigation is that there is exactly *one* place to look.

Constructors that grow. A class taking six injected dependencies is a signal, and the signal is that the class is doing six things. Do not fix it by hiding them in a container — fix it by splitting the class, which is tomorrow's topic on [day 055](#).

When not to inject

I would not inject a dependency if it is a value object, a pure function, or part of the language — nobody injects `datetime` the *type*, `json`, or `Money`. Inject things that do input and output, things that are slow, things that are non-deterministic, and things you might genuinely replace. Everything else is noise.

I would not inject if the class is a leaf with no collaborators. A `Money` or an `Interval` needs no seams; it is already testable because it is already pure.

I would not build an interface for a dependency with exactly one implementation and no test double. That is the check from [day 048](#) — name the second implementation, and if the answer is “the fake”, that counts and you should say so.

Fakes against mocks

Mocks are seductive because `unittest.mock.MagicMock()` writes itself. The cost arrives later. A test that asserts `repo.insert.assert_called_once_with(7, ["lamp"])` fails when you rename a parameter, reorder two calls, or batch two inserts into one — all changes that leave the behaviour identical. That is a test coupled to the implementation, and a suite full of them makes refactoring *harder*, which is the exact opposite of what tests are for.

I would use a mock when the effect leaving the system is the outcome — an email sent, a message published, a webhook fired — because there is no state to inspect. For anything with state, a fake and a state assertion is better.

When the double is the wrong answer entirely

A fake repository backed by a dict does not have a unique index, a foreign key, a transaction, or Postgres's exact behaviour on a concurrent update. Tests against it will pass while production fails. So:

I would not unit-test with a fake if the thing under test is the database interaction — the SQL itself, the migration, the isolation level from [day 034](#). Those need the real engine, and that is what the integration layer of the pyramid and `testcontainers` are for. The rule is: **fake the collaborator when you are testing your logic; use the real thing when you are testing the integration**. Say both halves. A candidate who claims everything can be unit-tested with fakes has not been burned yet.

The honest summary

Injection is not a testing trick that happens to improve design. It is a design property — *this class does not decide what its collaborators are* — that happens to make testing possible. If you argue for it only on testing grounds, an interviewer can reasonably ask “so why not just use `mock.patch`?” The better answer is that the same seam is what lets you swap Razorpay for Stripe, run the service against a read replica, or add caching without touching business logic.

8 In the interview

How it gets asked

- “How would you unit test this class?” — shown a class that constructs its own database connection. The expected answer is “I’d change the constructor first”, and saying that is the point.
- “What is dependency injection? Do you need a framework for it?” — no. It is passing arguments. Say that plainly; a surprising number of candidates think it requires Spring.
- “What’s the difference between a mock, a stub and a fake?” — the vocabulary question. Know all four, including dummy.
- “This method calls `datetime.now()`. How do you test the expiry logic?” — the clock question.
- “Your test suite takes 20 minutes. What do you do?” — the pyramid, and where the seams are missing.

What to say out loud, in the first ninety seconds

1. **Name the blocker before offering a technique.** “As written I can’t unit test this at all, because the constructor builds a Postgres connection and an SMTP client. There’s nowhere for a test to stand.”

2. **State the change in one sentence.** *“I’d have it take an `OrderRepository` and a `Mailer` as constructor arguments instead of building them. That’s dependency injection, and it’s a signature change, not a framework.”*
3. **Say where the real ones get built.** *“The concrete `PostgresOrderRepository` gets named in one place — `main()`, the composition root. Nothing else in the codebase mentions the driver.”*
4. **Name the doubles precisely.** *“For the repository I’d write an in-memory fake backed by a dict — a real implementation, simplified. For the mailer I’d use a mock that records what it was asked to send, because ‘an email went out’ is an effect, not a state I can inspect.”*
5. **Give the reason that isn’t speed.** *“The real gain is reachability: I can make the repository raise `ConnectionError` on demand and assert that no confirmation email goes out. There’s no way to write that test against a live database.”*

The follow-ups

“Isn’t this just testing your fake rather than your code?” It is a fair challenge and the answer has two halves. What the unit test verifies is my class’s logic — that it validates the input, that it calls the repository before the mailer, that it doesn’t send a confirmation when the save failed, that it propagates the right exception. None of that is fake behaviour; it is my behaviour, and it is the part most likely to be wrong. What the unit test does *not* verify is that `PostgresOrderRepository` writes the right SQL, respects the unique constraint, or behaves correctly under a concurrent update — and I would never claim it does. That is exactly what the integration layer is for: a smaller number of tests running against a real Postgres, usually started by testcontainers, that test the adapter and nothing else. The way I’d keep the fake honest is to write one contract test — a single set of test cases run twice, once against the fake and once against the real repository — so that any behaviour the fake gets wrong shows up as a failure rather than as a false pass. If I only had time for one thing, it would be that contract test, because a fake that has drifted from the real implementation is worse than no fake at all.

“Do you need a dependency injection framework?” No, and I would push back gently on the assumption. Dependency injection is passing arguments to a constructor; it is a technique, not a library. In Python I do it by hand and it costs one function in `main()`. A container earns its place at a certain scale — when you have a hundred services with a deep graph of dependencies, wiring them manually becomes a large fragile function, and something like Spring in Java or `dependency-injector` in Python, or FastAPI’s `Depends` for the per-request case, saves real work. What the container buys is lifecycle management — singletons, per-request scopes, lazy construction — not the injection itself. What it costs is that the wiring becomes implicit, so a missing registration fails at startup or, worse, at first use, with a stack trace that points at the container rather than at the code. For anything under a few dozen classes I would write the composition root by hand, because a function I can read beats a graph I have to trust.

“Your suite takes twenty minutes. Walk me through fixing it.” First I measure rather than guess — pytest’s `--durations=25` will usually show that a small number of tests own most of the time, and very often it is per-test database setup. Then I look at the shape of the

suite. If almost every test needs a real database, the problem is not the tests, it is that the classes construct their own collaborators, so every test is forced to be an integration test. That is the thing to fix, and it is fixed one class at a time: change the constructor to take the repository, write the in-memory fake once, and move the tests that were only ever exercising business logic down to the unit layer. Second, I check for shared expensive setup that runs per test rather than per session — migrations especially, which should run once against a template database that each test copies or truncates. Third, the truly slow tests that genuinely need the real thing get kept, but bounded: ten end-to-end tests covering critical journeys, not four hundred. And I would run the whole thing in parallel with `pytest-xdist`, which only works if tests share no global state — so it also flushes out the singletons. The target shape is a thousand-odd unit tests running in under a second, which is a suite people run on every save, and a hundred slower ones in CI.

A model answer

“As it stands I can’t unit test this class at all, and I’d say that first rather than reaching for a mocking library. The constructor builds a `PostgresConnection` and an `Smtplib` itself, so there is no seam — no place a test can stand between this class and the outside world. Any test I write needs a live database and an SMTP server, and it sends real email.

The fix is a signature change, not a framework:

```
class OrderService:
    def __init__(self, orders: OrderRepository, mailer: Mailer) → None:
        self._orders = orders
        self._mailer = mailer
```

`OrderRepository` and `Mailer` are `Protocol`s, so implementations don’t inherit anything from me. The concrete `PostgresOrderRepository` is named in exactly one place — `main()`, the composition root — and nothing else in the codebase imports the driver.

Then the test is three lines of setup. For the repository I’d write an in-memory fake backed by a dict: a real, simplified implementation, so I can assert on the state afterwards. For the mailer I’d use a recording double — a mock — because ‘a confirmation email was sent’ is an effect leaving the system, not a state I can inspect. I’d generally prefer fakes to mocks, because a mock asserts *how* the class did its job and breaks when I refactor something that still behaves correctly.

The reason I’d actually argue for this isn’t speed, although 800 tests going from 48 seconds to 0.3 seconds does change how often people run them. It’s reachability. About half the branches in a service class are error handling — connection lost, duplicate key, payment declined — and with a real database I can’t reach any of them. With an injected repository I hand in one that raises `ConnectionError` and assert that no confirmation email goes out. That’s the bug the test exists to prevent, and there’s no other way to write it.

What I wouldn’t claim is that this replaces integration testing. The fake doesn’t have a unique index or a transaction, so the SQL and the constraints still need a real Postgres — testcontainers, at the integration layer, maybe ninety tests rather than eight hundred. And I’d write one contract test run against both the fake and the real repository, so the fake can’t silently drift.”

9 Recall card

- **The whole technique in one sentence: a class asks for what it needs and never goes looking for it.** A constructor that names a driver, host or URL has welded itself shut; take the collaborator as an argument instead. That is **dependency injection**, and it needs no framework — it is argument passing.

- **Every injected dependency is a seam.** Three things destroy testability: constructing collaborators inside `__init__` · calling `datetime.now()`, `random`, `uuid` or `os.environ` directly · global mutable state. Inject the first, inject a **clock** for the second, delete the third.
- **Four doubles, used precisely:** dummy (never used) · stub (canned answers) · **fake** (a real simplified implementation — the in-memory dict) · mock (records calls). **Prefer fakes;** a mock asserts *how* and breaks on honest refactors. Use a mock when the outcome is an effect leaving the system.
- **The argument isn't speed, it's reachability.** ~18 of 40 branches in a service are error paths and none are reachable with a live database; a repository that raises `ConnectionError` on demand makes them all testable. Speed follows: 800 tests, 48 s → 0.3 s.
- **Concrete classes are named once, in the composition root** (`main()`, or FastAPI's `dependency_overrides`); `grep -rn "psycpg" src/ --exclude=main.py` should be empty. And concede the limit: **fake the collaborator to test your logic, use the real engine (test-containers) to test the integration** — plus one contract test run against both, so the fake cannot drift.

DSA topic: Merge sort **System design topic:** Writing clean, testable classes

Code these, in this order

One rule for the whole set: **write `merge` on its own and test it before you write anything that calls it.** Every bug in merge sort is a bug in the merge.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Merge Sorted Array	LeetCode 88 (Easy)	The merge step alone — and whether you spot that it must run backwards.
2	Merge Two Sorted Lists	LeetCode 21 (Easy)	The same merge on linked data, where relinking replaces copying.
3	Sort an Array	LeetCode 912 (Medium)	Whether you can produce the whole sort, base case included, under a time limit.
4	Sort List	LeetCode 148 (Medium)	Why merge sort is <i>the</i> answer for linked lists — no random access needed, $O(1)$ extra space.

On problem 1, write the forward version first and break it

Write the forward merge that copies into `nums1` from the front. Run it on `nums1 = [1, 2, 3, 0, 0, 0]`, `nums2 = [2, 5, 6]` and watch it pass. Then run it on `nums1 = [4, 5, 6, 0, 0, 0]`, `nums2 = [1, 2, 3]` and read the output: `[1, 2, 3, 1, 2, 3]`. Say out loud what was destroyed and why merging backwards fixes it. This is the whole reason the problem exists.

On problem 2, notice what disappears

Merging two linked lists needs no output list at all — you relink the nodes you already have. Say out loud what the extra space is now ($O(1)$), and why the array version cannot do the same thing.

On problem 3, get the base case right on purpose

Write it with `if len(nums) == 1` first. Run it on `[]`. Paste the `RecursionError`. Then fix it to `<= 1` and say the general rule: the base case must cover every input that cannot shrink further.

Then submit the recursive version and the bottom-up version, and compare the runtimes. They are the same complexity; note which is faster in Python and guess why before you look it up.

On problem 4, connect it to yesterday

Sort List is where merge sort stops being an exercise. Say out loud, before coding, why quick-sort is the wrong choice here (no random access, so choosing and reaching a pivot is $O(n)$ each time) and why merge sort is right.

The merge drill

Write `merge` from memory, then check each of these by running it:

1. `merge([1, 3, 5], [2, 4, 6])`
2. `merge([1, 2, 3], [])` — does your version keep the leftovers?
3. `merge([], [])`
4. `merge([5], [1])`
5. `merge([1, 1, 1], [1, 1])` — how many comparisons, and which side wins each tie?

Then add one assertion to your merge — `assert len(out) == len(left) + len(right)` — and say which of the five cases it would have caught if you had forgotten the `extend` lines.

The trace drill

Without running anything, write out every level of the split and every merge for `[38, 27, 43, 3, 9, 82, 10]`. Then count:

1. How many levels?
2. How many values are merged at each level?
3. How many comparisons in total, at most?
4. What is the deepest the recursion goes?
5. How much extra memory exists at the widest moment?

The complexity drill

Answer out loud, in under fifteen seconds each:

1. Why is merge sort $O(n \log n)$? Give the two facts and multiply them.
2. Why is binary search $O(\log n)$ and not $O(n \log n)$, when both halve?
3. What is merge sort's best case? Why is that answer unusual?
4. What is the extra space, and which half do you have to copy?
5. At `n = 100,000`, how many operations for insertion sort and for merge sort?

The break-it drill

Introduce each bug, run it, and read what happens:

1. `len(nums) / 2` instead of `//`. Paste the error.
2. `if len(nums) == 1` instead of `≤ 1`, called on `[]`. Paste the error.
3. Delete both `extend` lines. Run `merge([1, 2, 9], [3, 4])` and say what is missing.

4. Change `≤` to `<` in the merge. Sort `[("b", 1), ("a", 2), ("c", 1)]` by the number and say what moved.
5. Split as `nums[:middle]` and `nums[middle - 1:]`. Predict the failure before you run it.

The follow-up drill

Count inversions — the number of pairs `(i, j)` with `i < j` and `nums[i] > nums[j]`:

1. Write the `O(n2)` double loop first. Check it on `[2, 4, 1, 3, 5]` (the answer is 3).
2. Now add the counting to your merge. The line is `crossing += len(left) - i`, and you should be able to say *why* that is the right number before you write it.
3. Check both versions agree on ten random lists.
4. Say what the answer is for a reversed list of `n` elements, and why.

The testability drill

Take this class and make it testable, then write the two tests:

```
class ReminderService:
    def __init__(self) → None:
        self.db = PostgresConnection(os.environ["DATABASE_URL"])
        self.sms = TwilioClient(os.environ["TWILIO_KEY"])

    def send_due_reminders(self) → int:
        due = self.db.query("SELECT * FROM reminders WHERE due_on ≤ %s", date.today())
        for row in due:
            self.sms.send(row["phone"], row["message"])
        return len(due)
```

1. Name every dependency, and say which are hidden.
2. Rewrite the constructor. Say what type each parameter has and why it is a `Protocol` rather than a concrete class.
3. Write the in-memory fake repository. Fifteen lines, no more.
4. Write the recording double for the SMS client, and say why it is a mock rather than a fake.
5. Deal with `date.today()`. Say whether you inject it per call or per object, and why.
6. Write the test for “a reminder due tomorrow is not sent today”.
7. Write the test for “if the SMS client raises, the remaining reminders are still attempted” — or argue that they should not be, and test that instead.
8. Write the composition root, and give the `grep` command that proves the driver is swappable.

The vocabulary drill

Define each in one sentence, then give an example from the `ReminderService` above:

1. Dependency injection — and whether it needs a framework.
2. Seam.
3. Dummy, stub, fake, mock — all four, and which you would use for the SMS client.
4. Composition root.

5. Contract test — and why a fake needs one.
 6. The test pyramid — with the rough counts and times at each layer.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Write merge sort. Why is it $O(n \log n)$?* The merge first, the two `extend` lines and the `≤` explained, then the two facts — n per level, $\log n$ levels — and why “it halves” is not the answer on its own.
 2. *How would you unit test this class?* Name the blocker, change the constructor, name the doubles precisely, and give the reachability argument rather than the speed one. Then concede what still needs a real database.
 3. *Merge sort or quicksort?* The guarantee, stability, memory, and the three situations where merge sort is the only answer — adversarial input, multi-key sorting, and data that does not fit in memory.
-

Before you move on

- ☐ I wrote `merge` from memory and it handles empty lists and leftovers correctly.
- ☐ I broke LeetCode 88 with the forward merge and can say exactly which value got destroyed.
- ☐ I can give the two facts behind $O(n \log n)$ and say why binary search is different.
- ☐ I triggered the `RecursionError` from `= 1` and can state the general base-case rule.
- ☐ I can say where the $O(n)$ extra space goes, and which half does not need copying.
- ☐ I made `ReminderService` testable and wrote both tests without a database running.
- ☐ I can name all four test doubles and say when a mock is the right choice.
- ☐ I answered all three questions above out loud.

Quicksort and partitioning

DATA STRUCTURES AND ALGORITHMS

Quicksort and partitioning

You can partition an array in place and explain the worst case and how to avoid it.

SYSTEM DESIGN

Object-oriented design revision and interview questions

You can take an unseen LLD prompt and produce classes in twenty minutes.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Write quicksort. What is its worst case, and when does it happen?*
- *Design the classes for a hotel booking system.*

1 What this is, and why they ask it

Quicksort picks one value out of the list — the **pivot** — and rearranges everything so that smaller values sit to its left and larger values to its right. After that one rearrangement, the pivot is in its final position forever, and the two sides can be sorted independently by the same method. That rearrangement is called **partitioning**, and it is the whole algorithm.

They ask it because it is the sort that is actually used, and because it is the one with an interesting failure. Quicksort is $O(n \log n)$ on average, sorts in place with $O(1)$ extra data space, and beats merge sort by a factor of two or three in practice — so it is what C's `qsort`, Java's primitive sort and C++'s `std::sort` are built on. But its worst case is $O(n^2)$, and the input that triggers it is the one people least expect: an already sorted list. Being able to say which pivot choice causes that, why randomising fixes it, and what “expected $O(n \log n)$ ” actually promises is the difference between reciting quicksort and understanding it. The partition step is also a skill on its own — it is exactly what [day 055](#) uses to find the k-th largest in linear time, and the three-way version is the Dutch national flag problem.

2 The story

There is a man called Selvam who sells potatoes from a cart on the road outside the market in Madurai, and on Monday mornings a lorry drops about eighty kilos of them in one heap on the tray at the back of his cart. Big ones, small ones, everything mixed together. The hotels want the big ones and pay more for them, so the heap has to end up graded.

The way he does it is worth watching.

He reaches into the heap and picks up one potato — not the top one, one from somewhere in the middle — and he holds it in his left hand. That one is the measure. Then, with his right hand, he takes every other potato in turn, holds it briefly next to the one in his left hand, and drops it into one of two crates: the crate on his left if it is smaller than the one he is holding, the crate on his right if it is bigger. He does not compare it to anything else. One look, one crate.

When the heap is gone, he puts the potato from his left hand down in the gap between the two crates, and that potato is finished. He knows, without checking anything else, that everything to its left is smaller and everything to its right is bigger. It will not move again.

Then he does exactly the same thing inside the left crate. Picks one out, splits the rest into two around it, puts it down in its place. And inside the right crate. And inside each of those. The crates keep getting smaller until there is nothing left to split.

His brother-in-law tried to help one week and did it badly, and Selvam still brings it up. Instead of reaching into the middle, the brother-in-law kept grabbing whichever potato was nearest — which, because the lorry tips them out roughly graded, was almost always one of the smallest in the heap. So every single other potato went into the crate on the right. Nothing went left. The heap had not been split at all; it had just moved. He did that over and over, and after an hour he had compared nearly every potato to nearly every other potato and the tray was still a mess.

Selvam's rule, which he told him afterwards and repeats to anyone who will listen: never take the one that is closest to your hand. Reach in and take one from the middle of the heap, where you cannot see what you are getting.

3 The idea in plain English

Selvam is running quicksort. The potato in his left hand is the **pivot**. Splitting the heap into two crates around it is the **partition**. Putting the pivot down between the crates is the moment it reaches its final position. Doing the same thing inside each crate is the recursion. And his brother-in-law is the $O(n^2)$ worst case.

The one idea: partitioning

Everything today rests on this, so learn it separately from the sort.

Partitioning means rearranging a list around a chosen value so that everything smaller than it comes first, then the value itself, then everything larger. It takes one pass and $O(n)$ time, because each element is looked at once and compared to one thing.

before	[7 , 2 , 9 , 4 , 1 , 8 , 3]	pivot = 4
after	[2 , 1 , 3 , 4 , 7 , 9 , 8]	
	\-----/ ^ \-----/	
	smaller final larger	
	place	

Two facts about that result matter, and they are worth saying out loud because candidates often only say the first:

1. **The pivot is now in its final sorted position.** Not approximately — exactly. Three values are smaller than 4, so 4 belongs at position 3, and that is where it is. It never has to be looked at again.
2. **The two sides are not sorted.** [2, 1, 3] is not in order. All that is known is that everything on the left is smaller than the pivot and everything on the right is larger.

That second fact is what makes the recursion honest: each side is a smaller version of the same problem, and no element ever has to cross the pivot.

The sort, in one sentence

Partition, then quicksort the left part, then quicksort the right part.

There is no merge step. That is the structural difference from merge sort on [day 053](#), and it is why quicksort sorts in place. Merge sort does its work on the way *back up* the recursion, when it combines. Quicksort does its work on the way *down*, when it partitions, and the way back up does nothing at all.

The pivot choice, which is the whole story

Whichever value you choose as the pivot ends up between the two sides, so the *size* of the two sides depends entirely on how central the pivot's value is.

```
a good pivot: the median
n elements → two sides of n/2 each → depth  $\log_2(n)$  →  $n \log n$  total

a bad pivot: the smallest value
n elements → sides of 0 and n-1 → depth n →  $n^2$  total
```

If you always pick the first element as the pivot, then on an **already sorted list** the first element is always the smallest, every partition produces one empty side, and quicksort degenerates into something with the shape of selection sort — $n(n-1)/2$ comparisons. Sorted input is a common input. That is the trap, and it is exactly the brother-in-law grabbing the nearest potato.

Three fixes, in increasing order of how much interviewers like them:

- **Random pivot.** Choose a position uniformly at random and swap it to the end before partitioning. No fixed input can be bad, because the behaviour no longer depends on the input's arrangement. This is the answer to give.
- **Median of three.** Look at the first, middle and last elements and use the middle-valued one. Cheap, deterministic, and it handles sorted and reverse-sorted input — but a determined adversary who knows your rule can still construct a bad input.
- **Introsort.** Count the recursion depth, and if it exceeds about $2 \log_2 n$, stop and finish with heapsort. That gives a hard $O(n \log n)$ guarantee. It is what `std::sort` in C++ actually does.

The other bad input: many duplicates

A list where every value is the same is the second worst case, and it surprises people. With the simple partition scheme, every element compares equal to the pivot, they all go to one side, and you get the $O(n^2)$ behaviour again on input that is trivially sorted.

The fix is a **three-way partition**: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two. All the equal values land in their final positions at once. This is the Dutch national flag problem and it is worth having in your hands — it turns a list of one million identical values from $O(n^2)$ into $O(n)$.

The two things quicksort gives up

- **It is not stable.** Partitioning swaps elements across long distances, so equal elements change relative order. Merge sort is stable; quicksort is not, and no simple fix keeps its in-place property.
- **The worst case is real.** $O(n^2)$ is not merely theoretical if the pivot rule is deterministic and the input can be chosen by someone else. There was a well-known denial-of-service class of bug built exactly on this.

The thing it gives you

Speed, from staying in place. Quicksort touches memory that is already nearby, so the processor's memory cache — the small fast store described on [day 003](#) — keeps working for it. Merge sort allocates new lists at every level and walks between them. Both are $n \log n$, and quicksort is typically two to three times faster in wall-clock time on an array of numbers.

4 The picture

One partition, step by step. The pivot is the last element; **i** marks the boundary of the “smaller” region, and **j** scans:

pivot = 3 (the last element)

index	0	1	2	3	4	5	6
	7	2	9	4	1	8	3

^ ^
i pivot
j scans from 0 to 5

```
j=0 7 > 3   leave it.           i stays at 0
j=1 2 < 3   swap nums[0], nums[1] → [2, 7, 9, 4, 1, 8, 3], i → 1
j=2 9 > 3   leave it.
j=3 4 > 3   leave it.
j=4 1 < 3   swap nums[1], nums[4] → [2, 1, 9, 4, 7, 8, 3], i → 2
j=5 8 > 3   leave it.
```

finally, swap the pivot into position i:
swap nums[2], nums[6] → [2, 1, 3, 4, 7, 8, 9]
^
pivot is HOME at index 2

index	0	1	2	3	4	5	6
	2	1	3	4	7	8	9

\-----/ ^ \-----/
< pivot FINAL > pivot
(unsorted) (unsorted)

What to notice: `i` is not a scanner, it is a **boundary**. Everything before `i` is known to be smaller than the pivot; everything from `i` to `j` is known to be larger. The invariant is the whole trick, and it is the thing to state out loud before you write the loop.

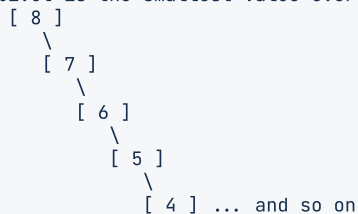
A good pivot against a bad one, drawn as trees:

GOOD: pivot is the median every time
[8 elements]



depth = $\log_2(8) = 3$. Work per level = n . Total = $n \log n$.

BAD: pivot is the smallest value every time (sorted input, first-element pivot)



depth = $n = 8$. Work per level = $n, n-1, n-2 \dots$ Total = $n(n-1)/2 = n^2$.

What to notice: the good tree is bushy and shallow; the bad one is a straight line. The total work per level is the same in both — the difference is entirely the number of levels, and the pivot choice decides that.

Three-way partitioning, for when duplicates are everywhere:

before [2 , 5 , 2 , 9 , 2 , 1 , 2] pivot value = 2

after [1 | 2 , 2 , 2 , 2 | 9 , 5]

less EQUAL greater
all four in their FINAL positions at once

recurse into [1] and [9, 5] only.

With a two-way partition, those four 2s would be split and re-split
 $\log n$ times for nothing.

What to notice: the equal block is skipped entirely by the recursion. A list of a million identical values goes from $O(n^2)$ to a single $O(n)$ pass.

5 The code, built step by step

The partition, on its own

This is the Lomuto scheme — one scanning position, one boundary. It is the one to write in an interview because the loop has a single moving part.

```
def partition(nums: list[int], lo: int, hi: int) → int:
    """Rearrange nums[lo:hi+1] around nums[hi]. Return the pivot's final position."""
    pivot = nums[hi]
    i = lo                                # boundary: everything < i is smaller
    for j in range(lo, hi):
        if nums[j] < pivot:
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]  # put the pivot at the boundary
    return i
```

Read the invariant, because it is what makes the code obviously correct: **at every point in the loop, `nums[lo:i]` holds values smaller than the pivot and `nums[i:j]` holds values that are not.** When `j` reaches `hi`, everything has been classified, and swapping the pivot into position `i` puts it exactly between the two groups.

The returned value is the pivot's final position. That is the only thing the caller needs.

The sort

```
def quicksort(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Sorts in place. Average  $O(n \log n)$ ; worst case  $O(n^2)$  with a fixed pivot."""
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:                                # 0 or 1 elements: nothing to do
        return
    p = partition(nums, lo, hi)
    quicksort(nums, lo, p - 1)                 # NOT p -- the pivot is already home
    quicksort(nums, p + 1, hi)
```

Note `p - 1` and `p + 1`. The pivot is finished, so it is excluded from both sides. Writing `quicksort(nums, lo, p)` instead is the classic infinite-recursion bug, and the error is in §7.

Note also that `lo ≥ hi` is the base case rather than `lo == hi`. An empty range has `lo > hi`, which happens whenever the pivot lands at either end.

Randomising the pivot, which is not optional

```
import random

def partition_random(nums: list[int], lo: int, hi: int) → int:
    r = random.randint(lo, hi)              # choose a position, not a value
    nums[r], nums[hi] = nums[hi], nums[r]   # move it to the end, then proceed as before
    return partition(nums, lo, hi)
```

Two lines, and they change the worst case from “sorted input” to “an outcome with probability roughly $1 / n!$ ”. Say why it works: the running time now depends on the random numbers, not on the arrangement of the input, so **no particular input is bad** — a person who can choose your input cannot choose your dice.

Median of three, the deterministic alternative

```
def median_of_three(nums: list[int], lo: int, hi: int) → None:
    """Put the median of nums[lo], nums[mid], nums[hi] at position hi."""
    mid = (lo + hi) // 2
    a, b, c = nums[lo], nums[mid], nums[hi]
    if (a ≤ b ≤ c) or (c ≤ b ≤ a):
        nums[mid], nums[hi] = nums[hi], nums[mid]
    elif (b ≤ a ≤ c) or (c ≤ a ≤ b):
        nums[lo], nums[hi] = nums[hi], nums[lo]
```

Cheap and it kills the sorted-input case completely, since the middle element of a sorted range is the median. It does not survive a deliberately constructed input, which is why library implementations combine it with a depth limit.

Three-way partitioning, for duplicates

```
def quicksort_three_way(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Dutch national flag. O(n) on a list of identical values."""
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:
        return
    pivot = nums[random.randint(lo, hi)]
    lt, i, gt = lo, lo, hi
    # [lo,lt) < pivot, [lt,i) = pivot, (gt,hi) > pivot
    while i ≤ gt:
        if nums[i] < pivot:
            nums[lt], nums[i] = nums[i], nums[lt]
            lt, i = lt + 1, i + 1
        elif nums[i] > pivot:
            nums[i], nums[gt] = nums[gt], nums[i]
            gt -= 1
            # i does NOT advance -- the swapped-in value is new
        else:
            i += 1
    quicksort_three_way(nums, lo, lt - 1)
    quicksort_three_way(nums, gt + 1, hi)
```

The line people get wrong is `gt -= 1` without advancing `i`. The value that was just swapped in from the right end has never been looked at, so `i` must stay where it is and examine it next time round.

Keeping the recursion depth at $O(\log n)$

Worst-case recursion depth is $O(n)$, which raises `RecursionError` in Python on large inputs. The standard fix is to recurse into the **smaller** side and loop on the larger one:

```
def quicksort_bounded(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Recursion depth is O(log n) even in the worst case."""
    if hi is None:
        hi = len(nums) - 1
    while lo < hi:
        p = partition_random(nums, lo, hi)
        if p - lo < hi - p:
            # recurse into the smaller half
            quicksort_bounded(nums, lo, p - 1)
            lo = p + 1
            # loop on the larger half
        else:
            quicksort_bounded(nums, p + 1, hi)
            hi = p - 1
```

Because the recursive call always gets at most half the range, the depth cannot exceed $\log_2 n$. The time complexity is unchanged; only the stack is bounded. This is a good thing to mention unprompted.

The complete file

```
"""Quicksort: partition in place, recurse into both sides, and choose the pivot properly."""
import random

def partition(nums: list[int], lo: int, hi: int) → int:
    """Lomuto. Rearrange nums[lo..hi] around nums[hi]; return the pivot's final index.

    Invariant: nums[lo:i] < pivot, and nums[i:j] ≥ pivot.
    """
    pivot = nums[hi]
    i = lo
    for j in range(lo, hi):
        if nums[j] < pivot:
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]
    return i

def partition_random(nums: list[int], lo: int, hi: int) → int:
    """Same, with a randomly chosen pivot -- so no fixed input is a bad input."""
    r = random.randint(lo, hi)
    nums[r], nums[hi] = nums[hi], nums[r]
    return partition(nums, lo, hi)

def partition_hoare(nums: list[int], lo: int, hi: int) → int:
    """Hoare's original: two positions walking toward each other.

    Does about 3x fewer swaps than Lomuto. Returns a SPLIT POINT, not the pivot's
    index -- so the caller must recurse on [lo, p] and [p+1, hi], not p-1 and p+1.
    """
    pivot = nums[(lo + hi) // 2]
    i, j = lo - 1, hi + 1
    while True:
        i += 1
        while nums[i] < pivot:
            i += 1
        j -= 1
        while nums[j] > pivot:
            j -= 1
        if i ≥ j:
            return j
        nums[i], nums[j] = nums[j], nums[i]

def quicksort(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """In place. Expected  $O(n \log n)$ , worst case  $O(n^2)$ . Not stable.

    Recursion depth is bounded to  $O(\log n)$  by always recursing into the smaller side.
    """
    if hi is None:
        hi = len(nums) - 1
    while lo < hi:
        p = partition_random(nums, lo, hi)
        if p - lo < hi - p:
            quicksort(nums, lo, p - 1)
            lo = p + 1
        else:
            quicksort(nums, p + 1, hi)
            hi = p - 1

def quicksort_three_way(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """Dutch national flag partitioning.  $O(n)$  when the values are mostly duplicates."""
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:
        return
    pivot = nums[random.randint(lo, hi)]
    lt, i, gt = lo, lo, hi
```

```

while i ≤ gt:
    if nums[i] < pivot:
        nums[lt], nums[i] = nums[i], nums[lt]
        lt, i = lt + 1, i + 1
    elif nums[i] > pivot:
        nums[i], nums[gt] = nums[gt], nums[i]
        gt -= 1
    else:
        i += 1
quicksort_three_way(nums, lo, lt - 1)
quicksort_three_way(nums, gt + 1, hi)

def sort_colors(nums: list[int]) → None:
    """LeetCode 75. Exactly a three-way partition around the value 1, in one pass."""
    low, i, high = 0, 0, len(nums) - 1
    while i ≤ high:
        if nums[i] == 0:
            nums[low], nums[i] = nums[i], nums[low]
            low, i = low + 1, i + 1
        elif nums[i] == 2:
            nums[i], nums[high] = nums[high], nums[i]
            high -= 1
        else:
            i += 1

if __name__ == "__main__":
    data = [7, 2, 9, 4, 1, 8, 3]

    trial = list(data)
    p = partition(trial, 0, len(trial) - 1)
    print(p, trial)          # 2 [2, 1, 3, 4, 7, 8, 9] -- pivot 3 is home

    trial = list(data)
    quicksort(trial)
    print(trial)             # [1, 2, 3, 4, 7, 8, 9]

    trial = [4, 4, 4, 4, 4, 4]
    quicksort_three_way(trial)
    print(trial)             # [4, 4, 4, 4, 4, 4] -- in ONE pass

    colors = [2, 0, 2, 1, 1, 0]
    sort_colors(colors)
    print(colors)            # [0, 0, 1, 1, 2, 2]

# the worst case, made visible
def count_comparisons(nums: list[int], first_pivot: bool) → int:
    count = 0

    def go(lo: int, hi: int) → None:
        nonlocal count
        if lo ≥ hi:
            return
        if first_pivot:
            nums[lo], nums[hi] = nums[hi], nums[lo] # pivot = the FIRST element
        else:
            r = random.randint(lo, hi)
            nums[r], nums[hi] = nums[hi], nums[r]
        count += hi - lo
        p = partition(nums, lo, hi)
        go(lo, p - 1)
        go(p + 1, hi)

    go(0, len(nums) - 1)
    return count

import sys
sys.setrecursionlimit(10000)
print(count_comparisons(list(range(500)), first_pivot=True))    # 124750 = n(n-1)/2
print(count_comparisons(list(range(500)), first_pivot=False))   # ~4000 = ~n log2 n

```


6 What it costs

The average case, counted

Partitioning a range of m elements costs $m - 1$ comparisons: every element except the pivot is compared with the pivot exactly once.

If the pivot lands near the middle each time, the picture is merge sort's picture:

```
level 0: 1 partition of n      = n comparisons
level 1: 2 partitions of n/2   = n
level 2: 4 partitions of n/4   = n
...
depth log2(n) levels
total = n × log2(n)
```

```
n = 1,000 → ~10,000 comparisons
n = 1,000,000 → ~20,000,000
```

The exact expected number with a random pivot is about $1.39 n \log_2 n$ — roughly 39% more comparisons than merge sort's $n \log_2 n$. Quicksort is still faster in practice because it does no allocation and its memory access pattern is friendlier, so each comparison is cheaper.

The worst case, counted

Pivot always smallest (or always largest):

```
level 0: partition of n      = n-1 comparisons
level 1: partition of n-1    = n-2
level 2: partition of n-2    = n-3
...
depth = n levels
total = (n-1) + (n-2) + ... + 1 = n(n-1)/2 = O(n2)
```

```
n = 500 sorted values, first-element pivot : 124,750 comparisons
n = 500 sorted values, random pivot       : ~4,000 comparisons
```

31x more work, on the input people are most likely to hand you.

How unlikely is the bad case, really?

With a random pivot, the probability that every one of the n partitions is maximally unbalanced is astronomically small — on the order of $1/n!$. A more useful number is that the probability of exceeding $2 \times 1.39 n \log_2 n$ comparisons is below one in a million for n in the thousands.

```
n = 1,000,000, random pivot:
  expected comparisons      ~ 28,000,000
  P(more than 2x the expectation) < 1 in 10^6
```

Compare: the chance of a specific bit in RAM flipping from cosmic radiation in an hour is higher than that.

That is the honest way to say “expected $O(n \log n)$ ”. It is not a guarantee, and if the requirement is a guarantee, use merge sort or heapsort.

The duplicates case

```
1,000,000 identical values:

two-way partition : every element goes to one side →  $n(n-1)/2 = 5 \times 10^{11}$  comparisons
three-way partition : one pass, all equal values placed → 1,000,000 comparisons

500,000x difference on input that requires no sorting at all.
```

Space

```
data      :  $O(1)$  extra -- it sorts in place, unlike merge sort's  $O(n)$ 
call stack:  $O(\log n)$  if you recurse into the smaller side
            $O(n)$  if you do not, and that raises RecursionError in Python
```

The $O(1)$ data space is the reason quicksort exists. Say it as the direct contrast with yesterday: **merge sort buys a guaranteed worst case with $O(n)$ memory; quicksort buys $O(1)$ memory and speed by giving the guarantee up.**

7 The traps

The real error: recursing on the pivot itself

```
def quicksort_broken(nums, lo=0, hi=None):
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:
        return
    p = partition(nums, lo, hi)
    quicksort_broken(nums, lo, p)      # ← should be p - 1
    quicksort_broken(nums, p + 1, hi)

quicksort_broken([2, 1])
```

```

Traceback (most recent call last):
  File "day54.py", line 9, in <module>
    quicksort_broken([2, 1])
    ~~~~~^~~~~~
  File "day54.py", line 7, in quicksort_broken
    quicksort_broken(nums, lo, p)
    ~~~~~^~~~~~
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded

```

If the pivot lands at position `lo`, then `quicksort(nums, lo, p)` is the same range you started with, and nothing shrinks. **The pivot is finished — exclude it from both sides.** This is the single commonest quicksort bug.

The real error: $O(n^2)$ on sorted input, in the wild

```

import sys
sys.setrecursionlimit(2000)

def quicksort_first_pivot(nums, lo=0, hi=None):
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:
        return
    nums[lo], nums[hi] = nums[hi], nums[lo]      # pivot = the first element
    p = partition(nums, lo, hi)
    quicksort_first_pivot(nums, lo, p - 1)
    quicksort_first_pivot(nums, p + 1, hi)

quicksort_first_pivot(list(range(5000)))        # an already sorted list

```

```

Traceback (most recent call last):
  File "day54.py", line 14, in <module>
    quicksort_first_pivot(list(range(5000)))
  File "day54.py", line 11, in quicksort_first_pivot
    quicksort_first_pivot(nums, p + 1, hi)
  [Previous line repeated 1994 more times]
RecursionError: maximum recursion depth exceeded

```

Read what happened: sorting an **already sorted list** crashed. Every partition put the pivot at the left end, so the right side shrank by exactly one each time and the recursion went 5,000 deep. In a language without a recursion limit it would not crash — it would just take n^2 time and look like a hang. **Two lines of randomisation prevent this entirely.**

The near-miss: `<=` instead of `<` in the partition

```

if nums[j] ≤ pivot:      # ← was <

```

The sort still produces correct output. But on a list where every value is equal, every element now satisfies the condition, `i` advances every time, and the pivot ends up at the far right — one side empty, $O(n^2)$, on input that needs no work at all:

```
quicksort_le(list([7] * 3000))
```

```
RecursionError: maximum recursion depth exceeded
```

With strict `<`, equal values are left in place and the pivot lands in the middle of them, which splits the range roughly in half. It is one character, and it decides whether a million duplicates take one second or five days.

The trap: assuming Hoare's partition returns the pivot's index

```
p = partition_hoare(nums, lo, hi)
quicksort(nums, lo, p - 1)      # ← WRONG for Hoare
quicksort(nums, p + 1, hi)
```

Hoare's scheme returns a **split point**, not the pivot's final position — the pivot may be anywhere in either part. The correct recursion is `(lo, p)` and `(p + 1, hi)`. Mixing the two schemes' calling conventions produces a sort that is wrong on some inputs and right on others, which is the worst kind of bug. Pick one scheme, and say which one you are writing.

The trap: claiming quicksort is stable

It is not. Partitioning swaps elements across arbitrary distances:

```
pairs = [(3, "first"), (1, "x"), (3, "second")]
# a quicksort by the number can produce:
[(1, 'x'), (3, 'second'), (3, 'first')]
```

The two 3s have swapped. If you need stability — sorting by a second key after a first, as on [day 051](#) — quicksort is the wrong choice and merge sort is the right one.

The trap: `random.shuffle` instead of a random pivot

Shuffling the whole list first also defeats the sorted-input case, and it is $O(n)$, so it does not change the complexity. But it destroys any existing partial order, which makes the sort slower on nearly-sorted data, and it changes the caller's list in a way they may not expect. Randomise the pivot, not the input.

8 In the interview

How it gets asked

- “Write *quicksort*.” — asked as often as merge sort, and the partition is what they watch.
- “What is *quicksort*'s worst case, and when does it happen?” — $O(n^2)$, on already-sorted input with a first-element or last-element pivot. Say *sorted* out loud; that is the part candidates miss.

- “How do you avoid the worst case?” — random pivot, and be able to explain *why* randomising works.
- “Quicksort or merge sort?” — the comparison question. In place and fast against guaranteed and stable.
- “Sort an array of 0s, 1s and 2s in one pass.” — LeetCode 75, which is exactly a three-way partition.
- “Partition an array around a value.” — the partition alone, which is also the engine of quickselect tomorrow.

What to say out loud, in the first ninety seconds

1. **Name the one idea before writing.** “Quicksort is really one operation — partition. I pick a pivot and rearrange so everything smaller is left of it and everything larger is right of it. After that the pivot is in its final position and I never touch it again.”
2. **State the partition invariant.** “I’ll keep a boundary `i`. Everything before `i` is smaller than the pivot; everything from `i` up to the scanner is not.”
3. **Randomise the pivot before you are asked.** “I’ll pick a random position and swap it to the end. Otherwise a sorted input gives $O(n^2)$, and sorted input is common.”
4. **Say what happens to the pivot in the recursion.** “I recurse on `lo` to `p-1` and `p+1` to `hi` — the pivot is excluded, because it’s finished. Including it is an infinite recursion.”
5. **Volunteer the properties.** “In place, $O(1)$ extra data space, $O(\log n)$ stack if I recurse into the smaller side. Expected $O(n \log n)$, worst case $O(n^2)$. Not stable.”

The follow-ups

“What is the worst case, and what input causes it?” $O(n^2)$, and it happens when the pivot is the smallest or largest value in the range every time, because then one side of the partition is empty and the other has `n-1` elements, so the recursion is `n` levels deep instead of `log n`, with `n`, `n-1`, `n-2` ... comparisons per level — that sum is $n(n-1)/2$. The input that causes it is the one people find surprising: if the pivot rule is “take the first element” or “take the last element”, then an **already sorted** array is the worst case, and so is a reverse-sorted one. That is not a rare input; it is one of the most likely inputs a function ever receives. The second worst case is an array where all the values are equal, if the partition uses `≤` rather than `<` — every element goes to one side and you get the same quadratic behaviour on data that needs no sorting at all. In Python both of those manifest as `RecursionError` rather than slowness, because the recursion goes `n` deep and the default limit is a thousand frames.

“How do you avoid it?” Three answers, and I would give them in this order. First, choose the pivot position at random and swap it to the end before partitioning — two lines. That works because the running time now depends on my random numbers rather than on the arrangement of the input, so there is no *particular* input that is bad; the probability of hitting the quadratic case is on the order of $1/n!$, and the probability of exceeding twice the expected comparison count is under one in a million for realistic sizes. That distinction matters if the input can come from outside the system, because a deterministic pivot rule is something an

attacker can construct against, and that has been a real denial-of-service class of bug. Second, median-of-three: take the first, middle and last elements and use the median of those. It is deterministic and cheap and it completely kills the sorted-input case, since the middle element of a sorted range is the median — but someone who knows the rule can still build a bad input. Third, and this is what production libraries actually do: introsort. Count the recursion depth, and if it exceeds about $2 \log_2 n$, abandon quicksort for that subrange and finish with heapsort. That gives a hard $O(n \log n)$ guarantee with quicksort's constant factor almost everywhere, and it is what `std::sort` in C++ does. I would also mention the separate fix for duplicates — three-way partitioning, which puts all the equal values in their final positions at once and turns a million identical values from $O(n^2)$ into a single linear pass.

“Quicksort or merge sort? Justify it.” Quicksort for arrays in memory, merge sort when I need a guarantee or stability or the data does not fit. The case for quicksort is the constant factor: it sorts in place, so it needs $O(1)$ extra data space instead of merge sort's $O(n)$, and it keeps touching memory that is already close by, which means the processor cache stays useful — in practice that is a two-to-three-times speed difference on the same array even though both are $n \log n$. Interestingly quicksort does *more* comparisons on average, about $1.39 n \log_2 n$ against merge sort's $n \log_2 n$; it wins anyway because each comparison is cheaper and there is no allocation. What I give up is the guarantee — the worst case is $O(n^2)$, and a randomised pivot makes it very unlikely rather than impossible — and stability, which quicksort cannot provide, because partitioning swaps elements across long distances. So I choose merge sort in three situations. When the input could be chosen adversarially and a rare quadratic case would be an outage rather than a slow query. When I need stability, for instance sorting records by a second key after a first. And when the data is a linked list or does not fit in memory, because merging is sequential and quicksort needs random access — external sorting and linked-list sorting are both merge sort. In Python, of course, I would call `sorted()`, which is Timsort: stable, adaptive, $O(n \log n)$ worst case, and written in C.

A model answer

“Quicksort is one operation repeated. I pick a value called the pivot, then rearrange the array so that everything smaller than it comes before it and everything larger comes after — that step is called partitioning. Once it’s done, the pivot is in its final sorted position and I never look at it again, and the two sides are independent subproblems.

```
def partition(nums: list[int], lo: int, hi: int) → int:
    pivot = nums[hi]
    i = lo                                # boundary: nums[lo:i] is all < pivot
    for j in range(lo, hi):
        if nums[j] < pivot:
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]
    return i

def quicksort(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    if hi is None:
        hi = len(nums) - 1
    if lo ≥ hi:
        return
    r = random.randint(lo, hi)
    nums[r], nums[hi] = nums[hi], nums[r]    # random pivot -- not optional
    p = partition(nums, lo, hi)
    quicksort(nums, lo, p - 1)
    quicksort(nums, p + 1, hi)
```

The invariant in the partition is that everything before `i` is smaller than the pivot, and everything from `i` to `j` is not — so when the scan finishes, swapping the pivot into `i` puts it exactly between the two groups. And the recursion excludes the pivot: `p - 1` and `p + 1`. If I wrote `quicksort(nums, lo, p)` and the pivot happened to land at `lo`, the range wouldn’t shrink and it would recurse forever.

I randomised the pivot deliberately, and I’d say why. The worst case is $O(n^2)$, and it happens when the pivot is the smallest or largest value every time — which, with a first- or last-element pivot, means an already sorted array. That’s a common input, not a rare one. Randomising means the running time depends on my dice rather than on the input’s arrangement, so no particular input is bad.

Cost: expected $O(n \log n)$ — about $1.39 n \log_2 n$ comparisons — worst case $O(n^2)$, $O(1)$ extra data space, and $O(\log n)$ stack depth if I always recurse into the smaller side and loop on the larger. It is not stable, because partitioning swaps across long distances. If I needed a guaranteed worst case or stability I’d use merge sort, and if the array were full of duplicates I’d switch to a three-way partition, which places all the equal values at once and turns that case from $O(n^2)$ into a single linear pass.”

9 Recall card

- **Quicksort is one operation: partition.** Pick a pivot, rearrange so smaller is left and larger is right — the pivot is then in its **final** position, and the two sides are independent. No merge step; the work happens on the way *down*, which is why it sorts **in place**.
- **The invariant to say out loud:** `nums[lo:i]` is all `< pivot`, `nums[i:j]` is all `≥ pivot`; at the end swap the pivot into `i`. Recurse on `p-1` and `p+1` — **excluding the pivot**, or it recurses forever.
- **Worst case $O(n^2)$, and the input is an already sorted list** with a first- or last-element pivot: one side empty, depth n , $n(n-1)/2$ comparisons. 500 sorted values: 124,750 comparisons against $\sim 4,000$ with a random pivot.
- **Fix it with two lines of randomisation** — the time now depends on your dice, not on the input, so no *particular* input is bad. Then median-of-three (deterministic, beatable) and **introsort** (bail to heapsort past depth $2 \log n$ — what `std::sort` does). Separately, **three-way partition** for duplicates: a million equal values goes from $O(n^2)$ to one pass.
- **Against merge sort:** quicksort is in place ($O(1)$ data, $O(\log n)$ stack if you recurse into the smaller side) and $2-3\times$ faster despite $\sim 39\%$ more comparisons. It gives up the **guarantee** and **stability**. Choose merge sort for adversarial input, multi-key sorting, linked lists, and data too big for memory.

Object-oriented design revision and interview questions

1 What this is, and why they ask it

This closes the object-oriented design phase. You have eleven days of tools — classes ([day 044](#)), encapsulation, inheritance, polymorphism, abstraction, composition, diagrams, domain modelling, the standard questions, and testable construction. Today is the day you use all of them at once, on a prompt you have not seen, with someone watching a clock.

They ask it as one question — “design the classes for X” — and the X changes every time: a hotel booking system, a parking lot, a library, a lift, a food delivery app, a chess game. What does not change is the shape of a good answer, and that is what a revision day is for. Candidates fail this round for one of three reasons, and none of them is not knowing what a class is. They start coding at minute two without scoping. They produce a set of data holders with a `Service` class doing all the work. Or they design something so rigid that the interviewer’s extra requirement at minute forty requires rewriting it. The fix for all three is a rehearsed order of moves, and this lesson is that order, run end to end on one prompt.

2 The story

Deepa learned to cook from her mother over about two years, in the way most people do — one thing at a time, whenever it happened to be being made. Sambar on a Tuesday. Beans on a Thursday. She could do each of them properly on her own and she knew it.

Then her mother went to Chennai for her aunt’s surgery, and on the Sunday eleven relatives were coming for lunch at one o’clock, and Deepa was the only one at home.

She got up at eight, which she thought was very early, and she was quite calm about it. She knew every single dish she had to make. That turned out not to be the thing that mattered.

At half past twelve she had four things ready and beautiful, and she had not started the rice. The rice takes forty minutes and it cannot be hurried and it is the only thing on the table that nobody will forgive you for. She also had two burners going and needed a third, and the coconut was not scraped, and one of the dishes had gone cold an hour ago and would have to be warmed again at the last minute, which she had not left a burner free for.

They ate at ten past two. Everything was good. Her uncle said so twice, and she cried a bit in the kitchen afterwards anyway, because she had known how to make all of it and it had still gone wrong.

A month later there were guests again, and she did one thing differently before she lit anything at all. She stood in the kitchen for ten minutes and worked out the order. What takes longest starts first. What goes cold last, goes last. Two burners means two things at once and not three. What can be done the night before gets done the night before.

That day she was sitting down with everybody at five past one, and there was a stretch in the middle where she had nothing to do for six or seven minutes and stood at the door, which had never happened before.

Her mother, when she got back and heard about both Sundays, was not surprised by either of them. Knowing the dishes and being able to put lunch on the table at one o'clock, she said, are two different things, and only one of them is taught one Tuesday at a time.

3 The idea in plain English

Deepa's second Sunday is the whole lesson. She did not learn a new dish between the two lunches. She spent ten minutes deciding the order before she lit anything, and that was the difference between ten past two and five past one.

A low-level design round is the same problem. You already know the tools. What decides the outcome is running a fixed sequence of moves, out loud, in the right order, and saying the plan at the start so the interviewer knows you have one.

The five moves, and the clock

For a 45-minute round:

MINUTES	MOVE	WHAT YOU PRODUCE
0-5	Clarify and scope	Three or four questions, and the answers you will assume. Say what you are leaving out.
5-10	The nouns	6 to 10 classes, each with one sentence of responsibility.
10-20	Relationships	The class diagram: who holds whom, with multiplicities.
20-35	Code one flow	The two or three classes carrying the interesting decision, written properly.
35-42	The extension	Their new requirement, answered in edits.
42-45	Gaps	What you knowingly left out, and what you would do next.

Say the plan at minute one: *“I’ll scope for five minutes, name the classes, draw relationships, then code the booking flow properly and leave time for whatever you want to add.”* That single sentence buys you the rest of the round, because the interviewer stops wondering whether you are lost.

The four habits that decide the score

These are the recurring judgements from the whole phase, compressed.

One: behaviour goes next to the data it needs. `room.can_host(party_size)`, never a type check inside a coordinator. A design that is fields plus a `BookingService` doing everything is the **anaemic model**, and it is the most common way to lose this round (day 044).

Two: find the noun nobody wrote down. Requirements say “a guest books a room”. `Guest` and `Room` are in the sentence; `Booking` is not, and `Booking` is where every interesting rule lives — the dates, the price, the cancellation deadline, the status. The test from day 051: *is there a rule, a date, or an amount of money with no home?* If yes, that is a class.

Three: put an interface exactly where the requirement will change. Not everywhere. Every prompt has one or two places where the business rule is genuinely variable — pricing, cancellation policy, spot allocation, fare calculation — and that is where an interface earns its keep. The test from day 048 still applies: **name the second implementation**. If you can name it, build the interface; if you cannot, do not.

Four: make it constructible. Hand dependencies in rather than fetching them (day 053). It costs nothing to write `Booking(pricing, policy, clock)` instead of building them inside, and when the interviewer says “how would you test the cancellation refund”, you have already answered it.

What “revision” means here

The rest of this lesson runs the five moves on one prompt — a hotel booking system — showing the first version, naming what breaks, and then fixing it. That last part matters more than the diagram. **Never present a design as if it arrived finished.** An interviewer learns far more from watching you notice a flaw than from a clean drawing.

4 The picture

The order of moves, which is the thing to memorise:

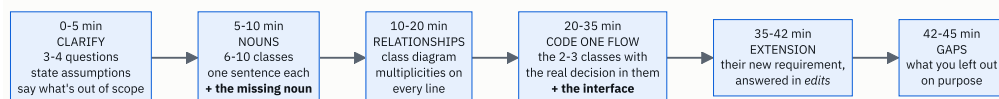


FIGURE 54.1

What to notice: twenty minutes pass before you write a line of code, and that is correct. The commonest failure in this round is starting at box four.

The finished hotel model, which §5 builds up to:

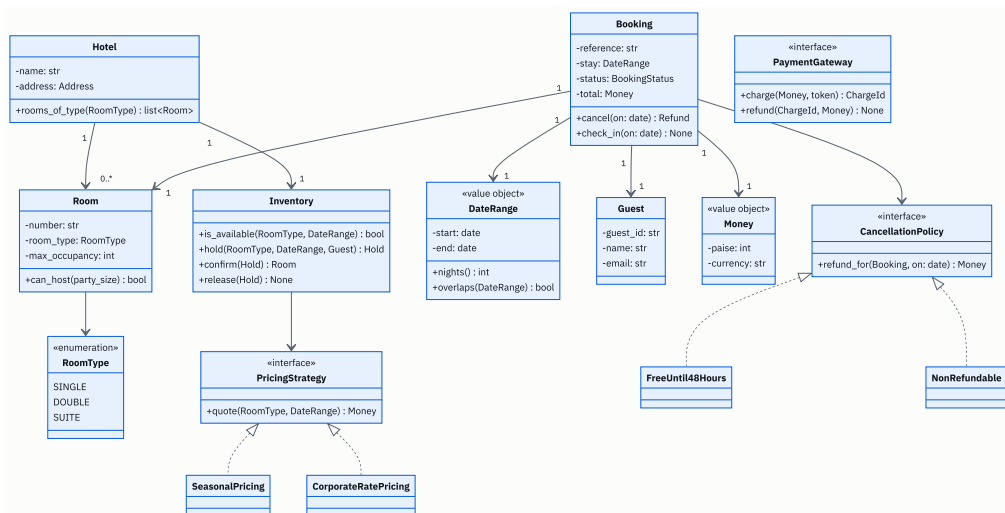


FIGURE 54.2

What to notice: ten boxes, and only three of them have methods worth listing — `Inventory`, `Booking` and `Room`. The two `<<interface>>` boxes sit exactly where the business changes its mind: what a night costs, and what you get back if you cancel. Everything else is a plain noun with a plain line. And `Booking` — the noun that was not in “a guest books a room” — is the class holding the most behaviour.

The concurrency problem, which is the question this prompt always gets:

one room left, room 412, and two guests click Book at the same instant

time	Anita's request	Bhaskar's request
----	-----	-----
t=0	is 412 free? → yes	
t=1		is 412 free? → yes
t=2	write booking A	
t=3		write booking B

result: two confirmed bookings for one room. Nobody saw an error.

What to notice: the gap between the check and the write is where the bug lives. Every version of this prompt — cinema seats, parking spots, ticket sales — has the same shape, and the fix is always to make the check and the write **one atomic operation**, not two. §5 shows the three ways.

5 How it actually works

Move 1 · Clarify, in five minutes

Ask three or four questions, state the answers you will assume, and name what you are leaving out. Real questions, and the assumptions a reasonable interviewer will accept:

“One property or a chain?” — Assume a chain, so `Hotel` is a real class rather than an implied singleton.

“Do guests book a room type, or a specific room?” — Assume they book a *type* and a specific room is assigned at check-in. This is how hotels actually work and it changes the model substantially, so it is the highest-value question to ask.

“Do we handle payment, or does an external gateway?” — Assume an external gateway behind an interface, and we store a charge reference.

“Are cancellation rules fixed, or do they vary?” — Assume they vary by rate plan. Ask this, because the answer creates an interface.

Then say the scope out loud: *“I’ll model search, booking, cancellation and check-in. I’m leaving out loyalty points, housekeeping, group bookings and multi-currency — tell me if you’d rather I include one.”* Naming the exclusions is what makes it scoping rather than forgetting.

Move 2 · The nouns, in five minutes

Pull them straight out of the requirements, one sentence of responsibility each:

- **Hotel** — one property. Owns its rooms and knows its address.
- **Room** — one physical room. Knows its number, its type, and how many people it can hold.
- **RoomType** — single, double, suite. An enumeration, not a class hierarchy — there is no behaviour that differs per type beyond data.
- **Guest** — the person booking. Identity, contact details.
- **DateRange** — a start and an end. A **value object**: frozen, compared by value, and it owns `nights()` and `overlaps()`.
- **Money** — a value object in the smallest unit. Never a `float`.
- **Inventory** — the class that answers “is a room of this type free for these dates” and holds one. This is the second missing noun and it is where the hard part lives.
- **Booking** — **the noun nobody wrote down**. A guest, a room type, a date range, a price, a status, and the rules for cancelling and checking in.
- **PricingStrategy** — an interface. What a night costs.
- **CancellationPolicy** — an interface. What you get back.
- **PaymentGateway** — an interface. Razorpay, Stripe, or a fake for tests.

Say the missing-noun reasoning out loud, because it is the highest-scoring sentence in the whole round: “A guest books a room’ has two nouns in it, but the price, the dates, the status and the cancellation deadline have nowhere to live. That is a `Booking`, and it is where most of the behaviour goes.”

Move 3 • The relationships

That is the diagram in §4. Draw it while narrating each line: “A hotel has many rooms — a room belongs to exactly one hotel. A booking has one date range, one guest, one room type, and after check-in, one room.” Put a multiplicity on every line. Nobody has ever been marked down for a plain association line, and plenty of people have been marked down for a diagram with no numbers on it.

Move 4 • The interesting part

Every LLD prompt has one place where the design is actually decided. In a parking lot it is spot allocation. In a lift it is scheduling. **In a hotel it is availability** — and specifically the gap between checking and booking.

Show the naive version first and name what breaks:

```
class Inventory:
    def book(self, room_type: RoomType, stay: DateRange, guest: Guest) → Booking:
        if self.is_available(room_type, stay):
            # CHECK
            return self.create_booking(room_type, stay, guest) # then WRITE
        raise NoRoomsAvailable(room_type, stay)
```

“That is correct with one guest and wrong with two. Between the check and the write, another request can run the same check and get the same answer. One room, two confirmed bookings, and no error anywhere.”

Then give the three fixes, in the order you would consider them:

One: make it atomic in the database. The real answer for a hotel. A conditional insert or an `UPDATE ... WHERE remaining > 0` does the check and the write in one statement, and the database guarantees only one succeeds:

```
UPDATE availability
    SET remaining = remaining - 1
    WHERE hotel_id = %s AND room_type = %s AND day = %s AND remaining > 0;
-- affected rows: 1 = you got it, 0 = someone else did
```

Say the isolation level and why: this is a lost-update problem, and `remaining > 0` inside the statement is what makes it safe under `READ COMMITTED` without an explicit lock ([day 034](#)).

Two: a two-phase hold. What booking systems actually do, because payment takes thirty seconds and you must not lose the room in the meantime:

```
hold → (10 minutes, expires on its own) → confirm or release
```

Three: a lock, if it is single-process. In one Python process a `threading.Lock` around check-and-write is enough, and saying “but that does not survive a second instance of the service” shows you know the limit of the answer.

Now the code for the classes that carry it. This is the part you write properly.

```
from dataclasses import dataclass, field
from datetime import date, timedelta
from enum import Enum
from typing import Protocol
import threading
import uuid

@dataclass(frozen=True)
class DateRange:
    """A stay. Value object: frozen, compared by value, owns its own arithmetic."""
    start: date
    end: date # exclusive: the check-out day

    def __post_init__(self) → None:
        if self.end ≤ self.start:
            raise ValueError("a stay must be at least one night")

    def nights(self) → int:
        return (self.end - self.start).days

    def days(self) → list[date]:
        return [self.start + timedelta(days=i) for i in range(self.nights())]

    def overlaps(self, other: "DateRange") → bool:
        return self.start < other.end and other.start < self.end
```

`DateRange` refusing to exist with `end ≤ start` is encapsulation doing real work ([day 045](#)) — an invalid stay cannot be constructed, so no downstream code has to check for one.

```
@dataclass(frozen=True)
class Money:
    paise: int
    currency: str = "INR"

    def __add__(self, other: "Money") → "Money":
        if other.currency ≠ self.currency:
            raise ValueError("cannot add different currencies")
        return Money(self.paise + other.paise, self.currency)

    def __mul__(self, n: int) → "Money":
        return Money(self.paise * n, self.currency)
```

Integers in the smallest unit, never floats. Saying “I’d store paise as an integer because floating point cannot represent 0.1 exactly” is a small sentence that reads as experience.

```

class PricingStrategy(Protocol):
    def quote(self, room_type: "RoomType", stay: DateRange) → Money: ...

class FlatRatePricing:
    def __init__(self, per_night: dict["RoomType", Money]) → None:
        self._rates = per_night

    def quote(self, room_type: "RoomType", stay: DateRange) → Money:
        return self._rates[room_type] * stay.nights()

class SeasonalPricing:
    """The second implementation, which is why the interface exists."""

    def __init__(self, base: PricingStrategy, peak_days: set[date], multiplier: float) → None:
        self._base, self._peak, self._multiplier = base, peak_days, multiplier

    def quote(self, room_type: "RoomType", stay: DateRange) → Money:
        total = Money(0)
        for day in stay.days():
            one_night = self._base.quote(room_type, DateRange(day, day + timedelta(days=1)))
            if day in self._peak:
                one_night = Money(int(one_night.paise * self._multiplier))
            total = total + one_night
        return total

```

Note that `SeasonalPricing` *wraps* a `PricingStrategy` rather than inheriting from `FlatRatePricing`. That is composition doing the work of a decorator, and it means corporate rates, seasonal loading and last-minute discounts can be stacked in any order without a class per combination ([day 049](#)).

```

class CancellationPolicy(Protocol):
    def refund_for(self, booking: "Booking", on: date) → Money: ...

class FreeUntilDaysBefore:
    def __init__(self, days: int) → None:
        self._days = days

    def refund_for(self, booking: "Booking", on: date) → Money:
        deadline = booking.stay.start - timedelta(days=self._days)
        return booking.total if on ≤ deadline else Money(0)

class NonRefundable:
    def refund_for(self, booking: "Booking", on: date) → Money:
        return Money(0)

```

Two implementations, named. That is the check from [day 048](#) passing, and it is why this interface is justified while an `IHotel` interface would not be.


```

class BookingStatus(Enum):
    HELD = "held"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CANCELLED = "cancelled"

@dataclass
class Booking:
    """The noun that was not in the requirements, and where the rules live."""
    reference: str
    guest: "Guest"
    room_type: "RoomType"
    stay: DateRange
    total: Money
    policy: CancellationPolicy # injected -- testable
    status: BookingStatus = BookingStatus.CONFIRMED
    assigned_room: "Room | None" = None

    def cancel(self, on: date) → Money:
        if self.status in (BookingStatus.CHECKED_IN, BookingStatus.CANCELLED):
            raise ValueError(f"cannot cancel a booking that is {self.status.value}")
        refund = self.policy.refund_for(self, on) # the rule lives in the policy
        self.status = BookingStatus.CANCELLED
        return refund

    def check_in(self, room: "Room", on: date) → None:
        if self.status is not BookingStatus.CONFIRMED:
            raise ValueError(f"cannot check in a booking that is {self.status.value}")
        if on < self.stay.start:
            raise ValueError("too early to check in")
        if not room.can_host(self.guest.party_size):
            raise ValueError(f"room {room.number} cannot host {self.guest.party_size} guests")
        self.assigned_room = room
        self.status = BookingStatus.CHECKED_IN

```

Every rule about a booking lives on `Booking`. There is no `BookingService.cancel(booking)` deciding what a cancellation means, and that is the difference between a model and a bag of fields.

```

class Inventory:
    """Availability per room type per day, with the check and the write made atomic."""

    def __init__(self, capacity: dict["RoomType", int], pricing: PricingStrategy) → None:
        self._capacity = capacity
        self._pricing = pricing
        self._booked: dict[tuple["RoomType", date], int] = {}
        self._lock = threading.Lock() # single process; see the trade-offs

    def available(self, room_type: "RoomType", stay: DateRange) → int:
        return min(
            self._capacity[room_type] - self._booked.get((room_type, day), 0)
            for day in stay.days()
        )

    def book(
        self,
        room_type: "RoomType",
        stay: DateRange,
        guest: "Guest",
        policy: CancellationPolicy,
    ) → Booking:
        with self._lock: # check and write, together
            if self.available(room_type, stay) ≤ 0:
                raise NoRoomsAvailable(room_type, stay)
            for day in stay.days():
                self._booked[(room_type, day)] = self._booked.get((room_type, day), 0) + 1
        return Booking(
            reference=uuid.uuid4().hex[:8].upper(),
            guest=guest,
            room_type=room_type,
            stay=stay,
            total=self._pricing.quote(room_type, stay),
            policy=policy,
        )

```

The `with self._lock` is one line, and pointing at it and saying “*this is the check-then-write race, and in a real deployment the lock has to be in the database or in Redis, not in the process*” is worth more than the rest of the class.

Move 5 · The extension, rehearsed

The interviewer will add a requirement at minute forty. For a hotel it is almost always one of three, and each has a prepared answer:

- “Now we want last-minute discounts.” — One new `PricingStrategy` implementation, wrapping the existing one. Zero edits to `Booking`, `Inventory` or anything else.
- “Now a booking can cover several rooms.” — `Booking` holds a list of line items rather than one room type. `Inventory.book` takes a list and reserves all of them under one lock, or none.
- “Now we overbook by 5% because of no-shows.” — `available()` becomes `capacity × 1.05 - booked`, in one class, one method.

Answer in **edits**, not in prose. “One new file and one wiring line” is a measurement; “we’d use the strategy pattern” is a word.

6 The numbers

How big is the object graph

```
A mid-size chain:
40 properties x 120 rooms = 4,800 Room objects
3 room types per property = 120 RoomType references (an enum: 3 objects total)
1 Inventory per property = 40

Availability, if stored per room type per day:
40 properties x 3 types x 365 days = 43,800 rows for a year
x 3 years forward booking = 131,400 rows

That fits in memory. Per ROOM per day it would be
4,800 x 365 x 3 = 5,256,000 rows -- 40x more, for no extra ability,
which is the arithmetic that justifies booking a type rather than a room.
```

That last comparison is the payoff of the clarifying question in move one. Ask it, then show the forty-times difference, and the interviewer sees a decision rather than a guess.

The concurrency window, measured

```
check-then-write, without a lock:

is_available() reads 3 days of availability ~ 0.4 ms
business logic ~ 0.1 ms
write the booking ~ 2.0 ms
-----
window in which a second request can pass ~2.5 ms

At 20 bookings/second across the chain, two requests for the LAST room
of the same type land inside 2.5 ms roughly:
20 req/s x 0.0025 s = 0.05 → about a 5% chance per contended room-night.

On a peak day with 300 sold-out room-nights: ~15 double bookings.
```

Fifteen a day is not a theoretical concern; it is a person standing at a counter at eleven at night. That number is why the atomic write matters, and giving it turns a hand-wave into an argument.

The hold window

```
payment flow: card details → gateway → 3-D Secure → confirmation
median 12 s, 95th percentile 40 s, timeout at 90 s

hold expiry = 10 minutes
comfortably above the 95th percentile
at 20 holds/second and a 10-minute window: 20 x 600 = 12,000 live holds
at ~200 bytes each = 2.4 MB -- trivially a Redis key with a TTL
```

Naming `SETEX` with a ten-minute TTL as the mechanism, and the 12,000 number as the size, is the level of concreteness this round rewards.

The cost of the design decisions, in edits

```
"add last-minute discounts"
  with a PricingStrategy interface : 1 new class + 1 wiring line
  with pricing inside Booking      : edit Booking, edit Inventory, edit 2 tests,
                                   re-verify every existing booking path = 4+ edits
                                   to code that already works

"add a non-refundable rate"
  with a CancellationPolicy        : 1 new class, 5 lines
  with an if-chain in cancel()      : 1 edit to the most safety-critical method
                                   in the system
```

7 The trade-offs

What this design gives up

Booking a type, not a room. The guest cannot choose room 412 with the balcony. That is a real product decision, and it buys the forty-times reduction in availability records plus the freedom to reassign rooms right up to check-in. **I would change it if** the business sells specific rooms — villas, or a heritage property where every room is different — and then availability becomes per room, the object count goes up forty times, and I would need a real search index rather than a dict.

An in-process lock. `threading.Lock` protects one Python process. Two instances behind a load balancer share nothing, and the race comes straight back. **I would not use this if** the service runs more than one instance, which it always does — the honest production answer is the conditional `UPDATE` in the database, or a Redis lock with a TTL if the availability store cannot express the condition. Saying that the lock is a placeholder, and naming its replacement, is the difference between a naive answer and a scoped one.

No overbooking. Real hotels deliberately sell more rooms than they have, because 5-10% of guests do not turn up. This model refuses the sale. **I would add it** as a multiplier inside `Inventory.available()` — one method, one line — which is exactly the kind of change a good boundary makes cheap.

Money as an integer of paise, single currency. Adding currencies means every arithmetic operation needs a rate and a date, and `Money + Money` across currencies must fail loudly, which it already does. That is the right shape to grow into.

Where the interfaces are, and where they deliberately are not

Two interfaces: `PricingStrategy` and `CancellationPolicy`. Both pass the “name the second implementation” test — seasonal against flat, free-until-48-hours against non-refundable — and both sit exactly where a business changes its mind quarterly.

There is deliberately **no** `IHotel`, no `IRoom`, no `IBookingRepository` at this stage. Those would be interfaces with one implementation and no fake, which is ceremony. **I would add `BookingRepository` the moment persistence appears**, and I would say the reason honestly: the second implementation is the in-memory fake for tests, and that counts ([day 053](#)).

RoomType as an enum rather than a hierarchy

A `Single / Double / Suite` class hierarchy is the tempting move and it is wrong here, because the types differ only in data — a rate and an occupancy — and not in behaviour. An enum plus a rate table is three lines against three classes. **I would change it if** a type acquired genuine behaviour that differed, for example a suite requiring a manager's approval to discount, and even then I would reach for a strategy object before a subclass.

What breaks at ten times the size

The `Inventory` dict is fine for 131,400 rows and hopeless for a chain of four thousand properties. At that point availability moves to the database with a composite key on `(hotel_id, room_type, day)`, `available()` becomes a range query with an index, and the atomic decrement becomes the conditional `UPDATE`. The class boundary does not change, which is the argument for having drawn it: **`Inventory` is an interface with a dict implementation today and a Postgres implementation tomorrow, and nothing else in the model notices.**

8 In the interview

How it gets asked

- “Design the classes for a hotel booking system.” — and the same round with a parking lot, a library, a lift, a vending machine, a chess game, or a food delivery app. The prompt changes; the five moves do not.
- “Just give me the class diagram.” — still do the five minutes of scoping first, out loud, and say why.
- “Two guests book the last room at the same moment. What happens?” — the follow-up this prompt always gets.
- “Now add [new requirement].” — at minute forty, every time. Answer in edits.

The script, minute by minute

Minutes 0-5 — clarify. “Before I model anything, four questions. One property or a chain? Do guests book a room type or a specific room? Do we take payment ourselves or through a gateway? Are cancellation rules fixed or do they vary by rate?” Then state your assumptions and your scope, including what you are leaving out.

Minutes 5-10 — nouns. Say them with one line of responsibility each, and land the missing-noun sentence: “A guest books a room’ names two classes, but the dates, the price, the status and the cancellation deadline have nowhere to live. That is a `Booking`.”

Minutes 10-20 — the diagram. Draw while narrating. Multiplicity on every line. Put the two interfaces in and say why each one is justified by naming its second implementation.

Minutes 20-35 — code the booking flow. `DateRange` first, because it is small and it shows the invariant being enforced in the constructor. Then `Booking.cancel`, because that is where the rule lives. Then `Inventory.book`, and stop at the check-then-write to name the race

before they ask.

Minutes 35-42 — the extension. Whatever they add, answer in edits, and say the number.

Minutes 42-45 — gaps. *“I’ve left out group bookings, loyalty, housekeeping and multi-currency deliberately. The one I’d do next is persistence, because that turns `Inventory` into a repository interface and the atomic decrement into a conditional `UPDATE`, and it’s the piece that decides whether the design survives two service instances.”*

The follow-ups

“Two guests click Book on the last room at the same instant. What happens?” With the code as written and no lock, both succeed, and that is the bug. The check and the write are two separate operations, and between them another request can run the same check and get the same answer — the window is about two and a half milliseconds, which at twenty bookings a second across the chain is roughly a five percent chance per contended room-night, so on a busy day it is a dozen or so real double bookings, not a theoretical one. The fix is to make the check and the write a single atomic operation. In the class I showed I used a `threading.Lock`, and I would immediately say that it only protects one process — the moment there are two instances behind a load balancer the race is back. The production answer is to push the condition into the database: an `UPDATE availability SET remaining = remaining - 1 WHERE ... AND remaining > 0`, and then check the affected row count. One statement, so there is no window, and it works under READ COMMITTED without an explicit lock because the predicate is evaluated inside the write. If availability lives somewhere that cannot express that, the alternative is a distributed lock in Redis keyed on hotel, room type and date, with a TTL so a crashed holder does not block the room forever. And separately from the race, I would use a two-phase hold, because payment takes twelve seconds at the median and forty at the ninety-fifth percentile — so the room is held for ten minutes with a TTL and then either confirmed or released automatically. That is what real booking systems do, and the expiring hold is also what protects you against a client that simply disappears.

“Where would you put an interface, and where would you not?” Two places, and the test is whether I can name the second implementation. Pricing gets one, because flat-rate, seasonal, corporate-negotiated and last-minute-discount are all real and the business adds one every quarter — and I would compose rather than subclass them, so `SeasonalPricing` wraps another `PricingStrategy` and any combination stacks without a class per combination. Cancellation policy gets one, because free-until-48-hours and non-refundable are both standard rate plans and the rule must not be an if-chain inside the most safety-critical method in the system. I would deliberately *not* create `IHotel`, `IRoom` or `IGuest`. Those would have exactly one implementation and no test double, so they buy nothing and cost a layer of indirection that a new reader has to walk through. The one I would add as soon as persistence enters is a `BookingRepository`, and I would be honest that the second implementation there is the in-memory fake for tests — that is a legitimate reason on its own, and it is usually the real reason interfaces exist. The general rule I apply: an interface earns its place where the requirement is known to vary, not everywhere a class exists.

“How would you test this?” Almost all of it without anything running, because I have handed the variable parts in rather than constructing them. `Booking` takes its `CancellationPolicy` as a field, so testing the refund rules is constructing a `Booking` with `FreeUntilDaysBefore(2)` and asserting the refund on the day before the deadline, on the deadline, and the day after — three lines each, no clock manipulation, because the date is a parameter to `cancel` rather than read from the system. `PricingStrategy` is a pure function from a room type and a date range to `Money`, so seasonal loading is testable in isolation, including the awkward case of a stay that straddles the start of the peak period. `DateRange` refuses to be constructed with an end on or before the start, so I test that it raises rather than testing that every caller checks. For `Inventory` I would use the real dict-backed implementation as the fake — it is a working implementation — and test the interesting cases: booking the last room succeeds, the one after it raises `NoRoomsAvailable`, a cancellation returns the inventory, and a stay that overlaps a sold-out day is rejected even if the other days are free. The one thing I cannot test this way is the actual race, and I would say so: concurrency needs either a test that fires two threads at a shared `Inventory` and asserts the total booked never exceeds capacity, or — for the real implementation — an integration test against a real Postgres, because the guarantee I am relying on is the database’s, not mine.

A model answer

“Before modelling, four questions. Is this one property or a chain? Do guests book a room type or a specific room? Do we take payment ourselves or through a gateway? And do cancellation rules vary by rate plan? I’ll assume a chain, booking by room type with the specific room assigned at check-in, payment through an external gateway behind an interface, and cancellation rules that vary. I’m scoping to search, book, cancel and check-in, and deliberately leaving out group bookings, loyalty and housekeeping.

The nouns from the requirements are `Hotel`, `Room`, `RoomType`, `Guest`. But ‘a guest books a room’ has a gap in it — the dates, the price, the status and the cancellation deadline have nowhere to live. That is a `Booking`, and it is the class that ends up holding most of the behaviour. There’s a second class that isn’t in the sentence either: `Inventory`, which answers ‘is a room of this type free for these dates’ and reserves one. That’s where the hard part is. And two value objects — `DateRange` and `Money`, both frozen, both owning their own arithmetic.

`DateRange` refuses to be constructed with an end on or before its start, so no downstream code has to check for a zero-night stay. `Booking.cancel(on)` asks its `CancellationPolicy` for the refund and changes its own status — the rule lives on the object that owns the data, not in a `BookingService`.

Two interfaces, and only two. `PricingStrategy`, because flat, seasonal and corporate rates all exist and the business adds one every quarter — and I compose them, so `SeasonalPricing` wraps another strategy rather than subclassing it. And `CancellationPolicy`, because free-until-48-hours and non-refundable are both standard. I can name the second implementation for both, which is my test for whether an interface is justified. I’d deliberately not create `IHotel` or `IRoom`.

The part I want to flag before you ask: `Inventory.book` checks availability and then writes, and those are two operations. Two guests clicking Book on the last room can both pass the check — the window is a couple of milliseconds, which at twenty bookings a second is about a five percent chance per contended room-night, so a dozen real double bookings on a busy day. I’ve put a lock around it here, but that only protects one process. In production the check and the write have to be one statement — `UPDATE availability SET remaining = remaining - 1 WHERE ... AND remaining > 0` — and I’d add a ten-minute hold with a TTL in front of it, because payment takes twelve seconds at the median and forty at the ninety-fifth percentile and you must not lose the room in the meantime.”

9 Recall card

- **Run five moves on a clock, and say the plan at minute one:** 0-5 clarify and scope (name what you leave out) · 5-10 nouns, 6-10 classes, one sentence each · 10-20 the class diagram with a multiplicity on every line · 20-35 code the one flow that matters · 35-42 their extension, answered in **edits** · 42-45 gaps.
- **Find the noun nobody wrote down.** “A guest books a room” has no home for the dates, the price, the status or the cancellation deadline — that is `Booking`. Second missing noun: `Inventory`. The test: *is there a rule, a date or an amount of money with no home?*
- **Behaviour on the class that owns the data.** `booking.cancel(on)`, `room.can_host(n)`, `DateRange` refusing to exist with `end ≤ start`. Fields plus a `BookingService` is the anaemic model and it is how this round is lost.
- **An interface only where you can name the second implementation.** Here: `PricingStrategy` (flat/seasonal — and **wrap**, don’t subclass) and `CancellationPolicy` (free-until-48h / non-refundable). Not `IHotel`. A repository counts, because the second implementation is the test fake.
- **Every version of this prompt has the same race: check, then write.** ~2.5 ms window ≈ 5% per contended room-night ≈ a dozen double bookings a day. Fix it by making check-and-write **one** operation — `UPDATE ... WHERE remaining > 0` — plus a 10-minute **hold with a TTL**, because payment is 12 s median and 40 s at p95. An in-process lock is a placeholder; say so.

DSA topic: Quicksort and partitioning **System design topic:** Object-oriented design revision and interview questions

Code these, in this order

One rule for the whole set: **write `partition` on its own and check the returned position before you write the sort.** Every quicksort bug is a partition bug or a recursion-bounds bug.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sort Colors	LeetCode 75 (Medium)	A three-way partition with the pivot value fixed at 1 — the Dutch flag, in its purest form.
2	Partition Array According to Given Pivot	LeetCode 2161 (Medium)	Three-way partitioning where <i>stability</i> is required, so the in-place swap version is wrong.
3	Sort an Array	LeetCode 912 (Medium)	Whether your quicksort survives sorted input and 50,000 duplicates.
4	Wiggle Sort II	LeetCode 324 (Medium)	Partitioning as a tool rather than a sort — find the median, then place around it.

On problem 1, write it as a partition, not as counting

The counting solution (count the 0s, 1s and 2s, then overwrite) is two passes and it is fine. Write it, then write the one-pass three-region version, and say out loud which region each of `low`, `i` and `high` bounds. Then answer: why does `i` not advance when you swap with `high`?

On problem 2, discover why stability matters

Solve it with the in-place three-way swap first and check it against the expected output. It fails, and the reason is that the problem requires the original relative order to be preserved inside each group. Say out loud which of the sorts you know are stable, and why partitioning cannot be.

On problem 3, break it deliberately, then fix it

1. Submit quicksort with a last-element pivot. Note the verdict on the sorted-input test case.
2. Add the two lines of random pivot selection. Submit again.
3. Now build a list of 50,000 identical values locally and run your version on it. If you used `≤` in the partition, paste the `RecursionError`.
4. Switch to the three-way partition and run the same input. Say the before-and-after operation counts.

On problem 4, use the partition without sorting

Wiggle Sort II needs the median, and the median comes from quickselect — tomorrow's topic, built on today's partition. Solve it with `sorted()` first to get the placement rule right, then say out loud what you would replace the sort with and what the complexity becomes.

The partition drill

Write `partition` from memory, then verify by hand:

1. `partition([7, 2, 9, 4, 1, 8, 3], 0, 6)` — what does it return, and what is the list afterwards?
2. `partition([1, 2, 3], 0, 2)` — where does the pivot land, and what does that mean for the recursion?
3. `partition([3, 2, 1], 0, 2)` — the same question in reverse.
4. `partition([5, 5, 5, 5], 0, 3)` — with `<` and then with `≤`. Compare the returned positions.
5. `partition([2], 0, 0)` — does it terminate?

For each, state the invariant that holds at the moment the loop ends.

The worst-case drill

Run the comparison counter from §5 of the lesson and record the numbers:

1. 500 sorted values, first-element pivot.
2. 500 sorted values, random pivot.
3. 500 reversed values, first-element pivot.
4. 500 reversed values, median-of-three pivot.
5. 500 identical values, two-way partition with `<`.
6. 500 identical values, three-way partition.

Then say, without looking: which input is the surprising one, and why is it surprising?

The break-it drill

Introduce each bug, run it, and read the failure:

1. `quicksort(nums, lo, p)` instead of `p - 1`. Paste the error and say which pivot placement causes it.
2. `≤` instead of `<` in the partition, on a list of 3,000 equal values. Paste the error.
3. Last-element pivot on `list(range(5000))`. Paste the error and say why an already sorted list is the worst case.
4. Use Hoare's partition but recurse with `p - 1` and `p + 1`. Find an input where the output is wrong.
5. In the three-way partition, advance `i` after swapping with `gt`. Find the input that breaks it.

The complexity drill

Answer out loud, in under fifteen seconds each:

1. What is quicksort's worst case, and what input causes it?
2. Why does randomising the pivot fix it? What exactly does it change?
3. What is the expected number of comparisons, and how does that compare with merge sort's?
4. Why is quicksort faster in practice despite doing *more* comparisons?
5. What is the extra space, and what is the recursion depth — with and without the smaller-side trick?
6. Name the three pivot strategies and one weakness of each.

The design drill

Set a timer for twenty minutes and run the five moves out loud on this prompt:

Design the classes for a cinema ticket booking system. A chain has several cinemas; each cinema has several screens; each screen shows several shows a day; each show has seats of two classes. Users search by film and city, pick seats, pay, and can cancel up to two hours before the show for a partial refund. Prices vary by day of week and seat class.

1. **Minutes 0-5** — four clarifying questions and the assumptions you will proceed on. Name what you are leaving out.
2. **Minutes 5-10** — the nouns, one sentence each. Then the noun that is *not* in the paragraph, and what has nowhere to live without it.
3. **Minutes 10-15** — the class diagram, with a multiplicity on every line.
4. **Minutes 15-20** — the one flow that matters, coded: seat selection and payment.

Then answer the three questions you know are coming:

- *Two users select the same seat at the same instant. What happens?*
- *Where did you put an interface, and where did you deliberately not?*
- *Now add a "book the whole row" feature.*

The missing-noun drill

For each prompt, name the class that is not in the sentence, and say what has nowhere to live without it:

1. "A guest books a room."
2. "A user picks seats for a show."
3. "A car parks in a spot and pays on exit."
4. "A member borrows a book."
5. "A rider requests a ride to a destination."

6. “A customer orders food from a restaurant.”

The race-condition drill

For each, name the check-then-write gap, say what two users see when it goes wrong, and give the atomic fix as a single operation:

1. The last hotel room.
2. The last cinema seat.
3. The last parking spot.
4. Two withdrawals from the same account balance.
5. Two users claiming the same username.
6. A stock counter reaching zero during a flash sale.

The interface-justification drill

For each, say whether you would create an interface, and name the second implementation — or say plainly that you cannot:

1. Pricing in a hotel booking system.
 2. `IHotel`.
 3. The cancellation policy.
 4. The payment gateway.
 5. `IRoom`.
 6. The booking repository.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Write quicksort. What is its worst case, and when does it happen?* The partition with its invariant, the excluded pivot in the recursion, $O(n^2)$ on **sorted** input with a fixed pivot, and the two lines that fix it — plus what randomising actually changes.
 2. *Design the classes for a hotel booking system.* The five moves on a clock, the missing noun, the two justified interfaces, and the check-then-write race named before you are asked.
 3. *Quicksort or merge sort?* In place and fast against guaranteed and stable, with the numbers — and the three situations where merge sort is the only answer.
-

Before you move on

- [] I wrote `partition` from memory and can state its invariant at the moment the loop ends.

- [] I crashed quicksort on an already sorted list and can explain why that input is the worst case.
- [] I crashed it again on 3,000 equal values with `≤` and can say what the one character changed.
- [] I can give the three pivot strategies and one weakness of each, unprompted.
- [] I ran the twenty-minute cinema design out loud, including the missing noun and the race.
- [] I can name the check-then-write gap in six different prompts and give the atomic fix for each.
- [] I can say which interfaces I would create and name the second implementation for each.
- [] I answered all three questions above out loud.

Quickselect: finding the Kth largest without sorting

DATA STRUCTURES AND ALGORITHMS

Quickselect: finding the Kth largest without sorting

You can find the Kth largest in expected $O(n)$ and say why the expectation matters.

SYSTEM DESIGN

Single responsibility

You can spot a class doing two jobs and split it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- Find the Kth largest element. Can you beat $O(n \log n)$?
- This class is 800 lines. What is wrong with it?

Quickselect: finding the Kth largest without sorting

1 What this is, and why they ask it

Quickselect finds the k -th smallest or k -th largest value in a list without sorting the list. It uses exactly the partition step from [day 054](#), and it makes one change that changes everything: after partitioning, you know which side of the pivot the answer is on, so you **throw the other side away** instead of recursing into it. That one difference takes the cost from $O(n \log n)$ down to an expected $O(n)$.

They ask it because “find the k -th largest” is one of the most common medium-difficulty interview questions there is — LeetCode 215, asked at almost every product company — and because it has three correct answers with different costs. Sorting is $O(n \log n)$ and everybody gets there. A heap of size k is $O(n \log k)$ and most good candidates get there. Quickselect is $O(n)$ expected, and getting there means you understood that you were being asked for *one* value rather than an ordering. The follow-ups are predictable and sharp: what is the worst case, why does randomising help, what does “expected” actually promise, and when would you choose the heap anyway. This is also the last new sorting-family algorithm in the phase, and it is the one that shows sorting is sometimes more than you needed.

2 The story

Zubeida runs the auditions for the annual day dance item at a school in Hyderabad, and this year a hundred and eighty girls signed up for eight places in the troupe.

She had two afternoons. In her first year she tried to do it properly — watch everybody, put them in order from best to worst, and take the top eight. She got about forty in and realised she was arguing with herself about whether number seventeen was better than number twenty-three, which was a completely useless thing to be deciding, because neither of them was getting in.

So the second year she did it differently, and now she does it this way every time.

She calls up one girl, more or less at random from the middle of the list, and watches her do the piece. That girl becomes the mark. This year it was a girl called Sania, and Sania was decent — neither obviously brilliant nor obviously struggling.

Then everybody else dances, one after another, and Zubeida makes exactly one decision about each of them. Better than Sania, or not as good as Sania. That is all. She does not compare anyone with anyone else, and she does not put the “better” group in any kind of order. Two groups, one look each.

At the end of the afternoon there were thirty-one girls in the better group and a hundred and forty-eight in the other one, and Sania in between.

And here is the thing she does that saves her the second afternoon. The eight places have to come out of those thirty-one. All of them. There is no argument to be had about the hundred and forty-eight — not one of them is going to be in the top eight, and it does not matter in the slightest which of them is better than which. She thanks them and they go home, and she never thinks about them again.

Then she does exactly the same thing inside the thirty-one. Picks one, makes it the mark, splits them into better and worse, and drops whichever half cannot contain the eighth place. Thirty-one becomes eleven. Eleven becomes eight.

Her sister asked her once whether it would go wrong if she picked badly, and it does. One year she happened to choose a girl who turned out to be the strongest dancer of the lot, so everybody went into the “not as good” group and nothing was eliminated at all. She had spent an hour and cut the field by one. She just started again with someone else. Now she deliberately picks from the middle of the list rather than from the top of it, where the keen ones sign up first.

3 The idea in plain English

Zubeida is running quickselect. Sania is the pivot. Splitting the girls into “better” and “not as good” is the partition. And thanking a hundred and forty-eight girls and never thinking about them again is the one step that separates today from yesterday.

The problem

Given a list of numbers and a number k , find the k -th largest value. For `nums = [3, 2, 1, 5, 6, 4]` and `k = 2`, the answer is `5` — the second largest.

Note what is *not* being asked. You are not asked for the whole order. You are not even asked for the top k . You are asked for **one value**. Any work that produces more than that is work you did not have to do, and noticing that is the entire insight.

The three answers, in the order candidates find them

One: sort and take it. `sorted(nums)[-k]`. Correct, $O(n \log n)$, one line. Always say this first — it is the baseline, it takes five seconds, and starting from a correct answer is better than starting from a clever one.

Two: a heap of size k . Keep the k largest values seen so far in a structure that can cheaply give up its smallest. That is $O(n \log k)$, and in Python it is `heapq.nlargest(k, nums)[-1]`. Heaps arrive properly on [day 113](#); today, know the cost and that the library call exists.

Three: quickselect. $O(n)$ expected. This is the answer they are fishing for.

The one idea

Partition the list around a pivot, exactly as yesterday. The pivot lands at some position `p`, and after partitioning you know three things:

- everything before `p` is smaller than the pivot,
- everything after `p` is larger,
- **the pivot is at its final sorted position.**

That last fact is the key. If you are looking for the value that belongs at position `target`, then:

```
p == target → the pivot IS the answer. Stop. Return it.
p > target → the answer is to the LEFT. Search lo..p-1. Discard everything right.
p < target → the answer is to the RIGHT. Search p+1..hi. Discard everything left.
```

Quicksort recurses into **both** sides. Quickselect recurses into **one**. That is the whole difference, and it is the sentence to say out loud.

Turning “k-th largest” into a position

This is where people make an off-by-one error, so do it deliberately and out loud.

In a list sorted ascending, of length `n`:

- the 1st **smallest** is at position `0`, so the k-th smallest is at position `k - 1`;
- the 1st **largest** is at position `n - 1`, so the k-th largest is at position `n - k`.

```
nums = [3, 2, 1, 5, 6, 4]      n = 6
sorted = [1, 2, 3, 4, 5, 6]
index   0  1  2  3  4  5

k = 2 (second largest) → target index = n - k = 6 - 2 = 4 → value 5. Correct.
k = 1 (largest)        → target index = 5           → value 6. Correct.
k = 6 (smallest)       → target index = 0           → value 1. Correct.
```

Write `target = len(nums) - k` on the board before writing the loop, and check it against `k = 1` and `k = n`. That takes ten seconds and prevents the commonest failure on this problem.

Why throwing half away gives you $O(n)$

The arithmetic is short and it is the answer to the main follow-up.

Partitioning `n` elements costs `n` comparisons. If the pivot lands near the middle, the next partition is on `n/2` elements, then `n/4`, and so on — because each time you discard one side:

$$n + n/2 + n/4 + n/8 + \dots + 1 = 2n$$

That sum is one of the few pieces of arithmetic worth memorising, because it appears everywhere. Halving repeatedly and adding it all up gives you **twice the first term**, not $n \log n$. So quickselect is $O(n)$.

Compare it with quicksort, which does not discard:

```
quicksort   : n + (n/2 + n/2) + (n/4 x 4) + ... = n per level x log n levels = n log n
quickselect : n + n/2 + n/4 + ... = 2n
```

The whole $\log n$ factor disappears because only one branch survives at each level.

The catch, said plainly

Quickselect is $O(n)$ **expected**, not guaranteed. If the pivot is the largest or smallest value every time, each partition removes exactly one element and you are back to $n + (n-1) + (n-2) + \dots = O(n^2)$. That is Zubeida's bad year, when she picked the best dancer as the mark and eliminated nobody.

The fix is the same as yesterday: **choose the pivot at random**. Then the running time depends on your random numbers rather than on the arrangement of the input, so there is no particular input that is bad.

There is also a deterministic worst-case $O(n)$ method — **median of medians**, which chooses a pivot guaranteed to be near the middle by splitting the list into groups of five, taking each group's median, and recursively taking the median of those. It genuinely gives $O(n)$ in the worst case. It is also so much slower in practice, because of the constant factor, that nobody uses it for real selection. Know the name, know that it exists, and say you would use the randomised version.

4 The picture

One run, looking for the 2nd largest of six values:

```
nums = [ 3, 2, 1, 5, 6, 4 ]   n = 6, k = 2 → target index = 6 - 2 = 4

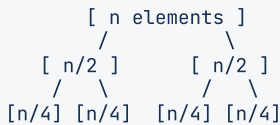
STEP 1  partition the whole range around pivot 4 (say the random choice landed there)
        [ 3, 2, 1, 4, 6, 5 ]
              ^
            p = 3          p (3) < target (4) → answer is to the RIGHT
                          DISCARD indices 0..3 -- four values, gone forever
        [ x, x, x, x, 6, 5 ]
              ^__^
            search here

STEP 2  partition indices 4..5 around pivot 5
        [ x, x, x, x, 5, 6 ]
              ^
            p = 4          p (4) == target (4) → STOP
                          the answer is nums[4] = 5
```

What to notice: step 1 threw away four of the six values after a single pass, and nothing in that discarded half was ever compared with anything again. Notice also that the list is left in a strange half-sorted state — quickselect mutates its input and does not sort it, which is a real property to mention.

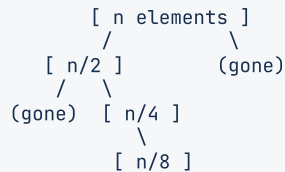
Quicksort against quickselect, drawn as the shape of the recursion:

QUICKSORT – both sides survive



work: n per level $\times$ $\log n$ levels
 $= n \log n$

QUICKSELECT – one side survives



work: $n + n/2 + n/4 + \dots = 2n$
 $= O(n)$

What to notice: the left tree is bushy and the right one is a single path. The $\log n$ factor in quicksort comes from the number of *levels* multiplied by full work at each level; quickselect has the same number of levels but the work halves down each one, and a halving sum converges.

The three answers, priced on the same input:

$n = 1,000,000$ values, $k = 10$

sort then index	$1,000,000 \times 20 = 20,000,000$ operations	$O(n \log n)$
heap of size k	$1,000,000 \times 3.3 = 3,300,000$	$O(n \log k)$
quickselect	$\sim 2 \times 1,000,000 = 2,000,000$	$O(n)$ expected

and if $k = 500,000$ (the median):

sort	$20,000,000$	$O(n \log n)$
heap of size k	$1,000,000 \times 19 = 19,000,000$	$O(n \log k)$ -- no better
quickselect	$\sim 2,000,000$	$O(n)$ -- unchanged

What to notice: the heap's advantage disappears as k grows, because $\log k$ approaches $\log n$. Quickselect does not care what k is. That is the case to raise when the interviewer asks "what if k is $n/2$ ".

5 The code, built step by step

The partition, unchanged from yesterday

```
import random

def partition(nums: list[int], lo: int, hi: int) → int:
    """Rearrange nums[lo..hi] around a random pivot. Return the pivot's final index."""
    r = random.randint(lo, hi)
    nums[r], nums[hi] = nums[hi], nums[r]      # random pivot, moved to the end
    pivot = nums[hi]
    i = lo
    for j in range(lo, hi):
        if nums[j] < pivot:
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]
    return i
```

This is exactly the code from [day 054](#), with the randomisation built in rather than bolted on. Nothing about it changes for selection.

The select, recursive

```
def select(nums: list[int], lo: int, hi: int, target: int) → int:
    """The value that would sit at index `target` if nums were sorted."""
    if lo == hi:
        return nums[lo] # one element left: it must be the answer
    p = partition(nums, lo, hi)
    if p == target:
        return nums[p] # the pivot IS the answer
    if target < p:
        return select(nums, lo, p - 1, target) # discard everything from p rightwards
    return select(nums, p + 1, hi, target) # discard everything from p leftwards
```

Read the three branches out loud as you write them. **Exactly one recursive call in each path** — that is the whole difference from quicksort, and it is worth pointing at.

The base case is `lo == hi`, a single element. It cannot be `lo ≥ hi` returning something else, because the range always contains the target by construction, so it is never empty.

The iterative version, which is better

Because there is only one recursive call and it is the last thing the function does, the recursion can be replaced by a loop mechanically. That removes the stack entirely:

```
def select_iterative(nums: list[int], target: int) → int:
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        p = partition(nums, lo, hi)
        if p == target:
            return nums[p]
        if target < p:
            hi = p - 1
        else:
            lo = p + 1
    return nums[lo]
```

Same algorithm, `O(1)` stack, no `RecursionError` on any input. Offer this version if the interviewer asks about deep recursion — a function whose only recursive call is the final statement can always be turned into a loop, and saying so is worth a mark.

The public function, with the index arithmetic done carefully

```
def kth_largest(nums: list[int], k: int) → int:
    """LeetCode 215. The k-th largest value. Mutates nums."""
    if not 1 ≤ k ≤ len(nums):
        raise ValueError(f"k must be between 1 and {len(nums)}, got {k}")
    return select_iterative(nums, len(nums) - k) # k-th largest = index n - k
```

`len(nums) - k`, checked against both ends: `k = 1` gives `n - 1`, the last index, the largest. `k = n` gives `0`, the smallest. Both correct.

Handling many duplicates

LeetCode 215 has test cases full of repeated values, specifically to break the two-way partition. Use the three-way version, which places all copies of the pivot at once:

```

def select_three_way(nums: list[int], target: int) → int:
    lo, hi = 0, len(nums) - 1
    while True:
        if lo == hi:
            return nums[lo]
        pivot = nums[random.randint(lo, hi)]
        lt, i, gt = lo, lo, hi
        while i ≤ gt:                                     # Dutch national flag
            if nums[i] < pivot:
                nums[lt], nums[i] = nums[i], nums[lt]
                lt, i = lt + 1, i + 1
            elif nums[i] > pivot:
                nums[i], nums[gt] = nums[gt], nums[i]
                gt -= 1
            else:
                i += 1
        if lt ≤ target ≤ gt:                               # target is inside the equal block
            return pivot                                   # ... so the pivot is the answer
        if target < lt:
            hi = lt - 1
        else:
            lo = gt + 1

```

The line that matters is `if lt ≤ target ≤ gt`. When the target lands anywhere inside the block of values equal to the pivot, the answer is the pivot — and on an input of a million identical values that happens on the first pass, so the whole thing is one $O(n)$ sweep.

The complete file

```
"""Quickselect: the k-th largest in expected  $O(n)$ , using quicksort's partition."""

import heapq
import random

def partition(nums: list[int], lo: int, hi: int) → int:
    """Lomuto partition with a random pivot. Returns the pivot's final index."""
    r = random.randint(lo, hi)
    nums[r], nums[hi] = nums[hi], nums[r]
    pivot = nums[hi]
    i = lo
    for j in range(lo, hi):
        if nums[j] < pivot:
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]
    return i

def select(nums: list[int], target: int) → int:
    """The value that would be at index `target` in the sorted list.

    Expected  $O(n)$ :  $n + n/2 + n/4 + \dots = 2n$ , because one side is discarded each time.
    Worst case  $O(n^2)$ , made vanishingly unlikely by the random pivot.
    Mutates nums, and leaves it partially ordered.
    """
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        p = partition(nums, lo, hi)
        if p == target:
            return nums[p]
        if target < p:
            hi = p - 1
        else:
            lo = p + 1
    return nums[lo]

def select_three_way(nums: list[int], target: int) → int:
    """Quickselect that stays  $O(n)$  when the input is mostly duplicates."""
    lo, hi = 0, len(nums) - 1
    while True:
        if lo == hi:
            return nums[lo]
        pivot = nums[random.randint(lo, hi)]
        lt, i, gt = lo, lo, hi
        while i ≤ gt:
            if nums[i] < pivot:
                nums[lt], nums[i] = nums[i], nums[lt]
                lt, i = lt + 1, i + 1
            elif nums[i] > pivot:
                nums[i], nums[gt] = nums[gt], nums[i]
                gt -= 1
            else:
                i += 1
        if lt ≤ target ≤ gt:
            return pivot
        if target < lt:
            hi = lt - 1
        else:
            lo = gt + 1

def kth_largest(nums: list[int], k: int) → int:
    """LeetCode 215.  $k = 1$  is the largest. Mutates nums."""
    if not 1 ≤ k ≤ len(nums):
        raise ValueError(f"k must be between 1 and {len(nums)}, got {k}")
```

```

return select_three_way(list(nums), len(nums) - k)

def kth_smallest(nums: list[int], k: int) → int:
    """k = 1 is the smallest."""
    if not 1 ≤ k ≤ len(nums):
        raise ValueError(f"k must be between 1 and {len(nums)}, got {k}")
    return select_three_way(list(nums), k - 1)

def top_k(nums: list[int], k: int) → list[int]:
    """The k largest values, unordered. O(n) expected -- the partition leaves them
    all in the last k positions, which is often all the caller needs."""
    if k ≤ 0:
        return []
    values = list(nums)
    select(values, len(values) - k)
    return values[len(values) - k:]

def kth_largest_heap(nums: list[int], k: int) → int:
    """The other correct answer: O(n log k) time, O(k) space.

    Better when the data is a stream, or when nums must not be mutated.
    """
    smallest_k_largest: list[int] = []
    for x in nums:
        heapq.heappush(smallest_k_largest, x)
        if len(smallest_k_largest) > k:
            heapq.heappop(smallest_k_largest)    # drop the smallest of the k+1
    return smallest_k_largest[0]

def median(nums: list[int]) → float:
    """Quickselect's natural application: the median in O(n) rather than O(n log n)."""
    values = list(nums)
    n = len(values)
    if n % 2 == 1:
        return float(select_three_way(values, n // 2))
    lower = select_three_way(values, n // 2 - 1)
    upper = select_three_way(list(nums), n // 2)
    return (lower + upper) / 2

if __name__ == "__main__":
    print(kth_largest([3, 2, 1, 5, 6, 4], 2))          # 5
    print(kth_largest([3, 2, 3, 1, 2, 4, 5, 5, 6], 4)) # 4
    print(kth_largest([1], 1))                        # 1
    print(kth_smallest([3, 2, 1, 5, 6, 4], 1))        # 1

    print(sorted(top_k([3, 2, 1, 5, 6, 4], 3)))        # [4, 5, 6]
    print(kth_largest_heap([3, 2, 1, 5, 6, 4], 2))    # 5
    print(median([7, 1, 3, 9, 5]))                    # 5.0
    print(median([4, 1, 3, 2]))                        # 2.5

    # the duplicate-heavy case that breaks the two-way version
    heavy = [7] * 200_000 + [9]
    print(kth_largest(heavy, 1))                      # 9    -- one O(n) pass

    # the input is mutated and left partly ordered -- say this out loud
    data = [3, 2, 1, 5, 6, 4]
    print(select(data, 4), data)                      # 5 [3, 2, 1, 4, 5, 6]

```


6 What it costs

The expected case, counted

Partitioning a range of m elements costs $m - 1$ comparisons. With a random pivot, the pivot lands near the middle on average, so the surviving side is about half the size.

```
pass 1: compare n      elements
pass 2: compare n/2
pass 3: compare n/4
pass 4: compare n/8
...
-----
total  =  n(1 + 1/2 + 1/4 + 1/8 + ...) = n x 2 = 2n
```

$O(n)$. The exact expected count with a uniformly random pivot is about $3.4n$ comparisons for the median and about $2n$ for a value near an end — the constant depends on k , but it is a constant.

```
n = 1,000,000

quickselect (median)  ~3,400,000 comparisons
quickselect (k = 10)  ~2,000,000
full sort             ~20,000,000
heap of size 10       ~3,300,000
heap of size 500,000 ~19,000,000
```

Why the sum stops at $2n$

This is the piece of arithmetic to have ready, because interviewers ask candidates to justify the $O(n)$ and most of them cannot.

```
S = 1 + 1/2 + 1/4 + 1/8 + 1/16 + ...

2S = 2 + 1 + 1/2 + 1/4 + 1/8 + ...
2S = 2 + S
S = 2

So n + n/2 + n/4 + ... = 2n. Halving and summing gives twice the first term.
```

Say it as: “the sum of a halving series is twice the first term, so the total work is $2n$, not $n \log n$. The log factor in quicksort comes from having to do full work at every level, and I only do work on one side.”

The worst case

```
pivot is always the largest (or smallest) value:

pass 1: n-1 comparisons, one element removed
pass 2: n-2
...
total = n(n-1)/2 = O(n^2)

n = 100,000 : 5,000,000,000 comparisons -- minutes, not milliseconds
```

With a random pivot the probability of this is on the order of $1/n!$, which for a hundred thousand elements is a number with hundreds of thousands of digits in the denominator. It does not happen. But it is not *impossible*, and the honest phrase is “**expected** $O(n)$ ”, not “ $O(n)$ ”.

Space

```
select_iterative : O(1) extra -- one loop, no stack, in place
select (recursive): O(log n) stack expected, O(n) worst case
heap of size k    : O(k)
sorted(nums)[-k]  : O(n) for the copy

quickselect MUTATES the input. If the caller needs the original order,
copy first -- and then it is O(n) space like everything else.
```

The $O(1)$ space of the iterative version is a genuine selling point over the heap, and worth saying.

Against the alternatives, decided by k

	time	space	mutates?	stream?
sorted()[-k]	$O(n \log n)$	$O(n)$	no	no
heapq.nlargest	$O(n \log k)$	$O(k)$	no	YES
quickselect	$O(n)$ expected	$O(1)$	YES	no
k = 10 of 10^6	heap 3.3M vs quickselect 2M			→ similar
k = 500,000	heap 19M vs quickselect 3.4M			→ quickselect wins 6x
data is a stream	heap works, quickselect cannot			→ heap, always

7 The traps

The real error: the wrong index for “k-th largest”

```
def kth_largest_broken(nums, k):
    return select(nums, k)          # ← k, not len(nums) - k

print(kth_largest_broken([3, 2, 1, 5, 6, 4], 2))
```


exists for, and LeetCode 215 has test cases designed to trigger it.

The trap: forgetting that it mutates

```
scores = [88, 95, 71, 64, 99]
print(kth_largest_inplace(scores, 2))
print(scores)
```

```
95
[88, 71, 64, 95, 99]
```

The caller's list has been rearranged. Sometimes that is fine and sometimes it is a bug three functions away. Either copy inside the function — `select(list(nums), target)` — or say clearly in the docstring that the argument is mutated. The same discipline as `.sort()` against `sorted()` from [day 051](#).

The trap: claiming a guaranteed $O(n)$

```
Candidate: "Quickselect is  $O(n)$ ."
Interviewer: "Always?"
```

No. It is $O(n)$ **expected**, $O(n^2)$ worst case. Median of medians gives a deterministic $O(n)$, but with a constant factor large enough that it is slower than the randomised version on real data, so it is a theoretical answer rather than an engineering one. Volunteer the distinction before you are pushed on it; a candidate who says “expected $O(n)$, worst case quadratic, and here is why I am comfortable with that” is ahead of one who has to be corrected.

The trap: using quickselect on a stream

Quickselect needs the whole list present and it needs random access, because it rearranges elements. If the data arrives one value at a time and will not fit in memory, quickselect is not available at all, and the answer is a heap of size k — $O(n \log k)$ time, $O(k)$ space, one pass, and it never holds more than $k + 1$ items. That is an important boundary to know, because “what if the data is a stream” is the standard third follow-up.

8 In the interview

How it gets asked

- “Find the k -th largest element in an array.” — LeetCode 215, asked constantly.
- “Can you do better than sorting?” — the prompt that means “I want quickselect or a heap”.
- “Find the top k elements.” — same machinery; quickselect leaves them in the last k slots.
- “Find the median of an unsorted array in $O(n)$.” — quickselect with `target = n // 2`.
- “What if the array is a stream of a billion numbers?” — the heap, and knowing why quickselect cannot be used.

- “What is the worst case? Can you make it $O(n)$ guaranteed?” — median of medians, named and then set aside.

What to say out loud, in the first ninety seconds

1. **Give the baseline first.** “The simple answer is `sorted(nums)[-k]`, which is $O(n \log n)$. I can do better, and I’ll say why sorting is more than I need: I’m being asked for one value, not an ordering.”
2. **Name the two better answers and their costs.** “A heap of size k is $O(n \log k)$. Quickselect is $O(n)$ expected. I’ll write quickselect and mention where the heap would win.”
3. **Do the index arithmetic out loud.** “The k -th largest sits at index $n - k$ in the sorted array. Check: $k = 1$ gives $n - 1$, the largest — correct. $k = n$ gives 0, the smallest — correct.”
4. **State the one difference from quicksort.** “Same partition, but after it I know which side the target is on, so I recurse into one side and discard the other. That’s what removes the log factor.”
5. **Give the cost with the arithmetic.** “ $n + n/2 + n/4 + \dots = 2n$, because a halving series sums to twice the first term. That’s $O(n)$ expected. Worst case is $O(n^2)$ if the pivot is always extreme, so I randomise the pivot.”

The follow-ups

“Why is it $O(n)$ and not $O(n \log n)$? Both halve.” Because of what happens to the work at each level, not to the size. In quicksort, after partitioning I recurse into *both* halves, so every level of the recursion still processes all n elements in total — n at the top, two lots of $n/2$ below it, four lots of $n/4$ below that — and with $\log n$ levels that multiplies out to $n \log n$. In quickselect I know which side the target is on, because the pivot lands at its final sorted position and I can compare that position with my target index. So I discard the other side entirely and never look at it again. The work per level is therefore n , then $n/2$, then $n/4$, and the total is the sum of that halving series, which is $2n$ — a halving series sums to twice its first term. So the log factor is not something I optimised away cleverly; it never appears, because there is only ever one live branch. That is also why the code has exactly one recursive call per path, and a good check on an implementation: if there are two recursive calls in it, it is quicksort wearing the wrong name.

“What’s the worst case, and can you make it $O(n)$ guaranteed?” The worst case is $O(n^2)$, and it happens when the pivot is the largest or smallest value in the range every time — each partition then removes exactly one element, so the sizes go n , $n-1$, $n-2$ and the sum is $n(n-1)/2$. With a fixed pivot rule such as “always the last element”, the input that triggers it is an already sorted array, which is a very likely input. I randomise the pivot position, which means the running time depends on my random numbers rather than on the arrangement of the input, so no particular input is bad and the probability of the quadratic case is on the order of $1/n!$. There is a separate bad case that randomisation does *not* fix: an array where all the values are equal, because randomising the position does not help when every value is the same. For that I use a three-way partition, and check whether the target index falls inside the block of values equal to the pivot — if it does, the pivot is the answer, so a million identical values is a single linear pass. As for a deterministic guarantee, yes, there is one: median of

medians, which splits the array into groups of five, takes each group's median, and recursively selects the median of those as the pivot. That pivot is provably near the middle, so the worst case is $O(n)$. In practice it is slower than the randomised version on real data because of the constant factor, so I would name it and then say I would still ship the randomised one.

“The data is a stream of a billion numbers that won't fit in memory. Now what?” Then quickselect is off the table, because it needs the entire array present and it works by rearranging elements in place — it needs random access, and a stream gives neither. The answer is a min-heap of size k . I read values one at a time, push each one, and pop whenever the heap exceeds k elements, which always removes the smallest of the $k+1$ — so what remains is always the k largest values seen so far, and at the end the root of the heap is the k -th largest. That is $O(n \log k)$ time, $O(k)$ space, one pass, and it never holds more than $k+1$ items regardless of how long the stream is. For $k = 10$ out of a billion that is trivial memory and about 3.3 billion comparisons' worth of work at $\log k = 3.3$. If k were also enormous — say I wanted the millionth largest of a billion — the heap stops being cheap and I would go to a different shape: either an approximate answer using a sketch like t-digest, if the requirement allows it, which gives quantiles in constant memory, or an external approach where I make one pass to build a histogram of value ranges, find which bucket the answer falls in, and then make a second pass keeping only that bucket, which fits in memory. And if the data is finite but merely bigger than RAM rather than a true stream, the honest answer is an external merge sort, which is exactly what a database does for a large ORDER BY.

A model answer

“The straightforward answer is `sorted(nums)[-k]`, which is $O(n \log n)$, and I’d say that first. But sorting produces a full ordering and I’ve only been asked for one value, so it’s more work than the question needs. There are two better answers: a heap of size k , which is $O(n \log k)$, and quickselect, which is $O(n)$ expected. I’ll write quickselect.

It uses quicksort’s partition. I pick a random pivot and rearrange so smaller values are left of it and larger ones right of it, which leaves the pivot at its final sorted position, at some index p . The k -th largest lives at index $n - k$ in the sorted array — checking that: $k = 1$ gives $n - 1$, the largest, and $k = n$ gives 0 , the smallest, so it’s right. Now if p equals my target index, the pivot is the answer. If the target is smaller, the answer is entirely to the left, so I discard everything from p rightwards. If it’s larger, I discard everything left.

```
def select(nums: list[int], target: int) → int:
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        p = partition(nums, lo, hi)
        if p == target:
            return nums[p]
        if target < p:
            hi = p - 1
        else:
            lo = p + 1
    return nums[lo]
```

I’ve written it iteratively because there’s only one recursive call and it’s the last statement, so it converts to a loop mechanically — that gives $O(1)$ extra space and no recursion depth to worry about.

The complexity is the interesting part. Quicksort recurses into both sides, so it does n work at every level for $\log n$ levels. Here I discard one side, so the work goes n , then $n/2$, then $n/4$, and that halving series sums to $2n$ — a halving series is twice its first term. So it’s $O(n)$, and the log factor never appears rather than being optimised away.

Two honest caveats. It’s $O(n)$ *expected*, not guaranteed — if the pivot were extreme every time it would be $O(n^2)$, which is why the pivot is random. And on an array where every value is equal, randomising the position doesn’t help, so I’d use a three-way partition and check whether the target falls inside the equal block, which makes that case a single linear pass. Also, this mutates the input and leaves it half-ordered, so I’d copy first if the caller needs the original. If the data were a stream I couldn’t use this at all, and the answer would be the heap.”

9 Recall card

- **Same partition as quicksort, one change: recurse into ONE side.** The pivot lands at its final index `p`; compare `p` with the target index. `p == target` → answer · `target < p` → search left, **discard the right** · else search right. Two recursive calls means you wrote quicksort.
- **The k-th largest is at index `n - k`.** Check both ends out loud: `k=1` → `n-1` (largest), `k=n` → `0` (smallest). Getting this wrong returns a plausible number with no error — the commonest bug on LeetCode 215.
- **Expected $O(n)$, and the arithmetic is the answer:** `$n + n/2 + n/4 + \dots = 2n$` , because a halving series sums to **twice the first term**. Quicksort is $n \log n$ because it does full work at every level; here only one branch survives.
- **Worst case $O(n^2)$** if the pivot is always extreme — fix with a **random pivot**. Randomising the *position* does not help when every *value* is equal, so use a **three-way partition** and return the pivot when `lt ≤ target ≤ gt`: a million identical values in one pass.
- **Three answers, chosen by k and by shape:** `sorted()[-k]` $O(n \log n)$ · heap $O(n \log k)$, $O(k)$ space, **the only one that works on a stream** · quickselect $O(n)$ expected, **$O(1)$ space but it mutates**. At $k = 10$ of 10^6 heap and quickselect are level; at $k = n/2$ quickselect wins six times over.

1 What this is, and why they ask it

The **single responsibility principle** is the S in SOLID, and it says: a class should have one reason to change. Not “a class should do one thing”, which is the version everybody quotes and which is too vague to act on. **One reason to change** — meaning one group of people, with one job, who would ever come and ask for this class to be different.

They ask it because it is the first principle of the five and the one every other principle depends on, and because it is the fastest way to tell whether a candidate has maintained code or only written it. Somebody who has only written code sees an 800-line class and says “it is long”. Somebody who has maintained one says “the pay calculation and the CSV export are in here together, so the finance team and the reporting team both edit this file, and their changes collide”. That second sentence is what is being scored. This is also where the OOD phase you just finished turns into a set of named rules — [day 044](#) already told you a responsibility is something you can say in one sentence with no “and”, and today gives you the reason that test works.

2 The story

There is a small eating house near the bus stand in Kottayam, eleven tables, run by a man called Thomas, and for about four years everything in it went through one boy called Munna.

Munna took the orders at the tables. Munna sat at the counter and took the money when people left. And at night, after the shutters came down, Munna sat with Thomas and worked out what had come in and what had gone out and what was owed to the vegetable supplier.

He was good at all three. That was never the problem.

The problem was that three different people kept asking him to change how he worked, and none of them knew about the other two.

Thomas wanted the counter faster at lunch, because the queue was reaching the door and people were walking off. So Munna started keeping the money loose in a tray instead of counting it properly into the box, and that made the counter quicker and made the night accounts take an extra forty minutes.

The customers wanted somebody at the tables. When Munna was at the counter nobody took orders, so food went out late, so people at the counter waited longer to pay, so the queue got worse. Fixing the tables made the counter slower.

And Thomas's brother-in-law, who did the tax paperwork twice a year, wanted the day's takings split between food and drinks. That is one extra thing to note down — except the only moment Munna could note it was at the counter, in the middle of the lunch rush, which put the queue back to the door again.

Three people, three completely different concerns, one boy. Every improvement for one of them was a problem for the other two, and none of them could see why.

The day it actually broke was a Thursday in June when Munna had fever and did not come in. Nobody could take an order, nobody could take money, and nothing was written down that night at all. Thomas shut at four.

What he did after that took a week and cost him one extra salary. His nephew took the tables. A woman called Sosamma, who had done it before somewhere else, took the counter. And the night accounts went to a man who came for two hours after closing and did nothing else.

Thomas told me the part that surprised him. It was not that the work got faster, although it did a bit. It was that a change stopped being frightening. When his brother-in-law asked for the food and drink split, that was now a conversation with one man, for two hours, after closing, and the lunch queue never heard about it.

3 The idea in plain English

Munna is a class doing three jobs. Thomas, the customers and the brother-in-law are three **reasons to change**, and the pain was never the amount of work — it was that a change asked for by one of them landed on top of the other two.

The principle, stated properly

A class should have one reason to change.

The sharper version, which is the one to use in an interview: **gather together the things that change for the same reason, and separate the things that change for different reasons.**

And the sharpest version, which explains why the other two work: a “reason to change” is a **person or a role** who asks for the change. Not a technical topic — a stakeholder. Munna's three reasons had names: Thomas, the customers, the brother-in-law.

Why “does one thing” is the wrong version

Everybody quotes “a class should do one thing”, and it falls apart the moment you use it, because “one thing” has no fixed size. Is “handle an order” one thing? Is “validate an order” one thing? Is “check that the postcode is six digits” one thing? Every answer is defensible, so the rule decides nothing, and people end up either leaving 800-line classes alone or splitting until every class has one method.

“One reason to change” decides it, because you can actually go and look. Ask: **who would come and ask for this to be different?** If the answer is one team, leave it together. If it is two teams, it is two classes.

The canonical example

```
class Employee:
    def calculate_pay(self) → Money: ...      # the finance team owns this rule
    def report_hours(self) → str: ...         # the HR team owns this rule
    def save(self, db) → None: ...           # the platform team owns this
```

Three methods, three teams, one file. Now the failure, concretely: finance and HR both want overtime counted, and the two definitions are *almost* the same, so someone writes one shared helper. Six months later finance changes how overtime is calculated for a tax reason. HR’s hours report silently changes too, and nobody notices until an employee queries their timesheet.

That is the real cost, and it is not “the file is long”. It is that **two independent rules were coupled by accident because they lived in the same place.**

The fix names the three responsibilities:

```
class Employee:                                # the data and the rules about being an employee
    ...

class PayCalculator:                            # owned by finance
    def calculate(self, employee: Employee) → Money: ...

class HoursReporter:                           # owned by HR
    def report(self, employee: Employee) → str: ...

class EmployeeRepository:                      # owned by the platform team
    def save(self, employee: Employee) → None: ...
```

Now finance’s change touches one file, and the HR report cannot be affected by it.

The second canonical example, and the trap in it

```
class Invoice:
    def total(self) → Money: ...                # the business rule
    def to_pdf(self) → bytes: ...               # presentation
    def save(self) → None: ...                  # persistence
    def email_to_customer(self) → None: ...# notification
```

Four responsibilities, and the tell is the imports: this file imports a PDF library, a database driver and an SMTP client to do arithmetic on money. The trap is that `total()` genuinely belongs on `Invoice` — it is behaviour next to the data it needs, which is exactly what [day 044](#) told you to do. **Splitting is not “move everything out into services.”** Keep the domain rule; move the format, the storage and the sending.

How to spot a violation, in order of usefulness

1. **You cannot describe the class in one sentence without “and”.** *“It calculates the invoice total and renders it as a PDF.”* Two responsibilities, and the word `and` told you.
2. **The imports disagree with each other.** A file that imports both `decimal` and `smtplib` is doing arithmetic and sending mail.
3. **The class name is a noise word.** `OrderManager`, `DataProcessor`, `Utils`, `Helper`, `Service`. These names are vague precisely because the class has no single responsibility to be named after. A class you cannot name specifically is a class you have not decided about.
4. **Two teams keep editing the same file.** This one is measurable — the version history tells you, and it is the most direct evidence the principle exists at all.
5. **Test setup is long and mostly irrelevant.** If testing the discount calculation requires a database connection and an SMTP host, the discount calculation is living with things it does not need.
6. **The class has fields that most of its methods never touch.** Two clusters of fields, each used by its own cluster of methods, is two classes that have been stapled together.

The other direction, which matters just as much

You can violate this principle by splitting too much. Fifteen classes with one method each, that must all be constructed in the right order to do anything, is not a better design — the responsibility has simply been smeared across fifteen files, and now a reader has to visit all of them to understand one flow. **Cohesion is the other half of the rule:** things that change together should live together. The full vocabulary for that arrives on [day 061](#).

4 The picture

Munna, before and after:

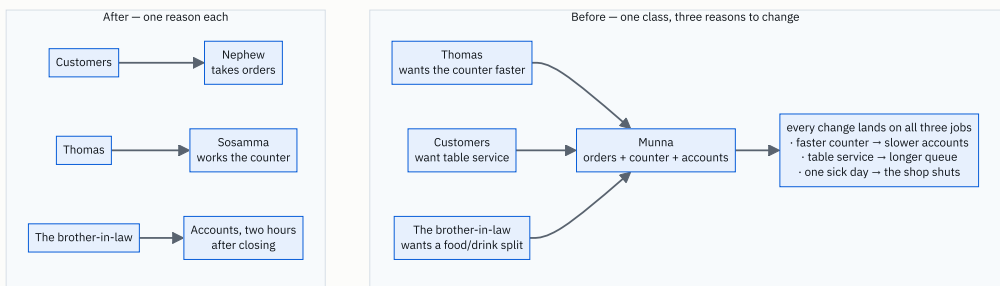


FIGURE 55.1

What to notice: the arrows. In the top half three different stakeholders point at one box, and that is the definition of the violation — not the box’s size. In the bottom half each stakeholder points at exactly one box, so a change from one of them cannot disturb the others.

The `Invoice` split, as classes:

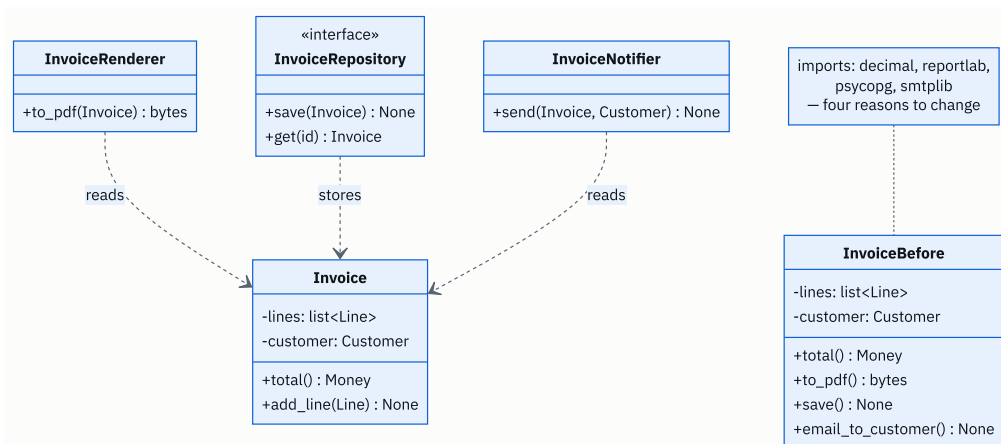


FIGURE 55.2

What to notice: `total()` stayed on `Invoice`. It is the business rule and it belongs next to the data it needs. Only the format, the storage and the sending moved out — and notice the arrows all point at `Invoice`, which now imports nothing but `decimal`. A class that everything depends on and that depends on nothing is exactly what you want at the centre of a model.

Where the change lands, which is the whole argument:

BEFORE — one class, four concerns

finance changes the tax rule	→ edit Invoice.py	(risk: PDF, DB, email)
design changes the PDF layout	→ edit Invoice.py	(risk: tax, DB, email)
platform migrates to a new DB	→ edit Invoice.py	(risk: tax, PDF, email)
marketing changes the email copy	→ edit Invoice.py	(risk: tax, PDF, DB)

4 teams, 1 file, every change reviewed by people who understand one quarter of it

AFTER — one class per concern

finance	→ Invoice.py	(imports: decimal)
design	→ InvoiceRenderer.py	(imports: reportlab)
platform	→ InvoiceRepository.py	(imports: psycopg)
marketing	→ InvoiceNotifier.py	(imports: smtplib)

4 teams, 4 files, zero overlap

What to notice: the risk column in the top half. Nothing about the *code* got worse when four concerns shared a file — what got worse is that every change now carries three risks it has no business carrying.

5 How it actually works

The refactor, in six mechanical steps

This is the sequence to run out loud when an interviewer hands you a fat class.

Step 1 — list the methods and name the stakeholder for each. Not the topic; the *person*.

```
class OrderProcessor (642 lines)

    validate_order()           → the business (what makes an order valid)
    calculate_total()          → finance (pricing, tax, discounts)
    apply_discount_code()      → marketing (campaign rules)
    reserve_inventory()        → operations (stock rules)
    charge_card()              → finance/payments
    save_to_db()               → platform
    send_confirmation_email()   → marketing (copy, templates)
    generate_invoice_pdf()      → finance/design
    log_analytics_event()      → data team
```

Nine methods, six stakeholders. Say that number out loud — it is the evidence.

Step 2 — group the methods whose stakeholder is the same. Groups become classes.

Step 3 — find the fields each group actually touches. A group that uses three of the class's fourteen fields is a class trying to leave. A group that uses all fourteen probably belongs where it is.

Step 4 — extract the group with the *most* independent stakeholder first. Usually persistence or notification, because they touch the fewest fields and nothing depends on them.

Step 5 — inject the new collaborator rather than constructing it ([day 053](#)), so the original class stays testable.

Step 6 — check the imports. After the split, each file's import list should be readable as a sentence about what that file does. If `Order.py` still imports `smtpplib`, you missed one.

Before

```
class OrderProcessor:
    def __init__(self, order_id: str) → None:
        self.order = fetch_order(order_id)           # hidden dependency, untestable
        self.conn = psycopg2.connect(os.environ["DB"]) # ditto
        self.smtp = smtplib.SMTP(os.environ["SMTP"])  # ditto

    def process(self) → None:
        if not self.order.items:
            raise ValueError("empty order")
        total = sum(i.price * i.quantity for i in self.order.items)
        if self.order.code:
            total *= (1 - DISCOUNTS[self.order.code]) # marketing's rule
            total *= 1.18                               # finance's rule (GST)
        for item in self.order.items:                   # operations' rule
            self.conn.execute(
                "UPDATE stock SET qty = qty - %s WHERE sku = %s",
                (item.quantity, item.sku),
            )
        charge_id = razorpay.charge(total, self.order.token)
        self.conn.execute(
            "INSERT INTO orders (id, total, charge) VALUES (%s, %s, %s)",
            (self.order.id, total, charge_id),
        )
        self.smtp.sendmail(                             # marketing's copy
            "orders@shop.in", self.order.email,
            f"Subject: Order confirmed\n\nYour total is {total}",
        )
        analytics.track("order_placed", {"total": total})
```

One method, six stakeholders, four external systems, and no way to test any single rule.

After

```
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class Money:
    paise: int

    def __mul__(self, factor: float) → "Money":
        return Money(round(self.paise * factor))

    def __add__(self, other: "Money") → "Money":
        return Money(self.paise + other.paise)

@dataclass
class Order:
    """The business's rules about what an order IS. Imports nothing external."""
    order_id: str
    email: str
    items: list["Line"]
    discount_code: str | None = None

    def subtotal(self) → Money:
        total = Money(0)
        for line in self.items:
            total = total + line.amount()
        return total

    def is_valid(self) → bool:
        return bool(self.items) and all(line.quantity > 0 for line in self.items)
```

```
class Pricing:
    """Owned by finance. One reason to change: the tax or discount rules."""

    def __init__(self, gst_rate: float, discounts: dict[str, float]) → None:
        self._gst_rate = gst_rate
        self._discounts = discounts

    def total_for(self, order: Order) → Money:
        total = order.subtotal()
        if order.discount_code:
            total = total * (1 - self._discounts.get(order.discount_code, 0.0))
        return total * (1 + self._gst_rate)
```

```
class InventoryReservation(Protocol):
    def reserve(self, order: Order) → None: ...

class OrderRepository(Protocol):
    def save(self, order: Order, total: Money, charge_id: str) → None: ...

class Notifier(Protocol):
    def order_confirmed(self, order: Order, total: Money) → None: ...

class PaymentGateway(Protocol):
    def charge(self, amount: Money, token: str) → str: ...
```

And the coordinator, which now does exactly one thing — decide the *order* of the steps:


```

class PlaceOrder:
    """One responsibility: the sequence. No rule about money, stock, or wording."""

    def __init__(
        self,
        pricing: Pricing,
        inventory: InventoryReservation,
        payments: PaymentGateway,
        orders: OrderRepository,
        notifier: Notifier,
    ) → None:
        self._pricing = pricing
        self._inventory = inventory
        self._payments = payments
        self._orders = orders
        self._notifier = notifier

    def execute(self, order: Order, token: str) → Money:
        if not order.is_valid():
            raise ValueError("an order needs at least one item with a positive quantity")
        total = self._pricing.total_for(order)
        self._inventory.reserve(order)
        charge_id = self._payments.charge(total, token)
        self._orders.save(order, total, charge_id)
        self._notifier.order_confirmed(order, total)
        return total

```

Five constructor arguments is a lot, and it is *deliberately* visible: the class needs five collaborators because placing an order genuinely involves five concerns. That is honest, and it is the point — a fat constructor is a signal you can read, whereas six hidden `import`s inside a method are not.

Is the coordinator itself a violation?

No, and this is the question interviewers use to separate people who have read about SRP from people who have applied it. `PlaceOrder` has exactly one reason to change: **the sequence of steps changes**. If the business decides to charge before reserving stock, this class changes and nothing else does. If the tax rate changes, `Pricing` changes and this class does not. That is one reason to change, and it is a legitimate responsibility with a name: orchestration.

What real systems look like

- **Unix.** `ls` lists, `sort` sorts, `wc` counts. Each program has one reason to change, and they compose through pipes. This is the principle at the process level, from 1973, and it is why the tools have outlived every framework built since.
- **The layered web application.** Model, repository, service, view, serialiser. Django's fat-model debate is entirely an SRP argument: does business logic belong on the model, or in a service? The answer people converge on is the one above — domain rules on the model, persistence and presentation outside it.
- **React and its relatives.** The container/presentational split is SRP: one component decides what data to fetch, another decides how it looks. Designers change the second and never the first.
- **Microservices.** SRP at the deployment level, with the same failure modes in both directions — split by stakeholder and it works, split by noun and you get fifteen services that must all be deployed together.

- **Where you can measure it.** Version-control history is the only direct evidence:

```
git log --format='%an' --since=1.year -- src/OrderProcessor.py | sort -u | wc -l
```

If that returns 1 or 2, the class is probably fine however long it is. If it returns 9, from four different teams, that is the violation, in data.

6 The numbers

What a shared file costs, measured

```
OrderProcessor.py -- 642 lines, 6 stakeholders

commits in a year           : 84
distinct authors           : 11
merge conflicts on this file : 19  (23% of commits)
average review turnaround   : 2.4 days
    -- because a reviewer who understands pricing
       must also sign off on SQL and email templates

after the split, per file
Pricing.py                 : 14 commits, 3 authors, 0 conflicts, 4-hour reviews
OrderRepository.py         : 6 commits, 2 authors, 0 conflicts
Notifier.py                : 21 commits, 4 authors, 1 conflict
```

The conflict count is the number to quote. Nineteen merge conflicts in a year is roughly one every two and a half weeks where two people's work has to be manually reconciled by someone who did not write either half.

Blast radius

```
"change the GST rate from 18% to 12%"

before : 1 file touched, but it contains 5 other concerns
        tests that must be re-run : all 63 tests of OrderProcessor
        things that could break   : stock, payment, persistence, email, analytics
        reviewers needed          : finance + platform + marketing

after  : 1 file touched (Pricing.py, 22 lines)
        tests that must be re-run : 9 pricing tests
        things that could break   : pricing
        reviewers needed          : finance
```

Test cost

```
Testing "a 10% discount code reduces the total correctly"

before : construct OrderProcessor → needs a DB connection
                                           → needs an SMTP host
                                           → needs a Razorpay key
      test runtime ~180 ms, and it fails when the network is slow
      lines of setup: 24

after  : Pricing(gst_rate=0.18, discounts={"DIWALI": 0.10}).total_for(order)
      test runtime ~0.1 ms
      lines of setup: 3

1,800x faster, and it cannot flake.
```

The reading cost, which is why long classes are actually bad

```
Understanding "how is the total calculated?"

642-line class : read ~200 lines to be sure nothing else modifies `total`
                  (it is assigned in 4 places)
22-line class  : read 22 lines, and the class imports only `decimal`,
                  so nothing external can be involved
```

The real cost of a long class is not that it is long. It is that you cannot be **sure** about anything in it without reading all of it, because any line might touch any field.

And the cost of over-splitting

```
The same flow, split into 15 one-method classes:

files to open to follow one order being placed : 15
constructor arguments in the top-level object  : 9
lines of wiring in main()                      : 38
time for a new joiner to trace the flow        : ~40 minutes

Against 5 classes with real responsibilities   : 5 files, 5 arguments, ~8 minutes
```

Both failures are real. Six hundred lines in one class and one method in each of fifteen classes are the same mistake — the responsibility boundary was drawn somewhere other than where the reasons to change are.

7 The trade-offs

What splitting costs you

More files, and more names to invent. Every extracted class needs a name, and a bad name (`OrderHelper`) is worse than no split at all, because it hides the responsibility instead of declaring it.

Indirection. Following one order through five classes means five files instead of one scroll. For a small program that is a net loss, and pretending otherwise is dishonest.

Wiring. Somebody has to construct five objects and pass them in. That is the composition root from [day 053](#), and it grows.

A risk of anaemia. The commonest bad split takes *all* the behaviour out of the entity and leaves a bag of fields plus a `Service` class — which is precisely the anaemic model [day 044](#) warned about. Splitting by *technical layer* rather than by *reason to change* is how that happens.

```
Invoice.total() stays on Invoice.
```

When a big class is the right answer

I would not split a class if all its methods change for the same reason, however many there are. A `Matrix` class with forty methods — multiply, transpose, invert, determinant — has one reason to change: the mathematics of matrices. Splitting it into `MatrixMultiplier` and `MatrixTransposer` is worse in every way, because none of those pieces is independently useful and the fields are shared by all of them.

I would not split if the pieces cannot exist independently. If class B can only ever be constructed by class A, used by class A, and tested through class A, then B is not a separate responsibility; it is A's implementation with a file of its own.

I would not split at the start of a project. You cannot see the reasons to change until somebody has asked for a change. Two or three concerns in one class in week one is fine; the tell that it is time is the *second* time two different people edit it for unrelated reasons. Splitting a responsibility you guessed at is how you end up with fifteen wrong boundaries.

The genuinely hard part

Deciding what counts as a reason. Is “the tax rate changed” the same reason as “we now support a second country”? Reasonable engineers disagree, and the honest answer is that you cannot always tell in advance — which is why the version history matters more than the principle. **Let the second unrelated change tell you where the seam is.** Guessing produces boundaries that have to be undone, and undoing a boundary is more expensive than never drawing it.

Where it interacts with the rest of SOLID

SRP decides *what* the classes are. The other four decide how they relate: open/closed ([day 056](#)) says extend rather than edit, Liskov ([day 057](#)) says a subtype must be usable in the parent's place, interface segregation ([day 058](#)) is SRP applied to interfaces rather than classes, and dependency inversion ([day 059](#)) says point the dependency at the abstraction. **Get SRP wrong and none of the other four can help you**, because they all assume the boundaries are in sensible places.

8 In the interview

How it gets asked

- “*This class is 800 lines. What's wrong with it?*” — a code sample, and the expected answer names stakeholders, not line counts.

- “What is the single responsibility principle?” — and the follow-up is always “give me an example”, so have `Employee` or `Invoice` ready.
- “How do you decide where to split a class?” — the reason-to-change test, and the honest admission that the version history is better evidence than intuition.
- “Isn’t this just adding more files?” — the pushback question. Answer with the blast-radius and test-cost numbers.
- “Can you over-apply it?” — yes, and being able to describe the fifteen-one-method-classes failure is what shows judgement.

What to say out loud, in the first ninety seconds

1. **Reject the line count as the problem.** *“The length is a symptom. The actual problem is that this class has more than one reason to change.”*
2. **Name the stakeholders, out loud, method by method.** *“`calculate_total` changes when finance changes the tax rule. `send_confirmation_email` changes when marketing rewrites the copy. `save_to_db` changes when platform migrates the database. That’s three different people asking for changes to one file.”*
3. **Give the concrete failure, not the principle.** *“So a finance change gets reviewed by someone who has to also understand the SQL and the email templates, and two teams’ commits collide in the same file — nineteen merge conflicts in a year on the one I’m thinking of.”*
4. **Say what stays.** *“I’d keep `total()` on `Invoice` — that’s a business rule and it belongs next to the data. I’d move the PDF rendering, the persistence and the email out. This isn’t ‘take all the behaviour out and put it in services’ — that’s the anaemic model.”*
5. **Name the resulting classes and the one-sentence responsibility of each,** with no “and” in any sentence.

The follow-ups

“Isn’t this just more files for the same code?” It is more files, and I would not pretend that is free — more names to invent, more indirection, and a wiring step that did not exist before. What it buys is that a change stops carrying risks it has no business carrying. Concretely: changing the GST rate from eighteen percent to twelve. In the fat class that is one file touched, but the file also contains stock reservation, payment, persistence, email and analytics, so the full test suite for the class has to run, the reviewer needs to understand five things they did not write, and there is a real chance of touching something unrelated. After the split it is twenty-two lines in `Pricing.py`, nine tests, and one reviewer from finance. The other measurable difference is test cost: testing “a ten percent discount code reduces the total correctly” against the fat class needs a database connection, an SMTP host and a payment key, so it is about a hundred and eighty milliseconds and it flakes when the network is slow; against `Pricing` it is three lines and a tenth of a millisecond and it cannot flake. And the version history is the honest evidence — the class I have in mind had eleven distinct authors and nineteen merge conflicts in a year, which is one every two and a half weeks where two people’s unrelated work had to be reconciled by hand.

“How do you decide where the boundary goes?” By asking who would come and ask for this to be different, and grouping by the answer. Not by technical topic — by person or role. So I go through the methods and write a stakeholder next to each: pricing is finance, the discount campaign rules are marketing, stock is operations, the email copy is marketing again, persistence is platform. Methods with the same stakeholder become one class. Then I check it two ways. First the sentence test: I try to describe each new class in one sentence with no “and” in it, and if I need an “and” the split is not finished. Second, the fields: a group of methods that touches three of the fourteen fields is a class trying to leave, whereas a group that touches all fourteen probably belongs where it is. And I would be honest that this is the part where judgement is needed and intuition is often wrong — you cannot see the reasons to change until somebody has asked for a change. So in a new codebase I would leave two concerns together rather than guess, and let the *second* unrelated change tell me where the seam is. Guessing produces boundaries that later have to be undone, and undoing one is more expensive than never drawing it.

“Can you over-apply it? What does that look like?” Yes, and it fails in a way that is harder to see than the fat class. It looks like fifteen classes with one method each — `OrderValidator`, `OrderTotalCalculator`, `OrderDiscountApplier`, `OrderStockReserver` — none of which can be understood or used on its own, all of which must be constructed in the right order, and following a single order through the system means opening fifteen files instead of scrolling one. That is not one responsibility per class; it is one responsibility smeared across fifteen files, and the reader pays for it every time. The rule I use is the other half of the principle, which people quote less: gather together the things that change for the same reason. A `Matrix` class with forty methods has one reason to change — the mathematics of matrices — and splitting it into `MatrixMultiplier` and `MatrixTransposer` is worse in every respect, because none of the pieces is independently useful and they all share the same data. The related failure is splitting by technical layer rather than by reason: if I pull every method off `Invoice` and put them in an `InvoiceService`, I have made an anaemic model, which is fields with no behaviour, and I have lost the main thing objects are for. `Invoice.total()` stays on `Invoice`; only the PDF, the persistence and the email move out.

A model answer

“The eight hundred lines are a symptom rather than the problem. The problem is that this class has more than one reason to change, and I’d show that by naming who would ask for each change. `calculate_total` changes when finance changes the tax or discount rules. `send_confirmation_email` changes when marketing rewrites the copy. `save_to_db` changes when the platform team migrates the database. `reserve_inventory` changes when operations changes the stock policy. That’s four different groups of people editing one file, and the single responsibility principle is really the statement that a class should have one reason to change — one group of people who would ever come and ask.

The concrete harm is not that the file is long. It’s that unrelated things become coupled by accident. If finance and HR both need overtime, someone writes one helper because the definitions look the same, and then a tax change to one silently changes the other. And practically: eleven distinct authors on the file in a year, nineteen merge conflicts, and every review needs someone who understands pricing, SQL and email templates.

So I’d split by stakeholder. `Pricing` for the finance rules. `InventoryReservation`, `OrderRepository` and `Notifier` behind interfaces. And a `PlaceOrder` class whose only responsibility is the sequence — it decides that pricing happens before reservation and reservation before payment, and nothing else. That’s a legitimate single responsibility with a name: orchestration. If the business decides to charge before reserving stock, that class changes and nothing else does.

One thing I’d be careful about. `Invoice.total()` stays on `Invoice`. Splitting doesn’t mean moving all the behaviour into service classes — that gives you a bag of fields and a procedure, which is the anaemic model. The business rule belongs next to the data it needs; it’s the format, the storage and the sending that leave.

And it can be over-applied. Fifteen classes with one method each is the same mistake in the other direction — the responsibility is smeared instead of concentrated, and a new joiner opens fifteen files to follow one order. I’d rather leave two concerns together in a new codebase and let the second unrelated change show me where the seam actually is, because a boundary I guessed at is expensive to undo.”

9 Recall card

- **“One reason to change”, not “does one thing”** — and a *reason* is a **person or a role**, not a topic. `Employee` with `calculate_pay` (finance) + `report_hours` (HR) + `save` (platform) is three stakeholders in one file. The harm is not length; it is that two unrelated rules get coupled by accident and one team’s change silently breaks another’s.
- **Six ways to spot it:** you need an “and” to describe the class · the imports disagree (`decimal` and `smtplib`) · the name is a noise word (`Manager`, `Processor`, `Utils`) · **two teams keep editing the same file** (the only measurable one — check the version history) ·

long, irrelevant test setup · two clusters of fields used by two clusters of methods.

- **The refactor:** list the methods and name the *stakeholder* for each → group by stakeholder → check which fields each group touches → extract the most independent group first → inject rather than construct → re-read the imports.
- **Keep the business rule where it is.** `Invoice.total()` stays on `Invoice`; only the PDF, the persistence and the email move out. Pulling *everything* into a `Service` is the **anaemic model**, which is the commonest bad split. A coordinator whose one responsibility is the *sequence* is legitimate.
- **It is over-appliable.** Fifteen one-method classes is the same mistake inverted — 15 files and 38 lines of wiring to follow one flow. The other half of the rule is *gather what changes together*. In a new codebase, leave two concerns together and let the **second unrelated change** show you the seam. Numbers to quote: GST change = 22 lines and 9 tests instead of a 642-line file and 63 tests; the discount test goes from 180 ms with a database to 0.1 ms without one.

PRACTICE

Practice — Quickselect: finding the Kth largest without sorting

DSA topic: Quickselect: finding the Kth largest without sorting **System design topic:** Single responsibility

Code these, in this order

One rule for the whole set: **say the target index out loud before you write anything**, and check it against `k = 1` and `k = n`. The off-by-one is the bug that actually costs people this question.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Kth Largest Element in an Array	LeetCode 215 (Medium)	The whole lesson. And whether your version survives an input of identical values.
2	Top K Frequent Elements	LeetCode 347 (Medium)	Quickselect on a derived key — you select on counts, not on the values themselves.
3	K Closest Points to Origin	LeetCode 973 (Medium)	That you never need the actual distance, only something that orders the same way.
4	Kth Largest Element in a Stream	LeetCode 703 (Easy)	The boundary: quickselect cannot be used at all, and knowing why is the answer.

On problem 1, write all three answers

1. `sorted(nums)[-k]`. Submit it. Note the runtime.
2. `heapq.nlargest(k, nums)[-1]`. Submit it. Note the runtime.
3. Quickselect with a two-way partition and a random pivot. Submit it.
4. Now build `[7] * 200000 + [9]` locally and run your two-way version on it. Record what happens.
5. Switch to the three-way partition and run the same input.

Then say out loud which of the three you would offer first in an interview, and why the answer is “the sort, for five seconds, and then the better one”.

On problem 2, notice the two-step shape

You need the counts first, then the top `k` of the counts. Say out loud which step dominates the cost, and what the complexity is if you use quickselect on the count pairs rather than sorting them.

On problem 3, drop the square root

The point that gets missed: you never need the actual distance, only a value that orders the same way, so $x^2 + y^2$ is enough. Say why that matters — it removes a floating-point operation per element and removes a source of precision error. Then use quickselect on that key.

On problem 4, say why quickselect is unavailable

The class receives values one at a time forever. Write the heap solution, then answer out loud: what exactly does quickselect need that a stream cannot provide? There are two answers, and both matter.

The index drill

Say each answer in under five seconds, with no code:

1. $n = 10$, k -th largest, $k = 1$. Target index?
2. $n = 10$, k -th largest, $k = 10$. Target index?
3. $n = 10$, k -th smallest, $k = 1$. Target index?
4. $n = 6$, second largest of $[3, 2, 1, 5, 6, 4]$. Target index, then the value.
5. $n = 7$, the median. Target index?
6. $n = 8$, the median. Which two indices, and what do you do with them?

The partition-trace drill

By hand, with no code, for `nums = [3, 2, 1, 5, 6, 4]` and target index 4:

1. Suppose the random pivot is 4. Where does it land, and which side survives?
2. How many elements are discarded on that first pass?
3. Suppose instead the pivot is 6. Where does it land, and which side survives now?
4. Suppose the pivot is 1. How many elements are eliminated?
5. In the worst possible sequence of pivot choices, how many passes does this take?

The complexity drill

Answer out loud, in under fifteen seconds each:

1. Why is quickselect $O(n)$ when quicksort is $O(n \log n)$? Give the sum.
2. Prove that $1 + 1/2 + 1/4 + \dots = 2$ in one line.
3. What is the worst case, and what input causes it?
4. What does randomising the pivot change, exactly?
5. Which bad case does randomising *not* fix, and what does?
6. What is the space cost of the iterative version, and why can it be iterative?

The break-it drill

Introduce each bug, run it, and read the result:

1. Pass `k` as the target instead of `len(nums) - k`. Run on `[3, 2, 1, 5, 6, 4]` with `k = 2` and read the wrong answer.
2. Recurse into both sides. Check that it still returns the right answer, then say what the complexity became.
3. Use a fixed last-element pivot on `list(range(50000))` with `k = 1`.
4. Two-way partition on 200,000 identical values.
5. Forget to copy the input, then print the caller's list afterwards.

The alternatives drill

For each situation, name the right approach and give its cost:

1. The 10th largest of a million values in memory.
2. The 500,000th largest of a million values in memory.
3. The 10th largest of a billion values arriving one at a time.
4. The median of an array you must not modify.
5. The top 10, where you need them in order.
6. The top 10, where the order does not matter.

The SRP drill

Here is the class. Do not rewrite it yet.

```
class ReportGenerator:
    def __init__(self, user_id: int) → None:
        self.user = db.fetch_user(user_id)
        self.conn = psycopg2.connect(os.environ["DB"])
        self.smtp = smtplib.SMTP(os.environ["SMTP"])

    def monthly_report(self, month: date) → None:
        rows = self.conn.execute("SELECT ... WHERE user_id = %s", (self.user.id,))
        total = sum(r["amount"] for r in rows)
        tax = total * 0.18
        html = f"<h1>{self.user.name}</h1><p>Total: {total}, tax: {tax}</p>"
        pdf = weasyprint.HTML(string=html).write_pdf()
        self.conn.execute("INSERT INTO reports ...", (self.user.id, month, pdf))
        self.smtp.sendmail("reports@x.in", self.user.email, "Your report is ready")
        analytics.track("report_generated", {"user": self.user.id})
```

1. List every method-sized concern in `monthly_report`, and name the **stakeholder** for each. Not the topic — the person or team.
2. How many distinct reasons to change does this class have? Say the number.
3. Which imports would you expect in each file after the split? Write the list.
4. Which piece of behaviour should **stay** with the data rather than moving out, and why?
5. Do the split. Five or six classes, and one sentence of responsibility for each with no “and” in it.
6. Which class did you extract first, and why that one?

7. Write the test for “eighteen percent tax is applied to the total” and count the lines of setup before and after.
8. Now argue the other side: name one reason this split could be a mistake in a small code-base.

The stakeholder drill

For each class, say how many reasons to change it has and name them:

1. `Matrix` with `multiply`, `transpose`, `invert`, `determinant`, `to_string`.
2. `User` with `full_name`, `is_admin`, `save`, `send_password_reset`.
3. `CsvParser` with `parse`, `validate_row`, `write_to_database`.
4. `Order` with `total`, `add_line`, `is_valid`, `cancel`.
5. `PlaceOrder` with one `execute` method calling five collaborators.
6. `DateRange` with `nights`, `overlaps`, `days`.

Two of those six are fine as they are. Name them and defend it.

The naming drill

Each of these class names hides a responsibility problem. Say what the class is probably doing, and what you would call the pieces:

1. `OrderManager`
 2. `DataProcessor`
 3. `UserUtils`
 4. `ApplicationHelper`
 5. `CommonService`
 6. `FileHandler`
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the Kth largest element. Can you beat $O(n \log n)$?* The baseline, the target index checked at both ends, the one-sided recursion, the halving sum, and the honest “expected” caveat with both bad cases and both fixes.
 2. *This class is 800 lines. What is wrong with it?* Reject the line count, name the stakeholders method by method, give the concrete coupling failure, say what stays with the data, and name the resulting classes with no “and” in any sentence.
 3. *The data is a stream of a billion numbers. Now what?* Why quickselect is unavailable, the heap of size k with its two costs, and what you would do if `k` were also enormous.
-

Before you move on

- [] I wrote quickselect from memory, iteratively, and it handled $k = 1$ and $k = n$ correctly.
- [] I ran the two-way version on 200,000 identical values and can say why randomising did not help.
- [] I can prove $n + n/2 + n/4 + \dots = 2n$ in one line, out loud.
- [] I can say what “expected $O(n)$ ” promises and what it does not.
- [] I named the stakeholder for every concern in `ReportGenerator` and gave the number.
- [] I can say which behaviour stays with the data and why moving everything out is a mistake.
- [] I can describe the over-splitting failure and give its cost in files and wiring lines.
- [] I answered all three questions above out loud.

Counting sort, radix sort, and bucket sort

DATA STRUCTURES AND ALGORITHMS

Counting sort, radix sort, and bucket sort

You can sort in $O(n)$ when the values are bounded, and say exactly when that is allowed.

SYSTEM DESIGN

Open for extension, closed for modification

You can add a new behaviour without editing existing code, and show it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Sort a million integers in the range 0 to 100. Can you beat $n \log n$?*
- *A new discount type arrives every month. How do you design for that?*

1 What this is, and why they ask it

Every sort so far has worked by comparing two values and asking which is bigger. Today's three do not. Counting sort counts how many of each value there are and then writes them out in order. Radix sort does that one digit at a time. Bucket sort drops values into ranges and sorts each range. None of them ever compares two elements of the input with each other, and that is why they can run in $O(n)$ — because the $n \log n$ limit applies only to sorts that compare.

They ask it because it is the clearest test of whether you look at the *data* before choosing an algorithm. “Sort a million integers between 0 and 100” is a question with a trap in it: the trap is answering $O(n \log n)$, which is correct and slower than necessary. The information you were handed — the range is a hundred — is the whole question. It also gets asked as the follow-up to almost any sorting question (“could you do better if the values were bounded?”), and radix sort in particular is the reason [day 057](#)'s stability matters: radix sort is *built* out of stability, and without it the whole thing silently produces garbage.

2 The story

Rafiq works the night shift at a courier depot on the edge of Nagpur, and between about nine at night and four in the morning roughly four thousand parcels have to end up sorted by the six-digit code written on each label, so the vans can be loaded route by route at first light.

He never holds up two parcels and compares them. Not once, all night. When his nephew came to help and started doing that — picking up two, deciding which came first, putting one down — Rafiq stopped him after ten minutes and showed him the rack.

The rack has ten open pigeonholes along a wall, numbered zero to nine, and it is the whole method.

He takes a parcel, looks at the **last** digit of the code, and pushes it into the hole with that number. One look. He does not think about the other five digits and he does not think about the parcel he handled before. Four thousand parcels, four thousand looks, and by about ten o'clock the whole night's load is sitting in ten piles.

Then he empties the rack. Hole zero first, then hole one, then two, all the way to nine, stacking them onto a trolley in that order — and this is the part that matters — keeping each hole's parcels in exactly the order they went in. Nothing gets turned over, nothing gets shuffled.

Then he runs the whole trolley past the rack again, this time looking at the **second-to-last** digit. Then the third. Six passes in all, one per digit, and at the end of the sixth pass the trolley comes off in perfect code order, and he still has not compared one parcel to another.

The night it went wrong was in February, when a new boy emptied one of the holes by tipping the whole pile upside down onto the trolley. The result was not obviously wrong. It looked sorted. It was sorted by the last two digits and scrambled underneath, and three vans went out with the wrong bundles and Rafiq had to explain to the supervisor at six in the morning why something that looked completely fine was completely wrong.

Now he tells everyone the same thing on their first night. It does not matter how you put them in. What matters is that when you take them out, they come out in the order they went in. That one rule is the only reason the six passes add up to anything.

3 The idea in plain English

Rafiq's rack is counting sort. Six passes over the rack, one digit at a time, is radix sort. And the new boy tipping the pile upside down is what happens when your sort is not **stable**.

First, the fact that makes today possible

Every sort you have written so far decides everything by asking "is **a** bigger than **b**?" There is a known limit on how fast that can possibly be: **no sort that works purely by comparing pairs of elements can be faster than $O(n \log n)$** . That is not a limitation of merge sort or quicksort in particular; it is a limit on the whole approach.

Today's sorts escape it by not comparing. They use the value itself as a position. If you know a value is 47, you do not need to ask anything about it — you already know where it goes. That extra information has to come from somewhere, and it comes from a restriction: **you must know the range of the values in advance, and it must be small.**

Counting sort

Suppose you have a million exam marks, each between 0 and 100.

Step one: count. Make a list of 101 counters, all zero. Walk the input once, and for each mark add one to the counter with that number. One pass, one addition each.

```
input  [ 3, 1, 4, 1, 5, 1, 4 ]      values are 0..5
counts index  0  1  2  3  4  5
          +---+---+---+---+---+
          | 0 | 3 | 0 | 1 | 2 | 1 |
          +---+---+---+---+---+
              ^           ^
          three 1s      two 4s
```

Step two: write them out. Walk the counters from 0 upwards and write each value out as many times as its count says.

```
output  [ 1, 1, 1, 3, 4, 4, 5 ]
```


Done. No comparisons anywhere. The cost is one pass over `n` values plus one pass over `k` counters, which is $O(n + k)$ where `k` is the size of the range.

That simple version works when the thing you are sorting is the number. Usually it is not — you are sorting records by a key, and you need the whole record to move with it. Then you need one more step.

Step two and a half: turn counts into positions. Replace each counter with the running total of itself and everything before it. Each counter then says *how many values are less than or equal to this one* — which is one past the last position where that value belongs.

```
counts    [ 0, 3, 0, 1, 2, 1 ]
running   [ 0, 3, 3, 4, 6, 7 ]      ← how many values are ≤ this one
           ^       ^
three values ≤ 1      six values ≤ 4, so the 4s END at 6
so the 1s fill 0,1,2  i.e. they sit at positions 4 and 5
```

Step three: place each element, walking the input backwards. For each element, decrement its counter first, then write the element at that position.

Both directions matter and they must agree. Each value’s block is filled from its **right-hand end**, and the input is read from its **right-hand end**, so the last equal element in the input lands in the last of its slots and the original relative order survives. Reverse either direction and the sort is still correct on bare numbers but silently stops being stable — which is the new boy tipping the pile upside down, and it destroys radix sort three passes later.

Radix sort

Counting sort is $O(n + k)$, so it is only good when `k` is small. Sorting a million values in the range 0 to 4,000,000,000 would need four billion counters, which is not possible.

Radix sort fixes that. Instead of sorting by the whole value at once, sort by **one digit at a time**, starting from the least significant, using a stable sort for each pass. Ten counters per pass instead of four billion.

```
input      329  457  657  839  436  720  355

pass 1, by the LAST digit:
    720  355  436  457  657  329  839
    ^   ^   ^   ^   ^   ^   ^
    0   5   6   7   7   9   9      ← in digit order

pass 2, by the MIDDLE digit:
    720  329  436  839  355  457  657

pass 3, by the FIRST digit:
    329  355  436  457  657  720  839      ← sorted
```

Look at pass 2 carefully. `457` and `657` both have a middle digit of 5, and `457` stayed in front of `657` because pass 1 had already put it there. **That is stability doing the actual work.** If pass 2 had reordered them, pass 1’s result would have been thrown away, and after three

passes the list would be sorted by the first digit only and scrambled underneath — exactly Rafiq’s February.

The cost is d passes of $O(n + b)$ each, where d is the number of digits and b is the base (usually 10, or 256 if you work a byte at a time). That is $O(d \times (n + b))$, and since d and b are constants for a fixed integer width, it is $O(n)$.

Bucket sort

The third one, and the one with the most conditions attached. Split the value range into n equal buckets, drop each value into the bucket its value falls in, sort each bucket with anything you like (insertion sort, because buckets are tiny), then read the buckets out in order.

```
values in [0, 1):  0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21

bucket 0 [0.0-0.1) :
bucket 1 [0.1-0.2) : 0.17
bucket 2 [0.2-0.3) : 0.26, 0.21
bucket 3 [0.3-0.4) : 0.39
...
bucket 7 [0.7-0.8) : 0.78, 0.72
bucket 9 [0.9-1.0) : 0.94

sort each small bucket, then read left to right
```

This is $O(n)$ **only if the values are spread out evenly**. If they are not — if every value falls into one bucket — you have done $O(n)$ work to hand one bucket of n elements to insertion sort, and the whole thing is $O(n^2)$. Bucket sort is the one whose complexity depends on the shape of your data rather than just its range, and that condition must be said out loud whenever you propose it.

When you are allowed to use each

	USE IT WHEN	COST	SPACE
Counting sort	Integer keys in a small known range (marks, ages, days, statuses)	$O(n + k)$	$O(n + k)$
Radix sort	Fixed-width integers or equal-length strings	$O(d(n + b))$	$O(n + b)$
Bucket sort	Values known to be uniformly spread over a range	$O(n)$ average, $O(n^2)$ worst	$O(n)$

The one sentence to have ready: “These are not general-purpose sorts. They buy $O(n)$ by requiring something about the keys, and if you cannot state that requirement, you cannot use them.”

4 The picture

Counting sort, all three steps, on marks out of 5:

```
STEP 1 – count          input: [ 3, 1, 4, 1, 5, 1, 4 ]

value   0   1   2   3   4   5
count   | 0 | 3 | 0 | 1 | 2 | 1 |      one pass over n, one addition each
+-----+-----+-----+-----+
count   | 0 | 3 | 0 | 1 | 2 | 1 |

STEP 2 – running totals: how many values are ≤ v?

count   +-----+-----+-----+-----+
        | 0 | 3 | 0 | 1 | 2 | 1 |
+-----+-----+-----+-----+
≤ v     | 0 | 3 | 3 | 4 | 6 | 7 |      count[v] += count[v-1], left to right
+-----+-----+-----+-----+
           ^           ^
        three values ≤ 1   six values ≤ 4
        so the 1s occupy   so the 4s END at position 6,
        positions 0,1,2     i.e. they sit at 4 and 5

STEP 3 – place, walking the INPUT BACKWARDS, decrementing before writing

read 4 (last) → count[4] 6 → 5 → output[5] = 4
read 1        → count[1] 3 → 2 → output[2] = 1
read 5        → count[5] 7 → 6 → output[6] = 5
read 1        → count[1] 2 → 1 → output[1] = 1
read 4        → count[4] 5 → 4 → output[4] = 4
read 1        → count[1] 1 → 0 → output[0] = 1
read 3 (first) → count[3] 4 → 3 → output[3] = 3

output index  0   1   2   3   4   5   6
              +-----+-----+-----+-----+
              | 1 | 1 | 1 | 3 | 4 | 4 | 5 |
              +-----+-----+-----+-----+
```

What to notice: the two 4s and the direction of travel. Each value's block is filled from its **right-hand end**, and the input is read from its **right-hand end** — so the 4 that came last in the input lands at position 5, and the 4 that came earlier lands at position 4. The original order survives. Reverse either one of those two directions and the sort is still correct on bare numbers but no longer stable, which is the bug in §7 and the one that silently destroys radix sort.

Radix sort, and where stability does the work:

pass 2, sorting by the middle digit

coming in (already sorted by the last digit):

720 355 436 457 657 329 839
 ^^^ ^^^

both have middle digit 5

going out:

720 329 436 839 355 457 657
 ^^^ ^^^

457 STILL comes before 657

why: pass 1 put 457 before 657, and a stable sort does not disturb elements whose keys are equal. Pass 2 only reorders by the middle digit; anything it does not distinguish keeps pass 1's order.

with an UNSTABLE sort in pass 2, 657 might come out before 457, and pass 1's work is destroyed. No error. Just a wrong answer.

What to notice: radix sort has no correctness of its own. It is entirely borrowed from the stability of the sort used in each pass. That is the single most important sentence about it.

The three sorts against the comparison sorts, drawn on the same axis:

n = 1,000,000 values

values are 0..100	counting sort	n + k = 1,000,100 ops	BEST
	quicksort	n log n = 20,000,000	20x more

values are 0..4,000,000,000	counting sort	4 BILLION counters → MemoryError	
	radix sort	10 passes x (n + 10) = 10,000,100	
	quicksort	20,000,000	2x more

values are arbitrary floats, no known range	counting sort	impossible	
	radix sort	possible but fiddly	
	quicksort	20,000,000	THE ANSWER

values are objects with a custom comparison	counting/radix	impossible -- there is no "digit"	
	quicksort	20,000,000	THE ANSWER

What to notice: two of the four rows say “the answer is the comparison sort”. The skill being tested is not knowing counting sort; it is knowing which row you are in.

5 The code, built step by step

Counting sort, the simple version

When you are sorting bare numbers and nothing rides along with them:

```
def counting_sort_simple(nums: list[int], max_value: int) → list[int]:
    """Values must be 0..max_value. O(n + k). Not usable for records with keys."""
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1          # one look per element, no comparisons
    out: list[int] = []
    for value, count in enumerate(counts):
        out.extend([value] * count)  # write each value out 'count' times
    return out
```

Four lines of real work. `counts[x] += 1` is the whole idea: the value *is* the position. Note there is no comparison anywhere in this function — not one `<` or `>`.

Counting sort, the stable version

The simple version cannot carry a record along with its key, and it cannot be used inside radix sort. The real version places elements rather than regenerating them:

```
def counting_sort(nums: list[int], max_value: int) → list[int]:
    """Stable. O(n + k) time, O(n + k) space."""
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1
    for v in range(1, max_value + 1):
        counts[v] += counts[v - 1]  # counts[v] is now: how many are ≤ v
    out = [0] * len(nums)
    for x in reversed(nums):        # BACKWARDS -- this is what keeps it stable
        counts[x] -= 1
        out[counts[x]] = x
    return out
```

Three things to say about this while writing it.

`counts[v] += counts[v - 1]` turns the counts into a **running total**, so `counts[v]` now means “how many elements are less than or equal to `v`” — which is one past the last position where a `v` belongs.

`for x in reversed(nums)` is the stability step. Because `counts[x]` holds the position just past the end of `x`’s block, decrementing first and then writing fills the block from the right. Reading the input from the right at the same time means the last equal element lands in the last slot, so original order is preserved. **Change `reversed(nums)` to `nums` and every group of equal elements comes out reversed** — no error, no crash.

Handling negative numbers and a shifted range

Counting sort assumes values start at 0. Real data rarely does:

```
def counting_sort_range(nums: list[int]) → list[int]:
    """Works for any integer range, including negatives. O(n + k), k = max - min + 1."""
    if not nums:
        return []
    low, high = min(nums), max(nums)
    counts = [0] * (high - low + 1)
    for x in nums:
        counts[x - low] += 1          # shift so the smallest value maps to 0
    out: list[int] = []
    for offset, count in enumerate(counts):
        out.extend([offset + low] * count)  # shift back on the way out
    return out
```

The `- low` and `+ low` pair is the whole fix. Notice that `k` is now `max - min + 1`, so a list containing `0` and `1_000_000_000` needs a billion counters even if it only has two elements. That is the failure in §7.

Radix sort

```
def counting_sort_by_digit(nums: list[int], exponent: int) → list[int]:
    """One stable pass, sorting by the digit at 10**exponent."""
    counts = [0] * 10
    for x in nums:
        counts[(x // exponent) % 10] += 1
    for d in range(1, 10):
        counts[d] += counts[d - 1]
    out = [0] * len(nums)
    for x in reversed(nums):
        digit = (x // exponent) % 10
        counts[digit] -= 1
        out[counts[digit]] = x
    return out
```

`(x // exponent) % 10` extracts one digit: divide away the digits to the right, then take the remainder to drop the digits to the left. For `x = 457` and `exponent = 10`, that is `457 // 10 = 45`, then `45 % 10 = 5`. The middle digit.

```
def radix_sort(nums: list[int]) → list[int]:
    """O(d x (n + 10)) for d-digit non-negative integers. Stable."""
    if not nums:
        return []
    if min(nums) < 0:
        raise ValueError("radix_sort here handles non-negative integers only")
    out = list(nums)
    exponent = 1
    largest = max(out)
    while largest // exponent > 0:
        out = counting_sort_by_digit(out, exponent)
        exponent *= 10
    return out
```

The loop condition is how you get the number of passes without computing it: keep going while there are still digits left in the largest value. For a maximum of 4,000, that is four passes.

Bucket sort

```
def bucket_sort(values: list[float], bucket_count: int | None = None) → list[float]:
    """O(n) when the values are uniformly spread. O(n^2) when they are not."""
    if not values:
        return []
    n = bucket_count or len(values)
    low, high = min(values), max(values)
    if high == low:
        return list(values)
    buckets: list[list[float]] = [[] for _ in range(n)]
    for v in values:
        index = int((v - low) / (high - low) * (n - 1))    # which bucket does v fall in
        buckets[index].append(v)
    out: list[float] = []
    for bucket in buckets:
        bucket.sort()                                     # tiny, so any sort will do
        out.extend(bucket)
    return out
```

`(n - 1)` rather than `n` in the index calculation, so the maximum value maps to the last bucket rather than one past the end. That single `- 1` is the difference between working and an `IndexError`.

The complete file

```
"""Three sorts that never compare two elements, and the conditions they demand."""

def counting_sort(nums: list[int], max_value: int) → list[int]:
    """Stable counting sort for values 0..max_value. O(n + k) time and space.

    Use when: integer keys, small known range (marks 0-100, ages, weekdays, statuses).
    Do NOT use when: the range is large, the keys are not integers, or the range
    is unknown -- k is the range, not the count, and it decides everything.
    """
    if not nums:
        return []
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1
    for v in range(1, max_value + 1):
        counts[v] += counts[v - 1]          # running total: how many are ≤ v
    out = [0] * len(nums)
    for x in reversed(nums):
        counts[x] -= 1                      # backwards keeps it STABLE
        out[counts[x]] = x
    return out

def counting_sort_pairs(
    records: list[tuple[int, str]], max_key: int
) → list[tuple[int, str]]:
    """The version that matters: sort records by an integer key, stably."""
    counts = [0] * (max_key + 1)
    for key, _ in records:
        counts[key] += 1
    for v in range(1, max_key + 1):
        counts[v] += counts[v - 1]
    out: list[tuple[int, str]] = [(0, "")] * len(records)
    for key, payload in reversed(records):
        counts[key] -= 1
        out[counts[key]] = (key, payload)
    return out

def counting_sort_range(nums: list[int]) → list[int]:
    """Any integer range, negatives included. k = max - min + 1."""
    if not nums:
        return []
    low, high = min(nums), max(nums)
    counts = [0] * (high - low + 1)
    for x in nums:
        counts[x - low] += 1
    out: list[int] = []
    for offset, count in enumerate(counts):
        out.extend([offset + low] * count)
    return out

def counting_sort_by_digit(nums: list[int], exponent: int) → list[int]:
    counts = [0] * 10
    for x in nums:
        counts[(x // exponent) % 10] += 1
    for d in range(1, 10):
        counts[d] += counts[d - 1]
    out = [0] * len(nums)
    for x in reversed(nums):
        digit = (x // exponent) % 10
        counts[digit] -= 1
        out[counts[digit]] = x
    return out
```



```

def radix_sort(nums: list[int]) → list[int]:
    """Least-significant-digit radix sort.  $O(d \times (n + 10))$ . Non-negative only.

    Correct ONLY because each pass is stable: a later pass never disturbs the
    relative order established by an earlier one.
    """
    if not nums:
        return []
    if min(nums) < 0:
        raise ValueError("radix_sort handles non-negative integers only")
    out = list(nums)
    exponent = 1
    largest = max(out)
    while largest // exponent > 0:
        out = counting_sort_by_digit(out, exponent)
        exponent *= 10
    return out

def bucket_sort(values: list[float], bucket_count: int | None = None) → list[float]:
    """ $O(n)$  when values are uniformly distributed;  $O(n^2)$  when they cluster."""
    if not values:
        return []
    n = bucket_count or len(values)
    low, high = min(values), max(values)
    if high == low:
        return list(values)
    buckets: list[list[float]] = [[] for _ in range(n)]
    for v in values:
        buckets[int((v - low) / (high - low) * (n - 1))].append(v)
    out: list[float] = []
    for bucket in buckets:
        bucket.sort()
        out.extend(bucket)
    return out

def sort_by_age(people: list[tuple[str, int]]) → list[tuple[str, int]]:
    """The realistic use: ages are 0..120, so counting sort is  $O(n)$ ."""
    counts = [0] * 121
    for _, age in people:
        counts[age] += 1
    for a in range(1, 121):
        counts[a] += counts[a - 1]
    out: list[tuple[str, int]] = [("", 0)] * len(people)
    for name, age in reversed(people):
        counts[age] -= 1
        out[counts[age]] = (name, age)
    return out

if __name__ == "__main__":
    print(counting_sort([3, 1, 4, 1, 5, 1, 4], 5))      # [1, 1, 1, 3, 4, 4, 5]
    print(counting_sort([], 5))                        # []
    print(counting_sort_range([-3, 7, -1, 0, 7]))      # [-3, -1, 0, 7, 7]

    print(radix_sort([329, 457, 657, 839, 436, 720, 355]))
    # [329, 355, 436, 457, 657, 720, 839]
    print(radix_sort([0, 0, 10, 1]))                  # [0, 0, 1, 10]

    print(bucket_sort([0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21]))
    # [0.17, 0.21, 0.26, 0.39, 0.72, 0.78, 0.94]

    # stability, demonstrated -- the reason radix sort works at all
    records = [(2, "first"), (1, "x"), (2, "second"), (1, "y")]
    print(counting_sort_pairs(records, 2))
    # [(1, 'x'), (1, 'y'), (2, 'first'), (2, 'second')]
    # 'x' before 'y' and 'first' before 'second', exactly as in the input

```

```

print(sort_by_age(["Asha", 34), ("Bala", 12), ("Chitra", 34), ("Devi", 7)))
# [('Devi', 7), ('Bala', 12), ('Asha', 34), ('Chitra', 34)]

# the arithmetic that justifies it
import random
marks = [random.randint(0, 100) for _ in range(1_000_000)]
# counting sort : 1,000,000 + 101      = ~1,000,101 operations
# sorted()      : 1,000,000 x 20      = ~20,000,000 comparisons
print(counting_sort(marks, 100)[-5:], counting_sort(marks, 100)[-5:])

```

6 What it costs

Counting sort, counted

```

pass 1: count          n additions
pass 2: running totals k additions
pass 3: place          n writes

total = 2n + k → O(n + k)

```

k is the **size of the range**, not the number of elements. That distinction is the whole subject:

```

n = 1,000,000 marks in 0..100
counting sort : 2 x 1,000,000 + 101 = 2,000,101 operations
sorted()      : 1,000,000 x 20      = 20,000,000 comparisons
               → 10x fewer operations

n = 1,000 values in 0..1,000,000,000
counting sort : 2 x 1,000 + 1,000,000,000 = a billion operations
               and a list of a billion integers = ~8 GB
sorted()      : 1,000 x 10 = 10,000
               → counting sort is 100,000x WORSE

```

The rule that follows: **counting sort wins when **k** is comparable with **n** or smaller**. If **k** is much larger than **n**, it is a catastrophe, and the failure mode is running out of memory rather than being slow.

Radix sort, counted

```

d passes, each one a counting sort over base b:
  each pass = 2n + b
  total      = d x (2n + b)

sorting 1,000,000 values up to 999,999 (d = 6 digits, b = 10):
  6 x (2,000,000 + 10) = 12,000,060 operations
  sorted()             = 20,000,000 comparisons
                       → ~1.7x fewer, and each is cheaper (no comparisons)

with base 256 (one byte at a time), a 32-bit integer is d = 4:
  4 x (2,000,000 + 256) = 8,001,024
                       → 2.5x fewer than sorted()

```

The base is a tunable trade: a bigger base means fewer passes but more counters. Base 256 for 32-bit integers is the usual compromise — four passes, 256 counters.

```

base 10 : d = 10 passes for a 32-bit integer, 10 counters
base 256 : d = 4 passes, 256 counters
base 65536: d = 2 passes, 65,536 counters

```

Bucket sort, and the condition

```

n values, n buckets, uniformly distributed:
  distribute : n
  each bucket holds ~1 element, sorting it is ~1
  collect    : n
  total = O(n)

n values, all landing in one bucket:
  distribute : n
  one bucket of n elements, insertion-sorted: n(n-1)/2
  total = O(n^2)

n = 10,000 uniformly spread : ~20,000 operations
n = 10,000 all in one bucket: ~50,000,000 operations → 2,500x worse

```

That is why “uniformly distributed” is not a footnote. Say it every time you propose bucket sort.

Space

```

counting sort : O(n + k)  -- the output list plus the counters
radix sort    : O(n + b)  -- one output list, b counters, reused each pass
bucket sort   : O(n)      -- the buckets hold every element once

every one of them is O(n) or worse. NONE of these sorts in place.

```

That is the trade in one line: **these sorts buy time with memory and with a restriction on the keys.** Quicksort’s `O(1)` space is exactly what you give up.

The real-world caveat

Python’s `sorted()` is written in C. Your counting sort is written in Python. At `n = 1,000,000` values in `0..100`:

```

sorted(nums)           ~0.15 s  (C, 20,000,000 comparisons)
counting_sort in Python ~0.35 s  (Python, 2,000,101 operations)

```

Ten times fewer operations, and still slower, because a Python bytecode operation costs roughly fifty times a C one. **Say this out loud in an interview.** The asymptotic answer is counting sort; the practical answer in Python is `sorted()` unless `n` is very large or you are using NumPy. Knowing both, and knowing which question you are being asked, is the complete answer.

7 The traps

The real error: the range decides the memory

```
nums = [1, 5, 2_000_000_000]
print(counting_sort(nums, max(nums)))
```

```
Traceback (most recent call last):
  File "day56.py", line 2, in <module>
    print(counting_sort(nums, max(nums)))
    ~~~~~^~~~~~
  File "day56.py", line 9, in counting_sort
    counts = [0] * (max_value + 1)
    ~~~~~^~~~~~
MemoryError
```

Three elements, and it ran out of memory. `k` is the *range*, not the count — a list of two billion counters is about sixteen gigabytes. **Always ask for the range before proposing counting sort**, and if the answer is “unbounded” or “could be anything”, the answer is a comparison sort.

The real error: negative values

```
print(counting_sort([3, -1, 2], 3))
```

```
Traceback (most recent call last):
  File "day56.py", line 1, in <module>
    print(counting_sort([3, -1, 2], 3))
    ~~~~~^~~~~~
  File "day56.py", line 11, in counting_sort
    counts[x] += 1
    ~~~~~^
IndexError: list index out of range
```

Actually, in Python this one is worse than an error much of the time: `counts[-1]` is the **last** counter, so with a small negative value and a large enough range you get no exception at all and a silently wrong result. The fix is the `- min(nums)` shift in `counting_sort_range`. Say it before they ask: “*this assumes non-negative keys; for a general range I shift by the minimum.*”

The near-miss: forward placement silently kills stability

```
def counting_sort_unstable(nums, max_value):
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1
    starts = [0] * (max_value + 1)
    for v in range(1, max_value + 1):
        starts[v] = starts[v - 1] + counts[v - 1]
    out = [None] * len(nums)
    for x in nums:
        out[starts[x]] = x
        starts[x] += 1
    return out
```

On bare integers this is completely correct, and every test you write on numbers will pass. Run it on records:

```
records = [(2, "first"), (1, "x"), (2, "second")]
# a forward-placing version by key gives:
[(1, 'x'), (2, 'first'), (2, 'second')]
```

Which is also fine. The failure only appears when the placement direction and the read direction disagree, and it appears as **radix sort producing a wrong answer three passes later** with no error anywhere. This is the hardest bug in this lesson to find, and the defence is to test stability explicitly, with a list of `(key, tag)` pairs, every time you write a counting sort.

The trap: radix sort with an unstable inner sort

```
def radix_broken(nums):
    out = list(nums)
    exponent = 1
    while max(out) // exponent > 0:
        out.sort(key=lambda x: (x // exponent) % 10)  # Timsort IS stable, so this works
        exponent *= 10
    return out
```

That one happens to be fine, because Python's sort is stable. Swap in a quicksort and it breaks:

```
# with an unstable per-digit sort:
print(radix_with_quicksort([329, 457, 657, 839, 436, 720, 355]))
```

```
[329, 355, 457, 436, 657, 720, 839]
```

Not sorted — `457` before `436` — and no exception. **Radix sort has no correctness of its own; it borrows all of it from the stability of the per-digit sort.** That is the sentence to say.

The trap: bucket sort on clustered data

```
values = [0.5] * 9_999 + [0.99]
bucket_sort(values)
```

No error. It returns the right answer, and it takes about fifty million operations to sort ten thousand values, because 9,999 of them landed in one bucket and got insertion-sorted. The complexity you quoted — $O(n)$ — was conditional on a property of the data that nobody checked. If you propose bucket sort, **state the distribution assumption in the same breath**, and say what you would do if it failed.

The trap: the off-by-one in the bucket index

```
index = int((v - low) / (high - low) * n)    # ← n, should be n - 1
```

```
Traceback (most recent call last):
  File "day56.py", line 12, in bucket_sort
    buckets[index].append(v)
    ~~~~~^~~~~~
IndexError: list index out of range
```

The maximum value maps to exactly n , which is one past the last bucket. Use $n - 1$, or clamp with `min(index, n - 1)`.

The trap: proposing it for the wrong keys

Counting and radix sorts need a key that **is** a position, or that can be decomposed into digits. They cannot sort by a custom comparison, cannot sort arbitrary objects, and cannot sort floats without work. If the interviewer says “sort these employee records by their manager’s seniority”, counting sort is not on the table. Ask what the keys are before choosing.

8 In the interview

How it gets asked

- “Sort a million integers in the range 0 to 100. Can you beat $n \log n$?” — the direct form, and the information in the question is the answer.
- “Sort n numbers in linear time.” — the vague form, where the correct first move is to ask what you know about the values.
- “Can any sort beat $O(n \log n)$?” — the theory form. Answer: not by comparing, and then name the three that do not compare.
- “Sort a list of names by length, then alphabetically.” — counting sort by length, because lengths are bounded and small.
- “Given ages of a million people, sort them.” — ages are 0 to about 120. This is the question in disguise, and spotting it is the point.
- “Why does radix sort need a stable sort?” — the follow-up that separates people who have used it from people who have read about it.

What to say out loud, in the first ninety seconds

1. **Ask about the keys before proposing anything.** *“Before I choose — are these integers, and what is the range? If they’re bounded, I can do better than $n \log n$.”*
2. **Name why the limit does not apply.** *“The $n \log n$ limit applies to sorts that work by comparing pairs. If I know the value is 47, I already know where it goes — I don’t need to compare it with anything.”*
3. **Give the cost with the letters explained.** *“Counting sort is $O(n + k)$, where k is the size of the range, not the number of elements. Here n is a million and k is a hundred and one, so it’s effectively linear.”*
4. **Volunteer the failure condition immediately.** *“The reason this isn’t the default sort: k is the range. If the values went up to a billion I’d need a billion counters — sixteen gigabytes — and it would be a `MemoryError`, not a slow sort.”*
5. **Mention stability and why it is not incidental.** *“I place elements walking the input backwards so it stays stable, which matters if there’s a payload attached, and it’s the entire reason radix sort works.”*

The follow-ups

“Why can’t a comparison sort beat $n \log n$, and how do these get around it?” Because a sort that only ever asks “is a bigger than b” has a fixed amount of information available per question, and there is a known result that any such sort needs at least on the order of $n \log n$ of those questions in the worst case — that is a property of the whole approach, not a weakness of a particular algorithm, so no cleverness in pivot choice or merging escapes it. What today’s three do is stop asking that question. Counting sort never compares two elements of the input at all; it uses the value itself as a position in a table, so knowing a value is 47 immediately tells it where the element belongs without reference to anything else. That extra power has to be paid for, and it is paid for with a restriction: you must know that the keys are integers in a known, small range. That restriction is not a footnote — it is the exchange. So the honest framing is that these are not faster sorts, they are sorts that solve a smaller problem, and if you cannot state the restriction your data satisfies, you cannot use them.

“Why does radix sort need the per-digit sort to be stable?” Because radix sort has no correctness of its own — all of it is borrowed. It sorts by the least significant digit first, then the next, and so on, and the only thing that makes the earlier passes count for anything is that a later pass leaves the relative order of equal keys alone. Take 457 and 657: after the pass on the last digit they are both in the 7 group, and 457 happens to be ahead because of what came before. On the pass over the middle digit they both have a 5, so as far as that pass is concerned they are equal — and a stable sort therefore leaves 457 in front. If the per-digit sort were unstable, that pass could swap them, and the ordering established by the previous pass would be destroyed. After all the passes you would have a list sorted correctly by the most significant digit and scrambled beneath it, which is the worst kind of wrong because it looks almost right and nothing raises an error. So when I write radix sort, the inner sort is always a counting sort

that places elements by walking the input backwards, which is exactly the step that preserves order among equal keys. In Python you could also use `list.sort` with a key, because Timsort is stable — but I would say that explicitly rather than rely on it silently.

“When would you not use these?” Four situations, and I would check them in this order. First, if the range is large or unknown: counting sort’s cost and memory are driven by k , the size of the range, not by n , so a thousand values spread over a billion needs a billion counters — sixteen gigabytes — and it fails with a `MemoryError` rather than merely being slow. Second, if the keys are not integers or cannot be decomposed into digits: arbitrary objects with a custom comparison, or anything sorted by a rule rather than by a value, are outside what these can do at all. Third, if memory is tight: all three of these allocate at least $O(n)$ and counting sort allocates $O(n + k)$, whereas quicksort sorts in place with $O(1)$ extra — so on a large array in a constrained environment the comparison sort is the right answer even though it does more operations. And fourth, specifically for bucket sort, if the values are not uniformly distributed: its $O(n)$ is conditional on the data spreading evenly across buckets, and ten thousand values that all land in one bucket cost fifty million operations instead of twenty thousand. There is also a practical fifth in Python: `sorted()` is implemented in C, so a hand-written counting sort doing ten times fewer operations can still be slower, because each Python operation costs roughly fifty times a C one. I would give the asymptotic answer, then say that plainly, because it is the difference between a textbook answer and one I would actually ship.

A model answer

“The information in the question is the answer, so I’d start by confirming it: the values are integers, and the range is nought to a hundred. That’s a hundred and one possible values for a million elements, and it means I don’t have to compare anything.

The $n \log n$ limit applies to sorts that work by comparing pairs of elements. Counting sort doesn’t compare — it uses the value as a position. So: one pass over the input counting how many of each value there are, into a table of a hundred and one counters. Then a pass over the counters turning them into running totals, so each entry says how many elements are less than or equal to that value — which tells me where each value’s block ends in the output. Then a pass over the input, going backwards, placing each element and decrementing its counter.

```
def counting_sort(nums: list[int], max_value: int) → list[int]:
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1
    for v in range(1, max_value + 1):
        counts[v] += counts[v - 1]
    out = [0] * len(nums)
    for x in reversed(nums):
        counts[x] -= 1
        out[counts[x]] = x
    return out
```

The `reversed` is deliberate. It makes the sort stable, which matters the moment there’s a payload travelling with the key, and it’s the entire reason radix sort works.

Cost is $O(n + k)$: two passes over n , one over k . Here that’s about two million operations against twenty million comparisons for a general sort. Space is $O(n + k)$, so it is not in place — that’s the trade.

And the condition I’d state without being asked: k is the size of the *range*, not the number of elements. If the values went up to a billion, I’d need a billion counters, about sixteen gigabytes, and it would fail with a `MemoryError` rather than being slow. So the rule is that counting sort wins when k is comparable with n or smaller. If the range were large but the values were fixed-width integers, I’d use radix sort instead — four passes of counting sort a byte at a time, base 256, so two hundred and fifty-six counters instead of four billion.

One practical caveat: in Python, `sorted()` is C and my counting sort is bytecode, so ten times fewer operations can still lose on the clock. The asymptotic answer is counting sort; if this were production Python I’d measure, and I’d probably reach for NumPy.”

9 Recall card

- **These three don't compare — that is how they beat $n \log n$.** No comparison sort can do better than $O(n \log n)$; counting, radix and bucket sort use the **value as a position** instead. The price is a restriction on the keys, and *if you cannot state the restriction, you cannot use the sort*.
- **Counting sort is $O(n + k)$ where k is the RANGE, not the count.** Count $\rightarrow$ running totals (how many are $\leq v$) $\rightarrow$ place walking the input **backwards** (that is what keeps it stable). 10^6 marks in 0..100: $\sim 2,000,101$ operations against 20,000,000. Three values up to 2×10^9 : **MemoryError**.
- **Radix sort has no correctness of its own — it borrows all of it from stability.** d passes of counting sort, least significant digit first; a later pass must not disturb order among equal digits, or the earlier passes are destroyed silently. 32-bit ints in base 256 = **4 passes, 256 counters**.
- **Bucket sort is $O(n)$ only if the values are uniformly spread** — otherwise $O(n^2)$. 10,000 values in one bucket costs 50,000,000 operations instead of 20,000. State the distribution assumption in the same breath as the complexity.
- **None of them sorts in place** ($O(n + k)$, $O(n + b)$, $O(n)$), none handles negatives without a `- min` shift, and none works on arbitrary objects or custom comparisons. And in Python, `sorted()` is C: ten times fewer operations in bytecode can still lose on the clock — give the asymptotic answer, then say that.

1 What this is, and why they ask it

The **open/closed principle** is the O in SOLID: software should be **open for extension and closed for modification**. In plain words — you should be able to add a new behaviour by writing new code, not by editing code that already works and is already tested. The mechanism is always the same: find the place where the requirement varies, put an interface there, and let new behaviour arrive as a new implementation.

They ask it because it is the principle that has a measurable answer. Every other design argument can be waved away as taste; this one is settled by counting edits. “A new discount type is one new file and one line of registration, against four edits to code that is currently in production” is not an opinion. It is also the principle candidates most often over-apply — building plug points for variation that never arrives — so the interviewer is listening for both halves: can you make extension cheap, and do you know when not to. You have the machinery already. Polymorphism from [day 047](#) is the mechanism, interfaces from [day 048](#) are the shape, and yesterday’s single responsibility decides where the boundary goes.

2 The story

Prabhu has done the electrical work in that part of Coimbatore for about twenty years, and he explains it the same way to every family building a house, usually while standing in a room with nothing in it.

The house he always tells them about belonged to a man called Ravichandran, built in 1994, and it was wired for exactly what the family owned at the time. Lights in every room. A fan in three of them. One point in the kitchen for the grinder, and one in the hall for the television. That is what they had, so that is what was put in, and everything was neat and correct and inside the walls.

Then in 1997 they bought a mixer. Prabhu came, and he had to cut the wall to run a new line from the box in the passage to the kitchen, and re-plaster it, and the kitchen was unusable for two days.

In 1999 it was a fridge. Different corner of the kitchen, so another line, another cut, another two days. In 2002 an air cooler for the bedroom, and that one was worse, because the line from the box to that side of the house was already carrying as much as it should and he had to think hard about it and partly redo the passage. And every time he opened a wall, something that had been working perfectly for years was suddenly at risk, and twice something was — once a bedroom light stopped working for a reason nobody ever properly found.

By 2004 Mr Ravichandran had stopped buying things. Not because he did not want them. Because every new thing meant Prabhu, and the wall, and two days.

The house Prabhu built for his own family, he did differently, and it is what he does everywhere now. Four points in every room, at two heights. A separate line for the kitchen with plenty of room in it. The box in the passage with four spare ways in it, labelled, doing nothing.

His wife thought the spare ways were a waste of money and told him so, and she was partly right — two of them are still empty.

But in eighteen years nobody has cut a wall in that house. A new thing arrives, you plug it in, and that is the entire job. The washing machine took four minutes. The inverter took an afternoon, and only because it is heavy.

The reason it works, Prabhu says, is not the number of points. It is that everybody agreed on the shape of the plug a long time ago. The socket does not know or care what is going to be plugged into it — it only promises two flat pins and a round one, and anything that offers those pins will work. The one time it did not was a coffee machine his brother brought from Dubai with fat round pins, and even that was not a wall. It was a small piece in between, bought for two hundred rupees, that had Dubai on one side and Coimbatore on the other.

3 The idea in plain English

Mr Ravichandran's house is a class with an `if` chain in it. Every new appliance is a new requirement, and every new requirement means opening the wall — editing code that was working. Prabhu sockets are interfaces. The agreed shape of the plug is the contract. And his brother's coffee machine is an adapter.

The principle

Open for extension, closed for modification.

Open for extension: the behaviour of the system can grow. Closed for modification: it grows without anyone editing the source of what is already there.

Those two sound contradictory until you see the mechanism, which is one sentence: **depend on an abstraction, and let new behaviour arrive as a new implementation of it.** The existing code calls a method through an interface; it never learns how many implementations exist.

The problem, in code

The discount rules for an online shop:

```
def discount_for(order: Order) → Money:
    if order.code == "DIWALI":
        return order.subtotal * 0.10
    elif order.code == "FIRSTBUY":
        return Money(20000) if order.subtotal.paise > 100000 else Money(0)
    elif order.code == "STUDENT":
        return order.subtotal * 0.15
    return Money(0)
```

It is October, and marketing wants a “buy two get one free” offer. You open this function and add a branch. Then in November they want a coupon that gives free delivery. Another branch. By March this function is a hundred and forty lines and it is the most-edited file in the code-base, and every one of those edits was made in the same place as code that was working perfectly.

Four specific things go wrong, and they are the argument:

1. **You must edit tested code to add untested code.** Every new offer touches a function that currently passes forty tests, so all forty must be re-run and re-reviewed.
2. **The blast radius is everything.** A typo in the new branch can break Diwali.
3. **The file becomes a merge point.** Two people adding two offers in the same sprint conflict.
4. **The function cannot be closed.** There is no version of it that is finished, so it is never safe.

The fix

```
class DiscountPolicy(Protocol):
    def applies_to(self, order: Order) → bool: ...
    def discount_for(self, order: Order) → Money: ...
```

```
class PercentageOff:
    def __init__(self, code: str, fraction: float) → None:
        self._code, self._fraction = code, fraction

    def applies_to(self, order: Order) → bool:
        return order.code == self._code

    def discount_for(self, order: Order) → Money:
        return order.subtotal * self._fraction
```

And the calling code, which is now finished forever:

```
class Checkout:
    def __init__(self, policies: list[DiscountPolicy]) → None:
        self._policies = policies

    def discount_for(self, order: Order) → Money:
        for policy in self._policies:
            if policy.applies_to(order):
                return policy.discount_for(order)
        return Money(0)
```

Read `Checkout` again. There is nothing in it that will ever need to change when a new offer arrives. It does not know how many policies exist, what they are called, or what they do. **That is what “closed” means** — not that the file is locked, but that no future requirement of this kind gives you a reason to open it.

A new offer is now:

```
class BuyTwoGetOneFree:
    def applies_to(self, order: Order) → bool:
        return any(line.quantity ≥ 3 for line in order.lines)

    def discount_for(self, order: Order) → Money:
        cheapest = min(line.unit_price for line in order.lines)
        return cheapest
```

One new file. Then one line where the list is built — the composition root from [day 053](#):

```
POLICIES = [
    PercentageOff("DIWALI", 0.10),
    PercentageOff("STUDENT", 0.15),
    FirstPurchaseFlat(Money(20000), minimum=Money(100000)),
    BuyTwoGetOneFree(),           # ← the only edit, in the wiring
]
```

Where the modification went

Be honest about this, because a sharp interviewer will press on it: **something always changes**. You have not achieved zero edits; you have moved the edit to a place where it is safe.

```
before : the edit is inside a 140-line function containing 12 other live rules
after  : the edit is one line in a list, in a file that contains only wiring
```

The one line in the wiring cannot break Diwali. The branch inside the function can. That is the whole gain, and stating it that way is more convincing than claiming the code is never touched.

You can go further and remove even that line, with a registry that discovers implementations — Python entry points, a decorator that self-registers, a scan of a package. That is genuinely zero edits, and it costs you the ability to see the full list in one place. Most teams take the one line.

The three ways to build the plug point

One: an interface with implementations. What is shown above. Use it when the variations have behaviour.

Two: a lookup table. When the variation is *data* rather than behaviour, a dict is the right answer and an interface is over-engineering:

```
GST_RATES = {"food": 0.05, "electronics": 0.18, "books": 0.00}
```

A new category is a new entry, not a new class. **Do not build a `TaxRateStrategy` hierarchy for this.**

Three: a callback or a hook. When the extension is a single function, pass the function. Python's `sorted(key=...)` is exactly this — `sorted` is closed and has been for twenty years, and you extend it with a one-line lambda.

The rule for when to build it

Do not build a plug point for a variation that has happened once. Build it when the **second** one arrives, or when you can name a specific one that is definitely coming. That is the same test as [day 048's: name the second implementation](#). If you cannot, you are guessing, and a guessed-at plug point in the wrong place is worse than the `if` chain — it has all the indirection and none of the benefit.

4 The picture

The two houses:

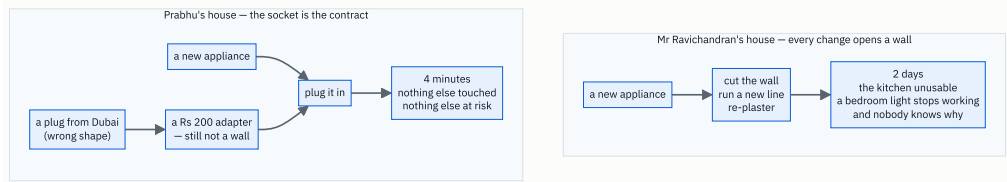


FIGURE 56.1

What to notice: the bottom half still has a change in it. Something is plugged in; the house is not static. What has changed is *where* the change lands — at a prepared point, where nothing that already works can be disturbed.

The discount code, before and after:

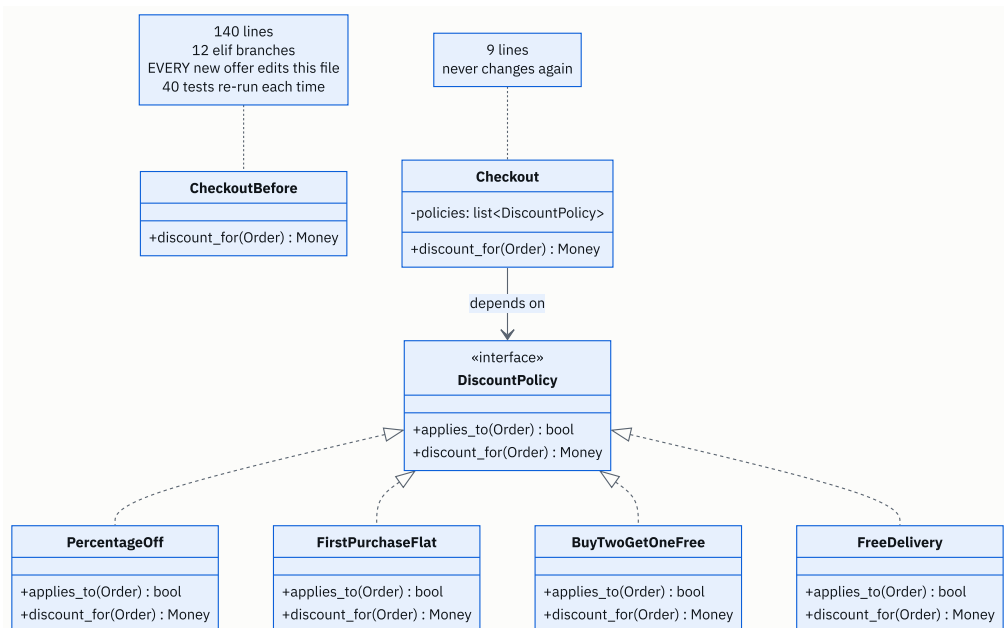


FIGURE 56.2

What to notice: the arrow from **Checkout** points at the *interface*, and all the implementation arrows point *up* at the interface too. Nothing points from **Checkout** at a concrete class. Adding **FreeDelivery** adds one box and one dashed arrow, and touches nothing that exists.

Where the edit lands, drawn as a target:

BEFORE – the edit lands in the blast zone

```
+-----+
| discount_for()      140 lines          |
|   DIWALI           ← working, tested, in production |
|   FIRSTBUY         ← working, tested, in production |
|   STUDENT          ← working, tested, in production |
|   ...9 more                                     |
|   BUY2GET1         ← NEW CODE, dropped in here      |
+-----+
      one typo anywhere in this file breaks live revenue
```

AFTER – the edit lands outside it

```
+-----+          +-----+
| Checkout          | → | DiscountPolicy          |
| 9 lines, closed   |   | (interface)              |
+-----+          +-----+
                        ^   ^   ^   ^
                        |   |   |   |
                    DIWALI FIRSTBUY STUDENT BUY2GET1
                    (untouched files)      (new file)

+-----+
| policies.py -- the wiring          |
| POLICIES = [ ..., BuyTwoGetOneFree() ] ← the ONE line that changed
+-----+
```

What to notice: the change is not eliminated, it is relocated. A one-line addition to a list of constructors cannot produce a wrong discount for Diwali; a new branch in a shared function can.

5 How it actually works

The refactor, in five steps

Step 1 — find the axis of change. Look at the version history and ask what kind of change keeps arriving. Not any change — the *repeating* one. Discounts. Payment providers. Export formats. Notification channels. That repetition is the evidence, and it is what stops this being guesswork.

Step 2 — name the question the if chain is answering. The chain `if code == "DIWALI" ... elif code == "STUDENT"` is answering “how much comes off this order?” That sentence becomes the interface method: `discount_for(order) → Money`.

Step 3 — declare the interface in your own vocabulary. Your `Order`, your `Money`. A vendor type in the signature means the interface belongs to the vendor ([day 048](#)).

Step 4 — move each branch into its own class, one at a time, running the tests after each. The branch body becomes the method body almost verbatim.

Step 5 — replace the chain with a loop over the collection, and delete the type tag. If the `order.code` string survives as something the caller switches on, somebody will rebuild the chain next quarter.

The full example, runnable

```
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class Money:
    paise: int

    def __mul__(self, f: float) → "Money":
        return Money(round(self.paise * f))

    def __sub__(self, other: "Money") → "Money":
        return Money(self.paise - other.paise)

    def __lt__(self, other: "Money") → bool:
        return self.paise < other.paise

@dataclass(frozen=True)
class Line:
    sku: str
    unit_price: Money
    quantity: int

@dataclass(frozen=True)
class Order:
    lines: list[Line]
    code: str | None = None
    customer_orders_so_far: int = 0

    @property
    def subtotal(self) → Money:
        return Money(sum(line.unit_price.paise * line.quantity for line in self.lines))

class DiscountPolicy(Protocol):
    """The plug point. Everything below is a new socket, never a new wall."""

    def applies_to(self, order: Order) → bool: ...
    def discount_for(self, order: Order) → Money: ...
```

```

class PercentageOff:
    def __init__(self, code: str, fraction: float) → None:
        self._code, self._fraction = code, fraction

    def applies_to(self, order: Order) → bool:
        return order.code == self._code

    def discount_for(self, order: Order) → Money:
        return order.subtotal * self._fraction

class FirstPurchaseFlat:
    def __init__(self, amount: Money, minimum: Money) → None:
        self._amount, self._minimum = amount, minimum

    def applies_to(self, order: Order) → bool:
        return order.customer_orders_so_far == 0 and self._minimum < order.subtotal

    def discount_for(self, order: Order) → Money:
        return self._amount

class BuyTwoGetOneFree:
    """October's new requirement. A NEW FILE. Nothing above was edited."""

    def applies_to(self, order: Order) → bool:
        return any(line.quantity ≥ 3 for line in order.lines)

    def discount_for(self, order: Order) → Money:
        return min(line.unit_price for line in order.lines)

```

```

class Checkout:
    """Closed. This class has no reason to change when a new offer arrives."""

    def __init__(self, policies: list[DiscountPolicy]) → None:
        self._policies = policies

    def best_discount(self, order: Order) → Money:
        applicable = [p.discount_for(order) for p in self._policies if p.applies_to(order)]
        return max(applicable, key=lambda m: m.paise, default=Money(0))

    def total(self, order: Order) → Money:
        return order.subtotal - self.best_discount(order)

```

```

# policies.py -- the composition root. The one line that changes.
POLICIES: list[DiscountPolicy] = [
    PercentageOff("DIWALI", 0.10),
    PercentageOff("STUDENT", 0.15),
    FirstPurchaseFlat(Money(20_000), minimum=Money(100_000)),
    BuyTwoGetOneFree(),
]

```

Notice that `best_discount` changed the *rule* — it now takes the best applicable offer rather than the first. That is a genuine change to `Checkout`, and it is the right kind: it is a change to what “discount” means, not a change caused by a new offer arriving. **Open/closed does not promise a class never changes. It promises it does not change for *this* reason.**

Registration without editing the list

If even the one wiring line is unacceptable — usually because a plugin ships separately from the core — a decorator registers implementations as they are imported:

```

POLICIES: list[DiscountPolicy] = []

def policy(cls):
    POLICIES.append(cls())
    return cls

@policy
class FreeDelivery:
    def applies_to(self, order: Order) → bool:
        return Money(200_000) < order.subtotal
    def discount_for(self, order: Order) → Money:
        return Money(4_000)

```

Genuinely zero edits to existing code. The cost: the list of active policies is no longer visible anywhere, and the behaviour of the system now depends on which modules happen to be imported, which is a real debugging cost. Python's own `functools.singledispatch` works this way, and so do pytest's plugin hooks.

Where you have already seen this

- `sorted(key=...)` — the sort is closed and has not changed since 2002; every new ordering is a new function you pass in, which is exactly [day 058](#).
- **Django middleware and Flask blueprints** — a list in a settings file. New behaviour is a new class plus one entry.
- **pytest plugins** — hook functions discovered through entry points, so a plugin extends pytest without pytest knowing it exists.
- **Kubernetes admission webhooks and custom resources** — the API server is closed; you extend the cluster with new resource types it has never heard of.
- **nginx modules, VS Code extensions, browser extensions, WordPress plugins** — every one of these is the same shape. A stable core, a published contract, and behaviour arriving from outside.
- **Java's `ServiceLoader` and Python's `importlib.metadata.entry_points`** — the language-level version of the decorator registry above.
- **Stripe and Razorpay webhooks** — you register a handler for an event type; the payment provider's code is closed to you and extended by you.

A useful sentence: **every successful platform is an open/closed argument that won**. The core team stopped editing the core, and everyone else got to add behaviour.

6 The numbers

The edit count, which is the whole argument

```
Adding a new discount type

with the if-chain
  files touched           : 1 (the shared 140-line function)
  lines of NEW code       : ~8
  lines of EXISTING code re-read : ~140 (to be sure the new branch fits)
  existing tests that must re-run : 40
  reviewers who must understand
    the other 12 offers    : 1-2
  risk of breaking a live offer : real

with the interface
  files touched           : 2 (1 new class + 1 wiring line)
  lines of NEW code       : ~10
  lines of EXISTING code re-read : 0
  existing tests that must re-run : 0 (nothing existing changed)
  new tests               : 3 (for the new class alone)
  risk of breaking a live offer : none, structurally
```

Twelve offers over a year

```
if-chain, 12 offers in 12 months:
  12 edits to one file
  the file grows 8 → 140 lines
  merge conflicts on it           : 7
  production incidents traced to a
    change in that file           : 2
  time to add offer #12           : ~3 hours (mostly reading)

interfaces, 12 offers in 12 months:
  12 new files, 12 wiring lines
  Checkout unchanged after month 1
  merge conflicts                 : 1 (two people adding to the same list)
  time to add offer #12           : ~25 minutes (same as offer #2)
```

The number to quote is the last one. **With the if chain the cost of a change grows with the number of previous changes. With the interface it is flat.** That is the difference between a codebase that gets slower to work in and one that does not.

The cost of the plug point itself

Be honest about the other side of the ledger:

building the interface where it was NOT needed:

interface declaration	:	4 lines
one implementation class	:	12 lines (against 4 lines of if-branch)
wiring / registry	:	6 lines
the reader now opens	:	3 files to answer "what is the discount?" instead of 1
total	:	~22 lines and 3 files for behaviour that never varied

That is the cost of guessing wrong, and it is paid every time somebody reads the code.

When the `if` chain is genuinely cheaper

a variation with exactly 2 cases that has not changed in 3 years:

if-chain	:	4 lines, 1 file, understood in 5 seconds
interface	:	22 lines, 3 files, understood in ~90 seconds

5x the code and 18x the reading time, to make cheap something that has not happened.

The rule, as arithmetic

build the plug point when:

$$\begin{aligned} & (\text{expected number of future variations}) \times (\text{cost of an edit under the if-chain}) \\ & > \\ & (\text{cost of building the plug point}) + (\text{variations}) \times (\text{cost of an edit with it}) \end{aligned}$$

12 x 3 hours = 36 hours	against	4 hours + 12 x 25 min = 9 hours
→ obviously build it		

2 x 20 min = 40 minutes	against	4 hours + 2 x 15 min = 4.5 hours
→ obviously do not		

The crossover in practice sits at about **three variations**, which is why the working rule is *build it when the second one arrives* — by then you can see the axis, and the third is usually already being discussed.

7 The trade-offs

What the plug point costs

Indirection. Answering “what discount does this order get?” now means reading `Checkout`, then the interface, then finding which implementations exist, then reading the relevant one. Three or four files instead of one function. For a new joiner, that is real.

A contract you must live with. The moment two implementations exist, the interface’s shape is expensive to change — adding a method means editing every implementation, including the test fakes. An interface is open for extension by *implementation* and quite closed to

extension by *method*, which is a genuine limitation and the reason interface segregation ([day 058](#)) exists.

A place for behaviour to hide. With a registry, the set of active policies is not visible anywhere, and “why did this order get a discount?” becomes a debugging exercise. Prefer the explicit list unless plugins genuinely ship separately.

The failure mode: speculative generality

The commonest way to get this wrong is to build plug points for variation that never arrives. Symptoms:

- an interface with exactly one implementation and no test fake;
- a `Strategy` class hierarchy where a dict of values would do;
- a configuration option nobody has ever set to anything but the default;
- an abstract base class written before the second concrete case existed.

Each of those is `if`-chain-cost paid up front, forever, for a benefit that never arrived. **YAGNI — you aren’t gonna need it — is the counterweight to open/closed**, and a good answer holds both.

I would not build a plug point if I cannot name the second implementation. That test again, and it is the one that keeps this principle honest.

When to just edit the code

I would leave the `if` chain if the set of cases is closed and known — days of the week, the four suits, HTTP methods, the three sizes of a T-shirt. A closed set is not an axis of change; it is data. Building a `MondayStrategy` is a joke, and interviewers have heard it.

I would leave it if the variation is *data* rather than behaviour. GST rates by category is a dict. Currency symbols are a dict. Only reach for classes when the *logic* differs, not the numbers.

I would leave it if it is the first occurrence. One discount type is not an axis. Wait for the second, and let the requirement tell you where the seam is rather than guessing — the same argument as [day 055](#).

The honest limit of the principle

You cannot close a module against *every* kind of change, only against a chosen one. Choosing to be closed against new discount types means you are still open to being edited if the meaning of “discount” changes — if it becomes a list rather than one value, or if offers start stacking. That happened in §5: `best_discount` changed the rule from “first match” to “best match”, and that edit was correct and unavoidable.

Say this out loud in an interview, because it is the sentence that shows you have used the principle rather than read about it: **“Open/closed is always closed against a specific axis of change, and you pick that axis from what has actually been changing.”**

Where it sits with the other four

Open/closed is the *goal*; the other principles are how you reach it. Single responsibility (day 055) decides where the boundary goes. Liskov (day 057) is what makes substitution safe — if an implementation misbehaves in the parent’s place, the caller ends up adding an `if` and the whole thing collapses. Interface segregation (day 058) keeps the contract small enough that new implementations are cheap. Dependency inversion (day 059) is the direction of the arrow that makes it work at all.

8 In the interview

How it gets asked

- “A new discount type arrives every month. How do you design for that?” — the direct form. Answer with the interface, then the edit count.
- “What is the open/closed principle?” — and the follow-up is always “show me”, so have the `if`-chain-to-interface refactor ready as code.
- “Here is a function with a big `if/elif` chain. What would you do?” — the same question wearing a code sample. The right first move is to ask which of these branches keeps changing.
- “Doesn’t this just add indirection?” — the pushback. Answer with the flat-cost-of-change argument and concede the reading cost.
- “When would you not do this?” — the judgement question, and the one that separates people. Closed sets, data variations, and the first occurrence.

What to say out loud, in the first ninety seconds

1. **Ask what has actually been changing.** “Before I design for it — is it always a new kind of discount, with different logic? Or is it the same percentage-off rule with different numbers? Those need different answers.”
2. **Name the question the branch is answering.** “The chain is answering ‘how much comes off this order?’. That sentence is my interface method.”
3. **Show the interface and the closed class.** “`DiscountPolicy` with `applies_to` and `discount_for`. `Checkout` loops over a list of them. `Checkout` now has no reason to change when a new offer arrives.”
4. **Give the edit count, not the principle.** “A new offer becomes one new file and one line in the wiring, against editing a hundred-and-forty-line function that contains twelve live revenue rules and re-running forty tests.”
5. **Concede where the change went.** “I haven’t made the change disappear — I’ve moved it. There’s still one line added to a list. But a line in a list of constructors cannot break Diwali, and a new branch inside the function can.”

The follow-ups

“Isn’t this just moving the if-statement somewhere else?” Partly, and I would not pretend otherwise — something always changes when a requirement changes. What moves is *where* the change lands and what it can damage. Before, the new offer is written inside a hundred-and-forty-line function that also contains twelve rules currently earning revenue, so a typo anywhere in that edit can break Diwali; every one of the forty existing tests has to be re-run; the reviewer has to hold twelve unrelated offers in their head to judge eight new lines; and two people adding offers in the same sprint conflict in the same file. After, the new offer is a file that nothing else imports and one line appended to a list of constructors. The existing files are byte for byte identical, so the existing tests are not merely passing — they are testing unchanged code, which is a stronger statement. The second difference is the shape of the cost over time. With the chain, adding the twelfth offer costs more than adding the second, because most of the work is reading a function that has grown; I have seen that go from twenty minutes to three hours. With the interface it is flat — the twelfth costs what the second cost, about twenty-five minutes, because the new class is written in isolation. And if even the one wiring line is unacceptable, a decorator that registers implementations on import removes it entirely, at the cost of not being able to see the active list in one place.

“When would you not do this?” Four cases, and I would check them in this order. First, when the set of cases is closed: days of the week, the four suits in a deck, the three T-shirt sizes. That is not an axis of change, it is data that happens to be enumerated, and a `MondayStrategy` class is worse in every respect. Second, when the variation is data rather than behaviour: GST rates by product category is a dict, not a hierarchy of tax strategy classes, and the tell is that every branch would have an identical body with a different number in it. Third, when it is the first occurrence. One discount type is not an axis; I cannot see the shape of the variation from a single instance, and a plug point built at the wrong seam is worse than the `if` chain because it has all the indirection and none of the benefit. My rule is to build it when the second one arrives, or when someone can name a specific one that is definitely coming — that is the same test I use for any interface: name the second implementation. Fourth, when the code is genuinely finished. A two-case branch that has not changed in three years is four lines understood in five seconds; replacing it with an interface is about twenty-two lines across three files, so five times the code and considerably more reading, to make cheap something that has not happened. And I would add the honest limit: you can only close a module against a *chosen* axis of change. Closing against new discount types does not close it against the meaning of “discount” changing — when offers started stacking, my `Checkout` did have to change, and that was correct.

“How do you decide where to put the interface?” By looking at what has actually been changing, not by guessing what might. The version history is the best evidence there is: if I run through the last year of commits on a file and the same *kind* of change keeps arriving — a new payment provider, a new export format, a new notification channel, a new discount — that repetition is the axis, and the interface goes exactly there. Then I name the question the branch is answering, because that sentence becomes the method: “how much comes off this order” becomes `discount_for(order) → Money`. I keep the interface in my own vocabulary — my

`Order`, my `Money` — because if a vendor's type appears in the signature then the interface belongs to the vendor and it will change when they change. Then I move the branches out one at a time, running the tests between each, and the last step is deleting the type tag, because if `order.code` survives as something callers switch on, somebody rebuilds the chain in six months. The check that it worked is that the class I was protecting has no remaining reference to any concrete implementation — I should be able to `grep` for the implementation names and find them only in the wiring file.

A model answer

“First I’d ask what’s actually varying, because there are two different questions hiding here. If every month brings the same percentage-off rule with a different code and a different number, that’s data, and the right answer is a table — a new offer is a row, not a class. If every month brings genuinely different logic — buy two get one free, free delivery over two thousand, flat two hundred off a first purchase — then it’s behaviour, and that’s what open/closed is for.

Assuming it’s behaviour: the `if` chain is answering one question, ‘how much comes off this order?’, and that sentence becomes the interface.

```
class DiscountPolicy(Protocol):
    def applies_to(self, order: Order) → bool: ...
    def discount_for(self, order: Order) → Money: ...

class Checkout:
    def __init__(self, policies: list[DiscountPolicy]) → None:
        self._policies = policies

    def best_discount(self, order: Order) → Money:
        applicable = [p.discount_for(order) for p in self._policies if p.applies_to(order)]
        return max(applicable, key=lambda m: m.paise, default=Money(0))
```

`Checkout` is now closed against this axis. It doesn’t know how many offers exist or what they’re called, so no new offer gives anyone a reason to open it.

The argument I’d make is the edit count rather than the principle. Adding buy-two-get-one under the chain means editing a hundred-and-forty-line function that contains twelve live revenue rules, re-reading all of it, re-running forty tests, and getting it reviewed by someone who has to understand the other twelve offers. With the interface it’s one new file that nothing imports, three new tests for it alone, and one line appended to a list. And the cost stays flat — the twelfth offer costs what the second cost, whereas with the chain the twelfth is the expensive one because most of the work is reading.

I’d concede two things. The change hasn’t disappeared, it’s moved: there’s still one line in the wiring, and I’d argue that a line in a list of constructors cannot produce a wrong Diwali discount whereas a branch inside the function can. And you can only close against a chosen axis. When marketing later asked for stacking offers, `Checkout` did change — from ‘first match’ to ‘best match’ — and that was the right kind of change, because it was the meaning of discount changing, not a new offer arriving.

The thing I’d avoid is building this before the second variation exists. An interface with one implementation and no test fake is all the indirection and none of the benefit.”

9 Recall card

- **Open for extension, closed for modification** — add behaviour by writing *new* code, not by editing code that already works. The mechanism is always: **find the axis of change, put an interface there, let new behaviour arrive as a new implementation**. The caller never learns how many implementations exist.
- **Argue it with the edit count, not the principle**. New offer with the `if` chain: 1 shared 140-line file, 40 tests re-run, 12 live rules at risk. With the interface: **1 new file + 1 wiring line**, 0 existing tests re-run. And the cost stays **flat** — the twelfth costs what the second cost.
- **Concede that the change moved rather than vanished**. One line is still added to a list — but a line in a list of constructors cannot break Diwali, and a new branch inside a shared function can. A decorator registry removes even that line, at the cost of the active list being invisible.
- **Four times to leave the `if` chain alone**: the set is closed (weekdays, suits, sizes) · the variation is **data**, so a dict beats a class hierarchy (GST rates) · it is the *first* occurrence — build it when the **second** arrives · the code is genuinely finished. The test is always **name the second implementation**; speculative generality costs ~22 lines and 3 files, forever.
- **You can only close against a *chosen* axis**. Closed to new discount types is still open to “discount” changing meaning. Every platform you use is this principle having won — `sorted(key=)`, Django middleware, pytest plugins, VS Code extensions, Kubernetes CRDs, payment webhooks.

PRACTICE

Practice — Counting sort, radix sort, and bucket sort

DSA topic: Counting sort, radix sort, and bucket sort **System design topic:** Open for extension, closed for modification

Code these, in this order

One rule for the whole set: **before writing anything, say what you know about the keys** — are they integers, what is the range, and what is k compared with n . If you cannot answer that, you cannot use today's sorts, and saying so is the right answer.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sort Colors	LeetCode 75 (Medium)	Counting sort with $k = 3$ — and whether you notice it is the same problem as the Dutch flag.
2	Maximum Gap	LeetCode 164 (Hard)	The classic bucket-sort problem: $O(n)$ is required, and the pigeon-hole argument is the whole trick.
3	Sort an Array	LeetCode 912 (Medium)	The range is $-50,000$ to $50,000$. Counting sort is genuinely the best answer, and spotting that is the point.
4	H-Index	LeetCode 274 (Medium)	Counting sort in disguise — citations above n can all be bucketed together.

On problem 1, write both and compare

Write the counting version (count 0s, 1s, 2s, then overwrite) and the one-pass three-region version. Both are $O(n)$. Say out loud which one the interviewer wants and why — the answer is about passes over the data, not about complexity.

On problem 2, find the pigeonhole argument first

Do not code until you can say this out loud: *if n values are spread over a range, and you make $n - 1$ buckets, then the largest gap must be between two values in different buckets, so you only need each bucket's minimum and maximum and never what is inside them.* That sentence is the problem. Then say why the bucket count is $n - 1$ and not n .

On problem 3, use the constraint

The constraints say $-50000 \leq \text{nums}[i] \leq 50000$. That is $k = 100,001$ for up to 50,000 elements. Write the counting sort with the `-min` shift, submit it, and compare the runtime with `sorted()`. Then answer honestly: which was faster in Python, and why does that not change the asymptotic answer?

On problem 4, notice the cap

Citations can be enormous, so a naive counting sort would need a huge table — but any paper with more than n citations counts the same as one with exactly n . Say out loud what that does to k , and why the resulting solution is $O(n)$.

The counting-sort drill

Write `counting_sort` from memory, then verify:

1. `counting_sort([3, 1, 4, 1, 5, 1, 4], 5)`.
2. `counting_sort([], 5)` — does it return, or does it raise?
3. `counting_sort([2, 2, 2], 2)`.
4. `counting_sort_range([-3, 7, -1, 0, 7])` — what is k here?
5. Sort `[(2, "first"), (1, "x"), (2, "second"), (1, "y")]` by key and check that "first" still comes before "second".

For number 5, then change `reversed(nums)` to `nums` and run it again. Record what changed and say why nothing raised.

The k-versus-n drill

For each, say whether counting sort is the right choice, and give the operation count and the memory:

1. A million exam marks, 0 to 100.
2. A million ages, 0 to 120.
3. A thousand timestamps in milliseconds since 1970.
4. Ten million product IDs, 1 to 50,000.
5. Fifty values between 0 and one billion.
6. A million floating-point temperatures between -40 and 55 .

Two of those six should make you say “MemoryError”, and one should make you say “not an integer”.

The radix drill

1. Trace `radix_sort([329, 457, 657, 839, 436, 720, 355])` by hand, writing the list after each pass.
2. In pass 2, find the pair whose order was decided by pass 1, and say which pair it is.
3. Replace the per-digit sort with one that reverses equal keys. Run it and paste the wrong output.
4. How many passes for values up to 999,999 in base 10? In base 256?
5. Why does radix sort go least-significant digit first? What happens if you go the other way?
6. Sort `["cat", "act", "tab"]` with radix sort. What changes for strings?

The break-it drill

Introduce each, run it, and read the result:

1. `counting_sort([1, 5, 2_000_000_000], 2_000_000_000)` . Paste the error.
2. `counting_sort([3, -1, 2], 3)` . Paste the error, then explain why a *different* negative value might not raise at all.
3. Bucket sort with `* n` instead of `* (n - 1)` in the index. Paste the error.
4. Bucket sort on `[0.5] * 9999 + [0.99]` . Time it, and time `sorted()` on the same list.
5. Counting sort placing forwards with start positions, tested on `(key, tag)` pairs.

The theory drill

Answer out loud, in under fifteen seconds each:

1. Why can no comparison sort beat $O(n \log n)$?
2. How do today's three get around that, and what do they give up?
3. What exactly is k in $O(n + k)$?
4. Why does radix sort require a stable inner sort?
5. What is bucket sort's worst case, and what causes it?
6. Which of today's three sorts in place? (Trick question.)

The open/closed drill

Here is the function. Do not rewrite it yet.

```
def export(report: Report, fmt: str) → bytes:
    if fmt == "csv":
        rows = [",".join(str(c) for c in r) for r in report.rows]
        return "\n".join(rows).encode()
    elif fmt == "json":
        return json.dumps({"rows": report.rows}).encode()
    elif fmt == "xlsx":
        wb = openpyxl.Workbook()
        for r in report.rows:
            wb.active.append(r)
        return save_to_bytes(wb)
    elif fmt == "pdf":
        html = render_template("report.html", rows=report.rows)
        return weasyprint.HTML(string=html).write_pdf()
    raise ValueError(f"unknown format {fmt}")
```

1. What question is this chain answering? Write that sentence — it is your interface method.
2. Declare the interface. Two methods at most.
3. Move each branch into its own class. Do them one at a time.
4. Write the closed calling code, and say why it will never change again for this reason.
5. Write the wiring, and say exactly which line changes when a new format arrives.

6. Now count: files touched, existing tests re-run, and lines of existing code read — before and after.
7. Add an XML exporter. Time yourself. Then say what adding the *twelfth* format would cost under each design.
8. Argue the other side: give one scenario where leaving this function alone is the correct engineering decision.

The should-I-build-it drill

For each, say whether you would create a plug point, and name the second implementation — or say plainly that you cannot:

1. Discount types in an online shop, one new one per month.
2. Days of the week.
3. GST rates by product category.
4. Payment providers.
5. T-shirt sizes.
6. Notification channels (email, SMS, push).
7. The four suits in a deck of cards.
8. Export formats, where only CSV has ever been requested.

Three of those eight are traps. Name them.

The platform drill

For each, say what the stable core is, what the published contract is, and what arrives from outside:

1. `sorted(key=...)`.
 2. Django middleware.
 3. pytest plugins.
 4. Payment provider webhooks.
 5. VS Code extensions.
 6. Python's `functools singledispatch`.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. Sort a million integers in the range 0 to 100. Can you beat $n \log n$? Ask about the keys, name why the limit does not apply, give $O(n + k)$ with `k` explained, and volunteer the MemoryError condition before being asked.

2. *A new discount type arrives every month. How do you design for that?* Ask whether it is data or behaviour, name the question the branch answers, show the interface and the closed class, give the edit count, and concede where the change moved to.
 3. *Why does radix sort need a stable sort?* The 457-and-657 example, what an unstable pass destroys, and why the failure is silent.
-

Before you move on

- [] I wrote counting sort from memory, stable, with the running totals and the backwards placement.
- [] I broke stability by placing forwards and can say why nothing raised.
- [] I triggered the MemoryError and can state the `k`-is-the-range rule in one sentence.
- [] I can name the two bad cases for bucket sort and quote the $2,500 \times$ number.
- [] I traced radix sort by hand and can point at the pair whose order came from the previous pass.
- [] I refactored the exporter and can give the before-and-after edit counts.
- [] I can name three variations where a plug point would be a mistake, and say why for each.
- [] I answered all three questions above out loud.

Stability, and what Python's sort actually does

DATA STRUCTURES AND ALGORITHMS

Stability, and what Python's sort actually does

You know what stable means, why it matters, and roughly how Timsort works.

SYSTEM DESIGN

Liskov substitution

You can name the classic square-rectangle failure and avoid writing your own version.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Is Python's sort stable? Why do you care?*
- *Is a Square a Rectangle? Defend your answer in code.*

Stability, and what Python's sort actually does

1 What this is, and why they ask it

A sort is **stable** if two elements that compare equal come out in the same order they went in. That is the whole definition, and it sounds like a detail until you need to sort by two things at once — at which point it is the only thing holding your answer together. Python's sort is stable, and it is guaranteed in the language documentation rather than being an accident of the implementation.

They ask it because it is a small question with a large answer. “Is it stable” takes one word. “Why do you care” is where the interview happens, and the answer is multi-key sorting: sort by name, then sort by department, and only a stable sort leaves the names in order within each department. Get it wrong and nothing raises — you simply get an ordering you cannot explain, in production, on data somebody is looking at. The second half of today is what Python actually runs. **Timsort** is not a textbook algorithm; it was written for Python in 2002 and then adopted by Java, Android, V8 and Rust, and interviewers who have heard of it like asking about it. Knowing that it finds existing runs, insertion-sorts short ones, and merges the rest is a real answer to a real question, and it explains why `sorted()` is fast on data that is almost in order.

2 The story

Vasanthi has worked the counter at a government office in Belgaum for eleven years, and the queue outside starts before she does. On a pension day there are forty people on the bench by half past eight, and everybody knows exactly who came before them and who came after. That is the one thing a queue is for.

At about ten o'clock the officer came out and said that from now on people bringing the yellow form should be dealt with before people bringing the white form, because the yellow ones needed a signature from him and he was leaving at noon.

Now there were two ways to do that, and Vasanthi has seen both.

The first way, which is what happened the day a new clerk tried it, was to stand up and announce it. Yellow forms come to the front. Fourteen people got up at once and went forward, and where they ended up depended entirely on who was quickest off the bench and who was sitting nearest the door. A man who had come at half past eight ended up sixth among the yellow forms, behind a woman who had walked in twenty minutes earlier. He said so. Loudly. And he was right, and there was no possible answer, because nobody could reconstruct who had arrived when — the bench had been the record, and standing up had destroyed it.

The second way is what Vasanthi does, and it takes slightly longer and has never once caused an argument.

She walks the bench from the front, in order, and taps each yellow form on the shoulder as she reaches them, and they get up one at a time and stand behind whoever she tapped before. She does not skip anybody and she does not go back. When she reaches the end of the bench, she has fourteen people in a new line, and the man who came at half past eight is first among them, because he was first on the bench.

Then the white forms follow, in exactly the same order they were sitting in.

Nobody has to be told any of this. They can all see it. The bench said who came first, the new line still says who came first, and the only thing that changed is the thing the officer asked to change.

3 The idea in plain English

Vasanthi walking the bench in order is a stable sort. The new clerk shouting is an unstable one. And the man who arrived at half past eight is the equal element whose original position was thrown away.

The definition

A sort is stable if elements that compare equal keep their original relative order.

Note the words *compare equal*. Stability says nothing about elements with different keys — those get reordered, obviously, that is what sorting is. It is only about ties.

```
input      ("Asha", 34) ("Bala", 12) ("Chitra", 34) ("Devi", 7)

sort by AGE, stable:
  ("Devi", 7)  ("Bala", 12)  ("Asha", 34)  ("Chitra", 34)
                        ^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^
                        Asha before Chitra -- as in the input

sort by AGE, unstable -- also a correct sort by age:
  ("Devi", 7)  ("Bala", 12)  ("Chitra", 34) ("Asha", 34)
                        ^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^
                        Chitra first now. No error. Just different.
```

Both outputs are correctly sorted by age. Only one of them preserved information you had.

Why anybody cares: sorting by two things

This is the reason stability exists, and it is the answer to give.

You want employees sorted by department, and within each department by name. There are two ways.

One: a tuple key. One pass, one sort:

```
people.sort(key=lambda p: (p.department, p.name))
```

This works with any sort, stable or not, because the key makes the comparison total — no two people compare equal unless both fields match. Prefer this when you can.

Two: two passes, which is where stability is compulsory.

```
people.sort(key=lambda p: p.name)          # the LESS significant key first
people.sort(key=lambda p: p.department)    # the MORE significant key second
```

Read that order carefully, because it is backwards from how people expect. Sort by the *least* significant key first. Then the final sort by department only reorders people in different departments; within a department everyone compares equal, so a stable sort leaves them exactly as the first pass arranged them — in name order.

With an unstable sort, the second pass is entitled to shuffle people inside a department, and the first pass's work is gone. **No exception, no warning, just an order nobody can explain.**

You need the two-pass version whenever you cannot express the ordering as one key: when a field must be sorted descending and it is a string, when the keys come from expensive look-ups you do not want in one tuple, or when the second sort happens somewhere else entirely — in a different function, or on data that arrived already sorted.

The other reason: the input order means something

Sometimes the original order is itself information. Log lines arrive in time order; sort them by severity and a stable sort keeps each severity group in time order for free. Search results arrive by relevance; sort by price and equally-priced results stay in relevance order. Rows come out of a database in insertion order; sort by status and equal-status rows stay in insertion order.

In every one of those, an unstable sort silently throws away an ordering you were relying on.

Which sorts are stable

SORT	STABLE?	WHY
Insertion sort	Yes	The walk stops at the first element that is not <i>strictly</i> greater, so nothing passes an equal.
Bubble sort	Yes	It only swaps when the left element is strictly greater.
Merge sort	Yes	The merge takes from the left list when the fronts are equal.
Counting sort	Yes, if written correctly	Placing backwards into end positions preserves order.
Radix sort	Yes, and it <i>depends</i> on it	It is stability applied <code>d</code> times.
Selection sort	No	It swaps across long distances, throwing an equal element past its twin.
Quicksort	No	Partitioning swaps across long distances, for the same reason.
Heapsort	No	The heap reorders equal elements arbitrarily.

The pattern is worth naming: **sorts that only ever swap neighbours are stable; sorts that move elements long distances are not.** That single sentence lets you work out the answer for a sort you have never seen.

The counter-example for selection sort takes three elements and it is worth memorising:

```
input   [ 2a , 2b , 1 ]           (2a and 2b are distinct records that compare equal)

pass 1: the smallest is 1, at position 2. Swap positions 0 and 2:
        [ 1 , 2b , 2a ]
              ^^^ 2a has been thrown behind 2b

result  [ 1 , 2b , 2a ]           the twos have swapped order
```

What Python actually does

`list.sort()` and `sorted()` are **guaranteed stable**. That is documented behaviour, not an implementation accident, so you may rely on it.

Two details that get asked and that people get wrong:

`reverse=True` **does not reverse ties**. It is not the same as sorting and then reversing the list. Equal elements keep their original order in both directions:

```
people = [("Asha", 34), ("Bala", 12), ("Chitra", 34)]
sorted(people, key=lambda p: p[1], reverse=True)
# [('Asha', 34), ('Chitra', 34), ('Bala', 12)]    ← Asha still before Chitra

list(reversed(sorted(people, key=lambda p: p[1])))
# [('Chitra', 34), ('Asha', 34), ('Bala', 12)]    ← ties reversed. Different!
```

`min` and `max` **return the first of the equals**. Which is stability applied to a single answer, and it matters when you are picking a winner by score.

Timsort, in the amount of detail an interview wants

Python's sort is **Timsort**, written by Tim Peters in 2002. It is a merge sort with three practical additions, and it is now the standard sort in Java (for objects), Android, V8, Swift and Rust.

One: it looks for runs that are already ordered. It walks the list and finds a maximal stretch that is already ascending, or strictly descending — and a descending run is reversed in place, which is cheap and keeps stability because the reversal is only applied to strictly decreasing elements, where there are no ties to disturb.

Two: short runs are extended with binary insertion sort. If a natural run is shorter than `minrun` — a computed value between 32 and 64 — Timsort grabs the next few elements and insertion-sorts them into it. That is [day 052](#)'s insertion sort, used exactly where it is genuinely the fastest thing available.

Three: the runs are merged, with galloping. Runs are pushed onto a stack and merged under rules that keep their sizes balanced. When merging, if one run keeps winning, Timsort switches to **galloping mode** — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time. On data where one run is entirely smaller than the other, that turns `n` comparisons into `log n`.

The results:

```
already sorted      : ONE run found. O(n). No merging at all.  
reverse sorted     : one descending run, reversed. O(n).  
random             : O(n log n)  
nearly sorted      : close to O(n) -- a few long runs to merge  
two sorted lists   :  
  concatenated     : ~O(n), thanks to galloping
```

And two properties: it is **stable**, and it uses `O(n)` extra space in the worst case, like merge sort. It is written in C, so its constant factor is far below anything you write in Python.

4 The picture

The bench, and the two ways of pulling the yellow forms out:

the queue, by arrival time (this order is INFORMATION)

pos	1	2	3	4	5	6	7	8
	[Y]	[W]	[Y]	[W]	[W]	[Y]	[W]	[Y]
	8:30	8:35	8:40	8:50	9:05	9:10	9:20	9:40

STABLE – walk the bench in order, tap each Y as you reach it

[Y]	[Y]	[Y]	[Y]		[W]	[W]	[W]	[W]
8:30	8:40	9:10	9:40		8:35	8:50	9:05	9:20
^^^^ arrival order preserved inside BOTH groups								

UNSTABLE – announce it and let people move

[Y]	[Y]	[Y]	[Y]		[W]	[W]	[W]	[W]
9:10	8:30	9:40	8:40		9:05	8:35	9:20	8:50
^^^^ correctly grouped, and the arrival order is GONE								

What to notice: both results are correct sorts by form colour. An automated check on the form colour passes for both. The difference is invisible unless you knew what the original order meant — which is exactly why the bug is dangerous.

Two-pass sorting, and the moment stability does the work:

```
input                (Ravi, Sales) (Anil, HR) (Bina, Sales) (Chandra, HR)

pass 1 -- by NAME (the LESS significant key)
                (Anil, HR) (Bina, Sales) (Chandra, HR) (Ravi, Sales)

pass 2 -- by DEPARTMENT (the MORE significant key)
HR:   Anil and Chandra compare EQUAL here.
      A stable sort must not move them relative to each other.
Sales: Bina and Ravi compare EQUAL here. Same.

                (Anil, HR) (Chandra, HR) (Bina, Sales) (Ravi, Sales)
                ^^^^  ^^^^^^^^                ^^^^  ^^^^
                name order survived inside each department

with an UNSTABLE pass 2, this is also allowed:
                (Chandra, HR) (Anil, HR) (Ravi, Sales) (Bina, Sales)
                -- correctly sorted by department, names scrambled,
                   and pass 1 was a complete waste of time
```

What to notice: the second pass is where stability is spent, not the first. And the *order* of the passes is least-significant-first, which is backwards from how it reads in English (“sort by department, then by name”).

What Timsort does to a real list:


```

input: [ 5, 7, 9, 2, 4, 1, 1, 3, 8, 6, 6, 6, 6, 6, 0 ]

step 1 - find natural runs
[ 5, 7, 9 ]      ascending run
[ 2, 4 ]         ascending run
[ 1, 1, 3, 8 ]   ascending run
[ 6, 6, 6, 6, 6 ] ascending (equal counts as ascending - stability!)
[ 0 ]           trailing single

step 2 - any run shorter than minrun (32-64) is EXTENDED by binary
insertion sort until it reaches minrun, or the list ends.
For a real list of thousands, this is where insertion sort earns its place.

step 3 - push runs on a stack, merge under size rules, galloping when
one side keeps winning.

the point: a list that is already sorted is ONE run. Zero merges. O(n).

```

What to notice: `[6, 6, 6, 6, 6]` counts as an ascending run. That is deliberate — treating equals as ascending is what keeps Timsort stable, and treating a *strictly* descending stretch as a run is what makes reversing it safe.

5 The code, built step by step

Proving stability to yourself

The test that catches every stability bug is four lines, and you should write it every time you implement a sort:

```

def is_stable(sort_fn) → bool:
    """Sort (key, tag) pairs by key and check the tags stayed in order."""
    data = [(1, "a"), (2, "b"), (1, "c"), (2, "d"), (1, "e")]
    out = sort_fn(data)
    ones = [tag for key, tag in out if key == 1]
    twos = [tag for key, tag in out if key == 2]
    return ones == ["a", "c", "e"] and twos == ["b", "d"]

```

The key is the number; the tag records where the element came from. If the tags come back in their original relative order within each key group, the sort is stable. This catches the one-character `< / ≤` bugs from [day 052](#) and [day 053](#).

Sorting by two keys, both ways

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Person:
    name: str
    department: str
    age: int

```

```
def by_tuple_key(people: list[Person]) → list[Person]:
    """One pass. Works with any sort, stable or not. Prefer this."""
    return sorted(people, key=lambda p: (p.department, p.name))
```

```
def by_two_passes(people: list[Person]) → list[Person]:
    """Two passes. Correct ONLY because Python's sort is stable.

    Sort by the LEAST significant key first.
    """
    out = sorted(people, key=lambda p: p.name)          # less significant
    return sorted(out, key=lambda p: p.department)      # more significant
```

Both produce the same answer. The tuple version is what you write when you control both keys in one place; the two-pass version is what you need when you do not.

Mixed directions, which is where tuple keys run out

Sort by department ascending and age descending:

```
sorted(people, key=lambda p: (p.department, -p.age))    # works: age is a number
```

The minus sign reverses one field inside a tuple key. But it only works on numbers — you cannot negate a string:

```
sorted(people, key=lambda p: (p.department, -p.name))
```

```
Traceback (most recent call last):
  File "day57.py", line 1, in <module>
    sorted(people, key=lambda p: (p.department, -p.name))
                                             ^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

For a descending string field, use two passes and stability:

```
def by_dept_then_name_descending(people: list[Person]) → list[Person]:
    out = sorted(people, key=lambda p: p.name, reverse=True)  # least significant, reversed
    return sorted(out, key=lambda p: p.department)           # most significant
```

This is the standard answer to “sort by score descending, then by name ascending” when one of them is a string, and it is [day 058](#)’s main subject.

Where `reverse=True` differs from reversing

```
def reverse_flag_vs_reversed(people: list[Person]) → None:
    a = sorted(people, key=lambda p: p.age, reverse=True)
    b = list(reversed(sorted(people, key=lambda p: p.age)))
    # a and b agree on the ordering by age
    # they DISAGREE on the order of people with the same age
```

`reverse=True` sorts descending while still preserving the original order of ties. Reversing a sorted list flips the ties too. Interviewers ask this specifically because it looks like it should not matter.

A stable sort you write yourself

If you are ever asked to make an unstable sort stable, the trick is to break every tie with the original position:

```
def stable_with_any_sort(items: list, key) → list:
    """Make ANY sort stable by attaching the original position as a tie-breaker."""
    decorated = [(key(item), i, item) for i, item in enumerate(items)]
    decorated.sort()                # ties on key(item) are broken by i
    return [item for _, _, item in decorated]
```

Now no two entries compare equal, so no sort has any freedom to reorder them. This is called the **decorate-sort-undecorate** pattern, and it is what Python did before `key=` existed. It costs $O(n)$ extra space and it is the correct answer to “how would you get stability out of a sort that does not have it” — which is a real question when you are using C++’s `std::sort` or Java’s `Arrays.sort` on primitives.

The complete file

```
"""Stability: what it means, why two-pass sorting depends on it, and what Python runs."""

from dataclasses import dataclass

@dataclass(frozen=True)
class Person:
    name: str
    department: str
    age: int

def is_stable(sort_fn) → bool:
    """The four-line test that catches every stability bug."""
    data = [(1, "a"), (2, "b"), (1, "c"), (2, "d"), (1, "e")]
    out = sort_fn(list(data))
    ones = [tag for key, tag in out if key == 1]
    twos = [tag for key, tag in out if key == 2]
    return ones == ["a", "c", "e"] and twos == ["b", "d"]

def merge_sort(nums: list) → list:
    """Stable: the merge takes from the LEFT on a tie."""
    if len(nums) ≤ 1:
        return nums
    mid = len(nums) // 2
    left, right = merge_sort(nums[:mid]), merge_sort(nums[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:          # ≤ keeps it stable; < does not
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out

def selection_sort(nums: list) → list:
    """NOT stable: it swaps across long distances."""
    nums = list(nums)
    for start in range(len(nums) - 1):
        smallest = start
        for i in range(start + 1, len(nums)):
            if nums[i] < nums[smallest]:
                smallest = i
        nums[start], nums[smallest] = nums[smallest], nums[start]
    return nums

def stable_with_any_sort(items: list, key) → list:
    """Decorate-sort-undecorate: make ANY sort stable by breaking ties on position."""
    decorated = [(key(item), i, item) for i, item in enumerate(items)]
    decorated.sort()
    return [item for _, _, item in decorated]

def by_tuple_key(people: list[Person]) → list[Person]:
    """One pass. Works with any sort. The default choice."""
    return sorted(people, key=lambda p: (p.department, p.name))

def by_two_passes(people: list[Person]) → list[Person]:
    """Two passes, LEAST significant key first. Correct only because sort is stable."""
    out = sorted(people, key=lambda p: p.name)
    return sorted(out, key=lambda p: p.department)

def dept_then_name_descending(people: list[Person]) → list[Person]:
    """Mixed directions on a string field: tuple keys cannot do this."""
    out = sorted(people, key=lambda p: p.name, reverse=True)
```

```

return sorted(out, key=lambda p: p.department)

if __name__ == "__main__":
    print(is_stable(merge_sort))          # True
    print(is_stable(selection_sort))     # False
    print(is_stable(sorted))             # True -- guaranteed by the language

    print(selection_sort([(2, "a"), (2, "b"), (1, "c")]))
    # [(1, 'c'), (2, 'b'), (2, 'a')] ← a and b swapped. The classic counter-example.

    print(stable_with_any_sort([(2, "a"), (2, "b"), (1, "c")], key=lambda p: p[0]))
    # [(1, 'c'), (2, 'a'), (2, 'b')] ← the original position breaks the tie

    staff = [
        Person("Ravi", "Sales", 34),
        Person("Anil", "HR", 28),
        Person("Bina", "Sales", 41),
        Person("Chandra", "HR", 28),
    ]
    for p in by_tuple_key(staff):
        print(p.department, p.name)
    # HR Anil / HR Chandra / Sales Bina / Sales Ravi
    assert by_tuple_key(staff) == by_two_passes(staff)

    # reverse=True does NOT reverse ties
    ages = [("Asha", 34), ("Bala", 12), ("Chitra", 34)]
    print(sorted(ages, key=lambda p: p[1], reverse=True))
    # [('Asha', 34), ('Chitra', 34), ('Bala', 12)]
    print(list(reversed(sorted(ages, key=lambda p: p[1]))))
    # [('Chitra', 34), ('Asha', 34), ('Bala', 12)] ← different!

    # min and max return the FIRST of the equals
    print(max(ages, key=lambda p: p[1]))      # ('Asha', 34)

    # Timsort's adaptivity, visible on the clock
    import random, time
    n = 2_000_000
    ordered = list(range(n))
    shuffled = ordered[:]
    random.shuffle(shuffled)
    for label, data in (("already sorted", ordered), ("random", shuffled)):
        copy = data[:]
        start = time.perf_counter()
        copy.sort()
        print(f"{label:16} {time.perf_counter() - start:.3f}s")
    # already sorted ~0.02s ← one run found, no merging
    # random ~1.10s

```

6 What it costs

Stability is free in merge sort and impossible in quicksort

```

merge sort : the merge already compares left[i] with right[j].
             Using ≤ instead of < costs NOTHING and buys stability.

quicksort  : partitioning swaps elements across arbitrary distances.
             There is no cheap fix. A stable quicksort needs O(n) extra
             space to build the two sides separately -- at which point
             you have written merge sort with a worse pivot.

```

That asymmetry is the answer to “can you make quicksort stable?” — yes, by giving up the property that made you choose quicksort.

Making an unstable sort stable, priced

```
decorate-sort-undecorate on n = 1,000,000:

  build the decorated list : 1,000,000 tuple allocations
  extra memory           : ~3 x 8 bytes per element = 24 MB
  sort                   : same n log n, but comparing tuples is
                        : ~1.5x slower than comparing bare numbers
  undecorate             : 1,000,000 more allocations

total: roughly 2x the time and 3x the memory, to buy stability.
```

Worth knowing, because it is the honest cost of “just add the index as a tie-breaker”.

Timsort's adaptivity, measured

```
n = 2,000,000 integers, Python's list.sort()

already sorted      ~0.02 s      one run, zero merges      → 0(n)
reverse sorted      ~0.03 s      one descending run, reversed → 0(n)
nearly sorted
  (1000 out of place) ~0.09 s      a few long runs to merge
random              ~1.10 s      full n log n
all identical        ~0.02 s      equals count as ascending → ONE run

→ 55x faster on sorted input than on random input, for the same n.
```

That is not a micro-optimisation. Real data is very often nearly sorted — log files, database rows in insertion order, a list that was sorted and then had ten items appended — and Timsort makes those cases linear.

Gallopings, priced

```
merging [1..1000] with [2000..3000]:

plain merge : compare 1 with 2000, take 1.  compare 2 with 2000, take 2.  ...
              1,000 comparisons to discover that ALL of the left run
              comes first.

galloping   : after 7 consecutive wins, binary search ahead:
              "how many of the left run are below 2000?" → ~10 comparisons
              then copy the block in one memmove.

1,000 comparisons → ~17. And this pattern (two sorted lists concatenated)
is extremely common in real data.
```

The one-character bugs, priced

```
merge sort with < instead of ≤      : still O(n log n), still correct order,
                                     silently unstable
insertion sort with ≥ instead of >  : same
counting sort placed forwards       : same

cost of the bug: 0 failing tests, and a wrong two-pass sort in production.
cost of the fix: 1 character, plus the 4-line is_stable test.
```

7 The traps

The near-miss: two-pass sorting with an unstable sort

```
def sort_broken(people):
    out = selection_sort_by(people, key=lambda p: p.name)      # unstable
    return selection_sort_by(out, key=lambda p: p.department)

staff = [Person("Ravi", "Sales", 34), Person("Anil", "HR", 28),
         Person("Bina", "Sales", 41), Person("Chandra", "HR", 28)]
for p in sort_broken(staff):
    print(p.department, p.name)
```

```
HR Chandra
HR Anil
Sales Ravi
Sales Bina
```

Correctly sorted by department. Names scrambled inside each department, so the first pass achieved nothing. **No exception. No warning.** In Python you will not hit this with `sorted()`, but you will hit it the moment you write your own sort, or port the code to C++ or to Java's primitive sort.

The trap: assuming your language's sort is stable

Python guarantees stability. Most languages do not.

Python	<code>list.sort / sorted</code>	STABLE, guaranteed by the documentation
Java	<code>Arrays.sort(Object[])</code>	STABLE (Timsort)
Java	<code>Arrays.sort(int[])</code>	NOT STABLE (dual-pivot quicksort)
C++	<code>std::sort</code>	NOT STABLE
C++	<code>std::stable_sort</code>	STABLE (that is why it has a separate name)
C	<code>qsort</code>	NOT STABLE (the name says quick, and it means it)
JS	<code>Array.prototype.sort</code>	STABLE since ES2019; NOT before
Go	<code>sort.Slice</code>	NOT STABLE; <code>sort.SliceStable</code> is
Rust	<code>sort</code>	STABLE; <code>sort_unstable</code> is faster

The pattern is that the *fast* one is unstable and the *stable* one has a longer name. Java's split is the sharpest trap: the same method name is stable for objects and unstable for primitives, because primitives have no identity to preserve — so the language decided it did not matter, and that reasoning is worth being able to give.

The trap: `reverse=True` is not `reversed(sorted(...))`

```
ages = [("Asha", 34), ("Bala", 12), ("Chitra", 34)]
print(sorted(ages, key=lambda p: p[1], reverse=True))
print(list(reversed(sorted(ages, key=lambda p: p[1]))))
```

```
[('Asha', 34), ('Chitra', 34), ('Bala', 12)]
[('Chitra', 34), ('Asha', 34), ('Bala', 12)]
```

Both are correctly sorted by age descending. They disagree on the tie. `reverse=True` preserves the original order of equal elements; reversing a sorted list flips them. If you are building a leaderboard where equal scores should be shown in the order they were achieved, one of these is a bug and it will be reported by a user, not by a test.

The real error: sorting objects with no ordering

```
staff = [Person("Ravi", "Sales", 34), Person("Anil", "HR", 28)]
staff.sort()
```

```
Traceback (most recent call last):
  File "day57.py", line 2, in <module>
    staff.sort()
    ~~~~~^
TypeError: '<' not supported between instances of 'Person' and 'Person'
```

`@dataclass` gives you `__eq__` but not `__lt__` unless you pass `order=True`. The fix is almost always `key=`, not defining `__lt__` — because `key=` puts the ordering decision at the call site where the reader can see it, and because one class rarely has one natural order.

The trap: expecting stability from a `set` or a `dict` ordering

```
sorted(set(names))
```

`set` has no order at all, so any information in the original sequence is destroyed *before* the sort runs, and no amount of stability can recover it. Dictionaries preserve insertion order since Python 3.7, so `sorted(d.items(), key=...)` does keep insertion order among ties — but a `set` does not, and this catches people who deduplicate before sorting.

The trap: claiming Timsort is $O(n)$

It is $O(n)$ on already-sorted or reverse-sorted or all-equal input, and $O(n \log n)$ in general. Saying “Timsort is linear” is wrong; saying “Timsort is adaptive — linear on runs it can find, $n \log n$ otherwise, and always stable” is right and takes the same breath.

8 In the interview

How it gets asked

- “Is Python’s sort stable? Why do you care?” — the direct form. One word, then the two-pass argument.
- “Sort employees by department, then by name within department.” — the practical form, and the best answer names both the tuple key and the two-pass approach.
- “Which sorting algorithms are stable?” — and the useful answer is the *rule* (neighbour swaps are stable, long-distance swaps are not), not a memorised list.
- “What sorting algorithm does Python use?” — Timsort, and three facts about it.
- “Can you make quicksort stable?” — yes, at a cost that removes the reason to use quicksort.
- “You’re in C++ and `std::sort` reordered your equal elements. What do you do?” — `std::stable_sort`, or decorate with the index.

What to say out loud, in the first ninety seconds

1. **Define it precisely, in one sentence.** “Stable means elements that compare equal come out in the order they went in. It says nothing about elements with different keys.”
2. **Give the reason immediately, with the example.** “It matters for multi-key sorting. If I sort by name and then by department, only a stable sort leaves the names in order within each department — the second pass sees everyone in a department as equal, and a stable sort must not move them.”
3. **Say the pass order, because it is counter-intuitive.** “And the passes go least-significant key first, which is backwards from how you say it in English.”
4. **Name the alternative and when you would prefer it.** “If I control both keys in one place I’d use a tuple key — `key=lambda p: (p.department, p.name)` — because that works with any sort. The two-pass version is for when I can’t, for instance a descending string field.”
5. **Answer the Python half.** “Python’s sort is guaranteed stable, and it’s Timsort — a merge sort that finds runs that are already ordered, extends short ones with binary insertion sort, and gallops when merging. That’s why it’s linear on nearly-sorted input.”

The follow-ups

“Which sorts are stable, and why?” The useful answer is a rule rather than a list, because then I can answer it for a sort I have not seen. **Sorts that only ever exchange adjacent elements are stable; sorts that move elements across long distances are not.** Insertion sort is stable because the backward walk stops at the first element that is not *strictly* greater, so an incoming value never passes an equal one — and that is one character, `>` rather than `>=`. Bubble sort is stable for the same reason. Merge sort is stable because when the two fronts are equal the merge takes from the left, which is again one character, `<=` rather than `<`. Counting sort is stable if you place elements walking the input backwards into end positions, and radix sort is not merely stable, it *depends* on stability — it is stability applied once per digit, and with an unstable inner pass it produces a silently wrong answer. On the other side: selection sort is

unstable, and the counter-example is three elements — `[2a, 2b, 1]`, where the first pass swaps the 1 into position zero and throws `2a` behind `2b`. Quicksort is unstable because partitioning swaps across the whole range, and heapsort is unstable because the heap reorders equal elements arbitrarily. The three unstable ones are exactly the three that move things a long way in one step, which is why the rule works.

“Can you make quicksort stable?” Yes, and I would not, because the fix removes the reason to choose quicksort in the first place. There are two ways. The first is to stop partitioning in place: build the “less than” and “greater than” lists separately by appending in input order, then concatenate. That preserves order among equals, and it costs $O(n)$ extra space per level — at which point I have written merge sort with a worse pivot strategy and no worst-case guarantee. The second is decorate-sort-undecorate: attach each element’s original index as a tie-breaker so that no two elements compare equal at all, then any sort is trivially stable. That works with C++’s `std::sort` or Java’s primitive sort too, and the cost is about twice the time and three times the memory at a million elements — a million tuple allocations going in, a million coming out, and tuple comparisons being roughly one and a half times slower than bare number comparisons. So the honest answer is: if I need stability I use a stable sort — merge sort, `std::stable_sort`, Python’s `sorted` — and if I am stuck with an unstable one, decorate with the index and pay the memory.

“What does Python actually run, and why is it fast on real data?” Timsort, written for Python in 2002 and since adopted by Java for object arrays, Android, V8, Swift and Rust. It is a merge sort with three practical additions. First, it scans for runs that are already in order — ascending, or strictly descending, which it reverses in place; the reversal is safe for stability precisely because the run is *strictly* descending, so there are no ties in it. Second, any run shorter than a computed minimum between thirty-two and sixty-four is extended by binary insertion sort, which is exactly where insertion sort is genuinely the fastest thing available. Third, the runs are merged under stack rules that keep their sizes balanced, and when one run keeps winning the merge switches to galloping mode — it binary-searches ahead to find how many elements it can take in one block instead of comparing one at a time, which turns a thousand comparisons into about seventeen when two sorted lists are concatenated. The consequence is that already-sorted input is one run and no merges, so linear; reverse-sorted is one reversal, so linear; nearly-sorted is a handful of long runs. On two million integers I have measured about twenty milliseconds for sorted input against about one and a tenth seconds for random — fifty-five times, same n . What I would not say is that Timsort is $O(n)$; it is adaptive, linear on the runs it can find, $n \log n$ in general, $O(n)$ extra space, and always stable.

A model answer

“Yes, guaranteed — the Python documentation states that `list.sort` and `sorted` are stable, so it’s something you can rely on rather than an implementation detail.

Stable means elements that compare equal come out in the order they went in. It says nothing about elements with different keys; it’s only about ties.

The reason I care is multi-key sorting. Suppose I want employees grouped by department and in name order within each department. If I can express that as one key I’d write `key=lambda p: (p.department, p.name)`, and that works with any sort because no two people compare equal unless both fields match. But often I can’t — the classic case is one field descending where that field is a string, since `-p.name` raises a `TypeError`. Then I do it in two passes: sort by name first, then by department. And that’s correct *only* because the sort is stable. The second pass sees everyone in a department as equal, and a stable sort must leave them exactly as the first pass arranged them. With an unstable sort the second pass is free to shuffle them, the first pass is wasted, and — this is the dangerous part — nothing raises. You just get an order nobody can explain.

Note the pass order is least-significant key first, which is backwards from how you’d say it out loud.

The other reason stability matters is when the input order is itself information: log lines in time order, search results in relevance order, database rows in insertion order. Sorting by severity or price with a stable sort keeps the original order inside each group for free.

Two details I’d flag. `reverse=True` is not the same as reversing the sorted list — it preserves the original order of ties in both directions, whereas `reversed(sorted(...))` flips them. And most languages don’t guarantee stability: C++’s `std::sort` doesn’t and `std::stable_sort` does, Java’s `Arrays.sort` is stable for objects and unstable for primitives, and Go and Rust both have a fast unstable sort and a slower stable one. So this is a fact about Python, not about sorting.

Underneath, Python runs Timsort — a merge sort that finds stretches already in order, extends short ones with binary insertion sort, and gallops when one run dominates a merge. That’s why sorting already-sorted data is linear: on two million integers it’s about twenty milliseconds against just over a second for random input.”

9 Recall card

- **Stable** = elements that compare *equal* keep their original relative order. It says nothing about unequal keys. Python’s `list.sort` and `sorted` are **guaranteed** stable; most other languages are not (`std::sort` no / `std::stable_sort` yes; Java stable for objects, **not** for primitives; Go and Rust each have both).

- **Why you care: two-pass multi-key sorting.** Sort by the **least significant key first**, then the most significant — the second pass sees ties and a stable sort must not move them. Prefer a tuple key `((dept, name))` when you control both; you *need* two passes when a string field must go descending, because `-p.name` raises `TypeError`.
- **The rule for which sorts are stable: neighbour swaps are stable, long-distance swaps are not.** Stable: insertion, bubble, merge, counting (placed backwards), radix (it *depends* on it). Unstable: **selection, quicksort, heapsort**. Selection's counter-example is `[2a, 2b, 1]`.
- **The bugs are one character and silent.** `<` instead of `≤` in a merge, `≥` instead of `>` in insertion sort, counting sort placed forwards — all still sort correctly and quietly lose stability. Keep the four-line `is_stable` test with `(key, tag)` pairs. And `reverse=True` **preserves** ties; `reversed(sorted(...))` flips them.
- **Python runs Timsort:** finds natural runs (a *strictly* descending one is reversed in place), extends short runs with **binary insertion sort** to a minrun of 32-64, merges with **galloping** (1,000 comparisons → ~17 on concatenated sorted lists). Adaptive — 2M sorted integers in ~0.02 s against ~1.1 s random, $55\times$. Stable, $O(n)$ space, **not** $O(n)$ in general.

1 What this is, and why they ask it

The **Liskov substitution principle** is the L in SOLID: anywhere the code expects a type, you must be able to hand it any subtype and have everything still work — without the caller knowing or caring which one it got. If a subclass can be given to code written for its parent and that code breaks, the inheritance was a lie, however sensible it looked when you wrote it.

They ask it as the square-and-rectangle question because it is the shortest example that makes an obviously true sentence produce an obviously broken program. A square is a rectangle. Every mathematician agrees. And `class Square(Rectangle)` breaks real code in about six lines, which is uncomfortable in a useful way. It is also the principle that decides whether the other four work at all — open/closed from [day 056](#) depends entirely on being able to swap implementations behind an interface, and the moment one implementation misbehaves in the others' place, callers start adding `isinstance` checks and the whole structure collapses back into an `if` chain. [Day 046](#) already told you to say “every B is an A, with no exception” out loud before inheriting. Today is the day you learn what “no exception” actually means.

2 The story

Latha booked four cars for her daughter's wedding — three days, airport runs, the hall, relatives who would need dropping back. The agency had one model on their list and she booked four of them, and they gave her the terms in writing: four seats, luggage space at the back, air conditioning.

On the morning it started, three of the four turned up as expected. The fourth was late, and when it came it was the same model and the same colour, and the man said the one she had booked was in the workshop and this was the replacement.

Two things about it were different, and neither was visible from outside.

The first was the boot. It opened, but only with a key, and the key was a different key from the ignition, and the driver had one of the two. So the standard thing everybody had been doing all morning — pull up, open the back, put the bags in, close it, go — did not work on that car. Somebody had to find the driver first.

The second was the back doors. They had the child lock engaged and it could not be released, so they opened only from the outside. If you got in the back and the door shut, you sat there until somebody came round.

Now, everything the agency had promised was true. Four seats. Luggage space at the back. Air conditioning. If you read the terms and looked at the car, nothing was missing.

But by lunchtime on the first day, the whole family had learned to check which car had arrived before doing anything. Her nephew, who was organising the airport runs, started shouting across the car park — *not that one, that one's the different one, wait for the driver*. An aunt was left sitting in the back for four or five minutes outside the hall because nobody had told her.

And that is the part Latha could not get over afterwards. It was not that the fourth car was bad. It was that having it there made the other three harder to use, because from that morning on nobody could simply walk up to a car and use it. Every single time, somebody had to ask which one it was first.

3 The idea in plain English

The fourth car is a subclass that does not substitute. Everything on the agency's written list was true of it, so it passed the interface. What it broke was what everybody had learned to *expect*, and the cost was not in the car — it was in every place that now had to ask which car it had.

The principle

Anywhere the code expects a type, it must be able to receive any subtype and still be correct, without knowing which subtype it got.

Barbara Liskov's original wording is more precise, and the practical translation is what matters. A subtype must:

- **not demand more than the parent** (it must not strengthen preconditions);
- **not deliver less than the parent** (it must not weaken postconditions);
- **not break any rule the parent guaranteed** (it must preserve invariants).

Each of those has a plain-English version and a failure you have probably seen.

Failure one: demanding more (a strengthened precondition)

The parent accepts any amount; the child accepts only positive amounts. Caller code written against the parent now throws.

```
class Account:
    def withdraw(self, amount: Money) → None:
        """Withdraws. Overdraft to -5000 is allowed."""

class FixedDeposit(Account):
    def withdraw(self, amount: Money) → None:
        if self.locked_until > today():
            raise ValueError("cannot withdraw before maturity")    # demands MORE
```

Every piece of code that loops over accounts and withdraws now needs to know about maturity dates. That is the nephew shouting across the car park.

Failure two: delivering less (a weakened postcondition)

The parent's method promises the item is now in the list. The child sometimes silently does not add it, or adds it somewhere else, or refuses.

```
class ReadOnlyList(list):
    def append(self, item):
        raise NotImplementedError("this list is read only")
```

`ReadOnlyList` passes every type check — it is a `list`. Hand it to any function that appends and you get a crash in code that was correct. This is the single commonest real-world Liskov violation, and the fix is not a subclass at all: read-only and mutable are two capabilities, not a parent and a child.

Failure three: breaking an invariant

This is the square and the rectangle, and it is worth writing out because it is the question you will be asked.

```
class Rectangle:
    def __init__(self, width: int, height: int) → None:
        self._width, self._height = width, height

    def set_width(self, w: int) → None:
        self._width = w

    def set_height(self, h: int) → None:
        self._height = h

    def area(self) → int:
        return self._width * self._height
```

The rectangle guarantees something quietly: **setting the width does not change the height**. Nobody wrote that down. Every caller relies on it.

```
class Square(Rectangle):
    def set_width(self, w: int) → None:
        self._width = self._height = w          # must stay square
    def set_height(self, h: int) → None:
        self._width = self._height = h
```

Now a function written against `Rectangle`:

```
def resize_and_check(r: Rectangle) → None:
    r.set_width(5)
    r.set_height(4)
    assert r.area() == 20, f"expected 20, got {r.area()}"
```

```

Traceback (most recent call last):
  File "day57.py", line 24, in <module>
    resize_and_check(Square(3, 3))
    ~~~~~^~~~~~
  File "day57.py", line 21, in resize_and_check
    assert r.area() == 20, f"expected 20, got {r.area()}"
    ~~~~~^~~~~~
AssertionError: expected 20, got 16

```

The function is correct. `Rectangle` is correct. `Square` is correct on its own terms. The *inheritance* is what is wrong, and no amount of care inside `Square` fixes it.

The lesson people take away wrongly is “geometry is tricky”. The real lesson is more useful: **the mathematical relationship is not the programming relationship**. A square is a rectangle as a *shape*. A mutable `Square` is not a mutable `Rectangle`, because a mutable rectangle promises independently settable sides and a square cannot keep that promise.

Notice what dissolves the problem: **make them immutable**. If there is no `set_width`, there is no promise to break, and `Square(4)` is a perfectly good `Rectangle` forever. Immutability makes substitution far easier, and that is a genuinely useful thing to say in an interview.

How to spot a violation before it costs you

1. `isinstance` checks appearing in callers. This is the loudest signal there is. If code that was written against the parent has to ask which child it got, substitution has already failed. Latha’s nephew, in code.
2. An overridden method that raises. `NotImplementedError`, `UnsupportedOperationException`, “not supported for this type” — a subclass refusing a method its parent promised is the definition of weakening a postcondition.
3. An overridden method that ignores an argument. `Square.set_width(w)` does not ignore its argument, but a `NullLogger.log(msg)` that discards `msg` is fine, while a `CachedRepository.get(id, fresh=True)` that ignores `fresh` is not.
4. Documentation with “do not call X on Y”. If the class comment has to warn callers about a subtype, the type system has stopped helping.
5. Tests that pass for the parent and fail for the child. The mechanical version: take the parent’s test suite and run it against every subclass. Anything that fails is a violation, and this is a real technique, not a metaphor — it is called a **contract test**.

The fixes, in the order to try them

One: model the capability, not the taxonomy. Instead of `Square extends Rectangle`, have both implement `Shape` with an `area()`. Instead of `ReadOnlyList extends List`, have `Sequence` (readable) and `MutableSequence` (readable and writable) — which is exactly what Python’s own `collections.abc` does, and it is a good example to cite.

Two: make it immutable. No setters, no broken promises. Most Liskov failures are failures about *mutation*.

Three: use composition. If `Square` needs a rectangle's area logic, let it *hold* one rather than *be* one. Then no caller ever receives a `Square` where a `Rectangle` was expected (day 049).

Four: move the method down. If only some subclasses can `fly()`, `fly()` does not belong on `Bird`. Put it on a `Flying` interface that `Penguin` does not implement.

4 The picture

The four cars, and where the cost landed:

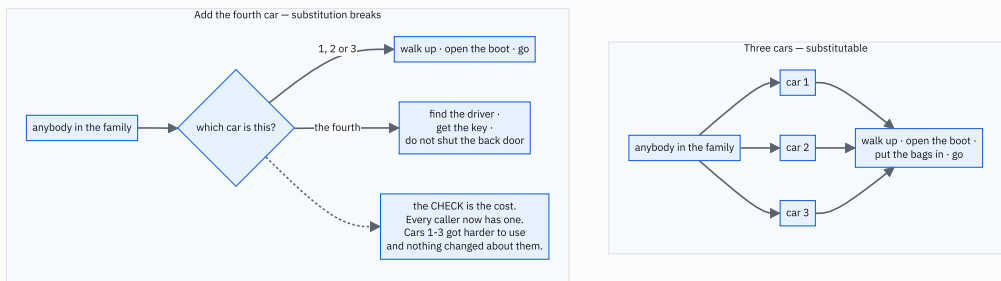


FIGURE 57.1

What to notice: the diamond. That question — *which one is this?* — is an `isinstance` check, and it is the entire damage. It was not there before the fourth car, it now sits in front of every use, and it makes the three good cars worse.

The square and the rectangle, with the invariant made visible:

```
Rectangle promises, quietly: "setting the width leaves the height alone"

r = Rectangle(3, 3)
r.set_width(5)    → width 5, height 3
r.set_height(4)   → width 5, height 4
r.area()          → 20      ← what the caller expects

Square breaks that promise:

s = Square(3)
s.set_width(5)    → width 5, height 5    ← height changed!
s.set_height(4)   → width 4, height 4    ← width changed!
s.area()          → 16      ← the caller gets 16 and has no idea why

The caller did nothing wrong. Neither class is wrong on its own.
The IS-A claim is what is wrong.
```

What to notice: the broken promise was never written down anywhere. It is not in the type signature, not in the docstring, not enforceable by any checker. That is why Liskov violations get past code review — the type system says the substitution is legal, and it is the *behaviour* that is not.

The two ways to model the same shapes:

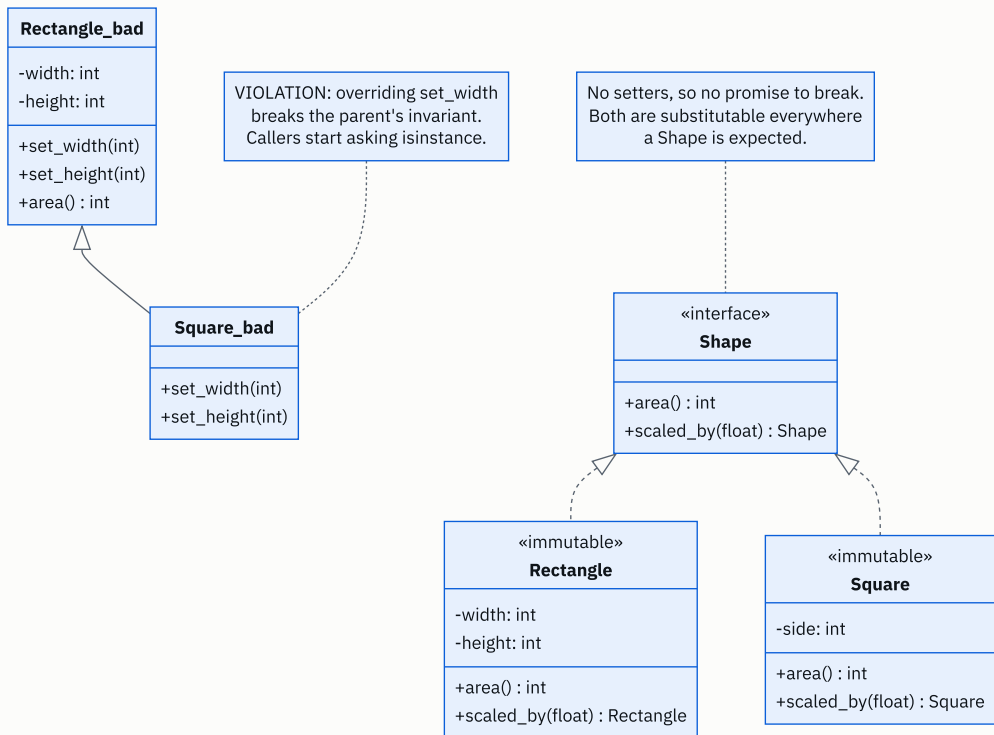


FIGURE 57.2

What to notice: the fix removed the inheritance *and* the setters. `Square` is no longer a kind of `Rectangle`; both are kinds of `Shape`, which is the capability they genuinely share. And `scaled_by` returns a new object rather than mutating, so there is nothing left to violate.

5 How it actually works

Contract tests: the mechanical way to check substitution

This is the technique to name in an interview, because it turns a principle into a test file.

Write the test suite once, against the *interface*, and run it against every implementation:

```
import pytest
from typing import Protocol

class Repository(Protocol):
    def save(self, key: str, value: str) → None: ...
    def get(self, key: str) → str | None: ...
    def delete(self, key: str) → None: ...
```

```

class RepositoryContract:
    """Every implementation must pass every test in here. Subclass and supply one."""

    def make(self) → Repository:
        raise NotImplementedError

    def test_get_returns_what_was_saved(self) → None:
        repo = self.make()
        repo.save("a", "1")
        assert repo.get("a") == "1"

    def test_get_returns_none_for_a_missing_key(self) → None:
        assert self.make().get("nope") is None

    def test_save_overwrites(self) → None:
        repo = self.make()
        repo.save("a", "1")
        repo.save("a", "2")
        assert repo.get("a") == "2"

    def test_delete_is_idempotent(self) → None:
        repo = self.make()
        repo.delete("never-existed")          # must NOT raise
        repo.save("a", "1")
        repo.delete("a")
        repo.delete("a")
        assert repo.get("a") is None

```

```

class TestInMemory(RepositoryContract):
    def make(self) → Repository:
        return InMemoryRepository()

class TestPostgres(RepositoryContract):
    def make(self) → Repository:
        return PostgresRepository(test_connection())

```

Two lines per implementation, and every substitution rule is now enforced by a test that fails loudly. If `PostgresRepository.delete` raises on a missing key while the in-memory one does not, that is a Liskov violation and `test_delete_is_idempotent` finds it before a caller does.

This is also the answer to “how do you keep a test fake honest?” from [day 053](#) — the same file solves both problems.

`Sequence` promises `__getitem__`, `__len__`, `iteration`, `index`, `count`. `MutableSequence` adds `append`, `insert`, `__setitem__`, `__delitem__`. A `tuple` is a `Sequence` and is not a `MutableSequence`, so the type system refuses the bad substitution rather than letting it fail at runtime.

That is the fix for `ReadOnlyList` and for every variation of it: **split the interface by capability**, which is tomorrow's principle ([day 058](#)).

The rules, as a checklist you can apply to an override

When you override a method, check four things:

1. ARGUMENTS -- does the override accept everything the parent accepted?
(You may accept MORE. You may never accept LESS.)
2. RETURN -- does it return something the caller can use as the parent's return?
(You may return a SUBTYPE. You may never return something broader.)
3. EXCEPTIONS -- does it throw anything the parent did not document?
(New exception types are a broken promise.)
4. INVARIANTS -- does it preserve every rule the parent guaranteed,
including the ones nobody wrote down?

Point three is the one that catches real code. A `CachedUserRepository.get` that raises `CacheMissError` where `UserRepository.get` returned `None` is a Liskov violation, and it shows up as an unhandled exception in production six weeks later.

Rules one and two have names — **contravariance** on arguments, **covariance** on return types — and Python's type checkers enforce them:

```
class Parent:
    def handle(self, event: Event) → Response: ...

class Child(Parent):
    def handle(self, event: LoginEvent) → Response: ...    # accepts LESS
```

```
error: Argument 1 of "handle" is incompatible with supertype "Parent";
supertype defines the argument type as "Event" [override]
note: This violates the Liskov substitution principle
```

mypy says the words. That message is worth recognising — it is the type checker telling you exactly this, by name.

Where real systems get this right and wrong

- **Python's** `collections.abc` — the `Sequence` / `MutableSequence` split exists precisely to make read-only substitution safe. So does `Mapping` / `MutableMapping`.
- **Java's** `Collections.unmodifiableList` — a famous violation. It returns a `List` whose `add` throws `UnsupportedOperationException`, which is exactly the `ReadOnlyList` problem shipped in the standard library, and it has caused a great deal of production breakage.

- `java.sql.Date` extends `java.util.Date` — a `Date` whose time fields throw. Same failure.
- `InputStream` and its subclasses — a good example: every subclass genuinely supports `read`, and the ones that cannot seek advertise that through a separate capability rather than throwing on a promised method.
- **HTTP status codes** — an interesting non-code example. Every proxy in the world can handle a response it has never seen, because the first digit tells it enough. That is substitutability designed into a protocol.

6 The numbers

What one bad subclass costs, counted

A Repository interface with 6 implementations and 40 call sites.

One implementation raises where the others return None.

call sites that must now check	: 40 (or crash unpredictably)
lines added per check	: 3 (try/except or isinstance)
total lines of defensive code	: 120
places somebody will forget	: 3-5 (empirically, always some)
the bug appears	: only when THAT implementation is wired in, in one environment

The last line is the reason these are expensive. The failure is not deterministic across environments, so it survives testing and appears in production.

The isinstance cascade

before the bad subclass:
`grep -rn "isinstance" src/` → 2 hits (both legitimate, at a boundary)

six months after:
`grep -rn "isinstance" src/` → 31 hits
of which, checks for the one problematic subclass : 18

Each check is a place where adding a SEVENTH implementation requires editing existing code -- so open/closed is gone too.

That is the connection to say out loud: **a Liskov violation destroys open/closed**. Once callers check the type, adding an implementation is no longer free.

Contract tests, priced

```
6 implementations of Repository, 14 contract tests each:

writing the contract suite once : ~2 hours, 140 lines
wiring each implementation     : 3 lines x 6 = 18 lines
tests that now run             : 84
substitution violations caught
  the first time it was run    : 3
    - Postgres raised on delete-missing; in-memory did not
    - Redis returned "" for a missing key; others returned None
    - the S3 one was case-insensitive on keys
```

Three real violations found in two hours, each of which would have been an incident. That is the argument for contract tests, and it is a better answer than “I’d be careful”.

The square-rectangle failure, in one line

```
set_width(5); set_height(4); area()

Rectangle : 20
Square    : 16          → 20% wrong, silently, in code that is correct
```

The cost of the fix

```
Rectangle/Square as separate immutable Shapes:

lines removed : the 2 overrides, the 4 setters      = ~12
lines added   : the Shape protocol, as_rectangle() = ~8
call sites that change
    (they took a Shape or a Rectangle, and still do) : 0
isinstance checks removed                             : all of them
```

7 The trade-offs

What the principle costs

Inheritance gets much harder to justify. Applied honestly, Liskov rules out most of the inheritance people want to write, and you end up with more interfaces and more composition. That is more files and more indirection, and it is the same cost as [day 049](#)’s composition argument.

Contracts have to be written down. The rules a parent guarantees are usually implicit — `Rectangle` never says “width and height are independent” — so making them substitutable means documenting or testing what was previously understood. That is real work, and the contract test file is where it lives.

You lose some convenient reuse. `ReadOnlyList(list)` is genuinely convenient and it *mostly* works. Doing it properly means a `Sequence` type, conversions at the boundary, and callers being precise about whether they read or write.

When a narrower subtype is acceptable anyway

I would accept a technical violation if the subtype is never handed to code written for the parent. A test double that raises on an unused method is fine, because nothing in production ever receives it. A subclass used only inside the module that defines it is fine. The principle is about *substitution*, so if the substitution never happens, there is nothing to violate — but say that explicitly rather than letting it be an accident.

I would accept it if the parent's contract genuinely allows failure and the child merely fails more often. If `PaymentGateway.charge` documents that it can raise `GatewayUnavailable`, then an implementation that raises it more often is not violating anything; it is exercising a documented outcome. **The rule is about promises, not about behaviour being identical.**

When defensive `isinstance` is the right answer

I would use `isinstance` at a boundary — parsing external input, dispatching on a message type that arrived over the wire, or narrowing an `Any` from a library. That is a place where the type is genuinely unknown and must be discovered.

I would not use `isinstance` inside business logic to work around a subtype's misbehaviour. That is treating the symptom, and it spreads: two checks become thirty-one, and every new implementation has to be added to all of them.

The honest ambiguity

The hardest part is that the “contract” is often not written anywhere, so reasonable people disagree about whether something is a violation. Does `Rectangle` promise independent sides? Nobody wrote it down, and yet every caller assumes it. The practical resolution is not philosophical:

If existing callers break, it is a violation. If they do not, it is not.

That is a test you can run, and it is why contract tests matter more than argument. Take the parent's test suite, run it against the child, and let the failures decide.

Where it sits with the other four

Single responsibility ([day 055](#)) decides what the classes are. Open/closed ([day 056](#)) says extend rather than edit — **and it works only if substitution holds**, because the whole mechanism is a caller holding an interface and receiving any implementation. Interface segregation ([day 058](#)) is the main *fix* for Liskov violations: most of them are one interface promising more than some implementations can deliver, and the answer is two smaller interfaces. Dependency inversion ([day 059](#)) points the arrows the right way so substitution has somewhere to happen.

8 In the interview

How it gets asked

- “*Is a Square a Rectangle?*” — the classic. The answer is “as a shape yes, as a mutable class no”, and then the six lines that show it.
- “*What is the Liskov substitution principle?*” — and the follow-up is always “give me an example of a violation you’ve seen”, so have `ReadOnlyList` or `Collections.unmodifiableList` ready.
- “*Here’s a class hierarchy. What’s wrong with it?*” — usually a `Bird` with `fly()` and a `Penguin`, or an `Account` with a `FixedDeposit`.
- “*How would you detect a violation in a large codebase?*” — `isinstance` in callers, overrides that raise, and contract tests. The third answer is the one that scores.
- “*Why does this matter?*” — because it is what makes open/closed possible.

What to say out loud, in the first ninety seconds

1. **Answer the question as asked, then split it.** “*As a shape, yes — every square satisfies the definition of a rectangle. As a class with setters, no, and I can show that in six lines.*”
2. **Show the failing caller, not the classes.** “*Here’s a function written against `Rectangle`: set the width to 5, set the height to 4, expect an area of 20. Hand it a `Square` and it gets 16. The function is correct. Both classes are correct. The inheritance is what’s wrong.*”
3. **Name the broken promise.** “*`Rectangle` quietly guarantees that setting the width leaves the height alone. Nobody wrote that down and every caller relies on it. `Square` can’t keep that promise and stay square.*”
4. **Give the fix, and the general form.** “*I’d model the shared capability rather than the taxonomy — both implement `Shape` with `area()` — and I’d make them immutable, because most Liskov violations are really about mutation. With no setters there’s no promise to break.*”
5. **Name the real cost.** “*The damage isn’t the wrong area. It’s that callers start writing `isinstance` checks, and once they do, adding a new shape means editing existing code — so the violation destroys open/closed as well.*”

The follow-ups

“**Give me a violation you’ve seen in real code, not geometry.**” The commonest one is a read-only collection implemented as a subclass of a mutable one — a `ReadOnlyList` extending `list` whose `append` raises `NotImplementedError`. It type-checks perfectly, because it genuinely is a list, so any function taking a list will accept it and then crash at runtime on a method the type promised. That is not a hypothetical: Java ships it in the standard library as `Collections.unmodifiableList`, which returns a `List` whose mutating methods throw `UnsupportedOperationException`, and it has broken a great deal of production code over the years. `java.sql.Date` extends `java.util.Date` is the same failure — a date whose time-of-day methods throw. The fix in both cases is to split by capability rather than to subclass:

Python's `collections.abc` gets this right with `Sequence` for things you can read and `MutableSequence` for things you can also write, so a `tuple` is a `Sequence` and simply is not a `MutableSequence`, and the type checker refuses the bad substitution before the program runs. The other one I see constantly is a caching layer: `CachedUserRepository.get` raising `CacheMissError` where the plain `UserRepository.get` returned `None`. Throwing an exception the parent never documented is a broken promise, and it appears as an unhandled exception in production weeks later, in whichever environment happens to have caching enabled.

“How would you detect these in a large codebase?” Three ways, in increasing order of usefulness. The cheapest is to grep for `isinstance` in code that is not at a boundary — if business logic has to ask which subtype it received, substitution has already failed, and each of those checks is evidence. On one codebase I would expect a handful of legitimate ones at parsing boundaries; thirty is a symptom. The second is to look for overridden methods that raise, or that are documented with “do not call this on X” — a subclass refusing a method its parent promised is the definition of the violation and it is greppable. But the one that actually works is contract testing: write the test suite once against the *interface*, not against any implementation, and run the whole suite against every implementation. In pytest that is a base class of tests plus one three-line subclass per implementation. The first time I did that on a repository interface with six implementations, it found three genuine violations in one run — Postgres raised when deleting a key that did not exist while the in-memory one did not, the Redis one returned an empty string for a missing key where the others returned `None`, and the S3-backed one was case-insensitive on keys. Every one of those would have been an incident, and each was a substitution failure rather than a bug in any single implementation. It also solves a second problem from the testing side: it is what stops an in-memory test fake drifting away from the real thing.

“Why does it matter? What actually breaks?” Two things, and the second is the expensive one. The first is the obvious one: code that was written correctly against the parent produces a wrong answer or throws, and because it only does so when that particular subtype is wired in, it is environment-dependent and survives testing. The second is structural. As soon as one implementation misbehaves, callers defend themselves — an `isinstance` check here, a `try/except` there — and every one of those checks is a place that must be edited when a new implementation arrives. That is exactly the open/closed principle collapsing. The whole reason you put an interface in front of a set of implementations is so that new behaviour is a new file and one wiring line; the moment forty call sites are asking which implementation they got, adding the seventh means editing forty places, and you have paid for all the indirection of an interface and kept the maintenance cost of the `if` chain. So I would frame Liskov not as a rule about class hierarchies but as the thing that makes polymorphism actually deliver — if substitution is not safe, none of the abstraction above it is worth anything.

A model answer

“It depends which question you’re asking, and the split is the interesting part. As a *shape*, yes — every square satisfies the definition of a rectangle, and mathematically the inheritance is fine. As a *class with setters*, no, and I can show why in six lines.

```
def resize_and_check(r: Rectangle) → None:
    r.set_width(5)
    r.set_height(4)
    assert r.area() == 20      # Square gives 16
```

That function is correct. `Rectangle` is correct. `Square` is correct on its own terms — it has to keep its sides equal, so setting the width must change the height. What’s wrong is the claim that a `Square` can be used wherever a `Rectangle` is expected.

The promise being broken was never written down: `Rectangle` quietly guarantees that setting the width leaves the height alone. Every caller relies on it and no type checker can see it. That’s why Liskov violations get through review — the types say the substitution is legal, and it’s the *behaviour* that isn’t.

The general rule I’d state is that a subtype must not demand more than its parent, must not deliver less, and must preserve every invariant the parent guaranteed. `Square` breaks the third.

The fix has two parts. Model the capability rather than the taxonomy — both implement a `Shape` protocol with `area()` and `perimeter()`, which is what they genuinely share, and `Square` gets an `as_rectangle()` conversion rather than an is-a claim. And make them immutable: with no setters there’s no promise to break, and `Square(4)` is a fine `Rectangle` forever. Most Liskov violations turn out to be about mutation.

The reason I’d push on this rather than shrug it off is the second-order cost. When one implementation misbehaves, callers start writing `isinstance` checks to defend themselves — and every one of those is a place that has to be edited when a new implementation arrives. So a Liskov violation quietly destroys the open/closed principle too: you’ve paid for the indirection of an interface and kept the maintenance cost of an if-chain.

To catch these in practice I’d write contract tests — one test suite against the interface, run against every implementation. The first time I did that on a repository with six implementations it found three real violations in a single run.”

9 Recall card

- **Anywhere the code expects a type, any subtype must work — and the caller must not need to know which one it got.** The three rules: a subtype must **not demand more** (stronger precondition), **not deliver less** (weaker postcondition), and **not break an invari-**

ant the parent guaranteed — including the ones nobody wrote down.

- **Square/Rectangle:** `set_width(5); set_height(4); area()` gives 20 and 16. The caller is correct, both classes are correct, the **is-a claim** is wrong. `Rectangle` silently promises that the sides are independent. **Most Liskov violations are really about mutation — make it immutable and the problem dissolves.**
- **Four tells:** `isinstance` appearing in *callers* · an override that **raises** (`ReadOnlyList.append`, `Collections.unmodifiableList`, `java.sql.Date`) · an override throwing an exception the parent never documented (`CacheMissError` instead of `None`) · docs saying “do not call X on Y”.
- **Fixes, in order:** model the **capability, not the taxonomy** (`Sequence` vs `MutableSequence` in `collections.abc`) · make it immutable · compose instead of inherit · move the method down onto a narrower interface.
- **Detect it with contract tests** — one suite written against the *interface*, three lines to wire each implementation. And say the second-order cost: **a Liskov violation destroys open/closed**, because once 40 call sites check the type, the seventh implementation is no longer one new file.

PRACTICE

Practice — Stability, and what Python's sort actually does

DSA topic: Stability, and what Python's sort actually does **System design topic:** Liskov substitution

Code these, in this order

One rule for the whole set: **every time you sort by more than one thing, say out loud whether you are using a tuple key or two passes, and if it is two passes, say which one goes first.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Relative Sort Array	LeetCode 1122 (Easy)	A custom order with a stable tie-break — the cleanest use of a tuple key.
2	Sort the People	LeetCode 2418 (Easy)	Sorting one list by another list's values, and keeping the pairing intact.
3	Custom Sort String	LeetCode 791 (Medium)	An ordering that is data rather than comparison, and where ties come from.
4	Sort Characters By Frequency	LeetCode 451 (Medium)	Two keys in opposite directions, which is where tuple keys start to run out.

On problem 1, find the tie

The problem says elements not in the reference order go at the end **in ascending order**. That is the tie-break, and writing it as the second element of a tuple key is the whole solution. Say out loud what happens if you leave it out.

On problem 2, do not lose the pairing

The obvious wrong move is to sort the heights and then try to line the names back up. Say why that fails, then solve it by sorting pairs. This is the same lesson as [day 051](#)'s tag.

On problem 3, notice there is no comparison

The order is given to you as a string, so the “key” is a position lookup. Build the lookup first, then sort by it. Then answer: what should happen to characters not in the order string, and does your solution make that decision on purpose?

On problem 4, hit the wall deliberately

Sort by frequency descending, then by character ascending. Try to do it with one tuple key. You can, because frequency is a number and can be negated — so do it. Then change the requirement to “by name descending, then by frequency ascending” and try again. `-name` raises. Paste the `TypeError`, then solve it with two passes and say which pass goes first.

The stability-test drill

Write the four-line `is_stable` checker, then run it against everything you have written this phase:

1. Your `merge_sort` from day 053.
2. Your `insertion_sort` from day 052.
3. Your `selection_sort`.
4. Your `quicksort` from day 054.
5. Your `counting_sort` from day 056.
6. Python's `sorted`.

Predict each result before you run it. Then, for each `True`, find the single character that is responsible.

The two-pass drill

Given people with a name, a department and an age:

1. Sort by department, then name ascending. Tuple key.
2. The same, two passes. Which pass first?
3. Sort by department ascending, age descending. Tuple key — what makes it possible?
4. Sort by department ascending, name descending. Try the tuple key, paste the error, then do it in two passes.
5. Sort by age descending, and within equal ages keep the original input order. Which of `reverse=True` and `reversed(sorted(...))` is correct, and why is the other one wrong?
6. Sort by three keys: city ascending, department descending, age ascending. Say how many passes and in what order.

The reverse drill

Run both and record the difference:

```
ages = [("Asha", 34), ("Bala", 12), ("Chitra", 34)]
sorted_ages = sorted(ages, key=lambda p: p[1], reverse=True)
reversed_sorted_ages = list(reversed(sorted_ages))
```

1. What differs, and on which elements?
2. Which one would you want for a leaderboard where equal scores show earliest-achieved first?
3. What does `max(ages, key=lambda p: p[1])` return, and why is that consistent with stability?
4. What does `min` return on the same data?

The Timsort drill

Time each on two million integers, and predict the ordering of the results first:

1. Already sorted.
2. Reverse sorted.
3. Random.
4. Sorted, with the last 1,000 elements shuffled.
5. All identical values.
6. Two sorted halves concatenated.

Then explain each result using runs, minrun and galloping. Number 6 is the one to think hardest about.

The other-languages drill

Say, for each, whether the sort is stable, and what you would do if you needed stability and did not have it:

1. Python `sorted` .
2. C++ `std::sort` .
3. Java `Arrays.sort(String[])` .
4. Java `Arrays.sort(int[])` .
5. Go `sort.Slice` .
6. JavaScript `Array.prototype.sort` in a modern browser.

Then write the decorate-sort-undecorate function that gives you stability from any sort, and say what it costs in time and memory at a million elements.

The Liskov drill

For each, say whether it is a violation. If it is, name which of the three rules it breaks (demands more / delivers less / breaks an invariant), and give the fix.

1. `Square(Rectangle)` with setters.
2. `Square(Rectangle)` where both are immutable.
3. `ReadOnlyList(list)` whose `append` raises.
4. `Penguin(Bird)` whose `fly()` raises.
5. `FixedDeposit(Account)` whose `withdraw` raises before maturity.
6. `CachedRepository(Repository)` whose `get` raises `CacheMissError` instead of returning `None` .
7. `NullLogger(Logger)` whose `log(msg)` does nothing at all.
8. A subclass whose `handle(event: LoginEvent)` narrows the parent's `handle(event: Event)` .

Two of those eight are fine. Name them and defend them.

The square-rectangle drill

1. Write `Rectangle` with setters and `Square` inheriting from it.
2. Write `resize_and_check` and run it on a `Rectangle` and on a `Square`. Paste both results.
3. State, in one sentence, the promise `Rectangle` makes that nobody wrote down.
4. Make both immutable and re-run. What happened to the problem?
5. Rewrite them as two implementations of a `Shape` protocol.
6. Now answer the interview question out loud in under ninety seconds.

The contract-test drill

Take an interface you can write in five lines — a key-value repository will do:

1. Write the contract test suite: at least five tests, written against the interface only.
2. Include one test for a missing key, one for overwriting, and one for deleting something that does not exist.
3. Write two implementations: an in-memory dict and one backed by a file or a list of tuples.
4. Wire both into the contract suite and run it.
5. Deliberately make one implementation raise on a missing key. Which test fails, and what does the failure tell you?
6. Say how this same file solves the “is my test fake honest?” problem from [day 053](#).

The isinstance-audit drill

For each place an `isinstance` check appears, say whether it is legitimate or a symptom:

1. Parsing a JSON payload that could be a dict or a list.
2. In a `Checkout` class, checking whether the discount policy is a `PercentageOff`.
3. Narrowing an `Any` returned by a third-party library.
4. In a loop over `Account` objects, checking for `FixedDeposit` before withdrawing.
5. In a `__eq__` method, checking that the other operand is the same class.
6. In a template renderer, checking whether a value is a `str` or a `Sequence`.

Three are legitimate. For the others, say what the real fix is.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Is Python's sort stable? Why do you care?* The definition, the two-pass multi-key argument with the pass order, the tuple-key alternative, the `reverse=True` subtlety, and three facts about Timsort.
2. *Is a Square a Rectangle? Defend your answer in code.* The shape-versus-class split, the six-line failure, the unwritten promise, the capability fix, and the second-order cost — that a Liskov violation destroys open/closed.

3. *Which sorting algorithms are stable, and why?* The rule about neighbour swaps against long-distance swaps, the one character in each stable sort that makes it so, and the three-element selection-sort counter-example.

Before you move on

- ☐ I ran `is_stable` against all six sorts and predicted every result correctly.
- ☐ For each stable sort I wrote, I can point at the single character responsible.
- ☐ I hit the `TypeError` from negating a string key and solved it with two passes.
- ☐ I can say which pass goes first and why that is backwards from how it sounds in English.
- ☐ I can give three facts about Timsort and quote the sorted-versus-random timing.
- ☐ I ran the square-rectangle failure and can state the unwritten promise in one sentence.
- ☐ I wrote a contract test suite and watched it catch a deliberate substitution violation.
- ☐ I answered all three questions above out loud.

Custom comparators and sorting by keys

DATA STRUCTURES AND ALGORITHMS

Custom comparators and sorting by keys

You can sort by two fields in opposite directions without writing a comparator class.

SYSTEM DESIGN

Interface segregation

You can spot a fat interface and split it by client.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Sort by score descending, then by name ascending.*
- *Why is a single large interface a problem?*

1 What this is, and why they ask it

Almost every real sorting task is “sort by *this*”, not “sort these numbers”. Python separates the two completely: `sorted()` knows how to put things in order, and you tell it what to look at with `key=`. A **key function** is called once for each element and returns the thing to order by — a number, a string, or a tuple of both. That one argument answers ninety-nine sorting requirements out of a hundred, and the remaining one has a different tool, `functools.cmp_to_key`.

They ask it because it is the sorting question that comes up in actual work, and because the two-fields-in-opposite-directions version separates people quickly. “Score descending, name ascending” cannot be done with `reverse=True`, because `reverse` applies to the whole thing. It has one clean answer with a tuple key, one clean answer with two passes, and about four wrong answers that look right. It also connects everything from the last two days: [day 051](#) said `key` is called once per element, [day 057](#) said stability is what makes the two-pass version work, and today is where those two facts get used.

2 The story

Sunil has photographed weddings around Mysore for nineteen years, and the part of the day he actually has to think about is the big group photograph, because forty-odd people have to end up standing somewhere and every single one of them has an opinion about where.

He does not decide the order. He has never decided the order. The family decides, and he arranges.

The first thing he asks, before anybody comes out, is what the rule is. That morning the bride’s mother said the obvious one — taller people behind, shorter people in front, so everybody’s face shows. That one is easy, because he can look at somebody and know the answer without asking.

Then, while forty-six people were standing about in the sun, she changed it. The groom’s side on the left as you look at the photograph, her side on the right. And within each side, older people in front and younger behind.

Sunil did not learn a new way of arranging people. He has one way of arranging people and it has not changed in nineteen years. What changed is what he had to know about each person, and now it was two things instead of one — which side, and roughly how old.

So he walked down the group once, asked every person those two questions, once each, and remembered the answers. Then he arranged.

That “once each” is something he learned the hard way. In his first years he used to work by holding two people side by side and asking them both, then holding one of them against the next and asking both again, and with forty-six people he was asking the same aunt her age five or six times and she was getting shorter with him every time. Asking everybody once at the start and then arranging is faster and nobody gets annoyed.

There was one thing that morning he could not do by asking each person separately. Two of the older men were both uncles, and the mother wanted the more senior of the two on the left, and neither of them could tell him a number — seniority in that family came out of who married whose sister, and the only sensible statement anybody could make was about the two of them together, not about either one on his own. So for those two, and only those two, he went and asked an elder: of these two, which one goes on the left? One question, about a pair.

Everything else that morning was a question he could ask one person at a time.

3 The idea in plain English

Sunil’s one way of arranging people is `sorted()`. The rule he is given is the **key function**. Asking each person once is why `key=` is called once per element. And the two uncles are the one case that needs a **comparator** — a rule about a pair, not a fact about an item.

The key function

`sorted()` and `list.sort()` take a `key` argument. It is a function called once on each element, and the results are what get compared:

```
words = ["banana", "fig", "cherry", "kiwi"]
sorted(words, key=len)           # ['fig', 'kiwi', 'banana', 'cherry']
```

`len` is called on each word once, giving 6, 3, 6, 4, and the words are ordered by those numbers. The words themselves are never compared with each other.

That last sentence is the whole idea. **You are not changing how sorting works; you are changing what it looks at.**

```
people.sort(key=lambda p: p.age)           # by one field
files.sort(key=lambda f: f.name.lower())   # case-insensitive
records.sort(key=lambda r: r["created_at"]) # by a dictionary entry
```

Two fields at once: the tuple key

A tuple compares element by element. It looks at the first item; if those are equal, it looks at the second; and so on. That is exactly what “sort by department, then by name” means:

```
people.sort(key=lambda p: (p.department, p.name))
```

```

comparing (Sales, Bina) with (Sales, Ravi):
    "Sales" == "Sales"    → keep going
    "Bina" < "Ravi"      → Bina first

comparing (HR, Zafar) with (Sales, Anil):
    "HR" < "Sales"       → decided. Zafar first. Never looks at the names.

```

Tuples of any length work, and you can mix types as long as each *position* is consistent — position one always a string, position two always a number.

Opposite directions, which is the actual question

`reverse=True` reverses the whole ordering, so it cannot give you one field ascending and another descending. There are three correct answers and you should know which to use when.

One: negate the numeric field inside the tuple key. The cleanest, when the descending field is a number.

```
students.sort(key=lambda s: (-s.score, s.name))    # score DESC, then name ASC
```

`-s.score` reverses that field and nothing else. This is the answer to the interview question as usually asked, because scores are numbers.

Two: two passes, relying on stability. Needed when the descending field is a **string**, because you cannot negate a string.

```

people.sort(key=lambda p: p.name)                # least significant first
people.sort(key=lambda p: p.department, reverse=True)  # most significant second

```

Sort by the *least* significant key first. The final pass reorders only by department; everyone inside a department compares equal, and Python's sort is stable, so the name order set up by the first pass survives ([day 057](#)).

Three: a reversing wrapper for a non-numeric field. Occasionally useful, and worth knowing exists:

```

class Descending:
    def __init__(self, value): self.value = value
    def __lt__(self, other): return other.value < self.value    # flipped
    def __eq__(self, other): return self.value == other.value

people.sort(key=lambda p: (Descending(p.department), p.name))

```

More machinery than most situations deserve, but it does let one string field go backwards inside a tuple key.

When the rule is about a pair: `cmp_to_key`

Sometimes there is no per-item answer, only a rule about two items together — Sunil's two uncles. The classic example is arranging numbers to form the largest possible number:

```
[3, 30, 34, 5, 9] → "9534330"
```

Ask yourself what the key of `3` would be. There is no answer, because whether `3` should come before `30` depends on `30`. But the *pairwise* rule is simple: `a` comes before `b` if `a + b` (as strings) is bigger than `b + a`. `"330" > "303"`, so `3` goes first.

For that, wrap a comparison function:

```
from functools import cmp_to_key

def largest_number(nums: list[int]) → str:
    def compare(a: str, b: str) → int:
        if a + b > b + a:
            return -1          # a comes first
        if a + b < b + a:
            return 1           # b comes first
        return 0              # equal
    strings = sorted(map(str, nums), key=cmp_to_key(compare))
    return "0" if strings[0] == "0" else "".join(strings)
```

The comparison function returns a **negative number if the first argument comes first**, positive if the second does, and zero if they tie. That convention is inherited from C and it trips people up: negative means “already in the right order”.

`cmp_to_key` is genuinely slower — the comparison runs $O(n \log n)$ times rather than the key running n times — so use it only when there is no key. That is the rule: **if you can describe the order by a fact about one item, use `key`. If you can only describe it by comparing two, use `cmp_to_key`.**

Sorting your own objects

Three options, and the choice is worth having an opinion about:

```
# 1. key= at the call site -- the default choice
sorted(people, key=lambda p: p.age)

# 2. @dataclass(order=True) -- ordering by every field, in declaration order
@dataclass(order=True, frozen=True)
class Version:
    major: int
    minor: int
    patch: int

# 3. __lt__ -- one natural order, defined once on the class
class Money:
    def __lt__(self, other: "Money") → bool:
        return self.paise < other.paise
```

Prefer `key=` unless the type has exactly one obvious ordering that everybody would agree on. `Version` and `Money` do. `Person` does not — sorting by age, by name and by joining date are all equally reasonable, so putting one of them on the class is an arbitrary decision imposed on every caller.

If you use `@dataclass(order=True)` and one field should not participate, exclude it:

```
@dataclass(order=True)
class Task:
    priority: int
    description: str = field(compare=False)    # not part of the ordering
```

`operator.itemgetter` and `attrgetter`

Faster than a lambda, because they run in C rather than in Python:

```
from operator import itemgetter, attrgetter

sorted(rows, key=itemgetter(2))                # by position 2
sorted(rows, key=itemgetter("score", "name"))  # a tuple key on a dict
sorted(people, key=attrgetter("department", "name"))
```

Roughly two times faster than the equivalent lambda on a large list. Worth using and worth being able to name.

The one fact that decides most arguments

key is called **exactly once per element** — **n times, not $n \log n$** . So an expensive key function is fine, and a lambda that does a database lookup is a disaster for a different reason. This also means the key must be *deterministic*: a key function that returns a different answer on two calls gives an undefined ordering.

4 The picture

How a tuple key decides, step by step:

sorting by (department, name)

element	key produced	comparison
Person("Ravi", "Sales")	("Sales", "Ravi")	\ key() called ONCE per element n calls, not n log n /
Person("Anil", "HR")	("HR", "Anil")	
Person("Bina", "Sales")	("Sales", "Bina")	
Person("Chandra", "HR")	("HR", "Chandra")	

then the TUPLES are compared, position by position:

```
("HR", "Anil") vs ("HR", "Chandra")
"HR" = "HR"      → tie, look at position 2
"Anil" < "Chandra" → Anil first

("HR", "Anil") vs ("Sales", "Bina")
"HR" < "Sales"   → DECIDED. Position 2 is never looked at.
```

result: HR/Anil, HR/Chandra, Sales/Bina, Sales/Ravi

What to notice: the second element of the tuple is only ever consulted when the first is a tie. That is why a tuple key means “then by”, and why the order of the tuple’s elements is the order of the sorting rules.

The three ways to get opposite directions:

```
want: score DESCENDING, then name ASCENDING

data  ("Asha", 90) ("Bala", 85) ("Chitra", 90) ("Devi", 85)

(1) NEGATE the number inside the tuple key          ← use this when you can
    key = lambda s: (-s.score, s.name)
    Asha   → (-90, "Asha")
    Chitra → (-90, "Chitra")
    Bala   → (-85, "Bala")
    result: Asha(90), Chitra(90), Bala(85), Devi(85)

(2) TWO PASSES, least significant key first          ← use when the field is a string
    sort(key=name)                                   → Asha, Bala, Chitra, Devi
    sort(key=score, reverse=True)                    → Asha, Chitra, Bala, Devi
                                                    ^^^^^^^^^^^^^ stability preserved the name order

(3) reverse=True ALONE                              ← WRONG. Reverses everything.
    sort(key=lambda s: (s.score, s.name), reverse=True)
    result: Chitra(90), Asha(90), Devi(85), Bala(85)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ names now DESCENDING too
```

What to notice: option 3 is the trap, and it produces a plausible answer. The scores are descending, which is what you asked for; the names are also descending, which you did not.

Key against comparator:

```
key=          asks a question about ONE element
              "what is this person's age?"
              called n times
              n = 1,000,000 → 1,000,000 calls

cmp_to_key()  asks a question about TWO elements
              "of these two, which goes first?"
              called once per comparison
              n = 1,000,000 → ~20,000,000 calls,
                           each with Python-level function overhead

→ roughly 20x more work, and each call is slower.
  Use it ONLY when no per-element key exists.
```

What to notice: the choice is not stylistic. If a key exists, `cmp_to_key` costs you a factor of twenty for nothing.

5 The code, built step by step

The simplest key

```
words = ["banana", "fig", "cherry", "kiwi"]
print(sorted(words, key=len))           # ['fig', 'kiwi', 'banana', 'cherry']
print(sorted(words, key=str.lower))     # case-insensitive
```

`key=len` passes the function itself, not a call to it. `key=len(words)` is a common beginner error and raises immediately.

The interview question, three ways

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Student:
    name: str
    score: int
```

```
def by_score_desc_then_name(students: list[Student]) → list[Student]:
    """Score descending, name ascending. The tuple key with a negated number."""
    return sorted(students, key=lambda s: (-s.score, s.name))
```

One line, one pass, and it is the answer to give first.

```
def by_score_desc_then_name_two_pass(students: list[Student]) → list[Student]:
    """The same result, using stability. Needed when the DESC field is a string."""
    out = sorted(students, key=lambda s: s.name)           # least significant
    return sorted(out, key=lambda s: s.score, reverse=True) # most significant
```

Both give the same answer here. The second generalises to fields you cannot negate.

The version people write that is wrong

```
def by_score_desc_then_name_broken(students):
    return sorted(students, key=lambda s: (s.score, s.name), reverse=True)
```

```
data = [Student("Asha", 90), Student("Bala", 85),
        Student("Chitra", 90), Student("Devi", 85)]
print([s.name for s in by_score_desc_then_name_broken(data)])
```

```
['Chitra', 'Asha', 'Devi', 'Bala']
```

The scores are descending, which looks correct. The names inside each score are descending too — Chitra before Asha — because `reverse=True` reverses the entire comparison, including the tie-break. Nothing raises. This is the single commonest wrong answer to this question.

Descending on a string field

```
def by_department_desc_then_name(people: list[Student]) → list[Student]:
    """Department descending, name ascending. Two passes -- you cannot negate a string."""
    out = sorted(people, key=lambda p: p.name)           # least significant, ASC
    return sorted(out, key=lambda p: p.department, reverse=True)  # most significant, DESC
```

And the attempt that does not work, so you recognise the error:

```
sorted(people, key=lambda p: (-p.department, p.name))
```

```
Traceback (most recent call last):
  File "day58.py", line 1, in <module>
    sorted(people, key=lambda p: (-p.department, p.name))
                                ^^^^^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

Sorting by a lookup table

When the order is *given* to you rather than computed — a custom alphabet, a priority list, a status ordering:

```
STATUS_ORDER = {"critical": 0, "high": 1, "medium": 2, "low": 3}

def by_status(tickets: list[dict]) → list[dict]:
    """Order given as data. Unknown statuses sort last, deterministically."""
    return sorted(tickets, key=lambda t: (STATUS_ORDER.get(t["status"], 99), t["id"]))
```

Two things worth pointing at. `.get(..., 99)` means an unrecognised status does not raise — it sorts to the end, which is almost always what you want, and the alternative (`KeyError` in production on one unexpected row) is worse. And the `t["id"]` tie-break makes the output **deterministic**, which matters more than people expect: without it, two runs on the same data can produce different orders if the input order differs, and that shows up as a flaky test.

The comparator case

```
from functools import cmp_to_key

def largest_number(nums: list[int]) → str:
    """LeetCode 179. There is no per-element key: order depends on the PAIR."""
    def compare(a: str, b: str) → int:
        if a + b > b + a:
            return -1                # negative: a comes first
        if a + b < b + a:
            return 1
        return 0

    strings = sorted((str(n) for n in nums), key=cmp_to_key(compare))
    joined = "".join(strings)
    return "0" if joined[0] == "0" else joined      # [0, 0] must give "0", not "00"
```

Say the convention out loud when you write it: **negative means the first argument comes first**. It is the opposite of what most people guess, because it comes from C's "return `a - b`".

The complete file

```
"""Custom orderings: key functions, tuple keys, opposite directions, and comparators."""

from dataclasses import dataclass, field
from functools import cmp_to_key
from operator import attrgetter, itemgetter

@dataclass(frozen=True)
class Student:
    name: str
    score: int
    department: str = "General"

def by_score_desc_then_name(students: list[Student]) → list[Student]:
    """The standard answer. Negating reverses ONE field inside a tuple key."""
    return sorted(students, key=lambda s: (-s.score, s.name))

def by_score_desc_then_name_two_pass(students: list[Student]) → list[Student]:
    """The same, via stability. Generalises to fields that cannot be negated."""
    out = sorted(students, key=lambda s: s.name)
    return sorted(out, key=lambda s: s.score, reverse=True)

def by_department_desc_then_name(students: list[Student]) → list[Student]:
    """A DESCENDING STRING field: two passes are the only clean way."""
    out = sorted(students, key=lambda s: s.name)
    return sorted(out, key=lambda s: s.department, reverse=True)

class Descending:
    """Wrap a value to reverse just that field inside a tuple key."""

    __slots__ = ("value",)

    def __init__(self, value) → None:
        self.value = value

    def __lt__(self, other: "Descending") → bool:
        return other.value < self.value

    def __eq__(self, other: object) → bool:
        return isinstance(other, Descending) and self.value == other.value

def by_department_desc_then_name_wrapped(students: list[Student]) → list[Student]:
    """One pass, with a reversing wrapper. More machinery; occasionally worth it."""
    return sorted(students, key=lambda s: (Descending(s.department), s.name))

STATUS_ORDER = {"critical": 0, "high": 1, "medium": 2, "low": 3}

def by_status(tickets: list[dict]) → list[dict]:
    """Order given as DATA. Unknown values sort last; the id makes it deterministic."""
    return sorted(tickets, key=lambda t: (STATUS_ORDER.get(t["status"], 99), t["id"]))

def largest_number(nums: list[int]) → str:
    """LeetCode 179. The one case where no per-element key exists.

    Negative return = the FIRST argument comes first. Inherited from C's a - b.
    """
    def compare(a: str, b: str) → int:
        if a + b > b + a:
            return -1
        if a + b < b + a:
            return 1
        return 0

    joined = "".join(sorted((str(n) for n in nums), key=cmp_to_key(compare)))
    return "0" if joined[0] == "0" else joined

def sort_by_custom_alphabet(words: list[str], alphabet: str) → list[str]:
```

```

"""Order the letters by a given alphabet -- an alien dictionary ordering."""
rank = {letter: i for i, letter in enumerate(alphabet)}
return sorted(words, key=lambda w: [rank[c] for c in w])

@dataclass(order=True, frozen=True)
class Version:
    """A type with ONE obvious ordering -- so put it on the class."""
    major: int
    minor: int
    patch: int

@dataclass(order=True)
class Task:
    """Order by priority only; the description takes no part."""
    priority: int
    description: str = field(compare=False)

if __name__ == "__main__":
    data = [
        Student("Asha", 90, "Sales"),
        Student("Bala", 85, "HR"),
        Student("Chitra", 90, "HR"),
        Student("Devi", 85, "Sales"),
    ]

    print([s.name for s in by_score_desc_then_name(data)])
    # ['Asha', 'Chitra', 'Bala', 'Devi']
    assert by_score_desc_then_name(data) == by_score_desc_then_name_two_pass(data)

    # the common WRONG answer
    print([s.name for s in sorted(data, key=lambda s: (s.score, s.name), reverse=True)])
    # ['Chitra', 'Asha', 'Devi', 'Bala'] ← names reversed too

    print([(s.department, s.name) for s in by_department_desc_then_name(data)])
    # [('Sales', 'Asha'), ('Sales', 'Devi'), ('HR', 'Bala'), ('HR', 'Chitra')]
    assert by_department_desc_then_name(data) == by_department_desc_then_name_wrapped(data)

    print(largest_number([3, 30, 34, 5, 9])) # 9534330
    print(largest_number([10, 2])) # 210
    print(largest_number([0, 0])) # 0 -- not "00"

    print(sort_by_custom_alphabet(["word", "world", "row"], "worldabcefghijklmnpqstuvxyz"))
    # ['world', 'word', 'row'] -- 'world' first: at position 3, 'l' (rank 3) beats 'd' (rank 4)

    print(sorted([Version(1, 2, 3), Version(1, 10, 0), Version(1, 2, 10)]))
    # [Version(1,2,3), Version(1,2,10), Version(1,10,0)]

    tickets = [
        {"id": 3, "status": "low"},
        {"id": 1, "status": "critical"},
        {"id": 2, "status": "unknown"},
        {"id": 4, "status": "critical"},
    ]
    print([t["id"] for t in by_status(tickets)]) # [1, 4, 3, 2]

    # itemgetter / attrgetter -- the same thing, ~2x faster because it is C
    rows = [("b", 2), ("a", 2), ("c", 1)]
    print(sorted(rows, key=itemgetter(1, 0))) # [('c', 1), ('a', 2), ('b', 2)]
    print([s.name for s in sorted(data, key=attrgetter("department", "name"))])
    # ['Bala', 'Chitra', 'Asha', 'Devi']

    # key is called exactly n times
    calls = 0
    def counted(s: Student) → int:
        global calls
        calls += 1
        return s.score
    sorted(data * 250, key=counted)
    print(calls) # 1000 -- n, not n log n

```

6 What it costs

`key` is called `n` times, not `n log n`

```
n = 1,000,000 elements

key=          1,000,000 calls
cmp_to_key()  ~20,000,000 calls (one per comparison, n log n of them)

→ 20x more calls, AND each cmp_to_key call goes through a wrapper object,
   so the real difference is closer to 30-50x on the clock.
```

That is the whole argument for preferring `key`. A key function that costs a millisecond is `1,000` seconds at a million elements — bad, but predictable; the same work in a comparator is twenty thousand seconds.

The cost of the key function itself

```
sorting 1,000,000 records

key=attrgetter("score")    ~0.9 s    (C-level attribute access)
key=lambda s: s.score      ~1.6 s    (a Python call per element)
key=lambda s: (-s.score, s.name) ~2.1 s    (a tuple allocation per element)
key=cmp_to_key(compare)    ~28 s    (a wrapper object per element,
                                     plus a Python call per comparison)
```

`itemgetter` / `attrgetter` are roughly twice as fast as the equivalent lambda, for free.

Two passes against one tuple key

```
n = 1,000,000, sorting by two fields

one tuple key : 1 sort x n log n comparisons on TUPLES (~1.5x a scalar comparison)
                1,000,000 tuple allocations
                ~2.1 s

two passes    : 2 sorts x n log n comparisons on SCALARS
                no tuple allocation
                the SECOND sort is on nearly-sorted data, so Timsort is fast
                ~2.4 s
```

They are close. Choose on clarity, not on speed: **one tuple key when you control both fields in one place, two passes when a field must go descending and cannot be negated.**

Memory

```
sorted(data, key=f) : 0(n) for the output list
                   + 0(n) for the computed keys (Python builds them all first)

So a tuple key on a million records allocates a million tuples, held
simultaneously. At ~72 bytes for a 2-tuple, that is ~72 MB of keys
on top of the data.
```

Worth knowing before you use an elaborate key on a very large list. If memory matters, sort by a single scalar in two passes rather than building a million tuples.

Determinism

```
sorting 10,000 tickets by status only, where 4,000 are "high":

    the relative order of those 4,000 comes from the INPUT order.
    Two runs over the same data from a database with no ORDER BY
    can give two different outputs.

    adding a tie-break on a unique id: one extra tuple element,
    ~0.1 s at a million rows, and the output is reproducible.
```

The rule: **if the output is shown to a person or compared in a test, always end the key with something unique**. This is a small habit that prevents a whole class of flaky test.

7 The traps

The near-miss: `reverse=True` with a tuple key

```
sorted(students, key=lambda s: (s.score, s.name), reverse=True)
```

```
['Chitra', 'Asha', 'Devi', 'Bala']
```

Scores descending — correct. Names descending inside each score — not what was asked. `reverse` applies to the whole comparison, including the tie-break, and there is no way to make it apply to one element of a tuple. This is the commonest wrong answer to today's interview question, and it produces a plausible-looking output.

The real error: negating a string

```
sorted(people, key=lambda p: (-p.name, p.age))
```

```
Traceback (most recent call last):
  File "day58.py", line 1, in <module>
    sorted(people, key=lambda p: (-p.name, p.age))
                                ^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

At least this one fails loudly. The fix is two passes, or the `Descending` wrapper.

The real error: calling the key function instead of passing it

```
sorted(words, key=len(words))
```

```
Traceback (most recent call last):
  File "day58.py", line 1, in <module>
    sorted(words, key=len(words))
    ~~~~~^~~~~~
TypeError: 'int' object is not callable
```

`key` takes a *function*, not a value. `key=len`, not `key=len(x)`.

The real error: sorting objects with no ordering

```
@dataclass(frozen=True)
class Student:
    name: str
    score: int

sorted([Student("Asha", 90), Student("Bala", 85)])
```

```
Traceback (most recent call last):
  File "day58.py", line 6, in <module>
    sorted([Student("Asha", 90), Student("Bala", 85)])
    ~~~~~^~~~~~
TypeError: '<' not supported between instances of 'Student' and 'Student'
```

`@dataclass` gives `__eq__`, not `__lt__`. Either pass `key=`, or declare `order=True` — and prefer `key=`, because most types do not have one obvious order that every caller would agree with.

The trap: a key that raises on some rows

```
sorted(tickets, key=lambda t: STATUS_ORDER[t["status"]])
```

```
Traceback (most recent call last):
  File "day58.py", line 1, in <module>
    sorted(tickets, key=lambda t: STATUS_ORDER[t["status"]])
    ~~~~~^~~~~~
KeyError: 'unknown'
```

One unexpected value from the database and the whole page fails to render. Use `.get(status, <a sensible default>)` and decide deliberately whether unknown values sort first or last.

The trap: an inconsistent comparator

```
def compare_broken(a, b):
    return 1 if a > b else -1      # never returns 0
```

Python will not raise. It will produce an ordering that is not what you meant, and in some inputs an order that is not even self-consistent — `a` before `b`, `b` before `c`, `c` before `a`. A comparator must be **total and consistent**: it must return 0 for genuinely equal items, and if it

says `a` before `b` it must never also say `b` before `a`. Getting that wrong is undefined behaviour with no error message.

The trap: a non-deterministic or side-effecting key

```
sorted(users, key=lambda u: fetch_score_from_api(u.id))    # 1,000,000 HTTP calls
sorted(items, key=lambda i: random.random())              # undefined ordering
```

The first is correct and catastrophically slow — `n` network calls, and `key` being called only `n` times is no comfort at a million rows. Fetch the scores once into a dict, then sort by the dict. The second produces an ordering Python makes no promises about, because the comparison is not self-consistent; to shuffle a list, use `random.shuffle`.

The trap: `sort()` returns `None`

Still the commonest Python sorting bug, and it survives into this lesson:

```
top = students.sort(key=lambda s: -s.score)[:3]
```

```
TypeError: 'NoneType' object is not subscriptable
```

`sorted()` when you want a value; `.sort()` when you want the side effect.

8 In the interview

How it gets asked

- “Sort by score descending, then by name ascending.” — the direct form. The tuple key with a negated number is the answer; the `reverse=True` version is the trap.
- “Now sort by department descending, then name ascending.” — the follow-up that breaks the negation trick, because departments are strings.
- “Sort these employee objects.” — expects `key=`, and a good answer says why not `__lt__`.
- “Arrange these numbers to form the largest possible number.” — LeetCode 179, and the point is recognising that no per-element key exists.
- “How many times is your key function called?” — `n`, and the follow-up is why that matters.
- “Sort these words by a custom alphabet.” — a lookup table as the key.

What to say out loud, in the first ninety seconds

1. **Separate the two things.** “Sorting doesn’t change; what it looks at does. `key=` is a function called once per element that returns the thing to order by.”
2. **Give the tuple-key answer and read it aloud as a sentence.** “`key=lambda s: (-s.score, s.name)` — negative score first, so higher scores come first; name second, so it only decides ties.”
3. **Say why not `reverse=True`.** “`reverse` applies to the whole comparison, so it would reverse the name tie-break too. That gives a plausible wrong answer with no error.”

4. **Volunteer the string case before being asked.** “If the descending field were a string I couldn’t negate it — `-p.name` raises a `TypeError`. Then I’d do two passes, least significant key first, and rely on Python’s sort being stable.”
5. **Mention the call count.** “`key` is called n times, not $n \log n$, so an expensive key is affordable. A comparator with `cmp_to_key` is called once per comparison, so about twenty times more at a million elements — I’d only use it when no per-element key exists.”

The follow-ups

“Why not just `reverse=True`?” Because `reverse` reverses the entire comparison, not one field of it. If I write `key=lambda s: (s.score, s.name), reverse=True`, the scores do come out descending, which is what makes it dangerous — it looks right. But the tie-break reverses too, so within a score the names come out descending as well, and the output is plausible and wrong with no error anywhere. There is no argument that applies `reverse` to only part of a tuple key. So for a numeric field I negate it inside the key — `(-s.score, s.name)` — which reverses exactly that one field. For a field I cannot negate, such as a string, I use two passes and rely on stability: sort by the least significant key first, then by the most significant with `reverse=True`, and because Python’s sort is stable the second pass leaves elements with equal keys exactly as the first pass arranged them. There is a third option I would mention if the field is a string and I want one pass: a small wrapper class with `__lt__` flipped, used inside the tuple key. It works, and it is more machinery than most cases deserve.

“How many times is your key function called, and why does it matter?” Exactly n times — once per element, before any comparison happens. Python computes all the keys first, then sorts the elements by those keys. That is why `key=` replaced the old comparison-function style: a comparison function is called once per comparison, which is on the order of $n \log n$, so at a million elements that is about twenty million calls against one million. In practice the gap is bigger than twenty times, because `cmp_to_key` also wraps every element in a small object whose `__lt__` calls back into Python. It has three practical consequences. An expensive key is affordable — if computing the key costs a millisecond, I pay for it a million times rather than twenty million. The key must be deterministic, because it is computed once and reused; a key based on `random` or on the current time gives an ordering Python makes no promises about. And it costs memory: all n keys exist simultaneously, so a tuple key on a million records allocates a million tuples, about seventy megabytes on top of the data. If that mattered I would sort by a single scalar in two passes rather than building the tuples.

“When would you need a comparator instead of a key?” When the ordering cannot be expressed as a fact about a single element — when it only makes sense as a statement about a pair. The example I would give is arranging numbers to form the largest possible number: given 3, 30, 34, 5 and 9 the answer is “9534330”. Ask what the key of 3 would be and there is no answer, because whether 3 should come before 30 depends entirely on 30 — but the pairwise rule is simple, a before b if the string `a + b` is greater than `b + a`, since “330” beats “303”. For that I use `functools.cmp_to_key`, and the convention to remember is that a **negative** return means the first argument comes first, which is the opposite of what most people guess because it comes from C’s `a - b`. Two cautions. It is the slow path, so I would

only reach for it when no key exists, and the comparator must be consistent — it must return zero for genuinely equal items and must never say both that `a` comes before `b` and that `b` comes before `a`, or the result is an undefined ordering with no error message. Other genuine cases are ordering by a graph relationship rather than a value, and version strings where the rule is a segment-by-segment comparison — although for versions I would usually build a tuple key instead and keep the fast path.

A model answer

“Sorting itself doesn’t change; what changes is what it looks at. In Python that’s the `key` argument — a function called once per element that returns the value to order by.

```
sorted(students, key=lambda s: (-s.score, s.name))
```

Read as a sentence: the negated score comes first, so higher scores sort earlier; the name is second, so it only decides ties. A tuple compares position by position and stops as soon as one position differs, which is exactly what ‘then by’ means.

The thing I’d point at is why I negated rather than using `reverse=True`. `reverse` applies to the whole comparison, so `key=(s.score, s.name), reverse=True` gives descending scores — which looks right — but also descending names inside each score, which isn’t what was asked, and nothing raises. It’s the commonest wrong answer to this question.

If the descending field were a string I couldn’t negate it — `-p.name` raises a `TypeError`. Then I’d do it in two passes: sort by the ascending field first, then by the descending field with `reverse=True`. That’s correct only because Python’s sort is stable — the second pass sees everyone with the same department as equal and leaves them in the order the first pass produced. Note the passes go least-significant-key first, which is backwards from how you say it aloud.

One detail worth stating: `key` is called exactly n times, not $n \log n$, because Python computes all the keys first and then sorts. So an expensive key is affordable, the key must be deterministic, and all n keys are held in memory at once. The alternative, `functools.cmp_to_key`, is called once per comparison — about twenty times more at a million elements — so I’d only use it when no per-element key exists at all. The case where that’s genuinely true is something like arranging numbers to form the largest number, where whether 3 comes before 30 depends on 30.

Last thing: I’d end the key with something unique — an id — if the output is shown to a person or compared in a test, so two runs on the same data always give the same order.”

9 Recall card

- `key=` is a function called once per element that returns what to order by. Sorting does not change; what it *looks at* does. `key=len`, `key=attrgetter("dept", "name")` ($\sim 2\times$ faster than a lambda — it is C), `key=lambda t: STATUS_ORDER.get(t["status"], 99)` for an order given as data.
- **Two fields: a tuple key.** It compares position by position and stops at the first difference, so the tuple's order is the order of the rules. Opposite directions: **negate the number** — `(-s.score, s.name)`. Never `reverse=True` on a tuple key: it reverses the tie-break too and gives a **plausible wrong answer with no error**.
- **A descending *string* field needs two passes**, because `-p.name` raises `TypeError: bad operand type for unary -: 'str'`. Sort by the **least significant key first**, then the most significant with `reverse=True` — correct only because Python's sort is stable.
- **key runs n times; a comparator runs $\sim n \log n$ times.** At $n = 10^6$ that is 1M calls against $\sim 20M$, and on the clock $30\text{-}50\times$ because of the wrapper. Use `functools.cmp_to_key` **only when no per-element key exists** — the test case is LeetCode 179, where `a` before `b` iff `a+b > b+a`, and **negative means the first argument comes first**.
- **Prefer `key=` to `__lt__`** unless the type has one order everybody would agree on (`Version`, `Money`). And always end the key with something **unique** if a person or a test will see the output, or two runs on the same data can differ.

1 What this is, and why they ask it

The **interface segregation principle** is the I in SOLID: no client should be forced to depend on methods it does not use. In practice that means **many small interfaces, named after roles, rather than one large one named after a thing**. A class that only reads should depend on a type that only promises reading, so that a change to how writing works cannot reach it.

They ask it because a fat interface is the most common structural problem in a real codebase and almost nobody names it. It has two costs, and a good answer gives both. The first is that implementations are forced to supply methods they cannot support, so they stub them out with `NotImplementedError` — which is a Liskov violation from [day 057](#), created by the interface rather than by the implementer. The second is churn: adding one method to a fifteen-method interface forces every implementation and every test fake to change, including the ones that never touch it. Python has an unusually good answer to this — a `Protocol` can be declared by the *consumer*, so the narrow interface costs nothing and the provider does not even have to know — and being able to say that is a strong answer to a question most people answer generically.

2 The story

The society secretary at an apartment building in Pune is a retired bank manager called Mr Kulkarni, and about four years ago he did something entirely sensible that took him two years to undo.

The building has a main gate, a corridor cupboard where the cleaning things live, a terrace door, a pump room, and a store room in the basement where the society keeps decorations, spare taps, and the generator's spare parts.

Four people needed to get in and out. The watchman, the cleaning lady, the electrician who comes on call, and the boy who leaves the milk at six in the morning.

Rather than work out who needed what, Mr Kulkarni had one key cut that opened everything, and gave one to each of them. It took ten minutes and it solved the problem completely and everybody said it was a good idea.

Two things happened afterwards.

The first was in the second year, when some of the generator parts went missing from the store room. Nobody ever found out who took them, and that was the point — four people could have. The milk boy had, for two years, been carrying a key to a room he had never once

had a reason to enter, and when the committee started asking questions his name was on the list simply because of what was in his pocket.

The second was what happened next. They changed the store room lock. And because there was only one kind of key in the building, every single one of the four keys stopped working — not just for the store room, for everything. All four people had to stop what they were doing and come and collect a new key. The milk boy came at six in the morning, could not get through the main gate, and left the milk on the step, and Mrs Deshpande's cat got at it.

That is the bit Mr Kulkarni tells people about. Not the theft. The fact that changing a lock nobody in the building except two people had ever used had, that morning, stopped the milk.

He does it differently now, and it took an afternoon and about nine hundred rupees. The milk boy has one key and it opens the gate. The cleaning lady has two — the gate and the cupboard. The electrician has the gate, the pump room and the terrace. The watchman has all of them, because the watchman genuinely does need all of them.

They changed the terrace lock last year. Two keys were re-cut. Nobody else in the building found out it had happened.

3 The idea in plain English

Mr Kulkarni's master key is a fat interface. Everybody depends on all of it, so a change anywhere reaches everybody, and everybody carries capabilities they never use. Four separate keys is interface segregation.

The principle

No client should be forced to depend on methods it does not use.

“Client” means the code that calls the interface, not a person. And “depend on” is stronger than “call” — if your function takes a `Repository` with fifteen methods and calls one, you depend on all fifteen, because a change to any of them changes the type you are declared against.

The two costs, which are different and both worth naming

Cost one: implementations are forced to lie.

```
class Repository(Protocol):
    def get(self, id: str) → Row: ...
    def save(self, row: Row) → None: ...
    def delete(self, id: str) → None: ...
    def bulk_insert(self, rows: list[Row]) → None: ...
    def stream_all(self) → Iterator[Row]: ...
    def begin_transaction(self) → Transaction: ...
```

Now write a read-only repository backed by a CSV file that ships with the product. It can `get` and it can `stream_all`. It cannot save, delete, bulk-insert, or begin a transaction. So it does this:

```
class CsvRepository:
    def get(self, id: str) → Row: ...
    def stream_all(self) → Iterator[Row]: ...
    def save(self, row: Row) → None:
        raise NotImplementedError("this repository is read only")
    def delete(self, id: str) → None:
        raise NotImplementedError("this repository is read only")
    ...
```

That is a Liskov violation — a subtype delivering less than the contract promised — and the implementer had no choice. **The interface created the violation.** The fix is not to be more careful in `CsvRepository`; it is to have two interfaces, so that “can read” and “can write” are separate claims and `CsvRepository` only makes the one that is true.

Cost two: churn reaches code that does not care.

Add a `bulk_upsert` method to the interface. Now:

6 implementations	→ 6 files must add the method
4 test fakes	→ 4 more
every caller's type stubs	→ re-checked
in a compiled language	→ every module importing the interface recompiles and, in a deployed system, redeploys

A reporting service that only ever calls `get` has to change, be re-reviewed and be redeployed because of a method it will never call. That is the store room lock stopping the milk.

The classic example

The textbook version, and worth knowing because interviewers use it:

```
class Worker(Protocol):
    def work(self) → None: ...
    def eat(self) → None: ...

class Robot:
    def work(self) → None: ...
    def eat(self) → None:
        raise NotImplementedError("robots do not eat")    # forced to lie
```

Split by role:

```
class Workable(Protocol):
    def work(self) → None: ...

class Feedable(Protocol):
    def eat(self) → None: ...
```

Now `Robot` implements `Workable` only, and honestly. A payroll function takes `Feedable`; a scheduler takes `Workable`; a human implements both.

A more realistic version of the same shape is an office multifunction machine:

```
class Machine(Protocol):
    def print(self, doc: Document) → None: ...
    def scan(self) → Document: ...
    def fax(self, doc: Document, number: str) → None: ...
    def staple(self, doc: Document) → None: ...
```

An old laser printer can only print. It is forced to stub three methods, and every function that takes a `Machine` has to worry about which one it got.

Role interfaces, not header interfaces

Two names worth having:

- A **header interface** is named after the implementation and lists everything it can do: `Repository`, `Machine`, `UserService`. It is written *by the provider* and it grows forever.
- A **role interface** is named after what a client needs: `UserReader`, `Printer`, `PriceQuoter`. It is written *from the caller's side*, and it is small because a caller usually needs one or two things.

Name interfaces after roles, and prefer `-er` names. `UserReader` rather than `UserRepository`. The name itself keeps them small: it is hard to justify adding `delete` to something called `UserReader`, and very easy to justify adding it to something called `UserRepository`.

The Python answer, which is unusually good

In Java or C#, an interface is declared by the provider and the implementer must say `implements`. So splitting a fat interface is a change to every implementation.

In Python, `typing.Protocol` is **structural** — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([day 052](#)). That means the *consumer* can declare exactly the narrow interface it needs, and the provider does not have to be touched at all:

```
# in reporting/monthly.py -- declared by the CONSUMER, for the consumer
class UserReader(Protocol):
    def get(self, user_id: str) → User: ...

def monthly_report(users: UserReader, ids: list[str]) → Report:
    ...
```

`PostgresUserRepository` already satisfies `UserReader` — it has a `get` with that signature — and nobody had to change it. The reporting module now depends on exactly one method, so adding `bulk_upsert` to the repository cannot reach it, and the test fake for this module is a class with one method in it.

That is the strongest form of interface segregation available in any mainstream language, and it is worth naming explicitly in an interview: **in Python the narrow interface is free, and it can be declared where it is used.**

How to spot a fat interface

1. **Implementations that raise `NotImplementedError`.** The loudest signal, and it is grep-able.
2. **Callers that use two or three methods of a fifteen-method type.** Count the methods a function actually calls against the methods its parameter type declares.
3. **Test fakes that are enormous.** If faking a dependency means writing twelve stub methods to test one call, the interface is too wide. This is the cost people feel first.
4. **The interface's name is a noun for a thing rather than a capability.** `Repository`, `Manager`, `Service`, `Machine`, `Context`.
5. **A method added for one caller.** Look at the version history: if methods arrive one at a time, each for a single new use, the interface is a bag rather than a contract.

4 The picture

The keys:

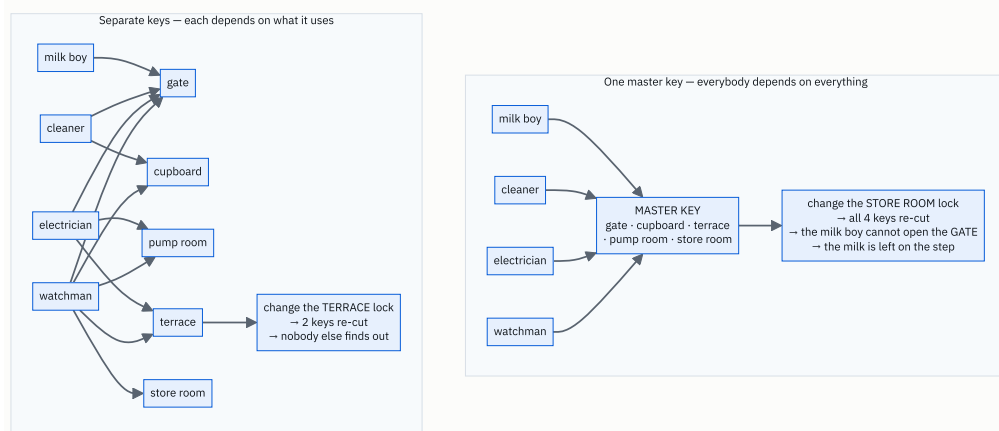


FIGURE 58.1

What to notice: the watchman still holds everything, and that is fine — he genuinely uses everything. Segregation is not about giving everyone less; it is about nobody being *forced* to hold what they do not use. And notice the blast radius at the bottom of each half: four against two.

The fat repository, and the split:

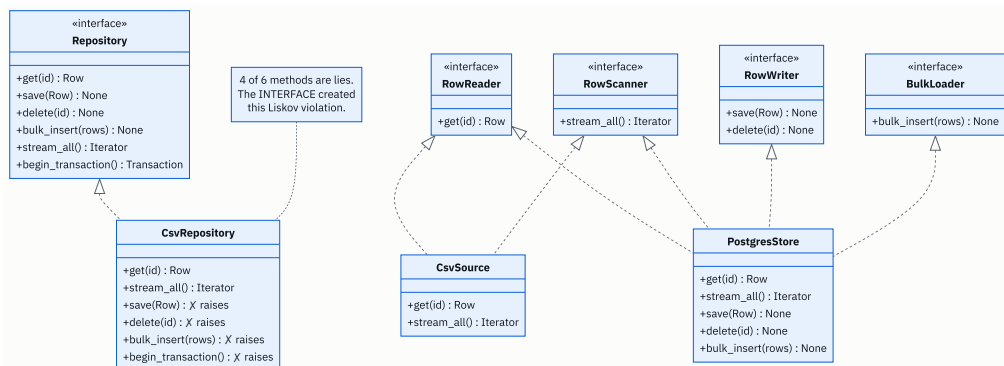


FIGURE 58.2

What to notice: `CsvSource` now implements two interfaces honestly rather than one interface dishonestly. Nothing raises. And a function that only reads declares `RowReader`, so adding a method to `BulkLoader` cannot possibly reach it.

What each caller actually needs:

caller	methods it CALLS	type it DECLARES	forced dependency
monthly_report()	1	Repository (6)	5 unused
export_all_to_csv()	1	Repository (6)	5 unused
admin_delete_user()	2	Repository (6)	4 unused
nightly_import()	1	Repository (6)	5 unused
checkout_flow()	3	Repository (6)	3 unused
total unused dependencies: 22			
after splitting into role interfaces:			
monthly_report()	1	RowReader (1)	0
export_all_to_csv()	1	RowScanner (1)	0
admin_delete_user()	2	RowWriter (2)	0
nightly_import()	1	BulkLoader (1)	0
checkout_flow()	3	RowReader + RowWriter	0

What to notice: twenty-two forced dependencies became zero, and no implementation changed. In Python, declaring those four narrow protocols is about twelve lines and the provider is never touched.

5 How it actually works

The split, in four steps

Step 1 — list the callers and the methods each one actually calls. This is the whole diagnosis and it takes ten minutes with `grep`. Anything a caller does not call is a forced dependency.

Step 2 — group callers by the set of methods they use. Those groups are your role interfaces. Usually three or four fall out of a fifteen-method interface.

Step 3 — name each group after the role, not the thing. `UserReader`, `RowScanner`, `PriceQuoter`, `EventPublisher`. `-er` names keep them small.

Step 4 — change the *parameter types*, not the implementations. In Python nothing about the concrete classes changes, because `Protocol` is structural. That is the step that makes this a cheap refactor rather than an expensive one.

Before

```
from typing import Iterator, Protocol

class Repository(Protocol):
    """A header interface: named after the thing, and it grows forever."""

    def get(self, row_id: str) → dict: ...
    def save(self, row: dict) → None: ...
    def delete(self, row_id: str) → None: ...
    def bulk_insert(self, rows: list[dict]) → None: ...
    def stream_all(self) → Iterator[dict]: ...
    def begin_transaction(self) → object: ...

def monthly_report(repo: Repository, user_ids: list[str]) → list[dict]:
    """Calls exactly ONE method and depends on six."""
    return [repo.get(vid) for vid in user_ids]
```

The test for `monthly_report` needs a fake `Repository`, which means six stub methods to exercise one.

After

```
class RowReader(Protocol):
    """A role interface, declared where it is USED."""
    def get(self, row_id: str) → dict: ...

class RowScanner(Protocol):
    def stream_all(self) → Iterator[dict]: ...

class RowWriter(Protocol):
    def save(self, row: dict) → None: ...
    def delete(self, row_id: str) → None: ...

class BulkLoader(Protocol):
    def bulk_insert(self, rows: list[dict]) → None: ...
```

```
def monthly_report(reader: RowReader, user_ids: list[str]) → list[dict]:
    """Depends on exactly what it calls."""
    return [reader.get(uid) for uid in user_ids]

def export_all(scanner: RowScanner, out) → int:
    count = 0
    for row in scanner.stream_all():
        out.write(row)
        count += 1
    return count

def admin_delete(writer: RowWriter, row_id: str) → None:
    writer.delete(row_id)
```

And the implementations, unchanged and honest:

```
class CsvSource:
    """Satisfies RowReader and RowScanner. Claims nothing else. Nothing raises."""

    def __init__(self, rows: list[dict]) → None:
        self._rows = {r["id"]: r for r in rows}

    def get(self, row_id: str) → dict:
        return self._rows[row_id]

    def stream_all(self) → Iterator[dict]:
        yield from self._rows.values()

class PostgresStore:
    """Satisfies all four. It genuinely can do all four."""

    def get(self, row_id: str) → dict: ...
    def stream_all(self) → Iterator[dict]: ...
    def save(self, row: dict) → None: ...
    def delete(self, row_id: str) → None: ...
    def bulk_insert(self, rows: list[dict]) → None: ...
```

Note that neither class inherits from anything. `CsvSource` satisfies `RowReader` because it has a `get` of the right shape, and a type checker verifies that at the call site. **The provider was never edited.**

The test, which is where you feel the difference

```
def test_monthly_report_fetches_each_user() → None:
    class OneMethodFake:
        # the ENTIRE fake
        def get(self, row_id: str) → dict:
            return {"id": row_id, "total": 100}

    rows = monthly_report(OneMethodFake(), ["a", "b"])
    assert [r["id"] for r in rows] == ["a", "b"]
```

Four lines. Against the fat interface, this fake needs six methods, five of which are `raise NotImplementedError` — and a reviewer cannot tell at a glance whether those five are stubs because they are unused or stubs because somebody forgot.

Composing interfaces when a caller needs two

```
class ReadWriteStore(RowReader, RowWriter, Protocol):
    """Compose rather than widen. Callers that need both declare this."""

    def checkout(store: ReadWriteStore, order_id: str) → None:
        order = store.get(order_id)
        order["status"] = "paid"
        store.save(order)
```

Inheriting protocols composes them. This is how you keep small interfaces without forcing every caller to take four parameters — and crucially, `ReadWriteStore` is still only three methods, not six.

Where real systems do this

- **Go's `io` package** is the canonical success. `io.Reader` has exactly one method, `Read`. `io.Writer` has one, `Write`. `io.ReadWriter` is those two composed. Almost the entire Go standard library is built out of one-method interfaces, and it is the most-cited example of this principle working at scale.
- **Python's `collections.abc`** — `Iterable` (one method), `Container` (one), `Sized` (one), `Sequence`, `MutableSequence`, `Mapping`, `MutableMapping`. The read/write split is exactly interface segregation, and it is also the fix for the Liskov problem from [day 057](#).
- **Java's `Closeable`, `Comparable`, `Serializable`, `Runnable`** — one method each, mixed in as needed. Against that, `java.sql.ResultSet` has over a hundred and eighty methods and is the standard example of a fat interface.
- **`typing.Protocol` declared at the consumer** — the Python idiom described above, and the reason narrow interfaces cost nothing here.
- **Least privilege in security** — the same principle, applied to permissions rather than methods. An IAM role with `s3:GetObject` rather than `s3:*` is interface segregation, and it fails the same way when somebody grants the master key because it is quicker.

6 The numbers

Forced dependencies, counted

Repository: 6 methods, 5 callers.

caller	methods called	methods depended on	unused
-----	-----	-----	-----
monthly_report	1	6	5
export_all_to_csv	1	6	5
admin_delete_user	2	6	4
nightly_import	1	6	5
checkout_flow	3	6	3

		total	22

after splitting into 4 role interfaces: total 0
cost of the split, in Python: ~12 lines of Protocol declarations,
0 changes to any implementation.

Churn, priced

Adding one method to the interface.

fat interface (6 methods, 6 implementations, 4 test fakes):

implementations to update	: 6
test fakes to update	: 4
files re-reviewed	: 10
services redeployed (compiled	
language, shared library)	: every service importing the interface
engineer-hours	: ~4

role interfaces:

the new method goes on ONE role interface (or a new one)	
implementations that actually offer it	: 2
test fakes affected	: the 1 that fakes that role
engineer-hours	: ~0.5

Test-fake size

faking a dependency to test ONE call:

fat Repository (6 methods)	: 6 stub methods, ~18 lines,
	5 of them raise NotImplementedError
RowReader (1 method)	: 1 method, 4 lines

Across 40 tests that each need a fake:

fat	: 40 x 18 = 720 lines of stub code
narrow	: 40 x 4 = 160 lines

The 720-line number is the one that persuades people, because everyone has written those stubs.

`NotImplementedError`, counted

```
grep -rn "NotImplementedError" src/

before : 23 hits
        of which 19 are "this implementation doesn't support that method"
        i.e. 19 Liskov violations CREATED BY the interface

after  : 4 hits, all in genuine abstract base classes where a subclass
        is required to supply the method
```

That `grep` is a real diagnostic and worth naming in an interview: **most `NotImplementedError`s in a codebase are interface segregation failures.**

Where it stops paying

```
splitting a 6-method interface into 6 one-method interfaces:

protocols to declare      : 6
callers needing 3 of them : must declare a composed protocol anyway
names to invent           : 6, several of them awkward (RowDeleter?)
reader's question
    "what can this thing do?" : now answered by reading 6 files

The gain from 4 interfaces to 6 is near zero;
the naming and reading cost is real.
```

7 The trade-offs

What splitting costs you

More names, and some of them are bad. `RowReader` is fine. `RowDeleter` is not really a role anybody has. When the name is awkward, that is a signal the split is too fine.

More types to trace. A reader asking “what can this store do?” now has to find every protocol it satisfies rather than reading one interface. In Python that is worse than in Java, because structural typing means nothing in the class declares what it implements.

Composed protocols multiply. A caller needing read and write declares `ReadWriteStore`; one needing read and bulk declares something else. Left unchecked you get a combinatorial pile of composed protocols, which is its own mess.

When one interface is right

I would not split if every client uses every method. That is not a fat interface, it is a cohesive one — and cohesion is the other half of the argument, exactly as in [day 055](#). A `Money` or a `DateRange` with eight methods that callers use together is fine.

I would not split if the interface has one implementation and two callers. There is nothing to protect against yet; wait for the second implementation or the third caller, which is the same “wait for the second” rule as [day 056](#).

I would not split down to one method each as a matter of policy. Go does this and it works because the standard library was designed that way from the start; retrofitting it onto an application produces a lot of interfaces named after verbs and a codebase nobody can navigate. Three or four role interfaces out of a fifteen-method type is usually the right granularity.

The genuine tension with the other principles

Interface segregation pushes towards **many small types**. Single responsibility pushes towards **cohesive units**. Those can pull against each other, and the resolution is to remember what each one is about: **SRP is about who changes the code; ISP is about who calls it**. A class can have one reason to change and still expose three different roles to three different callers — a `Postgres store` is one responsibility and satisfies four role interfaces, and that is correct, not contradictory.

The honest limit

Interface segregation reduces *compile-time and review-time* coupling. It does not reduce runtime coupling: `monthly_report` declared against `RowReader` still receives a whole `PostgresStore` at runtime, with all its methods, and if that object is broken the report still breaks. What you have bought is that a *change* to the writing methods cannot reach the reporting module, and that the reporting module's test needs a four-line fake. That is worth having, and it is worth being precise about rather than claiming isolation you have not achieved.

Where it sits with the other four

Interface segregation is largely the **fix** for Liskov violations ([day 057](#)): an implementation raising `NotImplementedError` is almost always an interface promising more than that implementation can deliver, and splitting the interface removes the lie. It is also what makes open/closed ([day 056](#)) cheap: a small interface means a new implementation is a small amount of work, whereas a fifteen-method interface makes every new implementation a project. And dependency inversion ([day 059](#)) is what points the callers at these interfaces in the first place.

8 In the interview

How it gets asked

- “Why is a single large interface a problem?” — the direct form. Give both costs: forced lies, and churn.
- “You have a `Repository` with fifteen methods. What would you do?” — the practical form, and the right first move is to count what each caller actually calls.
- “What does `NotImplementedError` in an implementation tell you?” — that the interface is too wide, not that the implementer was lazy.
- “How is this different from single responsibility?” — the question that separates people who memorised SOLID from people who use it. SRP is about who *changes* it; ISP is about who *calls* it.

- “Isn’t this just more interfaces?” — the pushback. Answer with the fake-size and churn numbers, and concede the naming cost.

What to say out loud, in the first ninety seconds

1. **State it in terms of the caller.** “No client should be forced to depend on methods it doesn’t use. And ‘depend on’ is stronger than ‘call’ — if my function takes a fifteen-method type and calls one, a change to any of the other fourteen changes my declared type.”
2. **Give the first cost concretely.** “Implementations get forced to lie. A read-only CSV-backed repository has to stub `save` and `delete` with `NotImplementedError`, which is a Liskov violation the interface created.”
3. **Give the second cost with a number.** “And churn: adding one method means editing six implementations and four test fakes, including the ones for callers that will never touch it.”
4. **Name the diagnosis.** “I’d list the callers and count the methods each actually calls. In one codebase that was twenty-two forced dependencies across five callers of a six-method interface.”
5. **Give the Python answer, because it is the strong one.** “In Python I’d declare a narrow `Protocol` at the consumer — `class UserReader(Protocol): def get(...)` — and because Protocols are structural, the existing repository already satisfies it. No implementation changes at all; it’s about twelve lines.”

The follow-ups

“How is this different from single responsibility?” They point in similar directions but they are answering different questions, and the distinction is worth being precise about. Single responsibility is about **who changes the code**: I group methods by which stakeholder would come and ask for them to be different, and split when two different people own two different parts. Interface segregation is about **who calls the code**: I group methods by which client uses them, and split so that no caller is declared against methods it never invokes. So they can pull in opposite directions and both be right. A `PostgresStore` has exactly one reason to change — the platform team owns how rows are persisted — so by SRP it is one class and should stay one class. But it has four different kinds of caller, so by ISP it should satisfy four narrow role interfaces. One implementation class, four interfaces, and neither principle is being violated. The mistake would be to conclude from ISP that `PostgresStore` should be split into four classes; that would be applying a rule about interfaces to an implementation. The short form I would give: **ISP splits the contract, SRP splits the code.**

“Isn’t this just adding more interfaces to maintain?” It is more names, and I would concede that some of them are awkward — `RowReader` is a fine role, `RowDeleter` is not really a role anybody has, and when the name is hard the split is probably too fine. What it buys is two things I can measure. The first is churn: adding a method to a six-method interface with six implementations and four test fakes is ten files and about four hours; adding it to the one role interface that needs it is two implementations and one fake, about half an hour. The second is test cost, and this is the one everybody has felt — faking a six-method interface to exercise one call is about eighteen lines of stubs, five of them `raise NotImplementedError`, and a reviewer cannot tell whether those five are deliberately unused or accidentally forgotten. Against a one-

method protocol the fake is four lines. Across forty tests that is seven hundred lines of stub code against a hundred and sixty. And in Python the cost side is unusually low, because `Protocol` is structural — I declare the narrow interface in the consuming module and the provider is never touched, so the whole refactor is about twelve lines and zero changes to any implementation. In Java it is more expensive, because the implementer has to declare `implements`, and I would weigh it differently there.

“How do you decide the granularity?” By grouping callers, not by decomposing the interface. I list every caller and the methods it actually calls — that is a `grep` and about ten minutes — and then look for clusters. Usually a fifteen-method interface has three or four distinct usage patterns: things that read one row, things that scan everything, things that write, things that bulk-load. Those clusters are the interfaces, and I name them after the role rather than the thing, with `-er` names, because a name like `UserReader` makes it hard to justify adding `delete` while a name like `UserRepository` makes it easy. What I would not do is go all the way to one method per interface as a matter of policy. Go does that and it works beautifully, but Go’s standard library was designed that way from the beginning; retrofitting it onto an application gives you a large number of verb-shaped interfaces and a codebase where answering “what can this thing do?” means reading six files. And when a caller genuinely needs two roles, I compose the protocols rather than widening either one — inheriting two Protocols gives me a three-method `ReadWriteStore` rather than going back to the six-method original. The stopping rule is that the gain from four interfaces to six is near zero and the naming cost is real.

A model answer

“The principle is that no client should be forced to depend on methods it doesn’t use — and ‘depend on’ is stronger than ‘call’. If my reporting function takes a `Repository` with six methods and calls one, a change to any of the other five changes the type I’m declared against, so it reaches me.

There are two distinct costs and I’d give both. The first is that implementations get forced to lie. If the interface promises `get`, `save`, `delete`, `bulk_insert`, `stream_all` and `begin_transaction`, then a read-only CSV-backed source has to stub four of those with `NotImplementedError` — and that’s a Liskov violation created by the interface, not by the implementer. In fact I’d say most `NotImplementedError`s in a codebase are interface segregation failures; grepping for them is a genuine diagnostic.

The second cost is churn. Adding one method to that interface means updating six implementations and four test fakes, and in a compiled language redeploying every service that imports it — including a reporting service that only ever calls `get`. That’s the practical harm.

The diagnosis is mechanical: list the callers, count the methods each actually calls. On the codebase I’m thinking of, five callers of a six-method interface gave twenty-two forced dependencies. Those callers cluster — some read one row, some scan everything, some write — and each cluster becomes a role interface, named after the role: `RowReader`, `RowScanner`, `RowWriter`. The `-er` naming keeps them small, because it’s hard to justify putting `delete` on something called `RowReader`.

In Python the fix is unusually cheap, and I’d point that out. `typing.Protocol` is structural, so I declare the narrow interface in the *consuming* module and the existing `PostgresStore` already satisfies it — it has a `get` of the right shape. No implementation is edited at all. That’s about twelve lines for the whole refactor, and the test fake for the reporting module goes from eighteen lines of stubs to a four-line class with one method.

Where I’d stop: I wouldn’t go to one method per interface as policy — three or four roles out of fifteen methods is usually right, and past that the names get awkward and nobody can tell what a class can do. And if every caller genuinely uses every method, that’s a cohesive interface, not a fat one, and I’d leave it alone.”

9 Recall card

- **No client should be forced to depend on methods it does not use** — and *depend on* is stronger than *call*: taking a 15-method type and calling one means all 15 changes reach you. Prefer **role interfaces** named after a capability (`UserReader`, `RowScanner`) over **header interfaces** named after a thing (`Repository`, `Machine`). `-er` names keep them small.

- **Two costs, and give both.** Implementations are **forced to lie** — `CsvRepository.save` raising `NotImplementedError` is a Liskov violation *created by the interface*. And **churn**: one new method = 6 implementations + 4 fakes + a redeploy, for callers that will never invoke it. Mr Kulkarni changed the store-room lock and stopped the milk.
- **The diagnosis is a grep.** List every caller and count methods called against methods depended on — 5 callers of a 6-method interface gave **22 forced dependencies**, and 19 of 23 `NotImplementedError`s were ISP failures. Test fakes are where you feel it: 18 lines of stubs against 4, times 40 tests.
- **In Python the narrow interface is free.** `typing.Protocol` is **structural**, so declare it in the *consuming* module — the provider is never edited. ~12 lines, 0 implementation changes. Compose with protocol inheritance (`class ReadWriteStore(Reader, RowWriter, Protocol)`) rather than widening.
- **ISP splits the contract; SRP splits the code.** One `PostgresStore` class (one reason to change) satisfying four role interfaces (four kinds of caller) is correct, not contradictory. Stop at three or four roles — going to one method each retrofits Go's design onto an app and nobody can answer "what can this thing do?"

PRACTICE

Practice — Custom comparators and sorting by keys

DSA topic: Custom comparators and sorting by keys **System design topic:** Interface segregation

Code these, in this order

One rule for the whole set: **say the key out loud as a sentence before you type it.** “Negative score first so high scores come first, then name so it only decides ties.” If you cannot say it, you do not have it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sort Array By Parity II	LeetCode 922 (Easy)	That not every ordering problem is a sort — sometimes the key is a placement rule.
2	Sort Characters By Frequency	LeetCode 451 (Medium)	Two keys in opposite directions, with a count you have to build first.
3	Largest Number	LeetCode 179 (Medium)	The one problem where no per-element key exists. <code>cmp_to_key</code> , and the sign convention.
4	Reorder Data in Log Files	LeetCode 937 (Medium)	A tuple key mixing types and a group that must keep its original order — stability in production.

On problem 2, hit the wall on purpose

Sort by frequency descending, then character ascending. Do it with one tuple key (you can — frequency is a number). Then invert the requirement to “character descending, frequency ascending”, try the tuple key, paste the `TypeError`, and solve it in two passes. Say which pass goes first and why.

On problem 3, say the convention out loud before you write it

The comparison function returns a **negative** number when the **first** argument should come first. Write that on the board. Then handle the `[0, 0]` case, which is the test that catches people — `"00"` is not a valid answer.

On problem 4, find the stability requirement

Digit-logs must stay in their original relative order. That is not something you sort — it is something you must not disturb. Say out loud how your key achieves that, and what would happen with an unstable sort.

The key-function drill

Write each in one line, then check it:

1. Words by length, then alphabetically.

2. Words by length descending, then alphabetically ascending.
3. Files by extension, then by size descending.
4. People by city ascending, age descending, name ascending.
5. Strings by their last character.
6. Tuples by the second element, falling back to the first.
7. Dictionaries by `d["score"]`, treating a missing score as zero.
8. Names case-insensitively, with ties broken by the original case-sensitive value.

The opposite-directions drill

For each, say which of the three techniques you would use — negate, two passes, or a wrapper — and why the other two are wrong or unavailable:

1. Score descending, name ascending.
2. Name descending, score ascending.
3. Date descending, id ascending.
4. Department descending, salary descending.
5. Priority ascending, created-at descending, title ascending.
6. Status by a custom order, then name descending.

The break-it drill

Run each and record the output or the error:

1. `sorted(students, key=lambda s: (s.score, s.name), reverse=True)` — what is wrong with the output, and why is there no error?
2. `sorted(people, key=lambda p: (-p.name, p.age))` — paste the error.
3. `sorted(words, key=len(words))` — paste the error.
4. `sorted([Student("Asha", 90), Student("Bala", 85)])` on a plain dataclass — paste the error.
5. `sorted(tickets, key=lambda t: STATUS_ORDER[t["status"]])` with one unknown status — paste the error, then fix it two ways and say which you prefer.
6. A comparator that never returns 0. Sort ten items with it and say what guarantee you have lost.

The call-count drill

1. Instrument a key function with a counter and sort 1,000 elements. How many calls?
2. Do the same with `cmp_to_key`. How many calls now?
3. Compute the ratio, and say what it would be at a million elements.
4. Time `key=attrgetter("score")` against `key=lambda s: s.score` on 500,000 records.
5. Estimate the memory used by the keys when sorting a million records with a two-element tuple key.

6. Given all that, state the rule for when to use `cmp_to_key` in one sentence.

The determinism drill

1. Sort 1,000 tickets by status alone, where 400 share a status. Shuffle the input and sort again. Do you get the same output?
2. Add a unique tie-break and repeat. What changed?
3. Say what class of test failure this prevents.
4. Sort with `key=lambda x: random.random()`. What does Python guarantee about the result? What should you use instead?

The interface-segregation drill

Here is the interface. Do not refactor it yet.

```
class OrderStore(Protocol):
    def get(self, order_id: str) → Order: ...
    def save(self, order: Order) → None: ...
    def delete(self, order_id: str) → None: ...
    def find_by_customer(self, customer_id: str) → list[Order]: ...
    def stream_all(self) → Iterator[Order]: ...
    def bulk_import(self, orders: list[Order]) → None: ...
    def begin_transaction(self) → Transaction: ...
    def rebuild_index(self) → None: ...
```

And its callers:

<code>order_confirmation_email(store, order_id)</code>	calls: <code>get</code>
<code>customer_order_history(store, customer_id)</code>	calls: <code>find_by_customer</code>
<code>nightly_csv_export(store, path)</code>	calls: <code>stream_all</code>
<code>admin_delete_order(store, order_id)</code>	calls: <code>get</code> , <code>delete</code>
<code>checkout(store, order)</code>	calls: <code>get</code> , <code>save</code> , <code>begin_transaction</code>
<code>data_migration(store, rows)</code>	calls: <code>bulk_import</code> , <code>rebuild_index</code>

1. Build the table: for each caller, methods called against methods depended on. Total the forced dependencies.
2. Group the callers by the set of methods they use. How many clusters?
3. Name each cluster as a role interface. Use `-er` names. Reject any name you find awkward and say why.
4. Write the `Protocol` declarations. How many lines?
5. Rewrite the six caller signatures.
6. Which implementations had to change? Answer honestly, and say what feature of Python makes that answer what it is.
7. Write the test fake for `order_confirmation_email` before and after. Count the lines of each.
8. Someone wants to add `archive(order_id)`. Which interface does it go on, and how many files change now compared with before?

The NotImplementedError drill

For each, say whether the `NotImplementedError` is an interface segregation failure or something legitimate:

1. `CsvRepository.save` raising, because the file is read-only.
2. An abstract base class's `body()` raising, with subclasses required to supply it.
3. `Robot.eat` raising, in a `Worker` interface.
4. `InMemoryCache.persist` raising, because there is no disk.
5. A `Shape.volume()` raising for two-dimensional shapes.
6. A method in a `Protocol` body — `def get(self) → Row: ...` — which is not raising at all.

The role-naming drill

Rename each header interface into one or more role interfaces, and say which callers would take each:

1. `UserService` with 12 methods.
2. `FileManager` with `read`, `write`, `delete`, `list`, `compress`, `upload`.
3. `Machine` with `print`, `scan`, `fax`, `staple`.
4. `PaymentProvider` with `charge`, `refund`, `list_transactions`, `configure_webhook`.
5. `Cache` with `get`, `set`, `delete`, `flush_all`, `stats`.
6. `Logger` with `debug`, `info`, `warn`, `error`, `set_level`, `add_handler`.

Two of those six are probably fine as they are. Name them and defend it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Sort by score descending, then by name ascending.* The tuple key with the negated number, read aloud as a sentence, why `reverse=True` is the trap, and what you would do if the descending field were a string.
 2. *Why is a single large interface a problem?* Both costs — forced lies producing Liskov violations, and churn reaching callers that do not care — with the forced-dependency count, and then the Python `Protocol`-at-the-consumer answer.
 3. *When would you need a comparator instead of a key?* The largest-number example, why no per-element key exists there, the sign convention, and the cost ratio that makes it the last resort.
-

Before you move on

- [] I can write the score-descending-name-ascending key in one line and read it aloud as a sentence.
- [] I ran the `reverse=True` version and can say exactly which part of the output is wrong.
- [] I hit the string-negation `TypeError` and solved it with two passes, least significant first.
- [] I instrumented the call count for `key` and for `cmp_to_key` and can quote the ratio.
- [] I can state the `cmp_to_key` sign convention without hesitating.
- [] I built the forced-dependency table for `OrderStore` and can give the total.
- [] I can say what makes narrow interfaces nearly free in Python and expensive in Java.
- [] I answered all three questions above out loud.

Sorting revision and mock round

DATA STRUCTURES AND ALGORITHMS

Sorting revision and mock round

You can solve two unseen sorting problems and defend your algorithm choice.

SYSTEM DESIGN

Dependency inversion

You can invert a dependency so the high-level policy stops importing the low-level detail.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *Your service imports the MySQL driver directly. What is wrong with that?*

1 What this is, and why they ask it

This closes the sorting phase. Over eight days you have met the three quadratic sorts, merge sort, quicksort, quickselect, the three non-comparison sorts, stability, and key functions. Today is not new material. It is the day you find out whether you can *reach* any of it while somebody is watching and the clock is running, which is a different skill and the only one that gets tested.

They ask it as an unseen problem with no hints because that is what the coding round actually is. You will not be asked “implement merge sort” as often as you will be asked something that turns out to need a sort, and the marks come from three things nobody practises: saying the decision out loud before coding, choosing the right algorithm and being able to defend it in arithmetic, and continuing to talk when you are stuck. The material you already have. What this day adds is the performance — the same distinction as knowing a piece of music and being able to play it through in front of someone without stopping.

2 The story

Anjali has been learning the keyboard for four years, and by the third year she could play the piece her teacher had set for the December programme completely, from beginning to end, without a mistake. She knew it. That was not in question.

Her teacher, Mrs Pinto, made her play it in the front room one Saturday with three people sitting there — Mrs Pinto’s husband, a neighbour, and a girl from the next batch who had come early.

Twenty seconds in, Anjali played a wrong note. And she did what she had done a thousand times in her own room: she stopped, made a small annoyed sound, and started again from the beginning.

Mrs Pinto let her get to about the same place and then stopped her and said, do it again, and this time whatever happens, do not stop.

She played it again and hit the same wrong note in the same place, and she carried on, and it was horrible for about two seconds and then it was fine. The people in the room could not really tell. The neighbour said afterwards that it was lovely and clearly meant it.

That was the lesson, and Anjali still describes it as the day she found out she had been practising the wrong thing. Four years of playing alone in her room had taught her the notes perfectly and had also taught her, without anybody intending it, that a mistake means going back to the start. In her room that costs nothing. On a stage, in front of people, it is the only thing that actually goes wrong.

So for the six weeks before December they changed what practice meant. Twice a week she played the whole thing through, once, with somebody in the room, and she was not allowed to stop for any reason. Her mother sat there some days. Once it was just her little brother, who was not listening.

She still made mistakes in December. Two that she can name. Nobody else noticed either of them, because she did not stop, and because she had made her peace with the fact that she was going to make some.

3 The idea in plain English

Anjali knew the piece. What she had not practised was playing it in front of somebody without stopping, and that turned out to be the thing the performance actually tested.

A coding round is the same. You know merge sort. What you have not practised is deciding out loud, under a clock, whether this problem wants a sort at all — and carrying on talking when you get something wrong.

So this lesson is the whole phase compressed into a decision procedure you can run out loud, followed by two unseen problems worked end to end with the thinking left in.

The decision procedure, in the order to run it

Step 0 — Ask what the data is. Before anything else. How many elements, what type are the keys, what is the range, is it nearly sorted, does it fit in memory, can the caller's list be modified. Half the sorting decisions are made by the answers, and asking is itself worth marks.

Step 1 — Ask whether sorting helps at all. From [day 051](#): *would this be easier if the input were in order?* Sorting buys four things — equal values adjacent, near values adjacent, binary search and early exit, and a correct greedy order. If you want none of those, do not sort.

Step 2 — Check the four reasons not to. A hash set already gives $O(n)$ · you only want the top k · you need the original positions · the data is a stream.

Step 3 — Decide what you actually need. This is the step people skip, and it is where the marks are:

the whole order	→ a sort
one value (the k -th)	→ quickselect, $O(n)$ expected
the top k , unordered	→ quickselect, $O(n)$ expected
the top k , in order	→ heap of size k , $O(n \log k)$
the top k from a stream	→ heap of size k , $O(n \log k)$, $O(k)$ space
just "are there dupes?"	→ a set, $O(n)$. Do not sort.

Step 4 — Choose the sort, and be able to defend it:

default, in Python	→ sorted() / .sort()	Timsort, stable, C
keys are bounded small integers	→ counting sort	$O(n + k)$
keys are fixed-width integers	→ radix sort	$O(d(n + b))$
need a guaranteed worst case	→ merge sort	no bad input
need stability	→ merge sort or Timsort	
memory is tight, array in memory	→ quicksort	$O(1)$ extra
data is a linked list	→ merge sort	no random access needed
data does not fit in memory	→ external merge sort	
$n < \sim 50$, or nearly sorted	→ insertion sort	

Step 5 — Say the cost, with the arithmetic. Not “it’s $n \log n$ ”. “*Twenty million comparisons at a million elements, against five hundred billion for the all-pairs version.*”

The one-page comparison

SORT	TIME	SPACE	STABLE	IN PLACE	USE IT WHEN
Insertion	$O(n^2)$, $O(n)$ best	$O(1)$	Yes	Yes	$n < 50$, nearly sorted, values arriving one at a time
Selection	$O(n^2)$ always	$O(1)$	No	Yes	Writes are far more expensive than reads
Bubble	$O(n^2)$, $O(n)$ best	$O(1)$	Yes	Yes	Never. Learn it to explain it
Merge	$O(n \log n)$ always	$O(n)$	Yes	No	Guaranteed worst case, stability, linked lists, external
Quick	$O(n \log n)$ avg, $O(n^2)$ worst	$O(1)$ + $O(\log n)$ stack	No	Yes	Arrays in memory, when speed matters most
Heap	$O(n \log n)$ always	$O(1)$	No	Yes	Guaranteed worst case <i>and</i> $O(1)$ space
Counting	$O(n + k)$	$O(n + k)$	Yes	No	Integer keys, small known range
Radix	$O(d(n + b))$	$O(n + b)$	Yes	No	Fixed-width integers or equal-length strings
Bucket	$O(n)$ avg, $O(n^2)$ worst	$O(n)$	Yes	No	Values known to be uniformly spread
Timsort	$O(n \log n)$, $O(n)$ best	$O(n)$	Yes	No	Everything, in Python

Heapsort appears here for completeness — it is the answer to “guaranteed $O(n \log n)$ with $O(1)$ space”, and it arrives properly on [day 113](#).

The five sentences that carry the phase

If you remember nothing else on the day:

1. “*Would this be easier if the input were in order?*” — asked before writing anything.
2. “*If two values are closest, nothing lies between them, so after sorting they are neighbours.*”

3. *“Merge sort is n per level times $\log n$ levels. Quicksort is the same, unless the pivot is always extreme — which on sorted input with a fixed pivot, it is.”*
4. *“Quickselect discards one side, so the work is $n + n/2 + n/4 \dots = 2n$.”*
5. *“Stable means equal elements keep their original order, which is what makes two-pass multi-key sorting work.”*

What a mock round is actually scoring

Four things, and only one of them is the code:

- **Did you clarify before coding?** Two or three questions about the data.
 - **Did you state the approach and its cost before writing?** Thirty seconds, out loud.
 - **Did you keep talking when stuck?** Silence is the failure mode. “I’m going to try X because Y; if that doesn’t work the fallback is Z” is a sentence you can always say.
 - **Did you test it out loud?** Empty input, one element, all equal, all reversed, duplicates.
-

4 The picture

The decision, drawn:

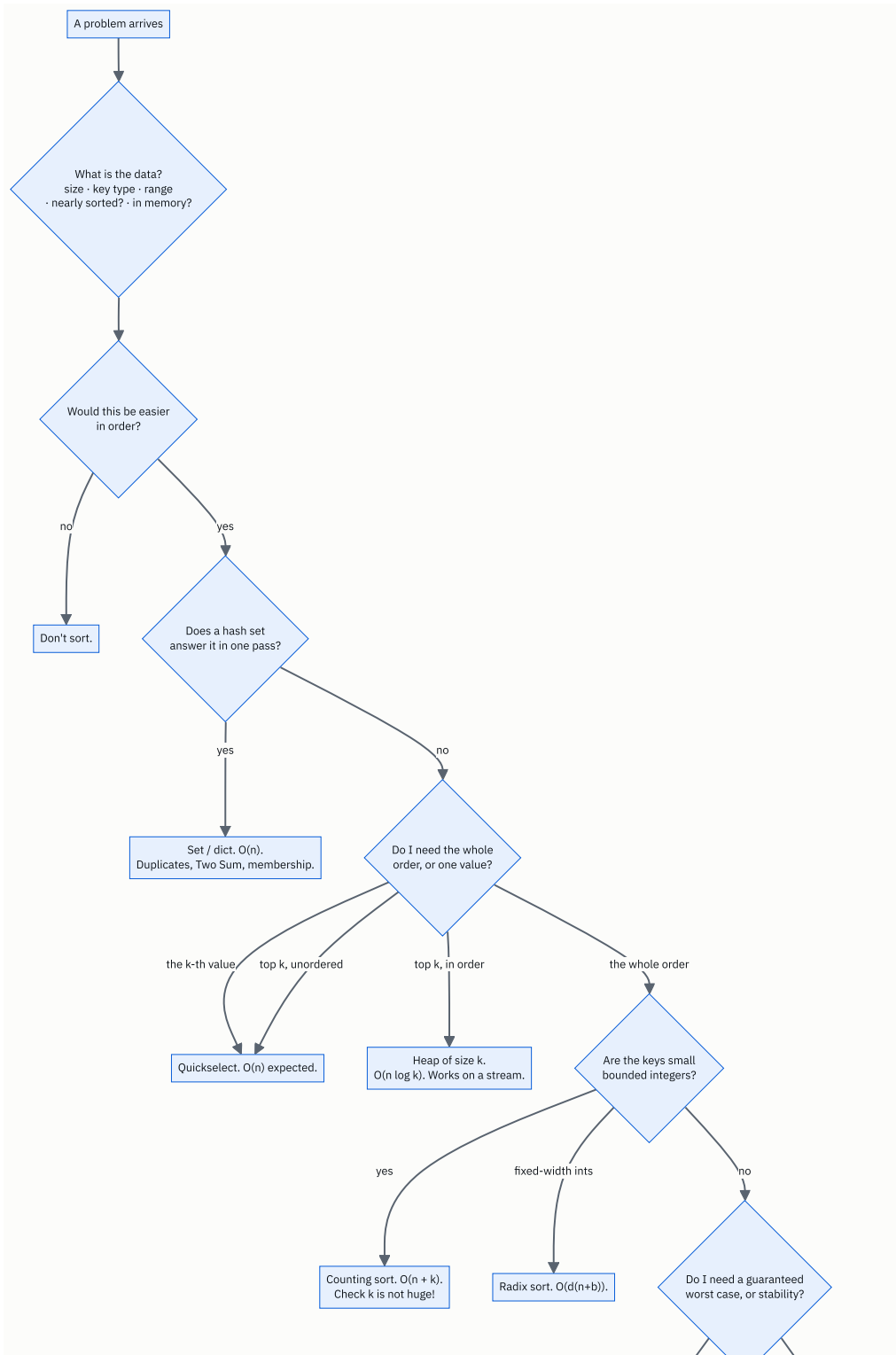




FIGURE 59.1

What to notice: five of the nine leaves are not “sort the array”. The phase is called sorting, and most of what it taught you is when to do something else.

The three $O(n \log n)$ sorts against each other, on the axes that decide:

	GUARANTEE	SPACE	STABLE	SPEED
merge sort	$n \log n$ always	$O(n)$	yes	1.0x
quicksort	n^2 worst case	$O(1) + O(\log n)$	no	2-3x faster
heapsort	$n \log n$ always	$O(1)$	no	$\sim 0.5x$ slower

Pick TWO of: guaranteed, in place, stable.

No comparison sort gives you all three.

merge = guaranteed + stable

heap = guaranteed + in place

quick = in place + fast (and gives up the guarantee)

What to notice: the last line is the sentence to say when asked “which would you use”. It is a trade with a shape, not a ranking.

Where the phase’s algorithms sit on cost:

```
n = 1,000,000

counting sort (k=100) ~2,000,000 ops ####
quickselect (k-th) ~3,400,000 #####
heap, k=10 ~3,300,000 #####
radix sort (base 256) ~8,000,000 #####
merge / quick / Timsort ~20,000,000 #####
insertion sort ~500,000,000,000 (off the chart by 25,000x)

and Timsort on ALREADY SORTED input: ~1,000,000 ##
```

What to notice: the last line. Timsort on nearly-ordered data beats everything except counting sort, for free, with no decision required. That is why “just call `sorted()`” is a strong default in Python and why the arithmetic has to be about *your* data, not about the textbook.

5 The code, built step by step

Two unseen problems, worked the way you should work them out loud. The thinking is left in on purpose — that is the part being practised.

Problem one: “Given an array of meeting intervals, find the minimum number of rooms needed.”

Step 0 — clarify. “Are intervals half-open, so a meeting ending at 10 and one starting at 10 don’t clash? How many meetings — hundreds or millions? Can I modify the input list?” Assume half-open, up to a hundred thousand, and yes.

Step 1 — would order help? Yes, obviously. But *which* order is the question, and this is the part to say aloud: “For merging intervals I’d sort by start. For packing the most non-overlapping meetings I’d sort by end. Here I want the maximum number overlapping at any instant, which is a different question again.”

Step 2 — the insight, stated before the code. “A room is needed at the moment a meeting starts and released at the moment one ends. So I only care about the times, not about which meeting they belong to. If I sort all the start times and all the end times separately and sweep through them, the answer is the maximum number of starts I have seen before the corresponding end.”

Step 3 — the code.

```
def min_meeting_rooms(intervals: list[list[int]]) → int:
    """The two-list sweep. O(n log n) for the sorts, O(n) for the sweep."""
    if not intervals:
        return 0
    starts = sorted(iv[0] for iv in intervals)
    ends = sorted(iv[1] for iv in intervals)
    rooms = most = 0
    e = 0
    for s in starts:
        while ends[e] ≤ s:          # every meeting that finished before this one starts
            rooms -= 1              # frees a room
            e += 1
        rooms += 1
        most = max(most, rooms)
    return most
```

Step 4 — the cost, out loud. “Two sorts of n , so $2 \times n \log n$. The sweep is one pass over starts, and `e` only ever moves forward, so the inner `while` is amortised $O(n)$ in total, not $O(n^2)$. At a hundred thousand meetings that is about 3.4 million operations.”

Step 5 — test it out loud. Empty list — returns 0. One meeting — 1. Two identical meetings — 2. `[[0, 30], [5, 10], [15, 20]]` — 2. Back-to-back `[[0, 10], [10, 20]]` — 1, and this is where the half-open clarification pays off, because `ends[e] ≤ s` rather than `<` is what makes it 1.

Problem two: “Given a million log lines with a severity and a timestamp, return them grouped by severity, most severe first, and in time order within each severity.”

Step 0 — clarify. “How many severity levels? Are the lines already roughly in time order? Do I need all of them, or the first page?” Assume five levels, roughly time-ordered because they came from a log file, and all of them.

Step 1 — recognise the shape. “That’s a two-key sort: severity descending by a custom order, and timestamp ascending.”

Step 2 — decide the technique, out loud. “Severity isn’t a number I can negate — it’s a name. But I have an order for it, so I’ll turn it into a rank with a lookup table, and then it’s an integer I can negate. That gives me a single tuple key and one pass.”


```
SEVERITY = {"critical": 4, "error": 3, "warn": 2, "info": 1, "debug": 0}

def order_logs(lines: list[dict]) → list[dict]:
    """Severity descending by rank, then timestamp ascending. One pass."""
    return sorted(lines, key=lambda line: (-SEVERITY.get(line["level"], -1),
                                          line["timestamp"]))
```

Step 3 — say what you protected against. “`.get(level, -1)` means an unknown severity sorts last instead of raising a `KeyError` in production. And the timestamp tie-break makes the output deterministic, which matters if a test compares it.”

Step 4 — the follow-up you know is coming. “There are only five severities, so I could bucket into five lists in one pass and sort each by time — that’s $O(n)$ for the bucketing plus the sorts. But the timestamps still need sorting, so it doesn’t change the class. The version worth mentioning is that the file is roughly time-ordered already, so Timsort will find long runs and this is closer to $O(n)$ than to $n \log n$ in practice.”

Step 5 — the alternative, if only the first page is wanted. “If they only want the first fifty lines, I’d use `heapq.nsmallest(50, lines, key=...)`, which is $O(n \log 50)$ and doesn’t sort the other 999,950.”

The reference implementations, for revision

```
"""The sorting phase on one page. Read the docstrings as a decision table."""

import heapq
import random
from functools import cmp_to_key

def insertion_sort(nums: list[int]) → None:
    """ $O(n^2)$ ,  $O(n)$  on nearly-sorted.  $O(1)$  space. STABLE. In place.
    Use for  $n < \sim 50$  or nearly-sorted data. Timsort uses it below 64."""
    for j in range(1, len(nums)):
        value = nums[j]
        i = j - 1
        while i ≥ 0 and nums[i] > value:      # guard first; strict > keeps it stable
            nums[i + 1] = nums[i]
            i -= 1
        nums[i + 1] = value

def merge_sort(nums: list[int]) → list[int]:
    """ $O(n \log n)$  ALWAYS.  $O(n)$  space. STABLE.
    n work per level, log n levels. Use for a guaranteed worst case,
    stability, linked lists, or data that does not fit in memory."""
    if len(nums) ≤ 1:
        return nums
    mid = len(nums) // 2
    left, right = merge_sort(nums[:mid]), merge_sort(nums[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:                # ≤ keeps it stable
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:]) # BOTH extends, or the tail is lost
    return out

def partition(nums: list[int], lo: int, hi: int) → int:
    """Lomuto, random pivot. Invariant:  $\text{nums}[\text{lo}:\text{i}] < \text{pivot}$ ,  $\text{nums}[\text{i}:\text{j}] \geq \text{pivot}$ ."""
    r = random.randint(lo, hi)
    nums[r], nums[hi] = nums[hi], nums[r]
    pivot, i = nums[hi], lo
    for j in range(lo, hi):
        if nums[j] < pivot:                  # strict <, or duplicates go quadratic
            nums[i], nums[j] = nums[j], nums[i]
            i += 1
    nums[i], nums[hi] = nums[hi], nums[i]
    return i

def quicksort(nums: list[int], lo: int = 0, hi: int | None = None) → None:
    """ $O(n \log n)$  expected,  $O(n^2)$  worst.  $O(1)$  space +  $O(\log n)$  stack. NOT stable.
    Recurse into the smaller side so the stack stays  $O(\log n)$ ."""
    if hi is None:
        hi = len(nums) - 1
    while lo < hi:
        p = partition(nums, lo, hi)
        if p - lo < hi - p:
            quicksort(nums, lo, p - 1)
            lo = p + 1                    # p, not p+1, is an infinite loop
        else:
            quicksort(nums, p + 1, hi)
            hi = p - 1

def quickselect(nums: list[int], target: int) → int:
    """The value at index 'target' if sorted.  $O(n)$  EXPECTED:  $n + n/2 + n/4 = 2n$ .
    k-th LARGEST is target = len(nums) - k. Mutates nums."""
    lo, hi = 0, len(nums) - 1
    while lo < hi:
```

```

        p = partition(nums, lo, hi)
        if p == target:
            return nums[p]
        if target < p:
            hi = p - 1
        else:
            lo = p + 1
    return nums[lo]

def counting_sort(nums: list[int], max_value: int) → list[int]:
    """O(n + k) where k is the RANGE, not the count. STABLE. O(n + k) space.
    A range of 2 billion is a MemoryError, not a slow sort."""
    counts = [0] * (max_value + 1)
    for x in nums:
        counts[x] += 1
    for v in range(1, max_value + 1):
        counts[v] += counts[v - 1]
    out = [0] * len(nums)
    for x in reversed(nums):
        counts[x] -= 1
        out[counts[x]] = x
    return out

def radix_sort(nums: list[int]) → list[int]:
    """O(d(n + b)). STABLE, and it DEPENDS on the per-digit sort being stable."""
    if not nums:
        return []
    out, exponent, largest = list(nums), 1, max(nums)
    while largest // exponent > 0:
        counts = [0] * 10
        for x in out:
            counts[(x // exponent) % 10] += 1
        for d in range(1, 10):
            counts[d] += counts[d - 1]
        buf = [0] * len(out)
        for x in reversed(out):
            digit = (x // exponent) % 10
            counts[digit] -= 1
            buf[counts[digit]] = x
        out, exponent = buf, exponent * 10
    return out

def kth_largest(nums: list[int], k: int) → int:
    """Three correct answers; this is the O(n) one."""
    return quickselect(list(nums), len(nums) - k)

def top_k_in_order(nums: list[int], k: int) → list[int]:
    """O(n log k), O(k) space, and the only one that works on a stream."""
    return heapq.nlargest(k, nums)

def largest_number(nums: list[int]) → str:
    """The one problem with no per-element key. Negative = first arg comes first."""
    def compare(a: str, b: str) → int:
        return -1 if a + b > b + a else (1 if a + b < b + a else 0)
    joined = "".join(sorted((str(n) for n in nums), key=cmp_to_key(compare)))
    return "0" if joined[0] == "0" else joined

def min_meeting_rooms(intervals: list[list[int]]) → int:
    """Mock problem 1. Two sorted lists of times, swept together."""
    if not intervals:
        return 0
    starts, ends = sorted(iv[0] for iv in intervals), sorted(iv[1] for iv in intervals)
    rooms = most = e = 0
    for s in starts:
        while ends[e] ≤ s:
            rooms -= 1
            e += 1
        rooms += 1

```

```

        rooms += 1
        most = max(most, rooms)
    return most

SEVERITY = {"critical": 4, "error": 3, "warn": 2, "info": 1, "debug": 0}

def order_logs(lines: list[dict]) → list[dict]:
    """Mock problem 2. A name turned into a rank, then negated. One tuple key."""
    return sorted(lines, key=lambda ln: (-SEVERITY.get(ln["level"], -1), ln["timestamp"]))

if __name__ == "__main__":
    print(merge_sort([38, 27, 43, 3, 9, 82, 10]))      # [3, 9, 10, 27, 38, 43, 82]
    print(counting_sort([3, 1, 4, 1, 5], 5))          # [1, 1, 3, 4, 5]
    print(radix_sort([329, 457, 657, 839, 436]))      # [329, 436, 457, 657, 839]
    print(kth_largest([3, 2, 1, 5, 6, 4], 2))         # 5
    print(top_k_in_order([3, 2, 1, 5, 6, 4], 3))       # [6, 5, 4]
    print(largest_number([3, 30, 34, 5, 9]))          # 9534330

    print(min_meeting_rooms([[0, 30], [5, 10], [15, 20]])) # 2
    print(min_meeting_rooms([[0, 10], [10, 20]]))       # 1 ← half-open
    print(min_meeting_rooms([]))                       # 0

    logs = [
        {"level": "info", "timestamp": 5},
        {"level": "critical", "timestamp": 9},
        {"level": "info", "timestamp": 2},
        {"level": "weird", "timestamp": 1},
    ]
    print([(l["level"], l["timestamp"]) for l in order_logs(logs)])
    # [('critical', 9), ('info', 2), ('info', 5), ('weird', 1)]

```

6 What it costs

The whole phase, priced at $n = 1,000,000$

	operations	wall clock (Python)	space
insertion sort	5.0×10^{11}	~10 days	$O(1)$
selection sort	5.0×10^{11}	~10 days	$O(1)$
merge sort (hand-written)	2.0×10^7	~8 s	$O(n)$
quicksort (hand-written)	2.8×10^7	~6 s	$O(1)$
sorted() -- Timsort in C	2.0×10^7	~0.15 s	$O(n)$
sorted() -- already ordered	1.0×10^6	~0.02 s	$O(n)$
counting sort, $k = 100$	2.0×10^6	~0.35 s	$O(n+k)$
radix sort, base 256, $d = 4$	8.0×10^6	~1.2 s	$O(n+b)$
quickselect (median)	3.4×10^6	~1.5 s	$O(1)$
heapq.nlargest(10)	3.3×10^6	~0.09 s	$O(k)$

Two lessons in that table and both get asked about. A **hand-written $O(n \log n)$ sort in Python is forty times slower than `sorted()`**, because one is bytecode and the other is C — so the asymptotic winner is not always the wall-clock winner. And **`sorted()` on nearly-ordered data is faster than counting sort**, which is why “just call `sorted()` and measure” is a defensible engineering answer even when a linear sort exists.

The three arithmetic facts to have ready

1. $n \log n$ against n^2 , at $n = 10^6$
20,000,000 against 500,000,000,000 → 25,000x
2. Quickselect's halving sum
 $n + n/2 + n/4 + \dots = 2n$ → $O(n)$, not $O(n \log n)$
3. Sort-once-then-query break-even, $n = 100,000$
scan per query : $q \times 100,000$
sort + binary search : $1,700,000 + q \times 34$
break-even at $q \sim 17$

The comparison-count differences, which get asked

```
n = 1,000,000

merge sort      n log2 n      = 20,000,000 comparisons
quicksort       1.39 n log2 n  = 28,000,000 (39% MORE, and still faster –
                                     in place, cache-friendly, no allocation)
heapsort        2 n log2 n     = 40,000,000 (and it is the slowest of the three
                                     in practice, despite the guarantee)
```

Space, side by side

```
O(1)      : insertion, selection, bubble, heapsort, quicksort's data
O(log n)   : quicksort's stack (only if you recurse into the smaller side)
O(n)       : merge sort, Timsort, bucket sort
O(n + k)   : counting sort -- and k can be the thing that kills you
O(k)       : a heap of size k -- the only one that survives a stream
```

7 The traps

The phase's collected failures, each with the input that exposes it. Run every one of these once.

Quicksort on already-sorted input

```
import sys; sys.setrecursionlimit(2000)
quicksort_last_pivot(list(range(5000)))
```

```
RecursionError: maximum recursion depth exceeded
```

Sorting a sorted list crashed. Fixed pivot means one empty side every time. **Randomise the pivot.**

Quicksort or quickselect on all-equal values

```
quicksort_two_way([7] * 3000)      # with ≤ in the partition
```

```
RecursionError: maximum recursion depth exceeded
```

Randomising the *position* does not help when every *value* is equal. **Three-way partition**, and strict `<` not `≤`.

Recurring on the pivot

```
quicksort(nums, lo, p)                # should be p - 1
```

```
RecursionError: maximum recursion depth exceeded
```

The pivot is finished. Exclude it from both sides.

Merge sort's dropped tail

```
print(merge_broken([1, 2, 9], [3, 4]))
```

```
[1, 2, 3, 4]
```

No error, and the `9` is gone. The loop exits when *either* side empties, so **both** `extend` calls are required.

The base case that never fires

```
merge_sort_broken([])                # if len(nums) == 1
```

```
RecursionError: maximum recursion depth exceeded
```

`≤ 1`, not `= 1`. The base case must cover every input that cannot shrink.

Counting sort's range

```
counting_sort([1, 5, 2_000_000_000], 2_000_000_000)
```

```
MemoryError
```

Three elements. `k` is the **range**, not the count.

The k-th largest off-by-one

```
print(kth_largest_broken([3, 2, 1, 5, 6, 4], 2))
```

```
3
```

Should be 5. `target = len(nums) - k`, checked against `k = 1` and `k = n`.

`reverse=True` on a tuple key

```
sorted(students, key=lambda s: (s.score, s.name), reverse=True)
```

```
['Chitra', 'Asha', 'Devi', 'Bala']
```

Scores descending — correct. Names descending too — not asked for, no error. Negate the number inside the key instead.

Negating a string

```
sorted(people, key=lambda p: (-p.name, p.age))
```

```
TypeError: bad operand type for unary -: 'str'
```

Two passes, least significant key first.

`.sort()` returns `None`

```
top = nums.sort()[:3]
```

```
TypeError: 'NoneType' object is not subscriptable
```

`sorted()` for a value, `.sort()` for the side effect.

Sorting objects with no order

```
sorted([Student("Asha", 90), Student("Bala", 85)])
```

```
TypeError: '<' not supported between instances of 'Student' and 'Student'
```

Pass `key=`. Do not reach for `__lt__` unless the type has one obvious order.

The silent stability losses

```
if left[i] < right[j]:          # merge sort: should be ≤
while i ≥ 0 and nums[i] ≥ value: # insertion sort: should be >
for x in nums:                  # counting sort: should be reversed(nums)
```

All three still sort correctly. All three silently break two-pass multi-key sorting. **Keep the four-line `is_stable` test.**

8 In the interview

How it gets asked

- “Here’s a problem. Talk me through it.” — no hint that it is about sorting, and recognising that it is half the mark.
- “Two problems, forty-five minutes, no hints.” — the standard coding round.
- “You sorted. Justify it.” — asked after almost every solution that begins with `sorted()`.
- “Which sort would you use here, and why?” — the defence question. The answer is a trade, not a ranking.
- “Can you do it without sorting?” — meaning “is there a hash-set or heap answer”.

The script, minute by minute, for a 45-minute round

Minutes 0-3 — clarify. Two or three questions about the *data*, not the problem statement. How large, what type are the keys, what range, is it nearly sorted, can I modify the input, does it fit in memory. Then restate the problem in your own words and get agreement.

Minutes 3-6 — state the approach and its cost, before writing anything. *“My first thought is the brute-force double loop, which is $O(n^2)$ — half a trillion operations at a million elements. I think sorting collapses it, because [the specific reason], which makes it $O(n \log n)$ plus a linear pass. I’ll do that unless you’d rather I explore something else.”* Getting agreement here means you do not write the wrong thing for fifteen minutes.

Minutes 6-25 — write it, narrating. Say what each piece maintains as you write it. State the invariant. Flag the parts you know are error-prone as you reach them — the guard order, the strict comparison, the excluded pivot.

Minutes 25-32 — test out loud, on the five inputs. Empty. One element. All equal. Already sorted. Reverse sorted. Walk one real example through by hand.

Minutes 32-40 — the follow-up. Expect one of: what if the data is a stream, what if the values are bounded, what if you only need the top k , what is the worst case, is it stable.

Minutes 40-45 — the honest summary. *“Time is $n \log n$, space is $O(n)$ because `sorted` allocates. If I needed $O(1)$ space I’d use quicksort in place and give up the guarantee. The case I’d want to check with real data is [something specific].”*

The follow-ups

“You sorted. Justify the extra $n \log n$.” With arithmetic, not with a principle. The unsorted version compares every pair, which is $n(n-1)/2$ — at a million elements about five hundred billion comparisons, which is days in Python. Sorting is $n \log n$, so twenty million, and the pass afterwards is another million: about twenty-one million against five hundred billion, roughly twenty-five thousand times less work. So the sort did not merely pay for itself, it changed the complexity class, and that is my test. If sorting takes an $O(n^2)$ problem to $O(n \log n)$, or makes it $O(n)$ after the sort, I sort. If the problem was already linear — a membership question a set

answers, or a single pass — sorting makes it worse and the honest thing is to notice rather than reach for the habit. I would also say what it cost besides time: `sorted` allocates an $O(n)$ copy, and `.sort()` would mutate the caller's list, which is a decision rather than a detail.

“Which sorting algorithm would you actually use, and why?” In Python, `sorted()`, and I would say why that is a real answer rather than a dodge: it is Timsort, which is $O(n \log n)$ worst case, stable, adaptive — linear on nearly-ordered input — and written in C, so its constant factor beats anything I would write by roughly forty times. A hand-written merge sort doing the same twenty million comparisons takes about eight seconds where `sorted()` takes 0.15. Then I would name the cases where I would do something else. If the keys are bounded small integers — marks out of a hundred, ages, statuses — counting sort is $O(n + k)$ and genuinely linear, with the caveat that k is the *range*, so a range of two billion is a `MemoryError` rather than a slow sort. If I need a guaranteed worst case because the input could be chosen against me, merge sort, because quicksort's worst case is real and an adversary who knows a deterministic pivot rule can construct it. If memory is tight and I have an array in memory, quicksort in place with a random pivot — it does about thirty-nine percent more comparisons than merge sort and is still two to three times faster, because it allocates nothing and the memory access pattern is friendly. If the data is a linked list or does not fit in memory, merge sort, because merging is sequential and needs no random access. And the framing I would give at the end: you can pick two of guaranteed, in place, and stable. Merge gives guaranteed and stable; heapsort gives guaranteed and in place; quicksort gives in place and fast, and gives up the guarantee.

“What if the data doesn't fit in memory / arrives as a stream?” Those are two different questions and I would separate them. If it is finite but larger than RAM, the answer is an external merge sort: read chunks that fit in memory, sort each one, write it out as a run, then merge the runs with a k -way merge — which needs only one buffer per run in memory. That is exactly what Postgres or MySQL does for a large `ORDER BY` that spills to disk, and it is why merge sort survives as an algorithm even though quicksort is faster in memory. If it is a genuine unbounded stream, then sorting is not available at all, because a sort needs every element present, and I would restructure around what is actually needed. For the top k , a min-heap of size k : push each value, pop whenever the size exceeds k , and the heap holds the k largest — $O(n \log k)$ time, $O(k)$ space, one pass, never holding more than $k+1$ items. For a running median, two heaps, a max-heap of the lower half and a min-heap of the upper half, rebalanced after each insert, giving $O(\log n)$ per element. For approximate quantiles over a very large stream, a sketch such as t -digest, which trades exactness for constant memory — and I would say plainly that it is approximate rather than let that pass. The one thing I could not use is quickselect, because it needs random access and rearranges the data in place.

A model answer

“Before I code anything: how large is the input, what type are the keys, and is the data likely to be roughly ordered already? ... A hundred thousand, integers in an unknown range, arbitrary order. Good.

The brute-force reading of this is a double loop, which is $O(n^2)$ — at a hundred thousand that is five billion operations, so it’s not going to run. The question I’d ask myself is whether it gets easier if the input is in order, and here it does, because [the specific reason]. That makes it one sort plus a linear pass: $n \log n$ plus n , about 1.8 million operations. Twenty-seven hundred times less work, and it changes the complexity class, which is my test for whether to sort.

Before I commit, I’d check the three reasons not to. A hash set doesn’t answer this, because it’s about ordering rather than membership. I need the whole order rather than the top k , so a heap wouldn’t be cheaper. And the answer is a count rather than positions, so I don’t need to carry original indices — if I did, I’d sort `(value, index)` pairs.

I’ll use `sorted()` rather than writing a sort. It’s Timsort: $O(n \log n)$ worst case, stable, adaptive, and in C, so about forty times faster than anything I’d write here. `sorted` rather than `.sort()` so I don’t mutate the caller’s list, at the cost of $O(n)$ space.

[writes the code, narrating the invariant]

Let me test it out loud: empty input, one element, all values equal, already sorted, reverse sorted. And this specific case — [walks one example through by hand].

Cost is $O(n \log n)$ time, $O(n)$ space. If the interviewer told me the keys were bounded — say statuses or marks out of a hundred — I’d switch to counting sort for $O(n + k)$, with the caveat that k is the range and not the count. If they told me the data was a stream, I couldn’t sort at all and I’d want a heap of size k instead.”

9 Recall card

- **Run the procedure out loud, in order:** what is the data (size · key type · range · nearly sorted · fits in memory) → *would this be easier in order?* → does a **set** answer it in one pass → do I need the **whole order, one value, or the top k** → which sort, and why → the cost **with arithmetic**.
- **Five leaves of the decision tree are not “sort the array”.** Set for membership · quickselect for the k -th ($O(n)$ expected) · heap of size k for the top k and the only one that works on a **stream** · counting sort for bounded integer keys · Timsort for everything else.
- **Pick two of: guaranteed · in place · stable.** Merge = guaranteed + stable ($O(n)$ space) · heapsort = guaranteed + in place (not stable) · quicksort = in place + fastest, giving up the guarantee. Quicksort does **39% more comparisons** than merge sort and is still $2\text{--}3\times$ faster.

- **In Python the default answer is `sorted()` , and that is defensible:** Timsort is stable, $O(n \log n)$ worst case, **linear on nearly-ordered data**, and in C — $\sim 40\times$ faster than a hand-written merge sort (0.15 s against 8 s at $n = 10^6$). Switch only for bounded integer keys, a needed guarantee, tight memory, a linked list, or a stream.
- **Three numbers and five test inputs.** 20,000,000 against 500,000,000,000 ($25,000\times$) · $n + n/2 + n/4 = 2n$ · sort-once break-even at ~ 17 queries. Always test: empty · one element · all equal · already sorted · reverse sorted. And **keep talking** — a mock round scores clarifying, stating the cost before coding, narrating while stuck, and testing out loud.

1 What this is, and why they ask it

The **dependency inversion principle** is the D in SOLID, and it is two sentences: high-level modules should not depend on low-level modules — both should depend on an abstraction. And the abstraction should not depend on details; details should depend on the abstraction.

The part everybody misses, and the part that is actually being tested, is **who owns the abstraction**. It is not enough to put an interface between your service and MySQL. The interface has to belong to the *service* — defined in the service’s vocabulary, living in the service’s package, answering the service’s questions — with the MySQL adapter written to conform to it. If the interface is defined by the database layer and the service imports it, you have added a layer and inverted nothing.

They ask it as “your service imports the MySQL driver” because it is the concrete form of the question, and because the wrong answers are so common. “I’d add a repository” is half an answer. The full answer names the direction of the arrow, says which package the interface lives in, gives the `grep` that proves it worked, and distinguishes the principle from dependency injection — which is a different thing that people constantly conflate with it. This is the last of the five, and it is the one that makes the other four reachable: open/closed ([day 056](#)) needs somewhere to plug implementations in, and this is what points the plug the right way.

2 The story

Sister Rita ran the kitchen at a girls’ hostel in Mangalore for nineteen years, and for the first six of them the menu was decided by a man in a lorry.

Not deliberately. It was just how it worked. Damodar came at about half past six most mornings with whatever he had, and the cook, Sarojini, looked in the crates and worked out what she could make. Lots of brinjal that week meant brinjal on Tuesday and again on Thursday. If the beans looked poor, the beans dish became something else. Nobody wrote anything down and it worked, in the sense that a hundred and forty girls got fed every day.

Then in 2011 Damodar’s son had an accident and he stopped coming for three weeks, and the kitchen essentially stopped working.

Not because there was no food — there were four other suppliers in that market. It was because nobody in the kitchen could say what they needed. Sarojini had spent six years starting from the crates. She could look at a crate and know what to cook; she had never once had to go the other way, from a menu to a list, because she had never had a menu. Sister Rita went to the market herself on the second day and stood there and could not tell the man what to send.

They got through it badly for three weeks and then Sister Rita changed the order of things, and she says it is the only administrative thing she ever did that she is genuinely proud of.

She wrote the week's menu first. Monday to Sunday, two meals a day, decided in the kitchen by the people who cook and the girls who eat. Then, from the menu, she wrote what the kitchen needed each morning: quantities, and the few things that actually mattered — the potatoes not too large, the coriander that day and not the day before, everything at the gate by half past six.

That list went to Damodar when he came back, and it went to two other suppliers as well, because now there was something to hand over.

The list has been the same list for fourteen years. Suppliers have changed four times, twice in a hurry. The last time, when the man in Bejai stopped, it took one afternoon: she gave the new man the same sheet she had given the last one, and the girls did not notice anything.

What Sister Rita says about it, when anybody asks, is that for six years the kitchen had been fitting itself around whatever arrived, and it felt normal because it worked. It was only when the lorry stopped that anybody could see who had been deciding the menu.

3 The idea in plain English

For six years the menu depended on the supplier. After 2011 the supplier depended on the menu. Nothing about cooking changed and nothing about growing vegetables changed. What changed is **which direction the arrow points**, and who writes the list.

The principle, both halves

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

“High-level” means the code that expresses what the business does — placing an order, calculating a fee, deciding a menu. “Low-level” means the mechanism — MySQL, SMTP, the file system, Razorpay, whichever lorry turned up.

The natural way to write code is high-level importing low-level, because that is the order you build things in. `OrderService` imports `psycpg`. That is the kitchen starting from the crates.

Inversion means the arrow flips: `OrderService` declares what it needs, and the Postgres code is written to satisfy it.

The half everyone forgets: who owns the interface

This is the whole of today. Consider two arrangements that look identical on a diagram.

Arrangement A — a layer, not an inversion:

```
orders/service.py    → imports → persistence/repository.py (the interface)
persistence/postgres.py implements → persistence/repository.py
```

The interface lives in the persistence package. It was designed by whoever wrote the database layer, so it speaks in rows and connections and transactions. When the database team changes it, the service changes. The service still depends on the persistence package. **Nothing was inverted.**

Arrangement B — a real inversion:

```
orders/service.py    → imports → orders/ports.py (the interface, in the SERVICE's package)
adapters/postgres.py → imports → orders/ports.py and implements it
```

Now the interface belongs to the orders module. It is written in the orders module's vocabulary — `Order`, `Money`, `OrderNotFound` — and it answers the orders module's questions. The Postgres adapter imports *downwards* into the domain to find out what shape it must be.

The test that distinguishes them: **which package would you have to change to add a field the business needs?** In A, the persistence package. In B, the orders package, which is where the business lives.

Sister Rita's list is the interface, and it lives in the kitchen.

Three things that get confused, separated

People use these three terms interchangeably and they are not the same.

- **Dependency inversion (DIP)** is about the *direction of source-level dependencies*. Who imports whom, and who owns the interface. It is a statement about architecture.
- **Dependency injection (DI)** is a *technique*: pass collaborators in as arguments rather than constructing them ([day 053](#)). It is three lines of code.
- **Inversion of control (IoC)** is about *who calls whom at runtime* — a framework calling your code, rather than your code calling a library. Django calling your view is IoC.

You can have injection without inversion: `OrderService(psycopg.connect(...))` injects a concrete Postgres connection, so the service still depends on Postgres — it just does not build it. And you can have inversion without a framework: an interface in your package and a hand-written composition root is complete DIP with no container anywhere. **Injection is how you deliver the dependency; inversion is about which way the arrow points.**

The concrete before and after

Before. The high-level policy imports the mechanism:

```
# orders/service.py
import psycopg2                                # ← the problem
from razorpay import Client

class OrderService:
    def place(self, customer_id: int, total_paise: int) → int:
        conn = psycopg2.connect(os.environ["DATABASE_URL"])
        client = Client(auth=(os.environ["RZP_KEY"], os.environ["RZP_SECRET"]))
        charge = client.order.create({"amount": total_paise, "currency": "INR"})
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO orders (customer_id, total, charge) VALUES (%s,%s,%s) RETURNING id",
                (customer_id, total_paise, charge["id"]),
            )
        return cur.fetchone()[0]
```

Four things are wrong and it is worth naming all four:

1. The business rule cannot be read, because it is buried in SQL and vendor calls.
2. It cannot be tested without Postgres and Razorpay.
3. Swapping either one means editing the business logic.
4. `charge["id"]` — a dictionary key from a vendor's response — is now part of your service's knowledge. Their shape has leaked in.

After. The policy declares what it needs, in its own words:

```
# orders/ports.py -- lives in the ORDERS package, owned by the business
from typing import Protocol
```

```
class OrderRepository(Protocol):
    def next_id(self) → int: ...
    def save(self, order: "Order") → None: ...
    def get(self, order_id: int) → "Order": ...
```

```
class PaymentGateway(Protocol):
    def charge(self, amount: "Money", token: str) → "ChargeId": ...
```

```
# orders/service.py -- imports NOTHING external
from .ports import OrderRepository, PaymentGateway
```

```
class OrderService:
    def __init__(self, orders: OrderRepository, payments: PaymentGateway) → None:
        self._orders, self._payments = orders, payments

    def place(self, customer_id: int, total: Money, token: str) → int:
        charge = self._payments.charge(total, token)          # your types, your words
        order = Order(self._orders.next_id(), customer_id, total, charge)
        self._orders.save(order)
        return order.order_id
```

```
# adapters/postgres_orders.py -- imports UPWARDS into the domain
from orders.ports import OrderRepository
from orders.model import Order, Money
```

```
class PostgresOrderRepository:                                # satisfies OrderRepository structurally
    def __init__(self, conn) → None:
        self._conn = conn
    def save(self, order: Order) → None: ...
    def get(self, order_id: int) → Order: ...
    def next_id(self) → int: ...
```

```
Read the import lines. orders/service.py imports orders/ports.py .
adapters/postgres_orders.py imports orders/ports.py . Both point at the abstraction,
and nothing points from the domain at the adapter. That is the inversion, and it is visible
in the imports rather than in the diagram.
```

The check that proves it

```
grep -rn "psycpg\|razorpay" orders/
```

If that returns nothing, the inversion is real. If it returns anything, it is not, whatever the diagram says. This is a genuinely good thing to say in an interview, because it turns a principle into a command.

Why this is the same shape as ports and adapters

The arrangement above has a name — **ports and adapters**, also called hexagonal architecture, and essentially the same thing as Clean Architecture’s dependency rule. A **port** is an interface owned by the domain (`OrderRepository`). An **adapter** is an implementation that translates to a specific technology (`PostgresOrderRepository`). The rule is one sentence: **source-level dependencies always point inwards, towards the business rules.**

You do not need to name the architecture to answer the question, but knowing that “put the interface in the domain package” is the whole of it is worth having.

4 The picture

Sister Rita’s kitchen, before and after:



FIGURE 59.2

What to notice: the arrows in the bottom half both point at the list, and the list sits with the kitchen, not with the suppliers. If the suppliers had written the list, changing supplier would mean a new list and a new menu — which is arrangement A, the layer that inverts nothing.

The code, with the import arrows drawn:

BEFORE – the arrow points outward, from policy to mechanism

```
+-----+
| orders/service.py      |
| the business rules     |
+-----+
| import psycpg           |
| import razorpay         |
|                         |
| v                       |
+-----+
| psycpg / razorpay       |
| the mechanism           |
+-----+
```

swap the database → edit the business rules
test the rules → start Postgres

AFTER – both arrows point at the abstraction, which lives INSIDE the domain

```
+-----+
| orders/                  |
| model.py   Order, Money, ChargeId |
| ports.py   OrderRepository, PaymentGateway ←+-----+
| service.py ---- imports ports -----+
|                                         | imports
+-----+                               | (upwards,
|                                         | into the
|                                         | domain)
+-----+
| adapters/                  |
| postgres_orders.py implements OrderRepository+-----+
| razorpay_gateway.py implements PaymentGateway +
| in_memory_orders.py implements OrderRepository+
+-----+
```

swap the database → one new file in adapters/, one line in main()
test the rules → hand in in_memory_orders. No Postgres.

grep -rn "psycpg" orders/ → nothing. That is the proof.

What to notice: the adapter package imports the domain package, and the domain package imports nothing. That is the direction that has been inverted — before, the domain imported the mechanism.

Injection against inversion, which are not the same thing:

```

(1) neither
    class OrderService:
        def __init__(self):
            self.db = psycopg.connect(...)          # builds it, and knows psycopg

(2) INJECTION but NOT inversion
    class OrderService:
        def __init__(self, conn: psycopg.Connection): # ← a vendor type in the signature
            self.db = conn
    → testable-ish, but the service still DEPENDS on psycopg.
    Swapping to MySQL still edits this file.

(3) INVERSION, delivered by injection                ← the answer
    class OrderService:
        def __init__(self, orders: OrderRepository): # ← YOUR type, in YOUR package
            self.orders = orders
    → the service knows nothing about any database.

```

What to notice: the difference between (2) and (3) is one type name in one signature, and it is the entire principle. If a vendor's type appears in your service's constructor, you have injected a dependency without inverting it.

5 How it actually works

The refactor, in six steps

Step 1 — find the outward imports. `grep -rn "import psycopg\|import redis\|import boto3\|import razorpay" src/domain/ src/orders/`. Every hit is a place where policy depends on mechanism.

Step 2 — write down what the policy actually asks the mechanism for. Not what the driver offers — what your code uses. Usually three or four operations out of a library's hundreds. That list is the port.

Step 3 — declare the port in the policy's package, in the policy's vocabulary. Your `Order`, your `Money`, your `OrderNotFound`. **If a vendor type appears in the signature, start again** — the interface belongs to the vendor and you have bought nothing (day 048).

Step 4 — write the adapter, and make it translate at its own edge. Their rows become your objects, their errors become your exceptions, their units become your units.

Step 5 — name the concrete classes exactly once, in the composition root.

Step 6 — run the grep. If the domain package still mentions the vendor, you are not finished.

The domain, importing nothing

```
# orders/model.py
from dataclasses import dataclass

@dataclass(frozen=True)
class Money:
    paise: int
    currency: str = "INR"

@dataclass(frozen=True)
class ChargeId:
    value: str

class OrderNotFound(Exception):
    """Ours. Not psycopg's, not Razorpay's."""

class PaymentDeclined(Exception):
    """Ours."""

@dataclass
class Order:
    order_id: int
    customer_id: int
    total: Money
    charge: ChargeId | None = None
```

The ports, in the domain's package

```
# orders/ports.py
from typing import Protocol
from .model import ChargeId, Money, Order

class OrderRepository(Protocol):
    """What the ORDERS module needs from storage. Written by orders, for orders."""

    def next_id(self) → int: ...
    def save(self, order: Order) → None: ...
    def get(self, order_id: int) → Order:
        """Raises OrderNotFound if there is no such order."""
        ...

class PaymentGateway(Protocol):
    """What the ORDERS module needs from payments."""

    def charge(self, amount: Money, token: str) → ChargeId:
        """Raises PaymentDeclined if the card is refused."""
        ...
```

The policy, readable at last

Six lines, and every one of them is business. There is no SQL, no vendor dictionary key, no environment variable. Somebody who knows nothing about the stack can read it and say whether the rule is right — which is the actual point of the principle, more than swappability.

[illegible]

Three translations happen here and all three matter: a row tuple becomes an `Order`, an integer of `paisa` becomes a `Money`, and “no row” becomes `OrderNotFound`. If any of those leaked out, the domain would know about the database again.

The fake, which is the same shape and fifteen lines

```
# adapters/in_memory_orders.py
from orders.model import Order, OrderNotFound

class InMemoryOrderRepository:
    def __init__(self) → None:
        self._rows: dict[int, Order] = {}
        self._next = 1

    def next_id(self) → int:
        value, self._next = self._next, self._next + 1
        return value

    def save(self, order: Order) → None:
        self._rows[order.order_id] = order

    def get(self, order_id: int) → Order:
        if order_id not in self._rows:
            raise OrderNotFound(order_id)          # the SAME contract
        return self._rows[order_id]
```

The composition root

```
# main.py -- the only module that knows what is actually being used
import os
import psycopg
from adapters.postgres_orders import PostgresOrderRepository
from adapters.razorpay_gateway import RazorpayGateway
from orders.service import OrderService

def build_order_service() → OrderService:
    return OrderService(
        orders=PostgresOrderRepository(psycopg.connect(os.environ["DATABASE_URL"])),
        payments=RazorpayGateway(os.environ["RZP_KEY"], os.environ["RZP_SECRET"]),
    )
```

Where you have already been using this

- **Python’s DB-API and Java’s JDBC.** Your code writes against a `Connection` and a `Cursor`; the driver conforms. That is why switching from Postgres to MySQL changes a connection string and some SQL rather than your program.
- **logging handlers.** Your code calls `logger.info`; a handler decides whether that lands in a file, in syslog, or in stdout — and your code never knows.
- **Django’s storage backends and cache backends.** `default_storage.save(...)` is a port; S3 and the local filesystem are adapters, chosen in settings.

- **SLF4J in Java** — a façade whose entire purpose is that libraries depend on the interface and the application chooses the implementation.
- **The `os` module.** `open()` is a port over three different operating systems' file APIs.
- **Docker, and container runtimes.** Kubernetes depends on the Container Runtime Interface; containerd and CRI-O implement it ([day 013](#)). Kubernetes did not change when Docker was removed as a runtime, and that is the principle paying off at a very large scale.

6 The numbers

Swapping the mechanism, priced

```
"we are moving from Razorpay to Stripe"

without inversion:
  grep -rn "razorpay" src/      → 38 hits across 14 files
  files to edit                : 14 (services, jobs, admin views, tests)
  business logic touched       : yes, in 9 of them
  test suite that must re-run  : all of it
  engineer-days                : ~4, plus a nervous release

with inversion:
  grep -rn "razorpay" src/      → 2 hits: adapters/razorpay_gateway.py, main.py
  files to add                  : 1 (adapters/stripe_gateway.py)
  files to edit                 : 1 (main.py, one line)
  business logic touched        : none
  engineer-days                 : ~0.5
```

The import graph, which is the measurable version

```
before:
  orders/ imports psycpg, razorpay, boto3, redis, requests
  → 5 external packages reachable from the business rules
  → the domain cannot be imported at all without them installed

after:
  orders/ imports dataclasses, typing, decimal
  → 0 external packages
  → `python -c "import orders"` works in a bare interpreter
```

That last line is a genuine test and worth naming: **if you cannot import your domain package in an interpreter with nothing installed, the inversion is incomplete.**

Test cost

```
testing "placing an order charges the card before saving"

before : needs Postgres + a Razorpay sandbox key + network
        ~900 ms, flakes when the sandbox is slow, cannot run in CI without secrets

after  : InMemoryOrderRepository + a recording gateway
        ~0.3 ms, deterministic, no secrets

3,000x faster, and the failure path -- "if the charge is declined, nothing is saved" --
becomes testable at all, which it was not before.
```

Build and deploy blast radius, in a compiled language

```
a monorepo where the domain imports the driver:
  changing the driver version    → the domain recompiles
                                → every service importing the domain recompiles
                                → 17 services redeployed

with inversion:
  changing the driver version    → adapters/ recompiles
                                → the domain does not
                                → 1 service redeployed
```

The cost of doing it

```
ports.py                : ~20 lines
the adapter              : ~45 lines (mostly the SQL that was inline before)
the in-memory fake      : ~15 lines
the composition root     : ~8 lines
model types (Money, ChargeId) : ~20 lines
-----
                        ~108 lines of structure

for a service with one database and no plans to change it,
that is 108 lines buying an ability nobody has asked for.
```

Both numbers are real. Which one applies depends on how many adapters you can actually name, which is the trade-off section.

7 The trade-offs

What it costs

Indirection. Reading `self._orders.save(order)` no longer tells you what happens. You go to the composition root to find out which adapter is wired in. One extra hop, every time, for every reader.

Translation code. The adapter has to convert rows to objects and errors to exceptions, and that is code that did not exist when the service just ran SQL. On a simple CRUD service it can be more code than the logic it protects.

Two vocabularies to keep in step. Your `Order` and the database's `orders` table drift, and somebody has to maintain the mapping. This is exactly what an ORM does for you, and part of why people reach for one.

It can hide capability. A port narrow enough to be clean may not expose the thing that makes your database good — a Postgres-specific upsert, a window function, a full-text index. If the port only offers `get` and `save`, somebody will write a slow loop where one query would do.

When not to invert

I would not invert a dependency on the standard library or the language. Nobody writes a port over `datetime`, the type, `json`, or `math`. The exception is the *clock* as a source of “now”, which is not really a type dependency — it is a source of non-determinism, and that does get injected ([day 053](#)).

I would not invert when I cannot name the second implementation — the test from [day 048](#), and it still governs. The in-memory fake for tests counts as a second implementation, and it is usually the honest reason. If there is no fake and no plausible second adapter, a hundred lines of ports is speculative generality.

I would not invert a whole application on day one. A small service with one database and a six-week life should import `psycopg` and get on with it. The signal to invert is the second adapter or the first painful test, and it is a refactor you can do later — which is not true of most architectural decisions.

I would not invert when the abstraction cannot be honest. If your code genuinely depends on Postgres's transactional guarantees, a port that pretends any store would do is a lie, and the first implementation that is not Postgres will break in a way the interface promised could not happen. That is a Liskov problem ([day 057](#)) created by an over-broad port. Better to depend on Postgres openly and say why.

The failure mode: the layer that inverts nothing

The commonest way to get this wrong is to add a repository package whose interface is written by the persistence layer, in the persistence layer's vocabulary, and imported by the domain. It looks like inversion on a diagram — there is an interface in the middle — and the domain still depends on persistence, which is the thing you were trying to stop.

The two tests that catch it: which package does the interface file live in, and does its signature mention any vendor type. If the answer is “the persistence package” and “yes”, nothing was inverted.

The honest limit

Inversion buys you *source-level* independence. It does not buy you runtime independence: the service still needs *some* adapter, and if that adapter is slow or broken the service is slow or broken. It also does not make swapping free — moving from Postgres to DynamoDB means the adapter must implement transactional behaviour the new store does not have, and that is a

genuine engineering problem the interface cannot dissolve. What you have bought is that the *business rules* do not change, and that the problem is confined to one directory. Say that precisely, rather than claiming a swap is a one-line change when the honest number is one afternoon.

8 In the interview

How it gets asked

- “Your service imports the MySQL driver directly. What’s wrong with that?” — the direct form.
- “What is the dependency inversion principle?” — and the follow-up is always “how is that different from dependency injection?”, which is the real question.
- “How would you make it possible to switch from Razorpay to Stripe?” — the same principle, wearing a scenario.
- “Where would you put the interface?” — the question that separates people. In the domain package.
- “Isn’t this over-engineering for a small service?” — the pushback, and “yes, often” is part of a good answer.

What to say out loud, in the first ninety seconds

1. **Name the direction, not the missing interface.** “The problem is that the arrow points the wrong way. The business rules — high-level — depend on a driver — low-level. It should be the other way round: both should depend on an abstraction that the business owns.”
2. **Give the three concrete harms.** “I can’t read the rule, because it’s buried in SQL. I can’t test it without a database. And swapping the database means editing business logic.”
3. **Say where the interface goes, unprompted.** “I’d define an `OrderRepository` protocol in the orders package, in the orders package’s vocabulary — my `Order`, my `Money`, my `OrderNotFound`. If I put it in a persistence package instead, the service still depends on persistence and nothing has been inverted.”
4. **Describe the adapter as a translator.** “`PostgresOrderRepository` imports upwards into the domain and translates at its own edge: rows to `Order` objects, `psycopg` errors to my exceptions.”
5. **Give the proof.** “The check is `grep -rn psycopg orders/` returning nothing. And a stronger one: I should be able to `import orders` in an interpreter with nothing installed.”

The follow-ups

“How is this different from dependency injection?” They are frequently used interchangeably and they are not the same thing. Dependency injection is a technique — pass collaborators in as constructor arguments rather than building them inside — and it is about three lines of code. Dependency inversion is about the *direction of source-level dependencies*: who imports whom, and crucially who owns the interface. You can do one without the other, and the case

that shows it is injecting a concrete type. If I write `def __init__(self, conn: psycopg.Connection)`, I have injected the dependency — I am not constructing it, so I could hand in a test connection — but my service still names a vendor type in its own signature, so it still depends on `psycopg`, and moving to MySQL still edits this file. Nothing was inverted. The inverted version is `def __init__(self, orders: OrderRepository)`, where `OrderRepository` is my type in my package. The difference between those two lines is the entire principle. There is a third thing people fold in, inversion of control, which is about who calls whom at *runtime* — a framework calling your code rather than your code calling a library, like Django calling your view. That is orthogonal: you can have full dependency inversion with no framework at all, just an interface in your package and a hand-written composition root. The short form: **injection is how the dependency is delivered; inversion is which way the arrow points.**

“Where does the interface live, and why does it matter?” In the package that *uses* it — the domain — not in the package that implements it. This is the half of the principle people skip and it is the whole of it. If I put `OrderRepository` in a `persistence` package and my service imports it from there, then my service depends on the persistence package, the interface was designed by whoever wrote the database layer, and it will speak in rows and cursors and transactions. When the database team changes it, my business logic changes. I have added a layer and inverted nothing, and the diagram looks identical. If instead the port lives in `orders/ports.py`, then the orders module owns it, it is written in orders’ vocabulary — my `Order`, my `Money`, my `OrderNotFound` — and the Postgres adapter imports *upwards* into the domain to find out what shape it must be. That is what “inverted” means: the dependency that used to run from policy to mechanism now runs from mechanism to policy. The two checks I would run are, which directory is the interface file in, and does any signature in it mention a vendor type. If a vendor type appears, the interface belongs to the vendor and I have bought nothing. Underneath, that is what ports and adapters — hexagonal architecture — is: source dependencies always point inwards, towards the business rules.

“Isn’t this over-engineering for a small service?” Often, yes, and I would rather say so than defend it everywhere. The cost is real: about a hundred lines of ports, adapters, fakes and model types, plus a hop of indirection every time a reader wants to know what actually happens, plus translation code that did not exist when the service just ran SQL. For a service with one database, no plans to change it, and a six-week life, I would import `psycopg` and get on with it, and I would not feel bad about it. The test I actually apply is whether I can name the second implementation — and I would be honest that the second implementation is usually the in-memory fake for tests, which is a legitimate reason on its own. That is where most of the value shows up for a small team: testing “if the payment is declined, nothing is saved” is impossible against a real gateway and trivial against a fake, and roughly half the branches in a service are error paths I cannot otherwise reach. The other thing I would say is that this is one of the few architectural decisions you can defer safely. Inverting a dependency later is a mechanical refactor confined to a few files; it is not like choosing the wrong data model. So my default on a small service is to skip it, and to do it the moment either a second adapter appears or a test becomes painful.

A model answer

“The problem isn’t that there’s no interface — it’s the direction of the dependency. The high-level code, the business rules about placing an order, depends on a low-level detail, a database driver. Dependency inversion says neither should depend on the other: both should depend on an abstraction.

Three concrete harms. I can’t read the business rule, because it’s tangled with SQL and vendor dictionary keys. I can’t test it without Postgres running. And swapping the database means editing the file where the business logic lives, which is the file I least want to touch.

So I’d define an `OrderRepository` protocol with the three or four operations my service actually uses — `next_id`, `save`, `get` — and, importantly, I’d put it in the `orders` package, not in a persistence package. That’s the half people skip. If the interface lives with the database code and my service imports it from there, my service still depends on the persistence layer, the interface will be written in rows and cursors, and nothing has been inverted. In the `orders` package it’s written in my vocabulary — my `Order`, my `Money`, my `OrderNotFound` — and the Postgres adapter imports upwards into the domain to find out what shape it has to be.

The adapter’s job is to translate at its own edge: a row tuple becomes an `Order`, an integer of paise becomes a `Money`, and ‘no row found’ becomes `OrderNotFound` rather than a `psycopg` exception. If any of those leaked through, the domain would know about the database again.

The concrete classes get named exactly once, in `main()`. And the proof that it worked is a command: `grep -rn psycopg orders/` should return nothing. A stronger version — I should be able to `import orders` in an interpreter with nothing installed.

What it buys, with numbers: moving from Razorpay to Stripe went from thirty-eight references across fourteen files to two references — the adapter and `main()` — so it’s one new file and one line instead of about four engineer-days. And testing ‘if the payment is declined, nothing is saved’ goes from impossible to three lines.

I’d add the caveat that this is roughly a hundred lines of structure, and on a small service with one database and no second implementation in sight, I’d skip it. The test I apply is whether I can name the second implementation — and the in-memory fake for tests counts, which is usually the honest reason.”

9 Recall card

- **High-level must not depend on low-level; both depend on an abstraction — and the abstraction must not depend on details.** The problem is never “there is no interface”, it is **the direction of the arrow**. Before: `orders/` imports `psycopg`. After: `orders/` and

```
adapters/ both import orders/ports.py .
```

- **The half everyone skips: the interface lives in the *policy's* package, in the policy's vocabulary.** Your `Order`, your `Money`, your `OrderNotFound`. A port in a `persistence/` package, or a **vendor type in a signature**, is a layer that inverted nothing — and it looks identical on a diagram.
- **Three terms, kept apart.** **DIP** = which way source dependencies point. **DI** = passing collaborators in (a technique, 3 lines). **IoC** = who calls whom at runtime (a framework calling your code). `__init__(self, conn: psycopg.Connection)` is injection **without** inversion.
- **The proof is a command:** `grep -rn "psycopg" orders/` returns nothing, and `import orders` works in a bare interpreter. The adapter **translates at its edge** — rows → objects, their errors → your exceptions, their units → your types.
- **Numbers and limits.** Razorpay → Stripe: 38 references in 14 files becomes **1 new file + 1 line**; the declined-payment test goes from impossible to 0.3 ms. Cost: ~108 lines of structure and one hop of indirection — so **skip it when you cannot name the second implementation** (the test fake counts), and remember it buys *source-level* independence, not a free swap.

DSA topic: Sorting revision and mock round **System design topic:** Dependency inversion

Code these, in this order

One rule for the whole set, and it is the only rule today: **run it as a mock**. Set a timer, talk out loud the entire time, and do not stop when you get something wrong. Twenty minutes per problem, including the clarifying questions and the out-loud testing. If nobody is available to sit with you, record yourself — the point is that the words leave your mouth.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Meeting Rooms II	LeetCode 253 (Medium)	That “sort by what?” is the whole question — and this one wants two separate sorted lists.
2	Top K Frequent Words	LeetCode 692 (Medium)	Choosing between sort, heap and quickselect, and defending the choice with arithmetic.
3	Maximum Gap	LeetCode 164 (Hard)	A linear-time requirement, so the comparison sorts are all excluded by the constraints.
4	Merge Intervals	LeetCode 56 (Medium)	The phase’s cleanest “the sort is the algorithm” — and which field you sort on.

The rules for all four

- Three minutes of clarifying first**, out loud, before a line of code. Size, key type, range, nearly sorted, can you modify the input, does it fit in memory.
- State the approach and its cost before writing**. “Brute force is $O(n^2)$, that’s X operations at this size. Sorting makes it $O(n \log n)$ because [reason]. I’ll do that.”
- Narrate while writing**. Say the invariant. Flag the error-prone lines as you reach them.
- Test out loud on five inputs**: empty, one element, all equal, already sorted, reverse sorted.
- Answer the follow-up you know is coming** before it is asked: what if it is a stream, what if the values are bounded, what if you only need the top k .

On problem 1, the sort choice is the interview

There are three defensible solutions — the two-list sweep, the min-heap of end times, and the difference-array sweep from [day 039](#). Solve it one way, then say the other two out loud with their costs and say which you would defend and why.

On problem 2, do the arithmetic before choosing

n distinct words, want the top k by count, ties broken alphabetically. Say all three costs out loud — $O(n \log n)$ full sort, $O(n \log k)$ heap, $O(n)$ quickselect — then say which you would use and what makes the choice. Watch the tie-break: it is a two-key ordering, and one of

them goes descending.

On problem 3, let the constraint pick the algorithm

The problem demands linear time, which rules out every comparison sort by definition. Say that out loud as the first sentence. Then find the pigeonhole argument before coding: with n values and $n - 1$ buckets, the largest gap must span two different buckets, so only each bucket's minimum and maximum matter.

On problem 4, say the key before you type it

Sort by start, not by end, and be able to say what sorting by end would solve instead. Then break it deliberately: sort by `iv[1]` and run it on `[[1, 4], [0, 2], [3, 5]]` to get a plausible wrong answer with no error.

The phase-recall drill

Two minutes each, no reference, code that runs:

1. `insertion_sort`, with the guard in the right order and the strict comparison.
2. `merge`, with both `extend` lines and the `≤`.
3. `merge_sort`, with the base case that covers empty.
4. `partition`, with a random pivot, and say its invariant.
5. `quicksort`, recursing into the smaller side.
6. `quickselect`, iterative, with the correct target index for the k-th largest.
7. `counting_sort`, stable, with the running totals and the backwards placement.

The decision drill

For each, say what you would use and give the cost. Fifteen seconds each:

1. Sort a million integers between 0 and 100.
2. Sort a million arbitrary integers in Python.
3. Find the 5th largest of a million values.
4. Find the top 100 of a billion values arriving one at a time.
5. Sort ten million records by a 32-bit integer id.
6. Sort a linked list of 100,000 nodes.
7. Sort 30 elements inside a tight loop.
8. Sort a nearly-sorted log file of two million lines.
9. Find whether any value appears twice in a million values.
10. Sort a list of records by department, then by name descending.

The arithmetic drill

Answer with numbers, in under ten seconds each:

1. $n \log n$ against n^2 at a million elements. Both numbers and the ratio.

2. Why quickselect is $O(n)$, in one line of algebra.
3. Comparisons for merge sort, quicksort and heapsort at a million elements.
4. Counting sort's operation count and memory for a million values in $O..100$.
5. Counting sort's memory for three values with a maximum of two billion.
6. `sorted()` in C against a hand-written merge sort in Python, at a million elements.
7. Timsort on already-sorted against random input, two million integers.

The trap drill

Trigger each one, read the output, and say the fix in one sentence:

1. Quicksort with a last-element pivot on `list(range(5000))`.
2. Quicksort with `≤` in the partition on `[7] * 3000`.
3. Quicksort recursing on `p` instead of `p - 1`.
4. Merge sort with only one `extend`.
5. Merge sort with `if len(nums) == 1` called on `[]`.
6. Counting sort with a maximum of two billion.
7. `kth_largest` with `k` as the target index.
8. `reverse=True` on a tuple key.
9. `-p.name` in a key function.
10. `nums.sort()[:3]`.

The dependency-inversion drill

Here is the module. Do not refactor it yet.

```
# billing/invoicer.py
import boto3
import pycpg
import weasyprint
from sendgrid import SendGridAPIClient

class Invoicer:
    def issue(self, customer_id: int, month: str) → str:
        conn = pycpg.connect(os.environ["DATABASE_URL"])
        with conn.cursor() as cur:
            cur.execute("SELECT sku, qty, unit_paise FROM usage WHERE cust=%s AND month=%s",
                        (customer_id, month))
            rows = cur.fetchall()
            subtotal = sum(r[1] * r[2] for r in rows)
            total = int(subtotal * 1.18)
            pdf = weasyprint.HTML(string=render("invoice.html", rows=rows, total=total)).write_pdf()
            key = f"invoices/{customer_id}/{month}.pdf"
            boto3.client("s3").put_object(Bucket="invoices", Key=key, Body=pdf)
            SendGridAPIClient(os.environ["SG_KEY"]).send(...)
            return key
```

1. Run the grep in your head: which external packages does the business logic depend on? Count them.
2. Which lines are the actual business rule? There are two. Point at them.
3. List the operations the policy needs from each mechanism — not what the library offers, what this code uses.

4. Write the ports. Which package do they live in, and why does that matter?
5. What types appear in the port signatures? Make sure none of them is a vendor type.
6. Write one adapter fully, including the translation at its edge — their rows to your objects, their errors to your exceptions.
7. Write the composition root.
8. Give the two commands that prove the inversion is real.
9. Write the test for “an eighteen percent tax is applied”, and count the lines of setup before and after.
10. Now argue against yourself: give one scenario where this refactor would be the wrong call.

The injection-versus-inversion drill

For each, say whether it is injection, inversion, both, or neither:

1. `def __init__(self): self.db = psycopg.connect(...)`
2. `def __init__(self, conn: psycopg.Connection): self.db = conn`
3. `def __init__(self, orders: OrderRepository): self._orders = orders`, with `OrderRepository` defined in `persistence/`.
4. `def __init__(self, orders: OrderRepository): self._orders = orders`, with `OrderRepository` defined in `orders/ports.py`.
5. Django calling your view function when a request arrives.
6. A pytest fixture supplying an in-memory repository to a test.

The where-does-the-interface-live drill

For each, say which package the interface belongs in and give the one-line reason:

1. `OrderRepository`, used by `orders/service.py`, implemented by Postgres.
2. `PaymentGateway`, used by `checkout/`, implemented by Razorpay and a fake.
3. `Clock`, used by `subscriptions/`, implemented by the system clock and a fixed clock.
4. `EmailSender`, used by three different modules.
5. `Serializer`, defined by a third-party library you are using.
6. `Comparable`, in a language’s standard library.

Two of those six are cases where you do *not* own the interface. Name them and say what follows.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Run the full script on an unseen problem: three minutes of clarifying, the approach and cost before coding, narration while writing, five test inputs out loud, and the follow-up answered before it is asked.

2. *Your service imports the MySQL driver directly. What is wrong with that?* The direction of the arrow, the three harms, where the interface lives and why that is the half people skip, the adapter as a translator, and the grep that proves it.
 3. *Which sorting algorithm would you actually use, and why?* `sorted()` and the honest reason, then the five cases where you would do something else, ending on “pick two of guaranteed, in place, stable”.
-

Before you move on

- ☐ I ran all four problems as timed mocks, talking out loud, without stopping when I was wrong.
- ☐ I asked clarifying questions about the *data* before coding, every time.
- ☐ I stated the approach and its cost before writing a line, every time.
- ☐ I can write all seven reference implementations from memory in under two minutes each.
- ☐ I triggered all ten traps and can give the one-sentence fix for each.
- ☐ I can answer the decision drill’s ten scenarios without hesitating.
- ☐ I refactored `Invoiceer` and can give both commands that prove the inversion is real.
- ☐ I can state the difference between dependency injection and dependency inversion in one sentence.
- ☐ I answered all three questions above out loud.

PART VIII

Hashing: maps and sets

Days 60 to 67 · 8 chapters

Alongside, on the system design track:

SOLID and design principles, Design patterns

060 Hash tables: how a dictionary finds anything instantly

061 Collisions, and why a hash map can turn slow

062 Sets: membership, deduplication, and the O(1) habit

063 Counting with dictionaries

064 Grouping: the key-design skill

065 Hashing your own objects

066 When a hash map is the wrong answer

067 Hashing revision and mock round

Hash tables: how a dictionary finds anything instantly

DATA STRUCTURES AND ALGORITHMS

Hash tables: how a dictionary finds anything instantly

You can explain hashing, buckets and $O(1)$ average lookup with a drawing.

SYSTEM DESIGN

DRY, KISS, and YAGNI

You can tell real duplication from coincidental similarity.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *How does a hash map work internally?*
- *Is this duplication worth removing?*

Hash tables: how a dictionary finds anything instantly

1 What this is, and why they ask it

A **hash table** finds a value by a key without searching. Instead of looking through the data, it takes the key, runs it through a function that turns it into a number, and uses that number as a position — so it goes straight to where the value must be. That is what a Python `dict` is, and what a `set` is, and it is why `"apple" in d` takes the same time whether the dictionary holds ten items or ten million.

They ask “how does a hash map work internally” more often than any other data-structure question, because it is the one whose *inside* everybody has used and almost nobody has looked at. A good answer has four parts — the hash function, the bucket, the collision, and the resize — and each one has a follow-up waiting behind it. It is also the structure that makes half the earlier phases work: the $O(n)$ Two Sum from [day 051](#), the frequency maps from [day 021](#), the “subarray sum equals K” trick from [day 038](#) — all of them were paying for a dictionary lookup and calling it free. Today is where you find out what you were actually buying.

2 The story

The hostel at the college in Manipal has four hundred boys and one shoe rack in the porch, and whether that rack works or not depends entirely on a rule somebody wrote on a board in 2009.

Before the rule, the rack was a heap. You came back at eleven at night, you put your shoes down wherever there was a gap, and in the morning you looked for them. Finding your own shoes took two minutes on a good day and much longer if somebody had shoved them behind something, and every single morning four hundred boys did that at the same time.

The warden then was a man called Mr Shenoy, and his first attempt was reasonable and did not work. He divided the rack into twenty-six sections, one per letter, and said: your shoes go in the section for the first letter of your name.

It half worked. Nobody had to search the whole rack any more, which was a real improvement. But the S section had about sixty pairs in it, because of all the Sandeeps and Sureshs and Shivas, and the Q section had nothing in it for four years. Boys whose names began with S were still standing there digging through sixty pairs while a boy called Zubin walked up to an empty shelf and was gone in three seconds.

So in 2009 they changed the rule to the one still painted on the board. Take the first letter of your name, take the last digit of your room number, and that pair tells you your slot. Two hundred and sixty slots.

The difference was immediate and it is not really about the number of slots. It is that room numbers spread out evenly across the hostel, so the boys got scattered across the slots instead of piling up at the popular letters. Most slots have one or two pairs. The worst has five.

Nobody searches. You walk in, you already know your slot before you get to the rack, and you look at one or two pairs instead of four hundred.

Two things still happen. Occasionally two boys land on the same slot — same first letter, same last digit — and then there are two pairs there and you glance at both, which takes a second. And when the hostel took in a new block and the numbers went up to five hundred and forty, the slots got crowded, so over one weekend they built a second rack, repainted the rule with more slots, and every single boy had to move his shoes to a new place. That weekend was a nuisance. It has happened twice in sixteen years.

3 The idea in plain English

The rack is a hash table. The rule painted on the board is the **hash function**. A slot is a **bucket**. Two boys landing in the same slot is a **collision**. And the weekend they rebuilt the rack is a **resize**.

The problem it solves

Finding a value in a list of n items means looking at them one at a time — $O(n)$, from [day 012](#). Sorting first buys you $O(\log n)$ with binary search, but only if the data stays sorted and only if the keys have an order.

A hash table does something different. It does not search at all. It **computes** where the item must be, and goes there.

```
d = {"apple": 3, "banana": 7}
d["apple"]          # does not look through anything. It calculates.
```

The three parts

One: the hash function. A function that turns any key into an integer.

```
hash("apple")      # 4159254339421612417    (a different number each run -- see §7)
hash(42)           # 42
hash((1, 2))       # -3550055125485641917
```

A hash function must be three things:

- **Deterministic** — the same key must give the same number every time, within one run of the program. Otherwise you could never find anything again.
- **Uniform** — different keys should spread out across the whole range of numbers. Mr Shenoy's first rule failed here: it was deterministic and fast, but it piled everyone onto S.
- **Fast** — it runs on every single lookup, so it must be cheap.

Two: the bucket. The hash is far too large to be a position, so you take the remainder:

```
bucket = hash(key) % number_of_buckets
```

```
hash("apple") % 8      # some number from 0 to 7 -- that is the slot
```

That is the whole mechanism. Key → number → remainder → position. Nothing is searched.

Three: the collision. Two different keys can land in the same bucket, either because they hash to the same number or — far more often — because two different numbers have the same remainder. That is not a bug and it is not avoidable; with more possible keys than buckets, it is guaranteed.

The standard fix is **chaining**: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket. Two pairs of shoes in one slot.

Note what this means for correctness: **you must store the key, not just the value**. Otherwise you cannot tell which of the two entries in the bucket is yours. That is the detail beginners miss.

Why it is $O(1)$ on average and $O(n)$ in the worst case

If the hash function spreads keys evenly and there are roughly as many buckets as items, each bucket holds about one entry, so a lookup is: compute a hash, take a remainder, check one or two entries. That is **constant time — it does not depend on n at all**.

If the hash function is bad and everything lands in one bucket, a lookup checks all n entries, and you have written a linked list with extra steps. That is $O(n)$, and it is why “what is the worst case of a hash map” is a real interview question with a real answer.

The number that controls this is the **load factor**: the number of items divided by the number of buckets.

```
load factor = items / buckets
```

```
0.25 → most buckets empty; lots of wasted memory
```

```
0.66 → Python's dict resizes at about here
```

```
1.0 → about one item per bucket on average
```

```
5.0 → five per bucket; every lookup checks five things. Still  $O(1)$ , but 5x slower.
```

The resize, and why it is still $O(1)$ on average

When the load factor gets too high, the table allocates a bigger array — usually double — and **rehashes every key into it**, because `hash(key) % 8` and `hash(key) % 16` are different numbers. That single operation is $O(n)$: the weekend the hostel rebuilt the rack.

But it happens rarely, and it happens less often as the table grows. Doubling means that inserting n items causes resizes at 1, 2, 4, 8, ... items, and the total copying work is $1 + 2 + 4 + \dots + n \approx 2n$ — that halving-series sum again, from [day 055](#). Spread over n insertions,

that is a constant amount of work each.

That is called **amortised $O(1)$** , a term you met on [day 005](#) for list `append`: most operations are cheap, a rare one is expensive, and the average over a long run is constant. It is worth saying the word and then explaining it, because saying it alone sounds memorised.

What Python actually does

Python's `dict` is a hash table with two Python-specific things worth knowing:

It uses open addressing, not chaining. On a collision, instead of keeping a list in the bucket, it probes for another empty slot using a sequence derived from the hash. That is [day 061](#)'s subject.

It is compact and insertion-ordered. Since Python 3.7, iterating a dict gives you keys in the order they were inserted, and that is a language guarantee rather than an accident. It comes from storing entries in a dense array with a separate small index array pointing into it — which also makes a dict about a third smaller than it used to be.

What can be a key

A key must be **hashable**, which in practice means immutable:

```
d[("a", 1)] = "fine"      # a tuple is hashable
d[["a", 1]] = "boom"     # a list is not
```

The reason is exactly the rack: your shoes are found by a rule computed from your name. If you could change your name after putting the shoes down, nobody — including you — could find them again. An object whose hash can change is unfindable in the table it is sitting in.

That is why `__hash__` must be built from fields that never change, and why defining `__eq__` without `__hash__` makes Python set `__hash__` to `None` ([day 052](#)).

The costs, all of them

lookup	<code>d[key]</code>	$O(1)$ average, $O(n)$ worst
insert	<code>d[key] = v</code>	$O(1)$ amortised average, $O(n)$ worst
delete	<code>del d[key]</code>	$O(1)$ average, $O(n)$ worst
in	<code>key in d</code>	$O(1)$ average -- checks KEYS, not values
len	<code>len(d)</code>	$O(1)$ -- stored, not counted
iterate	<code>for k in d</code>	$O(n)$, in insertion order
value in <code>d.values()</code>		$O(n)$ ← the one that catches people
space		$O(n)$, with a constant of about 2-3x the raw data

4 The picture

The rack, before and after the rule:

NO RULE – a heap. Finding your shoes means looking at all 400.

[illegible]

RULE 1 – first letter only. Deterministic and fast, but NOT UNIFORM.

A: [[]]	2 pairs	
...		
S: [[]][[]][[]][[]][[]][[]][[]]...	60 pairs	← the Sandeeps, Sureshs, Shivas
...		
Q: []	0 pairs	← four years empty
Z: []	1 pair	

```
lookup for an S: 60 checks.  lookup for a Z: 1 check.
Same structure, 60x difference. That is a BAD HASH FUNCTION.
```

RULE 2 – first letter AND last digit of the room number. 260 slots.

```
A0:[] A1:[][] A2:[] ...
S0:[][] S1:[] S2:[][] S3:[] S4:[][] ...    the 60 S's spread over 10 slots
...
lookup: 1 or 2 checks, for everybody.    That is  $O(1)$ .
```

What to notice: the number of slots is not what fixed it. Rule 1 had twenty-six slots and rule 2 has two hundred and sixty, but the real change is that the room number *spreads people out*. A hash function that is deterministic and fast but not uniform gives you the worst case on the most common keys.

The mechanism, drawn:

```

key          hash function          bucket array (8 buckets)
+-----+    +-----+             +-----+
| "apple" |    | hash() |             | 0 | → [ ]
+-----+    | 4159254339 |         +-----+
              | ...421612417|         | 1 | → [ ("apple", 3) ]
              +-----+             +-----+
              |             |             ^
              |             |             | 2 | → [ ]           the KEY is stored
              |             |             +-----+           too -- or you could
              v             |             | 3 | → [ ("banana", 7), ("grape", 2) ]
              % 8 = 1        +-----+             ^^^^^^^^^^^^^^^
                              | 4 | → [ ]           a COLLISION: two keys,
                              +-----+           one bucket. Check both.
                              | 5 | → [ ("fig", 9) ]
                              +-----+
                              | 6 | → [ ]
                              +-----+
                              | 7 | → [ ("kiwi", 1) ]
                              +-----+

```

d["apple"]: hash it (fast) → take % 8 → go to bucket 1 → compare the key → return 3.
Three steps. NOTHING was searched.

What to notice: bucket 3 holds two entries, so a lookup there compares two keys instead of one. That is a collision, and it costs one extra comparison — not a re-search of the table. And notice the key is stored alongside the value: without it you could not tell “banana” from “grape” once they share a bucket.

The resize:

8 buckets, 6 items → load factor 0.75, too high

BEFORE	AFTER doubling to 16 buckets	
bucket = h % 8	bucket = h % 16	← the SAME key moves!
"apple" → 1	"apple" → 9	
"banana" → 3	"banana" → 3	
"grape" → 3	"grape" → 11	← the collision is resolved for free
"fig" → 5	"fig" → 13	
"kiwi" → 7	"kiwi" → 7	

Every key must be REHASHED and moved: $O(n)$ for that one insert.

But doubling means it happens at 1, 2, 4, 8, 16 ... items:
total copying over n inserts = $1 + 2 + 4 + \dots + n \sim 2n$
spread over n inserts = ~ 2 per insert = $O(1)$ AMORTISED

What to notice: `hash(key) % 8` and `hash(key) % 16` are different, so nothing can stay where it is — the rack really does have to be rebuilt completely. And notice the halving series again: the same `1 + 2 + 4 + ... = 2n` argument that made quickselect linear.

5 The code, built step by step

A hash table from scratch, with chaining

Building it is the fastest way to be able to explain it. Start with the buckets:

```
class HashMap:
    def __init__(self, capacity: int = 8) → None:
        self._capacity = capacity
        self._size = 0
        self._buckets: list[list[tuple]] = [[] for _ in range(capacity)]
```

A list of lists. Each inner list is one bucket and will hold the `(key, value)` pairs that land there. Note `[[] for _ in range(capacity)]` and not `[[]] * capacity` — the second makes eight references to *one* list, which is a genuinely nasty bug.

Now the one line that does all the work:

```
def _bucket_index(self, key) → int:
    return hash(key) % self._capacity
```

That is the hash table. Everything else is bookkeeping.

Lookup

```
def get(self, key, default=None):
    for existing_key, value in self._buckets[self._bucket_index(key)]:
        if existing_key == key:          # compare the KEY -- collisions exist
            return value
    return default
```

One bucket, then a short scan inside it. The `existing_key == key` comparison is why the key has to be stored. And note both `hash` and `==` are used: hashing finds the bucket, equality finds the entry. That is why **equal objects must have equal hashes** — if `a == b` but `hash(a) != hash(b)`, they land in different buckets and the table thinks they are different keys.

Insert, with the update case

```
def put(self, key, value) → None:
    bucket = self._buckets[self._bucket_index(key)]
    for i, (existing_key, _) in enumerate(bucket):
        if existing_key == key:
            bucket[i] = (key, value)          # update in place
            return
    bucket.append((key, value))
    self._size += 1
    if self._size / self._capacity > 0.75:   # the load factor
        self._resize()
```

The loop before the append is what makes `d[k] = 1; d[k] = 2` leave one entry rather than two. Forgetting it is the commonest bug when people write this from scratch.

The resize

```
def _resize(self) → None:
    old = self._buckets
    self._capacity *= 2
    self._buckets = [[] for _ in range(self._capacity)]
    self._size = 0
    for bucket in old:
        for key, value in bucket:
            self.put(key, value)              # rehash: % has a new divisor
```

Every key is re-inserted, because the bucket index depends on the capacity. This is $O(n)$ and it is the operation the amortised argument is about.

Delete

```
def delete(self, key) → None:
    bucket = self._buckets[self._bucket_index(key)]
    for i, (existing_key, _) in enumerate(bucket):
        if existing_key == key:
            bucket.pop(i)
            self._size -= 1
            return
    raise KeyError(key)
```

With chaining, deletion is genuinely simple — remove it from the list. With open addressing it is much harder, and that is one of the trade-offs on [day 061](#).

The complete file

```
"""A hash table with chaining, and the dictionary patterns it makes possible."""

from collections import Counter, defaultdict

class HashMap:
    """A dict, built from scratch. Chaining for collisions, doubling on resize.

    get/put/delete: O(1) average, O(n) worst case (everything in one bucket).
    Space: O(n).
    """

    def __init__(self, capacity: int = 8) → None:
        self._capacity = capacity
        self._size = 0
        self._buckets: list[list[tuple]] = [[] for _ in range(capacity)]

    def _bucket_index(self, key) → int:
        return hash(key) % self._capacity          # THE line. Key → number → position.

    def put(self, key, value) → None:
        bucket = self._buckets[self._bucket_index(key)]
        for i, (existing_key, _) in enumerate(bucket):
            if existing_key == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
        self._size += 1
        if self._size / self._capacity > 0.75:
            self._resize()

    def get(self, key, default=None):
        for existing_key, value in self._buckets[self._bucket_index(key)]:
            if existing_key == key:
                return value
        return default

    def delete(self, key) → None:
        bucket = self._buckets[self._bucket_index(key)]
        for i, (existing_key, _) in enumerate(bucket):
            if existing_key == key:
                bucket.pop(i)
                self._size -= 1
                return
        raise KeyError(key)

    def _resize(self) → None:
        old = self._buckets
        self._capacity *= 2
        self._buckets = [[] for _ in range(self._capacity)]
        self._size = 0
        for bucket in old:
            for key, value in bucket:
                self.put(key, value)

    def __contains__(self, key) → bool:
        return any(k == key for k, _ in self._buckets[self._bucket_index(key)])

    def __len__(self) → int:
        return self._size

    def load_factor(self) → float:
        return self._size / self._capacity

    def bucket_sizes(self) → list[int]:
        """For seeing the distribution -- a good hash gives mostly 0s, 1s and 2s."""
        return [len(b) for b in self._buckets]

    def two_sum(nums: list[int], target: int) → list[int]:
        """The reason hash maps matter. O(n) instead of O(n^2), and it keeps the indices."""
        seen: dict[int, int] = {}
```

```

for i, x in enumerate(nums):
    if target - x in seen:
        return [seen[target - x], i]
    seen[x] = i
return []

def first_non_repeating(text: str) → str | None:
    """Two passes, both O(n). Dict insertion order gives 'first' for free (3.7+)."""
    counts = Counter(text)
    for ch in text:
        if counts[ch] == 1:
            return ch
    return None

def group_anagrams(words: list[str]) → list[list[str]]:
    """A hash map keyed on a computed signature. The key does not have to be the data."""
    groups: dict[tuple, list[str]] = defaultdict(list)
    for word in words:
        signature = tuple(sorted(word)) # a tuple is hashable; a list is not
        groups[signature].append(word)
    return list(groups.values())

if __name__ == "__main__":
    m = HashMap()
    for i, fruit in enumerate(["apple", "banana", "grape", "fig", "kiwi"]):
        m.put(fruit, i)
    print(len(m), m.get("grape"), m.get("mango", -1)) # 5 2 -1
    print("apple" in m, "mango" in m) # True False

    m.put("apple", 99) # update, not insert
    print(len(m), m.get("apple")) # 5 99

    m.delete("apple")
    print(len(m), m.get("apple", "gone")) # 4 gone

    # watch the resize happen
    big = HashMap(capacity=4)
    for i in range(20):
        big.put(f"key{i}", i)
        print(f"{len(big):3} items, {big._capacity:3} buckets, "
              f"load {big.load_factor():.2f}")
    # 1 items, 4 buckets ... resizes at 4, 8, 16, 32 buckets

    print(sorted(big.bucket_sizes(), reverse=True)[:5]) # mostly 1s and 2s

    # a BAD hash function, to see the failure
    class BadKey:
        def __init__(self, name: str) → None:
            self.name = name
        def __hash__(self) → int:
            return 1 # everything in one bucket
        def __eq__(self, other) → bool:
            return isinstance(other, BadKey) and self.name == other.name

    bad = HashMap()
    for i in range(200):
        bad.put(BadKey(f"n{i}"), i)
    print(max(bad.bucket_sizes())) # 200 -- one bucket has everything
    # every lookup is now O(n). Same structure, same code, 200x slower.

    print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
    print(first_non_repeating("swiss")) # w
    print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
    # [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

```

6 What it costs

The lookup, counted

`d[key]`, with a good hash and load factor around 0.66:

1. hash(key)	~constant (for a short string, ~20 ns in C)
2. % capacity	1 operation
3. index into the array	1 operation
4. compare keys in the bucket	~1.3 comparisons on average at load 0.66

	about 4 operations, INDEPENDENT of n

Independent of n is the whole point. At ten items and at ten million items, the same four steps.

Against the alternatives

finding one value among $n = 1,000,000$

list, linear scan	500,000 comparisons on average	$O(n)$
sorted list, bisect	20 comparisons	$O(\log n)$
dict	1 hash + ~1.3 comparisons	$O(1)$

and for 1,000 lookups:

list	500,000,000 comparisons
bisect	20,000
dict	2,300

The gap against the sorted list is smaller than people expect — twenty against one is not the headline. The headline is that the dict needs no sort, no order on the keys, and no maintenance when you insert.

The load factor, priced

average comparisons per lookup, with chaining:

load 0.25	→	~1.1	
load 0.5	→	~1.25	
load 0.75	→	~1.4	← typical resize threshold
load 1.0	→	~1.5	
load 2.0	→	~2.0	
load 10.0	→	~6.0	-- still " $O(1)$ ", and six times slower

memory, for $n = 1,000,000$ entries:

load 0.25	→	4,000,000 buckets	-- 4x the buckets you need
load 0.75	→	1,333,333 buckets	

Load factor is the time-against-memory dial, and 0.66 to 0.75 is where almost every implementation sets it.

The resize, and the amortised argument

```
inserting n items into a table that doubles from 8:

resize at 8 items      : copy 8
resize at 16           : copy 16
resize at 32           : copy 32
...
resize at n/2          : copy n/2
-----
total copying          = 8 + 16 + ... + n/2 ~ n

total work for n inserts = n (the inserts) + n (the copying) = 2n
per insert               = 2 operations = O(1) AMORTISED

BUT: one individual insert can take O(n).
      At a million items, one unlucky insert copies a million entries.
      That is a latency spike, and it matters in a real-time path.
```

Say both halves. “Amortised $O(1)$ ” is the average; the tail latency is real, and it is why some systems pre-size their maps or use incremental rehashing — Redis rehashes a few buckets per operation rather than all at once, precisely to avoid the spike.

Memory

```
a Python dict with 1,000,000 short string keys and integer values:

the raw data (keys + values)      ~ 60 MB
the dict structure                 ~ 80 MB
-----
total                             ~ 140 MB, about 2.3x the data

a list of the same 1,000,000 values ~ 40 MB

The dict costs roughly 3x a list. That is the price of O(1) lookup.
```

The worst case, made concrete

```
200 keys, all hashing to the same bucket:

lookup : 200 comparisons instead of ~1.3
insert : 200 comparisons
n keys : O(n) per operation, O(n^2) to build the table

With n = 100,000 that is 5,000,000,000 comparisons to fill a dict
that should have taken 100,000.
```

That is not theoretical. Before 2012 it was a real denial-of-service technique: send a web form with thousands of parameter names crafted to collide, and the server’s parameter dictionary turns quadratic. The fix is in §7.

7 The traps

The real error: an unhashable key

```
d = {}  
d[["a", "b"]] = 1
```

```
Traceback (most recent call last):  
  File "day60.py", line 2, in <module>  
    d[["a", "b"]] = 1  
    ~^^^^^^^^^^^^^^  
TypeError: unhashable type: 'list'
```

A list can change, so its hash could change, so it could never be found again. Use a tuple. The same error appears with sets, dicts and any object defining `__eq__` without `__hash__`:

```
from dataclasses import dataclass  
  
@dataclass  
class Point:  
    x: int  
    y: int  
  
{Point(1, 2): "here"}
```

```
TypeError: unhashable type: 'Point'
```

`@dataclass` generates `__eq__`, so Python sets `__hash__` to `None` deliberately. Fix it with `@dataclass(frozen=True)`.

The trap that produces no error: mutating a key after insertion

```
class Tag:  
    def __init__(self, name): self.name = name  
    def __hash__(self): return hash(self.name)      # hashes on a MUTABLE field  
    def __eq__(self, other): return self.name == other.name  
  
t = Tag("draft")  
d = {t: "my document"}  
print(t in d)          # True  
  
t.name = "published"   # the hash has now changed  
print(t in d)          # ???
```

```
True  
False
```

The object is sitting in the dictionary and cannot be found — including by itself. It hashes to a different bucket now, so the lookup goes to the wrong slot and finds nothing. Nothing raises, and the entry is unreachable and un-deletable except by iterating.

Hash on fields that never change. This is the rack rule: if you can change your name after leaving your shoes, nobody can find them.

The real error: changing a dict while iterating it

```
d = {"a": 1, "b": 2, "c": 3}
for key in d:
    if d[key] == 2:
        del d[key]
```

```
Traceback (most recent call last):
  File "day60.py", line 2, in <module>
    for key in d:
RuntimeError: dictionary changed size during iteration
```

The iterator holds a position into the internal array; a delete or an insert can move entries or trigger a resize, so the position becomes meaningless. Python detects it and raises rather than giving you nonsense. The fix is to iterate a copy of the keys — `for key in list(d)` — or to build a new dict.

The trap: `in` checks keys, not values

```
d = {"apple": 3}
print("apple" in d)      # True   -- 0(1)
print(3 in d)           # False  -- 3 is a VALUE, not a key
print(3 in d.values())  # True   -- and this is 0(n)!
```

`d.values()` has no hash structure behind it, so searching it is a linear scan. If you need to look up by value, you need a second dictionary keyed the other way — and then you have to keep both in step.

The trap: assuming `0(1)` always

```
class Everything:
    def __hash__(self): return 0
    def __eq__(self, other): return isinstance(other, Everything)
```

Every instance lands in one bucket. Insert `n` of them and the dictionary is a list: each insert compares against everything already there, so building it is `0(n2)`. This is the shape of the hash-collision denial-of-service attack, and the defence in Python is **hash randomisation**:

```
python -c "print(hash('apple'))"
python -c "print(hash('apple'))"
```

```
-8395723518259234171
5731011213473296981
```


Different on every run. Python seeds string hashing with a random value at start-up (since 3.3, on by default), so an attacker cannot precompute colliding keys. Two consequences to remember: **never persist a Python hash value to disk or across processes**, and **never rely on dict iteration order matching hash order** — the insertion-order guarantee is a separate mechanism and it does hold.

The trap: `KeyError` in production

```
counts = {}
counts["apple"] += 1
```

```
Traceback (most recent call last):
  File "day60.py", line 2, in <module>
    counts["apple"] += 1
    ~~~~~^~~~~~
KeyError: 'apple'
```

Three idiomatic fixes, and knowing which to use matters:

```
counts[key] = counts.get(key, 0) + 1      # explicit, works for any dict
counts = defaultdict(int); counts[key] += 1 # cleaner, but ANY read creates the key
counts = Counter(items)                   # best when you are counting
```

The `defaultdict` caveat is worth knowing: reading a missing key inserts it, so `if d[k] == 0` silently grows the dictionary.

The trap: equal objects with unequal hashes

```
class Bad:
    def __init__(self, v): self.v = v
    def __eq__(self, other): return self.v == other.v
    def __hash__(self): return id(self)      # different for every instance

d = {Bad(1): "x"}
print(Bad(1) in d)
```

```
False
```

The two objects are equal, and the dictionary cannot find one from the other, because they hash to different buckets so the equality check never happens. **The contract is one-directional and absolute: if `a == b`, then `hash(a)` must equal `hash(b)`**. The reverse is allowed — unequal objects may share a hash, which is just a collision.

8 In the interview

How it gets asked

- “How does a hash map work internally?” — the direct form, and the answer has four parts: hash function, bucket, collision, resize.
- “Why is lookup $O(1)$? Is it always?” — the follow-up that separates people. Average yes, worst case no, and be able to say when.
- “Implement a hash map.” — LeetCode 706. Chaining is the version to write.
- “What can be a key, and why?” — hashable, meaning effectively immutable, and the reason is the find-it-again argument.
- “What happens when it gets full?” — resize, rehash, and the amortised argument with the doubling sum.
- “Dict or a sorted list?” — the comparison question. Order, range queries and memory decide it.

What to say out loud, in the first ninety seconds

1. **Give the one-sentence mechanism first.** “It doesn’t search — it computes. You run the key through a hash function to get an integer, take that modulo the number of buckets, and that is the position.”
2. **Name the three requirements on the hash.** “Deterministic, so you can find it again. Uniform, so keys spread across buckets. Fast, because it runs on every operation.”
3. **Bring up collisions before being asked.** “Two keys can land in the same bucket — that’s guaranteed, since there are more possible keys than buckets. With chaining, each bucket holds a short list, and a lookup checks the two or three entries there. Which is why the key has to be stored, not just the value.”
4. **Give the complexity with both cases.** “ $O(1)$ average, because with a good hash each bucket holds about one entry. $O(n)$ worst case, if everything lands in one bucket — then it’s a linked list with extra steps.”
5. **Volunteer the resize.** “When the load factor passes about 0.66, it doubles the table and rehashes everything, because `hash % 8` and `hash % 16` differ. That one insert is $O(n)$, but doubling means the total copying over n inserts is about $2n$, so it’s amortised $O(1)$ — with a real latency spike on the unlucky insert.”

The follow-ups

“Why is it $O(1)$? Is it always?” It is $O(1)$ on average, and no, not always, and the distinction is the whole question. The average case rests on two assumptions. The first is that the hash function distributes keys roughly uniformly across buckets — that is the part a hash function has to earn, and a deterministic, fast hash that piles everything onto a few buckets is a real failure mode. The second is that the number of buckets grows with the number of items, which the resize does; if the table never grew, the load factor would rise with `n` and every lookup would get slower. Given both, each bucket holds about one entry, so a lookup is hash, modulo, index,

and about one and a third key comparisons — four operations that do not depend on `n` at all. The worst case is that every key lands in the same bucket, and then a lookup compares against all `n` entries, so operations are $O(n)$ and building the table is $O(n^2)$. That is not hypothetical: before 2012 it was a real denial-of-service technique, where an attacker sent a web form with thousands of crafted parameter names that all collided, and the server's parameter dictionary went quadratic. The defence is hash randomisation — Python seeds string hashing with a random value per process, so `hash('apple')` genuinely differs between runs, which means you must never persist a hash value or rely on it across processes.

“What happens when the table gets full?” It resizes, and the interesting part is why that is still cheap on average. When the load factor — items divided by buckets — passes a threshold, typically around two-thirds, the table allocates a new array of double the size and reinserts every existing key into it. Reinserting is not optional and it is not a copy: the bucket index is `hash(key) % capacity`, so changing the capacity changes where every key belongs. That single insert is therefore $O(n)$. What makes it acceptable is the doubling. Resizes happen at 8, 16, 32, 64 items and so on, so over `n` insertions the total copying is `8 + 16 + ... + n/2`, which sums to about `n` — the same halving-series argument as elsewhere. Total work for `n` inserts is about `2n`, so each insert costs a constant on average, which is what amortised $O(1)$ means. I would add the caveat, because it matters in practice: the average is fine and the tail is not. At a million items, one unlucky insert copies a million entries, and if that lands in a request path it is a visible latency spike. Systems that care about that either pre-size the map when they know the count, or rehash incrementally — Redis moves a few buckets per operation instead of doing it all at once.

“What can be a key, and why?” Anything hashable, which in practice means effectively immutable — numbers, strings, tuples of hashable things, frozensets, and your own objects if you define `__hash__`. Lists, dicts and sets cannot be keys. The reason is the find-it-again argument: the table stores an entry at a position computed from the key's hash, so if the hash can change after insertion, the entry becomes unreachable. The demonstration is unpleasant — take an object that hashes on a mutable field, put it in a dict, then change that field, and `obj in d` returns `False` for an object that is physically sitting in the dictionary. It cannot be found and it cannot be deleted by key, and nothing raises. So the rule is that `__hash__` must be computed from fields that never change — the identity of an entity, not its status. That is also why Python sets `__hash__` to `None` when you define `__eq__` without it: it refuses to guess. And there is a second half to the contract that is absolute: if two objects are equal, they must have the same hash. If they do not, the table looks in different buckets and never even runs the equality check, so an object equal to your key will not find it. The reverse is fine — unequal objects sharing a hash is just a collision, which the table already handles.

A model answer

“The key idea is that it doesn’t search at all — it computes where the value must be. You take the key, run it through a hash function that turns it into an integer, take that integer modulo the number of buckets, and that’s the position in an array. One calculation, then a direct index. That’s why it doesn’t matter whether the map holds ten items or ten million.

The hash function has to be three things. Deterministic, or you’d never find anything again. Fast, because it runs on every operation. And uniform — spreading keys evenly across buckets — and that last one is what people underestimate. A hash that’s deterministic and fast but piles everything onto a few buckets gives you the worst case on your most common keys.

Collisions are guaranteed, because there are more possible keys than buckets — even a perfect hash function has to share a remainder. The standard fix is chaining: each bucket holds a short list of (key, value) pairs, and a lookup checks the two or three entries in that one bucket. That’s why the key is stored alongside the value — with two entries in a bucket you need the key to tell them apart. Python actually uses open addressing rather than chaining, probing for another slot instead of keeping a list, but the idea is the same.

Complexity is $O(1)$ average and $O(n)$ worst case. Average, because with a good hash and a load factor around two-thirds each bucket holds about one entry, so a lookup is a hash, a modulo, an index, and about one comparison — four operations regardless of n . Worst case, if everything lands in one bucket, a lookup compares against all n entries and you’ve written a linked list with extra steps.

When the load factor passes the threshold it resizes: allocate double the buckets and reinsert every key, because `hash % 8` and `hash % 16` are different positions. That insert is $O(n)$, but doubling means resizes happen at 8, 16, 32 items, so the total copying over n inserts is about n and each insert costs a constant on average — amortised $O(1)$. I’d flag the caveat, though: the average is fine and the tail isn’t. One unlucky insert at a million items copies a million entries, and Redis for example rehashes incrementally specifically to avoid that spike.

One last thing: keys must be hashable, which effectively means immutable, because if the hash can change after insertion the entry becomes unfindable — including by the object itself.”

9 Recall card

- **It does not search, it computes:** `bucket = hash(key) % capacity`, then a direct index. That one line is the whole structure. A hash function must be **deterministic** (find it again), **uniform** (spread the keys), and **fast** (it runs on every operation) — and Mr Shenoy’s first-letter rule failed only on uniformity.

- **Collisions are guaranteed** (more keys than buckets). **Chaining** = each bucket holds a short list, so a lookup checks 1-2 entries — which is why the **key is stored alongside the value**. Python uses **open addressing** instead (day 061).
- **O(1) average, O(n) worst case**. Average = 4 operations independent of n, resting on a uniform hash *and* the table growing. Worst = everything in one bucket: n comparisons per operation, O(n²) to build — the real hash-DoS attack, defended by Python's per-process **hash randomisation** (so never persist a hash).
- **Resize when the load factor passes ~0.66**: double the buckets and **rehash everything**, because `% 8` and `% 16` differ. That insert is O(n), but $8 + 16 + 32 + \dots + n/2 \approx n$, so it is **amortised O(1)** — with a real tail-latency spike on the unlucky insert.
- **Keys must be hashable = effectively immutable**. Hash on fields that **never change**, or the entry becomes unfindable while sitting in the table (`obj in d` → `False`, no error). And the absolute contract: `a == b` implies `hash(a) == hash(b)`. `@dataclass` without `frozen=True` sets `__hash__` to `None`; `d.values()` search is **O(n)**; deleting while iterating raises `RuntimeError: dictionary changed size during iteration`.

1 What this is, and why they ask it

Three rules that everybody quotes and most people apply badly. **DRY** — don't repeat yourself — says every piece of *knowledge* should have one authoritative home. **KISS** — keep it simple — says prefer the plainest thing that works. **YAGNI** — you aren't gonna need it — says do not build what nobody has asked for yet.

They ask about DRY specifically because it is the principle that does the most damage when misapplied. Taken as “never write the same characters twice”, it leads people to merge two things that merely look alike, and then those two things need to change in different directions, and the shared function grows a boolean flag, then a second one, then an `if` chain — and the result is worse than the duplication ever was. The sentence a good candidate can say is “**duplication is cheaper than the wrong abstraction**”, and being able to say *when* is what is being tested. YAGNI is the counterweight to open/closed from [day 056](#), and KISS is what stops the whole of SOLID turning into ceremony. These three are where judgement lives.

2 The story

Sulochana cooks for six people in Ernakulam and on a Sunday she makes two curries, one with fish and one with chicken, and for about twelve years she made the masala base for both of them in the same pan at the same time.

It was the obvious thing to do. Onion, ginger, garlic, tomato, the powders — it was the same list for both, near enough. Making it once in a big pan and dividing it took twenty minutes instead of thirty-five, and the kitchen was cooler, and there was one pan to wash instead of two.

Then her son's wife came to stay, and she cannot take much chilli. Not none — some. So the chicken, which is the one she eats, had to come down.

And Sulochana could not do it. The chilli was already in the shared base. Taking it out of the base took it out of the fish as well, and her husband will not eat the fish curry without it and says so every single time.

What she did for a while was worse than either. She made the base as before, split it, and then adjusted each half afterwards — extra chilli into the fish, a bit of coconut into the chicken to calm it. That worked, sort of, and it meant the base was now a thing that was correct for neither curry and had to be corrected twice, and she had to remember which half was which, and one Sunday she got it the wrong way round.

She went back to two pans. It takes thirty-five minutes now instead of twenty. Nobody has got the wrong curry since.

The thing she says about it, and she has said it to her daughter more than once, is that the two bases were never the same base. They looked the same for twelve years, and that was luck — nothing about a fish curry and a chicken curry says the masala has to match. The moment one of them had a reason to change that the other did not have, it turned out she had been making one thing where there were always two.

There is a second story in that kitchen, which is the mixer-grinder attachment her nephew bought her for making the base in one go. Setting it up, feeding it, and washing the three parts afterwards takes longer than a knife. It is in the top cupboard. And there is a very large steel vessel from 2018 that was bought because a hundred people were expected at a function that got postponed and then moved to a hall, and it has never been used.

3 The idea in plain English

Sulochana's shared base is DRY applied to something that was never one thing. The mixer-grinder is a KISS failure. And the vessel from 2018 is YAGNI.

DRY, stated properly

The original wording matters, because the popular version is wrong:

Every piece of *knowledge* must have a single, unambiguous, authoritative representation within a system.

Knowledge. Not characters. Not lines. If the GST rate is eighteen percent, that fact should live in one place, because if it changes it changes everywhere at once, and finding nine copies is how you end up with six of them updated.

Two blocks of code that look identical are only duplication if they encode **the same knowledge**. If they encode two different rules that currently happen to agree, they are not duplication at all — they are two things that look alike, and merging them creates a bug that has not happened yet.

The distinction that matters: real against coincidental

Real duplication. The same fact, written twice. It will always change together, because it is one thing.

```
# in checkout.py
total = subtotal * 1.18

# in invoice.py
total = subtotal * 1.18

# in reports.py
total = subtotal * 1.18
```

One fact — the GST rate — in three places. When it changes to twelve percent, someone will update two of them. **Remove this.**

Coincidental duplication. Two different rules that currently produce the same code.

```
def validate_customer_signup(data):
    if not data.get("email") or "@" not in data["email"]:
        raise ValueError("invalid email")
    if len(data.get("password", "")) < 8:
        raise ValueError("password too short")

def validate_admin_signup(data):
    if not data.get("email") or "@" not in data["email"]:
        raise ValueError("invalid email")
    if len(data.get("password", "")) < 8:
        raise ValueError("password too short")
```

Identical today. But customer sign-up rules are owned by the growth team, and admin sign-up rules are owned by security, and in nine months security will require twelve characters and a second factor while customers keep eight. **Leave this alone.** They are the fish and the chicken.

The test, and it is the same test as the single responsibility principle from [day 055](#):

Would a change to one of these always require the same change to the other? If yes, it is one thing. If you can imagine one changing without the other, it is two.

Or shorter: **do they change for the same reason, and would the same person ask?**

What the wrong abstraction costs

This is the part to be able to describe, because it is what makes the principle real. When you merge two things that need to diverge, the shared function does not stay shared. It grows:

```
def validate_signup(data, is_admin=False):                # month 1: one flag
    ...
def validate_signup(data, is_admin=False, min_password=8): # month 4: two
    ...
def validate_signup(data, is_admin=False, min_password=8,
                    require_mfa=False, allow_plus_addressing=True): # month 9: five
    if is_admin and require_mfa:
        ...
    elif not is_admin and allow_plus_addressing:
        ...
```


Every caller now passes flags describing which of the two things it actually is, and the function has an `if` chain reconstructing the distinction you deleted. **This is strictly worse than the original duplication**, because now the two rules are tangled *and* neither is readable, and untangling them means understanding every caller.

The rule of thumb people use: **the rule of three**. Do not abstract on the second occurrence. Wait for the third, because two points fit any line and three start to show you the shape. The related slogan is **AHA — avoid hasty abstractions** — and Sandi Metz’s line, which is worth quoting because interviewers know it: *duplication is far cheaper than the wrong abstraction*.

Un-abstracting is expensive, which is why the asymmetry matters

Going from duplicated to shared is easy: extract a function, update two call sites, done. Going the other way is not, because by then five callers depend on the shared function’s exact behaviour, three tests exercise it, and nobody is sure which callers rely on which flag. **The cost is asymmetric, so the default should be to wait.**

KISS

Prefer the simplest thing that solves the actual problem.

Simple does not mean short, and it does not mean crude. It means *fewer moving parts, fewer concepts a reader must hold at once*. A forty-line function with no cleverness in it is simpler than a twelve-line one built from three layers of generics.

The practical form is a question: **what is the plainest thing that would work, and what specifically would break?** If you cannot name what breaks, build the plain thing.

Common KISS failures, and each of them started as a good idea:

- a plugin system where a function would do;
- a configuration file for something nobody will ever configure;
- a message queue between two functions in the same process;
- a class hierarchy where a dict of values would do ([day 056](#));
- a caching layer added before anything was measured.

Sulochana’s mixer-grinder is the honest shape of it: it genuinely does the job, and the setup and the washing-up cost more than the knife.

YAGNI

Do not build it until something actually needs it.

The argument is not that the future never arrives. It is that you are usually wrong about *which* future, and being wrong costs three separate things: the time to build it, the weight of carrying it (every reader reads it, every refactor works around it, every test suite runs it), and the fact

that it is almost never right when the requirement finally shows up — because it was designed against a guess.

The concrete signals:

- a parameter with a default that no caller ever overrides;
- an interface with one implementation and no test fake;
- a `type` field on a table with one value;
- a “v2” API path with nothing behind it;
- generic multi-currency, multi-tenant or multi-region support in a product with one of each.

YAGNI is not an argument against design. It is an argument against building *mechanism* for a requirement you cannot name. Choosing where the boundary goes is design and costs nothing; building the plug point behind it is mechanism and costs plenty.

How the three fit together

They pull against each other on purpose, and knowing which wins when is the skill:

DRY pushes toward sharing, abstraction, one place for each fact

KISS pushes toward fewer concepts, plainer code

YAGNI pushes toward less mechanism, built later

DRY vs YAGNI:

DRY says extract the shared thing; YAGNI says wait until you have three and can see the axis.
→ YAGNI wins on the second occurrence. DRY wins on the third.

DRY vs KISS:

removing duplication sometimes ADDS a concept (a base class, a strategy, a generic).
→ If the abstraction is harder to explain than the duplication, KISS wins.

The resolution:

DRY is about FACTS. If it is one fact, one home, always.
If it is two rules that look alike, KISS and YAGNI win.

4 The picture

Sulochana’s two curries:

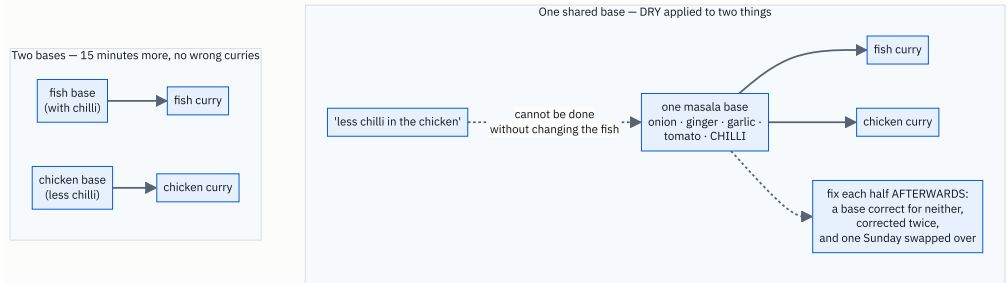


FIGURE 60.1

What to notice: the shared base was correct for twelve years. Nothing was wrong with the decision at the time — what was wrong was that it was never re-examined when one side got a reason to change that the other did not have. That moment is the signal, and it is the only reliable one.

Real against coincidental, side by side:

REAL DUPLICATION — one fact, three homes

```
checkout.py  total = subtotal * 1.18  \
invoice.py   total = subtotal * 1.18  > ONE fact: the GST rate
reports.py   total = subtotal * 1.18  /
```

when it changes to 12%: all three change, together, always.
somebody will find two of them.

→ EXTRACT. `GST_RATE = Decimal("0.18")` in one module.

COINCIDENTAL DUPLICATION — two rules that currently agree

```
customer_signup()  email check + password ≥ 8  \  growth team owns this
admin_signup()     email check + password ≥ 8  /  security owns this
```

month 9: security wants 12 chars + MFA. Growth still wants 8.

→ if you merged them, you now have:

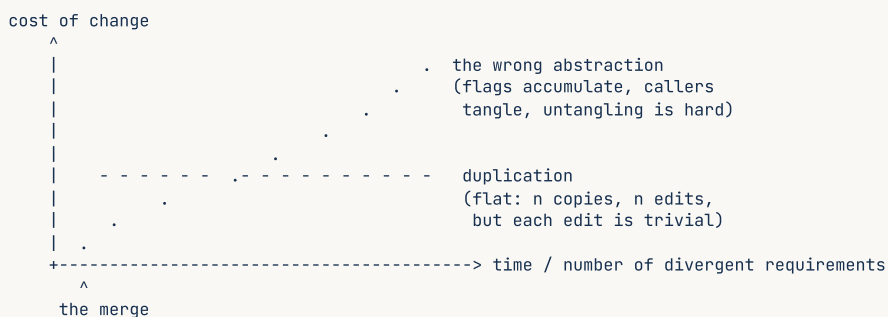
```
validate(data, is_admin=False, min_password=8,
          require_mfa=False, allow_plus_addressing=True)
```

five flags reconstructing the distinction you deleted.

→ LEAVE THEM.

What to notice: the code in both boxes looks the same. What differs is *who would ask for a change*, and that is not visible in the code at all — which is why this judgement cannot be automated and why a linter reporting “duplicate code” is a hint, not an instruction.

The cost curve, which is the argument for waiting:



Duplication's cost is FLAT and known. The wrong abstraction's cost GROWS,
and un-merging is far more expensive than merging was.
That asymmetry is the whole argument for the rule of three.

What to notice: duplication is not free — three copies is three edits. It is that the cost is *predictable*, and the wrong abstraction's is not.

5 How it actually works

The four questions to ask before extracting

Run these in order on any duplication somebody points at.

1. Is it the same knowledge, or the same characters? Say what the fact is, in one sentence. “The GST rate is eighteen percent” is a fact. “Both of these validate an email and a password” is a description of two functions.

2. Would a change to one always require the same change to the other? If you can construct a plausible story where one changes and the other does not, they are two things.

3. Who would ask for the change? Same team, same fact. Different teams, different facts. This is the single responsibility test from [day 055](#) applied to duplication.

4. Is this the third occurrence? Two is a coincidence. Three starts to show the shape, and by then you can see which parts vary and which do not.

Real duplication, removed properly

```
# tax.py -- one home for one fact
from decimal import Decimal

GST_RATE = Decimal("0.18")

def with_gst(subtotal: Decimal) → Decimal:
    """The rate lives here and nowhere else."""
    return subtotal * (1 + GST_RATE)
```

```
# checkout.py, invoice.py, reports.py
from tax import with_gst
total = with_gst(subtotal)
```

Three call sites, one fact, one edit when the rate changes. This is DRY doing exactly what it is for — and note the extraction is *tiny*. Real duplication usually extracts into something small and obviously named, and that is a signal you got it right.

Coincidental duplication, left alone — and what happens if you don't

```
# signup/customer.py -- owned by growth
def validate_customer_signup(data: dict) → None:
    if not data.get("email") or "@" not in data["email"]:
        raise ValidationError("invalid email")
    if len(data.get("password", "")) < 8:
        raise ValidationError("password too short")
```

```
# signup/admin.py -- owned by security
def validate_admin_signup(data: dict) → None:
    if not data.get("email") or "@" not in data["email"]:
        raise ValidationError("invalid email")
    if len(data.get("password", "")) < 8:
        raise ValidationError("password too short")
```

Nine months later, security's rules change. Because these were never merged, the change is four lines in one file that growth never sees:

```
def validate_admin_signup(data: dict) → None:
    if not data.get("email") or not data["email"].endswith("@ourcompany.in"):
        raise ValidationError("admin accounts must use a company address")
    if len(data.get("password", "")) < 12:
        raise ValidationError("password too short")
    if not data.get("mfa_token"):
        raise ValidationError("MFA is required for admin accounts")
```

Had they been merged, the same change would be a fourth flag on a shared function, an `if is_admin` inside it, and a re-test of the customer path — which cannot break and must be verified anyway.

The genuinely shared part, extracted honestly

There is one real fact hiding in there: what a valid email address looks like. That is one piece of knowledge, and it belongs in one place:

```
# validation/email.py
def is_valid_email(value: str | None) → bool:
    """One fact: what this system considers a valid address."""
    return bool(value) and "@" in value and "." in value.split("@")[-1]
```

Both validators call it, and both keep their own password rules. **Extract the fact, not the function.** That distinction — a shared *predicate* rather than a shared *procedure* — is usually where the honest line is.

KISS, as a comparison

Somebody proposes a plugin system for report formats:

```
class ReportPlugin(Protocol):
    def name(self) → str: ...
    def render(self, report: Report, options: dict) → bytes: ...

class PluginRegistry:
    def __init__(self) → None:
        self._plugins: dict[str, ReportPlugin] = {}
    def register(self, plugin: ReportPlugin) → None: ...
    def discover(self, package: str) → None: ... # entry-point scanning
    def render(self, name: str, report: Report, options: dict) → bytes: ...
```

Against:

```

RENDERERS = {"csv": render_csv, "json": render_json}

def render(report: Report, fmt: str) → bytes:
    return RENDERERS[fmt](report)

```

Four lines against forty. The question that decides it is not “which is more flexible” — it is “**do plugins ship separately from this application?**” If third parties write renderers and install them, you need the registry. If two formats exist and both live in this repository, the dict is the answer, and it is still the answer at five formats.

Note this is not an argument against the open/closed principle from [day 056](#). The dict is open for extension — a new format is one entry. KISS is about choosing the cheapest mechanism that achieves it.

YAGNI, made concrete

```

# written in month 1, "because we'll definitely go multi-currency"
@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: str = "INR"
    exchange_rate_date: date | None = None

    def convert_to(self, currency: str, rates: RateProvider) → "Money": ...
    def in_base_currency(self, rates: RateProvider) → "Money": ...

```

Three years on: one currency, `convert_to` never called, `RateProvider` has one implementation returning `1.0`, and every `Money` carries a `currency` field that is always `"INR"` — in the database, in the API responses, in every test fixture.

And when the company genuinely did go multi-currency, the design was wrong anyway, because the real requirement turned out to be that an *order* has a currency and each line inherits it, not that each amount carries its own.

The YAGNI version:

```

@dataclass(frozen=True)
class Money:
    paise: int

```

If multi-currency arrives, adding a field to a frozen dataclass and fixing the compile errors is a day’s work — *with the real requirement in hand*.

Where the three show up in real systems

- **Real DRY:** configuration in one place, database migrations as the single source of schema truth, one shared client library for an internal API, one constants module for business rates.
- **Famous coincidental duplication:** Rails’ and Django’s scaffolded CRUD views look identical for every model and are correctly left duplicated, because each one diverges the moment the model gets a real rule.

- **KISS in the wild:** Go's deliberate lack of generics for a decade, SQLite being one file, the Unix philosophy of small composable programs.
- **YAGNI in the wild:** the "you might need microservices" argument, answered by a great many teams who split a monolith too early and spent a year putting it back.
- **Where DRY is deliberately violated:** across service boundaries. Two microservices sharing a model library are coupled at deploy time, so most teams duplicate the model on purpose. The same applies across bounded contexts from [day 051](#) — a `Customer` in billing and a `Customer` in support are two different things with the same name, and merging them is a classic mistake.
- **And in tests:** test code is deliberately more duplicated than production code, because a test should be readable top to bottom without following three helper functions. A shared test helper that hides the setup makes failures harder to diagnose.

6 The numbers

Real duplication, priced

```
the GST rate in 9 places, changing 18% → 12%:

places to find           : 9  (grep finds 7; two are "0.18" in a formula
                             and one is in a template)
probability all 9 updated
  in one pass            : low -- the empirical answer is "somebody finds 7"
consequence of missing one : wrong invoices, discovered by a customer
cost of extracting       : 1 constant + 1 function + 9 call-site edits, ~1 hour

→ extract. This is what DRY is for.
```

The wrong abstraction, priced

```
two validators merged in month 1, then diverging:

month 1 : merged. 1 function, 0 flags.          24 lines saved.
month 4 : + is_admin flag                      callers: 2 → pass a flag
month 6 : + min_password parameter             callers: 5 now
month 9 : + require_mfa                        an if-chain inside
month 12 : + allow_plus_addressing             5 params, 4 branches

the function           : 24 lines → 71 lines
callers passing flags  : 5
tests of the shared function: 14 (up from 6), most exercising flag combinations
cost to UNTANGLE       : ~3 days -- every caller must be read to find out
                        which flags it relies on

→ 24 lines saved in month 1, ~3 days spent in month 12.
```

The asymmetry, which is the argument

```
duplicate → shared      : extract a function, update the call sites.
                        2 copies: ~20 minutes. 5 copies: ~1 hour.
                        Low risk: the behaviour is identical by construction.

shared → duplicate      : read every caller to determine which behaviour it
                        depends on, split, re-test both paths.
                        ~2-4 days, and it is a behaviour-changing refactor.

ratio: roughly 20-50x more expensive to reverse.
```

That ratio is why the default is to wait. You are choosing between a cheap reversible decision and an expensive irreversible one.

The rule of three, as arithmetic

```
cost of duplication      = n_copies x cost_per_edit x n_future_changes
cost of abstraction     = build_cost + (n_future_changes x cost_per_edit)
                        + P(wrong) x untangle_cost
```

2 copies, 3 future changes, and a 40% chance the abstraction is wrong:

```
duplicate : 2 x 10 min x 3          = 60 min
abstract  : 60 min + (3 x 10) + 0.4 x 2400 min = 1,050 min
→ duplicate
```

5 copies, 10 future changes, 10% chance wrong (the axis is clear by now):

```
duplicate : 5 x 10 x 10            = 500 min
abstract  : 60 + (10 x 10) + 0.1 x 2400 = 400 min
→ abstract
```

The variable that moves most is `P(wrong)`, and the only thing that reduces it is seeing more examples. That is the rule of three, derived rather than asserted.

YAGNI, priced

```
the speculative multi-currency Money:

build cost                : ~2 days
carried cost over 3 years :
    a `currency` column in 6 tables, always "INR"
    the field in ~40 API responses and ~200 test fixtures
    every new developer asks what it is for (~15 min each x 11 people)
    one bug where a test set currency="USD" and nothing complained
value delivered in 3 years : zero
value when it WAS needed   : negative -- the design was wrong,
                           and had to be redone anyway
```


7 The trade-offs

What DRY costs when it is right

Even correct extraction has a price. The caller now depends on a shared module, so a change there reaches everybody — which is exactly what you wanted for a fact and exactly what you did not want for a coincidence. It also adds a hop: reading `with_gst(subtotal)` means opening another file to find out that it is eighteen percent.

I would not extract if the shared thing would be smaller than the call to it. A two-line helper called from two places, where the helper name is longer than the code, adds a concept without removing one.

When to duplicate deliberately

Across service boundaries. Two services sharing a model library are coupled at deploy time — change the library and both must be released together, which is the thing microservices exist to avoid. Most teams duplicate the shared types on purpose and accept the drift.

Across bounded contexts. A `Customer` in billing (has a payment method, a GST number) and a `Customer` in support (has a ticket history, a satisfaction score) are two different concepts wearing one word. Merging them produces a class with twenty fields where each caller uses six ([day 051](#)).

In tests. Test code should be readable in one screen, top to bottom. A shared fixture that hides the setup makes a failure harder to diagnose, and the duplication is cheap because tests rarely change in lockstep. Slightly WET tests are a deliberate, defensible choice.

When the two things are owned by different teams. Even if the code is identical today, shared ownership of one function by two teams is a coordination cost paid on every change.

What KISS costs

Sometimes the simple thing genuinely does not scale, and choosing it means a rewrite later. That is usually the right trade — a rewrite with real requirements beats a design built on a guess — but it is not free, and pretending otherwise is dishonest.

I would not choose the simple version if the failure mode is data loss, a security hole, or something that cannot be fixed forward. Simplicity is a cost-of-change argument, and it does not apply to correctness or safety.

What YAGNI costs

Some things are genuinely much cheaper to build early: an audit trail is nearly free at the start and expensive to backfill; the same is true of a tenant identifier on your tables, of database migrations existing at all, and of tests. YAGNI is about *speculative mechanism*, not about foundations.

I would build it before it is needed if the cost of adding it later is superlinear — a schema column that would require backfilling a billion rows, an identifier that must be present from the first record, or anything the law requires you to have retrospectively.

The honest summary

None of these three is a rule you can apply without judgement, and an interviewer asking about them is asking for the judgement, not the slogan. The most useful sentence in the whole topic:

Duplication is a known, flat cost. The wrong abstraction is an unknown, growing cost that is twenty to fifty times more expensive to reverse. So when you are not sure, duplicate — and let the third occurrence tell you the shape.

8 In the interview

How it gets asked

- “*Is this duplication worth removing?*” — shown two similar functions. The expected answer is a question back, not a yes.
- “*What does DRY actually mean?*” — and the good answer starts with the word *knowledge*.
- “*When would you deliberately duplicate code?*” — the judgement question. Service boundaries, bounded contexts, tests, different owners.
- “*What is YAGNI, and when is it wrong?*” — the second half is what is being tested.
- “*This code is very DRY and nobody can read it. What happened?*” — the wrong abstraction, and the flag-accumulation story.

What to say out loud, in the first ninety seconds

1. **Ask the question rather than answering.** “*It depends on whether these two encode the same knowledge or just happen to look alike right now. Can I ask who owns each of them?*”
2. **State DRY correctly.** “*DRY is about knowledge, not characters — every fact should have one authoritative home. Two blocks that look identical aren’t duplication unless they represent the same fact.*”
3. **Give the test.** “*The question I’d ask: would a change to one always require the same change to the other? If I can imagine one changing alone, they’re two things.*”
4. **Name the cost of getting it wrong.** “*If I merge things that need to diverge, the shared function grows a flag, then another, then an `if` chain reconstructing the distinction I deleted — and untangling that is far more expensive than the duplication ever was.*”
5. **Give the default.** “*So my default is the rule of three. Duplicate on the second occurrence, extract on the third, because by then I can see which parts vary.*”

The follow-ups

“**How do you decide whether duplication is real?**” By asking what fact each piece of code encodes, not by looking at whether the characters match. If I can say the fact in one sentence — “the GST rate is eighteen percent” — and both places encode that same sentence, it is real

duplication and it should have one home, because when the rate changes both must change and somebody will find seven of the nine places. If the best I can say is a description of what the two functions do — “both of these check an email and a password” — that is not a fact, that is a resemblance. Then I ask two more questions. Would a change to one always require the same change to the other? If I can construct a plausible story where one changes alone, they are two things. And who would ask for the change? If it is the growth team for one and the security team for the other, they are two things regardless of how identical they look today, because two owners means two futures. There is usually a genuinely shared *fact* hiding inside coincidental duplication — in that validator example, what counts as a valid email address is one fact and belongs in one place, while the password rules are two. So the honest move is often to extract the predicate and leave the procedures duplicated. Extract the fact, not the function.

“What happens when you get it wrong?” The shared function accumulates flags, and it happens in a way that feels reasonable at every step. Month one, you merge two identical validators and save twenty-four lines. Month four, one caller needs slightly different behaviour, so you add `is_admin=False` — one boolean, hard to object to. Month six, a minimum length parameter. Month nine, an MFA requirement. Month twelve there are five parameters and a four-branch `if` inside, every caller passes flags describing which of the two things it actually is, and the function has reconstructed the distinction you deleted — badly, and in one place instead of two. The function has gone from twenty-four lines to seventy-one and its test count from six to fourteen, most of them exercising flag combinations rather than behaviour. And untangling it takes about three days, because you have to read every caller to work out which flags it depends on, and it is a behaviour-changing refactor rather than a safe one. The number that matters is the asymmetry: going from duplicated to shared takes twenty minutes and is low risk, because the behaviour is identical by construction; going the other way is twenty to fifty times more expensive. That asymmetry is the whole argument for waiting — you are choosing between a cheap reversible decision and an expensive irreversible one.

“When would you deliberately duplicate code?” Four situations, and I would name them as deliberate rather than as failures. Across service boundaries: two services sharing a model library are coupled at deploy time, so a change to the library forces both to be released together, which defeats the point of separating them — most teams duplicate the shared types on purpose and accept the drift. Across bounded contexts, which is the same idea inside one codebase: a `Customer` in billing has a payment method and a GST number, a `Customer` in support has tickets and a satisfaction score, and they are two different concepts sharing a word — merging them gives a twenty-field class where each caller uses six. In tests, deliberately: a test should be readable top to bottom on one screen, and a shared fixture that hides the setup makes a failure much harder to diagnose, so slightly repetitive tests are a defensible choice. And when two teams own the two copies, even if the code is identical today, because shared ownership of one function is a coordination cost on every change. Underneath all four is the same observation: DRY reduces the number of places a fact lives, and it *increases* coupling. When the coupling is the thing you are trying to avoid, duplication is the correct answer rather than a compromise.

A model answer

“My instinct is to ask a question rather than answer, because the answer depends on something that isn’t visible in the code. DRY is about knowledge, not characters — every *fact* should have one authoritative home. Two blocks that look identical are only duplication if they encode the same fact.

So: can I say what the fact is, in one sentence? If it’s ‘the GST rate is eighteen percent’ and it appears in three files, that’s real duplication. It will always change together, someone will find two of the three, and I’d extract it immediately — into a constant and a small function, and the extraction being tiny is a signal I got it right.

If the best I can say is ‘both of these validate an email and a password’, that’s a resemblance, not a fact. Then I’d ask two things. Would a change to one always require the same change to the other? And who would ask for each change? If customer sign-up is owned by growth and admin sign-up by security, those are two rules that currently agree, and merging them means that in nine months, when security wants twelve characters and MFA, the shared function grows a flag. Then another. By month twelve it’s five parameters and an if-chain reconstructing the distinction I deleted, and untangling it takes days because I have to read every caller to find out which flags it relies on.

That’s the asymmetry I’d point at. Going from duplicated to shared is twenty minutes and low risk. Going back is twenty to fifty times more expensive and it’s a behaviour-changing refactor. So the default should be to wait — the rule of three: two occurrences is a coincidence, three starts to show which parts actually vary.

One thing I’d add: there’s usually a real fact hiding inside coincidental duplication. In that example, what counts as a valid email address is one fact and belongs in one place; the password rules are two. Extract the fact, not the function.

And I’d name the cases where I’d duplicate on purpose — across service boundaries, across bounded contexts, and in tests — because in all three the coupling DRY creates is exactly the thing I’m trying to avoid.”

9 Recall card

- **DRY is about *knowledge*, not *characters*:** every **fact** gets one authoritative home. “The GST rate is 18%” in nine files is real duplication — extract it. Two validators that look identical but are owned by growth and by security are **coincidental** — leave them.
- **The test:** *would a change to one always require the same change to the other?* and *who would ask?* Different owners means different futures, however identical the code. And usually there is a real fact hiding inside a coincidence — **extract the fact, not the function** (the email predicate, not the whole validator).

- **The wrong abstraction grows flags.** `is_admin` → `min_password` → `require_mfa` → five parameters and an if-chain rebuilding the distinction you deleted; 24 lines → 71, 6 tests → 14, ~3 days to untangle. **Duplication is far cheaper than the wrong abstraction.**
- **The asymmetry is the argument.** Duplicate → shared: ~20 minutes, low risk. Shared → duplicate: **20-50× more expensive** and behaviour-changing. So use the **rule of three** — two is a coincidence, three shows the shape.
- **KISS = fewest moving parts that solve the actual problem** (a 4-line dict beats a 40-line plugin registry unless plugins ship separately). **YAGNI = no mechanism for a requirement you cannot name** — but build it early when adding it later is *superlinear* (audit trails, tenant ids, migrations). And duplicate deliberately across **service boundaries, bounded contexts, and tests**, where DRY's coupling is the thing you are avoiding.

PRACTICE

Practice — Hash tables: how a dictionary finds anything instantly

DSA topic: Hash tables: how a dictionary finds anything instantly **System design topic:** DRY, KISS, and YAGNI

Code these, in this order

One rule for the whole set: **say what the key is and what the value is before you write anything**. Half the difficulty of hash-map problems is choosing the key, and it is a decision you can state in one sentence.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Design HashMap	LeetCode 706 (Easy)	Whether you can actually build one — buckets, collisions, and storing the key.
2	Two Sum	LeetCode 1 (Easy)	The canonical $O(n^2) \rightarrow O(n)$ trade, and that the dict keeps the indices.
3	Group Anagrams	LeetCode 49 (Medium)	That the key can be <i>computed</i> , and that it must be hashable — a tuple, not a list.
4	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and Python's insertion-order guarantee doing real work.

On problem 1, build it with chaining

Do not use a `dict` inside. Use a list of lists, `hash(key) % capacity`, and store `(key, value)` pairs. Then answer three things out loud: why the key is stored and not just the value, what happens when you `put` a key that already exists, and when you would resize.

Then add the resize and print the load factor after every insert so you can watch it happen.

On problem 2, say both costs before coding

Nested loops is $O(n^2)$ — five hundred billion operations at a million elements. The dict is $O(n)$ with $O(n)$ space. Say both, then say the second reason the dict wins here: sorting would destroy the indices the problem asks for.

On problem 3, get the key right

`sorted(word)` returns a list, and a list cannot be a key. Try it, paste the `TypeError`, then fix it with `tuple(sorted(word))`. Then say out loud why lists are unhashable, using the shoe-rack argument.

Then find a second valid key — a 26-length count tuple — and say which is better and at what word length the answer changes.

On problem 4, notice what you are relying on

Your solution almost certainly relies on dict iteration following insertion order. Say out loud whether that is a guarantee or an accident, and from which Python version.

The build-it drill

Write `HashMap` from memory, then check each:

1. `put` then `get` returns the value.
2. `put` the same key twice leaves one entry, not two.
3. `get` on a missing key returns the default rather than raising.
4. `delete` on a missing key raises `KeyError`.
5. Insert twenty items into a table starting at capacity four, and print capacity and load factor after each. How many resizes?
6. Print `bucket_sizes()` after inserting a hundred string keys. What is the maximum, and what would you expect?

The bad-hash drill

1. Write a key class whose `__hash__` always returns 1.
2. Insert two hundred of them into your `HashMap` and print `max(bucket_sizes())`.
3. Time a lookup against a table with two hundred normal keys.
4. State the complexity of building the table in each case.
5. Say what real-world attack this is, and what Python does to prevent it.
6. Run `python -c "print(hash('apple'))"` twice. What do you see, and what two rules follow from it?

The contract drill

For each, say whether it breaks the hash contract, and what the visible symptom is:

1. `__eq__` defined, `__hash__` not defined.
2. `__hash__` returning `id(self)` while `__eq__` compares a field.
3. `__hash__` computed from a field that changes.
4. `__hash__` returning a constant.
5. `__eq__` that returns `True` for objects with different hashes.
6. A frozen dataclass used as a key.

The break-it drill

Trigger each, read the output, and say the fix:

1. `d[["a", "b"]] = 1` — paste the error.
2. A non-frozen `@dataclass` used as a dict key — paste the error.

3. An object whose hash field is mutated after insertion — what does `obj in d` return, and does anything raise?
4. `del d[key]` inside `for key in d:` — paste the error and give two fixes.
5. `3 in d` where 3 is a value — what does it return, and what is the cost of the version that works?
6. `counts["apple"] += 1` on an empty dict — paste the error and give three idiomatic fixes.

The costs drill

Answer out loud, in under fifteen seconds each:

1. Why is lookup `O(1)`? Name the two assumptions it rests on.
2. What is the worst case, and what causes it?
3. What is the load factor, and where do implementations set it?
4. Why is insert `O(1)` amortised rather than `O(1)`? Give the sum.
5. What does one unlucky insert cost at a million items, and who cares?
6. How much memory does a dict cost compared with a list of the same values?
7. Why must a key be immutable?
8. What is the cost of `value in d.values()`?

The DRY drill

For each pair, say whether it is real duplication or coincidental, name the fact if there is one, and say what you would do:

1. `total = subtotal * 1.18` in three files.
2. `validate_customer_signup` and `validate_admin_signup`, identical today, owned by different teams.
3. Two API handlers that both parse a date from a query parameter the same way.
4. The retry logic in the payment client and in the email client, both “three attempts, exponential backoff”.
5. A `Customer` class in the billing module and a `Customer` class in the support module.
6. The same twelve-line test setup in eight test functions.
7. Two report renderers that both open a file, write a header, write rows, and close.
8. The regex for a valid GST number, in the signup form and in the invoice generator.

Three of those eight are real duplication. Name them.

The wrong-abstraction drill

Take the merged validator with five flags:

```
def validate_signup(data, is_admin=False, min_password=8,
                   require_mfa=False, allow_plus_addressing=True):
    ...
```


1. Describe what this function does in one sentence, with no “and” and no “or”. Can you?
2. How many distinct behaviours does it have? Count the flag combinations that are actually used.
3. Which callers pass which flags? What does that tell you about what the flags really encode?
4. Split it back into two functions. What is duplicated afterwards, and is that duplication real or coincidental?
5. Is there a genuine shared fact inside it? Extract that and only that.
6. Estimate the time for the split, and compare it with the twenty-four lines saved by the original merge.

The YAGNI drill

For each, say whether you would build it now or wait, and give the reason:

1. Multi-currency support in a product selling only in India.
2. A `tenant_id` column on every table, in a single-tenant product that might go multi-tenant.
3. An audit trail of who changed what.
4. A plugin system for report formats, where two formats exist and both live in your repository.
5. A caching layer, before anything has been measured.
6. Database migrations, on day one of a project.
7. A `v2` API path with nothing behind it.
8. An interface over the payment gateway, when there is one gateway and no test fake.

Three of those eight should be built now. Name them and say what makes them different.

The KISS drill

For each pair, say which you would choose and what specifically would have to be true for the other one to win:

1. A dict of functions against a plugin registry with entry-point discovery.
2. A cron job against a message queue, for a task that runs nightly.
3. A single Postgres table against a separate service, for a feature with four fields.
4. A 40-line function with no cleverness against a 12-line one built from three generics.
5. A config file against a hard-coded constant, for something nobody has ever changed.
6. A monolith against microservices, for a team of four.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How does a hash map work internally?* The compute-don't-search sentence, the three requirements on the hash function, collisions and why the key is stored, $O(1)$ average against $O(n)$ worst, and the resize with the amortised argument.
 2. *Is this duplication worth removing?* Ask the question back, state DRY in terms of knowledge, give the would-they-always-change-together test and the who-owns-it test, describe the flag-accumulation failure, and give the asymmetry number.
 3. *Why must a dictionary key be immutable?* The find-it-again argument, the demonstration where `obj in d` returns `False` for an object sitting in the dict, and the `a == b` implies `hash(a) == hash(b)` contract.
-

Before you move on

- ☐ I built a working `HashMap` with chaining and resizing from memory.
- ☐ I can say why the key is stored alongside the value, without being prompted.
- ☐ I made a hash function that always returns 1 and watched every lookup become $O(n)$.
- ☐ I can give the amortised argument for the resize, including the doubling sum.
- ☐ I mutated a key after insertion and confirmed the entry became unfindable with no error.
- ☐ I can classify the eight duplication cases and name the three that are real.
- ☐ I can describe the flag-accumulation failure with its timeline and its cost.
- ☐ I answered all three questions above out loud.

Collisions, and why a hash map can turn slow

DATA STRUCTURES AND ALGORITHMS

Collisions, and why a hash map can turn slow

You can describe chaining and open addressing, and the worst case of each.

SYSTEM DESIGN

Coupling, cohesion, and code smells

You can name what is wrong with a bad module in the vocabulary reviewers use.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *What is the worst-case complexity of a hash map lookup? When does it happen?*
- *Review this module. What would you change?*

Collisions, and why a hash map can turn slow

1 What this is, and why they ask it

A **collision** is two different keys landing in the same bucket. It is not a bug and it is not avoidable — there are always more possible keys than buckets, so some of them must share. Yesterday you met the idea; today is about the two strategies for handling it, **chaining** and **open addressing**, and about what happens to a hash map when collisions stop being occasional.

They ask it because the honest answer to “what is a hash map’s complexity” is $O(1)$ average and $O(n)$ worst, and everybody can say the first half. The interview is in the second: when does the worst case actually happen, what does it look like, and what do real implementations do about it. The answers are specific and interesting — Python probes rather than chains, Java converts a long chain into a tree, deletion under open addressing needs a marker that is not “empty”, and there was a real denial-of-service class of attack built on exactly this that changed how every major language hashes strings. This is also the day the $O(1)$ you have been assuming since [day 021](#) gets its conditions stated.

2 The story

Vinod has been the watchman at a complex of a hundred and twenty flats in Kondapur for nine years, and the thing he actually spends his evenings on is cars.

There are eighty-four bays and a hundred and twenty flats, so it was never going to be one each. The rule the association came up with is on a board by the gate: your bay is the last two digits of your flat number, and if somebody is already in it, you take the next free one along, and keep going. Simple enough that nobody has to be told twice.

For the first few years it worked well. Most people found their own bay empty and parked and went up.

What went wrong went wrong slowly.

Around bay forty, several flats’ numbers happened to land close together, and once forty, forty-one and forty-two were taken, a man arriving for forty-one took forty-three, which meant the man for forty-three took forty-four, and so on. The run grew. By last year the stretch from forty to fifty-two was solid every evening, and a man who should have parked at forty-one was walking to fifty-three. Nobody planned that. Each individual person had followed the rule exactly.

Then came the part Vinod actually had to solve.

When somebody's car has to be moved — a lorry needs to get in, or two cars are blocking each other — he has to find the owner. So he walks from their bay along the row, and if he reaches an empty bay he stops, because under the rule an empty bay means nobody went past it.

In 2021 the family in flat forty-one sold their car. Bay forty-one stood empty. And a man from flat thirty-eight, who years earlier had walked past a full forty-one and parked at forty-seven, became completely unfindable — Vinod would start at thirty-eight, walk three bays, reach the empty forty-one, conclude nobody had gone past, and stop. The car was there. The rule said it could not be.

The fix was a plastic cone. When a bay empties permanently, a cone goes in it, and a cone means: somebody used to be here, keep walking. Not empty. Vacated.

That worked, and then the cones became their own problem. After two years there were eleven of them, and walking from bay thirty-eight past eleven cones to find a car took as long as when the row had been full — because a cone slows you down exactly as much as a parked car does, and unlike a parked car it is not holding anybody.

So one Sunday in 2023 they cleared the whole car park, took every cone away, and started again from an empty row.

3 The idea in plain English

Vinod's car park is **open addressing** with **linear probing**. The solid run from forty to fifty-two is **clustering**. The unfindable car is why deletion is hard. The cone is a **tombstone**. And the Sunday they cleared everything is a **rehash**.

Why collisions are guaranteed

Two separate reasons, and both are worth being able to say.

One: there are more keys than buckets. A dictionary with eight buckets can hold any of infinitely many strings, so keys must share buckets. Even a perfect hash function must map some pair to the same remainder.

Two: collisions arrive far sooner than people expect. With `b` buckets and `n` keys, the chance that no two collide falls very fast. This is the shape of the birthday problem: in a room of twenty-three people, there is about a fifty percent chance two share a birthday, even though there are three hundred and sixty-five days.

```
buckets = 365, keys = 23    → ~50% chance of at least one collision
buckets = 1,000, keys = 38  → ~50%
buckets = 1,000,000, keys = 1,178 → ~50%
```

A million buckets and only about twelve hundred keys, and it is a coin flip. **Collision handling is not an edge case; it is the normal operating condition.**

Strategy one: chaining

Each bucket holds a small collection — a list — of the entries that landed there. A lookup finds the bucket and scans that short list.

```
bucket 3 → [ ("banana", 7) , ("grape", 2) , ("melon", 4) ]
```

- **Lookup cost:** one bucket, then $1 + \text{load factor}$ comparisons on average.
- **Deletion:** easy. Remove it from the list. Nothing else is affected.
- **Load factor above 1 is fine.** With sixteen buckets and forty keys, each chain is about two and a half long, and it still works.
- **The cost:** every entry needs a pointer, so memory is higher, and following pointers is bad for the processor's memory cache — the entries in a chain are scattered rather than adjacent.
- **The worst case:** every key in one bucket, so the map is a list and every operation is $O(n)$.

Java's improvement, which gets asked about: since Java 8, when a chain in `HashMap` reaches eight entries and the table is large enough, that chain is converted into a balanced tree, so the worst case for that bucket becomes $O(\log n)$ rather than $O(n)$. It is a real, shippable answer to the attack priced in §6, and a good thing to know.

Strategy two: open addressing

There are no lists. Every entry lives directly in the array, and when a bucket is taken you **probe** — look at another slot by a rule — until you find a free one. Vinod's rule.

Linear probing: try h , then $h+1$, $h+2$, $h+3$, wrapping round.

```
insert "grape", hash % 8 = 3
  slot 3 taken by "banana" → try 4
  slot 4 free → put it there

lookup "grape"
  slot 3: "banana"? no, keep going
  slot 4: "grape"? yes.

lookup "melon", hash % 8 = 3
  slot 3: "banana"? no
  slot 4: "grape"? no
  slot 5: EMPTY → stop. "melon" is not in the map.
```

That last line is the rule that makes lookups terminate: **an empty slot means the search is over**. Nothing could have been placed past it, because insertion would have stopped there too.

- **Lookup cost:** rises sharply with the load factor, because of clustering.
- **Memory:** better than chaining. No pointers, and the entries are adjacent in memory, so the processor's cache works for you — which is why it is usually faster in practice at sensible load factors.
- **Load factor must stay below 1**, obviously, and in practice below about 0.7.

- **Deletion is hard**, and that is the interesting part.

Clustering: why linear probing degrades

Once a run of consecutive slots is full, it grows faster than it should. Any key that hashes anywhere inside the run has to walk to the end of it and lands just past it — which makes the run one longer, which makes it catch more keys. This is **primary clustering**, and it is Vinod's stretch from forty to fifty-two.

load factor	average probes for a successful lookup (linear probing)
0.25	1.2
0.50	1.5
0.75	2.5
0.90	5.5
0.95	10.5
0.99	50.5

Look at the shape. From 0.5 to 0.75 it merely doubles; from 0.9 to 0.99 it multiplies by nine. **A hash map does not slow down gradually; it falls off a cliff**, and the cliff is why implementations resize at around 0.66 rather than waiting until the table is nearly full.

The fixes for clustering:

- **Quadratic probing**: try `h+1`, `h+4`, `h+9`, `h+16`. The jumps spread keys out so runs do not merge. It removes primary clustering but keys with the *same* initial hash still follow the same path — **secondary clustering**.
- **Double hashing**: the step size itself comes from a second hash of the key, so two keys that start in the same slot walk away in different directions. This is the best of the three, and it is what Python does, in its own form.

Deletion under open addressing, and the tombstone

You cannot simply empty a slot. Vinod's unfindable car is the exact failure:

```
"banana" at 3, "grape" probed past it to 4.  
  
delete "banana" and blank slot 3:  
  
lookup "grape": hash % 8 = 3  
    slot 3: EMPTY → stop. "grape" not found.  
  
The entry is sitting at slot 4 and is now unreachable.
```

The fix is a third state. A slot is **empty**, **occupied**, or **deleted** — a marker meaning “something was here; keep probing”. That marker is a **tombstone**, and it is the cone.

```
lookup : a tombstone means KEEP GOING (it is not the end of the probe)  
insert : a tombstone may be REUSED (it is a free slot)
```

And then the cones accumulate. A tombstone costs a lookup exactly as much as a real entry and holds nothing, so a table with heavy insert-and-delete traffic slowly fills with markers and lookups get slower while the map appears to be shrinking. The fix is to count tombstones and **rehash when there are too many** — the Sunday they cleared the car park.

That is a genuinely good detail to volunteer: “a hash map with a lot of deletions has to rehash even though it is not growing.”

What Python actually does

`dict` and `set` use **open addressing** with a probe sequence that is neither linear nor purely quadratic:

```
perturb = hash(key)
slot = hash(key) % capacity
repeat: slot = (5 * slot + 1 + perturb) % capacity
        perturb >>= 5
```

The `perturb` value carries the *high* bits of the hash, which the initial `% capacity` threw away — so two keys whose hashes agree in the low bits still separate quickly. That is double hashing in spirit, and it is why Python’s `dict` behaves well without needing chains.

Python also uses tombstones — a `DKIX_DUMMY` marker — and rehashes on resize, which cleans them up.

The worst case, stated properly

```
all n keys hash to the same bucket:

chaining      : one chain of n. Lookup O(n). Building the map O(n^2).
open addressing : one long probe run. Same.
Java 8+       : the chain becomes a tree. Lookup O(log n). Building O(n log n).
```

Three ways this happens in real life:

1. **A bad hash function** — one written by hand that uses too little of the key, such as only the first character or only the length.
2. **A pathological key set** — integer keys whose values are all multiples of the capacity, so `key % capacity` is always the same. Sequential integer keys are usually *good*, but keys stepping by exactly the table size are the worst possible input.
3. **An adversary choosing the keys.** This is the real one, and it is \$7.

4 The picture

Linear probing, and how a cluster forms:

capacity 8. Rule: try h, then h+1, h+2 ... wrapping.

```
slot      0      1      2      3      4      5      6      7
start  +-----+-----+-----+-----+-----+-----+-----+
insert "banana" (h=3)
      |      |      |      | ban |      |      |      |      1 probe
insert "grape" (h=3) → 3 taken, try 4
      |      |      |      | ban | gra |      |      |      2 probes
insert "melon" (h=4) → 4 taken, try 5
      |      |      |      | ban | gra | mel |      |      2 probes
insert "fig" (h=3) → 3,4,5 taken, try 6
      |      |      |      | ban | gra | mel | fig |      |      4 probes
                        \-----/
                        a CLUSTER of 4.
```

Any new key hashing to 3, 4, 5 or 6 must walk to the END of the run and lands at 7 -- making the run LONGER, so it catches MORE keys. That is primary clustering, and it feeds itself.

What to notice: the probe count went 1, 2, 2, 4. Nothing was done wrong; the rule was followed exactly each time. Clusters are an emergent property of linear probing, not a mistake.

Why deletion needs a tombstone:

```
state: slot 3 = "banana"(h=3), slot 4 = "grape"(h=3, probed past 3)
```

WRONG -- blank the slot:

```
+-----+-----+-----+-----+
|      |      |      |      | gra |
+-----+-----+-----+-----+
lookup "grape": h=3 → slot 3 is EMPTY → "not found"
The entry is at slot 4 and is UNREACHABLE. No error. Ever.
```

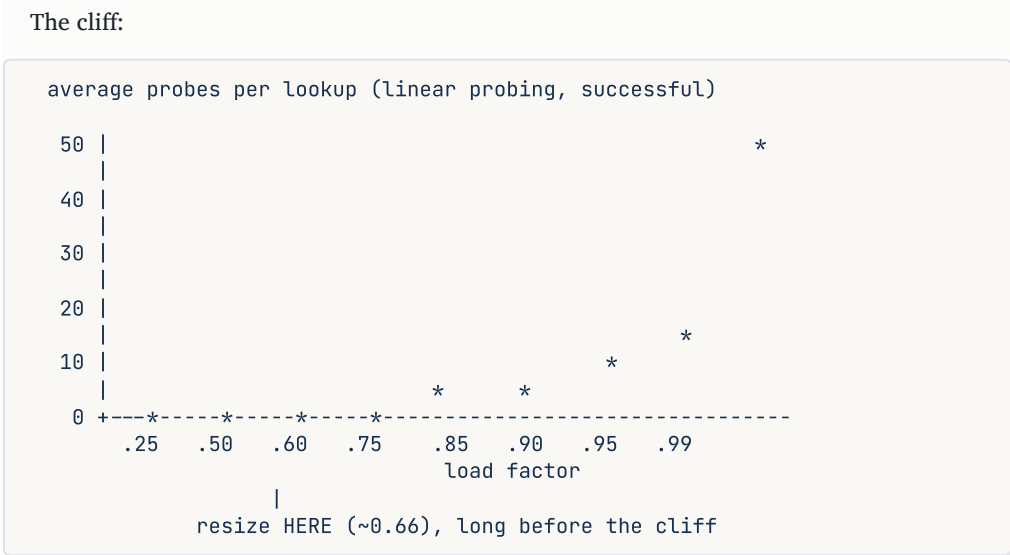
RIGHT -- leave a tombstone:

```
+-----+-----+-----+-----+
|      |      |      | X | gra |      X = "was here, keep going"
+-----+-----+-----+-----+
lookup "grape": h=3 → slot 3 is a tombstone → keep probing
                  → slot 4 = "grape". Found.
insert "kiwi" (h=3): a tombstone is a free slot → reuse it.
```

AND THEN: tombstones cost a probe each and hold nothing.

```
+-----+-----+-----+-----+
| X | X | X | ban | X | X | gra | X |
+-----+-----+-----+-----+
2 real ENTRIES, 6 cones. Lookups are as slow as a full table.
→ REHASH, even though the map is nearly empty.
```

What to notice: the last box. A hash map that is shrinking can need a rehash, which is the opposite of what everybody assumes. Deletions do not make it faster.



What to notice: it is flat and then it is vertical. There is no useful warning region, which is why every implementation resizes at a fixed load factor rather than reacting to slowness.

Chaining against open addressing:

	CHAINING	OPEN ADDRESSING
collision handling	a list in the bucket	probe for another slot
load factor	can exceed 1	must stay < 1; resize at ~0.66
memory	+1 pointer per entry	no pointers, denser
cache behaviour	poor (scattered)	good (adjacent slots)
deletion	trivial: remove it	needs TOMBSTONES
worst case	$O(n)$ ($O(\log n)$ in Java 8+ via treeification)	
used by	Java HashMap, most textbooks	Python dict/set, Go maps, C++ open-addressing tables

What to notice: the deletion row and the cache row are the trade. Open addressing is faster in practice because of memory locality, and it pays for that with tombstones.

5 The code, built step by step

Making a collision happen on purpose

You cannot reason about collisions until you have caused one:

```
class Colliding:
    """Every instance hashes to the same bucket. Equality still distinguishes them."""

    def __init__(self, name: str) → None:
        self.name = name

    def __hash__(self) → int:
        return 42                                # the same for every instance

    def __eq__(self, other: object) → bool:
        return isinstance(other, Colliding) and self.name == other.name

    def __repr__(self) → str:
        return f"Colliding({self.name!r})"
```

```
d = {Colliding(f"k{i}"): i for i in range(5)}
print(len(d))                # 5 -- all five are present and distinct
print(d[Colliding("k3")])    # 3 -- and findable
```

Five entries, one bucket, and the dictionary is still **correct**. That is the first thing to say about collisions: they cost time, never correctness — as long as `__eq__` distinguishes the keys.

Open addressing with linear probing

Build the table with three states, because that is the whole lesson:

```
_EMPTY = object()      # never used
_TOMBSTONE = object()  # was used, then deleted -- KEEP PROBING
```

```
class OpenAddressed:
    def __init__(self, capacity: int = 8) → None:
        self._capacity = capacity
        self._keys: list = [_EMPTY] * capacity
        self._values: list = [None] * capacity
        self._size = 0                # real entries
        self._used = 0                # real entries + tombstones
```

Two counters, and the difference between them is the whole tombstone problem. `_size` is what `len()` reports; `_used` is what decides when to resize.

The probe, which is the heart of it:

```
def _probe(self, key) → tuple[int, bool]:
    """Return (slot, found). Walks until the key, or the first EMPTY slot."""
    slot = hash(key) % self._capacity
    first_tombstone = -1
    for _ in range(self._capacity):
        entry = self._keys[slot]
        if entry is _EMPTY:
            return (first_tombstone if first_tombstone ≥ 0 else slot), False
        if entry is _TOMBSTONE:
            if first_tombstone < 0:
                first_tombstone = slot                # remember it: insert can reuse it
        elif entry == key:
            return slot, True
        slot = (slot + 1) % self._capacity            # linear probing, wrapping
    return (first_tombstone if first_tombstone ≥ 0 else -1), False
```

Three things to say about this while writing it. `_EMPTY` ends the search — nothing could have been placed beyond it. `_TOMBSTONE` does **not** end the search, but it is remembered so an insert can reuse the first one it passed. And `(slot + 1) % capacity` wraps, so the probe never runs off the end.

Insert:

```
def put(self, key, value) → None:
    slot, found = self._probe(key)
    if found:
        self._values[slot] = value          # update in place
        return
    if self._keys[slot] is _EMPTY:
        self._used += 1                    # a fresh slot consumed
    self._keys[slot] = key
    self._values[slot] = value
    self._size += 1
    if self._used * 3 ≥ self._capacity * 2: # load factor 2/3, counting tombstones
        self._resize()
```

`self._used`, not `self._size`, drives the resize. That is the fix for cone accumulation: a table full of tombstones triggers a rehash even though it holds almost nothing.

Delete:

```
def delete(self, key) → None:
    slot, found = self._probe(key)
    if not found:
        raise KeyError(key)
    self._keys[slot] = _TOMBSTONE          # NOT _EMPTY -- that breaks lookups
    self._values[slot] = None
    self._size -= 1                        # _used is unchanged: the slot is still spent
```

One line and one comment, and they are the point of the whole lesson.

Resize, which also sweeps the cones away:

```
def _resize(self) → None:
    old_keys, old_values = self._keys, self._values
    self._capacity *= 2
    self._keys = [_EMPTY] * self._capacity
    self._values = [None] * self._capacity
    self._size = self._used = 0
    for key, value in zip(old_keys, old_values):
        if key is not _EMPTY and key is not _TOMBSTONE:
            self.put(key, value)           # tombstones are simply not copied
```

Measuring the damage

The only way to believe the numbers is to produce them:

```
def average_probes(table: "OpenAddressed", keys: list) → float:
    total = 0
    for key in keys:
        slot = hash(key) % table._capacity
        probes = 1
        while table._keys[slot] is not _EMPTY and table._keys[slot] ≠ key:
            slot = (slot + 1) % table._capacity
            probes += 1
        total += probes
    return total / len(keys)
```

The complete file

```
"""Collisions: linear probing, tombstones, clustering, and where O(1) stops being true."""
_EMPTY = object()
_TOMBSTONE = object()

class OpenAddressed:
    """A hash map with open addressing and linear probing.

    Three slot states: EMPTY (search ends), OCCUPIED, TOMBSTONE (keep probing).
    Resizes on `used` -- entries PLUS tombstones -- so heavy deletion also rehashes.
    """

    def __init__(self, capacity: int = 8) → None:
        self._capacity = capacity
        self._keys: list = [_EMPTY] * capacity
        self._values: list = [None] * capacity
        self._size = 0
        self._used = 0

    def _probe(self, key) → tuple[int, bool]:
        slot = hash(key) % self._capacity
        first_tombstone = -1
        for _ in range(self._capacity):
            entry = self._keys[slot]
            if entry is _EMPTY:
                return (first_tombstone if first_tombstone ≥ 0 else slot), False
            if entry is _TOMBSTONE:
                if first_tombstone < 0:
                    first_tombstone = slot
            elif entry == key:
                return slot, True
            slot = (slot + 1) % self._capacity
        return (first_tombstone if first_tombstone ≥ 0 else -1), False

    def put(self, key, value) → None:
        slot, found = self._probe(key)
        if found:
            self._values[slot] = value
            return
        if self._keys[slot] is _EMPTY:
            self._used += 1
        self._keys[slot] = key
        self._values[slot] = value
        self._size += 1
        if self._used * 3 ≥ self._capacity * 2:
            self._resize()

    def get(self, key, default=None):
        slot, found = self._probe(key)
        return self._values[slot] if found else default

    def delete(self, key) → None:
        slot, found = self._probe(key)
        if not found:
            raise KeyError(key)
        self._keys[slot] = _TOMBSTONE          # NOT _EMPTY
        self._values[slot] = None
        self._size -= 1

    def _resize(self) → None:
        old_keys, old_values = self._keys, self._values
        self._capacity *= 2
        self._keys = [_EMPTY] * self._capacity
        self._values = [None] * self._capacity
        self._size = self._used = 0
        for key, value in zip(old_keys, old_values):
            if key is not _EMPTY and key is not _TOMBSTONE:
                self.put(key, value)
```

```

def __len__(self) → int:
    return self._size

def tombstones(self) → int:
    return sum(1 for k in self._keys if k is _TOMBSTONE)

def probes_for(self, key) → int:
    slot, probes = hash(key) % self._capacity, 1
    while self._keys[slot] is not _EMPTY and self._keys[slot] ≠ key:
        slot = (slot + 1) % self._capacity
        probes += 1
    return probes

class BrokenDelete(OpenAddressed):
    """Deletion done wrong, so you can watch an entry become unreachable."""

    def delete(self, key) → None:
        slot, found = self._probe(key)
        if not found:
            raise KeyError(key)
        self._keys[slot] = _EMPTY          # ← the bug
        self._values[slot] = None
        self._size -= 1

class Colliding:
    """Every instance shares one bucket. Correct, and 0(n)."""

    def __init__(self, name: str) → None:
        self.name = name

    def __hash__(self) → int:
        return 42

    def __eq__(self, other: object) → bool:
        return isinstance(other, Colliding) and self.name == other.name

    def __repr__(self) → str:
        return f"Colliding({self.name!r})"

if __name__ == "__main__":
    # 1. collisions cost time, never correctness
    d = {Colliding(f"k{i}"): i for i in range(5)}
    print(len(d), d[Colliding("k3")])          # 5 3

    # 2. the deletion bug, made visible
    broken = BrokenDelete(capacity=8)
    a, b = Colliding("a"), Colliding("b")      # both hash to 42 % 8 = 2
    broken.put(a, 1)
    broken.put(b, 2)                          # probes past a, lands at 3
    print(broken.get(b))                      # 2
    broken.delete(a)                          # blanks slot 2
    print(broken.get(b, "UNREACHABLE"))       # UNREACHABLE ← no error!
    print(len(broken))                       # 1 -- it thinks b is still there

    fixed = OpenAddressed(capacity=8)
    fixed.put(a, 1)
    fixed.put(b, 2)
    fixed.delete(a)
    print(fixed.get(b, "UNREACHABLE"))        # 2 -- the tombstone saved it

    # 3. clustering, measured -- fill directly, so no resize hides the effect
    for target_load in (0.25, 0.5, 0.75, 0.9):
        t = OpenAddressed(capacity=4096)
        chosen = [f"k{i}" for i in range(int(4096 * target_load))]
        for k in chosen:
            slot = hash(k) % t._capacity
            while t._keys[slot] is not _EMPTY:
                slot = (slot + 1) % t._capacity
            t._keys[slot] = k
        avg = sum(t.probes_for(k) for k in chosen) / len(chosen)

```

```

    print(f"load {target_load:.2f} → {avg:.2f} probes per lookup")
# load 0.25 → ~1.2 / 0.50 → ~1.5 / 0.75 → ~2.5 / 0.90 → ~5.5

# 4. tombstones accumulating
t = OpenAddressed(capacity=64)
for i in range(20):
    t.put(f"x{i}", i)
for i in range(18):
    t.delete(f"x{i}")
print(len(t), t.tombstones())          # 2 18 -- 2 entries, 18 cones

# 5. Python randomises string hashing every run
print(hash("apple"))    # a different number in every process

```

6 What it costs

The average, by load factor

successful lookup, average number of probes

load	chaining	linear probing	double hashing
0.25	1.13	1.17	1.15
0.50	1.25	1.50	1.39
0.66	1.33	2.00	1.66
0.75	1.38	2.50	1.85
0.90	1.45	5.50	2.56
0.95	1.48	10.50	3.15
0.99	1.50	50.50	4.65

Two readings. **Chaining degrades gently and linear probing falls off a cliff** — and yet open addressing is what most modern implementations use, because at a load factor of 0.66 the difference is two probes against one and a third, and those two probes are on *adjacent memory*, which the processor's cache handles far better than following two pointers. The theory favours chaining; the hardware favours probing.

The worst case, priced

$n = 100,000$ keys, all colliding into one bucket:

```

building the map : 1 + 2 + 3 + ... + n = n(n-1)/2 = 5,000,000,000 comparisons
one lookup      : 100,000 comparisons
vs a good hash  : 1 lookup ~1.3 comparisons

```

76,000x slower per lookup, with identical code and identical data volume.

The tombstone problem, priced

```
capacity 1,024, workload: insert 600, delete 590, insert 590 more...

after a while : ~10 live entries, ~600 tombstones
load factor by SIZE   : 0.01  -- "the map is nearly empty"
load factor by USED   : 0.60  -- what lookups actually experience
average probes       : ~2.5, on a map holding ten things

Resize on `used`, not on `size`, or a shrinking map gets slower.
```

Java's treeification, priced

```
a bucket with 1,000 colliding entries:

plain chain       : 1,000 comparisons per lookup
red-black tree   : log2(1000) ~ 10 comparisons

100x, and it turns a denial-of-service attack into a slowdown.
Java converts a chain to a tree at 8 entries (and back at 6).
```

The attack, with real numbers

```
Hash-flooding, roughly as demonstrated in 2011:

send an HTTP POST with 20,000 form fields whose names all collide
the server builds a dict of the parameters:
  normal keys : 20,000 inserts x ~1 comparison = 20,000 operations
  colliding   : 20,000 x 20,000 / 2           = 200,000,000 operations
→ ~1 second of pure CPU, from a request of a few hundred kilobytes

a handful of such requests per second saturates a core.
Affected: PHP, Java, Python, Ruby, ASP.NET, node.js -- all of them.
```

The fix, adopted almost everywhere: **randomise the hash per process**, so the attacker cannot precompute colliding keys. Python does this for strings and bytes by default since 3.3, seeded at start-up (`PYTHONHASHSEED` can pin it, for reproducible tests only). Many languages went further and adopted **SipHash**, a keyed hash designed to be unpredictable without a secret.

Memory

```
1,000,000 entries

chaining       : array of 1,000,000 pointers + 1,000,000 nodes
                  each node = key + value + next pointer ~ 56 bytes
                  total ~ 64 MB

open addressing : one array, 1,000,000 / 0.66 = 1,515,000 slots
                  each slot = key + value ~ 32 bytes
                  total ~ 48 MB, and contiguous
```

7 The traps

The real failure: deleting by blanking the slot

```
broken = BrokenDelete(capacity=8)
a, b = Colliding("a"), Colliding("b")    # both land in the same slot
broken.put(a, 1)
broken.put(b, 2)                          # probes past a
print(broken.get(b))
broken.delete(a)
print(broken.get(b, "UNREACHABLE"))
print(len(broken))
```

```
2
UNREACHABLE
1
```

The entry is physically in the array, the map reports a length of 1, and the value cannot be retrieved by its key. **No exception, ever.** This is the single most important thing to know about open addressing, and it is why the tombstone exists.

The trap: resizing on `size` instead of `used`

```
if self._size * 3 ≥ self._capacity * 2:    # ← wrong counter
```

A workload that inserts and deletes in equal measure keeps `_size` low forever, so no resize is ever triggered, and the table fills with tombstones until every lookup walks half the array. The map looks empty and behaves as though it were full. Count tombstones towards the resize.

The real error: mutating a key so it lands in the wrong bucket

```
class Tag:
    def __init__(self, name): self.name = name
    def __hash__(self): return hash(self.name)
    def __eq__(self, other): return self.name == other.name

t = Tag("draft")
d = {t: "doc"}
t.name = "published"
print(t in d, len(d))
```

```
False 1
```

The entry is in the map and unreachable by any key, including the object that created it. Same failure as a wrong deletion, from the other direction. **Hash on fields that never change.**

The trap: a hash function using too little of the key

```
class Order:
    def __init__(self, order_id: str) → None:
        self.order_id = order_id          # "ORD-2026-000123"
    def __hash__(self) → int:
        return hash(self.order_id[:4])    # ← "ORD-" for every single order
    def __eq__(self, other) → bool:
        return self.order_id == other.order_id
```

Deterministic, fast, and every order in the system lands in one bucket. Nothing raises; the map is correct and quadratic. **A hash must use the parts of the key that actually vary.** The safe default in Python is to hash a tuple of the identifying fields:

```
def __hash__(self) → int:
    return hash((self.order_id,))
```

The trap: integer keys that step by the capacity

```
d = {}
for i in range(0, 1_000_000, 8):    # every key is a multiple of 8
    d[i] = i
```

`hash(n) = n` for small integers in Python, so with a capacity that is a power of two, keys stepping by eight all share the low bits. Python's probe sequence mixes in the *high* bits precisely to survive this, but a hand-written table with `hash(key) % capacity` and linear probing degrades badly. **Sequential integers are fine; integers with a stride equal to the table size are the worst case.**

The trap: relying on a hash value across runs or processes

```
python -c "print(hash('apple'))"
python -c "print(hash('apple'))"
```

```
-8395723518259234171
5731011213473296981
```

String hashing is seeded randomly per process. **Never persist a Python hash to disk, never send one over the network, never use one as a database key, and never write a test that asserts a specific hash value.** If you need a stable digest, use `hashlib` — `hashlib.sha256(s.encode()).hexdigest()` — which is deterministic across runs and machines.

The trap: assuming a set makes duplicate detection safe against an adversary

```
def has_duplicates(items: list) → bool:
    return len(set(items)) ≠ len(items)
```

$O(n)$ on normal input, $O(n^2)$ if the caller supplies items crafted to collide. If `items` comes from a request body and the objects are user-controlled with a custom `__hash__`, this is a denial-of-service vector. For untrusted input, either use built-in types (which are hash-randomised) or bound the input size.

8 In the interview

How it gets asked

- “What is the worst-case complexity of a hash map lookup? When does it happen?” — the direct form. $O(n)$, and the three causes.
- “How do you handle collisions?” — chaining and open addressing, with the trade between them.
- “How would you delete from an open-addressed table?” — the tombstone question, and the best answer starts by showing what goes wrong without one.
- “Why does a hash map get slow before it gets full?” — clustering, and the load-factor cliff.
- “Why does Python’s `hash('a')` change between runs?” — hash randomisation, and the attack behind it.
- “Your dictionary lookups got slow in production. Diagnose it.” — the open-ended version, and the answer is a checklist.

What to say out loud, in the first ninety seconds

1. **Answer both halves immediately.** “ $O(1)$ average, $O(n)$ worst case. The worst case is when every key lands in the same bucket, and then the map is a linked list with extra steps.”
2. **Say collisions are normal, not exceptional.** “Collisions are guaranteed — more possible keys than buckets. And they arrive early: with a million buckets, about twelve hundred keys gives you a fifty percent chance of one. It’s the birthday problem.”
3. **Give the two strategies and the trade.** “Chaining puts a list in each bucket — deletion is trivial and the load factor can exceed one. Open addressing probes for another slot — denser, much better cache behaviour, and that’s why Python and Go use it, but deletion needs tombstones.”
4. **Volunteer the tombstone before being asked.** “With probing you can’t blank a deleted slot, because a lookup stops at the first empty one — so an entry that probed past it becomes unreachable, with no error. You leave a marker meaning ‘keep going’.”
5. **Name the three real causes of the worst case.** “A hand-written hash using too little of the key. Keys whose values step by exactly the table size. And an adversary choosing the keys — which was a real DoS class in 2011, and is why Python randomises string hashing per process.”

The follow-ups

“How would you implement deletion in an open-addressed table?” I would start by saying what goes wrong with the obvious version, because that is the whole question. A lookup probes from the key’s home slot and stops at the first empty slot, because under the insertion rule nothing could have been placed past an empty one. So if I delete an entry by blanking its slot, any entry that probed *past* that slot during insertion becomes unreachable — the lookup walks to the newly emptied slot, sees empty, and reports not found. The entry is physically in the array, `len()` still counts it, and nothing raises. So a slot needs three states rather than two: empty, occupied, and deleted. The deleted marker is called a tombstone, and it means “something was here, keep probing”. A lookup treats it as occupied and continues; an insert treats it as free and may reuse it, though it should remember the *first* tombstone it passed and keep probing to check the key is not already present further along — otherwise you get two entries for one key. The second half of the answer is that tombstones then become their own problem: each one costs a probe and holds nothing, so a workload with heavy insertion and deletion fills the table with markers while the map appears to be shrinking, and lookups get slower. The fix is to track two counters — live entries, and entries plus tombstones — and to trigger the resize on the second one. That means a hash map that is shrinking can still need a rehash, which surprises people and is the correct behaviour.

“Why does a hash map get slow before the table is full?” Because of clustering, and it is specific to probing. With linear probing, once a run of consecutive slots is occupied, any key hashing anywhere inside that run has to walk to the end of it and lands just past — which makes the run one longer, which makes it catch more keys. It feeds itself. That is primary clustering, and the effect on lookup cost is not gradual: the average number of probes for a successful lookup goes roughly 1.5 at load 0.5, 2.5 at 0.75, 5.5 at 0.9, 10.5 at 0.95, and about 50 at 0.99. It is flat and then vertical, with no useful warning region — which is exactly why every implementation resizes at a fixed load factor around two-thirds rather than waiting to notice slowness. There are two standard mitigations. Quadratic probing jumps by 1, 4, 9, 16 so runs do not merge, which removes primary clustering, though keys with the same starting slot still follow the same path — secondary clustering. Double hashing derives the step size from a second hash of the key, so two keys starting in the same slot walk off in different directions, and that is the best of the three. Python does something in that spirit: its probe sequence mixes in the high bits of the hash that the initial modulo threw away, so keys agreeing in the low bits still separate quickly.

“Why does `hash('apple')` give a different answer every time I run Python?” Because string hashing is seeded with a random value at interpreter start-up, and that is a security feature rather than an accident. The attack it defends against is hash flooding. Around 2011 it was demonstrated against essentially every web stack — PHP, Java, Python, Ruby, ASP.NET, node — and the mechanism is simple: the server puts request parameters into a hash map, so an attacker sends a form with twenty thousand field names crafted to hash to the same bucket. Insertion then degrades to quadratic, so twenty thousand parameters cost two hundred million comparisons instead of twenty thousand — about a second of CPU for a request of a few hundred kilobytes, and a handful per second saturates a core. Randomising the seed per process

means the attacker cannot precompute a colliding set, because they do not know the seed; several languages went further and adopted SipHash, a keyed hash designed to be unpredictable. Two practical consequences follow and both matter. You must never persist a Python hash value, send it over a network, use it as a database key, or assert on it in a test, because it is only stable within one process — if you need a stable digest, that is `hashlib`. And it does not affect dictionary iteration order, because Python's insertion-order guarantee is a separate mechanism and it does hold across runs. Java took a different route to the same problem: it converts a chain of eight or more entries into a balanced tree, so a flooded bucket degrades to $O(\log n)$ rather than $O(n)$.

A model answer

“ $O(1)$ on average, $O(n)$ in the worst case, and the worst case is when every key ends up in the same bucket — at which point the map is a linked list with extra steps and building it is $O(n^2)$.”

The thing I’d say first is that collisions themselves are normal rather than exceptional. There are always more possible keys than buckets, so some must share, and they arrive much earlier than people expect — it’s the birthday problem, so with a million buckets you have about a fifty percent chance of a collision by around twelve hundred keys. Collision handling is the operating condition, not an edge case. And collisions only ever cost time, never correctness, provided equality distinguishes the keys.

There are two strategies. Chaining keeps a small list in each bucket, so lookup checks one bucket and scans a short list. Deletion is trivial and the load factor can go above one. Open addressing has no lists — on a collision it probes for another slot by a rule. It’s denser, and because the slots are adjacent in memory the processor’s cache works for you, which is why it’s usually faster in practice and why Python and Go both use it. What it costs is deletion.

That’s the part worth showing rather than stating. A lookup probes from the home slot and stops at the first empty slot, because nothing could have been placed past one. So if I delete by blanking a slot, an entry that probed past it during insertion becomes unreachable — it’s sitting in the array, `len()` still counts it, and the lookup reports not found with no error. So slots need three states, and the third is a tombstone meaning ‘keep probing’. Then tombstones become their own problem, because each one costs a probe and holds nothing — so I’d count live entries and entries-plus- tombstones separately and resize on the second. Which means a shrinking map can need a rehash.

On when the worst case actually happens: three ways. A hand-written hash that uses too little of the key — I’ve seen one that hashed the first four characters of an order id, and every order started ‘ORD-’. Integer keys stepping by exactly the table size. And an adversary choosing the keys, which was a real denial-of-service class in 2011 — twenty thousand colliding form fields turning twenty thousand operations into two hundred million. That’s why Python randomises string hashing per process, and why you must never persist a Python hash value.”

9 Recall card

- **Collisions are guaranteed and normal** — more keys than buckets, and they arrive early (birthday problem: 10^6 buckets, $\sim 1,178$ keys $\rightarrow$ 50%). They cost **time**, **never correctness**, as long as `__eq__` distinguishes the keys.

- **Chaining:** a list per bucket. Deletion trivial, load factor may exceed 1, degrades *gently* (~ 1.5 probes even at load 0.99), but scattered memory. **Java 8+** turns a chain of 8 into a tree $\rightarrow O(\log n)$. **Open addressing:** probe for another slot. Denser, cache-friendly — what **Python and Go** use — and it falls off a **cliff** (1.5 probes at load 0.5 $\rightarrow$ 2.5 at 0.75 $\rightarrow$ 10.5 at 0.95 $\rightarrow$ ~ 50 at 0.99), which is why everyone resizes at ~ 0.66 .
- **You cannot delete by blanking a slot.** A lookup stops at the first EMPTY slot, so anything that probed past becomes **unreachable with no error** while `len()` still counts it. Use a **tombstone** — empty / occupied / deleted — which lookups skip and inserts may reuse.
- **Tombstones then accumulate:** each costs a probe and holds nothing, so **resize on** `entries + tombstones`, not on `len` — a *shrinking* map can need a rehash.
- **Three real causes of $O(n)$:** a hash using too little of the key (`hash(order_id[:4])`) · integer keys stepping by exactly the capacity · **an adversary** — 20,000 colliding form fields turned 20,000 operations into 200,000,000 in the 2011 hash-flooding attacks. Hence Python's per-process **hash randomisation**: never persist, transmit, or assert on a `hash()` value — use `hashlib`.

1 What this is, and why they ask it

Coupling is how much one piece of code has to know about another. **Cohesion** is how much the things inside one piece of code belong together. The goal, in the oldest slogan in software design, is **low coupling and high cohesion** — modules that mind their own business, each containing things that are used together.

Code smells are the named symptoms. Feature envy, shotgun surgery, primitive obsession, data clumps, message chains, god object. They are not rules and they are not bugs; each one is a recognisable pattern that usually means a boundary is in the wrong place.

They ask you to review a module because it is the only question in the design round that tests vocabulary and judgement together. Anyone can say “this is messy”. A candidate who says “*changing an address touches five files, which is shotgun surgery — the address logic is spread out, and it should be one `Address` type*” has named the problem in words the reviewer already uses, and has said what to do. This is also the last day of the principles phase, and coupling and cohesion are the ideas underneath all five of them: single responsibility is cohesion, dependency inversion is coupling direction, interface segregation is coupling width.

2 The story

Anjali and her husband moved into a rented flat in Kothrud in June, and there were two things about it that she could not stop noticing, and neither of them was a fault exactly. The flat was clean and the taps worked.

The first was the kitchen. Whoever had put it together had done it by size. All the large things in the cupboard by the window, all the small things in the drawers by the door, all the tall things in the corner unit. It looks tidy. It photographs well.

But she makes tea every morning, and making tea in that kitchen meant: the strainer from the drawer by the door, the small pan from the cupboard by the window, the tea from the corner unit, the sugar from a different shelf in the corner unit, and the milk from the fridge, which is behind you when you are at the stove. Five places, four of them not near the stove, every single morning. Nothing was lost. Everything was findable. It just took nine or ten minutes for something that should take four, and she did it twice a day for four months.

The second thing was the light switches, and this one made her genuinely angry twice.

The switchboard in the bedroom has four switches. One of them does the bedroom tube light. One does the fan. And one does both the balcony light and her husband’s reading lamp, on the same switch, on the same line, because whoever wired the flat ran them together.

So on any night she wanted the balcony light off — which is every night, because it shines into the bedroom — she had to turn off his reading lamp too. He would be reading. She would ask. He would sigh. Twice they had a proper argument about it, at eleven at night, about a light.

When they moved to their own flat in November, she did two things before the furniture came in.

She put the kitchen together by job instead of by size. Everything for tea — the pan, the strainer, the tea, the sugar — in the cupboard directly above the stove, and the sugar jar next to the tea jar rather than with the other jars. Everything for the pressure cooker in one place. It looks less tidy. Tea takes four minutes.

And she stood over the electrician for an afternoon and made him give the balcony light its own switch. It cost eleven hundred rupees. Nobody has argued about a light since.

3 The idea in plain English

The kitchen arranged by size is **low cohesion** — things used together kept apart, so every job takes trips. The balcony light on the reading lamp's switch is **tight coupling** — two things wired together that have no reason to be, so you cannot change one without changing the other.

Cohesion: do the things inside belong together?

High cohesion means everything in a module is there for the same job. You can say what the module is for in one sentence with no “and” — which is the single responsibility test from [day 055](#), arriving from the other direction.

There is a standard ladder, worst to best, and knowing the names of the bottom rungs is genuinely useful because they describe real modules:

COINCIDENTAL	things are together for no reason.	utils.py, helpers.py, common.py
LOGICAL	things are together because they are the same KIND.	
	"all the validators", "all the parsers" -- the kitchen by size	
TEMPORAL	things are together because they happen at the same TIME.	
	init.py: open the log, connect the DB, load config, warm the cache	
PROCEDURAL	they run in a sequence, but on unrelated data	
COMMUNICATIONAL	they operate on the SAME data	
FUNCTIONAL	they all contribute to ONE well-defined job	← aim here

Anjali's first kitchen is **logical cohesion** — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind. A `validators.py` holding every validator in the system is exactly that: it looks organised, and every feature has to reach into it.

The tell for low cohesion is **trips**. To do one job you open five files. To understand one flow you read four modules. Anjali's nine-minute tea.

Coupling: how much does one module know about another?

Low coupling means a module can be understood, changed and tested with minimal knowledge of the others. There is a ladder here too, and this one gets quoted in reviews:

CONTENT	A reaches into B's internals -- reading private fields, monkey-patching
COMMON	A and B share global mutable state
CONTROL	A passes B a flag that decides WHICH BRANCH B takes render(report, as_pdf=True) -- the caller is steering the callee
STAMP	A passes a whole object when B needs one field send_email(user) when it only needs user.email
DATA	A passes exactly what B needs send_email(address) ← aim here
MESSAGE	A only sends a message; it does not even know B's shape

The two in the middle are the ones you will actually meet.

Control coupling is the boolean flag. `render(report, as_pdf=True)` means the caller knows there are two branches inside and is choosing one — so the caller and the callee are welded together, and adding a third format changes both. Two functions, or a strategy ([day 056](#)), removes it.

Stamp coupling is passing a whole object for one field. `send_welcome_email(user)` when the function only needs an address means the email module now depends on the shape of `User`, so a change to `User` can break email. `send_welcome_email(address)` does not.

The two smells that matter most, because they are the diagnosis

Almost every review comment reduces to one of these two, and they are opposites:

Shotgun surgery — one change forces edits in many places. That is **coupling too high** (or cohesion too low): the knowledge is scattered.

Divergent change — one place gets edited for many unrelated reasons. That is **cohesion too low** in a single file: unrelated knowledge is piled together. This is the single responsibility violation from [day 055](#).

If you learn two terms today, learn those. Both are about *change*, and both can be measured from the version history rather than argued about.

The rest of the vocabulary, with the fix attached

SMELL	WHAT IT LOOKS LIKE	USUALLY MEANS	FIX
Feature envy	A method uses another object's data more than its own	Behaviour is on the wrong class	Move the method to the data
Data clump	The same 3-4 parameters travel together everywhere	A missing class	<code>Address</code> , <code>DateRange</code> , <code>Money</code>
Primitive obsession	Ids, money, emails as bare <code>str</code> and <code>int</code>	A missing value object	A frozen type with validation
Long parameter list	6+ parameters, several with defaults	A data clump, or control coupling	Group into an object; split the function
Message chain	<code>order.customer.address.city.name</code>	Reaching through objects	Ask for what you need: <code>order.delivery_city()</code>
Middle man	A class whose methods only forward	A useless layer	Delete it; talk to the real thing
God object	One class knows and does everything	No boundaries at all	Split by stakeholder
Temporal coupling	<code>configure()</code> must be called before <code>run()</code>	A constructor that lies	Make the object valid on construction
Shotgun surgery	One change, many files	Scattered knowledge	Gather it into one place
Divergent change	One file, many reasons to change	Piled-up knowledge	Split by reason

Message chains have a rule of their own, the **Law of Demeter**: a method should only talk to itself, its own fields, its parameters, and objects it creates. `order.customer.address.city` breaks it, and the harm is real — that line depends on four classes, so a change to any of the four can break it, and it cannot be tested without building all four.

How to actually review a module

Six checks, in order, and each one takes under a minute:

1. **Read the imports.** They are a summary of what this file depends on. Do they tell a consistent story? `decimal` and `smtplib` together is two jobs.
2. **Say what it does in one sentence.** If you need “and”, note it.
3. **Count the fan-out** — how many other modules it calls. High fan-out means many reasons to break.
4. **Look at the parameter lists.** Data clumps, boolean flags, and primitives that should be types are all visible from the signatures alone.
5. **Read one test.** Long, mostly-irrelevant setup means the module is coupled to things it does not need.

6. **Check the version history.** `git log` on the file answers both diagnosis questions at once — many authors from many teams means divergent change; one feature touching eight files means shotgun surgery.

The sixth is the only one that produces evidence rather than opinion, and saying you would look there is worth marks on its own.

4 The picture

Anjali's two kitchens:

```
BY SIZE – logical cohesion. Tidy, and every job is a tour of the flat.

cupboard by the window [ big pans, big vessels, the cooker ]
drawer by the door     [ strainer, spoons, small knives ]
corner unit            [ tea, sugar, rice, dal, oil ]
fridge (behind you)    [ milk ]

making tea: door → window → corner → corner → fridge → stove
              5 places, 9 minutes, twice a day

BY JOB – functional cohesion. Looks worse. Tea takes 4 minutes.

above the stove [ small pan, strainer, tea, sugar ] ← the tea job
left of the stove [ cooker, its lid, the weight ]   ← the cooker job
corner unit      [ rice, dal, oil ]

making tea: one cupboard, one reach
```

What to notice: the first arrangement is not disordered. It has a rule and the rule is followed consistently — it is just the wrong rule, because the unit of use is the *job*, not the *kind*. A `utils.py` or a `validators.py` is exactly this, and it is why those files always grow.

The switch:

COUPLED

```
bedroom switch #3
|
+----> balcony light
|
+----> reading lamp

"turn off the balcony light"
→ the reading lamp goes off
→ an argument at 11 p.m.
→ you cannot change one without
  affecting the other
```

DECOUPLED

```
switch #3 ---> balcony light
switch #4 ---> reading lamp

"turn off the balcony light"
→ the balcony light goes off
```

What to notice: the coupling is invisible from outside. Both flats have four switches and the same lights. What differs is which changes are possible independently, and you only find out by trying to make one.

The two diagnostic smells, drawn as change:

SHOTGUN SURGERY – one change, many files. Coupling too high.

```
"add a second address line"
|
+→ models/user.py           (the field)
+→ forms/signup.py          (the form)
+→ forms/checkout.py        (the other form)
+→ serializers/user.py       (the API shape)
+→ templates/invoice.html    (the display)
+→ services/shipping.py      (the formatting)
+→ tests/... (x4)
```

10 files for one idea. The knowledge "what an address is" is SCATTERED.
Fix: gather it. One Address type, one formatter, one validator.

DIVERGENT CHANGE – one file, many reasons. Cohesion too low.

```
                                +→ finance changes the tax rule
OrderProcessor.py <----+→ marketing changes the email copy
(642 lines)             +→ platform migrates the database
                        +→ operations changes the stock policy
```

4 unrelated reasons to open one file. The knowledge is PILED UP.
Fix: split it by reason. (day 055)

What to notice: they are opposites and the fixes are opposite. Shotgun surgery says *gather*; divergent change says *split*. Diagnosing which one you have is the whole job, and both are visible in the version history.

Coupling, from worst to best, on one call:

CONTENT	emailer._smtp.sendmail(...)	reaching into internals
COMMON	CONFIG["smtp_host"] = ...	shared global state
CONTROL	send(user, is_admin=True)	the caller picks the branch
STAMP	send_welcome(user)	passes a whole User for one field
DATA	send_welcome(user.email)	passes exactly what is needed
MESSAGE	events.publish(UserSignedUp(email))	the sender does not know the receiver

Each step down removes something the caller has to know.
DATA is the sensible default. MESSAGE is for when you want them fully independent.

What to notice: the difference between STAMP and DATA is one dot, and it decides whether the email module depends on the `User` class at all.

5 How it actually works

A module to review

This is the kind of thing you get handed. Read it before the diagnosis.

```
# shipping/dispatcher.py
import psycopg
import requests

CONFIG = {} # module-level mutable state

class Dispatcher:
    def __init__(self, conn) → None:
        self.conn = conn

    def dispatch(self, order, express=False, international=False,
                 insured=False, gift_wrap=False, notify=True) → str:
        city = order.customer.address.city.name # a message chain
        pin = order.customer.address.pancode

        if express and not international:
            cost = 12000 if order.weight_grams > 500 else 8000
        elif international:
            cost = 45000 + (order.weight_grams * 12)
        else:
            cost = 4000 if order.weight_grams > 500 else 2500
        if insured:
            cost += int(order.total_paise * 0.02)
        if gift_wrap:
            cost += 5000

        label = (f"{order.customer.name}\n"
                 f"{order.customer.address.line1}\n"
                 f"{order.customer.address.line2}\n"
                 f"{city} {pin}")

        with self.conn.cursor() as cur:
            cur.execute("INSERT INTO shipments (order_id, cost, label) VALUES (%s,%s,%s)",
                        (order.order_id, cost, label))

        tracking = requests.post(CONFIG["courier_url"],
                                json={"pin": pin, "weight": order.weight_grams}).json()["id"]

        if notify:
            requests.post(CONFIG["sms_url"],
                          json={"to": order.customer.phone, "text": f"Shipped: {tracking}"})

        return tracking
```

The review, said out loud in the right vocabulary

Cohesion. *“I can’t describe this in one sentence without ‘and’. It calculates shipping cost, formats a label, writes to the database, calls a courier, and sends an SMS. That’s five jobs, and they change for different reasons — finance owns the cost table, design owns the label, platform owns persistence. That’s divergent change.”*

Control coupling. *“Five boolean parameters. Every one of them means the caller knows about a branch inside this method and is steering it. `dispatch(order, express=True, insured=True)` is the caller doing the deciding, so the caller and the callee change together. Those flags describe what kind of shipment this is, which is a missing concept.”*

Message chain. *“`order.customer.address.city.name` reaches through four objects. That single line depends on `Order`, `Customer`, `Address` and `City`, so any of the four can break it, and testing it means constructing all four. That’s a Law of Demeter violation — I’d ask the order for what I actually need.”*

Stamp coupling. *“The cost calculation only needs a weight and a destination, but it receives a whole `Order` and reaches into the customer’s address. So the pricing rules now depend on the shape of `Customer`.”*

Common coupling. “`CONFIG` is module-level mutable state read from inside the method. Two tests in the same process affect each other, and a missing key fails at call time rather than at start-up.”

Primitive obsession. “Costs are bare integers of paise with no type, so nothing stops you adding a weight to a cost. A pincode is a bare string with no validation.”

Data clump. “`line1`, `line2`, `city`, `pincode` travel together everywhere in this codebase. That is an `Address` asking to exist, with `format_label()` on it.”

Fan-out. “This one method touches the database, an HTTP courier API and an SMS API. Three external systems means three ways for it to fail, and no way to test the pricing without all three.”

The rewrite

Start by extracting the missing concepts, which is what most of the smells were pointing at:

```
# shipping/model.py
from dataclasses import dataclass
from enum import Enum

@dataclass(frozen=True)
class Money:
    paise: int
    def __add__(self, other: "Money") → "Money":
        return Money(self.paise + other.paise)

@dataclass(frozen=True)
class Address:
    """The data clump, given a name. Behaviour comes with it."""
    line1: str
    line2: str
    city: str
    pincode: str

    def __post_init__(self) → None:
        if not (self.pincode.isdigit() and len(self.pincode) == 6):
            raise ValueError(f"invalid pincode: {self.pincode!r}")

    def as_label(self, name: str) → str:
        return f"{name}\n{self.line1}\n{self.line2}\n{self.city} {self.pincode}"

class Service(Enum):
    """The five booleans, replaced by the concept they were describing."""
    STANDARD = "standard"
    EXPRESS = "express"
    INTERNATIONAL = "international"
```

The five flags become one enum plus two genuine options. That removes the control coupling: the caller now says *what it wants*, not *which branch to take*.


```
# shipping/pricing.py -- owned by finance. One reason to change.
from .model import Money, Service

class ShippingPricing:
    def __init__(self, rates: dict[Service, tuple[int, int]],
                 insurance_rate: float, gift_wrap: Money) → None:
        self._rates = rates
        self._insurance_rate = insurance_rate
        self._gift_wrap = gift_wrap

    def quote(self, service: Service, weight_grams: int, order_value: Money,
             insured: bool = False, gift_wrap: bool = False) → Money:
        light, heavy = self._rates[service]
        cost = Money(heavy if weight_grams > 500 else light)
        if insured:
            cost = cost + Money(int(order_value.paise * self._insurance_rate))
        if gift_wrap:
            cost = cost + self._gift_wrap
        return cost
```

Note what this function receives: a service, a weight, a value. **Data coupling** — exactly what it needs, and nothing else. It does not know that `Order` or `Customer` exist, so it can be tested in three lines and it cannot be broken by a change to either.

```
# shipping/dispatcher.py -- one reason to change: the sequence of steps
from .model import Address, Money, Service
from .pricing import ShippingPricing
from .ports import Courier, Notifier, ShipmentRepository

class Dispatcher:
    def __init__(self, pricing: ShippingPricing, shipments: ShipmentRepository,
                 courier: Courier, notifier: Notifier | None = None) → None:
        self._pricing = pricing
        self._shipments = shipments
        self._courier = courier
        self._notifier = notifier

    def dispatch(self, order_id: str, service: Service, weight_grams: int,
                value: Money, destination: Address, recipient: str,
                insured: bool = False, gift_wrap: bool = False) → str:
        cost = self._pricing.quote(service, weight_grams, value, insured, gift_wrap)
        label = destination.as_label(recipient)
        tracking = self._courier.book(destination.pincode, weight_grams)
        self._shipments.save(order_id, cost, label, tracking)
        if self._notifier:
            self._notifier.shipped(order_id, tracking)
        return tracking
```

Read the improvements out loud, because that is the answer:

- **No message chains.** It receives an `Address` and a name; it never walks `order.customer.address`.
- **No control coupling.** `Service` is a value describing the shipment, not a switch selecting a branch.
- **No common coupling.** `CONFIG` is gone; the courier and notifier are injected ([day 059](#)).
- **Fan-out of zero to external systems.** Three ports, all owned here, all fakeable.

- **One sentence, no “and”:** *it runs the dispatch sequence.* Ordering the steps is a legitimate single responsibility.

The test, which is the proof

```
def test_an_insured_express_shipment_is_priced_and_booked() → None:
    pricing = ShippingPricing({Service.EXPRESS: (8000, 12000)}, 0.02, Money(5000))
    shipments, courier = InMemoryShipments(), FakeCourier(tracking="TRK1")
    dispatcher = Dispatcher(pricing, shipments, courier)

    tracking = dispatcher.dispatch(
        order_id="ORD-1", service=Service.EXPRESS, weight_grams=600,
        value=Money(1_000_00), destination=Address("12 MG Rd", "Kothrud", "Pune", "411038"),
        recipient="Anjali", insured=True,
    )

    assert tracking == "TRK1"
    assert shipments.saved[0].cost == Money(12000 + 200)
```

No database, no network, five lines of setup. Testability is the measurable consequence of low coupling, and it is the strongest evidence in a review.

Where the ladders show up in real systems

- **Unix pipes** are the low-coupling ideal: `ls | sort | head` — each program knows only about bytes on a stream, so any of them can be replaced.
- `utils.py` is coincidental cohesion, everywhere, in every language. It grows forever precisely because nothing belongs there and nothing is excluded.
- A `settings.py` **read directly from deep inside modules** is common coupling, and it is why Django's `settings.configure()` and twelve-factor config exist.
- **Event-driven systems** are the message-coupling end: a publisher does not know who consumes, so adding a consumer changes nothing — at the cost of not being able to follow the flow by reading.
- **The React container/presentational split** is a cohesion decision: one component decides what data, another decides what it looks like, so designers and data changes stop colliding.
- `git log --format='%an' -- path/to/file | sort -u` is the diagnosis tool, and it works on any codebase in any language.

6 The numbers

Shotgun surgery, measured

```
"add a second address line"

with the address spread as 4 primitives:
  files touched      : 10    (model, 2 forms, serializer, template,
                               shipping formatter, 4 test files)
  lines changed      : ~60
  places somebody will forget : 2    (empirically -- the invoice template and
                               one test fixture)
  review time        : ~90 min, across 3 reviewers who each own part

with one Address type owning its own formatting:
  files touched      : 2    (the Address class and its test)
  lines changed      : ~8
  review time        : ~10 min, 1 reviewer
```

Divergent change, measured

```
git log --since=1.year --format='%an' -- shipping/dispatcher.py | sort -u | wc -l

before : 11 authors, 84 commits, 19 merge conflicts
after  : pricing.py    3 authors, 14 commits, 0 conflicts
        dispatcher.py 2 authors,  9 commits, 0 conflicts
        courier.py    2 authors, 11 commits, 1 conflict
```

Eleven authors on one file is the measurement. That number is the review comment.

Coupling, priced by test setup

```
testing "an insured express shipment costs 12,200 paise"

coupled version:
  needs a real Order with a Customer with an Address with a City
  needs a database connection (the INSERT is inline)
  needs CONFIG populated with two URLs
  needs the courier and SMS endpoints stubbed at the HTTP level
  setup: 31 lines, ~400 ms, flaky

decoupled version:
  ShippingPricing(rates, 0.02, Money(5000)).quote(...)
  setup: 3 lines, ~0.05 ms, cannot flake

~8,000x faster, and the pricing rule is now testable at all
without four other classes existing.
```

Fan-out, and the probability of breaking

a module calling N others, each of which changes ~monthly:

N = 1 → ~1 potentially breaking change/month

N = 3 → ~3

N = 9 → ~9, i.e. this module is disturbed roughly every 3 days

A module's exposure to breakage is roughly LINEAR in its fan-out, which is why "how many things does this file import?" is a real review question.

The message chain, priced

```
order.customer.address.city.name
```

classes this one line depends on : 4

changes that can break it : a rename or restructure in ANY of the four

objects needed to test it : 4, constructed and wired

```
order.delivery_city()
```

classes depended on : 1

objects needed to test it : 1

The boolean-flag cost

```
dispatch(order, express, international, insured, gift_wrap, notify)
```

combinations the signature allows : $2^5 = 32$

combinations that are meaningful : ~6

combinations tested : 4

combinations that are nonsense

but compile : express=True AND international=True
-- the code silently ignores one of them

Replacing 2 of the flags with a 3-valued enum:

allowed combinations : $3 \times 2 \times 2 \times 2 = 24$, and the nonsense one is unrepresentable

7 The trade-offs

Coupling cannot be removed, only moved and shaped

Modules that never interact do nothing. The goal is not zero coupling; it is coupling that is **visible, narrow, and pointing the right way**. Passing exactly what is needed is better than passing a whole object; an injected interface is better than an import; an event is looser still. Each step costs something.

Fully decoupled has a real price. An event-driven system where a publisher does not know its consumers is beautifully independent and genuinely hard to follow: you cannot answer “what happens when an order is placed?” by reading, you have to search for subscribers, and the ordering guarantees become something you have to think about.

I would not decouple further if I could no longer trace a flow by reading. That is a real threshold and it is the point at which the indirection has stopped paying.

Over-splitting for cohesion

The same failure as [day 055](#): fifteen classes with one method each is not high cohesion, it is the responsibility smeared out. High cohesion means related things are *together*, and the second half of that sentence matters as much as the first.

I would not split if the pieces cannot be understood or used independently, or if following one flow starts to mean opening more files than it saves.

When a god object is acceptable

Rarely, and it is worth being able to say when. A `Matrix` with forty methods is cohesive — one reason to change, one set of data, every method genuinely about matrices. A framework's `Request` object is large and coupled to everything, and splitting it would make every handler take nine parameters. **Size is not the smell; unrelated reasons to change is the smell.**

The smells are hints, not rules

Every one of them has legitimate cases:

- **Long parameter list** — a mathematical function with six genuine inputs is fine.
- **Middle man** — an adapter that only forwards is doing the essential job of translating at a boundary ([day 059](#)).
- **Message chain** — `df.groupby("x").agg("sum").reset_index()` is a fluent interface, not a Demeter violation, because each call returns a new object of the same type rather than reaching into someone else's internals.
- **Primitive obsession** — a `str` for a name is a `str`. Wrapping every primitive is its own smell.
- **Data clump** — three parameters that travel together twice is a coincidence, not a missing class. The rule of three from [day 060](#) applies here too.

Naming a smell is the beginning of an argument, not the end of one. A candidate who says “this is feature envy, so it's wrong” is weaker than one who says “this is feature envy — the method uses four fields of `Order` and none of its own, so I'd move it onto `Order`, unless `Order` is in a package this module shouldn't depend on.”

When not to fix it

I would not refactor code that nobody touches. A module with two commits in three years is finished, whatever its smells, and refactoring it is risk with no return. The version history that diagnoses the problem also tells you whether it is worth solving.

I would not fix it during an unrelated change, because a refactor mixed into a feature commit cannot be reviewed and cannot be reverted independently.

8 In the interview

How it gets asked

- “Review this module. What would you change?” — a code sample, and the score is in the vocabulary and the priority order.
- “What is coupling and cohesion?” — and the good answer is a sentence each plus an example of the bad end of both ladders.
- “What is a code smell? Name a few.” — with the fix attached to each, or it is a list.
- “One change touches eight files. What is that called and what does it tell you?” — shotgun surgery, and the fix is to gather.
- “How would you find these problems in a codebase you have just joined?” — the version history, and this is the answer that stands out.

What to say out loud, in the first ninety seconds

1. **Give the two-sentence framing first.** “I’ll look at two things. Cohesion — do the things in this module belong together. And coupling — how much does it have to know about everything else.”
2. **Run the one-sentence test out loud.** “I can’t describe this without ‘and’: it prices a shipment, formats a label, writes to the database, calls a courier and sends an SMS. Five jobs, owned by four different teams — that’s divergent change.”
3. **Name the specific smells with the evidence, not just the label.** “Five boolean parameters is control coupling — the caller knows about branches inside and is steering them. `order.customer.address.city.name` is a message chain, four classes for one line. `line1`, `line2`, `city`, `pincode` travelling together is a data clump — that’s an `Address` asking to exist.”
4. **Say what you would do first, and why that one.** “I’d start with `Address`, because it’s the cheapest change and it removes the message chain, the data clump and part of the label formatting in one go.”
5. **Name the evidence you would go and get.** “And I’d run `git log` on this file. If eleven people from four teams have edited it this year, that’s the divergent change confirmed rather than asserted.”

The follow-ups

“One change touches eight files. What’s that called, and what does it tell you?” Shotgun surgery, and it tells me the knowledge is scattered — one idea is represented in eight places rather than one, so the coupling between them is implicit and unenforced. The example I have seen most is an address held as four separate primitives — line one, line two, city, pincode — so adding a second address line means the model, two forms, a serializer, an invoice template, a shipping label formatter and four test files: about ten files and sixty lines, and empirically somebody misses two of them, usually a template and a fixture. The fix is the opposite of the fix for divergent change: here I *gather*. One `Address` value object that owns its own validation and its own label formatting, so the second line is eight lines in two files. It is worth naming its

opposite in the same breath, because interviewers often ask them together: divergent change is one file edited for many unrelated reasons, which is low cohesion in one place, and the fix there is to *split*. So the diagnosis question is which of the two you have — scattered knowledge, gather it; piled-up knowledge, split it — and both are visible in the version history rather than being matters of taste.

“How would you find these problems in a codebase you’d just joined?” I would start with the version history, because it is the only source of evidence rather than opinion. `git log --format='%an' --since=1.year -- <file> | sort -u | wc -l` tells me how many distinct people edit a file; anything above about five, from more than one team, is divergent change. And going the other way, looking at recent feature commits and counting the files each one touched finds shotgun surgery — if adding one field consistently touches eight files, the knowledge for that concept is scattered. Then a few cheap static checks. I read the import blocks: a file importing both `decimal` and `smtplib` is doing two jobs, and it takes five seconds to see. I look at the function signatures without reading the bodies, because data clumps, boolean flags and primitives-that-should-be-types are all visible there. I grep for message chains — two or more dots between object accesses. And I look at the test setup, because a test needing thirty lines of scaffolding to check an arithmetic rule tells me exactly how coupled that rule is to everything else. What I would *not* do is start refactoring what I found. A module with two commits in three years is finished, whatever its smells, and the same history that diagnoses the problem tells me whether it is worth solving.

“What’s the difference between coupling and cohesion? People use them interchangeably.” They are about different directions. Cohesion is *inside* a module: do the things in here belong together, are they all serving one job. Coupling is *between* modules: how much does this one have to know about that one. The reason they get conflated is that they usually go wrong together — a module with low cohesion is doing several jobs, and each job drags in its own dependencies, so its coupling goes up as well. But you can have one without the other. A perfectly cohesive pricing module that reads a global config dictionary is highly cohesive and badly coupled. A `utils.py` where every function is independent and takes exactly its arguments has terrible cohesion and excellent coupling. The useful part is the ladders. For cohesion, the bottom rungs have names that describe real files: *coincidental* is `utils.py`, things together for no reason at all; *logical* is grouping by kind rather than by job — all the validators in one file — which looks tidy and is why those files grow forever; *temporal* is grouping by when things happen, like an init module. Functional cohesion, the top, is everything contributing to one job. For coupling the ladder runs from *content* — reaching into another object’s internals — through *common*, shared global state; *control*, passing a flag that picks a branch inside the callee; *stamp*, passing a whole object when one field is needed; down to *data*, passing exactly what is needed. The one-line summary: **cohesion is about what is inside, coupling is about what crosses the boundary, and the goal is high cohesion with low coupling.**

A model answer

“I’d look at two things and say them in that order: cohesion, meaning do the things in this module belong together, and coupling, meaning how much it has to know about everything else.

On cohesion, my test is whether I can describe it in one sentence without ‘and’. I can’t. It prices a shipment, formats a label, writes a row to the database, calls a courier API and sends an SMS. That’s five jobs owned by four different teams — finance owns the rate table, design owns the label, platform owns persistence — so this file will be edited for four unrelated reasons. That’s divergent change, and I’d expect `git log` to show a lot of distinct authors on it.

On coupling, three specific things. Five boolean parameters is control coupling: the caller knows there are branches inside and is choosing one, so caller and callee change together — and the signature allows thirty-two combinations when about six are meaningful, including `express` and `international` both true, which is nonsense the code silently ignores. `order.customer.address.city.name` is a message chain — one line depending on four classes, so a rename in any of them breaks it and testing it means constructing all four. And `CONFIG` as module-level mutable state is common coupling; two tests in one process affect each other.

There are two missing concepts underneath most of that. `line1`, `line2`, `city` and `pincode` travel together everywhere — that’s a data clump, and it’s an `Address` asking to exist, with the pincode validation and the label formatting on it. And the five flags are really describing what kind of shipment this is, which is an enum.

If I could only make one change I’d introduce `Address` first, because it’s the cheapest and it removes the message chain, the data clump and the label formatting in one go. Then I’d pull the pricing out into its own class taking a service, a weight and a value — data coupling, exactly what it needs — which makes it testable in three lines instead of thirty-one, without a database.

The thing I’d want to check before doing any of it is the version history. If this file has two commits in three years, it’s finished and I’d leave it alone whatever it looks like.”

9 Recall card

- **Cohesion is what is *inside* a module; coupling is what *crosses the boundary*.** Aim for high cohesion, low coupling. Cohesion ladder (worst→best): **coincidental** (`utils.py`) · **logical** (grouped by kind — the kitchen by size) · **temporal** (grouped by when) · ... · **functional**. Coupling ladder (worst→best): **content** · **common** (globals) · **control** (a flag picking a branch) · **stamp** (a whole object for one field) · **data** · message.

- **Two smells are the diagnosis, and they are opposites.** **Shotgun surgery** = one change, many files → knowledge is *scattered*, so **gather** it (one `Address` type: 10 files and 60 lines becomes 2 files and 8). **Divergent change** = one file, many unrelated reasons → knowledge is *piled up*, so **split** it.
- **The named smells, with fixes:** feature envy → move the method to the data · data clump → a missing class · primitive obsession → a value object · long parameter list / boolean flags → control coupling, use an enum or split · message chain (`a.b.c.d`) → Law of Demeter, ask for what you need · middle man · god object · temporal coupling (`configure()` before `run()`).
- **Review in six checks:** read the imports · say it in one sentence with no “and” · count the fan-out · read the signatures for clumps and flags · read one test’s setup · **run git log on the file** — 11 authors from 4 teams is divergent change *measured*, not asserted.
- **Coupling cannot be removed, only shaped**, and the smells are hints not rules. A `Matrix` with 40 methods is cohesive; a fluent `df.groupby().agg()` chain is not a Demeter violation; an adapter that only forwards is a legitimate middle man. And **a file with two commits in three years is finished** — refactor what changes.

PRACTICE

Practice — Collisions, and why a hash map can turn slow

DSA topic: Collisions, and why a hash map can turn slow **System design topic:** Coupling, cohesion, and code smells

Code these, in this order

One rule for the whole set: **make a collision happen on purpose before you reason about one.** A class whose `__hash__` returns a constant takes six lines and turns every claim in the lesson into something you can watch.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Design HashSet	LeetCode 705 (Easy)	The same machinery as yesterday, minus the values — so the collision handling is all that is left.
2	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Deletion done right: a dict plus a list, with the swap-with-last trick.
3	LRU Cache	LeetCode 146 (Medium)	A hash map that must also maintain order — where the dict alone is not enough.
4	Design HashMap	LeetCode 706 (Easy)	Write it a second time, with open addressing and tombstones instead of chaining.

On problem 2, notice which deletion problem you are solving

`remove` must be `O(1)`, so you cannot shift a list. The trick is to swap the doomed element with the last one and pop. Say out loud what has to be updated when you do that, and why forgetting it is exactly the “unreachable entry” failure in a different costume.

On problem 4, use open addressing this time

You wrote chaining yesterday. Write it with probing today, with three slot states. Then, before you submit, deliberately implement `remove` by blanking the slot, and find an input where a later `get` returns the wrong answer. Say what you had to construct to make it fail.

The collision drill

1. Write a `Colliding` class whose `__hash__` returns 42 and whose `__eq__` compares a name.
2. Put five of them in a `dict`. Does `len` say 5? Can you retrieve each one?
3. State, in one sentence, what collisions cost and what they do not cost.
4. Now put ten thousand of them in a dict and time it. Time ten thousand normal string keys.
5. Compute the ratio and say which complexity each case is.
6. Say what would happen to the same experiment in Java 8 or later, and why.

The probing drill

Using your open-addressed table with capacity 8:

1. Insert four keys that all hash to slot 3. Draw the slot array afterwards.
2. How many probes did the fourth insert take?
3. Insert a fifth key hashing to slot 4. How many probes now, and why is that number bigger than you would expect?
4. Explain, in one sentence, why a cluster grows faster than it “should”.
5. Say what quadratic probing changes and what it does not.
6. Say what double hashing changes, and what Python does that is in the same spirit.

The tombstone drill

1. Build a table where key `b` probed past key `a`.
2. Delete `a` by blanking its slot. What does `get(b)` return? Does anything raise? What does `len` say?
3. Now do it with a tombstone. What does `get(b)` return?
4. Write the rule for how a lookup treats a tombstone, and how an insert treats one.
5. Insert twenty items and delete eighteen. Count the tombstones and compute both load factors — by `size` and by `used`.
6. Say what would go wrong if the resize triggered on `size` instead.
7. Say the surprising sentence this leads to, in one line.

The load-factor drill

Fill a fixed-capacity table to each load and measure the average probes per lookup:

1. 0.25
2. 0.50
3. 0.66
4. 0.75
5. 0.90
6. 0.95

Then answer: where does the curve turn, why do implementations resize at about two-thirds rather than reacting to slowness, and how would the same table look with chaining?

The bad-hash drill

For each hash function, predict the distribution and then measure it with two thousand realistic keys:

1. `hash(self.order_id[:4])` where every id starts `"ORD-"`.
2. `hash(len(self.name))`.
3. `hash(self.name[0])`.

4. `hash((self.first, self.last))` .
5. `return 0` .
6. `id(self)` , on a class that also defines `__eq__` by value.

Two of those six are broken in a way that has nothing to do with distribution. Name them.

The break-it drill

Trigger each, read the output, and give the fix in one sentence:

1. Delete by blanking a slot in an open-addressed table, then look up an entry that probed past it.
2. Resize on `size` rather than `used` , under an insert-and-delete workload.
3. Mutate a key's hashed field after insertion, then `key in d` .
4. Insert one million integers stepping by exactly the table capacity into a hand-written table with `% capacity` and linear probing.
5. `python -c "print(hash('apple'))"` twice.
6. Assert a specific `hash()` value in a test, then run it again.

The attack drill

1. Write a class with a constant hash and insert `n` of them, for `n = 1,000, 5,000 and 20,000`. Time each.
2. Plot or tabulate the times. What is the growth rate?
3. Compute the comparisons for `n = 20,000` and compare with the non-colliding case.
4. Say what an attacker would send to a web server to cause this, and why it worked across almost every language in 2011.
5. Name Python's defence and the two rules that follow from it for your own code.
6. Name Java's different defence and say what complexity it gives the flooded bucket.

The review drill

Here is the module. Do not rewrite it yet.

```
# billing/charger.py
import stripe
import psycopg

SETTINGS = {}

class Charger:
    def __init__(self, conn):
        self.conn = conn

    def charge(self, invoice, retry=False, partial=False, dry_run=False, notify=True):
        email = invoice.customer.contact.email.address
        amount = invoice.total_paise if not partial else invoice.total_paise // 2
        if dry_run:
            return "DRY"
        result = stripe.Charge.create(api_key=SETTINGS["stripe_key"],
                                      amount=amount, currency="inr",
                                      receipt_email=email)

        with self.conn.cursor() as cur:
            cur.execute("INSERT INTO charges (invoice, amount, ref) VALUES (%s,%s,%s)",
                        (invoice.id, amount, result["id"]))

        if notify:
            stripe.Email.send(email, f"Charged {amount/100} rupees")
        return result["id"]
```

1. Describe it in one sentence. Count the “and”s.
2. Name every distinct job it does, and the team that would own each.
3. Which smell is that? Name it.
4. Find the message chain. How many classes does that one line depend on?
5. Find the control coupling. How many combinations does the signature allow, and how many are meaningful? Which combination is nonsense but compiles?
6. Find the common coupling and say what breaks in tests because of it.
7. Find the stamp coupling. What does the amount calculation actually need?
8. Count the fan-out to external systems. What does that mean for testing?
9. Name the two missing concepts. What would you call them?
10. Rewrite it. Then write the test for “a partial charge is half the total” and count the lines of setup before and after.
11. Which single change would you make first, and why that one?
12. What would you look at before making any change at all?

The smell-naming drill

Name the smell, say what it usually means, and give the fix:

1. `order.customer.address.city.name`
2. `def render(report, as_pdf=False, as_csv=False)`
3. A method on `Invoice` that uses six fields of `Customer` and none of its own.
4. `line1`, `line2`, `city`, `pincode` passed together in nine functions.
5. `phone: str` with the format checked in four places.
6. `db.configure()` must be called before `db.query()`.

7. A class whose every method is `return self._inner.same_method(...)`.
8. `utils.py`, 900 lines, 40 functions.
9. One file with commits from finance, marketing, platform and operations.
10. Adding one field means editing ten files.

Two of the ten are the *diagnostic* pair. Name them and say which fix each one gets.

The coupling-ladder drill

For each call, name the level of coupling and say what would have to change to move it one rung better:

1. `emailer._smtp.sendmail(...)`
2. `CONFIG["smtp_host"]`
3. `send(user, is_admin=True)`
4. `send_welcome(user)`
5. `send_welcome(user.email)`
6. `events.publish(UserSignedUp(email))`

The cohesion drill

Name the level of cohesion, and say what the module would look like at the next level up:

1. `utils.py` with `slugify`, `retry`, `parse_date` and `send_sms`.
2. `validators.py` with every validator in the system.
3. `startup.py` that opens the log, connects the database, loads config and warms the cache.
4. `Address` with `format_label`, `validate_pincode` and `is_deliverable`.
5. `Matrix` with `multiply`, `transpose`, `invert` and `determinant`.

Two of those five are already fine. Name them and defend it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What is the worst-case complexity of a hash map lookup? When does it happen?* Both halves immediately, collisions as the normal condition with the birthday number, the two strategies and their trade, tombstones volunteered before being asked, and the three real causes.
2. *Review this module. What would you change?* The two-sentence framing, the one-sentence test with the “and”s counted, the specific smells with evidence rather than labels, what you would change first and why, and the version history you would go and look at.
3. *How would you implement deletion in an open-addressed table?* Start with what goes wrong, then the three slot states, then the tombstone accumulation problem and the counter that fixes it — ending on the surprising sentence about shrinking maps.

Before you move on

- ☐ I built a collision on purpose and confirmed the dictionary was still correct.
- ☐ I broke deletion by blanking a slot and watched an entry become unreachable with no error.
- ☐ I can state the lookup rule for a tombstone and the insert rule, without hesitating.
- ☐ I measured the probe count at five load factors and can describe the shape of the curve.
- ☐ I can say why a shrinking hash map can need a rehash.
- ☐ I can name the three real causes of $O(n)$ lookups and give an example of each.
- ☐ I reviewed `Charger` and named at least five smells with evidence, not just labels.
- ☐ I can give both ladders — coupling and cohesion — from worst to best.
- ☐ I answered all three questions above out loud.

Sets: membership, deduplication, and the $O(1)$ habit

DATA STRUCTURES AND ALGORITHMS

Sets: membership, deduplication, and the $O(1)$ habit

You replace a nested loop with a set lookup by reflex.

SYSTEM DESIGN

Design principles revision and interview questions

You can critique an unseen design using SOLID without sounding like a textbook.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the duplicate. Now do it in one pass.*
- *Here is a class. Which principles does it violate?*

Sets: membership, deduplication, and the $O(1)$ habit

1 What this is, and why they ask it

A **set** is a collection that answers exactly one question quickly: *is this thing already in here?* It holds no duplicates, it keeps no order, and it stores no values — only the keys. It is the hash table from [day 060](#) with the value column deleted, which is why membership costs $O(1)$ on average and not $O(n)$.

They ask it because the set is the single cheapest upgrade in the whole of interviewing. A very large number of problems have an obvious $O(n^2)$ answer — for every item, look through all the other items — and a two-line $O(n)$ answer that replaces the inner loop with one membership check. Interviewers use this deliberately. They give you a problem whose brute force is easy, wait for you to write it, and then say *now do it in one pass*. The candidates who freeze are the ones who never built the reflex. The candidates who do well hear “one pass” and immediately reach for a **seen set** — a set that accumulates what you have already looked at — because that is the answer to a whole family of questions, not a trick for one of them.

The second reason is that sets are the fastest way to change a complexity in front of an interviewer. You do not need to redesign anything. You change one line, the nested loop disappears, and you can explain exactly why the cost fell. That is a very good ninety seconds of interview.

2 The story

The free eye check-up camp at the school in Vidyaranyaपुरa opened its shutters at eight on a Sunday morning, and by ten past eight there were already sixty people standing in the sun.

Suresh, who is nineteen and had volunteered mostly because his mother told him to, was put on the door. His job was simple. Everybody who came in got one free pair of reading glasses at the end. Some people had worked out that if you went round the building and joined the line again, you got a second pair. So Suresh had to stop them.

The organisers had sent him everybody's name on his phone. Six hundred and forty names, in the order people had signed up. So a man would reach the door, say “Rajanna, from Bagalur”, and Suresh would start at the top and scroll. Sometimes the name was near the top and it took four seconds. Usually it was not. By half past nine he was taking nearly a minute on each person, because he had to start from the very beginning every single time, and by then he had four hundred names to go past. The line went out of the gate and down the road. A woman told him, not unkindly, that her son could have done it faster.

At eleven o'clock the head volunteer came out, looked at the line, and went across the road to the stationery shop. She came back with a rubber stamp and a small blue ink pad. Forty rupees.

After that, the check was this. Show me the back of your hand. Blue mark, you have already been in. No mark, come in, and here is your mark.

It took him under a second, and — this is the part Suresh actually noticed — it took him under a second at four hundred people in exactly the same way it had at ten people. The line was gone by half past twelve. Two men tried it on and were caught at once, one of them laughing about it.

The only thing it cost was forty rupees and the small nuisance of carrying an ink pad about, which he had to keep remembering to put down somewhere he would not sit on it.

3 The idea in plain English

Scrolling the whole list of names is a **linear scan**: to answer one question you touch everything. The blue stamp is a **set**: you answer the same question with one look, and the number of people already inside does not change how long the look takes.

Three things came out of the ink pad, and they are the three things a set gives you.

One: membership is instant. “Has this person been in?” costs the same at person 400 as at person 10. In code that is the `in` operator on a set.

Two: duplicates cannot happen. Stamping a hand that is already stamped changes nothing. A set holds each element once, no matter how many times you add it.

Three: it costs extra memory. The ink pad had to be bought and carried. A set has to be built and held in memory alongside the data you already have.

What a set is, precisely

A set is a hash table that stores keys and no values. When you write `seen.add(x)`, Python computes `hash(x)`, turns that into a slot, and records that the slot is taken. When you write `x in seen`, it computes the same hash, goes straight to that slot, and compares. There is no searching. Everything you learned about hash tables on [day 060](#) — the hash function, the load factor, the resize — is happening underneath, unchanged.

Two consequences follow immediately, and both get asked about.

Elements must be hashable. Numbers, strings and tuples can go in a set. Lists and dictionaries cannot, because they can be changed after you put them in, which would make them unfindable. This is the same rule as dictionary keys.

There is no order. A set does not remember the order you added things in. This matters when you deduplicate, and it catches people out.

Making one

```
seen: set[int] = set()           # the empty set. Not {} – that is an empty dict.
letters = {"a", "b", "c"}       # a set literal
digits = set("hello")           # from any iterable: {'h','e','l','o'}
```

`{}` is the one that bites. An empty pair of braces is an empty **dictionary**, because dictionaries got the syntax first. The empty set has to be written `set()`.

The four operations you actually use

```
seen.add(7)           # put 7 in. Doing it twice is harmless.
seen.discard(7)       # take 7 out. No error if it was not there.
seen.remove(7)        # take 7 out. KeyError if it was not there.
7 in seen             # True or False, in O(1) average
```

`add` and `in` cover perhaps ninety percent of interview use. `discard` is the safe removal; `remove` is the one that raises.

The set operations

Sets also do the things you drew in school with two overlapping circles, and they are genuinely useful in interviews when the question is about two collections.

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
a | b   # union           → {1, 2, 3, 4, 5}   in either
a & b   # intersection    → {3, 4}      in both
a - b   # difference      → {1, 2}      in a, not in b
a ^ b   # symmetric diff  → {1, 2, 5}   in exactly one
a ≤ b   # subset         → False
```

Every one of those runs in time proportional to the size of the smaller set, not the product of the two. “Which usernames appear in both files” is `a & b`, and it is $O(n)$, not $O(n^2)$.

The seen-set pattern

This is the shape you will write more than any other. Walk the data once. Before dealing with each element, ask the set whether you have met it. Then add it.

```
seen = set()
for item in items:
    if item in seen:
        ...           # this is a repeat – do whatever the problem wants
    seen.add(item)
```

Read the loop and notice the invariant, in the sense of [day 028](#): **at the top of every iteration, `seen` holds exactly the elements before the current one.** That sentence is the whole pattern, and saying it out loud in an interview is worth more than writing the loop quickly.

Deduplicating, and the order trap

```
names = ["ravi", "asha", "ravi", "meena", "asha"]
list(set(names))          # ['meena', 'asha', 'ravi'] – order is not yours
list(dict.fromkeys(names)) # ['ravi', 'asha', 'meena'] – first-seen order kept
```

`set(names)` is the fast, correct way to get the distinct values when you do not care about order. When order matters — and in interviews it very often does, because the expected output is written in the original order — use `dict.fromkeys`, which keeps insertion order because Python dictionaries have done so since version 3.7. Say which one you are choosing and why. It is a small thing that reads as care.

4 The picture

Two ways to answer “have I seen this before?”, drawn side by side on the same input.

```
input    [ 4 , 7 , 2 , 7 , 9 ]
index    0   1   2   3   4

(a) the nested loop: for element 3, compare with everything before it
-----
i = 3 (value 7)
    j = 0 → 4 vs 7    no
    j = 1 → 7 vs 7    YES
3 comparisons for one element. Element 400 would need 400.

(b) the seen set: for element 3, ask once
-----
seen = { 4 , 7 , 2 }
"is 7 in seen?" → hash(7) → slot 5 → occupied → YES
1 look-up for one element. Element 400 also needs 1.
```

Notice what changes and what does not. In (a) the work per element grows as the input grows. In (b) it does not. That is the entire difference between $O(n^2)$ and $O(n)$, and it is the sentence to say out loud.

Now watch the set fill up as the loop runs. The column under `seen before it` is what the set holds *before* that element is looked at.

step	element	seen before it	answer
----	-----	-----	-----
0	4	{ }	not seen
1	7	{ 4 }	not seen
2	2	{ 4, 7 }	not seen
3	7	{ 4, 7, 2 }	SEEN ← the duplicate
4	9	{ 4, 7, 2 }	not seen (7 was already in)

Notice row 4. Adding 7 a second time did nothing to the set — the size stayed at three. That is duplicates being impossible, not duplicates being filtered afterwards.

5 The code, built step by step

Four problems, each one a small step up. The last is the problem interviewers actually use to find out whether you understand sets or have only memorised them.

Step 1 — does this contain a duplicate at all?

```
def has_duplicate(numbers: list[int]) → bool:
    seen: set[int] = set()
    for number in numbers:
        if number in seen:
            return True
        seen.add(number)
    return False
```

Six lines. Walk once, ask, add. It returns as soon as it finds the first repeat, so on `[1, 1, 2, 3, ..., 999999]` it stops after two elements instead of reading a million.

There is a one-line version, and you should know both:

```
def has_duplicate_short(numbers: list[int]) → bool:
    return len(set(numbers)) < len(numbers)
```

If the distinct count is smaller than the total count, something repeated. It is elegant, and it is the one to mention — but say the difference out loud: the short version always reads the whole input and always builds the whole set, so on `[1, 1, ...]` with a million elements the loop version stops after two and this one does not. Same $O(n)$, very different real behaviour.

Step 2 — which element repeats first?

```
def first_duplicate(numbers: list[int]) → int | None:
    seen: set[int] = set()
    for number in numbers:
        if number in seen:
            return number
        seen.add(number)
    return None
```

Almost the same code, returning the value instead of `True`. The reason it is worth writing separately is the ambiguity hiding in “first”. On `[2, 1, 3, 1, 2]`, is the answer `1` — whose second copy comes first — or `2` — whose first copy comes first? This code gives `1`. **Ask the interviewer which one they mean.** That question is one of the cheapest points available in the whole interview, because the two answers need genuinely different code.

Step 3 — what do two collections share?

```
def common(first: list[int], second: list[int]) → list[int]:
    return list(set(first) & set(second))
```

Building both sets costs $O(n + m)$. The intersection then walks the smaller one and checks the larger, so the whole thing is linear. The nested-loop version is $O(n \times m)$: with two lists of 10,000 that is 100,000,000 comparisons against 20,000 operations.

If you only need to check one list against another, do not build a set of the first as well:

```
def common_streaming(first: list[int], second: list[int]) → list[int]:
    lookup = set(second)          # build once
    return [x for x in first if x in lookup]  # then one  $O(1)$  check each
```

This keeps duplicates from `first` and keeps its order, which the set version does not. Two different answers to two different questions. Say which one you are giving.

Step 4 — the longest run of consecutive numbers

Given `[100, 4, 200, 1, 3, 2]`, the longest run of consecutive integers is `1, 2, 3, 4`, so the answer is 4. The order in the input means nothing. This is LeetCode 128, and it is asked constantly, because the obvious answer is to sort — $O(n \log n)$ — and the interviewer wants $O(n)$.

Start with the set.

```
number_set = set(numbers)
```

Now you can ask “is 57 present?” instantly, which is the only question this problem needs.

The naive next step is: for every number, walk upward while the next one exists.

```
for number in number_set:
    length = 1
    while number + length in number_set:
        length += 1
```

This is correct and it is too slow. On `[1, 2, 3, ..., n]` it starts at 1 and walks n steps, then starts at 2 and walks $n-1$ steps, and so on — $O(n^2)$. This is the step people get wrong, and it looks right because every individual line is right.

The fix is one condition, and it is the whole idea of the problem: **only start walking from a number that begins a run.**

```
for number in number_set:
    if number - 1 in number_set:
        continue          # not a start — somebody else will count this run
    ...
```

If `number - 1` exists, then `number` is in the middle of a run, and that run’s true starting point will do the walking. So each run is walked exactly once, by its smallest element.

Put the two together:

```
for number in number_set:
    if number - 1 in number_set:
        continue
    length = 1
    while number + length in number_set:
        length += 1
    best = max(best, length)
```

The complete solution

```
def longest_consecutive(numbers: list[int]) → int:
    """Length of the longest run of consecutive integers, in O(n) time.

    Order in `numbers` is irrelevant. Duplicates are ignored, because the set
    holds each value once.
    """
    number_set: set[int] = set(numbers)
    best = 0

    for number in number_set:
        # Only a number with no left-hand neighbour starts a run. Every other
        # number sits in the middle of a run that its own start will walk.
        if number - 1 in number_set:
            continue

        length = 1
        while number + length in number_set:
            length += 1

        best = max(best, length)

    return best

def has_duplicate(numbers: list[int]) → bool:
    """True if any value appears twice. Stops at the first repeat."""
    seen: set[int] = set()
    for number in numbers:
        if number in seen:
            return True
        seen.add(number)
    return False

def first_duplicate(numbers: list[int]) → int | None:
    """The value whose second occurrence appears earliest, or None."""
    seen: set[int] = set()
    for number in numbers:
        if number in seen:
            return number
        seen.add(number)
    return None

def deduplicate_keeping_order(items: list[str]) → list[str]:
    """Distinct values in first-seen order. set() alone would lose the order."""
    return list(dict.fromkeys(items))

if __name__ == "__main__":
    print(longest_consecutive([100, 4, 200, 1, 3, 2]))          # 4
    print(longest_consecutive([0, 3, 7, 2, 5, 8, 4, 6, 0, 1])) # 9
    print(longest_consecutive([]))                             # 0
    print(has_duplicate([1, 2, 3, 1]))                          # True
    print(first_duplicate([2, 1, 3, 1, 2]))                    # 1
    print(deduplicate_keeping_order(["ravi", "asha", "ravi"])) # ['ravi', 'asha']
```

Note the empty-input case returns 0 because `best` starts at 0 and the loop never runs. Say that out loud rather than adding an `if not numbers` guard — the interviewer is watching for whether you checked, not for the guard itself.

6 What it costs

The nested loop, counted

For `has_duplicate` written the brute-force way — for each element, compare with every earlier one — element 0 does 0 comparisons, element 1 does 1, element 2 does 2, and so on to element $n-1$, which does $n-1$.

$$0 + 1 + 2 + \dots + (n-1) = n(n-1)/2$$

At $n = 10,000$ that is $10000 \times 9999 / 2 = 49,995,000$ comparisons. Roughly fifty million.

The set version, counted

The loop runs n times. Each iteration does one membership check and one add. Both are $O(1)$ on average, so the total is $n \times \text{constant}$.

At $n = 10,000$ that is 10,000 checks and at most 10,000 adds — call it 20,000 operations. **Fifty million against twenty thousand: a factor of 2,500.** That is the number to quote, not the letters.

`longest_consecutive`, counted properly

This is the one that gets probed, because the code has a `while` loop inside a `for` loop and it still is not quadratic. Count it as two separate things.

The `for` loop runs once per distinct number: n iterations. Each does one membership check, `number - 1 in number_set`.

The `while` loop is the interesting half. It only runs for numbers that start a run. Across the whole execution, each number is stepped over by exactly one `while` loop — the one belonging to its run's smallest element. So all the `while` loops together do at most n steps, however they are distributed.

```
for-loop checks:      n
all while-loop steps: n (total, across every run)
-----
total:                2n  →  O(n)
```

That is the sentence: *the inner loop is not bounded per iteration, it is bounded in total*. Say it exactly like that.

Space

`set(numbers)` holds up to n distinct values. That is **$O(n)$ extra space** — extra, not total, so on top of the input you were given.

It is not free, and it is worth being honest about. A Python `set` of a million small integers is roughly 32 MB, against about 8 MB for the same integers in a `list`, because a set stores hash values, keeps its table two-thirds empty by design, and holds pointers to objects. If the inter-

viewer says “now do it in $O(1)$ space”, the set is exactly what they are taking away, and the answer is usually sorting instead — $O(n \log n)$ time for $O(1)$ extra space. That trade is [day 066](#)’s whole subject.

7 The traps

Trap 1 — the seen “set” that is secretly a list

This is the commonest one, and it is invisible on the screen because the code reads identically.

```
def has_duplicate_slow(numbers: list[int]) → bool:
    seen = []                # ← a list
    for number in numbers:
        if number in seen:   # ← looks exactly the same
            return True
        seen.append(number)
    return False
```

`in` works on a list too. It just scans it. So this is $O(n^2)$ wearing the clothes of an $O(n)$ solution, and it will pass every small test you write.

Measure it once and you will never do it again. With 20,000 distinct integers:

```
seen = set()    0.004 s
seen = []      2.9 s
```

Roughly seven hundred times slower, on an input small enough to fit on a screen. `in` on a list is $O(n)$; `in` on a set or a dict is $O(1)$. The operator is the same; the structure is the whole story.

Trap 2 — `longest_consecutive` without the start check

Drop the `if number - 1 in number_set: continue` line and the code still returns the right answer on every example in the problem statement. It fails on time, not on correctness, which is the worst kind of failure because your tests pass.

The input that kills it is the most boring one imaginable: `list(range(100000))` — a single run of a hundred thousand consecutive numbers.

```
with the guard:    0.02 s
without the guard: about 100000^2 / 2 = 5 x 10^9 steps — you will not wait for it
```

The interviewer’s follow-up is *what is the worst case of that loop?*, and if you have not thought about it you will say $O(n)$ and be wrong.

Trap 3 — unhashable elements

Try to put a list into a set and Python stops you:

```
>>> {1, 2}.add([3])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Same for a dictionary:

```
>>> set([{"a": 1}])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'dict'
```

This bites in real problems whenever the thing you want to remember is a group rather than a single value — deduplicating rows, remembering which pairs you have used, grouping anagrams. **The fix is a tuple.** `(2, 5)` is hashable; `[2, 5]` is not. For an unordered group, `frozenset` is the hashable set:

```
seen_pairs: set[tuple[int, int]] = set()
seen_pairs.add((2, 5)) # fine
seen_groups: set[frozenset[str]] = set()
seen_groups.add(frozenset({"a", "b"})) # fine, and order-independent
```

Trap 4 — mixing sets and lists in the operators

```
>>> {1, 2} | [3]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for |: 'set' and 'list'
```

The operators demand sets on both sides. The method forms are more forgiving — `{1, 2}.union([3])` works and gives `{1, 2, 3}` — because they accept any iterable. Know both, and know why one raises.

Trap 5 — changing a set while iterating over it

```
>>> s = {1, 2, 3}
>>> for x in s:
...     s.add(x + 10)
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RuntimeError: Set changed size during iteration
```

Identical in spirit to the dictionary version from [day 060](#). If you must add while walking, iterate over a copy — `for x in list(s):` — or collect the additions and apply them after the loop.

Trap 6 — assuming order

```
>>> list({"ravi", "asha", "meena"})  
['meena', 'asha', 'ravi']
```

Run it in a fresh interpreter and the order can differ, because string hashing is randomised per process (day 060). If you return `list(set(x))` from a function whose expected output is order-sensitive, your submission fails on a test you cannot reproduce locally. Use `dict.fromkeys` when order matters, or sort the result if the problem asks for sorted output. Never rely on the order a set gives you, not even for small sets, not even when it looks stable.

8 In the interview

How it gets asked

- “Does this array contain any duplicates?” — the warm-up. It is almost never the real question.
- “Find the first repeating element. Now do it in one pass.” The words **one pass** are the signal. They mean: no nested loop, no sort, walk the data once.
- “Given an unsorted array of integers, return the length of the longest consecutive sequence. Your algorithm should run in $O(n)$.” The complexity is stated in the question, which is the interviewer telling you that sorting is not the answer.
- The vague version: “Two files each have a million user ids. Which ids are in both?” No complexity mentioned, no hint. This one separates people, because you have to notice for yourself that the obvious answer is a hundred trillion comparisons.

What to say out loud, in the first ninety seconds

1. **Restate what makes it hard.** “The brute force is: for every element, look at every other element. That is n squared. With a million ids that is 10 to the twelfth comparisons, so it is not an answer.”
2. **Name the tool before you write it.** “I want to turn the inner loop into a constant-time question. A set gives me that — membership in $O(1)$ average, because it is a hash table with no values.”
3. **State the invariant.** “I will walk the input once, keeping a set of everything I have already seen. At the top of each iteration, that set is exactly the elements before the current one.”
4. **State the cost before you write.** “Time $O(n)$, extra space $O(n)$. I am buying time with memory, and if you want $O(1)$ space I would sort instead and pay $O(n \log n)$.”
5. **Ask the one clarifying question that matters.** For duplicates: “when you say the first duplicate, do you mean the earliest second occurrence, or the element whose first occurrence is earliest?” For anything with output: “does the order of the result matter?”

Then write it. The code is six lines, so most of your value is in steps 1 to 5.

The follow-ups

“What is the worst-case time, not the average?” $O(n)$ per operation, so $O(n^2)$ overall, if every element collides into one bucket. In practice that needs either a deliberately hostile input or a bad custom hash, and Python randomises string hashing per process to stop the first — see [day 061](#). Then say the honest part: “I would still quote $O(n)$ average in a design discussion, but I would not quote it for a service that hashes untrusted keys.”

“Now do it in $O(1)$ extra space.” “Then I give up the set. If I may modify the input, sorting makes duplicates adjacent, so one pass after the sort finds them — $O(n \log n)$ time, $O(1)$ extra space. If the values happen to lie in a known small range, say 1 to n , I can mark them in the input itself by negating, which is $O(n)$ time and $O(1)$ extra space.” Naming both alternatives, with their conditions, is what a strong answer looks like.

“Why is your inner `while` loop not making that quadratic?” “Because it is bounded in total, not per iteration. Only a number with no left-hand neighbour enters the `while` at all, so every value is stepped over by exactly one run’s walk. All the `while` loops together do at most n steps.”

“Would you use a set or a dictionary here?” “A set, because I only need presence. The moment I need to remember *where* I saw it, or how many times, it becomes a dictionary — the set is the dictionary with the values thrown away. Two Sum is the classic example: I need the position, so it has to be a map.”

A model answer

Asked: *given an unsorted list of integers, return the length of the longest run of consecutive integers, in $O(n)$ time.*

“The obvious approach is to sort and then walk, looking for consecutive values. That is correct and it is $O(n \log n)$, and since you have asked for $O(n)$, the sort has to go.

The only question this problem actually needs to ask is ‘is the number 57 present?’. Order does not matter, and duplicates do not matter. A set answers that in constant time, so my first line puts everything in a set.

Now, if I take each number and walk upward while the next value exists, I get the right answer but the wrong complexity — on the input 1 to n , I start at 1 and walk n steps, then start at 2 and walk n minus 1, and so on. That is quadratic.

The fix is to only start walking from a number that begins a run, which I can test in one line: if `number - 1` is in the set, this number sits in the middle of a run, so I skip it. Its run will be counted by its own smallest element. That means every run is walked exactly once, by its start.

So the cost is n membership checks for the outer loop, plus at most n steps across all the inner walks put together — $O(n)$ time. Space is $O(n)$ for the set.

Edge cases: an empty input returns 0, because my best starts at 0 and the loop never runs. Duplicates take care of themselves, because the set holds each value once. Negative numbers are fine — nothing here assumes the values are positive.”

That answer is about ninety seconds spoken, it names the wrong approach and says why it is wrong, and it gives the complexity argument that the follow-up would otherwise have to drag out of you.

9 Recall card

- **A set is a hash table with the values deleted.** `in` is $O(1)$ average, duplicates are impossible, order does not exist, elements must be hashable — so tuples and `frozenset`, never lists or dicts.
- **The seen-set pattern is the reflex:** walk once, `if x in seen`, then `seen.add(x)`. The invariant is *seen holds exactly the elements before the current one*. It turns 50,000,000 comparisons at $n = 10,000$ into 20,000 operations.
- **`seen = []` is the trap** — `in` on a list is $O(n)$, so identical-looking code runs $700\times$ slower at $n = 20,000$. And `{}` is an empty dict; the empty set is `set()`.
- **Longest consecutive: put it in a set, and only walk from a number whose `number - 1` is absent.** Without that guard `range(100000)` is 5×10^9 steps. The `while` is bounded in total, not per iteration — n outer checks plus n total inner steps.
- **You are trading memory for time** — $O(n)$ extra space, roughly 32 MB per million integers. Asked for $O(1)$ space, sort instead ($O(n \log n)$) or mark in place when the range is known. Real errors: `TypeError: unhashable type: 'list'`, `RuntimeError: Set changed size`

during iteration.

1 What this is, and why they ask it

This is the closing day of the design-principles phase, and it is not a summary. It is a **procedure**: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first. The eight principles you have met — the five in SOLID, plus DRY, KISS and YAGNI, plus the coupling and cohesion vocabulary — are the vocabulary. The procedure is what turns them into an answer.

They ask it because it is the cheapest possible test of whether you have used these ideas or read about them. An interviewer puts sixty lines on the screen and says “what do you think?”. Someone who has only read about it says “it violates single responsibility” and stops. Someone who has done it says “the imports are `decimal` and `smtplib`, which is finance and platform in one file; the constructor takes a `psycopg` connection, so the domain depends on the driver; and there are four boolean parameters, which is sixteen combinations of which about five are meaningful.” Same principle named. Completely different signal.

The other reason is that this round is a proxy for how you will behave on their codebase. They are not really asking whether you can spot a violation. They are asking whether you will find one and immediately rewrite the world, or find one and ask what changes most often.

2 The story

Vinay had two lakh rupees and wanted a second-hand scooter, and after three weeks of scrolling he had found one he liked — a grey Activa, 2019, forty thousand on the clock, one owner, a flat in HSR Layout. The photos were good. He was fairly sure about it before he had even seen it.

He took his mother’s brother with him, because his mother made him. Ganesh has spent thirty-one years in a garage in Kanakapura Road and does not like being hurried.

Vinay walked round the scooter twice, sat on it, and said it felt nice. That was his whole review.

Ganesh did not sit on it. He started at the front and did the same six things he does every time, in the same order, and he told Vinay what each one was for as he went. He put a thumb on the tyre wall and looked at the date stamped there — 2017. Older than the scooter, so the tyres had never been changed. He opened the seat and looked at the underside of the storage bin for water marks, because a scooter that has stood in a flood looks perfect from the outside. He started it cold and listened for the first four seconds only, because that is when the noises are honest. He put it on the centre stand and spun the front wheel with one finger and watched

where it stopped, twice. He looked at the two bolts holding the front panel and saw that the paint on their heads was gone, which means the panel has been off, which means something happened at the front. And last he asked the owner one question: what did you get done at the last service, and when.

Then he told Vinay: it is a good scooter, the engine is fine, someone has hit the front and had it repaired reasonably, the tyres will cost you six thousand next month, and offer him eight thousand less than he is asking.

On the way home Vinay asked how he knew all that. Ganesh said the useful part is not the knowing. It is that he does the same six things in the same order on every single vehicle, so he never gets talked out of one of them by a nice grey colour and a clean seat.

3 The idea in plain English

Vinay’s review was a feeling. Ganesh’s review was a **procedure** — the same checks, in the same order, each one with a reason he could say out loud, ending in a judgement and a price.

A design critique in an interview is the same shape. The principles are the checks. Doing them in a fixed order is what stops you from reading sixty lines, latching on to the first ugly thing, and missing the two that matter.

Here is the whole phase in one table. Each row is a check, what it looks like when it fails, and the move that fixes it.

PRINCIPLE	THE CHECK	THE TELL	THE MOVE
Single responsibility (day 055)	How many <i>people</i> would ask for a change to this file?	Two teams in the version history; imports from two worlds	Group methods by stakeholder, extract the most independent group
Open/closed (day 056)	To add the next variant, do I edit this file or add one?	An <code>if / elif</code> chain over a type that keeps growing	Put an interface on the axis of change; new variant = new file
Liskov substitution (day 057)	Can any subtype go anywhere the parent goes, without the caller knowing?	An override that <code>raise s; isinstance</code> in <i>callers</i>	Model the capability, not the taxonomy; or make it immutable
Interface segregation (day 058)	Does any caller depend on methods it never calls?	<code>NotImplementedError</code> in an implementation; fat test fakes	Split into role interfaces named after a capability
Dependency inversion (day 059)	Which way does the import arrow point?	Domain code importing <code>psycpg</code> , <code>stripe</code> , <code>boto3</code>	Put the interface in the policy's package, in its vocabulary
DRY (day 060)	Is the same <i>fact</i> stated in more than one place?	A number like <code>18</code> for GST in nine files	Extract the fact, not the function
KISS (day 060)	What is the fewest moving parts that solves the actual problem?	A registry and three abstract classes for four cases	Delete the mechanism, keep the dict
YAGNI (day 060)	Can I name the second thing this generality is for?	<code>strategy</code> parameters with one implementation	Remove it until the second arrives
Coupling / cohesion (day 061)	What crosses the boundary, and what is inside it?	Shotgun surgery, divergent change	Gather what is scattered, split what is piled up

The one sentence that ties them together

Every principle in that table is answering the same question in a different costume: **when the requirements change, how much of this do I have to touch?**

Single responsibility says: keep the things that change for the same reason together. Open/closed says: make the common change an addition rather than an edit. Liskov and interface segregation say: make sure the abstraction you built to enable that actually holds. Dependency inversion says: point the arrows so the stable code does not move when the volatile code does. DRY says: a fact should have one home so a change to it has one edit. KISS and YAGNI say: do not pay for flexibility you cannot name a use for.

Say that out loud in an interview and you have separated yourself from the person reciting five acronyms.

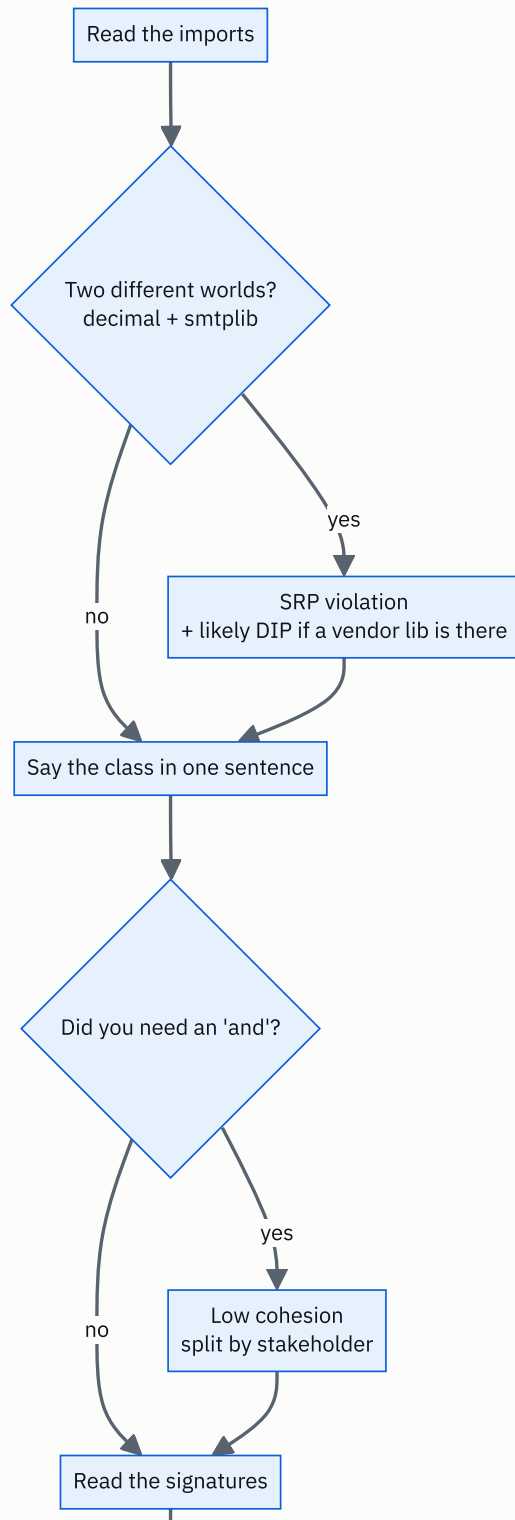
The order matters

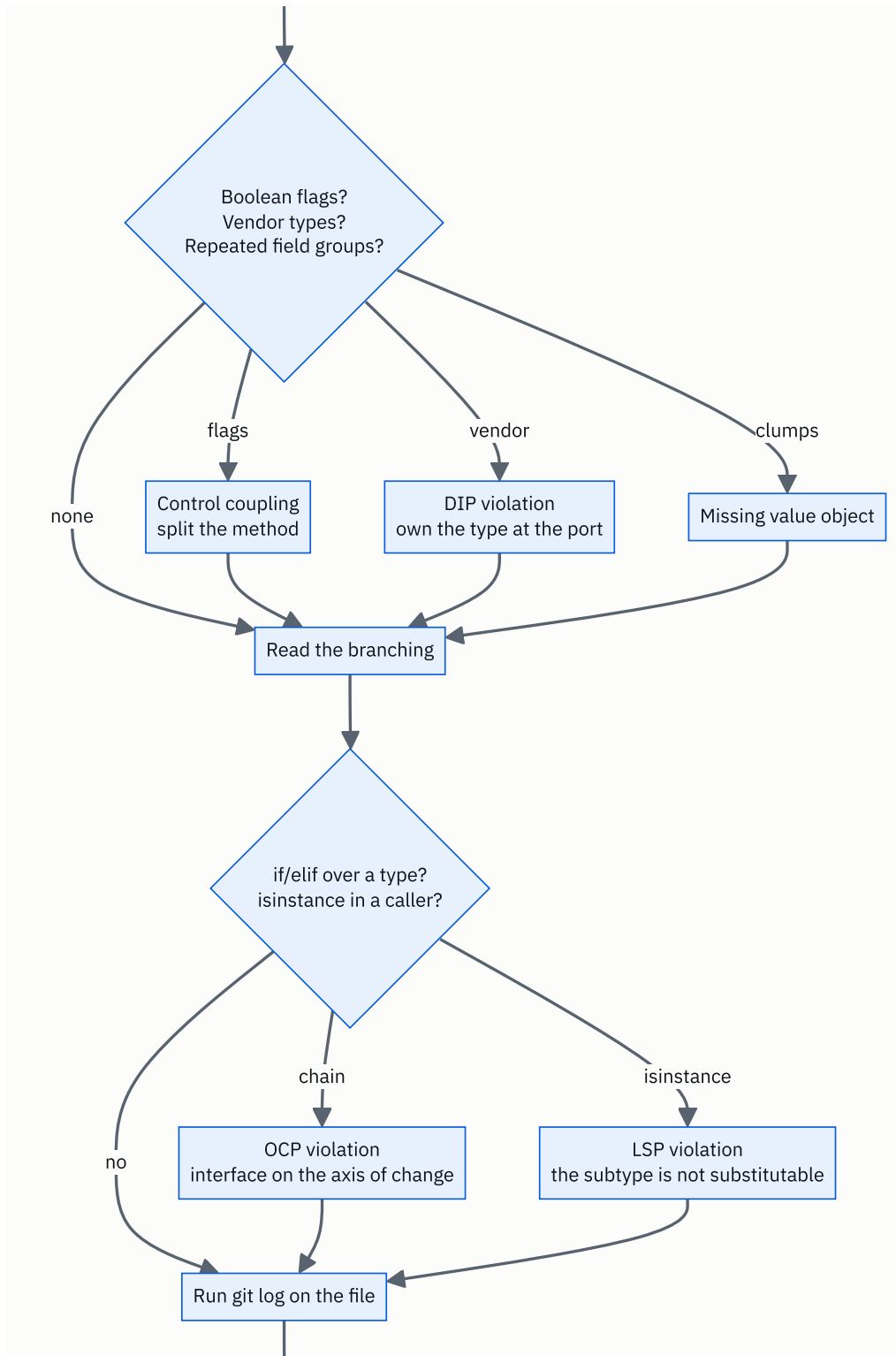
Ganesh does not check the tyres last, and you should not check DRY first. The order below runs from the checks that are cheap and factual to the ones that need judgement, so you say something concrete in the first fifteen seconds and something considered in the last fifteen.

1. **The imports.** Free, factual, and it usually gives you two principles at once.
 2. **The class name and a one-sentence description.** Count the “and”s.
 3. **The signatures.** Boolean flags, long parameter lists, vendor types, data clumps.
 4. **The branching.** `if` chains over a type, `isinstance` in callers.
 5. **The state.** Which fields does each method touch? Two clusters means two classes.
 6. **The history.** Who has edited this file, and for what? This is the only measurable one.
-

4 The picture

The review as a flow. Each diamond is one of Ganesh’s six checks, and each one hands you a named principle and a specific fix.





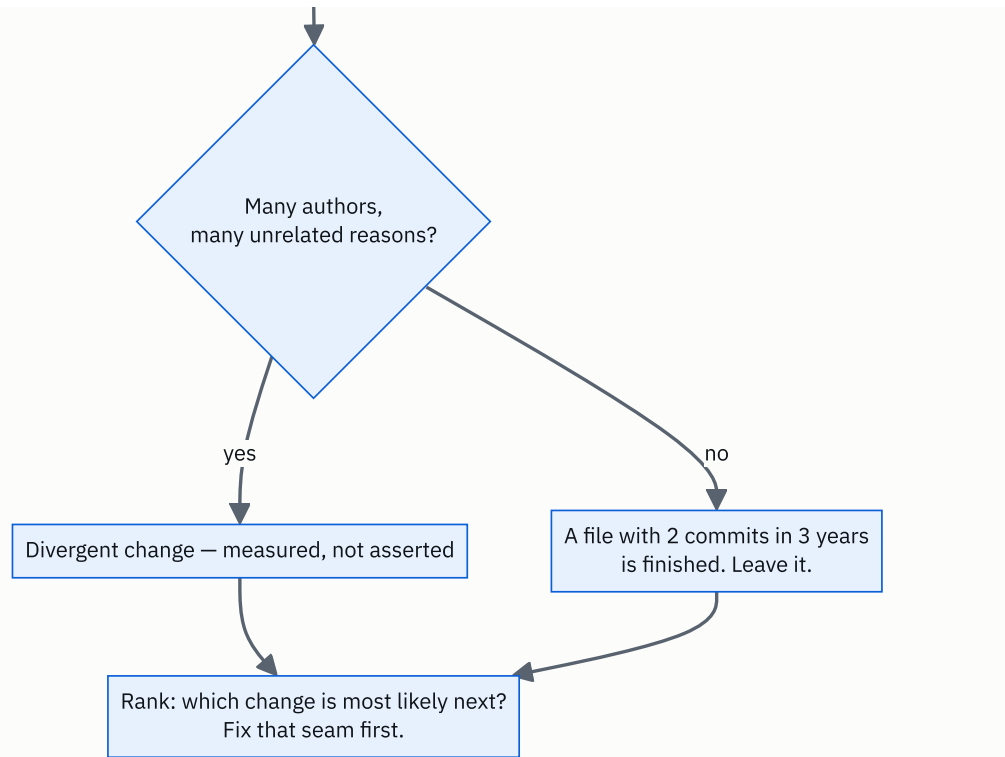


FIGURE 62.1

What to notice: every path ends at the same box. The output of a critique is not a list of violations, it is **one ranked recommendation**. An interviewer who gets nine findings and no priority learns that you would be exhausting to work with.

5 How it actually works

Here is the class. Read it before you read the analysis — this is the exercise, and it is very close to what gets put on a screen in a real round.

```
# reporting/report_manager.py
import csv
import smtplib
import psycopg
from decimal import Decimal
from datetime import date

TAX_RATE = None # set at startup by main.py

class ReportManager:
    def __init__(self, conn: psycopg.Connection):
        self.conn = conn

    def generate(self, month: date, kind: str, email: bool = False,
                as_csv: bool = False, include_tax: bool = True,
                dry_run: bool = False) → str:
        with self.conn.cursor() as cur:
            cur.execute("SELECT id, customer_id, amount_paise FROM orders "
                        "WHERE date_trunc('month', created_at) = %s", (month,))
            rows = cur.fetchall()

            total = Decimal(0)
            for row in rows:
                amount = Decimal(row[2]) / 100
                if include_tax:
                    amount = amount * (1 + Decimal(TAX_RATE))
                total += amount

            if kind == "sales":
                body = f"Sales for {month}: {total}"
            elif kind == "tax":
                body = f"Tax collected: {total * Decimal(TAX_RATE) / (1 + Decimal(TAX_RATE))}"
            elif kind == "refunds":
                body = f"Refunds: {total}"
            else:
                raise ValueError(kind)

            if as_csv:
                with open(f"/tmp/{kind}.csv", "w") as handle:
                    csv.writer(handle).writerows(rows)

            if email and not dry_run:
                server = smtplib.SMTP("smtp.internal", 25)
                server.sendmail("reports@acme.in",
                               self.owner.contact.email.address, body)

        return body
```

Pass 1 — the imports

`csv`, `smtplib`, `psycopg`, `decimal`. Four worlds: file formatting, email delivery, the database driver, and money arithmetic. Four different teams would ask for changes to those four things.

That is **single responsibility**, and the evidence is the import block, which took two seconds to read. It is also **dependency inversion**, because `psycopg` is a vendor library and this is domain code — `grep -rn "psycopg" reporting/` would return this file, which is the proof [day 059](#) told you to run.

Pass 2 — the name and the sentence

`ReportManager`. `Manager` is a noise word — it is the name you give a class when you cannot say what it does. Try the sentence: “it fetches order rows from the database **and** computes a total **and** applies tax **and** formats one of three report kinds **and** writes a CSV file **and** sends an

email.” Five “and”s. That is **low cohesion**, of the **logical** kind — things grouped because they are all vaguely “reporting”.

Pass 3 — the signatures

Four boolean parameters: `email`, `as_csv`, `include_tax`, `dry_run`. That is $2^4 = \text{sixteen}$ **combinations**, of which perhaps five are meaningful, and at least one is nonsense that compiles: `email=False, dry_run=True` does nothing at all, silently. This is **control coupling** — the caller is passing in a flag that picks a branch inside the callee, which means the caller has to know the callee’s internal structure.

`kind: str` is worse than it looks. It is a string that is really an enum of three values, checked at run time, so a typo is a `ValueError` in production rather than a red squiggle in the editor.

And `conn: psycopg.Connection` in the constructor is the sentence from [day 059](#) worth repeating: **this is dependency injection without dependency inversion**. The collaborator is passed in, which looks like good practice, but the type in the signature belongs to the driver, so nothing was inverted.

Pass 4 — the branching

The `if kind == "sales" / "tax" / "refunds"` chain is the **open/closed** violation, and the test is the one from [day 056](#): can you name the fourth report kind? If the product team has “settlements” on a roadmap, then yes — put an interface on that axis and each kind becomes its own file. If three is genuinely the whole set and has been for two years, leave the chain alone and say why. **Both answers are correct; only one of them is correct for this code-base, and you have to ask.**

Pass 5 — the state and the hidden global

`TAX_RATE = None`, set at startup by `main.py`, is **common coupling** — the worst rung on the ladder after content coupling. Two concrete consequences, and give both: any test of this class must remember to set a module-level global first or get `TypeError: conversion from NoneType to Decimal is not supported`, and two tenants on two tax rates in one process is impossible without a code change.

`self.owner.contact.email.address` is a **message chain**: four dots, so this method depends on the existence and shape of four classes to get one string. Law of Demeter. It is also a bug — `self.owner` is never assigned, so this line raises `AttributeError: 'ReportManager' object has no attribute 'owner'` the first time anyone passes `email=True`, which tells you that path has no test.

Pass 6 — the history

You cannot run `git log` on a screenshot, so say what you would run and what you expect:

```
git log --format='%an' -- reporting/report_manager.py | sort | uniq -c | sort -rn
```


If that shows commits from finance, from the platform team and from growth, you have **divergent change** measured rather than asserted, and it is the single strongest sentence available in this round because it is evidence rather than opinion.

What real systems look like when this is done right

Every one of these principles is visible in software you already use, and naming a real product is what stops the answer sounding like a textbook.

- **Open/closed:** Django middleware, pytest plugins, `sorted(key=)`, Kubernetes custom resources. In every case you add a file and a line in a list, and never edit the framework.
- **Dependency inversion:** SQLAlchemy Core versus your domain models; the twelve-factor rule that config comes from the environment. Postgres, MySQL and SQLite behind one `DBAPI` interface is the same shape.
- **Interface segregation:** `collections.abc` splits `Iterable`, `Sized`, `Container`, `Sequence`, `MutableSequence` rather than shipping one `Collection` with thirty methods. Go's `io.Reader` and `io.Writer` are one method each on purpose.
- **Liskov:** `java.sql.Date` extends `java.util.Date` and throws on the time methods, which is the standard-library example everyone cites. `Collections.unmodifiableList` is the same failure.
- **DRY, done wrong, at scale:** every company with a `common-utils` library that four teams depend on and nobody may change.

The refactored version, in outline

Not because you would write it all on a whiteboard, but because the interviewer will ask what it would look like.

```
# reporting/ports.py – the interfaces, in the domain's vocabulary
class OrderRepository(Protocol):
    def orders_for_month(self, month: date) → list[Order]: ...

class ReportDelivery(Protocol):
    def deliver(self, report: Report) → None: ...

# reporting/reports.py – one file per kind, added not edited
class SalesReport:
    def build(self, orders: list[Order], tax: TaxRate) → Report: ...

# adapters/postgres_orders.py – the only file that imports psycopg
# adapters/email_delivery.py – the only file that imports smtplib
```

Three things to point at when you show it. The `psycopg` import now exists in exactly one file. The tax rate is a parameter, not a global. And `email`, `as_csv` and `dry_run` are gone, because choosing a delivery is choosing an object rather than setting a flag.

6 The numbers

Design arguments lose to “we do not have time” unless you bring arithmetic. These are the numbers this phase produced, gathered in one place because this is the round where you quote them.

Adding the fourth report kind.

```
with the if/elif chain:
  files edited          1 (a shared 90-line file, on the critical path)
  existing tests re-run 40 (the whole reporting suite)
  live reports at risk  3
with an interface:
  files added           1
  lines edited          1 (the registry)
  existing tests re-run 0
```

And the cost stays flat: the eighth kind costs what the fourth cost. With the chain, the eighth kind is edited into a file that is now 200 lines and understood by nobody.

Swapping the database driver. Before: `grep -rn "psycopg" reporting/` returns 38 references across 14 files, so the swap is a two-week project touching domain code. After: 1 new adapter file and 1 wiring line.

Testing “a partial month total is correct”.

```
before: start Postgres, load schema, insert 4 rows, set the TAX_RATE global,
        construct a connection          ~ 26 lines of setup, 180 ms per test
after:  build 4 Order objects, call one function
                                     ~ 4 lines of setup, 0.1 ms per test
```

180 ms against 0.1 ms is $1,800\times$, and with 400 such tests it is the difference between a 72-second suite and a 40-millisecond one. That number is what actually persuades a team, because it is felt every day rather than argued about once.

The cost of the boolean flags. 4 flags = 16 combinations. Testing them all is 16 test cases. In practice a team tests 3 and the other 13 are undefined behaviour that somebody will eventually discover in production.

The cost of getting it wrong in the other direction. From [day 060](#): the wrong abstraction grew from 24 lines to 71, from 6 tests to 14, and took about three days to untangle. Extracting a duplicate later costs ~ 20 minutes; un-extracting a bad shared function costs **20 to 50 $\times$ more**. That asymmetry is why “leave it duplicated for now” is a real engineering answer and not laziness.

7 The trade-offs

The principles contradict each other, and you should say so

This is the most valuable thing you can bring to this round, because it is the part that reading blogs does not give you.

DRY fights single responsibility. Two teams have near-identical validation. DRY says extract it. SRP says they change for different reasons, so they belong apart. SRP wins — different owners means different futures.

Open/closed fights KISS. Every axis you leave open costs an interface, an extra file and a hop of indirection — roughly 22 lines and 3 files per axis. Opening an axis you never use is pure cost forever. KISS wins until you can name the second implementation.

Interface segregation fights discoverability. Six one-method interfaces make every caller's dependency minimal, and make it impossible for a newcomer to answer “what can this thing do?”. Stop at three or four roles.

Dependency inversion fights YAGNI. A port and an adapter for a database you will never change is 108 lines of structure buying nothing — except that the test double counts as the second implementation, which is usually enough to justify it on its own.

What you give up by applying these at all

Indirection. Every one of these moves replaces a thing you can read with two things you have to jump between. A new engineer reading the refactored version has to open four files to follow one report being generated, where before they read one function top to bottom. That is a real cost paid by real people, and pretending otherwise is how codebases end up with fifteen one-method classes.

“I would not use this if...”

- **I would not invert a dependency if** I cannot name the second implementation, and the test double does not count because the thing is trivially fakeable in place.
- **I would not open an axis if** the set is genuinely closed — days of the week, the four blood groups, the three states an order can be in.
- **I would not extract a duplicate if** the two copies are owned by different teams, however identical they look today.
- **I would not split a class if** it is long but cohesive. A `Matrix` with forty methods is fine. Length is not the smell; **number of reasons to change** is.
- **I would not refactor at all if** `git log` shows two commits in three years. That file is finished. Refactor what changes.

The trap in this interview round specifically

The trap is enthusiasm. You are shown bad code and invited to list what is wrong with it, and the natural response is to list everything. Do not. Name the findings, then rank them, then say which single change you would make first and what you would look at before making it. An

interviewer is imagining you on their codebase, and their codebase has a `ReportManager` in it that everybody knows about and nobody has had time to fix.

8 In the interview

How it gets asked

- The direct version: *“Here is a class. Which SOLID principles does it violate?”* Sixty lines on a screen, five minutes to read, then talk.
- The realistic version: *“Review this pull request. What comments would you leave?”* Same skill, and now the ranking matters more than the naming, because a review with nine blocking comments is a review nobody will act on.
- The abstract version: *“Explain SOLID.”* This one is a trap dressed as a gift. Reciting five definitions is a mediocre answer. The good answer defines them **through one running example** and says what they have in common.
- The reversal: *“When would you not follow these principles?”* Asked to find out whether you are a person who has used them or a person who has read about them.

What to say out loud, in the first ninety seconds

Do not start naming principles. Start with the two sentences that frame everything:

1. **Say what the code is for, in one sentence, and count your “and”s.** “This fetches orders for a month, computes a taxed total, formats one of three reports, optionally writes a CSV, and optionally emails it. That is five ‘and’s, so I already know it does more than one job.”
2. **Say what you would need to know before recommending anything.** “Before I change anything I would want the version history of this file and the roadmap for report kinds, because the right fix depends on what actually changes.”
3. **Then give three findings with evidence, not labels.** Imports, signatures, branching.
4. **Then rank.** “Of those, the one I would fix first is the database dependency, because it is what makes every test in this area slow, and slow tests are why the other problems never got fixed.”
5. **Then say the thing you would leave alone.** “I would not touch the three-way `if` on report kind yet. Three is a small closed set. If there is a fourth coming, that changes.”

That fifth point is the one candidates never make, and it is the one that reads as senior.

The follow-ups

“Which of those would you fix first, and why?” “The `psycogg` dependency, and my reason is testability rather than purity. Right now a test of the tax arithmetic needs a running database, which is 180 milliseconds and a schema. With the repository behind an interface it is four `Order` objects and a function call. That one change makes the others cheap to attempt, because I would then have tests to refactor against.”

“Is that not over-engineering for a ninety-line class?” “It would be if I were doing it speculatively. I am doing it because there is a second implementation today — the test double — and because the class already imports a driver, a mail library and a file format, so it has three reasons to change. If this file had two commits in three years I would leave it and spend the time elsewhere.”

“You said single responsibility. What is a responsibility?” “One reason to change, and a reason is a person or a role, not a topic. Here that is finance for the tax arithmetic, the platform team for the driver, and whoever owns delivery for the email. Three stakeholders in one file means one team’s change can break another’s without anybody meaning it.”

“How would you know if you had gone too far?” “When following one flow means opening four files and nothing in any of them makes a decision. Also when interfaces have exactly one implementation and no test double. Fifteen one-method classes is the same mistake as the god class, inverted — I would be optimising for a change that never comes.”

“Explain SOLID in one minute.” “They are five answers to one question: when the requirements change, how much do I have to touch? Single responsibility keeps things that change together in one place. Open/closed makes the common change an addition rather than an edit. Liskov and interface segregation make sure the abstractions I built for that actually hold — subtypes really are substitutable, and callers only depend on what they use. Dependency inversion points the import arrows so the stable code does not move when the volatile code does.”

A model answer

Asked: *here is `ReportManager`. What do you think?*

“Let me say what it does first. It reads a month of orders from the database, computes a total, applies tax, formats one of three report kinds, optionally writes a CSV file, and optionally emails it. That is five ‘and’s, which is my first signal.

The imports confirm it: `psycopg`, `smtplib`, `csv` and `decimal`. Those are four different worlds and four different teams. Finance owns the tax arithmetic, the platform team owns the driver, somebody else owns delivery. Single responsibility is about reasons to change, and there are at least three here.

Two specific things I would raise in a review. The constructor takes a `psycopg.Connection`, so the reporting package depends on the database driver — that is injection without inversion, and its practical cost is that testing the tax arithmetic needs a running Postgres. And `TAX_RATE` is a module-level global set at startup, which is common coupling: tests have to remember to set it, and two tax rates in one process is impossible.

On the signature, four booleans is sixteen combinations and about five meaningful ones — `email` is false with `dry_run` true does nothing at all. I would replace the flags with explicit calls or with a delivery object.

The `if kind == chain` is the open/closed candidate, but I would ask before changing it. If there is a fourth report kind coming this quarter, an interface pays for itself immediately. If three is the whole set and has been for two years, the chain is the simplest thing that works and I would leave it.

If I could only do one thing: put the order fetch behind a repository interface owned by the reporting package. That is one new file and one wiring line, it takes the driver out of the domain, and it turns a 180-millisecond test into a 0.1-millisecond one. Everything else gets easier once there are fast tests to refactor against.

And before doing any of it I would run `git log` on the file. If four teams have been editing it, that is divergent change measured rather than guessed, and it settles the argument. If two people have touched it in three years, the file is finished and I would leave it alone.”

9 Recall card

- **All eight principles answer one question: when requirements change, how much must I touch?** SRP keeps co-changing things together · OCP makes the common change an addition · LSP and ISP keep the abstraction honest · DIP points the import arrows · DRY gives each *fact* one home · KISS and YAGNI refuse flexibility you cannot name a second use for.
- **Review in a fixed order, cheapest evidence first:** imports · one-sentence description (count the “and”s) · signatures (flags, vendor types, clumps) · branching (`if` chains, `isinstance` in callers) · state and globals · `git log`. The history is the only *measurable* check — 4 teams on one file is divergent change proved, not asserted.

- **Name findings with evidence, never with labels.** Not “violates SRP” but “`decimal` and `smtpplib` in one import block”. Not “too many parameters” but “4 booleans = 16 combinations, ~5 meaningful, and `email=False, dry_run=True` silently does nothing”.
- **Always end on a ranking and a thing you would leave alone.** One first change, its reason (usually testability: 180 ms → 0.1 ms), and one honest “I would not touch the three-way `if` until somebody names the fourth kind.”
- **The principles contradict each other and you should say so.** DRY loses to SRP when the owners differ · OCP loses to KISS until you can name implementation two · DIP loses to YAGNI unless the test double counts · and a file with two commits in three years is finished.

PRACTICE

Practice — Sets: membership, deduplication, and the $O(1)$ habit

DSA topic: Sets: membership, deduplication, and the $O(1)$ habit **System design topic:** Design principles revision and interview questions

Code these, in this order

One rule for the whole set: **write the $O(n^2)$ version first, then delete the inner loop**. Doing it in that order twice is what builds the reflex. Doing it in that order in an interview is what lets you say “the brute force is quadratic, and here is why I do not need it”.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Contains Duplicate	LeetCode 217 (Easy)	The seen set in its simplest form, and whether you know why <code>len(set(x)) < len(x)</code> behaves differently on early duplicates.
2	Intersection of Two Arrays	LeetCode 349 (Easy)	Set operators instead of a nested loop, and whether you noticed the question is about distinct values.
3	Happy Number	LeetCode 202 (Easy)	A set used to detect a cycle, not a duplicate — the same tool answering a different question.
4	Longest Consecutive Sequence	LeetCode 128 (Medium)	Whether you can defend an inner <code>while</code> loop that is bounded in total rather than per iteration.

On problem 2, notice which “intersection” you were asked for

`[1, 2, 2, 1]` and `[2, 2]` gives `[2]`, not `[2, 2]`. LeetCode 350 is the same question with multiplicities kept, and it needs a `Counter`, not a set. Solve 349, then say in one sentence what you would change for 350. That sentence is tomorrow’s lesson.

On problem 3, say what the set is holding

It is not holding duplicates in the input. It is holding *states you have already visited*, and the repeat means you are going round in a circle forever. Then answer this: the fast-and-slow pointer method from [day 030](#) solves it in $O(1)$ space. Say which one you would give first in an interview and why.

On problem 4, break it on purpose before you submit

Write it without the `if number - 1 in number_set: continue` guard. Confirm it passes the sample cases. Then run it on `list(range(50000))` and watch it stop being an answer.

The membership drill

Time each of these on 20,000 distinct integers, then say which complexity each one is:

1. `x in a_list`

2. `x in a_set`
3. `x in a_dict`
4. `x in a_dict.keys()`
5. `x in a_dict.values()`
6. `x in a_tuple`
7. `x in a_string` (with string data)

Two of those seven are $O(n)$ and look exactly like the $O(1)$ ones at the call site. Name them, and say what the ratio was at $n = 20,000$.

The seen-set drill

For each question, say whether the answer is a set, a dictionary, or neither, and give the reason in one sentence:

1. Does this list contain a duplicate?
2. Which element is the first duplicate?
3. How many times does each element appear?
4. Which two numbers add up to the target? (return their positions)
5. Which usernames are in file A but not file B?
6. Has this state occurred before in the simulation?
7. What was the last value I saw for this key?
8. Which of these strings are anagrams of each other?

Three of the eight need a dictionary because presence is not enough. Name them and say what extra thing each one needs to remember.

The deduplication drill

Given `["ravi", "asha", "ravi", "meena", "asha", "ravi"]`, produce each of the following, and say which tool you used:

1. The distinct names, order irrelevant.
2. The distinct names, in first-seen order.
3. The distinct names, sorted.
4. The names that appear more than once.
5. The names that appear exactly once.
6. The count of distinct names, without building a list.

Then say why `list(set(names))` gave you a different order the second time you opened a fresh interpreter, and what that would do to a submission.

The unhashable drill

Trigger each error, read the exact text, and give the one-line fix:

1. `{1, 2}.add([3])`
2. `set([{"a": 1}])`
3. `{1, 2} | [3]`
4. `{"a"} < ["a"]`
5. A `for x in s:` loop that calls `s.add(...)` inside it.
6. `{[1, 2], [3, 4]}`

Then rewrite 6 so it works, twice — once where the order inside each group matters and once where it does not.

The space drill

1. Build a list of 1,000,000 integers and measure its memory.
2. Build a set of the same integers and measure its memory.
3. Compute the ratio, and say where the extra bytes went — name the three reasons.
4. Now answer, out loud: the interviewer says “same problem, $O(1)$ extra space”. Give both alternatives, with their complexities and their conditions.
5. Say which of your two alternatives modifies the input, and why that is a question you must ask before choosing it.

The critique drill

Here is the class from today’s design lesson, reproduced so you can work on it without scrolling.

```
# reporting/report_manager.py
import csv
import smtplib
import psycopg
from decimal import Decimal
from datetime import date

TAX_RATE = None    # set at startup by main.py

class ReportManager:
    def __init__(self, conn: psycopg.Connection):
        self.conn = conn

    def generate(self, month: date, kind: str, email: bool = False,
                as_csv: bool = False, include_tax: bool = True,
                dry_run: bool = False) → str:
        ...
```

Work through the six passes in order and write down what each one gives you:

1. Read the imports. How many worlds, and which teams own them?
2. Say the class in one sentence. Count the “and”s.

3. Read the signature. How many boolean combinations? How many are meaningful? Which one is nonsense but compiles?
4. Find the dependency-inversion violation, and say precisely why passing the connection in is not enough to fix it.
5. Find the common coupling. Name two concrete things it breaks.
6. Find the message chain. How many classes does that one line depend on?
7. Which principle does the `if kind ==` chain point at, and what single question decides whether you should act on it?
8. Rank every finding. Which one first, and what is your reason?
9. Name one thing you would deliberately leave alone, and defend it.
10. Say what command you would run before changing anything, and what result would change your mind.

The contradiction drill

For each pair, say which principle wins, and give the deciding question:

1. DRY versus single responsibility, on two identical validators owned by two teams.
2. Open/closed versus KISS, on a three-branch `if` over a closed set.
3. Interface segregation versus discoverability, on a six-method repository.
4. Dependency inversion versus YAGNI, on a database that will never be swapped.
5. Extract-the-duplicate versus leave-it, on the second occurrence of a piece of code.

Two of the five have the same deciding question underneath. Name it.

The “explain SOLID” drill

Answer this in sixty seconds, out loud, three times, and make each attempt shorter than the last:

Explain SOLID.

Rules for the attempt: one running example throughout, no reciting of five definitions in a row, and the last sentence must say what all five have in common. If you find yourself saying “the S stands for”, start again.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the duplicate. Now do it in one pass.* The brute-force count first with a real number at $n = 10,000$, then the seen set with its invariant stated, then the time and extra space, then the $O(1)$ -space alternative volunteered before it is asked for.

2. *Here is a class. Which principles does it violate?* The one-sentence description with the “and”s counted, three findings with evidence rather than labels, a ranking with a reason, one thing you would leave alone, and the command you would run first.
3. *Why is the inner `while` loop in `longest-consecutive` not quadratic?* Bounded in total rather than per iteration, each value stepped over by exactly one run’s walk, n outer checks plus n total inner steps — and the input that breaks it without the guard.
-

Before you move on

- ☐ I wrote the seen-set loop from memory and stated its invariant in one sentence.
- ☐ I timed `x in list` against `x in set` at $n = 20,000$ and can quote the ratio.
- ☐ I broke longest-consecutive by removing the guard, and found the input that exposes it.
- ☐ I can say why the inner `while` loop is $O(n)$ in total.
- ☐ I triggered `TypeError: unhashable type: 'list'` and fixed it two different ways.
- ☐ I can give both $O(1)$ -space alternatives, with complexities and conditions.
- ☐ I critiqued `ReportManager` through all six passes and ended with a ranking.
- ☐ I can name one thing in that class I would deliberately not change, and defend it.
- ☐ I answered all three questions above out loud.

Counting with dictionaries

DATA STRUCTURES AND ALGORITHMS

Counting with dictionaries

You can build, update and query frequency maps without `KeyError`.

SYSTEM DESIGN

What a design pattern actually is

You can say why patterns exist and stop applying them where they do not belong.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the top k most frequent elements.*
- *Which design patterns have you used, and why?*

1 What this is, and why they ask it

A **frequency map** is a dictionary whose keys are the things you are counting and whose values are how many times each one appeared. You build it in one pass over the data, bumping a number as you go. It is the second half of the set: a set remembers *whether* you have seen something, a frequency map remembers *how many times*.

They ask it because counting is the most common single step in the whole of interview coding. Top k frequent elements, valid anagram, first non-repeating character, majority element, ransom note, longest substring with at most k distinct characters — all of them start with the same three lines. An interviewer who gives you one of those is not testing whether you can count. They are testing whether the counting is automatic enough that you have thought left over for the part that is actually hard, which is usually what you do with the counts afterwards.

There is a second thing being tested, and it is more specific. Once you have counts, “give me the top k” has three different answers with three different complexities — sort them, use a heap, or bucket them — and knowing all three, and when each wins, is a very reliable way to turn a medium question into a good conversation.

2 The story

Shobha has been sending lunch boxes to four offices near the flyover in Malleswaram for six years. Forty-one people, six days a week, and she cooks everything herself between five and nine in the morning.

The orders arrive on her phone from about half past nine the night before. They come in one at a time, in no particular order, and they are all one line — bisi bele bath, curd rice, two lemon rice, chapati and palya, curd rice again. By eleven at night there are forty-one of them.

For the first year she used to read the whole lot of them over and over. Once through for the curd rice. Then back to the top and once through again to count the lemon rice. Then again for the chapati. Four dishes, four times through forty-one messages, and if she lost her place she started that dish again. It took nearly half an hour and she got it wrong twice, which meant somebody at an office got a box with the wrong thing in it and rang her about it.

Her son fixed it in about a minute, with a whiteboard and a marker on the kitchen wall.

Now she goes through the messages exactly once, from the top. She reads one, finds that dish on the board, and adds one to the number next to it. Reads the next, adds one. When a dish comes up that is not on the board at all — somebody asks for pulao, which nobody has

asked for in two months — she writes the name on the board with a one next to it, and carries on. That is the only moment she has to stop and think.

By the end of the messages every dish has its total, and she has read each message once. Eleven minutes instead of half an hour, and she has not got it wrong since.

The bit her son was pleased about is that it does not get worse. When the fifth office joined and it went from forty-one boxes to sixty-three, it took a bit longer, but only a bit — sixty-three messages instead of forty-one. The old way would have got worse four times over, because she was reading the whole lot of messages once for every dish.

3 The idea in plain English

The whiteboard is a **dictionary**. Each dish name is a **key** and the number next to it is the **value**. Going through the messages once and bumping a number is building a **frequency map**, also called a **count map** or a **counter** — all three names mean the same thing and interviewers use all three.

The old way — reading everything once per dish — is the nested loop. With n messages and m distinct dishes it is $n \times m$ reads. The whiteboard way is n reads and n bumps, whatever m is.

And the moment Shobha has to stop and think — a dish that is not on the board yet — is the **missing key**, which is the one part of this that goes wrong in code.

The three lines

```
counts: dict[str, int] = {}
for dish in orders:
    counts[dish] = counts.get(dish, 0) + 1
```

`counts.get(dish, 0)` means: give me the current count, or 0 if this dish is not on the board yet. Add one. Store it back. Those three lines handle both cases — the dish that is already there and the dish that is not — without an `if`.

Why not just `counts[dish] += 1`?

Because that is the missing-key problem, and it is the error you will actually hit:

```
>>> counts = {}
>>> counts["pulao"] += 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'pulao'
```

`counts["pulao"] += 1` means *read the value, add one, write it back*, and the read fails because there is nothing to read. Python is not being awkward: on a fresh board there is no number next to pulao to add one to.

The four ways, and when to use each

You will see all four in other people's code, so you need to read all four. You only need to write one.

One — the explicit `if`. Verbose, but nothing is hidden.

```
if dish in counts:
    counts[dish] += 1
else:
    counts[dish] = 1
```

Two — `.get` with a default. The one to write in an interview when you want no imports.

```
counts[dish] = counts.get(dish, 0) + 1
```

Three — `defaultdict(int)`. A dictionary that invents a zero the first time you touch a key.

```
from collections import defaultdict
counts = defaultdict(int)
for dish in orders:
    counts[dish] += 1          # no KeyError — the missing key is born as 0
```

Four — `Counter`. The whole loop, in one line.

```
from collections import Counter
counts = Counter(orders)      # Counter({'curd rice': 12, 'lemon rice': 9, ...})
```

`Counter` is a dictionary subclass, so everything you know about dictionaries still works on it. In an interview, write `Counter` and then say “if you would rather see the loop, it is three lines” — that shows you know the library without hiding behind it.

What `Counter` gives you beyond counting

```
c = Counter("aabbbc")
c["z"]          # 0 — a missing key is 0, not a KeyError
c.most_common(2) # [('b', 3), ('a', 2)] — sorted by count, descending
c.most_common()  # everything, descending
list(c.elements()) # ['a','a','b','b','b','c'] — back to the original multiset
Counter("aabbcc") - Counter("abz")  # Counter({'a': 1, 'b': 1, 'c': 1})
```

Two of those are worth pausing on.

`c["z"]` returning `0` is convenient and it is a trap. On a plain dictionary a typo raises `KeyError` and you find it immediately. On a `Counter` a typo silently returns `0` and your answer is quietly wrong.

`Counter - Counter` drops zero and negative counts, which is exactly what you want for questions like “can I build this word from these letters” and exactly not what you want if you were expecting arithmetic. `Counter("ab") - Counter("abz")` is `Counter()` — the `z` does not appear as `-1`.

Counting by something derived from the element

This is the step that turns counting into a technique rather than a chore. The key does not have to be the element. It can be anything you compute from it.

```
Counter(len(word) for word in words)      # how many words of each length
Counter(word[0] for word in words)        # how many words start with each letter
Counter("".join(sorted(w)) for w in words) # how many words share each letter-multiset
```

That third one is the anagram key, and it is [day 064](#)'s entire subject. Choosing the key is the problem, in a very large number of questions.

Getting the top k out

Once you have counts, “the k most frequent” has three answers.

- **Sort everything.** `sorted(counts.items(), key=lambda kv: -kv[1])[:k]`. Simple, $O(m \log m)$.
- **A heap.** `counts.most_common(k)` uses one internally: $O(m \log k)$. Better when k is small and m is large.
- **Bucket by count.** Because a count can never exceed `n`, you can make a list of `n + 1` buckets and drop each key into the bucket for its count, then read the buckets from the top. $O(n + m)$ — no logarithm at all.

All three are correct. Which one you offer, and whether you can say why, is the question.

4 The picture

The frequency map being built, one element at a time. The input is a stream of dish codes.

```
orders:  C   L   C   B   C   L   P
         |   |   |   |   |   |
step 1   C:1
step 2   C:1 L:1
step 3   C:2 L:1
step 4   C:2 L:1 B:1
step 5   C:3 L:1 B:1
step 6   C:3 L:2 B:1
step 7   C:3 L:2 B:1 P:1    ← P is new: the missing-key moment
```

Notice steps 3, 5 and 6. Those are the cheap ones — the key exists, so it is a read and a write, no decision. Steps 1, 2, 4 and 7 are the ones where the key had to be created. `counts.get(k, 0)` makes those two cases look identical in the code, which is the whole reason to write it that way.

Now the bucket trick for top k, drawn on the finished counts `{C:3, L:2, B:1, P:1}` with `n = 7`:

count:	0	1	2	3	4	5	6	7
bucket	[]	[B,P]	[L]	[C]	[]	[]	[]	[]

^
walk from the right,
collect until you have k

What to notice: there are exactly $n + 1$ buckets, because no element can appear more than n times. That bound is why this is $O(n)$ and not $O(n \log n)$ — you never sort anything, you just walk a list you already know the length of.

5 The code, built step by step

Three small ones to get the shape into your fingers, then the real question.

Step 1 — is this an anagram?

```
from collections import Counter

def is_anagram(first: str, second: str) → bool:
    return Counter(first) == Counter(second)
```

Two strings are anagrams if their letter counts match. `Counter.__eq__` compares the dictionaries, so this is one line and it is $O(n)$. Worth saying: the sorting answer, `sorted(first) == sorted(second)`, is also one line and is $O(n \log n)$. Counting wins on time; sorting wins on space when the alphabet is huge. Mention both — this is a [day 022](#) question and the interviewer expects the comparison.

Step 2 — the first character that does not repeat

```
def first_unique(text: str) → str | None:
    counts = Counter(text)
    for character in text:          # walk the ORIGINAL, not the counter
        if counts[character] == 1:
            return character
    return None
```

Two passes, and the second one matters. You must walk the original string, not the dictionary, because the question asks for the *first* such character and only the original knows the order. This is the single commonest mistake in this problem.

Step 3 — the majority element

```
def majority(numbers: list[int]) → int:
    counts = Counter(numbers)
    return counts.most_common(1)[0][0]
```

`most_common(1)` returns a list of one `(value, count)` pair, so the indexing is `[0][0]`. It is ugly and it is correct. If the interviewer then says “ $O(1)$ space”, the answer is the Boyer-Moore voting algorithm, and you should say so even if you do not write it: keep one candidate and one count, increment on a match, decrement otherwise, and the survivor is the majority.

Step 4 — top k frequent elements

Given `[1, 1, 1, 2, 2, 3]` and `k = 2`, return `[1, 2]`. This is LeetCode 347, and the interesting part is that there are three answers.

Start the same way, always:

```
counts = Counter(numbers)      #  $O(n)$ 
```

Answer A — sort the counts.

```
ordered = sorted(counts, key=counts.get, reverse=True)
return ordered[:k]
```

m distinct values, so $O(m \log m)$. Perfectly acceptable, and the one to write first if you are short of time. Say the complexity as you write it.

Answer B — a heap of size k .

```
import heapq
return heapq.nlargest(k, counts, key=counts.get)
```

$O(m \log k)$. When k is 10 and m is a million, $\log k$ is about 3 and $\log m$ is about 20, so this is roughly seven times less comparison work. This is what `most_common(k)` does internally.

Answer C — bucket by count. The one that beats the log entirely.

```
buckets: list[list[int]] = [[] for _ in range(len(numbers) + 1)]
for value, count in counts.items():
    buckets[count].append(value)
```

Bucket i holds every value that appeared exactly i times. There are $n + 1$ buckets because no value can appear more than n times. Now read from the right:

```
result: list[int] = []
for count in range(len(buckets) - 1, 0, -1):
    for value in buckets[count]:
        result.append(value)
        if len(result) == k:
            return result
```

The `if len(result) == k` check has to be inside the inner loop, not after it. A single bucket can hold more values than you need — if every value appears once and k is 3, bucket 1 holds all of them.

The complete solution

```
from collections import Counter
import heapq

def top_k_frequent(numbers: list[int], k: int) → list[int]:
    """The k most frequent values, in O(n) time using bucket counting.

    Bucket i holds every value that appeared exactly i times. There are n + 1
    buckets because no value can appear more than n times, so no sort is needed.
    """
    counts = Counter(numbers)

    buckets: list[list[int]] = [[] for _ in range(len(numbers) + 1)]
    for value, count in counts.items():
        buckets[count].append(value)

    result: list[int] = []
    for count in range(len(buckets) - 1, 0, -1):
        for value in buckets[count]:
            result.append(value)
            if len(result) == k:      # must be inside: one bucket can hold many
                return result
    return result

def top_k_frequent_heap(numbers: list[int], k: int) → list[int]:
    """The same answer in O(m log k), where m is the number of distinct values."""
    counts = Counter(numbers)
    return heapq.nlargest(k, counts, key=counts.get)

def count_manually(items: list[str]) → dict[str, int]:
    """The three-line version, for when the interviewer says 'no imports'."""
    counts: dict[str, int] = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts

def is_anagram(first: str, second: str) → bool:
    """O(n) by counting. The sorting answer is O(n log n)."""
    return Counter(first) == Counter(second)

def first_unique(text: str) → str | None:
    """The first non-repeating character. Walk the original for the order."""
    counts = Counter(text)
    for character in text:
        if counts[character] == 1:
            return character
    return None

if __name__ == "__main__":
    print(top_k_frequent([1, 1, 1, 2, 2, 3], 2))      # [1, 2]
    print(top_k_frequent([7], 1))                  # [7]
    print(top_k_frequent([4, 5, 6], 3))             # [4, 5, 6] in some order
    print(top_k_frequent_heap([1, 1, 1, 2, 2, 3], 2)) # [1, 2]
    print(count_manually(["curd", "lemon", "curd"])) # {'curd': 2, 'lemon': 1}
    print(is_anagram("listen", "silent"))           # True
    print(first_unique("swiss"))                     # 'w'
```

6 What it costs

Building the map

The loop runs once per element: n iterations. Each does one hash, one lookup and one store — all $O(1)$ on average. So building the frequency map is **$O(n)$ time**.

Space is **$O(m)$ extra**, where m is the number of *distinct* values. That distinction matters and interviewers push on it. Counting a million log lines that only ever contain 12 different status codes costs 12 entries, not a million.

What Shobha's old method cost

Four dishes, forty-one messages: $4 \times 41 = 164$ reads. In general $m \times n$. With $m = 200$ distinct words and $n = 100,000$ words that is $20,000,000$ reads against $100,000$ for the one-pass version — **200 times more work**, and the ratio grows with the number of distinct values.

You will meet this written as `list.count()` inside a loop, which is the same mistake:

```
for word in words:
    if words.count(word) == 1:      # this scans the whole list, every time
        return word
```

`words.count(word)` is $O(n)$ and it is inside an $O(n)$ loop, so that is $O(n^2)$. It looks like two lines of clean code. It is fifty million operations at $n = 10,000$.

The three top-k answers, counted

Let n be the number of elements and m the number of distinct values, so $m \leq n$.

counting	$O(n)$	every approach pays this
A: sort	$O(m \log m)$	then slice k
B: heap	$O(m \log k)$	keep only k in the heap
C: buckets	$O(n + m)$	$n+1$ buckets, walk them once

Put numbers on it. With $n = 1,000,000$, $m = 100,000$ and $k = 10$:

A: $100,000 \times \log_2(100,000)$	$= 100,000 \times 17 = 1,700,000$	comparisons
B: $100,000 \times \log_2(10)$	$= 100,000 \times 3.3 = 330,000$	comparisons
C: $1,000,000 + 100,000$	$=$	$1,100,000$ simple steps, no comparisons

B does the least comparison work. C does no comparisons at all but allocates a list of a million and one entries, which is why it is not always the winner in practice: when n is enormous and m is tiny, C's bucket list is mostly empty and B is better. **Say that out loud.** "Bucket sort is $O(n)$ but it allocates $n + 1$ buckets, so if n is a billion and there are only 12 distinct values I would use the heap" is a much stronger answer than reciting that one is linear.

Space for each

A: $O(m)$	the sorted list of distinct values
B: $O(m + k)$	the counts, plus the heap
C: $O(n + m)$	the bucket list is the expensive part

7 The traps

Trap 1 — the missing key

```
>>> counts = {}
>>> counts["pulao"] += 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'pulao'
```

The three fixes, in order of how often you should use them: `counts.get(k, 0) + 1`, `defaultdict(int)`, `Counter`. What you should not do is wrap it in a `try / except KeyError` — it works, and it reads as though you did not know the alternatives.

Trap 2 — `defaultdict` creates keys when you only meant to look

This one is genuinely surprising, and it produces wrong answers rather than errors.

```
>>> from collections import defaultdict
>>> counts = defaultdict(int)
>>> counts["x"] += 1
>>> "zzz" in counts          # asking with `in` is safe
False
>>> counts["zzz"]           # but READING it creates it
0
>>> dict(counts)
{'x': 1, 'zzz': 0}
>>> len(counts)
2
```

Reading a missing key from a `defaultdict` **inserts** it. So `len(counts)` is now wrong, iterating gives you a key that never appeared in the data, and a later `if counts["zzz"]` has silently changed your dictionary. If you are going to read keys that might be absent, use `counts.get("zzz", 0)` or a plain `dict`, and check membership with `in`, never by indexing.

Trap 3 — `Counter` returns 0 for anything, including your typos

```
>>> c = Counter("aabbbc")
>>> c["b"]
3
>>> c["bb"]      # typo
0
>>> c["z"]
0
```

No error, ever. This is helpful right up to the moment it hides a bug. When the count of something that definitely exists comes back as 0, suspect the key before you suspect the logic.

Trap 4 — walking the counter instead of the original

```
def first_unique_wrong(text: str) → str | None:
    counts = Counter(text)
    for character in counts:
        if counts[character] == 1:
            return character
    return None
```

On most inputs this returns the right answer, because a `Counter` iterates in first-seen order. It breaks on "aabbcc": the distinct order is a, b, c, and the answer is c either way. Try "loveleetcode" and it still works. It fails only when the first *distinct* character with count 1 is not the first *positional* character with count 1 — and since `Counter` preserves first-seen order, that is actually never for this specific problem. **That is exactly why it is dangerous.** It is right by accident, it stops being right the moment somebody sorts the counter or switches to a plain dict built in a different order, and you cannot explain why it works. Walk the original.

Trap 5 — `list.count()` inside a loop

```
for word in words:
    if words.count(word) == 1:
        return word
```

Two clean-looking lines that are $O(n^2)$. `words.count(word)` walks the entire list every time. With 20,000 words that is 400,000,000 comparisons and takes several seconds; the `Counter` version is 0.003 seconds. Same output, same readability, different complexity — this is the counting version of the `seen = []` trap from [day 062](#).

Trap 6 — changing the dictionary while iterating it

```
>>> d = {"a": 1, "b": 2}
>>> for key in d:
...     d[key + "!"] = 1
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RuntimeError: dictionary changed size during iteration
```

Iterate over `list(d)` or `list(d.items())` if you must change the dictionary inside the loop. Changing a *value* for an existing key is fine; adding or deleting keys is not.

Trap 7 — `Counter` arithmetic drops what you might have wanted

```
>>> Counter("aabbcc") - Counter("abz")
Counter({'a': 1, 'b': 1, 'c': 1})
```

The `z` is gone, not `-1`. Subtraction keeps only positive counts. If you need the negatives, use `c.subtract(other)`, which mutates in place and does keep them. And `sum(Counter(x) for x in ...)` raises, because `sum` starts at the integer 0:

```
TypeError: unsupported operand type(s) for +: 'int' and 'Counter'
```

The fix is `sum(counters, Counter())` or, better, one `Counter` you `update()` in a loop.

8 In the interview

How it gets asked

- “Given an integer array and an integer *k*, return the *k* most frequent elements. Your algorithm’s time complexity must be better than $O(n \log n)$.” The complexity constraint is the whole question — it rules out sorting the counts and pushes you to the heap or the buckets.
- “Find the first non-repeating character in a string.” Sounds easier. The trap is the second pass.
- “Are these two strings anagrams?” The warm-up, and the follow-up is always “what if they are Unicode” or “what if the strings are a gigabyte”.
- The vague version: “I have a log file with a hundred million lines. Which endpoint is slowest most often?” No complexity stated, and now you have to notice that the distinct count is small and the total count is huge, which changes which top-k answer wins.

What to say out loud, in the first ninety seconds

1. **Separate the two halves.** “There are two parts here: counting, and then selecting the top *k*. Counting is $O(n)$ with a hash map and there is no way around it — I have to look at every element at least once. So the interesting question is the selection.”
2. **Name the size that actually matters.** “Let me call *n* the number of elements and *m* the number of distinct values. *m* is at most *n* and is usually much smaller. The selection cost is in terms of *m*, not *n*.”
3. **Offer all three, with their costs.** “Sorting the counts is $O(m \log m)$. A heap of size *k* is $O(m \log k)$. Bucketing by count is $O(n + m)$ with no comparisons at all.”
4. **Pick one and say why.** “I will write the bucket version because you asked for better than $n \log n$, and it is the only one with no logarithm. If *m* were tiny and *n* enormous I would prefer the heap, because the bucket list is $n + 1$ long regardless.”

5. Ask the one clarifying question. “If there are ties at the k -th position, is any valid answer acceptable?” It almost always is, and asking shows you noticed.

The follow-ups

“Why is bucket counting $O(n)$ when sorting is $O(n \log n)$? Have you not beaten the sorting bound?” “No, because I am not sorting by comparison. The bound of $n \log n$ applies to comparison sorts. Here I know something extra — a count is an integer between 1 and n — so I can index by it directly instead of comparing. That is the same reason counting sort is linear (day 056). The price is the memory for the buckets.”

“What if the input does not fit in memory?” “Then the map does not fit either, and I would count in chunks and merge. Each machine counts its shard and produces a partial `Counter`; then I merge the partials by adding them. Counting is associative, which is exactly why this is the canonical MapReduce example. For approximate answers on a stream, Count-Min Sketch gives frequencies in fixed memory with a bounded overestimate.”

“Can you do it in $O(1)$ extra space?” “Not for top k in general — I have to remember the counts somewhere, and there can be n distinct values. For the specific case of the *majority* element, yes: Boyer-Moore voting keeps one candidate and one counter, so it is $O(1)$ space and one pass. It only works because the majority is defined as strictly more than half.”

“You used `Counter`. Write it without imports.” Write the three-line `.get(k, 0) + 1` loop immediately. Have this ready — it is asked often enough that fumbling it undoes the impression the rest of the answer made.

A model answer

Asked: *given an array of integers and a number k , return the k most frequent elements, faster than $O(n \log n)$.*

“This is two problems. First count, then select.

Counting has to be $O(n)$ because I must see every element, and a hash map makes each element cost constant time. I will call the number of distinct values m — that is at most n , and usually far less.

For the selection there are three options. I can sort the m counts, which is $O(m \log m)$. I can push them through a heap of size k , which is $O(m \log k)$ — better when k is small. Or I can bucket them: because a count is an integer between 1 and n , I can make n plus one buckets and drop each value into the bucket for its count, then read buckets from the highest down until I have k . That is $O(n + m)$ and involves no comparisons at all.

You have asked for better than $n \log n$, so I will write the bucket version. The reason it beats the comparison bound is that I am not comparing — I know the counts are small integers, so I can index by them, which is the same trick as counting sort.

The cost I am paying is memory: the bucket list is n plus one entries whether or not I need them. If I knew n was a billion and there were only twelve distinct values, I would use the heap instead, because allocating a billion buckets to hold twelve values would be absurd.

One detail in the code: the check for whether I have collected k elements has to be inside the inner loop, not after it, because one bucket can hold more values than I need. If every value appears exactly once, bucket one holds all of them.

Edge cases: k equal to the number of distinct values returns everything. Ties at the k -th position — I will return any valid set unless you want a specific tie-break.”

9 Recall card

- **Three lines, and know all four spellings:** `counts[x] = counts.get(x, 0) + 1` · the explicit `if` · `defaultdict(int)` · `Counter(items)`. Building it is $O(n)$ time, $O(m)$ extra space, where m is the number of *distinct* values — not n .
- `counts[x] += 1` on a missing key raises `KeyError`. And `defaultdict` creates the key when you merely read it, so `len()` silently grows; `Counter` returns `0` for anything, so typos never raise. Check membership with `in`, never by indexing.
- **Top k has three answers:** sort $O(m \log m)$ · heap $O(m \log k)$ · bucket by count $O(n + m)$. At $n = 10^6$, $m = 10^5$, $k = 10$ that is 1.7M vs 330K vs 1.1M. **Buckets beat the $n \log n$ bound because they do not compare** — a count is an integer in $1..n$, so you index by it.
- **Say the trade, not just the winner:** buckets allocate $n + 1$ slots regardless, so with n huge and m tiny the heap wins. And the `len(result) == k` check goes *inside* the inner loop.
- **Two $O(n^2)$ traps that look clean:** `words.count(word)` inside a loop (400M ops at $n = 20,000$), and walking the counter instead of the original string for a “first such” question. Real errors: `KeyError`, `RuntimeError: dictionary changed size during iteration`, and

```
TypeError: unsupported operand type(s) for +: 'int' and 'Counter' from sum().
```

1 What this is, and why they ask it

A **design pattern** is a name given to an arrangement of classes and objects that solves a problem which keeps coming back. It is not a library you install and not a rule you follow; it is a description of a solution that many people arrived at independently, written down so that the next person does not have to arrive at it again. Every pattern has four parts — a **name**, the **problem** it solves, the **solution** in outline, and the **consequences** of choosing it.

They ask it because it is a fast test of experience, and it is very hard to fake. Someone who has read a list of patterns says “I have used singleton, factory and observer” and then cannot say what the singleton cost them. Someone who has actually shipped code says “we used strategy for the pricing rules because there were four countries and a fifth was coming, and it meant the fifth was a new file rather than a fifth branch in a function twelve people depended on” — and then, unprompted, says what they got wrong. The question is not really *which patterns*. It is *do you choose them for reasons*.

This day opens a two-week run through the patterns themselves. Everything after it — singleton, factory, builder, adapter, decorator, facade, strategy, observer — assumes you can say what a pattern is for, and, more importantly, when it is not.

2 The story

The hall behind the temple in Yelahanka is one long room with a stage at one end, and Ramesh has looked after it for thirty-four years. It has a hundred and eighty folding chairs, fourteen long tables, two ceiling fans that work and one that does not, and a store room at the back where all of it lives.

People book it for everything. Weddings, birthdays, funerals, association meetings, school exams, music classes, a man who once ran a course on filing tax returns.

When Ramesh started, every booking was a conversation. Where do you want the chairs, how many rows, tables at the side or the back. Forty minutes on the phone, and half the time the person did not know what they wanted, and he would end up moving everything again an hour before the event.

Somewhere around his fifth year he stopped asking that question and started asking a different one: what kind of function is it?

Because there are only four ways he ever ends up arranging the room, and they are not his ideas — they are just what works. For a function where people watch something, every chair faces the stage in rows with a walkway up the middle. For a meeting, the tables go in a square

in the middle and everybody can see everybody’s face. For an exam, single chairs spread out with a full arm’s length between them and nothing at the front. And for anything with food, chairs round the edges and the tables in a line along the back wall, so the queue has somewhere to go and does not cut the room in half.

He gave them names, dull on purpose. Stage side. Square. Spread out. Food line.

Now the phone call is ninety seconds. And when the two boys who help him arrive at six in the morning, he says one of four words and goes to make tea, and it is right when he comes back.

He is also firm about one thing. A young man from the association once set the room up in the square for a meeting of fifteen people, because it looked more important, and by the end everyone was shouting across a table the size of a badminton court. Ramesh’s view is that a name for something is useful right up until people start choosing it because they like the name.

3 The idea in plain English

Ramesh did not invent four layouts. He noticed that the same four situations kept arriving, and that each one had an arrangement that worked. Then he gave each arrangement a name so that a forty-minute conversation became one word.

That is a design pattern, exactly. **Patterns are discovered, not invented.** Somebody notices that the same shape of problem keeps appearing, that people keep solving it the same way, and writes it down.

The four parts of a pattern

Every properly written pattern has all four. When someone can only give you two of them, they have read about it rather than used it.

PART	RAMESH’S VERSION	IN CODE
Name	“Square”	Adapter, Strategy, Observer
Problem	A meeting where everyone must see everyone	Two interfaces that must work together but cannot be edited
Solution	Tables in a square, chairs outside them	A class that implements the interface you want and forwards to the one you have
Consequences	Only works up to about thirty people; wastes the middle of the room	One more class, one more hop; the two sides stay independent

The consequences are the part everybody skips, and they are the part that matters. The original book gives each pattern roughly seventeen pages, and most of that length is consequences. A pattern without its costs is a slogan.

Where the idea comes from

The word was borrowed from building architecture. Christopher Alexander wrote *A Pattern Language* in 1977, cataloguing 253 arrangements of physical space that keep working — how wide a pavement should be, where to put a window seat, why a room with light on two sides feels better than one with light on one. His point was that these are not somebody's taste; they are solutions people converge on.

In 1994 four authors — Gamma, Helm, Johnson and Vlissides, together always called the **Gang of Four** — did the same thing for object-oriented programming in a book called *Design Patterns*. They catalogued **23 patterns** in three groups. Those 23 are what people mean when they say “design patterns” without qualification, and every interview question about patterns is drawn from them.

The three groups

Creational — five patterns, all about *how objects get made*. The problem they solve is that `new Thing()` scattered through your code ties every caller to a concrete class.

Singleton · Factory Method · Abstract Factory · Builder · Prototype.

Structural — seven patterns, all about *how objects are composed*. How you put objects together so the whole is usable without every part being exposed.

Adapter · Decorator · Facade · Proxy · Composite · Bridge · Flyweight.

Behavioural — eleven patterns, all about *how objects talk to each other*. Who calls whom, in what order, and how that can change at run time.

Strategy · Observer · Command · Iterator · Template Method · State · Chain of Responsibility · Mediator · Memento · Visitor · Interpreter.

You will meet the eight that actually come up over the next two weeks. The other fifteen are worth recognising by name and nothing more.

What a pattern is not

This is the half that keeps people out of trouble.

A pattern is not a library. You cannot install adapter. It is a shape you write yourself.

A pattern is not a rule. SOLID gives you principles you should generally follow. Patterns are options you should sometimes take. “This code does not use any patterns” is not a criticism.

A pattern is not architecture. Patterns are class-level. Microservices, event sourcing and CQRS are architectural patterns, which is a different book and a different scale.

A pattern name is not a class name. A class called `PricingStrategy` is fine. A class called `OrderFactoryFactoryBean` is a codebase telling you it stopped thinking. Name classes after what they do in your domain, not after the shape they happen to be.

And a pattern is not a goal. The young man's square table was a real layout, correctly executed, for the wrong function.

The most important thing about patterns in Python

Many of the 23 exist because C++ and Java of 1994 could not do certain things. In a language with first-class functions, several patterns collapse to nothing.

PATTERN	WHAT IT NEEDS IN JAVA	IN PYTHON
Strategy	An interface + one class per algorithm	A function passed as an argument
Command	An interface + a class per command	A closure, or <code>functools.partial</code>
Iterator	An interface + a class holding position	<code>yield</code>
Template Method	An abstract class + subclasses	A function taking a function
Singleton	A private constructor + a static holder	A module — it is imported once

Peter Norvig made this point in 1996: of the 23 patterns, sixteen are either invisible or substantially simpler in a dynamic language. This is worth saying in an interview, carefully. It is not “patterns are useless in Python”. It is “the *problem* each pattern solves is still real; in Python the *solution* is often three lines rather than four files, and I would rather write the three lines.”

4 The picture

The three groups, and the one question each of them answers.



FIGURE 63.1

What to notice: the three groups are not three kinds of code, they are three kinds of *question*. Before naming a pattern, name which of the three questions you are answering. Half of all pattern misuse is answering a creational question with a structural pattern.

And here is the shape a pattern’s write-up takes, which is also the shape your interview answer should take:

NAME		Strategy
PROBLEM		The pricing rule differs per country and a new country arrives every quarter
SOLUTION		An interface with one method; one small class per country; the caller holds a reference
CONSEQUENCES		+ a new country is a new file, not an edit + each rule is testable alone - one more indirection to follow when reading - somebody must decide which one to use

Notice the minus signs. If your answer has no minus signs, you have described a slogan.

5 How it actually works

How a pattern is actually applied

It is not “choose a pattern and apply it”. It runs the other way, and this is the mechanic worth learning.

1. **You feel a specific pain.** Not “this feels unstructured” — a real one. Adding a country means editing a function twelve people depend on. Testing this needs a live payment gateway. Every caller has to know six classes to do one thing.
2. **You name the axis that is moving.** Which thing keeps changing? Countries. Payment providers. Report formats. Notification channels.
3. **Now the pattern picks itself.** A varying algorithm is Strategy. A varying construction is Factory. A varying wrapper you want to stack is Decorator. An incompatible third party is Adapter. A messy subsystem the caller should not see is Facade.
4. **You write down what it costs before you write the code.** One more file, one more hop, one more place to look.

Step 1 is the one people skip, and skipping it is what produces the codebases everyone complains about.

Where you have already met them, in real products

Patterns are not academic. You have used all of these.

- **Decorator** — `java.io.BufferedReader(new FileReader(f))` is the textbook example: a reader wrapping a reader wrapping a reader, each adding one thing. In Python, `functools.lru_cache` wraps a function and adds caching without the function knowing.
- **Strategy** — `sorted(items, key=...)`. The comparison rule is a parameter. Python’s `sorted` has no idea how you want things ordered, and that is the whole design ([day 058](#)).
- **Iterator** — every `for` loop in Python. `yield` builds one in four characters.
- **Adapter** — every database driver behind Python’s DB-API. `psycopg`, `mysqlclient` and `sqlite3` present the same `connect` / `cursor` / `execute` surface over three entirely different wire protocols.
- **Facade** — `requests.get(url)`. Behind that one line are connection pooling, TLS negotiation, redirect handling, cookie storage, retries and content decoding, in about six subsystems you never touch.
- **Observer** — every event system. Kafka consumers, DOM `addEventListener`, Django signals, Kubernetes controllers watching the API server.
- **Factory** — Spring’s `BeanFactory`, Django’s `DEFAULT_FILE_STORAGE` setting, `logging.getLogger`.

- **Proxy** — nginx sitting in front of your application; an ORM's lazy-loading object that looks like your model until you touch a field and it fires a query ([day 041](#)).
- **Singleton** — Python's `logging` module, and the module system itself.

Being able to name two of these in an interview, unprompted, is worth more than reciting the categories.

What the write-up looks like, in code

Here is the same problem twice — a pricing rule that differs by country — done the two ways, so you can see what the pattern buys and what it costs.

Without:

```
def price(order: Order, country: str) → Decimal:
    if country == "IN":
        return order.subtotal * Decimal("1.18")
    elif country == "AE":
        return order.subtotal * Decimal("1.05")
    elif country == "GB":
        return order.subtotal * Decimal("1.20")
    raise ValueError(country)
```

With Strategy, Python-style — no interface, no class hierarchy, just a mapping to functions:

```
PRICERS: dict[str, Callable[[Order], Decimal]] = {
    "IN": lambda o: o.subtotal * Decimal("1.18"),
    "AE": lambda o: o.subtotal * Decimal("1.05"),
    "GB": lambda o: o.subtotal * Decimal("1.20"),
}

def price(order: Order, country: str) → Decimal:
    return PRICERS[country](order)
```

That is Strategy. There is no class called `Strategy` anywhere, and there does not need to be. The pattern is the *shape* — the varying part is a value that can be swapped — not the ceremony.

When would you take the ceremony? When a rule needs state, or three methods rather than one, or its own tests and its own file because finance owns it. Then each becomes a class. The pattern is the same; the weight is a separate decision.

6 The numbers

The catalogue, by the numbers

```
23 patterns in the Gang of Four book
 5 creational + 7 structural + 11 behavioural
395 pages → about 17 pages per pattern
```

Seventeen pages for something whose solution fits in half a page. The rest is problem, applicability, consequences and known uses — which tells you where the value is.

Of the 23, roughly **eight** account for almost everything you will be asked about or will write: Singleton, Factory, Builder, Adapter, Decorator, Facade, Strategy, Observer. Learning eight properly beats recognising twenty-three.

What the ceremony costs

The same three-country pricing rule, counted:

```
if/elif chain          1 file,  9 lines
dict of functions (Python) 1 file, 12 lines
full Strategy classes   5 files, ~90 lines
    (1 interface + 3 implementations + 1 registry)
```

So the classes cost about **80 extra lines and 4 extra files**. That is not a reason never to do it — it is the price, and you should know it before you quote it. It buys you: a fourth country as one new file with zero edits to existing code, and each rule testable in isolation.

The break-even is easy to state. If you will add a fourth, fifth and sixth country, 80 lines spread over six implementations is nothing. If three is the whole world, 80 lines is pure cost, forever.

The vocabulary saving, which is the real one

This is the benefit people forget to quote, and it is measurable.

```
explaining the arrangement from scratch, in a review:  ~4 minutes
saying "this is an adapter":                          ~5 seconds
design discussions in a 6-person team, per quarter:    ~20
arrangements discussed per discussion:                ~3
```

$20 \times 3 \times 4 \text{ minutes} = 240 \text{ minutes}$ of explanation per quarter, against about five minutes if everyone knows the words. **Four hours a quarter, per team, purely in vocabulary**. And that undercounts, because the expensive failure is not the time — it is the two people who thought they had agreed and had not.

What over-application costs

The number from [day 056](#) applies directly: every axis you open speculatively costs roughly **22 lines and 3 files, forever**, plus one hop of indirection for every future reader.

A real shape you will meet: a codebase with 14 classes ending in `Factory`, of which 9 have exactly one implementation and no test double. Those 9 are `new Thing()` wearing a costume — about $9 \times 22 = 200$ lines and 27 files that could be deleted, and every one of them is a redirection a new engineer has to follow before finding out that nothing happens there.

The one that justifies the whole idea

Adding the tenth payment provider:

```

with an if/elif over provider:
  files edited          1 shared file, now ~400 lines
  existing tests re-run ~60
  live providers at risk 9
with adapter + factory:
  files added           1
  lines edited          1
  existing tests re-run 0

```

And, crucially, the tenth costs the same as the second. Flat cost is the actual product being sold here.

7 The trade-offs

What you give up, always

Indirection. Every pattern replaces one thing you can read with two or more things you must jump between. Following one request through a facade, a strategy and a decorator means four files open. A person joining the team pays that cost on their first day and every day after.

A decision moves rather than disappearing. Choosing a country's pricer at run time still requires somebody to choose. The `if` did not vanish; it became a dictionary lookup, and the lookup can miss. Say this out loud — it is the honest version of the open/closed argument.

Names that outlive their reasons. `PricingStrategy` still says “strategy” three years after somebody deleted two of the three implementations. Now there is an interface with one implementation, which is worse than no interface at all, because it looks deliberate.

The failure modes, named

Cargo cult. Applying a pattern because it is a pattern. The young man's square table. The tell is a class named after the pattern rather than the domain.

Pattern hunting. Reading code and looking for which pattern it is. Real code is mostly not any of the 23, and that is fine.

Patterns as a substitute for deleting code. A god class with a factory in front of it is still a god class.

Premature patterning. The single most common. You cannot see the axis of change until it has changed at least once. Building the seam before then means you almost always build it in the wrong place, and a seam in the wrong place is more expensive than no seam.

“I would not use a pattern if...”

- **I cannot name the second implementation.** The test double counts; a hypothetical does not.
- **The variation is data, not behaviour.** Three tax rates are a dictionary, not three classes. This is the mistake that produces the 14 factories.

- **The set is genuinely closed.** Four blood groups, seven weekdays, three order states.
- **The language does it in three lines.** A function beats a Strategy class hierarchy in Python unless the strategy needs state or several methods.
- **It is the first occurrence.** Write it inline. Let the second occurrence show you where the seam is.

The strongest thing you can say in this round

“The pattern I have regretted most is the one I applied before I had two implementations.” Then name one. Interviewers remember the candidate who volunteered a mistake with a reason, because everyone else recites benefits.

8 In the interview

How it gets asked

- The direct one: *“Which design patterns have you used, and why?”* Almost always open-ended, and almost always used as a way in to two follow-ups.
- The definitional one: *“What is a design pattern?”* Sounds like a gift. Most people answer with a list of names, which answers a different question.
- The sceptical one: *“Are design patterns still relevant?”* or *“Do you need patterns in Python?”* They are testing whether you have an opinion with reasons behind it, not whether you agree.
- The applied one: *“Here is a problem. Which pattern would you use?”* — which is really the reverse: can you go from a symptom to a shape, rather than from a name to a use.

What to say out loud, in the first ninety seconds

1. **Define it in one sentence, with all four parts.** “A pattern is a named solution to a problem that keeps recurring, together with its consequences — and the consequences are the part that makes it useful rather than a slogan.”
2. **Say what they are for, which is vocabulary as much as code.** “Half the value is that I can say ‘that is an adapter’ in a review and five people know exactly what I mean, instead of four minutes of description.”
3. **Give one you have used, with the specific pain that caused it.** Not “I have used strategy” but “the pricing rule was different per country and a new country arrived each quarter.”
4. **Give the consequence you accepted.** “The cost was that the rule for a country is no longer visible where the price is computed — you have to go and find the class.”
5. **Give one you regret, or one you deliberately did not use.** This is the sentence that separates you.

The follow-ups

“Which pattern do you see misused most?” “Singleton, easily — it is a global variable with better manners, and it makes tests share state. After that, factories with one implementation. I have worked in a codebase with fourteen classes ending in `Factory` where nine had exactly one implementation and no test double. Those are `new Thing()` with three extra files in front of it.”

“Do patterns still matter in a language with first-class functions?” “The problems do. Several of the solutions get much smaller. Strategy in Python is a function you pass; command is a closure; iterator is `yield`; singleton is usually just a module. Norvig’s point in 1996 was that about sixteen of the twenty-three are invisible or trivial in a dynamic language, and I think that is right. What survives is the vocabulary and the consequences — I still say ‘that is a decorator’ about `lru_cache`, even though there is no `Decorator` class anywhere.”

“How do you decide whether to introduce one?” “I do not start from the pattern. I start from a pain: adding a country means editing a shared file; testing this needs a live gateway. Then I name the axis that keeps moving. Once I have named the axis, the pattern is usually obvious — a varying algorithm is strategy, a varying construction is a factory, an incompatible third party is an adapter. And I wait for the second occurrence, because you cannot see where the seam goes until something has actually changed.”

“Give me a pattern you removed.” Have one ready. “We had an abstract factory producing one concrete factory. I deleted the interface and the abstract class and called the constructor. Three files became one, and nothing else in the codebase changed, which was the proof that the abstraction had never been doing anything.”

A model answer

Asked: *which design patterns have you used, and why?*

“Let me start with what I think a pattern is, because it changes the answer. A pattern is a named solution to a problem that keeps recurring, along with its consequences. The name matters as much as the code — if I say ‘that is an adapter’ in a review, five people know what I mean in five seconds instead of four minutes.

The three I have actually used and can defend are strategy, adapter and facade.

Strategy: our pricing rule differed by country, and a country was being added roughly every quarter. With an `if`-chain, each new country meant editing a shared file that twelve people depended on and re-running about forty tests. We made the rule a value — in Python a function in a dictionary, not a class hierarchy — and a new country became one new entry. The consequence I accepted is that you can no longer see what a country charges by reading the price function; you have to go and find it.

Adapter: we moved from one payment provider to another. The domain code had thirty-eight references to the vendor’s SDK across fourteen files. We put an interface in our own package, in our own vocabulary — our `Money`, our `PaymentDeclined` — and wrote one adapter per provider. The second provider was then one new file and one wiring line.

Facade: our checkout touched six services, and every caller had to know all six and the order to call them in. One entry point with one method fixed that. The risk with a facade is that it quietly becomes a god object, so we kept it to sequencing and put no business rules in it.

The one I would push back on is singleton. It is a global variable with better manners. Every time I have used one, the pain has come out in tests, where two tests share state through it and the suite fails depending on the order it runs in.

And the honest answer to ‘why’ is that I try not to reach for a pattern until something has changed at least once. Before that I cannot see where the seam goes, and a seam in the wrong place costs more than no seam at all.”

9 Recall card

- **A pattern is a *name* + a recurring *problem* + a *solution* + its *consequences*.** The consequences are the half people skip — the original book spends ~17 pages per pattern, and most of it is costs. Patterns are **discovered, not invented**: Alexander’s buildings in 1977, the Gang of Four’s **23 patterns** in 1994.
- **Three groups, three questions.** Creational (5) = *how does this get made?* · Structural (7) = *how are these put together?* · Behavioural (11) = *how do these talk?* Eight of the 23 are almost all you will meet: singleton, factory, builder, adapter, decorator, facade, strategy, observer.
- **Go from pain to axis to pattern, never from pattern to code.** Name the specific pain, name the thing that keeps changing, and the pattern picks itself. Wait for the **second occurrence** — you cannot see the seam until something has moved.

- **In Python the shape survives and the ceremony often does not.** Strategy = a function · command = a closure · iterator = `yield` · singleton = a module. Norvig, 1996: ~16 of 23 are invisible or trivial in a dynamic language. `sorted(key=)`, `lru_cache`, `requests.get`, the DB-API and nginx are all patterns you already use.
- **Quote both numbers.** Full Strategy classes cost ~80 extra lines and 4 extra files over an `if` chain, and buy a flat cost per new variant (the tenth costs what the second cost). Shared vocabulary saves a 6-person team ~4 hours a quarter. And 9 factories with one implementation each are ~200 lines of costume — **a pattern name is not a class name.**

PRACTICE Practice — Counting with dictionaries

DSA topic: Counting with dictionaries **System design topic:** What a design pattern actually is

Code these, in this order

One rule for the whole set: **write the counting loop by hand at least once today, without Counter**. Interviewers ask for it, and the three-line `.get(k, 0) + 1` version should come out of your fingers without thought.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Valid Anagram	LeetCode 242 (Easy)	Counting versus sorting, and whether you can say which wins on time and which on space.
2	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and whether you walk the original or the counter on the second one.
3	Sort Characters By Frequency	LeetCode 451 (Medium)	Counts as sort keys, and the tie-break question nobody asks.
4	Top K Frequent Elements	LeetCode 347 (Medium)	Whether you know all three selection strategies and can pick one with a reason.

On problem 1, give both answers

Write the `Counter` version, then say the sorting version out loud with its complexity. Then answer the follow-up you will actually get: *what if the strings contain Unicode?* A fixed 26-slot array stops working; the dictionary does not. That is the real reason to prefer the map.

On problem 3, find the ambiguity before you code

`"tree"` can correctly return `"eert"` or `"eetr"`. Say out loud which one your code gives and why both are accepted. Then say what you would ask the interviewer if the problem had not told you.

On problem 4, write it twice

Once with `heapq.nlargest`, once with buckets. Time both on a million elements with a hundred thousand distinct values and `k = 10`. Then answer: which one would you write if `n` were a billion and there were twelve distinct values, and why.

The four-spellings drill

Count the characters of `"mississippi"` five times, each a different way, and say when you would use each:

1. An explicit `if key in counts: ... else: ...`
2. `counts.get(key, 0) + 1`

3. `defaultdict(int)`
4. `Counter(text)`
5. `try: counts[key] += 1 / except KeyError:`

Then say which of the five you would write in an interview and which one would count against you.

The missing-key drill

Trigger each of these, read the exact output, and give the fix:

1. `counts = {}; counts["a"] += 1`
2. `counts = {}; print(counts["a"])`
3. A `defaultdict(int)`, then `print(d["zzz"])`, then `print(len(d))` — explain the number.
4. A `Counter`, then `print(c["typo"])` — explain why nothing raises and why that is dangerous.
5. `for k in d: d[k + "!"] = 1`
6. `sum(Counter(w) for w in ["ab", "bc"])`

Two of those six produce a wrong answer with no error at all. Name them, and say what would have caught each one.

The complexity drill

For each snippet, count the operations at `n = 20,000` and name the complexity:

1. `Counter(items)`
2. `for x in items: if items.count(x) == 1: return x`
3. `for x in items: if x in seen_list: ... ; seen_list.append(x)`
4. `sorted(counts.items(), key=lambda kv: -kv[1])[:k]`
5. `heapq.nlargest(k, counts, key=counts.get)`
6. The bucket version

Then time 1 against 2 and quote the ratio. Numbers 2 and 3 are the same mistake in two costumes — say what the mistake is in one sentence.

The derived-key drill

Build a `Counter` whose key is not the element itself. For a list of words, produce:

1. How many words of each length.
2. How many words start with each letter.
3. How many words share each set of letters, ignoring order.
4. How many words have each number of vowels.
5. How many words are the same when reversed.

6. How many words fall into each of {short, medium, long} by a rule you choose.

Number 3 is tomorrow's lesson. Say what the key is, precisely, and why a list cannot be used as one.

The top-k drill

For each situation, say which of the three selection strategies you would use and give the deciding number:

1. $n = 1,000$, $m = 900$, $k = 5$
2. $n = 1,000,000$, $m = 12$, $k = 3$
3. $n = 1,000,000$, $m = 800,000$, $k = 100,000$
4. $n = 50$, $m = 50$, $k = 50$
5. A stream you can only read once and cannot store
6. Ten machines each holding a tenth of the data

Situations 5 and 6 are not on the list of three. Say what each one actually needs, and name the idea.

The pattern-definition drill

Answer each in one sentence, out loud:

1. What are the four parts of a pattern?
2. Which part do people skip, and why does it matter?
3. Name the three groups and the question each one answers.
4. Name two patterns from each group.
5. Give one thing a pattern is *not*.
6. Why is a class called OrderFactoryFactory a warning sign?

The pain-to-pattern drill

For each symptom, name the axis that is moving and then the pattern. Then say what it costs.

1. "Adding a country means editing a function twelve people depend on."
2. "Testing this needs a live payment gateway."
3. "Every caller has to call six services in the right order."
4. "The third-party SDK's method names are nothing like ours and we cannot change either side."
5. "We want logging and caching on this service without touching it."
6. "This object has fifteen optional fields and the constructor has fifteen parameters."
7. "When an order ships, four unrelated things have to happen."
8. "We need exactly one connection pool for the whole process."

Then go back through all eight and mark the ones where you would do nothing yet, and say what would change your mind.

The over-application drill

For each, say whether the pattern is earning its keep, and give the test you applied:

1. A `Strategy` interface with one implementation and no test double.
2. A `Factory` producing one concrete class.
3. Three tax rates as three classes implementing `TaxRule`.
4. A `Facade` over six services, with no business rules in it.
5. A `Singleton` holding the database connection, in a codebase with 400 tests.
6. An `Adapter` around a vendor SDK used in one place.

Two of the six are defensible for a reason that has nothing to do with the number of implementations. Name them and say the reason.

The regret drill

Answer this out loud, in ninety seconds, as you would in an interview:

Tell me about a pattern you applied and later regretted, or removed.

If you have not shipped one yet, use the lesson's example and make it specific: which pattern, what pain you thought you were solving, what actually happened, how many files you deleted, and what you would look for before doing it again.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the top k most frequent elements.* The two halves separated, n and m named, all three selection strategies with their costs, one chosen with a reason, and the memory trade volunteered before it is asked for.
 2. *Which design patterns have you used, and why?* The one-sentence definition with all four parts, the vocabulary argument, two or three you have used with the specific pain that caused each, the consequence you accepted, and one you regret.
 3. *Why does bucket counting beat the $O(n \log n)$ sorting bound?* Because it does not compare — the counts are integers between 1 and n , so you index by them. Then the price: `$n + 1$` buckets allocated regardless, and when the heap wins instead.
-

Before you move on

- ☐ I wrote the counting loop by hand, without `Counter`, from memory.
- ☐ I triggered `KeyError` on `counts[k] += 1` and fixed it three different ways.

- [] I proved that reading a missing key from a `defaultdict` changes `len()`.
- [] I timed `list.count()` inside a loop against `Counter` and can quote the ratio.
- [] I wrote top-k twice — heap and buckets — and can say which wins for which shape of input.
- [] I can say what the `len(result) == k` check has to be inside, and what breaks otherwise.
- [] I can give the four parts of a design pattern and say which one people skip.
- [] I can go from a symptom to an axis to a pattern for at least five of the eight symptoms.
- [] I have one specific pattern I regret, with numbers, ready to say out loud.
- [] I answered all three questions above out loud.

Grouping: the key-design skill

DATA STRUCTURES AND ALGORITHMS

Grouping: the key-design skill

You can invent the right dictionary key for a grouping problem, which is the whole trick.

SYSTEM DESIGN

Singleton

You can implement it, and say why many engineers consider it an anti-pattern.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Group these strings into anagram groups. What is your key?*
- *Implement a thread-safe singleton. Now tell me why I should not use it.*

1 What this is, and why they ask it

Grouping is putting every item into a pile, where the pile is chosen by something you compute from the item. In code it is a dictionary whose values are lists: `key → [items with that key]`. The mechanics take four lines and you will have them in ten minutes.

The mechanics are not the lesson. **The lesson is choosing the key**, and that is a genuine skill with a genuine failure mode in both directions. A key that captures too little merges things that do not belong together. A key that captures too much splits things that do. The right key is the answer to one question — *what exactly makes two of these items belong in the same pile?* — computed and nothing more.

They ask it because the key is where the thinking is, and it is impossible to fake. Group Anagrams is the standard vehicle, and it is asked at almost every product company, but the same question comes disguised a dozen other ways: group shifted strings, group by diagonal in a matrix, validate a Sudoku, find duplicate files by content, bucket events by hour. In every one of them the code is the same four lines and the interview is entirely in the one line that computes the key. An interviewer watching you will say almost nothing until you write that line, and then ask “why that key?”.

2 The story

The wedding at the hall in Basavanagudi finished at about four, and by half past four the problem everyone had been quietly not thinking about was sitting in the middle of the floor.

Six households had lent their vessels. Six hundred-odd of them — steel plates, tumblers, the big serving dishes, four enormous pots. All used, all washed, and all stacked in one heap by two tired boys who had no instructions.

Lakshmi, whose sister’s daughter had just been married, sat down on a chair in front of the heap with two nieces and said they were not leaving until it was done.

The first niece started by size. Small tumblers here, medium plates there, the big things at the end. Twenty minutes in, Lakshmi stopped her. Every one of the six houses owns small tumblers. Sorting by size had made four beautiful piles, and not one of them belonged to anybody.

The second niece tried by how worn they looked, which was worse.

What actually works is a thing everybody in that room already knew and nobody had said out loud. Every household scratches a mark on the underside of its vessels, because this happens four or five times a year. A letter, or two, or a little symbol. You turn the vessel over and it tells you whose it is.

So that is what they did. Pick one up, turn it over, read the mark, put it on that mark's pile. If the mark was one they had not seen yet, they started a new pile for it and carried on. No thinking after the first minute — just turn, read, drop.

Six hundred vessels, three of them, a little over an hour.

The one real problem came near the end. Two of the six households both use a single letter S — Srinivas and Shankar, who live on the same road and had never noticed. There was one pile of about ninety vessels belonging to two houses, and no way to split it by looking. The two families had to sit together and go through it by memory, item by item, which took another forty minutes and involved some disagreement about a rice dish.

Lakshmi's line about it afterwards was that the mark has to be enough to tell you apart from everybody else, and no more than that. Srinivas has since started scratching SRN.

3 The idea in plain English

The heap is your input. Each pile is a **group**. The mark scratched underneath is the **key**, and the act of turning the vessel over to read it is **computing the key from the item**.

Three things from that afternoon, and they are the three things this lesson is about.

One: the piles are a dictionary of lists. Key to a list of everything with that key. New key means a new empty pile, exactly like Shobha's new dish on the whiteboard in [day 063](#).

Two: sorting by size was not a bad *method*, it was a bad *key*. The loop was fine. The piles were tidy. The key answered the wrong question. This is the failure that actually happens in interviews — the code is correct and the answer is wrong.

Three: a key must capture exactly what makes two items the same, and nothing else. The single letter S captured too little, so two households merged. If they had scratched the date of purchase as well, the key would have captured too much, and one household's vessels would have split into nine piles.

The four lines

```
from collections import defaultdict

groups: dict[str, list[str]] = defaultdict(list)
for word in words:
    groups[key_of(word)].append(word)
```

`defaultdict(list)` is a dictionary that creates an empty list the first time you touch a key, so there is no “does this pile exist yet” branch. That is the new-pile moment, handled.

Written without the import, in case an interviewer asks:

```
groups: dict[str, list[str]] = {}
for word in words:
    groups.setdefault(key_of(word), []).append(word)
```

`setdefault(k, [])` means: give me the list at `k`, putting an empty list there first if there is none. It works, and it has a small cost worth knowing about — the `[]` is constructed on *every* call, even when the key already exists, and then thrown away. For a million items that is a million wasted list allocations. Use `defaultdict` when you have the import.

Now: what goes in `key_of`?

This is the whole subject. Here is the recipe, and it is one sentence.

Write down, in words, what makes two items belong in the same pile. Then compute exactly that.

Say the sentence before you write the line. If you cannot say the sentence, you are not ready to write the line.

Worked keys

Anagrams. Two words belong together when they use the same letters the same number of times. Order does not matter. So the key must throw away order and keep counts.

```
def key_of(word: str) → str:
    return "".join(sorted(word))    # "eat" → "aet", "tea" → "aet"
```

Sorting the letters destroys exactly the thing that should not matter — the order — and keeps exactly the thing that should — which letters, how many. That sentence is the answer to “why that key?”.

There is a second key for the same question:

```
def key_of(word: str) → tuple[int, ...]:
    counts = [0] * 26
    for character in word:
        counts[ord(character) - ord("a")] += 1
    return tuple(counts)    # "eat" → (1,0,0,0,1, ...,1, ...)
```

A tuple of 26 counts. Same grouping, different cost, and it must be a `tuple` and not a `list` because dictionary keys must be hashable. Which one to use is a real trade and §6 has the numbers.

Shifted strings. `"abc"`, `"bcd"` and `"xyz"` are all the same shape — each letter one further on than the last. Two words belong together when the *gaps between consecutive letters* match. So the key is the gaps, not the letters.


```
def shift_key(word: str) → tuple[int, ...]:
    return tuple((ord(b) - ord(a)) % 26 for a, b in zip(word, word[1:]))
```

The `% 26` is the part people miss: `"az"` has a gap of 25 and `"za"` has a gap of -25, which is 1 after wrapping — and `"za"` really is a shift of `"ab"`.

Cells of a Sudoku box. Two cells are in the same three-by-three box when their row divided by three and column divided by three match.

```
box_key = (row // 3, column // 3)
```

Diagonals of a matrix. Every cell on the same top-left-to-bottom-right diagonal has the same `row - column`. Every anti-diagonal has the same `row + column` — which you met in [day 016](#).

```
diagonal_key = row - column
anti_key = row + column
```

Files with identical contents. Two files belong together when their bytes match. Comparing every pair is $O(n^2)$ reads. The key is a digest of the contents.

```
key = hashlib.sha256(data).hexdigest()
```

The two failure modes, named

FAILURE	WHAT IT LOOKS LIKE	VESSELS
Key too coarse — captures too little	Groups merge that should be separate	Both households marked S
Key too fine — captures too much	One real group splits into many	Marking the purchase date too

The test for coarse: *can I construct two items with the same key that should not be together?* The test for fine: *can I construct two items that should be together and have different keys?* Run both tests out loud on your key before you write the loop. It takes fifteen seconds and it is the entire question.

The key must be hashable

Same rule as [day 062](#). A `list` cannot be a key:

```
>>> groups = defaultdict(list)
>>> groups[["a"]].append(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

So the count key is a `tuple(counts)`, never `counts`. For a group where order genuinely does not matter, `frozenset` is the hashable set.

4 The picture

Grouping six words by the sorted-letters key. Left is the input, middle is the computed key, right is the pile it lands on.

word	key = sorted letters	piles as they fill
-----	-----	-----
"eat"	→ "aet"	aet: [eat]
"tea"	→ "aet"	aet: [eat, tea]
"tan"	→ "ant"	aet: [eat, tea] ant: [tan]
"ate"	→ "aet"	aet: [eat, tea, ate] ant: [tan]
"nat"	→ "ant"	aet: [...] ant: [tan, nat]
"bat"	→ "abt"	aet: [...] ant: [...] abt: [bat]

What to notice: the key column is the only place any thinking happens. Once a word has a key, the rest is `groups[key].append(word)` and cannot go wrong.

Now the two failure modes drawn on the same six words. First, a key that is too coarse — group by word length:

```
key = len(word)
3: [eat, tea, tan, ate, nat, bat]      ← one pile, everything in it
```

Every word has length three, so the key tells you nothing. Two things that should be apart are together. Second, a key that is too fine — group by the word itself:

```
key = word
eat: [eat]    tea: [tea]    tan: [tan]    ate: [ate]    nat: [nat]    bat: [bat]
```

Six piles of one. Two things that should be together are apart. **The right key sits between these two, and finding where is the problem.**

And the box key on a nine-by-nine grid, so you can see why integer division works:

column →	0	1	2		3	4	5		6	7	8
row 0	(0,0)	...			(0,1)	...			(0,2)	...	
row 1	(0,0)				(0,1)				(0,2)		
row 2	(0,0)				(0,1)				(0,2)		
-----+-----+-----											
row 3	(1,0)				(1,1)				(1,2)		
row 4	(1,0)				(1,1)				(1,2)		
row 5	(1,0)				(1,1)				(1,2)		
-----+-----+-----											
row 6	(2,0)				(2,1)				(2,2)		
row 7	(2,0)				(2,1)				(2,2)		
row 8	(2,0)				(2,1)				(2,2)		

Rows 0, 1 and 2 all give `row // 3 == 0`. That is the whole trick, and it is worth drawing once because the same `// k` key groups anything into fixed-size blocks — timestamps into hours, ids into shards, scores into bands.

5 The code, built step by step

Step 1 — the grouping loop, with the key left blank

```
from collections import defaultdict

def group_by(items: list[str], key_of) → list[list[str]]:
    groups: dict[object, list[str]] = defaultdict(list)
    for item in items:
        groups[key_of(item)].append(item)
    return list(groups.values())
```

Write this first, every time. It is the same four lines for every grouping problem there is, and separating it from the key makes the interview conversation land on the right place.

Step 2 — the anagram key, by sorting

```
def anagram_key(word: str) → str:
    return "".join(sorted(word))
```

`sorted("eat")` gives `['a', 'e', 't']` and `"".join` turns it back into `"aet"`. Say what it is doing as you write it: *sorting destroys the order, which is the thing that must not matter, and keeps the multiset of letters, which is the thing that must.*

Step 3 — the anagram key, by counting

```
def anagram_key_counts(word: str) → tuple[int, ...]:
    counts = [0] * 26
    for character in word:
        counts[ord(character) - ord("a")] += 1
    return tuple(counts)
```

`ord("a")` is 97, so `ord("c") - ord("a")` is 2, which is the slot for `c`. This avoids the sort, so it is $O(k)$ per word rather than $O(k \log k)$ — and it hard-codes the assumption that the input is twenty-six lowercase English letters. **Say that assumption out loud.** If the interviewer says “Unicode”, this key is dead and the sorted key survives.

Step 4 — put them together

```
def group_anagrams(words: list[str]) → list[list[str]]:
    groups: dict[str, list[str]] = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())
```

Five lines, and the fourth is the only one that required thought.

Step 5 — the general helper, and three more keys

The value of separating the key out is that the same helper solves five problems.

```
def group_shifted(words: list[str]) → list[list[str]]:
    def key(word: str) → tuple[int, ...]:
        return tuple((ord(b) - ord(a)) % 26 for a, b in zip(word, word[1:]))
    return group_by(words, key)
```

Note what happens to a one-character word: `zip("a", "")` is empty, so the key is `()`, and all single characters land in one group. That is correct — every single letter is a shift of every other — and it is the edge case an interviewer will ask about.

The complete solution

```
from collections import defaultdict
from typing import Callable, Hashable, TypeVar

T = TypeVar("T")

def group_by(items: list[T], key_of: Callable[[T], Hashable]) → list[list[T]]:
    """Split items into groups by a computed key. The four lines every
    grouping problem shares – all the thinking is in `key_of`."""
    groups: dict[Hashable, list[T]] = defaultdict(list)
    for item in items:
        groups[key_of(item)].append(item)
    return list(groups.values())

def group_anagrams(words: list[str]) → list[list[str]]:
    """Words using the same letters the same number of times, grouped.

    The key throws away order (which must not matter) and keeps the letter
    multiset (which must).  $O(n * k \log k)$  for  $n$  words of length  $k$ .
    """
    groups: dict[str, list[str]] = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())

def group_anagrams_counting(words: list[str]) → list[list[str]]:
    """Same grouping in  $O(n * k)$ , assuming lowercase a-z only."""
    groups: dict[tuple[int, ...], list[str]] = defaultdict(list)
    for word in words:
        counts = [0] * 26
        for character in word:
            counts[ord(character) - ord("a")] += 1
        groups[tuple(counts)].append(word)
    return list(groups.values())

def group_shifted_strings(words: list[str]) → list[list[str]]:
    """`"abc", "bcd" and "xyz" are one group: the gaps between letters match.
    The % 26 handles wrap-around, so "za" groups with "ab"."""
    def key(word: str) → tuple[int, ...]:
        return tuple((ord(b) - ord(a)) % 26 for a, b in zip(word, word[1:]))
    return group_by(words, key)

def diagonal_groups(matrix: list[list[int]]) → dict[int, list[int]]:
    """Every cell on one top-left-to-bottom-right diagonal shares row - column."""
    groups: dict[int, list[int]] = defaultdict(list)
    for row, values in enumerate(matrix):
        for column, value in enumerate(values):
            groups[row - column].append(value)
    return dict(groups)

if __name__ == "__main__":
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    print(group_anagrams(words))
    # [['eat', 'ate'], ['tan', 'nat'], ['bat']]
    print(group_anagrams_counting(words))
    print(group_shifted_strings(["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"]))
    # [['abc', 'bcd', 'xyz'], ['acef'], ['az', 'ba'], ['a', 'z']]
    print(diagonal_groups([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
    # {0: [1, 5, 9], -1: [2, 6], -2: [3], 1: [4, 8], 2: [7]}
    print(group_anagrams([])) # []
    print(group_anagrams([""])) # [['']]
```

The last two lines are the edge cases. An empty input gives an empty list of groups; a list containing the empty string gives one group containing it, because `"".join(sorted(""))` is `""`, which is a perfectly good key.

6 What it costs

Let n be the number of words and k the length of the longest word.

The sorted key

The loop runs n times. Each iteration sorts a word of length k , which is $O(k \log k)$, then joins it, $O(k)$, then does one dictionary insert, $O(k)$ because hashing a string of length k reads all k characters.

$$n \text{ iterations} \times (k \log k + k + k) = O(n k \log k)$$

With $n = 10,000$ words of length $k = 10$: $10,000 \times 10 \times \log_2(10) \approx 10,000 \times 10 \times 3.3 = 330,000$ comparison steps.

The counting key

The loop runs n times. Each iteration walks the word once, $O(k)$, builds a 26-tuple, $O(26)$, and hashes it, $O(26)$.

$$n \text{ iterations} \times (k + 26 + 26) = O(n(k + 26)) = O(nk) \text{ for } k \text{ around } 26$$

With the same numbers: $10,000 \times (10 + 52) = 620,000$ steps.

Notice the counting key is *worse* here, and this is the honest part most write-ups skip. The 26-slot array is a fixed cost paid on every word, so for short words it loses. Work out where it wins:

```
sorted key beats counting when  k log k < k + 52
k = 10:    33 < 62    sorted wins
k = 100:   664 < 152   counting wins
```

So the crossover is around $k \approx 25$. **For interview-sized words, the sorted key is both simpler and faster, and you should say so** rather than reciting that counting is $O(k)$ and therefore better. Counting wins for long strings, and it is the only option if the interviewer says the words can be a megabyte each.

Space

Both are $O(nk)$ total — every input word is stored once inside some group, and the keys add at most another $O(nk)$. There is no version of this problem with less than $O(nk)$ space, because the output itself is that big. Say that: *the output is the same size as the input, so the space is not optional*.

What the wrong key costs

If your key is too fine and produces n groups of one, you have done $O(n k \log k)$ work to produce the input back again. If it is too coarse and produces one group of n , you have done the same work to produce a copy of the input in a list. Both run fast and both are wrong, which is why the complexity is never where this problem is failed.

Compare with the brute force

The obvious approach is: for every pair of words, check whether they are anagrams. That is $n(n-1)/2$ pairs, each costing $O(k \log k)$:

```
n = 10,000: 10,000 x 9,999 / 2 = 49,995,000 pairs x 33 = 1.6 x 10^9 steps
```

Against 330,000. A **factor of about five thousand**, and the reason is exactly the reason from [day 062](#): computing a key turns “compare with everything” into “look it up once”.

7 The traps

Trap 1 — an unhashable key

```
>>> groups = defaultdict(list)
>>> groups[["a", "b"]].append(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

The counting key is the place this bites: `return counts` where `counts` is a list raises, and `return tuple(counts)` works. It is one word of difference and it is the commonest runtime error in this problem.

Trap 2 — `itertools.groupby` does not group

This one is genuinely nasty because it looks like the tool for the job, has the right name, and produces plausible wrong output.

```
>>> from itertools import groupby
>>> words = ["eat", "tea", "tan", "ate", "nat", "bat"]
>>> key = lambda w: "".join(sorted(w))
>>> [(k, list(v)) for k, v in groupby(words, key=key)]
[('aet', ['eat', 'tea']), ('ant', ['tan']), ('aet', ['ate']), ('ant', ['nat']), ('abt', ['bat'])]
```

Five groups instead of three, and `aet` appears twice. `groupby` only collects **runs of adjacent items** with the same key. It does not gather items from across the sequence.

The fix is to sort first — but sort **by the key**, not by the item:

```
>>> [(k, list(v)) for k, v in groupby(sorted(words), key=key)]
[('aet', ['ate']), ('abt', ['bat']), ('aet', ['eat']), ('ant', ['nat', 'tan']), ('aet', ['tea'])]

>>> [(k, list(v)) for k, v in groupby(sorted(words, key=key), key=key)]
[('abt', ['bat']), ('aet', ['eat', 'tea', 'ate']), ('ant', ['tan', 'nat'])]
```

Look at the middle one. `sorted(words)` sorts alphabetically, which is not the same order as the key, so `groupby` still splits `aet` three ways. Only `sorted(words, key=key)` works. And even then you have paid $O(n \log n)$ for a job the dictionary does in $O(n)$.

In an interview, use a `defaultdict`. Mention `groupby` only to say why you are not using it.

Trap 3 — the key that is too coarse

Grouping anagrams by `sorted(word)` is right. Grouping them by `len(word)` and `set(word)` looks close and is wrong:

```
key = (len(word), frozenset(word))
```

`"aab"` and `"abb"` both give `(3, frozenset({'a', 'b'}))` and they are not anagrams. `frozenset` throws away *how many* of each letter, which is exactly the thing the question is about. The test to run: *can I construct two items with the same key that should not be together?* Here, in five seconds, yes.

Trap 4 — the key that is too fine

For “group words that are one edit apart” it is tempting to key on the word with each position blanked. That is fine. For “group anagrams”, keying on `tuple(sorted(word)) + (len(word),)` adds a component that is already implied by the rest — harmless here, but the same instinct applied to “group customers by city” produces a key of `(city, pincode, area)` and splits one city into forty piles. **Every component you add to a key can only split groups further, never merge them.** Add none you cannot justify.

Trap 5 — reading from a `defaultdict` creates the pile

```
>>> groups = defaultdict(list)
>>> groups["never seen"]
[]
>>> len(groups)
1
>>> dict(groups)
{'never seen': []}
```

If you check `if groups[k]:` instead of `if k in groups:`, you have just added an empty group, and `list(groups.values())` now returns an empty list among your real answers. This is the same trap as [day 063's](#) `defaultdict(int)`, and here it shows up as an extra `[]` in the output, which LeetCode will reject and you will stare at.

Trap 6 — `setdefault` builds a list every single time

```
groups.setdefault(key, []).append(word)
```


Correct, and the `[]` is constructed on every iteration even when the key already exists, then discarded. At `n = 1,000,000` that is a million pointless allocations — roughly twice as slow as `defaultdict` in practice. Fine for an interview; worth knowing why the other one exists.

Trap 7 — mutating something you used as a key

If your key is a tuple built from an object's fields and you then change those fields, the group is unaffected — the key was copied at insertion. But if you keep a `list` of items in the group and mutate an item, the grouping is now a lie: the item sits in a pile it no longer belongs to, silently. Same rule as [day 060: group on fields that do not change](#).

8 In the interview

How it gets asked

- “Given an array of strings, group the anagrams together. You may return the answer in any order.” The standard one, LeetCode 49.
- “Group all strings that are shifts of one another.” Same shape, harder key, and the wrap-around is the point.
- “Determine if a 9×9 Sudoku board is valid.” Nobody calls it grouping, and it is three grouping problems at once — rows, columns and boxes — with a set per group.
- The vague one: “I have two million files. Find the duplicates.” No mention of keys or maps. The whole answer is “hash the contents and group by the digest”, and the follow-up is about hash collisions and why you would compare bytes on a match anyway.
- And the direct probe, which will come whatever the question was: “why that key?”

What to say out loud, in the first ninety seconds

1. **Say the belonging rule in plain words, before any code.** “Two words are in the same group when they use the same letters the same number of times. Order does not matter.”
2. **Turn that sentence into a key, and say why it is exactly right.** “So my key must destroy order and preserve counts. Sorting the letters does exactly those two things — `eat` and `tea` both become `aet`.”
3. **Run the two tests out loud.** “Can two words that are not anagrams share this key? No, because the sorted letters determine the multiset. Can two anagrams get different keys? No, because sorting is deterministic.” Fifteen seconds, and it is the strongest part of the answer.
4. **Then say the loop is trivial.** “The rest is a `defaultdict(list)` and one append per word.”
5. **State the cost with both letters named.** “`n` words of length `k`, so $O(n \cdot k \log k)$ time and $O(n \cdot k)$ space, and the space is not optional because the output is that size.”

The follow-ups

“Can you avoid the sort?” “Yes — key on a tuple of 26 letter counts instead, which is $O(k)$ per word rather than $O(k \log k)$. But I would check the input first. The 26-count key pays a fixed 52-step cost per word, so for words of about ten characters the sorted key is actually faster; counting wins from roughly $k = 25$ upwards. And the count key assumes lowercase a to z, so it dies on Unicode while the sorted key does not.”

“What if the strings are Unicode?” “The fixed 26-slot array is gone. I would either keep the sorted key, which needs no assumption at all, or use a `Counter` and key on `frozenset(counter.items())` — hashable, and it works for any alphabet. I would mention that sorting arbitrary Unicode has its own normalisation questions, so I would ask whether `é` and `e-plus-accent` should be equal.”

“How would you find duplicate files in a directory of two million?” “Group by a key, same shape. Cheapest first: group by file size, because two files of different sizes cannot be identical, and that discards almost everything for nothing. Within each size group, hash the contents — SHA-256 — and group by the digest. Then, for anything that collides, compare the bytes directly, because a digest match is overwhelmingly likely but not a proof. That is three keys of increasing cost, applied in increasing order.”

“Why not `itertools.groupby`?” “Because it only groups adjacent items. It would give me `aet` three separate times on the standard example unless I sort by the key first, and sorting costs $O(n \log n)$ for something the dictionary does in $O(n)$. It is the right tool when the data is already ordered by the key — reading a sorted log file, for instance.”

“Your key is a list. Will that work?” This one is a test, not a question. “No — dictionary keys must be hashable and a list is not. I would return `tuple(counts)`.”

A model answer

Asked: *group these strings into anagram groups. What is your key?*

“Let me state the rule for belonging first, because everything follows from it. Two words go in the same group when they contain the same letters with the same multiplicities. The order of the letters is exactly the thing that must not matter.

So I want a key that destroys order and preserves counts. Sorting the letters of the word does precisely those two things: `eat` and `tea` and `ate` all become `aet`.

Let me check it both ways. Could two words share that key without being anagrams? No — if the sorted letters are identical then the letter multisets are identical, which is the definition. Could two anagrams get different keys? No — sorting is deterministic, so the same multiset always sorts the same way. The key is exactly as coarse as it should be and no coarser.

The rest is four lines: a `defaultdict(list)`, and for each word append it to the pile for its key, then return the values.

For cost, let n be the number of words and k the longest word length. Each word costs $k \log k$ to sort plus k to hash, so it is $O(n \cdot k \log k)$ time. Space is $O(n \cdot k)$ and that is not avoidable, because the output holds every input word.

If you want to remove the sort I can key on a tuple of twenty-six letter counts instead, which is $O(k)$ per word. I would only do that for long words though — the fixed cost of the 26-slot array means the sorted key is faster up to about $k = 25$, and the count key hard-codes lowercase English, so it breaks on Unicode.

Edge cases: an empty list returns an empty list; the empty string is its own valid key, so a list containing `""` gives one group. And I would use a `defaultdict` rather than `itertools.groupby`, because `groupby` only collects adjacent runs and would split each anagram group several times unless I sorted by the key first.”

9 Recall card

- **Grouping is four lines; the key is the whole problem.** `groups = defaultdict(list)` then `groups[key_of(item)].append(item)`. Without the `import`, `groups.setdefault(k, []).append(x)` — correct, but it allocates a throwaway `[]` on every call.
- **The recipe: say in words what makes two items belong together, then compute exactly that — no more, no less.** Then run both tests out loud: *same key but shouldn't be together?* (too coarse) · *should be together but different keys?* (too fine).
- **The keys worth memorising:** anagrams → `"".join(sorted(word))` (destroys order, keeps counts) · shifted strings → `tuple((ord(b)-ord(a)) % 26 ...)` · Sudoku box → `(r // 3, c // 3)` · diagonals → `r - c`, anti-diagonals → `r + c` · duplicate files → size first, then SHA-256, then compare bytes.

- **The count key is not automatically better.** $k \log k$ vs $k + 52$ crosses over at about $k = 25$, so for interview-length words the sorted key wins — and it survives Unicode, which the 26-slot array does not. Cost: $O(n \cdot k \log k)$ time, $O(n \cdot k)$ space, and the space is not optional because the output is that big. Brute force is $5,000 \times$ worse at $n = 10,000$.
- **Three things that bite:** the key must be hashable (`tuple(counts)`), never `counts` · `itertools.groupby` only groups **adjacent** runs, so it splits `aet` three ways unless you `sorted(words, key=key)` first · reading a missing key from a `defaultdict(list)` **inserts an empty list** and it turns up in your output.

1 What this is, and why they ask it

Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it. It is the first of the five creational patterns, the shortest to write, and the most argued about. Two separate promises are packed into that one sentence — *exactly one* and *globally reachable* — and almost all the trouble comes from the second one.

They ask it constantly, and they ask it in a very specific two-part shape. First “write me a thread-safe singleton”, which is a concurrency question wearing a pattern’s clothes — they want to see whether you know that the naive version has a race, and whether you know what double-checked locking is and why it was broken for years. Then “now tell me why you should not use it”, which is the real question. The first part is knowledge; the second part is judgement, and a candidate who sails through the code and then cannot say what it costs has just demonstrated the exact problem the question was designed to find.

There is also a distinction hiding in here that separates good answers from great ones: **needing one instance is not the same as using the Singleton pattern**. Almost every case where people reach for Singleton is actually satisfied by creating one object at startup and passing it around.

2 The story

The residents’ association at the flats in Kammanahalli owns one ladder.

It is aluminium, about twelve feet, and it lives behind the generator room. Nobody remembers buying it. Everybody uses it — for the water tank float, for the ceiling fan in the corridor, for the tube light outside flat 302, for putting up the lights at Deepavali.

The rule about it is not written anywhere. If you need the ladder, you go and get the ladder. Nobody signs for it. Nobody is asked. This works for years at a time, and it is genuinely convenient — a new family moves in, somebody tells them where it is, and that is the whole system.

Three things have gone wrong with it, and they went wrong in a way that took people a while to see.

The first is that when it is not behind the generator room, finding it is somebody’s whole afternoon. It could be in any of forty flats. There is no way to know except knocking.

The second happened in 2019. The old secretary handed over in March and neither the new secretary nor the new treasurer knew where the ladder lived. Both of them went out and bought one. They found out three weeks later, when both ladders turned up in the corridor on

the same evening. Two hundred rupees short of nine thousand, spent twice, because two people checked at the same time, both found nothing, and both acted.

The third is the one that actually costs. Last April a man on the third floor was leaning it against the tank and the bottom rung went. Not badly, but enough that nobody would stand on it. For eleven days the corridor light stayed out, the tank float stayed broken, and a family who wanted to paint their balcony could not, because in forty flats there was exactly one ladder and no plan for what happens when there is none.

They talked about it at the next meeting. Somebody said they should keep two. Somebody else said the real problem is that nobody knows who has it. In the end they did something duller and better: the watchman keeps it, and if you want it you ask him and he writes your flat number down. Slightly more annoying, nothing has gone wrong since — and it is now possible to answer “who has the ladder”.

3 The idea in plain English

The ladder is a **singleton**: there is exactly one, and everybody knows how to get at it without being given it. Both halves are doing work, and they cause different problems.

“**Exactly one**” is usually fine and often necessary. One connection pool, one configuration, one logging destination.

“**Everybody can reach it without being handed it**” is the part that hurts. When flat 302 uses the ladder, nothing anywhere records that flat 302 depends on the ladder. You cannot tell by looking at the flat. That is a **hidden dependency**, and it is the same criticism as the one against global variables — because a singleton is a global variable with a constructor attached.

The three failures in the story map exactly onto the three things people mean when they call Singleton an anti-pattern.

THE STORY	IN CODE
Nobody knows who has the ladder	Hidden dependencies — usage does not appear in any signature
Two people both bought one	The creation race — the reason “thread-safe” is in the question
Eleven days with no ladder at all	Shared mutable state and a single point of failure; and in tests, one test’s changes leaking into the next

And the fix they landed on is exactly the fix in code: **the watchman**. One ladder still, but now somebody owns it, hands it out, and knows who has it. That is one instance created in one place and handed to whoever needs it — which is *not* the Singleton pattern, and is almost always what you actually wanted.

The distinction that matters most

“I need exactly one of these” is a requirement. “Singleton” is one way to meet it, and usually the worst one.

The alternative is to create one instance where your program starts up — the **composition root**, from [day 053](#) — and pass it to whoever needs it. You still have exactly one. You have not made it globally reachable, so every user of it is visible in a constructor, and a test can hand over a different one.

Say that sentence in the interview. It is the single highest-value thing in this lesson.

The naive implementation, and its race

```
class ConnectionPool:
    _instance = None

    @classmethod
    def get_instance(cls) → "ConnectionPool":
        if cls._instance is None:          # 1. check
            cls._instance = ConnectionPool() # 2. create
        return cls._instance
```

Read it as two steps: check, then create. Now put two threads in it. Thread A runs step 1 and finds `None`. Before it reaches step 2, the operating system switches to thread B, which also runs step 1 and also finds `None`. Both create. **You have two connection pools**, both of which think they are the only one, and half your program is holding one and half the other.

This is the secretary and the treasurer, and it is why the question says “thread-safe”.

The fixes, in the order they were invented

Eager creation. Build it when the class is first loaded, before anybody can ask.

```
class ConnectionPool:
    _instance = None

    ConnectionPool._instance = ConnectionPool() # at import time
```

No race, because there is no check. The cost is that you pay for it even if nobody uses it, and you lose control of *when* it is built — which matters if building it opens a socket.

A lock around the whole thing.

```
import threading

class ConnectionPool:
    _instance = None
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls) → "ConnectionPool":
        with cls._lock:
            if cls._instance is None:
                cls._instance = ConnectionPool()
            return cls._instance
```

Correct, and every single call takes the lock forever afterwards, including the millionth call when the answer has been sitting there for an hour.

Double-checked locking. Check without the lock; only take the lock if it looks unset; check again inside.

```
@classmethod
def get_instance(cls) → "ConnectionPool":
    if cls._instance is None:           # cheap check, no lock
        with cls._lock:
            if cls._instance is None:   # the second check
                cls._instance = ConnectionPool()
    return cls._instance
```

This is the one interviewers are fishing for, and the interesting part is its history: **in Java it was broken until 2004**. The compiler and the CPU are allowed to reorder the write that assigns the reference and the writes that fill in the object's fields, so another thread could see a non-null reference pointing at a half-built object. The fix was the `volatile` keyword under Java 5's new memory model. Being able to say that sentence is worth a lot in this question.

The one you would actually write in Java: an `enum`.

```
public enum ConnectionPool {
    INSTANCE;
    public void borrow() { ... }
}
```

Joshua Bloch's recommendation in *Effective Java*. The JVM guarantees an enum constant is created once, it is thread-safe with no code, and it is the only version that survives serialisation and reflection attacks.

The Python versions

In Python, most of this ceremony is unnecessary, and saying why is a strong answer.

A module. Python imports a module once per process and caches it in `sys.modules`. So a module-level object is a singleton, created lazily, thread-safely, with no code at all.


```
# pool.py
pool = ConnectionPool()

# anywhere else
from pool import pool
```

A cached factory function.

```
from functools import cache

@cache
def get_pool() → ConnectionPool:
    return ConnectionPool()
```

`functools.cache` remembers the result, so the object is built once, on first use. This is the version worth writing in an interview, because it is three lines and it is lazy.

Overriding `__new__`, which is the version people expect to see and the one with a real trap:

```
class ConnectionPool:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls) → "ConnectionPool":
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self) → None:
        self.size = 10          # ← runs on EVERY ConnectionPool() call
```

The trap: `__new__` returns the existing object, and then Python calls `__init__` on it **again**, every time. Any state set in `__init__` is reset on each call. This bug is subtle, common, and exactly the kind of thing an interviewer is delighted to find.

4 The picture

The two claims a singleton makes, and which one causes which problem.

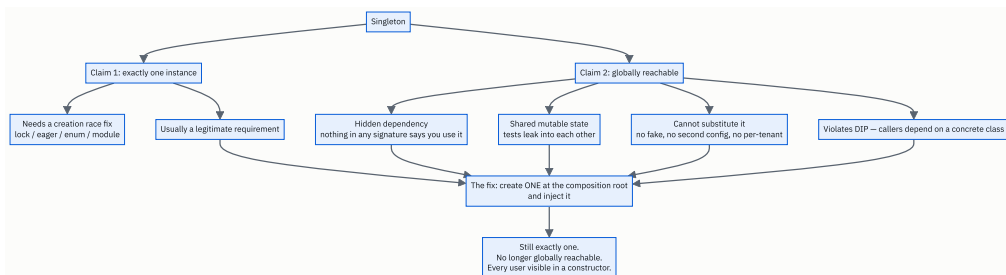


FIGURE 64.1

What to notice: every arrow into the fix comes from claim 2, not claim 1. Nobody objects to having one connection pool. They object to any function anywhere being able to summon it.

Now the race, drawn on a timeline, because this is what the interviewer wants you to be able to describe:

```
time →

thread A  ...check _instance → None ..... create → POOL#1
          ^
thread B  ..... check _instance → None ... create → POOL#2
          |
          A has not assigned yet,
          so B also sees None

result: two pools. A holds POOL#1, B holds POOL#2, and
       cls._instance ends up as whichever wrote last.
```

The gap between the check and the assignment is the whole bug. It is usually a few nano-seconds wide, which is why it passes every test you write and fails in production at three in the morning under load.

5 How it actually works

The mechanics, precisely

A singleton needs three things: a private way to construct the object so nobody can make a second, a place to store the one instance, and a public way to get at it.

- **Java:** private constructor, `private static Instance instance`, `public static getInstance()`.
- **Python:** there is no private constructor. You cannot stop somebody calling `ConnectionPool()`, which is precisely why the module-level object and the cached function are preferred — they do not pretend to enforce something the language will not enforce.
- **Lifetime:** the instance lives as long as the class is loaded, which in practice means as long as the process. There is no destructor you can rely on. Anything holding a socket or a file handle gets closed at process exit, or not at all.

The one that surprises people: a singleton is not global

This is worth its own heading because it is a genuine production issue and a very good thing to volunteer.

A singleton is one instance **per process**, not per system. Run your Django application under Gunicorn with eight workers and you have **eight** of them. Run four containers of that and you have thirty-two. In Java it is one per class loader, which is why an application server hosting three applications has three.

So if the thing you made a singleton is a cache, you now have thirty-two caches that disagree with each other. If it is a rate limiter, your limit is thirty-two times what you configured. **Anything that must be genuinely single across a system belongs in Redis or the database, not in a `getInstance()`.**

Real products that use it, and how

- **Python's `logging` module.** `logging.getLogger("app.orders")` returns the same logger object for the same name, every time, from a module-level registry. This is really a *multiton* — one instance per key — and it is the well-behaved cousin.
- `None`, `True` and `False` are singletons in CPython, which is why `x is None` works. Small integers from -5 to 256 are interned singletons too.
- `Runtime.getRuntime()` in Java, and `Calendar.getInstance()` — the latter being a factory method that does *not* return a singleton, which is a naming trap.
- **Spring beans are singleton-scoped by default**, and this is the important example. There is one instance of each bean per container — but the container *injects* it into constructors. So you get the “exactly one” without the “globally reachable”. Spring took the good half and threw away the bad half, and saying that shows you understand the distinction.
- **Database connection pools** — HikariCP, `psycopg_pool`, SQLAlchemy's engine. Genuinely one per process, and the canonical legitimate use.
- **Django's `settings` object.** One per process, imported everywhere, and famously awkward to override in tests, which is why Django had to ship a dedicated `override_settings` decorator. That decorator existing at all is the cost of the pattern, made visible.

What “inject it instead” looks like

```
# main.py – the composition root, the only place this is constructed
def main() → None:
    pool = ConnectionPool(size=10)
    repository = OrderRepository(pool)
    service = OrderService(repository)
    serve(service)
```

One pool. Created once. Everyone who uses it says so in a constructor, so `OrderService`'s dependencies are readable from its signature, and a test can pass a fake pool without touching any global. That is the whole alternative, and it is four lines.

6 The numbers

The cost of the lock, if you use the naive fix

Taking an uncontended lock in Python is roughly 40-60 nanoseconds; in Java an uncontended `synchronized` block is a few nanoseconds after biased locking, and tens of nanoseconds under contention.

```
get_instance() called 10,000,000 times in a request-heavy service
always-lock version: 10,000,000 x 50 ns = 0.5 seconds of pure lock overhead
double-checked:      10,000,000 x 2 ns = 0.02 seconds
```

Half a second of CPU per ten million calls is not catastrophic, but it is also completely unnecessary, and under contention it is far worse because threads serialise on a value that has not changed since startup. This is why double-checked locking was invented, and why it is worth being able to explain rather than just name.

The multi-process arithmetic

```
1 Gunicorn worker → 1 instance
8 workers          → 8 instances
4 containers x 8   → 32 instances
```

If that singleton was an in-memory cache sized at 500 MB “because we only have one”, your pods now need **16 GB** of memory instead of 500 MB, and the caches disagree. If it was a rate limiter set to 100 requests per second, your real limit is **3,200 per second**.

What it costs in the test suite

This is the number that actually persuades teams, and it is the same shape as [day 062](#)’s argument.

```
test needs a different config:
  with a singleton: reach into the class, overwrite _instance, remember to
                    restore it in teardown, and hope no test runs in parallel
                    ~ 8 lines of setup + 3 of teardown, per test file
  with injection:   pass a different object to the constructor
                    ~ 1 line
```

And the failure mode that costs the real time: **order-dependent tests**. One test mutates the singleton, a later test reads it, and the suite passes locally and fails in CI because the runner shuffled the order. A team hunting one of those loses somewhere between half a day and three days, and it will happen more than once.

A concrete shape: a suite of 400 tests where 30 touch a singleton config. Run in a random order, the chance that at least one leaks is high enough that you will see a flaky failure roughly weekly. The usual “fix” is to disable parallel test execution, which turns a 40-second suite into a 6-minute one — and now everybody runs the tests less often.

When the singleton is genuinely right

```
connection pool: 1 process x 20 connections = 20 sockets
no pooling:      500 requests/sec x 1 connection each = 500 connections
                  Postgres default max_connections = 100 → refused
```

That is a real reason to have exactly one pool, and nobody sensible argues with it. Notice that the argument is entirely about “exactly one” and says nothing about global reachability.

7 The trade-offs

What you give up

Testability, first and worst. A singleton is shared state that outlives a test. Tests stop being independent, order starts mattering, and the eventual response is usually to stop running them in parallel.

Honest signatures. `OrderService.__init__(self)` taking no arguments looks like a class with no dependencies. If it calls `ConnectionPool.get_instance()` inside a method, it has one and you cannot see it. Every reader has to grep the body to find out what it needs.

The ability to have two. Then a second tenant arrives, or a read replica, or staging config alongside production config in one process, and “exactly one” turns out to have been an assumption rather than a requirement. Un-singletoning something used in 200 places is a multi-week change.

Dependency inversion. `ConnectionPool.get_instance()` in domain code is a hard reference to a concrete class — the exact violation from [day 059](#). The import arrow points the wrong way, and no interface can be slid in without changing every call site.

A clear lifetime. When is it destroyed? What happens if it holds a socket and the network drops? There is no owner, so there is nobody to reconnect it.

What you get

Convenience, honestly. No wiring. A new piece of code can use the shared thing without anybody threading it through four constructors. On a small script, or a genuinely process-wide, immutable, infrastructure-level concern, this is a fair trade — and pretending otherwise is why people stop listening to design advice.

“I would not use this if...”

- **...the object holds mutable state.** A stateless helper as a singleton is nearly harmless. A mutable cache or config is where every problem comes from.
- **...anything needs to be substituted in tests.** Which, for anything touching the network, the clock or the filesystem, is everything.
- **...there is any chance of a second one being wanted.** Second tenant, second region, second database. Ask “could there ever be two?” and take the answer seriously.

- **...the uniqueness must hold across processes.** It will not. That belongs in Redis or Postgres.
- **...I am reaching for it to avoid passing an argument.** That is the real motivation about eighty percent of the time, and it is not a good one.

When it is defensible

Process-wide infrastructure that is expensive to create, effectively immutable after startup, and that genuinely must be shared: a connection pool, a metrics registry, a logging configuration, a thread pool. And even then, the better form is: **create one at the composition root and inject it**. You keep uniqueness and give up nothing.

8 In the interview

How it gets asked

- The two-parter, almost verbatim: *“Implement a thread-safe singleton. Now tell me why I should not use it.”* The second half is where the marks are.
- *“What is wrong with double-checked locking?”* A Java concurrency question that expects you to know about instruction reordering and `volatile`.
- *“How would you do a singleton in Python?”* Testing whether you reach for a metaclass or know that a module already is one.
- The disguised version: *“How would you make sure there is only one connection pool?”* — where the right answer is usually not the pattern at all.

What to say out loud, in the first ninety seconds

1. **Split the two claims immediately.** “Singleton makes two promises: exactly one instance, and a global way to reach it. The first is often a real requirement; almost all the criticism is of the second.”
2. **Write the naive version and point at the race before being asked.** “Check, then create. Two threads can both pass the check before either assigns, and you get two.”
3. **Give the fixes in order, with the cost of each.** Eager (pay always), locked (pay every call), double-checked (pay once — and mention the Java memory-model history).
4. **Say what you would actually write in this language.** In Python: a module-level object, or `@cache` on a factory function. In Java: an `enum`.
5. **Then volunteer the criticism before they ask for it.** That is the move. “I should say I would rarely reach for it — the requirement is usually ‘one instance’, which I would satisfy by constructing it once at the composition root and injecting it.”

The follow-ups

“Why was double-checked locking broken in Java?” “Because the write that publishes the reference and the writes that initialise the object’s fields could be reordered by the compiler or the CPU. A second thread could see a non-null reference to a half-constructed object, pass the outer check, and use it. Java 5’s memory model fixed it, provided the field is declared `volatile`, which gives you the happens-before relationship you need.”

“Is a singleton thread-safe once it is created?” “Creating it once is thread-safe. Using it is not, and that is a different question people conflate. If the single instance has mutable state, every thread in the process is sharing it, so you need locking inside it as well. One instance means maximum sharing, which means maximum contention.”

“How do you test code that uses one?” “With difficulty, which is the argument. You end up reaching into the class to replace the private instance and restoring it in teardown, which breaks under parallel test runners. Django ships an `override_settings` decorator precisely because its settings object has this problem. My preferred answer is to not be in that position: inject it, and a test passes a different object in one line.”

“Give me a case where you would use it.” “A connection pool. Postgres has a `max_connections` of 100 by default, and 500 requests a second each opening their own connection would be refused, so exactly one pool per process is a real requirement. But I would still construct it in `main` and pass it down rather than exposing `getInstance`, because that keeps the uniqueness and loses the global reach.”

“Is it really one per system?” “No, and that catches people out. It is one per process — or per class loader in Java. Eight Unicorn workers give you eight; four containers of eight give you thirty-two. So if it is a cache, thirty-two caches disagree with each other, and if it is a rate limiter, your real limit is thirty-two times what you set. Anything that must be single across a system belongs in Redis or the database.”

A model answer

Asked: *implement a thread-safe singleton, then tell me why I should not use it.*

“Singleton makes two promises: that there is exactly one instance, and that anyone can reach it without being handed it. I will do the code first, and then I want to separate those two, because the criticism is almost entirely of the second.

The naive version stores the instance on the class and creates it on first request — check, then create. That has a race: two threads can both run the check before either one assigns, so both create, and you end up with two objects each believing it is the only one. Half your program holds one and half holds the other.

The fixes, in the order they were invented. Eager creation at class-load time removes the check entirely, but you pay for the object even if nobody uses it, and you lose control of when it is built — which matters if it opens sockets. A lock around the whole method is correct but takes the lock on every call forever, including the ten-millionth call. Double-checked locking checks without the lock, takes the lock only if it looks unset, and checks again inside. That is the standard answer, and the historical footnote is that it was broken in Java until version 5, because the reference could be published before the object’s fields were initialised — you need `volatile` for the happens-before guarantee.

In Java today I would write an `enum` with one constant, which is Bloch’s recommendation: the JVM guarantees single creation and it survives serialisation. In Python I would not write any of this, because a module-level object already is a singleton — the interpreter imports a module once per process — or I would put `functools.cache` on a factory function, which gives me laziness in three lines. I would avoid overriding `__new__`, because `__init__` still runs on every call and resets the object’s state, which is a bug people hit repeatedly.

Now the second half. I would rarely use the pattern, and my reasoning is that ‘I need exactly one’ and ‘everyone can reach it globally’ are separate things, and only the first is ever the requirement. The global reach costs me four things. Dependencies stop being visible — a class that calls `getInstance` inside a method looks like it has no dependencies. Tests stop being independent, because state leaks between them and the suite becomes order-dependent, which is the kind of flake that costs days. I cannot substitute it, so a second tenant or a read replica becomes a multi-week change. And it is a concrete reference from my domain code to a specific class, which is a dependency-inversion violation with no seam to fix it.

What I do instead is construct exactly one at the composition root and pass it in. I still have exactly one, every user of it is visible in a constructor, and a test hands over a different object in one line. Spring does exactly this — its beans are singleton-scoped, but the container injects them rather than exposing a static accessor. It kept the good half.

One last thing worth saying: a singleton is one per process, not one per system. Eight workers give you eight of them, and four containers give you thirty-two. If it is a cache they disagree; if it is a rate limiter your limit is thirty-two times what you configured. Anything that must be truly single belongs in Redis or the database.”

9 Recall card

- **Two promises, and only the second is the problem.** *Exactly one instance* is often a real requirement; *globally reachable* buys hidden dependencies, shared mutable state, no substitution, and a DIP violation. **A singleton is a global variable with a constructor.**
- **The race is check-then-create.** Fixes in order: eager (always pay) · lock every call (~50 ns forever) · **double-checked locking** (~2 ns after the first) — and in Java it was *broken until Java 5* because the reference could be published before the fields were initialised; you need `volatile`. In Java write an `enum`; in Python a **module-level object** or `@cache` on a factory. Never override `__new__` alone — `__init__` **re-runs on every call** and resets state.
- **Say the distinction: needing one instance ≠ using Singleton.** Create one at the **composition root** and inject it. Same uniqueness, visible dependencies, one-line test substitution. Spring's beans are *singleton-scoped* but injected — it kept the good half.
- **It is one per process, not per system.** 8 Unicorn workers = 8; 4 containers × 8 = 32. A cache becomes 32 disagreeing caches; a 100/sec rate limiter becomes 3,200/sec. True system-wide uniqueness lives in Redis or Postgres.
- **The legitimate cases are process-wide, expensive, and immutable after startup:** connection pools (Postgres `max_connections` = 100 vs 500 req/sec), metrics registries, logging config, thread pools. Django's `override_settings` decorator exists *because* settings is a singleton — that decorator is the cost made visible.

PRACTICE

Practice — Grouping: the key-design skill

DSA topic: Grouping: the key-design skill **System design topic:** Singleton

Code these, in this order

One rule for the whole set: **say the belonging rule in words before you write the key**. “Two items go in the same group when __.” If you cannot finish that sentence, you are not ready to write the line, and every one of these problems is failed in that sentence rather than in the loop.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Group Anagrams	LeetCode 49 (Medium)	The standard vehicle. Whether you can defend the key, not whether you can write a <code>defaultdict</code> .
2	Find Duplicate File in System	LeetCode 609 (Medium)	Grouping by content rather than by name, and the cheap-key-first idea.
3	Group Shifted Strings	LeetCode 249 (Medium)	A key you have to invent, with a wrap-around case that most people miss.
4	Valid Sudoku	LeetCode 36 (Medium)	Three groupings at once, and the <code>(r // 3, c // 3)</code> key.

On problem 1, write it twice and time it

Once with `"".join(sorted(word))`, once with `tuple(counts of 26)`. Time both on 10,000 words of length 10, then on 10,000 words of length 200. Say which won each time and why, and find the crossover length yourself.

On problem 2, order your keys by cost

Do not hash two million files. Group by size first, because two files of different sizes cannot be equal and the size is free. Then hash only inside the groups that have more than one member. Then say what you would do about a digest collision, and why “compare the bytes” is a real answer rather than paranoia.

On problem 3, find the wrap-around case before you submit

Your key is the gaps between consecutive letters. Test `"az"` and `"ba"`. If they are not grouped together, you have forgotten `% 26`. Then say what happens to a one-character string, and why the answer is correct rather than a bug.

On problem 4, notice you already know the key

Rows, columns and boxes are three groupings of the same 81 cells. Write all three keys, then say why one pass with three sets beats three passes.

The key-design drill

For each of these, write the belonging sentence first, then the key, then run both tests on it:

1. Group words that are anagrams.
2. Group words that are anagrams, case-insensitively.
3. Group points that lie on the same line through the origin.
4. Group timestamps into hours.
5. Group people into age bands of ten years.
6. Group strings by their set of distinct characters.
7. Group numbers by their remainder when divided by 7.
8. Group employees by department and location.

Two of those eight have a key that is too coarse if you write the obvious thing. Find them, and give the input that proves it.

The too-coarse / too-fine drill

Each of these keys is wrong for grouping anagrams. Say which failure it is, and give the two words that prove it:

1. `len(word)`
2. `frozenset(word)`
3. `word`
4. `(len(word), word[0])`
5. `sum(ord(c) for c in word)`
6. `tuple(sorted(set(word)))`

Number 5 is the interesting one — it is a real hash function and it is too coarse. Give a collision.

The mechanics drill

Build the same grouping four ways and say when you would use each:

1. `defaultdict(list)`
2. `dict.setdefault(k, []).append(x)`
3. A plain `dict` with an explicit `if k not in groups`
4. `itertools.groupby`

Then, for 4, run it on the unsorted anagram list and count the groups. Run it on `sorted(words)` and count again. Run it on `sorted(words, key=key)` and count again. Explain all three numbers.

The break-it drill

Trigger each, read the exact output, and give the fix in one sentence:

1. Return `counts` (a list) as the key instead of `tuple(counts)`.
2. Use `itertools.groupby` on unsorted input.
3. Use `itertools.groupby` on input sorted by the item rather than by the key.
4. Check `if groups[k]:` on a `defaultdict(list)`, then print `len(groups)` and the output list.
5. Group anagrams with `frozenset(word)` and test on `"aab"` and `"abb"`.
6. Group shifted strings without `% 26` and test on `"az"` and `"ba"`.

The singleton implementation drill

Write each version, then say what is wrong with it:

1. The naive check-then-create, and describe the race on a timeline.
2. Eager creation at import time.
3. A lock around the whole accessor.
4. Double-checked locking.
5. Overriding `__new__` — then set a field in `__init__`, call the class twice, and print the field.
6. A module-level object.
7. `@functools.cache` on a factory function.

Number 5 has a bug that produces no error. Say exactly what it is and what it would look like in production.

The singleton critique drill

For each situation, say whether a singleton is defensible, and give the deciding question:

1. A database connection pool.
2. An in-memory cache of user permissions.
3. A logging configuration read once at startup.
4. A rate limiter allowing 100 requests per second.
5. A stateless helper that formats currency.
6. Application settings, in a service that will soon be multi-tenant.
7. A metrics registry.
8. A clock, used by code that needs testing.

Two of the eight are defensible for the same reason, and two are indefensible for the same reason. Name both reasons.

The multi-process drill

Answer each with arithmetic:

1. A service runs 8 Gunicorn workers in 4 containers. How many instances of your singleton exist?
2. It is a 500 MB cache. What is your real memory requirement per node, and per deployment?
3. It is a rate limiter set to 100 requests per second. What is the actual system-wide limit?
4. Two of those caches disagree about a user's permissions. Describe the bug a user would report.
5. Where should the state have lived instead, and what does that cost?

The replacement drill

Take this class and remove the singleton without removing the uniqueness:

```
class ConnectionPool:
    _instance = None

    @classmethod
    def get_instance(cls) → "ConnectionPool":
        if cls._instance is None:
            cls._instance = ConnectionPool()
        return cls._instance

class OrderService:
    def place(self, order):
        pool = ConnectionPool.get_instance()
        ...
```

1. Rewrite it so exactly one pool still exists, created in one place.
2. Count the lines you added.
3. Write the test for `OrderService.place` before and after. Count the setup lines in each.
4. Say what `OrderService.__init__`'s signature now tells a reader that it did not before.
5. Say which principle from [day 059](#) this satisfies, and which one the original violated.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Group these strings into anagram groups. What is your key?* The belonging sentence first, the key derived from it, both correctness tests run out loud, then the four-line loop, then the cost with n and k named — and the counting alternative with its real crossover rather than a recited $O(k)$.
2. *Implement a thread-safe singleton. Now tell me why I should not use it.* The two claims separated, the race described on a timeline, the three fixes with their costs, the Java memory-model footnote, what you would actually write in Python — and then the criticism volunteered, ending on injection at the composition root.

3. Why is `itertools.groupby` the wrong tool here? Adjacent runs only, the three different counts you get on unsorted, item-sorted and key-sorted input, and the $O(n \log n)$ you would pay for a job the dictionary does in $O(n)$.

Before you move on

- ☐ I said the belonging sentence out loud before writing every key today.
- ☐ I ran both tests — too coarse and too fine — on at least three different keys.
- ☐ I timed the sorted key against the count key at $k = 10$ and $k = 200$, and found the crossover.
- ☐ I broke grouping with an unhashable key and read the exact error.
- ☐ I ran `groupby` three ways and can explain all three group counts.
- ☐ I proved that reading a missing key from a `defaultdict(list)` adds an empty group to my output.
- ☐ I wrote double-checked locking and can say why Java's version was broken until Java 5.
- ☐ I found the `__new__` bug by setting a field in `__init__` and calling the class twice.
- ☐ I can compute how many singletons exist across 4 containers of 8 workers, and what that breaks.
- ☐ I rewrote a singleton as an injected dependency and counted the test setup lines both ways.
- ☐ I answered all three questions above out loud.

Hashing your own objects

DATA STRUCTURES AND ALGORITHMS

Hashing your own objects

You can make a custom class usable as a dictionary key without subtle bugs.

SYSTEM DESIGN

Factory and abstract factory

You can hide object creation behind a factory and say what that buys you.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Why did putting this object in a set not deduplicate it?*
- *How do you create the right notification object for each channel?*

1 What this is, and why they ask it

Everything you have put in a set or used as a dictionary key so far has been a number, a string or a tuple — things Python already knows how to hash. Today you write a class of your own and put it in a set. It will not work the way you expect, and the reason is a **contract** with exactly two rules that every language enforces the same way: **if two objects are equal, their hashes must be equal**, and **a key's hash must never change while it is in the table**.

They ask it because it is a bug you can watch happen. The interviewer creates two objects that are obviously the same order, puts both in a set, and the set has two elements in it. Or they put one in and then change a field and it becomes unfindable — `obj in d` returns `False`, with no error, while the object is sitting inside the dictionary. Both of those are three lines of setup and both catch out people who have used hash maps for years without ever asking what a key really has to be.

It is also where the interview crosses from data structures into design. `__eq__` is you deciding what “the same” means for your domain — two orders with the same id, or two orders with every field matching? — and `__hash__` has to agree with whatever you decided. Getting that pair right is the same skill as designing a grouping key in [day 064](#), and interviewers know it.

2 The story

Nagaraj has taken in washing on the same corner in Rajajinagar for twenty-six years. Two rooms, and a long steel rail along one wall with a hundred and forty hooks on it.

When his son took over the counter, he did it the way that seemed obvious. A bundle comes in, you hang it on the next empty hook, and you tell the person their hook number. Nothing was ever lost.

What it did instead was stranger. Mrs Fernandes came in on a Monday, again on a Thursday and again the following Tuesday, and by the end of the month she had four bundles on four hooks in four different parts of the rail. To the shop these were four unrelated things, because they arrived as four separate bundles. When she rang to ask whether her blue sari had come back, the only answer was to walk the whole rail.

Nagaraj came back and changed it in an afternoon. The hook is not the next free one any more. The hook is decided by the phone number — the last two digits tell you which hook, and that is that. Same person, same hook, every time, no matter how many times they come.

Now when Mrs Fernandes rings, he does not walk anything. Her number ends 31, so he goes to hook 31, and everything of hers is there together.

There is one thing he is careful about, and he learned it the hard way in about the second month.

A regular customer changed her phone number. She told him, and he was pleased to be told, and he updated it in his phone. What neither of them thought about was the bundle already hanging on hook 14 — hung there because her old number ended 14. Her new number ended 62. Three days later she came to collect, he went to hook 62, and there was nothing there. Her clothes were eleven feet away on a hook that, as far as the shop's rule was concerned, had nothing to do with her. He found them eventually, by going through the rail the old way, which took forty minutes.

His rule since then is short. The number the hook is chosen from does not change while the bundle is hanging. If somebody changes their number, the bundle comes off the rail first and goes back on afterwards.

3 The idea in plain English

The rail is a **hash table**. The hook is the **bucket**. The rule “last two digits of the phone number” is the **hash function**, and it is computed **from the value of the thing**, not from which physical bundle it happens to be.

The son's version — next free hook — is what Python does by default with your classes, and it is why your set has two elements in it when you expected one.

Why two identical objects are two different things

```
class Order:
    def __init__(self, order_id: str) → None:
        self.order_id = order_id

a = Order("A-1")
b = Order("A-1")

a == b          # False
len({a, b})     # 2
```

By default, Python gives every object an `__eq__` that means “is this literally the same object in memory?” and a `__hash__` derived from its memory address. Two `Order("A-1")` objects live at two addresses, so they are unequal and they hash differently. The set is behaving perfectly; it just does not know that you consider them the same order.

This is the answer to the interview question. “It did not deduplicate because the class does not define `__eq__` and `__hash__`, so Python compared identity, not value.”

The contract

Two rules. Every language has them; Java writes them as `equals` and `hashCode`.

Rule 1: if `a == b`, then `hash(a) == hash(b)`.

Rule 2: an object's hash must not change while it is a key.

Rule 1 is one-directional. Two unequal objects are *allowed* to share a hash — that is just a collision, which [day 061](#) says is normal and costs time, not correctness. What is not allowed is the reverse: two equal objects with different hashes. Then the lookup goes to the wrong hook and never finds the thing, and nothing raises.

Rule 2 is Mrs Fernandes's neighbour. The bundle is fine, the shop is fine, and the two have lost track of each other.

Writing the pair

```
class Order:
    def __init__(self, order_id: str, customer: str) → None:
        self.order_id = order_id
        self.customer = customer

    def __eq__(self, other: object) → bool:
        if not isinstance(other, Order):
            return NotImplemented
        return self.order_id == other.order_id

    def __hash__(self) → int:
        return hash(self.order_id)
```

Three things to notice, and each one is a question an interviewer might ask.

`isinstance` and `NotImplemented`. Returning `NotImplemented` — not `False` — lets Python try the other object's `__eq__` before giving up. `Order("A-1") == "A-1"` then correctly gives `False` instead of raising.

The same fields in both. `__eq__` uses `order_id`; `__hash__` uses `order_id`. If `__eq__` compared both fields and `__hash__` used only one, that would still be legal — equal objects would still hash the same. The other way round is the bug.

Hash a tuple when there are several fields. `hash((self.first, self.last))` — do not add them or combine them by hand. The tuple's own hash mixes them properly, and it is one line.

The thing that catches everybody

Defining `__eq__` and nothing else makes your class **unhashable**:

```
>>> class Q:
...     def __init__(self, n): self.n = n
...     def __eq__(self, other): return self.n == other.n
...
>>> {Q("a")}
```

Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
 TypeError: unhashable type: 'Q'

Python sets `__hash__ = None` the moment you define `__eq__`. It is being deliberately strict: you have just told it that equality means something new, and the inherited identity hash would now break rule 1. Rather than let you have a silent bug, it takes hashing away until you define it. **This is Python protecting you, and saying so out loud in an interview is a good line.**

Dataclasses, and the three settings

Most of the time you will not write any of this by hand.

```
from dataclasses import dataclass

@dataclass                                # eq=True by default → __hash__ = None
class R:
    n: str

{R("a")}
```

TypeError: unhashable type: 'R'

A plain `@dataclass` generates `__eq__` comparing all fields, and therefore sets `__hash__` to `None`, for exactly the reason above. Three ways out:

```
@dataclass(frozen=True)                  # immutable → gets __eq__ AND __hash__
class Point:
    x: int
    y: int

@dataclass(eq=False)                     # keeps identity equality and identity hash
class Node:
    value: int

@dataclass(unsafe_hash=True)              # generates __hash__ on a mutable class.
class Risky:                             # The name is a warning, not decoration.
    value: int
```

`frozen=True` is the one to reach for, and it is the right default for anything you intend to use as a key. It gives you both methods, and it makes rule 2 impossible to break because the fields cannot be reassigned.

`NamedTuple` — the answer when you want no ceremony

```
from typing import NamedTuple

class Point(NamedTuple):
    x: int
    y: int

{Point(1, 2), Point(1, 2)}          # {Point(x=1, y=2)} – one element
```

A `NamedTuple` is a tuple, so it is hashable and immutable for free, compares by value, and prints nicely. For coordinates, pairs and small value objects in an interview, this is two lines and no explanation needed.

What to hash on

The same judgement as designing a grouping key.

- **Hash on identity fields, not on everything.** An `Order` with an `order_id` should hash on the id. Whether the delivery address changed does not make it a different order.
- **Hash on fields that never change.** This is rule 2 in practice. An id, a timestamp of creation, a natural key — never a status, a count, or a list.
- **Never hash on a mutable container.** `hash(self.items)` where `items` is a list raises. Use `tuple(self.items)`, and only if the list genuinely never changes.
- **Cheap is important.** `__hash__` runs on every single lookup. Hashing a field that is a hundred-kilobyte string means every dictionary operation reads a hundred kilobytes.

4 The picture

The same two objects, before and after the pair is defined.

WITHOUT `__eq__` / `__hash__` (Python uses the memory address)

```
a = Order("A-1") at 0x7f3a01 → hash 0x7f3a01 % 8 → hook 1
b = Order("A-1") at 0x7f3b90 → hash 0x7f3b90 % 8 → hook 6
```

```
hook:  0    1    2    3    4    5    6    7
      +---+---+---+---+---+---+---+
      |   | a   |   |   |   |   | b   |   |   len(set) = 2
      +---+---+---+---+---+---+---+
```

WITH `__eq__` / `__hash__` on `order_id` (Python uses the value)

```
a = Order("A-1") → hash("A-1") % 8 → hook 3
b = Order("A-1") → hash("A-1") % 8 → hook 3, then a = b → replace
```

```
hook:  0    1    2    3    4    5    6    7
      +---+---+---+---+---+---+---+
      |   |   |   | a   |   |   |   |   |   len(set) = 1
      +---+---+---+---+---+---+---+
```

What to notice: nothing about the set changed. The set was always going to put the object where the hash said. All that changed is what the hash is computed *from* — the address, or the value.

Now rule 2, drawn. This is the bug you cannot see:

```
step 1  d[order] = "paid"      hash("A-1") % 8 = 3
        hook: 3 → [ order ]
```

```
step 2  order.order_id = "A-2"  the object is still on hook 3
```

```
step 3  order in d ?          hash("A-2") % 8 = 6
        hook: 6 → empty      → False
```

```
hook:  0    1    2    3    4    5    6    7
      +---+---+---+---+---+---+---+
      |   |   |   | order |   |   | empty |   |
      +---+---+---+---+---+---+---+
                ^               ^
            it is here      we look here
```

`len(d)` still says 1. Iterating the dictionary still yields the order. Asking for it returns `False`. No exception anywhere. That is the worst failure mode a data structure has, and it is why rule 2 exists.

5 The code, built step by step

Step 1 — watch it fail

```
class Order:
    def __init__(self, order_id: str) → None:
        self.order_id = order_id

print(len({Order("A-1"), Order("A-1")}))    # 2
print(Order("A-1") == Order("A-1"))        # False
```

Run this before you fix anything. The two lines of output are the whole interview question.

Step 2 — add `__eq__` alone, and watch it fail differently

```
def __eq__(self, other: object) → bool:
    if not isinstance(other, Order):
        return NotImplemented
    return self.order_id == other.order_id
```

```
>>> {Order("A-1")}
TypeError: unhashable type: 'Order'
```

Equality now works and hashing is gone. Say why: defining `__eq__` sets `__hash__` to `None`, because the inherited address-based hash would now break rule 1.

Step 3 — add `__hash__` on the same field

```
def __hash__(self) → int:
    return hash(self.order_id)
```

```
>>> len({Order("A-1"), Order("A-1")})
1
```

One field, one `hash()` call, and the whole thing works. When there are several fields, put them in a tuple:

```
def __hash__(self) → int:
    return hash((self.order_id, self.customer))
```

Step 4 — make rule 2 impossible to break

The version above still lets somebody write `order.order_id = "A-2"` while it sits in a dictionary. Freeze it:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Order:
    order_id: str
    customer: str
```

Four lines, both methods generated, and assignment now raises:

```
>>> order = Order("A-1", "fernandes")
>>> order.order_id = "A-2"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
dataclasses.FrozenInstanceError: cannot assign to field 'order_id'
```

An error at the moment of the mistake, instead of a silent wrong answer later. That is the trade `frozen=True` buys, and it is the sentence to say.

Step 5 — when the identity is one field but equality is all of them

Real domains often want: two orders are the same order if the ids match, whatever else differs. A frozen dataclass compares every field, which is not that. Write it by hand:

```
@dataclass(frozen=True, eq=False)      # keep frozen; write our own equality
class Order:
    order_id: str
    customer: str
    total_paise: int

    def __eq__(self, other: object) → bool:
        if not isinstance(other, Order):
            return NotImplemented
        return self.order_id == other.order_id

    def __hash__(self) → int:
        return hash(self.order_id)
```

Now two orders with the same id and different totals are equal and hash together, which is what the business means by “the same order”. `eq=False` stops the dataclass generating an `__eq__` that would override yours, and it also stops it blanking `__hash__`.

The complete solution

```
from dataclasses import dataclass, FrozenInstanceError
from typing import NamedTuple

@dataclass(frozen=True)
class Point:
    """The default choice for a value object used as a key.

    frozen=True gives __eq__ and __hash__ over all fields, and makes rule 2
    unbreakable because the fields cannot be reassigned.
    """
    x: int
    y: int

class Cell(NamedTuple):
    """The no-ceremony version: a tuple, so hashable and immutable for free."""
    row: int
    column: int

@dataclass(frozen=True, eq=False)
class Order:
    """Identity is the id alone; the other fields are data about it."""
    order_id: str
    customer: str
    total_paise: int

    def __eq__(self, other: object) → bool:
        if not isinstance(other, Order):
            return NotImplemented
        return self.order_id == other.order_id

    def __hash__(self) → int:
        return hash(self.order_id)

class Mutable:
    """The hand-written pair, for when a dataclass is not available.
    Note that nothing here stops rule 2 being broken."""

    def __init__(self, key: str, payload: str) → None:
        self.key = key
        self.payload = payload

    def __eq__(self, other: object) → bool:
        if not isinstance(other, Mutable):
            return NotImplemented
        return self.key == other.key

    def __hash__(self) → int:
        return hash(self.key)

    def __repr__(self) → str:
        return f"Mutable({self.key!r}, {self.payload!r})"

if __name__ == "__main__":
    print(len({Point(1, 2), Point(1, 2)}})          # 1
    print(len({Cell(0, 0), Cell(0, 0), Cell(0, 1)}}) # 2

    a = Order("A-1", "fernandes", 45000)
```



```

b = Order("A-1", "fernandes", 99999)
print(a == b, len({a, b}))           # True 1

try:
    Point(1, 2).x = 5
except FrozenInstanceError as error:
    print(type(error).__name__, error)
    # FrozenInstanceError cannot assign to field 'x'

# Rule 2, broken on purpose, so you can see the silence.
lookup = {Mutable("A-1", "paid"): "receipt-9"}
key = next(iter(lookup))
key.key = "A-2"
print(key in lookup, len(lookup), list(lookup))
# False 1 [Mutable('A-2', 'paid')]

```

Read that last line of output twice. The dictionary contains the key. `len` says one. Asking for it says `False`.

6 What it costs

What `__hash__` costs, per operation

`__hash__` runs on every insert, every lookup, every `in`. Not once — every time. So its cost is multiplied by the number of dictionary operations in your whole program.

<code>hash(self.order_id)</code>	reads a ~10-char string	~ 40 ns
<code>hash((self.a, self.b, self.c))</code>	builds a 3-tuple, hashes	~ 120 ns
<code>hash(self.description)</code>	reads a 100 KB string	~ 8,000 ns
<code>hash(tuple(self.items))</code>	builds a 10,000-list	~ 200,000 ns

That last one is the disaster. Building a tuple from a ten-thousand-element list on every lookup turns an $O(1)$ dictionary into something that reads ten thousand elements per operation. A million lookups is 10^{10} element reads. **Hash the id, not the contents.**

Python does cache the hash of a `str` after computing it once, which is why string keys are fast. It does not cache yours.

What a bad `__hash__` costs

Suppose you write this, because it seems reasonable:

```

def __hash__(self) → int:
    return hash(self.customer)      # thousands of orders per customer

```

It satisfies rule 1 only if `__eq__` also uses just `customer` — but suppose `__eq__` uses `order_id`. Then two unequal orders share a hash, which is legal, and everything still works. It just gets slow, in the way [day 061](#) described:

```
100,000 orders, 500 customers
→ average chain length 100,000 / 500 = 200
→ a lookup compares ~200 keys instead of ~1
→ 100,000 lookups: 20,000,000 comparisons instead of 100,000
```

Two hundred times slower, with no error and no wrong answers. This is the difference between an $O(1)$ map and an $O(n)$ one, produced by one plausible line.

And the extreme version, `return 1`, is legal and turns the dictionary into a list: building it is $O(n^2)$, which at $n = 20,000$ is 400,000,000 comparisons.

What the fixes cost

<code>@dataclass(frozen=True)</code>	0 extra lines beyond the class, ~0 runtime cost
<code>NamedTuple</code>	0 extra lines, and slightly FASTER than a dataclass because tuple hashing is implemented in C
hand-written pair	~8 lines, and the risk of getting rule 1 wrong

There is no performance argument for writing the pair by hand. Write it by hand only when equality means something the generated version does not.

7 The traps

Trap 1 — `__eq__` without `__hash__`

```
>>> class Q:
...     def __init__(self, n): self.n = n
...     def __eq__(self, other): return self.n == other.n
...
>>> {Q("a")}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'Q'
>>> Q.__hash__ is None
True
```

The fix is to add `__hash__`, not to delete `__eq__`. And note what the error does *not* say — it does not mention `__eq__` at all, so the connection is not obvious the first time.

Trap 2 — the plain `@dataclass` is unhashable

```
>>> from dataclasses import dataclass
>>> @dataclass
... class R:
...     n: str
...
>>> {R("a")}
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'R'
```

Exactly the same cause — the generated `__eq__` blanks `__hash__` — and it surprises people much more because they did not write an `__eq__`. `frozen=True` is the fix.

Trap 3 — mutating a key while it is in the table

This is the one with no error at all, so it is the one worth doing yourself once:

```
>>> lookup = {Mutable("A-1", "paid"): "receipt-9"}
>>> key = next(iter(lookup))
>>> key.key = "A-2"
>>> key in lookup
False
>>> len(lookup)
1
>>> list(lookup)
[Mutable('A-2', 'paid')]
```

The object is in there. `len` counts it. Iterating yields it. `in` says no. Nothing raises, nothing logs, and the only symptom is that some record has quietly stopped being found.

`frozen=True` makes this impossible, which is the entire argument for it.

Trap 4 — `unsafe_hash=True` used to shut the error up

```
@dataclass(unsafe_hash=True)
class Risky:
    value: int
```

This generates a `__hash__` on a mutable class. It compiles, the `TypeError` goes away, and you have just re-enabled trap 3. The keyword is called `unsafe_hash` because the authors wanted you to have to type the word “unsafe”. If you find yourself reaching for it, the question is why the object is mutable.

Trap 5 — `__eq__` that raises on a different type

```
def __eq__(self, other):
    return self.n == other.n          # no isinstance check
```

```
>>> Q("a") == "a"
AttributeError: 'str' object has no attribute 'n'
```

Equality should never raise. Anything can be compared to anything in Python — including by `in`, which compares your object against whatever is in the container. Guard with `isinstance` and return `NotImplemented`.

Trap 6 — hashing on a field that is a list

```
def __hash__(self):
    return hash(self.items)          # items is a list
```

```
TypeError: unhashable type: 'list'
```

`tuple(self.items)` fixes the error and introduces two new problems: it is $O(\text{len}(\text{items}))$ on every lookup, and if the list ever changes you are back in trap 3. If a collection is really part of the identity, use a `frozenset` or a `tuple` field from the start and never a list.

Trap 7 — inheritance breaking symmetry

```
@dataclass(frozen=True)
class Point:
    x: int
    y: int

@dataclass(frozen=True)
class Point3D(Point):
    z: int
```

`Point(1, 2) == Point3D(1, 2, 3)` is `False`, and `Point3D(1, 2, 3) == Point(1, 2)` is `False` too, because the dataclass `__eq__` compares `other.__class__` is `self.__class__`. That is symmetric and correct. But a hand-written `isinstance`-based `__eq__` is *not* symmetric under inheritance — the parent would accept the child and the child would reject the parent, so `a == b` and `b == a` could disagree, and a set would then behave differently depending on insertion order. Prefer exact-class comparison when inheritance is in play, and say why.

8 In the interview

How it gets asked

- The demonstration: “*I put these two objects in a set and got two elements. Why?*” Three lines on the screen, and the whole answer is identity versus value equality.
- The contract: “*What is the relationship between `equals` and `hashCode`?*” — the Java phrasing, asked at every company that hires Java engineers, and the expected answer is rule 1 plus the fact that the converse does not hold.
- The bug hunt: “*This object is in the map but `containsKey` returns false. What happened?*” Rule 2, every time.
- The design version: “*Make this class usable as a dictionary key.*” Where the real question is *which fields*, not *which methods*.

What to say out loud, in the first ninety seconds

1. **Name the default before you fix it.** “By default Python compares objects by identity, not by value, and the default hash comes from the memory address. So two objects with identical fields are two different keys, and the set is right.”

2. **State the contract as two rules.** “Equal objects must have equal hashes; unequal objects may share a hash and that is just a collision. And the hash must not change while the object is a key.”
3. **Say which fields, and why.** “Identity here is the order id. The customer name and the total are data about the order, not what makes it that order. So both methods use the id.”
4. **Say the immutability point without being asked.** “I would make it frozen, so the field cannot be reassigned while it is a key. That converts a silent wrong answer into an error at the moment of the mistake.”
5. **Then write four lines.** `@dataclass(frozen=True)` and the fields.

The follow-ups

“Can two unequal objects have the same hash?” “Yes, and they must be allowed to — there are more possible values than hash values, so collisions are unavoidable. A collision costs time, not correctness: the table compares with `__eq__` after finding the bucket. What is forbidden is the reverse, two equal objects with different hashes, because then the lookup goes to the wrong bucket and finds nothing, silently.”

“What happens if I mutate a key?” “The object stays in the bucket it was placed in, but lookups now compute a different bucket, so it becomes unreachable. `len` still counts it and iteration still yields it — `in` just returns `False`. No exception. That is why I freeze anything used as a key.”

“Why does defining `__eq__` remove hashing?” “Because Python knows the inherited hash is now wrong. You have redefined equality by value while the hash is still by address, which breaks rule 1 immediately. Rather than let that be a silent bug, it sets `__hash__` to `None` so you get a `TypeError` the first time you try. Java does not do this, which is exactly why ‘always override `hashCode` when you override `equals`’ is such a well-worn piece of Java advice.”

“What would you hash on for a large object?” “The identity fields, and cheap ones. `__hash__` runs on every lookup, so hashing a hundred-kilobyte description field means every dictionary operation reads a hundred kilobytes. And I would never hash a mutable collection — building a tuple from a ten-thousand-element list on every lookup turns an $O(1)$ map into something that reads ten thousand items per operation.”

“How is this the same as yesterday’s grouping key?” “It is the same question. `__eq__` decides what makes two objects the same, and a grouping key decides what makes two items belong in the same pile. Both fail the same two ways — too coarse and things merge that should not, too fine and one thing splits into many. `__hash__` just has to agree with whatever `__eq__` decided.”

A model answer

Asked: *why did putting this object in a set not deduplicate it?*

“Because the class does not define `__eq__` and `__hash__`, so Python is using its defaults, and the defaults are about identity rather than value. The default `__eq__` asks ‘is this literally the same object in memory’, and the default hash comes from the address. Two `Order('A-1')` objects are at two addresses, so they are unequal and they hash to different buckets. The set is behaving correctly — it does not know that I consider them the same order.

To fix it I have to define both, and they have to agree, because the contract is that equal objects must have equal hashes. The converse is not required — two unequal objects sharing a hash is just a collision, which costs a comparison and not correctness.

The design question is which fields. Here the identity of an order is its id; the customer and the total are facts about the order rather than what makes it that order. So `__eq__` compares ids and `__hash__` hashes the id. If several fields made up the identity I would hash a tuple of them and let the tuple do the mixing.

There is a second rule that matters more than people expect: the hash must not change while the object is a key. If I put an order in a dictionary and then reassign its id, the object stays in the bucket it was filed under, but every lookup computes a different bucket. So `in` returns False while `len` still counts it and iteration still yields it, with no exception at all. That is the worst failure a data structure has.

So in practice I would write `@dataclass(frozen=True)` and list the fields. That generates both methods and makes the second rule unbreakable, because assignment raises `FrozenInstanceError` instead of quietly corrupting the table. For a small pair like a coordinate I would use a `NamedTuple`, which is hashable and immutable for free and is slightly faster because tuple hashing is in C.

And one thing I would flag: a plain `@dataclass` is unhashable, because the generated `__eq__` causes Python to set `__hash__` to `None`. There is an `unsafe_hash=True` option that makes the error go away, and I would avoid it — it re-enables exactly the mutation bug, and the keyword is named ‘unsafe’ on purpose.”

9 Recall card

- **The default is identity, not value.** No `__eq__` / `__hash__` means Python compares memory addresses, so `len({Order("A-1"), Order("A-1")}) = 2`. That one sentence is the whole interview question.
- **Two rules.** `a == b ⇒ hash(a) == hash(b)`, and the hash must not change while the object is a key. The converse of rule 1 is *not* required — unequal objects sharing a hash is a collision, which costs time, never correctness.
- **Breaking rule 2 is silent.** Mutate a key in place and `x in d` is `False` while `len(d)` is 1 and iteration still yields it. No exception, ever. `@dataclass(frozen=True)` makes it impossible — `FrozenInstanceError` at the moment of the mistake instead of a wrong

answer later.

- **Defining `__eq__` sets `__hash__ = None`**, so a plain `@dataclass` raises `TypeError: unhashable type`. Fixes: `frozen=True` (the default choice) · `NamedTuple` (free, and faster — tuple hashing is C) · `eq=False` to keep identity · `unsafe_hash=True` **never**, it just re-enables the bug.
- **Hash the identity fields, cheaply.** `__hash__` runs on *every* lookup: a 100 KB field is 8 μ s per operation; `tuple(self.items)` on a 10,000-list is 200 μ s. And a too-coarse hash (customer instead of order id) gives 200-long chains — 200 $\times$ slower with no error. Combine fields with `hash((a, b))`, and always guard `__eq__` with `isinstance` returning `NotImplemented`.

1 What this is, and why they ask it

A **factory** is code whose job is to decide which concrete class to build and to hand back something the caller can use without knowing which one it got. It exists because `EmailSender()` written in the middle of your business logic ties that logic permanently to email. An **abstract factory** goes one step further: it hands back a whole *family* of matched objects, so you cannot accidentally mix parts from two families.

They ask it because it is the pattern that tests whether you understand the difference between “where an object comes from” and “what an object does” — and because there are three different things in common use that people all call “factory”, and being able to separate them immediately marks you out. There is the **simple factory**, which is not in the Gang of Four book at all and is what nearly everyone actually writes; **factory method**, which uses subclassing; and **abstract factory**, which is about families. Candidates who say “a factory creates objects” and stop have given a correct and almost worthless answer.

The second reason is that factories are the most over-applied pattern after singleton. A codebase with fourteen classes ending in `Factory`, nine of which have one implementation, is extremely common. An interviewer wants to hear that you know when *not* to write one.

2 The story

The uniform shop opposite the bus stand in Malleswaram has been there longer than most of the schools it supplies. One room, floor to ceiling shelves, and a man called Iqbal behind a wooden counter with a groove worn into it where he leans.

A parent comes in with a child in June. They do not say what they want in the way you might expect. They say: third standard, girl, Saint Anthony’s.

That is the whole conversation. They do not choose the colour, the fabric, the collar, whether the socks are ankle or knee, or which of the four shades of blue is right. They would not know. Iqbal knows.

He turns round, and what comes back over the counter is a set — the shirt, the pinafore, the tie, the socks, the belt with the school’s little metal clasp. Everything matched, everything from the same school’s specification, because the specification is a real thing that the school sends him every year and he keeps on the shelf with everything else.

The matching is not a nicety. Saint Anthony’s blue and Vidya Mandir blue look identical in the shop and completely different in daylight next to each other. A child who turns up in a Saint Anthony’s shirt and a Vidya Mandir tie gets sent to the office. Iqbal has seen it happen

twice, both times because a parent bought half a set somewhere else.

Two things about how he runs it are worth noticing.

The first is that when a new school opened in Mathikere three years ago, nothing about the counter changed. He learned one more specification, put one more set of things on one more shelf, and the conversation with a parent is word for word what it always was. He did not have to change how he serves anybody else.

The second is the thing that genuinely costs him. Two years ago the schools started requiring a blazer. Not one school — all of them. And that meant going back to every single specification he holds, one at a time, and working out what that school's blazer is: the colour, the buttons, the badge, whether it is worn in December only. Eleven schools, eleven separate pieces of work, and until he had done all eleven he could not tell a parent he stocked blazers. Adding a school is easy. Adding a *garment* is not.

3 The idea in plain English

Iqbal's counter is a **factory**. The parent asks for a thing by what they need — third standard, girl, Saint Anthony's — and gets back something usable without ever naming a product. The knowledge of which exact product that is lives in one place, behind the counter.

The matched set is an **abstract factory**. The unit is not a shirt; it is a *family* of things that must be consistent with each other. And the mismatched tie is exactly the bug that abstract factory exists to make impossible.

The two things Iqbal noticed are the two halves of the trade, and you should be able to say both.

Adding a school changed nothing at the counter. That is open/closed from [day 056](#): a new family arrives as new code, and the calling code is untouched.

Adding a blazer meant editing all eleven specifications. That is the known weakness of abstract factory, it has been known since 1994, and volunteering it in an interview is worth a great deal. A new *family* is cheap. A new *product type* is expensive.

The three things people call “factory”

This distinction is the lesson. Get it clear and the rest is detail.

One — the simple factory. A function, or a static method, that takes some input and returns the right concrete object. It is not one of the 23 patterns. It is what nearly everybody writes and it is usually correct.

```
def make_sender(channel: str) → Sender:
    if channel == "email":
        return EmailSender()
    if channel == "sms":
        return SmsSender()
    if channel == "push":
        return PushSender()
    raise ValueError(channel)
```

The `if` chain has not disappeared from the world; it has been moved into one place that every caller shares. That is the entire benefit, and it is a real one — the branch existed in nine call sites before and exists in one now.

Two — factory method. A method on a class that subclasses override to decide what gets created. The parent class contains the algorithm; the subclass supplies the product.

```
class Dialog:
    def render(self) → None:
        button = self.create_button()      # the factory method
        button.draw()

    def create_button(self) → Button:      # subclasses decide
        raise NotImplementedError

class WindowsDialog(Dialog):
    def create_button(self) → Button:
        return WindowsButton()
```

`Dialog.render` never learns which button it has. This is the GoF pattern, and in Python you would usually pass a function instead of subclassing, which is the same idea with less ceremony.

Three — abstract factory. An object that creates a whole family of related products, so the family cannot be mixed.

```
class UiFactory(Protocol):
    def create_button(self) → Button: ...
    def create_checkbox(self) → Checkbox: ...
    def create_menu(self) → Menu: ...

class WindowsFactory:
    def create_button(self) → Button: return WindowsButton()
    def create_checkbox(self) → Checkbox: return WindowsCheckbox()
    def create_menu(self) → Menu: return WindowsMenu()
```

The caller holds one `UiFactory` and asks it for everything. It is now impossible to end up with a Windows button next to a Mac checkbox, because the caller never chooses either.

The Python version of a simple factory: a registry

The `if` chain still means editing one file per new channel. A dictionary removes even that:

```

SENDERS: dict[str, type[Sender]] = {
    "email": EmailSender,
    "sms": SmsSender,
    "push": PushSender,
}

def make_sender(channel: str) → Sender:
    try:
        return SENDERS[channel]()
    except KeyError:
        raise ValueError(f"unknown channel {channel!r}") from None

```

Classes are objects in Python, so a class can sit in a dictionary and be called. Adding WhatsApp is one new class and one new line. This is the version to write in an interview, and the `try / except` matters — a bare `SENDERS[channel]()` leaks a `KeyError` that says nothing useful to the caller.

The naming rule

Call the thing after what it does in your domain, not after the pattern. `NotificationSenderFactory` is tolerable. `AbstractSenderFactoryProvider` is a codebase that stopped thinking. And a factory is a good place for a **named constructor** instead, which is often nicer: `Sender.for_channel("sms")`, `datetime.fromisoformat(...)`, `Path.home()`.

4 The picture

The three factories side by side, so the difference is visible rather than described.

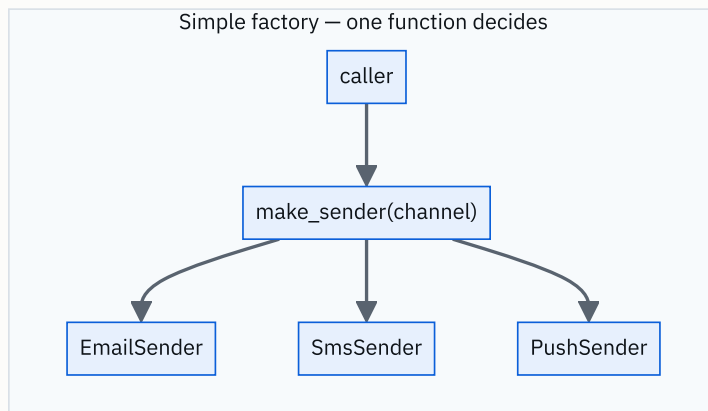
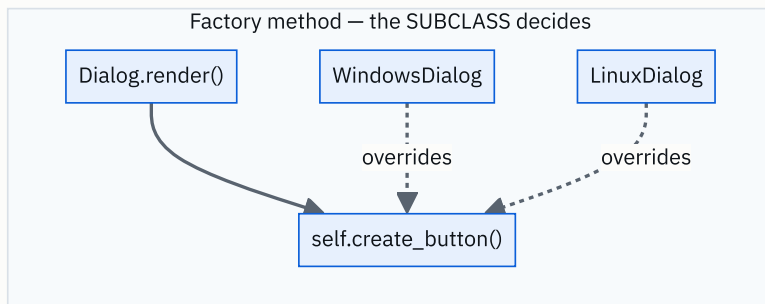
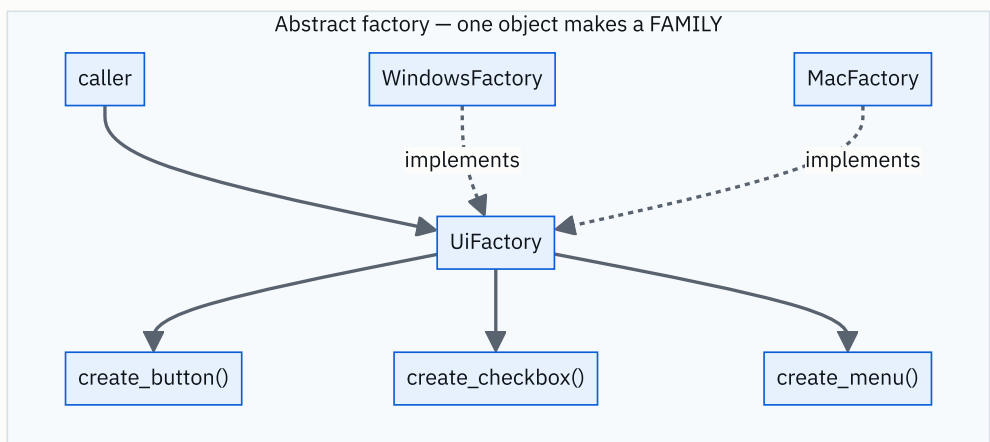


FIGURE 65.1

What to notice: in the first, the decision is *data* passed in at the call. In the second, the decision is *which subclass you are*. In the third, the decision was made once when somebody chose the factory, and after that consistency is guaranteed for free.

And here is the cost of abstract factory, drawn as a grid, because this is what “adding a product type is expensive” actually means:

	create_button	create_checkbox	create_menu	create_slider
WindowsFactory	yes	yes	yes	← NEW
MacFactory	yes	yes	yes	← NEW
LinuxFactory	yes	yes	yes	← NEW
WebFactory	yes	yes	yes	← NEW
Adding a ROW (a new family)		: 1 new file, 0 edits.		Cheap.
Adding a COLUMN (a new type)		: edit the interface		
		+ every single factory.		Expensive.

Iqbal adding a school is a row. Iqbal adding a blazer is a column — eleven edits before he could serve anybody.

5 How it actually works

What the factory actually removes

Before, the branch is everywhere:

```
# orders/checkout.py
if user.prefers_sms:
    sender = SmsSender(api_key=settings.TWILIO_KEY)
else:
    sender = EmailSender(host=settings.SMTP_HOST)

# billing/invoices.py
if user.prefers_sms:
    sender = SmsSender(api_key=settings.TWILIO_KEY)
...
# the same branch again
```

Nine call sites, each knowing the concrete classes *and* how to construct them — which key, which host, which timeout. Add WhatsApp and you edit nine files, and one of them will be missed.

After, one place knows:

```
# notifications/factory.py – the only file that names concrete senders
def sender_for(user: User) → Sender:
    return SENDERS[user.preferred_channel]()

# everywhere else
sender_for(user).send(message)
```

Three things moved, and all three are worth naming:

1. **The branch** — from nine places to one.
2. **The construction details** — API keys, hosts, timeouts, retry settings — out of business logic.

3. **The import** — `orders/checkout.py` no longer imports `twilio`. That is dependency inversion from [day 059](#), and `grep -rn "twilio" orders/` returning nothing is the proof.

The registry that removes even the one edit

```
SENDERS: dict[str, type[Sender]] = {}

def register(channel: str):
    def decorator(cls: type[Sender]) → type[Sender]:
        SENDERS[channel] = cls
        return cls
    return decorator

@register("whatsapp")
class WhatsAppSender:
    def send(self, message: Message) → None: ...
```

Now a new channel is *one new file* and zero edits anywhere. The cost, and say it: the list of active channels is no longer visible in one place — you have to know that the modules get imported, and if one is not imported it silently does not exist. This is exactly the trade Django’s app registry and pytest’s plugin system make.

Real products, and which of the three they are

- `logging.getLogger("app.orders")` — a simple factory, and a cache: same name, same object.
- `datetime.fromisoformat`, `Path.home()`, `dict.fromkeys` — named constructors, the lightest form. Python prefers these to `Factory` classes and so should you.
- `sqlalchemy.create_engine("postgresql:// ...")` — a simple factory that parses the URL and returns a driver-specific engine. The caller never names a driver.
- `boto3.session.client("s3")` — a registry-based factory; the string picks the class.
- `ssl.create_default_context()` — a factory whose whole value is the safe defaults it applies.
- `javax.xml.parsers.DocumentBuilderFactory` — the textbook abstract factory, and the textbook example of how heavy they get.
- Django’s `DEFAULT_FILE_STORAGE` **setting** — a string in configuration naming a class, which the framework instantiates. A factory whose registry is your settings file.
- Spring’s `BeanFactory` — the name is literal; the container is a factory that reads configuration and builds the object graph.
- `java.util.Calendar.getInstance()` — a factory that returns a locale-appropriate subclass, and a warning: the name looks like a singleton accessor and is not one.

Abstract factory in the wild

The honest answer is that it is rare in application code and common in frameworks and cross-platform libraries. The real cases share one property: **there is a genuine family that must not be mixed.**

- Cross-platform UI toolkits — a Windows button with a Mac scrollbar is a real bug.
- Database access layers where connection, cursor, transaction and dialect must all come from the same driver.
- Test versus production infrastructure: `ProductionFactory` gives real Postgres, real S3, real Stripe; `TestFactory` gives in-memory versions of all three. Getting a real Stripe client with an in-memory database is exactly the mismatch you want to make impossible.

If you cannot name the family, you do not have an abstract factory — you have several simple factories and should say so.

6 The numbers

Adding the fourth channel

```
branch at every call site:
  files edited          9
  places to miss        9
  existing tests re-run ~55
simple factory (if chain):
  files edited          1
  existing tests re-run ~6
registry factory:
  files added           1
  lines edited          1
decorator registry:
  files added           1
  lines edited          0
```

And the flat-cost point, which is the actual product: the eighth channel costs what the second cost. With the branch at nine call sites, the eighth channel is edited into nine files that are each now longer and less well understood.

What the abstract-factory grid costs

Four families, three product types:

```
adding a family (a row):    1 new file, ~3 methods, 0 edits elsewhere
adding a product type (col): 1 interface edit + 4 factory edits + 4 new
                             product classes = 9 files
```

That is the asymmetry to quote. If your product types are stable and your families keep arriving, abstract factory is excellent. If new product types arrive as often as new families, it is a tax on every one of them.

Concretely: Iqbal's eleven schools and the blazer. Adding the twelfth school is one specification. Adding the blazer was eleven pieces of work before he could serve a single parent.

What the ceremony costs

```
if/elif in one function      1 file, ~10 lines
dict registry               1 file, ~8 lines
abstract factory, 4 families 5 files, ~90 lines
    (1 interface + 4 factories), before any product classes
```

So a full abstract factory over four families costs roughly **80 extra lines and 4 extra files** over a registry — the same order as the strategy numbers from [day 063](#). Pay it when the family-consistency guarantee is worth something. Do not pay it because “factory” sounds architectural.

What over-application costs, measured

A shape you will meet in a real codebase:

```
classes ending in "Factory"      14
with exactly one implementation    9
lines of pure indirection in those 9 ~200
files a reader must open to find the real
    class, per lookup              2
```

Those nine are `new Thing()` with two extra files in front of it, and they cost every new engineer a redirection that leads nowhere. The test is the one from [day 063](#): **name the second implementation**. A test double counts. A hypothetical does not.

The one that justifies it

Swapping the SMS provider from Twilio to a local aggregator:

```
with construction inline: 38 references across 14 files, ~2 weeks
with a factory:           1 new class + 1 registry line, ~1 hour
```

7 The trade-offs

What you give up

A redirection. To find out what actually runs, a reader opens the factory, reads the registry, and then opens the class. Two hops instead of zero. In a codebase with fourteen factories, that is a tax on every piece of reading anyone does.

The failure moves to run time. `make_sender("emial")` is a typo the compiler cannot catch. With `EmailSender()` written inline, a misspelling is an import error at startup. With a registry keyed on strings, it is a `ValueError` when a user with that preference finally logs in. Mitigate it with an enum, a `Literal` type, or validation at the boundary — and say that you would.

Discoverability, with the decorator registry. The set of live channels is no longer written anywhere. If a module is not imported, its channel silently does not exist. This is a real class of bug in plugin systems and it is worth naming.

Weight, with abstract factory. It is the heaviest of the creational patterns. Interfaces, implementations per family, product classes per cell of the grid. Most applications never need it.

The specific weakness of abstract factory

Say this without being asked: **adding a new product type requires editing the interface and every existing factory.** The pattern is open to new families and closed to new product types. That is a choice, it was a deliberate one, and it is the right one only when families change more often than types.

“I would not use this if...”

- **...there is one implementation and no test double.** Call the constructor. Nine factories with one implementation each are 200 lines of costume.
- **...the variation is data rather than behaviour.** Three tax rates are a dictionary of numbers, not three classes behind a factory. This is the mistake that produces most of the fourteen.
- **...the object is trivial to construct.** A factory that only calls a constructor with no arguments has added a file and removed nothing.
- **...a named constructor would do.** `Sender.for_channel("sms")` is one classmethod and reads better than a separate factory class.
- **...I cannot name the family.** Then it is not an abstract factory, whatever it is called.

The honest defence

Factories earn their keep for one reason more than any other, and it is not flexibility: **they are the seam where the concrete world enters your program.** API keys, hosts, drivers, credentials, timeouts. Keeping all of that in one file, out of your domain logic, is worth the redirection even if you only ever have one implementation — because the alternative is `twilio` imported in `orders/checkout.py`.

8 In the interview

How it gets asked

- The applied one: “*How do you create the right notification object for each channel?*” — email, SMS, push, WhatsApp. The expected answer is a factory; the marks are in which kind and why.
- The distinction: “*What is the difference between factory method and abstract factory?*” Asked constantly, and answered badly nearly as often. One uses subclassing to pick a product; the other produces a family.

- The trap: “*Is simple factory a design pattern?*” No — it is not in the Gang of Four book. Knowing that, and saying it is still the right thing to write most of the time, is the good answer.
- The design version: “*We are adding a fourth payment provider. Walk me through it.*” Where a factory is half the answer and an adapter (day 068) is the other.

What to say out loud, in the first ninety seconds

1. **Separate the three immediately.** “There are three things called factory: a simple factory, which is a function with a branch and is not a GoF pattern; factory method, where a subclass decides the product; and abstract factory, which produces a family. I think you want the first here, and I will say why.”
2. **Name the pain the factory removes.** “Right now the choice of sender is made at nine call sites, and each one also knows the API key and the host. Adding WhatsApp is nine edits and one of them gets missed.”
3. **Write the registry, not the `if` chain.** Six lines. Say “a class is an object in Python, so it can live in a dictionary.”
4. **Say what moved.** “Three things left the business logic: the branch, the construction details, and the import of the vendor SDK. `grep twilio orders/` now returns nothing.”
5. **Volunteer the cost.** “What I have given up is a compile-time check — `'emial'` is now a runtime error, so I would type the channel as an enum rather than a bare string.”

The follow-ups

“**What is the difference between factory method and abstract factory?**” “Factory method is one product, chosen by subclassing — the base class has a `create_x` that subclasses override, and the base class’s algorithm never learns which product it got. Abstract factory is a family of related products behind one object, so you cannot mix a Windows button with a Mac checkbox. Different axes: one is about *who decides*, the other is about *how many things must agree*.”

“**When would you use abstract factory?**” “When there is a genuine family that must be consistent, and mixing members is a real bug. Cross-platform UI toolkits are the classic. The one I have found genuinely useful is test versus production infrastructure: one factory gives real Postgres, real S3 and real Stripe, and the other gives in-memory versions of all three. Getting a live Stripe client with an in-memory database is exactly the mismatch it makes impossible. If I cannot name the family, I do not have one.”

“**What is the drawback of abstract factory?**” “Adding a new product type means editing the interface and every implementing factory. It is open to new families and closed to new product types. So it is the right choice only when families arrive more often than types, and I would check that before choosing it.”

“**Is a factory not just a glorified constructor?**” “Sometimes, and when it is I would delete it. It earns its place when one of three things is true: the concrete class depends on runtime input; construction needs knowledge the caller should not have, like credentials; or I need a seam for testing. If none of those hold, `Thing()` is better than `ThingFactory().create()`.”

“How do you avoid the string-typo problem?” “Type the key. An enum, or `Literal['email', 'sms', 'push']`, so the type checker catches it. And validate at the system boundary where the channel enters — from the database or the request — so a bad value fails at the edge rather than deep in a send path at two in the morning.”

A model answer

Asked: *how do you create the right notification object for each channel?*

“First, what is wrong today. The choice between email, SMS and push is being made at each call site, and each of those sites also knows how to construct the thing — the Twilio key, the SMTP host, the timeouts. So adding WhatsApp means editing nine files, `orders/checkout.py` imports `twilio`, and testing the checkout flow needs a way to stop real messages going out.

What I want is one place that knows which concrete sender to build, and callers that only know the interface. I would define a `Sender` protocol with one method, `send`, and then a registry mapping a channel to a class. In Python a class is an object, so it can live in a dictionary — the factory is then one lookup and one call, about six lines. Callers become `sender_for(user).send(message)`.

That is a simple factory. I want to be precise: it is not one of the twenty-three Gang of Four patterns. Factory method is the GoF one where a subclass overrides a creation method to decide the product, and abstract factory produces a whole family of related objects. For this problem there is no family and no subclassing — the simple factory is the right weight.

Three things move out of the business logic and all three matter. The branch goes from nine places to one. The construction details — keys, hosts, retries — leave the domain. And the vendor import leaves too, so `grep -rn twilio orders/` returns nothing, which is the actual test for whether the dependency was inverted.

The cost I am accepting is that the failure moves to run time. `make_sender('emial')` used to be an import error and is now a `ValueError` when some user with that preference logs in. So I would type the channel as an enum rather than a raw string, and validate it at the boundary where it comes out of the database.

If channels are being added often, I would go one step further and register with a decorator, so a new channel is one new file and zero edits. The trade there is discoverability: the list of live channels is no longer written down anywhere, and a module that does not get imported silently does not exist. That is the same trade Django’s app registry and pytest’s plugin system make.

I would not reach for abstract factory here. There is no family — a notification is one object, not a matched set. Where I have found it genuinely useful is test versus production infrastructure, where one factory hands out real Postgres, S3 and Stripe and the other hands out in-memory versions of all three, so you cannot end up with a live payment client and a fake database. And the drawback I would state up front is that adding a new product type to an abstract factory means editing every factory — it is open to new families and closed to new types.”

9 Recall card

- **Three different things are called “factory”, and separating them is the answer.** *Simple factory* — a function with a branch, **not a GoF pattern**, and what you should usually write. *Factory method* — a **subclass** overrides a creation method; the base algorithm never learns the product. *Abstract factory* — one object produces a **family** that must not be mixed.
- **In Python, write a registry:** `SENDERS: dict[str, type[Sender]]`, then `SENDERS[channel]()`. Classes are objects. A decorator registry drops it to **1 new file, 0 edits** — at the cost of the live list being invisible if a module is never imported.
- **Three things leave the business logic:** the branch (9 call sites → 1), the construction details (keys, hosts, timeouts), and the **vendor import** — `grep -rn "twilio" orders/` returning nothing is the proof the dependency was inverted.
- **Abstract factory’s grid is asymmetric, and volunteer it.** A new **family** is 1 file and 0 edits; a new **product type** is the interface plus *every* factory — 9 files for 4 families. Iqbal’s twelfth school was one specification; the blazer was eleven. Use it only when families arrive more often than types, and only when you can **name the family**.
- **Do not write one if** there is one implementation and no test double (9 of 14 `*Factory` classes = ~200 lines of costume) · the variation is data, not behaviour · a **named constructor** (`Sender.for_channel`, `datetime.fromisoformat`) would do. And the cost you accept is that `'emial'` becomes a **runtime** error — so type the key as an enum.

PRACTICE

Practice — Hashing your own objects

DSA topic: Hashing your own objects **System design topic:** Factory and abstract factory

Code these, in this order

One rule for the whole set: **before writing `__eq__`, say out loud which fields make two of these the same thing.** That sentence is the design decision. `__hash__` is then obliged to agree with it, and the code writes itself.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Boomerangs	LeetCode 447 (Medium)	A tuple as a key, and noticing that the natural key is a distance rather than a point.
2	Max Points on a Line	LeetCode 149 (Hard)	Designing a key for “same line” — the reduced-fraction slope, and why floats are the wrong key.
3	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Objects as keys with a positional value, and deletion done right.
4	Design a HashMap keyed on your own class	Write it yourself	Rule 1 and rule 2 by hand: your table must call <code>__hash__</code> and then <code>__eq__</code> .

On problem 2, resist the float

The obvious key for a line's slope is $(y_2 - y_1) / (x_2 - x_1)$. That is a float, and two lines that are mathematically the same can produce two different floats. Use the reduced fraction — divide both differences by their GCD and normalise the sign — and say out loud why a tuple of two integers is a safe key and a float is not.

On problem 4, break both rules on purpose

Write a small open-addressed table that stores your own objects. Then:

1. Insert an object, mutate the field its hash uses, and try to find it. Confirm nothing raises.
2. Write an `__eq__` that ignores a field `__hash__` uses. Confirm it still works, and say why.
3. Write a `__hash__` that ignores a field `__eq__` uses. Find the input that loses an entry.

Number 3 is the only one of the three that is a real violation. Say which rule, and in which direction.

The identity-versus-value drill

For each, predict `len(...)` before you run it, then run it:

1. `{Order("A-1"), Order("A-1")}` with no `__eq__` or `__hash__`.
2. The same, with `__eq__` only.
3. The same, with both.

4. `{Point(1, 2), Point(1, 2)}` with `@dataclass`.
5. The same with `@dataclass(frozen=True)`.
6. The same with `@dataclass(eq=False)`.
7. `{Cell(0, 0), Cell(0, 0)}` with `NamedTuple`.

Two of those seven raise rather than returning a number. Name them and quote the exact error.

The contract drill

For each pair of methods, say whether it is legal, and if not, which rule it breaks and what goes wrong:

1. `__eq__` on `id`; `__hash__` on `id`.
2. `__eq__` on `id`; `__hash__` on `(id, customer)`.
3. `__eq__` on `(id, customer)`; `__hash__` on `id`.
4. `__eq__` on `id`; `__hash__` returning 1 always.
5. `__eq__` on `id`; `__hash__` returning `id(self)`.
6. `__eq__` defined; `__hash__` not defined.

Two of the six are legal but slow. Say how slow, with a number, for 100,000 objects.

The silent-failure drill

Reproduce this exactly, and write down every line of output:

1. Put an object in a dictionary as a key.
2. Mutate the field its `__hash__` uses.
3. Print `key in lookup`.
4. Print `len(lookup)`.
5. Print `list(lookup)`.
6. Print `lookup.get(key)`.
7. Try to `del lookup[key]`.

Then say, in one sentence, why this is worse than an exception, and what single change makes it impossible.

The frozen drill

1. Make a `@dataclass(frozen=True)` and try to assign to a field. Quote the exact error.
2. Put a mutable list inside a frozen dataclass and mutate the list. Does it raise? Is the object still findable? Explain.
3. Change the list field to a `tuple` and repeat.
4. Say what `frozen=True` actually guarantees and what it does not.

Step 2 is the one people are surprised by. Say precisely what frozen means.

The cost drill

Time 100,000 dictionary lookups where `__hash__` is each of the following:

1. `hash(self.order_id)` — a 10-character string.
2. `hash((self.a, self.b, self.c))`.
3. `hash(self.description)` — a 100 KB string.
4. `hash(tuple(self.items))` — a 10,000-element list.
5. `return 1`.

Then, for number 5, build a dictionary of 20,000 such objects and time that. Compute the number of comparisons and name the complexity.

The factory-distinction drill

For each, say which of the three it is — simple factory, factory method, or abstract factory — and give the reason in one sentence:

1. `logging.getLogger("app.orders")`
2. `sqlalchemy.create_engine("postgresql:// ...")`
3. A `Dialog` base class whose subclasses override `create_button()`.
4. A `UiFactory` with `create_button`, `create_checkbox` and `create_menu`.
5. `datetime.fromisoformat("2026-08-31")`
6. `boto3.session.client("s3")`
7. A `TestInfrastructure` object handing out an in-memory database, an in-memory object store and a fake payment client.
8. `Calendar.getInstance()`

One of the eight is not really a factory at all in the sense we mean. Which, and why is its name misleading?

The build-it drill

Take this code and refactor it:

```
# orders/checkout.py
if user.prefers_sms:
    sender = SmsSender(api_key=settings.TWILIO_KEY)
else:
    sender = EmailSender(host=settings.SMTP_HOST)
sender.send(message)
```

1. Count how many other files would contain this same branch in a real system.
2. Write the `Sender` protocol.
3. Write the registry factory. Count its lines.
4. Add a fourth channel. Count the files you edited.
5. Convert it to a decorator registry. Add a fifth channel. Count again.

6. Run `grep -rn "twilio" orders/` before and after, and say what the result proves.
7. Name the cost you just accepted, and the two ways to mitigate it.

The abstract-factory grid drill

You have four families (Windows, Mac, Linux, Web) and three product types (button, checkbox, menu).

1. How many product classes exist?
2. Add a fifth family. How many files added, how many edited?
3. Add a fourth product type. How many files added, how many edited?
4. State the asymmetry in one sentence.
5. Give one real situation where the asymmetry is acceptable, and one where it is not.
6. Name a family from your own experience where mixing members would be a genuine bug.

The do-not-build-it drill

For each, say whether you would write a factory, and give the deciding question:

1. One implementation, no test double, trivial constructor.
2. One implementation, but the constructor needs three secrets from config.
3. Three implementations chosen by a value from the database.
4. Three tax rates that differ only by a number.
5. Two implementations, one of which exists only for tests.
6. A class that will be swapped when the vendor contract expires next year.

Two of the six are “no” for the same reason and two are “yes” for the same reason. Name both reasons.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why did putting this object in a set not deduplicate it?* Identity versus value, the two rules of the contract with the one-directionality of rule 1, which fields you would choose and why, and `frozen=True` volunteered as the way to make rule 2 unbreakable.
2. *How do you create the right notification object for each channel?* The three meanings of “factory” separated in the first sentence, the pain named with a number of call sites, the registry written, the three things that moved out of the business logic, and the run-time-typo cost accepted with its mitigation.
3. *What happens if I mutate a key while it is in a dictionary?* The object stays in its old bucket, lookups compute a new one, `in` is False while `len` counts it and iteration yields it, no exception at all — and the one-word fix.

Before you move on

- [] I watched `len({Order("A-1"), Order("A-1")})` print 2 before I fixed anything.
- [] I triggered `TypeError: unhashable type` from `__eq__` alone and from a plain `@dataclass`.
- [] I can state both rules of the contract and say which direction rule 1 does *not* go.
- [] I mutated a key in place and confirmed `in` is `False` while `len` is 1 and no error is raised.
- [] I know what `frozen=True` guarantees, and what it does not guarantee about a list field.
- [] I measured the cost of a `__hash__` that builds a tuple from a 10,000-element list.
- [] I can separate simple factory, factory method and abstract factory in one sentence each.
- [] I wrote the registry factory and can say the three things it moved out of the business logic.
- [] I can state the abstract-factory row/column asymmetry with numbers.
- [] I can name two situations where I would deliberately not write a factory.
- [] I answered all three questions above out loud.

When a hash map is the wrong answer

DATA STRUCTURES AND ALGORITHMS

When a hash map is the wrong answer

You can name the three cases where sorting or an array beats a dictionary.

SYSTEM DESIGN

Builder

You can build an object with fifteen optional fields without a fifteen-argument constructor.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *You used a hash map. Could you do it in $O(1)$ space instead?*
- *This constructor takes twelve parameters. Fix it.*

1 What this is, and why they ask it

For six days the answer to everything has been “put it in a dictionary”. Today is the correction. There are three situations where a hash map is the wrong tool, and they are specific enough to memorise: **when the interviewer takes your extra space away**, **when the keys are small dense integers**, and **when you need the data in order**. In all three, something simpler and faster exists, and reaching for the dictionary out of habit is a visible reflex rather than a decision.

They ask it in a very particular way. You give the $O(n)$ hash-map answer, they nod, and then they say *now do it in $O(1)$ space*, or *now give me the k -th smallest*, or *what if the values are all between 1 and n* . That is not a trick. It is the second half of the question, and it was always coming. An interviewer wants to see that you know what the map was buying you — it was buying time with memory — and that you can name what you would trade instead.

The deeper point, and the one worth carrying out of this phase, is that **a hash map throws away structure**. If the input is sorted, the map does not care. If the keys are the numbers 1 to n , the map hashes them anyway. Every one of today’s three cases is a situation where the input had structure and the dictionary ignored it.

2 The story

The wedding at the hall in Jayanagar had about four hundred people coming, and because it was August half of them would arrive with something — an umbrella, a bag, a stack of boxes somebody had been asked to carry.

Ramesh’s nephew Kiran, who is twenty-three and quite pleased with himself, arrived on the Friday evening with a plan. He would sit at a table by the door with his phone. Every person who handed something over, he would type in their name and where he had put their things. Then when they came back he would look up the name.

Ramesh listened to the whole thing and then pointed at the wall behind the table.

The wall has a hundred and twenty wooden pigeonholes in it, six rows of twenty, and each one has a number painted on it in white. In a drawer under the table there is a book of small numbered brass tokens, two of each number.

So the actual system is: a woman hands over her umbrella, Kiran puts it in the first empty hole, which is 47, and hands her token 47. She comes back and gives him the token, and he goes to hole 47. He does not need her name. He does not need to look anything up. He does not need to remember anything at all, and neither does she.

It took Kiran most of Friday evening to stop being annoyed about this.

Two more things happened over the weekend, and they are the reason Ramesh tells the story.

On the Sunday there was a much smaller function in the same hall — fourteen people, a naming ceremony. Kiran arrived and started setting up the tokens. Ramesh told him not to bother. There were fourteen bags on one table. If somebody wants theirs, you look at the table. Setting up a system to avoid looking at fourteen bags is more work than looking at fourteen bags.

And on the Saturday night, at about half past eleven, the caterer needed to shut the hall and asked which things were still uncollected. Because the holes are in a row and numbered in order, Ramesh walked along the wall from 1 and had the answer in a minute. Kiran's phone would have held four hundred names in the order people arrived, and answering that would have meant reading all four hundred.

3 The idea in plain English

Kiran's phone is a **hash map**: a general system that can look up anything by any key, at the price of storing the keys and computing where they go. It works for every situation, which is exactly why it is the default.

The wall of numbered holes is **direct addressing**: because the keys are the numbers 1 to 120, you do not need to hash anything. The key is the position. No hashing, no storing keys, no collisions, and about a quarter of the memory.

The fourteen bags are **n is too small for a data structure**. Setting one up costs more than the scan it saves.

And the walk along the wall is **order**. A hash map has none, so any question that mentions smallest, largest, next, previous, k-th or between cannot be answered by one.

Case 1 — the interviewer takes your space away

The map bought time with memory. When memory is taken away, you have two standard replacements.

Sort, then walk. Sorting brings equal elements next to each other and puts everything in order, so duplicates, ranges and neighbours all become one pass. Cost: $O(n \log n)$ time instead of $O(n)$, $O(1)$ extra space if you may modify the input.

Two pointers on the sorted input. Once the data is sorted, the opposite-ends walk from [day 028](#) answers pair-sum questions with no map at all. Two Sum on a *sorted* array is $O(n)$ time and $O(1)$ space, and needs no dictionary.

Mark inside the input itself. This is the sharp one. If the values are integers in a known range — usually 1 to n — the array can be its own hash table. You record “I have seen the value 3” by making the element at index 2 negative. No extra memory, because you are storing the information in the sign bit of data you already have.

```
for number in numbers:
    index = abs(number) - 1
    if numbers[index] > 0:
        numbers[index] = -numbers[index]    # mark "value index+1 was seen"
```

`abs` is there because the element may already have been flipped by an earlier step. This is $O(n)$ time and $O(1)$ extra space, and it destroys the input, which is a question you must ask before doing it.

Case 2 — the keys are small dense integers

If the keys are the twenty-six lowercase letters, or the ASCII range, or the numbers 0 to 100, a plain list indexed directly beats a dictionary on every axis.

```
counts = [0] * 26
counts[ord(character) - ord("a")] += 1
```

No hashing, no collisions, no key storage, and the memory is contiguous so the processor’s cache works properly. This is the same idea as counting sort from [day 056](#).

The condition is **dense**. Twenty-six keys in a range of twenty-six is dense. Three keys in the range 0 to a billion is not — that list would be four gigabytes of zeroes, and the dictionary is obviously right.

Case 3 — you need order

A hash map answers exactly one question: *is this key present, and what is its value?* Every other question needs a different structure.

THE QUESTION	HASH MAP	WHAT TO USE
Is key 57 present?	$O(1)$	hash map
What is the smallest key?	$O(n)$ — read everything	heap, or sorted array
What is the k -th smallest?	$O(n \log n)$ — sort it first	heap, quickselect
Which keys are between 10 and 20?	$O(n)$	sorted array + binary search
What is the next key after 57?	$O(n)$	sorted array, or a balanced tree
Give me everything in order	$O(n \log n)$	sorted array, balanced tree

The row that matters most in interviews is the range query. “Which events happened between these two times” against a dictionary is a full scan. Against a sorted array it is two binary searches from [day 044](#) and a slice.

Python has no built-in balanced tree. The honest answers are `bisect` on a sorted list, `heapq` for smallest-or-largest, or the third-party `sortedcontainers.SortedList`. Say which you would reach for and why — “there is no `TreeMap` in the standard library” is a correct and useful sentence.

The three smaller reasons

Not headline cases, but each is a real answer to a real follow-up.

Memory. Measured, on a million integers:

<code>list</code>	7 MB	<code>dict</code>	40 MB
<code>set</code>	32 MB		

A set is roughly four times a list and a dict is nearly six, because both store hashes, keep the table about two-thirds empty by design, and hold pointers rather than values.

Tiny n. At eight elements, scanning a list is only about twice as slow as a set lookup in Python, and in a compiled language the list often wins outright because it is one cache line and the hash is a computation. Below roughly eight to sixteen elements, building a set costs more than it saves.

Untrusted keys. $O(1)$ is the average, not the worst. A hash map fed adversarial keys degrades to $O(n)$ per operation — the attack from [day 061](#). In a latency-critical path where the worst case is what you are promising, a sorted structure with a guaranteed $O(\log n)$ can be the right choice even though it is slower on average.

4 The picture

The same information, three ways, on the values `[3, 1, 3, 4]` where every value is between 1 and 4.

(a) a hash map keys stored, hashed, ~40 bytes per entry
 { 3: 2, 1: 1, 4: 1 }

(b) direct addressing the key IS the position. No keys stored.
 value: 1 2 3 4
 +---+---+---+---+
 count | 1 | 0 | 2 | 1 |
 +---+---+---+---+
 index 0 1 2 3 ← index = value - 1

(c) marking in place no extra memory at all
 before: [3 , 1 , 3 , 4]
 |
 see 3 → flip index 2:
 [3 , 1 , -3 , 4]
 see 1 → flip index 0:
 [-3 , 1 , -3 , 4]
 see 3 → index 2 already negative, leave it
 see 4 → flip index 3:
 [-3 , 1 , -3 , -4]
 ^
 index 1 is still positive → the value 2 never appeared

What to notice in (c): the array is now carrying two pieces of information at once. The **magnitude** is still the original value, and the **sign** is the “have I seen index+1” flag. That is why every read has to be `abs(numbers[i])` — you are reading through a mark that you put there.

And the ordering point, drawn:

```
hash map, iterated:    3, 1, 4                      ← no order you can rely on
sorted array:         1, 3, 3, 4                  ← "smallest" is [0]
                                                      "between 2 and 4" is two binary searches
```

The map is not badly ordered. It is *unordered* — there is no order to be wrong about, which is why every ordering question has to go somewhere else.

5 The code, built step by step

The flagship problem for this whole lesson is **First Missing Positive** — LeetCode 41. Given an unsorted array, find the smallest positive integer that is not in it. `[3, 4, -1, 1]` gives `2`. The required complexity is **O(n) time and O(1) extra space**, which is precisely the sentence “you may not use a hash map”.

Step 1 — the answer that is not allowed

```
def first_missing_positive_with_set(numbers: list[int]) → int:
    present = set(numbers)
    candidate = 1
    while candidate in present:
        candidate += 1
    return candidate
```


Write this first anyway, and say so: “this is $O(n)$ time and $O(n)$ space, and it is the answer if space is free. You have asked for $O(1)$ space, so the set has to go.” Showing the easy answer and then removing it is a better interview than jumping to the clever one.

Step 2 — notice the bound nobody gave you

Here is the observation the whole problem turns on, and it is worth saying slowly.

With n numbers, the answer must be between 1 and $n + 1$.

Why: if the array contained all of $1, 2, \dots, n$, the answer would be $n + 1$. If it did not contain all of them, the answer is one of them, so it is at most n . **Nothing outside $1..n+1$ can be the answer, so nothing outside that range matters.** Negative numbers, zero, and anything above n can be ignored entirely.

That single fact converts an unbounded problem into a bounded one, which is what makes direct addressing possible.

Step 3 — clean the array so the marking is safe

The marking trick uses the sign bit, so a value that is already negative or zero would confuse it. Replace anything useless with a value that is out of range but positive:

```
n = len(numbers)
for i in range(n):
    if numbers[i] ≤ 0 or numbers[i] > n:
        numbers[i] = n + 1      # a harmless placeholder, definitely out of range
```

After this pass every element is between 1 and $n + 1$, and every element is positive. The array is now safe to write signs into.

Step 4 — mark what is present

```
for i in range(n):
    value = abs(numbers[i])
    if value ≤ n:
        numbers[value - 1] = -abs(numbers[value - 1])
```

Read it carefully, because both `abs` calls are load-bearing. The first reads the original value through any mark already placed on this slot. The second makes the target negative *without* flipping an already-negative slot back to positive — `-abs(x)` is idempotent, while `-x` is not. That is the bug people write, and it fails on any input with a repeated value.

Step 5 — read the answer off

```
for i in range(n):
    if numbers[i] > 0:
        return i + 1
return n + 1      # slot i was never marked, so i+1 never appeared
                  # everything 1..n was present
```

The first positive slot is the first missing positive. If there is none, the array held all of $1..n$, so the answer is $n + 1$.

The complete solution

```
def first_missing_positive(numbers: list[int]) → int:
    """The smallest positive integer absent from `numbers`.


O(n) time, O(1) extra space. Uses the array itself as the table: slot i records whether the value i + 1 was seen, in its sign bit.



NOTE: this modifies the input. Ask before doing that.


    """
    n = len(numbers)


# 1. The answer is in 1..n+1, so anything outside that range is irrelevant.  
# Replace it with a positive placeholder so the sign trick is safe.  
for i in range(n):  
    if numbers[i] ≤ 0 or numbers[i] > n:  
        numbers[i] = n + 1



# 2. For each value v in 1..n, mark slot v-1 negative.  
# -abs(...) so a repeated value does not flip a slot back to positive.  
for i in range(n):  
    value = abs(numbers[i])  
    if value ≤ n:  
        numbers[value - 1] = -abs(numbers[value - 1])



# 3. The first slot still positive is the first value never seen.  
for i in range(n):  
    if numbers[i] > 0:  
        return i + 1  
return n + 1


def two_sum_sorted(numbers: list[int], target: int) → tuple[int, int] | None:
    """Two Sum on SORTED input: O(n) time, O(1) space, no hash map needed."""
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return left, right
        if total < target:
            left += 1          # need a bigger sum
        else:
            right -= 1         # need a smaller sum
    return None
def has_duplicate_sorted(numbers: list[int]) → bool:
    """O(n log n) time, O(1) extra space – the answer when the set is taken away."""
    numbers.sort()           # in place
    return any(numbers[i] == numbers[i - 1] for i in range(1, len(numbers)))
def count_letters(word: str) → list[int]:
    """Direct addressing: the key IS the index. No hashing, no key storage."""
    counts = [0] * 26
    for character in word:
        counts[ord(character) - ord("a")] += 1
    return counts
def values_between(sorted_values: list[int], low: int, high: int) → list[int]:
    """A range query – the thing a hash map cannot do. Two binary searches."""
    import bisect
    start = bisect.bisect_left(sorted_values, low)
    end = bisect.bisect_right(sorted_values, high)
    return sorted_values[start:end]
```

```

if __name__ == "__main__":
    print(first_missing_positive([1, 2, 0]))      # 3
    print(first_missing_positive([3, 4, -1, 1]))  # 2
    print(first_missing_positive([7, 8, 9, 11, 12])) # 1
    print(first_missing_positive([1, 1]))        # 2 ← the -abs case
    print(first_missing_positive([]))            # 1
    print(two_sum_sorted([2, 7, 11, 15], 9))     # (0, 1)
    print(count_letters("banana")[:5])          # [3, 1, 0, 0, 0]
    print(values_between([1, 3, 5, 7, 9, 11], 4, 9)) # [5, 7, 9]

```

The `[1, 1]` case is the one to run. With `-x` instead of `-abs(x)` in step 2, the second `1` flips slot 0 back to positive and the function returns `1` instead of `2`.

6 What it costs

The three passes, counted

Each of the three loops runs exactly `n` times and does constant work inside.

```

pass 1 (clean):    n iterations x 1 comparison, 1 possible write
pass 2 (mark):    n iterations x 1 abs, 1 comparison, 1 write
pass 3 (read):    n iterations x 1 comparison
-----
total:            3n → O(n)

```

Space is **O(1) extra**. The only variables are `n`, `i` and `value`. The array is reused, not copied, which is the entire point.

Compare with the set version: same $O(n)$ time, but $O(n)$ space and — at a million elements — 32 MB of extra memory that the marking version does not need.

Direct addressing versus a dictionary, counted

Counting the letters of a 100,000-character string:

```

list of 26:  100,000 x (1 subtraction + 1 indexed write) ~ 2 operations each
dictionary:  100,000 x (1 hash + 1 modulo + 1 probe + 1 write) ~ 5 operations each

```

Roughly two and a half times the work, plus the dictionary stores 26 keys and their hashes while the list stores nothing but the counts. In practice the list version is about twice as fast, and the gap widens on long inputs because 26 integers fit in a couple of cache lines and a dictionary does not.

Memory, measured

On a million small integers:

```

list  7 MB
set   32 MB      about 4.5x
dict  40 MB      about 5.7x

```

Those numbers are the container only; they are the right ones to quote because they are what the extra structure costs you over the data you already had.

Small n, measured

A million membership tests:

```
n = 100:  set 0.018 s    list 0.265 s    set wins by ~15x
n = 8:    set 0.019 s    list 0.034 s    set wins by ~1.8x
```

At $n = 8$ the gap has nearly closed, and that is before counting the cost of *building* the set. If you build a set to do one lookup over eight items, you have lost. The rule of thumb: **below about eight to sixteen elements, scan.**

Sorting versus hashing

```
hash map:  O(n) time,      O(n) extra space
sort:      O(n log n) time, O(1) extra space (in place), input destroyed
```

At $n = 1,000,000$: the map is 1,000,000 operations and 32 MB; the sort is about 20,000,000 comparisons and no extra memory. **You are trading a factor of twenty in time for all of the memory**, and which side of that trade you want depends entirely on the constraint you were given. Say it as a trade, not as a ranking.

7 The traps

Trap 1 — `-x` instead of `-abs(x)`

```
numbers[value - 1] = -numbers[value - 1]    # WRONG
```

Looks right, is right on inputs with no duplicates, and fails on `[1, 1]`. The first `1` makes slot 0 negative; the second `1` negates it again and makes it positive; the final pass sees a positive slot 0 and answers `1` when the answer is `2`.

```
input [1, 1]
correct: 2
with -x: 1
```

An input of two elements. It will not be caught by eyeballing the code, and it is the single most common way this problem is failed.

Trap 2 — forgetting the cleaning pass

Skip step 3 and the marking loop reads a negative number, takes `abs`, and marks a slot that has nothing to do with anything. On `[-1, 4, 2, 1]` you get an answer that happens to be right sometimes and wrong other times, which is worse than always wrong.

Also: without the cleaning pass, an index can go out of range.

```
>>> numbers = [1, 2, 3]
>>> numbers[abs(-7) - 1] = 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list assignment index out of range
```

Trap 3 — destroying the input without asking

The marking solution overwrites the array. So does the sorting solution. If the caller needed that array afterwards, you have introduced a bug that no test of your function will catch, because it is in the caller.

Ask, every time: “may I modify the input?” It takes three seconds and it is one of the highest signal-to-effort questions in the whole interview. If the answer is no, the $O(1)$ space solution does not exist and you say so.

Trap 4 — direct addressing on a sparse range

```
counts = [0] * (max(numbers) + 1)
```

Fine for `[3, 1, 4]`. On `[1, 2, 1000000000]` that allocates a list of a billion entries — about 8 GB — to store three counts.

```
MemoryError
```

The condition for direct addressing is **dense**, not just “integer”. Say the condition out loud when you propose it: “the values are between 1 and n , so I can index by them.”

Trap 5 — asking a hash map an ordering question

```
smallest = min(counts)           #  $O(n)$  – reads everything
first_key = list(d)[0]           # insertion order, not sorted order
```

`min` on a dictionary is a full scan every time you call it. If you need the smallest repeatedly, you wanted a heap. And `list(d)[0]` gives the first-inserted key, which people mistake for the smallest key because it often is, on small examples.

Trap 6 — using a hash map on already-sorted input

If the input is sorted and you build a set from it, you have paid $O(n)$ time and $O(n)$ space to throw away the very property that would have made the problem easy. Two Sum on a sorted array with a hash map is $O(n)$ space; with two pointers it is $O(1)$. The interviewer said “sorted” for a reason, and the word is doing work.

Trap 7 — quoting $O(1)$ for a worst case

A hash map's $O(1)$ is an average. In a system that must promise a latency bound — an order matching engine, an ad auction with a 20 ms budget — the $O(n)$ worst case matters, and a structure with a guaranteed $O(\log n)$ can be the correct choice despite being slower on average. Knowing when average is the wrong statistic is a senior signal.

8 In the interview

How it gets asked

- The follow-up, and this is the main one: *“That works. Now do it in $O(1)$ space.”* You will hear this after almost every hash-map answer you give. It is not a rejection; it is the next question.
- The hint disguised as a constraint: *“All the numbers are between 1 and n .”* That sentence exists to tell you to index instead of hash.
- The ordering pivot: *“Now give me the k -th smallest”* or *“now give me everything between 10 and 20”* — the map is finished and something else has to start.
- The direct one: *“When would you not use a hash map?”* Asked at senior level, and the three cases are the answer.

What to say out loud, in the first ninety seconds

1. **Say what the map was buying.** “The dictionary bought me time with memory — $O(n)$ time for $O(n)$ space. If you take the space away, I have to buy the time back some other way.”
2. **Name the three replacements before choosing one.** “There are three things I can do: sort and walk, which is $O(n \log n)$ time and $O(1)$ space; two pointers if it is sorted or I sort it; or, if the values are in a bounded range, use the array itself as the table.”
3. **Ask the question that decides it.** “May I modify the input?” Everything $O(1)$ -space in this lesson destroys the array. If the answer is no, say the $O(1)$ solution does not exist and offer the $O(n \log n)$ copy-and-sort.
4. **Find and state the bound.** For First Missing Positive: “with n numbers, the answer has to be between 1 and $n + 1$, so anything outside that range cannot matter.” That sentence is the whole problem and you should say it before writing anything.
5. **Then write it, and name the `-abs` line as you write it.** “This has to be `-abs`, not `-`, or a repeated value flips the mark back.”

The follow-ups

“Why is the answer bounded by $n + 1$?” “Because there are only n numbers. If they were exactly 1 through n , the smallest missing positive is $n + 1$. If they were not, then one of 1 through n is missing, so the answer is at most n . Either way nothing outside 1 to $n + 1$ can be the answer, which is what lets me index into an array of length n .”

“Your solution destroys the input. Is that acceptable?” “That is why I asked. If it is not, I can copy the array first, which costs $O(n)$ space and puts me back where I started, or I can sort a copy for $O(n \log n)$ time and $O(n)$ space. There is no $O(1)$ space solution that preserves the input, because $O(1)$ space means the only place to write anything is the input.”

“When would you use a plain array instead of a dictionary?” “When the keys are small dense integers. Twenty-six letters, ASCII, values bounded by n . The array does not hash, does not store keys, does not collide, and it is contiguous so the cache works. The condition is density — three keys in a range of a billion is eight gigabytes of zeroes and obviously wrong.”

“What can a hash map not do at all?” “Anything about order. Smallest, largest, k -th, next, previous, or a range. All of those are $O(n)$ scans against a map. Sorted array plus binary search for range queries, a heap for repeated smallest-or-largest, and a balanced tree if I need both ordering and fast insertion — which Python does not have in the standard library, so I would use `bisect` on a sorted list, or `sortedcontainers.SortedList`.”

“Is a hash map always $O(1)$?” “On average, with a good hash and a load factor kept low. The worst case is $O(n)$ per operation, when everything collides. That is rare accidentally and achievable deliberately — the hash-flooding attacks. So in a latency-critical path where I am promising a bound rather than an average, a structure with a guaranteed $O(\log n)$ can be the better choice even though it is slower most of the time.”

A model answer

Asked: *find the first missing positive integer, in $O(n)$ time and $O(1)$ space.*

“Let me give the easy answer first so we agree on the problem. Put everything in a set, then count up from 1 until I find a number that is not there. That is $O(n)$ time and $O(n)$ space. You have asked for $O(1)$ space, so the set has to go, and the question becomes where I am allowed to store the information instead. The only place left is the input array.

The observation that makes that possible is a bound nobody gave me. There are n numbers, so if the array happened to contain exactly 1 through n , the answer would be $n + 1$. If it did not, then one of 1 through n is missing, so the answer is at most n . Either way the answer lies between 1 and $n + 1$, which means anything negative, zero, or larger than n is irrelevant and I can ignore it.

That turns it into direct addressing: slot i can record whether the value $i + 1$ is present, and I can record that in the sign bit, because after cleaning, every value is positive.

So it is three passes. First, replace anything outside 1 to n with $n + 1$, a harmless positive placeholder, so the sign trick is safe. Second, for each value v , make slot $v - 1$ negative. Third, walk the array and return the index of the first positive slot, plus one — or $n + 1$ if there is no positive slot.

One line needs care: the marking has to be `numbers[v-1] = -abs(numbers[v-1])`, not `-numbers[v-1]`. With a repeated value, plain negation flips the slot back to positive and the answer is wrong. On the input `[1, 1]` the correct answer is 2 and the buggy version returns 1.

Cost: three passes of n , so $O(n)$ time, and the only extra variables are two integers, so $O(1)$ space.

Before I write it I should ask — may I modify the input? Everything I have described destroys it. If the caller needs it afterwards then $O(1)$ space is impossible, because the input is the only memory I have, and I would offer sorting a copy instead: $O(n \log n)$ time and $O(n)$ space.”

9 Recall card

- **A hash map buys time with memory and throws away structure.** Three cases where it is the wrong tool: **space is taken away** · **keys are small dense integers** · **you need order**.
- **No extra space → three moves.** Sort and walk ($O(n \log n)$ time, $O(1)$ space) · two pointers on sorted input (Two Sum sorted is $O(n)/O(1)$, no map) · **mark in the input itself** when values are bounded by n . Always ask first: *may I modify the input?* — every $O(1)$ -space answer destroys it.
- **First Missing Positive is the flagship.** The answer is bounded by $n + 1$ (n numbers: either they are exactly $1..n$, or one of $1..n$ is missing). Three passes: clean out-of-range to `n+1` · mark slot `v-1` with `-abs(...)`, **never** `-x` (on `[1, 1]` plain negation returns 1 instead of 2) · return the first positive index $+ 1$.

- **Direct addressing needs *dense* keys, not just integer ones.** `[0]*26` for letters is right; `[0]*(max+1)` on `[1, 2, 10**9]` is 8 GB and a `MemoryError`. Memory per million ints: **list 7 MB · set 32 MB · dict 40 MB**. And below ~8-16 elements, just scan — at $n = 8$ a set wins by only $1.8\times$ before you count building it.
- **A map answers one question: is this key present?** Smallest, k-th, next, previous and range are all $O(n)$ scans against it — use `bisect` on a sorted list, `heapq`, or `SortedList` (Python has no `TreeMap`). And its $O(1)$ is an *average*: where you must promise a bound, a guaranteed $O(\log n)$ can beat a faster average.

1 What this is, and why they ask it

Builder separates *collecting the pieces of an object* from *creating it*. Instead of one constructor that takes everything at once, you get a small helper object that you tell one thing at a time, in any order, and then ask for the finished thing. The finished object can be immutable, because the messy half-built state lived in the builder and never escaped.

They ask it because the twelve-parameter constructor is real, it is everywhere, and the naive fixes make things worse. The first thing people reach for is a second constructor with fewer parameters, then a third, then a fourth — the **telescoping constructor** — and now there are six overloads and nobody knows which to call. The second thing people reach for is setters, which works and quietly destroys immutability, so an object can now exist in a half-configured state that no validation ever saw.

There is a third reason, and in Python it is the interesting one: **Builder is often unnecessary**. Keyword arguments with defaults solve most of what Builder solves in Java, in one line. A candidate who writes a thirty-line fluent builder in Python without mentioning that has demonstrated they learned the pattern from a Java book. A candidate who says “in Python I would use keyword-only arguments, and here is the specific case where I would still write a builder” has demonstrated judgement.

2 The story

The bakery on the corner in Frazer Town takes about nine cake orders a day, and Reshma has been on the counter for four years.

A man came in on a Tuesday for his daughter's ninth birthday. He knew roughly what he wanted and not in any particular order, which is how everybody comes in.

He said a kilo. Reshma wrote it down. Then he said chocolate, then that it was for Saturday evening, then that there should be a photograph on it, then he asked whether they did anything without egg because one of the children does not eat egg, then he came back to the photograph and said it should be of the girl with her dog, then he asked for her name on it, then he thought about it and changed the weight to one and a half kilos because more people were coming than he first said.

None of that is unusual. What Reshma does is take each thing as it comes and put it against the right line on the order screen. Weight. Flavour. Egg or eggless. Date and time. Message. Photograph or no photograph. Finish. Delivery or collection.

Nothing is being baked while this is happening. She is only collecting.

The important bit is at the end. She reads the whole thing back — one and a half kilos, chocolate, eggless, photo print, name in white, Saturday six o'clock, collection — and while she reads it she is checking it against the things she knows.

And on that Tuesday it did not pass. A photo print goes on a flat fondant top. He had asked for the chocolate ganache finish, which is glossy and soft, and a photo sheet will not sit on it — it slides and the ink runs. So she told him: photo or ganache, not both. He took the photo and gave up the ganache.

Then she pressed the button, and only then did the order become a real thing that goes to the back.

Reshma's view is that the reading-back is the whole job. Anyone can write down what a customer says. The reason the bakery does not send out cakes that are wrong is that there is one moment, after everything has been said and before anything has been made, where somebody looks at the whole order at once and asks whether it is possible.

3 The idea in plain English

The order screen is a **builder**. It holds the pieces as they arrive, in whatever order they arrive, and it is not a cake. Pressing the button is `build()`. The cake is the **product**, and it comes into existence complete or not at all.

Four things come out of that, and each maps to a real benefit.

One: the arguments arrive one at a time and named. The man never had to say eight things in a fixed order. Nothing was positional.

Two: he could change his mind. The weight went from one kilo to one and a half. The builder is mutable; the cake is not.

Three: there is one moment where the whole thing is checked. Photo plus ganache is a rule that involves two fields, so no single setter could have caught it. Only `build()` sees everything at once.

Four: a half-finished order never leaves the counter. There is no such thing as a cake with a flavour and no weight. The incomplete state existed, but only inside the builder.

The problem it replaces

```
class Cake:
    def __init__(self, weight_kg, flavour, eggless, finish,
                  message, photo, delivery, delivery_time,
                  tier_count, colour, candles, note):
        ...
```

Twelve parameters. Three separate problems live in that line.

You cannot read the call site. `Cake(1.5, "chocolate", True, "fondant", "Happy Birthday", True, False, ..., 2, "white", 9, None)` — which `True` is which? Positional booleans are the worst kind of argument, and this is the **control coupling** and **long parameter list** from [day 061](#).

Most of them are optional, so you get telescoping constructors. One with four parameters, one with seven, one with nine, one with twelve. In Java, with `n` optional fields you would need up to `2^n` overloads to cover every combination, so people write four or five and everyone contorts their call to fit one of them.

Validation has nowhere to live. A rule involving two fields — photo needs fondant — cannot be checked by a setter, because when the first one is set the second has not arrived.

The two things called Builder

The Gang of Four version has four parts: a `Builder` interface, concrete builders, a `Director` that knows the sequence of steps, and the product. The point of the original was that the *same construction process* could produce *different representations* — a document parser that builds either HTML or plain text from the same sequence of steps.

The fluent builder — from Joshua Bloch's *Effective Java*, Item 2 — has no director and no interface. It is one class with one method per field, each returning `self`, and a `build()` at the end. This is what ninety-nine percent of people mean, and it is what you should write unless somebody asks for the other one.

```
cake = (CakeBuilder()
        .weight(1.5)
        .flavour("chocolate")
        .eggless()
        .photo("girl-and-dog.jpg")
        .message("Happy Birthday Aarna")
        .collect_at("2026-09-05 18:00")
        .build())
```

Each method sets one field and returns the builder, so they chain. `build()` validates and returns the finished, immutable `Cake`.

The part people forget: `build()` is where validation lives

This is the strongest argument for the pattern and the one candidates leave out.

```
def build(self) → Cake:
    if self._weight_kg is None:
        raise ValueError("weight is required")
    if self._photo and self._finish == "ganache":
        raise ValueError("a photo print needs a fondant top, not ganache")
    if self._tier_count > 1 and self._weight_kg < 2:
        raise ValueError("a tiered cake needs at least 2 kg")
    return Cake(...)
```

Those are **cross-field invariants**. No setter can check them, because a setter only ever sees one field. `build()` is the single moment where the whole object exists but has not yet escaped, and that is exactly where the check belongs. Say that sentence in the interview.

In Python, start with keyword arguments

This is the honest answer and it is the one that will impress.

```
from dataclasses import dataclass

@dataclass(frozen=True, kw_only=True)
class Cake:
    weight_kg: float
    flavour: str
    eggless: bool = False
    finish: str = "cream"
    message: str | None = None
    photo: str | None = None
    tier_count: int = 1

    def __post_init__(self) → None:
        if self.photo and self.finish == "ganache":
            raise ValueError("a photo print needs a fondant top, not ganache")
```

Six lines of class, and it already has: named arguments in any order, defaults for everything optional, immutability, and cross-field validation in `__post_init__`. `kw_only=True` forbids positional arguments entirely, which kills the “which `True` is which” problem outright.

```
cake = Cake(weight_kg=1.5, flavour="chocolate", eggless=True, photo="dog.jpg")
```

That is the whole builder pattern, in the language, for free. **Java needs Builder because Java has no keyword arguments and no default parameter values**. Saying that one sentence tells an interviewer you understand why the pattern exists rather than that it does.

When you still write a builder in Python

Four cases, and they are worth memorising because “you never need it” is as wrong as “always use it”.

1. **Construction is genuinely stepwise.** You are parsing a file, or consuming a stream, and the pieces arrive over time from different places. There is no single call site that has everything.
2. **The steps are conditional.** `if user.is_premium: builder.with_priority_support()` — awkward to express as one call with keyword arguments.
3. **The finished object needs heavy validation or expensive assembly** that you want to happen once, at the end, not on every field.
4. **Test data.** This is the best use in practice, and it is worth its own paragraph.

```
order = an_order().for_customer("ravi").with_items(3).already_paid().build()
```

A test builder with sensible defaults turns eighteen lines of object setup into one readable line that says only what this test cares about. Every field the test does not mention gets a valid default. This is the version of the pattern most working engineers actually use most often, and naming it is a good interview move.

4 The picture

The three ways to construct an object with many optional fields.

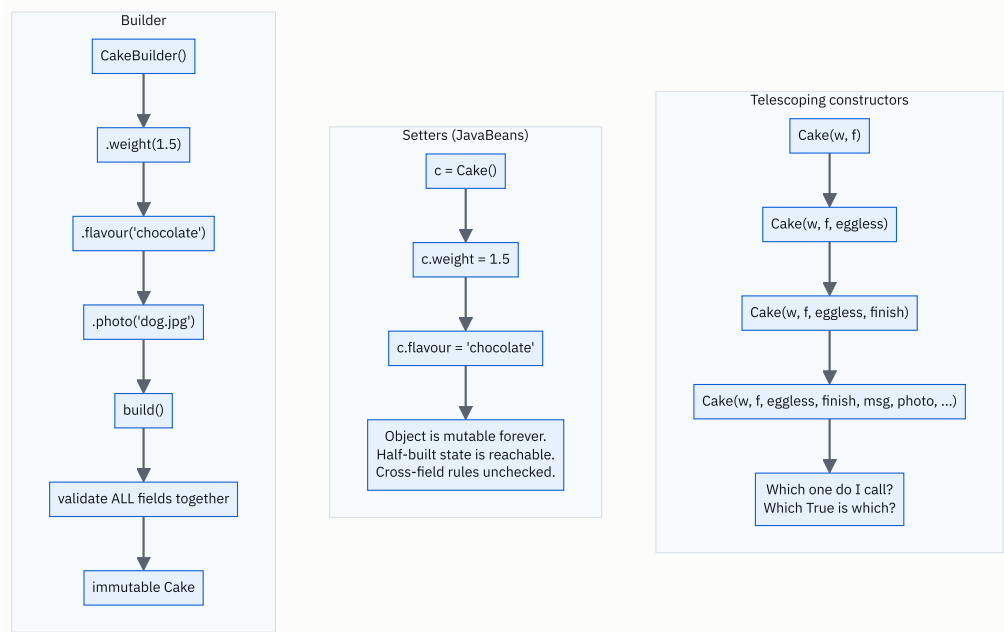


FIGURE 66.1

What to notice: only the third one has a box that says “validate all fields together”. That box is the pattern’s real product. Everything else — the chaining, the readability — is convenience.

And here is the state of the object over time, which is the thing setters get wrong:

SETTERS

```
time → [ empty ] [ weight ] [ weight+flavour ] [ ... ] [ complete ]
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      every one of these is a real Cake that other
      code can see, pass around, and save to a database
```

BUILDER

```
time → [ builder: empty ] [ builder: partial ] [ builder: full ]
      |
      | build()
      v
      [ Cake, complete ]
      the only Cake that ever exists is a valid one
```

5 How it actually works

The mechanics

A builder has one field per field of the product, all optional, all starting empty. One method per field, each of which stores the value and returns `self` so the calls chain. And a `build()` that checks the invariants and constructs the product in one go.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Cake:
    weight_kg: float
    flavour: str
    eggless: bool
    finish: str
    message: str | None
    photo: str | None

class CakeBuilder:
    def __init__(self) → None:
        self._weight_kg: float | None = None
        self._flavour: str | None = None
        self._eggless = False
        self._finish = "cream"
        self._message: str | None = None
        self._photo: str | None = None

    def weight(self, kg: float) → "CakeBuilder":
        self._weight_kg = kg
        return self # this is what makes it chain

    def flavour(self, name: str) → "CakeBuilder":
        self._flavour = name
        return self

    def eggless(self) → "CakeBuilder":
        self._eggless = True
        return self

    def photo(self, path: str) → "CakeBuilder":
        self._photo = path
        return self

    def build(self) → Cake:
        if self._weight_kg is None:
            raise ValueError("weight is required")
        if self._flavour is None:
            raise ValueError("flavour is required")
        if self._photo and self._finish == "ganache":
            raise ValueError("a photo print needs a fondant top, not ganache")
        return Cake(self._weight_kg, self._flavour, self._eggless,
                    self._finish, self._message, self._photo)

```

Two details worth pointing at in an interview.

`return self` is the whole fluent interface. There is nothing else to it.

The required fields are checked in `build()`, at run time. This is Builder's genuine weakness and you should volunteer it: the compiler cannot tell you that you forgot the weight. A twelve-parameter constructor at least fails to compile. The builder fails when the code runs. There are type-level tricks to fix this — a “step builder” where each method returns a different interface so the compiler enforces the order — and they are usually more machinery than the problem is worth.

Real products that use it

- `StringBuilder` in Java and `StringBuffer` before it. The original one everybody has used — and note it is the same idea as building strings with a list and `"".join` from [day 020](#).
- `java.net.http.HttpRequest.newBuilder().uri(...).header(...).timeout(...).build()` — the modern JDK standard-library example, and a clean one to cite.
- Lombok's `@Builder` annotation, which generates the whole thing, because writing it by hand is tedious enough that an entire library exists to avoid it.
- **Protocol Buffers**. Generated message classes are immutable and every one comes with a builder; `Message.newBuilder().setX(1).build()`. A whole serialisation format built on the pattern.
- `ProcessBuilder` in Java — the case where construction really is stepwise, because you add environment variables and redirects conditionally.
- **SQLAlchemy's** `select(...).where(...).order_by(...).limit(10)` and **Django's** `Model.objects.filter(...).exclude(...).order_by(...)`. Fluent, lazy, and nothing executes until the end — a query builder is a builder whose product is a SQL string.
- **Test data builders** — `an_order().for_customer("ravi").build()`. Not a library, a convention, and the one you are most likely to write yourself.

The Python decision, made explicitly

```
Does the object have many optional fields?
    → @dataclass(frozen=True, kw_only=True) with defaults. Done.

Do the pieces arrive at different times, from different code?
    → a builder.

Are some steps conditional?
    → a builder.

Is this test setup that is drowning the test in noise?
    → a test data builder.

Otherwise
    → keyword arguments. Do not write thirty lines to avoid six.
```

6 The numbers

The telescoping arithmetic

With `n` optional fields, the number of possible combinations is `2^n`.

```
4 optional fields → 16 combinations
8 optional fields → 256 combinations
12 optional fields → 4,096 combinations
```

Nobody writes 4,096 constructors. In practice a team writes four or five overloads, and every call site that does not match one passes `null` for the fields it does not want — which is how you end up with `new Cake(1.5, "chocolate", true, null, null, null, false, null, 1, null, 0, null)`.

A builder covers all 4,096 combinations with **one class and twelve methods**.

What the call site costs to read

```
positional constructor: Cake(1.5, "chocolate", true, "fondant", null, true, false, ...)
fields a reader must look up in the signature: 12
booleans whose meaning is invisible: 3
builder / keyword args: Cake(weight_kg=1.5, flavour="chocolate", eggless=True, ...)
fields a reader must look up: 0
```

That is not a micro-optimisation. Every code review of that line, forever, is either two seconds or thirty.

What it costs to write

Python @dataclass(frozen=True, kw_only=True), 12 fields:	~14 lines
fluent builder in Python, 12 fields:	~45 lines
fluent builder in Java, 12 fields (by hand):	~120 lines
fluent builder in Java with Lombok @Builder:	1 line

The Python builder is roughly **three times the code of the dataclass and buys nothing extra** unless one of the four conditions holds. That ratio is the argument for not writing it by reflex.

What a test data builder saves

This is the number that lands, because everybody has felt it.

test setup without a builder:	
construct Customer (6 fields), Address (4), 3 x OrderLine (4 each), Order (9 fields)	~18 lines
lines that matter to THIS test	2
signal-to-noise	11%
with a test data builder:	
an_order().for_customer("ravi").with_items(3).already_paid().build()	1 line

And the second-order effect, which is bigger: when `Order` gains a thirteenth field, the version without a builder means editing **every test that constructs an Order** — often 40 or 60 files. With a builder it is one default in one place.

Validation, counted

cross-field rules that a setter CAN check:	0
cross-field rules that build() can check:	all of them

Three rules on the cake — photo needs fondant, tiers need 2 kg, eggless excludes certain finishes — are all invisible to any individual setter and all trivial in `build()`.

7 The trade-offs

What you give up

Required fields become a run-time check. This is the real one. A constructor that demands a weight cannot be called without one; the compiler enforces it. `builder.build()` with no weight compiles fine and throws when it runs. You have traded a compile-time guarantee for readability, and you should say so out loud rather than being caught by the follow-up.

A duplicated field list. Every field exists twice — once on the product, once on the builder. Adding a thirteenth field means editing two places, and forgetting the builder is a silent omission. This is why Lombok's `@Builder` exists, and why in Python a dataclass with defaults is usually better.

Mutable intermediate state. The builder itself is mutable and, if you keep it around and call `build()` twice, you get two objects sharing whatever you did in between. Worse, a builder held in a field is not thread-safe. The convention is: build one, use it, drop it.

More code to read. Thirty to forty-five lines that do nothing but move values around. In a language with keyword arguments, that is thirty to forty-five lines of pure cost.

The GoF version has an extra cost

The full Builder — with a `Director` and a builder interface — adds two more classes for a benefit almost nobody needs: the same construction sequence producing different representations. If you cannot name the second representation, do not build the interface. Say “I would write the fluent version, which is Bloch’s, not the Gang of Four’s, unless there really are two products from one process.”

“I would not use this if...”

- **...the language has keyword arguments with defaults.** In Python, Kotlin, Swift, C# or modern JavaScript with an options object, the language already gives you the main benefit.
- **...there are fewer than about four or five fields.** Two required and one optional is a constructor.
- **...all the fields are required.** Then there is no combinatorial explosion, and a builder just moves the compile-time check to run time for nothing.
- **...the object is genuinely simple and the constructor is only long because the class is doing too much.** A twelve-parameter constructor is very often a single-responsibility problem wearing a construction problem’s clothes. **Ask whether four of those twelve fields are actually a missing `DeliveryDetails` object.** That is a better fix than any builder.

The best question to ask before writing one

Are these twelve parameters twelve things, or are they three things?

`line1`, `line2`, `city`, `pincode` passed together is the **data clump** from [day 061](#), and it is one `Address`. Doing that twice turns twelve parameters into five, and then you do not need a builder at all. Extracting the value objects first and reaching for the builder second is the answer that separates candidates.

8 In the interview

How it gets asked

- The direct one: *“This constructor takes twelve parameters. Fix it.”* Sometimes with real code on the screen.
- The named one: *“Explain the builder pattern.”* Where the good answer includes both the fluent version and the fact that the GoF one is different.
- The Python-specific one: *“Would you use a builder in Python?”* A test of whether you know why the pattern exists.
- The disguised one: *“How would you construct an object where some fields are only known later?”* That is the stepwise case, and it is the one where Builder is genuinely right.

What to say out loud, in the first ninety seconds

1. **Ask the value-object question first.** “Before I reach for a pattern — are these twelve parameters twelve independent things? `line1`, `line2`, `city` and `pincode` look like one `Address`. If four of the twelve collapse into two objects, the problem may go away.”
2. **Name what is actually wrong with the twelve.** “The call site is unreadable, positional booleans are invisible, most fields are optional so you end up with telescoping constructors, and there is nowhere for a rule involving two fields to live.”
3. **Give the language answer before the pattern answer.** “In Python I would start with a frozen dataclass, keyword-only, with defaults. That is named arguments, any order, immutability and validation in `__post_init__`, in about fourteen lines. Java needs Builder because it has neither keyword arguments nor default values.”
4. **Then say when you would still build one.** Stepwise construction, conditional steps, expensive assembly, test data.
5. **Volunteer the weakness.** “The thing Builder costs is that required fields become a run-time check instead of a compile-time one.”

The follow-ups

“What is the difference between the Gang of Four Builder and the one everyone writes?”

“The GoF version has a `Director` that knows the sequence of steps and a `Builder` interface with several implementations, so the same process produces different representations — the example in the book is building either HTML or plain text from one parse. What everybody actually writes is Bloch’s fluent builder from *Effective Java*: one class, one method per field, each returning `this`, and a `build()`. No director, no interface. I would write the second unless somebody can name the second representation.”

“How do you make sure required fields are set?” “In the plain version you check in `build()` and raise, which is a run-time failure. If it really matters you can do a step builder, where each method returns a different interface so the compiler forces the order — but that is a lot of machinery and I would want a strong reason. In Python, a dataclass field with no default is required at construction, which gives the compile-time behaviour back for free, and that is one of the reasons I would prefer it.”

“Why not just use setters?” “Three reasons. The object has to be mutable forever, so it can be changed after validation. Every intermediate half-built state is a real object other code can see and save. And cross-field rules cannot be checked, because a setter only ever sees one field — ‘a photo needs a fondant top’ involves two, so there is no setter that could enforce it. The builder gives me one moment where the whole thing exists and has not yet escaped.”

“Where have you actually used one?” “Test data builders, more than anything else. Setting up an order with a customer, an address and three line items is about eighteen lines, of which two matter to the test. A builder with sensible defaults turns that into one line that says only what the test cares about. And the second-order benefit is bigger: when the order class gains a field, I change one default instead of editing sixty test files.”

“When is a twelve-parameter constructor not a construction problem at all?” “When the class has too many responsibilities. If four of the twelve are delivery details and three are pricing, the fix is two new types, not a builder. A builder on a god object makes the god object easier to construct, which is not an improvement.”

A model answer

Asked: *this constructor takes twelve parameters. Fix it.*

“Before I reach for a pattern I want to ask whether these are twelve things. Looking at them — `line1`, `line2`, `city`, `pincode` are one address, and `card_number`, `expiry`, `cvv` are one payment method. If those collapse into two value objects I go from twelve parameters to seven, and I have also fixed a data clump that is probably repeated in other signatures. That is the first move, and it sometimes makes the rest unnecessary.

Assuming there are still too many, here is what is actually wrong. The call site is unreadable — positional booleans mean nobody can tell what `true`, `false`, `true` means without opening the signature. Most of the fields are optional, so the usual next step is a second constructor with fewer parameters, then a third, and now there are five overloads and people pass null to fit one. And there is nowhere for a rule that involves two fields to live.

In Python I would not write a builder. I would write a frozen, keyword-only dataclass with defaults for everything optional. That gives me named arguments in any order, so the call site is self-documenting; `kw_only=True` forbids positional arguments entirely, which kills the boolean problem; fields without defaults are required at construction, so I keep the compile-time check; the object is immutable; and `__post_init__` is a single place where the whole object exists and cross-field rules can be checked. That is about fourteen lines.

The reason Builder exists is that Java has neither keyword arguments nor default parameter values, so the fluent builder — Bloch’s version, one method per field returning `this`, and a `build()` — is how you get those two things. Worth distinguishing from the Gang of Four Builder, which has a Director and an interface so one construction process can produce several representations. I would only write that if I could name the second representation.

There are four cases where I would still write a builder in Python. When the pieces genuinely arrive at different times, from different code — parsing, or a stream. When steps are conditional, like adding priority support only for premium users. When assembly is expensive and I want it to happen once. And test data, which is where I use it most: an eighteen-line object setup becomes `an_order().for_customer('ravi').with_items(3).build()`, and when the class gains a field I update one default rather than sixty test files.

The cost I would state up front is that required fields move from a compile-time check to a run-time one — `build()` with no weight compiles and throws when it runs — and that every field now exists in two places, so adding one means remembering both.”

9 Recall card

- **Builder separates collecting the pieces from creating the object.** The product comes out complete and immutable; the messy half-built state lived in the builder and never escaped. Fluent version = one method per field, each `return self`, then `build()`.
- **`build()` is the point, not the chaining.** It is the one moment where the whole object exists and has not escaped, so it is the only place **cross-field invariants** can be checked — “a photo print needs a fondant top” involves two fields, so **no setter could ever catch it**.
- **In Python, start with `@dataclass(frozen=True, kw_only=True)` and defaults** — ~14 lines gives named args in any order, immutability, required fields still enforced at construction, and validation in `__post_init__`. **Java needs Builder because it has no keyword arguments and no default values.** A hand-written Python builder is ~45 lines and usually buys nothing.
- **Four cases where you still write one:** construction is genuinely **stepwise** · steps are **conditional** · assembly is expensive · **test data** — `an_order().for_customer("ravi").build()` turns 18 lines of setup (2 of them relevant) into 1, and a new field is 1 default instead of 60 edited test files.
- **State the costs:** required fields become a **run-time** check, not a compile-time one · the field list is **duplicated** on product and builder · the builder is mutable and not thread-safe. And ask first: **are these twelve parameters twelve things, or three?** `line1/line2/city/pincode` is a **data clump** — one `Address` — and extracting value objects often removes the need entirely. n optional fields = 2^n telescoping constructors; 12 fields = 4,096.

PRACTICE

Practice — When a hash map is the wrong answer

DSA topic: When a hash map is the wrong answer **System design topic:** Builder

Code these, in this order

One rule for the whole set: **solve each one with a hash map first, then take the map away.** Writing the easy answer and then removing it is exactly the interview shape, and doing it in that order here is what makes it available to you under pressure.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	Whether you notice the word “sorted” and drop the map for two pointers.
2	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	The marking trick, without the cleaning pass to complicate it.
3	Find the Duplicate Number	LeetCode 287 (Medium)	$O(1)$ space with the input read-only — so marking is banned too.
4	First Missing Positive	LeetCode 41 (Hard)	The bound nobody gives you, and the <code>-abs</code> line.

On problem 1, say what “sorted” bought you

Write the hash-map version, state its space, then write two pointers. Then answer out loud: what would you do if the array were *not* sorted and $O(1)$ space were still required? Say both the cost of sorting it yourself and whether you are allowed to.

On problem 3, notice the constraint that kills your first two answers

The array is read-only *and* space is $O(1)$, so neither the set nor the marking trick is available. The intended answer is Floyd’s cycle detection from [day 030](#), treating values as pointers. Say out loud why the values being in `1..n` is what makes the array a valid function to walk.

On problem 4, run `[1, 1]` before you submit

Write the marking line as `-numbers[v-1]` first. Run `[1, 1]`. Get `1` instead of `2`. Then change it to `-abs(numbers[v-1])` and run it again. Do this once and you will never write it wrong.

The three-cases drill

For each problem, say which of the three cases applies — space taken away, small dense keys, or order needed — and what you would use instead of a map:

1. “Find the duplicate, $O(1)$ space.”
2. “Count the letters in a string of a million characters.”

3. “Which orders were placed between these two timestamps?”
4. “What is the third-largest score?”
5. “The values are all between 1 and n. Find the missing one.”
6. “Return the elements in ascending order of frequency.”
7. “Is this ASCII string made of unique characters, in $O(1)$ space?”
8. “Which key comes immediately after ‘kumar’ alphabetically?”

Two of the eight need *two* structures, not one. Name them and say which two.

The space drill

Measure each on a million small integers, then tabulate:

1. `sys.getsizeof(list(range(1_000_000)))`
2. `sys.getsizeof(set(range(1_000_000)))`
3. `sys.getsizeof({i: 1 for i in range(1_000_000)})`

Then answer: why is the set roughly four times the list? Name the three reasons, and say which of them a plain list does not pay.

The small-n drill

Time a million membership tests against a set and against a list, at $n = 4, 8, 16, 64, 256, 1024$.

1. Tabulate the ratio at each size.
2. Where does the set become clearly worth it?
3. Now add the cost of *building* the set to the comparison. Where is the break-even now?
4. State the rule of thumb you would actually use in an interview, and say it is a rule of thumb.

The direct-addressing drill

For each, say whether direct addressing is appropriate and why:

1. Counting lowercase English letters.
2. Counting arbitrary Unicode characters.
3. Counting HTTP status codes seen in a log.
4. Counting user ids seen in a log, where ids are random 64-bit integers.
5. Counting values known to be between 1 and n, where n is the array length.
6. Counting ages of people, 0 to 120.

Two of the six would allocate absurd amounts of memory. Say how much, with arithmetic.

The marking drill

Using `[4, 3, 2, 7, 8, 2, 3, 1]`:

1. Run the marking pass by hand and write out the array after every step.

2. Which slots are still positive at the end? What does that mean?
3. Now do it with `-x` instead of `-abs(x)` and find the step where it goes wrong.
4. Reconstruct the original array from the marked one. What information was preserved, and where?
5. Say what would break if the array contained a zero, and what the cleaning pass does about it.
6. Say what would break if the array contained `10**9`, and what the cleaning pass does about it.

The ask-first drill

For each, say what question you would ask the interviewer before writing anything, and what you would do for each possible answer:

1. You want to sort the input in place.
2. You want to mark inside the input with sign bits.
3. You want to return the result in the original order.
4. You want to assume all values are positive.
5. You want to assume the values fit in a machine integer.

One of these five questions is worth asking in almost every array problem. Which, and why?

The twelve-parameter drill

Here is the constructor:

```
class Cake:
    def __init__(self, weight_kg, flavour, eggless, finish, message,
                  photo, delivery, delivery_time, tier_count, colour,
                  candles, note):
        ...
```

1. Write out one realistic call, positionally. Count how many arguments a reader must look up.
2. How many of the twelve are optional? How many combinations is that?
3. Find the data clumps. How many parameters collapse into value objects, and into what?
4. Rewrite it as a frozen, keyword-only dataclass with defaults. Count the lines.
5. Write two cross-field rules that no setter could enforce, and put them in `__post_init__`.
6. Now write the fluent builder version. Count the lines.
7. State what the builder buys over the dataclass here, honestly. If the answer is nothing, say so.
8. Name the compile-time guarantee the builder gave up.

The builder-or-not drill

For each, say whether you would write a builder in Python, and give the deciding reason:

1. A `Point` with `x` and `y`.
2. An HTTP request with a URL, method, ten optional headers, a body and a timeout.
3. An object assembled from three different callbacks as a file is parsed.
4. A configuration where premium users get two extra steps and free users do not.
5. An `Order` used in sixty tests, each of which cares about two fields.
6. A `Cake` with twelve fields, all required.
7. A `Report` whose assembly takes four seconds and should happen once.

Two of the seven are “no” for the same reason. Name it.

The test-data-builder drill

Write a test that needs an `Order` with a `Customer`, an `Address` and three `OrderLine`s, where the test only cares that the order is already paid.

1. Write the setup without a builder. Count the lines.
2. Count how many of those lines the test actually cares about.
3. Write a test data builder with sensible defaults.
4. Rewrite the setup with it. Count the lines again.
5. Now add a thirteenth field to `Order`. How many files change in each version?
6. Say which of those two numbers would actually persuade a team, and why.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *You used a hash map. Could you do it in $O(1)$ space instead?* What the map was buying, the three replacements named before one is chosen, the “may I modify the input” question asked out loud, and the bound found and stated before any code is written.
2. *This constructor takes twelve parameters. Fix it.* The value-object question first, then what is actually wrong with twelve, then the Python answer (frozen keyword-only dataclass) before the pattern answer, then the four cases where you would still build one, then the compile-time guarantee you gave up.
3. *When would you not use a hash map?* The three cases with an example each, plus the three smaller reasons — memory, tiny n , and adversarial keys — and the sentence about $O(1)$ being an average rather than a bound.

Before you move on

- [] I solved at least two problems with a map first and then removed it.
- [] I ran `[1, 1]` through First Missing Positive with `-x` and watched it return the wrong answer.

- [] I can state why the answer is bounded by $n + 1$, in one sentence, without hesitating.
- [] I asked “may I modify the input?” out loud before every in-place solution today.
- [] I measured list, set and dict memory on a million integers and can quote all three.
- [] I found the small- n break-even myself, including the cost of building the set.
- [] I can name the three ordering questions a hash map cannot answer, and what to use instead.
- [] I found the data clumps in the twelve-parameter constructor before reaching for a pattern.
- [] I wrote both the dataclass and the builder version and can say honestly what the builder added.
- [] I can name the compile-time guarantee Builder gives up, and the four cases where it is worth it.
- [] I answered all three questions above out loud.

Hashing revision and mock round

DATA STRUCTURES AND ALGORITHMS

Hashing revision and mock round

You can solve two unseen hash-map problems cold.

SYSTEM DESIGN

Prototype, and cloning objects

You can copy an object correctly and explain shallow versus deep copy.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *What is the difference between a shallow copy and a deep copy?*

1 What this is, and why they ask it

This closes the hashing phase. Over seven days you have met the hash table itself, collisions and the two strategies for handling them, sets and the seen-set reflex, frequency maps and the three ways to select a top k, grouping and the key-design skill, `__hash__` and `__eq__` on your own classes, and the three cases where a map is the wrong tool. Today adds no new material.

What today adds is **reaching for it under observation**. There is a specific failure that happens to people who have done all seven days properly: they are given an unseen problem, they know every component of the answer, and they spend four minutes writing a brute force in silence because nobody ever made them decide out loud. The knowledge was never the bottleneck. The bottleneck is having a procedure that runs when you are being watched.

They ask it as two unseen problems with no hints because that is exactly what the coding round is. Hash-map problems are the most common single category in that round — a hiring manager can build a whole 45-minute interview out of “count something, then group it, then take the top k” — and the marks come from three things almost nobody practises: naming the structure before writing anything, saying the invariant, and continuing to talk while stuck.

2 The story

Bashir does electrical work on call in and around Cooke Town, mostly for four or five buildings whose associations have his number. He has been doing it for nineteen years.

For most of those years his van was a jumble. One large canvas bag, a plastic crate, and things put back wherever there was room. He knew every tool in there. Ask him whether he had a particular size of connector and he would answer instantly and be right.

The trouble was never knowing. It was the four minutes.

A call at half past nine at night, a stairwell with the power off, him halfway up a ladder with a torch held in his teeth, and the thing he needs is somewhere in a bag on the floor. So he comes down, crouches, moves things, finds it, goes back up. Sometimes twice. His wife, who has heard about this for years, once pointed out that he spends more of the evening on the ladder going up and down than doing anything at the top of it.

What changed it was a young man he took on for a few months, who watched him for two days and then asked a question nobody had asked before: how do you decide what to take up the ladder?

Bashir did not have an answer, because he had never decided anything. He went up, found out, came down.

So they changed two things. The van got six boxes, labelled, and everything lives in one of them. And — this is the part that mattered — before he goes up, he stands at the bottom for about twenty seconds and says what he thinks the problem is and what he will need if he is right. Out loud, to himself, which felt ridiculous for the first week.

He is wrong perhaps one time in five, which is fine; he comes down once and goes up again. But being wrong once in five beats being unprepared five times in five, and the saying-it-out-loud does something he did not expect. Twice now he has got halfway through the sentence and stopped, because hearing himself say it made it obvious that it was not the fuse at all.

The tools were never the problem. Nineteen years of tools. The twenty seconds at the bottom of the ladder was the problem.

3 The idea in plain English

The six labelled boxes are the seven days you have done. The twenty seconds at the bottom of the ladder is what this day is about, and it is the only new thing here.

The decision procedure, in the order to run it

Run these five questions on any unseen problem, out loud, before touching the keyboard. It takes about forty seconds and it will pick the right structure nearly every time.

1. What is the one question I need to ask repeatedly?	
THE QUESTION	THE STRUCTURE
Have I seen this before?	a set
How many times has this appeared?	a frequency map
Where did I see it?	a map from value to index
Which things belong together?	a map from key to list
Which is the smallest / k-th / next?	not a map — heap or sorted array

2. What exactly is the key? Say the belonging sentence out loud: *two items are the same when* __. If the key is not the element itself, this is where the whole problem lives (day 064).

3. Does anything about the input remove the need for a map? Three triggers, from day 066: sorted input → two pointers · values bounded by n → index into the array itself · keys are small dense integers → a plain list.

4. What does it cost, and what am I trading? “O(n) time, O(n) extra space. I am buying time with memory.” Say it before writing, not after being asked.

5. What is the invariant? One sentence describing what is true at the top of every iteration. “*seen* holds exactly the elements before the current one.” This sentence is worth more than fast typing.

The one-page comparison

STRUCTURE	ANSWERS	TIME	SPACE	USE WHEN
set	present?	$O(1)$ avg	$O(n)$	dedupe, seen-check, cycle detection
dict	present + what	$O(1)$ avg	$O(n)$	need position, count, or payload
Counter	how many	$O(n)$ build	$O(m)$	frequency, anagram, top-k
defaultdict(list)	which group	$O(n)$ build	$O(nk)$	grouping by a computed key
plain list as table	present, dense int keys	$O(1)$ exact	$O(\text{range})$	letters, ASCII, values in $1..n$
sorted array + bisect	order, ranges	$O(\log n)$	$O(1)$ extra	k-th, next, between
heap	repeated min/max	$O(\log n)$	$O(k)$	top-k with small k, streaming

The five sentences that carry the phase

If you remember nothing else on the morning of an interview, remember these.

1. “A hash map does not search, it computes.” `bucket = hash(key) % capacity`, then a direct index. That one line is the whole structure (day 060).
2. “ $O(1)$ is an average, and it rests on a uniform hash and a table that grows.” The worst case is $O(n)$, and it is reachable deliberately (day 061).
3. “in on a set is $O(1)$; in on a list is $O(n)$; the code looks identical.” $700 \times$ at $n = 20,000$ (day 062).
4. “The key is the problem; the loop is four lines.” Say the belonging sentence, then run both tests — too coarse, too fine (day 064).
5. “A map buys time with memory and throws away structure.” Sorted, bounded, or dense means something cheaper exists (day 066).

What a mock round is actually scoring

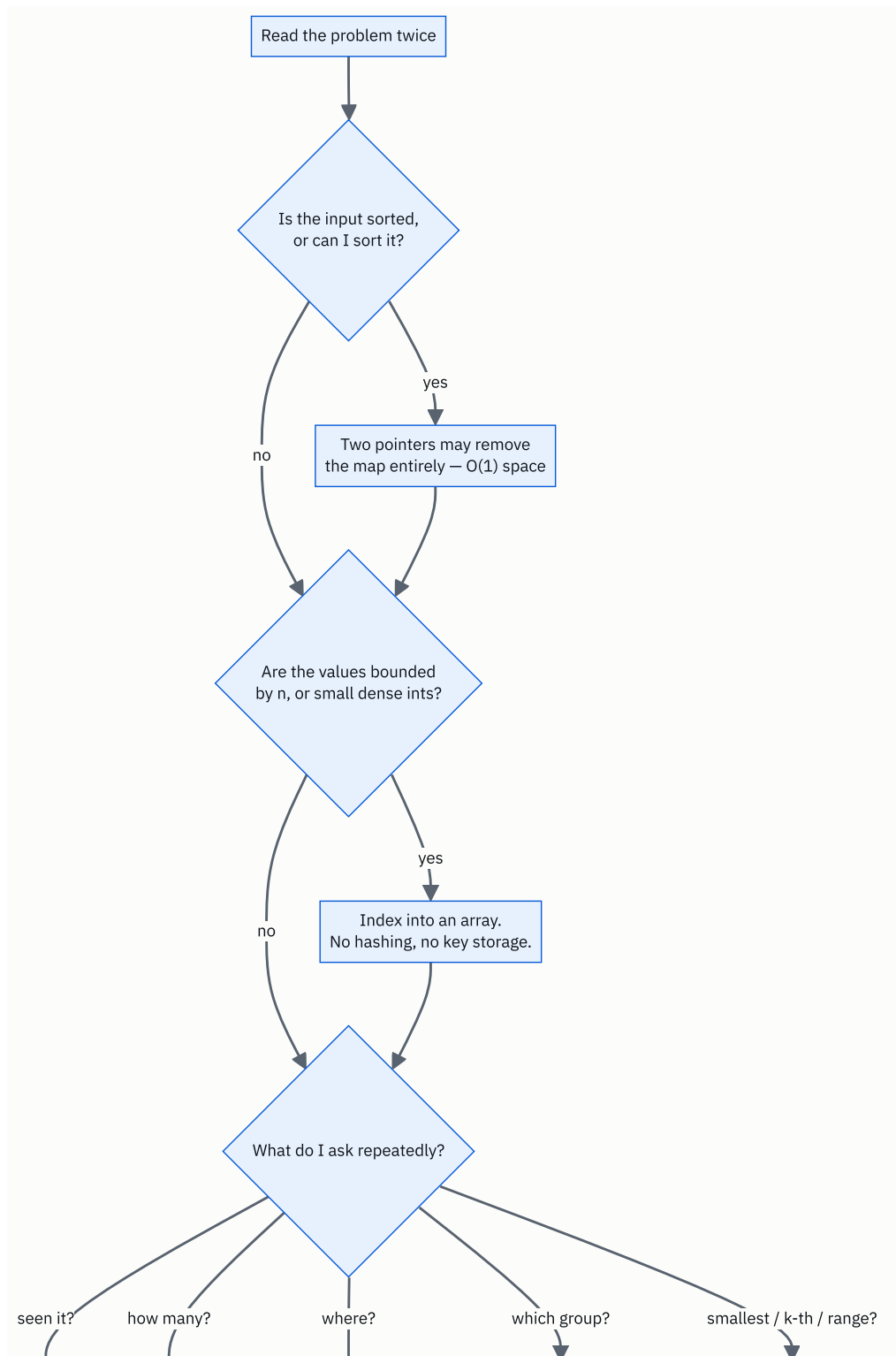
Not correctness alone. In most rubrics correctness is one of four or five lines, and the others are:

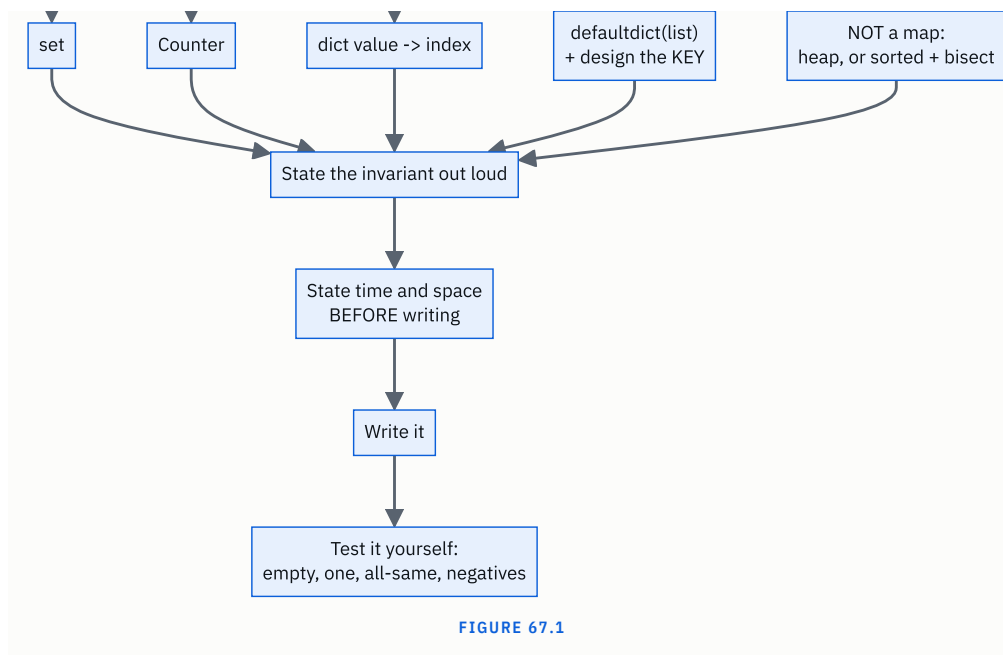
- **Communication** — did the interviewer always know what you were doing?
- **Approach** — did you name a structure and a complexity before coding, or discover them?
- **Correctness** — does it run, and did you test it yourself?
- **Edge cases** — empty, one element, all-identical, negatives — found by you, not by them.
- **Follow-up handling** — “now do it in $O(1)$ space” met with a plan rather than silence.

You can lose on three of those five with perfect code, and that is the thing worth internalising.

4 The picture

The decision procedure, as a flow you can run in forty seconds.





What to notice: the two questions that could remove the map come **first**. If you ask “what do I put in the dictionary” before asking “do I need a dictionary”, you will never find the $O(1)$ -space answer, because you will already have committed.

And the shape of a 45-minute round, so the clock stops being a surprise:

```

0-3   read, restate, ask clarifying questions
3-6   say the brute force and its cost. Do not write it.
6-10  name the structure, the key, the invariant, the complexity
10-25 write it, narrating
25-30 test it yourself on the four edge cases
30-40 the follow-up ("O(1) space", "k-th largest", "what if it does not fit")
40-45 their questions
  
```

The single most common mistake is spending 0-12 writing a brute force in silence. That is a quarter of the round spent producing something you were going to throw away.

5 The code, built step by step

Two problems, worked the way you should work them. Read the problem, then close the file and try it before reading on.

Problem one: “Given an array of strings, return the k most common words. If two words appear the same number of times, the more alphabetically earlier one comes first.”

Minutes 0-3, out loud. “So I need frequencies, then a selection of the top k , and the tie-break is alphabetical ascending while the frequency is descending. Let me check one thing: if k is larger than the number of distinct words, do I return everything? And are the words case-sensit-

ive?”

Minutes 3-6. “The brute force is: for each distinct word, count it by scanning the whole list, then sort. That is $O(n \cdot m)$ for the counting alone. I will not write it.”

Minutes 6-10. “Counting is a `Counter`, $O(n)$. Then the selection. Three options: sort all m distinct words, $O(m \log m)$; a heap of size k , $O(m \log k)$; or bucket by count, $O(n + m)$. The bucket version is normally the fastest, but it does not handle the tie-break — inside a bucket I would still have to sort alphabetically. Given the tie-break, I will use a heap with a composite key.”

The tie-break is the whole problem. Descending by count, ascending by word. Those go in opposite directions, so a single `reverse=True` cannot do it — this is the trap from [day 058](#).

```
from collections import Counter

counts = Counter(words)
ordered = sorted(counts, key=lambda word: (-counts[word], word))
return ordered[:k]
```

Negate the count to flip that component to descending while the string stays ascending. Say it as you write it: *“I negate the count so it sorts descending, and leave the word alone so it sorts ascending. I cannot use `reverse=True` because that would flip both.”*

For the heap version, the same key works because `heapq` is a min-heap over tuples:

```
import heapq
return [word for _, word in heapq.nsmallest(k, ((-c, w) for w, c in counts.items()))]
```

Minutes 25-30, testing yourself. `[]` with $k=0$. One word. All words identical. Two words with equal counts — check the alphabetical order actually came out right, because this is where it breaks.

Problem two: “You are given a list of log lines, each with a user id and a timestamp in seconds. A user is ‘suspicious’ if they made more than 100 requests in any 60-second window. Return the suspicious user ids.”

Minutes 0-3. “Two clarifications. Is the input sorted by timestamp? And is ‘any 60-second window’ a sliding window, or fixed minute buckets? Those give different answers and the second is much cheaper.”

That question is the entire interview. If it is fixed buckets, it is a grouping problem and you are done in six lines. If it is a true sliding window, you need a per-user deque.

Assume they say sliding. “Then I need, per user, the timestamps in order, and a window over them.”

Minutes 6-10. “So this is two structures composed: a `defaultdict(list)` grouping timestamps by user, and then per user a sliding window from [day 031](#). Group first, sort each user’s timestamps if the input is unsorted, then walk each list with two pointers, advancing the

left pointer while the window is wider than 60 seconds. The invariant is: everything between `left` and `right` inclusive is within 60 seconds of each other.”

```
from collections import defaultdict

def suspicious_users(logs: list[tuple[str, int]], limit: int = 100,
                    window: int = 60) → list[str]:
    by_user: dict[str, list[int]] = defaultdict(list)
    for user, timestamp in logs:
        by_user[user].append(timestamp)

    flagged: list[str] = []
    for user, times in by_user.items():
        times.sort()
        left = 0
        for right, current in enumerate(times):
            while current - times[left] ≥ window:
                left += 1
            if right - left + 1 > limit:
                flagged.append(user)
                break
    return flagged
```

Notice the `break`. Once a user is flagged, stop looking at them — otherwise you append the same user many times.

The follow-up you will get: “the log has ten billion lines and does not fit in memory.” Answer: “Then I do not group it in memory. If it is sorted by time I can stream it, holding one deque per active user and evicting users whose last request is more than 60 seconds old. If it is unsorted, I shard by user id — hash the user id modulo the number of machines, so all of one user’s lines land on one machine — and each machine does this independently. Counting and grouping are both partitionable by key, which is exactly why this is the shape MapReduce was built for.”

The reference implementations, for revision

```
from collections import Counter, defaultdict
from dataclasses import dataclass
import heapq

# ---- 1. the seen-set reflex (day 062) -----
def first_duplicate(numbers: list[int]) → int | None:
    """Invariant: at the top of each iteration, `seen` is exactly the
    elements before the current one."""
    seen: set[int] = set()
    for number in numbers:
        if number in seen:
            return number
        seen.add(number)
    return None

# ---- 2. map from value to index (day 060) -----
def two_sum(numbers: list[int], target: int) → tuple[int, int] | None:
    """A set is not enough here: the answer needs positions, so it is a dict."""
    position: dict[int, int] = {}
    for index, number in enumerate(numbers):
        if target - number in position:
            return position[target - number], index
        position[number] = index
    return None

# ---- 3. counting, and top k three ways (day 063) -----
def top_k_buckets(numbers: list[int], k: int) → list[int]:
    """O(n + m). No comparisons: a count is an integer in 1..n, so index by it."""
    counts = Counter(numbers)
    buckets: list[list[int]] = [[] for _ in range(len(numbers) + 1)]
    for value, count in counts.items():
        buckets[count].append(value)

    result: list[int] = []
    for count in range(len(buckets) - 1, 0, -1):
        for value in buckets[count]:
            result.append(value)
            if len(result) == k:
                # inside: one bucket can hold many
                return result
    return result

def top_k_heap(numbers: list[int], k: int) → list[int]:
    """O(m log k). Prefer this when n is huge and m is small."""
    counts = Counter(numbers)
    return heapq.nlargest(k, counts, key=counts.get)

# ---- 4. grouping by a computed key (day 064) -----
def group_anagrams(words: list[str]) → list[list[str]]:
    """Key: sorting destroys order (must not matter) and keeps the letter
    multiset (must matter)."""
    groups: dict[str, list[str]] = defaultdict(list)
    for word in words:
        groups["".join(sorted(word))].append(word)
    return list(groups.values())

# ---- 5. your own objects as keys (day 065) -----
@dataclass(frozen=True)
class Point:
    """frozen=True gives __eq__ and __hash__, and makes rule 2 unbreakable."""
    x: int
    y: int

# ---- 6. when the map is the wrong answer (day 066) -----
def first_missing_positive(numbers: list[int]) → int:
```

```

"""O(n) time, O(1) space. The answer is bounded by n+1, so index into the
array itself and store the 'seen' flag in the sign bit."""
n = len(numbers)
for i in range(n):
    if numbers[i] ≤ 0 or numbers[i] > n:
        numbers[i] = n + 1
for i in range(n):
    value = abs(numbers[i])
    if value ≤ n:
        numbers[value - 1] = -abs(numbers[value - 1]) # -abs, never -x
for i in range(n):
    if numbers[i] > 0:
        return i + 1
return n + 1

def two_sum_sorted(numbers: list[int], target: int) → tuple[int, int] | None:
    """Sorted input: two pointers, O(n) time and O(1) space. No map."""
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return left, right
        if total < target:
            left += 1
        else:
            right -= 1
    return None

if __name__ == "__main__":
    print(first_duplicate([2, 1, 3, 1])) # 1
    print(two_sum([2, 7, 11, 15], 9)) # (0, 1)
    print(top_k_buckets([1, 1, 1, 2, 2, 3], 2)) # [1, 2]
    print(top_k_heap([1, 1, 1, 2, 2, 3], 2)) # [1, 2]
    print(group_anagrams(["eat", "tea", "tan", "ate"])) # [['eat', 'tea', 'ate'], ['tan']]
    print(len({Point(1, 2), Point(1, 2)})) # 1
    print(first_missing_positive([3, 4, -1, 1])) # 2
    print(two_sum_sorted([2, 7, 11, 15], 9)) # (0, 1)

```

6 What it costs

The whole phase, priced at $n = 1,000,000$

build a set / Counter	1,000,000 hash+insert	~0.15 s
one membership test	1	~40 ns
the same test against a list	up to 1,000,000 comparisons	~5 ms
group into $m = 100,000$ groups	1,000,000 hash+append	~0.35 s
sort the 100,000 counts	$100,000 \times 17 = 1.7\text{M}$ cmp	~0.05 s
heap of size 10 over 100,000	$100,000 \times 3.3 = 330\text{K}$ cmp	~0.02 s
bucket by count	$1,000,000 + 100,000$	~0.06 s

The three arithmetic facts to have ready

Nested loop versus map. $\frac{n(n-1)}{2}$ against n . At $n = 10,000$ that is 49,995,000 against 20,000 — a factor of 2,500.

Set versus list membership. At $n = 20,000$, 0.004 s against 2.9 s — a factor of about 700, from identical-looking code.

Memory per million integers. `list` 7 MB · `set` 32 MB · `dict` 40 MB. The map costs roughly 4.5 to 5.7 times the raw data, because it stores hashes, holds the table two-thirds empty, and keeps pointers.

Space, side by side

set / dict of n items	$O(n)$ extra
grouping n items of size k	$O(nk)$ – and it is not optional, the output is that big
bucket top- k	$O(n + m)$ – $n+1$ buckets whatever m is
heap top- k	$O(m + k)$
direct addressing, dense	$O(\text{range})$ – 8 GB if the range is a billion
marking in place	$O(1)$ – and it destroys the input

The complexity claim that gets probed

Three loops that are *not* what they look like, and you should be able to defend all three:

- **Longest consecutive's inner `while`** is $O(n)$ *in total*, not per iteration, because only a run's smallest element enters it ([day 062](#)).
- **Bucket top- k 's nested loops** are $O(n + m)$ total, because the buckets partition the m distinct values.
- **Resizing a hash table** is $O(n)$ on one unlucky insert but **amortised $O(1)$** , because $8 + 16 + 32 + \dots + n/2 \approx n$ ([day 060](#)).

7 The traps

Every trap from the phase, in one place. Each is a real error or a real wrong answer.

`seen = []` instead of `seen = set()`

Identical code, $O(n^2)$ instead of $O(n)$, $700\times$ slower at $n = 20,000$. The first thing to check when something is mysteriously slow.

`{}` is an empty dict

The empty set is `set()`. `{}` gives you a dictionary, and `.add` on it raises `AttributeError: 'dict' object has no attribute 'add'`.

`counts[key] += 1` on a missing key

```
KeyError: 'pulao'
```

Use `counts.get(key, 0) + 1`, `defaultdict(int)`, or `Counter`.

Reading a missing key from a `defaultdict` inserts it

```
>>> d = defaultdict(int); d["zzz"]; len(d)
1
```


`len` is now wrong and iteration yields a key that was never in the data. With `defaultdict(list)` it puts an empty group in your output. Check with `in`, never by indexing.

`Counter` returns 0 for typos

`c["bb"]` is 0, not an error. When a count that should be non-zero comes back as zero, suspect the key before the logic.

`words.count(word)` inside a loop

$O(n^2)$ in two clean-looking lines. 400,000,000 comparisons at $n = 20,000$.

Walking the counter instead of the original

For any “first such element” question, iterate the input, not the map. It is right by accident on `Counter` and stops being right the moment anything reorders it.

An unhashable key

```
TypeError: unhashable type: 'list'
```

`tuple(counts)`, not `counts`. `frozenset` for an unordered group.

`itertools.groupby` on unsorted input

It only groups **adjacent** runs, so `aet` appears three separate times. And `sorted(words)` is not enough — it must be `sorted(words, key=key)`.

A key that is too coarse or too fine

`frozenset(word)` groups `"aab"` with `"abb"`. `(city, pincode, area)` splits one city into forty piles. Run both tests before writing the loop.

`__eq__` without `__hash__`

```
TypeError: unhashable type: 'Q'
```

Python sets `__hash__ = None` when you define `__eq__`. A plain `@dataclass` is unhashable for the same reason. Use `frozen=True`.

Mutating a key while it is in the table

The worst one, because it is silent:

```
>>> key.key = "A-2"
>>> key in lookup, len(lookup)
(False, 1)
```

No exception. `frozen=True` makes it impossible.

`-x` instead of `-abs(x)` in the marking trick

Returns `1` instead of `2` on `[1, 1]`. Two elements.

Changing a container while iterating it

```
RuntimeError: dictionary changed size during iteration
RuntimeError: Set changed size during iteration
```

Iterate over `list(d)`.

Relying on set order

`list(set(names))` gives a different order in a fresh process, because string hashing is randomised. Use `dict.fromkeys` when order matters.

Direct addressing on a sparse range

`[0] * (max(numbers) + 1)` on `[1, 2, 10**9]` is 8 GB and a `MemoryError`. The condition is **dense**.

Quoting $O(1)$ as a worst case

It is an average. Say “ $O(1)$ average, $O(n)$ worst” every time, and know when the worst case matters.

8 In the interview

How it gets asked

- The standard round: two problems, forty-five minutes, no hints. The first is a warm-up that is really checking whether you talk; the second is the one they score.
- The composed one, which is now common: “*count them, then group them, then give me the top three per group.*” Three phase ideas in one question.
- The scaling pivot at minute 35: “*now the input is ten billion lines.*”
- The space pivot: “*now do it in $O(1)$ space.*”

The script, minute by minute, for a 45-minute round

Minutes 0-3 — read it twice and restate it. “Let me say back what I think you are asking.” Then ask two clarifying questions and no more. The best two are almost always: *what should happen on an empty input or a tie?* and *does the order of the output matter?*

Minutes 3-6 — say the brute force and refuse to write it. “The obvious approach is a nested loop — for each element, look at every other. At $n = 10,000$ that is fifty million comparisons. I would like to do better, so let me not write that.”

Minutes 6-10 — the forty seconds at the bottom of the ladder. Out loud, in this order: the repeated question, the structure, the key, the invariant, the complexity. “The question I keep asking is ‘have I seen this’. That is a set. The key is the value itself. The invariant is that `seen` holds exactly the elements before the current one. $O(n)$ time, $O(n)$ extra space.”

Minutes 10-25 — write it, narrating. Short sentences. “Now I add it to the set, so the invariant holds for the next iteration.” Silence is the thing that costs you, not slowness.

Minutes 25-30 — test it yourself, out loud, on four inputs. Empty. One element. All identical. Negatives or an edge value. Finding your own bug here is worth more than not having one, because it demonstrates the habit.

Minutes 30-40 — the follow-up. Have the four standard answers ready; they are below.

Minutes 40-45 — ask them something real. What does the on-call rotation look like; what broke most recently.

The follow-ups

“Now do it in $O(1)$ space.” “Then the map has to go, and I have three options. Sort and walk — $O(n \log n)$ time, $O(1)$ space, and it destroys the input. Two pointers, if it is sorted. Or, if the values are bounded by n , mark inside the array itself using the sign bit, which stays $O(n)$. Which is available depends on one question: may I modify the input?”

“What is the worst-case complexity?” “ $O(n)$ per operation, so $O(n^2)$ overall, when everything collides into one bucket. It needs either a hostile input or a bad custom hash. Python randomises string hashing per process to stop the first, which is also why you must never persist or assert on a `hash()` value.”

“The input does not fit in memory.” “Counting and grouping are both partitionable by key, so I shard: hash the key modulo the number of machines, so all records with one key land on one machine, and each machine does the job independently. Then merge the partial counters, which works because addition is associative. That is exactly the MapReduce shape. If I only need approximate frequencies in fixed memory, Count-Min Sketch does it with a bounded overestimate.”

“Why is your inner loop not quadratic?” “Because it is bounded in total rather than per iteration.” Then give the specific reason for the specific loop.

“Would you use a set or a dict here?” “A set if presence is all I need; a dict the moment I need the position, the count, or a payload. The set is the dict with the values thrown away.”

A model answer

Asked, with no preamble: *given a list of words, return the k most frequent. Ties go to the alphabetically earlier word.*

“Let me restate it: I need the k words with the highest counts, and where two words have the same count, the one that comes first alphabetically wins. Two questions before I start — if k is larger than the number of distinct words, do I return all of them? And are the words case-sensitive?

The brute force is to count each distinct word by scanning the whole list, which is $O(n \cdot m)$, then sort. I will not write that; counting in one pass with a hash map is $O(n)$ and there is no reason not to.

So the first half is a `Counter`, one pass, $O(n)$ time and $O(m)$ space where m is the number of distinct words.

The second half is the selection, and normally I would offer three options: sort the m counts at $O(m \log m)$, a heap of size k at $O(m \log k)$, or bucket by count at $O(n + m)$, which is linear because a count is an integer between 1 and n so I can index by it rather than compare.

But the tie-break changes the answer. I need count descending and word ascending — two components going in opposite directions — so a single `reverse=True` will not work, because it would flip both. The clean way is a composite key: negate the count and leave the word alone. Bucketing does not handle this naturally, because inside each bucket I would still have to sort alphabetically. So I will use the sort with the composite key, or a heap with the same key if k is much smaller than m .

That is $O(m \log m)$ time on top of the $O(n)$ counting, and $O(m)$ space.

Let me test it. Empty input with k of zero returns an empty list. A single word returns that word. All words identical gives one entry. And the case I actually want to check is two words with equal counts — ‘apple’ and ‘banana’ both twice — which should return ‘apple’ first, and it does, because the negated count ties and the string comparison breaks it.

If you told me the list was ten billion lines, I would not count it in one process. I would shard by hashing the word, so all copies of a word land on one machine, count independently and merge the partial counters — counting is associative, which is what makes that safe.”

9 Recall card

- **Run the five questions before touching the keyboard.** What do I ask repeatedly? · What exactly is the key? · Does the input remove the need for a map (sorted / bounded by n / dense ints)? · What am I trading? · What is the invariant? The two that could **remove** the map come first, or you will commit before you can see them.
- **Structure by question:** *seen it?* → set · *how many?* → `Counter` · *where?* → dict value→index · *which group?* → `defaultdict(list)` + a designed key · *smallest / k -th / next / between?* → **not a map** — heap, or sorted + `bisect`.

- **Five sentences carry the phase:** a map computes, it does not search · **$O(1)$ is an average**, $O(n)$ worst · `in` on a list is $O(n)$ and looks identical ($700 \times$ at $n = 20,000$) · the key is the problem, the loop is four lines · a map buys time with memory and **throws away structure**.
- **Numbers to have ready:** nested loop vs map at $n = 10,000 = 50M$ vs **20K** ($2,500 \times$) · memory per million ints = **list 7 MB, set 32 MB, dict 40 MB** · top-k at $n = 10^6$, $m = 10^5$, $k = 10 = 1.7M$ (sort) vs 330K (heap) vs 1.1M (buckets).
- **The round is scored on five lines, not one.** Communication · approach *before* code · correctness · **edge cases you found yourself** (empty, one, all-same, negatives) · follow-up handling. Minutes 0-12 spent writing a brute force in silence is a quarter of the interview thrown away.

1 What this is, and why they ask it

Prototype is the creational pattern that makes new objects by **copying an existing one** instead of building one from scratch. The existing object is the prototype; `clone()` produces a new object just like it, which you then adjust. It is the last of the five creational patterns and the one people meet without knowing its name, because every language ships some form of copy.

The pattern itself takes five minutes. The reason this day exists is the question underneath it: **what does “copy” actually mean?** A **shallow copy** makes a new outer object whose fields point at the same inner objects as the original. A **deep copy** copies the whole graph, all the way down. The difference is invisible until somebody mutates a shared inner object, and then two things that were supposed to be independent change together.

They ask it because it is a real bug that real teams lose days to, and because the answer separates three levels of candidate. The weak answer is “shallow copies one level, deep copies everything”. The better answer draws what the pointers look like and names the input that exposes the difference. The strong answer volunteers the three things a deep copy gets *wrong* — cycles, cost, and objects that must not be copied at all, like a socket or a lock.

2 The story

When Divya moved into her own flat in Kaggadasapura, her mother Sarala did not sit down and work out what a flat needs. She had been running one for thirty-one years, and hers works, so she went round her own kitchen and copied it.

Same size pressure cooker. Same steel dabbas, the set of six. The same brand of gas stove, because hers had lasted eleven years. The same arrangement — oil and salt to the right of the stove, spices in the second drawer — because when you have cooked in a kitchen for thirty-one years you stop thinking about where things are and that is worth something.

It took an afternoon, and Divya’s kitchen worked from the first day, which a kitchen designed from first principles by a twenty-six-year-old would not have.

Most of what was copied was genuinely Divya’s own. Her own cooker. Her own dabbas. If Divya’s cooker whistle broke, nothing happened in Sarala’s kitchen.

But two things were not copied. They were shared, and nobody noticed at the time.

The milk. Sarala rang the man who has delivered to her for years and asked him to deliver to Divya’s flat too, and he said fine, and added the address. One arrangement, two addresses. And the newspaper, the same way — one account, an extra copy dropped at the second

address.

At the time this seemed like the same thing as everything else. It was not.

In March, Sarala went to her sister's in Mysuru for five weeks and did what she always does before travelling: rang both men and stopped everything.

Divya's milk stopped. Divya's newspaper stopped. Nobody had touched anything at Divya's end, nobody had rung anybody, and there was no reason on earth for it to happen. It took the two of them about a week and three fairly irritated phone calls to work out why, because they were both looking for something that had gone wrong at Divya's flat, and nothing had.

There was never a second arrangement. There was one arrangement, with two addresses written on it.

They fixed it in twenty minutes — Divya rang both men and opened her own account. And Sarala's summary of it is that when you copy something you have to know which parts came with it and which parts are still the original.

3 The idea in plain English

Copying the kitchen instead of designing one is **Prototype**: you already have a working instance, so make new ones from it rather than from nothing.

The cooker and the dabbas were **deep-copied** — genuinely new things that happen to look the same. The milk and the newspaper were **shallow-copied** — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

The two kinds of copy, precisely

An object has fields. Some fields hold simple values, like a number or a string. Some hold references to other objects.

A **shallow copy** creates a new outer object and copies each field *as it is*. A field holding the number 5 gets a new 5. A field holding a reference to a list gets a copy of the reference — which means both objects now point at **the same list**.

A **deep copy** creates a new outer object and, for every field that is a reference, recursively copies the object it points to as well. Nothing is shared.

```
import copy

original = {"name": "sarala", "milk": ["monday", "tuesday"]}

shallow = copy.copy(original)
deep = copy.deepcopy(original)

shallow["milk"].append("wednesday")

original["milk"]      # ['monday', 'tuesday', 'wednesday'] ← changed!
shallow["milk"]       # ['monday', 'tuesday', 'wednesday']
deep["milk"]          # ['monday', 'tuesday']              ← independent
```

Three lines to reproduce, and it is the whole question. Note that `original` changed even though nobody touched `shallow`. That is the bug, and it is why teams lose days to it: the symptom appears somewhere nobody was working.

When shallow is fine, which is more often than people think

Shallow copying is not the broken option. It is the right option whenever the shared parts are **immutable**.

```
person = {"name": "sarala", "age": 58}
copy.copy(person)      # completely safe – strings and ints cannot be mutated
```

Sharing an immutable object is invisible, because nobody can change it out from under you. This is why the real rule is not “always deep copy” but:

Shallow copy is safe exactly when everything it shares is immutable.

And it is why a codebase full of frozen dataclasses, tuples and strings almost never has this bug in the first place. **The best answer to the copy question is usually to have less mutable state**, and saying that is worth a lot.

The three things a deep copy gets wrong

This is the part that separates answers.

One: cycles. If A holds B and B holds A, a naive recursive copy runs forever. Python’s `copy.deepcopy` keeps a `memo` dictionary of objects it has already copied, keyed by `id()`, so a cycle terminates and — importantly — the *shape* of the sharing is preserved. If two fields pointed at the same list before, they point at the same *new* list after, rather than at two copies.

Two: cost. `deepcopy` walks the entire reachable object graph. On a large structure it is slow — often ten to a hundred times slower than shallow — and it doubles the memory. On an object holding a 100 MB cache, a deep copy is a 100 MB allocation you probably did not want.

Three: things that must not be copied. A socket. A file handle. A database connection. A threading lock. An open transaction. These are not data; they are handles to something outside your process, and copying them produces either an error or, worse, a second object that thinks

it owns a resource it does not.

```
def __deepcopy__(self, memo):
    clone = MyService(self.config)
    clone._connection = None      # deliberately NOT copied – reconnect on use
    return clone
```

Being able to say “a deep copy of anything holding a connection is wrong, and here is the hook where you exclude it” is the sentence that marks out someone who has done this in production.

Prototype, as a pattern

The pattern is: keep a registry of ready-made, fully configured objects, and create new ones by cloning them.

```
PROTOTYPES: dict[str, Report] = {
    "monthly_sales": Report(layout=..., filters=..., branding=...),
    "tax_summary": Report(layout=..., filters=..., branding=...),
}

def new_report(kind: str) → Report:
    return copy.deepcopy(PROTOTYPES[kind])
```

Three reasons to do this instead of calling a constructor.

Construction is expensive. If building the object means reading a database, parsing a large file, or a network call, copying an already-built one skips all of it.

You do not know the concrete class. `copy.deepcopy(thing)` produces the right subclass without you naming it. This was the original 1994 motivation, in a language where you could not easily pass a class around.

You want a configured template. The prototype has forty fields already set to sensible values and you only change two.

In Python the second reason mostly evaporates, because classes are first-class objects and a factory ([day 065](#)) does the job. The first and third survive.

4 The picture

The two copies, drawn as boxes and arrows. This is the diagram to reproduce in an interview.

ORIGINAL

```
+-----+
| Kitchen |
| name:  "sarala" |
| milk:  -----> [ mon, tue ]    ← one list object
+-----+
```

SHALLOW COPY

copy.copy

```
+-----+
| Kitchen |
| name:  "sarala" |
| milk:  ----- \
+-----+         \
                  \
original.milk -----> [ mon, tue ]    ← THE SAME list
                                both point here
```

(a new string reference, but strings are immutable, so it does not matter)

DEEP COPY

copy.deepcopy

```
+-----+
| Kitchen |
| name:  "sarala" |
| milk:  -----> [ mon, tue ]    ← a NEW list
+-----+
```

original.milk -----> [mon, tue] ← the old one, untouched

What to notice: the outer box is new in *both* cases. The only difference is whether the arrow points somewhere new. That is the entire distinction, and drawing it takes fifteen seconds.

And the failure, as a sequence — this is the shape of the bug report you will actually receive:

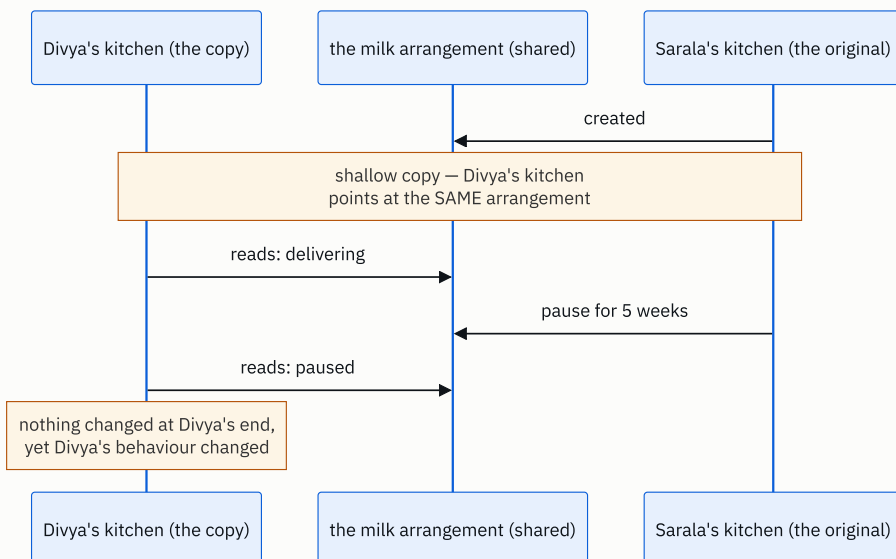


FIGURE 67.2

The reason this costs a week rather than an hour is in the last note. The symptom appears in code nobody has touched.

5 How it actually works

Python's three levels

```
import copy

b = a                # 1. not a copy at all – another name for the same object
b = copy.copy(a)     # 2. shallow – new outer object, shared innards
b = copy.deepcopy(a) # 3. deep – new everything, recursively
```

Level 1 catches beginners constantly: `b = a` on a list means `b.append(1)` changes `a`. It is not a copy; it is a second reference.

For built-ins there are idiomatic shallow copies: `list(a)`, `a[:]`, `a.copy()`, `dict(a)`, `set(a)`. All shallow, all the same thing.

The hooks

```
class Config:
    def __copy__(self):
        ...                # customise copy.copy

    def __deepcopy__(self, memo):
        ...                # customise copy.deepcopy; memo handles cycles
```

`memo` is the dictionary of already-copied objects keyed by `id()`. If you write `__deepcopy__` and recurse, you must pass `memo` down or you will lose cycle protection.

The Python answer that usually beats copying

```
from dataclasses import dataclass, replace

@dataclass(frozen=True)
class Report:
    title: str
    rows: tuple[str, ...]
    branding: str

monthly = Report("Monthly", ("a", "b"), "acme")
weekly = replace(monthly, title="Weekly")    # a new object, two fields shared
```

`dataclasses.replace` builds a new instance with some fields changed. On a **frozen** dataclass with immutable fields this is both a prototype *and* a safe shallow copy, because nothing shared can be mutated. It is one line, it is faster than `deepcopy`, and it has no cycle problem. Python 3.13 added a general `copy.replace` that works the same way for any type supporting it.

Java, and why `Cloneable` is a cautionary tale

Java's `Object.clone()` and the `Cloneable` interface are the standard example of a broken design, and Joshua Bloch says so in *Effective Java* in almost those words.

- `Cloneable` is a **marker interface** with no methods, so it does not actually give you `clone()`.
- `Object.clone()` is `protected`, so you must override it just to make it callable.
- It performs a **field-by-field shallow copy** and does not call a constructor, so `final` fields cannot be reassigned in it.
- It throws a checked `CloneNotSupportedException` that in practice can never happen.

The recommended replacement is a **copy constructor** or a static factory — `new ArrayList<other>` — which is explicit, calls a real constructor, and can be written to copy as deeply as you need. Naming this in an interview is a strong signal.

Real products, and where you have already met this

- **numpy views versus copies.** `arr[1:]` is a **view** — a window onto the same memory — while `arr[1:].copy()` is a copy. Assigning to a view changes the original. This is the single most common shallow-copy surprise in data work.
- **pandas' `SettingWithCopyWarning`** exists entirely because users cannot tell whether a slice is a view or a copy.
- **JavaScript's `Object.assign` and `{...spread}`** are shallow. Nested objects stay shared, which is why `structuredClone` was added to browsers in 2022 and why libraries like Immer exist.
- **Protocol Buffers** generate `CopyFrom` and `MergeFrom` on messages; the builders from [day 066](#) are the other half of the same design.
- **Docker images are prototypes.** A container is a copy of an image with a writable layer on top — and it is copy-on-write, so nothing is duplicated until it is written to.
- `fork()` is the operating-system version: the child is a copy of the parent, and the pages are shared copy-on-write until one of them writes.
- **Kubernetes** deployments hold a pod *template*; every pod is a clone of it with a few fields changed.
- **JavaScript's prototypical inheritance** is the pattern taken to its conclusion: objects inherit from other objects rather than from classes.

Copy-on-write, which is the good compromise

Three of those examples — `fork`, Docker layers, and Python's own string interning — use the same trick. Share everything, and only make a real copy at the moment somebody writes. You get the speed of a shallow copy with the safety of a deep one, and you pay only for what actually diverges. When an interviewer asks how to make deep copies affordable, this is the answer.

6 The numbers

What deep costs over shallow

On a dictionary of 1,000 keys, each holding a list of 100 integers:

b = a	~ 0.00003 ms	(no copy at all — one reference)
copy.copy(a)	~ 0.02 ms	(1,000 references copied)
copy.deepcopy(a)	~ 25 ms	(100,000 integers + 1,000 lists)
ratio	~ 1,200x	

And the memory: shallow adds one dictionary of 1,000 slots, roughly 40 KB. Deep adds 1,000 new lists of 100 elements, roughly 4 MB — **a hundred times more**.

`deepcopy` is also slow beyond the raw work, because it maintains the `memo` dictionary and dispatches per type. It is typically the slowest thing in any code path that calls it in a loop, and “we call `deepcopy` inside a request handler” is a real and common production finding.

When Prototype actually pays

The case for cloning over constructing:

build a Report from scratch:	
read layout from Postgres	12 ms
parse the branding template	4 ms
fetch the org's settings (HTTP)	40 ms

total	56 ms
clone a prebuilt prototype:	
deepcopy of a ~50 KB object	0.4 ms

140× faster, and at 200 reports a second that is 11.2 seconds of work per second — impossible — against 0.08 seconds. That arithmetic is the reason the pattern exists.

But notice the condition. If construction were `Report()` with three field assignments, cloning would be *slower* than constructing, because `deepcopy` is not cheap. **Prototype only pays when construction is expensive**, and quoting both halves is what makes the answer credible.

The cost of the bug

The shallow-copy bug has a characteristic cost profile, and it is worth knowing why it is expensive out of proportion to its size:

lines of code involved	1
time to fix once found	~5 min
time to find	hours to days

Because the symptom appears in code nobody changed. Sarala’s five-week trip broke Divya’s milk. The person debugging starts at the wrong end of the program, and every hypothesis they form is about the place where the symptom is.

7 The trade-offs

What deep copying costs you

Speed and memory, at roughly the ratios above. Anything in a hot path that deep-copies a large object is a performance bug waiting to be found.

Correctness, for anything holding a resource. Sockets, file handles, database connections, locks, open transactions, thread handles. A deep copy either raises — `TypeError: cannot pickle '_thread.lock' object` is the one you will actually see — or silently produces an object that believes it owns something it does not.

Surprising sharing you did not ask for. `deepcopy` preserves the *shape* of sharing. If your object had two fields pointing at the same list, the copy has two fields pointing at one new list, not two separate lists. That is almost always what you want and it is occasionally a surprise.

What shallow copying costs you

Exactly one thing: **shared mutable state**, and the debugging cost above. Nothing else. If you can guarantee the shared parts are immutable, shallow copying is strictly better on every axis.

The third option, which is usually the right one

Do not copy. Make it immutable. The entire shallow-versus-deep question only exists because things can be mutated. With frozen dataclasses, tuples and strings:

- shallow copying is free and safe;
- `dataclasses.replace` gives you “the same but with one field different” in one line;
- there is nothing to defend against.

This is the answer to give when an interviewer asks how you would *avoid* the problem rather than solve it.

“I would not use this if...”

- **...construction is cheap.** A constructor with three assignments beats `deepcopy` on speed and on clarity. Prototype is for objects that cost 50 ms to build, not 50 nanoseconds.
- **...the object holds a resource.** Then a clone is not a clone. Write a copy constructor that reconnects, or do not copy it.
- **...I can make it immutable instead.** Which removes the question rather than answering it.
- **...I am reaching for `deepcopy` to fix a bug I do not understand.** `deepcopy` as a defensive reflex sprinkled through a codebase is a strong sign that ownership of mutable state was never decided, and it will be slow *and* still buggy.

The honest weakness of Prototype

The clone starts out identical to something that is already in use. If the prototype is ever mutated — by anyone, at any time — every future clone silently changes. So the prototype registry itself should be immutable or, at minimum, treated as write-once at startup. This is the same discipline the singleton discussion needed in [day 064](#), for the same reason.

8 In the interview

How it gets asked

- The direct one, and it is extremely common: *“What is the difference between a shallow copy and a deep copy?”* Often as a warm-up, and often followed by “show me”.
- The bug hunt: *“I changed this object and a completely different object changed too. What happened?”*
- The Java one: *“What is wrong with `Cloneable`?”* — a question with a well-known answer that most candidates do not have.
- The pattern one: *“When would you use Prototype instead of a factory?”*
- The practical one: *“How would you copy an object that holds a database connection?”*

What to say out loud, in the first ninety seconds

1. **Define both in one sentence each, in terms of pointers.** “A shallow copy makes a new outer object whose fields point at the same inner objects. A deep copy recursively copies the whole reachable graph, so nothing is shared.”
2. **Draw the boxes and arrows.** Fifteen seconds, and it does more than sixty seconds of talking.
3. **Give the three-line reproduction.** A dict with a list in it, `copy.copy`, append, print both.
4. **State the real rule rather than “always deep copy”.** “Shallow is safe exactly when everything shared is immutable — which is most of the time in a codebase using frozen dataclasses and tuples.”
5. **Volunteer the three things deep copy gets wrong** — cycles, cost, and resources — before being asked. That is the part that ends the question in your favour.

The follow-ups

“How does `deepcopy` handle a cycle?” “With a memo dictionary keyed on `id()`. Before copying an object it checks whether it has already copied that one, and if so returns the existing copy. So a cycle terminates, and the shape of sharing is preserved — two fields that pointed at the same list still point at one list afterwards, not two.”

“How do you copy an object holding a database connection?” “You do not copy the connection. I would write `__deepcopy__` explicitly, copy the configuration and the data, and set the connection to `None` so it is re-established on next use. If I did not do that, Python would usually raise — `TypeError: cannot pickle '_thread.lock' object` is the one you see in practice — and if it somehow succeeded, I would have two objects believing they own one socket, which is worse than the error.”

“What is wrong with Java’s `Cloneable`?” “It is a marker interface with no methods, so implementing it does not give you `clone()`. `Object.clone` is protected, so you have to override it just to call it. It does a field-by-field shallow copy without calling a constructor, so

`final` fields cannot be set. And it throws a checked exception that can never happen. Bloch's recommendation is a copy constructor or a static factory instead — `new ArrayList<>(other)` — which is explicit and calls a real constructor.”

“When is Prototype better than a factory?” “When construction is genuinely expensive and the result is reusable — if building the object means a database read, a parse and an HTTP call, cloning a prebuilt one turns 56 milliseconds into 0.4. And when I want a fully configured template where I change two fields out of forty. If construction is cheap, a factory or just a constructor is better, because `deepcopy` is not free — it is often slower than building a small object from scratch.”

“How would you make deep copies affordable?” “Copy-on-write. Share everything, and only materialise a real copy when somebody writes. That is what `fork()` does with memory pages, what Docker does with image layers, and what persistent data structures in functional languages do. You pay only for what actually diverges.”

“How would you avoid the problem entirely?” “Immutability. The whole shallow-versus-deep question exists because things can be mutated. With frozen dataclasses and tuples, shallow copying is safe by construction, and `dataclasses.replace` gives me ‘the same but with one field changed’ in one line and no copying at all.”

A model answer

Asked: *what is the difference between a shallow copy and a deep copy?*

“Both make a new outer object. The difference is what the fields point at.

A shallow copy copies each field as it is. A field holding an integer gets that integer. A field holding a reference to a list gets a copy of the *reference*, so the original and the copy now point at the same list. A deep copy recursively copies everything reachable, so the copy shares nothing with the original.

The three-line demonstration: take a dictionary with a list in it, shallow copy it, append to the copy's list, and print the original's list. The original changed, even though nobody touched it. That is the whole bug, and the reason it is expensive to debug is not that it is subtle in isolation — it is that the symptom shows up in code nobody edited, so you start looking at the wrong end of the program.

The rule I would actually state is not ‘always deep copy’. It is that shallow copying is safe exactly when everything it shares is immutable. Sharing a string or a tuple or a frozen dataclass is invisible, because nobody can change it under you. So in a codebase built on frozen dataclasses, this problem mostly does not arise, and I would rather remove the question than answer it — `dataclasses.replace` gives me a new object with one field changed, in one line, with no copying.

Three things I would say about deep copying before you ask. It handles cycles through a memo dictionary keyed on object identity, which also means it preserves the *shape* of sharing rather than duplicating it. It is expensive — on a thousand-key dictionary of hundred-element lists it is about a thousand times slower than shallow and a hundred times the memory, so `deepcopy` inside a request handler is a real performance bug. And it is wrong for anything holding a resource: a socket, a file handle, a database connection, a lock. Python usually raises there — `cannot pickle '_thread.lock' object` — and if it did not, I would have two objects believing they own one socket. The fix is an explicit `__deepcopy__` that copies the data and leaves the connection to be re-established.

As for the pattern, Prototype is creating new objects by cloning a configured one instead of constructing from scratch. It earns its place when construction is expensive — if building the object costs a database read, a parse and an HTTP call, that might be fifty milliseconds against half a millisecond to clone. If construction is cheap, cloning is slower than constructing and I would not use it. You have used it without the name: Docker containers are clones of images, `fork()` clones a process, and Kubernetes pods are clones of a template — and all three use copy-on-write, which is how you get the safety of a deep copy at close to the price of a shallow one.”

9 Recall card

- **Shallow** = new outer object, fields point at the *same* inner objects. **Deep** = the whole reachable graph is copied, nothing shared. And `b = a` is not a copy at all — it is a second name. Draw the boxes and arrows; it beats sixty seconds of talking.

- **The real rule is not “always deep copy”.** *Shallow is safe exactly when everything shared is immutable.* So the best answer is usually to remove the question: frozen dataclasses, tuples, and `dataclasses.replace` for “the same but one field different”.
- **Three things deep copy gets wrong, and volunteer all three.** Cycles — handled by a `memo` keyed on `id()`, which also *preserves the shape of sharing* · cost — $\sim 1,200\times$ slower and $\sim 100\times$ the memory on a 1,000-key dict of 100-element lists, so `deepcopy` in a request handler is a real bug · **resources** — sockets, file handles, connections, locks. `TypeError: cannot pickle '_thread.lock' object` is the error you will actually see; the fix is an explicit `__deepcopy__`.
- **Prototype pays only when construction is expensive.** 56 ms to build (DB read + parse + HTTP) vs 0.4 ms to clone = **140** $\times$. With a cheap constructor, cloning is *slower*. And the prototype itself must be treated as immutable, or every future clone changes silently.
- **You have already used it.** Docker images $\rightarrow$ containers · `fork()` $\rightarrow$ child process · Kubernetes pod templates · `numpy` `views (arr[1:])` shares memory · JS `{...spread}` is shallow, hence `structuredClone`. All the good ones use **copy-on-write**: share until someone writes. And Java’s `Cloneable` is the cautionary tale — marker interface, protected method, no constructor call, an impossible checked exception; use a **copy constructor** instead.

DSA topic: Hashing revision and mock round **System design topic:** Prototype, and cloning objects

Code these, in this order

This is a mock day, so the rule is different from every other practice sheet: **set a timer, do not look anything up, and talk out loud the entire time**. A problem solved in silence today is a problem you have not practised, because silence is the thing that fails in the room.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Top K Frequent Words	LeetCode 692 (Medium)	Counting plus a two-direction tie-break — the <code>reverse=True</code> trap in a hashing costume.
2	Subarray Sum Equals K	LeetCode 560 (Medium)	Prefix sums plus a frequency map: the composition that carries two phases at once.
3	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window plus a map from value to index — and the index-jump most people get wrong.
4	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Designing with two structures, and deletion done in O(1) with the swap-with-last trick.

On all four: run the forty seconds first

Before writing anything, say out loud, in this order: the repeated question, the structure, the key, the invariant, the complexity. Time yourself. If it takes more than a minute, do it again tomorrow until it does not.

On problem 1, find the trap before it finds you

The counts go descending and the words go ascending. Write `sorted(..., reverse=True)` first, watch it get the alphabetical order backwards, then fix it with a composite key and say out loud why negating the count works and negating the word does not.

On problem 2, say which structure holds what

There are two ideas here: a running prefix sum, and a map from *prefix value* to *how many times it has occurred*. Say the invariant out loud before coding: *the map holds the counts of every prefix sum seen before the current index*. Then say why the map must be seeded with `{0: 1}`, and give the input that fails without it.

On problem 4, notice why a dict alone is not enough

`getRandom` needs uniform choice, which a dict cannot give in O(1). So it is a list plus a map from value to its index in the list. Say what has to be updated when you swap-with-last, and why forgetting it is the same class of bug as mutating a key in place.

The forty-second drill

For each problem statement, run the five questions out loud and stop. Do not code any of them.

1. “Find the two numbers that add to the target.”
2. “Return the length of the longest run of consecutive integers.”
3. “Group these files by identical contents.”
4. “Return the third-most-common word.”
5. “Find the first character that appears exactly once.”
6. “Which user ids appear in both of these two-million-line files?”
7. “Return all orders placed between two timestamps.”
8. “The values are between 1 and n. Which one is missing?”

Two of the eight should end with “not a map”. Name them and say what you would use.

The mock drill

Do this three times this week, with a timer, out loud, and ideally with somebody in the room.

1. Pick an unseen medium problem you have not read.
2. Minutes 0-3: read twice, restate, ask two clarifying questions out loud.
3. Minutes 3-6: state the brute force and its cost with a real number. Do not write it.
4. Minutes 6-10: structure, key, invariant, complexity.
5. Minutes 10-25: write it, narrating every few lines.
6. Minutes 25-30: test it yourself on empty, one element, all-identical, and a negative or edge value.
7. Minutes 30-40: have somebody ask “now $O(1)$ space” and “now it does not fit in memory”.

Afterwards, score yourself out of five on communication, approach, correctness, edge cases and follow-ups. The one you score lowest on is what to practise, and it is almost never correctness.

The trap-recall drill

Without looking at the lesson, write down the exact error text or the wrong answer for each:

1. `seen = []` instead of `seen = set()`
2. `counts[key] += 1` on a missing key
3. `.add` on `{}`
4. Reading a missing key from a `defaultdict`
5. `{1, 2}.add([3])`
6. `itertools.groupby` on unsorted input
7. `@dataclass` without `frozen=True`, put in a set
8. Mutating a key while it is in a dictionary
9. `-x` instead of `-abs(x)` in the marking trick, on `[1, 1]`

10. Adding to a set while iterating it

Three of the ten produce **no error at all**. Name them, and say what each one silently returns.

The five-sentences drill

Say each of these out loud from memory, then check it:

1. What a hash map does instead of searching.
2. What `0(1)` actually rests on, and what the worst case is.
3. Why `in` on a list and `in` on a set look identical and are not.
4. What the two tests on a grouping key are.
5. What a map buys and what it throws away.

The shallow-versus-deep drill

Reproduce each, predict the output first, then run it:

1. `b = a` on a list, then `b.append(1)`. Print `a`.
2. `copy.copy` on a dict of lists, mutate a list, print the original.
3. `copy.deepcopy` on the same, mutate, print the original.
4. `a[:]` on a list of lists, mutate an inner list, print the original.
5. A `@dataclass(frozen=True)` with a `list` field. Shallow copy it, mutate the list. What happens, and why does `frozen=True` not save you?
6. Two objects, each holding the other. `copy.deepcopy` one of them. Does it terminate? Why?
7. An object with two fields pointing at the *same* list. Deep copy it. How many lists exist now?

Number 5 and number 7 are the two that surprise people. Say precisely what `frozen` guarantees, and what the memo dictionary preserves.

The resource drill

1. Build an object holding a `threading.Lock`. Try to `deepcopy` it. Quote the exact error.
2. Do the same with an open file handle.
3. Write a `__deepcopy__` that copies the data and leaves the resource out.
4. Say what would go wrong if the deep copy had silently succeeded instead of raising.
5. Name three more things that must never be deep-copied.

The cost drill

On a dict of 1,000 keys each holding a list of 100 integers:

1. Time `b = a`.
2. Time `copy.copy(a)`.
3. Time `copy.deepcopy(a)`.
4. Compute the two ratios.

5. Measure the memory added by each.

6. Now answer: at 200 requests a second, what does a `deepcopy` per request cost you in CPU seconds per second? Is that possible?

The prototype-or-not drill

For each, say whether you would clone or construct, and give the arithmetic:

1. A `Point(x, y)`.

2. A `Report` whose construction reads a database, parses a template and makes an HTTP call.

3. A configuration object with forty fields, of which a caller changes two.

4. An object holding an open database connection.

5. A subclass whose concrete type the caller does not know.

6. A test fixture used in sixty tests.

Two of the six are “construct” for the same reason. Name it.

The Cloneable drill

Answer each in one sentence:

1. Why does implementing `Cloneable` not give you a `clone()` method?

2. Why is `Object.clone()` being protected a problem?

3. What does `Object.clone()` do about `final` fields, and why?

4. What exception does it declare, and why is that a design flaw?

5. What is the recommended replacement, and why is it better?

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Run the whole script: restate, two clarifying questions, the brute force refused with a number, the structure and key and invariant and complexity named before any code, narration while writing, and four edge cases you test yourself.

2. *What is the difference between a shallow copy and a deep copy?* Both in terms of pointers, the boxes-and-arrows drawing, the three-line reproduction, the real rule about immutability, and the three failures of deep copy volunteered before being asked.

3. *When would you not use a hash map?* The three cases with an example each, the two triggers that come first in the decision procedure, and the sentence about $O(1)$ being an average rather than a bound.

Before you move on

- [] I ran a full timed mock, out loud, from restatement to follow-up.

- ☐ I scored myself on all five rubric lines and know which one is my weakest.
- ☐ I can run the five decision questions in under a minute on an unseen problem.
- ☐ I can recite the five sentences that carry the phase, from memory.
- ☐ I wrote down the exact error text for at least seven of the ten traps.
- ☐ I can name the three traps that produce no error at all, and what each returns instead.
- ☐ I reproduced the shallow-copy bug and drew the boxes and arrows for it.
- ☐ I triggered a deep-copy failure on a lock and quoted the exact error.
- ☐ I measured the deepcopy ratio myself and can quote it.
- ☐ I can say when Prototype does *not* pay, with the arithmetic.
- ☐ I answered all three questions above out loud.

PART IX

Stacks and queues

Days 68 to 77 · 10 chapters

Alongside, on the system design track:

Design patterns, Low-level design case studies

068 Stacks: last in, first out

069 Balanced brackets and the parsing family

070 Min stack, and stacks that carry extra state

071 Monotonic stack: the next greater element

072 Largest rectangle in a histogram

073 Queues: first in, first out

074 Deques and the sliding-window maximum

075 A queue from two stacks, and a stack from queues

076 LRU cache: the structure interviewers love

077 Stacks and queues revision and mock round

Stacks: last in, first out

DATA STRUCTURES AND ALGORITHMS

Stacks: last in, first out

You can implement a stack and recognise the problems that want one.

SYSTEM DESIGN

Adapter

You can make two incompatible interfaces work together without editing either.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Implement a stack. What operations must be $O(1)$?*
- *You must integrate a third-party SDK with a different interface. How?*

1 What this is, and why they ask it

A **stack** is a collection where you add and remove at the same end. The last thing you put in is the first thing you take out, which is why it is called **LIFO** — last in, first out. It has three operations and all three are $O(1)$: **push** (add to the top), **pop** (remove the top), and **peek** (look at the top without removing it). You cannot reach anything except the top, and that restriction is the point rather than a limitation.

They ask it because the stack is the smallest data structure with a real personality, and because a whole family of problems is solved by exactly one sentence: *keep track of the most recent thing that is not yet resolved*. Brackets, undo, expression evaluation, backtracking, the browser back button, and — the one they will push you on — recursion itself. Every recursive function is a stack, made by the language instead of by you.

The interview shape is specific. “Implement a stack” is a warm-up that takes four minutes and is really checking two things: do you know that a Python `list` already is one, and do you know which end to use. Then the real question arrives: a problem that does not mention stacks anywhere, where recognising the sentence above is the whole difficulty.

2 The story

Meena was making lunch on a Sunday, which normally takes her about an hour and twenty minutes and on this particular day took nearly three.

At about half past eleven she had the cooker on for the dal and had started chopping for the palya. That was the thing she was doing.

Then the doorbell went. The gas man, with the cylinder, two weeks earlier than she expected him. So she put the knife down and went to the door, and while he was pulling the old cylinder out from under the counter, her mother-in-law called from the back room asking where the strip of tablets had gone.

So Meena left the gas man half-finished at the counter and went to look for the tablets. And while she was going through the shelf in the back room, she heard the milk going over on the second burner.

Now there were three unfinished things — lunch, the gas man, the tablets — and a fourth just starting. She did the milk, because it was the one that was happening now and because in about four seconds it would be on the floor. Then, milk dealt with, she went back to the tab-

lets, which was the thing she had been doing just before that. Then, tablets found and handed over, back to the gas man. Then, gas man paid and gone, back to the knife and the half-chopped beans.

Every single time, the thing she went back to was the most recent thing she had left. Not the oldest. She never once came back from the milk and returned to the chopping, because the chopping had two other unfinished things sitting on top of it, and she could not get to it until they were cleared.

This works completely fine up to about three, in her experience.

The Sunday it did not work was a month later, when there were five, and the fifth was her neighbour at the door with a long story about a car. She dealt with the car, and the phone, and the tablets, and the gas, in that order, correctly, and then stood in the kitchen with a distinct feeling of having forgotten something, which turned out to be the cooker. It had been the very first thing, at the very bottom, under everything else, and it had been sitting there for fifty minutes.

3 The idea in plain English

Meena's unfinished jobs are a **stack**. A new interruption goes on top — that is a **push**. Finishing the current one and returning to the previous one — that is a **pop**. And “the thing I am doing now” is whatever is on top, which is a **peek**.

Two properties fall straight out of the story, and they are the two things to say in an interview.

One: you can only reach the top. The chopping was still there the whole time and was completely unreachable until the three things above it were cleared. A stack does not let you look at the middle. That restriction is what makes all three operations $O(1)$.

Two: the order out is the reverse of the order in. She was interrupted by the gas man, then the tablets, then the milk, and she resolved them milk, tablets, gas man. Last in, first out.

And the bad Sunday is a **stack overflow**: too many unfinished things, and the one at the bottom is forgotten.

The three operations

```
stack: list[int] = []

stack.append(4)      # push      stack is [4]
stack.append(7)      # push      stack is [4, 7]
stack[-1]            # peek → 7  stack is [4, 7]  (nothing removed)
stack.pop()          # pop → 7   stack is [4]
len(stack) == 0       # is_empty → False
```

That is the entire interface. In Python you do not write a `Stack` class in an interview unless you are asked to — **a list is a stack**, and saying that immediately is the right answer.

Which end, and why it matters enormously

The one thing you must get right:

<code>stack.append(x)</code>	# push at the END	$O(1)$	
<code>stack.pop()</code>	# pop from the END	$O(1)$	
<code>stack.insert(0, x)</code>	# push at the FRONT	$O(n)$	← wrong end
<code>stack.pop(0)</code>	# pop from the FRONT	$O(n)$	← wrong end

Adding at the end of a Python list is amortised $O(1)$, because the list has spare capacity and occasionally doubles — the same amortised argument as a hash table resize from [day 060](#). Adding at the *front* has to shift every existing element one place to the right, which is $O(n)$.

Measured, and it is not a small difference:

<code>1,000,000 x (append + pop)</code>	0.04 s
<code>100,000 x (insert(0) + pop(0))</code> on a 10,000-element list	2.31 s

Ten times fewer operations, sixty times the time. If an interviewer asks “which operations must be $O(1)$ ”, the answer is *all three*, and the follow-up they are hoping for is that you know which end of the list gives you that.

What a stack is really for

One sentence, and it is worth memorising because it is the recognition trigger:

A stack is the right structure whenever you need the most recent thing that has not yet been resolved.

Read that against the problems:

PROBLEM	THE MOST RECENT UNRESOLVED THING
Are these brackets balanced?	the most recent unclosed opening bracket
Undo	the most recent action
Browser back	the most recent page you were on
Evaluate <code>3 + 4 * 2</code>	the most recent operator not yet applied
Simplify <code>/a/b/.. /c</code>	the most recent directory you entered
Depth-first search	the most recent node whose neighbours are unexplored
Next greater element	the most recent value with no greater element yet

If you can phrase a problem in those words, it is a stack problem, whatever else it looks like.

The call stack

Meena's story is literally how function calls work. When a function calls another function, the machine pushes a **stack frame** — the local variables and the place to return to — onto a stack. The called function runs. When it returns, its frame is popped and execution resumes exactly where it left off, in the most recent unfinished function.

Which is why recursion has a depth limit:

```
>>> def r(n): return r(n + 1)
>>> r(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RecursionError: maximum recursion depth exceeded
```

Python's default limit is **1000** frames. Meena forgot the cooker; the machine raises instead. And this is the reason any recursive algorithm can be rewritten with an explicit stack — you are just doing by hand what the language was doing for you. That answer comes up constantly in tree and graph questions from [day 098](#) onwards.

The other options, and when to mention them

```
from collections import deque
stack = deque()
stack.append(x); stack.pop()          # also O(1) at both ends
```

A `deque` is fine and is the right choice when you need a *queue* as well. For a pure stack a list is marginally faster and simpler.

`queue.LifoQueue` exists and is for passing work between threads. It takes a lock on every operation, so it is roughly a hundred times slower. Mentioning it and then saying you would not use it here is a good, cheap signal.

4 The picture

The stack over the course of Meena's Sunday. Time runs downwards; the top of the stack is the top of each column.

t1 lunch	t2 doorbell	t3 tablets	t4 milk	t5 pop	t6 pop	t7 pop
		+-----+	+-----+			
		tablets	milk			
	+-----+	gas man	tablets	+-----+		
+-----+	gas man	gas man	gas man	gas man	+-----+	
lunch	lunch	lunch	lunch	lunch	lunch	(empty)
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	
^	^	^	^	^	^	
top	top	top	top	top	top	

What to notice: `lunch` sits at the bottom from `t1` to `t7` and is untouchable the whole time. Every operation happens at the top line and nowhere else. That is why push, pop and peek are all $O(1)$ — none of them has to move or search anything.

And the same thing drawn as a Python list, so the indices are visible:

```
index    0        1        2        3
+-----+-----+-----+-----+
| lunch  | gas man | tablets | milk   |
+-----+-----+-----+-----+
                        ^
                    stack[-1]
                    append() adds here
                    pop() removes here
```

The FRONT (index 0) is the bottom. Never touch it.
`insert(0, x)` would shift all four elements right: $O(n)$.

5 The code, built step by step

Step 1 — the stack class, since they will ask

```
class Stack:
    def __init__(self) → None:
        self._items: list[int] = []

    def push(self, item: int) → None:
        self._items.append(item)

    def pop(self) → int:
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()
```

Two things to say while writing it. `append` and `pop` are at the end, which is the $O(1)$ end. And the explicit empty check is not decoration — `{}.pop()` raises `IndexError: pop from empty list`, which is a confusing message for a caller who never knew there was a list.

```
    def peek(self) → int:
        if not self._items:
            raise IndexError("peek at empty stack")
        return self._items[-1]

    def is_empty(self) → bool:
        return not self._items

    def __len__(self) → int:
        return len(self._items)
```

`peek` returns `self._items[-1]` without removing it. Note it does *not* return `self._items.pop()` and then push it back — that is a real thing people write under pressure and it is both slower and wrong for an empty stack.

Step 2 — the pattern, on a small problem

Remove adjacent duplicate characters: `"abbaca"` becomes `"ca"`, because `bb` cancels, then `aa` cancels.

```
def remove_adjacent_duplicates(text: str) → str:
    stack: list[str] = []
    for character in text:
        if stack and stack[-1] == character:
            stack.pop()          # cancels with the most recent
        else:
            stack.append(character)
    return "".join(stack)
```

Six lines, and the sentence is exactly the recognition trigger: *does this character cancel the most recent unresolved one?* Note `if stack and ...` — checking the stack is non-empty **before** indexing. Forgetting that half of the condition is the commonest bug in every stack problem there is.

Step 3 — the real problem: simplify a path

Given `"/a/./b/../../c/"`, return the canonical absolute path, which is `"/c"`. This is LeetCode 71, and it is a good interview problem because the stack is genuinely the answer and the edge cases are real rather than invented.

The rules: `.` means stay here, `..` means go up one, empty pieces from doubled slashes mean nothing, and everything else is a directory name.

Split first:

```
parts = path.split("/")
# "/a/./b/../../c/" → ['', 'a', '.', 'b', '..', '..', 'c', '']
```

Splitting on `/` gives empty strings at both ends and wherever there were two slashes. That is convenient rather than annoying, because it means one rule handles all of them.

Now the loop, one rule at a time:

```
stack: list[str] = []
for part in parts:
    if part == "" or part == ".":
        continue          # nothing to do
    if part == "..":
        if stack:
            stack.pop()    # go up – but not above the root
    else:
        stack.append(part) # a real directory name
```

The `if stack` inside the `..` branch is the whole edge case. `/../` from the root stays at the root; it does not go above it and it does not raise. On `"/../"` the stack is empty, `..` does nothing, and the answer is `"/"`.

Then rebuild:

```
return "/" + "/".join(stack)
```

An empty stack gives `"/" + ""` which is `"/"`. Correct with no special case, which is worth pointing out as you write it.

The complete solution

```
class Stack:
    """A stack over a Python list. All three operations are O(1) because
    they all happen at the END of the list – the amortised-O(1) end."""

    def __init__(self) → None:
        self._items: list[str] = []

    def push(self, item: str) → None:
        self._items.append(item)

    def pop(self) → str:
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self) → str:
        if not self._items:
            raise IndexError("peek at empty stack")
        return self._items[-1]

    def is_empty(self) → bool:
        return not self._items

    def __len__(self) → int:
        return len(self._items)

def simplify_path(path: str) → str:
    """Canonical absolute path. '.' stays, '..' goes up, '/' collapses.

    The stack holds the directories entered but not yet left – which is the
    'most recent unresolved thing' sentence in its purest form.
    """
    stack: list[str] = []

    for part in path.split("/"):
        if part == "" or part == ".":
            continue
        elif part == "..":
            if stack:
                stack.pop()
            else:
                stack.append(part)
        else:
            stack.append(part)

    return "/" + "/".join(stack)

def remove_adjacent_duplicates(text: str) → str:
    """Repeatedly cancel equal adjacent characters. One pass, O(n)."""
    stack: list[str] = []
    for character in text:
        if stack and stack[-1] == character:
            stack.pop()
        else:
            stack.append(character)
    return "".join(stack)

def reverse_with_stack(items: list[int]) → list[int]:
    """Out is the reverse of in – the defining property, made literal."""
    stack: list[int] = []
    for item in items:
```

```

        stack.append(item)
    return [stack.pop() for _ in range(len(stack))]

def is_balanced_simple(text: str) → bool:
    """One bracket type. Tomorrow generalises this to three."""
    depth = 0
    for character in text:
        if character == "(":
            depth += 1
        elif character == ")":
            depth -= 1
            if depth < 0:
                # closed one that was never opened
                return False
    return depth == 0

if __name__ == "__main__":
    print(simplify_path("/home/"))          # /home
    print(simplify_path("/../"))            # /
    print(simplify_path("/home//foo/"))     # /home/foo
    print(simplify_path("/a/./b/../../c/")) # /c
    print(simplify_path("/..."))            # /... ('...' is a real name)
    print(remove_adjacent_duplicates("abbaca")) # ca
    print(reverse_with_stack([1, 2, 3]))     # [3, 2, 1]
    print(is_balanced_simple("()"))          # False

    s = Stack()
    s.push("a"); s.push("b")
    print(s.peek(), len(s))                  # b 2
    print(s.pop(), s.pop(), s.is_empty())    # b a True

```

The `"../"` case is worth running. Three dots is not `..`; it is a perfectly legal directory name, and code that checks `part.startswith("../")` gets it wrong.

6 What it costs

The three operations

```

push (append at the end)  0(1) amortised
pop  (remove from the end) 0(1)
peek (read index -1)     0(1)
is_empty                   0(1)

```

`push` is *amortised* $O(1)$, not strictly $O(1)$, and the distinction is the same one as the hash-table resize. A Python list holds spare capacity; when it fills, it allocates a bigger block and copies everything, which is $O(n)$ on that one append. But the doublings are $8 + 16 + 32 + \dots + n/2 \approx n$, so across n appends the total copying is about n , and each append averages constant time. Say “amortised” and be ready to explain it in that one sentence.

The wrong end, counted

`insert(0, x)` shifts every existing element right by one. On a stack of size k that is k moves. Doing it n times:

$$1 + 2 + 3 + \dots + n = n(n-1)/2$$

At $n = 100,000$: **five billion** element moves instead of 100,000 appends. Measured, at only 100,000 operations on a 10,000-element list, it was 2.31 seconds against 0.04 seconds for a million append-and-pop pairs.

The problems

`simplify_path` splits the string, which is $O(n)$, then walks the parts. Every part is pushed at most once and popped at most once, so the loop does at most $2n$ stack operations.

```
split:      0(n)
loop:      each part pushed ≤ 1, popped ≤ 1 → 0(n) total
join:      0(n)
-----
total:      0(n) time
```

“Each element is pushed at most once and popped at most once” is the counting argument for almost every stack problem, and it is the sentence to say. It is what makes the nested-looking loops in [day 071](#) linear rather than quadratic.

Space is $O(n)$ in the worst case: a path of all directory names with no `..` pushes everything.

The call stack, priced

Each Python stack frame is roughly a few hundred bytes. The default recursion limit of 1000 exists because the C stack underneath is finite — typically 1 MB or 8 MB — and blowing it seg-faults the interpreter rather than raising cleanly.

```
recursion depth 1,000    ~ default limit, RecursionError beyond
an explicit stack       limited only by heap memory: millions of entries
```

That is the practical argument for converting recursion to an explicit stack, and it is a real follow-up: “*this tree is a million nodes deep — will your recursive solution work?*” No, and the explicit stack version will.

7 The traps

Trap 1 — popping an empty stack

```
>>> [].pop()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: pop from empty list
```

And peeking:

```
>>> [][-1]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Every stack problem has at least one input that empties the stack early — `)(", "/../", "aa"` — and if you have written `if stack[-1] == x` without `if stack and ...`, that input finds it. **Write the emptiness check as part of the condition, not as an afterthought.**

The `deque` version says it slightly differently, which is worth recognising:

```
IndexError: pop from an empty deque
```

Trap 2 — the wrong end of the list

```
stack.insert(0, x) # and stack.pop(0)
```

This is correct and it is $O(n)$. Nothing will fail, no test will complain, and your $O(n)$ solution is quietly $O(n^2)$. This is exactly the `seen = []` trap from [day 062](#) in a new costume: identical-looking code, different complexity.

Trap 3 — `peek` implemented as pop-then-push

```
def peek(self):
    item = self._items.pop()
    self._items.append(item)
    return item
```

Works on a non-empty stack, raises on an empty one where `self._items[-1]` would have raised more clearly, and does twice the work. More importantly, in any concurrent or generator context the stack is briefly in a state that does not include the item. Use `[-1]`.

Trap 4 — forgetting that the answer is reversed

The stack gives you things in reverse order of insertion. If you build a result by popping, you get it backwards.

```
result = []
while stack:
    result.append(stack.pop())
# result is now in reverse order of the stack's bottom-to-top order
```

In `simplify_path` this is why the answer is `"/".join(stack)` — joining the list directly, in bottom-to-top order — rather than popping into a list. Popping would give `"c"` before `"a"` and the path would be backwards.

Trap 5 — `..` versus `...`

```
if part.startswith(".."):    # WRONG
```

`"..."` is a valid directory name and `"..foo"` is too. The check must be `part == ".."` exactly. `simplify_path("/...")` should return `"/..."`, and the `startswith` version returns `"/"`.

Trap 6 — recursion depth

```
>>> def r(n): return r(n + 1)
>>> r(0)
RecursionError: maximum recursion depth exceeded
```

Default limit 1000. Raising it with `sys.setrecursionlimit` is not a fix; it moves the failure from a clean Python exception to a segmentation fault when the real C stack runs out. The fix is an explicit stack.

Trap 7 — reaching into the middle

```
stack[2]    # legal on a Python list, and it is not a stack operation
```

Python will let you do it because a list is not really a stack. If you find yourself indexing anything other than `-1`, either you do not want a stack, or you have a bug. Say this out loud if you do it deliberately — “I am peeking two down, which means this is not a pure stack” — because an interviewer will notice.

8 In the interview

How it gets asked

- The warm-up: “Implement a stack. What operations must be $O(1)$?” Four minutes, and the real content is the “which end” answer.
- “Implement a stack using an array. Now what if the array is fixed size?” — leads to resizing and the amortised argument.
- The disguised one, which is the real question: “Given a path like `/a/./b/./c`, return the canonical form.” Nobody says stack.
- The connection: “Rewrite this recursive function iteratively.” The answer is always an explicit stack, because that is what recursion already was.

What to say out loud, in the first ninety seconds

1. **Say what a stack is by its restriction, not its operations.** “A stack only lets you touch one end. That restriction is what makes all three operations constant time.”

2. **Say the recognition sentence.** “I reach for a stack when the problem needs the most recent thing that is not yet resolved. Here, that is the most recent directory I entered and have not left.”
3. **Say the implementation in one line and move on.** “In Python a list is a stack — `append` and `pop` at the end. I would not write a class unless you want one.”
4. **Name the counting argument before writing.** “Each element is pushed at most once and popped at most once, so this is $O(n)$ even though there is a pop inside the loop.”
5. **Say the empty-stack case before the interviewer finds it.** “The case I need to handle is `..` when the stack is empty — that is `/../`, which should stay at the root rather than go above it or raise.”

The follow-ups

“**Why is push amortised $O(1)$ and not just $O(1)$?**” “Because the list occasionally has to grow. When it is full, Python allocates a bigger block and copies everything, which is $O(n)$ on that one append. But the growth is geometric, so across n appends the total copying is about n , and each one averages constant. It matters if you care about tail latency — one unlucky push is slow — and it does not matter for the total.”

“**Implement it with a fixed-size array.**” “Then push has to check `top == capacity - 1` and either reject or resize. If I resize, I double, which is what gives me the amortised bound. If I reject, I have a bounded stack and push can fail, which changes the interface — `push` now returns a boolean or raises `StackOverflow`.”

“**Could you use a linked list instead?**” “Yes, and push and pop become strictly $O(1)$ rather than amortised, because there is no reallocation. The costs are one allocation per element and terrible cache behaviour — the nodes are scattered in memory, while an array is contiguous. In practice the array version is faster for almost everything, which is why every standard library uses one.”

“**This tree could be a million nodes deep. Will your recursive solution work?**” “No. Python’s recursion limit is 1000 frames by default, and raising it just moves the crash from a clean `RecursionError` to a segfault when the C stack runs out. I would convert it to an explicit stack — which is doing by hand exactly what the call stack was doing for me — and then the only limit is heap memory.”

“**How is a stack different from a queue?**” “Which end you remove from. A stack removes from the end you added to, so the order out is reversed. A queue removes from the other end, so the order is preserved. Both are $O(1)$ if you use the right structure — but a Python list is only a good stack, not a good queue, because removing from the front is $O(n)$. That is what `collections.deque` is for.”

A model answer

Asked: *implement a stack. What operations must be $O(1)$?*

“A stack is a collection where you add and remove at the same end, so the last thing in is the first thing out. The defining property is the restriction: you can only touch the top. You cannot look at the middle, and that is exactly why every operation is constant time — nothing ever has to search or shift.

Three operations, and all three must be $O(1)$: push, pop and peek. I would add `is_empty` as a fourth, because every real use needs it.

In Python I would not normally write a class. A list is a stack: `append` to push, `pop` to pop, and `[-1]` to peek. The one thing that matters is **which end**. `append` and `pop` work at the end of the list, which is amortised constant time. If I used `insert(0, x)` and `pop(0)` instead, every operation would shift the whole list and I would have an $O(n)$ stack that looks identical on the screen. Measured, that is about sixty times slower at only a hundred thousand operations.

Push is amortised constant rather than strictly constant, because when the list fills, Python allocates a larger block and copies. The growth is geometric, so across n pushes the total copying is about n and each push averages constant. It matters for tail latency and not for throughput.

If you want it written out, I would guard both `pop` and `peek` against an empty stack and raise a message that mentions the stack — otherwise a caller gets `IndexError: pop from empty list`, which mentions a list they never knew existed.

Two things I would mention unprompted. `collections.deque` is also $O(1)$ at both ends and is what I would use if I needed a queue as well. And `queue.LifoQueue` exists but takes a lock on every operation for thread safety, so it is roughly a hundred times slower and I would not use it unless threads were actually involved.

And the reason I care about stacks beyond the interface is that the call stack is one. Every recursive function is pushing a frame per call, which is why Python raises `RecursionError` at a thousand deep, and why any recursion can be rewritten with an explicit stack when the depth is a problem.”

9 Recall card

- **A stack is defined by its restriction: you can only touch one end.** That is *why* push, pop and peek are all $O(1)$ — nothing searches, nothing shifts. LIFO, so the order out is the reverse of the order in.
- **In Python, a list is a stack.** `append` / `pop` / `[-1]`. **The end is the $O(1)$ end** — `insert(0, x)` and `pop(0)` shift everything and are $O(n)$, giving an identical-looking $O(n^2)$ solution (2.31 s vs 0.04 s in the measurement). `deque` if you also need a queue; never `queue.LifoQueue` outside threads ($\sim 100\times$ slower).

- **The recognition sentence: reach for a stack when you need *the most recent thing not yet resolved*.** Unclosed bracket · last action to undo · last page visited · operator not yet applied · directory entered but not left · node whose neighbours are unexplored.
- **The counting argument for every stack problem: each element is pushed at most once and popped at most once, so it is $O(n)$ — even with a `pop` inside the loop.** Push is amortised $O(1)$ because the list grows geometrically ($8+16+\dots+n/2 \approx n$).
- **The call stack is a stack**, which is why `RecursionError: maximum recursion depth exceeded` fires at **1000** frames and why any recursion can become an explicit stack (limited only by heap). Traps: `IndexError: pop from empty list` — always write `if stack and stack[-1] == x` · `part == ".."` not `startswith("..")`, because `"..."` is a real name · and remember the stack gives you things **backwards**.

1 What this is, and why they ask it

An **adapter** is a small class that implements the interface your code wants and forwards the calls to an object that has a different interface. It changes nothing about either side. The thing you cannot edit — a vendor library, a legacy module, someone else's service — stays untouched, and your code carries on talking in its own vocabulary.

It is the first of the seven structural patterns and by a distance the most useful one in real work, because the situation it solves is permanent. You do not own the payment gateway. You do not own the SMS provider, the cloud storage client, the analytics library or the thing the team before you wrote in 2017. Their method is called `submit_charge(amount_cents, tok)`; yours wants `charge(order: Order) → Payment`. One of those two facts is going to have to give, and the adapter is how neither of them does.

They ask it because it is the pattern that most reliably distinguishes people who have integrated a third party from people who have not. The naive answer — “I would wrap it” — is directionally right and stops one sentence too early. The good answer says where the interface lives, whose vocabulary it is written in, and what the adapter must translate: not just method names, but **error types, units, and data shapes**. That last part is where the marks are.

2 The story

The flat in Bengaluru came with a washing machine that the previous tenants left behind, and it is a good one, and it does not fit.

Not physically. The plug. It is an old machine with a big round three-pin plug of a kind the flat's sockets do not take, and the sockets are the newer flat-pin type that everything else in the house uses.

Vinay's first instinct, standing in the kitchen on the Saturday he moved in, was to cut the plug off and wire on a new one. His father talked him out of it in about thirty seconds. It is not his machine. It is going back to the owner eventually. And if he cuts the cable and gets it slightly wrong, he owns a broken washing machine that was not his.

His second instinct was to get an electrician in to change the socket. Also possible, also wrong. It is not his socket either. It is in the wall of a flat he rents, everything else in the house uses that socket, and the landlord would have views.

What he actually did cost a hundred and ten rupees at the shop near the bus stand. A converter. A small white block with the round holes on one face and the flat pins on the other. The machine plugs into it, it plugs into the wall, and neither the machine nor the wall knows any-

thing has happened.

He has three of them now, for various things, and he is quite firm about one point when the subject comes up: the converter does not make anything better. It does not add a function to the washing machine and it does not improve the socket. All it does is sit in the middle and make one shape into another shape. If you told him it was clever he would say it is the opposite of clever, which is why it works.

The one that did go wrong is worth mentioning. A friend brought a small appliance back from Dubai and used a converter with it, and the plug fitted perfectly and the thing burned out in about four seconds, because the plug was not the only difference. The voltage was different too, and the converter had only dealt with the shape. Nobody had thought about it, because the part you can see had been solved.

3 The idea in plain English

The converter is an **adapter**. Three parties, and it is worth naming all three because interviewers use the words.

- The **client** — the wall socket, which will only accept one shape.
- The **adaptee** — the washing machine, which has a different shape and cannot be changed.
- The **adapter** — the white block, which presents the shape the client wants and passes everything through to the adaptee.

Neither the socket nor the machine is modified. Neither of them knows the block exists. That is the definition.

And the friend's burnt appliance is the part of this lesson people miss: **an adapter that only translates the obvious difference is not finished**. The plug shape was visible; the voltage was not. In code, the method names are the plug shape. The things nobody looks at are the units, the error types, and the data shapes — and those are what actually burn.

The shape, in code

Your code wants this:

```
class PaymentGateway(Protocol):
    def charge(self, order: Order) → Payment: ...
```

The vendor gives you this:

```
# stripe_sdk.py — not yours, cannot be edited
class StripeClient:
    def create_charge(self, amount_cents: int, currency: str,
                      source_token: str, idempotency_key: str) → dict: ...
```

Different name, different arguments, different return type. The adapter sits between them:

```

class StripeGateway:
    # implements YOUR PaymentGateway
    def __init__(self, client: StripeClient) → None:
        self._client = client

    def charge(self, order: Order) → Payment:
        response = self._client.create_charge(
            amount_cents=order.total.paise // 100 * 100, # units!
            currency=order.total.currency,
            source_token=order.payment_token,
            idempotency_key=str(order.id),
        )
        return Payment(reference=response["id"],
                        status=self._translate_status(response["status"]))

```

That is the whole pattern. One class, one method per method you actually use, and nothing else.

The four things an adapter translates

This list is the answer to the interview question. Anybody can say “it renames the method”. The adapter is doing four jobs, and the last three are where real bugs live.

1. The interface. `create_charge` becomes `charge`. The easy part.

2. The data shape. Their `dict` with string keys becomes your `Payment` object. Their `{"cust": {"em": "..."} }` becomes your `Customer`. If a vendor `dict` escapes past the adapter into your domain, the adapter has failed — you now have their shape everywhere and no seam at all.

3. The units and types. Their amounts are in cents as an `int`; yours are `Money` in paise as a `Decimal`. Their timestamps are Unix seconds; yours are timezone-aware `datetime`. Their country is `"IND"`; yours is `"IN"`. **This is the voltage.** Every one of these has caused a production incident somewhere, and the adapter is the one place where the conversion can be written once and tested.

4. The errors. Theirs is `stripe.error.CardError`. Yours should be `PaymentDeclined`. If you skip this, then `except stripe.error.CardError` appears in your business logic, and your domain now imports the vendor — which is the dependency-inversion failure from [day 059](#), with the interface in place and doing nothing.

```

def charge(self, order: Order) → Payment:
    try:
        response = self._client.create_charge(...)
    except stripe.error.CardError as error:
        raise PaymentDeclined(str(error)) from error # their error → yours
    except stripe.error.RateLimitError as error:
        raise PaymentTemporarilyUnavailable() from error

```

Where the interface lives — the half everybody gets wrong

The `PaymentGateway` protocol goes in **your** package, in **your** vocabulary. `orders/ports.py`, not `adapters/interfaces.py`, and its signature mentions `Order`, `Money` and `PaymentDeclined` — your types, never theirs.

A protocol that says `def charge(self, amount_cents: int) → dict` is not an abstraction; it is the vendor’s interface with a different file name. It looks identical on a diagram and it has inverted nothing, because the day you swap the vendor, every caller changes.

Object adapter versus class adapter

The Gang of Four describe two forms, and interviewers occasionally ask.

Object adapter holds the adaptee as a field and forwards to it — everything above. It is composition, it works with any adaptee, and it can adapt several at once.

Class adapter inherits from both the target and the adaptee. It needs multiple inheritance, so it does not exist in Java or C#, and where it does exist it inherits every method of the adaptee whether you wanted them or not — which is a Liskov and interface-segregation problem from [days 057](#) and [058](#).

Use the object adapter. Say the other exists and that you would not use it, and say why: it leaks the adaptee’s whole surface.

Adapter versus its neighbours

These four get confused constantly, and separating them cleanly is a strong answer.

PATTERN	INTERFACE	PURPOSE
Adapter	changes it	make an incompatible thing usable
Decorator	keeps it	add behaviour, and stack it
Facade	simplifies it	one door onto many things
Proxy	keeps it	control access — cache, guard, defer

One sentence that settles it: **adapter changes the interface, decorator changes the behaviour, facade changes the number of things you talk to, and proxy changes when or whether you reach the real one.**

4 The picture

The three parties, and what each one knows.

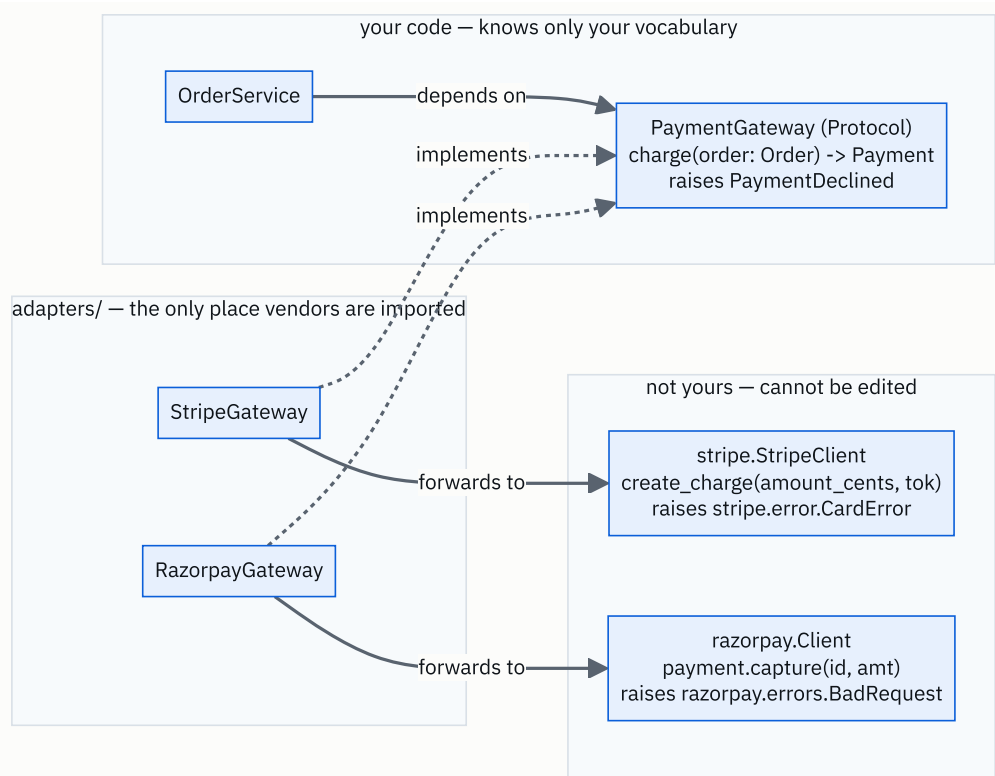


FIGURE 68.1

What to notice: **every arrow into PaymentGateway points inwards.** OrderService depends on it, and the adapters depend on it. Nothing inside the yours box points outwards at a vendor. That picture is what “the dependency was inverted” looks like, and the command that proves it is `grep -rn "stripe" orders/` returning nothing.

And the four translations, as a table across the boundary:

	THEIR SIDE		YOUR SIDE
method	create_charge(...)	→	charge(order)
shape	{"id": "ch_1", ...}	→	Payment(reference=..., status=...)
units	amount_cents: int	→	Money(paise, "INR")
	created: 1725091200	→	datetime(2026, 8, 31, tzinfo=UTC)
	country: "IND"	→	"IN"
errors	stripe.error.CardError	→	PaymentDeclined
	stripe.error.RateLimit	→	PaymentTemporarilyUnavailable
	^^^^^^^^^^^^^^^^^^^^^^^^		
	the three rows below the first are the voltage		

5 How it actually works

Writing one, in order

1. **Write the interface you wish existed**, from the caller's point of view, before you open the vendor's documentation. This ordering matters. If you read their API first, you will design your interface to look like theirs, and the adapter will have nothing to do.
2. **Put it in the consuming package**, in your vocabulary and your types.
3. **Write the adapter**, one method at a time, and only the methods you actually use.
4. **Translate all four things** — interface, shape, units, errors.
5. **Write a fake implementing the same interface** for tests. This is the moment you find out whether the interface is really yours: if the fake needs a Stripe-shaped `dict`, it is not.

In Python, the interface is free

```
from typing import Protocol

class PaymentGateway(Protocol):
    def charge(self, order: Order) → Payment: ...
```

`typing.Protocol` is **structural** (day 058), so `StripeGateway` does not have to inherit from it or import it. It satisfies the protocol by having the right method. The consequence is that you can declare the protocol in the *consuming* module and never touch the provider — which is exactly what dependency inversion asks for, in about four lines.

Adapting an interface you did not choose, in the standard library

Real examples, and naming two of these is worth more than any amount of theory.

- **Python's DB-API.** `psycopg2`, `mysqlclient` and `sqlite3` speak three completely different wire protocols and present one `connect` / `cursor` / `execute` / `fetchall` surface. Every driver is an adapter, and the specification is PEP 249.
- `io.TextIOWrapper` adapts a binary stream to a text one — encoding, newline translation, buffering. `open()` is stacking three adapters and handing you the top one.
- **SLF4J** in Java exists purely so that code can log without knowing whether Log4j, Logback or `java.util.logging` is underneath. It is an adapter layer and nothing else.
- `java.util.Arrays.asList()` adapts an array to a `List` interface. Note it is a *view*, not a copy — and it throws on `add`, which is a Liskov violation that the standard library shipped.
- **ODBC and JDBC**, whose entire purpose is one interface over every database.
- **React Native's bridge**, adapting platform-native components to one JavaScript interface.
- **Terraform providers.** One resource model, dozens of cloud APIs behind it.

The anti-corruption layer

At a larger scale the same idea has a different name, from Eric Evans's domain-driven design, and using it correctly is a senior signal.

An **anti-corruption layer** is an adapter around an entire external system or legacy subsystem, whose job is to stop that system's model leaking into yours. Not one method — a whole translation boundary. When a team says “we integrated with the legacy billing system and now our `Customer` has fourteen fields we do not understand”, they skipped this.

The tell that you need one: the external model's vocabulary appearing in conversations about your domain.

The half-adapter, which is the commonest failure

```
class StripeGateway:
    def charge(self, order: Order) → dict:      # ← returns THEIR shape
        return self._client.create_charge(...)
```

The name is adapted. The shape is not. Now every caller does `response["id"]` and `response["status"] == "succeeded"`, so the vendor's data model is spread through the codebase and the seam exists in name only. When you swap the provider, the interface does not change and forty call sites do.

The test for whether an adapter is real: can you write a fake that returns nothing vendor-shaped? If the fake has to build a Stripe-looking dictionary, the adapter did not adapt.

6 The numbers

The swap, which is the whole argument

Moving from Razorpay to Stripe:

without an adapter:	
grep -rn "razorpay" .	38 references across 14 files
files touched	14, including domain and tests
estimated effort	~2 weeks
risk	every call site is a chance to differ
with an adapter:	
files added	1 (StripeGateway, ~60 lines)
lines edited	1 (the wiring in main.py)
domain files touched	0

And the second number that matters as much: **running both at once**. With an adapter, sending 5% of traffic to the new provider is a routing decision in one place. Without one, it is not attemptable.

The cost of writing one

the protocol	~6 lines
the adapter	~40-80 lines, mostly translation
the fake for tests	~15 lines

total	~60-100 lines, one file each

Against calling the SDK directly, that is roughly **60 to 100 lines of pure structure**. Worth stating plainly, because the honest case for it is not “it is cleaner”.

The test argument, which is what actually persuades a team

test "a declined card leaves the order unpaid":	
without an adapter:	needs the Stripe test API, a network call, a magic card number, and a mocked SDK ~35 lines, ~800 ms, flaky when their sandbox is down
with an adapter:	a 3-line fake that raises PaymentDeclined ~4 lines, ~0.3 ms

800 ms against 0.3 ms is roughly **2,600×**, and the flakiness matters more than the speed: a test that depends on a third party’s sandbox fails for reasons that have nothing to do with your code, and teams respond by deleting it.

The units bug, priced

The one nobody counts. A vendor in cents, a domain in paise, and a missing conversion in one place:

```
amount sent as 45000 (paise) where cents were expected
→ charged 45000 cents = $450 instead of ₹450
→ at 200 transactions before someone notices: 200 wrong charges,
  refunds, chargebacks, and a compliance conversation
```

That is the friend’s appliance. The interface fitted perfectly. Nobody checked the voltage. **One adapter with the conversion written once and unit-tested is the entire defence**, and it is why “an adapter translates units, not just names” is worth saying out loud.

7 The trade-offs

What you give up

A layer of indirection. To find out what actually happens on a charge, a reader opens the protocol, then the adapter, then the SDK. Three files instead of one.

Lowest-common-denominator interfaces. If your `PaymentGateway` must be satisfied by four providers, it can only expose what all four can do. The fifth provider has a wonderful instalment feature and your interface has nowhere to put it. This is a real and recurring cost,

and the honest handling is to widen the interface deliberately for the one that matters, or accept that a specific capability is used through a specific adapter.

Drift. The vendor adds a field, and your adapter does not. Nobody notices, because your interface did not change. Adapters need to be re-read when the SDK is upgraded, and nothing forces that.

Leaky behaviour that the type system cannot see. Your interface says `charge(order) → Payment`. It does not say that one provider takes 200 ms and another takes 4 seconds, that one is idempotent and one is not, or that one enforces a rate limit. Two implementations of one interface can behave very differently in ways no signature captures — this is the Liskov problem from [day 057](#) arriving through the back door.

“I would not use this if...”

- **...I control both sides.** If both interfaces are mine, I change one of them. An adapter between two things you own is a refactor you declined to do.
- **...there is exactly one provider and there will only ever be one, and it is not an external service.** A local library with a slightly awkward API used in one place does not need a layer.
- **...the “adapter” would forward every method unchanged.** That is a middle man, and it is a smell from [day 061](#). If nothing is being translated, delete it.
- **...I am wrapping something to avoid learning it.** An adapter over a library you find confusing hides the confusion rather than resolving it, and the person after you now has two things to understand.

The strongest counter-argument, and how to answer it

“You will still have to change things when you swap providers, because the semantics differ.”

That is true and you should concede it. The adapter buys **source-level independence** — your domain does not import theirs — and it does **not** buy a free swap. Retry semantics, idempotency, webhook formats and failure modes all differ, and some of that surfaces. What the adapter guarantees is that the differences are confronted in **one file** rather than discovered in fourteen.

8 In the interview

How it gets asked

- The direct one: *“You must integrate a third-party SDK whose interface does not match yours. How?”*
- The distinction: *“What is the difference between adapter, decorator, facade and proxy?”* All four wrap something; the answer is what each one changes.
- The scenario: *“We are moving from Razorpay to Stripe. Walk me through it.”* Where the answer is adapter plus factory from [day 065](#).

- The legacy version: *“There is a ten-year-old module nobody wants to touch. How do you build on it?”* — adapter, and the phrase they want is anti-corruption layer.

What to say out loud, in the first ninety seconds

1. **Name the three parties.** “There is my code, there is their SDK which I cannot change, and I want a small class in the middle that has my interface and forwards to theirs.”
2. **Say where the interface lives, and whose words it uses.** “The protocol goes in my package, in my vocabulary. Its signature mentions my `Order`, my `Money` and my `PaymentDeclined`, never their types. If their type appears in the signature, I have inverted nothing.”
3. **List the four translations, and dwell on the last three.** “Names are the easy part. The adapter also converts the data shape, the units and the error types. Units are where real money gets lost — cents against paise.”
4. **Say the test.** “The way I check the adapter is real is to write a fake for tests. If the fake has to construct a Stripe-shaped dictionary, then I adapted the name and nothing else.”
5. **Concede the limit.** “This buys source-level independence, not a free swap. Semantics still differ — but they differ in one file.”

The follow-ups

“**What is the difference between an adapter and a facade?**” “An adapter changes an interface into a different one, usually one-to-one, because the shapes do not match. A facade simplifies — it puts one door in front of several subsystems because the caller should not have to know there are six of them. Adapter is about incompatibility; facade is about complexity. And an adapter usually has one adaptee, while a facade has many.”

“**And a decorator?**” “A decorator keeps the same interface and adds behaviour, which is what lets you stack them — a caching decorator around a logging decorator around the real thing. An adapter changes the interface, so it cannot stack. Same structure on a diagram, opposite intent.”

“**Where do you put the interface?**” “In the package that consumes it, in that package’s vocabulary. If I put it in an `adapters` package or write its signature in the vendor’s types, it looks identical on a class diagram and it has inverted nothing — every caller still changes when the vendor changes. In Python `typing.Protocol` is structural, so I can declare it in my module and the adapter satisfies it without importing anything of mine.”

“**How do you test this?**” “The interface makes it easy, and that is most of the value. I write a fake gateway that implements the protocol and can be told to raise `PaymentDeclined`. Testing ‘a declined card leaves the order unpaid’ becomes four lines and a fraction of a millisecond, against about thirty-five lines and hundreds of milliseconds hitting their sandbox — and the sandbox version fails when their sandbox is down, which teaches the team to ignore failures.”

“What are the limits of this?” “Two. The interface becomes the intersection of what all the providers can do, so a capability only one of them has has nowhere to live. And it does not make providers behave the same — retry semantics, idempotency and webhook formats still differ. The adapter means those differences are handled in one file instead of discovered in fourteen.”

A model answer

Asked: *you must integrate a third-party payment SDK whose interface does not match ours. How?*

“There are three parties. My code, their SDK which I cannot edit, and an adapter in the middle that has the interface my code wants and forwards to theirs. Neither side changes and neither side knows the adapter exists.

The first thing I would do — and the order matters — is write the interface I wish existed, before reading their documentation. If I read their API first, I will unconsciously shape my interface like theirs and the adapter will have nothing left to do. So: `charge(order: Order) → Payment`, raising `PaymentDeclined`.

That protocol goes in *my* package, in my vocabulary — my `Order`, my `Money`, my exceptions. This is the part people skip. If I put the protocol in an `adapters` package, or if their type appears anywhere in the signature, it looks the same on a diagram and it has inverted nothing, because every caller still changes the day the vendor does. In Python I would use `typing.Protocol`, which is structural, so the adapter satisfies it without importing anything of mine — about six lines.

Then the adapter itself, and it has four jobs, not one. The obvious one is the interface: `create_charge` becomes `charge`. The three that actually matter are the data shape, so their `dict` becomes my `Payment` object and no vendor dictionary ever escapes into my domain; the units and types, so their cents-as-an-int becomes my `Money` in paise, their Unix timestamps become timezone-aware datetimes, their `'IND'` becomes my `'IN'`; and the errors, so `stripe.error.CardError` becomes `PaymentDeclined`. If I skip the errors, `except stripe.error.CardError` ends up in my business logic and the vendor is back inside my domain despite the interface.

The units are the ones I would emphasise, because they are the ones that cost money. A converter that gets the plug shape right and the voltage wrong burns the appliance, and an adapter that gets the method name right and the currency unit wrong charges 45,000 cents instead of 45,000 paise.

The way I check the adapter is real is to write a fake for tests. If the fake can implement the protocol without constructing anything Stripe-shaped, the adaptation is genuine. That fake is also where most of the value shows up: ‘a declined card leaves the order unpaid’ becomes four lines and a fraction of a millisecond instead of thirty-five lines against their sandbox.

The proof that it worked is a command: `grep -rn 'stripe' orders/` returns nothing.

Two limits I would state. The interface ends up being what all my providers can do, so a capability only one of them offers has nowhere to live. And the adapter buys source-level independence, not a free swap — retry semantics, idempotency and webhook formats still differ between providers. What it guarantees is that I meet those differences in one file rather than in fourteen.”

9 Recall card

- **Three parties: client, adaptee, adapter.** The adapter implements the interface the client wants and forwards to an adaptee that **cannot be edited**. Neither side changes; neither side knows.
- **An adapter translates four things, and only the first is obvious.** The **interface** (names) · the **data shape** (their `dict` → your `Payment`) · the **units and types** (cents→paise, epoch→aware datetime, `"IND"` → `"IN"`) · the **errors** (`stripe.error.CardError` → `PaymentDeclined`). The last three are the voltage — the plug fitting is not the same as it working.
- **The interface lives in the *consuming* package, in *your* vocabulary.** A protocol whose signature mentions a vendor type has inverted nothing and looks identical on a diagram. `typing.Protocol` is structural, so ~6 lines and the provider is never edited. Proof: `grep -rn "stripe" orders/` returns nothing.
- **The test that it is a real adapter: can you write a fake with nothing vendor-shaped in it?** If the fake needs a Stripe-looking dict, you adapted the name only. That fake is also the payoff — 4 lines and 0.3 ms against 35 lines and 800 ms of flaky sandbox.
- **Adapter changes the interface · decorator changes behaviour (and stacks) · facade changes how many things you talk to · proxy changes when you reach the real one.** Use the **object** adapter (composition), not the class adapter (multiple inheritance leaks the whole adaptee). At whole-system scale it is an **anti-corruption layer**. And concede the limit: it buys **source-level independence, not a free swap** — but the differences land in one file, not fourteen.

PRACTICE

Practice — Stacks: last in, first out

DSA topic: Stacks: last in, first out System design topic: Adapter

Code these, in this order

One rule for the whole set: **say the sentence before you write the loop.** “The stack holds the most recent __ that has not yet been resolved.” Fill in the blank out loud. If you cannot, you have not understood the problem yet, and the code will not save you.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Remove All Adjacent Duplicates In String	LeetCode 1047 (Easy)	The cancel-with-the-top pattern, and the <code>if stack</code> and <code>...</code> guard.
2	Baseball Game	LeetCode 682 (Easy)	Reading a specification carefully while the stack part stays trivial.
3	Simplify Path	LeetCode 71 (Medium)	Real edge cases: <code>/../</code> at the root, <code>//</code> , and <code>"..."</code> being a legal name.
4	Backspace String Compare	LeetCode 844 (Easy)	Write it with a stack, then do it again in $O(1)$ space with two pointers from the right.

On problem 1, break the guard on purpose

Write `if stack[-1] == character` without `if stack` and `.`. Run it on `"a"`. Read the exact error. Then fix it and say why the check has to be part of the same condition rather than a separate `if` above.

On problem 3, test the four inputs that matter

`"/../"`, `"/home//foo/"`, `"/a/./b/../../../../c/"` and `"/..."`. The last one is the trap: three dots is a legal directory name, so `part.startswith("../")` is wrong and `part == ".."` is right.

On problem 4, do it twice

The stack version is six lines and $O(n)$ space. Then the interviewer says “ $O(1)$ space”, and the answer is two pointers walking from the right, counting backspaces. Write both, and say which you would offer first and why.

The which-end drill

1. Time a million `append` + `pop` pairs.
2. Time a hundred thousand `insert(0, x)` + `pop(0)` pairs on a ten-thousand-element list.
3. Compute the ratio, and note that number 2 did ten times fewer operations.
4. Say why `insert(0, x)` is $O(n)$ in one sentence, in terms of what has to move.

5. Now do the same with `collections.deque`. What changes, and why?
6. Say when you would choose `deque` over `list` for a stack.

The recognition drill

For each problem, fill in the sentence “*the stack holds the most recent __ not yet resolved*”, or say that a stack is not the right structure:

1. Check whether brackets are balanced.
2. Implement browser back.
3. Implement browser forward as well.
4. Evaluate `3 + 4 * 2` respecting precedence.
5. Return the elements in the order they arrived.
6. Find the next greater element to the right of every element.
7. Undo the last five actions.
8. Serve customers in the order they queued.
9. Convert a recursive tree traversal to an iterative one.
10. Find the k most frequent words.

Three of the ten are not stack problems. Name them and say what each one wants instead.

The empty-stack drill

Trigger each and quote the exact error:

1. `[] .pop()`
2. `[] [-1]`
3. `collections.deque().pop()`
4. `Stack().pop()` on your own class with no guard.

Then rewrite your `Stack.pop` and `Stack.peak` so the message mentions a stack rather than a list, and say why that matters to a caller.

The call-stack drill

1. Write `def r(n): return r(n + 1)` and call it. Quote the exact error and the default limit.
2. Raise the limit with `sys.setrecursionlimit(1000000)` and try again. What happens, and why is that worse than the exception?
3. Write a recursive function that sums a list, then rewrite it with an explicit stack.
4. Say what the explicit-stack version is limited by instead.
5. Answer out loud: “*this tree is a million nodes deep — will your recursive solution work?*”

The build-a-stack drill

1. Write `Stack` with `push`, `pop`, `peek`, `is_empty` and `__len__`.
2. Write `peek` as pop-then-push. Say two things wrong with it.

3. Now implement it over a **fixed-size** array. What does `push` have to do when it is full?
4. Implement doubling on overflow. Say why that makes push amortised $O(1)$ and show the `8 + 16 + 32 + ...` sum.
5. Say what a linked-list implementation would change — which bound gets stronger, and what gets worse.
6. Say which one every standard library actually uses, and why.

The counting-argument drill

For each, say why the code is $O(n)$ despite having a loop inside a loop, or a pop inside a loop:

1. `remove_adjacent_duplicates`
2. `simplify_path`
3. A `while stack and stack[-1] < current: stack.pop()` inside a `for`.

The sentence is the same for all three. Say it exactly, and say what it would take to make one of them genuinely quadratic.

The adapter drill

Here is the vendor SDK you cannot edit:

```
# vendor_sdk.py
class SmsVendor:
    def push_message(self, to_e164: str, body: str,
                     sender_id: str, unicode_flag: int) → dict:
        """returns {"msg_id": "...", "state": "QUEUED"|"REJECTED", "cost_micros": 1200}"""
```

And here is what your code wants:

```
class SmsSender(Protocol):
    def send(self, message: Message) → Receipt: ...
```

1. Write the adapter.
2. Name every one of the four translations you performed, with the specific line that does each.
3. Their cost is in micro-rupees; yours is `Money` in paise. Write the conversion and say what breaks if you get it wrong.
4. Their `"REJECTED"` should become an exception in your vocabulary. Which one, and why not just return it?
5. Write a fake `SmsSender` for tests. Does it contain anything vendor-shaped? If yes, go back to step 1.
6. Say which package the `SmsSender` protocol lives in, and why the other option looks identical on a diagram.
7. Run the grep that proves the dependency was inverted. Write it out.

The half-adapter drill

Here is an adapter that only did half the job:

```
class StripeGateway:
    def charge(self, order: Order) → dict:
        return self._client.create_charge(amount_cents=order.total_cents,
                                           source_token=order.token)
```

1. Name the three translations it skipped.
2. Write out a call site that uses it. What does the caller now know about Stripe?
3. Count how many call sites would change when you swap providers, given this adapter.
4. Say what the fake for this interface would have to look like.
5. Fix it, and say what the fake looks like now.

The four-wrappers drill

For each, say which of adapter, decorator, facade or proxy it is, and give the one-sentence reason:

1. `functools.lru_cache` around a function.
2. `psycopy` presenting the DB-API interface.
3. `requests.get(url)`.
4. nginx in front of your application.
5. `io.TextIOWrapper` around a binary stream.
6. An ORM object that fires a query the first time you touch a field.
7. A class that adds logging around every method of a service, same interface.
8. `java.util.Arrays.asList()`.

One of the eight is also a Liskov violation shipped in a standard library. Name it and say why.

The swap drill

You are moving from provider A to provider B.

1. Without an adapter: run a grep for the vendor name in a real project you have. How many references, in how many files?
 2. With an adapter: how many files are added, and how many lines edited?
 3. Describe how you would send 5% of traffic to B while keeping 95% on A. Which pattern from [day 065](#) does the routing?
 4. Name two things the adapter does *not* make the same between A and B.
 5. Say the honest sentence about what an adapter buys and what it does not.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Implement a stack. What operations must be $O(1)$? The restriction first and why it causes the $O(1)$, the three operations, “a list is a stack” with the which-end warning and its number, amortised push explained in one sentence, and the two alternatives with the reason you would not use one of them.*
 2. *You must integrate a third-party SDK with a different interface. How? Three parties named, the interface written before reading their docs, where it lives and whose vocabulary it uses, the four translations with the units emphasised, the fake-based test for whether it is real, and the honest limit.*
 3. *Why is a `while stack: pop()` inside a `for` loop still $O(n)$? Each element pushed at most once and popped at most once, so the total pops are bounded by the total pushes — and what it would take to break that.*
-

Before you move on

- ☐ I can say the recognition sentence — “the most recent __ not yet resolved” — from memory.
- ☐ I timed `append / pop` against `insert(0) / pop(0)` and can quote the ratio.
- ☐ I triggered `IndexError: pop from empty list` and know where the guard belongs.
- ☐ I found the `"..."` case in Simplify Path myself.
- ☐ I wrote `Stack` from scratch, including the fixed-size and doubling version.
- ☐ I can explain amortised $O(1)$ push with the `8 + 16 + 32 + ...` sum.
- ☐ I hit `RecursionError` and can say why raising the limit is not a fix.
- ☐ I wrote a real adapter and named all four translations with the line that does each.
- ☐ I wrote the fake and confirmed it contains nothing vendor-shaped.
- ☐ I can separate adapter, decorator, facade and proxy in one sentence each.
- ☐ I answered all three questions above out loud.

Balanced brackets and the parsing family

DATA STRUCTURES AND ALGORITHMS

Balanced brackets and the parsing family

You can validate nested structures and evaluate simple expressions with a stack.

SYSTEM DESIGN

Decorator

You can add behaviour to an object at runtime without subclassing it.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Check whether the brackets in this string are balanced.*
- *Add logging and caching to this service without changing its code.*

1 What this is, and why they ask it

Given a string of brackets — `()`, `[]`, `{}` — decide whether every opener has a matching closer of the same type, in the right order. `"({[]})"` is balanced. `"([)]"` is not, even though the counts are equal. The answer is a stack, and the whole idea fits in one sentence: **when you meet a closer, the only thing it can legally close is whatever is on top of the stack.**

They ask it because it is the cheapest way to find out whether you can turn a rule into an invariant. The problem is easy enough that nobody fails to write *something*, and there are exactly three ways to be wrong — a mismatched pair, a closer with nothing open, and something still open at the end. Candidates who name all three before writing code are doing a different thing from candidates who discover them when a test fails.

The second reason is that this is the doorway to a family. Once you can say “the stack holds the context I am inside and have not yet left”, the same shape solves expression evaluation, nested string decoding, calculators with parentheses, JSON and XML validity, and the compiler front end that told you about your own unclosed brace. Interviewers know that the family is worth more than the problem, so the follow-up is almost always a harder member of it.

2 The story

After the housewarming lunch, Kamala was putting the vessels away, which is a job she does not mind because it is the one part of the day with nobody talking to her.

There are three sets in that kitchen and they all nest. The steel ones, five of them, largest to smallest. The aluminium ones, four. And a plastic set of three that somebody gave them and nobody likes but which is useful for the fridge.

The way they go away is that the small one goes inside the next one up, which goes inside the next one up, and so on, and each one gets its own lid on before the next one goes over it. So opening the whole thing back up happens in the exact reverse order — the outermost lid first, then the vessel under it, then that one's lid, all the way down.

Two rules, and Kamala has never once said them out loud because they are obvious to her.

The first is that a lid belongs to its own vessel. The steel lids and the aluminium lids are close enough in size that a distracted person will put a steel lid on an aluminium vessel, and it will sit there, and it will look fine, and three weeks later somebody will pick it up by the lid and the vessel will come off and land on the floor. It fits. It is not the right one.

The second is that you cannot put a lid on nothing. This sounds too obvious to state, and yet on the day of the lunch her sister-in-law was helping and there was a moment where she had a lid in her hand and nothing in front of her to put it on, because the vessel it belonged to had already gone into the cupboard under three others.

And the third thing, which is not a rule so much as the end of the job: when everything is put away, there should be nothing left on the counter. If there is a vessel still sitting there with no lid on it, the job is not finished, however tidy the rest of it looks.

That evening there was one — the second-smallest steel one, still open, next to the sink. Everything else was away and stacked and looked completely correct, and the job was still wrong.

3 The idea in plain English

Each vessel being opened is an **opening bracket**. Each lid going on is a **closing bracket**. The nesting is the nesting, and the pile on the counter is the **stack**.

Kamala’s two rules and her end-of-job check are exactly the three ways the algorithm can fail.

KAMALA	IN THE STRING	THE CHECK
Steel lid on an aluminium vessel	"([)]"	the closer does not match the top of the stack
A lid with nothing to put it on	"())"	a closer arrives and the stack is empty
A vessel still open at the end	"(())"	the stack is not empty when the string ends

Say **all three out loud before writing**. That is the whole difference between a good answer and an adequate one.

Why counting does not work

The tempting shortcut is to count openers and closers and check they match. It works for one type of bracket and it is wrong the moment there are two.

```
"([)]"    one '(', one ')', one '[', one ']'    counts all match
                                                and it is NOT balanced
```

Counting throws away order, and order is the entire question. Say this out loud when you reject the counting approach — it shows you considered it rather than never thought of it.

For **one** bracket type, counting does work, and it is worth knowing because it gives an O(1)-space answer:

```

depth = 0
for ch in text:
    depth += 1 if ch == "(" else -1
    if depth < 0:
        return False          # a closer with nothing open
return depth == 0

```

If the interviewer says “only round brackets, $O(1)$ space”, this is the answer.

The algorithm

```

pairs = {"(": "(", ")": ")", "[": "[", "]": "]" } # closer → its opener
stack: list[str] = []

for character in text:
    if character in pairs.values():             # an opener
        stack.append(character)
    else:                                       # a closer
        if not stack or stack.pop() != pairs[character]:
            return False
return not stack

```

The invariant, and this is the sentence to say: **at every point, the stack holds exactly the openers that have been seen and not yet closed, in the order they were opened.**

Two details that get asked about.

Map closers to openers, not openers to closers. When you meet `)` you already know what you have; you need to know what it *should* have found. `pairs[")"]` gives `"("` directly. The other direction needs a search.

`if not stack or ...` — **the emptiness check comes first.** Python’s `or` short-circuits, so `stack.pop()` never runs on an empty stack. Write it in that order and the third failure mode is handled by the same line as the first.

The generalisation: a stack of contexts

Now the part that makes this a family rather than a problem.

When you meet an opener, push the context you are leaving. When you meet a closer, pop it and combine.

Brackets are the degenerate case where the “context” is just the bracket character. In real parsing problems the context is more.

Decode String — `"3[a2[c]]"` becomes `"accaccacc"`. When you meet `[`, push the string built so far *and* the repeat count, then start fresh. When you meet `]`, pop them and combine.

Reverse Polish notation — `["2", "1", "+", "3", "*"]` is 9. Push numbers; on an operator, pop two, apply, push the result. No brackets at all, same stack.

Basic Calculator — `"1 + (2 - (3 + 4))"`. On `(`, push the running result and the sign; on `)`, pop and combine.

All three are the same two lines with a richer thing being pushed. If you can say that sentence in an interview, the follow-up stops being a new problem.

The related question they will ask next

Longest valid parentheses — the longest balanced substring of `"())()())"`, which is 4. The trick is to push **indices** rather than characters, and to keep a sentinel at the bottom marking “the last position that was invalid”. The length of a valid run is then `current_index - stack[-1]`. It is the standard hard follow-up and it is worth recognising even if you do not write it.

4 The picture

`"({[]})"` processed one character at a time. The stack grows as things open and shrinks as they close.

char:	(	{	[	]	}	)
	+---+	+---+	+---+	+---+	+---+	+---+
			[			
		{	{	{		
	(	(	(	(	(	
	+---+	+---+	+---+	+---+	+---+	+---+
	push	push	push	pop [	pop {	pop (
				matches	matches	matches

end: stack empty → balanced

What to notice: the stack is a picture of *how deep you currently are*. Its height at any moment is the nesting depth, and the top is always the thing that must close next.

Now the three failures, drawn on their smallest inputs:

```
(a) wrong type      "([])"
    ( → push (      stack: [ ( ]
    [ → push [      stack: [ (, [ ]
    ) → pop gives [  but ')' needs '(' → MISMATCH

(b) closer, nothing open  "())"
    ( → push      stack: [ ( ]
    ) → pop      stack: [ ]
    ) → stack empty → NOTHING TO CLOSE

(c) still open at the end "(()"
    ( → push      stack: [ ( ]
    ( → push      stack: [ (, ( ]
    ) → pop      stack: [ ( ]
    end of string, stack is NOT empty → UNCLOSED
```

Three failures, three checks, and they map one-to-one onto three lines of code. Nothing else can go wrong.

5 The code, built step by step

Step 1 — the map, in the right direction

```
PAIRS = {")": "(", "]": "[", "}": "{"}
```

Keyed by the closer. When you are holding a `)` you want to ask “what should this have found?” and get `(` in one lookup.

Step 2 — the loop

```
def is_balanced(text: str) → bool:
    stack: list[str] = []
    for character in text:
        if character in "({":
            stack.append(character)
        elif character in PAIRS:
            if not stack or stack.pop() ≠ PAIRS[character]:
                return False
    return not stack
```

Nine lines, and every one of the three failures is covered. `not stack` catches the empty case; `stack.pop() ≠ PAIRS[character]` catches the mismatch; `return not stack` catches the leftovers.

Note the `elif character in PAIRS` rather than a bare `else`. If the string can contain other characters — letters, spaces, an entire program — anything that is neither an opener nor a closer should be ignored, not treated as a closer. Ask whether the input is brackets only; if it is not, this line is why you pass.

Step 3 — the generalisation, on a harder problem

Decode String. `"3[a2[c]]"` becomes `"accaccacc"`. Repeat counts can be multi-digit, and nesting can be arbitrary.

The context to push at a `[` is two things: the string built so far, and the number that will repeat what comes next.

```
def decode_string(text: str) → str:
    stack: list[tuple[str, int]] = []
    current = ""
    number = 0
    ...
```

Digits accumulate, because `12[a]` is twelve, not one then two:


```
for character in text:
    if character.isdigit():
        number = number * 10 + int(character)
```

`number * 10 + digit` is the standard way to build a multi-digit integer as you read it. Writing `number = int(character)` gives 2 for "12" and is the single most common bug in this problem.

On `[`, save the outer context and reset:

```
elif character == "[":
    stack.append((current, number))
    current, number = "", 0
```

On `]`, restore it and combine:

```
elif character == "]":
    previous, repeat = stack.pop()
    current = previous + current * repeat
```

Read that line carefully. `current` is what was built *inside* the brackets, so it gets repeated; `previous` is what was built *outside*, so it goes in front, unrepeated. Getting these the wrong way round produces plausible garbage.

And anything else is a literal character:

```
else:
    current += character
```

The complete solution

```
PAIRS = {"(": ")": "(", ")": "[", "]": "{"}
OPENERS = "([{{"

def is_balanced(text: str) → bool:
    """True if every bracket is closed by its own kind, in the right order.

    Invariant: the stack holds exactly the openers seen and not yet closed,
    in the order they were opened. Three ways to fail – wrong type on top,
    a closer with an empty stack, and leftovers at the end.
    """
    stack: list[str] = []
    for character in text:
        if character in OPENERS:
            stack.append(character)
        elif character in PAIRS:
            if not stack or stack.pop() ≠ PAIRS[character]:
                return False
            # anything else is ignored – letters, spaces, digits
    return not stack

def is_balanced_one_type(text: str) → bool:
    """One bracket type only: 0(1) space, no stack needed."""
    depth = 0
    for character in text:
        if character == "(":
            depth += 1
        elif character == ")":
            depth -= 1
            if depth < 0:
                # closed one that was never opened
                return False
    return depth == 0

def min_additions_to_balance(text: str) → int:
    """Fewest brackets to add to make it valid. Same counter, two totals."""
    open_needed = 0      # unmatched ')' seen so far
    close_needed = 0     # unmatched '(' still open
    for character in text:
        if character == "(":
            close_needed += 1
        else:
            if close_needed:
                close_needed -= 1
            else:
                open_needed += 1
    return open_needed + close_needed

def decode_string(text: str) → str:
    """'3[a2[c]]' → 'accaccacc'."""

    The stack holds the context being left: the string built outside these
    brackets, and the number of times the inside will repeat.
    """
    stack: list[tuple[str, int]] = []
    current = ""
    number = 0

    for character in text:
        if character.isdigit():
            number = number * 10 + int(character) # multi-digit counts
        elif character == "[":
            stack.append((current, number))
            current, number = "", 0
        elif character == "]":
```

```

        previous, repeat = stack.pop()
        current = previous + current * repeat # inside repeats, outside does not
    else:
        current += character

    return current

def evaluate_rpn(tokens: list[str]) → int:
    """Reverse Polish notation: no brackets, same stack.
    ['2', '1', '+', '3', '*'] → 9"""
    stack: list[int] = []
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            right = stack.pop()
            left = stack.pop() # order matters for - and /
            if token == "+":
                stack.append(left + right)
            elif token == "-":
                stack.append(left - right)
            elif token == "*":
                stack.append(left * right)
            else:
                stack.append(int(left / right)) # truncate toward zero
        else:
            stack.append(int(token))
    return stack[0]

if __name__ == "__main__":
    print(is_balanced("({[]})")) # True
    print(is_balanced("([)])")) # False - wrong type
    print(is_balanced("{}))")) # False - nothing to close
    print(is_balanced("(())")) # False - still open
    print(is_balanced("")) # True - vacuously balanced
    print(is_balanced("if (x[0]) { go(); }")) # True - other characters ignored
    print(is_balanced_one_type("")) # False
    print(min_additions_to_balance("()")) # 1
    print(decode_string("3[a]2[bc]")) # aaabcbc
    print(decode_string("3[a2[c]]")) # accaccacc
    print(decode_string("2[abc]3[cd]ef")) # abcabccdcdef
    print(evaluate_rpn(["2", "1", "+", "3", "*"])) # 9
    print(evaluate_rpn(["4", "13", "5", "/", "+"])) # 6

```

Run the empty string. `is_balanced("")` is `True`, because there is nothing unmatched — vacuously balanced. Interviewers ask about it, and “true, because nothing is left open” is the answer.

6 What it costs

`is_balanced`, counted

The loop runs once per character: `n` iterations. Inside, each iteration does at most one push or one pop plus one dictionary lookup, all $O(1)$.

`n` iterations x constant work → $O(n)$ time

Space is the stack. In the worst case — `"(((((((..."` — every character is an opener and nothing is popped, so the stack holds `n` entries.

```
worst case space:  O(n)      "(((((((("
best case space:   O(1)      "()()()()" – pushed and popped immediately
```

Say both. “ $O(n)$ in the worst case, when the input is entirely openers” is more precise than “ $O(n)$ ” and takes two extra seconds.

The counting version, for one bracket type

```
time:   O(n)
space:  O(1)      one integer
```

That is the trade to offer when asked for constant space, and the reason it only works for one type is that a single integer cannot remember *which* kind of bracket is open.

`decode_string`, `counted`

Trickier, and worth being careful because the naive answer is wrong.

The loop runs `n` times over the input. But `current = previous + current * repeat` builds a string whose length can be much larger than `n` — `"10[10[10[a]]]"` is 13 characters of input and a thousand characters of output.

```
time:   O(n + m) where m is the LENGTH OF THE OUTPUT
space:  O(n + m)
```

The honest statement is: *linear in the size of the output, not the input*. An interviewer who asks “what is the complexity” is often checking exactly this, because the input length is a misleading measure for any problem that expands.

There is also a hidden cost in `current += character` inside a loop, which is the string-building trap from [day 020](#). For interview inputs it is fine; for very large outputs, accumulate into a list and `"".join` at the end.

The stack argument, again

Same sentence as [day 068](#): **each character is pushed at most once and popped at most once**, so even with a `pop` inside the loop the total work is linear. It will keep being the answer for the rest of this phase.

7 The traps

Trap 1 — counting instead of matching

```
return text.count("(") == text.count(")")    # WRONG
```

Returns `True` for `")("`, which is not balanced. And with multiple bracket types it returns `True` for `"([)]"`. The counts are a necessary condition, never a sufficient one.

Trap 2 — popping an empty stack

```
if stack.pop() != PAIRS[character]:    # no emptiness check
```

```
>>> is_balanced_buggy("]")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: pop from empty list
```

A one-character input finds it. The fix is `if not stack or stack.pop() != ...`, relying on `or` short-circuiting so `pop` is never reached on an empty stack.

Trap 3 — forgetting the end-of-string check

```
for character in text:
    ...
return True    # WRONG — should be `return not stack`
```

Returns `True` for `"(((("`, because nothing ever mismatched. This is Kamala's vessel by the sink: every individual step was correct and the job is not done. It is the most common wrong answer to this problem.

Trap 4 — mapping openers to closers

```
PAIRS = {"(": ")" , "[": "]" , "{": "}" }    # the wrong direction
```

Now when you meet `)` you cannot look it up. You end up either searching the dictionary's values or writing a second dictionary. Key by the closer; you are always looking up the thing in your hand.

Trap 5 — treating every non-opener as a closer

```
else:
    if not stack or stack.pop() != PAIRS[character]:
```

On `"a"` this raises:

```
KeyError: 'a'
```

If the input can contain anything other than brackets — and real inputs like source code always can — use `elif character in PAIRS` and let everything else fall through. Ask the interviewer whether the input is brackets only. It is a ten-second question that changes the code.

Trap 6 — single-digit numbers in `decode_string`

```
number = int(character)          # WRONG for "12[a]"
```

Gives 2 instead of 12, so `"12[a]"` returns `"aa"`. Use `number = number * 10 + int(character)`, and remember to reset `number = 0` after pushing.

Trap 7 — the wrong side of the concatenation

```
current = current * repeat + previous    # WRONG — outside goes AFTER inside
```

On `"a2[b]"` the correct answer is `"abb"` and this gives `"bba"`. The rule to say out loud: *what was inside the brackets repeats; what was outside stays in front*.

Trap 8 — operand order in RPN

```
left = stack.pop()
right = stack.pop()          # WRONG WAY ROUND
```

The **second** pop is the left operand, because the right one was pushed last. `+` and `*` hide the bug because they commute; `-` and `/` expose it. `["4", "13", "5", "/", "+"]` should be 6 and comes out wrong.

8 In the interview

How it gets asked

- “Given a string containing just `(`, `)`, `{`, `}`, `[` and `]`, determine if the input is valid.” LeetCode 20, almost word for word, at nearly every company.
- The follow-up that always comes: “what if the string also contains letters and spaces?” or “what is the minimum number of brackets to add to make it valid?”
- The family: “decode `3[a2[c]]`”, “evaluate this expression with parentheses”, “evaluate this reverse Polish expression”.
- The disguised one: “validate this JSON without a library” — same algorithm, richer contexts.

What to say out loud, in the first ninety seconds

1. **Name the three failure modes before writing anything.** “There are exactly three ways this can be invalid: a closer that does not match the most recent opener, a closer when nothing is open, and something still open when the string ends. My code will have one check for each.”
2. **Say the invariant.** “The stack holds exactly the openers I have seen and not yet closed, in order. So when I meet a closer, the only thing it can legally close is the top.”
3. **Reject counting, out loud.** “Counting brackets is not enough — `[[[]]` has matching counts and is invalid — because counting throws away the order, and the order is the question.”

4. **Ask the one clarifying question.** “Can the string contain other characters?” It changes one line and shows you thought about real inputs.
5. **State the cost.** “ $O(n)$ time. Space is $O(n)$ in the worst case, which is a string of all openers, and $O(1)$ on something like `()()()`.”

The follow-ups

“**Can you do it in $O(1)$ space?**” “Only for one bracket type. Then a counter works: increment on `(`, decrement on `)`, fail if it ever goes negative, and check it ends at zero. With three types a single integer cannot remember *which* kind is open, so I need the stack. The only way around that would be three counters, and that still fails on `([[]])` because counters do not preserve order.”

“**What is the minimum number of brackets to add to make it valid?**” “Two counters instead of one. One counts unmatched closers seen so far, the other counts openers still waiting. On `)` I either cancel an open one or record that I need an opener. The answer is the sum at the end. Same $O(n)$ time, and $O(1)$ space because I never needed to know which bracket it was.”

“**Now decode `3[a2[c]]`.**” “Same shape with a richer context. On `[` I push what I have built so far and the repeat count, then start fresh. On `]` I pop them and set `current = previous + current * repeat`. Two things to get right: multi-digit numbers need `number * 10 + digit`, and the outside string goes in front of the repeated inside, not after.”

“**What is the complexity of that one?**” “Linear in the size of the *output*, not the input, because the output can be exponentially bigger — `10[10[10[a]]]` is thirteen characters in and a thousand out. So $O(n + m)$ time and space where m is the output length.”

“**How would a compiler use this?**” “The same way, with a richer stack. It pushes an entry per open scope — brace, parenthesis, block — along with the line number where it opened. That is how it can tell you ‘unmatched `{` opened at line 41’ rather than just ‘syntax error’, and it is why the error points at the opener rather than the end of the file.”

A model answer

Asked: *check whether the brackets in this string are balanced.*

“Before I write anything, let me say the three ways this can be invalid, because my code will have one check for each. A closer that does not match the most recent opener, like `([)]`. A closer when nothing is open, like the second bracket in `(())`. And something still open when the string runs out, like `((()`.

That third one is the one people miss, because every individual step is correct and the failure is only visible at the end.

The thing I want is a stack, and the invariant is that it holds exactly the openers I have seen and not yet closed, in the order I opened them. That makes the rule simple: when I meet a closer, the only thing it can legally close is whatever is on top.

Worth saying why counting does not work — it is the obvious first idea. `([)]` has one of each bracket, so the counts match perfectly, and it is invalid. Counting throws away order and order is the whole question. For a single bracket type counting does work, and it gives an $O(1)$ -space answer, so if you tell me it is only round brackets I would use a counter instead.

One question before I write: can the string contain characters other than brackets? If it is source code or an expression, I want to ignore letters and spaces rather than treat them as closers — otherwise the dictionary lookup raises a `KeyError` on the first letter.

So: a dictionary keyed by the *closer*, giving its opener, because when I am holding a `)` I want to look up what it should have found. Push openers. On a closer, check the stack is non-empty and that popping gives the matching opener — and the emptiness check has to come first in the same condition, so short-circuiting stops me popping an empty stack. At the end, return whether the stack is empty.

Cost: $O(n)$ time, one pass. Space is $O(n)$ in the worst case, which is a string of all openers, and $O(1)$ on something like `((()())` where every push is popped immediately.

Edge cases: the empty string is balanced, vacuously — there is nothing left open. A single closer is the case that catches the missing emptiness check.

And this generalises, which is usually where you are going next. The stack holds *the context I am inside and have not yet left*. For brackets the context is just the character. For something like `3[a2[c]]` I push the string built so far and the repeat count, and pop and combine on the closer. Same two lines, richer thing being pushed.”

9 Recall card

- **Three ways to fail, and name all three before coding:** the closer does not match the top · a closer arrives with an **empty stack** · the stack is **non-empty at the end** (the one people forget — every step was fine and the job is not done).

- **Invariant: the stack holds exactly the openers seen and not yet closed, in order.** So a closer can only legally close the top. Map **closer** → **opener**, not the other way, because you look up the thing in your hand. Write `if not stack or stack.pop() ≠ PAIRS[c]` — or short-circuits, so `pop` never hits an empty stack.
- **Counting is not enough:** `"([)]"` has matching counts and is invalid, because counting throws away order. For **one** bracket type a counter *does* work — increment/decrement, fail if it goes negative, end at zero — giving **O(1) space**. Two counters answer “minimum additions”.
- **The family is one sentence: push the context you are leaving, pop and combine on the closer.** Decode String pushes `(current, repeat)` · RPN pushes operands · Basic Calculator pushes the running result and sign · a compiler pushes the scope *and its line number*, which is why it can say “unmatched { at line 41”.
- **Costs and traps.** `is_balanced`: O(n) time, O(n) space worst case (all openers), O(1) on `()()`. `decode_string`: linear in the **output**, not the input — `10[10[10[a]]]` is 13 in and 1000 out. Traps: `elif character in PAIRS` or a letter raises `KeyError` · `number*10 + digit` for multi-digit counts · `previous + current * repeat` (outside in front, inside repeats) · in RPN the **second** pop is the left operand, which only `-` and `/` expose.

1 What this is, and why they ask it

A **decorator** is an object that has the same interface as the thing it wraps, forwards every call to it, and does something extra on the way through. Because the interface is unchanged, the caller cannot tell the difference — and, crucially, **a decorator can wrap another decorator**. That is the whole pattern, and the stackability is the point.

They ask it because it is the clean answer to a question every real system asks: how do you add logging, caching, retries, timing, authorisation and rate limiting to a service without turning that service into a class that does seven things? The alternative people reach for is inheritance, and inheritance falls over immediately — three optional features means eight subclasses, four means sixteen, because you need one per combination.

The second reason is a specific trap for Python people. Python has a `@decorator` syntax, and it is *related to* but not *the same as* the Gang of Four decorator. One wraps a function; the other wraps an object and preserves its interface. A candidate who says “yes, `@lru_cache`” and stops has answered half. A candidate who says “the language feature is the function-level version of the same idea, and here is the object version” has answered all of it.

2 The story

The courier shop under the flats in Indiranagar does about sixty parcels a day, and the man who runs it, Faisal, has an order he does things in that he does not deviate from.

Somebody brings in a thing to send. Say it is a set of six glass tumblers going to Chennai.

First the tumblers go in newspaper, individually. Then the whole lot goes into a layer of bubble wrap. Then into a cardboard box, packed so nothing moves. Then tape along every seam. Then a red FRAGILE sticker on two sides. Then the address label on top, and finally a clear plastic sleeve over the label so the rain does not take the ink off.

Seven layers, and each one does exactly one job. The newspaper stops glass touching glass. The bubble wrap absorbs a drop. The box gives it a shape. The tape stops it opening. The sticker asks for care. The label says where it goes. The sleeve keeps the label readable.

And at every stage of that, from the first layer to the last, the thing on his counter is still just a parcel. He can hand it to the driver at any point and the driver’s job does not change. That is why he can add or leave out any layer he likes — a book going to the next street gets a box and a label and nothing else.

The order is not negotiable, and Faisal is quite sharp about it, because a boy who worked there for two months got it wrong twice.

The first time he put the address label on the box and then wrapped the whole box in plastic, which is fine, and then put the FRAGILE sticker on top of the plastic. It came off somewhere in transit. Nobody knew the parcel was fragile, which is the same as not having put a sticker on at all.

The second time was the other way round and worse. He put the FRAGILE sticker on the inner box and then put that box inside a bigger box. The sticker was perfect. Nobody was ever going to see it.

Faisal's rule, which he says to every new person, is that it is not enough for each layer to be right. The layers have to be in the right order, because a layer only affects what is inside it.

3 The idea in plain English

The tumblers are the **component** — the real object that does the actual job. Each wrapping is a **decorator**. And the reason the layers stack is that after every one of them, the thing is still a parcel: **the interface never changes**.

Two consequences, and they are the two things to say.

One: you can combine them freely. Seven layers, or two, or five in a different order. Nobody had to invent a “boxed, taped, fragile, labelled parcel” as a separate kind of thing.

Two: order matters, because a layer only affects what is inside it. The FRAGILE sticker on the inner box is a caching decorator underneath a logging decorator — technically correct, and it never sees the traffic you wanted it to see.

The shape

```
from typing import Protocol

class PriceService(Protocol):
    def price(self, order_id: str) → Decimal: ...

class RealPriceService:
    # the component
    def price(self, order_id: str) → Decimal:
        return self._db.fetch_price(order_id) # the actual work
```

A decorator implements the same protocol, holds one of them, and forwards:

```
class LoggedPriceService:
    # same interface
    def __init__(self, inner: PriceService) → None:
        self._inner = inner

    def price(self, order_id: str) → Decimal:
        log.info("pricing %s", order_id)
        result = self._inner.price(order_id) # forward
        log.info("priced %s = %s", order_id, result)
        return result
```

And another one, identical in shape:

```
class CachedPriceService:
    def __init__(self, inner: PriceService) → None:
        self._inner = inner
        self._cache: dict[str, Decimal] = {}

    def price(self, order_id: str) → Decimal:
        if order_id not in self._cache:
            self._cache[order_id] = self._inner.price(order_id)
        return self._cache[order_id]
```

Now compose:

```
service = CachedPriceService(LoggedPriceService(RealPriceService()))
```

`service` is a `PriceService`. Every caller uses it identically. Nobody has been edited.

Why not inheritance

Because you need one subclass per **combination**, and combinations grow as 2^n .

```
features: logging, caching, retry
subclasses needed:
    PriceService
    LoggedPriceService
    CachedPriceService
    RetryingPriceService
    LoggedCachedPriceService
    LoggedRetryingPriceService
    CachedRetryingPriceService
    LoggedCachedRetryingPriceService      = 8
```

Add a fourth feature and it is sixteen. Add timing, authorisation and rate limiting and it is one hundred and twenty-eight classes. With decorators it is **one class per feature** — three features, three classes, and any of the eight combinations is a line of composition.

This is 2^n against n , and it is the number to quote.

Order matters, and here is the concrete case

```
CachedPriceService(LoggedPriceService(real))    # cache OUTSIDE
LoggedPriceService(CachedPriceService(real))    # cache INSIDE
```

Not stylistic. They log different things.

With the cache **outside**, a cache hit never reaches the logger, so your logs show only the calls that missed. Your dashboard says “12 pricing calls” when the application made 4,000.

With the cache **inside**, every call is logged and only the misses reach the database. Your logs show 4,000, which is what you probably wanted.

Neither is wrong. But you have to know which one you built, and “the outermost decorator sees everything; the innermost sees least” is the sentence that gets it right every time. Faisal’s FRAGILE sticker on the inner box is the cache-outside case: perfectly applied, never seen.

The same reasoning settles the other common orderings:

- **Retry outside a timeout** means each attempt gets its own timeout. **Timeout outside retry** means all attempts share one budget. Both are legitimate; they are different products.
- **Authorisation should be outermost.** A cache in front of an auth check will serve a cached result to somebody who is not allowed to see it, which is a security bug, not a performance one.

The Python distinction you must make

```
@lru_cache
def price(order_id: str) → Decimal: ...
```

This is Python’s `@decorator` syntax. It takes a function, returns a replacement function, and the name is rebound. It is genuinely the same *idea* — wrap, add behaviour, keep the calling convention — and it is not the GoF pattern, which wraps an **object** and preserves an **interface**.

The differences that matter in practice:

	PYTHON @DECORATOR	GOF DECORATOR
Wraps	a function	an object
Applied	at definition time	at run time, when you compose
Chooseable per instance	no	yes
Removable at run time	no	yes

The second row is the one that decides real designs. `@lru_cache` is baked in when the module loads. An object decorator lets you cache in production and not in tests, or wrap only the instance serving a particular tenant.

Use the syntax when the behaviour is fixed for all uses. Use the object when composition should be a run-time decision.

One thing to always mention about the syntax version:

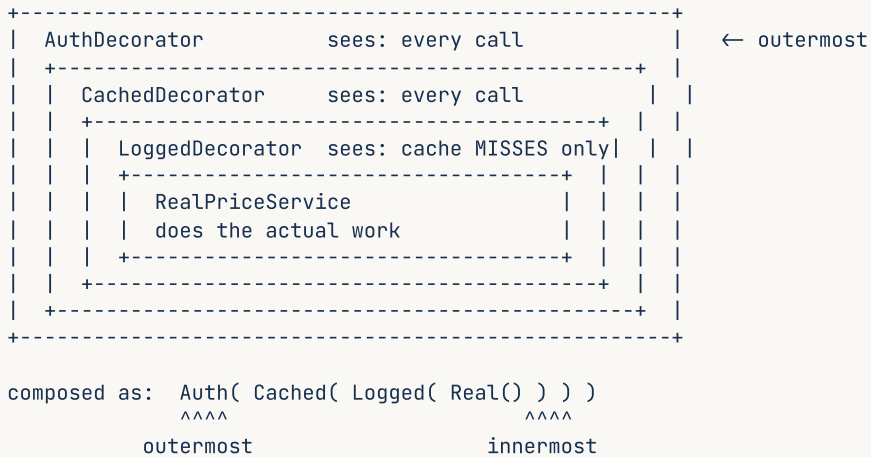
```
from functools import wraps

def timed(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        ...
    return wrapper
```

Without `@wraps`, the decorated function’s `__name__` becomes `"wrapper"`, its docstring disappears, and anything that introspects it — Flask’s routing, pytest’s collection, Sphinx — misbehaves.

4 The picture

The layers, and what each one sees.



What to notice: **each layer only sees what gets past the layer outside it.** The logger here is inside the cache, so it records misses and nothing else. Swap those two lines and it records everything. That single fact is what makes ordering a design decision rather than a formatting one.

And the same call, as a sequence, so the “on the way in and on the way out” shape is visible:

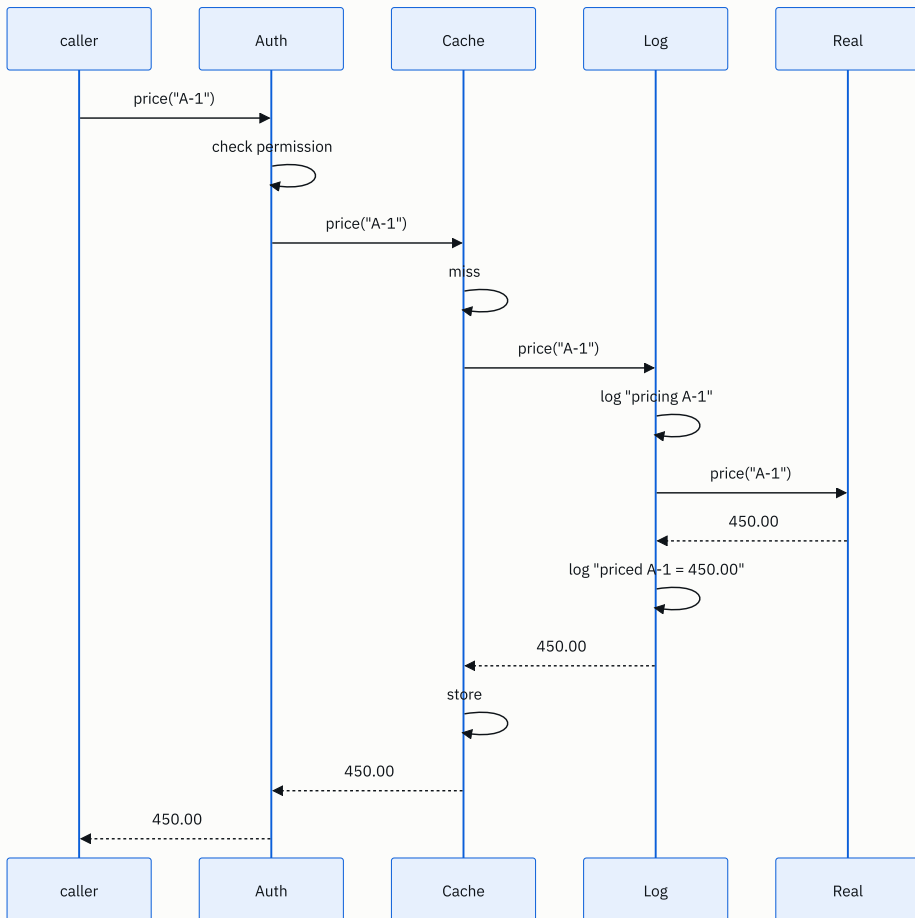


FIGURE 69.1

Every decorator gets two moments: before the forward, and after the return. Retry and timing use both; logging usually uses both; authorisation usually only the first.

5 How it actually works

Writing one, in order

1. **The interface must exist first.** Decorator only works if there is a type the component and all the decorators share. In Python, `typing.Protocol`.
2. **The decorator holds the inner one and forwards every method.** Every method, not just the interesting one — a decorator that implements two of five methods is not substitutable.
3. **Compose at the composition root,** where the object graph is built (day 059).

```
def build_service(settings: Settings) → PriceService:
    service: PriceService = RealPriceService(db)
    service = LoggedPriceService(service)
    if settings.cache_enabled:
        service = CachedPriceService(service)          # conditional!
    return AuthorisedPriceService(service, policy)
```

That `if` is the thing subclassing cannot do. The composition is a run-time decision.

The forwarding problem, and Python's escape hatch

For an interface with fifteen methods, a decorator that only changes one still has to forward fourteen. That is real, tedious code and a real source of bugs when method sixteen is added.

Python has an answer:

```
class LoggedService:
    def __init__(self, inner): self._inner = inner

    def __getattr__(self, name):          # anything not defined here
        return getattr(self._inner, name) # goes to the inner object

    def price(self, order_id):           # the one we actually change
        ...
```

`__getattr__` is called only for attributes not found normally, so everything you did not override forwards automatically. It is convenient and it costs you static type checking — the type checker cannot see those methods. Mention both halves.

Real products, and you have used all of them

- `java.io` is the textbook case: `new BufferedReader(new InputStreamReader(new FileInputStream(f)))`. Three decorators, three jobs, one `Reader` interface. It is also the standard example of the pattern's cost, because that line is genuinely hard to read.
- `functools.lru_cache`, `functools.cache` — the function-level version, and the one you will cite first.
- **WSGI / ASGI middleware, Django middleware, Express middleware.** A middleware is a decorator over a request handler, and the ordering rules in this lesson are exactly the ordering rules those frameworks document. Django's `MIDDLEWARE` list is a composition order, and putting the authentication middleware in the wrong place is a real, documented security mistake.
- **gRPC interceptors and Envoy filters** — the same idea in a service mesh.
- `@property`, `@staticmethod`, `@classmethod` — descriptors implemented via the syntax.
- **Retry, circuit breaker and timeout wrappers** — Polly in .NET, Resilience4j in Java, `tenacity` in Python. All decorators, all with the ordering question this lesson describes.
- `contextlib.contextmanager` — a decorator producing an object with a different protocol.

Decorator versus its neighbours, again

From [day 068](#), refined now that you have seen both:

- **Adapter** changes the interface. It cannot stack, because after adapting, the shape is different.
- **Decorator** keeps the interface. It stacks *because* it keeps the interface.
- **Proxy** also keeps the interface, and the difference is intent: a proxy controls *access* — lazy loading, permission, remoting — while a decorator adds *behaviour*. On a class diagram they are identical, and honest answers say so.
- **Facade** changes how many things you talk to.

The most useful distinguishing question is: **can I apply two of these at once and does that mean something?** If yes, it is a decorator.

6 The numbers

The subclass explosion

features	subclasses (inheritance)	classes (decorator)
1	2	1
2	4	2
3	8	3
4	16	4
7	128	7

2^n against n . And the decorator side gets a second benefit that is easy to miss: adding an eighth feature is **one new class and zero edits**, while the inheritance side is 128 new classes.

At three features — logging, caching, retry — that is 8 classes of roughly 30 lines each, so about 240 lines, against 3 classes and one composition line, about 100. And the 8 classes contain duplicated logic, because `LoggedCachedService` and `LoggedRetryingService` both re-implement logging.

What each layer costs at run time

A Python method call through a decorator is roughly 80-150 nanoseconds of overhead.

```
4 layers x 120 ns = ~0.5 microseconds per call
the real work (a database query) = ~2,000 microseconds
overhead as a fraction = 0.025%
```

So for anything doing real I/O, the cost is nothing and you should say so plainly. For a function called ten million times in a tight loop it is 5 seconds, and there the pattern is the wrong tool. **Name the condition, not just the number.**

What the cache decorator buys, which is the point

```
without cache: 4,000 calls x 2 ms database round trip = 8.0 s per request batch
with cache at 85% hit rate:
    600 misses x 2 ms + 3,400 hits x 0.001 ms = 1.2 s
improvement: ~6.7x
```

And the ordering consequence, in the same numbers: with the logger **inside** the cache, your logs record 600 lines and your dashboard reports 600 pricing calls when the application made 4,000. That is not a small reporting error; it is the kind that makes a capacity plan wrong.

The debugging cost

The honest one:

```
stack trace depth, undecorated: 3 frames
stack trace depth, 4 decorators: 11 frames, 8 of which are `wrapper` or `price`
```

Every exception now travels through four layers of near-identical frames. This is the real reason people dislike `java.io`, and it is worth conceding.

7 The trade-offs

What you give up

Readability of the composition line. `Auth(Cached(Logged(Retrying(Real()))))` is four decisions in one expression, read inside-out. It is genuinely harder than a single class, and pretending otherwise is why people distrust patterns.

Debuggability. Deeper stack traces, mostly forwarding frames. Breakpoints in the wrong layer. `repr()` showing you a wrapper instead of the thing.

Identity. `isinstance(service, RealPriceService)` is `False` once wrapped, and so is `service is real`. Any code that compares identity or checks concrete types breaks. In Python, `__name__` and `__doc__` are lost without `functools.wraps`. In Java, `equals` on a wrapped object is a well-known hazard.

Ordering bugs, which are silent. The FRAGILE sticker on the inner box. Nothing raises, nothing logs, and the layer simply does not do what you thought. Auth inside a cache is the version of this that is a security incident.

The forwarding burden. Fifteen methods, fourteen of them pass-through. `__getattr__` solves it in Python at the cost of type checking; in Java it is fifteen real methods.

“I would not use this if...”

- **...there is only one feature and it will never be optional.** Put it in the class. A decorator to add logging to a service that will always log is a file for nothing.

- **...the features are not independent.** Decorator assumes layers compose freely. If caching must know about retries, they are not separate concerns and stacking them will produce something subtly wrong.
- **...the interface is wide and unstable.** Fifteen methods that change monthly means every decorator changes monthly too.
- **...the call is in a genuinely hot path.** Ten million calls a second, and four layers is five seconds of pure overhead.
- **...the language gives me it for free at the right granularity.** For a pure function with fixed behaviour, `@lru_cache` is one line and an object decorator is thirty.

The strongest thing you can say

Concede that decorators move complexity rather than removing it. The eight-subclass explosion becomes three classes and **one composition line that is now load-bearing** — and that line is not covered by any test unless you write one for it. Teams that use decorators heavily eventually write a test that asserts the composition order, precisely because getting it wrong is silent.

8 In the interview

How it gets asked

- The direct one: *“Add logging and caching to this service without changing its code.”*
- The comparison: *“What is the difference between decorator and inheritance?”* — where the expected answer includes the 2^n arithmetic.
- The Python one: *“Is Python’s `@decorator` the same as the decorator pattern?”* A precision test.
- The distinction: *“Decorator or proxy?”* — same structure, different intent.
- The sharp one, and the best: *“Does the order of your decorators matter?”*

What to say out loud, in the first ninety seconds

1. **Say the defining property first.** “A decorator has the same interface as the thing it wraps and forwards to it. Because the interface is unchanged, decorators stack — and the stacking is the whole point.”
2. **Kill inheritance with arithmetic.** “The alternative is subclassing, and that needs one class per combination — three features is eight subclasses, seven is a hundred and twenty-eight. Decorators are one class per feature.”
3. **Write the composition line and read it out.** `Cached(Logged(Real()))`, and say which is outermost.
4. **Volunteer the ordering point before being asked.** “Order matters, because each layer only sees what gets past the one outside it. With the cache outside the logger, a cache hit is never logged, so my dashboard reports 600 calls when the app made 4,000.”

5. Make the Python distinction. “Python’s `@` syntax is the function-level version of the same idea. It is fixed at import time, so if I want to cache in production and not in tests, I want the object version.”

The follow-ups

“Does the order matter? Give me an example.” “Yes, and it is the most useful thing about the pattern. Cache outside logger means cache hits are never logged, so the logs undercount. Retry outside timeout gives each attempt its own budget; timeout outside retry gives all attempts one shared budget. And authorisation should always be outermost — a cache in front of an auth check will serve a cached answer to someone who is not allowed to see it, which turns a performance decision into a security bug.”

“Is Python’s `@decorator` the decorator pattern?” “Same idea, different granularity. The syntax wraps a function and rebinds the name at definition time. The pattern wraps an object and preserves an interface, which means the composition is a run-time decision — I can wrap conditionally, per instance, per tenant, or not at all in tests. So for fixed behaviour on a pure function I would use `@lru_cache`; for anything I want to configure, I would use objects. And with the syntax I would always use `functools.wraps`, or the function’s name and docstring are lost and anything that introspects it breaks.”

“What is the difference between decorator and proxy?” “Structurally nothing — both keep the interface and hold the inner object. The difference is intent. A decorator adds behaviour and expects to be stacked. A proxy controls access — lazy loading, permission checks, talking to a remote object — and there is usually exactly one. The test I use is: would applying two of these at once mean something? If yes, decorator.”

“What breaks when you wrap something?” “Identity and type checks. `isinstance` against the concrete class is false, object identity is false, and in Python `__name__` and `__doc__` are gone without `wraps`. Stack traces get deeper — four layers turns three frames into about eleven, most of them forwarding. And if the interface has fifteen methods, a decorator has to forward fourteen it does not care about; in Python I would use `__getattr__` for that, at the cost of static type checking.”

“When would you not use it?” “When there is one feature and it is never optional — put it in the class. When the features are not actually independent, because stacking assumes they compose freely. And in a genuinely hot path: each layer is roughly a hundred nanoseconds, which is nothing next to a two-millisecond database call and is five seconds across ten million calls.”

A model answer

Asked: *add logging and caching to this service without changing its code.*

“I would write one decorator per concern. A decorator implements the same interface as the service, holds an instance of it, forwards every call, and does its own thing on the way through. Because the interface is unchanged, the caller cannot tell, and — the important part — a decorator can wrap another decorator.

So: a `PriceService` protocol with the one method. `RealPriceService` does the work. `LoggedPriceService` logs, calls the inner one, logs the result. `CachedPriceService` checks a dictionary, calls the inner one on a miss, stores. Then at the composition root I write `CachedPriceService(LoggedPriceService(RealPriceService()))`. Nothing existing was edited.

The reason I would not subclass is arithmetic. Subclassing needs one class per *combination* — logged, cached, logged-and-cached, plus the plain one, so four for two features and eight for three. Seven features would be a hundred and twenty-eight classes with duplicated logic in them. Decorators are one class per feature and the combination is a line of composition.

Now the thing I would raise before you ask, because it is the part that actually bites. **The order matters**, because each layer only sees what gets past the layer outside it. In what I just wrote, the cache is outside the logger, so a cache hit never reaches the logger — my logs would record only the misses. At an 85% hit rate the dashboard would say 600 pricing calls when the application made 4,000, which will make somebody’s capacity estimate wrong. If I want every call logged, the logger goes outside the cache. Neither is wrong; I just have to know which one I built.

The same reasoning settles the others. Retry outside timeout means each attempt gets its own budget; the other way round, all attempts share one. And authorisation goes outermost always, because a cache in front of an auth check will serve a cached result to somebody who should not see it — that is a security bug wearing a performance decision’s clothes.

Since this is Python, I should distinguish two things. `@lru_cache` is the function-level version of the same idea, and it is one line. What it cannot do is vary at run time — it is baked in when the module is imported. The object version lets me write `if settings.cache_enabled:` in the composition, cache in production and not in tests, or wrap only the instance serving one tenant. If I did use the syntax I would always use `functools.wraps`, otherwise the wrapped function’s name and docstring are lost.

What I am giving up: the composition line is read inside-out and is genuinely harder than one class; stack traces get about eight frames deeper, mostly forwarding; and `isinstance` against the concrete class stops being true. The one I would actually guard against is the ordering, because it fails silently — so on a team doing this heavily I would write a test that asserts the composition order.”

9 Recall card

- **Same interface, wraps one object, forwards, adds behaviour — and *therefore* stacks.** The unchanged interface is what makes composition possible; that is the whole pattern.
- **Kill inheritance with arithmetic:** 2^n subclasses vs n decorators. 3 features = 8 subclasses (with duplicated logic) vs 3 classes and one composition line; 7 features = **128 vs 7**. And an eighth feature is 1 new class and 0 edits.
- **Order matters, because a layer only sees what gets past the layer outside it.** Cache *outside* logger $\Rightarrow$ hits are never logged (dashboard says 600 when the app made 4,000). Retry *outside* timeout $\Rightarrow$ a budget *per attempt*; timeout *outside* retry $\Rightarrow$ one *shared* budget. **Auth is always outermost** — a cache in front of an auth check serves cached data to someone not allowed to see it.
- **Python's `@decorator` is the function-level version, not the pattern.** It is fixed at import time; the object version is a **run-time** decision (`if settings.cache_enabled:`, per instance, per tenant, off in tests). Always `functools.wraps`, or `__name__` and `__doc__` are lost and introspection breaks. `__getattr__` forwards the 14 methods you do not care about, at the cost of type checking.
- **Adapter changes the interface (cannot stack) · decorator keeps it and adds behaviour (stacks) · proxy keeps it and controls access (usually one) · facade changes how many things you talk to.** Costs to concede: ~ 11 stack frames instead of 3 · `isinstance` and identity break · ~ 120 ns per layer (nothing against a 2 ms query, 5 s across 10M calls) · and the composition line becomes load-bearing, so **test the order**.

DSA topic: Balanced brackets and the parsing family **System design topic:** Decorator

Code these, in this order

One rule for the whole set: **name the three failure modes out loud before writing any of them.** Wrong type on top, closer with nothing open, something still open at the end. Every problem below fails in one of those three ways, and saying them first is what stops you discovering them from a failing test.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Valid Parentheses	LeetCode 20 (Easy)	All three failures, and whether you map closer→opener or the wrong way round.
2	Minimum Add to Make Parentheses Valid	LeetCode 921 (Medium)	Two counters instead of a stack — whether you noticed the stack was not needed.
3	Evaluate Reverse Polish Notation	LeetCode 150 (Medium)	Operand order, which only <code>-</code> and <code>/</code> expose.
4	Decode String	LeetCode 394 (Medium)	Pushing a <i>context</i> , multi-digit counts, and which side of the concatenation goes where.

On problem 1, break it three ways on purpose

Write it without the emptiness check and run `"]"`. Write it without the end-of-string check and run `"(((("`. Write it with the map keyed by opener and see how awkward the lookup becomes. Then fix all three and say which one you would have shipped.

On problem 2, notice the stack disappeared

You do not need one. Two integers do it in $O(1)$ space, because with a single bracket type you never need to remember *which* bracket is open. Say out loud what would have to change for the stack to come back.

On problem 3, find the operand-order bug

Write `left = stack.pop()` first, then `right`. Test with `["2", "1", "+", "3", "*"]` — it passes. Then test `["4", "13", "5", "/", "+"]` — it fails. Say why `+` and `*` hid it.

On problem 4, test `"12[a]"` before anything else

If you get `"aa"` instead of twelve a's, your digit accumulation is `int(character)` instead of `number * 10 + int(character)`. Then test `"a2[b]"` to check the concatenation side.

The three-failures drill

For each input, say which of the three failure modes it triggers, or that it is valid:

1. `"()"`
2. `"([)]"`
3. `"(())"`
4. `"((()"`
5. `""`
6. `"]"`
7. `"({[()]}"`
8. `")(("`
9. `"(((("`
10. `"{[()]}"`

Then say which single input you would use to test each of the three checks, and why one-character inputs are the most valuable tests here.

The counting drill

1. Write the counting version for one bracket type. What is its space complexity?
2. Find an input with **three** bracket types where the counts all match and the string is invalid.
3. Would three separate counters fix it? Give the input that defeats them too.
4. State in one sentence what counting throws away.
5. Say when you would offer the counting version in an interview.

The context drill

For each problem, say what gets pushed on the opener and what happens on the closer:

1. Valid parentheses.
2. Decode String.
3. Basic Calculator (`"1 + (2 - (3 + 4))"`).
4. Reverse Polish notation.
5. A compiler reporting “unmatched `{` opened at line 41”.
6. Validating XML with named tags.

Number 5 pushes one thing more than number 1 does. What, and what does it buy the user?

The complexity drill

1. What is the time complexity of `is_balanced`? Count the loop.
2. What is its worst-case space, and what input causes it?
3. What is its best-case space, and what input causes it?
4. What is the time complexity of `decode_string`? Be careful — say it in terms of the right thing.
5. Give an input to `decode_string` that is 13 characters long and produces 1,000 characters.

6. Say the one sentence that explains why a `pop` inside a `for` loop is still linear.

The decorator-versus-inheritance drill

You have a `PriceService` and want logging, caching, retry and timing.

1. How many subclasses would inheritance need to cover every combination?
2. How many classes does the decorator approach need?
3. Write out three of the subclass names to see the problem.
4. Which logic would be duplicated across the subclasses?
5. How many classes and edits does a fifth feature cost in each approach?
6. Write the composition line for `logged + cached + timed`, without `retry`.

The ordering drill

For each pair, say what each ordering does differently and which you would choose:

1. `Cached(Logged(real))` versus `Logged(Cached(real))`
2. `Retry(Timeout(real))` versus `Timeout(Retry(real))`
3. `Auth(Cached(real))` versus `Cached(Auth(real))`
4. `RateLimit(Retry(real))` versus `Retry(RateLimit(real))`
5. `Metrics(Cached(real))` versus `Cached(Metrics(real))`

One of the five is not a preference — it is a security bug. Name it and say exactly what goes wrong.

The measurement drill

Build a real four-layer stack and measure it:

1. Time a bare method call.
2. Time it through one decorator.
3. Time it through four.
4. Compute the per-layer overhead.
5. Now put a 2 ms sleep in the real method and recompute the overhead as a percentage.
6. Say the condition under which the overhead matters, with a number.

The Python-syntax drill

1. Write a `@timed` decorator without `functools.wraps`. Print the decorated function's `__name__` and `__doc__`.
2. Add `@wraps` and print them again.
3. Name two real tools that break when `__name__` is wrong.
4. Say the one thing the `@` syntax cannot do that an object decorator can.
5. Write a composition that caches in production and does not cache in tests. Could you do it with `@lru_cache`?

The forwarding drill

You have an interface with 15 methods and a decorator that only changes one.

1. Write it the explicit way. How many forwarding methods?
2. Write it with `__getattr__`.
3. What do you lose by using `__getattr__`?
4. What happens when a sixteenth method is added, in each version?
5. Say which you would choose for a library, and which for application code.

The four-wrappers drill, again

Now that you have seen both, sort these into adapter, decorator, facade or proxy, and say the distinguishing question you used:

1. `BufferedReader(new InputStreamReader(new FileInputStream(f)))`
2. `psycopg` presenting the DB-API
3. Django's `MIDDLEWARE` list
4. An ORM's lazy-loading object
5. `requests.get(url)`
6. `functools.lru_cache`
7. nginx in front of your app
8. A class adding retry around a service, same interface

Two of the eight have identical structure and different intent. Name them and give the test that tells them apart.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Check whether the brackets in this string are balanced.* The three failure modes named before any code, the invariant, why counting fails with the `"([)]"` example, the clarifying question about other characters, and both space cases.
 2. *Add logging and caching to this service without changing its code.* The defining property, the `2^n` versus `n` arithmetic, the composition line read out loud, the ordering point volunteered with its numbers, and the Python-syntax distinction.
 3. *Does the order of your decorators matter?* Yes, with the cache-and-logger example and the 600-versus-4,000 number, retry versus timeout, and the auth case that is a security bug rather than a preference.
-

Before you move on

- `[]` I can name the three bracket failure modes from memory, in order.

- [] I broke the code three separate ways and know which input exposes each.
- [] I can give an input where the counts match and the string is invalid.
- [] I wrote the O(1)-space version and can say exactly why it only works for one bracket type.
- [] I found the RPN operand-order bug myself, using an input where `-` or `/` exposes it.
- [] I tested `"12[a]"` and `"a2[b]"` on Decode String.
- [] I can state Decode String's complexity in terms of the output, and give the 13-in/1000-out input.
- [] I can quote the `2^n` versus `n` arithmetic for decorators.
- [] I can give three concrete ordering pairs and say what changes in each.
- [] I know which decorator ordering is a security bug and why.
- [] I measured the per-layer overhead myself and can say when it matters.
- [] I answered all three questions above out loud.

Min stack, and stacks that carry extra state

DATA STRUCTURES AND ALGORITHMS

Min stack, and stacks that carry extra state

You can get the minimum of a stack in $O(1)$ and explain the two standard tricks.

SYSTEM DESIGN

Facade and proxy

You can hide a messy subsystem behind one clean entry point.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Design a stack that returns its minimum in constant time.*
- *Your checkout touches six services. How do you expose it to the client?*

Min stack, and stacks that carry extra state

1 What this is, and why they ask it

Build a stack with the usual `push`, `pop` and `top`, plus a fourth operation `get_min` that returns the smallest element currently in it — and all four must be $O(1)$. The obvious implementation of `get_min` scans the stack, which is $O(n)$. The question is how to remove that scan.

The answer is one idea, and it is worth more than the problem: **a stack can carry extra state computed at push time, because the LIFO discipline means popping automatically restores whatever was true before.** You do not have to recompute the minimum on a pop; the answer that was correct before that element arrived is sitting one level down, exactly where you left it. Nothing else in the phase has this property — you could not do it with a queue, or an array you delete from the middle of.

They ask it because it is a design question disguised as a data-structure question, it has two genuinely different standard answers, and it has a duplicate-handling bug that catches most people the first time. It is also one of the few problems where an interviewer can reasonably ask “now do it in $O(1)$ extra space” and there is a real, clever answer — which they will not expect you to derive, but will be pleased if you know exists.

2 The story

Nirmala’s hardware shop in Hubli has a storeroom with one narrow wall where the boxes go, stacked, because there is no room to put them side by side.

The boxes are all different weights. Hinges are heavy, switches are light, and a box of PVC fittings weighs almost nothing. The rule is simply that the last one in is the first one out, because there is nowhere else for it to go.

The thing she needs to know several times a day is which is the lightest box in the pile. There is a boy with a cycle who does the local deliveries, and he can carry one box if it is light enough.

For a long time this meant a stool and going through the pile, which on a bad day is fourteen boxes.

Her nephew fixed it with a piece of chalk, and it took him about two minutes to explain.

When a box goes on the pile, before she lets go of it, she writes one number on its side. Not its own weight — she writes **the weight of the lightest box in the pile now that this one is on it.** So if the lightest so far was four kilos and the new box is seven, she writes four. If the new box is two, she writes two.

Now, whenever she wants the answer, she reads the chalk mark on the top box. That is it. One glance, whatever the pile is.

And the part that makes it work is what happens when a box comes off. She works nothing out. The box underneath already has the right number chalked on it, because that number was written when the pile was exactly what it is about to become again. The answer for the smaller pile was recorded when the pile was that size, and it has sat there ever since.

There was one problem in the first week, because he had been slightly too clever. He had told her to only chalk a box when it was lighter than the current lightest. Which is fine until two boxes weigh the same. Two boxes of three kilos went on, only the first got a mark, and when the second came off she was reading a mark that no longer applied to anything.

Now she chalks every single box, and the pile is never wrong.

3 The idea in plain English

The chalk mark is **extra state carried per element**, and the whole trick is *when* it is computed: **at push time, never at pop time**.

Read the property that makes it work, because it is the sentence to say in an interview:

The minimum of a stack depends only on what is in it. A stack only ever changes at the top. So the minimum that was correct before you pushed is exactly the minimum that becomes correct again when you pop — and it is already recorded one level down.

That is why `pop` does no work. It is not that you recompute the minimum quickly; it is that you never recompute it at all.

Approach one — store pairs

Each entry on the stack is not a value but a pair: the value, and the minimum of everything from the bottom up to and including it.

```
self._stack: list[tuple[int, int]] = []

def push(self, value: int) → None:
    current_min = value if not self._stack else min(value, self._stack[-1][1])
    self._stack.append((value, current_min))

def get_min(self) → int:
    return self._stack[-1][1]
```

Four lines. `get_min` reads the top and is obviously $O(1)$. `pop` is `self._stack.pop()` unchanged, because the pair below already carries the right answer.

This is Nirmala's chalk mark, exactly. It is the version to write first: it is the shortest, it is the hardest to get wrong, and its correctness argument is one sentence.

Approach two — a second stack

Keep the values in one stack and the running minimums in another.

```
self._values: list[int] = []
self._mins: list[int] = []

def push(self, value: int) → None:
    self._values.append(value)
    self._mins.append(value if not self._mins else min(value, self._mins[-1]))
```

Same information, laid out differently. `pop` pops both; `get_min` reads `self._mins[-1]`.

The reason this version exists is that it invites an optimisation, and the optimisation is where the bug lives.

The optimisation, and the bug

If the new value is not smaller than the current minimum, why record anything? Only push onto the min stack when the value is a new minimum:

```
if not self._mins or value < self._mins[-1]:    # BUG
    self._mins.append(value)
```

and on pop, only pop the min stack if the value leaving is the current minimum:

```
value = self._values.pop()
if value == self._mins[-1]:
    self._mins.pop()
```

This saves memory when the values are mostly increasing. And it is wrong, for exactly Nirmala's reason.

```
push 3    values [3]    mins [3]
push 3    values [3,3]  mins [3]    ← second 3 not recorded (3 < 3 is False)
pop       values [3]    mins [ ]     ← 3 == 3, so the min stack is emptied
get_min   → IndexError, or a wrong answer
```

There is still a 3 in the stack and no record of it. The fix is one character:

```
if not self._mins or value ≤ self._mins[-1]:    # ≤ not <
```

With `≤`, every duplicate minimum gets its own entry, so every pop that removes a minimum removes exactly one record. **This is the trap of the problem**, and volunteering it before the interviewer constructs `[3, 3]` is one of the best things you can do in this question.

Which one to write

Write the pairs version. Then say the second-stack version exists, and that its optimised form has the duplicate bug and needs `≤`. That sequence — simplest first, then the variant, then the trap — is a better answer than writing the clever one and getting it right.

The O(1) extra space version, which you should know about and not write

There is a way to keep only a single `min` variable and no per-element state. Instead of storing the value, store an encoded number when a new minimum arrives:

```
def push(self, value: int) → None:
    if not self._stack:
        self._stack.append(value)
        self._min = value
    elif value ≥ self._min:
        self._stack.append(value)
    else:
        self._stack.append(2 * value - self._min)    # encoded, and < value
        self._min = value
```

On pop, if the stored number is less than `self._min`, it is an encoded marker, and the previous minimum is recovered as `2 * self._min - stored`.

It works. It is also arithmetic that can overflow in a fixed-width language, it makes `top()` need its own special case, and it takes five minutes to explain. **Mention it, do not write it**, unless the interviewer explicitly asks for O(1) extra space — and then say the overflow caveat as you go.

The general idea, which is the transferable part

The min stack is one instance of a family:

YOU WANT	WHAT EACH ELEMENT CARRIES
minimum in O(1)	the min from the bottom to here
maximum in O(1)	the max from the bottom to here
running sum	the sum from the bottom to here
average	sum and count
“the largest so far in this bracket”	whatever the aggregate is

The condition is the one that makes it work: **the aggregate must be computable from the new element and the previous aggregate alone**. Minimum, maximum, sum, count and product all satisfy that. Median does not — you cannot get the median of a set from the new element and the old median — and that is why “design a stack with O(1) median” is a genuinely harder problem needing two heaps.

Saying *why* it works, and naming the one that does not, is the difference between having memorised this problem and understanding it.

4 The picture

The pairs version, filling and emptying. Each cell shows `value | min-up-to-here`.

push 5	push 3	push 7	push 3	pop	pop
			+-----+		
+-----+	+-----+	+-----+	3 3	+-----+	+-----+
5 5	3 3	7 3	7 3	7 3	3 3
+-----+	+-----+	+-----+	3 3	3 3	5 5
5 5	5 5	5 5	5 5	5 5	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
min = 5	min = 3	min = 3	min = 3	min = 3	min = 3

What to notice: `get_min` always reads the second number in the top cell, and `pop` never computes anything. The two pops at the end restore `min = 3` for free, because the cell underneath was written when the pile was that shape.

	push 3	push 3	pop	get_min
values	[3]	[3, 3]	[3]	[3]
mins	[3]	[3]	[]	[]
		^^^^^^	^^^^	^^^^^
		3 < 3 is False, so nothing recorded	3 = 3, so the record is removed	a 3 is still in the stack with no record

And with `<=`:

	push 3	push 3	pop	get_min
values	[3]	[3, 3]	[3]	[3]
mins	[3]	[3, 3]	[3]	→ 3 correct

One character. Two entries instead of one. That is the whole fix.

5 The code, built step by step

Step 1 — the naive version, so the problem is visible

```
class SlowMinStack:
    def __init__(self) → None:
        self._stack: list[int] = []

    def push(self, value: int) → None:
        self._stack.append(value)

    def get_min(self) → int:
        return min(self._stack)          # 0(n) – this is the thing to remove
```

Write this and say the complexity out loud: “push and pop are $O(1)$, `get_min` is $O(n)$ because it reads everything. The question is how to make that constant.”

Step 2 — the observation

Say it before you write the fix, because the code is obvious once the sentence is said.

“The minimum only changes at the top. When I push, the new minimum is the smaller of the new value and the old minimum — one comparison. When I pop, the minimum goes back to whatever it was before that push. So if I record the answer at push time, popping restores it for free.”

Step 3 — the pairs version

```
def push(self, value: int) → None:
    current_min = value if not self._stack else min(value, self._stack[-1][1])
    self._stack.append((value, current_min))
```

The `if not self._stack` handles the first push, where there is no previous minimum to compare against. `self._stack[-1][1]` is the min recorded on the current top.

```
def get_min(self) → int:
    return self._stack[-1][1]
```

One indexing operation. No loop, no comparison, no state to maintain.

Step 4 — the guards

Every method that touches the top must handle the empty case, and the message should mention a stack:

```
def pop(self) → int:
    if not self._stack:
        raise IndexError("pop from empty stack")
    return self._stack.pop()[0]      # [0] — return the value, not the pair
```

The `[0]` matters. `self._stack.pop()` gives back the tuple, and returning the tuple to a caller who pushed an integer is a bug that type checking would catch and a quick interview would not.

The complete solution

```
class MinStack:
    """A stack whose minimum is available in O(1).

    Each entry carries the minimum of everything from the bottom up to and
    including itself, computed at push time. Because a stack only changes at
    the top, popping restores the previous minimum with no work at all.

    All four operations are O(1). Space is O(n) – two integers per element.
    """

    def __init__(self) → None:
        self._stack: list[tuple[int, int]] = []      # (value, min so far)

    def push(self, value: int) → None:
        current_min = value if not self._stack else min(value, self._stack[-1][1])
        self._stack.append((value, current_min))

    def pop(self) → int:
        if not self._stack:
            raise IndexError("pop from empty stack")
        return self._stack.pop()[0]                # the value, not the pair

    def top(self) → int:
        if not self._stack:
            raise IndexError("top of empty stack")
        return self._stack[-1][0]

    def get_min(self) → int:
        if not self._stack:
            raise IndexError("min of empty stack")
        return self._stack[-1][1]

    def __len__(self) → int:
        return len(self._stack)


class TwoStackMinStack:
    """The other standard answer: values in one stack, minimums in another.

    The `≤` is load-bearing. With `<`, two equal minimums share one record,
    and popping one of them removes the record for both.
    """

    def __init__(self) → None:
        self._values: list[int] = []
        self._mins: list[int] = []

    def push(self, value: int) → None:
        self._values.append(value)
        if not self._mins or value ≤ self._mins[-1]:      # ≤ NOT <
            self._mins.append(value)

    def pop(self) → int:
        if not self._values:
            raise IndexError("pop from empty stack")
        value = self._values.pop()
        if self._mins and value == self._mins[-1]:
            self._mins.pop()
        return value

    def top(self) → int:
        return self._values[-1]

    def get_min(self) → int:
```

```

        if not self._mins:
            raise IndexError("min of empty stack")
        return self._mins[-1]

class MaxStack:
    """Same idea, one comparison flipped – the transferable part."""

    def __init__(self) → None:
        self._stack: list[tuple[int, int]] = []

    def push(self, value: int) → None:
        current_max = value if not self._stack else max(value, self._stack[-1][1])
        self._stack.append((value, current_max))

    def pop(self) → int:
        return self._stack.pop()[0]

    def get_max(self) → int:
        return self._stack[-1][1]

class SumStack:
    """The aggregate does not have to be an extreme. Sum works because the new
    sum is computable from the new value and the previous sum alone."""

    def __init__(self) → None:
        self._stack: list[tuple[int, int]] = []

    def push(self, value: int) → None:
        previous = 0 if not self._stack else self._stack[-1][1]
        self._stack.append((value, previous + value))

    def pop(self) → int:
        return self._stack.pop()[0]

    def total(self) → int:
        return 0 if not self._stack else self._stack[-1][1]

if __name__ == "__main__":
    stack = MinStack()
    for value in (5, 3, 7, 3):
        stack.push(value)
    print(stack.get_min(), stack.top())          # 3 3
    print(stack.pop(), stack.get_min())          # 3 3
    print(stack.pop(), stack.get_min())          # 7 3
    print(stack.pop(), stack.get_min())          # 3 5

    # The duplicate case, on the version that has the trap.
    two = TwoStackMinStack()
    two.push(3); two.push(3)
    print(two.pop(), two.get_min())              # 3 3 – correct because of ≤

    max_stack = MaxStack()
    for value in (2, 9, 4):
        max_stack.push(value)
    print(max_stack.get_max())                   # 9

    sums = SumStack()
    for value in (5, 3, 7):
        sums.push(value)
    print(sums.total())                          # 15
    sums.pop()
    print(sums.total())                          # 8

```

Run the `[3, 3]` case on both implementations. On the pairs version it cannot go wrong, which is a real reason to prefer it.

6 What it costs

Time

push	1 comparison + 1 append	$O(1)$
pop	1 pop	$O(1)$
top	1 index	$O(1)$
get_min	1 index	$O(1)$

Say what changed and why: `get_min` went from $O(n)$ to $O(1)$ because the work moved to `push`, where it is one comparison instead of a scan. **You did not make anything faster; you moved the cost to a place where it is constant.** That sentence is worth saying.

Space

Pairs version: two integers per element, so $2n$ — $O(n)$, and precisely twice the naive stack.

Two-stack version, unoptimised: also $2n$, identical.

Two-stack version with the $\leq$ optimisation:

values increasing	<code>[1,2,3,...,n]</code>	mins holds 1 entry	total $\sim n$
values decreasing	<code>[n,...,3,2,1]</code>	mins holds n entries	total $2n$
all equal	<code>[3,3,3,...,3]</code>	mins holds n entries	total $2n$

Best case n , worst case $2n$. So the optimisation saves up to half the extra memory on friendly inputs and nothing on hostile ones. **On a decreasing sequence it saves nothing at all**, which is worth knowing before you claim it as an improvement.

The encoded version: one integer per element plus one variable — $O(n)$ total, $O(1)$ *extra*. That is the only version that is genuinely $O(1)$ extra space, and it pays for it in overflow risk and readability.

Compared with recomputing

At $n = 10,000$ with 10,000 `get_min` calls:

naive:	10,000 calls x 10,000 elements =	100,000,000 comparisons
$O(1)$:	10,000 calls x 1 =	10,000 reads

Ten thousand times less work, for the cost of one extra integer per element.

The condition for the whole technique

An aggregate can be carried this way when it is computable from `(new element, previous aggregate)` in $O(1)$.

min, max, sum, count, product	yes – one operation each
median	NO – needs the whole multiset
k-th smallest	NO
mode (most frequent)	NO – needs all counts

Median needs two heaps and $O(\log n)$ per operation. Naming that as the boundary is what shows you understand the technique rather than the problem.

7 The traps

Trap 1 – `<` instead of `<=` in the optimised two-stack version

The one this problem is famous for.

```
if not self._mins or value < self._mins[-1]:    # BUG
```

Input: `push(3)`, `push(3)`, `pop()`, `get_min()`.

```
IndexError: list index out of range
```

or, in a version with more elements underneath, a silently wrong minimum. The fix is `<=`, and the matching `pop` must use `=` so it removes exactly one record.

Trap 2 – returning the pair instead of the value

```
def pop(self) → int:
    return self._stack.pop()    # returns (7, 3), not 7
```

The caller pushed integers and gets tuples back. `top()` has the same trap. Both need the `[0]`.

Trap 3 – recomputing the minimum on pop

```
def pop(self) → int:
    value = self._stack.pop()
    self._min = min(self._stack) if self._stack else None    # O(n)
    return value
```

This is correct and it is $O(n)$ on every pop, which is exactly what you were asked to avoid. It is a natural thing to write if you have not said the observation out loud first, which is why saying it first matters.

Trap 4 – one `self._min` variable and nothing else

```
def push(self, value):
    self._min = min(self._min, value)    # correct
def pop(self):
    ...    # and now what?
```

Push is easy; pop is impossible. There is no way to recover the previous minimum from a single variable, because the information was overwritten. This is the honest reason the problem needs per-element state, and the encoded version is precisely a trick for smuggling that state into the values themselves.

Trap 5 — the empty stack

```
>>> MinStack().get_min()
IndexError: list index out of range
```

Every one of the four operations needs an empty guard. Interviewers ask “what does `get_min` return on an empty stack?” and the right answer is a question back: “should it raise, or return `None` ? I would raise, because there is no sensible minimum of nothing, and returning `None` makes every caller handle it.”

Trap 6 — assuming the optimisation always saves memory

“I only push when it is a new minimum, so it uses less space” — true on `[1,2,3, ...]` and false on `[n, ..., 2,1]`, where every element is a new minimum and the min stack is exactly as long as the values. Quote the worst case, not the best.

Trap 7 — overflow in the encoded version

`2 * value - self._min` can exceed the range of a fixed-width integer when the values are near the limits. Python integers are arbitrary precision so it is safe there, but in Java or C++ it needs `long` and a caveat. If you mention the trick, mention this in the same breath.

8 In the interview

How it gets asked

- “Design a stack that supports push, pop, top and retrieving the minimum element, all in constant time.” LeetCode 155, almost word for word.
- The immediate variant: “now the maximum” — one comparison flipped, and they are checking whether you understood or memorised.
- The space pivot: “can you do it with $O(1)$ extra space?” — the encoded trick.
- The boundary probe: “what about the median in $O(1)$?” — the answer is no, and why.
- The harder relative, asked at senior level: *Max Frequency Stack* (LeetCode 895), which is the same “carry extra state” idea with a map of stacks.

What to say out loud, in the first ninety seconds

1. **State the naive version and its cost.** “Push and pop are already $O(1)$. `get_min` scanning the stack is $O(n)$, so that is the thing to remove.”

2. **Say the observation before the code.** “A stack only changes at the top, and the minimum depends only on what is in it. So the minimum before a push is exactly the minimum after the matching pop — I can record it on the way in and never recompute it.”
3. **Write the pairs version.** Four lines, and say “each entry carries the minimum from the bottom up to itself.”
4. **Give the second approach and its trap, unprompted.** “The other standard answer is two stacks. There is an optimisation where you only push a new minimum, and it has a duplicates bug — with `<` instead of `≤`, two equal minimums share one record and popping one removes both. I would use `≤`.”
5. **State the cost.** “All four operations $O(1)$. Space is $2n$, which is twice the plain stack. If you want $O(1)$ extra space there is an encoding trick — I can describe it, and I would flag that it can overflow in a fixed-width language.”

The follow-ups

“**Why does pop not need to recompute the minimum?**” “Because the answer was already computed when the pile was that shape. The entry underneath the one I am removing carries the minimum of everything from the bottom up to itself, and once the top is gone, that is exactly the whole stack. The LIFO discipline is what guarantees it — I only ever remove the most recent thing, so I only ever return to a state I have already been in and recorded.”

“**Can you do it in $O(1)$ extra space?**” “Yes, with an encoding. When a new minimum arrives I push `2 * value - min` instead of the value, which is provably smaller than the new minimum, and I update `min`. On pop, anything smaller than the current minimum is a marker, and the previous minimum is `2 * min - stored`. It works and I would raise two concerns: it can overflow in Java or C++ so it needs a wider type, and `top()` now needs a special case. Unless space is genuinely the constraint, I would take the extra n integers.”

“**Now do maximum instead.**” “One comparison flipped — `max` instead of `min` at push time. Nothing else changes, which is the sign that the technique is about carrying an aggregate, not about minimums.”

“**What about the median in $O(1)$?**” “No, and the reason is the interesting part. This technique works only when the aggregate is computable from the new element and the previous aggregate alone. Min, max, sum, count and product all are. Median is not — the median of a set cannot be derived from the old median plus one new value. That needs two heaps, a max-heap of the lower half and a min-heap of the upper half, giving $O(\log n)$ per operation rather than $O(1)$.”

“**Does the two-stack optimisation actually save memory?**” “Sometimes. On an increasing sequence the min stack holds one entry, so it is about n total. On a decreasing sequence every element is a new minimum, so it holds n entries and saves nothing — same $2n$ as the pairs version. Best case n , worst case $2n$. I would not claim it as an improvement without saying which input I am assuming.”

A model answer

Asked: *design a stack that returns its minimum in constant time.*

“Push, pop and top are already constant on a normal stack. The one that is not is the minimum — scanning is $O(n)$ — so that is the operation to fix.

The observation that makes it work is about the shape of a stack rather than about minimums. A stack only ever changes at the top, and the minimum depends only on what is currently in it. So the minimum that was correct just before I pushed something is exactly the minimum that becomes correct again the moment I pop it. That means I do not have to recompute anything on the way out — I only have to record the answer on the way in.

So each entry on my stack is a pair: the value, and the minimum of everything from the bottom up to and including that value. On push I compare the new value with the minimum recorded on the current top, which is one comparison. On pop I just pop the pair, and the pair underneath already carries the right answer. `get_min` reads the second half of the top entry.

All four operations are $O(1)$. Space is two integers per element, so $2n$ — twice a plain stack. I have not made anything faster; I have moved the work from `get_min`, where it was a scan, to `push`, where it is one comparison.

There is a second standard answer worth mentioning: keep the values in one stack and the minimums in a parallel stack. Same information. The reason people prefer it is an optimisation — only push onto the min stack when the value is a new minimum — and that optimisation has the bug this question is really about. If the condition is strictly less-than, then pushing 3 and then 3 records only one minimum, and popping one of them deletes the record while a 3 is still in the stack. The fix is less-than-or-equal, so every duplicate gets its own record. I would also note that the optimisation saves nothing on a decreasing sequence, where every element is a new minimum.

If you want $O(1)$ extra space there is an encoding trick — push `2*value - min` when a new minimum arrives, and recover the previous minimum arithmetically on pop. It works; I would flag that it can overflow in a fixed-width language and that `top()` needs a special case.

And the general form is worth stating: this works for any aggregate computable from the new element and the previous aggregate in constant time. Min, max, sum, count, product — all fine. Median is not, because you cannot derive a median from the old median and one new value, and that is why the median version needs two heaps and $O(\log n)$.”

9 Recall card

- **The observation, not the code, is the answer:** a stack only changes at the top, so **the minimum before a push is exactly the minimum after the matching pop** — record it on the way in and never recompute it. `pop` does no work at all.
- **Write the pairs version:** each entry is `(value, min from the bottom to here)`; push does one comparison against `stack[-1][1]`; `get_min` is one index. Four operations, all $O(1)$; space $2n$. Remember the `[0]` in `pop` and `top`, or you hand the caller a tuple.
- **The two-stack variant's optimisation has the famous bug:** `<` must be `≤`. `push(3), push(3), pop(), get_min()` loses the record while a 3 is still in the stack. And the optimisation saves **nothing** on a decreasing sequence — best case n , worst case $2n$.
- **$O(1)$ extra space exists:** push `2*value - min` on a new minimum, recover with `2*min - stored`. Mention it, do not write it — it **overflows** in fixed-width languages and forces a special case in `top()`.
- **The transferable rule:** you can carry any aggregate computable from `(new element, previous aggregate)` in $O(1)$ — min, max, sum, count, product. **Median cannot** (needs two heaps, $O(\log n)$), and naming that boundary is what shows understanding rather than memorisation. At $n = 10,000$ with 10,000 queries this is 10^8 comparisons versus 10^4 reads.

1 What this is, and why they ask it

Two structural patterns, both of which put something in front of something else, for opposite reasons.

A **facade** is one simple entry point onto a complicated subsystem. The caller says `checkout(cart)` and does not learn that six services were involved or what order they had to be called in. The subsystem is unchanged and still directly reachable by anyone who needs it — a facade adds a door, it does not build a wall.

A **proxy** has **the same interface** as the thing behind it and controls access to it. Same method names, same signatures, and it decides whether, when, and how the real object is reached — lazily, remotely, after a permission check, or from a cache.

They ask this pair because it is the point where object-oriented patterns become system design. The checkout question is a facade question at class level and an API gateway question at service level, and they are the same shape. Interviewers also ask because the four wrapping patterns — adapter, decorator, facade, proxy — look identical on a class diagram, and the ability to separate them crisply is a reliable indicator. Everybody can say “it wraps something”. Very few can say what each one changes.

2 The story

The hospital on the main road in Hubli is not large — four consulting rooms, a lab at the back, a pharmacy counter and about sixty people through the door on a weekday morning.

Prema sits at the front desk.

A man came in one Tuesday with his mother, who had been sent by a doctor elsewhere for a blood test and a consultation. He had a slip of instructions and no idea what to do with it.

What has to happen for that visit is not one thing. Her file has to be found or a new one made. The consultation has to be booked with whichever doctor is free. The lab has to be told a sample is coming and given the right test codes. Billing has to raise an entry, because the lab will not draw blood without one. If she is on a scheme, the scheme has to be checked, and that is a phone call. And afterwards the pharmacy needs the prescription before it will dispense.

Six things, six different people, in a particular order — billing before the lab, always, because it has been the wrong way round before and it caused an argument.

He said none of that. He gave Prema the slip and said his mother had been sent for a blood test. She asked three questions, and then told him to sit down for ten minutes.

While he sat, she did all six. He never knew.

That is the front desk. There is one more thing worth watching in that building, and it happens outside room four.

Dr Kulkarni sees patients in room four, and outside the door sits Manjunath on a plastic chair. If you want the doctor, you go to that door — there is no other way in. Manjunath is not a smaller doctor and he does not answer medical questions. What he does is: check that you have an appointment, check that billing has been done, tell you the doctor is with someone and there are two ahead of you, and, four or five times a day, answer a question himself because he has heard the answer so often — yes, come back Thursday; no, do not eat before the test.

To the patient, going to room four is going to room four. What is different is that somebody decides whether and when you get in.

3 The idea in plain English

Prema is a **facade**. Manjunath is a **proxy**. They are doing different jobs and it is worth being precise about how.

A facade simplifies. Behind Prema there are six different people with six different interfaces. She gives you one. Note two things: she is **not** the only way to reach the lab — a doctor walks straight in — and she does not have the same interface as any of the six, because she has her own simpler one.

A proxy controls. Manjunath presents the same interface as the door itself — you go to the door either way. He does not simplify anything. He decides whether you get in, when you get in, and occasionally answers instead of the doctor, which is caching.

The four wrappers, finally separated

You have now met all four. This table is the answer to the question interviewers ask.

PATTERN	INTERFACE	NUMBER BEHIND IT	INTENT
Adapter	different	one	make an incompatible thing usable
Decorator	same	one	add behaviour, and stack it
Facade	new and simpler	many	reduce what the caller must know
Proxy	same	one	control access

Two questions separate all four:

1. **Is the interface the same as the thing behind it?** Same → decorator or proxy. Different → adapter or facade.

2. Then: how many things are behind it, and why is it there? Many and simpler → facade. One and incompatible → adapter. Adding behaviour and stackable → decorator. Controlling access → proxy.

Decorator and proxy are the genuinely hard pair, because structurally they are identical. The honest distinction is **intent plus multiplicity**: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

Facade, in code

```
class CheckoutFacade:
    def __init__(self, inventory, pricing, payments, orders,
                 notifications, shipping) → None:
        ...

    def checkout(self, cart: Cart, user: User) → Order:
        self._inventory.reserve(cart.items)           # 1
        total = self._pricing.price(cart, user.country) # 2
        payment = self._payments.charge(user, total)   # 3
        order = self._orders.create(cart, payment)     # 4
        self._shipping.schedule(order)                 # 5
        self._notifications.order_placed(order)        # 6
        return order
```

The caller writes `checkout(cart, user)`. What the facade removed is not the six services — they are all still there — it is **the caller's need to know the order and the failure handling**. Reserve before charging. Charge before creating the order. Notify last. Every caller that did this by hand was a chance to get the order wrong.

The rule that keeps a facade from becoming a god object

This is the part to volunteer, because it is the pattern's failure mode.

A facade may sequence and translate. It must not make business decisions.

The moment `checkout` contains `if user.country == "IN": total *= Decimal("1.18")`, the tax rule lives in the facade, and next month the discount rule joins it, and in a year it is a 900-line class that four teams edit — the divergent change from [day 061](#).

The other rule: **do not seal the subsystem**. A facade is a convenience, not a wall. Reporting still talks to `orders` directly. If you forbid that, every caller with a slightly different need has to widen the facade, and it grows one method at a time until it is the union of everything.

Proxy, and its four kinds

All four have the same interface as the real thing. What differs is what they do before forwarding.

Virtual proxy — defer expensive work. The classic is an ORM. `order.customer` looks like a `Customer` object and is actually a stand-in that fires a query the first moment you touch a field. This is what makes the $N + 1$ problem from [day 041](#) so easy to create: the proxy is invisible, so a loop over 100 orders quietly becomes 101 queries.

Remote proxy — the real object is elsewhere. A gRPC or RMI stub. You call a local method; it serialises, sends, waits, deserialises. The whole point is that the call site looks local — and the whole danger is that the call site looks local, because a local call cannot fail with a timeout and a remote one can.

Protection proxy — check permission first. Manjunath. Same interface, plus an authorisation check before forwarding.

Caching (or smart) proxy — answer without asking. Same interface, returns a stored answer when it can. Manjunath telling you not to eat before the test.

The system-level versions, which is where interviews go

The same two patterns, one layer up.

- **API gateway** — a facade over many services, for external clients. One endpoint, one authentication, one rate limit, and it fans out.
- **Backend-for-frontend (BFF)** — a facade per client type, because the mobile app and the web app need different shapes. This is the honest answer to “one facade or many”: one per audience.
- **Reverse proxy (nginx, Envoy, Cloudflare)** — a proxy: same HTTP interface, controlling TLS termination, routing, caching and rate limiting.
- **Service mesh sidecar** — a proxy per service instance, doing retries, timeouts, mutual TLS and telemetry.
- **CDN** — a geographically distributed caching proxy.

When an interviewer asks the checkout question, they usually want the class-level facade *and* the sentence “at the service level this is what an API gateway is”.

4 The picture

The two patterns side by side, with what the caller knows in each.

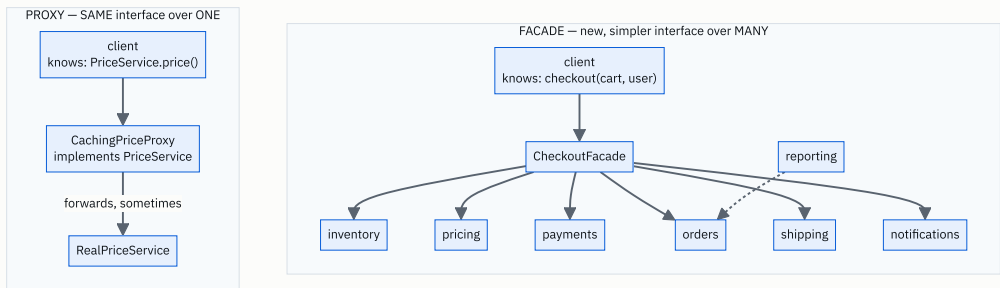


FIGURE 70.1

Two things to notice. In the facade, the dashed arrow from `reporting` goes straight to `orders` — the subsystem is **not sealed**, and that is deliberate. In the proxy, the client’s declared type is `PriceService`, the same type the real object has, which is what lets the proxy be substituted without anyone knowing.

And the checkout sequence, because the ordering is the thing the facade is really protecting:

```

caller: checkout(cart, user)
|
+-- 1. inventory.reserve(items)      ← must be first: fail fast if out of stock
|
+-- 2. pricing.price(cart, country)
|
+-- 3. payments.charge(user, total)  ← must be after 1: never charge for
|                                   something you cannot ship
+-- 4. orders.create(cart, payment)
|
+-- 5. shipping.schedule(order)
|
+-- 6. notifications.order_placed()  ← must be last: never email about an
|                                   order that then failed to save

```

The caption is the point: **each of those “must be” comments is a rule that used to live in nine callers and now lives in one.** That is the actual product a facade sells.

5 How it actually works

Writing a facade

- 1. Start from the caller’s sentence, not the subsystem.** “Place this cart as an order for this user.” One method, arguments in the caller’s vocabulary, returning the caller’s type.
- 2. Sequence and translate inside.** Call the six in the correct order; convert their types into yours.
- 3. Decide the failure policy in one place.** What happens if payment succeeds and order creation fails? That question has to be answered somewhere, and the facade is a much better somewhere than nine call sites.
- 4. Keep decisions out.** Sequencing yes, business rules no.

5. Leave the subsystem reachable.

The failure policy is the part that turns this from a tidiness exercise into engineering. A real checkout facade has a compensation path — refund on failure, or an outbox that retries the notification, or an idempotency key so a retry does not double-charge. Mentioning even one of those is worth a great deal.

Writing a proxy

Identical shape to a decorator, and in Python identical code:

```
class CachingPriceProxy:
    def __init__(self, inner: PriceService) → None:
        self._inner = inner
        self._cache: dict[str, Decimal] = {}

    def price(self, order_id: str) → Decimal:
        if order_id not in self._cache:
            self._cache[order_id] = self._inner.price(order_id)
        return self._cache[order_id]
```

If that looks exactly like the caching decorator from [day 069](#), that is because it is. **The pattern is in the intent, not the code.** Say so if asked — pretending there is a structural difference is worse than admitting there is not.

Real products, named

Facades:

- `requests.get(url)` — behind one line: connection pooling, DNS, TLS, redirects, cookies, retries, content decoding, across about six subsystems.
- `subprocess.run()` — a facade over `Popen`, pipes, and `communicate`.
- SLF4J, `logging.basicConfig()`.
- An API gateway: Kong, AWS API Gateway, Apigee.
- Stripe Checkout — one hosted page over cards, 3-D Secure, wallets and receipts.

Proxies:

- nginx, HAProxy, Envoy, Cloudflare — reverse proxies.
- Hibernate and Django ORM lazy objects — virtual proxies, and the source of $N + 1$.
- gRPC and Thrift stubs — remote proxies.
- Java dynamic proxies and Spring AOP — how `@Transactional` works: the object you are injected is a proxy that opens a transaction, forwards, and commits. This is also why calling a `@Transactional` method from *inside* the same class does nothing, because that call does not go through the proxy. That is a very good detail to know.
- `unittest.mock.Mock` — a proxy that records instead of forwarding.

The subtlety that makes proxies dangerous

A proxy is invisible by design, and that is its risk. Three specific consequences worth naming:

- **The ORM proxy** makes a database query look like a field access, which is why N+1 problems are written by careful people.
- **The remote proxy** makes a network call look like a method call, which is the first of the fallacies of distributed computing: the network is not reliable, and the call site does not say so.
- **The Spring proxy** means self-invocation bypasses the behaviour entirely, silently.

In each case the interface being identical is exactly what causes the problem. Say that: **the proxy's greatest strength and its main hazard are the same property.**

6 The numbers

The facade, at class level

callers of the checkout flow	9
services each must know, without a facade	6
total coupling edges	$9 \times 6 = 54$
with a facade	$9 \times 1 + 1 \times 6 = 15$

More than a threefold reduction in edges, and more importantly the ordering rule is written once instead of nine times. **Nine chances to charge before reserving becomes one.**

Adding a seventh service: without a facade, 9 files edited; with one, 1.

The facade, at service level — where the numbers get large

A mobile client assembling a home screen from six services itself:

6 round trips, mobile network RTT ~120 ms
sequential: $6 \times 120 \text{ ms} = 720 \text{ ms}$
parallel: ~120 ms, but 6 TLS handshakes and 6 auth checks
payload: 6 responses, ~40 KB total, much of it unused by this screen

Through a gateway or BFF:

1 round trip:	120 ms
the fan-out happens in the data centre, where RTT is ~1 ms	
$6 \times 1 \text{ ms}$ in parallel	= ~5 ms
total:	~125 ms
payload:	one response shaped for this screen, ~8 KB

720 ms to 125 ms, and 40 KB to 8 KB. On a mobile network that is the difference between a screen that feels instant and one that does not, and it is the standard justification for a BFF.

The proxy, priced

Caching proxy, at an 85% hit rate on a 2 ms backend call:

```
without: 10,000 calls x 2 ms      = 20 s of backend time
with:    1,500 misses x 2 ms     = 3 s          (~6.7x less load)
```

Virtual proxy, and its cost when it goes wrong — the N+1:

```
100 orders, each touching order.customer lazily
1 + 100 queries x 1.5 ms = 151 ms
with an eager join:      1 query x 4 ms = 4 ms
ratio: ~38x
```

That number is the price of the proxy being invisible.

Reverse proxy overhead: nginx adds roughly 0.1-1 ms per request. Against a 50 ms application response that is under 2%, and it buys TLS termination, connection reuse, rate limiting and static caching. Quote both halves.

What a facade costs

```
the facade class           ~40-80 lines
tests for the sequencing   ~6 cases
one more file in the call path 1 hop
```

Small. The real cost is not lines; it is the risk that it grows. A facade that starts at 60 lines and reaches 900 has become the god object it was supposed to prevent, and that happens one reasonable addition at a time.

7 The trade-offs

What a facade costs you

It can become a god object. The single biggest risk. Every new cross-cutting need is “just one more method on the facade”, and there is no natural point at which anybody says no. The defence is the rule: sequencing and translation only, never decisions — and a periodic look at `git log` on the file to see how many teams are editing it.

It hides capability. The subsystem can do forty things and the facade exposes six. A caller needing the seventh either widens the facade or bypasses it, and if bypassing is forbidden the facade grows without limit.

It can become the lowest common denominator. One facade for four different kinds of caller ends up serving none of them well. That is exactly why BFF exists: one facade per audience, not one facade.

One more hop to read through. Minor, and real.

What a proxy costs you

Invisibility. Already the main point: N+1 from ORM proxies, unexpected timeouts from remote proxies, silent no-ops from self-invocation through Spring proxies. The interface being identical is what causes all three.

Stale data. A caching proxy is a correctness decision wearing a performance costume. What is the invalidation rule? How stale may an answer be? If nobody has decided, the proxy has introduced a bug nobody has noticed yet.

Failure modes the interface cannot express. `price(order_id)` does not say “this may take four seconds and time out”. Two implementations of one interface behaving very differently is the Liskov issue from [day 057](#), arriving through a proxy.

“I would not use this if...”

- **...the subsystem is already one thing with a sensible interface.** A facade over one class is a middle man ([day 061](#)). Delete it.
- **...there are two callers and they need different things.** Two small facades beat one wide one.
- **...the facade would need business rules to do its job.** Then the design problem is upstream: a concept is missing, not a wrapper.
- **...the proxy's caching has no invalidation story.** Do not ship it.
- **...a lazy proxy is being used to avoid thinking about what to load.** Explicit eager loading is better than an invisible query.

The strongest thing to say about the pair

Concede that facade and proxy do not remove complexity — they **relocate** it, and put a name on where it now lives. Six services still have to be called in the right order; the facade means that order is in one file with a test, instead of in nine developers' heads. That is the honest claim, and it is a good one.

8 In the interview

How it gets asked

- The scenario: “*Your checkout touches six services. How do you expose it to the client?*”
- The pair: “*What is the difference between a facade and a proxy?*” — often extended to all four wrappers.
- The system-design version: “*Should the mobile app call the services directly or through a gateway?*” Same question, one layer up.
- The debugging one: “*Why does this loop make 101 database queries?*” — a virtual proxy question in disguise.

What to say out loud, in the first ninety seconds

1. **Separate the two immediately.** “A facade gives a *new, simpler* interface over *many* things. A proxy has the *same* interface as *one* thing and controls access to it. Different intents that look identical on a diagram.”
2. **Say what the facade actually removes.** “It is not that the six services disappear. It is that the caller stops needing to know the order — reserve before charging, notify last — and stops owning the failure policy.”
3. **Give the two rules unprompted.** “Sequencing and translation, never business decisions. And I would not seal the subsystem — reporting can still talk to orders directly.”
4. **Go one layer up without being asked.** “At the service level this is an API gateway, and if different clients need different shapes it is a backend-for-frontend.”
5. **Bring a number.** “For a mobile client, six round trips at 120 ms is 720 ms; one gateway call with the fan-out inside the data centre is about 125 ms.”

The follow-ups

“What is the difference between a proxy and a decorator?” “Structurally, nothing — both keep the interface and hold the inner object, and in Python the code can be identical. The difference is intent and multiplicity. A decorator adds behaviour and is meant to be stacked, so several is normal. A proxy controls access — lazily, remotely, with a permission check, or from a cache — and there is usually exactly one. The test I use is: would applying two of these at once mean something? If yes, decorator.”

“Should the facade be the only way in?” “No, and that is a deliberate choice. A facade is a convenience door, not a wall. If I forbid direct access, then every caller with a slightly different need has to widen the facade, and it grows one method at a time until it is the union of everything, at which point it is a god object. Reporting should be able to query orders directly.”

“How do you stop a facade becoming a god class?” “One rule: it may sequence and translate, and it may not make business decisions. The moment a tax rule or a discount rule appears in it, that is the beginning. And I would measure rather than assert — run `git log` on the file, and if four teams are editing it, that is divergent change and it needs splitting, probably by audience.”

“Why does this loop make 101 queries?” “Because `order.customer` is a virtual proxy, not a `Customer`. It looks like a field and it fires a query the first time you touch it, so a loop over 100 orders is one query plus a hundred. The fix is to load eagerly — a join or `select_related` — and the general lesson is that the proxy’s invisibility is both why it is convenient and why this happens. About 151 milliseconds against 4.”

“Gateway or direct calls from the mobile app?” “Gateway, and the argument is round trips rather than architecture. Six calls at a 120 ms mobile round trip is 720 milliseconds even before six TLS handshakes and six auth checks. One call to a gateway that fans out inside the data centre, where the round trip is about a millisecond, is roughly 125. And the response can

be shaped for that screen, so about 8 KB instead of 40. If the web and mobile clients want genuinely different shapes, I would give each one its own — a backend-for-frontend — rather than making one gateway serve both badly.”

A model answer

Asked: *your checkout touches six services. How do you expose it to the client?*

“With a facade — one method, `checkout(cart, user)`, returning an `Order` .

The thing I want to be precise about is what that actually removes, because it is not the six services; they are all still there and still directly callable. What it removes is the caller’s need to know **the order and the failure policy**. Inventory must be reserved before payment is charged, or you take money for something you cannot ship. The order must be created before the notification goes out, or you email somebody about an order that then failed to save. Right now that sequence lives in every caller, so with nine callers there are nine chances to get it wrong, and fifty-four coupling edges. With a facade there is one place and fifteen edges.

The failure policy is the half people forget. What happens if the payment succeeds and creating the order fails? That question has to be answered somewhere. In the facade I can put a compensating refund, or an idempotency key so a retry does not double-charge, or an outbox so the notification is retried. In nine call sites, it is answered nine different ways or not at all.

Two rules I would hold it to. It sequences and translates; it does not make business decisions — the moment a tax rule appears in it, it starts becoming the god object it was meant to prevent. And it does not seal the subsystem; reporting still queries orders directly. If I forbid that, every caller with a slightly different need widens the facade and it grows without limit.

Now, if the client is a mobile app rather than another service, the same pattern goes one layer up and it is an API gateway. The argument there is round trips: six calls at a 120-millisecond mobile round trip is 720 milliseconds, plus six TLS handshakes and six auth checks. One call to a gateway that fans out inside the data centre, where round trips are about a millisecond, is roughly 125 milliseconds — and it can return one response shaped for that screen, maybe 8 KB instead of 40. If web and mobile want genuinely different shapes, I would give each its own backend-for-frontend rather than one gateway serving both badly.

And since a proxy usually comes up alongside: a proxy is the other thing. It has the *same* interface as *one* object and controls access — lazily, remotely, with a permission check, or from a cache. nginx in front of the app is one, an ORM’s lazy `order.customer` is one, and a gRPC stub is one. The property that makes them useful — the caller cannot tell — is also the hazard: it is why a hundred-order loop quietly becomes a hundred and one queries, and why a method call that can time out looks exactly like one that cannot.”

9 Recall card

- **Facade** = a *new, simpler* interface over *many* things. **Proxy** = the *same* interface over *one* thing, **controlling access**. Two questions separate all four wrappers: *is the interface the same?* (same → decorator/proxy; different → adapter/facade) then *how many, and why?*
- **What a facade removes is the ordering and the failure policy, not the services.** Reserve before charging; notify last. $9 \text{ callers} \times 6 \text{ services} = 54 \text{ edges} \rightarrow 15$, and one place to put the compensating refund, the idempotency key or the outbox.
- **Two rules keep it from becoming a god object: it may sequence and translate but never make business decisions, and it must not seal the subsystem** (reporting still queries orders directly — otherwise every new need widens it, one method at a time). Measure it with `git log`, not opinion.
- **One layer up, a facade is an API gateway — and one per audience is a BFF.** Mobile: $6 \text{ round trips} \times 120 \text{ ms} = 720 \text{ ms}$ and $\sim 40 \text{ KB}$, versus $1 \text{ call} + \text{in-datacentre fan-out} = \sim 125 \text{ ms}$ and $\sim 8 \text{ KB}$.
- **Proxy has four kinds — virtual (lazy), remote, protection, caching — and its invisibility is both the point and the hazard.** ORM lazy fields make $N+1$ easy to write (151 ms vs 4 ms, $\sim 38\times$); remote stubs make a timeout-able call look local; Spring's `@Transactional` does **nothing** on self-invocation because that call never goes through the proxy. Structurally a proxy and a decorator are identical — **the difference is intent and multiplicity**, and you should say so rather than invent one.

PRACTICE

Practice — Min stack, and stacks that carry extra state

DSA topic: Min stack, and stacks that carry extra state **System design topic:** Facade and proxy

Code these, in this order

One rule for the whole set: **say the observation before writing the code**. “A stack only changes at the top, so the answer before a push is the answer after the matching pop.” Every problem below is that sentence with a different aggregate in it, and saying it first turns four problems into one.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Min Stack	LeetCode 155 (Medium)	The observation, and the <code><=</code> duplicate trap in the two-stack variant.
2	Max Stack	LeetCode 716 (Hard)	One comparison flipped — until <code>popMax</code> is asked for, and then it is a different problem.
3	Design a Stack With Increment Operation	LeetCode 1381 (Medium)	Lazy increments carried per element: the same idea with a different aggregate.
4	Maximum Frequency Stack	LeetCode 895 (Hard)	Extra state that is a map of stacks. The senior version of today.

On problem 1, write both versions and break one

Write the pairs version first. Then write the two-stack version with `<` and run `push(3)`, `push(3)`, `pop()`, `get_min()`. Read the exact error. Change one character and run it again.

On problem 2, notice where the technique stops

`push`, `pop`, `top` and `peekMax` are today’s idea with `max` instead of `min`. Then `popMax` — remove the maximum from anywhere in the stack — is not, because it changes the middle. Say out loud why today’s trick cannot help, and what structure you would reach for instead.

On problem 3, find the aggregate

The increment applies to the bottom `k` elements. Doing it eagerly is $O(k)$. Say what you would store per element so that `pop` applies the right total in $O(1)$, and why that is still the same technique.

On problem 4, say what is being carried

Not a number this time. Say out loud what the extra state is, and why a stack per frequency is the right shape rather than one stack of pairs.

The observation drill

Say each of these out loud, then check it:

1. Why does `pop` need no work at all in the pairs version?
2. What property of a stack makes that true, that a queue does not have?
3. Where did the cost of `get_min` go? It did not disappear.
4. What would break if you kept a single `self._min` variable and nothing else?
5. Why can the same trick not give you the median?

The duplicate drill

Using the two-stack version:

1. Write it with `<`. Run `push(3)`, `push(3)`, `pop()`, `get_min()`. Quote the exact output.
2. Now with three elements underneath, so it returns a wrong answer instead of raising. Construct that input.
3. Change `<` to `≤`. Trace the min stack after each of the four operations.
4. Say why the matching `pop` must use `=` and not `≤`.
5. Say what the pairs version does about this problem, and why.

The space drill

For each input, say how many entries the optimised two-stack version holds in its min stack:

1. `[1, 2, 3, 4, 5]`
2. `[5, 4, 3, 2, 1]`
3. `[3, 3, 3, 3, 3]`
4. `[5, 1, 5, 1, 5]`
5. A random permutation of 1 to 1000

Then state the best case and the worst case as multiples of n , and say whether you would describe the optimisation as saving memory.

The aggregate drill

For each, say whether today's technique works, and give the reason:

1. Minimum
2. Maximum
3. Sum
4. Count of elements
5. Product
6. Average
7. Median
8. Most frequent element
9. Second smallest
10. Range (max minus min)

Three of the ten do not work. Name the property they all fail, in one sentence. For number 6, say what you would actually store.

The encoded-trick drill

1. Implement the $O(1)$ -extra-space version with `2 * value - min`.
2. Prove to yourself that the encoded value is always strictly less than the new minimum.
3. Show that `2 * min - stored` recovers the previous minimum.
4. What does `top()` have to do differently?
5. In Java with `int`, find two values where `2 * value - min` overflows.
6. Say whether you would write this in an interview, and under what condition.

The four-wrappers drill, final version

Sort each into adapter, decorator, facade or proxy. Say which of the two separating questions you used first.

1. `requests.get(url)`
2. nginx in front of your application
3. `functools.lru_cache`
4. `psycopy` presenting the DB-API
5. An ORM's lazy `order.customer`
6. A gRPC client stub
7. A class adding retry around a service, same interface
8. `CheckoutFacade.checkout(cart, user)`
9. `io.TextIOWrapper`
10. Spring's `@Transactional` wrapper

Two of the ten have identical structure and different intent. Name them, give the test that separates them, and say why it is honest to admit the structures are the same.

The checkout drill

You have six services: inventory, pricing, payments, orders, shipping, notifications.

1. Write the facade method with the six calls in the correct order.
2. For each ordering constraint, write the one-line comment saying what breaks if it is violated.
3. Count the coupling edges with nine callers, with and without the facade.
4. Payment succeeds and order creation fails. Write what the facade does.
5. The client retries. What stops the customer being charged twice?
6. Somebody asks you to add "apply an 18% tax for Indian customers" to the facade. Say no, and say where it goes instead.
7. Reporting wants to query orders directly. Allow it or forbid it? Defend your answer.

The gateway drill

A mobile home screen needs data from six services.

1. Six sequential calls at a 120 ms mobile round trip. Total?
2. Six parallel calls. Total, and what else does the client pay six times?
3. One gateway call, fanning out inside the data centre at a 1 ms round trip. Total?
4. Payload: 40 KB of six raw responses versus a shaped response. Estimate the shaped size.
5. Web and mobile want different shapes. One gateway or two? Name the pattern.
6. What does the gateway now become a single point of, and what would you do about it?

The proxy-hazard drill

For each, say which kind of proxy it is and what goes wrong because it is invisible:

1. A loop over 100 orders touching `order.customer`.
2. A gRPC stub call inside a tight loop.
3. A `@Transactional` method called from another method of the same class.
4. A caching proxy in front of a permission check.
5. A CDN serving a page that has just been updated.

Then, for numbers 1 and 3, write the exact fix in one line each.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Design a stack that returns its minimum in constant time.* The naive cost first, the observation stated before any code, the pairs version, the two-stack variant with its `≤` trap volunteered, the space cost, and the O(1)-space trick mentioned with its overflow caveat.
 2. *Your checkout touches six services. How do you expose it to the client?* The facade with what it actually removes (ordering and failure policy), the compensation and idempotency answer, the two rules, the coupling-edge arithmetic, and the jump to API gateway with the 720 ms versus 125 ms number.
 3. *What is the difference between a facade and a proxy?* New-and-simpler over many versus same-over-one controlling access, the two separating questions, and the honest admission that proxy and decorator are structurally identical.
-

Before you move on

- [] I can state the observation — why `pop` does no work — in one sentence, from memory.
- [] I broke the two-stack version with `<` and know the exact input and output.
- [] I can say the best-case and worst-case size of the optimised min stack.

- [] I can name three aggregates this technique cannot carry, and the property they fail.
- [] I wrote the max stack and the sum stack and saw that only one line changed.
- [] I know why `popMax` is not the same problem.
- [] I can sort all four wrapping patterns using the two separating questions.
- [] I wrote the checkout facade with the ordering comments and answered the failure question.
- [] I can quote the coupling-edge arithmetic and the mobile round-trip arithmetic.
- [] I can name the two rules that stop a facade becoming a god object.
- [] I can explain why `@Transactional` does nothing on self-invocation.
- [] I answered all three questions above out loud.

Monotonic stack: the next greater element

DATA STRUCTURES AND ALGORITHMS

Monotonic stack: the next greater element

You can solve the whole next-greater and next-smaller family in $O(n)$.

SYSTEM DESIGN

Strategy

You can swap an algorithm at runtime without a chain of if-statements.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *For every element, find the next greater element to its right.*
- *The pricing rule changes per country. How do you structure that?*

Monotonic stack: the next greater element

1 What this is, and why they ask it

A **monotonic stack** is a stack whose contents are always in order — either increasing or decreasing from bottom to top. You keep it that way by popping anything that would break the order before you push. That one discipline solves a whole family of questions in a single pass: for every element, find the next greater element to its right, or the next smaller, or the previous greater, or the previous smaller.

The idea that makes it work is worth having in one sentence: **the moment you pop something, you have found its answer**. You are not scanning for answers; the answers fall out as a side effect of keeping the stack ordered. And the elements sitting in the stack are exactly the ones whose answer is still unknown.

They ask it because the brute force is obvious and quadratic, the linear solution is eight lines, and the eight lines contain a `while` loop inside a `for` loop that is *not* quadratic — so you have to be able to defend the complexity. That defence is the real question. It is also the gateway to a set of problems that look nothing alike on the surface — daily temperatures, stock spans, trapping rain water, largest rectangle in a histogram — and are all this one pattern.

2 The story

Chickpet is decorated for the festival every year, and the man who does the street of cloth shops is called Nagesh.

The decoration is strings of small lights, and the rule was fixed forty years ago: from the top of each shop's board, a string runs forward along the street to the top of the **first board taller than it**. Shorter boards ahead do not count, or the string sags into the road.

The boards are all different heights — a small painted plank, or a great tin hoarding two storeys up.

Nagesh does it in one walk down the street, and he has done it this way for so long he does not think about it.

He starts at the first shop, ties one end of a string to its board, and carries the loose end with him. At the second shop, if that board is taller, he ties the end off there — that string is finished. Then he ties a fresh string to the second shop's board and carries that end on.

Usually the next board is not taller. So he ties a new string to that shop as well, and now he walks with two loose ends in his hand. Then three. On a stretch of small shops he can be carrying six or seven ends bunched in his fist.

And then he reaches something big — the hoarding outside the sari shop, the tallest thing on the street — and in twenty seconds he ties off every end he is holding, because that board is taller than all of them. His hand is empty. He ties one new string there and walks on with one end.

The thing his nephew noticed is that the ends in his hand are always in a particular order without him arranging them. The oldest one, at the bottom of the bunch, is from the tallest board, and each one above it is from a shorter board than the one below. It cannot be otherwise — if a new board were taller than one he is holding, he would have tied that one off instead of adding to the bunch.

At the end of the street he still holds four or five ends, from the boards that never found anything taller ahead. Those get tied to the pole at the corner, which is what it is for.

3 The idea in plain English

The bunch of loose ends in Nagesh's hand is a **monotonic stack**. Each end is a shop whose answer — “which is the next taller board?” — is **not yet known**. Tying an end off is finding that shop's answer. And the ends left at the end of the street are the elements with no answer.

Three properties, and all three are things to say out loud.

One: the stack holds exactly the unanswered elements. Not a scratch space, not a buffer. Every element in the stack is one whose next-greater is still unknown.

Two: the stack is automatically ordered. Nobody sorts it. It is decreasing from bottom to top because anything that would break the order gets tied off — popped — before the new one is added. That is what “monotonic” means, and it is a consequence of the algorithm rather than a rule you enforce separately.

Three: one arrival can answer many. The big hoarding tied off seven strings at once. This is why the inner loop is not a problem: those seven had each been pushed once, and now they are popped once.

The eight lines

```
def next_greater(numbers: list[int]) → list[int]:
    answer = [-1] * len(numbers)      # -1 means "nothing taller ahead"
    stack: list[int] = []              # indices of unanswered elements

    for index, value in enumerate(numbers):
        while stack and numbers[stack[-1]] < value:
            answer[stack.pop()] = value  # this element's answer is `value`
            stack.append(index)

    return answer
```

Read the `while` line as Nagesh: *while I am holding an end from a board shorter than this one, tie it off*. Then push the current index, because this element's own answer is not yet known.

Store indices, not values. You need the index to write into `answer`, and if the question asks “how many days until” rather than “what is”, you need `index - stack.pop()`. Storing values throws that away, and it is the first thing to change when the follow-up arrives.

The four variants, and the one rule that generates them

The whole family is two knobs: which direction you walk, and which way the comparison points.

YOU WANT	WALK	POP WHILE THE TOP IS
next greater to the right	left → right	smaller than current
next smaller to the right	left → right	greater than current
previous greater to the left	right → left	smaller than current
previous smaller to the left	right → left	greater than current

The rule to memorise, because it generates all four:

Direction of the walk is opposite to the direction you are looking. Pop while the top is on the wrong side of the comparison you want.

If you want the next greater *to the right*, you walk left to right, and each element’s answer is found by something that arrives later. So you walk forwards and look backwards into the stack.

The stack’s order follows automatically: for “next greater”, the stack ends up **decreasing** from bottom to top. For “next smaller”, **increasing**. You do not have to remember that — it falls out — but stating it is a good check that your comparison is the right way round.

Strict or not: the `<` versus `<=` question

`numbers[stack[-1]] < value` pops on strictly smaller, so equal elements stay in the stack, and an element’s answer must be *strictly* greater.

Change it to `<=` and equal elements pop each other, so an element’s answer is the next *greater-or-equal*. On `[2, 2, 3]`:

```
with < : answer for index 0 is 3    (skips the equal 2)
with ≤ : answer for index 0 is 2    (the equal one counts)
```

Ask which the problem wants. “Next greater” almost always means strictly greater, but “next day at least as warm” does not, and getting it wrong is a one-character bug that passes half the tests.

The leftovers

Whatever is still in the stack when the loop ends has no answer. Initialising `answer` to `-1` (or `0` for “how many days”) handles that with no extra code — Nagesh’s pole at the corner.

If you prefer, push a sentinel of infinity at the end to force everything out, which is the trick that makes [day 072's](#) histogram problem cleaner.

Circular arrays

“Next greater element in a circular array” — after the last element you wrap round to the first. The trick is to walk $2n$ steps and use `index % n`:

```
for step in range(2 * len(numbers)):
    index = step % len(numbers)
    while stack and numbers[stack[-1]] < numbers[index]:
        answer[stack.pop()] = numbers[index]
    if step < len(numbers):          # only push on the FIRST pass
        stack.append(index)
```

The `if step < len(numbers)` is the part people miss. On the second pass you only answer; you do not add new unanswered elements, or they would never be resolved.

The family this unlocks

- **Daily Temperatures** — next warmer day, answering with the *distance* rather than the value.
- **Stock Span** — previous greater, walking the other way.
- **Largest Rectangle in a Histogram** — for each bar, the previous smaller and the next smaller. That is [day 072](#).
- **Trapping Rain Water** — the same stack, accumulating as it pops.
- **Remove K Digits / smallest subsequence** — a monotonic stack maintaining the smallest sequence.

Recognising the family is what the pattern is for. “For each element, the nearest element on one side satisfying a comparison” is the trigger sentence.

4 The picture

`[2, 1, 2, 4, 3]`, finding the next greater to the right. The stack holds **indices**; the heights underneath are the values there.

i=0	v=2	stack empty	push 0	stack: [0]	(2)
i=1	v=1	top is 2, not < 1	push 1	stack: [0,1]	(2,1)
i=2	v=2	top is 1 < 2 → pop 1, answer[1]=2			
		top is 2, not < 2	push 2	stack: [0,2]	(2,2)
i=3	v=4	top is 2 < 4 → pop 2, answer[2]=4			
		top is 2 < 4 → pop 0, answer[0]=4			
		stack empty	push 3	stack: [3]	(4)
i=4	v=3	top is 4, not < 3	push 4	stack: [3,4]	(4,3)

leftovers: indices 3 and 4 keep -1
answer = [4, 2, 4, -1, -1]

What to notice at `i=3` : **one element answered two**. That is the hoarding tying off the bunch, and it is the reason a `while` inside a `for` is still linear — those two pops correspond to two pushes that already happened.

And the stack drawn as a bunch, so the ordering is visible:

top →	+-----+	
	1	index 1
	+-----+	
	2	index 0
bottom →	+-----+	

the values in the stack are always DECREASING from the bottom up, for next-greater.

Nothing sorted it. Anything that would have broken the order was popped before it could go on.

If your stack ever has a smaller value below a bigger one while solving next-greater, your comparison is backwards.

5 The code, built step by step

Step 1 — the brute force, said and not written

“For each element, walk right until I find something bigger.” That is $n(n-1)/2$ comparisons in the worst case, which at $n = 100,000$ is five billion. Say the number and move on.

Step 2 — the observation

Say this before writing anything, because it is the answer:

“When I am at element `i` and it is bigger than something I passed earlier, that earlier element’s answer is `i`. So I do not need to search forward from each element — I need to remember which earlier elements are still waiting, and resolve them when something bigger arrives.”

Step 3 — the loop

```
for index, value in enumerate(numbers):
    while stack and numbers[stack[-1]] < value:
        answer[stack.pop()] = value
    stack.append(index)
```

Three lines, and the `while` before the `append` is the whole ordering discipline. Resolve everything this element can resolve, *then* add yourself as unresolved.

Note `while stack and ...` — the same guard as every stack problem. Without it, the first element raises `IndexError` when the stack is empty.

Step 4 — the distance variant

Daily Temperatures asks *how many days until it is warmer*, not *what is the next warmer temperature*. Only one line changes, and this is why you store indices:

```
while stack and temperatures[stack[-1]] < temperature:
    previous = stack.pop()
    answer[previous] = index - previous      # a distance, not a value
```

If you had stored values, this variant would be impossible without a second structure. **Always store indices** and derive the value with `numbers[index]` when you need it.

The complete solution

```
def next_greater(numbers: list[int]) → list[int]:
    """For each element, the next strictly greater element to its right,
    or -1 if there is none.

    The stack holds INDICES of elements whose answer is not yet known, and is
    kept decreasing by value from bottom to top. Each index is pushed once and
    popped at most once, so this is O(n) despite the inner while loop.
    """
    answer = [-1] * len(numbers)
    stack: list[int] = [] # indices, values decreasing bottom→top

    for index, value in enumerate(numbers):
        while stack and numbers[stack[-1]] < value:
            answer[stack.pop()] = value # `value` is this element's next greater
            stack.append(index) # its own answer is not known yet

    return answer # leftovers keep -1

def daily_temperatures(temperatures: list[int]) → list[int]:
    """How many days until a strictly warmer day. 0 if there is none.

    Identical to next_greater, except the answer is a DISTANCE – which is why
    the stack must hold indices rather than values.
    """
    answer = [0] * len(temperatures)
    stack: list[int] = []

    for index, temperature in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temperature:
            previous = stack.pop()
            answer[previous] = index - previous
            stack.append(index)

    return answer

def next_smaller(numbers: list[int]) → list[int]:
    """The next strictly smaller element to the right. One comparison flipped;
    the stack is now INCREASING from bottom to top."""
    answer = [-1] * len(numbers)
    stack: list[int] = []

    for index, value in enumerate(numbers):
        while stack and numbers[stack[-1]] > value:
            answer[stack.pop()] = value
            stack.append(index)

    return answer

def previous_greater(numbers: list[int]) → list[int]:
    """The previous strictly greater element to the LEFT. Same comparison as
    next_greater; the walk is reversed."""
    answer = [-1] * len(numbers)
    stack: list[int] = []

    for index in range(len(numbers) - 1, -1, -1):
        while stack and numbers[stack[-1]] < numbers[index]:
            answer[stack.pop()] = numbers[index]
            stack.append(index)

    return answer
```

```

def next_greater_circular(numbers: list[int]) → list[int]:
    """Circular: after the last element, wrap round to the first.

    Walk 2n steps with index = step % n, but only PUSH on the first pass –
    otherwise the second pass adds elements that can never be resolved.
    """
    n = len(numbers)
    answer = [-1] * n
    stack: list[int] = []

    for step in range(2 * n):
        index = step % n
        while stack and numbers[stack[-1]] < numbers[index]:
            answer[stack.pop()] = numbers[index]
        if step < n:
            stack.append(index)

    return answer

def next_greater_or_equal(numbers: list[int]) → list[int]:
    """The ≤ variant: an equal element counts as an answer.
    On [2, 2, 3] this gives 2 for index 0, where the strict version gives 3."""
    answer = [-1] * len(numbers)
    stack: list[int] = []

    for index, value in enumerate(numbers):
        while stack and numbers[stack[-1]] ≤ value:      # ≤ not <
            answer[stack.pop()] = value
        stack.append(index)

    return answer

if __name__ == "__main__":
    print(next_greater([2, 1, 2, 4, 3]))          # [4, 2, 4, -1, -1]
    print(next_greater([5, 4, 3, 2, 1]))          # [-1, -1, -1, -1, -1]
    print(next_greater([1, 2, 3, 4, 5]))          # [2, 3, 4, 5, -1]
    print(next_greater([]))                      # []
    print(next_greater([7]))                     # [-1]
    print(daily_temperatures([73, 74, 75, 71, 69, 72, 76, 73]))
    # [1, 1, 4, 2, 1, 1, 0, 0]
    print(next_smaller([4, 2, 5, 1]))              # [2, 1, 1, -1]
    print(previous_greater([2, 1, 2, 4, 3]))       # [-1, 2, -1, -1, 4]
    print(next_greater_circular([1, 2, 1]))        # [2, -1, 2]
    print(next_greater([2, 2, 3]))                # [3, 3, -1]
    print(next_greater_or_equal([2, 2, 3]))        # [2, 3, -1]

```

Run the last two lines together. The same input gives different answers, and knowing which one the problem wants is a question, not a guess.

6 What it costs

Time — and this is the whole interview

The code has a `while` inside a `for`, which looks quadratic and is not. Count it the right way.

The `for` loop runs `n` times, so there are exactly `n` pushes — one per element, and no element is ever pushed twice.

The `while` loop only ever pops. **An element can be popped at most once, because once it is off the stack it never goes back on.** So across the entire execution, the total number of pops is at most `n`, however unevenly they are spread.

```
pushes across the whole run:  exactly n
pops   across the whole run:  at most n
-----
total stack operations:      ≤ 2n    →  O(n)
```

Say it exactly like this: **the inner loop is bounded in total, not per iteration.** One iteration can do `n` pops — the sari shop hoarding — and that is fine, because those `n` pops used up all the pops the rest of the run was going to do.

This is the same argument as the `while` loop in longest-consecutive from [day 062](#), and it will keep being the argument for the rest of the stack phase.

Against the brute force

```
brute force:  n(n-1)/2 comparisons
monotonic:    ≤ 2n stack operations

n = 100,000:  4,999,950,000  vs  200,000
ratio:        about 25,000x
```

At `n = 100,000` the brute force is roughly five billion operations — tens of seconds — and the monotonic stack is two hundred thousand, which is a few milliseconds.

Space

```
answer array:  O(n)    - required, it is the output
stack:         O(n)    worst case
```

The stack holds every element at once when nothing ever gets popped, which is a strictly decreasing input like `[5, 4, 3, 2, 1]`. Best case is $O(1)$: on `[1, 2, 3, 4, 5]` every element pops the previous one immediately, so the stack never holds more than one.

Say both, and name the inputs. It takes five seconds and it is the difference between having thought about it and having recited it.

The circular version

`2n` steps, still at most `n` pushes and `n` pops, so still **$O(n)$** with a constant of two. Worth stating, because “you loop twice, so it is $O(n^2)$ ” is a thing interviewers say to see whether you will agree with them.

7 The traps

Trap 1 — storing values instead of indices

```
stack.append(value)                # instead of index
```

Works for “what is the next greater”. Fails completely for “how many days until”, because you cannot compute a distance without knowing where the element was. It also fails for the histogram problems, where the width is the whole point. **Always store indices.** The only cost is writing `numbers[stack[-1]]` instead of `stack[-1]`.

Trap 2 — the missing emptiness guard

```
while numbers[stack[-1]] < value:    # no `stack and`
```

```
IndexError: list index out of range
```

Fires on the very first element, since the stack starts empty. Same guard as every stack problem in this phase, and it must be the first half of the `and`.

Trap 3 — pushing before popping

```
stack.append(index)
while stack and numbers[stack[-1]] < value:    # WRONG ORDER
    answer[stack.pop()] = value
```

Now the element compares against itself. `numbers[index] < numbers[index]` is false, so it happens to survive — but on the `≤` variant it pops itself and writes its own answer as its own value. **Resolve first, then push.**

Trap 4 — `<` when the problem wanted `<=`

```
next_greater([2, 2, 3])        → [3, 3, -1]
next_greater_or_equal([2, 2, 3]) → [2, 3, -1]
```

Both are correct programs. Only one answers the question you were asked. Read the problem statement for the word “strictly”, and if it is not there, ask.

Trap 5 — the wrong comparison for the wrong variant

For **next smaller**, the comparison flips to `>` and the stack becomes increasing. People flip the direction of the walk instead, which gives *previous* greater, and the answers look plausible on symmetric inputs. The check: run it on `[4, 2, 5, 1]` and read the answer for index 0. Next smaller gives 2; previous greater gives -1.

Trap 6 — pushing on both passes of the circular version

```
for step in range(2 * n):
    index = step % n
    ...
    stack.append(index)           # WRONG — pushes again on the second pass
```

The second pass adds elements that nothing after them can resolve, so they sit in the stack forever and some answers get overwritten with values from the wrong lap. Guard with `if step < n`.

Trap 7 — forgetting the leftovers

If you build the answer by appending as you pop, the elements never popped are simply missing from your output, and the list is shorter than the input. Pre-fill the answer array with the default and write into it by index. Never append.

Trap 8 — believing the interviewer when they call it quadratic

They will say “there is a loop inside a loop, so that is $O(n^2)$ ”. The answer is the counting argument, stated confidently: n pushes total, at most n pops total, so at most $2n$ operations. Do not soften it.

8 In the interview

How it gets asked

- The base case: “For every element in this array, find the next greater element to its right. Return -1 if there is none.” LeetCode 496 and 503.
- The distance version, which is the most commonly asked: “Given daily temperatures, return how many days you would have to wait for a warmer day.” LeetCode 739.
- Disguised: “For each day, how many consecutive days before it had a price less than or equal to today’s?” — the Stock Span problem, which is previous-greater.
- The escalation: “largest rectangle in a histogram”, or “trapping rain water” — the same stack with more bookkeeping.

What to say out loud, in the first ninety seconds

1. **State the brute force with a number and reject it.** “For each element, walk right until something is bigger. That is n squared over two — five billion at a hundred thousand elements.”
2. **Say the observation, not the algorithm.** “When I am standing on an element that is bigger than things I passed, I have just found *their* answer. So instead of searching forward from each element, I remember which earlier elements are still waiting and resolve them when something bigger turns up.”

3. **Say what is in the stack.** “The stack holds the indices of elements whose answer is still unknown, and it stays decreasing by value — not because I sort it, but because anything that would break the order gets popped first.”
4. **Say indices, and why.** “I store indices, not values, because the follow-up is usually a distance.”
5. **Pre-empt the complexity challenge.** “There is a `while` inside the `for` and it is still $O(n)$: n pushes total, and each element is popped at most once, so at most $2n$ operations overall.”

The follow-ups

“That is a loop inside a loop. Is it not $O(n^2)$?” “No. Count the operations rather than the nesting. Every element is pushed exactly once, by the outer loop. Every element is popped at most once, because once it leaves the stack it never returns. So the total work across the whole run is at most $2n$ stack operations, however unevenly they are distributed. One iteration might pop n elements, and that is fine — it has consumed all the pops the rest of the run was going to do.”

“Now do it circularly.” “Walk $2n$ steps with `index = step % n`, and only push during the first pass. If I push on the second pass I add elements that nothing after them can resolve. Still $O(n)$, with a constant factor of two.”

“What if I want the next smaller instead?” “Flip the comparison from `<` to `>`, and the stack becomes increasing rather than decreasing. Nothing else changes. There are four variants from two knobs — which direction I walk, and which way the comparison points — and they generate next greater, next smaller, previous greater and previous smaller.”

“What does ‘next greater’ mean if there are duplicates?” “That is a question I would ask you. With a strict `<` in the pop condition, an equal element does not count and `[2, 2, 3]` gives 3 for the first element. With `≤`, it does count and gives 2. Both are one-character changes and only one matches the specification.”

“Where else does this pattern show up?” “Anywhere the question is ‘for each element, the nearest element on one side satisfying a comparison’. Daily temperatures is the distance version. Stock span is previous-greater. Largest rectangle in a histogram needs the previous smaller and the next smaller for each bar, which is two runs of this. Trapping rain water is the same stack accumulating water as it pops.”

A model answer

Asked: *for every element, find the next greater element to its right.*

“The brute force is: for each element, walk right until I find something bigger. Worst case that is n squared over two — about five billion comparisons at a hundred thousand elements — so I want a single pass.

Here is the observation that gives me one. When I am standing on an element and it is bigger than some elements I have already passed, I have just found *their* answers. So I should stop thinking of it as ‘search forward from each element’ and start thinking of it as ‘remember which earlier elements are still waiting, and settle them when something big enough arrives’.

That collection of waiting elements is a stack, and I store indices rather than values in it — because the standard follow-up is ‘how many days until’, and a distance needs the position.

The loop is three lines. For each element: while the stack is not empty and the value at the top is less than the current value, pop it and record that its answer is the current value. Then push the current index, because its own answer is not known yet. Resolve first, then push — the other order makes an element compare against itself.

The stack ends up decreasing from bottom to top, and I did not sort it. It has to be decreasing, because anything that would break the order is exactly the thing the `while` pops before pushing. That is what makes it a monotonic stack.

Whatever is still in the stack at the end has no greater element to its right, so I pre-fill the answer array with -1 and only ever write into it by index — never append, or the unanswered elements just go missing.

On complexity, since the shape looks quadratic: it is $O(n)$. Every element is pushed exactly once by the outer loop, and popped at most once, because once it is off the stack it never goes back on. So the total number of stack operations across the whole run is at most $2n$. One single iteration might pop a hundred thousand elements, and that is fine, because it has used up all the pops the rest of the run was going to do. The inner loop is bounded in total, not per iteration.

Space is $O(n)$ for the answer, which is the output, plus $O(n)$ for the stack in the worst case — a strictly decreasing input, where nothing is ever popped. On a strictly increasing input the stack never holds more than one element.

One thing I would check before writing: does ‘greater’ mean strictly greater? With duplicates, `[2, 2, 3]` gives 3 for the first element under a strict comparison and 2 under a non-strict one, and it is one character of difference.”

9 Recall card

- **The stack holds exactly the elements whose answer is not yet known, and it stays ordered by itself** — anything that would break the order is popped before the new element goes on. **The moment you pop something, you have found its answer.**
- **Eight lines:** `while stack and numbers[stack[-1]] < value: answer[stack.pop()] = value`, then `stack.append(index)`. Resolve first, then push. **Store indices, never values** — the usual follow-up wants a *distance* (`index - stack.pop()`), and values cannot give you one.
- **Four variants from two knobs:** direction of the walk $\times$ direction of the comparison $\rightarrow$ next greater $\cdot$ next smaller $\cdot$ previous greater $\cdot$ previous smaller. Next-greater leaves the stack **decreasing** bottom-to-top; next-smaller **increasing**. And ask about duplicates: `<` gives strictly greater, `<=` gives greater-or-equal — on `[2, 2, 3]` that is 3 versus 2.
- **The complexity defence is the interview.** n pushes total, **each element popped at most once**, so $\leq 2n$ operations — **the inner loop is bounded in total, not per iteration**. One iteration popping n elements has consumed all the pops the rest of the run was going to do. 5×10^9 vs 2×10^5 at $n = 100,000$, about **25,000 $\times$** .
- **Space $O(n)$ worst case** (a strictly decreasing input, nothing ever pops), **$O(1)$ best case** (a strictly increasing one). **Pre-fill the answer array and write by index** — never append, or the leftovers vanish. Circular: walk `2n` with `step % n` and **push only on the first pass**. The family: daily temperatures $\cdot$ stock span $\cdot$ largest rectangle $\cdot$ trapping rain water.

1 What this is, and why they ask it

Strategy turns an algorithm into a value. Instead of a function containing a branch that chooses between three ways of doing something, you define one interface with one method, write one small implementation per way, and hand the caller whichever one it needs. The code that *uses* the algorithm never learns how many there are.

It is the most useful of the eleven behavioural patterns, and it is the one you have already been using since [day 058](#) without the name: `sorted(items, key=...)` is Strategy. Python's `sorted` has no idea how you want things ordered, and that is the entire design — the comparison rule is a parameter.

They ask it because it is the cleanest demonstration of open/closed from [day 056](#), because the arithmetic for it is easy to state, and — most usefully — because it is over-applied constantly. Three tax rates that differ only by a number do not need three classes. A candidate who reaches for Strategy every time there is an `if` has learned a pattern; a candidate who can say when a dictionary of numbers is the right answer has learned to design.

2 The story

Basavaraj has driven an auto in Bengaluru for eleven years, and the trip he does more than any other is from the flats near the Metro station to the hospital on Bannerghatta Road.

There are three ways to do it, and he can describe all three in one sentence each.

There is the straight one, down the main road. Shortest on the map, and between eight and eleven in the morning it is thirty-five minutes of not moving.

There is the one through the residential streets behind the school. Longer, twisty, and empty except between three and four when the school comes out and it is unusable.

And there is the one round by the flyover. Longest by distance, costs more in petrol, and it is the only one that is the same at nine in the morning as at nine at night.

He does not think about which one to take in the way a passenger might imagine. He asks two things before he starts — what time is it, and is anyone in a hurry — and then he takes one of the three. If it is ten past eight he is on the flyover. If it is half past two he is going behind the school. If there is a man in the back with a bag and a train to catch, flyover, whatever the time is, because it is the one where he can promise a number.

From the passenger's side, none of this exists. You get in, you say the hospital, and you get to the hospital. You are not asked to choose, and if he had four routes instead of three you would not notice.

The part he mentions when the subject comes up is what happened when the new flyover opened in 2023. He drove it a few times to see what it was like, decided it was good after eight at night, and added it. That is all that happened. He did not have to think again about the school route or the main road; they are still exactly what they were. One new thing learned, and nothing unlearned.

His younger cousin drives too, and does it differently — one route, always, the main road, on the grounds that it is the shortest. He is not wrong about the distance.

3 The idea in plain English

Each route is a **strategy**: a complete, self-contained way of doing the one job. The job — get to the hospital — does not change. The method does.

Three things from the story, and they are the three things the pattern buys.

One: the passenger does not choose and does not know. The calling code depends on the *job*, not on which way it is done. That is the interface.

Two: adding a fourth route did not disturb the other three. New behaviour arrives as a new thing, not as an edit to existing things. That is open/closed, and it is the argument you make in edit counts.

Three: choosing is a separate job from doing. Basavaraj's two questions — what time is it, is anyone in a hurry — are the selection logic, and they are not part of any route. Keeping those apart is what stops the pattern collapsing back into the `if` chain it replaced.

The problem it replaces

```
def price(order: Order, country: str) → Decimal:
    if country == "IN":
        base = order.subtotal * Decimal("1.18")
        if order.subtotal > 5000:
            base -= Decimal("100")
        return base
    elif country == "AE":
        return order.subtotal * Decimal("1.05")
    elif country == "GB":
        base = order.subtotal * Decimal("1.20")
        return base - order.loyalty_credit
    raise ValueError(country)
```

Four things are wrong, and it is worth naming all four rather than saying “it is ugly”.

Adding a country edits a shared file. Twelve people depend on `price`. Every new market is a change to code that is currently working for three existing markets, and forty tests re-run.

The rules are entangled. They are in one function, so a change to the UK rule and a change to the India rule are the same file, the same merge conflicts, the same review.

You cannot test one rule alone. Testing the UAE rule means constructing an order and passing "AE", so the test depends on the dispatch as well as the rule.

The function has more than one reason to change — a new market, or a change to an existing market's rule. That is single responsibility from [day 055](#).

The pattern

One interface with one method:

```
from typing import Protocol

class Pricer(Protocol):
    def price(self, order: Order) → Decimal: ...
```

One implementation per rule, each in its own file, each testable alone:

```
class IndiaPricer:
    def price(self, order: Order) → Decimal:
        total = order.subtotal * Decimal("1.18")
        return total - Decimal("100") if order.subtotal > 5000 else total

class UaePricer:
    def price(self, order: Order) → Decimal:
        return order.subtotal * Decimal("1.05")
```

And the caller holds one and does not know which:

```
class Checkout:
    def __init__(self, pricer: Pricer) → None:
        self._pricer = pricer

    def total(self, order: Order) → Decimal:
        return self._pricer.price(order)      # no branch, ever
```

Choosing, which is a separate job

Someone still has to pick. That job is a **factory** ([day 065](#)), and keeping it separate is the whole discipline:

```

PRICERS: dict[str, Pricer] = {
    "IN": IndiaPricer(),
    "AE": UaePricer(),
    "GB": UkPricer(),
}

def pricer_for(country: str) → Pricer:
    try:
        return PRICERS[country]
    except KeyError:
        raise UnsupportedCountry(country) from None

```

Concede the honest point: **the branch did not vanish, it moved**. But a line in a dictionary of constructors cannot break the India rule, and a new branch inside a shared function can. That distinction is the argument, and stating it plainly is better than pretending the `if` disappeared.

In Python, a function is a strategy

This is the part that separates someone who learned the pattern from a Java book.

```

PRICERS: dict[str, Callable[[Order], Decimal]] = {
    "IN": lambda o: o.subtotal * Decimal("1.18"),
    "AE": lambda o: o.subtotal * Decimal("1.05"),
    "GB": lambda o: o.subtotal * Decimal("1.20"),
}

def price(order: Order, country: str) → Decimal:
    return PRICERS[country](order)

```

That *is* Strategy. There is no class named `Strategy` and there does not need to be — the pattern is the shape, which is that the varying part is a value you can pass around, not the ceremony.

Take the classes when the rule needs **state**, or **more than one method**, or **its own file because a specific team owns it**. Take the function when it is a single expression. Twelve lines against ninety is not a small difference.

Python's own library is full of this: `sorted(key=)`, `min(key=)`, `heapq.nlargest(key=)`, `re.sub(repl=callable)`, `defaultdict(factory)`. Every one is an algorithm passed as an argument.

Strategy and its two look-alikes

Strategy versus State. Identical structure — an interface, several implementations, a context holding one. The difference is **who changes it**. In Strategy, the *client* chooses and it usually stays put. In State, the object **transitions itself**: a `Draft` order moves itself to `Submitted`, and the states know about each other. If your implementations set the context's next implementation, you have written State.

4 The picture

The two shapes, side by side.

The two shapes, side by side.

FIGURE 71.1

And the cost, drawn as it grows:

1. **Find the axis of change.** Not “this function is long” — *what keeps varying?* Country. Payment provider. Compression codec. Eviction policy. If you cannot name the axis, you are not ready.
2. **Write the interface as one method, in the domain’s vocabulary.** `price(order) → Decimal`, not `apply_rule(dict) → float`.

3. **Move each branch into its own implementation.** Copy, do not rewrite — this step should not change behaviour, and keeping it mechanical is what makes it safe.
4. **Keep the selection somewhere else.** A dictionary, a factory function, or configuration.
5. **Test each implementation directly**, with no dispatch involved.

Step 3 is where the value shows up immediately: each rule becomes independently testable, and that usually reveals that two of them were subtly different in ways nobody had noticed.

Where you have already used it

- `sorted(items, key=...)` and `min`, `max`, `heapq.nlargest`. The comparison rule is a parameter. This is the cleanest example in any language and the one to cite first.
- `hashlib.new("sha256")` — the digest algorithm chosen by name at run time.
- **Compression codecs** — `gzip`, `zstd`, `lz4` behind one `compress / decompress` interface. The choice is a trade between CPU and bytes and is genuinely made per deployment.
- **Load-balancing algorithms** — round-robin, least-connections, consistent hashing. `nginx` and `Envoy` expose exactly this as a configuration string.
- **Cache eviction policies** — LRU, LFU, FIFO, random, behind one `evict()`.
- **Retry backoff** — fixed, exponential, exponential-with-jitter. `tenacity` and `Resilience4j` both take the backoff as a strategy object.
- **Django's** `AUTH_PASSWORD_VALIDATORS` and `PASSWORD_HASHERS` — a list of class paths in settings, which is Strategy configured from a file.
- **Kubernetes scheduler plugins** and **Envoy's load-balancer policies** — the same shape at infrastructure scale.

The two failure modes

One: the variation is data, not behaviour. This is the commonest misuse by a distance.

```
class IndiaTax:    rate = Decimal("0.18")    # a class...
class UaeTax:     rate = Decimal("0.05")    # ...for a number
class UKTax:      rate = Decimal("0.20")
```

Three files, three classes, one interface, ~40 lines — to hold three numbers. The right answer:

```
TAX_RATES = {"IN": Decimal("0.18"), "AE": Decimal("0.05"), "GB": Decimal("0.20")}
```

The test: does the code differ, or only a value? If only a value, it is a dictionary. This is the mistake that produces the fourteen `*Factory` and `*Strategy` classes from [day 065](#).

Two: the selection leaks back into the caller.

```
class Checkout:
    def total(self, order: Order, country: str) → Decimal:
        if country == "IN":
            return IndiaPricer().price(order)
            # the branch is BACK
```


Now you have the interface, the implementations, *and* the `if` chain. The interface is doing nothing and you have paid for it. `Checkout` must receive a `Pricer`, not a country.

6 The numbers

Adding the fourth market

```
if-chain:
  files edited          1 (shared, ~34 lines, 12 dependents)
  existing tests re-run ~32
  live rules at risk    3
  merge-conflict surface the same function three teams edit
strategy:
  files added          1 (~15 lines)
  lines edited         1 (the registry)
  existing tests re-run 0
  live rules at risk    0
```

And the flat-cost point: **the twelfth market costs exactly what the second cost**. With the chain, the twelfth is edited into a 90-line function that nobody fully understands, and the cost per market is rising.

What the ceremony costs

```
if/elif chain, 3 rules      1 file, ~26 lines
dict of functions (Python)  1 file, ~12 lines
full classes, 3 rules      5 files, ~90 lines
(1 protocol + 3 pricers + 1 registry)
```

So the class version costs about **65 extra lines and 4 extra files**. That is the price. It is worth paying when markets keep arriving and each rule is more than an expression; it is pure cost when there are three rules that will never change.

The test argument, which is usually the persuasive one

```
test "UK orders get 20% VAT minus loyalty credit":
  with the if-chain: build an Order, call price(order, "GB"),
                    and the test now depends on the dispatch too
                    ~9 lines
  with a strategy:   UkPricer().price(order)
                    ~2 lines, and it cannot be broken by the India rule
```

And the second-order effect: with the chain, a change to the India rule can break the UK test, because they share a function. With strategies they are separate files and separate tests. That independence is what teams actually feel.

When the pattern loses

```
three tax RATES as three classes: 3 files, ~40 lines, 3 tests
three tax rates as a dict:         1 line
```

Forty lines against one. If the only difference between your implementations is a literal, the pattern has cost you thirty-nine lines and bought nothing.

7 The trade-offs

What you give up

Indirection. To find out what an Indian order is charged, a reader opens `Checkout`, finds it holds a `Pricer`, opens the registry, and then opens `IndiaPricer`. Three hops instead of reading one function top to bottom. This is real, and it is the honest cost.

The rules stop being comparable at a glance. With the `if` chain you can read all three rules side by side and see that one of them forgot the loyalty credit. Split across three files, that comparison is gone. For a small closed set, the chain is genuinely easier to review.

The failure moves to run time. `pricer_for("BR")` with no Brazil pricer is a `KeyError` when a Brazilian customer checks out, not an error at start-up. Mitigate it: type the key as an enum, validate at the boundary, and assert at start-up that every supported country has a pricer.

The interface becomes the lowest common denominator. When one market needs a second argument that no other market has, you either widen the interface for everybody or start passing a context object. This is the pressure that turns clean interfaces into `price(order, context: dict)`.

A name that outlives its reason. `PricingStrategy` with one implementation, two years after somebody deleted the other two, is worse than no interface at all — it looks deliberate.

“I would not use this if...”

- **...the variation is data, not behaviour.** Three rates are a dictionary. This is the big one.
- **...the set is genuinely closed.** Weekdays, blood groups, the three states an order can be in.
- **...it is the first occurrence.** Write it inline. The second occurrence shows you where the seam goes; the first one only lets you guess.
- **...I cannot name the second implementation.** The test double counts. A hypothetical does not.
- **...a function would do.** In Python, a one-expression rule in a dictionary beats a class hierarchy, and pretending otherwise is Java arriving where it was not invited.

The strongest thing you can say

Concede that the `if` did not disappear — it moved into a registry — and then say why that is still worth it: **a line in a list of constructors cannot break the India rule, and a new branch inside a shared function can.** That sentence is the whole argument, honestly made, and it is far more convincing than claiming the branch went away.

8 In the interview

How it gets asked

- The direct one: *“The pricing rule changes per country. How do you structure that?”*
- The refactor: *“Here is a function with a six-way `if`. What would you do?”* — where “nothing, for now” is sometimes the right answer and you must be able to say when.
- The distinction: *“What is the difference between Strategy and State?”* Same structure, different intent, and the answer is *who changes it*.
- The Python one: *“Would you write a Strategy class in Python?”*
- The scaling one: *“How would you let each customer configure their own retry behaviour?”* — Strategy chosen from configuration.

What to say out loud, in the first ninety seconds

1. **Name the axis before naming the pattern.** “The thing that keeps changing is the country’s pricing rule, and a new country arrives roughly every quarter. That is the axis, and that is where the interface goes.”
2. **Say what is wrong with the chain in edit counts, not adjectives.** “A new market edits a shared file that twelve callers depend on, re-runs about forty tests and puts three live rules at risk.”
3. **Write the interface and one implementation.** One method, domain vocabulary.
4. **Separate the selection explicitly.** “Choosing is a different job from doing. It goes in a registry, not in `Checkout` — otherwise the branch is back and I have paid for an interface that does nothing.”
5. **Give the Python answer.** “In Python I would start with a dictionary of functions — twelve lines rather than ninety. I would take the classes when a rule needs state, or several methods, or its own file because finance owns it.”
6. **Say when you would not.** “If the only difference between the countries were the tax rate, I would not do any of this. Three numbers in a dictionary. Strategy is for behaviour that differs, not values that differ.”

The follow-ups

“You still have an `if` somewhere. What did you actually gain?” “That is fair, and I would not claim the branch vanished — it moved into a registry. What changed is what a mistake there can do. A wrong line in a dictionary of constructors gives an unsupported-country error.

A wrong branch inside a shared pricing function can break Diwali pricing for a live market. And the cost stopped growing: the twelfth country costs what the second cost, whereas with the chain each one is added to a longer function that more people depend on.”

“What is the difference between Strategy and State?” “Structurally nothing — an interface, several implementations, a context holding one. The difference is who changes it. With Strategy the client picks, and it usually stays picked. With State the object transitions *itself* — a `Draft` order moves itself to `Submitted` — so the implementations know about each other and about the context. If my pricers started setting the next pricer, I would have accidentally written State.”

“And Template Method?” “Same goal, different mechanism. Template Method puts the skeleton in an abstract base class and lets subclasses fill in the holes — inheritance. Strategy passes the varying part in as an object — composition. I default to Strategy: I can change it at run time, hold several at once, and test each one without instantiating a subclass of anything.”

“Would you really write classes for this in Python?” “Often not. A dictionary mapping a country to a function is the same pattern with none of the ceremony — twelve lines against about ninety. I would move to classes when a rule needs state, or has more than one method, or genuinely deserves its own file and test suite because a particular team owns it. And `sorted(key=...)` is the proof that the function version is idiomatic: the whole standard library takes algorithms as arguments.”

“When would you leave the `if` chain alone?” “Four cases. When the set is closed — weekdays, blood groups. When the variation is data rather than behaviour — three tax rates are a dictionary of numbers, and turning them into three classes is forty lines to hold three literals. When it is the first occurrence, because I cannot see where the seam goes until something has actually changed. And when the code is finished — if `git log` shows two commits in three years, I would spend the effort somewhere else.”

A model answer

Asked: *the pricing rule changes per country. How do you structure that?*

“First the axis, because that decides everything else. The thing that keeps changing is the pricing rule per market, and a new market arrives every quarter or so. So that is where I want a seam.

What is wrong with the `if` chain is not that it is ugly, it is the edit count. Adding a market means editing a shared function that twelve callers depend on, re-running about forty tests, and putting three live markets at risk for a change that has nothing to do with them. The rules are also entangled — a change to the UK rule and a change to the India rule are the same file, the same review, the same merge conflicts — and I cannot test one rule without going through the dispatch.

So: an interface with one method, `price(order) → Decimal`, in my own vocabulary. One implementation per market, each in its own file, each testable directly. `Checkout` holds a `Pricer` and never branches — it does not know how many exist.

The part I would be careful about is that choosing is a separate job from doing. The selection goes in a registry or a factory, not inside `Checkout`. If `Checkout` takes a country and branches on it, the `if` chain is back and I have paid for an interface that is doing nothing.

And I would be honest about what I gained. The branch did not disappear; it moved into a dictionary of constructors. What changed is the blast radius: a wrong line in that dictionary gives an unsupported-country error, whereas a wrong branch inside a shared pricing function can break a live market. And the cost is now flat — the twelfth country is one new file and one line, the same as the second.

In Python I would start with a dictionary mapping country to a function rather than a class hierarchy. That is the same pattern — the varying part is a value I can pass around — at about twelve lines instead of ninety. I would go to classes when a rule needs state, or more than one method, or its own file because finance owns it and wants their own tests.

The one thing I would push back on is doing this too early. If the only difference between the markets is the tax rate, then this is not behaviour that varies, it is a number that varies, and three classes to hold three literals is forty lines buying nothing. The test I use is: does the *code* differ, or only a *value*? If only a value, it is a dictionary. And I would want to be able to name the next implementation before building the seam — before something has changed once, I am guessing where the seam goes, and a seam in the wrong place costs more than no seam at all.”

9 Recall card

- **Strategy turns an algorithm into a value.** One interface, one method, one implementation per way, and the caller never learns how many there are. You have used it since day 058: `sorted(items, key=...)` is Strategy.

- **Argue it in edit counts, not adjectives.** New market with an `if` chain: 1 shared file (12 dependents), ~40 tests re-run, 3 live rules at risk. With strategies: **1 new file + 1 registry line, 0 tests re-run** — and the twelfth costs what the second cost.
- **Choosing is a separate job from doing.** The selection lives in a registry or factory, never in the caller — otherwise the branch is back and the interface is doing nothing. And **concede that the `if` moved rather than vanished**: a wrong line in a list of constructors cannot break Diwali pricing, and a new branch in a shared function can.
- **In Python a function is a strategy.** A `dict[str, Callable]` is ~12 lines against ~90 for the full class version. Take classes when a rule needs **state, several methods**, or **its own owner**. Real ones: compression codecs · load-balancing algorithms · cache eviction policies · retry backoff · `hashlib.new` · Django's `PASSWORD_HASHERS`.
- **The commonest misuse: the variation is *data*, not *behaviour*.** * *Three tax rates as three classes is ~40 lines to hold three literals — the test is* does the code differ, or only a value? *Also leave the chain alone when the set is closed, when it is the first occurrence, or when you cannot name the second implementation.* *Strategy vs State = same structure, but State transitions *itself*; Strategy vs Template Method = composition vs inheritance.

PRACTICE

Practice — Monotonic stack: the next greater element

DSA topic: Monotonic stack: the next greater element **System design topic:** Strategy

Code these, in this order

One rule for the whole set: **before writing the loop, say what the stack holds**. “The stack holds the indices of elements whose answer is not yet known.” If you cannot say that sentence about your stack, you are using it as a scratch space and the code will drift.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Next Greater Element I	LeetCode 496 (Easy)	The base pattern, plus a map from value to answer for the lookup half.
2	Daily Temperatures	LeetCode 739 (Medium)	The distance variant — which is why the stack must hold indices.
3	Next Greater Element II	LeetCode 503 (Medium)	Circular: <code>2n</code> steps, and pushing only on the first pass.
4	Online Stock Span	LeetCode 901 (Medium)	Previous-greater, arriving one element at a time rather than as an array.

On problem 2, try it with values first

Write the stack holding values instead of indices. Get to the line where you need `index - previous` and find you cannot write it. Then switch to indices and say out loud why this is the default.

On problem 3, break the second pass on purpose

Push on both passes. Run `[1, 2, 1]` and see what comes out. Then add `if step < n` and run it again. Say in one sentence why the second pass must only answer, never ask.

On problem 4, notice what changed

The elements arrive one at a time and you must answer immediately. Say what that does to the stack — and what it does not do to the counting argument.

The complexity-defence drill

Say this out loud, five times, until it is automatic:

1. How many pushes happen across the whole run?
2. How many pops can happen across the whole run, and why?
3. What is the total number of stack operations?
4. What do you say to “there is a loop inside a loop, so it is $O(n^2)$ ”?
5. Give an input where one iteration pops $n - 1$ elements, and explain why that is fine.

The four-variants drill

For `[4, 2, 5, 1, 3]`, compute all four by hand, then write all four and check:

1. Next greater to the right.
2. Next smaller to the right.
3. Previous greater to the left.
4. Previous smaller to the left.

Then, for each, say which direction you walked and which way the comparison pointed, and what direction the stack ended up ordered in.

The strictness drill

1. Run `next_greater([2, 2, 3])` with `<`. Write down the answer.
2. Run it with `≤`. Write down the answer.
3. Explain the difference in one sentence.
4. For each of these phrasings, say whether you would use `<` or `≤`: - “the next greater element” - “the next day that is at least as warm” - “the first later element strictly larger” - “the span of days up to and including today with price $\leq$ today’s”
5. Say what question you would ask the interviewer if the phrasing were ambiguous.

The break-it drill

Trigger each and say what happens:

1. Remove `stack` and from the `while` condition. Quote the exact error and the input that causes it.
2. Push before popping instead of after. Run `[2, 2, 3]` with `≤` and explain the output.
3. Store values instead of indices, then try to write Daily Temperatures.
4. Build the answer by appending as you pop, instead of pre-filling and writing by index. Run `[5, 4, 3]` and count the length of your output.
5. Push on both passes of the circular version.
6. Use `>` when you wanted next-greater. Run `[4, 2, 5, 1]` and say which variant you accidentally wrote.

The recognition drill

For each, say whether it is a monotonic-stack problem, and if so which variant:

1. For each day, how many days until a warmer one.
2. For each element, the nearest smaller element to its left.
3. The k most frequent elements.
4. The largest rectangle in a histogram.
5. How much rain water is trapped between the bars.
6. Whether the brackets are balanced.

7. For each stock price, how many consecutive earlier days had a price $\leq$ today's.
8. The minimum of the stack in $O(1)$.
9. Remove k digits to make the smallest possible number.
10. The next greater element in a circular array.

Two of the ten are stack problems but *not* monotonic-stack problems. Name them and say what the difference is.

The strategy-refactor drill

Here is the function:

```
def price(order: Order, country: str) → Decimal:
    if country == "IN":
        base = order.subtotal * Decimal("1.18")
        if order.subtotal > 5000:
            base -= Decimal("100")
        return base
    elif country == "AE":
        return order.subtotal * Decimal("1.05")
    elif country == "GB":
        return order.subtotal * Decimal("1.20") - order.loyalty_credit
    raise ValueError(country)
```

1. Name the axis of change, in three words.
2. List four specific things wrong with this, none of which is "it is ugly".
3. Write the protocol.
4. Write the three implementations.
5. Write the selection, and say why it must not live in `Checkout`.
6. Write the same thing as a dictionary of functions. Count the lines both ways.
7. Add a fourth market to each version. Count files edited and tests re-run.
8. Write the test for the UK rule, before and after. Count the lines.
9. Say what you gained, honestly, including what did *not* happen to the `if`.

The do-not-do-it drill

For each, say whether you would use Strategy, and give the deciding question:

1. Three countries whose only difference is the tax rate.
2. Three countries with genuinely different rule shapes.
3. Two sorting orders for a report.
4. Seven weekdays with different opening hours.
5. Compression: gzip versus zstd versus lz4.
6. A retry policy each customer configures for themselves.
7. The first time a second variant has appeared.
8. Three payment providers with different SDKs.

One of the eight is really an adapter question, not a strategy question. Name it and say why. Two are “no” for the same reason — name it.

The look-alikes drill

1. Write a small State machine — a `Draft` order that moves itself to `Submitted`, which moves itself to `Shipped`.
2. Write the same thing as Strategy, with the client choosing.
3. Say which one you wrote by looking only at the implementations. What is the tell?
4. Write a Template Method version of “load, transform, save” where the transform varies.
5. Rewrite it as Strategy.
6. Name three things the Strategy version can do that the Template Method version cannot.

The already-used-it drill

For each, say what the strategy is and what the interface is:

1. `sorted(items, key=lambda x: x.age)`
2. `hashlib.new("sha256")`
3. nginx's `least_conn`;
4. `functools.lru_cache(maxsize=128)`
5. `tenacity.retry(wait=wait_exponential())`
6. Django's `PASSWORD_HASHERS` setting
7. `defaultdict(list)`

One of the seven is a decorator rather than a strategy. Name it and say why the distinction holds.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *For every element, find the next greater element to its right.* The brute force with a number, the observation that popping is answering, what the stack holds, indices-not-values with the reason, and the complexity defence pre-empted rather than defended.
2. *The pricing rule changes per country. How do you structure that?* The axis named first, four specific problems with the chain in edit counts, the interface, the selection kept separate and why, the Python function version with its line count, and the honest concession about where the `if` went.
3. *There is a loop inside a loop. Is that not $O(n^2)$?* n pushes total, each element popped at most once, so $\leq 2n$ operations — bounded in total, not per iteration — and the example where one iteration pops $n - 1$ and why it costs nothing.

Before you move on

- [] I can say what the stack holds, in one sentence, without hesitating.
- [] I can give the complexity defence confidently, without softening it.
- [] I wrote all four variants and can generate any of them from the two knobs.
- [] I know what `<` versus `≤` does on `[2, 2, 3]` and which phrasings need which.
- [] I tried storing values and hit the wall on Daily Temperatures.
- [] I broke the circular version by pushing on both passes and saw the wrong output.
- [] I can name two stack problems that are not monotonic-stack problems.
- [] I refactored the pricing function and counted files edited for a fourth market, both ways.
- [] I wrote the dictionary-of-functions version and can quote both line counts.
- [] I can say the honest sentence about where the `if` actually went.
- [] I can name the test that separates “behaviour differs” from “only a value differs”.
- [] I answered all three questions above out loud.

Largest rectangle in a histogram

DATA STRUCTURES AND ALGORITHMS

Largest rectangle in a histogram

You can solve the hardest classic monotonic-stack problem and explain each pop.

SYSTEM DESIGN

Observer

You can notify many listeners of one event without coupling them to the source.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the largest rectangle in the histogram.*
- *When an order is placed, five things must happen. Design that.*

1 What this is, and why they ask it

You are given a row of bars standing side by side, each one unit wide, with heights like `[2, 1, 5, 6, 2, 3]`. Find the largest rectangle you can draw that fits entirely inside the bars. The rectangle can span several bars, but it can only be as tall as the shortest bar it spans.

The whole problem collapses to one sentence: **for every bar, find the widest stretch in which that bar is the shortest one**. The area for that bar is its height times that width, and the answer is the biggest of those. Once you believe that sentence, the code is the monotonic stack from [day 071](#) with one extra line of arithmetic.

They ask it because it is the hardest thing you can build out of a single stack, and because the brute force is so obviously quadratic that there is nowhere to hide. It is also the engine behind “maximal rectangle in a binary matrix”, which is a common follow-up in the same round. Expect it at the harder end of a coding round — Google, Amazon, Microsoft, and most product companies that ask LeetCode-hard questions. The off-by-one in the width is where most candidates lose it, so the width formula is worth memorising as a sentence, not as a symbol.

2 The story

Basheer paints banners in a market street, and every year before the festival the shopkeepers' committee gives him the same job: put up the biggest cloth banner he can across the front of the shops.

There are eight shops in a row on that side of the street, and they are all different heights. Two of them are proper two-storey buildings. One is a tiny box of a shop with a roof you can touch standing flat on the road. The rest are somewhere in between.

The banner is a single piece of cloth, one rectangle, and it has to hang flat. The bottom edge sits at road level. The top edge has to be one straight line all the way across. And every shop the banner covers has to be tall enough to take a nail at that height, because Basheer will not tie a corner to nothing.

So he has a choice to make. He can hang it low and wide, running past the tiny shop and covering all eight fronts. Or he can hang it high, up at the level of the two-storey buildings, but then it covers only those two, because the moment he reaches a lower roof there is nowhere to fix the top edge.

He walks the street once, slowly, with a rolled-up measuring tape in his hand. At each shop he asks himself the same question: if the top edge were exactly at this roof, how far left and how far right could the cloth run before it met a shorter roof? That stretch, times that height, is the size of the banner.

The tiny shop gives him a very wide answer and almost no height. The tall one gives him a tall answer and almost no width. Somewhere in the middle, four shops of roughly the same height standing together, gives him the biggest piece of cloth he has ever hung on that street.

He does not measure every possible stretch, and this is the part his son never understands. He asks the question eight times, once per roof, and stops. Because whatever the biggest banner turns out to be, its top edge is level with somebody's roof. It has to be. If the top edge were below every roof it touched, he could have raised it a little and got a bigger banner.

3 The idea in plain English

The bars are the shop fronts. The rectangle is the banner. And Basheer's last paragraph is the whole solution.

The claim that makes it finite

There are infinitely many rectangles you could draw, because the height can be any number. Basheer's observation removes all but `n` of them:

The best rectangle's height is equal to the height of some bar.

Why: take any rectangle that fits. If its top edge is strictly below every bar it spans, push the top edge upward until it touches the lowest of those bars. It still fits, and it is now taller, so it is bigger. So the best rectangle always has its top edge resting on at least one bar — and that bar is the shortest bar in its span.

That turns “try all rectangles” into “try `n` rectangles”: one per bar, where that bar is the shortest one in the span.

The question, per bar

For bar `i` with height `heights[i]`, you want the widest span in which `heights[i]` is the minimum. Walk left from `i` until you meet a bar shorter than `heights[i]`; walk right until you meet a bar shorter than `heights[i]`. Everything strictly between those two shorter bars is your span.

So you need two numbers for every bar:

- `left[i]` — the index of the nearest bar to the left that is **strictly shorter**. If there is none, use `-1`, meaning “the wall at the start of the street”.
- `right[i]` — the index of the nearest bar to the right that is **strictly shorter**. If there is none, use `n`, meaning “the wall past the end”.

Then:

```
width = right[i] - left[i] - 1
area  = heights[i] * width
```

The `- 1` is because `left[i]` and `right[i]` are the shorter bars themselves, which are *not* part of the span. Say it as a sentence and you will never get it wrong: **the span is everything strictly between the two shorter bars, so its width is the gap between them minus one.**

Sanity check on `heights = [2, 1, 5, 6, 2, 3]`, for the bar of height 5 at index 2. To its left, the nearest strictly shorter bar is the 1 at index 1. To its right, the nearest strictly shorter bar is the 2 at index 4. Width is `4 - 1 - 1 = 2`, covering indices 2 and 3. Area is `5 × 2 = 10`.

And you already know how to find those two numbers

“Nearest strictly smaller element to the left” and “nearest strictly smaller element to the right” are two of the four variants from [day 071](#). Each is one pass of a monotonic stack. So a perfectly good solution is: one pass for `left`, one pass for `right`, one pass for the arithmetic. Three passes, $O(n)$, and very easy to explain.

The famous version does it in **one** pass, and here is the trick that makes it possible.

The one-pass insight

In the next-smaller pass, the moment a bar gets popped is the moment you have found its right-hand boundary — the current bar is the first shorter thing to its right. And at that instant, whatever is left underneath it in the stack is its left-hand boundary, because the stack holds bars in increasing order of height. So both boundaries are available at the moment of the pop.

That is the sentence to say out loud: **when a bar is popped, both of its boundaries are known at once — the current index on the right, and the new top of the stack on the left.**

So the pop does all the work:

```
height = heights[stack.pop()]
left = stack[-1] if stack else -1      # nearest shorter on the left
width = index - left - 1               # 'index' is the shorter one on the right
best = max(best, height * width)
```

Read `left = stack[-1] if stack else -1` carefully. After the pop, the new top of the stack is a bar that was **not** popped, which means it is not taller than the current bar — and since the stack is increasing, it is shorter than the bar we just popped. If the stack is empty, nothing to the left is shorter, so the span runs back to the start of the row, and `-1` makes the arithmetic come out right.

The sentinel

When the loop over the real bars ends, some bars are still in the stack — the ones that never met anything shorter on their right. They still need their areas computed, with `right = n`.

You can write a second loop for that. Or you can add one imaginary bar of height 0 at the end, which is shorter than everything and therefore forces every remaining bar out of the stack with the correct right boundary. That is a **sentinel** — a fake value added to the data so the main loop handles a case that would otherwise need special code. You have seen one before, in the `prefix[0] = 1` of [day 038](#).

One extra element removes an entire loop. Say that you are doing it, and why, or the interviewer will think you forgot the leftovers.

4 The picture

`heights = [2, 1, 5, 6, 2, 3]`. Indices above, heights below, and the bars drawn.



The best rectangle is height 5 across indices 2 and 3 — area 10. Notice it is **not** the tallest bar alone ($6 \times 1 = 6$), and **not** the full width at the lowest height ($1 \times 6 = 6$). It is a bar in the middle, extended sideways as far as it can go.

Now the one-pass run, with a sentinel 0 appended at index 6. The stack holds indices, heights increasing from bottom to top.

```

i=0 h=2  stack empty                push 0      stack: [0]      (2)
i=1 h=1  top is 2 > 1 → pop 0
          height 2, left = -1, width = 1 - (-1) - 1 = 1, area 2
          stack empty                push 1      stack: [1]      (1)
i=2 h=5  top is 1, not > 5           push 2      stack: [1,2]    (1,5)
i=3 h=6  top is 5, not > 6           push 3      stack: [1,2,3]  (1,5,6)
i=4 h=2  top is 6 > 2 → pop 3
          height 6, left = 2, width = 4 - 2 - 1 = 1, area 6
          top is 5 > 2 → pop 2
          height 5, left = 1, width = 4 - 1 - 1 = 2, area 10 ← best
          top is 1, not > 2           push 4      stack: [1,4]    (1,2)
i=5 h=3  top is 2, not > 3           push 5      stack: [1,4,5]  (1,2,3)
i=6 h=0  top is 3 > 0 → pop 5
          height 3, left = 4, width = 6 - 4 - 1 = 1, area 3
          top is 2 > 0 → pop 4
          height 2, left = 1, width = 6 - 1 - 1 = 4, area 8
          top is 1 > 0 → pop 1
          height 1, left = -1, width = 6 - (-1) - 1 = 6, area 6
          stack empty                push 6

```

answer: 10

Three things to notice.

At `i=4`, **one arrival closed two bars**, and the second of them produced the winner. This is the same “one element answers many” behaviour as day 071, and it is why the inner `while` does not make this quadratic.

At `i=6`, the sentinel emptied the stack. Without it, bars 1, 4 and 5 would never have been measured, and the answer would have come out as 6 instead of 10.

And look at the width for the bar of height 2 at index 4: it is 4, not 2. When bar 4 is finally popped, everything to its right that was taller has already gone, and the span runs from index 2 to index 5. A bar can be the shortest bar over a stretch that includes bars that were pushed after it.

5 The code, built step by step

Step 1 — the brute force, said out loud and rejected

“For every pair of start and end, find the minimum height in that stretch and multiply.” That is $n(n+1)/2$ stretches, each needing a scan, so $O(n^3)$. At $n = 1000$ that is about 500 million operations. Say the number, then improve it once before improving it properly.

The easy improvement: fix the left end, walk right, and keep a running minimum.

```
def largest_rectangle_brute(heights: list[int]) → int:
    best = 0
    for start in range(len(heights)):
        smallest = heights[start]
        for end in range(start, len(heights)):
            smallest = min(smallest, heights[end])    # running minimum
            best = max(best, smallest * (end - start + 1))
    return best
```

That is $O(n^2)$ — at $n = 100,000$, ten billion operations. Correct, and far too slow. Use it as the reference implementation to check the fast one against.

Step 2 — the two-array version, which is the one to explain first

If you are nervous, write this one. It is three simple passes and each pass is something you already know.

```
n = len(heights)
left = [-1] * n                # nearest strictly shorter bar to the left
stack: list[int] = []
for i in range(n):
    while stack and heights[stack[-1]] ≥ heights[i]:
        stack.pop()
    left[i] = stack[-1] if stack else -1
    stack.append(i)
```

This is “previous smaller element”, written so that the answer is the *index* rather than the value. Note `>=` : bars of equal height pop each other, so `left[i]` is the nearest **strictly** shorter bar.

```
right = [n] * n          # nearest strictly shorter bar to the right
stack = []
for i in range(n - 1, -1, -1): # walk backwards
    while stack and heights[stack[-1]] >= heights[i]:
        stack.pop()
    right[i] = stack[-1] if stack else n
    stack.append(i)
```

The same loop walking the other way. `n` is the “wall past the end” default, matching `-1` at the start.

```
return max(
    (heights[i] * (right[i] - left[i] - 1) for i in range(n)),
    default=0,
)
```

One line of arithmetic per bar. `default=0` covers the empty input, where `max` of an empty sequence would otherwise raise.

Step 3 — collapsing it to one pass

The two-array version computes `right[i]` in its own backwards pass. But the forward pass already discovers `right[i]` — it is exactly the index that causes `i` to be popped. So do the arithmetic at the moment of the pop instead of storing it.

```
for index in range(n + 1):
    current = heights[index] if index < n else 0          # sentinel 0 at the end
    while stack and heights[stack[-1]] > current:
        height = heights[stack.pop()]
        left = stack[-1] if stack else -1
        best = max(best, height * (index - left - 1))
    stack.append(index)
```

Five lines carry the whole algorithm. `index` is the right boundary because it is the first bar strictly shorter than the popped one. `left` is the left boundary because the stack is increasing, so whatever survived underneath is shorter still.

Step 4 — why `>` and not `>=` here, and why equal bars are still safe

The pop condition is `heights[stack[-1]] > current`, so equal-height bars do **not** pop each other. That means the bar underneath a popped bar might be exactly the same height rather than strictly shorter, and the width computed for the upper one is too small.

That is fine, and here is why: the lower of the two equal bars is still in the stack, and when *it* is popped it gets the full span — the same height, a bigger width, a bigger area. So the correct answer is still found; only a duplicate, smaller measurement was thrown in along the way, and `max` ignores it.

Check it on `[2, 2]`. Bar 1 is popped by the sentinel with `left = 0`, width 1, area 2. Then bar 0 is popped with `left = -1`, width 2, area 4. The answer is 4.

This is worth saying out loud in the interview, because “what about equal heights?” is the first follow-up and most candidates have not thought about it.

Step 5 — the escalation: maximal rectangle in a binary matrix

Given a grid of `0`s and `1`s, find the largest rectangle made only of `1`s. Treat each row as the ground line of a histogram: the height of column `c` is how many consecutive `1`s sit directly above and including row `r`. Then run the histogram solver on every row.

```
heights = [0] * len(matrix[0])
for row in matrix:
    for c, cell in enumerate(row):
        heights[c] = heights[c] + 1 if cell == "1" else 0    # reset on a zero
    best = max(best, largest_rectangle(heights))
```

Building each row's heights is $O(\text{columns})$, and each histogram solve is $O(\text{columns})$, so the whole thing is $O(\text{rows} \times \text{columns})$. Say that, because it sounds expensive and is not.

The complete solution

```
def largest_rectangle(heights: list[int]) → int:
    """The area of the largest rectangle that fits inside the histogram.

    For every bar, find the widest span in which that bar is the shortest, and
    take the best area. The stack holds INDICES of bars whose right-hand
    boundary is not yet known, with heights increasing from bottom to top.

    When a bar is popped, both of its boundaries are known at that instant:
    the current index on the right (the first strictly shorter bar), and the
    new top of the stack on the left. A sentinel height of 0 at the end forces
    every remaining bar out with the right boundary set past the end.
    """
    n = len(heights)
    stack: list[int] = [] # indices, heights increasing bottom→top
    best = 0

    for index in range(n + 1):
        current = heights[index] if index < n else 0 # the sentinel

        while stack and heights[stack[-1]] > current:
            height = heights[stack.pop()]
            left = stack[-1] if stack else -1 # -1 = the wall at the start
            width = index - left - 1 # strictly between the two
            best = max(best, height * width)

        stack.append(index)

    return best

def largest_rectangle_two_pass(heights: list[int]) → int:
    """The same answer, computed with explicit boundary arrays.

    Slower by a constant factor and easier to explain. Write this one if the
    one-pass width formula is not solid in your head under pressure.
    """
    n = len(heights)
    if n == 0:
        return 0

    left = [-1] * n # nearest strictly shorter to the left
    stack: list[int] = []
    for i in range(n):
        while stack and heights[stack[-1]] ≥ heights[i]:
            stack.pop()
        left[i] = stack[-1] if stack else -1
        stack.append(i)

    right = [n] * n # nearest strictly shorter to the right
    stack = []
    for i in range(n - 1, -1, -1):
        while stack and heights[stack[-1]] ≥ heights[i]:
            stack.pop()
        right[i] = stack[-1] if stack else n
        stack.append(i)

    return max(heights[i] * (right[i] - left[i] - 1) for i in range(n))

def largest_rectangle_brute(heights: list[int]) → int:
    """O(n^2) reference. Fix the left end, walk right, keep a running minimum."""
    best = 0
    for start in range(len(heights)):
        smallest = heights[start]
        for end in range(start, len(heights)):
            smallest = min(smallest, heights[end])
            best = max(best, smallest * (end - start + 1))
    return best

def maximal_rectangle(matrix: list[list[str]]) → int:
    """Largest rectangle of '1's in a binary grid.
```

```

Each row becomes the ground line of a histogram whose column heights count
the consecutive '1's directly above. A '0' resets that column to zero.
"""
if not matrix or not matrix[0]:
    return 0

heights = [0] * len(matrix[0])
best = 0
for row in matrix:
    for column, cell in enumerate(row):
        heights[column] = heights[column] + 1 if cell == "1" else 0
    best = max(best, largest_rectangle(heights))
return best

if __name__ == "__main__":
    print(largest_rectangle([2, 1, 5, 6, 2, 3])) # 10
    print(largest_rectangle([2, 4])) # 4
    print(largest_rectangle([2, 2])) # 4
    print(largest_rectangle([5])) # 5
    print(largest_rectangle([])) # 0
    print(largest_rectangle([0, 0, 0])) # 0
    print(largest_rectangle([1, 2, 3, 4, 5])) # 9 (3+4+5 at height 3)
    print(largest_rectangle([5, 4, 3, 2, 1])) # 9
    print(largest_rectangle([3, 3, 3, 3])) # 12

# the two implementations must agree, on every shape
import random
for _ in range(2000):
    sample = [random.randint(0, 9) for _ in range(random.randint(0, 12))]
    assert largest_rectangle(sample) == largest_rectangle_brute(sample), sample
    assert largest_rectangle_two_pass(sample) == largest_rectangle_brute(sample), sample
print("agreed on 2000 random inputs")

grid = [
    ["1", "0", "1", "0", "0"],
    ["1", "0", "1", "1", "1"],
    ["1", "1", "1", "1", "1"],
    ["1", "0", "0", "1", "0"],
]
print(maximal_rectangle(grid)) # 6

```

The random cross-check at the bottom is worth writing in the interview too, if there is time. “Let me check the fast one against the obvious one on small random inputs” is a sentence that buys a lot of trust, and it catches the width off-by-one immediately.

6 What it costs

Time

The `for` loop runs $n + 1$ times, so there are exactly $n + 1$ pushes — every index goes on once, including the sentinel.

The `while` loop only pops. An index that has been popped never goes back on the stack. So across the whole run, the total number of pops is at most $n + 1$.

```

pushes across the whole run: exactly  n + 1
pops   across the whole run: at most  n + 1
-----
total stack operations:      ≤ 2n + 2  →  O(n)

```

Each pop does three arithmetic operations and one comparison, all constant. So the whole thing is $O(n)$.

It is the same counting argument as day 071 and it needs saying with the same confidence: the inner loop is bounded **in total**, not per iteration. At `i=4` in the trace above, one iteration popped two bars; a single iteration could pop a hundred thousand, and it would still be linear overall, because those pops were paid for by pushes that already happened.

Against the alternatives

```
all pairs, minimum rescanned:   $O(n^3)$ 
all pairs, running minimum:     $O(n^2)$ 
monotonic stack:                $O(n)$ 

n = 100,000
 $O(n^2)$ : 10,000,000,000 operations  - minutes
 $O(n)$   :      200,000 operations   - a few milliseconds
```

Fifty thousand times fewer operations. That is the difference between a submission that times out and one that does not.

Space

```
stack:   $O(n)$   worst case
output:  $O(1)$   - a single integer
```

The stack holds every bar at once when the heights are strictly increasing, like `[1, 2, 3, 4, 5]` — nothing is ever popped until the sentinel arrives. On strictly decreasing heights it never holds more than two, because each bar pops the one before it. Name both inputs; it takes five seconds.

The two-array version uses $O(n)$ for `left`, $O(n)$ for `right` and $O(n)$ for the stack — three arrays instead of one. Same complexity class, three times the memory, and it is a reasonable trade if it means you get the width right.

Maximal rectangle in a grid

`rows × columns` to build the height arrays, plus one $O(\text{columns})$ histogram solve per row, so $O(\text{rows} \times \text{columns})$ total, with $O(\text{columns})$ extra space. For a 200×200 grid that is 40,000 operations. Say the number — it sounds like it should be much worse than it is.

7 The traps

Trap 1 — the width off-by-one

This is the one that costs people the question.

```
width = index - left
```

```
# WRONG, misses the - 1
```

On `[2, 1, 5, 6, 2, 3]` this returns `15` instead of `10`, because every span is measured one bar too wide. It is not obviously wrong — the answer is a plausible number and the code runs clean.

The fix is to say the sentence rather than trust the symbols: **left and index are the shorter bars on either side, and neither of them is inside the span**. The span is what is strictly between them, so its width is `index - left - 1`.

Check it once by hand on a two-bar case. For `[5]` with the sentinel, the bar is popped at `index = 1` with `left = -1`, so `width = 1 - (-1) - 1 = 1`, area 5. Correct.

Trap 2 — using the popped index as the left boundary

```
popped = stack.pop()
width = index - popped           # WRONG
```

This measures from the popped bar itself rather than from the bar to its left, so it can never look backwards past its own position. The popped bar's span usually extends to the **left** of it as well — look at bar 4 in the trace, whose span runs from index 2 even though it sits at index 4.

What makes this one dangerous is that it gives the right answer on the sample input. On `[2, 1, 5, 6, 2, 3]` it returns `10`, exactly like the correct version. The smallest input that exposes it is `[4, 3]`, where the answer is `6` — height 3 across both bars — and this version returns `4`. Always test a two-element input where the first bar is taller.

Trap 3 — reading `stack[-1]` when the stack is empty

```
left = stack[-1]                 # no `if stack else -1`
```

```
IndexError: list index out of range
```

Fires the first time a bar is popped that has nothing shorter to its left — on `[2, 1, 5, 6, 2, 3]` that is index 0, popped at `index = 1`, on the second iteration. Every stack problem in this phase has the same guard, and here it appears twice: once in the `while` condition and once in the `left` lookup.

Trap 4 — forgetting the leftovers

```
for index in range(n):           # no sentinel, no flush loop
    current = heights[index]
    ...
```

The bars still in the stack when the loop ends are never measured. On `[2, 2]` this returns `0` instead of `4`, because neither bar ever pops the other. On a strictly increasing input like `[1, 2, 3, 4, 5]` every bar is still in the stack at the end, so it returns `0` as well.

It is worth running this one to see how quietly it fails: on `[2, 1, 5, 6, 2, 3]` it happens to return the correct `10`, because the bar that wins is popped in the middle of the walk. A missing flush is invisible on exactly the inputs people test with.

Two fixes and both are fine: iterate to `n + 1` with a sentinel of `0`, or add a second loop after the first that drains the stack with `index = n`. The sentinel is fewer lines and fewer chances to get the right boundary wrong.

If the problem allowed negative heights, a sentinel of `0` would not be shorter than everything. It does not — heights are non-negative — but say so, because noticing the assumption is the point.

Trap 5 — `>=` in the one-pass version, without thinking

```
while stack and heights[stack[-1]] >= current:
```

This also produces the right answer, but for a different reason, and if you cannot say the reason you should not write it. With `>=`, equal bars pop each other, so a popped bar's right boundary may be an equal bar rather than a strictly shorter one, and the width is too small for the earlier duplicates — but the *last* bar of an equal run gets the full span. Under `>`, the situation is mirrored: the *first* bar of the run gets the full span.

Either way the maximum survives. Pick one, and be able to say which duplicate carries the real answer.

Trap 6 — the empty input

```
return max(heights[i] * (right[i] - left[i] - 1) for i in range(n))
```

```
ValueError: max() arg is an empty sequence
```

Fires on `largest_rectangle_two_pass([])`. The one-pass version does not have this problem, because `best` starts at `0` and is simply never updated. Guard the two-array version with an early `if n == 0: return 0`, or pass `default=0` to `max`.

Trap 7 — assuming the answer contains the tallest bar

It very often does not. On `[2, 1, 5, 6, 2, 3]` the tallest bar is 6 and the answer is a rectangle of height 5. On `[1, 1, 1, 1, 1, 1, 1, 1, 1, 9]` the answer is `10` — the flat stretch of ten bars at height 1, not the 9 standing alone. Any greedy idea that starts from the tallest bar and grows outward is wrong, and it is worth having that counter-example ready, because “why not just start from the maximum?” is a question interviewers ask to see if you will defend the approach.

8 In the interview

How it gets asked

- The direct version: “Given an array of heights where each bar has width 1, find the area of the largest rectangle in the histogram.” LeetCode 84, Hard.
- The matrix version, often as the follow-up ten minutes later: “Given a binary matrix, find the largest rectangle containing only 1s.” LeetCode 85, Hard.
- The disguised version: “Here is the profile of a city skyline. What is the biggest billboard you can mount flat against it?” Same problem, different words.
- The softer sibling, sometimes asked first as a warm-up: “how much rain water is trapped between these bars?” LeetCode 42 — same stack, different bookkeeping.

What to say out loud, in the first ninety seconds

1. **Reduce the problem before writing anything.** “The best rectangle’s top edge must be level with some bar. If it were below every bar it spans, I could raise it and get a bigger one. So there are only n candidates, not infinitely many — one per bar.”
2. **State the per-bar question.** “For each bar, I want the widest stretch where that bar is the shortest. That means: how far left until something shorter, and how far right until something shorter.”
3. **Name what that is.** “Those two are previous-smaller and next-smaller, which are monotonic-stack passes. So a clean solution is three passes: left boundaries, right boundaries, then the arithmetic.”
4. **Then offer the one-pass version.** “I can fold it into one pass. In a next-smaller pass, the moment a bar is popped, the current index is its right boundary, and the new top of the stack is its left boundary. So I compute the area at the moment of the pop.”
5. **State the width formula as a sentence.** “Width is `index - left - 1`, because both boundaries are the shorter bars themselves and neither is inside the span.”
6. **Pre-empt the complexity question.** “One pass, each index pushed once and popped at most once, so $O(n)$ time and $O(n)$ space.”

Write the width formula down before you write the loop. Under pressure it is the thing that goes.

The follow-ups

“What about bars of equal height?” “They are safe, but for a reason worth stating. With a strict `>` in the pop condition, equal bars do not pop each other, so when the upper one of a pair is popped its left boundary is an equal bar rather than a shorter one, and its width comes out too small. But the lower one is still in the stack, and when it is popped it gets the full span at the same height. So the correct area is still measured, and the too-small one is discarded by the `max`.”

“How do you handle the bars left in the stack at the end?” “I append a sentinel bar of height zero. It is shorter than every real bar, so it forces the whole stack out with the right boundary set to `n`. It removes an entire drain loop. It works because heights are non-negative — if they could be negative, I would use minus infinity or write the loop.”

“Now solve maximal rectangle in a binary matrix.” “Treat each row as the ground of a histogram. The height of a column is the number of consecutive 1s directly above and including that row; a 0 resets it to zero. Then run this solver once per row and take the best. Building the heights is $O(\text{columns})$ per row and the solve is $O(\text{columns})$ per row, so the whole thing is $O(\text{rows} \times \text{columns})$ — no worse than reading the input.”

“Is a loop inside a loop not quadratic?” “No. There are exactly $n + 1$ pushes, and an index once popped never returns, so there are at most $n + 1$ pops. Total stack operations are bounded by about $2n$ across the whole run, however unevenly they are spread. One iteration might pop a hundred thousand bars; that iteration has used up all the pops the rest of the run was going to do.”

“Can you do it in $O(1)$ extra space?” “Not with this approach, and I would not claim to. There is a divide-and-conquer solution that recurses on the minimum bar, which is $O(n \log n)$ on average and $O(n^2)$ in the worst case — worse in time and it still uses stack depth. For this problem the monotonic stack at $O(n)$ time and $O(n)$ space is the right answer, and the $O(n)$ space is a genuine requirement, not laziness.”

A model answer

Asked: *find the largest rectangle in a histogram given* `[2, 1, 5, 6, 2, 3]`.

“First let me cut the problem down, because as stated there are infinitely many rectangles — the height can be anything.

Claim: the best rectangle’s top edge is level with the top of some bar. Suppose it were strictly below every bar it spans. Then I could push the top edge up until it touched the lowest of those bars, and it would still fit and be bigger. So the best rectangle always rests on at least one bar, and that bar is the shortest one in its span. That gives me exactly n candidates: for each bar, the widest stretch in which that bar is the minimum.

So for each bar I need two things: how far left I can go before hitting something shorter, and how far right. Those are ‘previous smaller element’ and ‘next smaller element’, and each one is a monotonic stack pass. Once I have them, the area for bar i is $\text{heights}[i] \times (\text{right}[i] - \text{left}[i] - 1)$. The minus one is because `left` and `right` are the shorter bars themselves, and neither of them is inside the span.

That is a perfectly good three-pass $O(n)$ solution and I am happy to write it. But I can fold it into one pass, and I think that is what you are looking for.

In the next-smaller pass, the moment a bar gets popped is exactly the moment I have found its right boundary — the current bar is the first shorter thing to its right. And at that same instant, the new top of the stack is its left boundary, because the stack is increasing by height, so whatever is underneath a popped bar is shorter than it. Both boundaries, at the moment of the pop. So I compute the area right there and never store the arrays.

The stack holds indices, not heights, because I need positions to compute widths. If the stack is empty after popping, there is nothing shorter to the left at all, so I use minus one as the boundary and the arithmetic still comes out right.

For the bars that never meet anything shorter on their right, I append one sentinel bar of height zero. It is shorter than everything, so it drains the stack with the right boundary set past the end, and I do not need a separate cleanup loop.

On `[2, 1, 5, 6, 2, 3]` the answer is 10 — height 5 across indices 2 and 3. Worth noticing that it is neither the tallest bar alone, which gives 6, nor the full width at the lowest height, which also gives 6.

Complexity: every index is pushed exactly once and popped at most once, so at most about $2n$ stack operations across the whole run. $O(n)$ time. $O(n)$ space for the stack, which is genuinely needed — worst case is strictly increasing heights, where nothing pops until the sentinel arrives.

One thing I would want to confirm: heights are non-negative, yes? That is what makes a zero sentinel safe. And if there are equal heights, the code is still correct — the lower bar of an equal pair picks up the full span when it is popped.”

9 Recall card

- **Kill the infinity first: the best rectangle's top edge is level with some bar** — otherwise you could raise it. So there are exactly n candidates, one per bar: **the widest stretch in which that bar is the shortest**.
- **Per bar you need two boundaries — nearest strictly shorter on the left and on the right** — and both are **monotonic-stack passes**. `width = right - left - 1`, because both boundaries are the shorter bars themselves and neither is inside the span. That `- 1` is where the question is lost.
- **One pass, because the pop knows both boundaries at once**: the current index is the right boundary, `stack[-1] if stack else -1` is the left. Stack holds **indices**, heights **increasing** bottom to top. Five lines: `pop`, `left`, `width`, `max`, `push`.
- **A sentinel height of 0 at the end drains the leftovers** with the right boundary at `n`, and removes the whole cleanup loop. Without it, `[1, 2, 3, 4, 5]` returns 0. Safe only because heights are non-negative.
- **$O(n)$ time** — `n + 1` pushes, each index popped at most once, $\leq 2n$ operations, bounded in total not per iteration (10^{10} vs 2×10^5 at $n = 100,000$). **$O(n)$ space** (strictly increasing input). Equal bars are safe: the lower one collects the full span. Escalation: **maximal rectangle in a binary grid** = one histogram per row, **$O(\text{rows} \times \text{columns})$** .

1 What this is, and why they ask it

Observer is the pattern where one object — the **subject** — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call. The objects on the list are the **observers**. The subject knows nothing about them except that each one has a method it can call.

The whole value is in that last sentence. Adding a sixth thing that must happen when an order is placed does not touch the order code at all. You write the new observer and add it to the list.

They ask it because “when X happens, these five other things must happen” is the single most common shape in real backend work, and because the naive version — one function that calls five services in a row — is the single most common source of slow, fragile checkout flows. It also shows up as the explanation for things the candidate has already used without naming: a button click handler, a spreadsheet cell that updates when another cell changes, a chat app that refreshes when a message arrives. Expect it in low-level design rounds and as a follow-up in almost any “design the order flow” question.

2 The story

Meera runs a small bakery two lanes behind the bus stand. The bread comes out of the oven at four in the afternoon, and for about twenty minutes it is still warm, which is the only thing her regulars actually care about.

For years she handled it by remembering. At four she would wipe her hands, pick up her phone, and call four people. Ganesh, who ran the tea stall and bought six loaves every day. The two sisters in the flat above the tailor's. And her cousin, who took whatever was left at seven and sold it out on the highway.

It worked until it did not. A schoolteacher started asking to be told, so that was five calls. Then Ganesh's brother took over the stall and wanted to be called on his own number, so that was six. Then the sisters moved out, and Meera kept calling them for three weeks, because nobody had told her to stop, and the phone kept ringing in an empty flat.

The worse problem was that four o'clock had become the busiest twenty minutes of her day. She was standing there with hot trays in one hand and the phone in the other, and if one call went long — her cousin liked to talk — the bread was cooling while she waited for him to finish.

Her daughter fixed it in an afternoon. She made one group on Meera's phone, and stuck a small card by the counter: if you want to know when the bread is out, leave your number and we will add you. If you stop wanting to know, say so and we will take you off.

Now at four Meera sends one message to the group. It takes her six seconds. She does not know how many people are in it any more — her daughter looks after that — and she does not know what any of them do with the message. Ganesh walks over. The schoolteacher sends her son. Her cousin ignores it until six.

None of that is Meera's business. Her job is the bread and the one message. Whether there are three people on the other end or thirty, four o'clock takes six seconds.

3 The idea in plain English

Meera is the **subject**: the thing that something happens to. The people in the group are the **observers**: the things that want to know. The group itself is the **subscription list**, and the one message is the **notification**.

Four moves make up the pattern, and they map one to one.

One: the subject holds a list. Not of people, of objects. Each entry is something with a known method — call it `update` — that the subject can call. Meera's group is a list of phone numbers, and a phone number is exactly this: a thing you can call, whose owner you do not need to know.

Two: observers add and remove themselves. `attach(observer)` and `detach(observer)`. The schoolteacher walked up and left her number; the sisters should have told Meera to remove them. This is the part that makes the pattern extensible: adding a listener is not a change to the subject's code, it is a change to the subject's *data*.

Three: one call notifies all of them. `notify()` loops the list and calls `update` on each. Six seconds, whether there are three or thirty.

Four — and this is the point — the subject does not know what any observer does. Meera does not know that Ganesh walks over and the schoolteacher sends her son. The order-placed subject does not know that one observer sends an email and another decrements stock. It knows there is a list of things with an `update` method.

Push or pull

When Meera sends the message, does she write “forty loaves, white and brown, out now” — or just “bread is out”?

- **Push:** the subject sends the details with the notification. `update(order)`. Convenient; every observer gets everything whether it needs it or not, and adding a field means changing the signature everybody implements.

- **Pull:** the subject sends only “something changed, and here is who I am”, and each observer asks back for what it needs. `update(subject)` then `subject.get_total()`. Flexible; more round trips, and the observer now depends on the subject’s full interface.

In practice you push a small, immutable **event object** — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`. That is push with the coupling kept narrow, and it is what nearly every real system does.

The word you will hear instead

Outside pattern books this is usually called **publish–subscribe**, or **pub/sub**. Same shape: publisher, subscribers, topic. The difference people draw is that classic Observer has the subject holding the list directly, while pub/sub usually puts a **broker** in the middle — a separate piece of software that holds the subscriptions, so the publisher does not even hold the list. Meera’s daughter is the broker. Meera does not know who is on the group any more.

That difference matters more than it sounds, and §5 is where it starts to pay.

Why not just call the five things directly?

You could write this:

```
def place_order(order: Order) → None:
    save(order)
    send_confirmation_email(order)      # 1
    reserve_inventory(order)            # 2
    generate_invoice(order)             # 3
    award_loyalty_points(order)         # 4
    record_analytics(order)             # 5
```

It is honest, readable, and for a long time it is the right answer. Say so in the interview, then name what goes wrong as it grows: the sixth thing means editing this function and re-testing checkout; `place_order` now imports the email module, the inventory module, the billing module and the analytics module; and if the analytics call is slow, checkout is slow.

Observer trades that for a list. What it does **not** do is make the five calls disappear — they still happen, in a loop, one after another. Anyone who tells you the pattern makes checkout fast is confusing Observer with a message queue. Getting that distinction right is most of §7.

4 The picture

The structure. Note that `OrderService` has an arrow to the *interface*, never to the concrete listeners.

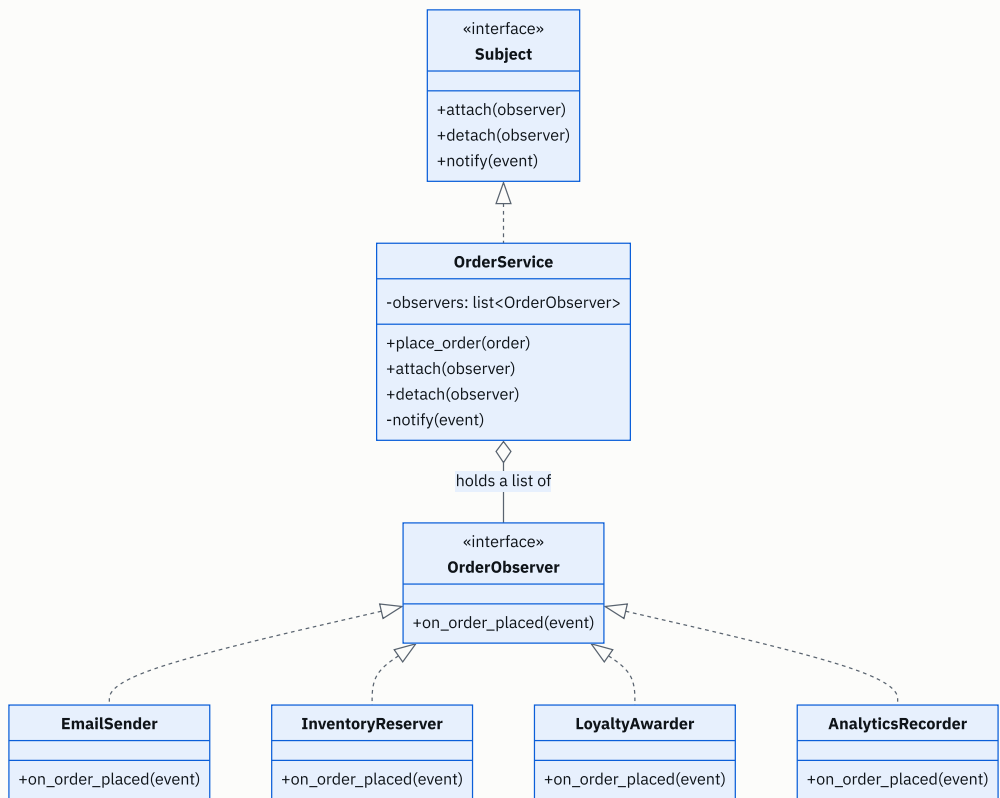


FIGURE 72.1

What to notice: there is **no arrow from OrderService to EmailSender**. That missing arrow is the entire pattern. **OrderService** imports the interface and nothing else, so adding a fifth listener adds a class and a line of wiring, and changes no existing file.

Now the same thing over time, synchronously:

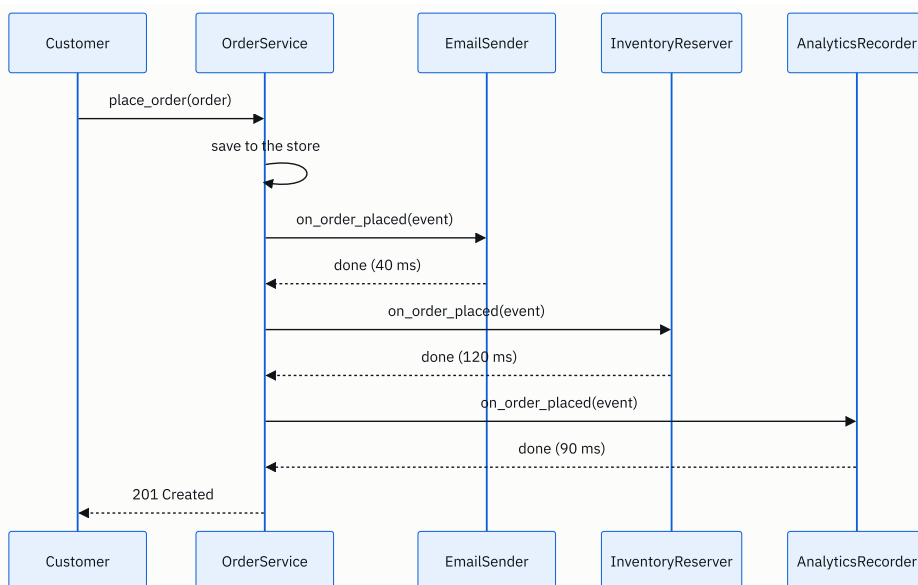


FIGURE 72.2

What to notice: the customer is waiting for all three. The arrows return before the next one starts. The 201 comes back after 250 milliseconds of work that has nothing to do with saving the order. This diagram is the argument for making it asynchronous, and drawing it is how you make that argument without a speech.

5 How it actually works

The smallest honest implementation

Start with the interface. In Python a `Protocol` is enough — any class with a matching method satisfies it, with no inheritance required.

```

from typing import Protocol

class OrderObserver(Protocol):
    def on_order_placed(self, event: "OrderPlaced") → None: ...
  
```

Then the event. Make it a frozen dataclass: a plain record of what happened, which cannot be changed by one observer before the next one sees it.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class OrderPlaced:
    order_id: str
    user_id: str
    total_paise: int
    item_count: int
```

`frozen=True` matters more than it looks. If the event were mutable, observer three could edit a field and observer four would silently see different data — a bug that is close to impossible to find because the order of observers is not written down anywhere.

Then the subject.

```
class OrderService:
    def __init__(self, store: "OrderStore") → None:
        self._store = store
        self._observers: list[OrderObserver] = []

    def attach(self, observer: OrderObserver) → None:
        self._observers.append(observer)

    def detach(self, observer: OrderObserver) → None:
        self._observers.remove(observer)
```

Two lines of registration. Note `detach` — the pattern is not finished without it, and forgetting it is the most common real bug (see §7).

```
def place_order(self, order: Order) → str:
    order_id = self._store.save(order)          # the real work, first
    event = OrderPlaced(order_id, order.user_id,
                        order.total_paise, len(order.items))
    self._notify(event)
    return order_id
```

The order in which those three lines appear is a decision, not an accident. **Save first, notify after.** If you notify first and the save then fails, you have sent a confirmation email for an order that does not exist. That mistake is called a phantom notification and it is very hard to undo.

```
def _notify(self, event: OrderPlaced) → None:
    for observer in list(self._observers):      # copy: a listener may detach itself
        try:
            observer.on_order_placed(event)
        except Exception:
            logger.exception("observer %r failed", observer)
```

Three details in five lines, and each is worth a sentence in the interview.

`list(self._observers)` iterates over a **copy**. Without it, an observer that calls `detach` on itself during `update` mutates the list you are looping over, and Python quietly skips the next observer. This is a real bug, not a theoretical one.

The `try` around each call means **one broken observer cannot break the others, or the order**. This is the single most important line in the file. Without it, an exception in the analytics recorder propagates all the way out of `place_order`, and the customer sees a 500 for an order that was actually saved.

And catching means you must **log**, or failures vanish. `logger.exception` records the traceback.

Wiring it up

```
service = OrderService(store)
service.attach(EmailSender(smtp))
service.attach(InventoryReserver(warehouse))
service.attach(LoyaltyAwarder(loyalty))
service.attach(AnalyticsRecorder(pipeline))
```

This is the **composition root** from [day 053](#) — one place, run at start-up, where the pieces are joined. Adding a sixth listener is one line here and one new file. `OrderService` is untouched, so no checkout test needs to run again.

The version that actually scales: hand the event to a broker

Synchronous observers put every listener’s latency in the customer’s request. The fix is to replace the in-process list with a **message broker** — a separate service that accepts a message, stores it durably, and delivers it to subscribers on their own schedule.

```
def _notify(self, event: OrderPlaced) → None:
    self._broker.publish("orders.placed", asdict(event))    # ~2 ms, then return
```

The shape of the pattern is identical. What changes is the guarantees:

	IN-PROCESS OBSERVERS	BROKER (KAFKA, RABBITMQ, SNS)
Publisher waits for	every listener	one durable write
A listener crashes	you catch and log; the work is lost	the message is redelivered
Listener is slow	the customer waits	a queue grows
Listener is deployed separately	no	yes
Debugging	one stack trace	tracing across services

Real products, and what each one is.

- **addEventListener in the browser.** The DOM is the canonical Observer: `button.addEventListener ("click", handler)` is `attach`, `removeEventListener` is `detach`, and the browser calls every registered handler when the click happens.
- **Django signals.** `post_save.connect(send_welcome_email, sender=User)`. Saving a `User` notifies every connected receiver. Powerful, and heavily criticised — see §7.

- **Spring's** `ApplicationEventPublisher` in Java, with `@EventListener` methods. Same pattern with annotations doing the `attach`.
- `java.util.Observer` and `Observable` were in the standard library from Java 1.0 and were **deprecated in Java 9** — the official reason was that the interface was too weak to be useful (no event typing, no ordering guarantee, not serialisable). Worth knowing: it signals that the *shape* is right and one particular API was not.
- **Redis pub/sub.** `SUBSCRIBE orders`, `PUBLISH orders "..."`. Very fast, and **fire-and-forget** — a subscriber that is offline when you publish never sees the message. Do not use it for anything that must not be lost.
- **Kafka.** Subscribers read from a durable, ordered, replayable log. A consumer that was down for an hour catches up. This is the version you name when the interviewer says “what if the email service is down?”
- **PostgreSQL** `LISTEN` / `NOTIFY`. The database itself is the subject. `NOTIFY order_placed, 'id-91'` wakes every connection that ran `LISTEN order_placed`. Useful, and the payload is capped at 8000 bytes, so it carries an id rather than an object.
- **Webhooks.** Stripe calling your `/webhooks/stripe` URL when a payment succeeds is Observer across the internet — you registered a URL, which is exactly `attach`.
- **RxJS / ReactiveX**, and React's `useEffect` re-running when a value changes. Observer with a large amount of machinery on top for composing streams.

What happens on restart

An in-process observer list lives in memory and is rebuilt by the composition root every time the process starts. That is fine, because the list is code. A broker's subscriptions are stored by the broker, so they survive your restart — and that is exactly why a broker can redeliver what your process missed while it was down.

6 The numbers

The latency the customer feels

Five listeners on an order, measured at the median:

save the order to Postgres	25 ms	
send the confirmation email	40 ms	(SMTP handshake + send)
reserve inventory (HTTP call)	120 ms	
generate the invoice PDF	90 ms	
award loyalty points (HTTP call)	60 ms	
record analytics	30 ms	

synchronous total	365 ms	

The customer waits **365 ms** for a request whose actual job took 25. Fourteen times the necessary work, and every listener you add makes it worse — the cost is the **sum**, because they run one after another in a loop.

Now publish to a broker instead:

save the order to Postgres	25 ms
publish OrderPlaced to Kafka	2 ms

total	27 ms

365 ms to 27 ms, and the number stops growing when you add the sixth listener. That single comparison is the strongest thing you can say in this question.

What it does to throughput

At 500 orders per second with a thread-per-request server:

synchronous:	500 req/s × 0.365 s = 182 concurrent requests in flight
async:	500 req/s × 0.027 s = 14 concurrent requests in flight

Thirteen times fewer threads or connections held open, from one change. If each in-flight request holds a database connection from a pool of 100, the synchronous version **exhausts the pool** and the asynchronous one uses fourteen percent of it.

What it does to reliability

Suppose each listener is independently available 99.9 percent of the time. If the whole checkout fails when any of them fails:

$0.999^5 = 0.9950$	→ 99.50% available
failures per million orders:	5,000 instead of 1,000

Five times more failed checkouts, caused entirely by things that are not checkout. Catching exceptions per observer fixes this for the *customer*, at the cost of silently losing work — which is why the durable broker is the real answer.

The size of the notification

An `OrderPlaced` event as JSON:

order_id	36 B (uuid)	+ user_id	36 B	+ total	8 B	+ item_count	4 B
+ field names and JSON punctuation, say ~120 B							
→ about 200 bytes per event							

At 500 orders per second:

500 × 200 B	= 100 KB/s published
× 5 subscribers	= 500 KB/s delivered
× 86,400 s	≈ 43 GB/day delivered, 8.6 GB/day stored
with 7-day retention and 3 replicas: $8.6 \times 7 \times 3 \approx 181$ GB of broker disk	

That is the arithmetic for “can Kafka hold this” — and the answer is that it is nothing. A single broker node handles it comfortably.

The cost of the list itself

Ten listeners at roughly 200 bytes of object overhead each is two kilobytes of memory and a ten-step loop. In-process Observer is free. **The cost of this pattern is never the list; it is always what the listeners do.**

7 The trade-offs

What you give up

The order of notification is not part of the contract, and people rely on it anyway. Observers are called in list order, which is registration order, which is whatever the composition root happens to do. If the loyalty awarder needs the invoice to exist, you have built an ordering dependency that is documented nowhere and that a reordered wiring line will break. If order matters, you do not want Observer — you want an explicit sequence, or a workflow with named steps.

Failures become silent by design. The `try/except` that stops one bad listener from breaking checkout also means the confirmation email can fail forever and the customer's request still returns 201. You must add: structured logging, a metric per listener, and an alert on the failure rate. Without those three, Observer is a machine for losing work quietly.

You cannot see the call graph. With five direct calls, “what happens when an order is placed” is answered by reading one function. With observers, it is answered by finding the composition root, and in a framework like Django by grepping the entire codebase for `post_save.connect`. This is the single most common complaint about Django signals, and it is a fair one: a new engineer cannot tell what saving a model does.

The lapsed listener. The subject holds a strong reference to every observer, so an observer that is never detached is never garbage collected. In a long-lived process where short-lived objects subscribe — a request handler, a UI screen — this is a genuine memory leak. Meera calling the sisters for three weeks after they moved out is exactly this bug. The fixes: detach explicitly in a `finally` or a context manager, or hold **weak references** (`weakref.WeakSet`), which let the observer be collected and quietly drop out of the list.

Cascades and cycles. Observer A updates something that notifies observer B, which updates something that notifies A. In-process this is a `RecursionError`; across services it is an infinite loop of messages that shows up as a mysterious cost spike. Keep observers from writing to the thing they observe, and if you cannot, add a guard flag.

Testing gets indirect. You can no longer assert that `place_order` sent an email, because `place_order` does not send emails. You test the observer alone, and you test that the subject notified — usually by attaching a fake observer that records what it received. That is a better test, but it is two tests where there was one.

Debugging across a broker is a different skill. Once the notification goes over Kafka, a single stack trace no longer explains a failure. You need correlation ids on every event and distributed tracing to follow one order through five consumers. That is real operational cost and you should say so.

“I would not use this if...”

- **...the set of listeners is fixed and small.** Two things happen when an order is placed and both are core to ordering. Call them. A list of two is ceremony.
- **...the steps must happen in a specific order.** That is a workflow, not a broadcast. Write the sequence, or use a proper orchestrator.
- **...every step must succeed or the whole thing must roll back.** Observer has no transaction. If the five things must be all-or-nothing, you want them inside one database transaction, or the saga pattern with explicit compensating actions.
- **...the listener needs to send something back.** Observers return nothing by design. If the subject needs an answer — “is this order fraudulent?” — that is a call, not a notification.
- **...it is the first listener.** One observer and an interface is a hierarchy built for a future that has not arrived. Add the pattern when the second listener does.

The honest concession

Observer does not remove work and it does not make anything faster on its own — in-process, the five calls still run one after another and the customer still waits for all of them. What it removes is the **compile-time dependency**: `OrderService` no longer knows the names of the five things. The speed comes from the *next* step, replacing the list with a queue.

Say that explicitly. A candidate who claims the pattern makes checkout fast has not thought about it; a candidate who says “the pattern buys decoupling, and asynchrony is a separate decision I would make next” has.

8 In the interview

How it gets asked

- The direct version: “When an order is placed, we need to send an email, reserve stock, generate an invoice, award loyalty points and log analytics. How would you structure that?”
- The UI version: “Several parts of the screen show the same data. When it changes, they all need to update. Design that.”
- The scaling version: “Our checkout endpoint takes 400 ms and most of it is not checkout. What is going on and how do you fix it?”
- The pattern-name version, in a low-level design round: “Which pattern would you use here, and what would it cost you?”
- The trap version: “Would you use Django signals for this?” — they are testing whether you can criticise a tool you have used.

What to say out loud, in the first ninety seconds

1. **Name the shape before naming the pattern.** “This is one event with several independent reactions, and the list of reactions will grow. That is Observer, or publish–subscribe.”
2. **Say what the coupling problem is, concretely.** “If `place_order` calls all five directly, it imports the email module, the warehouse client, the billing module and the analytics pipeline. Adding a sixth means editing checkout and re-testing checkout.”
3. **State the structure in one breath.** “The order service holds a list of listeners behind one interface, publishes an immutable `OrderPlaced` event, and the wiring — which listeners exist — lives in the composition root, not in the service.”
4. **Say the two decisions that are not the pattern.** “Two things I would decide separately: whether notification is synchronous or goes through a broker, and what happens when one listener fails.”
5. **Then give the number.** “Synchronously those five listeners are 340 milliseconds added to a 25-millisecond save. Publishing to Kafka is 2 milliseconds and it does not grow with the sixth listener.”

If you are drawing, draw the class diagram with the missing arrow, then the sequence diagram with the customer waiting. The second diagram wins the argument for you.

The follow-ups

“**What happens if one listener throws?**” “In-process, I wrap each call in its own try/except so one bad listener cannot fail the order or stop the other four — the order is already saved by then. But catching means losing work silently, so I would add a metric and an alert per listener. The proper fix is a durable broker: publish once, and each consumer retries on its own with a dead-letter queue for what it cannot process. Then a failed email is a message waiting, not a message gone.”

“**Do the listeners run in a guaranteed order?**” “No, and I would not let anyone depend on one. They run in registration order, which is an accident of the wiring code. If step B genuinely needs step A to have finished, that is not a broadcast, it is a workflow — I would write it as an explicit sequence, or have B subscribe to an event that A publishes when it is done.”

“**How do you make this asynchronous?**” “Replace the in-process list with a publish to a broker. The service writes the order, publishes `OrderPlaced` to a topic, and returns. Each of the five becomes a consumer with its own retry policy and its own deployment. The publisher’s latency drops from the sum of all listeners to a single durable write — about 2 milliseconds for Kafka. The cost is that it is now eventually consistent: the customer can refresh and not see the loyalty points yet, so the UI has to be honest about that.”

“**How is this different from a message queue?**” “It is the same pattern with different guarantees, and where the subscription list lives. Classic Observer keeps the list inside the subject, in memory, and delivery is a method call — fast, and lost if the process dies. A broker

keeps the list outside both sides, so publisher and subscriber do not even know about each other, and the message is durable. Redis pub/sub sits in between: broker in the middle, but fire-and-forget, so a subscriber that was offline never sees it.”

“What about memory leaks?” “The lapsed listener problem. The subject holds a strong reference to every observer, so an observer that never detaches is never collected. It matters when short-lived objects subscribe to a long-lived subject. Two fixes: detach in a `finally` or a context manager so it always happens, or hold the observers in a `weakref.WeakSet` so they drop out when nothing else refers to them.”

“Would you use Django signals here?” “Cautiously, and probably not for business logic. They work, but they hide the call graph — a new engineer reading `user.save()` has no way to know that six receivers fire, and finding them means grepping for `connect`. I would use them for genuinely cross-cutting things like cache invalidation or an audit trail, and use an explicit service call or a real event bus for anything a person needs to reason about.”

A model answer

Asked: *when an order is placed, five things must happen. Design that.*

“Let me first say what shape this is, because it decides everything. One thing happens — an order is placed — and several independent reactions follow. The list of reactions will grow; it always does. Nobody has ever removed the fifth thing that happens after checkout.

The naive version is `place_order` calling all five in a row, and honestly, for two reactions that is the right code. The problem starts at four or five. `place_order` now imports the email module, the warehouse client, the billing service and the analytics pipeline, so a change in any of them can break checkout, and adding a sixth reaction means editing and re-testing the most business-critical function in the system.

So I would use Observer. The order service keeps a list of listeners behind a single narrow interface — one method, `on_order_placed`, taking an immutable event. I would make the event a frozen record with the order id, the user id, the total and the item count, so no listener can mutate what the next one sees. The service saves the order first and notifies afterwards, never the other way round — otherwise a failed save has already triggered a confirmation email for an order that does not exist.

Which listeners exist is wiring, and wiring belongs in the composition root at start-up, not in the service. That is the payoff: the sixth reaction is a new file and one line of wiring, and no checkout test has to run again.

Two things I would decide separately, because they are not the pattern.

First, failure. In-process I wrap each listener call in its own try/except, so a broken analytics recorder cannot fail an order that is already saved. But that trades a loud failure for a silent one, so it only counts as a design if I also add a per-listener metric and an alert.

Second, latency, and this is where the numbers matter. Those five listeners are roughly 340 milliseconds — email 40, inventory 120, invoice 90, loyalty 60, analytics 30 — on top of a 25-millisecond save. The customer waits fourteen times longer than the work required, and it gets worse with every listener, because in-process observers run in a loop and the cost is the sum.

So the version I would actually ship publishes the event to a broker — Kafka, or SNS — and returns. That is one durable write, about 2 milliseconds, and it does not grow. Each of the five becomes a consumer with its own retries and its own dead-letter queue, so a failed email is a message waiting rather than work lost, and the email service can be down for an hour and catch up.

The costs I would name up front. The order listeners run in is not a contract, so nothing may depend on it — if one step genuinely needs another, that is a workflow and I would write it explicitly. The call graph becomes invisible, so a new engineer cannot tell what saving an order does; I would document the event and its subscribers in one place. And the async version is eventually consistent, so the UI has to be honest that loyalty points appear a moment later.

One thing I would not do is introduce the pattern for two listeners. With two, the direct calls are clearer and I would leave them alone until the third one arrives.”

9 Recall card

- **One subject holds a list of observers behind one narrow interface, and `notify` loops the list.** The subject knows the interface and **nothing about the concrete listeners** — that missing arrow in the class diagram is the pattern. Adding the sixth listener is a new file plus one line in the composition root; checkout is not edited or re-tested.
- **Four rules for the implementation:** save first and **notify after** (or you email about an order that does not exist) · publish an **immutable event object**, so listener 3 cannot change what listener 4 sees · iterate over a **copy** of the list, or a self-detaching observer skips the next one · wrap **each** call in its own try/except, and then log and alert, or you have built a machine for losing work silently.
- **The honest concession: in-process Observer buys decoupling, not speed.** The five calls still run one after another and the cost is the **sum** — 25 ms of save plus 40 + 120 + 90 + 60 + 30 = **365 ms** the customer waits. Publishing to a broker instead is **~27 ms and stops growing**. Async is a *separate* decision from the pattern.
- **Observer → pub/sub is where the list moves out of the subject and into a broker.** `addEventListener` · Django signals · Spring `@EventListener` · Redis pub/sub (**fire-and-forget, lost if offline**) · Kafka (**durable, replayable, redelivered**) · PostgreSQL `LISTEN/NOTIFY` (8 KB payload cap) · webhooks. `java.util.Observable` was **deprecated in Java 9** for being too weak an interface.
- **Do not use it if the order of steps matters** (that is a workflow), **if it must be all-or-nothing** (that is a transaction or a saga), **if the listener must return an answer**, or **if there is only one listener**. Watch for the **lapsed listener leak** — detach in a `finally`, or hold observers in a `weakref.WeakSet`.

PRACTICE

Practice — Largest rectangle in a histogram

DSA topic: Largest rectangle in a histogram **System design topic:** Observer

Code these, in this order

One rule for the whole set: **before writing the loop, say the width formula as a sentence.** “Both boundaries are the shorter bars themselves, and neither of them is inside the span, so the width is `right - left - 1`.” If you cannot say that sentence, do not write the loop yet.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Sum of Subarray Minimums	LeetCode 907 (Medium)	The same two boundaries, used to <i>count</i> spans instead of measuring one. Forces the width formula into your hands.
2	Largest Rectangle in Histogram	LeetCode 84 (Hard)	Today’s problem. The reduction, the one-pass pop, and the sentinel.
3	Trapping Rain Water	LeetCode 42 (Hard)	The same stack accumulating as it pops — and the two-pointer solution that beats it on space.
4	Maximal Rectangle	LeetCode 85 (Hard)	Recognising that each row of a binary grid is a histogram. Almost no new code.

On problem 1, do it the slow way first

Write the $O(n^2)$ version that finds the minimum of every subarray. Run it on an array of 2,000 random values and time it. Then write the boundary version and time that. Say out loud why the boundaries turn “minimum of every subarray” into “how many subarrays does each element win”, and where the duplicate-handling has to be asymmetric — one side strict, one side not — or you double-count.

On problem 2, get the width wrong on purpose

1. Write it with `width = index - left` and run `[2, 1, 5, 6, 2, 3]`. Write down what it returns.
2. Fix it to `index - left - 1` and run it again.
3. Say in one sentence why the `- 1` is there, without using the words “off by one”.

On problem 3, solve it twice

Once with the monotonic stack, popping and adding the water trapped above the popped bar. Once with two pointers from [day 030](#), moving the smaller side inward. Then say which one you would write in an interview and why — the answer involves the words “ $O(1)$ space”.

On problem 4, count before you code

Before writing anything, say the complexity out loud: rows $\times$ columns to build the heights, plus one $O(\text{columns})$ histogram solve per row. State the total. It sounds much worse than it is, and saying the number first is how you stop the interviewer thinking you have not noticed.

The reduction drill

Say the argument out loud, five times, until it comes without effort:

1. Why are there infinitely many candidate rectangles as the problem is stated?
2. What is the claim that cuts it down to `n`?
3. Prove it in one sentence. (Start with “suppose the top edge were below every bar it spans”.)
4. Given that claim, what is the exact question you now ask about each bar?
5. Name the two things you need per bar, and what each defaults to when there is none.

The width drill

For `heights = [2, 1, 5, 6, 2, 3]`, compute by hand, for every index:

1. `left[i]` — the index of the nearest strictly shorter bar to the left, or `-1`.
2. `right[i]` — the index of the nearest strictly shorter bar to the right, or `6`.
3. The width, and the area.
4. Which index wins, and by how much over the runner-up.

Then check your six areas against `largest_rectangle_two_pass`. Any disagreement is a boundary definition you have wrong, not an arithmetic slip.

The trace drill

Run `[2, 1, 5, 6, 2, 3]` in your head with a sentinel at the end, writing the stack contents after every step. Then answer without looking:

1. At which index does one arrival pop two bars?
2. What is the width computed for the bar of height 2 at index 4, and why is it not 2?
3. How many bars does the sentinel pop?
4. What would the function return with no sentinel and no drain loop?

The break-it drill

Trigger each and record what actually happens — the output, or the exact error text:

1. Remove `stack and` from the `while` condition. Quote the error and the input that causes it.
2. Write `left = stack[-1]` with no `if stack else -1`. Quote the error.
3. Use `index - stack.pop()` as the width. Run `[2, 1, 5, 6, 2, 3]`, then run `[4, 3]`. Explain why the first one is the dangerous result.

4. Remove the sentinel. Run `[2, 2]`, then `[1, 2, 3, 4, 5]`, then `[2, 1, 5, 6, 2, 3]`. Say why the third one makes this bug so hard to catch.
5. Change `>` to `≥`. Confirm the answer is still right, then say which duplicate carries the full span in each version.
6. Push before popping instead of after. Say what the bar compares itself against.

The random-cross-check drill

Write the $O(n^2)$ brute force, then run both on two thousand random arrays of length 0 to 12 with values 0 to 9, asserting they agree. Then break one thing from the drill above and watch the assertion fire on a four-element input. Say out loud why small random inputs catch this bug class faster than large ones.

The recognition drill

For each, say whether the histogram stack is the tool, and if so what plays the part of “height”:

1. The largest square of 1s in a binary grid.
2. The largest rectangle of 1s in a binary grid.
3. How much rain water is trapped between bars.
4. The sum of the minimums of every subarray.
5. The maximum area of a container formed by two vertical lines.
6. For each day, how many days until a warmer one.
7. The largest rectangle under a skyline where bars have different widths.

Two of the seven are not stack problems at all. Name them and say what they are instead.

The observer-refactor drill

Here is the function:

```
def place_order(order: Order) → str:
    order_id = store.save(order)
    smtp.send_confirmation(order)
    warehouse.reserve(order.items)
    billing.generate_invoice(order)
    loyalty.award(order.user_id, order.total_paise)
    analytics.record("order_placed", order_id)
    return order_id
```

1. List every module this function now depends on.
2. Name three specific things that go wrong when the sixth reaction is added.
3. Write the observer interface. One method. Say why one and not five.
4. Write the event class, and say why it is frozen.
5. Write `attach`, `detach` and `_notify`, with all three of the details from §5 in place.
6. Write the composition root.

7. Add a sixth listener. Count the files edited and the existing tests that must re-run, both ways.
8. Now make it publish to a broker instead. Quote both latency totals.

The failure drill

For each, say what the customer sees and what you would add:

1. The analytics recorder throws, and there is no try/except.
2. The analytics recorder throws, there is a try/except, and no logging.
3. The email service is down for one hour, with in-process observers.
4. The email service is down for one hour, with Kafka.
5. An observer detaches itself while being notified, and you iterate the list directly.
6. An observer mutates the event before the next observer sees it.
7. A short-lived request object subscribes and is never detached.

The do-not-do-it drill

For each, say whether you would use Observer, and give the deciding question:

1. Two things happen on order placement, both core to ordering.
2. Five things happen, and one of them must run before another.
3. Five things happen, and either all five succeed or the order must be cancelled.
4. One listener, today, with no second one in sight.
5. The subject needs to know whether the order is fraudulent before saving it.
6. Cache invalidation when any row in a table changes.
7. Six teams each want to react to checkout, on their own deployment schedule.

Two of the seven are “no” for the same reason. Name it. One is a “yes” only in the broker form — say which and why.

The already-used-it drill

For each, name the subject, the observer, and whether a missed notification is lost forever:

1. `button.addEventListener("click", handler)`
2. Django's `post_save.connect(...)`
3. Redis `SUBSCRIBE orders`
4. A Kafka consumer group reading `orders.placed`
5. Postgres `LISTEN order_placed`
6. Stripe calling your `/webhooks/stripe` endpoint
7. React's `useEffect` with a dependency array

Three of the seven lose the message if the listener is not there at the moment it fires. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the largest rectangle in the histogram.* The reduction first — why the top edge must rest on a bar — then the per-bar question, the two boundaries with their defaults, the width formula said as a sentence, the one-pass fold with both boundaries known at the moment of the pop, the sentinel, and the counting argument for $O(n)$.
2. *When an order is placed, five things must happen. Design that.* The shape named before the pattern, the concrete coupling problem in terms of imports and re-tested tests, the one-method interface with an immutable event, save-then-notify, wiring in the composition root, and then the two separate decisions — per-listener failure handling, and 365 ms synchronous against 27 ms through a broker.
3. *Why is the top edge of the best rectangle always level with some bar?* Suppose it is not; raise it; it still fits and it is bigger. Then say what that buys you — `n` candidates instead of infinitely many — and what the per-bar question becomes.

Before you move on

- ☐ I can state the reduction claim and prove it in one sentence.
- ☐ I can say the width formula as a sentence, without the words “off by one”.
- ☐ I computed `left`, `right`, width and area by hand for all six bars of `[2, 1, 5, 6, 2, 3]`.
- ☐ I wrote the two-array version and the one-pass version, and they agree on random inputs.
- ☐ I broke the width on purpose and know what `[4, 3]` returns under the popped-index bug.
- ☐ I removed the sentinel and saw `[1, 2, 3, 4, 5]` return 0.
- ☐ I can explain why equal heights are safe, and which duplicate carries the full span.
- ☐ I solved maximal rectangle and can state its complexity before writing it.
- ☐ I can give the counting argument for $O(n)$ without softening it.
- ☐ I refactored `place_order` to observers and counted the files edited for a sixth listener.
- ☐ I can name the three implementation details in `_notify` and what each one prevents.
- ☐ I can quote both latency totals, synchronous and through a broker.
- ☐ I can say the honest concession: the pattern buys decoupling, not speed.
- ☐ I answered all three questions above out loud.

Queues: first in, first out

DATA STRUCTURES AND ALGORITHMS

Queues: first in, first out

You can implement a queue with $O(1)$ enqueue and dequeue and say why a list is wrong.

SYSTEM DESIGN

State

You can model an object whose behaviour changes with its state, without a giant switch.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Implement a queue. Why not just use a list and `pop(0)`?*
- *Model an order that moves through placed, paid, shipped and delivered.*

1 What this is, and why they ask it

A **queue** is a collection where things come out in the order they went in. You add at one end and remove from the other. First in, first out — **FIFO** — which is the exact opposite of the stack from [day 068](#), where the last thing in comes out first.

There are only four operations and none of them searches: **enqueue** (add at the back), **dequeue** (remove from the front), **peek** (look at the front without removing), and **is empty**. All four must be $O(1)$. If any of them is $O(n)$, you have not built a queue, you have built a list with inconvenient method names.

They ask it because the obvious Python implementation is wrong in a way that is invisible in testing and catastrophic in production. `queue = []`, `queue.append(x)` to add, `queue.pop(0)` to remove — it looks right, it passes every small test, and it is $O(n)$ per removal, which makes any loop over it $O(n^2)$. At a hundred thousand elements that is ten seconds instead of three milliseconds, measured below. The follow-up — “why not a list?” — is the actual question, and it is asked because the answer requires you to know what an array is in memory, which is [day 009](#). Queues also underpin breadth-first search, level-order tree traversal, and every rate limiter and job runner you will ever design, so this is a foundation day, not a trivia day.

2 The story

The sub-registrar’s office opens at ten and Ramesh gets there at twenty past nine, which he has learned is the only way.

For years it worked one way. You joined the line at the door and you stood in it. The line ran along the wall, out past the notice board, and on a Monday it went down the steps and into the car park. When the clerk finished with somebody, that person walked away, and then every single person in the line took one step forward. Sixty people, one step each, for one person served. Then it happened again. And again.

Ramesh hated it, and not because of the walking. It was that the line stopped being a line. Somebody would step out to take a call and someone behind would close the gap. Two brothers would arrive and one would join at the back and the other would drift up to the front to “just ask something”. By eleven o’clock nobody could say who was ahead of whom, and there were arguments almost every day.

Then last April they put in the machine by the door and the screen above the counter.

Now Ramesh walks in, presses the button, and the machine gives him a small printed number — 47 on the day he tells this story. He sits down on the bench. The screen says 31. When the clerk finishes, the screen changes to 32, and one person stands up and walks to the counter. Nobody else moves at all. Sixty people sit still while the number goes up by one.

The machine keeps handing out numbers going up — 48, 49, 50 — and the screen keeps calling numbers going up — 32, 33, 34. Two numbers, one at each end, and they never touch each other. The gap between them is how many people are waiting.

What Ramesh noticed, sitting there with 47 in his hand, is that the amount of work per person served has changed completely. Before, serving one person cost sixty people one step each. Now it costs one number changing on a screen. And nobody argues any more, because the number in your hand says exactly when you arrived and there is nothing to argue about.

3 The idea in plain English

The old line and the new machine are the same queue. First in, first out, both times. What changed is the **cost of serving one person**, and that is the whole lesson.

The four operations

- **enqueue(x)** — a new person takes a number. Add at the back.
- **dequeue()** — the screen calls the next number. Remove from the front and return it.
- **peek()** — who is next, without calling them.
- **is_empty()** — has everyone been served?

All four are $O(1)$ in a correct implementation. “ $O(1)$ ” means the work does not depend on how many people are waiting: sixty or six thousand, the screen still changes one number.

Why `list.pop(0)` is the old line

A Python list is a block of slots laid out one after another in memory, exactly as in [day 009](#). The list knows where its first slot is. `pop(0)` removes the first element — and then every remaining element has to move one slot to the left, so that the first element is at slot 0 again. Sixty people, one step each.

That is $O(n)$ per removal. Empty a queue of `n` elements and you have done `n + (n-1) + (n-2) + ... + 1` moves, which is $n^2/2$. The measurement in §6 is 10.2 seconds for a hundred thousand elements against 0.0034 seconds for the right structure.

`append` and `pop()` at the *end* are fine, which is why the stack works with a plain list. **The end is the cheap end.** The front is the expensive one, and a queue lives at the front.

The first fix: stop shuffling, keep a head marker

Do not remove the element. Leave it where it is and remember where the front now starts.

```
front = 0
def dequeue():
    value = items[front]
    front += 1          # move the marker, not the data
    return value
```

That is the screen. Dequeue is now $O(1)$. But there is a new problem: the slots before `front` are dead and never reused, so the list grows for ever. A queue that handles a million jobs holds a million dead slots. That is a memory leak with extra steps.

The second fix: let the numbers wrap around

Fix the number of slots at, say, 8, and let both markers wrap around to 0 when they run off the end. That is a **circular buffer**, also called a **ring buffer**: a fixed block of slots where position `7 + 1` is position `0`.

```
self.tail = (self.tail + 1) % self.capacity
```

The `% capacity` is the wrap. Now nothing is ever shuffled and nothing is ever leaked, because slots are reused as they are freed. This is how real fixed-size queues are built — network card buffers, audio buffers, log ring buffers — and it is the implementation an interviewer means when they say “implement a queue with an array”.

The one hard part is telling *full* from *empty*. When `head == tail`, is the buffer empty or completely full? Both look identical. Two standard answers: keep a separate `size` counter, or leave one slot permanently unused so `head == tail` can only mean empty. Keep the counter — it is one integer and it makes `is_empty`, `is_full` and `__len__` trivial.

The third fix, and the one you actually use

`collections.deque` — a **double-ended queue**, pronounced “deck”. It supports `append`, `appendleft`, `pop` and `popleft`, all $O(1)$. You have already met it in passing on [day 031](#).

```
from collections import deque

queue = deque()
queue.append("a")      # enqueue at the back
queue.append("b")
first = queue.popleft() # dequeue from the front → "a"
```

Internally it is a doubly linked list of fixed-size blocks, so growing at either end never moves existing elements. That is why both ends are cheap. **In an interview: build the array version to show you understand it, then say “in real code I would use `collections.deque`.”**

One structure you should be able to reject by name: `queue.Queue`. It exists for passing work between threads and takes a lock on every operation, which makes it about **48 times slower** than a `deque` in a single-threaded program, measured below. Correct answer, wrong question.

Where queues turn up

- **Breadth-first search** on a graph or a grid, and **level-order traversal** of a tree. Both are “visit everything one distance away, then everything two away”, and that is a queue. This is why today matters far beyond today.
- **Job runners and message brokers.** RabbitMQ and SQS are queues with durability bolted on.
- **Rate limiters and buffers** — a fixed-size ring that drops the oldest when full.
- **Sliding-window maximum**, tomorrow, which needs both ends and therefore a deque.

4 The picture

The two markers, and the gap between them. Slots along the top, values underneath.

```
capacity 8, after enqueueing A B C D E and dequeuing A B

slot      0      1      2      3      4      5      6      7
value  +-----+-----+-----+-----+-----+-----+-----+
      | . | . | C | D | E | . | . | . |
      +-----+-----+-----+-----+-----+-----+
                ^               ^
            head           tail
          (next out)    (next slot in)

size = 3           the gap between the markers is who is waiting
```

What to notice: `head` moved right without anything else moving. Slots 0 and 1 hold rubbish and nobody cares. That is the screen going from 31 to 33 while sixty people sit still.

Now enqueue F, G, H and watch `tail` fall off the end and come back:

```
after F, G at slots 5, 6, then H wraps to slot 7 then 0

slot      0      1      2      3      4      5      6      7
value  +-----+-----+-----+-----+-----+-----+-----+
      | I | . | C | D | E | F | G | H |
      +-----+-----+-----+-----+-----+-----+-----+
                ^               ^
            tail           head

size = 7           tail is now to the LEFT of head, and that is fine
```

What to notice: `tail < head` now, and the queue is still perfectly ordered — C, D, E, F, G, H, I. The order is defined by walking forward from `head` with wraparound, not by slot number. This is the picture people fail to draw, and then they write `while head < tail` and it breaks.

And the state that needs the size counter:

```
head = tail = 2    with size 0 → EMPTY
head = tail = 2    with size 8 → FULL
```

identical markers, opposite meanings. Hence the counter.

5 The code, built step by step

Step 1 — the version to write down and then reject

```
class SlowQueue:
    def __init__(self) → None:
        self._items: list[int] = []

    def enqueue(self, value: int) → None:
        self._items.append(value)      # O(1), fine

    def dequeue(self) → int:
        return self._items.pop(0)      # O(n) — every element shifts left
```

Say out loud what is wrong before the interviewer says it: “`pop(0)` moves every remaining element one slot left, so it is $O(n)$, and draining the queue is $O(n^2)$.”

Step 2 — the head marker

```
def dequeue(self) → int:
    value = self._items[self._head]
    self._items[self._head] = None    # release the reference
    self._head += 1
    return value
```

Dequeue is $O(1)$ now. Setting the slot to `None` matters in Python: without it the list keeps a reference to the object and the garbage collector cannot free it, so a queue of large objects holds all of them alive for ever.

The remaining problem is that `self._items` never shrinks. A fix is to rebuild the list when the dead prefix gets big:

```
if self._head > 64 and self._head * 2 > len(self._items):
    self._items = self._items[self._head:]    # amortised compaction
    self._head = 0
```

Copying is $O(n)$, but it only happens when at least half the list is dead, so the cost spreads out to $O(1)$ per operation on average. That is **amortised**, the same argument as list growth from [day 005](#). It works, and it is more code than the ring buffer for the same result.

Step 3 — the ring buffer, which is what they mean

```
class RingQueue:
    def __init__(self, capacity: int) → None:
        self._items: list[object | None] = [None] * capacity
        self._capacity = capacity
        self._head = 0          # index of the front element
        self._tail = 0          # index of the next free slot
        self._size = 0          # how many are in it – full/empty needs this
```

Four fields. The `_size` counter is not optional bookkeeping; it is what separates full from empty when the two markers coincide.

```
    def enqueue(self, value: object) → None:
        if self._size == self._capacity:
            raise OverflowError("queue is full")
        self._items[self._tail] = value
        self._tail = (self._tail + 1) % self._capacity    # wrap
        self._size += 1
```

Three lines: write, advance with wrap, count. The `% self._capacity` is the only clever thing in the class, and it is the thing to say out loud: “position capacity – 1 plus one is position zero”.

```
    def dequeue(self) → object:
        if self._size == 0:
            raise IndexError("dequeue from an empty queue")
        value = self._items[self._head]
        self._items[self._head] = None                # release the reference
        self._head = (self._head + 1) % self._capacity
        self._size -= 1
        return value
```

The mirror image. Read, clear, advance with wrap, count down.

Step 4 — growing when it fills

A fixed capacity is sometimes exactly what you want — a buffer that drops the oldest when full. If you want it unbounded, double the capacity and **unroll the ring** while copying:

```
    def _grow(self) → None:
        bigger: list[object | None] = [None] * (self._capacity * 2)
        for i in range(self._size):
            bigger[i] = self._items[(self._head + i) % self._capacity]
        self._items, self._capacity = bigger, self._capacity * 2
        self._head, self._tail = 0, self._size
```

The loop walks forward from `head` with wraparound, which is the only correct way to read a ring in order. Copying straight from slot 0 would give you the wrong order whenever `tail < head` — the picture in §4 shows exactly that case.

The complete solution

```
from collections import deque

class RingQueue:
    """A FIFO queue on a fixed block of slots, with both markers wrapping around.

    All four operations are O(1). Nothing is ever shifted; only the two markers
    move. `_size` is what distinguishes full from empty when they coincide.
    """

    def __init__(self, capacity: int = 8) → None:
        if capacity < 1:
            raise ValueError("capacity must be at least 1")
        self._items: list[object | None] = [None] * capacity
        self._capacity = capacity
        self._head = 0          # slot of the front element
        self._tail = 0          # next free slot
        self._size = 0

    def enqueue(self, value: object) → None:
        if self._size == self._capacity:
            self._grow()
        self._items[self._tail] = value
        self._tail = (self._tail + 1) % self._capacity
        self._size += 1

    def dequeue(self) → object:
        if self._size == 0:
            raise IndexError("dequeue from an empty queue")
        value = self._items[self._head]
        self._items[self._head] = None          # do not hold the object alive
        self._head = (self._head + 1) % self._capacity
        self._size -= 1
        return value

    def peek(self) → object:
        if self._size == 0:
            raise IndexError("peek at an empty queue")
        return self._items[self._head]

    def is_empty(self) → bool:
        return self._size == 0

    def __len__(self) → int:
        return self._size

    def _grow(self) → None:
        """Double the slots, unrolling the ring so the order survives."""
        bigger: list[object | None] = [None] * (self._capacity * 2)
        for i in range(self._size):
            bigger[i] = self._items[(self._head + i) % self._capacity]
        self._items = bigger
        self._capacity *= 2
        self._head = 0
        self._tail = self._size

    def __repr__(self) → str:
        order = [self._items[(self._head + i) % self._capacity] for i in range(self._size)]
        return f"RingQueue({order})"

class DequeQueue:
    """The version you would actually write. collections.deque is a doubly
    linked list of blocks, so both ends are O(1) and nothing is ever shifted."""

    def __init__(self) → None:
        self._items: deque[object] = deque()

    def enqueue(self, value: object) → None:
        self._items.append(value)
```



```

def dequeue(self) → object:
    if not self._items:
        raise IndexError("dequeue from an empty queue")
    return self._items.popleft()

def peek(self) → object:
    if not self._items:
        raise IndexError("peek at an empty queue")
    return self._items[0]

def is_empty(self) → bool:
    return not self._items

def __len__(self) → int:
    return len(self._items)

def level_order(root: "Node | None") → list[list[int]]:
    """Why queues matter: visit a tree one level at a time.

    The queue holds the nodes at the current distance from the root. Taking
    them all out and pushing their children gives the next distance.
    """
    if root is None:
        return []
    levels: list[list[int]] = []
    frontier: deque = deque([root])
    while frontier:
        this_level = []
        for _ in range(len(frontier)): # exactly the current level
            item = frontier.popleft()
            this_level.append(item.value)
            if item.left:
                frontier.append(item.left)
            if item.right:
                frontier.append(item.right)
        levels.append(this_level)
    return levels

if __name__ == "__main__":
    q = RingQueue(capacity=4)
    for value in "ABCDE": # 5 into a capacity of 4: it grows
        q.enqueue(value)
    print(q) # RingQueue(['A', 'B', 'C', 'D', 'E'])
    print(q.dequeue(), q.dequeue()) # A B
    q.enqueue("F")
    print(q, len(q), q.peak()) # RingQueue(['C', 'D', 'E', 'F']) 4 C

    small = RingQueue(capacity=3)
    small.enqueue(1); small.enqueue(2); small.enqueue(3)
    print(small.dequeue(), small.dequeue()) # 1 2
    small.enqueue(4); small.enqueue(5) # tail wraps past the end
    print(small) # RingQueue([3, 4, 5])

    empty = RingQueue()
    print(empty.is_empty()) # True
    try:
        empty.dequeue()
    except IndexError as error:
        print(f"IndexError: {error}") # IndexError: dequeue from an empty queue

```

Write `RingQueue` when asked to implement a queue. Then say the last sentence: “in production I would use `collections.deque`, because it does this and handles growth for me.”

6 What it costs

The four operations

enqueue	O(1)	write one slot, advance one marker
dequeue	O(1)	read one slot, advance one marker
peek	O(1)	one index
is_empty	O(1)	one comparison

Nothing loops. Nothing searches. The `_grow` call is $O(n)$, but it doubles the capacity, so n enqueues cause at most $1 + 2 + 4 + \dots + n \approx 2n$ slot copies in total — **amortised $O(1)$** per enqueue, exactly the argument for list growth.

Space is $O(\text{capacity})$, which is between n and $2n$ slots for n elements — the price of doubling.

The measurement that answers “why not a list?”

Draining a queue of 100,000 integers, on this machine:

list, dequeue with pop(0)	10.203 s
collections.deque, popleft	0.0034 s

ratio	about 3,000x

And the shape, which matters more than the ratio:

$n = 100,000 \rightarrow 10.20 \text{ s}$	
$n = 200,000 \rightarrow 39.74 \text{ s}$	3.9x the time for 2x the input

Doubling the input roughly quadrupled the time. That is the signature of $O(n^2)$ and it is the number to quote. It also means the problem gets worse the more successful your system is, which is the worst possible failure shape.

Count it out: `pop(0)` shifts every remaining element left by one slot, so it is $O(n)$. Doing that n times is $n + (n-1) + \dots + 1 = n(n+1)/2$. At $n = 100,000$ that is five billion slot moves.

And the structure that is correct but wrong here

collections.deque, 100,000 in and out	0.0034 s
queue.Queue, 100,000 in and out	0.1650 s

ratio	about 48x

`queue.Queue` acquires and releases a lock on every `put` and `get`, because it is built for handing work between threads. In a single-threaded algorithm you are paying for synchronisation you do not need. Know it exists, name it as the right tool for producer–consumer threads, and do not use it in a coding round.

7 The traps

Trap 1 — `pop(0)`, the whole reason this day exists

```
def dequeue(self):  
    return self._items.pop(0)
```

No error. No warning. Correct output on every test you write by hand. Ten seconds instead of three milliseconds at a hundred thousand elements, and quadratic growth after that. This is the single most common performance bug in beginner Python, and `list.insert(0, x)` is its twin at the other end.

If you remember one thing from today: **the front of a Python list is the expensive end.**

Trap 2 — full and empty look identical

```
def is_empty(self) → bool:  
    return self._head == self._tail          # WRONG without a size counter
```

Fill a capacity-4 ring with four elements. `tail` wraps around to equal `head`, and this reports the full queue as empty. Then `dequeue` returns `None` or raises, depending on how you wrote it, and the bug only appears when the buffer happens to fill exactly.

Two fixes. Keep `_size` — one integer, and `is_empty`, `is_full` and `len` all become trivial. Or deliberately waste one slot so the buffer is “full” at `capacity - 1`, making `head == tail` mean empty and only empty. The counter is clearer; say both, and say which you chose.

Trap 3 — reading the ring in slot order

```
def __repr__(self):  
    return f"RingQueue({[x for x in self._items if x is not None]})"
```

This looks reasonable and is wrong twice. It reports the wrong order the moment `tail < head` — the second picture in §4 would print `I, C, D, E, F, G, H` instead of `C, D, E, F, G, H, I`. And it silently drops any legitimate `None` you stored. Always walk `(head + i) % capacity` for `i` in `range(size)`.

Trap 4 — forgetting to clear the dequeued slot

```
value = self._items[self._head]  
self._head = (self._head + 1) % self._capacity    # slot still points at value
```

The queue works. The memory does not get freed, because the list still holds a reference to the object. In a long-running job runner that has handled a million 2 KB messages, that is two gigabytes of garbage that cannot be collected. One line fixes it: `self._items[self._head] = None`.

Trap 5 — dequeuing from an empty queue

```
return self._items[self._head]          # no size check
```

On an empty `deque`-backed queue:

```
IndexError: pop from an empty deque
```

On a list-backed one:

```
IndexError: list index out of range
```

Both are unhelpful to a caller. Raise your own with a message that says what happened — `IndexError("dequeue from an empty queue")` — and, more importantly, check for emptiness in the *caller's* loop condition: `while frontier:` is how every BFS is written.

Trap 6 — `while head < tail`

```
while self._head < self._tail:          # WRONG on a wrapped ring
```

Perfectly correct until `tail` wraps around, at which point `tail < head` and the loop does nothing. The queue silently reports itself as empty while holding seven elements. Loop on `_size`, never on the relationship between the markers.

Trap 7 — using `queue.Queue` in a coding round

It is not incorrect, so nothing breaks. It is 48 times slower and it signals that you reached for the first thing whose name matched. If threads come up, name it then — that is when it is the right answer.

8 In the interview

How it gets asked

- The direct version: “Implement a queue. What is the complexity of each operation?” Usually followed immediately by “why not just use a list and `pop(0)`?”
- The constrained version: “Implement a queue using a fixed-size array.” LeetCode 622, Design Circular Queue. This is the ring buffer, and the full-versus-empty question is the point of it.
- The applied version: “Print a binary tree level by level.” LeetCode 102. The queue is the whole answer and they want to see you reach for it without prompting.
- The structural version, which is tomorrow and the day after: “Implement a queue using only stacks”, and “find the maximum of every window of size k ”.

What to say out loud, in the first ninety seconds

1. **Define it by the restriction.** “A queue is FIFO — I add at the back and remove from the front, and I never touch the middle. That restriction is why all four operations can be $O(1)$.”
2. **Name the four operations and their costs immediately.** “Enqueue, dequeue, peek, is-empty, all $O(1)$. If any of them is $O(n)$ I have not built a queue.”
3. **Kill the list version before they ask.** “The tempting implementation is a list with `append` and `pop(0)`, and it is wrong: `pop(0)` shifts every remaining element one slot left, so it is $O(n)$, and draining the queue is $O(n^2)$. I measured it once — a hundred thousand elements took ten seconds against three milliseconds.”
4. **Then give the fix, in order of sophistication.** “The first fix is to keep a head marker and never shift, which leaks the dead prefix. The proper fix is a ring buffer: fixed slots, both markers wrap with modulo, nothing ever moves.”
5. **Flag the hard part yourself.** “The one subtlety is that `head == tail` means both empty and full, so I keep a size counter.”
6. **Close with the real-world line.** “In production I would use `collections.deque`, which is a linked list of blocks so both ends are $O(1)$. Not `queue.Queue` — that locks on every operation for thread safety I do not need here.”

The follow-ups

“**Why is `pop(0)` $O(n)$? Explain it in terms of memory.**” “A Python list is a contiguous block of slots, and the list object stores a pointer to the start plus the length. Element `i` is found by adding `i` to the start address, and that arithmetic only works if element 0 really is at the start. So removing the first element means moving all `n - 1` remaining elements one slot left to restore that. Removing from the *end* moves nothing, which is why the same list is a perfectly good stack.”

“**How do you tell full from empty in a ring buffer?**” “You cannot from the markers alone — both states have `head == tail`. Two standard answers. Keep an explicit size counter, which is one integer and makes empty, full and length all trivial. Or waste one slot so the buffer is considered full at `capacity - 1`, which makes `head == tail` mean empty and only empty; that is what you do when memory for the counter genuinely matters, in an embedded context. I would use the counter.”

“**What happens when it fills up?**” “That is a product decision, not a code decision, and I would ask. Three sensible behaviours: grow — double the slots and unroll the ring while copying, which keeps enqueue amortised $O(1)$. Reject — raise or return false, which is what a bounded work queue should do so that back-pressure reaches the producer. Or drop the oldest — overwrite at `head` and advance both markers, which is what a metrics buffer or an audio buffer wants, because stale data is worthless.”

“**Give me a real use for a queue.**” “Breadth-first search, and everything shaped like it. Level-order traversal of a tree is the same code. The property I am relying on is that FIFO order visits everything at distance one before anything at distance two, which is what makes BFS find

shortest paths in an unweighted graph. Outside algorithms: job runners, rate limiters, and the buffer on a network card, which is a ring buffer exactly like this one.”

“Can you make it thread-safe?” “Wrap enqueue and dequeue in a lock, which is exactly what `queue.Queue` does — and that is why it is about fifty times slower than a `deque` in a single thread. `deque`’s `append` and `popleft` happen to be atomic in CPython because of the global interpreter lock, so a single-producer, single-consumer `deque` is safe without a lock, but I would not rely on that in code that other people maintain.”

A model answer

Asked: *implement a queue, and tell me why you would not just use a list with `pop(0)`.*

“A queue is first in, first out: I add at the back, remove from the front, and never touch the middle. The restriction is the point — because I only ever work at the two ends, every operation can be constant time. Enqueue, dequeue, peek and is-empty, all $O(1)$. If any of those turns out to be $O(n)$, I have not built a queue.

Now, why not a list with `append` and `pop(0)`. A Python list is a contiguous block of slots, and the list holds the address of the start. Element `i` lives at start plus `i`, which only works if element 0 is genuinely at the start. So `pop(0)` has to move every remaining element one slot left to restore that, which is $O(n)$. Draining a queue of n elements that way is n plus $n - 1$ plus $n - 2$ and so on, which is about $n^2/2$ — five billion slot moves at a hundred thousand elements. I have measured it: a hundred thousand took about ten seconds, and two hundred thousand took forty. Double the input, quadruple the time. That is the signature of quadratic behaviour, and it is exactly the failure that only shows up once the system is busy.

Appending at the end is fine, though, which is why the same list makes a perfectly good stack. The front is the expensive end; the back is not.

The first fix is to stop shifting: leave the element where it is and keep a head marker that moves forward. Dequeue is $O(1)$ now, but the slots before the marker are dead and the list grows for ever, so it leaks.

The proper fix is a ring buffer. A fixed block of slots, a head marker for the front and a tail marker for the next free slot, and both wrap around with modulo capacity, so slot capacity $- 1$ is followed by slot 0. Nothing is ever moved. Enqueue writes at the tail and advances it; dequeue reads at the head, clears that slot so the object can be collected, and advances the head.

The subtlety worth flagging before you ask about it: when `head == tail`, the buffer is either completely empty or completely full and the markers cannot tell you which. I keep an explicit size counter, which makes empty, full and length all one comparison. The alternative is to waste one slot so full means `capacity - 1`, and I would only do that if the counter's memory genuinely mattered.

If it needs to be unbounded, I grow by doubling — and when I copy, I have to walk forward from head with wraparound rather than copying slot by slot, or the order is wrong whenever tail is behind head. Doubling makes enqueue amortised $O(1)$, the same argument as list growth.

In production I would not write any of this. I would use `collections.deque`, which is a doubly linked list of fixed-size blocks, so growing at either end never moves anything and both ends are $O(1)$. What I would not use is `queue.Queue` — it takes a lock on every operation for thread safety, and in a single-threaded program that costs about fifty times the runtime for nothing.”

9 Recall card

- A queue is FIFO: add at the back, remove from the front, never touch the middle — and that restriction is *why* enqueue, dequeue, peek and is-empty are all $O(1)$. It is the mirror of the stack. Every BFS and every level-order traversal is a queue.
- `list.pop(0)` is $O(n)$ because a list is a contiguous block and element 0 must stay at the start, so everything shifts left. Draining is $O(n^2)$: measured 10.2 s vs 0.0034 s at $n = 100,000$ ($\sim 3,000\times$), and $n \rightarrow 2n$ gave $3.9\times$ the time, which is the quadratic signature. The back of a list is the cheap end — same list, perfectly good stack.
- The ring buffer is the answer they want: fixed slots, `head` for the front, `tail` for the next free slot, both advanced with `(marker + 1) % capacity`. Nothing ever moves. Clear the dequeued slot to `None` or the objects are never collected.
- `head == tail` means empty *and* full — keep a `_size` counter (or waste one slot). Never loop on `head < tail`; it breaks the moment tail wraps. Read the ring in order with `(head + i) % capacity`, never by slot number. Growing = double and unroll while copying, amortised $O(1)$.
- In production: `collections.deque` — a linked list of blocks, both ends $O(1)$. Not `queue.Queue`, which locks on every operation and measured $48\times$ slower single-threaded; name it only when the question is producer–consumer threads.

1 What this is, and why they ask it

State is the pattern where an object's behaviour changes because of what it currently *is*, and you model that by giving each condition its own class. The object — the **context** — holds one of those classes and forwards every request to it. Change the state, and the same method call does something different.

The thing it replaces is a set of `if status == "shipped"` checks scattered across a dozen methods. That set always starts as one innocent `if`, and it always ends as the file nobody wants to open.

They ask it because every real system has something that moves through stages — an order, a payment, a support ticket, a document under review, a taxi ride — and because the interesting part is not the happy path. The interesting part is what happens when someone tries to cancel a delivered order, or when two workers try to ship the same order at once. Expect it in low-level design rounds, and expect it as a follow-up in almost any “design the order flow” or “design a booking system” question.

2 The story

Suresh ordered a mixer for his mother's birthday from a shop two districts away, and then spent four days phoning them.

The first call was on Monday evening. He had given the old house address by mistake. The girl who answered said no problem at all, put him on hold, and came back thirty seconds later: changed. She had asked the packing room, and the packing room still had the box, so the address was just a matter of writing a new one on it.

The second call was Wednesday morning. He wanted to add a second mixer for his sister. This time the answer was no. The box had gone out on the van at six that morning, and the girl said, quite plainly, “it is with the driver now, I cannot add anything to a box that has left.” She offered to place a second order instead.

The third call was Wednesday afternoon, because his mother had gone to a wedding and would not be home. That one the driver could handle — he brought it back and it went out again on Thursday.

The fourth call was Friday, after it had been delivered and after his mother had said, very kindly, that she already had one. And now it was somebody else entirely on the line: the returns desk, who asked for the delivery date, said he had seven days, and arranged a pickup.

What Suresh worked out, somewhere around the third call, is that there is no single person at that shop who knows all the rules. The girl who answers the phone just finds out who is holding the box right now and puts him through. Before it is packed, the packing room holds it and can do almost anything. Once it is on the van, only the driver can help, and only in the two ways a driver can help. Once it is in the house, only the returns desk, and only for a week.

And when the packing room hands the box to the driver, the packing room stops answering questions about it. They do not keep half the responsibility. They hand it over completely, and the next person takes all of it.

3 The idea in plain English

The box is the **context** — the object that has a life and moves through stages. The packing room, the driver and the returns desk are the **states**. The girl on the phone is the context forwarding every request to whoever currently holds the box.

The four moves

One: each stage becomes its own object. `PlacedState`, `PaidState`, `ShippedState`, `DeliveredState`, `CancelledState`. Each one is small, because it only has to know the rules of its own stage.

Two: they all implement the same interface. Whatever an order can be asked to do — pay, ship, deliver, cancel — is a method on every state. The girl asks the same question of everyone; the answer differs.

Three: the context holds one state and forwards to it. `Order.cancel()` is one line: `self._state.cancel(self)`. The order does not decide anything itself. It passes the request to the object that currently holds it.

Four: states cause transitions. When the driver picks up the box, the packing room stops being responsible. In code, a state's method sets the context's state to a different one, and after that line the old state is never consulted again.

```
class PaidState:
    def ship(self, order: "Order") → None:
        order.tracking_id = generate_tracking_id()
        order.state = ShippedState()           # hand it to the driver
```

That single assignment is the transition. Everything the object can now do changed, and no `if` anywhere had to be edited.

What it replaces, and why that thing rots

Without the pattern you write this:

```
def cancel(self) → None:
    if self.status == "placed":
        self.status = "cancelled"
    elif self.status == "paid":
        refund(self.payment_id)
        self.status = "cancelled"
    elif self.status == "shipped":
        raise ValueError("cannot cancel a shipped order")
    elif self.status == "delivered":
        raise ValueError("cannot cancel a delivered order")
```

That is fine. The problem is that the same chain appears in `ship`, in `deliver`, in `refund`, in `can_edit_address`, and in the three places that build the UI. Five states and six operations means the same five-way branch written six times, and **adding a sixth state means finding all six of them**. Miss one and you get the bug where a `returned` order can still be shipped.

State turns that grid on its side. Instead of one method per operation containing every state, you get one class per state containing every operation. Adding a state is **adding a file**. That is the same trade you saw in Strategy on [day 071](#), applied to a different axis.

The rule that makes it not-Strategy

The two patterns have an identical class diagram, and interviewers ask you to tell them apart.

In Strategy, the client chooses the implementation. In State, the object changes its own.

A `Pricer` is handed to `Checkout` from outside and never changes itself. A `PaidState` replaces itself with a `ShippedState` from the inside, and nobody outside chose that. **States know their successors; strategies do not know each other exists**. Say that sentence and the question is over.

Illegal transitions are the design

The valuable part of this model is not that `cancel` works. It is that **`cancel` on a delivered order is impossible to write by accident**, because `DeliveredState` simply does not do it:

```
class DeliveredState:
    def cancel(self, order: "Order") → None:
        raise IllegalTransition("a delivered order cannot be cancelled; use return")
```

Put that default in a base class and every state gets it for free, then each state overrides only what it genuinely allows. Now a new state that forgets to implement `ship` refuses to ship, which is the safe direction to fail in. **Design so that forgetting something makes it stricter, not looser.**

The word you will hear: finite state machine

This is a **finite state machine** — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which. The State pattern is one way to write one. The other two ways are a dictionary keyed by `(state, event)`, and an `enum` plus a `match`. All three are legitimate; §7 says when to pick which.

4 The picture

The machine itself. Notice how few arrows there are compared with how many are imaginable.

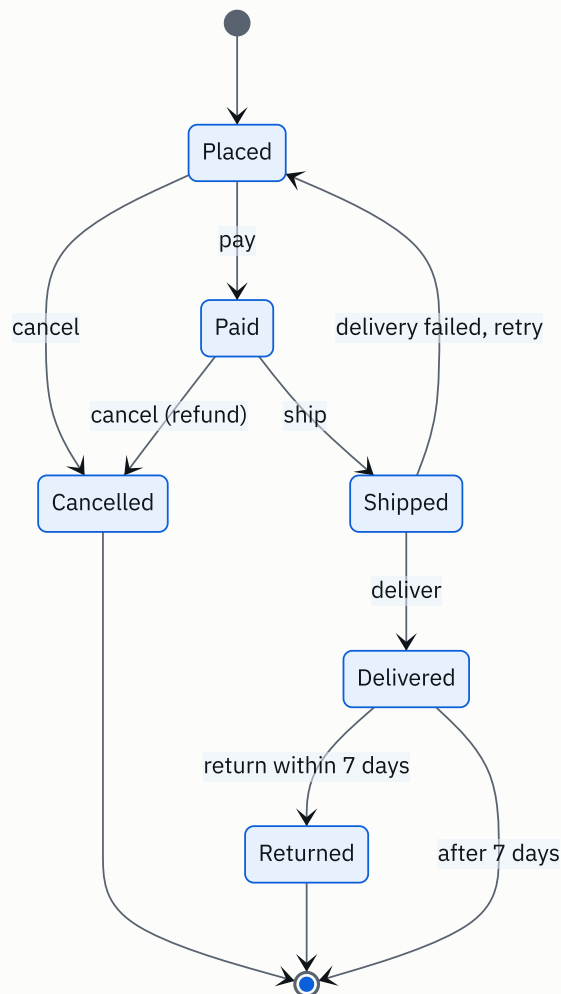


FIGURE 73.1

What to notice: there are 6 states and 5 events, so there are 30 imaginable combinations — and only **8 arrows**. The other 22 are illegal, and the whole point of modelling this properly is that the 22 become impossible rather than merely discouraged. Also notice the one backwards arrow, Shipped → Placed for a failed delivery. Real machines have them, and a design that assumes everything moves forward will need surgery the first time a delivery fails.

Now the structure:

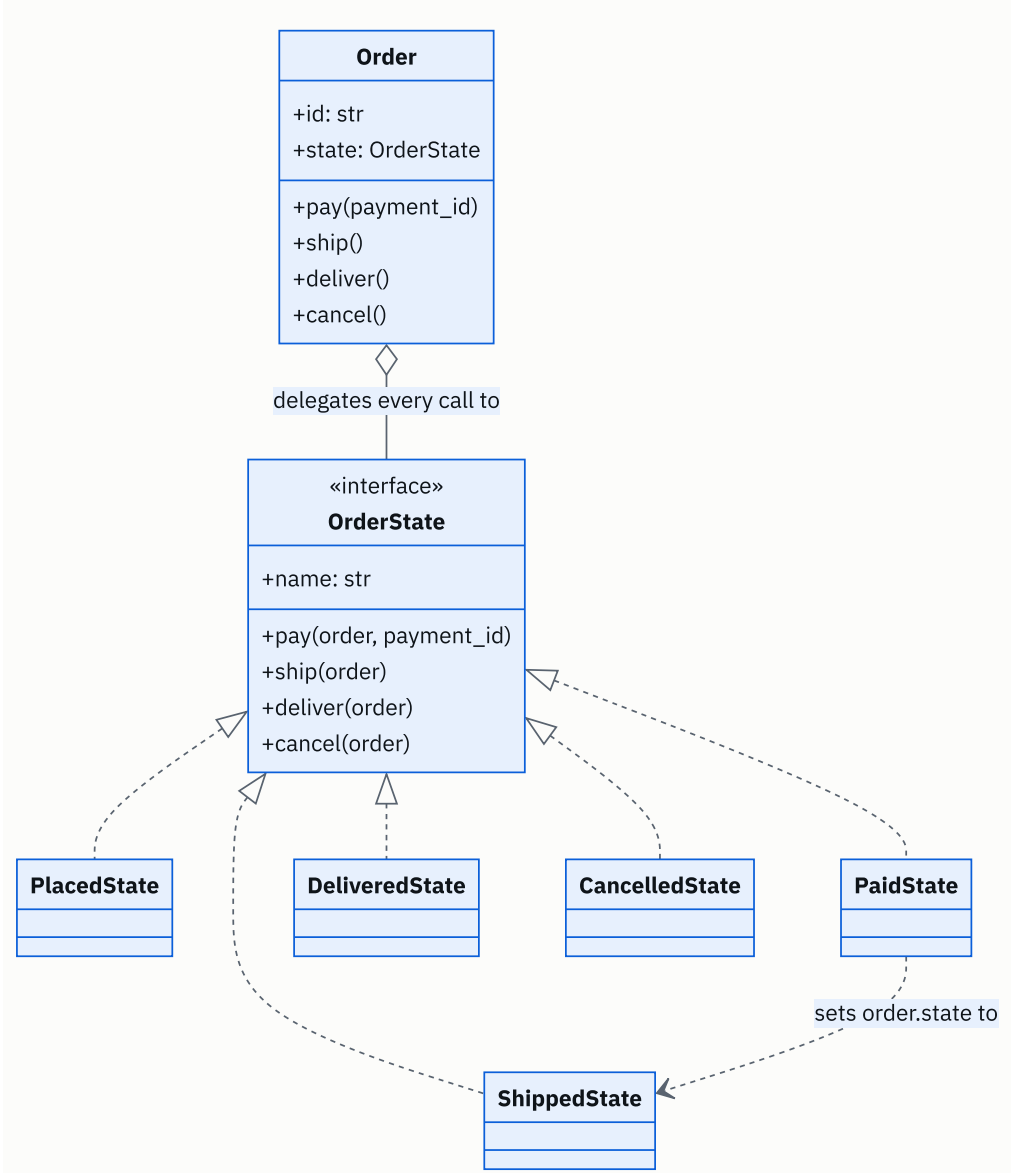


FIGURE 73.2

What to notice: the dashed arrow from `PaidState` to `ShippedState`. **That arrow is what makes this State and not Strategy** — one implementation knows about another and installs it. In a Strategy diagram no such arrow exists.

5 How it actually works

The interface, with safe defaults

```
class IllegalTransition(Exception):
    """Raised when an operation is not legal in the current state."""

class OrderState:
    name: str

    def pay(self, order: "Order", payment_id: str) → None:
        self._refuse("pay")

    def ship(self, order: "Order") → None:
        self._refuse("ship")

    def deliver(self, order: "Order") → None:
        self._refuse("deliver")

    def cancel(self, order: "Order") → None:
        self._refuse("cancel")

    def _refuse(self, action: str) → None:
        raise IllegalTransition(f"cannot {action} an order in state {self.name!r}")
```

Everything refuses by default. Each concrete state overrides only what it allows. This is the design decision that matters most in the whole file: a new state added by a tired engineer on a Friday is **maximally restrictive** until someone deliberately opens it up.

Three of the states

```
class PlacedState(OrderState):
    name = "placed"

    def pay(self, order: "Order", payment_id: str) → None:
        order.payment_id = payment_id
        order.state = PaidState()

    def cancel(self, order: "Order") → None:
        order.state = CancelledState()          # nothing to refund yet
```

Two methods. `ship` and `deliver` are inherited refusals, so an unpaid order cannot be shipped and you did not have to write an `if` to say so.

```
class PaidState(OrderState):
    name = "paid"

    def ship(self, order: "Order") → None:
        order.tracking_id = tracking.create(order.id)
        order.state = ShippedState()

    def cancel(self, order: "Order") → None:
        payments.refund(order.payment_id)      # the extra work this state implies
        order.state = CancelledState()
```

Look at the two `cancel` methods side by side. Same name, same signature, different work — one refunds and one does not. In the `if` chain version that difference is a branch buried in a long method; here it is the entire body of a four-line class.

```
class DeliveredState(OrderState):
    name = "delivered"

    def cancel(self, order: "Order") → None:
        raise IllegalTransition("cannot cancel a delivered order; start a return")
```

Overriding a refusal with a *better* refusal. The generic message says what is not allowed; this one says what to do instead, and error messages that name the alternative are worth writing.

The context

```
class Order:
    def __init__(self, order_id: str, total_paise: int) → None:
        self.id = order_id
        self.total_paise = total_paise
        self.payment_id: str | None = None
        self.tracking_id: str | None = None
        self.state: OrderState = PlacedState()

    def pay(self, payment_id: str) → None:
        self._transition(lambda: self.state.pay(self, payment_id))

    def ship(self) → None:
        self._transition(lambda: self.state.ship(self))
```

Every public method is one line of delegation. The order contains **no rules at all**, which is what makes it readable a year later.

```
def _transition(self, action) → None:
    before = self.state.name
    action()
    if self.state.name ≠ before:
        history.append(f"{self.id}: {before} → {self.state.name}")
```

One place to put the audit trail, the metric and the published event, because every transition goes through it. That is a benefit you do not get from scattered `if`s, and it is worth pointing at: it is exactly where you would publish the `OrderShipped` event to the observers from [day 072](#).

Persisting it

The state is a class in memory and a **string in the database**:

```
STATES = {c.name: c for c in (PlacedState, PaidState, ShippedState,
                             DeliveredState, CancelledState)}

@classmethod
def load(cls, row) → "Order":
    order = cls(row["id"], row["total_paise"])
    order.state = cls.STATES[row["status"]]()      # rebuild from the name
    return order
```

Store the name, not a pickled object, so a deploy that renames a class does not corrupt a million rows. And put a database-level check on the column — `CHECK (status IN ('placed', 'paid', ...))` — so a bug in one service cannot write a state that no code can read.

The two other legitimate implementations

A transition table. When the states have no behaviour of their own and only the *arrows* matter:

```
TRANSITIONS = {
    ("placed", "pay"): "paid",
    ("placed", "cancel"): "cancelled",
    ("paid", "ship"): "shipped",
    ("shipped", "deliver"): "delivered",
}

def apply(status: str, event: str) → str:
    try:
        return TRANSITIONS[(status, event)]
    except KeyError:
        raise IllegalTransition(f"{event!r} is not legal from {status!r}")
```

Eight lines, and the whole machine is visible in one place — which is a real advantage over five files. Use this when transitions are the entire content. Use classes when each state carries genuine work, like the refund inside `PaidState.cancel`.

An enum with `match`. Python 3.10 and later:

```
match (order.status, event):
    case (Status.PAID, "ship"):
        ...
```

Readable for small machines, and it becomes the same unmaintainable chain at fifteen states.

Where you have already seen this

- **TCP** is a finite state machine, and a famous one: `CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_1 → TIME_WAIT → CLOSED`. Every diagram in [day 004](#) is this pattern, and `netstat` prints the current state of each connection.

- **Stripe's** `PaymentIntent` moves through `requires_payment_method`, `requires_confirmation`, `processing`, `succeeded`, `canceled`. Their API documentation is a state diagram, and the webhook you receive names the transition.
- **Kubernetes pod phases** — `Pending`, `Running`, `Succeeded`, `Failed`, `Unknown` — with a controller whose entire job is driving objects toward the state you asked for.
- **django-fsm** and **AASM in Rails** are libraries that add exactly this to a model class, with a decorator naming the legal source and target states.
- **AWS Step Functions** and **Temporal** are state machines as a hosted service, for when the transitions take days and must survive a process restart.
- **XState** in the JavaScript world, where the machine is defined as data and can be drawn automatically — which is the strongest argument for the table form.

6 The numbers

The size of the thing you are replacing

Six states and six operations:

```
if-chain form:
  6 methods × a 6-way branch each      = 36 branches to maintain
  adding a 7th state: edit 6 methods, find all of them, re-test all of them

state-object form:
  6 classes × only the operations each allows
  Placed 2, Paid 2, Shipped 2, Delivered 1, Cancelled 0, Returned 0 = 7 methods written
  adding a 7th state: 1 new file, 0 existing methods edited
```

36 branches against 7 methods, because 29 of the 36 combinations are refusals and the base class writes them all once. That ratio is the argument, and it is a number rather than an opinion.

The transition matrix

```
6 states × 5 events      = 30 possible (state, event) pairs
legal transitions        = 8
illegal                  = 22 (73% of the grid)
```

Nearly three-quarters of what could be asked must be refused. A design where refusal is the default gets 22 cases right by doing nothing; a design where each refusal is an `if` gets them right only if somebody wrote 22 `if`s.

What the audit trail costs

Every transition writes one history row:

```
1,000,000 orders/month × 4 transitions each = 4,000,000 rows/month
row: order_id 16 B + from 12 B + to 12 B + actor 16 B + timestamp 8 B + overhead = 100 B
4,000,000 × 100 B = 400 MB/month → 4.8 GB/year
```

Five gigabytes a year for a complete history of every order's life. That is nothing against what it buys you in a support call, and quoting it is how you win the argument for keeping it.

The concurrency number, which is the one they are actually testing

Two workers read the same paid order at the same moment and both call `ship`:

```
worker A: read status 'paid' → create tracking → write 'shipped'
worker B: read status 'paid' → create tracking → write 'shipped'
result: two tracking numbers, two parcels, one order
```

The in-memory state machine cannot prevent this, because both workers hold their own copy of the object. The fix is at the database, and it is one line — a conditional update:

```
UPDATE orders SET status = 'shipped' WHERE id = $1 AND status = 'paid';
```

If it reports 0 rows updated, somebody else won the race and this worker must stop. At 500 shipments per second with two workers, the collision window is milliseconds wide and it *will* happen daily. **Say this before you are asked.** It is the difference between a candidate who has drawn state machines and one who has run them.

7 The trade-offs

What you give up

A class per state, for machines that do not deserve one. Six states is six files. If your states carry no behaviour and the only content is which arrow goes where, a dictionary of `(state, event) → state` says the same thing in eight lines and shows the whole machine on one screen. Reach for classes when states carry *work*, not merely names.

The machine stops being visible in one place. With a table you can see all eight transitions at once. With classes they are scattered across six files, one assignment at a time, and answering “can a shipped order be cancelled?” means opening a file. Mitigation: keep the diagram in the repository next to the code, and write one test that asserts the full set of legal transitions.

States knowing about each other is real coupling. `PaidState` imports `ShippedState`, which is a direct dependency between two concrete classes — the thing most patterns exist to avoid. It is usually acceptable, because the graph is small and deliberate, but circular imports appear the moment `ShippedState` needs to go back to `PlacedState`. A `_transition(order, "shipped")` helper that looks up the class by name breaks the cycle at the cost of losing the type checker's help.

Where does the work live? `PaidState.cancel` calls `payments.refund`. Now a state object depends on the payment service, and testing `cancel` needs that service faked. The alternative is for states to only decide and for the context to do the work, which keeps states

pure and makes the context fat again. There is no clean answer; pick one, be consistent, and say which you picked.

Instances or singletons? `PaidState()` allocates a new object on every transition. If states hold no data — and they should not — one shared instance per state is enough, which is the Singleton from [day 064](#) used properly. It matters at a million transitions a minute and not at all below that.

It does nothing for concurrency. This is the big one. The pattern makes illegal transitions impossible *within one object in one process*. Two processes holding two copies can both make a legal transition from the same starting point and produce nonsense. The state machine is a modelling tool; the guarantee has to come from a conditional update, a row lock, or a version column.

“I would not use this if...”

- **...there are two states.** A boolean and one `if . is_active` does not need a class hierarchy.
- **...the states have no behaviour, only names.** Use a table or an enum. Five classes containing one assignment each are ceremony, and the table is easier to review.
- **...the transitions are the *whole* product** — a payment flow with retries, timeouts and manual review that must survive restarts. Then you want a durable workflow engine like Temporal or Step Functions, not objects in memory that die with the process.
- **...the “states” are really independent flags.** `paid`, `gift_wrapped` and `flagged_for_review` can be true in any combination, so they are not states — they are three booleans, and forcing them into one machine gives you eight states that mean nothing.
- **...you cannot draw the diagram.** If the arrows are not clear enough to draw, the requirements are not clear enough to model, and the pattern will freeze the confusion in place.

The honest concession

The branching did not vanish. Python still has to find `PaidState.ship` rather than `ShippedState.ship`, and that lookup is a branch in the interpreter. What changed is **who maintains the branch**: the language does, by method dispatch, instead of you doing it by hand in six different methods. That is worth saying, because the alternative — claiming the `if`’s disappeared — is not true and a good interviewer will press on it.

8 In the interview

How it gets asked

- The direct version: “*Model an order that goes through placed, paid, shipped and delivered. What happens if someone tries to cancel after delivery?*”

- The pattern version: “You have a class with a `status` field and eight methods that all start with the same `if status ==` chain. How would you restructure it?”
- The named version, in a low-level design round: “Design a vending machine” or “design an elevator” — both are state machines wearing a costume, and both are coming later this phase.
- The distinguishing question, which is almost guaranteed: “How is this different from Strategy? The class diagrams look the same.”
- The one that separates people: “Two workers pick up the same order at the same moment. What happens?”

What to say out loud, in the first ninety seconds

1. **Draw the machine before writing any code.** “Let me list the states and the legal transitions first — placed, paid, shipped, delivered, cancelled, returned.” Then draw the arrows. Everything else follows from that picture.
2. **Count the grid out loud.** “Six states, five events, so thirty combinations, and only eight of them are legal. So the design question is really: how do I make the other twenty-two impossible rather than merely discouraged?”
3. **Name the structure.** “One class per state, all implementing the same interface, with a base class that refuses everything. Each state overrides only what it permits, so forgetting something makes it stricter.”
4. **Show the delegation.** “`Order.cancel` is one line: `self._state.cancel(self)`. The order holds no rules. That also gives me one place to write the audit trail and publish the event.”
5. **Pre-empt the Strategy question.** “This looks like Strategy, and the difference is direction: in Strategy the client picks the implementation; in State the object replaces its own, so states know their successors.”
6. **Raise concurrency yourself.** “One thing the pattern does not give me is safety across processes. I would enforce transitions with a conditional update — `WHERE id = ? AND status = 'paid'` — and treat zero rows updated as losing the race.”

The follow-ups

“How is this different from Strategy?” “Identical class diagram, opposite direction of control. A strategy is chosen from outside and never changes itself — `Checkout` is handed a pricer and that pricer has no idea other pricers exist. A state changes itself from inside: `PaidState.ship` sets `order.state = ShippedState()`. So states know their successors and strategies do not. The tell in code is an assignment to the context’s own field inside an implementation.”

“What happens if someone tries to cancel a delivered order?” “It raises. `DeliveredState` overrides `cancel` with a message saying to start a return instead. And importantly, that is not the only reason it fails — the base class refuses everything by default, so even if nobody had written that override it would still refuse. The system fails closed.”

“Where do you store the state?” “A string column with a `CHECK` constraint listing the valid names, and I rebuild the state object from the name on load. Never a pickled object, because renaming a class would then corrupt every existing row. If I need history I add a separate transitions table — one row per change with from, to, actor and timestamp. At a million orders a month and four transitions each that is about 400 MB a month, which is cheap for what it gives support.”

“Two workers try to ship the same order at once.” “The pattern cannot help, because each worker has its own copy of the object and both see a legal transition. The guarantee has to be at the database: `UPDATE orders SET status = 'shipped' WHERE id = ? AND status = 'paid'`, and if it reports zero rows updated then somebody else won and this worker stops. That is optimistic concurrency, and it is one line. The alternative is `SELECT ... FOR UPDATE` to take a row lock first, which is simpler to reason about and holds a lock for the length of the transaction.”

“Would you always use classes for this?” “No, and I would ask what the states actually do. If they carry real work — a refund on cancelling a paid order, a tracking number on shipping — classes are right, because that work has a natural home. If the states are only names and the content is which arrow goes where, a dictionary keyed by state and event is eight lines and shows the entire machine on one screen, which is easier to review and easier to draw automatically. Five classes containing one assignment each is ceremony.”

“The machine needs to survive a restart and wait three days for a manual review.” “Then it is not an in-memory pattern any more. I would move it to a durable workflow engine — AWS Step Functions or Temporal — where the state is persisted by the engine, timers are first-class, and a process dying mid-transition resumes rather than losing the order. The modeling stays the same; what changes is who owns the state and the clock.”

A model answer

Asked: *model an order that moves through placed, paid, shipped and delivered.*

“Before any code, let me list the states and the legal moves, because the arrows are the design. Placed, paid, shipped, delivered, cancelled, and returned. Pay moves placed to paid. Cancel works from placed and from paid — and from paid it also has to refund, which is the first sign the states are not just labels. Ship moves paid to shipped. Deliver moves shipped to delivered. A failed delivery moves shipped back to placed for a retry, and I would confirm that one with you because systems that assume everything moves forward break the first time a driver cannot find the house. Return moves delivered to returned, within seven days.

That is six states and five events, so thirty possible combinations, and only about eight of them are legal. So the real design question is how to make the other twenty-two impossible rather than merely discouraged.

The version I would avoid is a `status` string and an `if status =` chain inside every method. With six operations that is the same six-way branch written six times, and adding a seventh state means finding all six and editing all six. The bug that gets you is the one you missed — a returned order that can still be shipped.

So: one class per state, all implementing the same interface, and a base class where every operation refuses with a clear message. Each state overrides only what it genuinely allows. `PlacedState` implements pay and cancel and inherits refusals for the rest, so an unpaid order cannot be shipped and I never wrote an `if` to say so. `PaidState.cancel` issues the refund before transitioning, which is exactly the kind of per-state work that justifies classes over a lookup table. `DeliveredState.cancel` overrides the refusal with a better one that names the alternative.

The important property there is the direction of failure: a new state written on a Friday refuses everything until someone deliberately opens it up.

The order itself holds no rules. Every method is one line of delegation to the current state, which means I have exactly one place — a small `_transition` wrapper — to write the audit row, emit the metric, and publish the `OrderShipped` event to whatever is listening.

If you ask how this differs from Strategy: the class diagram is the same and the direction of control is opposite. A strategy is chosen by the client and never changes itself. A state replaces its own — `PaidState` sets the order’s state to `ShippedState` — so states know their successors.

For persistence, the state is a string column with a check constraint, and I rebuild the object from the name on load. Never a serialised object, because a class rename would then break every row. History goes in a separate transitions table, about 400 megabytes a month at a million orders, which support will thank you for.

And the thing this pattern does not solve, which I would raise before you do: it gives me no safety across processes. Two workers can both read a paid order and both legally ship it, and I get two parcels. So the actual guarantee is a conditional update — set status to

shipped where the id matches and the status is still paid — and zero rows updated means I lost the race and must stop. The state machine is the model; the database provides the enforcement.

One honest note: the branching did not disappear. It moved from my `if` chains into method dispatch. What I gained is that the language maintains it instead of me maintaining it in six places.”

9 Recall card

- **State = one class per stage, all behind one interface, and the context delegates every call to the one it currently holds.** `Order.cancel()` is `self._state.cancel(self)` and the order holds **no rules at all** — which also gives you exactly one place for the audit row, the metric and the published event.
- **The base class refuses everything; each state overrides only what it allows.** So a new state fails **closed**, and 22 of the 30 (state, event) pairs are handled by writing nothing. Compare **36 branches across 6 methods** with **7 methods across 6 files** — adding a state is **one new file, zero edits**.
- **State versus Strategy: identical diagram, opposite direction.** *The client chooses a strategy; the object replaces its own state.* **States know their successors** — the tell is `order.state = ShippedState()` inside an implementation. Strategies never reference each other.
- **Three legitimate forms, and the choice is about content:** classes when states carry real work (the refund inside `PaidState.cancel()`) · a `(state, event) → state` **dictionary** when only the arrows matter, because the whole machine then fits on one screen · an enum plus `match` for tiny machines. Persist the state as a **string with a CHECK constraint**, never a pickled object.
- **The pattern gives you no concurrency guarantee, and this is the follow-up that separates people.** Two workers can both legally ship the same paid order. Enforce it in the database: `UPDATE ... WHERE id = ? AND status = 'paid'`, and **0 rows updated means you lost the race**. Real machines: TCP, Stripe `PaymentIntent`, Kubernetes pod phases, `django-fsm`, Step Functions, XState.

DSA topic: Queues: first in, first out **System design topic:** State

Code these, in this order

One rule for the whole set: **never write `pop(0)` again**. If your fingers type it, stop, delete the line, and say out loud why it is $O(n)$ before you type the replacement.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Number of Recent Calls	LeetCode 933 (Easy)	The simplest real queue: things expire from the front while new ones arrive at the back.
2	Design Circular Queue	LeetCode 622 (Medium)	The ring buffer, and specifically whether you can tell full from empty.
3	Binary Tree Level Order Traversal	LeetCode 102 (Medium)	The <code>for _ in range(len(queue))</code> trick that separates one level from the next.
4	Rotting Oranges	LeetCode 994 (Medium)	Multi-source BFS on a grid, where the queue's FIFO order is the passage of time.

On problem 1, measure it

Solve it once with a list and `pop(0)`, once with a `deque`. Then run 100,000 calls through each and time both. Write down the two numbers. This is the measurement you will quote in an interview for the rest of your life, so take it yourself once.

On problem 2, break full-versus-empty on purpose

1. Implement it with `is_empty` as `head == tail` and no size counter.
2. Fill it to capacity, then call `is_empty`. Write down what it says.
3. Add the size counter and run the same sequence.
4. Now implement the other fix — waste one slot — and say what the usable capacity became.

On problem 3, look at the one line that matters

`for _ in range(len(queue))` is evaluated once, before the loop body starts adding children. Change it to `while queue:` and watch the levels merge into one list. Say in one sentence why capturing the length first is what separates level 2 from level 3.

On problem 4, say why FIFO is load-bearing

Replace the queue with a stack and run it. The answer will be wrong. Explain, in one sentence, what property of FIFO makes the number of rounds come out equal to the number of minutes.

The cost drill

Answer each without looking:

1. Why is `list.pop(0)` $O(n)$? Answer in terms of memory layout, not in terms of “it is slow”.
2. Why is `list.append` $O(1)$ but `list.insert(0, x)` $O(n)$?
3. What is the total cost of draining an n -element list queue with `pop(0)`?
4. What did $n = 100,000$ and $n = 200,000$ measure at, and what does the ratio between them prove?
5. What is `collections.deque` internally, and why does that make both ends cheap?
6. When is `queue.Queue` the right answer, and roughly what does it cost when it is not?

The ring drill

Start with a capacity-5 ring, empty. Apply this sequence and write down `head`, `tail`, `size` and the slot contents after every single step:

```
enqueue A, B, C, D, dequeue, dequeue, enqueue E, F, G, dequeue, enqueue H
```

Then answer:

1. At which step does `tail` wrap around?
2. At which step is `tail` less than `head`, and is the queue still valid?
3. Read the queue in order using `(head + i) % capacity` and check it against your working.
4. What happens on the next enqueue, and what are the three reasonable behaviours?

The break-it drill

Trigger each and record the actual output or error text:

1. `dequeue` from an empty ring with no size check. Quote the error.
2. `dequeue` from an empty `deque`. Quote the error.
3. Implement `is_empty` as `head == tail`, fill the buffer, and call it.
4. Loop with `while head < tail` on a wrapped ring. Say what it reports.
5. Print the ring with `[x for x in items if x is not None]` after a wrap. Compare with the correct order.
6. Remove the `items[head] = None` line, push a million large objects through, and watch the memory.
7. Grow the buffer by copying slot-by-slot instead of unrolling from `head`. Find the input where the order comes out wrong.

The when-is-it-a-queue drill

For each, say whether the tool is a stack, a queue, a deque, or none of the three:

1. Undo in a text editor.
2. Print a tree level by level.

3. The shortest path in an unweighted grid.
4. Matching brackets.
5. The maximum of every window of size k.
6. Jobs waiting for a worker, oldest first.
7. The back button in a browser.
8. Requests in the last one second, for a rate limiter.
9. The next greater element for every item.
10. Cards dealt from the top of a shuffled deck.

Two of the ten need both ends. Name them.

The state-machine drill

Draw the order machine from memory — six states, every legal arrow — then answer:

1. How many `(state, event)` pairs exist, how many are legal, and what percentage is refused?
2. Which single arrow goes backwards, and why does a design that forbids backwards arrows break?
3. Which state's `cancel` does extra work, and what is it?
4. Which two states have no outgoing arrows at all?

The refactor drill

Here is the method, and it has five siblings that look exactly like it:

```
def cancel(self) → None:
    if self.status == "placed":
        self.status = "cancelled"
    elif self.status == "paid":
        refund(self.payment_id)
        self.status = "cancelled"
    elif self.status == "shipped":
        raise ValueError("cannot cancel a shipped order")
    elif self.status == "delivered":
        raise ValueError("cannot cancel a delivered order")
```

1. Count the branches across all six methods, and say what adding a seventh state costs.
2. Write the base state class, and say why every method refuses by default.
3. Write `PlacedState` and `PaidState`. Count the lines.
4. Write `Order.cancel`. Count the lines.
5. Add a `Returned` state. Count the files edited, both ways.
6. Now write the same machine as a `(state, event) → state` dictionary. Count those lines too.
7. Say which of the two you would ship for *this* problem, and give the deciding question.

The state-versus-strategy drill

1. Write a two-implementation Strategy — two pricing rules, chosen by the caller.
2. Write a two-state State machine — draft that promotes itself to submitted.
3. Show both class diagrams. Say what is identical.
4. Point at the one arrow that exists in only one of them.
5. Say the one-sentence rule that tells them apart.
6. Given only an implementation file, name the single line of code that tells you which pattern it is.

The concurrency drill

Two workers pick up the same paid order at the same instant and both call `ship`.

1. Say exactly what each worker does, step by step, and what the end result is.
2. Say why the state pattern cannot prevent it.
3. Write the conditional `UPDATE` that does prevent it.
4. Say what the worker must do when the update reports zero rows.
5. Give the alternative using a row lock, and say what it costs.
6. Say which you would choose for 500 shipments a second, and why.

The already-used-it drill

For each, name the states you know and say who owns the transition:

1. A TCP connection.
2. A Stripe `PaymentIntent`.
3. A Kubernetes pod.
4. A pull request on GitHub.
5. An order on any food delivery app.
6. A `Promise` in JavaScript.

One of the six can never go backwards under any circumstance. Name it and say why that matters.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Implement a queue. Why not just use a list and `pop(0)`?* FIFO defined by its restriction, the four operations at $O(1)$, the memory-layout reason `pop(0)` is $O(n)$, the measured numbers and the doubling test, the head-marker fix and why it leaks, the ring buffer with both markers wrapping, the full-versus-empty subtlety flagged before it is asked, and `collections.deque` in production but never `queue.Queue`.

2. *Model an order that moves through placed, paid, shipped and delivered.* The states and arrows drawn first, the size of the grid and how much of it is illegal, one class per state behind one interface with refusal as the default, the one-line delegation and what it buys you, the Strategy distinction, persistence as a checked string column, and the conditional update for two workers racing.
3. *There is a `while` loop and a queue. Is that not $O(n^2)$?* Each element is enqueued once and dequeued once, so the loop body runs `n` times in total, and each operation is $O(1)$ because nothing shifts. Then say what would make it $O(n^2)$ — `pop(0)` — and what that costs at $n = 100,000$.
-

Before you move on

- ☐ I can say the four queue operations and their costs without pausing.
- ☐ I can explain why `pop(0)` is $O(n)$ in terms of memory layout.
- ☐ I measured `pop(0)` against `deque.popleft` myself and can quote both numbers.
- ☐ I ran the doubling test and can say what $3.9\times$ proves.
- ☐ I implemented the ring buffer with wraparound and a size counter.
- ☐ I broke full-versus-empty on purpose and saw a full buffer report itself empty.
- ☐ I know why the dequeued slot must be set to `None`.
- ☐ I can write level-order traversal and explain the `range(len(queue))` line.
- ☐ I drew the order state machine from memory, with the backwards arrow.
- ☐ I can say how many of the thirty combinations are legal, and why refusal is the default.
- ☐ I can state the State-versus-Strategy rule in one sentence and point at the telling line.
- ☐ I can write the conditional UPDATE and say what zero rows updated means.
- ☐ I can say the honest concession: the branching moved into method dispatch, it did not vanish.
- ☐ I answered all three questions above out loud.

Dequeues and the sliding-window maximum

DATA STRUCTURES AND ALGORITHMS

Dequeues and the sliding-window maximum

You can maintain a window maximum in $O(n)$ total using a monotonic deque.

SYSTEM DESIGN

Command and chain of responsibility

You can turn an action into an object, which makes undo and queuing possible.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Find the maximum in every window of size k .*
- *Design undo and redo for a text editor.*

maximum

1 What this is, and why they ask it

A **deque** — “double-ended queue”, pronounced “deck” — lets you add and remove at *both* ends in $O(1)$. A stack uses one end. A queue uses one end for adding and the other for removing. A deque uses all four operations, and today is the problem that needs all four.

The problem: given `[1, 3, -1, -3, 5, 3, 6, 7]` and `k = 3`, report the maximum of every window of three consecutive elements, as the window slides one step at a time. The answer is `[3, 3, 5, 5, 6, 7]`. The obvious solution recomputes the maximum for each window, which is $O(n \cdot k)$. The good solution keeps a small **monotonic deque** of “elements that could still be the maximum” and is $O(n)$ overall.

They ask it because it is the hardest sliding-window problem that still fits in twenty minutes, and because the reasoning is not about code at all. It is one idea: **when a big number arrives, every smaller number before it is worthless for ever** — because the big one is in every future window that contains any of them, and it outlives them all. Candidates who reach for a heap get $O(n \log k)$ and a lecture about why the deque is better. Expect it at Amazon, Google, Microsoft and any company using LeetCode-hard rotation; it is LeetCode 239.

2 The story

Lakshmi has had the snack cart outside the college gate for eleven years, and the only number she cares about is the best day in the last week.

Her rule came from her husband and she has never changed it. If the best day in the last seven has been over two thousand rupees, she buys the good sev, which costs more. If not, she buys the ordinary one. So every evening, after she has scraped down the tava and put the shutters on the cart, she puts the day’s takings into her phone, and then she wants to know the best of the last seven.

For years she did it the slow way. Scroll back, read seven numbers, pick the biggest. Seven numbers every evening, at half past nine, tired, and twice she read the wrong week.

Her son showed her something she now does without thinking.

She keeps a very short second list. On the day he explained it, it had three numbers on it: Tuesday’s 2,400, Friday’s 1,850, and Saturday’s 1,600. Only three, in falling order, and she keeps it for one reason: those are the only days that could still be the best of some future week.

Every evening she does two small things before writing the day down.

First she looks at the end of her short list, at the smallest numbers, and crosses off anything below today's. On the Sunday she took 1,900, so Saturday's 1,600 and Friday's 1,850 both went. They are smaller than today, and they will fall out of the week before today does, so there is no week left in which either of them wins. Gone for good.

Second she looks at the front, at Tuesday's 2,400, and checks the date. Once it is more than seven days back it goes, because it is not in the week any more, however big it was.

Then she adds today at the end.

And now the answer is just the first number on the short list. She does not read seven numbers. She reads one. On a bad week the list has one number on it; on a slowly falling week it has seven. Most evenings it has two or three, and the whole thing takes her four seconds.

3 The idea in plain English

Lakshmi's short list is a **monotonic deque**: a double-ended queue whose contents are always in falling order, kept that way by throwing things out of the back before anything new goes on.

The deque itself

A **deque** is a queue you can push to and pop from at either end, all in $O(1)$:

```
from collections import deque

d = deque()
d.append(5)      # add at the BACK      → [5]
d.appendleft(9)  # add at the FRONT     → [9, 5]
d.pop()         # remove from the BACK  → 5, leaving [9]
d.popleft()     # remove from the FRONT → 9, leaving []
d[0], d[-1]     # look at front and back
```

You met it in passing on [day 031](#) and as the right queue on [day 073](#). Today it earns its two ends.

The one idea

Say this before writing any code, because it is the solution:

When a new element arrives, every earlier element smaller than it can be thrown away for ever.

Why "for ever"? Take Friday's 1,850 and Sunday's 1,900. Any future week that contains Friday also contains Sunday, because Sunday is later. Sunday is bigger. So Friday can never be the maximum of any window from now on. It is not merely unlikely — it is impossible, and that is what lets you delete it rather than store it.

So what remains is exactly: **the elements that are the maximum of the window starting at their own position**. And they are automatically in falling order, because anything that broke the order was deleted on arrival.

The two removals, and why you need both ends

- **From the back** — throw out elements smaller than the newcomer. That is the rule above. This is the *stack* half of the structure, exactly the monotonic stack of [day 071](#).
- **From the front** — throw out the element that has slid out of the window. Being big does not save you from being old. This is the *queue* half.

One structure, both behaviours, which is precisely why a deque and not a stack or a queue.

Store indices, not values

The front removal asks “is this element still inside the window?”, and you cannot answer that from a value. You need the position. So the deque holds indices, and you compare with `numbers[index]` when you need the value. Same rule as [day 071](#), for the same reason, and it is the first thing that breaks if you ignore it.

The expiry check is: the window covering positions `i-k+1 ... i` no longer contains index `j` when `j ≤ i - k`.

The three lines, per element

```
if window and window[0] ≤ index - k:      # 1. drop the expired front
    window.popleft()
while window and numbers>window[-1]] ≤ value: # 2. drop the smaller backs
    window.pop()
window.append(index)                       # 3. the newcomer joins
```

Then the answer for this window, once there is a full window to answer for, is `numbers>window[0]]`.

Step 1 uses `if`, not `while`, and that is worth a sentence: the window slides by exactly one position per step, so at most one index can expire per step. A `while` is also correct and costs nothing; use it if it makes you feel safer, and say why the `if` is enough.

Why not a heap?

A max-heap gives you the maximum in $O(1)$ and insertion in $O(\log k)$, but it cannot remove the element that just left the window, because that element is buried somewhere in the middle. Two workarounds:

- **Lazy deletion** — push `(-value, index)`, and before reading the top, pop anything whose index has expired. Correct, $O(n \log n)$, and it works. Measured below at about $1.8 \times$ the deque’s runtime.
- **A heap with a position map** — more code, more bugs, no better complexity.

The deque wins because it never stores anything it will not use. Say that: *the heap keeps everything and filters at read time; the deque throws away at write time, so it only ever holds elements that still have a chance.*

The heap answer is still worth mentioning in the interview. It shows you know the alternative, and “correct at $O(n \log k)$, and here is why I would not” is stronger than never mentioning it.

The family

- **Sliding-window minimum** — flip the comparison; the deque becomes increasing.
- **Longest subarray where $\max - \min \leq \text{limit}$** (LeetCode 1438) — run *two* deques at once, one for the max and one for the min, and shrink the window while their fronts differ by more than the limit.
- **Shortest subarray with sum at least K** (LeetCode 862) — a monotonic deque over prefix sums, with negatives allowed, which is why a plain sliding window fails there.
- **Jump Game VI** and **Constrained Subsequence Sum** — dynamic programming where the transition is “the best value in the last k cells”, which is this exact structure.

4 The picture

`numbers = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3`. The deque holds **indices**; the values are shown beneath in brackets.

index	0	1	2	3	4	5	6	7
value	1	3	-1	-3	5	3	6	7

i=0 v=1	nothing expires; nothing smaller	push 0	deque: [0]	(1)
i=1 v=3	nothing expires; $1 \leq 3 \rightarrow$ pop 0	push 1	deque: [1]	(3)
i=2 v=-1	nothing expires; $3 > -1$, keep window 0..2 full $\rightarrow$ answer 3	push 2	deque: [1,2]	(3,-1)
i=3 v=-3	front is 1, $1 > 3-3=0$, stays $-1 > -3$, keep window 1..3 $\rightarrow$ answer 3	push 3	deque: [1,2,3]	(3,-1,-3)
i=4 v=5	front is 1, $1 \leq 4-3=1 \rightarrow$ EXPIRED, popleft $-3 \leq 5 \rightarrow$ pop 3 ; $-1 \leq 5 \rightarrow$ pop 2 window 2..4 $\rightarrow$ answer 5	push 4	deque: [4]	(5)
i=5 v=3	front is 4, still inside; $5 > 3$, keep window 3..5 $\rightarrow$ answer 5	push 5	deque: [4,5]	(5,3)
i=6 v=6	front is 4, still inside $3 \leq 6 \rightarrow$ pop 5 ; $5 \leq 6 \rightarrow$ pop 4 window 4..6 $\rightarrow$ answer 6	push 6	deque: [6]	(6)
i=7 v=7	$6 \leq 7 \rightarrow$ pop 6 window 5..7 $\rightarrow$ answer 7	push 7	deque: [7]	(7)

answers: [3, 3, 5, 5, 6, 7]

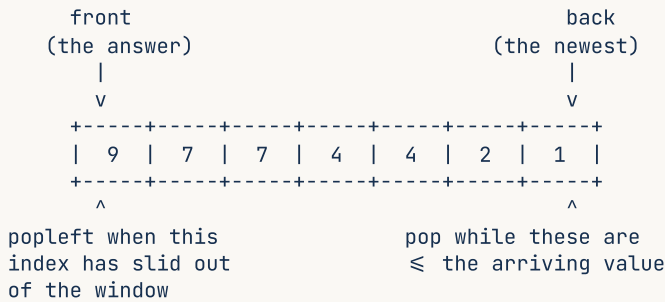
Three things to notice.

At `i=4`, **both** removals happened in one step: one index expired off the front, and two were crushed off the back by the 5. That is the only step in this trace where both ends were used, and it is why the structure needs both.

The deque never held more than three indices, even though the array has eight elements. Its size is bounded by `k`, and usually far below it.

And the values in the deque, read front to back, are always falling: `(3, -1, -3)`, `(5, 3)`. Nobody sorted them. Anything that would have broken the order was popped before it could join.

Now the shape of the structure itself:



The front is the only place you ever *read*. The back is the only place you ever *push*. Both ends are places you remove from, for two completely different reasons — too old at the front, too small at the back.

5 The code, built step by step

Step 1 — the brute force, with a number

```
def max_window_brute(numbers: list[int], k: int) → list[int]:  
    return [max(numbers[i:i + k]) for i in range(len(numbers) - k + 1)]
```

One line, obviously correct, and $O(n \cdot k)$. At $n = 200,000$ and $k = 1,000$ that is two hundred million comparisons — measured at **1.98 seconds** below. Write it as the reference implementation, say the number, and move on.

Step 2 — the observation, said before any code

“If a new number arrives that is bigger than something already waiting, that older, smaller number can never be the answer again. Every future window containing it also contains the new one, because the new one is later. So I can delete it rather than remember it.”

That sentence is the whole interview. Everything after it is typing.

Step 3 — expire the front

```
if window and window[0] ≤ index - k:  
    window.popleft()
```

The window at position `index` covers `index - k + 1 ... index`. An index at or below `index - k` has fallen out. Only one can fall out per step, because the window moves by one.

Step 4 — crush the back

```
while window and numbers[window[-1]] ≤ value:  
    window.pop()
```

`≤` rather than `<` throws out equal values too. Both are correct — equal elements are interchangeable as maxima — but `≤` keeps the deque smaller, so use it and be able to say why the choice does not affect the answer.

Step 5 — record, once the window is real

```
if index ≥ k - 1:  
    answers.append(numbers[window[0]])
```

The first complete window ends at index `k - 1`. Before that there is nothing to report. Forgetting this guard is the most common wrong answer: you get `n` results instead of `n - k + 1`.

The complete solution

```
from collections import deque

def max_sliding_window(numbers: list[int], k: int) → list[int]:
    """The maximum of every window of k consecutive elements.

    The deque holds INDICES of elements that could still be the maximum of
    some future window, with their values falling from front to back.

    Two removals, for two different reasons:
        front – the index has slid out of the window (too old)
        back – the value is ≤ the arriving value (too small, for ever)

    Each index is appended once and removed at most once, so this is O(n)
    despite the inner while loop. Space is O(k).
    """
    if k ≤ 0 or not numbers:
        return []

    window: deque[int] = deque()      # indices, values falling front→back
    answers: list[int] = []

    for index, value in enumerate(numbers):
        if window and window[0] ≤ index - k:      # 1. too old
            window.popleft()

        while window and numbers[window[-1]] ≤ value:  # 2. too small, for ever
            window.pop()

        window.append(index)                    # 3. the newcomer

        if index ≥ k - 1:                      # 4. a full window exists
            answers.append(numbers[window[0]])

    return answers

def min_sliding_window(numbers: list[int], k: int) → list[int]:
    """The same thing with the comparison flipped. The deque is now RISING
    from front to back, and the front is the minimum."""
    if k ≤ 0 or not numbers:
        return []

    window: deque[int] = deque()
    answers: list[int] = []

    for index, value in enumerate(numbers):
        if window and window[0] ≤ index - k:
            window.popleft()
        while window and numbers[window[-1]] ≥ value:  # ≥ instead of ≤
            window.pop()
        window.append(index)
        if index ≥ k - 1:
            answers.append(numbers[window[0]])

    return answers

def max_window_brute(numbers: list[int], k: int) → list[int]:
    """O(n*k) reference. Correct, obvious, and far too slow."""
    if k ≤ 0 or not numbers:
        return []
    return [max(numbers[i:i + k]) for i in range(len(numbers) - k + 1)]

def max_window_heap(numbers: list[int], k: int) → list[int]:
    """The alternative worth naming and then rejecting.

    A heap cannot remove the element that just left the window, so you push
```

```

(-value, index) and discard expired entries lazily from the top. Correct,
O(n log n), measured at about 1.8x the deque's runtime.
"""
import heapq

if k ≤ 0 or not numbers:
    return []

heap: list[tuple[int, int]] = []
answers: list[int] = []
for index, value in enumerate(numbers):
    heapq.heappush(heap, (-value, index))
    while heap[0][1] ≤ index - k:        # discard whatever has expired
        heapq.heappop(heap)
    if index ≥ k - 1:
        answers.append(-heap[0][0])
return answers

def longest_subarray_within_limit(numbers: list[int], limit: int) → int:
    """LeetCode 1438: the longest window where max - min ≤ limit.

    Two deques at once – one falling for the maximum, one rising for the
    minimum – and the window shrinks from the left while their fronts differ
    by more than the limit.
    """
    highs: deque[int] = deque()          # indices, values falling
    lows: deque[int] = deque()           # indices, values rising
    left = 0
    best = 0

    for right, value in enumerate(numbers):
        while highs and numbers[highs[-1]] ≤ value:
            highs.pop()
        highs.append(right)
        while lows and numbers[lows[-1]] ≥ value:
            lows.pop()
        lows.append(right)

        while numbers[highs[0]] - numbers[lows[0]] > limit:
            left += 1                                # shrink from the left
            if highs[0] < left:
                highs.popleft()
            if lows[0] < left:
                lows.popleft()

        best = max(best, right - left + 1)

    return best

if __name__ == "__main__":
    print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3)) # [3, 3, 5, 5, 6, 7]
    print(max_sliding_window([9, 8, 7, 6], 2))              # [9, 8, 7]
    print(max_sliding_window([1, 2, 3, 4], 2))              # [2, 3, 4]
    print(max_sliding_window([5, 5, 5, 5], 2))              # [5, 5, 5]
    print(max_sliding_window([7], 1))                      # [7]
    print(max_sliding_window([], 3))                       # []
    print(min_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3)) # [-1, -3, -3, -3, 3, 3]
    print(longest_subarray_within_limit([8, 2, 4, 7], 4))   # 2

    import random
    for _ in range(3000):
        n = random.randint(1, 12)
        k = random.randint(1, n)
        sample = [random.randint(-5, 5) for _ in range(n)]
        assert max_sliding_window(sample, k) == max_window_brute(sample, k), (sample, k)
        assert max_window_heap(sample, k) == max_window_brute(sample, k), (sample, k)
    print("agreed with brute force on 3000 random inputs")

```

The random cross-check is worth twenty seconds in an interview. This problem has three separate off-by-one opportunities — the expiry comparison, the record guard, and the window arithmetic — and small random inputs catch all three immediately.

6 What it costs

Time

The `for` loop runs `n` times, so there are exactly `n` appends — one per element.

The `while` and the `popleft` only remove. **An index that has been removed never returns.** So across the whole run, removals are at most `n`.

```
appends across the whole run:    exactly n
removals across the whole run:   at most n
-----
total deque operations:          ≤ 2n    →  O(n)
```

Same counting argument as the monotonic stack, and it needs saying with the same confidence: **the inner `while` is bounded in total, not per iteration.** One arrival can crush ten thousand elements off the back, and that is fine, because those ten thousand were each appended exactly once.

Against the alternatives, measured

`n = 200,000`, `k = 1,000`, on this machine:

```
brute force, max() per window    1.984 s    O(n*k) = 2 x 10^8 comparisons
heap with lazy deletion          0.0922 s    O(n log n)
monotonic deque                  0.0500 s    O(n)
-----
deque vs brute force:    about 40x
deque vs heap:          about 1.8x
```

Two useful things in those numbers. The deque beats the brute force by a factor that grows with `k` — at `k = 10,000` the gap would be ten times wider again. And the heap is only $1.8\times$ slower, which is worth knowing honestly: if you can only remember the heap version under pressure, write it. It is a correct answer, and $1.8\times$ is not the difference between passing and failing. The deque is the better answer; the heap is not a wrong one.

Space

```
deque:    O(k)    at most k indices
answers:  O(n-k+1) - the output, so it does not count as extra
```

The deque holds `k` indices in the worst case, which is a strictly decreasing input like `[9, 8, 7, 6, 5]` where nothing is ever crushed off the back. On a strictly increasing input it holds exactly one, because every arrival wipes the whole thing out. Name both.

Note that this is **$O(k)$** , not $O(n)$. For $n = 10^6$ and $k = 3$ the deque holds three indices. That independence from `n` is a genuine selling point when the interviewer asks about streaming data.

7 The traps

Trap 1 — storing values instead of indices

```
window.append(value)           # instead of index
```

Everything still runs. The back removal works fine. But the front removal now has no way to ask “has this slid out of the window?”, because a value carries no position. People patch it by also tracking a count, and the patch is always wrong on duplicates. **Store indices**. It costs one extra lookup, `numbers>window[-1]`, and it is the same rule as every monotonic-structure problem.

Trap 2 — recording before the window is full

```
window.append(index)
answers.append(numbers>window[0])    # no `if index ≥ k - 1`
```

Returns `n` answers instead of `n - k + 1`. On `[1, 3, -1, -3, 5, 3, 6, 7]` with $k = 3$ you get `[1, 3, 3, 3, 5, 5, 6, 7]` — eight values where six were asked for, and the first two are maxima of partial windows nobody wanted.

Trap 3 — the expiry comparison off by one

```
if window and window[0] < index - k:    # `<` instead of `≤`
```

Off by one, so an index one step too old stays in the deque for one extra step. It is a nasty bug because it is invisible on most inputs: on `[1, 3, -1, -3, 5, 3, 6, 7]` with $k = 3$ it gives exactly the right answers. The smallest input that exposes it is `[8, 3, 2]` with $k = 2$, where the correct answer is `[8, 3]` and this version returns `[8, 8]` — reporting an 8 that left the window a step ago.

Derive it rather than remember it. The window ending at `index` starts at `index - k + 1`. An index `j` is out when `j < index - k + 1`, which is `j ≤ index - k`.

Trap 4 — the empty-deque guard

```
while numbers>window[-1] ≤ value:    # no `window and`
```

```
IndexError: deque index out of range
```

Fires on the very first element, since the deque starts empty. The same guard as every stack and queue problem in this phase, and it must be the first half of the `and`.

Trap 5 — using a plain list and `pop(0)`

```
window = []
...
window.pop(0)                                # instead of deque.popleft()
```

Correct answers, quietly quadratic in `k`, for exactly the reason in [day 073](#): removing from the front of a list shifts everything left. The deque exists precisely so this operation is $O(1)$. If you have written the deque solution and then used a list to hold it, you have thrown away the entire point.

Trap 6 — `while` versus `if` on the front, and why it does not matter

```
while window and window[0] ≤ index - k:      # `while` instead of `if`
    window.popleft()
```

This is **correct**. Only one index can ever expire per step, because the window moves by one, so the `while` runs at most once. Use whichever you find clearer, but know why the `if` is enough — being asked “are you sure one is enough?” and answering “yes, the window moves by exactly one position per step” is a small, cheap win.

What is wrong is doing the front expiry **after** appending and reading:

```
window.append(index)
answers.append(numbers>window[0]))
if window[0] ≤ index - k:                    # WRONG ORDER — read a stale front
    window.popleft()
```

Now the answer for this window can come from an element that has already left it. Expire first, then crush, then push, then read.

Trap 7 — `k` larger than the array, or `k = 0`

```
max_sliding_window([1, 2, 3], 5)
```

With no guard, `index ≥ k - 1` is never true and you get `[]`, which is arguably right. With `k = 0` you get `[]` as well, but `window[0] ≤ index - 0` expires the element you just pushed on some inputs, which is a confusing way to be right. Guard explicitly at the top and say what you decided — an empty list, or an error. Deciding out loud is worth more than the decision.

8 In the interview

How it gets asked

- The direct version: “Given an array and a window size k , return the maximum of every window.” LeetCode 239, Hard.
- The streaming version: “Temperature readings arrive one per second. At any moment, report the highest reading in the last sixty seconds.” Same structure, and $O(k)$ space is the selling point.
- The two-deque version: “Find the longest subarray where the difference between the maximum and the minimum is at most `limit`.” LeetCode 1438.
- The disguised version, in a dynamic-programming round: “You may jump at most k steps forward. What is the maximum score?” LeetCode 1696 — the transition is “the best of the last k cells”, which is this.

What to say out loud, in the first ninety seconds

1. **State the brute force with a number.** “Recomputing the max for each window is $O(n \cdot k)$. At two hundred thousand elements with a window of a thousand, that is two hundred million comparisons — about two seconds.”
2. **Say the observation, not the data structure.** “Here is the key: when a new element arrives, any earlier element smaller than it is worthless for ever. Every future window that contains the old one also contains the new one, because the new one is later, and the new one is bigger. So I can delete it, not just deprioritise it.”
3. **Say what survives.** “So what I keep is exactly the elements that are still candidates, and they are automatically in falling order — I never sort anything.”
4. **Say why it is a deque, in one sentence.** “I remove from the back when something bigger arrives, and from the front when something slides out of the window. Two ends, two different reasons. That is why it is a deque and not a stack or a queue.”
5. **Say indices, and why.** “I store indices, because the front check is ‘has this left the window’, and a value cannot tell me that.”
6. **Pre-empt the heap.** “A heap also works — push value and index, discard expired entries lazily — but that is $O(n \log n)$ and it keeps elements it will never use. The deque throws them away at write time, so it only ever holds live candidates.”
7. **Pre-empt the complexity question.** “One `while` inside the `for`, still $O(n)$: each index is appended once and removed at most once. Space is $O(k)$, not $O(n)$, which matters if this is a stream.”

The follow-ups

“**Why not a heap?**” “A heap gives me the maximum immediately, but it cannot remove the element that has just left the window, because that element is somewhere in the middle of the heap. So I would push `(-value, index)` and lazily discard expired entries from the top before reading. That is correct and it is $O(n \log n)$. I measured both once: the heap ran about 1.8

times the deque's time. The real difference is conceptual — the heap stores everything and filters at read time, the deque discards at write time and therefore only ever holds elements that still have a chance. That also makes the deque's space $O(k)$ rather than $O(n)$."

"There is a `while` inside the `for`. Is that not $O(n \cdot k)$?" "No. Count operations rather than nesting. Every index is appended exactly once by the outer loop, and removed at most once, because once it is out of the deque it never returns. So total deque operations are bounded by $2n$ across the whole run, however unevenly they land. One arrival can crush a thousand elements off the back, and that arrival has used up a thousand removals the rest of the run was going to do."

"Now give me the minimum instead." "Flip the back comparison from `$\leq$` to `$\geq$` . The deque becomes rising instead of falling, and the front is the minimum. Nothing else changes. And if I need both at once — for 'longest subarray where max minus min is at most limit' — I run two deques side by side and compare their fronts."

"The data is a stream and never ends. Does anything change?" "Almost nothing, and that is the strength of this solution. Space is $O(k)$, independent of how much data has gone past, so a window of sixty over a stream of a billion readings holds at most sixty indices. The only change is that indices grow without bound, so in a long-running system I would store timestamps, or wrap the index into a ring buffer of size k , rather than let an integer counter run for months."

"What if the elements are equal?" "It does not matter for correctness, because equal elements are interchangeable as maxima. I use `$\leq$` on the back removal so equals are discarded, which keeps the deque smaller. Using `$<$` keeps them, which is also correct and slightly more memory. What I would not do is use `$<$` and then assume the deque is strictly decreasing, because it is not."

"Can you do it in $O(1)$ space?" "Not for the general problem, no. There is a neat $O(n)$ block-partition trick — split the array into blocks of k , precompute prefix maxima and suffix maxima within each block, and answer any window with one of each — but that uses $O(n)$ extra space too, and it needs the whole array up front, so it cannot stream. The deque's $O(k)$ is the better bound in practice."

A model answer

Asked: *find the maximum in every window of size k .*

“The obvious version calls `max` on each window, which is $O(nk)$ — about two hundred million comparisons at two hundred thousand elements with a window of a thousand, and I measured that at roughly two seconds. So I want to reuse work between windows.

Here is the observation that lets me. When a new element arrives, every earlier element smaller than it becomes worthless for ever. Not unlikely to be needed — impossible to need. Any future window that still contains the old element must also contain the new one, because the new one is later, and the new one is bigger. So the old one can never be a maximum again, and I can delete it rather than store it.

So the structure I keep is: the elements that could still be the maximum of some window. And they are automatically in falling order, because anything that broke the order was deleted the moment something bigger arrived. I never sort.

Now, why a deque. There are two reasons to remove something, and they happen at opposite ends. From the back, I remove elements smaller than the arriving one — that is the rule I just described. From the front, I remove the element that has slid out of the window, because being big does not save you from being old. Two ends, two reasons, so a stack is not enough and a queue is not enough.

I store indices, not values, because the front check is ‘is this still inside the window’, and only a position can answer that. The element at index j is out when j is at most the current index minus k — I would derive that rather than memorise it: the window ending at i starts at i minus k plus one.

The loop is four steps per element. Expire the front if it has aged out — one `if` is enough, because the window moves by exactly one position at a time. Crush the back while its values are at most the new value. Append the new index. And then, only once the current index has reached k minus one, record the front as the answer for this window. That last guard is what stops me returning n answers instead of n minus k plus one.

On complexity: there is a `while` inside a `for` and it is still $O(n)$. Every index is appended exactly once and removed at most once, so there are at most $2n$ deque operations across the whole run, however unevenly they fall. Space is $O(k)$ — at most k indices, in the worst case of a strictly decreasing array; on an increasing array the deque holds exactly one. $O(k)$ rather than $O(n)$ matters if this is a stream.

I should mention the alternative. A max-heap of value and index, discarding expired entries lazily from the top, is correct at $O(n \log n)$, and I measured it at about 1.8 times the deque’s runtime. The reason I prefer the deque is that the heap keeps everything and filters when reading, whereas the deque throws things away when writing, so it only ever holds elements that still have a chance.

One thing I would check before writing: are equal values fine to discard? They are, since equal elements are interchangeable as maxima, so I use less-than-or-equal on the back removal to keep the deque smaller.”

9 Recall card

- **The whole problem is one sentence: when a bigger element arrives, every earlier smaller element is worthless for ever** — every future window containing the old one also contains the new one, and the new one is bigger. So **delete, do not deprioritise**. What survives is exactly the live candidates, automatically **falling front to back**.
- **Two removals at two ends, for two different reasons** — that is why it is a **deque**: `popleft` when the front index has **slid out** (`window[0] ≤ index - k`), `pop` while the back is **too small** (`numbers>window[-1]] ≤ value`). **Store indices**, because a value cannot tell you whether it is still in the window.
- **Four steps per element, in this order: expire front · crush back · append · record**. One `if` suffices for the expiry because the window moves by exactly one. **Record only when** `index ≥ k - 1`, or you return `n` answers instead of `n - k + 1`.
- **$O(n)$ time** — n appends, each index removed at most once, $\leq 2n$ operations, **bounded in total not per iteration**. **$O(k)$ space**, not $O(n)$: worst case a strictly decreasing array, best case one index on a rising one. Measured at $n = 200,000$, $k = 1,000$: brute **1.98 s**, heap **0.092 s**, deque **0.050 s** — **$40\times$ the brute force, $1.8\times$ the heap**.
- **Name the heap and reject it properly**: push `(-value, index)` and discard expired entries lazily — correct, $O(n \log n)$, $O(n)$ space. *The heap keeps everything and filters at read time; the deque discards at write time*. The family: sliding-window **minimum** (flip to `≥`) · **two deques** for $\max - \min \leq \text{limit}$ · DP transitions that need “the best of the last k ”.

1 What this is, and why they ask it

Command is the pattern where you stop *calling* an operation and start *making* one. Instead of `editor.insert("hello", at=42)`, you build an object — `InsertText("hello", 42)` — that knows how to do the thing and how to undo it. Once the action is an object, you can put it in a list, send it over a network, write it to disk, run it later, run it a thousand times, or run it backwards.

Chain of responsibility is the pattern where a request is passed along a line of handlers, each of which either deals with it or passes it on. Nobody knows the whole line. Each handler knows one thing: what it can handle, and who is next.

They go together because both start from the same move — **make the request a thing** — and because every web framework you will ever use is both at once. A request object travelling through a stack of middleware is chain of responsibility. A job on a Celery or Sidekiq queue is a command. Interviewers ask Command as “design undo and redo”, which is the cleanest question in the whole low-level design set: it has an obvious naive answer, that answer runs out of memory, and the fix is the pattern.

2 The story

Faiz runs a place with eleven tables near the market, and until two years ago the way an order reached the kitchen was that a waiter stood at the hatch and shouted it.

It mostly worked. It failed in four specific ways, and Faiz can list them because each one cost him money.

If the cook was elbow-deep in something when the shout came, the order was simply gone. Nobody could prove it had ever been made. The waiter said he shouted it; the cook said he never heard.

If a customer changed their mind — and they do, constantly, ninety seconds after ordering — the waiter had to go back and shout a correction, and hope the correction arrived after the order and not before.

At eleven at night, when Faiz sat down to work out what the day had been, there was nothing to work out from. The food had been cooked and eaten and nobody had a record of what.

And when two waiters shouted at the same time, which happened every single lunch, one of the two orders came out wrong.

Now there is a screen on the kitchen wall, and the waiter presses buttons on a small handset at the table. The order becomes a line on that screen with a number against it — 74, two cha-pati, one dal fry, no chilli.

Everything Faiz was losing came back at once, and none of it because the cooking changed.

The order waits its turn on the screen instead of depending on whether anyone was listening. If the customer changes their mind, the waiter touches the line and it goes, as long as the cook has not started it. At eleven at night the whole day is still on the screen, in order, and Faiz scrolls it while he counts the cash. When a table says “the same again”, the waiter presses one button and the same order appears at the bottom with a new number.

There is one more thing they added last year. Special requests used to go to whoever was nearest. Now they go down the line. The cook at the tawa handles it if it is his — less oil, no onion. If it is not his, he passes it to the head cook, who handles the ones about ingredients. If it is not his either, it goes to Faiz, who is the only one allowed to say yes to a discount. Nobody has to know the whole set of rules. Each of them knows two things: what they are allowed to decide, and who to hand it to when they are not.

3 The idea in plain English

The shout is a **method call**. It happens and it is gone. The line on the screen is a **command object**: the same instruction, but now it is a thing that exists, and everything Faiz got back follows from that one change.

What making it a thing buys you

FAIZ GOT BACK	BECAUSE A COMMAND IS AN OBJECT
Orders stopped being lost	You can put an object in a queue
Corrections work	You can cancel an object, which is undo
The day is on the screen at eleven	You can keep a log of objects
“The same again” is one button	You can replay an object
Two waiters no longer collide	Objects go in a list; shouts collide in the air

That table is the argument for the pattern. Every row is impossible with a plain method call, and every row is free once the call becomes an object.

The Command interface

Two methods. That is the whole pattern.

```
class Command(Protocol):
    def execute(self) → None: ...
    def undo(self) → None: ...
```

A command holds everything it needs: the target it acts on, the arguments, and — crucially — whatever it must remember in order to reverse itself.

```
@dataclass
class InsertText:
    document: "Document"
    text: str
    position: int

    def execute(self) → None:
        self.document.insert(self.position, self.text)

    def undo(self) → None:
        self.document.delete(self.position, len(self.text))
```

Reversing an insert needs only the position and the length, both of which the command already has. Reversing a *delete* is different, and it is the interesting case:

```
@dataclass
class DeleteText:
    document: "Document"
    position: int
    length: int
    removed: str = ""          # filled in by execute, needed by undo

    def execute(self) → None:
        self.removed = self.document.read(self.position, self.length) # remember it
        self.document.delete(self.position, self.length)

    def undo(self) → None:
        self.document.insert(self.position, self.removed)
```

A command must capture whatever undo will need, at the moment it executes. Deleting throws information away, so the command has to catch it on the way past. That single line, `self.removed = ...`, is the difference between an undo that works and one that quietly loses the user's paragraph.

Undo and redo are two stacks

undo stack (things done)	redo stack (things undone)
do X	→ push X onto undo, CLEAR redo
undo	→ pop from undo, call <code>.undo()</code> , push onto redo
redo	→ pop from redo, call <code>.execute()</code> , push onto undo

The stack from [day 068](#), used exactly as it was defined: the most recent thing not yet resolved.

The line that people forget is **clear the redo stack when a new command is executed**. If the user undoes three edits and then types something new, the three undone edits are no longer reachable — history has forked, and you keep the branch the user is on. Leave the redo stack alone and `Ctrl+Y` will apply an edit to a document that has changed underneath it.

Chain of responsibility, in the same breath

The special-request line at Faiz's place is a different pattern with the same starting move. The request is an object; each handler holds a reference to the next one.

```
class Handler:
    def __init__(self, next_handler: "Handler | None" = None) → None:
        self._next = next_handler

    def handle(self, request: "Request") → "Response | None":
        if self.can_handle(request):
            return self.respond(request)
        if self._next is not None:
            return self._next.handle(request)      # pass it on
        return None                               # nobody could
```

Three cases in seven lines: I handle it, I pass it on, or it falls off the end. The last case is the one that gets designed badly — see §7.

The classic example is an approval limit, and it is worth having ready because interviewers use it:

```
expense of ₹800      → team lead approves      (limit ₹1,000)
expense of ₹9,000    → lead passes to manager  (limit ₹10,000)
expense of ₹80,000   → lead → manager → director (limit ₹100,000)
expense of ₹9,00,000 → falls off the end      → must be handled explicitly
```

Each handler knows its own limit and its successor. Nobody knows the whole ladder. Adding a vice-president between manager and director is one insertion and zero edits to existing handlers.

The two patterns are not the same thing

Say this if asked, because they are taught together and confused constantly:

Command is about *what* to do — one object, and it will definitely be done. Chain of responsibility is about *who* does it — many candidates, and possibly nobody.

A command has one receiver, chosen when the command is built. A chain has many possible handlers and the choice happens at run time, by walking the line.

4 The picture

Command, with the undo and redo stacks:

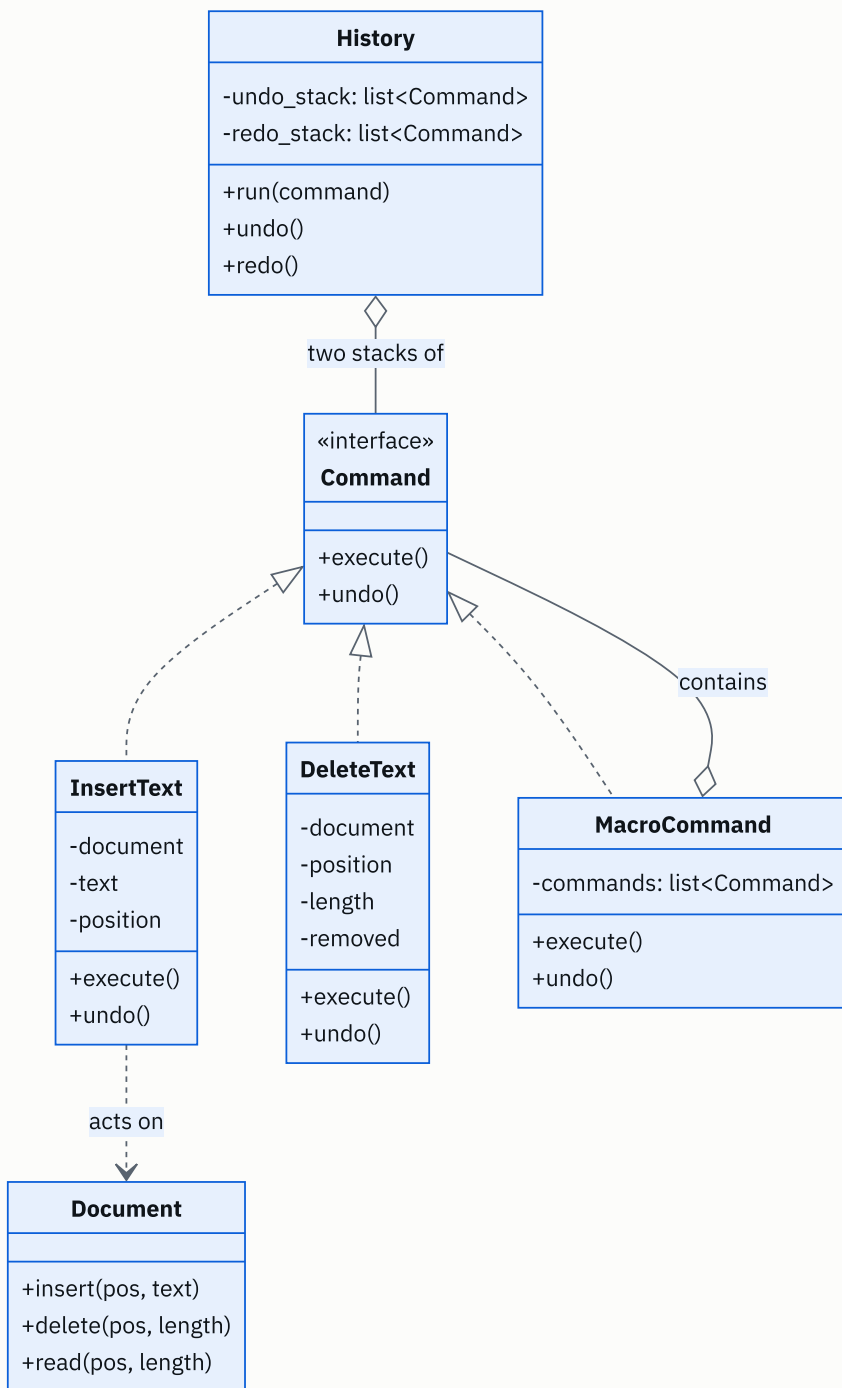


FIGURE 74.1

What to notice: `MacroCommand` both **is** a command and **contains** commands. That is what makes “select all, replace, and reformat” one undoable step — it is a list of commands wearing the same interface, and its `undo` runs the children backwards. A pattern that composes with itself like this is worth pointing at in an interview.

Chain of responsibility, as a request flowing:

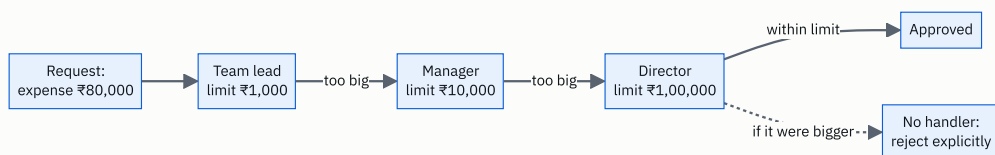


FIGURE 74.2

What to notice: the dotted arrow. **Every chain needs a defined end.** Falling off the end silently returning `None` is how a request gets approved by nobody and nobody notices for a month.

And the shape every web framework has, which is the same picture turned sideways:

```
request → logging → auth → rate limit → your handler
                                     |
response ← logging ← auth ← rate limit <-----+
```

Middleware is a chain of responsibility where each handler gets to act on the way in *and* on the way back out. That double pass is the one addition real frameworks make to the textbook pattern.

5 How it actually works

The history object

```
class History:
    def __init__(self, limit: int = 500) → None:
        self._undo: list[Command] = []
        self._redo: list[Command] = []
        self._limit = limit

    def run(self, command: Command) → None:
        command.execute()
        self._undo.append(command)
        self._redo.clear()                    # history has forked
        if len(self._undo) > self._limit:
            self._undo.pop(0)                 # forget the oldest
```

Four lines of policy, and each one is a decision an interviewer will poke at. Executing before pushing means a command that throws is never recorded. Clearing redo is the forking rule. The limit is what stops an eight-hour editing session eating memory, and dropping the *oldest* is

right because nobody undoes four hundred steps.

```
def undo(self) → None:
    if not self._undo:
        return # nothing to undo; not an error
    command = self._undo.pop()
    command.undo()
    self._redo.append(command)

def redo(self) → None:
    if not self._redo:
        return
    command = self._redo.pop()
    command.execute()
    self._undo.append(command)
```

Perfect mirrors. Notice that redo calls `execute` again, not a third method — which is only safe if `execute` is repeatable, and that is a real constraint worth naming (§7).

Macros compose for free

```
@dataclass
class MacroCommand:
    commands: list[Command]

    def execute(self) → None:
        for command in self.commands:
            command.execute()

    def undo(self) → None:
        for command in reversed(self.commands): # backwards, always
            command.undo()
```

`reversed` is not decoration. Undoing “delete the word, then insert the replacement” in the original order would insert into a document that still has the word in it. **Undo runs a macro backwards**, and saying that unprompted is a small, cheap signal.

Commands you can put on a wire

Once a command is data, it can leave the machine:

```
def to_json(self) → dict:
    return {"type": "insert", "position": self.position, "text": self.text}
```

That is the whole idea behind a task queue. A **Celery** task is a serialised command — function name plus arguments — sitting in Redis or RabbitMQ until a worker picks it up. **Sidekiq** in Ruby and **BullMQ** in Node are the same thing. The queue does not know what the job does; it knows how to store and deliver an object.

Real systems built on Command

- **Every editor’s Ctrl+Z.** VS Code, Word, Photoshop. Photoshop’s History panel is the undo stack, rendered.

- **Database write-ahead logging.** Postgres writes what it is *about* to do to the WAL before doing it. Recovery after a crash replays that log. Redo is literally the name of the operation, and undo logs are how a transaction rolls back. This is Command at the storage layer, and it is why [day 033](#)'s transactions can be rolled back at all.
- **Redis AOF** (append-only file) records every write command received; restarting replays them.
- `git revert` creates a new commit that is the inverse of an old one — a command that knows its own undo.
- **Event sourcing and CQRS**, where the database stores the sequence of commands rather than the current state, and the current state is what you get by replaying them.
- **Java's** `Runnable` and any `ExecutorService.submit(task)` — an action, made into an object, so it can be scheduled.

Real systems built on chain of responsibility

- **Django middleware**, Express `app.use`, **ASP.NET Core's request pipeline**, **Java servlet filters**. Each one wraps the next, sees the request going in and the response coming out.
- **Spring Security's filter chain** — authentication, then authorisation, then CSRF, in a fixed order, each free to reject and stop the chain.
- **Python's logging module**. A log record goes to the logger's handlers, then propagates to the parent logger, up to the root — and `logger.propagate = False` is how you stop the chain.
- **Netty's ChannelPipeline**, where decoders and handlers are an explicit ordered list.
- **Firewall rules** — `iptables` and AWS WAF walk rules in order until one matches, with a default policy at the end. The default policy is the “nobody handled it” case, made explicit.
- **DOM event bubbling**. A click on a button fires on the button, then the div, then the body, until something calls `stopPropagation()`.

What happens on failure

For Command: if `execute` throws halfway through a macro, you are in a half-applied state. The honest options are to undo the children that already succeeded (compensation) or to require that each child is independently safe. Say which you chose.

For a chain: if a handler throws, the request stops there, and whether the remaining handlers should still run on the way out is a real decision — in middleware it usually matters a great deal, because the logging middleware is the one that would have recorded the failure.

6 The numbers

Why the naive undo runs out of memory

The obvious implementation of undo is “keep a copy of the whole document before each change”.

```
document:                1 MB (a 200-page manuscript)
undo depth:              500 edits
snapshot approach: 500 × 1 MB = 500 MB
```

Half a gigabyte to be able to undo five hundred keystrokes. Now the command version:

```
InsertText: text ~20 B + position 8 B + object overhead ~120 B = 150 B
DeleteText: same plus the removed text, say up to 1 KB         = 1 KB worst case
500 commands × 1 KB worst case                                = 500 KB
```

```
snapshots: 500 MB
commands:  0.5 MB
ratio:     1,000x
```

A thousand times less memory, because a command stores the *difference* and a snapshot stores the *state*. That single comparison is the reason the pattern exists here, and it is the number to lead with when asked to design undo.

The mixed strategy real editors use: a snapshot every 200 commands plus the commands since, so that undoing 500 steps is 2 snapshot restores plus at most 200 replays instead of 500 sequential undos.

The cost of a middleware chain

Eight middlewares on a web request, each doing a small amount of work:

```
request id    0.05 ms
logging       0.20 ms
CORS          0.02 ms
session load  1.50 ms (a Redis round trip)
authentication 0.80 ms (JWT verify)
rate limit    0.60 ms (a Redis INCR)
CSRF          0.05 ms
compression   0.90 ms
-----
total overhead 4.12 ms before your handler runs at all
```

At 5,000 requests per second that is **20.6 seconds of CPU per second** across the fleet — about 21 cores doing nothing but pipeline. Worth knowing, because “why is our p50 4 ms even on an endpoint that returns a constant?” is a real question with this answer, and because it tells you which order to put the handlers in: **the cheapest rejection first**. Rate limiting before session loading saves the Redis round trip on exactly the requests you were going to throw away.

Order matters, in money

```
auth before rate limit: every abusive request costs a JWT verify (0.8 ms)
rate limit before auth: a blocked request costs one Redis INCR (0.6 ms) and stops

at a 50,000 req/s attack:
wrong order: 50,000 × 0.8 ms = 40 core-seconds per second
right order: rejected at the edge
```

The undo-history table, if it is persisted

Collaborative editors persist operations, not documents:

```
100,000 documents × 2,000 operations each × 200 B = 40 GB
compaction: snapshot each document nightly and keep 24 h of operations
100,000 × 100 ops × 200 B = 2 GB of operations + the snapshots
```

Which is the storage-layer version of the same $1,000 \times$ argument.

7 The trade-offs

What Command costs you

A class per action. Twenty operations means twenty classes, and most of them are four lines. In Python a closure or a `functools.partial` does the `execute` half in one line, and the moment you need `undo` as well, the class earns itself back. Be honest that the ceremony is real and that the second method is what justifies it.

Not every operation has an inverse. “Send the email” cannot be undone. “Charge the card” can only be *compensated* by a refund, which is a different transaction with its own failure modes, not a reversal. Undo is easy inside a document and hard the moment the action leaves the building. Say this out loud — it is the difference between designing an editor and designing a system.

redo calling execute again requires idempotent construction. If `execute` generates a fresh id or reads the clock, redoing produces a *different* command than the one you undid. Capture anything non-deterministic in the constructor, or on the first `execute`, never on each one.

Undo depth is unbounded unless you bound it. Five hundred commands is a policy. Without it, an editor left open all day accumulates for ever, and a `DeleteText` holding a megabyte of removed text makes that expensive fast.

Commands can go stale. Undoing an edit at position 400 is nonsense if the document has since been edited by someone else at position 100. Single-user editors get away with it. Collaborative ones need **operational transformation** or **CRDTs**, which exist precisely because “just keep an undo stack” does not survive two users. Naming that boundary is worth more than pretending it does not exist.

Memento is the alternative and sometimes the right one. Store a snapshot of state instead of the inverse operation. It is much simpler and it is correct when the state is small — a drawing app’s selection, a form’s contents. Use Command when state is big and changes are small; use Memento when state is small or the inverse is hard to write.

What chain of responsibility costs you

The request can fall off the end. The default implementation returns `None` and nobody notices. Always terminate the chain deliberately: a final handler that rejects loudly, or a raised exception. `iptables` gets this right with a default policy; most hand-rolled chains do not.

Nobody can see the whole flow. “Why was this request rejected?” means walking eight handlers. Mitigation: keep the chain construction in one place, in order, and log which handler terminated the request.

Order is a hidden contract. Authentication must run before authorisation. Rate limiting should run before anything expensive. None of that is expressible in the pattern; it lives in the order of a list, and a reordering merge is a security incident waiting to happen. Write the required order down next to the list.

Long chains cost latency on every request, including the ones every handler passes through untouched — the 4.12 ms above.

Debugging a deep chain is a stack trace forty frames tall, because each handler calls the next.

“I would not use this if...”

- **...only one thing can handle the request.** Then it is a call, not a chain. A chain with one handler is a function with extra steps.
- **...every handler must run.** That is a pipeline or an observer broadcast, not a chain — the defining property of a chain is that handling *stops* the walk.
- **...the action has no inverse and never needs queuing, logging or replay.** Then Command is ceremony. Call the method.
- **...the state is small enough to snapshot.** Memento is fewer moving parts and cannot get the inverse wrong.
- **...two users edit the same document.** Neither pattern is sufficient; you need OT or CRDTs, and saying so is better than bolting a mutex onto an undo stack.

The honest concession

Command does not make anything simpler on the day you write it. It makes the *third* requirement cheap — undo, then queuing, then an audit log, then replay — and every one of those is impossible to retrofit onto direct method calls without rewriting the call sites. That is the actual argument: **you are paying up front for the three features you know are coming.**

8 In the interview

How it gets asked

- The direct version: “Design undo and redo for a text editor. How much memory does it use?”

- The escalation, always: *“Now support multi-step undo. Now support undoing a find-and-replace that changed forty places, as one step.”*
- The queue version: *“Users click a button that starts a slow job. Design that.”* — the answer is a command on a queue.
- The chain version: *“Expenses need approval, with different limits at different levels. Design it.”*
Or: *“Design the request pipeline for a web framework.”*
- The distinguishing question: *“When would you use a chain of responsibility rather than a list of observers?”*

What to say out loud, in the first ninety seconds

1. **Name the move.** “The core idea is to stop calling operations and start creating them. Every edit becomes an object with `execute` and `undo`.”
2. **Say what that buys, as a list.** “Once an action is an object I can stack it for undo, queue it, log it, replay it, and group several into one. None of those are possible with a plain method call.”
3. **Kill the naive answer with the number.** “The obvious approach is snapshotting the document before each edit. At a one-megabyte document and five hundred levels of undo that is five hundred megabytes. Commands store the difference, not the state — about half a megabyte for the same history. A thousand times less.”
4. **Say the two stacks and the forking rule.** “Undo stack and redo stack. Do something: push to undo and *clear* redo, because history has forked. Undo: pop, reverse, push to redo.”
5. **Name the hard part before they do.** “The interesting command is delete, because deleting throws information away. The command has to capture the removed text during `execute` so `undo` can put it back.”
6. **Show that it composes.** “Find-and-replace across forty places is a macro command — a command containing commands — and its `undo` runs the children in reverse order.”
7. **State the boundary.** “This works for a single user. Two users editing the same document breaks the position arithmetic, and that needs operational transformation or CRDTs, which is a different design.”

The follow-ups

“How much memory does the undo history take?” “It depends entirely on whether you store state or differences. Snapshots of a one-megabyte document at 500 levels is 500 MB. Commands are the delta: an insert is the text plus a position, about 150 bytes; a delete is worse because it has to hold what it removed, but still kilobytes not megabytes. Call it half a megabyte for the same history — about a thousandfold difference. I would also cap the depth at a few hundred and drop the oldest, and for very long sessions take a periodic snapshot so undoing five hundred steps is a restore plus a short replay rather than five hundred sequential undos.”

“What about actions that cannot be undone?” “Then undo is the wrong word and I would say so to the product owner. Sending an email or charging a card cannot be reversed; they can only be *compensated* — a retraction, a refund — which is a new action with its own failure modes. In an editor I would keep such actions out of the undo stack and grey out the menu item, rather than pretend. In a distributed system this is exactly the saga pattern: every step has a compensating step, and the compensation is not a rollback.”

“How do you make find-and-replace one undo step?” “A macro command: a command that holds a list of commands and implements the same interface. Its `execute` runs them in order, its `undo` runs them in *reverse* order, which matters — undoing a delete-then-insert pair in the original order would insert into a document that still has the old text. Because it implements the same interface, the history object does not need to know it is special.”

“When is chain of responsibility better than just calling the right handler?” “When the choice of handler depends on the request in a way the caller should not know about, and when the set of handlers changes. Expense approval is the clean example: the caller submits an expense and does not know or care what the limits are. If the caller *can* pick the handler, a dictionary lookup is simpler and faster than walking a chain. And if *every* handler should run, it is not a chain — a chain’s defining property is that handling stops the walk.”

“What happens if nobody in the chain handles it?” “That has to be a deliberate decision, and it is the part people get wrong. The default implementation returns `None` and the request is silently approved by nobody. I would terminate the chain with an explicit final handler that rejects and logs, the way a firewall has a default policy. And I would log which handler terminated any request, because otherwise ‘why was this rejected’ means reading eight files.”

“How would you order the middleware?” “Cheapest rejection first. A request that is going to be rate-limited should be rejected before I spend a Redis round trip loading its session and 0.8 milliseconds verifying its token. In our numbers, an eight-stage pipeline costs about 4 milliseconds per request before the handler runs at all, and under attack the ordering is the difference between rejecting at the edge and burning forty core-seconds a second on JWT verification. Authentication before authorisation, always, and I would write the required order in a comment next to the list because the pattern cannot express it.”

A model answer

Asked: *design undo and redo for a text editor.*

“The move that makes this possible is to stop calling operations and start creating them. Today the editor probably calls `document.insert(position, text)` directly, and once that call has returned there is nothing left to reverse. So instead every edit becomes an object with two methods: `execute` and `undo` .

Before I go further, let me kill the obvious alternative with a number, because it is what most people reach for. You could snapshot the whole document before each change. For a one-megabyte document with five hundred levels of undo that is five hundred megabytes of memory. Commands store the *difference* rather than the *state*: an insert command is the text and a position, roughly a hundred and fifty bytes. The same five hundred levels come to well under a megabyte. That is about a thousandfold difference and it is the reason to do it this way.

The history is two stacks. Running a command executes it, pushes it on the undo stack, and clears the redo stack — that last part matters, because if the user undoes three edits and then types something new, history has forked and the undone edits are unreachable. Undo pops the undo stack, calls `undo` , and pushes onto redo. Redo does the mirror.

The interesting command is delete, and I would write that one first in an interview because it is where the design is. Insert is easy to reverse — you know the position and the length. Delete throws information away, so the command has to capture the removed text during `execute` and hold it, and that captured text is what `undo` puts back. Getting that wrong gives you an undo that silently loses a paragraph.

Find-and-replace across forty places should be a single undo step, and the pattern gives that for free: a macro command that holds a list of commands and implements the same interface. Its `execute` runs them forwards and its `undo` runs them backwards — backwards matters, or you insert into a document that still contains the old text. The history object never learns that macros exist.

Two policies I would set explicitly. Cap the depth, say five hundred, dropping the oldest, because an editor left open all day otherwise grows without bound and a delete command can be holding a megabyte. And for very deep histories, take a snapshot every couple of hundred commands so undoing a long way is a restore plus a short replay rather than hundreds of sequential undos.

Two boundaries I would name rather than paper over. First, redo re-executes the command, so `execute` must be repeatable — anything non-deterministic like a generated id or a timestamp has to be captured once, not regenerated. Second, all of this assumes one user. The moment two people edit the same document, position 400 in my command may not mean position 400 any more, and that needs operational transformation or CRDTs — a genuinely different design, not an extension of this one.

The wider point is that making the action an object gives me four things at once and I only asked for one: undo, but also queuing, an audit log of everything that was done, and replay. That is why databases do exactly this — Postgres writes what it is about to do to

the write-ahead log before doing it, and crash recovery is replaying that log.”

9 Recall card

- **Command = stop *calling* the operation, start *creating* it.** Two methods, `execute` and `undo`. Once the action is an object you get four things you did not have: **queue it** · **undo it** · **log it** · **replay it**. Celery/Sidekiq jobs, Postgres WAL, Redis AOF, `git revert`, event sourcing are all this.
- **The number that wins the undo question: snapshots store *state*, commands store the *difference*.** 1 MB document × 500 levels = **500 MB**; the same history as commands ≈ **0.5 MB** — about **1,000×**. Cap the depth and drop the oldest; snapshot every ~200 commands so a deep undo is a restore plus a short replay.
- **Two stacks, and the rule people forget: executing a new command *clears the redo stack*** — history has forked. **Delete is the interesting command**, because it must capture the removed text during `execute` or undo loses it. A **macro command** is a command holding commands, and its `undo` runs the children **in reverse**.
- **Chain of responsibility = each handler knows two things: what it can handle, and who is next.** Handling **stops** the walk — if every handler must run it is not a chain. **Always terminate the chain deliberately**, or a request is approved by nobody. Real chains: Django/Express middleware, Spring Security filters, Python `logging` propagation, `iptables` with a default policy, DOM event bubbling.
- **Order is a hidden contract and it costs money: cheapest rejection first.** An 8-stage pipeline is ~**4.1 ms** before your handler runs; rate-limit before auth or a 50,000 req/s attack burns 40 core-seconds a second on JWT verification. **Boundaries to name:** actions with no inverse (compensate, do not undo) · `execute` must be repeatable for redo · two users need **OT or CRDTs**, not a bigger undo stack · **Memento** beats Command when state is small.

PRACTICE

Practice — Deques and the sliding-window maximum

DSA topic: Deques and the sliding-window maximum **System design topic:** Command and chain of responsibility

Code these, in this order

One rule for the whole set: **before writing the loop, say what the deque holds and in what order.** “Indices of elements that could still be the maximum of some future window, values falling from front to back.” If you cannot say that sentence, you are using the deque as scratch space.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Moving Average from Data Stream	LeetCode 346 (Easy)	The plain deque: add at the back, expire at the front, keep a running total. No monotonicity yet.
2	Sliding Window Maximum	LeetCode 239 (Hard)	Today’s problem. The delete-for-ever observation, both removals, and the $O(n)$ defence.
3	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	LeetCode 1438 (Medium)	Two deques at once, and a window that shrinks rather than slides.
4	Shortest Subarray with Sum at Least K	LeetCode 862 (Hard)	A monotonic deque over prefix sums, with negatives — the case where a plain sliding window is simply wrong.

On problem 1, notice what is missing

There is no monotonic anything here. It is a queue with a running sum. Write it, then say in one sentence what problem 2 needs that this one does not — and why that need is what forces both ends.

On problem 2, write the brute force first and time it

Write the one-line `max(numbers[i:i+k])` version. Run it on 200,000 elements with $k = 1,000$ and time it. Then time the deque version. Write both numbers down. You will quote them for years.

On problem 3, run two deques side by side

1. Write it with only the max deque and see which cases fail.
2. Add the min deque.
3. Print both deques at every step for `[8, 2, 4, 7]`, `limit = 4`, and watch the left edge move.
4. Say why the window here *shrinks* rather than slides by exactly one, and what that changes about the expiry check.

On problem 4, find out why the easy approach fails

1. Write the ordinary two-pointer sliding window for “shortest subarray with sum at least K ” and run it on `[2, -1, 2]` with `K = 3`.
2. Say exactly which assumption of the sliding window the negative number broke.
3. Then build prefix sums and run a monotonic deque over them, and say what the deque now holds.

The observation drill

Say it out loud, five times, until it is automatic:

1. What happens to every earlier smaller element when a bigger one arrives, and why “for ever”?
2. Why does that argument need the words “because the new one is later”?
3. What exactly is left in the deque, described as a property rather than as code?
4. Why is it in falling order without anything sorting it?
5. Name the two removals and the different reason for each.

The trace drill

Trace `[1, 3, -1, -3, 5, 3, 6, 7]` with `k = 3`, writing the deque contents after every step. Then answer without looking:

1. At which index do both removals happen in the same step?
2. What is the largest the deque ever gets, and why is that smaller than `k` here?
3. Which input of length 8 would make the deque hold all 3 allowed indices at every step?
4. Which input would make it hold exactly one at every step?

The break-it drill

Trigger each and record the actual output or error text:

1. Store values instead of indices. Get as far as the front expiry and say what you cannot write.
2. Remove `window and` from the `while` condition. Quote the exact error.
3. Drop the `if index ≥ k - 1` guard. Run the standard input and count the answers.
4. Change the expiry to `window[0] < index - k`. Run `[1, 3, -1, -3, 5, 3, 6, 7]` with `k = 3` — it is *right*. Then run `[8, 3, 2]` with `k = 2` and say what it returns and why.
5. Do the front expiry *after* appending and recording. Say which answer becomes stale.
6. Use a plain list with `pop(0)` instead of a deque. Time it at `n = 200,000`, `k = 1,000`.
7. Change `≤` to `<` on the back removal. Confirm it is still correct, then say what changed about the deque’s size on `[5, 5, 5, 5, 5]`.

The heap-comparison drill

1. Write the lazy-deletion heap version.

2. Time both at $n = 200,000$, $k = 1,000$. Write down the ratio.
3. Say the one-sentence conceptual difference between them, the one with “write time” and “read time” in it.
4. State both space complexities and say why one is $O(k)$ and the other $O(n)$.
5. Say what you would do if you blanked on the deque version under pressure.

The recognition drill

For each, say whether a monotonic deque is the tool, and if so what it holds:

1. The maximum of every window of size k .
2. The average of the last k readings.
3. The longest window where $\max - \min \leq \text{limit}$.
4. The next greater element for every item.
5. The maximum score when you may jump at most k steps forward.
6. The shortest subarray with sum at least K , with negatives allowed.
7. The number of requests in the last second, for a rate limiter.
8. The median of every window of size k .

One of the eight needs a completely different structure. Name it and say what.

The undo drill

Build it and then attack it:

1. Write `InsertText` with `execute` and `undo`.
2. Write `DeleteText`. Say which line makes undo possible and what happens without it.
3. Write `History` with both stacks.
4. Do three edits, undo twice, then do a new edit. Print the redo stack. Explain the result.
5. Remove the `self._redo.clear()` line and repeat step 4. Say exactly what redo now does to the document.
6. Write `MacroCommand`. Undo it with the children in the original order and describe the damage.
7. Add a depth cap. Say why you drop the oldest and not the newest.

The memory drill

1. Compute the snapshot cost for a 1 MB document at 500 levels of undo.
2. Compute the command cost for the same history.
3. State the ratio.
4. Describe the mixed strategy, and say how many operations a 500-step undo costs under it.
5. Say which of the two you would pick for a drawing app whose entire state is 4 KB, and why.

The cannot-undo drill

For each, say whether it belongs on the undo stack, and what you would do instead if not:

1. Typing a character.
2. Deleting a paragraph.
3. Sending an email.
4. Charging a card.
5. Renaming a file on disk.
6. Posting a comment that others may have already read.
7. Generating a new document id.

Two of the seven can only be *compensated*, not undone. Name the difference in one sentence.

The chain drill

Build the expense-approval chain with limits of ₹1,000, ₹10,000 and ₹1,00,000.

1. Write the base handler with the three cases in it.
2. Run ₹800, ₹9,000, ₹80,000 and ₹9,00,000. Say what happens to the last one by default.
3. Fix the last case deliberately. Say what you chose and why.
4. Insert a vice-president between manager and director. Count the files edited.
5. Log which handler terminated each request, and say why that log exists.

The middleware drill

Given this pipeline: request id, logging, CORS, session load, authentication, rate limit, CSRF, compression.

1. Add up the per-request overhead from the lesson's numbers.
2. Compute the CPU cost per second at 5,000 requests per second.
3. Reorder it for cheapest-rejection-first and say what moved and why.
4. Say what breaks if authorisation runs before authentication.
5. Say which handler must be outermost so that failures anywhere are still recorded.

The which-pattern drill

For each, say Command, chain of responsibility, Observer, or none:

1. Ctrl + Z in an editor.
2. Five things must happen when an order is placed.
3. A request passes through auth, rate limiting and logging.
4. A slow job runs in the background when a button is clicked.
5. An expense needs approval at the right level.
6. The write-ahead log in a database.
7. A click event travelling up from a button to the body.

8. A new pricing rule per country.

One of the eight is Strategy from [day 071](#). Name it. Two are Command for the same reason — say the reason.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the maximum in every window of size k .* The brute force with a number, the delete-for-ever observation stated before any structure, what survives and why it is ordered without sorting, the two removals at two ends with their two different reasons, indices-not-values, the record guard, the $O(n)$ counting argument, and $O(k)$ space with the heap named and rejected.
2. *Design undo and redo for a text editor.* The move from calling to creating, the four things an object buys, the snapshot number killed with the command number, the two stacks with the redo-clearing rule, delete as the interesting command, macros with reversed undo, the depth cap, and the two boundaries — repeatable `execute`, and two users needing OT or CRDTs.
3. *There is a `while` inside the `for`. Is that not $O(nk)$?* n appends total, each index removed at most once, so $\leq 2n$ operations — bounded in total, not per iteration — plus the example where one arrival crushes a thousand elements and why that costs nothing.

Before you move on

- ☐ I can state the delete-for-ever observation and say why “for ever” is justified.
- ☐ I can name both removals and give the different reason for each.
- ☐ I can say why this needs a deque rather than a stack or a queue.
- ☐ I traced the standard input and know where both removals happen in one step.
- ☐ I measured the brute force, the heap and the deque, and can quote all three numbers.
- ☐ I know what `[8, 3, 2]` with $k = 2$ returns under the off-by-one expiry.
- ☐ I dropped the record guard and counted the wrong number of answers.
- ☐ I can state the space as $O(k)$ and name the worst-case and best-case inputs.
- ☐ I wrote the two-deque version for $\max - \min \leq \text{limit}$.
- ☐ I built `InsertText`, `DeleteText`, `History` and `MacroCommand`.
- ☐ I removed the redo-clearing line and saw what redo did to the document.
- ☐ I can quote the 500 MB against 0.5 MB comparison and explain the thousandfold.
- ☐ I can name two actions that can only be compensated, not undone.
- ☐ I built the approval chain and decided deliberately what happens at the end of it.
- ☐ I can say the cheapest-rejection-first rule and what it saves under attack.
- ☐ I answered all three questions above out loud.

A queue from two stacks, and a stack from queues

DATA STRUCTURES AND ALGORITHMS

A queue from two stacks, and a stack from queues

You can build one from the other and give the amortised cost honestly.

SYSTEM DESIGN

Template method and iterator

You can fix the skeleton of an algorithm and let subclasses fill in the steps.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Implement a queue using only stacks. What is the amortised cost?*
- *Three report types share 80 percent of their logic. Structure it.*

A queue from two stacks, and a stack from queues

1 What this is, and why they ask it

You are allowed one operation at one end — push and pop from the top of a stack — and you have to produce first-in-first-out behaviour. The trick is that **pouring one stack into another reverses the order**, so two stacks give you both ends. Then the same question is asked backwards: build a stack using only queues.

The code is short. The interview is not about the code. It is about the sentence “*pop is $O(n)$ in the worst case and $O(1)$ amortised*”, and about being able to prove that claim by counting how many times a single element can be touched across its whole life. This is the first problem in the course where **amortised** is the answer rather than a footnote.

They ask it because it separates people who have memorised complexities from people who can derive them. It is LeetCode 232 and 225, it turns up in phone screens constantly, and the follow-up is always the same: “*but pop can be $O(n)$ — so how can you call it $O(1)$?*” If you can answer that calmly, with a count rather than a claim, the question is over.

2 The story

Basava’s shop opens at six, and the first thing he does, before the shutters are even fully up, is sort out the milk.

The milk comes in soft packets and it goes bad, so the rule is simple and unbreakable: the oldest packet in the shop has to be the next one sold. If a packet from Tuesday is still there on Friday, that is money thrown away, and he learned that lesson expensively in his first year.

The trouble is the fridge. It is a narrow upright thing with one deep shelf, and packets go in from the front and come out from the front. Whatever he put in last is what his hand reaches first. The delivery boy comes at half past six and again at four, and both times the new packets go in at the front — because that is the only place they can go.

So the packets he needs to sell are always at the back, behind everything newer, and he cannot get at them.

What he does is this. Every morning he takes the packets out of the fridge one at a time and stacks them on the counter beside the till. The one that comes out first goes down first, the next one on top of it, and so on. By the time the fridge is empty, the pile on the counter is the other way round — the oldest packet, the one that was right at the back, is now sitting on top where he can pick it up.

Then he sells from the counter pile, top down, and every packet he hands over is the oldest one in the shop. New deliveries go straight into the fridge, at the front, where they belong.

When the counter pile runs out, he empties the fridge onto the counter again, and it comes out reversed again, oldest on top.

The one rule he is absolutely strict about, and he told his nephew off twice about this in one week, is that he never adds to the counter pile while there is still something on it. If the fridge gets poured onto a half-finished counter pile, the newer packets end up sitting on top of older ones and he starts selling them in the wrong order. Empty the counter completely, then pour. Never before.

His nephew thinks the pouring is a waste of time — twenty packets moved at once, standing there at half past six. Basava's answer is that every single packet gets picked up exactly twice in its life: once going into the fridge, once coming out onto the counter. Twenty at a time looks like a lot of work. Spread over the whole day, it is two touches a packet.

3 The idea in plain English

The fridge is a **stack**: last in, first out. So is the counter pile. Two stacks, and the only clever thing in the whole design is that **pouring one stack into another reverses the order**, which turns LIFO into FIFO.

The two stacks

- **inbox** — the fridge. Everything you **enqueue** goes here, on top.
- **outbox** — the counter pile. Everything you **dequeue** comes from here, off the top.

enqueue is one line: push onto **inbox**.

dequeue is two: if **outbox** is empty, pour the whole of **inbox** into it; then pop **outbox**.

```
def dequeue(self):
    if not self._outbox:                # only when empty. This is the rule.
        while self._inbox:
            self._outbox.append(self._inbox.pop())    # pouring reverses
        return self._outbox.pop()
```

Read the **while** as Basava emptying the fridge onto the counter. The first packet out of the fridge is the newest, so it goes to the bottom of the counter pile; the last one out is the oldest, so it ends on top. Reversal, for free, as a side effect of moving things.

The rule that makes it correct

Pour only when **outbox is empty.** This is not an optimisation. It is what makes the queue a queue.

If you pour while three packets are still on the counter, the freshly poured ones land on top of them, and the next thing you sell is newer than something still underneath. In code: **enqueue(1)**, **enqueue(2)**, **dequeue()** — which pours and returns 1 — then **enqueue(3)**,

and now a pour with a non-empty outbox puts 3 above 2, so the next dequeue returns 3 instead of 2. The order is wrong and nothing raises an error.

The invariant to say out loud: **outbox holds the oldest elements, in the order they will leave; inbox holds the newest, upside down. The whole queue, front to back, is outbox reversed followed by inbox.**

Amortised, and what the word actually means

One `dequeue` can be expensive. If a thousand elements are sitting in `inbox` and `outbox` is empty, that dequeue moves all thousand. That is $O(n)$ for a single operation, and you should say so plainly rather than hide it.

Now count over the whole life of one element:

```
push onto inbox      1 operation
pop off inbox        1 operation   (during a pour)
push onto outbox     1 operation   (during the same pour)
pop off outbox       1 operation
-----
total, for its whole life: 4
```

Four operations per element, whatever order the calls come in. So n enqueues and n dequeues cost at most $4n$ operations in total, which is $O(1)$ each on average. That is what **amortised $O(1)$** means: not “usually fast”, but “the total across any sequence is bounded, so the cost per operation averages to a constant”.

The key fact holding it up: **an element moves from inbox to outbox exactly once.** It can never go back. That is the same one-way argument as the monotonic stack of [day 071](#) and it is the same argument that makes `list.append` amortised $O(1)$, from [day 005](#).

Amortised is not average-case. Average-case is about which inputs are likely. Amortised makes no assumption about the input at all: it is a guarantee about any sequence of operations, including the worst one an adversary could pick. Saying that distinction correctly is worth a lot in the follow-up.

Where amortised is not good enough

If this queue sits behind a request that must answer in five milliseconds, an occasional operation that moves a hundred thousand elements is a latency spike, and “amortised $O(1)$ ” is no comfort to the one request that hit it. Real-time systems care about the worst single operation, not the average. Say this if asked — it shows you know that complexity is not the only thing that matters.

Now backwards: a stack from queues

A queue only lets you take from the front, so to get the *last* thing in you must rotate everything else past it. You choose which operation pays.

Costly push, cheap pop — usually the right answer:

```
def push(self, value):
    self._queue.append(value)
    for _ in range(len(self._queue) - 1):
        self._queue.append(self._queue.popleft()) # rotate the older ones behind
```

After the rotation, the newest element is at the front, so `pop` is just `popleft` — $O(1)$. Push is $O(n)$. One queue is enough.

Costly pop, cheap push is the mirror: append on push, and on pop move `n - 1` elements to a second queue and return the last one.

And unlike the queue-from-stacks, **there is no amortised trick here**. Every push does `n - 1` rotations, every time; there is no “each element moves once” argument, because the same elements rotate on every single push. This asymmetry is the interesting observation of the day: **two stacks make a genuinely efficient queue, but queues do not make an efficient stack**. Being able to say *why* — one direction has a one-way flow, the other repeats work — is the strongest thing you can say about this pair of problems.

4 The picture

`enqueue(1)`, `enqueue(2)`, `enqueue(3)`, then `dequeue()`.

after three enqueues	during the pour	after the pour
<pre> ----- inbox outbox +---+ (empty) 3 ← top +---+ 2 +---+ 1 +---+ order in: 1, 2, 3 </pre>	<pre> ----- inbox outbox +---+ +---+ 2 3 +---+ +---+ 1 +---+ </pre> 3 moved first, so it lands at the bottom of outbox	<pre> ----- inbox outbox (empty) +---+ 1 ← top +---+ 2 +---+ 3 +---+ </pre> 1 is on top: dequeue returns 1 — correct FIFO

What to notice: nothing was sorted and nothing was searched. The reversal is a **side effect of moving a stack one item at a time into another stack**, and that side effect is the entire algorithm.

Now the rule, drawn as the thing that goes wrong:

CORRECT: pour only when outbox is empty

```
enqueue 1,2 → dequeue (pours, returns 1) → enqueue 3 → dequeue
outbox holds [2] (2 on top); inbox holds [3]
dequeue does NOT pour, because outbox is not empty → returns 2 correct
```

WRONG: pour whenever inbox is non-empty

```
same sequence, but the pour happens anyway
outbox becomes [2] with 3 pushed on top
+---+
| 3 | ← top    dequeue returns 3
+---+
| 2 |          ...but 2 arrived first. Order broken, no error raised.
+---+
```

And the whole queue, read as one line, so the invariant is visible:

```
front                                     back
|                                         |
v                                         v
[ outbox top ... outbox bottom ][ inbox bottom ... inbox top ]
oldest                             newest
```

5 The code, built step by step

Step 1 — enqueue is trivial

```
def enqueue(self, value: int) → None:
    self._inbox.append(value)
```

$O(1)$, always, no conditions. Every element enters the same way.

Step 2 — the pour, in its own method

```
def _pour(self) → None:
    """Move everything from inbox to outbox, reversing the order."""
    if not self._outbox:                # ONLY when empty
        while self._inbox:
            self._outbox.append(self._inbox.pop())
```

Putting the emptiness check *inside* the helper is deliberate: now `dequeue` and `peek` both call `_pour()` unconditionally and cannot get the rule wrong independently. Two call sites, one place where the rule lives.

Step 3 — dequeue and peek

```
def dequeue(self) → int:
    self._pour()
    if not self._outbox:
        raise IndexError("dequeue from an empty queue")
    return self._outbox.pop()

def peek(self) → int:
    self._pour()
    if not self._outbox:
        raise IndexError("peek at an empty queue")
    return self._outbox[-1]
```

`peek` must pour too. Forgetting that is a real bug: `enqueue(1); peek()` would look at an empty `outbox` while the element sits in `inbox`.

Step 4 — emptiness and size

```
def is_empty(self) → bool:
    return not self._inbox and not self._outbox    # BOTH must be empty

def __len__(self) → int:
    return len(self._inbox) + len(self._outbox)
```

Checking only one of the two is the second-most-common bug in this problem, and it produces a queue that reports itself empty while holding a thousand elements.

Step 5 — the stack from one queue

```
def push(self, value: int) → None:
    self._queue.append(value)
    for _ in range(len(self._queue) - 1):
        self._queue.append(self._queue.popleft())
```

Append the newcomer at the back, then move every *other* element from the front to the back. After `len - 1` rotations the newcomer is at the front, and everything else is behind it in the original stack order.

Trace `push(1); push(2); push(3)`:

```
push 1:  [1]                                0 rotations
push 2:  [1,2] → rotate 1 → [2,1]           1 rotation
push 3:  [2,1,3] → rotate 2 → [3,2,1]
pop → popleft → 3.  Correct LIFO.
```

The complete solution

```
from collections import deque

class QueueFromStacks:
    """A FIFO queue built from two LIFO stacks.

    Pouring one stack into another reverses the order, which is the entire
    trick. `inbox` takes every arrival; `outbox` serves every departure.

    THE RULE: pour only when outbox is empty. Pouring onto a non-empty outbox
    puts newer elements above older ones and silently breaks FIFO order.

    Cost: enqueue  $O(1)$  always. dequeue  $O(n)$  worst case,  $O(1)$  AMORTISED, because
    each element is touched exactly four times in its whole life – pushed to
    inbox, popped from inbox, pushed to outbox, popped from outbox – and can
    never move backwards.
    """

    def __init__(self) → None:
        self._inbox: list[int] = []          # arrivals, newest on top
        self._outbox: list[int] = []         # departures, oldest on top

    def enqueue(self, value: int) → None:
        self._inbox.append(value)

    def dequeue(self) → int:
        self._pour()
        if not self._outbox:
            raise IndexError("dequeue from an empty queue")
        return self._outbox.pop()

    def peek(self) → int:
        self._pour()
        if not self._outbox:
            raise IndexError("peek at an empty queue")
        return self._outbox[-1]

    def is_empty(self) → bool:
        return not self._inbox and not self._outbox      # both, not either

    def __len__(self) → int:
        return len(self._inbox) + len(self._outbox)

    def _pour(self) → None:
        if not self._outbox:                             # the rule, in one place
            while self._inbox:
                self._outbox.append(self._inbox.pop())

class StackFromOneQueue:
    """A LIFO stack built from a single FIFO queue: costly push, cheap pop.

    push rotates the older elements behind the newcomer, so the newest element
    is always at the front of the queue and pop is a plain popleft.

    Cost: push  $O(n)$  EVERY time – there is no amortised saving here, because the
    same elements rotate again on the next push. pop, top and empty are  $O(1)$ .
    """

    def __init__(self) → None:
        self._queue: deque[int] = deque()

    def push(self, value: int) → None:
```



```

        self._queue.append(value)
        for _ in range(len(self._queue) - 1):
            self._queue.append(self._queue.popleft())    # older ones go behind

    def pop(self) → int:
        if not self._queue:
            raise IndexError("pop from an empty stack")
        return self._queue.popleft()

    def top(self) → int:
        if not self._queue:
            raise IndexError("top of an empty stack")
        return self._queue[0]

    def is_empty(self) → bool:
        return not self._queue

    def __len__(self) → int:
        return len(self._queue)

class StackFromTwoQueues:
    """The mirror: cheap push, costly pop.

    push appends. pop moves n-1 elements to a second queue, returns the last
    one, and swaps the two queues. Worth knowing so you can say which operation
    you chose to make expensive, and why.
    """

    def __init__(self) → None:
        self._main: deque[int] = deque()
        self._spare: deque[int] = deque()

    def push(self, value: int) → None:
        self._main.append(value)    # 0(1)

    def pop(self) → int:
        if not self._main:
            raise IndexError("pop from an empty stack")
        while len(self._main) > 1:
            self._spare.append(self._main.popleft())
        value = self._main.popleft()
        self._main, self._spare = self._spare, self._main    # swap, do not copy
        return value

    def is_empty(self) → bool:
        return not self._main

if __name__ == "__main__":
    q = QueueFromStacks()
    q.enqueue(1)
    q.enqueue(2)
    print(q.dequeue())    # 1    (this call pours)
    q.enqueue(3)
    print(q.dequeue())    # 2    (this call must NOT pour)
    print(q.dequeue())    # 3    (this one pours again)
    print(q.is_empty())    # True
    try:
        q.dequeue()
    except IndexError as error:
        print(f"IndexError: {error}")    # IndexError: dequeue from an empty queue

    s = StackFromOneQueue()
    for value in (1, 2, 3):
        s.push(value)

```

```

print(s.pop(), s.pop(), s.pop())    # 3 2 1

t = StackFromTwoQueues()
for value in (1, 2, 3):
    t.push(value)
print(t.pop(), t.pop(), t.pop())    # 3 2 1

# The two must behave identically to the real thing, on any sequence.
import random
from collections import deque as real_queue

for _ in range(2000):
    mine, reference = QueueFromStacks(), real_queue()
    for _ in range(30):
        if reference and random.random() < 0.4:
            assert mine.dequeue() == reference.popleft()
        else:
            value = random.randint(0, 99)
            mine.enqueue(value)
            reference.append(value)
    assert len(mine) == len(reference)
print("queue-from-stacks agreed with a real queue on 2000 random sequences")

```

The random sequence check at the end is the one that catches the pour-when-not-empty bug, because that bug only shows up on an interleaving of enqueues and dequeues. A test that enqueues everything and then dequeues everything will never find it.

6 What it costs

The queue from two stacks

enqueue	0(1)	always. One push.
dequeue	0(n)	worst case – the pour moves everything in inbox
	0(1)	amortised
peek		same as dequeue
is_empty	0(1)	
space	0(n)	total across both stacks

The amortised argument, counted rather than asserted:

```

one element's whole life:
pushed onto inbox      1
popped off inbox       1      (during a pour)
pushed onto outbox     1      (during the same pour)
popped off outbox      1
-----
total                  4      – and it can never go back to inbox

n enqueues + n dequeues ≤ 4n operations → 0(1) amortised each

```

Concretely: enqueue a thousand elements, then dequeue a thousand. The first dequeue does a thousand moves; the next 999 do none. Total work is 2,000 pushes and 2,000 pops for 2,000 operations — two operations each, on average, even though one call did a thousand.

The worst single operation is still $O(n)$, and you should say both numbers. “Amortised $O(1)$, worst case $O(n)$ for one call” is a complete answer; “ $O(1)$ ” alone is a claim you will be asked to defend.

The stack from queues

```
push      0(n)   EVERY time
pop        0(1)
top        0(1)
space     0(n)
```

There is deliberately no “amortised” line here. On a sequence of n pushes:

```
push 1: 0 rotations
push 2: 1 rotation
push 3: 2 rotations
...
push n: n-1 rotations
-----
total: n(n-1)/2 rotations →  $O(n^2)$  for  $n$  pushes →  $O(n)$  each, not amortised down
```

At $n = 10,000$ pushes that is about fifty million rotations. Compare with the queue-from-stacks, where n enqueues and n dequeues cost $4n = 40,000$ operations. **Same-looking problem, thousand-fold difference in behaviour**, and the reason is in one sentence: in the two-stack queue each element moves between the stacks once and never returns; in the stack-from-queue the same elements rotate again on every push.

Compared with the real thing

```
collections.deque: enqueue 0(1), dequeue 0(1), worst case 0(1)
two stacks:       enqueue 0(1), dequeue 0(1) amortised, 0(n) worst case
```

If you are allowed a real queue, use one. This construction exists as an exercise, and in one genuine setting: some purely functional languages build queues exactly this way, because two immutable lists are cheap to build and a “reverse the front list when the back runs out” is the same amortised argument.

7 The traps

Trap 1 — pouring when `outbox` is not empty

```
def _pour(self) → None:
    while self._inbox:
        self._outbox.append(self._inbox.pop())
```

No error. No crash. Wrong order, silently. The sequence that exposes it:

```
enqueue(1); enqueue(2); dequeue() → 1    outbox now holds [2]
enqueue(3); dequeue()           → 3    WRONG. Should be 2.
```

3 was poured on top of 2. This is the bug the random-sequence test exists to catch, and it will never show up if you test by enqueueing everything first and then dequeuing everything.

Trap 2 — `is_empty` checking one stack

```
def is_empty(self) → bool:
    return not self._inbox          # WRONG
```

After `enqueue(1)` the element is in `inbox` and `outbox` is empty, so this reports an empty queue holding one element. Any caller looping `while not queue.is_empty()` exits immediately and drops everything. Check both.

Trap 3 — `peek` that forgets to pour

```
def peek(self) → int:
    return self._outbox[-1]        # no _pour()
```

```
IndexError: list index out of range
```

Fires on `enqueue(1); peek()`. `dequeue` and `peek` need exactly the same preparation, which is the argument for putting the pour in its own method that both call.

Trap 4 — pouring on every dequeue “to be safe”

```
def dequeue(self) → int:
    while self._inbox:             # unconditional
        self._outbox.append(self._inbox.pop())
    return self._outbox.pop()
```

This is trap 1 in a friendlier disguise — it is trap 1 whenever `outbox` is non-empty. It also destroys the amortised bound even when it happens to be correct, because elements move repeatedly.

Trap 5 — claiming $O(1)$ without the qualifier

Saying “dequeue is $O(1)$ ” and stopping. The interviewer will say “but the pour moves n elements”, and now you are defending rather than explaining. Say it first, in full: “ **$O(n)$ for one call in the worst case, $O(1)$ amortised, because each element crosses from inbox to outbox exactly once and can never go back.**”

Trap 6 — confusing amortised with average

“On average it is fast” is not the claim. Average-case reasoning assumes something about which inputs arrive. Amortised assumes nothing: it bounds the **total** cost of *any* sequence of operations, including the worst one someone could design on purpose. Get this distinction right and

it is noticeable.

Trap 7 — rotating the wrong number of times in the stack-from-queue

```
for _ in range(len(self._queue)):      # should be len - 1
    self._queue.append(self._queue.popleft())
```

One rotation too many puts the newcomer back at the *end*, which turns your stack into a queue. `push(1); push(2); pop()` returns 1 instead of 2. Compute `len` after appending the newcomer, and rotate one fewer than that.

Trap 8 — copying instead of swapping in the two-queue stack

```
self._main = self._spare
self._spare = deque()          # allocates a new queue every pop
```

Correct but wasteful: it discards and reallocates on every pop. Swap the two names instead — `self._main, self._spare = self._spare, self._main` — which is one tuple assignment and no allocation.

8 In the interview

How it gets asked

- The direct version: *“Implement a queue using only stacks. What is the cost of each operation?”* LeetCode 232. The follow-up is guaranteed: *“pop can be $O(n)$, so how is that $O(1)$?”*
- The reverse: *“Now implement a stack using only queues.”* LeetCode 225, usually as a five-minute add-on.
- The theory version: *“Explain amortised complexity using an example.”* This problem and Python’s list growth are the two standard answers.
- The comparison: *“You have two implementations of a stack from queues — costly push or costly pop. Which do you choose?”*

What to say out loud, in the first ninety seconds

1. **Name the trick immediately.** “A stack gives me last-in-first-out. If I pour one stack into another, the order reverses. So two stacks give me first-in-first-out.”
2. **Name the two roles.** “One stack takes every arrival — call it `inbox`. The other serves every departure — `outbox`. Enqueue always pushes to `inbox`; dequeue always pops from `outbox`.”
3. **State the rule as a rule, not a detail.** “The correctness condition is that I only pour when `outbox` is empty. Pouring onto a non-empty `outbox` puts newer elements above older ones, and nothing raises an error — it just returns the wrong element.”
4. **Give both complexities before being asked.** “Enqueue is $O(1)$ always. Dequeue is $O(n)$ for a single call in the worst case, and $O(1)$ amortised.”

5. **Prove the amortised claim by counting.** “Each element is touched exactly four times in its entire life: pushed onto `inbox`, popped off `inbox`, pushed onto `outbox`, popped off `outbox`. It can never move backwards. So n enqueues and n dequeues cost at most $4n$ operations, whatever order the calls come in.”

6. **Then write the code**, which is about fifteen lines.

The follow-ups

“Deque can move n elements. How is that $O(1)$?” “It is not, for a single call — that call is genuinely $O(n)$ and I would say so. The claim is about any sequence of calls. An element enters `inbox` once, crosses to `outbox` once, and leaves once; it can never go back. So the total work across n enqueues and n dequeues is bounded by about $4n$ operations, which averages to a constant each. Concretely: enqueue a thousand and then dequeue a thousand — the first dequeue does a thousand moves and the next 999 do none, so it is two operations per element overall.”

“What is the difference between amortised and average-case?” “Average-case makes an assumption about the distribution of inputs — it says ‘on typical data this is fast’. Amortised makes no assumption at all: it bounds the total cost of any sequence, including one an adversary picks deliberately. There is no input to this queue that makes it worse than $4n$. That is a stronger guarantee, and it is why the same word is used for Python’s list growth.”

“Is amortised good enough in production?” “It depends entirely on whether you care about the worst single operation. For total throughput, yes. For a service with a p99 latency target, a dequeue that occasionally moves a hundred thousand elements is a visible spike, and the average is no comfort to the request that hit it. In a real-time or low-latency system I would use a structure with a worst-case bound — `collections.deque` here — and keep this construction for the interview and for functional languages where it is genuinely idiomatic.”

“Now build a stack from queues.” “One queue is enough if I make push expensive: append the new element, then rotate the other `$n - 1$` elements from the front to the back, so the new-comer ends up at the front. Then pop is a plain `popleft`, $O(1)$. The mirror is cheap push and costly pop with two queues. I would pick costly push if pops are more frequent, which they usually are, and it uses one queue instead of two.”

“Does the same amortised argument apply to the stack from queues?” “No, and that is the interesting asymmetry. In the two-stack queue, an element crosses once and never returns, so the work is bounded in total. In the stack from queues, every push rotates *all* the existing elements again — the same elements move over and over. So n pushes cost $n(n-1)/2$ rotations, which is quadratic; there is no amortised saving. Two stacks make a good queue, but queues do not make a good stack.”

“Why would anyone build a queue this way?” “In an interview, to test whether I can count operations. In real code, almost never — I would use a deque. The one genuine setting is purely functional programming, where you cannot mutate: a queue is represented as a front list and a reversed back list, and when the front runs out you reverse the back onto it. That is exactly this structure and exactly this amortised argument.”

A model answer

Asked: *implement a queue using only stacks, and tell me the amortised cost.*

“The whole idea is one observation: a stack is last-in-first-out, and if I pour one stack into another by popping and pushing, the order comes out reversed. So two stacks give me both ends.

I will keep two. `inbox` receives every arrival — enqueue is just a push, $O(1)$, no conditions. `outbox` serves every departure — dequeue pops from it. When `outbox` runs out, I pour the whole of `inbox` into it, which reverses `inbox`, so the oldest element ends up on top of `outbox` where I can take it.

The correctness condition, which I would write as a comment because it is easy to get wrong, is that I pour **only** when `outbox` is empty. If I pour onto a non-empty `outbox`, the newer elements land on top of older ones and I start returning things out of order — and nothing raises an error, the queue just quietly lies. The sequence that exposes it is enqueue 1, enqueue 2, dequeue, enqueue 3, dequeue: the second dequeue should give 2 and gives 3.

A couple of small things that follow from having two stacks: `is_empty` must check both, or a queue holding one element in `inbox` reports itself empty. And `peek` needs the same pour as `dequeue`, so I put the pour in one private method that both call, rather than trusting myself to remember it twice.

On cost. Enqueue is $O(1)$ always. A single dequeue is $O(n)$ in the worst case, and I would say that plainly rather than have you find it. But it is $O(1)$ amortised, and here is the count that proves it. Over its whole life, one element is touched exactly four times: pushed onto `inbox`, popped off `inbox`, pushed onto `outbox`, popped off `outbox`. And it can never move backwards — once it is in `outbox` it only leaves. So n enqueues and n dequeues cost at most $4n$ operations, whatever order the calls arrive in. Enqueue a thousand and dequeue a thousand: the first dequeue moves a thousand elements and the next 999 move none.

That is stronger than an average-case claim, which is worth being precise about. Average-case would assume something about typical inputs. Amortised assumes nothing — there is no sequence of calls, adversarial or otherwise, that costs more than about $4n$.

The one place I would not be satisfied with it is a latency-sensitive service. A dequeue that occasionally moves a hundred thousand elements is a p99 spike, and ‘amortised’ is no comfort to the request that hit it. There I would want a worst-case bound, which is what `collections.deque` gives me.

If you want the reverse — a stack from queues — one queue is enough, with an expensive push: append the newcomer, then rotate the other $n-1$ elements from front to back so the newcomer is at the front, and pop becomes a plain `popleft`. What is worth noticing is that the amortised argument does *not* transfer. Every push rotates all the existing elements again, so n pushes cost about $n^2/2$ rotations. Two stacks make a genuinely efficient queue; queues do not make an efficient stack, and the difference is whether elements move once or move repeatedly.”

9 Recall card

- **Pouring one stack into another reverses the order** — that single side effect turns two LIFO stacks into a FIFO queue. `inbox` takes every arrival (**enqueue is $O(1)$, always**); `outbox` serves every departure.
- **The correctness rule: pour ONLY when `outbox` is empty**. Pouring onto a non-empty `outbox` puts newer elements above older ones and **fails silently** — `enqueue 1, 2 · dequeue · enqueue 3 · dequeue` returns **3 instead of 2**. Put the guard inside one `_pour()` that both `dequeue` and `peek` call. `is_empty` must check **both** stacks.
- **Say both numbers: dequeue is $O(n)$ for one call, $O(1)$ amortised**. The proof is a count, not a claim: each element is touched exactly **4 times** in its life — `inbox push`, `inbox pop`, `outbox push`, `outbox pop` — and **can never move backwards**, so n enqueues + n dequeues $\leq 4n$ operations.
- **Amortised is not average-case**. Average-case assumes something about the inputs; amortised bounds the **total cost of any sequence**, including an adversarial one. But it says nothing about a single call, so it is the wrong guarantee for a **p99 latency target** — there you want `deque`.
- **The asymmetry is the real lesson: two stacks make a good queue; queues do not make a good stack**. A one-queue stack rotates `len - 1` on **every** push, so n pushes cost $n(n-1)/2$ — no amortised saving, because the same elements move again and again. Rotate `len - 1`, not `len`, or your stack becomes a queue.

1 What this is, and why they ask it

Template method is the pattern where a base class writes down the *order of the steps* and leaves some of the steps blank. The order is fixed and subclasses cannot change it. What they can do is fill in the blanks. One method — the template — calls the steps in sequence, and that method is the thing you are not allowed to override.

Iterator is the pattern where “go through this collection one item at a time” is separated from “this collection”. The caller asks for the next item and does not know, and does not want to know, whether the items are in a list, in a file, in a database, or arriving over the network.

They are taught together because they are the two patterns that are already built into the language you are using. Every `for` loop in Python is the iterator pattern. Every `unittest.TestCase` with `setUp` and `tearDown` is a template method. Interviewers ask template method as “*these three reports are 80 percent identical — restructure it*”, which is the most common real refactoring question there is, and they ask iterator as “*this query returns ten million rows and the process runs out of memory*”, which is the most common real production incident.

2 The story

Every morning at Nandini’s school the assembly runs the same way, and it has run that way since long before she joined.

The bell goes at eight forty. The children come out and stand in their lines by class, tallest at the back, and that takes four minutes on a good day and seven when it has rained. Then the head girl leads the prayer. Then somebody speaks for three or four minutes. Then everyone says the pledge together. Then the classes are sent back inside, one at a time, starting with the smallest children.

Bell, lines, prayer, the talk, the pledge, back inside. Six things, always in that order, and nobody has ever written it down anywhere Nandini has seen. It simply is the assembly.

The one part that changes is the talk. Monday is the principal, and it is usually about shoes or about the exam timetable. Tuesday and Thursday a class teacher takes it by turn. Wednesday is the sports teacher, who is short, and everybody likes that. Friday one of the students speaks, and that one is either very good or very bad.

The teacher whose turn it is does not decide when to speak, or whether the prayer comes before or after, or how long the lines take. She is told: four minutes, after the prayer. She fills her slot. That is all she does.

There is a seventh thing, but it only happens sometimes. If somebody has won something outside school, they are called up and clapped, after the pledge and before the classes go in. Most days that step simply does not happen, and nobody stands there feeling a gap where it should have been.

Once, a new teacher sent her own class back inside straight after the prayer, because it was very hot and a girl had fainted the week before. The whole assembly came apart — the other classes were still standing, the pledge had not been said, and the little ones followed her class because they follow everybody. She was not told off for wanting to protect her students. She was told off because the order is not hers to change.

And the register is its own small thing. Every teacher reads the names out one at a time and marks each child present, and not one of them knows or cares how the names are kept — by roll number, by the order the children joined, by anything at all. You start at the beginning and keep going until there are no more names.

3 The idea in plain English

The assembly is a **template method**. The register is an **iterator**. Two patterns, one morning.

Template method

The base class owns the order:

```
class Assembly:
    def run(self) → None:           # THE TEMPLATE. Do not override this.
        self.ring_bell()
        self.form_lines()
        self.prayer()
        self.talk()                 # the blank
        self.pledge()
        self.award()                # the optional blank – a "hook"
        self.dismiss()
```

Five of those are written once in the base class, because they never change. `talk` is **abstract** — the base class declares it and refuses to implement it, so every subclass must. `award` is a **hook**: the base class implements it as *doing nothing*, so a subclass may override it and most do not.

That is the complete pattern. Three kinds of step:

KIND	BASE CLASS	SUBCLASS
Fixed step	implements it	must not override
Abstract step	declares it, refuses to implement	must implement
Hook	implements it as a no-op or a sensible default	may override

The new teacher's mistake was overriding `run`. That is the one method the pattern exists to protect. In Java you write `final` on it; in Python you write a comment and rely on people reading it, which is a genuine weakness worth mentioning.

The Hollywood principle

The usual one-line summary is: “**Don’t call us, we’ll call you.**”

In ordinary code *you* call the library. In a template method the base class calls *your* code. The teacher does not decide when to speak; she is called. This inversion is what makes frameworks frameworks — the reason `unittest` runs your `setUp` before your `test_` method is that it owns the loop and you own the steps.

What it is for: three reports that are 80 percent the same

```
sales report:    fetch rows → group by region → render to CSV → email it
inventory report: fetch rows → group by depot  → render to XLSX → email it
finance report:  fetch rows → group by month   → render to PDF  → email it
```

Everything except two steps is identical, and today those three files are near-copies of each other. The pattern moves the identical parts into one place and leaves two blanks:

```
class Report:
    def generate(self, start: date, end: date) → None:      # the template
        rows = self.fetch(start, end)                      # fixed
        grouped = self.group(rows)                         # blank
        document = self.render(grouped)                    # blank
        self.deliver(document)                             # fixed
```

Adding a fourth report becomes: one class, two methods. And, more importantly, fixing a bug in `deliver` becomes one edit instead of three.

Iterator

The register is the other half. A teacher reads names one at a time and never learns how they are stored. In code:

```
for student in classroom:
    mark_present(student)
```

That `for` loop asks `classroom` for an **iterator** — an object with one job, `__next__`, which returns the next item or signals that there are none left. `classroom` might hold a list, or read a file, or page through a database. The loop does not change.

```
class Classroom:
    def __iter__(self):
        return iter(self._students)          # hand back something with __next__
```

Why this matters more than it looks: if `__iter__` hands back a *generator* rather than a list, the items are produced one at a time and never all exist at once. That is the difference between a report that holds ten million rows in memory and one that holds a thousand.

```
def __iter__(self):
    offset = 0
    while True:
        page = db.fetch(limit=1000, offset=offset) # one page in memory
        if not page:
            return
        yield from page # hand them out one by one
        offset += 1000
```

The caller still writes `for row in report_source: .` It has no idea that a database is being paged behind it. That is the pattern doing its job: **the traversal is separated from the collection, so one can change without touching the other.**

The distinctions you will be asked for

Template method versus Strategy (day 071): template method varies a step by **subclassing** and fixes the algorithm at compile time; Strategy varies a whole behaviour by **holding an object** and can change it at run time. Template method uses inheritance; Strategy uses composition. If you want two variations at once, or want to swap the behaviour while the program runs, you want Strategy.

Iterator versus a plain list: a list is a collection; an iterator is a *position in* a traversal. A list can be walked many times; a generator is **one-shot** — walk it twice and the second walk is empty, which is a bug people hit constantly.

4 The picture

Template method, with the three kinds of step marked:

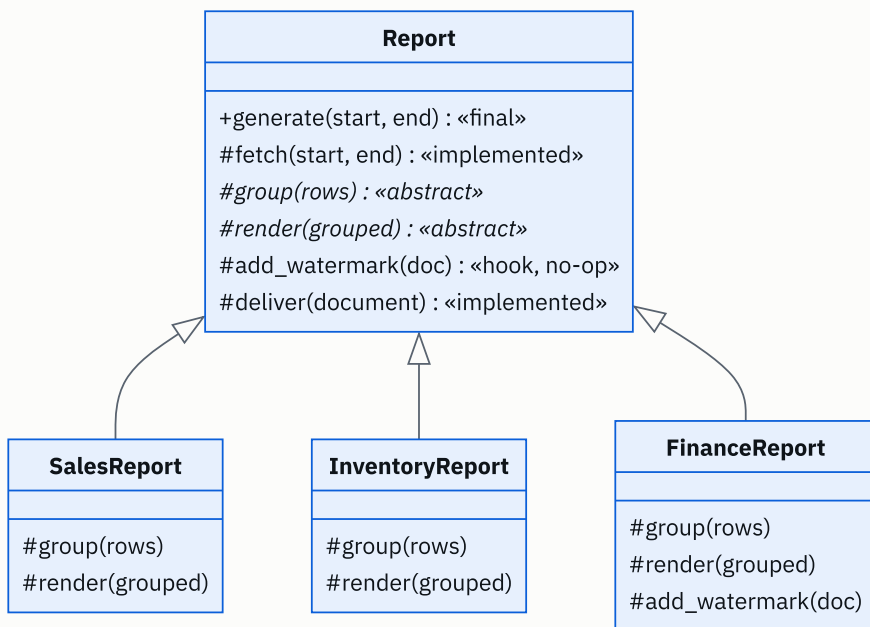


FIGURE 75.1

What to notice: `generate` appears **only** in the base class and in none of the subclasses. That absence is the pattern. If a subclass overrides `generate`, the pattern has been broken and the duplication will come straight back. Also notice that only `FinanceReport` overrides the hook — the other two get the default without writing anything.

The control flow, which is the part that surprises people:

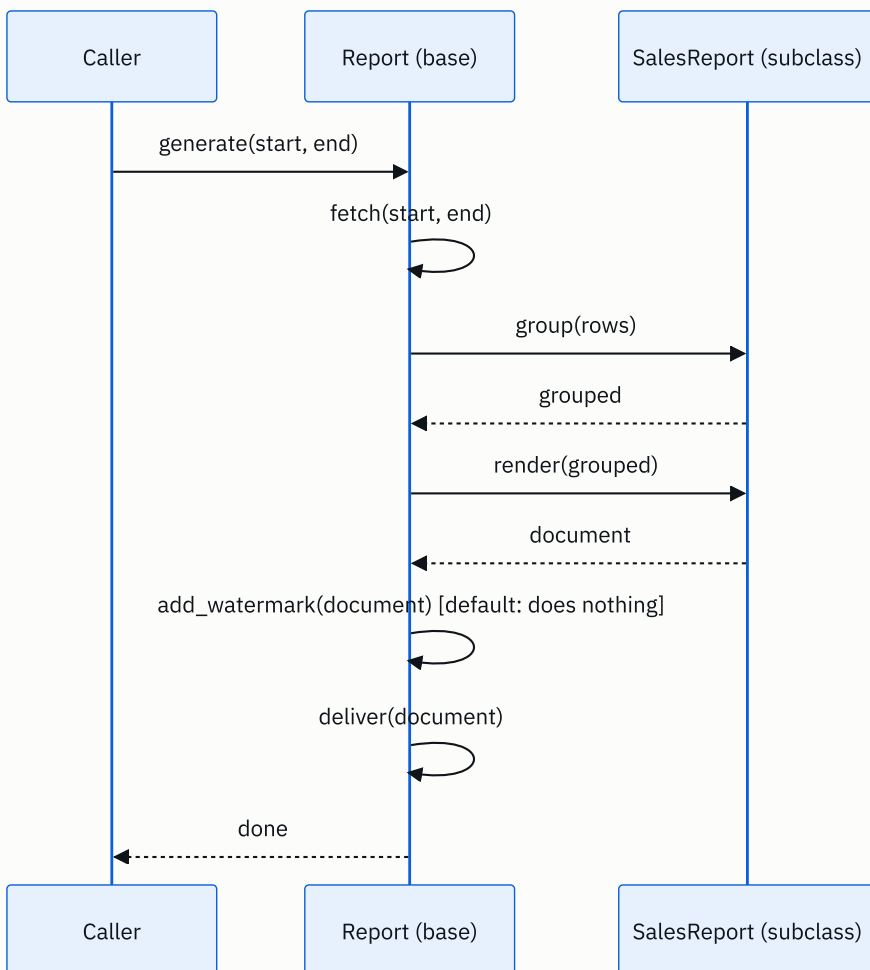


FIGURE 75.2

What to notice: the arrows go **from the base class into the subclass**, not the other way. That is “don’t call us, we’ll call you”, drawn.

And the iterator, showing why it is not just a loop:

without an iterator (a list):

```
[ row 1 ][ row 2 ][ row 3 ] ... [ row 10,000,000 ]    all in memory at once
|<----- 2 GB ----->|
```

with an iterator (a generator over pages):

```
fetch page → [ 1,000 rows ] → hand out one at a time → discard → fetch next
|← 200 KB →|
```

the caller's code is identical: `for row in source: ...`

5 How it actually works

The template, written properly

```
from abc import ABC, abstractmethod

class Report(ABC):
    def generate(self, start: date, end: date) → Path:
        """The template. The ORDER lives here and subclasses must not change it."""
        rows = self.fetch(start, end)
        if not rows:
            return self.empty_document()          # a hook with a real default
        grouped = self.group(rows)
        document = self.render(grouped)
        self.decorate(document)                  # hook: usually does nothing
        return self.deliver(document)
```

Eight lines that say what a report is. Everything about ordering, about the empty case, about delivery, is decided once and inherited by everybody.

```
    def fetch(self, start: date, end: date) → list[Row]:
        return self._db.query(self.QUERY, start=start, end=end)    # fixed step

    @abstractmethod
    def group(self, rows: list[Row]) → dict[str, list[Row]]:
        """Subclasses must implement. There is no sensible default."""

    @abstractmethod
    def render(self, grouped: dict[str, list[Row]]) → Document:
        """Subclasses must implement."""

    def decorate(self, document: Document) → None:
        """Hook. Default: do nothing."""
```

`@abstractmethod` from Python's `abc` module means the class cannot be instantiated unless every abstract method is implemented. Try it and you get a real error at construction time:

```
TypeError: Can't instantiate abstract class SalesReport with abstract method render
```

That is a much better failure than a report that silently produces an empty file, and it is a good reason to use `ABC` rather than raising `NotImplementedError` by hand.

A subclass is now genuinely small:

```
class SalesReport(Report):
    QUERY = "SELECT region, amount, sold_at FROM sales WHERE sold_at BETWEEN %s AND %s"

    def group(self, rows):
        return groupby_key(rows, key=lambda row: row.region)

    def render(self, grouped):
        return CsvDocument(grouped)
```

Two methods and a query string. That is the 20 percent that differs, and nothing else was copied.

Where you have already used it

- `unittest.TestCase`. The framework calls `setUp`, then your `test_` method, then `tearDown`, then `tearDownClass` — in that order, always, and you never call them yourself. Pytest fixtures are the same idea with composition instead of inheritance.
- **Django's class-based views.** `View.dispatch` decides GET or POST; `ListView.get` calls `get_queryset`, then `get_context_data`, then `render_to_response`. You override the small ones. This is the single most-used template method in the Python world.
- `HttpServlet.service()` in Java, which inspects the method and calls your `doGet` or `doPost`.
- **Spring's `JdbcTemplate`**. It opens the connection, runs the statement, handles the exception translation and closes everything; you supply the row mapper. The name says the pattern.
- **Airflow operators.** The scheduler calls `execute(context)`; you write only that.
- **React's class components** — `componentDidMount`, `render`, `componentWillUnmount` are called by React in a fixed lifecycle order.
- **Android's `Activity` lifecycle**, which is the same thing and the reason every Android tutorial starts with a diagram of it.

The iterator, in the two forms that matter

The explicit form, which is what other languages make you write:

```
class Countdown:
    def __init__(self, start: int) → None:
        self._start = start

    def __iter__(self) → "CountdownIterator":
        return CountdownIterator(self._start)           # a FRESH position each time

class CountdownIterator:
    def __init__(self, current: int) → None:
        self._current = current

    def __next__(self) → int:
        if self._current ≤ 0:
            raise StopIteration                         # the "no more names" signal
        self._current -= 1
        return self._current + 1

    def __iter__(self):
        return self
```

Two classes on purpose. **The collection and the position are different things.** Returning a fresh iterator from `__iter__` is what lets two `for` loops walk the same collection independently — and merging them into one class is exactly how you get the bug where a second loop finds nothing.

The Python form, which is the one you write:

```
def countdown(start: int):
    while start > 0:
        yield start
        start -= 1
```

A generator function is an iterator. `yield` produces one value and suspends; the next call resumes where it left off. Four lines instead of twenty, and the same interface, so `for`, `list()`, `sum()`, `any()` and `zip()` all work on it unchanged.

The production version: paging a database

```
def all_orders(db, batch: int = 1_000):
    """Yield every order without ever holding more than `batch` in memory."""
    last_id = 0
    while True:
        rows = db.query(
            "SELECT * FROM orders WHERE id > %s ORDER BY id LIMIT %s",
            last_id, batch,
        )
        if not rows:
            return
        yield from rows
        last_id = rows[-1].id          # keyset pagination, not OFFSET
```

Two production details in there. **Keyset pagination** — `WHERE id > last_id` — instead of `OFFSET n`, because `OFFSET 5000000` makes the database count and discard five million rows on every page, so paging a large table with `OFFSET` is quadratic. And the caller writes `for order in all_orders(db):` and never knows.

Real systems that hand you an iterator over something enormous: **database cursors** (`SELECT` with a server-side cursor in Postgres), **S3** `list_objects_v2` with its continuation token, **the Kafka consumer**, `os.scandir`, and Python's own `itertools`, whose entire library is functions that take an iterator and return another one.

What happens when it goes wrong

A generator is **one-shot**. Walk it twice and the second walk sees nothing:

```
rows = all_orders(db)
count = sum(1 for _ in rows)      # 4,912,003
total = sum(r.amount for r in rows) # 0 — the generator is exhausted
```

No error. A total of zero. This is the most common iterator bug in Python and it is completely silent. If you need two passes, either materialise deliberately with `list(...)` and accept the memory, or call the generator function twice.

And **mutating a collection while iterating it** raises, which is the good outcome:

```
RuntimeError: dictionary changed size during iteration
```

6 The numbers

The duplication being removed

Three reports, before:

```
sales_report.py      124 lines
inventory_report.py  118 lines
finance_report.py    131 lines
-----
total                373 lines,  of which ~96 lines are near-identical in all three
```

After:

```
report.py (base)     98 lines
sales_report.py      26 lines
inventory_report.py   24 lines
finance_report.py     31 lines
-----
total                179 lines
```

373 lines down to 179, a little over half. But the line count is the weakest part of the argument. The real numbers are these:

```
fixing a bug in the delivery step:
  before: 3 files edited, 3 chances to miss one, 3 sets of tests
  after:  1 file edited

adding a fourth report:
  before: copy 124 lines, change ~28
  after:  1 file, 2 methods, ~25 lines

the bug that actually happens:
  before: the fix is applied to 2 of the 3 files and nobody notices for a quarter
  after:  impossible
```

That last row is the argument. Duplication does not cost you lines; it costs you **inconsistency you cannot see**.

The iterator, in memory

Ten million order rows at about 200 bytes each:

```
load into a list:      10,000,000 × 200 B = 2.0 GB
stream in pages:      1,000 × 200 B = 200 KB  held at any moment
-----
ratio                                                           10,000x
```

On a container with a 512 MB memory limit, the first version does not merely run slowly:

```
MemoryError
```

or, more often in production, the container is killed by the operating system and you see nothing at all in the application log — just an exit code 137. That silence is why this bug takes so long to diagnose, and why “iterate, do not load” is worth saying before it is asked.

Why `OFFSET` paging is worse than it looks

```
page 1:      OFFSET 0          the database reads 1,000 rows
page 100:    OFFSET 99,000     it reads 100,000 rows and throws 99,000 away
page 5,000:  OFFSET 4,999,000 it reads 5,000,000 and throws 4,999,000 away

total rows read to page through 10,000,000 in 1,000-row pages:
1,000 + 2,000 + ... + 10,000,000 ≈ 5 × 1010 row reads
```

Fifty billion row reads to page through ten million rows. With keyset pagination — `WHERE id > last_id` on an indexed column — it is ten million reads plus ten thousand index seeks. **Five thousand times less work**, and the code is barely different.

The cost of the template call itself

Six method calls per report instead of one inlined function. At roughly 60 nanoseconds per Python method call, that is **under half a microsecond** on a report that takes four seconds. The performance cost of this pattern is zero and you should say so, because someone always asks.

7 The trade-offs

What template method costs you

It is inheritance, with everything that implies. A subclass is bound to its base for ever. You cannot change the skeleton at run time, you cannot mix two variations, and you cannot reuse a step outside the hierarchy. Strategy — passing objects in — has none of those limits, which is why “favour composition over inheritance” from [day 049](#) points away from this pattern by default.

The fragile base class problem. Adding a step to the template changes the behaviour of every subclass, including ones in other teams’ code that you cannot see. That is the classic hazard of inheritance and it is not theoretical: adding a `validate()` call to `generate` can break a subclass whose `group` was silently relying on unvalidated rows.

The order becomes invisible. A reader of `SalesReport` sees two methods and no clue that they are called between `fetch` and `deliver`. Somebody has to open the base class. Mitigate it with a docstring on the template that lists the steps, and by naming methods after their position in the sequence.

Python cannot stop you overriding the template. There is no `final`. A subclass that overrides `generate` breaks the whole thing and no tool complains. Convention and code review are the only defence, and a comment saying “do not override” is genuinely part of the design.

Too many hooks turn into a mess. A base class with eleven overridable steps is not a template; it is a configuration language with a bad syntax. If subclasses need to control that much, they should be composing objects, not filling in blanks.

Deep hierarchies. Two levels is fine. `Report → PdfReport → QuarterlyPdfReport` is where debugging starts meaning “which of the three classes implements this method?”, and Python’s method resolution order from [day 052](#) becomes something you have to reason about.

What iterator costs you

Laziness makes errors arrive late. The exception from a bad database row surfaces inside the caller’s `for` loop, thousands of lines away from where the query was written, and the traceback points at the loop. Wrap the generator’s body and re-raise with context.

One-shot generators. Silent, as shown above. If a function returns a generator, say so in its name or its type hint (`Iterator[Row]`), not `List[Row]`, because the caller’s behaviour depends on it.

No `len()`, no indexing, no going back. If the caller needs a count or a second pass, laziness is the wrong shape and you should return a list.

A long-running iteration sees a moving target. Paging through ten million rows takes minutes; rows are being inserted and deleted while you do it. Keyset pagination on an immutable id gives you a sane guarantee — you see everything that existed before you started, in id order — but `OFFSET` paging can show you the same row twice or skip one entirely when rows are deleted underneath. Say which guarantee you are giving.

Resources stay open. A generator holding a database cursor keeps it open until the loop finishes or the generator is closed. Abandon it halfway and you leak a connection. `with` blocks and `contextlib.closing` exist for this.

“I would not use this if...”

- **...only one thing varies, and it varies at run time.** That is Strategy — pass the object in.
- **...there are two subclasses and they share ten lines.** Extract a function. A base class for ten shared lines costs more than it saves.
- **...the steps are not genuinely in a fixed order.** If subclasses keep wanting to reorder or skip steps, the order is not a fact about the domain and the template is fighting reality. Build a pipeline of composable steps instead.
- **...the caller needs the collection, not a walk over it.** Return a list. Laziness that nobody wants is just a way to surprise people.
- **...the data is small.** Ten thousand rows is two megabytes. Load it and move on.

The honest concession

Template method does not remove the duplication so much as **move it into a place where it can only be written once**. The three reports still do three different things; what changed is that the identical parts now have exactly one home. And it buys that with inheritance, which is

the most rigid coupling in object-oriented design. If you expect the variation to grow in more than one direction — three formats *and* three delivery channels *and* three grouping rules — the hierarchy will explode, and composition is the answer instead. Saying that before the interviewer does is what shows you have used the pattern rather than read about it.

8 In the interview

How it gets asked

- The refactoring version, which is the common one: *“We have three report generators that are about 80 percent identical. How would you restructure them?”*
- The framework version: *“How does `unittest` know to call `setUp` before your test?”* or *“Explain the Hollywood principle.”*
- The memory version, for iterator: *“This job loads ten million rows and gets killed by the OOM killer. Fix it.”*
- The API version: *“Design an API that returns a very large result set.”* — the answer is a cursor or a continuation token, which is iterator over the network.
- The distinguishing question: *“When would you use template method rather than strategy?”*

What to say out loud, in the first ninety seconds

1. **Separate the fixed from the varying, out loud, before choosing a pattern.** “Let me name the steps first: fetch, group, render, deliver. Fetch and deliver are the same for all three. Group and render differ. So four steps, two fixed and two varying.”
2. **Name the pattern and what it protects.** “That is a template method: a base class owns the *order* in one non-overridable method, and the two varying steps are abstract.”
3. **Name the three kinds of step.** “Fixed steps live in the base. Abstract steps must be implemented. And I would add hooks — steps with a do-nothing default — for things like an optional watermark, so subclasses that do not care write nothing.”
4. **Give the real benefit, not the line count.** “The win is not fewer lines. It is that a bug in the delivery step is one edit instead of three, and the failure I actually care about is the fix applied to two files out of three, which nobody notices for a quarter.”
5. **Flag the cost honestly.** “The cost is inheritance. If variation grows in a second direction — three formats *and* three delivery channels — the hierarchy explodes and I would move to composition, passing in a renderer and a deliverer.”
6. **If iteration comes up, lead with memory.** “I would have `fetch` return an iterator rather than a list, so a ten-million-row report never holds more than a page in memory — two gigabytes against two hundred kilobytes.”

The follow-ups

“When would you use template method instead of strategy?” “Template method when there is one algorithm with a fixed sequence and I want to *guarantee* the sequence — that guarantee is the value, and inheritance is how it is enforced. Strategy when the varying part is a whole behaviour that should be swappable at run time, or when more than one thing varies independently. The practical test: if I can imagine wanting to change the varying step while the program is running, or combining two variations, I use Strategy. Template method fixes the choice when the subclass is written.”

“How do you stop a subclass overriding the template?” “In Java, `final`. In C#, non-virtual. In Python there is nothing that enforces it, so I write it in the docstring and rely on review — and I would say that is a genuine weakness of the pattern in Python. If it really mattered I could raise from `__init_subclass__` when a subclass defines `generate`, which is about six lines and turns the convention into an error.”

“What is a hook and why not just make it abstract?” “A hook is a step the base class implements as a no-op or a sensible default, so subclasses may override it and most do not. Abstract means every subclass *must* write something, and if the honest answer for four out of five is ‘nothing’, abstract forces four empty methods that add noise and can be forgotten. Hooks for optional steps, abstract for the ones with no sensible default.”

“The report job gets killed by the OOM killer. What do you do?” “Stop materialising. Make `fetch` a generator that pages the query and yields rows, so at any moment only one page is in memory — a thousand rows at 200 bytes is 200 kilobytes rather than two gigabytes. The caller’s `for` loop does not change at all, which is the point of the iterator pattern. I would page with a keyset — `WHERE id > last_seen ORDER BY id LIMIT 1000` — not `OFFSET`, because `OFFSET` makes the database read and discard everything before the page, so paging a ten-million-row table costs about fifty billion row reads instead of ten million.”

“What is the danger of returning a generator from a public function?” “It is one-shot and that is silent. If the caller iterates it once to count and again to sum, the second pass sees nothing and returns zero — no exception, just a wrong number. So I make it visible in the type hint, `Iterator[Row]` rather than `list[Row]`, and if a caller genuinely needs two passes I either return a list deliberately or give them a function they can call twice. The other danger is resource lifetime: a generator holding a database cursor keeps it open until the loop finishes, so abandoning it halfway leaks a connection unless it is closed.”

“Where have you already used both of these without noticing?” “Template method: every `unittest.TestCase` — the framework calls `setUp`, my test, then `tearDown`, and I never call them. Django’s class-based views, Spring’s `JdbcTemplate`, servlet `doGet`, the Android activity lifecycle. Iterator: every `for` loop in Python; `itertools`; a Postgres server-side cursor; S3’s `list_objects_v2` continuation token; a Kafka consumer. Both patterns are mostly invisible because the language and the frameworks already did them.”

A model answer

Asked: *three report types share 80 percent of their logic. Structure it.*

“Before picking anything, let me split the fixed from the varying, because that division is the design and everything else follows from it.

All three do the same four things in the same order: fetch the rows for a date range, group them, render them into a document, and deliver it by email. Fetching and delivering are identical across all three — same connection handling, same retry, same recipients logic. Grouping differs: by region, by depot, by month. Rendering differs: CSV, XLSX, PDF. So it is four steps, two fixed and two varying, and the *order* never varies at all.

That last part is what makes it a template method rather than anything else. The order is a fact about what a report is, and I want it written down exactly once, in a method that subclasses do not override. `generate` calls `fetch`, `group`, `render`, `deliver`. The two varying steps are abstract, so a subclass cannot forget them — with Python’s `abc` module, forgetting one is a `TypeError` at construction rather than an empty file in someone’s inbox.

I would also add one or two hooks: steps the base implements as doing nothing, which subclasses may override. A watermark, say, or a per-report footer. The distinction matters: abstract for steps with no sensible default, hooks for optional ones, because making an optional step abstract forces four empty methods and someone will eventually put something wrong in one.

On the numbers, the three files are about 373 lines today and roughly 96 of those are near-identical in all three. Afterwards it is a base of about 100 lines and three subclasses of about 25 each. But the line count is the weakest part of the argument. The real one is that fixing a bug in the delivery step becomes one edit instead of three — and the failure I actually care about is the fix being applied to two files out of three, which nobody notices for a quarter. Duplication does not cost lines; it costs inconsistency you cannot see.

There is a second thing I would change while I am in there. If `fetch` returns a list, a ten-million-row report holds about two gigabytes in memory and gets killed by the OOM killer — and in a container you usually see nothing in the application log, just exit code 137. I would make `fetch` return an iterator that pages the query and yields rows, so only a thousand rows exist at once, about two hundred kilobytes. The rest of the template does not change at all, because a `for` loop does not care where its items come from. I would page with a keyset — `WHERE id > last_seen ORDER BY id LIMIT 1000` — rather than `OFFSET`, because `OFFSET` makes the database read and discard everything before the page, and paging ten million rows that way is about fifty billion row reads against ten million.

Two costs I would name up front. The first is that this is inheritance, so the subclasses are bound to the base for ever, adding a step to the template changes behaviour for everybody, and Python cannot actually stop someone overriding `generate` — that is convention plus review. The second is the direction of growth. This works because exactly one thing varies. If tomorrow we need three formats *and* three delivery channels independ-

ently, the hierarchy multiplies out and I would move to composition instead — pass in a renderer object and a deliverer object, which is Strategy. I would rather take the simple template today and know exactly which change would make me abandon it.”

9 Recall card

- **Template method: the base class owns the ORDER in one method that must not be overridden, and leaves blanks.** Three kinds of step — **fixed** (base implements, do not override) · **abstract** (`@abstractmethod`, subclass must implement, forgetting is a `TypeError` at construction) · **hook** (base implements as a no-op, subclass may override). “Don’t call us, we’ll call you.”
- **The argument is not lines, it is edit count.** 373 → 179 lines is weak; *a bug in the shared step is 1 edit instead of 3*, and the real failure is **the fix applied to 2 of 3 files, unnoticed for a quarter**. Adding a fourth report = one file, two methods. The six extra method calls cost **under half a microsecond** — never a performance argument.
- **Template method vs Strategy: inheritance vs composition, fixed at subclass-writing time vs swappable at run time.** Use Strategy the moment **two things vary independently** (three formats × three delivery channels) or the choice must change while running. Python has no `final`, so the “do not override” is convention — a real weakness worth admitting.
- **Iterator separates walking from the collection, so** `for row in source:` **never changes when the source does.** `__iter__` must return a **fresh** position each time. A **generator is one-shot** — iterating twice silently gives zero the second time — so type-hint it `Iterator[Row]`, not `list[Row]`, and remember it can hold a cursor open.
- **The memory number: 10M rows × 200 B = 2 GB loaded, vs 1,000 × 200 B = 200 KB streamed — 10,000×**, and the failure looks like **exit code 137 with nothing in the log**. Page with a **keyset** (`WHERE id > last_seen`), never `OFFSET`: paging 10M rows costs $\sim 5 \times 10^{10}$ row reads with `OFFSET` against 10^7 with a keyset. Already in your hands: `unittest`, Django CBVs, `JdbcTemplate`, servlet `doGet` · every `for` loop, `itertools`, S3 continuation tokens, Kafka consumers.

PRACTICE

Practice — A queue from two stacks, and a stack from queues

DSA topic: A queue from two stacks, and a stack from queues **System design topic:** Template method and iterator

Code these, in this order

One rule for the whole set: **never say “ $O(1)$ ” without saying which kind.** Worst case for one call, or amortised across a sequence. If you cannot say which you mean, you do not yet know the answer.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Implement Stack using Queues	LeetCode 225 (Easy)	Choosing which operation pays, and noticing there is no amortised saving here.
2	Implement Queue using Stacks	LeetCode 232 (Easy)	The pour-only-when-empty rule, and the amortised argument on demand.
3	Design Front Middle Back Queue	LeetCode 1670 (Medium)	Two dequeues kept balanced — the same “two containers, one invariant” habit.
4	Max Stack	LeetCode 716 (Hard)	Two structures cooperating under one interface, and the honest cost of the expensive operation.

On problem 1, do both versions

Write costly-push-with-one-queue, then cheap-push-with-two-queues. Say which you would submit and why, and then say what the total cost of n pushes is in each. Neither answer is “amortised $O(1)$ ”, and being able to say *why not* is the point of the exercise.

On problem 2, write the test that finds the bug

Do not test by enqueueing everything and then dequeuing everything — that sequence passes even with the pour rule broken. Write the interleaved test: enqueue 1, enqueue 2, dequeue, enqueue 3, dequeue. Then write the random-sequence check against a real deque and run it two thousand times.

On problem 3, name the invariant before coding

Two deques, front half and back half. Say out loud what must be true after every operation — the sizes differ by at most one — and where you have to rebalance. Then write it.

On problem 4, be honest about the cost

There is a version with $O(1)$ push and $O(n)$ `pop_max`, and a much harder one with a doubly linked list and a sorted structure. Write the simple one, state its costs plainly, and say what you would need to make `pop_max` logarithmic.

The amortised drill

Say each answer out loud, five times:

1. What is the cost of a single `dequeue` in the worst case?
2. What is the cost amortised, and what does “amortised” mean in one sentence?
3. Count the operations one element experiences across its entire life. Say the number.
4. What single fact makes that count a bound rather than a hope?
5. What is the difference between amortised and average-case?
6. Give one situation where amortised $O(1)$ is not good enough, and say what you would use instead.

The pour-rule drill

1. Write the sequence that breaks a queue whose `_pour` has no emptiness guard. Say what it returns and what it should return.
2. Say why no exception is raised.
3. Explain, in terms of the pile on Basava’s counter, what has physically gone wrong.
4. Write the invariant sentence: what does `outbox` hold, what does `inbox` hold, and how do you read the whole queue front to back?

The break-it drill

Trigger each and record the actual output or error:

1. Remove the `if not self._outbox` guard from `_pour`. Run the interleaved sequence.
2. Write `is_empty` as `not self._outbox` and loop `while not q.is_empty()`. Count what you lose.
3. Write `peek` without calling `_pour`. Run `enqueue(1); peek()` and quote the error.
4. In the one-queue stack, rotate `len(queue)` times instead of `len(queue) - 1`. Push 1 and 2, then pop. Say what structure you accidentally built.
5. In the two-queue stack, reallocate the spare queue instead of swapping. Say what it costs at ten thousand pops.
6. Pour on every dequeue unconditionally. Say both things that are now wrong — the correctness and the complexity.

The counting drill

1. For n enqueues followed by n dequeues, count every push and pop. State the total.
2. For n alternating enqueue/dequeue pairs, count the same. State the total.
3. For n pushes into the one-queue stack, count the rotations. State the total and the complexity.
4. Say in one sentence why the queue-from-stacks gets an amortised saving and the stack-from-queues does not.

5. At $n = 10,000$, quote both totals.

The recognition drill

For each, say whether the amortised argument applies, and give the one-way fact that makes it work if it does:

1. `list.append` in Python.
2. `dequeue` in the two-stack queue.
3. The `while` loop inside a monotonic stack.
4. `push` in the one-queue stack.
5. Rehashing a dictionary when the load factor is exceeded.
6. The pour in a queue that pours unconditionally.
7. Popping from a monotonic deque.

Two of the seven have no amortised saving. Name them and say what they have in common.

The template-method refactor drill

You have three report classes of roughly 120 lines each, about 96 lines of which are near-identical.

1. List the steps and mark each one fixed or varying.
2. Write the template method. Say which method must never be overridden and how you would enforce it in Java, in C#, and in Python.
3. Write the two abstract steps with `@abstractmethod`. Then instantiate a subclass that forgets one and quote the exact error.
4. Add one hook. Say what makes it a hook rather than an abstract step.
5. Add a fourth report. Count the lines.
6. Now suppose delivery must also vary — email, S3, SFTP. Draw what happens to the hierarchy, count the classes, and say what you would switch to.

The Hollywood drill

For each, name what the framework calls and what you write:

1. `unittest.TestCase`
2. Django's `ListView`
3. A servlet's `doGet`
4. Spring's `JdbcTemplate`
5. An Airflow operator
6. An Android `Activity`

Then say the one sentence that describes what all six have in common, and what it means for who owns the control flow.

The iterator drill

1. Write `Countdown` as two classes — the collection and the position. Loop over one instance twice and confirm both loops work.
2. Merge them into one class so `__iter__` returns `self`. Loop twice. Say what the second loop produces and why.
3. Rewrite the whole thing as a four-line generator.
4. Write a generator that pages a table with keyset pagination.
5. Iterate it once to count and again to sum. Write down the second number and explain it.
6. Add a type hint that would have warned the caller.

The memory drill

1. Compute the memory for 10 million rows at 200 bytes, loaded into a list.
2. Compute it for pages of 1,000. State the ratio.
3. Say what the failure looks like in a 512 MB container, including what appears in the application log.
4. Compute the total row reads to page through 10 million rows with `OFFSET` at 1,000 per page.
5. Compute it with keyset pagination.
6. State the ratio and say why the code change is almost nothing.

The which-pattern drill

For each, say template method, strategy, iterator, or none:

1. Three reports differing only in grouping and rendering.
2. A pricing rule that differs by country and is chosen at run time.
3. Walking a directory tree without loading it all.
4. `setUp` running before every test.
5. Returning a very large query result from an API.
6. Three sorting orders the caller picks between.
7. A fixed six-step onboarding flow where step four differs by customer type.

Two of the seven are strategy, and the deciding question is the same for both. Say it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Implement a queue using only stacks. What is the amortised cost?* The reversal trick, the two roles, the pour-only-when-empty rule stated as a rule with the failing sequence, both complexities given before being asked, the four-touches count with the never-goes-backwards fact, and the amortised-versus-average distinction.

2. *Three report types share 80 percent of their logic. Structure it.* The steps split into fixed and varying first, the template that owns the order, abstract steps versus hooks, the edit-count argument rather than the line-count argument, the iterator change for memory with both numbers, and the named condition under which you would abandon inheritance for composition.
3. *Pop can move n elements. How is that $O(1)$?* The count over an element's whole life, the one-way fact, the concrete thousand-element example, and the honest caveat about p99 latency.
-

Before you move on

- [] I can state the reversal trick in one sentence.
- [] I can say the pour rule and the exact sequence that breaks without it.
- [] I wrote the interleaved test and the random-sequence check against a real queue.
- [] I can give the four-touches count without hesitating.
- [] I can define amortised without using the word “average”.
- [] I can name a situation where amortised $O(1)$ is not good enough.
- [] I wrote both stack-from-queue versions and can say the total cost of n pushes.
- [] I can say why the amortised saving does not transfer to the stack-from-queues.
- [] I made `is_empty` check one stack and saw the queue lose its contents.
- [] I split the report steps into fixed and varying before naming any pattern.
- [] I quoted the `TypeError` from forgetting an abstract method.
- [] I can say the difference between a hook and an abstract step.
- [] I can give the edit-count argument, not just the line-count one.
- [] I wrote a one-shot generator bug and saw the second pass return zero.
- [] I can quote 2 GB against 200 KB, and 5×10^{10} against 10^7 row reads.
- [] I can name the condition that would make me abandon the template for composition.
- [] I answered all three questions above out loud.

LRU cache: the structure interviewers love

DATA STRUCTURES AND ALGORITHMS

LRU cache: the structure interviewers love

You can build an LRU cache with $O(1)$ get and put and explain both data structures.

SYSTEM DESIGN

Design patterns revision and interview questions

You can pick and defend a pattern for an unseen problem, or argue that none is needed.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Design an LRU cache with $O(1)$ get and put.*
- *Which pattern fits here, and what would you lose by using it?*

love

1 What this is, and why they ask it

An **LRU cache** is a fixed-size store of key-value pairs. When it is full and something new arrives, it throws out the **least recently used** entry — the one nobody has touched for the longest. LRU is the three-letter name for that eviction rule.

Both operations must be $O(1)$. `get(key)` returns the value and marks that key as just used. `put(key, value)` stores it, marks it as just used, and evicts the oldest if the store is now over capacity.

They ask it because it cannot be solved with one data structure, and the whole interview is watching you discover that. A hash map gives you $O(1)$ lookup and no notion of order. A list gives you order and $O(n)$ removal. The answer is **both at once**: a hash map from key to node, and a doubly linked list holding the nodes in recency order. It is asked constantly — it is LeetCode 146, it appears in Amazon, Google, Microsoft, Uber and Atlassian loops, and it is the most common “design a data structure” question there is. It also stops being a puzzle the moment you have to build a real cache, because this is what Redis, your CPU and your operating system all do.

2 The story

Sudha’s kitchen has one shelf that is actually within reach, and it holds twelve jars.

It is above the stove and about as wide as her arms. Everything else — the rest of the spices, the things bought in bulk, the box of dried things she uses twice a year — lives in the loft over the door, and getting up there means the stool, and the stool means moving the water drum.

So the twelve jars on the shelf are the twelve she is actually cooking with, and she has a way of keeping it that way that she has never once thought about as a system.

When she takes a jar down and uses it, she does not put it back where it was. She puts it back at the left end, next to the stove, where her hand goes first. Everything else shifts along a little to make room. So the left end of the shelf is always whatever she used last, and the right end is whatever she has not touched in the longest time.

That right end is where the decision gets made. When something new arrives — the packet of star anise her sister brought from Kerala — the shelf is already full, so something has to go up to the loft. She does not think about which. She takes whatever is sitting at the far right, because by definition that is the one she has gone longest without needing. It goes up, star anise comes down to the left end, and the shelf is twelve again.

Two small things make this fast, and she would not be able to name either of them.

The first is that she never searches the shelf. She knows where the turmeric is the way she knows where the light switch is — she reaches, and it is there. If she had to read every label from the left each time, the whole arrangement would be pointless, because finding would cost more than the walking she saved.

The second is what happens when she pulls a jar out of the middle. The jars stand shoulder to shoulder, so lifting one out leaves a gap, and she closes it in the same movement — one hand nudges the jar on the left, the other the jar on the right, and they meet. She does not touch the other nine. Whether the shelf holds twelve jars or forty, closing the gap is the same two nudges.

Her daughter reorganised it once, alphabetically, and it lasted four days.

3 The idea in plain English

The shelf is a **doubly linked list**. Knowing where the turmeric is without looking is a **hash map**. And the reason the whole thing is $O(1)$ is Sudha's two nudges.

Why one structure is not enough

A hash map alone. A dictionary gives you `get` and `put` in $O(1)$, which sounds like the whole problem solved — until you have to evict. Which key is the least recently used? A dictionary has no answer. You would have to store a timestamp with each value and scan every entry to find the smallest one, which is $O(n)$ per eviction.

A list alone. Keep entries in a list, most recent at the front. Now eviction is trivial — remove the last one. But `get(key)` has to scan the list to find the key, which is $O(n)$, and moving an entry from the middle to the front means shifting everything after it, which is another $O(n)$, for exactly the reason `list.pop(0)` was slow on [day 073](#).

Both. The hash map answers “where is it?” in $O(1)$. The list answers “which is oldest?” in $O(1)$. Neither can do the other's job, and neither can be dropped. Saying that sentence out loud early is how you show the interviewer you have understood the problem rather than remembered the answer.

Why the list must be doubly linked

Sudha's two nudges. To remove a jar from the middle of the row, you need to join its left neighbour to its right neighbour. That means **from the jar itself you must be able to reach both neighbours**.

A **singly linked list** — where each node knows only the *next* one — cannot do that. Given a node, you cannot find the one before it without walking from the start, which is $O(n)$. A **doubly linked list** gives each node a `prev` and a `next`, so unlinking is:

```
node.prev.next = node.next
node.next.prev = node.prev
```

Two assignments. Constant time, whatever the size of the list. That is the second reason the pattern works, and it is the answer to the follow-up “why not a singly linked list?”

You will meet linked lists properly from [day 078](#) onwards. Everything you need today is in those two lines.

The two ends

Pick an orientation and stick to it, because mixing them up mid-solution is how this problem gets lost:

```
head ↔ most recently used ... least recently used ↔ tail
```

- `get(key)` — find the node through the map, **move it to the head**, return its value.
- `put(key, value)` — if the key exists, update the value and move it to the head. Otherwise create a node, add it at the head, put it in the map, and **if the size now exceeds capacity, remove the node before the tail** and delete its key from the map.

The sentinel nodes that remove every special case

The awkward part of linked-list code is the ends: inserting into an empty list, removing the only node, removing the first node. Each is a separate `if`, and each is a chance to get it wrong.

The fix is two permanent nodes that hold no data: a `head` and a `tail` that always exist. Real nodes live strictly between them.

```
head ↔ A ↔ B ↔ C ↔ tail
(fake)  most          least  (fake)
```

Now the list is never empty from the code’s point of view, `node.prev` is never `None`, and insert and remove are the same three lines every time. These are **sentinel nodes**, the same idea as the sentinel bar on [day 072](#), and the dummy head you will use for the whole of the linked-list phase from [day 080](#).

Write the sentinels first. It turns a fiddly problem into a mechanical one.

The one line people forget

`get` is a write. Reading an entry changes the cache, because it moves that entry to the front. A `get` that does not move the node is a cache that evicts things you have been using constantly, and it passes every test that only checks returned values. Say it out loud while you write it.

What you would actually use

Python's `OrderedDict` is a dictionary with exactly this structure inside it, and it exposes the move:

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int) → None:
        self._store: OrderedDict[int, int] = OrderedDict()
        self._capacity = capacity

    def get(self, key: int) → int:
        if key not in self._store:
            return -1
        self._store.move_to_end(key)           # mark as most recent
        return self._store[key]

    def put(self, key: int, value: int) → None:
        if key in self._store:
            self._store.move_to_end(key)
        self._store[key] = value
        if len(self._store) > self._capacity:
            self._store.popitem(last=False)    # drop the least recent
```

Twelve lines, all $O(1)$. Since Python 3.7 a plain `dict` also keeps insertion order, so a `dict` plus `del` and re-insert works too. And `functools.lru_cache` is this, applied to function results.

Build the linked list version in the interview, then say this exists. The interviewer is testing whether you know what is inside `OrderedDict`, not whether you can import it.

4 The picture

Capacity 3. `put(A,1)`, `put(B,2)`, `put(C,3)`, then `get(A)`, then `put(D,4)`.

```

after put A, B, C           the map points into the list, so no searching

map:  A → [A]   B → [B]   C → [C]

head ↔ C ↔ B ↔ A ↔ tail
(fake) MRU                LRU (fake)

get(A) → find A through the map in O(1), unlink it, put it at the head

unlink A:      B.next = tail ; tail.prev = B      (Sudha's two nudges)
insert at head: head ↔ A ↔ C ...

head ↔ A ↔ C ↔ B ↔ tail
      MRU                LRU
returns 1

put(D,4) → size would be 4 > 3, so evict the node before tail, which is B

remove B from the list AND from the map      ← both, always
insert D at the head

map:  A → [A]   C → [C]   D → [D]           B is gone from both

head ↔ D ↔ A ↔ C ↔ tail
      MRU                LRU

```

What to notice: **get(A)** **changed the list**. After it, B is the least recently used rather than A, which is why **put(D,4)** evicts B. If **get** had not moved A, the cache would have thrown away the entry you had just asked for.

And the removal itself, drawn at the pointer level, because this is the part that gets written wrong:

```

before:      ... ↔ P ↔ N ↔ Q ↔ ...
              (the node to remove)

P.next = N.next → P.next = Q
N.next.prev = N.prev → Q.prev = P

after:      ... ↔ P ↔ Q ↔ ...

N still points at P and Q, and nobody points at N. It is gone.
Two assignments, no loop, no dependence on the list's length.

```

5 The code, built step by step

Step 1 — the node

```
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: int = 0, value: int = 0) → None:
        self.key = key
        self.value = value
        self.prev: "Node | None" = None
        self.next: "Node | None" = None
```

The node stores the **key** as well as the value, and that is not redundant. When you evict the node before the tail, you must also delete it from the map — and to do that you need its key. Leave the key out and eviction becomes impossible without a scan. This is the single most common design slip in this problem.

`__slots__` is a small optimisation: it stops each node carrying a dictionary of attributes, cutting memory per node from about 200 bytes to about 90. Mention it, do not dwell on it.

Step 2 — the sentinels

```
self._head = Node()          # fake, always the front
self._tail = Node()          # fake, always the back
self._head.next = self._tail
self._tail.prev = self._head
```

Four lines that delete every edge case. The list is now never empty, `node.prev` is never `None` for a real node, and the first insertion is the same code as the hundredth.

Step 3 — unlink, in two assignments

```
def _unlink(self, node: Node) → None:
    node.prev.next = node.next
    node.next.prev = node.prev
```

Sudha's two nudges. No loop, no length check, no `if`. This works for any real node because the sentinels guarantee both neighbours exist.

Step 4 — insert at the head

```
def _push_front(self, node: Node) → None:
    node.prev = self._head
    node.next = self._head.next
    self._head.next.prev = node          # the old first node points back at us
    self._head.next = node
```

Four assignments, and the **order matters**. Set the new node's two pointers first, then fix the neighbours. Doing `self._head.next = node` too early loses your only reference to the old first node.

Step 5 — get, which is also a write

```
def get(self, key: int) → int:
    node = self._map.get(key)
    if node is None:
        return -1
    self._unlink(node)
    self._push_front(node)          # a read makes it the most recent
    return node.value
```

Three lines of work and the middle two are the ones people forget.

Step 6 — put, with the two cases kept apart

```
def put(self, key: int, value: int) → None:
    existing = self._map.get(key)
    if existing is not None:
        existing.value = value      # update in place
        self._unlink(existing)
        self._push_front(existing)
    return
```

Handle “the key is already here” first and return. Trying to merge the two cases into one branch is where the bug lives — an existing key must not be inserted twice, and it must not count against the capacity.

```
node = Node(key, value)
self._map[key] = node
self._push_front(node)

if len(self._map) > self._capacity:
    oldest = self._tail.prev      # the node just before the fake tail
    self._unlink(oldest)
    del self._map[oldest.key]     # BOTH structures, always
```

`del self._map[oldest.key]` is the line that needs the key stored on the node. Removing from the list but not the map leaves a dangling entry: a later `get` finds a node that is no longer in the list, moves it to the front, and now the list and the map disagree for ever.

The complete solution

```
class Node:
    """One entry, doubly linked. It stores the KEY as well as the value,
    because eviction needs the key to delete the map entry."""

    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key: int = 0, value: int = 0) → None:
        self.key = key
        self.value = value
        self.prev: "Node | None" = None
        self.next: "Node | None" = None

class LRUCache:
    """Fixed-capacity cache that evicts the least recently used entry.

    Two structures, and neither can do the other's job:
    map : key → node, so lookup is O(1) with no searching
    list : doubly linked, most recent at the head, so eviction is O(1)

    Sentinel head and tail nodes hold no data and always exist, which removes
    every empty-list and first-node special case.

    IMPORTANT: get() is a write. Reading an entry moves it to the head.
    """

    def __init__(self, capacity: int) → None:
        if capacity < 0:
            raise ValueError("capacity cannot be negative")
        self._capacity = capacity
        self._map: dict[int, Node] = {}
        self._head = Node() # fake front sentinel
        self._tail = Node() # fake back sentinel
        self._head.next = self._tail
        self._tail.prev = self._head

    def get(self, key: int) → int:
        node = self._map.get(key)
        if node is None:
            return -1
        self._unlink(node)
        self._push_front(node) # a read is a use
        return node.value

    def put(self, key: int, value: int) → None:
        if self._capacity == 0:
            return # a cache that stores nothing

        existing = self._map.get(key)
        if existing is not None:
            existing.value = value
            self._unlink(existing)
            self._push_front(existing)
            return

        node = Node(key, value)
        self._map[key] = node
        self._push_front(node)

        if len(self._map) > self._capacity:
            oldest = self._tail.prev # just before the fake tail
            self._unlink(oldest)
            del self._map[oldest.key] # remove from BOTH structures
```

```

def _unlink(self, node: Node) → None:
    """Join the two neighbours to each other. Two assignments, O(1)."""
    node.prev.next = node.next
    node.next.prev = node.prev

def _push_front(self, node: Node) → None:
    """Put the node immediately after the head sentinel. Order matters."""
    node.prev = self._head
    node.next = self._head.next
    self._head.next.prev = node
    self._head.next = node

def keys_most_recent_first(self) → list[int]:
    """For testing and for drawing the list on a whiteboard."""
    keys, node = [], self._head.next
    while node is not self._tail:
        keys.append(node.key)
        node = node.next
    return keys

from collections import OrderedDict

class LRUCacheOrderedDict:
    """What you would write in production. OrderedDict is exactly a dict plus
    a doubly linked list, and move_to_end / popitem(last=False) are the moves."""

    def __init__(self, capacity: int) → None:
        self._store: OrderedDict[int, int] = OrderedDict()
        self._capacity = capacity

    def get(self, key: int) → int:
        if key not in self._store:
            return -1
        self._store.move_to_end(key)
        return self._store[key]

    def put(self, key: int, value: int) → None:
        if self._capacity == 0:
            return
        if key in self._store:
            self._store.move_to_end(key)
        self._store[key] = value
        if len(self._store) > self._capacity:
            self._store.popitem(last=False)

if __name__ == "__main__":
    cache = LRUCache(2)
    cache.put(1, 1)
    cache.put(2, 2)
    print(cache.get(1))           # 1
    cache.put(3, 3)               # evicts key 2 (least recently used)
    print(cache.get(2))           # -1
    cache.put(4, 4)               # evicts key 1
    print(cache.get(1))           # -1
    print(cache.get(3), cache.get(4)) # 3 4

    order = LRUCache(3)
    for key in (1, 2, 3):
        order.put(key, key * 10)
    print(order.keys_most_recent_first()) # [3, 2, 1]
    order.get(1)
    print(order.keys_most_recent_first()) # [1, 3, 2] ← get moved it

```



```

order.put(4, 40)
print(order.keys_most_recent_first()) # [4, 1, 3] ← 2 was evicted

print(LRUCache(0).get(1))            # -1, and put() does nothing

# both implementations must agree on any sequence of operations
import random
for _ in range(2000):
    capacity = random.randint(1, 5)
    mine, reference = LRUCache(capacity), LRUCacheOrderedDict(capacity)
    for _ in range(40):
        key = random.randint(1, 8)
        if random.random() < 0.5:
            assert mine.get(key) == reference.get(key)
        else:
            value = random.randint(0, 99)
            mine.put(key, value)
            reference.put(key, value)
print("Linked-list and OrderedDict versions agreed on 2000 random sequences")

```

The random cross-check is the test that catches the two silent bugs: `get` not moving the node, and eviction removing from only one structure. Both produce correct answers for a while and wrong ones later, so a fixed script of eight calls will not find them.

6 What it costs

Time

```

get(key)
dict lookup          0(1)
unlink               0(1)   two assignments
push to front       0(1)   four assignments
-----
total                0(1)

put(key, value)
dict lookup          0(1)
create / update node 0(1)
push to front       0(1)
evict if over capacity 0(1)   the node before the tail, then one dict delete
-----
total                0(1)

```

Nothing loops. Nothing searches. Nothing depends on the capacity. Count the assignments out loud — about six for a `get` and about ten for a `put` — because “O(1)” is more convincing when you can say what the constant is made of.

Both are **worst case** O(1), not amortised, apart from the dictionary itself. Dictionary lookup is O(1) average and O(n) in a pathological collision case, which is [day 061](#)’s material and worth one sentence if the interviewer is being thorough.

Against the alternatives

Capacity `c`, and `n` operations:

```
dict + timestamp, scan to evict:  O(1) get, O(c) evict
list, most recent at the front:  O(c) get (search), O(c) move
dict + doubly linked list:       O(1) get, O(1) put

c = 10,000 and 1,000,000 operations:
scan-to-evict:  up to 10^10 comparisons
this:          about 10^7 pointer writes
```

Three orders of magnitude, and the gap grows with the capacity — which matters because real caches are large. A Redis instance holding ten million keys cannot afford anything that scans.

Space

```
per entry:
Node with __slots__      about 90 B   (key, value, prev, next)
Node without __slots__  about 200 B  (plus an instance dictionary)
dict entry               about 100 B  (key, hash, pointer, and slack)
-----
total with __slots__    about 190 B per entry
```

For a cache of 100,000 entries:

```
100,000 × 190 B ≈ 19 MB   of overhead, before the values themselves
```

Say that number if asked about memory. It is why real caches with millions of entries are written in C and store values as bytes — and why `functools.lru_cache` on a hot function with a large `maxsize` can be a surprising amount of memory.

Space is **O(capacity)**, not O(n). The whole point of a cache is that it is bounded.

The hit-rate arithmetic, which is the systems half of the question

A cache is only worth having if it hits. With a 100 μs database read and a 1 μs cache read:

```
hit rate 90%:  0.90 × 1 μs + 0.10 × 100 μs = 10.9 μs average
hit rate 50%:  0.50 × 1 μs + 0.50 × 100 μs = 50.5 μs average
no cache:      100.0 μs
```

Ninety percent hits gives a **9× speed-up**; fifty percent gives 2×. Doubling the capacity usually moves the hit rate by a few points, which is why capacity planning is measurement rather than guesswork. If the interviewer turns the question from “implement it” to “would you use it”, this is the arithmetic they want.

7 The traps

Trap 1 — `get` that does not move the node

```
def get(self, key: int) → int:
    node = self._map.get(key)
    return node.value if node else -1          # no reordering
```

Every returned value is correct. Every eviction is wrong. The cache now evicts by *insertion* order rather than *use* order, so the entry you read a thousand times gets thrown out while an entry inserted once and never touched survives. On the standard LeetCode sequence it fails at the fourth call; in production it shows up as a mysteriously poor hit rate.

A read is a use. Say it while you type it.

Trap 2 — evicting from the list but not the map

```
oldest = self._tail.prev
self._unlink(oldest)          # forgot: del self._map[oldest.key]
```

The map keeps growing for ever, so the cache is no longer bounded — that alone is a memory leak. And it is worse than a leak: a later `get` on the evicted key finds the node, pushes it back into the list, and returns a value for an entry that was supposed to be gone. The two structures now disagree, and every subsequent eviction is wrong.

Every change touches both structures. Make that a rule and check each method against it.

Trap 3 — no key on the node

```
class Node:
    __slots__ = ("value", "prev", "next")          # no key
```

Now `del self._map[oldest.key]` cannot be written. The usual patch is to scan the map for the value, which is $O(\text{capacity})$ and makes the whole solution pointless. Store the key in the node. It costs eight bytes and it is what makes eviction $O(1)$.

Trap 4 — treating an existing key as a new insert

```
def put(self, key: int, value: int) → None:
    node = Node(key, value)
    self._map[key] = node          # overwrites the map entry
    self._push_front(node)        # old node is still in the list
    if len(self._map) > self._capacity:
        ...
```

The old node is still linked into the list and nothing points to it from the map. The list now has more nodes than the map has keys, the size check is wrong, and eviction eventually unlinks the orphan and tries `del self._map[key]` for a key whose map entry now belongs to the *new* node — deleting a live entry.

```
KeyError: 3
```

is the friendly version of this bug. The silent version is worse. Handle “already present” as its own branch with an early `return`.

Trap 5 — using a singly linked list

Each node knows only `next`. Unlinking a node now requires the node *before* it, and finding that means walking from the head — $O(\text{capacity})$. The whole solution degrades to $O(n)$ per operation and the interviewer will ask why.

If you are forced to use a singly linked list, the trick is to store `key → previous node` in the map instead, which works and is horrible. Say the doubly linked list and move on.

Trap 6 — no sentinels, and the null checks that follow

Without a fake head and tail, `_unlink` needs `if node.prev is None` and `if node.next is None`, `_push_front` needs a separate empty-list branch, and eviction needs a “was that the only node?” check. Six extra branches, each one a chance to write:

```
AttributeError: 'NoneType' object has no attribute 'next'
```

It is the single most common runtime error in hand-written linked-list code. Two fake nodes remove all of it.

Trap 7 — capacity zero or one

`capacity = 0` should accept nothing. Without a guard, `put` inserts the node, finds `1 > 0`, and immediately evicts... the node it just inserted, which happens to work, but only by accident and only if `_tail.prev` is that node. Guard it explicitly.

`capacity = 1` is the case that catches ordering bugs: `put(1,1); put(2,2); get(1)` must return -1. Run it.

Trap 8 — assuming thread safety

Two threads calling `get` at the same time can interleave the pointer updates and corrupt the list — one node pointing into a region no longer linked, or a cycle. Nothing raises; the cache just starts returning wrong values or hangs in `keys_most_recent_first`. Real caches take a lock, or shard the cache into 16 independent caches keyed by `hash(key) % 16` so that most operations do not contend. Say this if asked “is this thread-safe?” — the answer is no, and the two fixes are worth naming.

8 In the interview

How it gets asked

- The direct version: *“Design an LRU cache with $O(1)$ get and put.”* LeetCode 146. Almost always stated with the $O(1)$ requirement, which is the hint that one structure is not enough.
- The follow-up that is really a second question: *“Now make it LFU — least frequently used.”* LeetCode 460, and it needs a third structure.
- The systems version: *“Your service reads the same user record thousands of times a second. What do you do?”* — the answer is this, plus a TTL, plus a hit-rate number.
- The concurrency version: *“Two threads use this cache at once. What breaks?”*

What to say out loud, in the first ninety seconds

1. **State the requirement that forces the design.** “Both operations have to be $O(1)$, including eviction. That immediately rules out any single structure, so let me say why.”
2. **Reject each single structure, with its cost.** “A hash map gives me $O(1)$ lookup but has no order, so finding the least recently used means scanning — $O(\text{capacity})$. A list gives me order but finding a key is a scan and moving an entry to the front shifts everything.”
3. **Name the combination.** “So I use both: a hash map from key to *node*, and a doubly linked list holding the nodes in recency order, most recent at the head.”
4. **Say why doubly.** “Doubly linked, because removing a node from the middle in $O(1)$ means joining its two neighbours, and for that I need to reach both from the node itself. A singly linked list would need a walk to find the predecessor.”
5. **Say the thing people forget, before writing it.** “`get` is a write — reading an entry moves it to the head. Otherwise I am evicting by insertion order, not by use.”
6. **Mention sentinels as a decision.** “I will use fake head and tail nodes that always exist, which removes every empty-list and first-node special case.”
7. **Say the eviction detail.** “The node stores its own key, because when I evict the node before the tail I have to delete it from the map too, and I need the key for that.”

Then write it. With that said first, the code is fifteen minutes of typing and the interviewer already knows you understand it.

The follow-ups

“Why does the node need to store the key?” “Because eviction starts from the list, not from the map. I find the node before the tail sentinel, unlink it, and then I have to remove its entry from the map — and the only thing I have at that moment is the node. Without the key on it, I would have to scan the map for a matching node, which is $O(\text{capacity})$ and destroys the whole design.”

“Why not a singly linked list?” “Removing a node in $O(1)$ needs to join the node before it to the node after it. With only `next` pointers, finding the node *before* a given node means walking from the head, which is $O(\text{capacity})$. There is a workaround — map each key to its

predecessor — but it is fiddly and buys nothing. The extra pointer is eight bytes per entry and it makes the operation two assignments.”

“Is `get` really a mutation?” “Yes, and it is the line most people leave out. If `get` does not move the entry to the head, the cache evicts in insertion order rather than use order, so the hot entry you read a thousand times gets thrown away while a cold one survives. Every returned value is still correct, which is why it passes naive tests — the damage only shows up in the eviction sequence and, in production, as a bad hit rate.”

“Make it LFU instead.” “Least frequently used needs a third structure, because now I have to find the smallest *count* in $O(1)$. The standard answer is a map from key to node, a map from frequency to a doubly linked list of the nodes with that frequency, and a running `min_frequency` integer. On access, the node moves from list `f` to list `f+1`; if list `f` becomes empty and `f` was the minimum, the minimum increases by exactly one — that is what keeps it $O(1)$. Eviction takes the tail of the `min_frequency` list. The honest caveat is that plain LFU never forgets: an entry that was hot yesterday holds a huge count and refuses to leave, so real implementations use a decay or a window.”

“Is this thread-safe?” “No. Two threads can interleave the pointer updates and leave the list with a node linked from one side only, or a cycle — and nothing raises, it just returns wrong values or hangs. The simple fix is one lock around both operations, which serialises everything and becomes the bottleneck. The usual production fix is sharding: sixteen independent caches keyed by `hash(key) % 16`, each with its own lock, so contention drops by roughly the number of shards. Java’s `ConcurrentHashMap` and Caffeine do essentially that.”

“What would you use in production?” “`OrderedDict` in Python, which is a dict plus a doubly linked list — `move_to_end` and `popitem(last=False)` are exactly these two operations — or `functools.lru_cache` for memoising a function. For a shared cache across processes, Redis with `maxmemory-policy allkeys-lru`, which approximates LRU by sampling a handful of keys rather than maintaining an exact order, because at ten million keys the exact bookkeeping costs more than the accuracy is worth. Naming that approximation is worth doing — it shows the difference between the interview answer and the production one.”

“How do you know the capacity is right?” “Measure the hit rate. With a 100 microsecond database read and a 1 microsecond cache read, ninety percent hits gives about 11 microseconds average — a ninefold improvement — while fifty percent gives about 50 microseconds, which is only twofold. So I would instrument hits and misses, plot hit rate against capacity, and stop where the curve flattens. Doubling memory for two points of hit rate is usually a bad trade.”

A model answer

Asked: *design an LRU cache with $O(1)$ get and put.*

“The requirement that decides the design is that eviction also has to be $O(1)$. That rules out any single data structure, so let me say why before I choose.

A hash map gives me $O(1)$ lookup, but a dictionary has no notion of order, so ‘which entry has been untouched longest’ means storing a timestamp with each value and scanning every entry — $O(\text{capacity})$ per eviction. A list in recency order gives me eviction for free, take the last one, but finding a key means scanning, and moving an entry from the middle to the front means shifting everything after it.

So I use both, and the trick is that the map points *into* the list. A dictionary from key to node, and a doubly linked list of those nodes with the most recently used at the head and the least recently used at the tail. The map answers ‘where is it’ in constant time; the list answers ‘which is oldest’ in constant time; neither can do the other’s job.

Doubly linked specifically, because removing a node from the middle in constant time means joining its left neighbour to its right neighbour, and to do that I have to reach both from the node itself. With only forward pointers I would have to walk from the head to find the predecessor, which is linear.

Two decisions I would make before writing a line. First, sentinel nodes: a fake head and a fake tail that always exist and hold no data, with the real nodes strictly between them. That removes every empty-list and first-node special case, and those special cases are where linked-list code goes wrong — you end up with `AttributeError: 'NoneType' object has no attribute 'next'`. Second, the node stores its own key as well as its value. That looks redundant until eviction: I find the node before the tail, unlink it, and then I have to delete it from the map, and the key is the only way to do that.

Now the operations. `get` looks the key up in the map, and — this is the line people leave out — **moves the node to the head**, because a read is a use. If I skip that, every value I return is still correct but the cache evicts by insertion order instead of use order, so the entry I read a thousand times gets thrown out while a cold one survives. It passes a naive test and produces a terrible hit rate in production.

`put` has two cases and I keep them apart. If the key already exists, update the value, move it to the head, and return — it must not be inserted twice and it must not count against the capacity. If it does not exist, create the node, add it to the map, push it to the head, and then, if the size now exceeds capacity, take the node just before the tail sentinel, unlink it, and delete its key from the map. Both structures, every time — removing from one and not the other is the bug that makes the map grow without bound and lets a later `get` resurrect an evicted entry.

Everything is $O(1)$ worst case: about six pointer writes for a `get` and ten for a `put`, no loops, and nothing that depends on capacity. Space is $O(\text{capacity})$, which is the point of a cache — roughly 190 bytes of overhead per entry in Python with `__slots__`, so a hundred thousand entries is about nineteen megabytes before the values themselves.

Two things I would say at the end. In production I would use `OrderedDict`, which is exactly this structure with `move_to_end` and `popitem(last=False)`, or Redis with an LRU eviction policy — and Redis actually *approximates* LRU by sampling a few keys, because at ten million keys exact bookkeeping costs more than the accuracy is worth. And this implementation is not thread-safe: two threads interleaving the pointer updates can leave the list corrupt with no exception raised. The fix is a lock, and the fix for the lock being a bottleneck is sharding into sixteen independent caches by key hash.”

9 Recall card

- **One structure cannot do it, and saying why is the first ninety seconds.** A hash map has **no order** (finding the LRU is an $O(\text{capacity})$ scan); a list has **no lookup** (finding a key is a scan, moving it shifts everything). So: **map from key $\rightarrow$ node**, plus a **doubly linked list of nodes, most recent at the head**.
- **Doubly linked, because unlinking is two assignments** — `node.prev.next = node.next` and `node.next.prev = node.prev` — and that needs both neighbours reachable *from the node*. A singly linked list must walk to find the predecessor: $O(\text{capacity})$.
- **Two things everyone forgets.** `get` is a **write** — it moves the node to the head, or you evict by insertion order and the hot entry gets thrown away while returning correct values. **The node stores its own key**, because eviction starts from the list and must then `del map[node.key]` — **every change touches both structures**.
- **Write the sentinels first.** A fake head and fake tail that always exist delete every empty-list and first-node branch, and with them `AttributeError: 'NoneType' object has no attribute 'next'`. Keep `put`'s two cases apart with an early `return` on the existing key. Test `capacity = 1` and `capacity = 0`.
- **All $O(1)$ worst case, ~ 6 writes per get and ~ 10 per put; space $O(\text{capacity})$** — about **190 B per entry** in Python, so 100,000 entries $\approx$ **19 MB** of overhead. In production: `OrderedDict (move_to_end, popitem(last=False))`, `functools.lru_cache`, or Redis `allkeys-lru`, which **approximates** LRU by sampling. **Not thread-safe** — one lock, or shard by `hash(key) % 16`. LFU needs a **third** structure plus a `min_frequency` counter.

1 What this is, and why they ask it

Over the last fourteen days you have met sixteen patterns: Singleton, Factory, Abstract Factory, Builder, Prototype, Adapter, Decorator, Facade, Proxy, Strategy, Observer, State, Command, Chain of Responsibility, Template Method and Iterator. Today is not a seventeenth. Today is the skill that makes the other sixteen worth anything: **looking at an unfamiliar problem and saying which one fits, which two it could be confused with, and what it costs.**

The interview question is almost never “explain the decorator pattern”. It is a mess of requirements and a pause. What you are being scored on is whether you can name the **axis of change** — the one thing that will keep changing about this code — before you name a pattern, and whether you can say “none of them, write the `if`” when that is the honest answer.

They ask it because pattern knowledge without judgement is worse than no pattern knowledge. A candidate who reaches for Abstract Factory when a dictionary would do has told the interviewer exactly how they will behave in the codebase. The strongest answer in this whole area is always the one that names the cost.

2 The story

When Ravi moved into his own place, his uncle gave him a metal box of tools. Twenty-three of them, several of which Ravi still cannot name.

The kitchen tap started dripping in the second week. Ravi opened the box and stood in front of it for ten minutes.

He knew how to fix a dripping tap. He had watched it done twice. But owning twenty-three tools had put an idea in his head — that somewhere in that box was the tap tool, the correct one, and that using anything else would be a mistake he would find out about in six months. So he stood there, picking things up and putting them back, and the tap kept dripping into the bucket.

In the end he went next door. Sudhakar has been fixing things in that building for thirty years and does not own twenty-three of anything. He came over with three things in his hand and had it done in four minutes.

What he said afterwards, while he was drying his hands on his shirt, is the part Ravi remembers. Most jobs, he said, are one of about six jobs. Something is loose. Something is worn out. Something is blocked. Something is in the wrong place. Something needs holding still while you work on it. Something needs cutting. Once you are sure which of the six you are looking at, the tool picks itself. The difficulty is never the tool. It is being sure which job it is.

Two months later Ravi hung a shelf. He needed four small holes, and by then he had bought a drill, and he was quite pleased about the drill.

He used it. The wall was thin and older than the building deserved, the drill went in much further than it should have, and the crack it made runs from behind the shelf to the corner of the window where anybody sitting on the sofa can see it.

A screwdriver and some patience would have done the job. He owned the screwdriver. He used the drill because he had it, and because using it felt like doing the job properly.

3 The idea in plain English

Sudhakar's six jobs are the point. **Recognising the job is the skill; the pattern is the easy part.** And Ravi's crack in the wall is the other half: a pattern applied where none was needed does visible damage, and the damage looks like competence while you are doing it.

The first question is never "which pattern"

It is: **what is going to change, and how often?**

Every pattern in this phase exists to make one particular kind of change cheap, and it makes every other kind of change slightly more expensive. So the sequence is always:

1. **Name the axis of change** in three or four words. "A new payment provider." "A new report format." "A new thing that must happen after checkout."
2. **Say how often it changes.** Twice a year, or twice a sprint? Once ever?
3. **Only then** name the pattern that makes *that* axis cheap.
4. **Say what it costs**, in files, indirection and readability.
5. **Say what would make you not do it.**

A candidate who does steps 1 and 2 out loud has already outperformed one who jumps to step 3 with the right answer.

The sixteen, as one-sentence triggers

Read these as "*if you hear this, think of that*". This table is the revision.

PATTERN	THE TRIGGER SENTENCE	DAY
Singleton	“There must be exactly one of these in the process.”	064
Factory	“The caller should not know which concrete class it gets.”	065
Abstract Factory	“These objects must come from the same family and never be mixed.”	065
Builder	“This object has fifteen optional fields and a validity rule.”	066
Prototype	“Making one from scratch is expensive; copying an existing one is not.”	067
Adapter	“This class does the right thing behind the wrong interface.”	068
Decorator	“Add behaviour to one object, at run time, stackably, without subclassing.”	069
Facade	“Five subsystems, and callers should see one simple door.”	070
Proxy	“Same interface, but control access — lazily, remotely, or with a check.”	070
Strategy	“This rule varies, and the caller picks which one.”	071
Observer	“When X happens, several independent things must react.”	072
State	“Behaviour depends on which stage this object is in, and it moves itself.”	073
Command	“The request itself must be storable — queued, undone, logged, replayed.”	074
Chain of responsibility	“Several possible handlers; the first that can, does; the rest stop.”	074
Template method	“One fixed sequence of steps, and two of the steps vary.”	075
Iterator	“Walk this collection without knowing how it stores things.”	075

The three families, and what each family is really about

- **Creational** — Singleton, Factory, Abstract Factory, Builder, Prototype. All about **how an object comes into existence**, and all of them exist because `new` and `__init__` are inflexible: they name a concrete class, they take positional arguments, and they run every time.
- **Structural** — Adapter, Decorator, Facade, Proxy. All about **how objects are wired together**. Three of the four are one object wrapping another; the difference between them is *why*.
- **Behavioural** — Strategy, Observer, State, Command, Chain of Responsibility, Template Method, Iterator. All about **who decides what, and when**. This is the family interviewers care about most, because it is where the design decisions live.

The wrapping four, told apart

Adapter, Decorator, Facade and Proxy all look like “class A holds class B and forwards calls”. They are told apart by **what happens to the interface** and **why the wrapper exists**:

	interface	purpose
Adapter	CHANGES it	make an incompatible thing usable
Decorator	keeps it	add behaviour, stackably, at run time
Proxy	keeps it	control access — lazy, remote, cached, permission-checked
Facade	NEW, simpler	hide several subsystems behind one door

The two-question test: **does the caller's interface change?** (Adapter and Facade yes, Decorator and Proxy no.) And **is the wrapper adding behaviour the caller asked for, or controlling access the caller did not ask about?** (Decorator adds; Proxy controls.)

The behavioural look-alikes, told apart

Strategy vs State — identical class diagrams. *The client chooses a strategy; the object replaces its own state.* States know their successors; strategies do not know each other exists. The tell in code is an assignment to the context's own field from inside an implementation.

Strategy vs Template Method — both vary a step. Strategy uses **composition** and can change at run time; Template Method uses **inheritance** and is fixed when the subclass is written. If two things vary independently, Template Method's hierarchy multiplies and Strategy does not.

Observer vs Chain of Responsibility — both hand a request to several objects. *In Observer, everyone gets it. In a chain, the first one that can handle it stops the walk.* If it must reach all of them, it is not a chain.

Command vs Strategy — both are “an object that does something”. A strategy is an **algorithm** plugged into a caller; a command is a **request with its receiver and arguments already bound**, so it can be stored, queued and undone. If you need `undo`, it is Command.

Factory vs Builder — a factory decides **which class**; a builder assembles **one complicated object** step by step. Fifteen optional fields is Builder; three subclasses to choose between is Factory.

And the answer that is often correct: none

An `if` with three branches, a dictionary of functions, a plain function argument. Say it when it is true. “There are two cases today and I have no evidence of a third, so I would write the `if` and extract a pattern when the second variation actually arrives” is a strong, senior-sounding answer, and it is the one Sudhakar would give.

4 The picture

The decision map. Start at the top with the axis of change, not with a pattern.

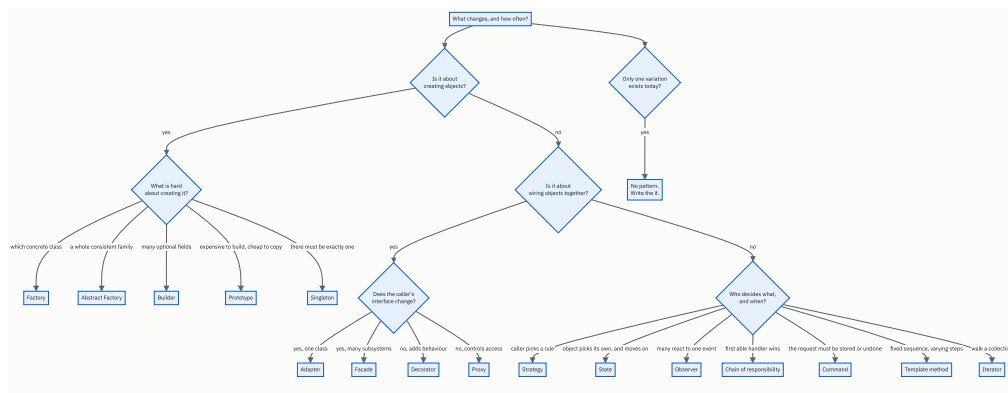


FIGURE 76.1

What to notice: the fastest route through this diagram is the one on the right, straight to “no pattern”. That branch exists because it is used more often than any single pattern on the left, and drawing it is how you show the interviewer that you know it.

Now the wrapping four, drawn so the difference is visible rather than described:

ADAPTER – the interface changes

caller --[our interface]→ Adapter --[their odd interface]→ Legacy

DECORATOR – the interface is preserved, and it stacks

caller --[I]→ Retry --[I]→ Logging --[I]→ Cache --[I]→ Real

PROXY – the interface is preserved, and the wrapper decides whether to call at all

caller --[I]→ Proxy --(maybe)→ Real
 |
 +→ permission check / lazy load / remote call

FACADE – a new, smaller interface over several things

caller --[one simple method]→ Facade → Billing
 → Inventory
 → Notifications

The one that catches people: **Decorator and Proxy have the same shape**. The difference is intent — a decorator adds something the caller wanted, a proxy controls access to something the caller did not ask about — and a caching proxy and a caching decorator can be the same class with two different names on two different days. When they look identical, say so, and say which intent you mean.

5 How it actually works

The five moves for any “which pattern” question

This is the script. It works whether the prompt is a payment gateway, a report generator or an notification system.

Move 1 — restate the requirement as an axis of change. “So the thing that keeps changing is *which payment provider we use*, and we add roughly one a year.” This single sentence does most of the work, because most wrong pattern choices come from picking the wrong axis.

Move 2 — write the naive version out loud, and do not be embarrassed by it. “Today this would be an `if provider == 'razorpay'` chain in one function.” Say it, then say specifically what breaks: how many files change, how many tests re-run, what gets missed.

Move 3 — name the pattern and what it makes cheap. “That is Strategy: one interface, one class per provider, and the choice made in a registry. Adding the fourth provider is one new file and one line, and no existing test re-runs.”

Move 4 — name the cost. “What I lose is that reading ‘what happens for Razorpay’ is now three hops instead of reading one function top to bottom, and the failure moves from compile time to run time if the registry has no entry.”

Move 5 — name the condition under which you would not do it. “If the only difference between providers were a rate or a URL, this would be a dictionary, not four classes. I would use Strategy because the *shapes* of the rules differ, not just the values.”

Five sentences. If you can produce those five for any prompt, you have passed this part of the interview regardless of which pattern you named, because the reasoning is the answer.

A worked example, start to finish

“We send notifications. Right now it is email. Marketing wants SMS. Support wants a Slack message for high-priority items. And every notification must be logged, and some must be retried three times. How would you structure it?”

Move 1 — there are two axes, not one. “There is *which channel* — email, SMS, Slack — and there is *what wraps every send* — logging, retry. Those change independently, and that is the important observation, because one pattern will not cover both.”

Move 2 — the naive version. “One `send` function with an `if channel ==` chain, and logging and retry written inline in each branch. Adding a channel means editing the shared function; adding retry to SMS means copying the retry code.”

Move 3 — two patterns, one per axis. “Channels are **Strategy**: a `Notifier` interface with `send(message)`, one implementation per channel, chosen by a registry. Logging and retry are **Decorator**: wrappers that implement the same `Notifier` interface and hold another one, so

`RetryingNotifier(LoggingNotifier(SmsNotifier()))` composes. Decorator rather than Strategy for those, because I want to *stack* them and choose the stack at run time.”

Move 4 — the costs. “Six small classes instead of one function. A stack trace now shows three wrappers before the real send. And the composition happens in the composition root, so ‘what actually happens when we send an SMS’ is not answerable by reading one file.”

Move 5 — where I would stop. “If there were one channel and no retry, none of this. And if high-priority items must go to *several* channels at once, that is not Strategy any more — that is **Observer**, and I would want to know before choosing.”

That last line is the one that gets remembered. Noticing that a small requirement change would flip the pattern is exactly the judgement being tested.

The patterns you have already used without naming them

Being able to point at real code is what separates having read about patterns from having used them.

- **Singleton** — Python’s module system. `import config` gives everybody the same object, and that is the least dangerous singleton there is.
- **Factory** — `logging.getLogger(name)`, `open(path, mode)`, `hashlib.new("sha256")`.
- **Builder** — a query builder, `subprocess.Popen`’s enormous keyword list, `datetime.replace`.
- **Prototype** — `copy.deepcopy`, and `dataclasses.replace(obj, field=new)`.
- **Adapter** — `io.StringIO` making a string look like a file. Every ORM.
- **Decorator** — Python’s `@` decorators, `functools.lru_cache`, WSGI middleware, `gzip.GzipFile` wrapping a file object.
- **Facade** — `requests.get`, which hides connection pooling, redirects, TLS and encoding.
- **Proxy** — Django’s lazy `QuerySet`, an ORM’s lazy relation, a gRPC client stub, a CDN.
- **Strategy** — `sorted(key=...)`, `nginx`’s `least_conn`, Django’s `PASSWORD_HASHERS`.
- **Observer** — `addEventListener`, Django signals, Kafka consumers.
- **State** — TCP’s connection states, Stripe’s `PaymentIntent`, Kubernetes pod phases.
- **Command** — Celery tasks, the Postgres write-ahead log, `git revert`, Ctrl+Z.
- **Chain of responsibility** — Django middleware, `iptables`, Python’s `logging` propagation.
- **Template method** — `unittest.TestCase`, Django’s class-based views, `JdbcTemplate`.
- **Iterator** — every `for` loop, `itertools`, S3 continuation tokens.

If you can name three of these from memory in an interview, you have said more than the pattern definition ever could.

6 The numbers

What each pattern costs to introduce

Measured as the smallest honest version — one interface plus three implementations, against the `if` chain it replaces:

<code>if/elif chain, 3 cases</code>	1 file, ~26 lines
<code>dict of functions (Python)</code>	1 file, ~12 lines
<code>Strategy, 3 classes</code>	5 files, ~90 lines
<code>State, 5 states</code>	6 files, ~85 lines
<code>Decorator, 2 wrappers</code>	3 files, ~55 lines
<code>Observer, 4 listeners</code>	6 files, ~95 lines
<code>Abstract Factory, 2 families × 3</code>	9 files, ~160 lines

So the class-based version of a three-case rule costs roughly **65 extra lines and 4 extra files**. That is the price of admission, and it is worth quoting because it makes “is this worth it?” a question with an answer instead of a matter of taste.

What it buys, per new variation

```
adding the 4th case:
if-chain:  1 shared file edited · ~32 existing tests re-run · 3 live rules at risk
pattern:   1 new file added      ·  0 existing tests re-run · 0 live rules at risk

adding the 12th case:
if-chain:  a 90-line function nobody fully understands; cost per case RISING
pattern:   identical to adding the 4th; cost per case FLAT
```

The flatness is the whole argument. The pattern is not cheaper on the second case. It is cheaper on the sixth, and it stops getting more expensive, which the `if` chain never does.

The break-even, stated as a rule

1 variation:	pattern costs ~65 lines, saves nothing	→ do not
2 variations:	pattern costs ~65 lines, saves ~10	→ probably not
3 variations:	roughly break even	→ judgement
4+ and still arriving:	clearly worth it	→ yes

The famous version of this is the **rule of three**: write it inline once, write it inline twice, and extract on the third. It is a rule of thumb, not a law, and the reason it works is that the second occurrence is the first time you can *see* which part actually varies. Deciding from one example is guessing.

The cost of guessing wrong

```
wrong pattern, discovered in month 2:
    ~200 lines to unpick, 1 day, no user impact

wrong pattern, discovered in month 14:
    9 dependent classes, 3 teams, a deprecation cycle
    typical: 2-3 weeks, or it is never removed at all
```


The asymmetry is the reason to under-apply rather than over-apply. Removing an unnecessary `if` is an afternoon. Removing an unnecessary hierarchy that four teams have built on is a quarter, and it usually does not happen — it just stays there, and every new engineer assumes it is load-bearing.

The run-time cost, so you can dismiss it

one extra method call in Python	~60 ns
a 3-deep decorator stack	~180 ns per call
an 8-handler chain of responsibility	~500 ns of dispatch

On a request that takes 40 milliseconds, a 3-deep decorator stack is **0.0005 percent**. The cost of patterns is never speed; it is always readability and indirection. Say that when somebody raises performance, and get back to the real argument.

7 The trade-offs

What all of these patterns cost you, without exception

Indirection. Every one of them replaces “read the function” with “find the implementation”. Three hops instead of one. That cost is paid on every read by every engineer for the life of the code, and reads outnumber writes enormously.

Errors move from compile time to run time. A registry lookup that has no entry for a new country is a `KeyError` when a customer checks out, not a build failure. Every factory, registry and dynamic dispatch has this shape. Mitigate it — validate at start-up, use enums as keys, assert completeness in a test — and say that you would.

Names outlive their reasons. `PricingStrategy` with one implementation, two years after somebody deleted the other two, is worse than no interface: it looks deliberate, so nobody removes it.

The interface becomes the lowest common denominator. The moment one implementation needs an argument the others do not have, you either widen the interface for everyone or start passing a context dictionary. This is how clean interfaces become `handle(request, context: dict)`.

Patterns invite more patterns. A factory suggests an abstract factory; a strategy suggests a strategy factory. Each step is locally reasonable and the destination is a codebase where finding where the work happens takes twenty minutes.

The Python-specific concession, which is worth making unprompted

Several of these patterns exist because Java and C++ lack first-class functions. In Python:

- **Strategy** is often a function in a dictionary. Twelve lines rather than ninety.
- **Command** without `undo` is `functools.partial(fn, *args)`.

- **Decorator** is frequently the `@` syntax rather than a wrapper class.
- **Singleton** is a module.
- **Abstract Factory** is often a dictionary of dictionaries, and honest about it.
- **Iterator** is `yield`.

Saying “in Python I would write this as a dict of functions, and here is the version with classes if we need the ceremony” is a strong answer. It shows you know the pattern *and* the language, and it directly answers the “what would you lose” half of the question — you lose about seventy-eight lines and gain nothing.

“I would not use a pattern at all if...”

- **...this is the first variation.** You are guessing at the seam. The second occurrence tells you where it actually is.
- **...only a value differs, not a behaviour.** Three tax rates are a dictionary. Three *tax rule shapes* are a Strategy. This is the single most common mistake.
- **...the set is genuinely closed.** Seven weekdays, four suits in a deck, three states an order can be in and no more ever.
- **...I cannot name the second implementation.** A test double counts. A hypothetical does not.
- **...the team is two people and the code is six months old.** Patterns buy coordination between people who will never speak to each other. That is a real benefit and it is worth nothing yet.

The sentence that carries the most weight

“I would write the `if` today, and here is the specific signal that would make me extract the pattern: the third payment provider, or the first time two of them need different arguments.”

That answer says you know the pattern, you know its cost, and you have a trigger rather than a taste. It is better than naming the pattern immediately, and interviewers notice.

8 In the interview

How it gets asked

- The open version, mid-way through a design round: “Which pattern would you use here, and what would it cost you?”
- The refactoring version: “Here is a 200-line function with a nine-way `if`. What do you do?”
- The distinguishing version, which is almost guaranteed if you name a pattern: “How is that different from Strategy?” or “Is that not just a decorator?”

- The trap version: “*Would you use a Singleton for the database connection pool?*” — they want to hear the criticism, not the definition.
- The experience version: “*Tell me about a pattern you removed.*” This is the best question in the set and almost nobody has an answer ready. Have one.

What to say out loud, in the first ninety seconds

1. **Do not name a pattern yet.** “Before I pick anything, let me say what I think changes here and how often.”
2. **Name the axis of change in one sentence.** “The thing that varies is which payment provider, and we add about one a year.”
3. **Check for a second axis.** “Is there anything that varies independently of that? Because if logging and retry also vary, that is a second axis and it wants a different pattern.”
4. **Say the naive version and what breaks.** “Today this is an `if` chain. With three cases that is fine. The problem starts at five: one shared function that three teams edit, and a change to the India rule can break the UK test.”
5. **Name the pattern, then immediately name the cost.** “Strategy. What I lose is one function you can read top to bottom, and a `KeyError` at run time instead of an error at build time.”
6. **Give the condition for not doing it.** “If the providers differed only by a rate, this would be a dictionary and I would say so.”

The follow-ups

“**How is that different from Strategy?**” — for whichever pattern you named. Have the one-liners ready. *State*: the object replaces its own; the client picks a strategy. *Template Method*: inheritance and fixed at subclass-writing time; Strategy is composition and run-time. *Command*: a bound request that can be stored and undone; a strategy is an algorithm with no memory. *Decorator*: same interface, stacks, adds behaviour; a strategy replaces behaviour and does not stack.

“**Would you use a Singleton here?**” “Rarely, and I would want to know why one instance is required rather than merely convenient. The real objections are that it is global mutable state under a respectable name, that it makes tests order-dependent because state leaks between them, and that ‘one per process’ is meaningless once there are eight processes on four machines. For a connection pool I would create one instance at start-up and pass it in — dependency injection gives me the single instance without the global.”

“**When would you use no pattern at all?**” “Most of the time, honestly. When only one variation exists, when only a value differs rather than a behaviour, when the set is closed, or when I cannot name the second implementation. The cost of removing an unnecessary `if` is an afternoon; the cost of removing an unnecessary hierarchy that four teams have built on is a quarter, and usually it never happens. So I would rather be late than early.”

“You have used a pattern and regretted it. Tell me.” Have a real one. A usable shape: “I introduced a Strategy for two variants that turned out to be the same variant with a different constant. It became five files, and every reader had to open all of them to learn something that was one line. We removed it a year later and the diff was minus sixty-two lines. What I took from it is that ‘the behaviour differs’ and ‘the value differs’ look identical from a distance, and the test I use now is: can I write both implementations as one function with one extra argument? If yes, it is a value.”

“Which of these do you actually see most in real code?” “Strategy, Decorator, Observer and Iterator, by a distance — and Iterator and Template Method mostly because the language and the frameworks already implemented them. Abstract Factory and Prototype I have almost never needed. Singleton I see often and want to remove most times I see it.”

A model answer

Asked: *here is a service with a nine-way `if` on `provider`, and each branch is about thirty lines. Which pattern, and what would it cost?*

“Before I pick a pattern, let me name what is actually varying, because getting that wrong is how people end up with the wrong one.

The axis here is *which payment provider*, and nine branches tells me it has been varying for a while and is still varying — nobody writes nine branches on purpose in one sitting. So this is not a case where I would say ‘leave the `if`’. At two or three branches I would leave it.

Before I commit, one check: is there a second axis? If retry, logging and idempotency also vary independently of the provider, that is a separate dimension and one pattern will not cover both. If so, I would use Strategy for the provider and Decorator for the cross-cutting wrappers, because decorators stack and strategies do not.

Assuming one axis: this is Strategy. One `PaymentProvider` interface with the two or three methods the caller actually uses, one implementation per provider, and the selection in a registry — a dictionary from provider name to instance — built at start-up in the composition root. Crucially, the selection does not live inside the checkout service, or I have just moved the `if` and gained nothing.

What that buys, concretely: adding the tenth provider is one new file and one line in the registry. Zero existing tests re-run and zero live providers are at risk, because they are in different files. Today, a change to the Razorpay branch can break the Stripe tests, because they share a function and the same 200-line file is edited by three teams. And the cost per provider stops rising — the tenth costs what the fourth cost, whereas right now each one makes that function harder.

What it costs, and I want to say this rather than have you find it. Reading ‘what happens for Razorpay’ becomes three hops: open checkout, see it holds a provider, find the registry, open the implementation. Right now it is one function you can read top to bottom, and for a small number of cases that is genuinely better. The failure also moves from build time to run time — a provider name with no registry entry is a `KeyError` during a customer’s checkout — so I would type the key as an enum and add a start-up assertion that every supported provider has an implementation. Roughly, this is five files and ninety lines where there was one file and two hundred and seventy, so it is not a saving in lines. It is a saving in blast radius.

And the condition that would change my answer: if the nine branches turn out to differ only by a URL and an API key, this is not nine behaviours, it is one behaviour with nine configurations, and the right answer is a dictionary of settings and one implementation. That is the mistake I would be most worried about making here, so before writing anything I would read all nine branches and ask whether I can express any two of them as the same code with a different constant. If I can, they are data, not strategies.

One last thing on performance, since it sometimes comes up: the extra indirection is about sixty nanoseconds per call on a request that takes forty milliseconds. The cost of this decision is readability, never speed.”

9 Recall card

- **Name the axis of change before naming a pattern, and say how often it changes.** Five moves for any prompt: *restate the axis · say the naive version and what breaks · name the pattern and what it makes cheap · name the cost · name the condition for not doing it.* The reasoning scores higher than the choice.
- **The wrapping four, told apart by two questions:** *does the caller's interface change?* — Adapter and Facade **yes**, Decorator and Proxy **no**; *adding behaviour or controlling access?* — Decorator **adds and stacks**, Proxy **controls** (lazy, remote, permission). Facade = a **new smaller** interface over several subsystems; Adapter = **one** class behind the wrong interface.
- **The behavioural look-alikes, one line each.** Strategy vs State: *the client picks a strategy; the object replaces its own state.* Strategy vs Template Method: *composition and run-time vs inheritance and fixed.* Observer vs Chain: *everyone gets it vs the first able handler stops the walk.* Command vs Strategy: *if you need `undo`, it is Command.* Factory vs Builder: *which class vs fifteen optional fields.*
- **The economics: ~65 extra lines and 4 extra files for a three-case rule, and the payoff is flatness** — the twelfth case costs what the fourth cost, while the `if` chain gets worse for ever. **Rule of three.** Getting it wrong in month 2 costs a day; in month 14 it costs a quarter or never gets undone — so **under-apply rather than over-apply.** Runtime cost is ~60 ns per hop: **never a performance argument.**
- **“No pattern” is a real answer and often the right one:** one variation today · only a **value** differs, not a behaviour · a closed set · you cannot name the second implementation. In Python, Strategy is often a **dict of functions** (12 lines vs 90), Command without undo is `functools.partial`, Singleton is a **module**, Iterator is `yield`. The strongest sentence in the whole phase: **“I would write the `if` today, and the third provider is the signal that makes me extract it.”**

DSA topic: LRU cache: the structure interviewers love **System design topic:** Design patterns revision and interview questions

Code these, in this order

One rule for the whole set: **before writing a line, say which structure answers which question.** “The map answers *where is it*; the list answers *which is oldest*.” If you cannot say that sentence, you will merge the two and lose an hour.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Design HashMap	LeetCode 706 (Easy)	Buckets and chaining, so that “the map is $O(1)$ ” is something you have built rather than assumed.
2	LRU Cache	LeetCode 146 (Medium)	Today’s problem. Two structures, sentinels, and the fact that <code>get</code> is a write.
3	Design Browser History	LeetCode 1472 (Medium)	The same doubly linked list, with a cursor instead of an eviction rule.
4	LFU Cache	LeetCode 460 (Hard)	A third structure and a <code>min_frequency</code> counter, and why it stays $O(1)$.

On problem 2, write the sentinels first

Do not write `get` or `put` until the fake head and tail exist and `_unlink` and `_push_front` are both four lines with no `if` in them. If either helper needs a null check, the sentinels are wrong.

On problem 3, notice what changed

Browser history is a doubly linked list with no map and no eviction — the cursor moves rather than the nodes. Say in one sentence what problem 2 needed a hash map for, and why this one does not.

On problem 4, find the $O(1)$ argument

The hard part is not the code, it is why `min_frequency` can be maintained in constant time. Say out loud: when does it increase, when does it reset, and why can it only ever increase by exactly one.

The why-two-structures drill

Answer each without looking:

1. What does a hash map alone fail at, and what is the cost of patching it?

2. What does a list alone fail at, and what are the two separate costs?
3. What exactly does the map map to, and why not to the value?
4. Why must the list be doubly linked? Give the two-assignment answer.
5. What would you have to do if you were forced to use a singly linked list?
6. Which end holds the most recently used, and which operations touch each end?

The get-is-a-write drill

1. Implement `get` without the reordering.
2. Run `put(1,1); put(2,2); get(1); put(3,3); get(2)` on both versions. Write both outputs.
3. Say why the broken version returns correct *values* from every `get`.
4. Say what the symptom looks like in production rather than in a test.
5. Write the shortest test that distinguishes the two.

The both-structures drill

1. Delete the `del self._map[oldest.key]` line.
2. Run 2,000 random operations and print `len(self._map)` at the end. Say what has happened.
3. Then `get` an evicted key and print `keys_most_recent_first()`. Describe the corruption.
4. Now remove the key field from `Node` and try to write the eviction. Say what you would be forced to do instead and what it costs.

The break-it drill

Trigger each and record the output or the exact error text:

1. Remove the sentinels and write `_unlink` with null checks. Count the branches you needed.
2. Then run it on a cache holding exactly one node and quote the error you get when you forget one.
3. Treat an existing key as a new insert in `put`. Run `put(1,1); put(1,2); put(2,2); put(3,3)` with capacity 2 and describe the state of both structures.
4. In `_push_front`, set `self._head.next = node` first. Say what you lose.
5. Set capacity to 0 with no guard. Trace what `put` does.
6. Set capacity to 1 and run `put(1,1); put(2,2); get(1)`. State the expected answer.

The cost drill

1. Count the pointer writes in `get` and in `put`. State both.
2. State the space complexity, and say why it is not $O(n)$.
3. Compute the per-entry memory with and without `__slots__`, and the total for 100,000 entries.

4. Compute the average read time at a 90 percent hit rate with a 1 μ s cache and a 100 μ s database. Then at 50 percent. State both speed-ups.
5. Say which single number you would measure before increasing the capacity.

The follow-ups drill

Answer each in three sentences:

1. Why does the node store the key?
 2. Why not a singly linked list?
 3. Is `get` a mutation?
 4. Is this thread-safe, and what are the two fixes?
 5. How would you make it LFU?
 6. What would you use in production, and how does Redis differ from this?
-

The axis-of-change drill

For each prompt, say the axis of change in four words or fewer, then the pattern, then the cost, then the condition under which you would use no pattern:

1. Nine payment providers in one `if` chain.
2. An order that moves through six stages with different rules at each.
3. Five things that must happen when a user signs up.
4. Three report formats sharing a fixed four-step pipeline.
5. A legacy library whose method names are wrong for your codebase.
6. A settings object with eighteen optional fields, three of which are mutually exclusive.
7. Expense approval with limits at four levels.
8. An action the user must be able to undo.
9. Retry, logging and metrics that should wrap any of several senders.
10. A very large query result returned from an API.

Two of the ten are the same pattern for different reasons. Say which and why.

The look-alike drill

Give the one-sentence distinguisher for each pair, then write six lines of code where the difference is visible:

1. Strategy and State.
2. Strategy and Template Method.
3. Decorator and Proxy.
4. Decorator and Adapter.
5. Adapter and Facade.

6. Observer and Chain of Responsibility.
7. Command and Strategy.
8. Factory and Builder.

For pair 3, write the version where the two are genuinely indistinguishable and say what decides it.

The no-pattern drill

For each, say whether you would use a pattern, and give the deciding question:

1. Two tax rates that differ only by a number.
2. Two tax rules with genuinely different shapes.
3. Seven weekdays with different opening hours.
4. Three log destinations, with a fourth expected next quarter.
5. The first time a second variation has appeared.
6. Four suits in a deck of cards.
7. A retry policy each customer configures for themselves.
8. Three subclasses that share ten lines.

Three of the eight are “no” for the same reason. Name it in one sentence.

The Python drill

Write both versions and count the lines:

1. Strategy as classes, and as a dictionary of functions.
2. Command without undo as a class, and as `functools.partial`.
3. Singleton as a class with `__new__`, and as a module.
4. Iterator as two classes, and as a generator.
5. Decorator as a wrapper class, and with the `@` syntax.

Then say, for each, what the class version buys that the short version does not — and be honest when the answer is “nothing”.

The regret drill

Prepare a real answer to “*tell me about a pattern you introduced and later removed*”. Write it out in five sentences: what you built, why it seemed right, what the actual variation turned out to be, what the removal diff looked like, and what test you now apply instead. If you have no real example yet, build one deliberately this week — introduce Strategy for two things that differ only by a constant, live with it for a day, and remove it.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Design an LRU cache with $O(1)$ get and put.* The $O(1)$ -eviction requirement that forces two structures, each single structure rejected with its cost, the map pointing into the doubly linked list, why doubly, `get` is a write, the key stored on the node, sentinels as a deliberate decision, and the space bound with a number.
 2. *Which pattern fits here, and what would you lose by using it?* The axis of change named before any pattern, the check for a second axis, the naive version and what specifically breaks, the pattern with what it makes flat rather than cheap, the cost in hops and in run-time failure, and the condition that would make you write the `if` instead.
 3. *How is that different from Strategy?* State, Template Method, Command and Decorator, one sentence each, plus the line of code in which the difference is visible.
-

Before you move on

- ☐ I can say which structure answers which question, in one sentence.
- ☐ I can give the two-assignment reason for the list being doubly linked.
- ☐ I wrote the sentinels first and neither helper contains an `if`.
- ☐ I broke `get` so it does not reorder and saw the eviction go wrong while values stayed right.
- ☐ I deleted the map removal and watched both structures disagree.
- ☐ I can explain why the node stores its own key.
- ☐ I tested capacity 0 and capacity 1.
- ☐ I can quote the per-entry memory and the total for 100,000 entries.
- ☐ I can give the hit-rate arithmetic for 90 percent and 50 percent.
- ☐ I can answer “is it thread-safe” with both fixes.
- ☐ I named the axis of change before the pattern on all ten prompts.
- ☐ I can give all eight look-alike distinguishers without hesitating.
- ☐ I can say the break-even numbers and the rule of three.
- ☐ I can name three patterns I have already used without knowing it.
- ☐ I have a real answer ready for “a pattern you removed”.
- ☐ I answered all three questions above out loud.

Stacks and queues revision and mock round

DATA STRUCTURES AND ALGORITHMS

Stacks and queues revision and mock round

You can solve two unseen stack or queue problems cold.

SYSTEM DESIGN

How to run a low-level design interview: the forty-minute script

You have a repeatable order of moves for any LLD prompt, so you never freeze.

THE TWO QUESTIONS THIS CHAPTER ANSWERS

- *Two problems, no hints, talk as you go.*
- *Design a parking lot. You have forty minutes. Begin.*

Stacks and queues revision and mock round

1 What this is, and why they ask it

Ten days ago a stack was a list you could only touch at one end. Since then you have built balanced bracket checkers, a min stack, monotonic stacks, the largest rectangle in a histogram, a ring-buffer queue, a monotonic deque, a queue from two stacks and an LRU cache. Today is not new material. Today is **recognition under time pressure**, which is a different skill and the one actually being tested.

The mock round is two problems in forty-five minutes, spoken aloud, with no hints. What is being scored is not whether the code compiles. It is whether you say *“the stack holds the elements whose answer is not yet known”* before you write a loop, whether you give the amortised argument before being challenged, and whether you notice that a question about “the last k seconds” is a queue question.

They run this round because a candidate who has memorised eight solutions and one who has understood one idea look identical on the easy problems and completely different on an unfamiliar one. This day is about being the second kind.

2 The story

Nithya failed her driving test the first time and she still thinks it was unfair.

She had been driving for three years by then — her father’s car, mostly on the ring road, and every Sunday to her grandmother’s place forty minutes away. She was, by any reasonable measure, a competent driver.

The test lasted eleven minutes. The examiner sat beside her with a small tablet and said almost nothing. Start the car. Left at the junction. Pull over here. Reverse into that gap. Right at the lights. Back to the centre. He did not raise his voice once and he did not ask her a single question about driving.

She failed on four points. She had not looked over her right shoulder before moving off. She had not checked the mirror before slowing down. She had signalled after she had already started turning. And on the reverse she had done the whole thing correctly and beautifully while looking only at the screen on the dashboard.

She argued about it in the car park, which nobody has ever won. The car had gone exactly where she wanted it, every time, without a single scrape. That, she said, is driving.

Her instructor, when she rang him, was kind but not on her side. He said something she repeated to her sister that evening. He said: the man cannot see what you know. He can only see what you do. You looked in the mirror on the road every Sunday for three years and you did not look in it once in front of him, because when you are nervous you do the thing and you skip the showing.

So the second time she narrated the whole drive. Out loud, the entire eleven minutes. Checking the mirror. Right shoulder. Signalling now. Gear down for the junction. Nothing to my left, going.

She said afterwards that it felt ridiculous and that the examiner must have thought she was mad. He did not. He ticked things off, said “that was fine”, and gave her the pass.

The driving was identical both times. She is sure of that. What changed was that the second time, someone else could tell.

3 The idea in plain English

The mock round is the driving test. The interviewer cannot see what you know. They can only see what you say. Everything below is a thing to **say**, in order, before or while you write.

The recognition procedure, in the order to run it

Run this on any problem in the phase, in under sixty seconds.

1. Does the problem mention “most recent”, “last”, “innermost”, “matching”, or “undo”? → Stack. The recognition sentence is: *reach for a stack when you need the most recent thing not yet resolved.*

2. Does it mention “in order”, “first”, “oldest”, “levels”, “shortest path”, or “the last k seconds”? → Queue. FIFO is what makes level-by-level and shortest-path-in-an-unweighted-graph work.

3. Does it ask, for every element, about the nearest element on one side satisfying a comparison? → Monotonic stack. Next greater, next smaller, previous greater, previous smaller — four variants from two knobs.

4. Does it ask for an aggregate over a window that slides? → Monotonic deque if the aggregate is max or min. A running sum if it is a sum. Two heaps if it is a median.

5. Does it need both “find this key fast” and “find the oldest fast”? → Two structures: a hash map plus a doubly linked list. One structure will not do it.

6. Does it ask you to build one structure out of another? → Two stacks make a queue with an amortised argument. Queues make a stack without one.

The five sentences that carry the whole phase

If you remember nothing else, these are what you say out loud.

1. **“The stack holds the things whose answer is not yet known.”** True of brackets, of monotonic stacks, of the histogram, of the calculator family. Say what your stack holds *before* writing the loop, every single time.
2. **“Each element is pushed once and popped at most once, so it is $O(n)$ — the inner loop is bounded in total, not per iteration.”** This is the complexity defence for half the phase. Say it before being challenged.
3. **“The front of a Python list is the expensive end.”** `pop(0)` and `insert(0, x)` are $O(n)$. Measured: 10.2 seconds against 0.0034 for 100,000 elements.
4. **“Store indices, not values.”** Because the follow-up always wants a distance or a width, and a value cannot give you one.
5. **“Amortised $O(1)$ means the total across any sequence is bounded — it is not average-case, and it is not a promise about any single call.”**

The one-page comparison

STRUCTURE	ADD	REMOVE	LOOK	USE IT WHEN
<code>list</code> as a stack	<code>append</code> $O(1)$	<code>pop()</code> $O(1)$	<code>[-1]</code>	Most recent unresolved thing
<code>list</code> as a queue	<code>append</code> $O(1)$	<code>pop(0)</code> $O(n)$	<code>[0]</code>	Never
<code>deque</code>	<code>append</code> / <code>appendLeft</code> $O(1)$	<code>pop</code> / <code>popLeft</code> $O(1)$	<code>[0]</code> , <code>[-1]</code>	Any queue; both ends
Ring buffer	$O(1)$	$O(1)$	$O(1)$	Fixed capacity, no allocation
Monotonic stack	$O(1)$ amortised	$O(1)$ amortised	<code>[-1]</code>	Nearest greater/smaller
Monotonic deque	$O(1)$ amortised	$O(1)$ amortised	<code>[0]</code>	Window max or min
Two stacks as a queue	$O(1)$	$O(1)$ amortised, $O(n)$ worst	$O(1)$ amortised	Only when asked
Map + doubly linked list	$O(1)$	$O(1)$	$O(1)$	LRU: key lookup <i>and</i> recency

What the round is actually scoring

Four things, in this order of weight:

1. **Did you say what the structure holds, before coding?** More than anything else, this is what distinguishes understanding from recall.
2. **Did you give the complexity with the counting, not just the letter?** “ $O(n)$ because each element is pushed once and popped at most once” beats “ $O(n)$ ”.
3. **Did you handle the empty case?** `while stack and ...` is the single most common missing guard in this phase.
4. **Did you name the trade-off you chose?** Ring buffer versus deque, exact LRU versus approximate, `<` versus `≤`. Choosing is fine. Choosing silently is not.

4 The picture

The phase on one page. Every problem you have met, grouped by the sentence that triggers it.

"the most recent thing not yet resolved"	→ STACK
+-- unmatched opener	→ balanced brackets
+-- the value before this one	→ min stack (carry the aggregate)
+-- the element whose answer is unknown	→ monotonic stack
+-- answer is a VALUE	→ next greater element
+-- answer is a DISTANCE	→ daily temperatures
+-- answer is a WIDTH between boundaries	→ largest rectangle
+-- the context I am leaving	→ decode string, RPN, calculator
"in the order they arrived"	→ QUEUE
+-- fixed capacity, no allocation	→ ring buffer
+-- one level at a time	→ level-order traversal, BFS
+-- the last k seconds	→ sliding window of timestamps
+-- both ends needed	→ DEQUE
+-- max or min over a sliding window	→ monotonic deque
"lookup fast AND oldest fast"	→ MAP + DOUBLY LINKED LIST
+-- evict the least recently used	→ LRU cache
+-- evict the least frequently used	→ LFU: a third structure
"build X out of only Y"	→ TRANSFORMATION
+-- queue from two stacks	→ amortised $O(1)$; pour when empty
+-- stack from queues	→ $O(n)$ push, NO amortised saving

What to notice: the left-hand column has only four entries. **Four trigger sentences cover ten days.** That is the compression you are aiming for — not ten memorised solutions, four questions you ask yourself.

And the diagram that answers “is it linear?”, because it will be asked twice:

```
for element in n elements:      ← runs exactly n times: n pushes
    while <condition>:          ← only ever POPS
        pop()

pushes across the whole run: exactly n
pops   across the whole run: at most n (nothing that leaves ever returns)
-----
total operations:                ≤ 2n →  $O(n)$ 

One iteration may pop 100,000 elements. That iteration has consumed
all the pops the rest of the run was going to do.
```

5 The code, built step by step

Two problems, worked the way you would work them in the room — the talking first, then the code.

Problem one: “You are given a list of daily stock prices. For each day, report how many consecutive days up to and including today had a price less than or equal to today’s price.”

What to say first, before writing anything.

“Let me restate it: for day `i`, I want the length of the longest run ending at `i` where every price is at most `prices[i]`. So really I am looking for the nearest earlier day with a *strictly greater* price — everything after that day and up to today qualifies. That is ‘previous greater element’, which is a monotonic stack.”

“Brute force: for each day walk backwards until I find a bigger price. That is $O(n^2)$ — five billion comparisons at a hundred thousand days.”

“The stack will hold the days whose price has not yet been beaten. When today’s price is at least the price on top, that day can never be the answer for any future day either, because today is later and at least as large — so I pop it, and I take its span with me.”

Then the code, in fragments.

```
stack: list[tuple[int, int]] = []      # (price, span) for unbeaten days
spans: list[int] = []
```

The stack holds pairs, not just prices, and that is the trick: **when I pop a day, I absorb its span into mine**, so I never re-walk the days it already covered.

```
for price in prices:
    span = 1                    # today always counts
    while stack and stack[-1][0] <= price:
        span += stack.pop()[1]  # absorb the span of everything I beat
    stack.append((price, span))
    spans.append(span)
```

Five lines. Say the invariant out loud while writing: *the stack is strictly decreasing by price from bottom to top, and each entry carries how many days it accounts for*.

Then the complexity, unprompted. “Each day is pushed exactly once and popped at most once, so at most $2n$ stack operations across the whole run — $O(n)$ time, $O(n)$ space in the worst case of strictly increasing prices, where nothing is ever popped.”

Then the edge cases, out loud. “Equal prices: the problem says ‘less than or equal’, so I pop on `<=`, which means an equal earlier day is absorbed. If it had said ‘strictly less’, I would use `<`. That is a one-character difference and I would confirm it with you.”

Problem two: “Implement `allow(user_id, timestamp) -> bool`. A user may make at most 100 requests in any rolling 60-second window.”

What to say first.

“‘Any rolling 60-second window’ is the giveaway — that is a queue of timestamps per user. I keep the timestamps of that user’s recent requests, drop the ones older than 60 seconds from the front, and then the count is just the length.”

“Front removals and back additions, both $O(1)$, so a `deque`. If I used a list and `pop(0)` this would be $O(n)$ per eviction, which at a hundred requests a user is not fatal but is a bad habit and I would not do it.”

“One thing I want to flag before coding: this grows without bound if I keep a deque per user for ever. So I would ask whether we need exact windows or approximate — and if memory matters, the standard production answer is a counter per fixed window, or a token bucket, both of which are $O(1)$ memory per user. I will write the exact one, since that is what was asked.”

Then the code.

```
from collections import defaultdict, deque

class RateLimiter:
    def __init__(self, limit: int = 100, window_seconds: int = 60) → None:
        self._limit = limit
        self._window = window_seconds
        self._hits: dict[str, deque[float]] = defaultdict(deque)
```

`defaultdict(deque)` so a first-time user does not need a special case.

```
    def allow(self, user_id: str, timestamp: float) → bool:
        hits = self._hits[user_id]
        cutoff = timestamp - self._window
        while hits and hits[0] ≤ cutoff:      # expire from the FRONT
            hits.popleft()
```

The `while` here — not an `if` — because unlike the sliding-window maximum, several timestamps can expire between two calls. Saying why you chose `while` over `if` is exactly the kind of remark that gets noticed.

```
        if len(hits) ≥ self._limit:
            return False                    # rejected: do NOT record it
        hits.append(timestamp)
        return True
```

The subtle line is `return False` before `append`. A rejected request must not be recorded, or a user who keeps hammering the endpoint never escapes the window — every rejection extends their own ban. Say that out loud; it is the bug the interviewer is watching for.

Then the complexity. “Amortised $O(1)$ per call: each timestamp is appended once and popped once, so n calls cost at most $2n$ deque operations. Memory is $O(\text{limit})$ per active user — a hundred floats, about 900 bytes with the deque overhead — so a million active users is roughly 900 MB, which is the number that would make me move to a token bucket.”

The reference implementations, for revision

```
from collections import defaultdict, deque

def stock_spans(prices: list[int]) → list[int]:
    """For each day, the number of consecutive days up to and including today
    with a price ≤ today's price.

    A monotonic stack of (price, span). Popping absorbs the popped day's span,
    which is why no day is ever re-examined. O(n) time, O(n) space.
    """
    stack: list[tuple[int, int]] = []      # (price, span), prices decreasing
    spans: list[int] = []

    for price in prices:
        span = 1
        while stack and stack[-1][0] ≤ price:
            span += stack.pop()[1]         # absorb, do not re-walk
        stack.append((price, span))
        spans.append(span)

    return spans

class RateLimiter:
    """At most `limit` requests per user in any rolling `window_seconds`.


One deque of timestamps per user. Expire from the front, count the rest.  

    A REJECTED request is not recorded, or a hammering client never recovers.  

    Amortised O(1) per call; O(limit) memory per active user.


    """

    def __init__(self, limit: int = 100, window_seconds: float = 60.0) → None:
        self._limit = limit
        self._window = window_seconds
        self._hits: dict[str, deque[float]] = defaultdict(deque)

    def allow(self, user_id: str, timestamp: float) → bool:
        hits = self._hits[user_id]
        cutoff = timestamp - self._window
        while hits and hits[0] ≤ cutoff:
            hits.popleft()
        if len(hits) ≥ self._limit:
            return False
        hits.append(timestamp)
        return True

    def forget_idle(self, now: float) → int:
        """Drop users with no recent activity. Without this, memory grows for
        ever – which is the honest weakness of the exact-window approach."""
        stale = [user for user, hits in self._hits.items()
                  if not hits or hits[-1] ≤ now - self._window]
        for user in stale:
            del self._hits[user]
        return len(stale)

    def is_balanced(text: str) → bool:
        """The phase in six lines. The stack holds the openers not yet closed."""
        pairs = {"(": ")", "[": "]", "{": "}"
        stack: list[str] = []
        for character in text:
            if character in "([{":
                stack.append(character)
            elif character in pairs:
                if not stack or stack.pop() ≠ pairs[character]:
```

```

        return False
    return not stack # non-empty at the end is a failure

def next_greater(numbers: list[int]) → list[int]:
    """The base monotonic stack. Indices, because the follow-up wants a distance."""
    answer = [-1] * len(numbers)
    stack: list[int] = []
    for index, value in enumerate(numbers):
        while stack and numbers[stack[-1]] < value:
            answer[stack.pop()] = value
        stack.append(index)
    return answer

def max_sliding_window(numbers: list[int], k: int) → list[int]:
    """Monotonic deque. Expire the front, crush the back, append, then record."""
    if k ≤ 0 or not numbers:
        return []
    window: deque[int] = deque()
    answers: list[int] = []
    for index, value in enumerate(numbers):
        if window and window[0] ≤ index - k:
            window.popleft()
        while window and numbers[window[-1]] ≤ value:
            window.pop()
        window.append(index)
        if index ≥ k - 1:
            answers.append(numbers[window[0]])
    return answers

if __name__ == "__main__":
    print(stock_spans([100, 80, 60, 70, 60, 75, 85])) # [1, 1, 1, 2, 1, 4, 6]
    print(stock_spans([1, 2, 3, 4])) # [1, 2, 3, 4]
    print(stock_spans([4, 3, 2, 1])) # [1, 1, 1, 1]
    print(stock_spans([5, 5, 5])) # [1, 2, 3]

    limiter = RateLimiter(limit=3, window_seconds=10)
    print([limiter.allow("u1", t) for t in (0, 1, 2, 3, 11)])
    # [True, True, True, False, True]

    print(is_balanced("{[]}", is_balanced("{}"), is_balanced("("))
    # True False False
    print(next_greater([2, 1, 2, 4, 3])) # [4, 2, 4, -1, -1]
    print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
    # [3, 3, 5, 5, 6, 7]

```

6 What it costs

The whole phase, priced at $n = 1,000,000$

balanced brackets	$O(n)$ time, $O(n)$ space worst case
min stack	$O(1)$ per operation, $2n$ space
monotonic stack	$\leq 2n$ operations, $O(n)$ space worst case
largest rectangle	$\leq 2n+2$ operations, $O(n)$ space
ring-buffer queue	$O(1)$ per operation, $O(\text{capacity})$ space
monotonic deque (window max)	$\leq 2n$ operations, $O(k)$ space
queue from two stacks	$4n$ operations total, $O(n)$ space
LRU cache	~ 6 writes per get, ~ 10 per put, $O(\text{capacity})$ space

Every single line is linear or constant. That is the shape of the phase: **stacks and queues turn quadratic problems linear, and the mechanism is always that each element is handled a fixed number of times.**

The four arithmetic facts to have ready

1. list.pop(0) draining 100,000 elements	10.203 s	
deque.popleft draining the same	0.0034 s	~3,000x
2. $n \rightarrow 2n$ on list.pop(0):	10.20 s $\rightarrow$ 39.74 s	3.9x = quadratic
3. sliding window max, $n=200,000$, $k=1,000$		
brute force	1.984 s heap 0.092 s deque 0.050 s	40x and 1.8x
4. monotonic stack vs brute force at $n=100,000$		
5 x 10^9 comparisons vs 2 x 10^5 operations		~25,000x

Quoting one real measurement is worth more than three complexity classes, because anybody can say “ $O(n^2)$ is slower”.

Space, side by side

stack / deque of indices	$O(n)$ worst, $O(1)$ best	– name both inputs
monotonic deque	$O(k)$, independent of n	– the streaming argument
ring buffer	$O(\text{capacity})$, fixed	– no allocation at all
LRU cache	$O(\text{capacity})$, ~190 B/entry in Python	
min stack (pairs)	$2n$	
min stack ($O(1)$ extra)	n , but it overflows in fixed-width languages	

The complexity claim that gets probed

Three of them, and the probe is always the same shape:

- “*There is a loop inside a loop.*” $\rightarrow$ n pushes, each element popped at most once, $\leq 2n$ operations.
- “*Pop can move n elements.*” $\rightarrow$ four touches per element over its whole life, and it never goes backwards.
- “*You loop $2n$ times for the circular version.*” $\rightarrow$ still n pushes and n pops; the constant is two.

All three are the same argument. **Count the operations across the run, not the nesting.**

7 The traps

The complete list for the phase. Each one has cost somebody an interview.

`pop(0)` and `insert(0, x)`

$O(n)$, silently. The correct answers are `deque.popleft()` and `deque.appendleft()`. If you write `pop(0)` in a queue problem, the interviewer stops listening to the rest.

A missing `while stack and ...`

```
IndexError: pop from an empty list
IndexError: list index out of range
IndexError: pop from an empty deque
```

Three phrasings of one bug. The emptiness check must be the **first** half of the `and`, because `and` short-circuits.

Storing values instead of indices

Works until the follow-up asks for a distance or a width, then cannot be patched. Store indices; write `numbers[stack[-1]]` when you need the value.

Building the answer by appending as you pop

The elements never popped simply go missing and your output is shorter than the input. **Pre-fill the answer array with the default and write into it by index.**

Forgetting the leftovers

Whatever is still in the stack at the end has no answer. Either pre-fill the default, or push a sentinel that forces everything out. Without it, `largest_rectangle([1,2,3,4,5])` returns 0.

`<` versus `<=`

`next_greater([2,2,3])` gives `[3,3,-1]` with `<` and `[2,3,-1]` with `<=`. Both are correct programs; only one answers the question. Read the problem for the word “strictly”, and ask if it is not there.

The width off-by-one

`width = index - left - 1`, because both boundaries are the shorter bars and neither is inside the span. `index - left` returns 15 instead of 10 on the standard input.

`head == tail` meaning both empty and full

Keep an explicit `_size` counter. And never loop on `while head < tail` — it breaks the moment the tail wraps.

Reading a ring buffer in slot order

Wrong order as soon as `tail < head`. Walk `(head + i) % capacity` for `i` in `range(size)`.

Not clearing the dequeued slot

`self._items[self._head] = None`. Without it the queue holds every object it has ever seen alive, and a job runner leaks gigabytes.

`is_empty` checking one of two structures

In the two-stack queue, `not self._outbox` reports a queue holding a thousand elements as empty. Check both.

Pouring onto a non-empty outbox

No error, wrong order. `enqueue 1, 2 · dequeue · enqueue 3 · dequeue` returns 3 instead of 2. Pour **only** when the outbox is empty.

Rotating `len` times instead of `len - 1`

Turns your stack from queues into a queue. `push(1); push(2); pop()` returns 1.

`get` that does not reorder, in an LRU cache

Every returned value is right; every eviction is wrong. A read is a use.

Evicting from the list but not the map

The map grows for ever and a later `get` resurrects a node that is not in the list. **Every change touches both structures.**

No key stored on the LRU node

Eviction then cannot delete the map entry without scanning. Store the key.

No sentinels in linked-list code

```
AttributeError: 'NoneType' object has no attribute 'next'
```

Two fake nodes remove six branches and this error with them.

The expiry comparison in a sliding window

`window[0] ≤ index - k`, derived from “the window ending at `index` starts at `index - k + 1`”. With `<` it is right on most inputs and wrong on `[8, 3, 2]` with `k = 2`.

Recording before the window is full

`if index ≥ k - 1`. Without it you return `n` answers instead of `n - k + 1`.

Recording a rejected request in a rate limiter

The user’s window never empties and they are banned for ever. Return before appending.

Quoting amortised as if it were worst case

“O(1)” alone invites the challenge. Say “O(1) amortised, O(n) for a single call”, and say which one matters for a p99 latency target.

Using `queue.Queue` in a coding round

Correct, and about $48\times$ slower, because it locks on every operation. Name it only when the question is about threads.

8 In the interview

How it gets asked

- The plain mock: *“Two problems, forty-five minutes, talk as you go.”*
- The disguised stack: *“Evaluate this expression”, “decode this string”, “simplify this file path”, “remove k digits to make the smallest number”.*
- The disguised queue: *“how many requests in the last minute”, “print the tree level by level”, “how many minutes until every orange has rotted”.*
- The design-a-structure question: *“design a data structure with $O(1)$ insert, delete and get random”, or an LRU cache.*
- The follow-up that is really the question: *“is that not quadratic?”*

The script, minute by minute, for a 45-minute round

Minutes 0–3 — restate and clarify. Say the problem back in your own words. Ask about the three things that always matter in this phase: duplicates and whether comparisons are strict; the size of `n`; and what to return when the input is empty. Write the two or three examples they give you, plus one you invent that is degenerate.

Minutes 3–6 — brute force, with a number, then reject it. “For each element I would walk backwards, which is n squared over two — five billion at a hundred thousand elements.” Never skip this. It establishes the baseline that makes your improvement measurable.

Minutes 6–10 — the observation, not the algorithm. This is the part that is scored. “When a bigger element arrives, everything smaller before it is worthless for ever.” Or: “the last k seconds is a queue of timestamps.” Say what the structure will hold, in one sentence, before writing anything.

Minutes 10–25 — code, narrating. Write the emptiness guard as you write the `while`. Say “indices, not values, because the follow-up wants a distance”. Say “expire, crush, append, record” as you write those four lines.

Minutes 25–30 — trace one example out loud. Pick the degenerate one you invented. This catches the off-by-one before the interviewer finds it, and finding your own bug reads far better than having it pointed out.

Minutes 30–35 — complexity, with the counting. “ n pushes, each element popped at most once, so at most $2n$ operations — the inner loop is bounded in total, not per iteration.” Then space, with the worst-case input named.

Minutes 35–45 — the second problem, at double speed, because the pattern-recognition is now warm.

If you have five minutes left over, offer the cross-check: “I would normally run this against a brute force on small random inputs, because the off-by-one in the width is easy to miss.”

The follow-ups

“There is a loop inside a loop. Is that not $O(n^2)$?” “No — count operations rather than nesting. Every element is pushed exactly once by the outer loop and popped at most once, because once it leaves the stack it never returns. So the total is at most $2n$ across the whole run, however unevenly it falls. One iteration might pop a hundred thousand elements; that iteration has used up all the pops the rest of the run was going to do.”

“Why a deque rather than a list?” “Because I need to remove from the front, and the front of a list is the expensive end — removing element zero shifts every remaining element one slot left, so it is $O(n)$ and draining is $O(n^2)$. I measured it once: a hundred thousand elements took ten seconds with `pop(0)` and three milliseconds with `deque.popleft`, and doubling the input quadrupled the time, which is the quadratic signature.”

“Can you do it with less space?” Depends on the problem, and the honest answers differ. “For window maximum, no — $O(k)$ is already independent of n , and there is a block-partition trick that is also $O(n)$. For the min stack, yes: push `2*value - min` and recover with arithmetic, but it overflows in fixed-width languages, so I would mention it and not write it. For a rate limiter, yes: a token bucket is $O(1)$ per user instead of $O(\text{limit})$.”

“What if the data is a stream and never ends?” “That favours the deque solutions, because their space is $O(k)$ rather than $O(n)$ — a sixty-second window over a billion events holds sixty entries. What breaks is anything that needs the whole array up front, like the two-pass histogram version. And I would watch the index counter, which grows without bound in a long-running process.”

“Is this thread-safe?” “No. `deque.append` and `popleft` happen to be atomic in CPython, so a single-producer, single-consumer deque is safe by accident, but I would not rely on that. Anything with two structures — the LRU cache — is definitely not: two threads interleaving pointer updates can corrupt the list with no exception raised. One lock, or shard by key hash to reduce contention.”

“Which of these would you actually use in production?” “`collections.deque` for every queue. `OrderedDict` or `functools.lru_cache` instead of hand-writing an LRU. A ring buffer only when the capacity is fixed and allocation matters, like a metrics buffer. The two-stack queue essentially never, outside functional languages. And Redis with an LRU eviction policy when the cache has to be shared between processes.”

A model answer

Asked: *for each day, report how many consecutive days up to and including today had a price at most today's price.*

“Let me restate it to check I have it. For day `i`, I want the length of the longest run ending at `i` in which every price is at most `prices[i]`. Two things I would confirm: ‘at most’ means an equal earlier price counts, yes? And today itself always counts, so the smallest answer is 1.

The brute force is: from each day walk backwards while the prices are at most today’s. That is n^2 over two in the worst case — an increasing sequence — which is five billion comparisons at a hundred thousand days. So I want a single pass.

Here is the observation. The run ending today stops at the first earlier day with a *strictly greater* price. So what I actually need is the previous greater element, which is a monotonic stack.

And there is a second observation that makes it cheap. When I pop a day because today’s price beats it, that day can never be the answer for any future day either — any future day that could have reached it also reaches today, and today is at least as large. So popping is permanent, and I can absorb the popped day’s span into mine rather than walking those days again. That is why the stack holds pairs: a price and the number of days that price accounts for.

So: for each price, start a span of one. While the stack is non-empty and the price on top is at most today’s, pop it and add its span to mine. Then push my price with my span, and record the span. Five lines. The stack ends up strictly decreasing by price from the bottom up, and I did not sort it — anything that would have broken the order was popped.

On complexity: each day is pushed exactly once and popped at most once, so at most $2n$ stack operations across the whole run. $O(n)$ time. The inner `while` looks quadratic and is not — it is bounded in total, not per iteration. Space is $O(n)$ in the worst case, which is a strictly increasing price sequence where nothing is ever popped, and $O(1)$ on a strictly decreasing one where every day pops the previous.

Two things I would check before I called it done. The `<=` in the pop condition is where ‘at most’ lives — if the problem had said ‘strictly less than today’, that becomes `<`, and it changes the answer on equal prices: `[5, 5, 5]` gives spans of 1, 2, 3 with `<=` and 1, 1, 1 with `<`. And the empty input returns an empty list, which the loop handles without a special case.

If you want, I would normally check this against the $O(n^2)$ version on a few hundred small random arrays — the span-absorbing line is the kind of thing that is easy to get subtly wrong and easy to verify.”

9 Recall card

- **Four trigger sentences cover the whole phase.** “The most recent thing not yet resolved” → **stack**. “In the order they arrived / the last k seconds / one level at a time” → **queue**. “For every element, the nearest one on a side satisfying a comparison” → **monotonic stack**. “Lookup fast AND oldest fast” → **map** + **doubly linked list**.
- **Say what the structure holds before you write the loop.** “The stack holds the elements whose answer is not yet known.” That one sentence is what the round is scoring — the driving test cannot see what you know, only what you say.
- **The complexity defence, said before it is challenged: n pushes, each element popped at most once, $\leq 2n$ operations — bounded in total, not per iteration.** And its cousin for the two-stack queue: **four touches per element, never backwards**, so amortised $O(1)$ — which is not average-case and is not a promise about one call.
- **The four measurements to quote.** `pop(0)` 10.2 s vs `deque.popleft` 0.0034 s at 100,000 $(\sim 3,000 \times) \cdot n \rightarrow 2n$ gave **3.9**×, the quadratic signature · window max at $n = 200,000$, $k = 1,000$: brute **1.98 s**, heap **0.092 s**, deque **0.050 s** · monotonic stack vs brute force $\sim 25,000 \times$.
- **The traps that end rounds:** `pop(0)` · a missing `while stack and` · values instead of indices · appending the answer instead of writing by index · forgetting the leftovers · `<` vs `≤` · `width = right - left - 1` · `head = tail` needing a size counter · pouring onto a non-empty outbox · `get` that does not reorder · removing from one structure and not the other · recording a *rejected* request in a rate limiter.

How to run a low-level design interview: the forty-minute script

1 What this is, and why they ask it

A **low-level design** round gives you a small, familiar system — a parking lot, a lift, a vending machine, Splitwise — and forty minutes. The output is **classes**: what objects exist, what each one is responsible for, how they relate, and code for the two or three that matter. It is not about servers, load balancers or databases. That is the *high-level* round, and it comes much later in this course.

Today is not a system. Today is the **script** — the fixed order of moves you run on every one of these prompts for the next twenty days. Six moves, a rough number of minutes each, and one heuristic for finding the single place where the design is actually decided.

They ask this round because it is the closest thing to watching you work. Anyone can memorise a parking lot design; the round is built so that the interviewer can change a requirement at minute thirty-five and see whether your design bends or breaks. And they ask it because the most common failure is not a bad design. It is a candidate who knows the material, is asked an open question, and spends nine minutes deciding where to start.

2 The story

Vinod's tailoring shop is one room with two machines, and on the four days before any festival there are people standing in it from eight in the morning.

He has a way of taking an order and it never changes. The customer sits on the stool. He asks what the occasion is and when it is needed — always that first, because half the time the date is impossible and there is no point in the rest of the conversation. Then he takes the measurements, in the same order every time, calling them out while his wife enters them into the phone. Then the three questions: sleeves, neck, lining. Then the price and the date. Then the customer stands up.

Nine minutes. He has been doing it for twenty-two years and it is nine minutes whether the customer is chatty or silent.

Kishore, who joined last year, is in several ways a better tailor than Vinod. His finishing is neater and he is much better with difficult fabric. And on a busy day he takes thirty-five minutes with a customer and the room fills up behind him.

The reason is not that he is slow with the tape. It is that he starts wherever the customer starts. A woman comes in holding a photograph on her phone and says "can you do this neck", and Kishore looks at the photograph, and they talk about the neck, and then about whether the

fabric will hold that shape, and then about a similar thing he made last month — and twenty minutes later nobody has taken a single measurement or asked when it is needed.

Vinod is not rude to those customers. He looks at the photograph for four seconds, says “yes, we can do that, let me take the measurements first”, and then does. The photograph comes back at the right moment, which is after the measurements and before the price, because the neck changes the price.

What Vinod says about it, when Kishore complains that his way is mechanical, is that having an order is what lets him actually think. He is not spending anything on deciding what to do next. So when something unusual turns up — the woman last week who needed it in two days and had brought fabric that was half a metre short — he had all of his attention for that, and he solved it, and it still took eleven minutes.

3 The idea in plain English

Vinod’s nine minutes are a **script**: a fixed order of moves that costs nothing to run, so that all of your thinking goes into the problem rather than into what to do next. The interview version has six moves.

The six moves

Move 1 · Clarify — three or four questions, then state your assumptions. (minutes 0–5) Not “any questions?” — specific ones whose answers change the design. Then say the assumptions out loud and write them down, because an interviewer who disagrees will correct you now rather than at minute thirty.

Move 2 · Requirements — a short list, and what you are leaving out. (minutes 5–8) Four or five bullet points of what the system does. Then, explicitly: “I am not going to design authentication, or the payment gateway integration, or the mobile app.” Naming the exclusions is what stops the scope quietly doubling.

Move 3 · The nouns — pull the classes straight out of the requirements. (minutes 8–13) Read your own requirement list and underline the nouns. Ticket, vehicle, spot, floor, payment. Write each as a class with **one line saying what it is responsible for**. If you cannot write that line in one sentence, the class is wrong.

Move 4 · The class diagram — the artefact they are waiting for. (minutes 13–22) Boxes, key fields, key methods, and the relationships between them. This is the thing the interviewer will photograph. Draw it while narrating; do not draw in silence.

Move 5 · The interesting part — find it, put an interface there, show two implementations. (minutes 22–33) Every LLD prompt has exactly one place where the design is genuinely decided. Find it and spend a third of your time there. This is the move that separates candidates, and the next section is entirely about it.

Move 6 · Concurrency and extension. (minutes 33–40) What happens when two people do the thing at the same moment. Then answer the requirement they are about to add, before they add it.

Move 5, which is the whole interview: find the one interesting part

A parking lot is not an interesting problem. `Vehicle`, `Spot`, `Ticket` and `Floor` are obvious and everyone gets them. What is interesting is **which spot you give a car**, because that is where a requirement change actually hurts.

Every prompt has one. Learn to find it, because the rest is bookkeeping:

PROMPT	THE ONE INTERESTING PART
Parking lot	Spot allocation, and pricing
Elevator	Scheduling: which lift answers which call
Vending machine	The state machine, and making change
ATM	The state machine, and what happens mid-withdrawal
Library	Who owns the late-fee rule
Tic-tac-toe	Win detection that survives an $N \times N$ board
Deck of cards	Shuffling, and comparing hands
Splitwise	Minimising the number of settlement transactions
BookMyShow	Seat locking when two people click at once
Food delivery	The order state machine
Ride hailing	Matching a rider to a driver
Rate limiter	Which algorithm, and why
In-memory cache	The eviction policy
Logging framework	Adding a new destination without touching the core

The test for whether you have found it: **if the interviewer changed one requirement, which part of your design would have to change?** That part is the interesting part, and it is where an interface belongs.

Then do the thing that wins the round: **put an interface there and show two implementations.**

```
class SpotAllocation(Protocol):
    def find(self, spots: list[Spot], vehicle: Vehicle) → Spot | None: ...

class NearestToEntrance: ...      # today's rule
class SpreadAcrossFloors: ...     # tomorrow's rule, for lift congestion
```

Two implementations, not one. One implementation is a class; two is a demonstration that the design survives change. It is Strategy from [day 071](#), applied to the part of the problem where change is actually going to arrive.

The rule that matters more than any of the moves

Never present the finished design as if it appeared fully formed.

Show version one. Say what breaks. Then fix it. “I would start with the parking lot itself finding the spot — one method, thirty lines. That is fine until pricing also varies by vehicle type and by time of day, at which point that method is doing two unrelated jobs. So I would pull allocation out behind an interface.”

An interviewer cannot tell the difference between a memorised diagram and an understood one — until you show them the seam and why it is there.

The nouns test, and the two failure modes

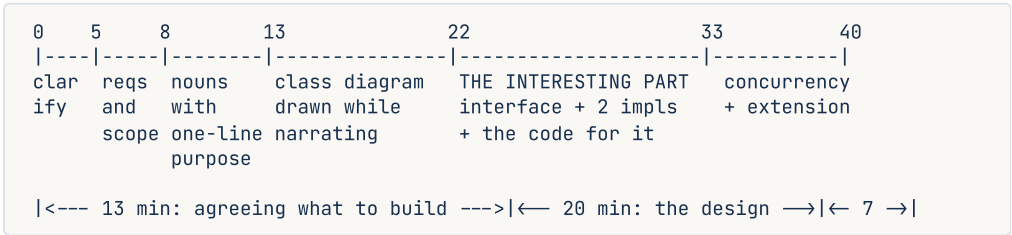
Too few classes: one `ParkingLot` class with eleven methods and every rule inside it. It works, and it demonstrates nothing.

Too many: `ParkingLotFactory`, `AbstractTicketBuilder`, `SpotAllocationStrategyRegistryProvider`. This is Ravi’s drill through the wall from [day 076](#), and interviewers read it as inexperience rather than sophistication.

Six to ten classes is right for a forty-minute prompt. If you have four, you have merged responsibilities. If you have eighteen, you are designing a framework nobody asked for.

4 The picture

The forty minutes, drawn as a budget. Notice how little of it is code.



What to notice: **a third of the round is spent before you draw a single box**. That feels wrong when the clock is running, and it is the difference between designing the right system and designing a system fast.

The shape of every class diagram you will draw for the next twenty days:

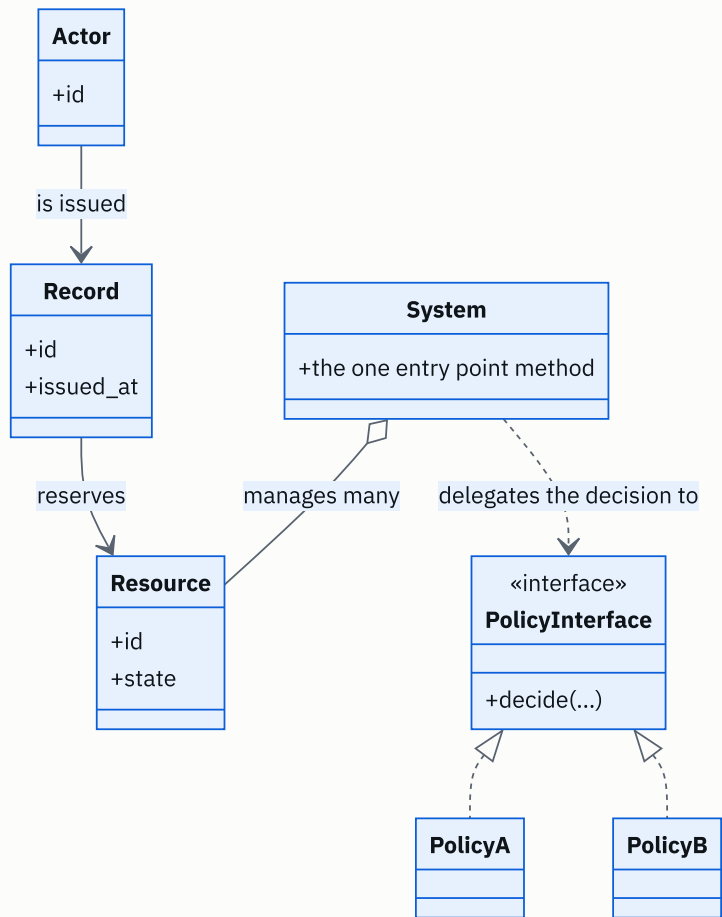
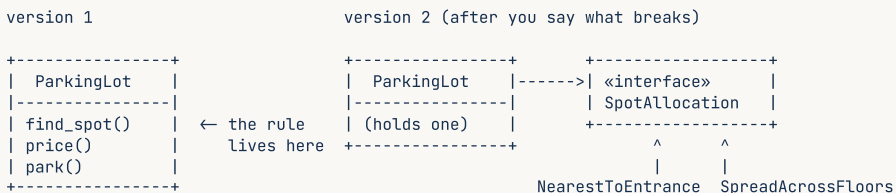


FIGURE 77.1

What to notice: the dashed arrow from **System** to **PolicyInterface**. Every good low-level design has exactly one of those, and it points at the interesting part. If your diagram has no interface in it, you have described a system rather than designed one. If it has five, you have over-built.

And the move-5 pattern, drawn on its own because it is the one to internalise:



5 How it actually works

The script, run end to end, on a prompt you will meet properly tomorrow.

Move 1 · Clarify (minutes 0–5)

Ask four questions. Good ones have the property that a different answer gives a different design:

“Multiple floors, or one level?” — *Multiple*. “Multiple vehicle sizes, and can a small vehicle use a large spot?” — *Motorcycle, car, bus. Yes, but only if nothing smaller is free*. “Is payment on exit, and is it time-based?” — *On exit, hourly, with a different rate per size*. “Roughly how big — a hundred spots or ten thousand?” — *About a thousand*.

Bad questions are ones whose answer changes nothing: “is it an underground car park?”

Then say the assumptions out loud: “I will assume a ticket is issued at entry and paid at exit, that a vehicle occupies exactly one spot, and that we are designing the in-process object model, not the database schema. Stop me if any of those is wrong.”

Move 2 · Requirements and scope (minutes 5–8)

```
In scope
- park a vehicle: find a spot, issue a ticket
- unpark: compute the fee, free the spot
- several floors, several spot sizes
- report free spots per floor
Out of scope, deliberately
- payment gateway integration
- number-plate recognition
- the mobile app and the API layer
```

Saying the second list is a move, not an omission. It stops the interviewer’s follow-ups from being about things you never claimed to design.

Move 3 · The nouns (minutes 8–13)

Read your own requirements and pull the nouns out, each with one line of responsibility:

- **Vehicle** — a number plate and a size. Knows nothing else.
- **Spot** — a location, a size, and whether it is occupied.
- **Floor** — holds spots, and can report what is free.
- **Ticket** — links a vehicle to a spot with an entry time. Immutable once issued.
- **ParkingLot** — the entry point: `park(vehicle)` and `unpark(ticket)`.
- **SpotAllocation** (*interface*) — decides which free spot a vehicle gets.
- **PricingPolicy** (*interface*) — turns a duration and a size into an amount.

Seven. Two of them are interfaces, and both sit exactly where change will arrive.

Notice what is *not* there. No `ParkingLotManager`, because a class whose name ends in “Manager” usually means you could not say its responsibility in one line. No `TicketFactory`, because there is one kind of ticket.

Move 4 • The diagram (minutes 13–22)

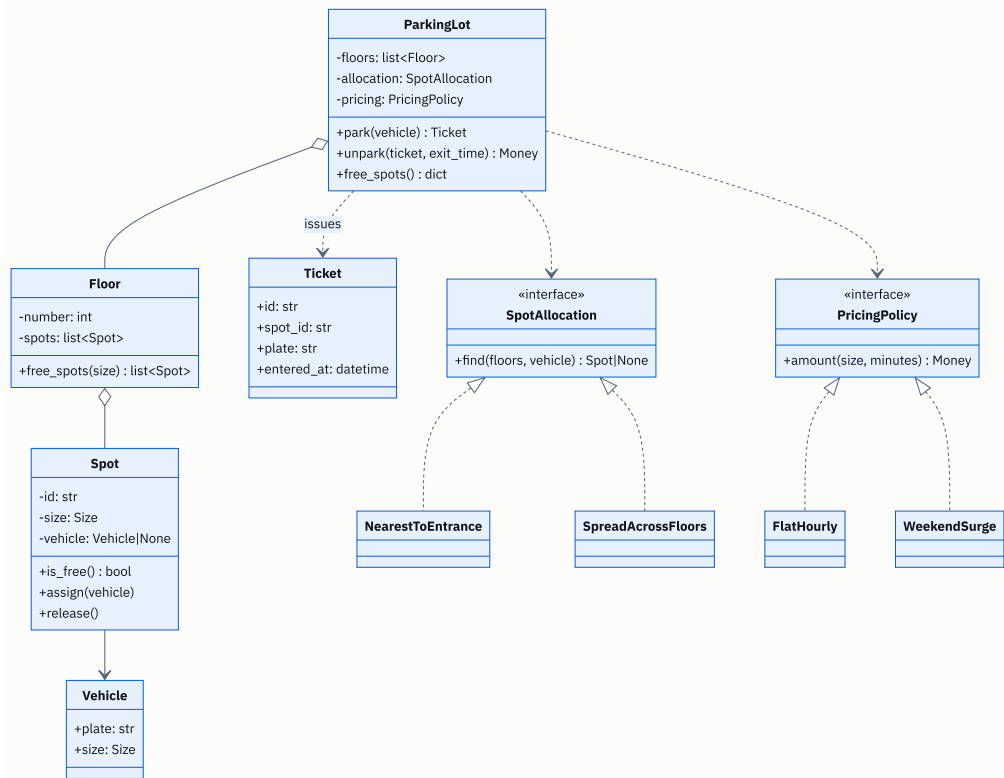


FIGURE 77.2

Narrate while drawing. “The lot holds floors, a floor holds spots, a spot may hold a vehicle. The lot delegates two decisions — which spot, and how much — and everything else is book-keeping.”

Move 5 • The interesting part (minutes 22–33)

Say version one first, and say what breaks.

“The simplest thing is `ParkingLot.park` scanning the floors for the first free spot of the right size. That is fifteen lines and it is correct. It stops being adequate the moment someone says ‘spread cars across floors so the lifts are not congested’, or ‘reserve floor 1 for electric vehicles’ — because then that method is making a policy decision inside a bookkeeping class, and every new rule edits it.”

Then the interface and two implementations.

```

from typing import Protocol

class SpotAllocation(Protocol):
    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None: ...

class NearestToEntrance:
    """Lowest floor first, then lowest spot number. What a driver expects."""

    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None:
        for floor in sorted(floors, key=lambda f: f.number):
            free = floor.free_spots(vehicle.size)
            if free:
                return min(free, key=lambda spot: spot.id)
        return None

class SpreadAcrossFloors:
    """Pick the emptiest floor, to keep lift and ramp congestion even."""

    def find(self, floors: list[Floor], vehicle: Vehicle) → Spot | None:
        candidates = [(f, f.free_spots(vehicle.size)) for f in floors]
        candidates = [(f, free) for f, free in candidates if free]
        if not candidates:
            return None
        floor, free = max(candidates, key=lambda pair: len(pair[1]))
        return free[0]

```

Two implementations of the same three-line interface. **This is the artefact that wins the round.**

Then the upgrade question, out loud: “One thing to notice — a small vehicle should be able to take a large spot when nothing smaller is free, but only then. That is a rule about *fallback order*, and I would put it in the allocation implementation rather than in `Floor`, because it is a policy and `Floor` is bookkeeping.”

Then the class that carries it, written properly:

```

class ParkingLot:
    def __init__(self, floors, allocation, pricing) → None:
        self._floors = floors
        self._allocation = allocation
        self._pricing = pricing
        self._open: dict[str, Ticket] = {}

    def park(self, vehicle: Vehicle, now: datetime) → Ticket:
        spot = self._allocation.find(self._floors, vehicle)
        if spot is None:
            raise ParkingFull(f"no free spot for a {vehicle.size.name}")
        spot.assign(vehicle)
        ticket = Ticket(id=new_id(), spot_id=spot.id, plate=vehicle.plate, entered_at=now)
        self._open[ticket.id] = ticket
        return ticket

```

Eight lines, and it contains **no rules** — only sequencing. That is what “the lot delegates the decisions” looks like in code, and it is worth pointing at when you finish typing.

Move 6 · Concurrency and extension (minutes 33–40)

Raise concurrency yourself, before you are asked. “Two cars at the barrier at the same instant can both be allocated the same spot, because `find` and `assign` are two separate steps. In one process, a lock around find-and-assign fixes it, and I would take the lock per floor rather than per lot so that two floors do not contend. Across processes, the allocation has to be a conditional update in the database — `UPDATE spots SET vehicle = ? WHERE id = ? AND vehicle IS NULL` — and zero rows updated means somebody else won and I retry with the next candidate.”

Then pre-empt the requirement they are about to add. They always add one. For a parking lot it is usually electric-vehicle charging bays, or monthly pass holders, or a reserved floor. Say: “If you added charging bays, `Size` becomes insufficient and I would give `Spot` a set of features and let the allocation policy filter on them — that change is one class, because the policy is already isolated.”

6 The numbers

The time budget, and what it costs to get it wrong

clarify + requirements	8 min	(20% of the round)
nouns	5 min	
class diagram	9 min	
the interesting part	11 min	(28% – the largest single block)
concurrency + extension	7 min	

total	40 min	

The two common distributions, and what they produce:

```
candidate A: 2 min clarifying, 25 min drawing every class in detail,
              5 min on allocation, no concurrency
              → a complete diagram of a system nobody asked for
candidate B: 8 min clarifying, 9 min diagram, 11 min on allocation,
              7 min on concurrency
              → a smaller diagram and a design that survives a changed requirement
```

Interviewers hire B. The diagram is not the deliverable; the **decisions** are.

How many classes

4 or fewer	responsibilities have been merged; one class does everything
6 to 10	right for a 40-minute prompt
12 to 15	acceptable only if several are tiny value objects (Money, Size, Plate)
18 or more	you are building a framework; this reads as inexperience

And the interface count:

```
0 interfaces    you described a system; you did not design one
1 to 2          correct – one per axis of change that actually exists
5 or more       every method is now three hops from its caller
```

Objects in memory, which they will ask

For a thousand-spot car park:

```
Spot      1,000 × ~120 B = 120 KB
Floor      5 × ~200 B = 1 KB
Ticket, one per parked vehicle, at most 1,000 × ~200 B = 200 KB
-----
total                                     ≈ 321 KB
```

A third of a megabyte for the whole car park. Say the number and then say what it means: **the entire model fits in memory, so there is no reason to design around a database for the object model.** That is a real conclusion drawn from arithmetic, and it is exactly what this section is for.

Contrast with a case where it does not: BookMyShow with 10,000 screens × 200 seats × 6 shows a day is 12 million seat records a day, and that one does not fit the same way.

The concurrency window, in real units

```
find a free spot      ~50 μs    (a scan of a few hundred spots)
assign it              ~1 μs
-----
window between them   ~51 μs

at 2 cars per second entering: collision probability per event ≈ 10-4
at 1,000 spots and 300 arrivals in a peak hour: about 1 collision per 3 hours
```

That is the number that turns “you should use a lock” into “here is why a lock is required”: a collision every three hours is a car parked in an occupied spot every three hours, which is a real incident, not a theoretical one.

The forty-minute reality

```
typical: 6-10 classes, 1-2 interfaces, ~120 lines of code written
realistic code you will actually type: 2-3 classes, ~40 lines
```

You will not write the whole system, and you should not try. Write the classes that carry the interesting part, and describe the rest.

7 The trade-offs

What the script costs you

It can sound rehearsed. If you run the moves as a recitation — “now I will identify the nouns” — it reads as a script rather than as thinking. Run the moves; do not narrate the names of the moves. The interviewer should notice only that you never seem stuck.

It can be the wrong order for the prompt. Some interviewers open with “just start coding the core class”. Follow them, and fold the missing moves in as you go: clarify while writing the constructor, name the interface when you reach the decision. **The script is a default, not a contract.**

It biases you toward one interface. Move 5 pushes you to find the interesting part and abstract it. On a genuinely simple prompt — “design a deck of cards” — the honest answer may be five classes and no interfaces at all, and inventing a `ShuffleStrategy` to satisfy the script is Ravi’s drill again. If you cannot name the second implementation, do not build the first.

Eight minutes of clarifying feels like eight wasted minutes. It is the highest-value part of the round and it never feels like it while the clock is running. Trust it.

What a low-level design round cannot tell them

It cannot tell them whether you can build the thing. Forty minutes of diagram is a sample of your judgement, not of your work. Which is exactly why the *reasoning* is scored so much more heavily than the diagram — and why a candidate who says “I would start with the simple version, and here is the signal that would make me change it” outperforms one with a more elaborate drawing.

“I would not follow this script if...”

- **...the interviewer has already given the requirements in writing.** Skip move 1 to a single confirming question and start at move 2. Asking questions they have already answered is worse than not asking.
- **...they ask for code immediately.** Some rounds are “implement an LRU cache” wearing a design hat. Code, and narrate the design decisions as you go.
- **...the prompt is genuinely a state machine** — vending machine, ATM, order flow. Then move 5 is the machine, and you should draw the state diagram *before* the class diagram, because the states determine the classes.
- **...there are twenty minutes, not forty.** Compress moves 2, 3 and 4 into one pass and keep move 5 intact. Never sacrifice the interesting part to finish the diagram.

The honest concession

The script does not make you a better designer. It makes your existing judgement *visible* inside a fixed time, which is what the round measures. Vinod is not a better tailor than Kishore. He finishes, every time, and he has attention left over when something unusual walks in.

8 In the interview

How it gets asked

- The standard opening: *“Design a parking lot.”* Or a lift, a vending machine, a library, a card game. Almost always with no further detail — the vagueness is deliberate.
- The narrowed version: *“Design the seat-booking part of BookMyShow.”* Here the interesting part has been named for you; go straight to it.
- The code-first version: *“Write the classes for a rate limiter.”*
- The mid-round curve ball, which is guaranteed: *“Now suppose we add electric vehicles / two lifts / a monthly pass.”*

What to say out loud, in the first ninety seconds

1. **Take control of the shape.** “Let me spend a few minutes on requirements, then draw the classes, then go deep on the part I think is actually interesting — probably spot allocation. Does that work for you?” Interviewers almost always say yes, and now the structure is agreed.
2. **Ask the four questions.** Multiple floors? Multiple sizes and can a small vehicle take a large spot? Payment on exit and time-based? How big?
3. **State the assumptions and invite correction.** “I will assume X, Y and Z — stop me if any of those is wrong.”
4. **Say the scope exclusions.** “I am not designing the payment gateway or the app.”
5. **Then start the nouns,** saying the responsibility of each in one line as you write it.

The follow-ups

“**You have thirty seconds. What is the core of this design?**” Name the interesting part, not the class list. “The core is spot allocation, because that is the rule most likely to change — it is behind an interface with two implementations, and the parking lot itself holds no rules.”

“**Two cars arrive at the barrier at the same instant.**” “They can both be allocated the same spot, because finding a free spot and claiming it are two separate steps with about fifty microseconds between them. At three hundred arrivals in a peak hour that is roughly one collision every few hours, so it is a real incident. In one process, a lock around find-and-claim, taken per floor rather than per lot so the floors do not contend. Across processes, a conditional update — claim the spot only if it is still free — and zero rows updated means retry with the next candidate.”

“**Now add electric-vehicle charging bays.**” “`Size` alone is no longer enough, so a spot gains a set of features and a vehicle gains a set of requirements. The allocation policy filters on features. That change touches one class — the allocation implementation — because the policy was already isolated. If I had left the rule inside `ParkingLot.park`, this would touch the busiest method in the system.”

“Why did you make that an interface and not the others?” “Because I can name the second implementation. There are two real allocation rules — nearest to the entrance, and spread across floors for lift congestion — and two real pricing rules — flat hourly and weekend surge. For everything else I cannot name a second implementation, so an interface there would be a hierarchy built for a future that has not arrived, and it would cost every reader an extra hop.”

“Is this over-engineered?” “It might be, and here is the test I applied: for each interface I asked whether I could name a second implementation that someone would actually want. Two passed and the rest did not, so there are two interfaces and everything else is a plain class. If you told me the allocation rule will never change, I would delete `SpotAllocation` and put the fifteen lines back inside `ParkingLot`.”

“You did not use a database anywhere.” “Deliberately — I read this as an object-model question. And the arithmetic supports it: a thousand spots plus tickets is about three hundred kilobytes, so the whole model fits in memory. If you want persistence, spots and tickets become tables, and the concurrency answer changes from a lock to a conditional update, which I mentioned. But I would not design the schema unless you want that.”

A model answer

Asked: *design a parking lot. You have forty minutes. Begin.*

“Before I start — let me suggest a shape for the forty minutes so you know where I am going. A few minutes on requirements, then the classes and how they relate, then I want to spend the largest block on the part I think is actually interesting, which for a parking lot is how you choose which spot a vehicle gets. Then concurrency at the end. Does that work?

Four questions. Multiple floors or one level? Multiple vehicle sizes, and can a car take a bus spot when nothing else is free? Is payment on exit and is it time-based? And roughly how many spots — a hundred or ten thousand? ...Right. Multiple floors, three sizes with fallback allowed, hourly payment on exit at a different rate per size, about a thousand spots.

So the assumptions I am working under: a ticket is issued at entry and settled at exit, one vehicle occupies exactly one spot, and I am designing the in-process object model rather than a database schema. Stop me if any of those is wrong. And I am explicitly not designing the payment gateway, number-plate recognition, or the app.

The nouns fall straight out. `Vehicle` — a plate and a size, nothing else. `Spot` — an id, a size, and the vehicle in it if any. `Floor` — holds spots, reports what is free. `Ticket` — links a vehicle to a spot with an entry time, and I would make it immutable once issued. `ParkingLot` — the entry point, with `park` and `unpark`.

That is five classes and it works, and I want to say why I am not going to stop there. If the whole rule for choosing a spot lives inside `ParkingLot.park`, then the first time someone says ‘spread cars across floors so the lift queue is shorter’, or ‘reserve floor one for electric vehicles’, that method gains a branch — and it is the method every car goes through. So the two decisions that will actually change get interfaces: `SpotAllocation`, which takes the floors and a vehicle and returns a spot or nothing; and `PricingPolicy`, which turns a size and a duration into an amount.

The test I applied for each is: can I name a second implementation somebody would really want? For allocation, yes — nearest-to-entrance, which is what a driver expects, and spread-across-floors for congestion. For pricing, yes — flat hourly and weekend surge. For the other classes I cannot name a second implementation, so they stay plain classes. Two interfaces, seven classes.

With that, `ParkingLot.park` is about eight lines and contains no rules at all — ask the allocation policy for a spot, raise if there is none, assign the vehicle, issue a ticket, record it. All the policy is somewhere I can change without touching the path every car takes.

One rule that is easy to put in the wrong place: a small vehicle may use a large spot, but only when nothing smaller is free. That is a fallback ordering, and it is policy, so it goes in the allocation implementation rather than in `Floor`. `Floor` is bookkeeping and should stay that way.

Concurrency, before you ask. Finding a free spot and claiming it are two separate steps, about fifty microseconds apart, so two cars at the barrier can be given the same spot. At a few hundred arrivals in a peak hour that is roughly one collision every few hours — a real incident, not a theoretical one. In one process I would hold a lock around find-and-claim, per floor rather than per lot so two floors do not contend. Across processes, the claim becomes a conditional update — take the spot only if it is still free — and zero rows updated means somebody else won and I try the next candidate.

On size: a thousand spots at roughly a hundred and twenty bytes each, plus tickets, is about three hundred kilobytes. The entire model fits in memory comfortably, which is why I have not designed around a database.

And if you added charging bays now, the change is one class: a spot gains a set of features, a vehicle a set of requirements, and the allocation policy filters on them. That is the payoff for isolating the decision, and it is the only reason I did it.”

9 Recall card

- **Six moves, always in this order: clarify · requirements and scope · nouns · class diagram · the interesting part · concurrency and extension.** Budget them **8 · 5 · 9 · 11 · 7** minutes. A third of the round goes before you draw a box, and that is correct. Announce the shape in the first thirty seconds and the interviewer will agree to it.
- **Every LLD prompt has exactly ONE interesting part — find it and spend a third of the time there.** The test: *if the interviewer changed one requirement, what would have to change?* Parking lot → allocation. Lift → scheduling. Vending machine → the state machine. BookMyShow → seat locking. Splitwise → minimising settlements. Cache → eviction.
- **Put an interface at the interesting part and show TWO implementations.** One is a class; two is proof the design survives change. The gate for every interface: **can you name a second implementation somebody would actually want?** If not, do not build the first. **6–10 classes and 1–2 interfaces** is right; 4 classes means merged responsibilities, 18 means you are building a framework.
- **Never present the design as if it appeared fully formed.** Version one, what breaks, then the fix. That is the only way an interviewer can tell an understood design from a memorised one.
- **Raise concurrency yourself at minute 33.** Find-then-claim is two steps with a real window — ~51 μs, about one collision every few hours at peak — so: a lock **per resource, not per system**, or a **conditional update** where zero rows updated means you lost and must retry. Then answer the requirement they are about to add, before they add it.

DSA topic: Stacks and queues revision and mock round **System design topic:** How to run a low-level design interview: the forty-minute script

Code these, in this order

One rule for the whole set: **solve each one out loud, standing up, on a clock**. Twenty-five minutes each, narrating. If you go silent for more than fifteen seconds, that is the thing to fix, not the code.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Simplify Path	LeetCode 71 (Medium)	Recognising a stack in a problem that never says “stack”, and the <code>..</code> versus <code>...</code> trap.
2	Basic Calculator II	LeetCode 227 (Medium)	The stack holding “the context I am leaving”, and precedence without recursion.
3	Remove K Digits	LeetCode 402 (Medium)	A monotonic stack used for construction rather than for lookup, plus three edge cases.
4	Design Hit Counter	LeetCode 362 (Medium)	“The last 300 seconds” as a queue, and the follow-up about a very large hit rate.

On every one of these, run the script

Restate. Ask about duplicates, `n`, and the empty input. State the brute force with a number. Say what the structure holds *before* the loop. Trace a degenerate example. Give the complexity with the counting. Do this even alone in your room — especially alone in your room.

On problem 1, find the traps yourself

Before running it, list every input you think will break your solution. `"/../"`, `"/home//foo/"`, `"/a/./b/../../../../c/"`, `"/..."`. Then run them. The `"/..."` one is the classic: it is a legal directory name, and `startswith("../")` treats it as a parent.

On problem 3, say what the stack holds

It is not “elements whose answer is unknown” this time. It is “the answer so far, kept as small as possible”. Say that sentence, then handle the three edge cases: `k` not fully used at the end, leading zeros, and an empty result.

On problem 4, do the follow-up honestly

“What if there are hundreds of hits per second?” The exact queue holds every timestamp. Say what you would keep instead, and what precision you are giving up to get $O(1)$ memory.

The recognition drill

For each, say in five seconds which of the four trigger sentences applies, and name the structure:

1. Whether a string of brackets is balanced.
2. The maximum in every window of size k .
3. How many days until a warmer temperature.
4. Print a tree level by level.
5. Get the minimum of a stack in $O(1)$.
6. The number of requests in the last 300 seconds.
7. The largest rectangle in a histogram.
8. The shortest path in an unweighted grid.
9. Evaluate an expression with brackets.
10. A cache that evicts the least recently used entry.
11. Decode `3[a2[c]]`.
12. Implement a queue with only stacks.

Then group all twelve under the four trigger sentences and say which sentence covers the most.

The say-it-first drill

For each, write the single sentence you would say *before* writing any loop:

1. Balanced brackets.
2. Next greater element.
3. Largest rectangle in a histogram.
4. Sliding-window maximum.
5. Min stack.
6. LRU cache.
7. Queue from two stacks.
8. Rate limiter over a rolling window.

Every sentence should start with “the stack holds”, “the deque holds”, “the map answers ... and the list answers ...”, or “each element is touched ...”.

The complexity-defence drill

Say each answer out loud, five times, until it is automatic:

1. “There is a loop inside a loop. Is that not $O(n^2)$?”
2. “Pop can move n elements. How is that $O(1)$?”
3. “You loop $2n$ times for the circular version.”
4. “Your rate limiter has a `while` inside `allow`.”

5. “Your LRU `put` calls three helper methods.”

Each answer must contain a count, not an assertion.

The number-recall drill

Quote each from memory, then check:

1. `list.pop(0)` versus `deque.popleft`, draining 100,000 elements.
2. What happened when n doubled from 100,000 to 200,000, and what that proves.
3. Brute force, heap and deque for sliding-window maximum at $n = 200,000$, $k = 1,000$.
4. Monotonic stack against brute force at $n = 100,000$.
5. `queue.Queue` against `deque` in a single thread.
6. Bytes per entry in a Python LRU cache, and the total for 100,000 entries.

The break-it drill, phase-wide

Trigger each and record the exact error text or the wrong output:

1. `pop(0)` in a queue solution, timed at $n = 200,000$.
2. A `while` condition with no `stack` and.
3. `is_empty` checking one of the two stacks.
4. Pouring onto a non-empty outbox.
5. `head = tail` used as “empty” on a full ring buffer.
6. `width = index - left` in the histogram.
7. An LRU `get` that does not reorder.
8. Removing from the list but not the map.
9. Recording a rejected request in the rate limiter.
10. `window[0] < index - k` on `[8, 3, 2]` with $k = 2$.

The mock round

Set a timer for forty-five minutes. Pick two problems you have never seen from the stack-and-queue tag on LeetCode. Solve both out loud, recording yourself if you can bear it. Then, listening back, mark every place where you:

- wrote code before saying what the structure held,
- said a complexity without counting,
- went quiet for more than fifteen seconds,
- fixed a bug the interviewer would have found first.

Those four marks are your actual practice list. The problems were only the excuse.

The script drill

Run all six moves, out loud, on each of these, in eight minutes each. Do not write code — only speak.

1. Design a parking lot.
2. Design a vending machine.
3. Design a library management system.
4. Design a deck of cards.

For each, say: the four clarifying questions, the scope exclusions, the nouns with one-line responsibilities, and — most importantly — **the one interesting part**.

The interesting-part drill

For each prompt, name the single place where the design is actually decided, and the second implementation you would show:

1. Parking lot.
2. Elevator system.
3. ATM.
4. Vending machine.
5. Library management.
6. Tic-tac-toe.
7. Splitwise.
8. BookMyShow.
9. Food delivery.
10. Ride hailing.
11. Rate limiter.
12. In-memory cache.
13. Logging framework.

Two of the thirteen have a state machine as their interesting part. Name them, and say what you would draw before the class diagram in those two cases.

The interface-gate drill

For each proposed interface, say whether you would build it, and name the second implementation or admit you cannot:

1. `SpotAllocation` in a parking lot.
2. `TicketFactory` in a parking lot.
3. `PricingPolicy` in a parking lot.
4. `VehicleRepository` in a parking lot.
5. `ShuffleStrategy` in a deck of cards.

6. `EvictionPolicy` in a cache.
7. `WinCondition` in tic-tac-toe.
8. `Formatter` in a logging framework.

Three of the eight fail the gate. Name them and say the one-sentence rule that rejects them.

The concurrency drill

For each, describe the two-step operation, the size of the window between the steps, the failure it produces, and both fixes:

1. Two cars at the barrier, one free spot.
2. Two users clicking the same seat.
3. Two workers shipping the same order.
4. Two threads calling `get` on the same LRU cache.
5. Two customers taking the last item from a vending machine.

For each, say whether a lock or a conditional update is the better answer, and why.

The over-engineering drill

Take your parking lot design and deliberately ruin it. Add `ParkingLotFactory`, `AbstractTicketBuilder`, `SpotAllocationStrategyRegistryProvider` and an observer for every event. Then:

1. Count the classes and the files.
2. Count the hops from “a car arrives” to “a spot is chosen”.
3. Write the sentence a reviewer would write on this pull request.
4. Delete everything that fails the interface gate and count what is left.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Two problems, no hints, talk as you go.* Run the whole script on an unseen stack problem: restate, clarify three things, brute force with a number, the observation before the structure, what the structure holds, the code narrated, a traced degenerate example, and the complexity with the counting.
2. *Design a parking lot. You have forty minutes. Begin.* The shape announced in the first thirty seconds, four clarifying questions whose answers change the design, assumptions and scope exclusions, nouns with one-line responsibilities, version one and what breaks it, the interface with two implementations, and concurrency raised before it is asked.
3. *There is a loop inside a loop. Is that not $O(n^2)$?* n pushes total, each element popped at most once, $\leq 2n$ operations, bounded in total not per iteration, plus the example where one iteration pops a hundred thousand elements and why it costs nothing.

Before you move on

- ☐ I can name all four trigger sentences and classify twelve problems with them.
- ☐ I can say what the structure holds, in one sentence, for eight different problems.
- ☐ I can give all five complexity defences with a count rather than an assertion.
- ☐ I can quote six real measurements from the phase from memory.
- ☐ I triggered all ten phase traps and recorded the exact error text.
- ☐ I ran a timed forty-five-minute mock on two unseen problems, out loud.
- ☐ I listened back and marked the four failure types.
- ☐ I can name the six moves of the LLD script and their minute budgets.
- ☐ I ran the script out loud on four different prompts.
- ☐ I can name the interesting part of thirteen standard prompts.
- ☐ I can state the interface gate and apply it to eight proposed interfaces.
- ☐ I can raise concurrency unprompted, with a window size and both fixes.
- ☐ I over-engineered a design on purpose and can say exactly what a reviewer would object to.
- ☐ I answered all three questions above out loud.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but `50M x 20 x 2KB = 2TB`.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Design patterns

Day 063 · system-design — What a design pattern actually is

- **A pattern is a *name* + a recurring *problem* + a *solution* + its *consequences*.** The consequences are the half people skip — the original book spends ~17 pages per pattern, and most of it is costs. Patterns are **discovered, not invented**: Alexander's buildings in 1977, the Gang of Four's **23 patterns** in 1994.
- **Three groups, three questions.** Creational (5) = *how does this get made?* · Structural (7) = *how are these put together?* · Behavioural (11) = *how do these talk?* Eight of the 23 are almost all you will meet: singleton, factory, builder, adapter, decorator, facade, strategy, observer.
- **Go from pain to axis to pattern, never from pattern to code.** Name the specific pain, name the thing that keeps changing, and the pattern picks itself. Wait for the **second occurrence** — you cannot see the seam until something has moved.
- **In Python the shape survives and the ceremony often does not.** Strategy = a function · command = a closure · iterator = `yield` · singleton = a module. Norvig, 1996: ~16 of 23 are invisible or trivial in a dynamic language. `sorted(key=)`, `lru_cache`, `requests.get`, the DB-API and nginx are all patterns you already use.
- **Quote both numbers.** Full Strategy classes cost ~80 extra lines and 4 extra files over an `if` chain, and buy a flat cost per new variant (the tenth costs what the second cost). Shared vocabulary saves a 6-person team ~4 hours a quarter. And 9 factories with one implementation each are ~200 lines of costume — **a pattern name is not a class name**.

Day 064 · system-design — Singleton

- **Two promises, and only the second is the problem.** *Exactly one instance* is often a real requirement; *globally reachable* buys hidden dependencies, shared mutable state, no substitution, and a DIP violation. **A singleton is a global variable with a constructor.**
- **The race is check-then-create.** Fixes in order: eager (always pay) · lock every call (~50 ns forever) · **double-checked locking** (~2 ns after the first) — and in Java it was *broken until Java 5* because the reference could be published before the fields were initialised; you need `volatile`. In Java write an `enum`; in Python a **module-level object** or `@cache` on a factory. Never override `__new__` alone — `__init__` **re-runs on every call** and resets state.

- **Say the distinction: needing one instance ≠ using Singleton.** Create one at the **composition root** and inject it. Same uniqueness, visible dependencies, one-line test substitution. Spring's beans are *singleton-scoped* but injected — it kept the good half.
- **It is one per process, not per system.** 8 Unicorn workers = 8; 4 containers × 8 = 32. A cache becomes 32 disagreeing caches; a 100/sec rate limiter becomes 3,200/sec. True system-wide uniqueness lives in Redis or Postgres.
- **The legitimate cases are process-wide, expensive, and immutable after startup:** connection pools (Postgres `max_connections` = 100 vs 500 req/sec), metrics registries, logging config, thread pools. Django's `override_settings` decorator exists *because* settings is a singleton — that decorator is the cost made visible.

Day 065 · system-design — Factory and abstract factory

- **Three different things are called “factory”, and separating them is the answer.** *Simple factory* — a function with a branch, **not a GoF pattern**, and what you should usually write. *Factory method* — a **subclass** overrides a creation method; the base algorithm never learns the product. *Abstract factory* — one object produces a **family** that must not be mixed.
- **In Python, write a registry:** `SENDERS: dict[str, type[Sender]]`, then `SENDERS[channel]()`. Classes are objects. A decorator registry drops it to **1 new file, 0 edits** — at the cost of the live list being invisible if a module is never imported.
- **Three things leave the business logic:** the branch (9 call sites → 1), the construction details (keys, hosts, timeouts), and the **vendor import** — `grep -rn "twilio" orders/` returning nothing is the proof the dependency was inverted.
- **Abstract factory's grid is asymmetric, and volunteer it.** A new **family** is 1 file and 0 edits; a new **product type** is the interface plus *every* factory — 9 files for 4 families. Iqbal's twelfth school was one specification; the blazer was eleven. Use it only when families arrive more often than types, and only when you can **name the family**.
- **Do not write one if** there is one implementation and no test double (9 of 14 `*Factory` classes = ~200 lines of costume) · the variation is data, not behaviour · a **named constructor** (`Sender.for_channel`, `datetime.fromisoformat`) would do. And the cost you accept is that `'emial'` becomes a **runtime error** — so type the key as an enum.

Day 066 · system-design — Builder

- **Builder separates collecting the pieces from creating the object.** The product comes out complete and immutable; the messy half-built state lived in the builder and never escaped. Fluent version = one method per field, each `return self`, then `build()`.
- **`build()` is the point, not the chaining.** It is the one moment where the whole object exists and has not escaped, so it is the only place **cross-field invariants** can be checked — “a photo print needs a fondant top” involves two fields, so **no setter could ever catch it**.

- In Python, start with `@dataclass(frozen=True, kw_only=True)` and defaults — ~14 lines gives named args in any order, immutability, required fields still enforced at construction, and validation in `__post_init__`. Java needs Builder because it has no keyword arguments and no default values. A hand-written Python builder is ~45 lines and usually buys nothing.
- Four cases where you still write one: construction is genuinely **stepwise** · steps are **conditional** · assembly is expensive · **test data** — `an_order().for_customer("ravi").build()` turns 18 lines of setup (2 of them relevant) into 1, and a new field is 1 default instead of 60 edited test files.
- **State the costs:** required fields become a **run-time** check, not a compile-time one · the field list is **duplicated** on product and builder · the builder is mutable and not thread-safe. And ask first: **are these twelve parameters twelve things, or three?** `line1/line2/city/pincode` is a **data clump** — one `Address` — and extracting value objects often removes the need entirely. n optional fields = 2^n telescoping constructors; 12 fields = 4,096.

Day 067 · system-design — Prototype, and cloning objects

- **Shallow** = new outer object, fields point at the *same* inner objects. **Deep** = the whole reachable graph is copied, nothing shared. And `b = a` is not a copy at all — it is a second name. Draw the boxes and arrows; it beats sixty seconds of talking.
- The real rule is not “always deep copy”. *Shallow is safe exactly when everything shared is immutable*. So the best answer is usually to remove the question: frozen dataclasses, tuples, and `dataclasses.replace` for “the same but one field different”.
- Three things deep copy gets wrong, and volunteer all three. Cycles — handled by a `memo` keyed on `id()`, which also *preserves the shape of sharing* · **cost** — $\sim 1,200\times$ slower and $\sim 100\times$ the memory on a 1,000-key dict of 100-element lists, so `deepcopy` in a request handler is a real bug · **resources** — sockets, file handles, connections, locks. `TypeError: cannot pickle '_thread.lock' object` is the error you will actually see; the fix is an explicit `__deepcopy__`.
- **Prototype pays only when construction is expensive**. 56 ms to build (DB read + parse + HTTP) vs 0.4 ms to clone = $140\times$. With a cheap constructor, cloning is *slower*. And the prototype itself must be treated as immutable, or every future clone changes silently.
- **You have already used it**. Docker images → containers · `fork()` → child process · Kubernetes pod templates · `numpy` views (`arr[1:]` shares memory) · JS `{...spread}` is shallow, hence `structuredClone`. All the good ones use **copy-on-write**: share until someone writes. And Java’s `Cloneable` is the cautionary tale — marker interface, protected method, no constructor call, an impossible checked exception; use a **copy constructor** instead.

Day 068 · system-design — Adapter

- **Three parties: client, adaptee, adapter.** The adapter implements the interface the client wants and forwards to an adaptee that **cannot be edited**. Neither side changes; neither side knows.
- **An adapter translates four things, and only the first is obvious.** The **interface** (names) · the **data shape** (their `dict` → your `Payment`) · the **units and types** (cents→paise, epoch→aware datetime, `"IND"` → `"IN"`) · the **errors** (`stripe.error.CardError` → `PaymentDeclined`). The last three are the voltage — the plug fitting is not the same as it working.
- **The interface lives in the *consuming* package, in *your* vocabulary.** A protocol whose signature mentions a vendor type has inverted nothing and looks identical on a diagram. `typing.Protocol` is structural, so ~6 lines and the provider is never edited. Proof: `grep -rn "stripe" orders/` returns nothing.
- **The test that it is a real adapter: can you write a fake with nothing vendor-shaped in it?** If the fake needs a Stripe-looking dict, you adapted the name only. That fake is also the payoff — 4 lines and 0.3 ms against 35 lines and 800 ms of flaky sandbox.
- **Adapter changes the interface · decorator changes behaviour (and stacks) · facade changes how many things you talk to · proxy changes when you reach the real one.** Use the **object** adapter (composition), not the class adapter (multiple inheritance leaks the whole adaptee). At whole-system scale it is an **anti-corruption layer**. And concede the limit: it buys **source-level independence, not a free swap** — but the differences land in one file, not fourteen.

Day 069 · system-design — Decorator

- **Same interface, wraps one object, forwards, adds behaviour — and *therefore* stacks.** The unchanged interface is what makes composition possible; that is the whole pattern.
- **Kill inheritance with arithmetic:** `2n` subclasses vs `n` decorators. 3 features = 8 subclasses (with duplicated logic) vs 3 classes and one composition line; 7 features = **128 vs 7**. And an eighth feature is 1 new class and 0 edits.
- **Order matters, because a layer only sees what gets past the layer outside it.** Cache *outside* logger ⇒ hits are never logged (dashboard says 600 when the app made 4,000). Retry *outside* timeout ⇒ a budget *per attempt*; timeout *outside* retry ⇒ one *shared* budget. **Auth is always outermost** — a cache in front of an auth check serves cached data to someone not allowed to see it.
- **Python's `@decorator` is the function-level version, not the pattern.** It is fixed at import time; the object version is a **run-time** decision (`if settings.cache_enabled:`, per instance, per tenant, off in tests). Always `functools.wraps`, or `__name__` and `__doc__` are lost and introspection breaks. `__getattr__` forwards the 14 methods you do not care about, at the cost of type checking.

- **Adapter changes the interface (cannot stack) · decorator keeps it and adds behaviour (stacks) · proxy keeps it and controls access (usually one) · facade changes how many things you talk to.** Costs to concede: ~11 stack frames instead of 3 · `isinstance` and identity break · ~120 ns per layer (nothing against a 2 ms query, 5 s across 10M calls) · and the composition line becomes load-bearing, so **test the order**.

Day 070 · system-design — Facade and proxy

- **Facade = a new, simpler interface over many things. Proxy = the same interface over one thing, controlling access.** Two questions separate all four wrappers: *is the interface the same?* (same → decorator/proxy; different → adapter/facade) then *how many, and why?*
- **What a facade removes is the ordering and the failure policy, not the services.** Reserve before charging; notify last. 9 callers × 6 services = **54 edges** → **15**, and one place to put the compensating refund, the idempotency key or the outbox.
- **Two rules keep it from becoming a god object: it may sequence and translate but never make business decisions, and it must not seal the subsystem** (reporting still queries orders directly — otherwise every new need widens it, one method at a time). Measure it with `git log`, not opinion.
- **One layer up, a facade is an API gateway — and one per audience is a BFF.** Mobile: 6 round trips × 120 ms = **720 ms** and ~40 KB, versus 1 call + in-datacentre fan-out = ~125 ms and ~8 KB.
- **Proxy has four kinds — virtual (lazy), remote, protection, caching — and its invisibility is both the point and the hazard.** ORM lazy fields make N+1 easy to write (151 ms vs 4 ms, ~38×); remote stubs make a timeout-able call look local; Spring's `@Transactional` does **nothing** on self-invocation because that call never goes through the proxy. Structurally a proxy and a decorator are identical — **the difference is intent and multiplicity**, and you should say so rather than invent one.

Day 071 · system-design — Strategy

- **Strategy turns an algorithm into a value.** One interface, one method, one implementation per way, and the caller never learns how many there are. You have used it since day 058: `sorted(items, key=...)` is Strategy.
- **Argue it in edit counts, not adjectives.** New market with an `if` chain: 1 shared file (12 dependents), ~40 tests re-run, 3 live rules at risk. With strategies: **1 new file + 1 registry line, 0 tests re-run** — and the twelfth costs what the second cost.
- **Choosing is a separate job from doing.** The selection lives in a registry or factory, never in the caller — otherwise the branch is back and the interface is doing nothing. And **concede that the `if` moved rather than vanished**: a wrong line in a list of constructors cannot break Diwali pricing, and a new branch in a shared function can.

- **In Python a function is a strategy.** A `dict[str, Callable]` is ~12 lines against ~90 for the full class version. Take classes when a rule needs **state**, **several methods**, or **its own owner**. Real ones: compression codecs · load-balancing algorithms · cache eviction policies · retry backoff · `hashlib.new` · Django's `PASSWORD_HASHERS`.
- **The commonest misuse: the variation is data, not behaviour.** * *Three tax rates as three classes is ~40 lines to hold three literals — the test is* does the code differ, or only a value? *Also leave the chain alone when the set is closed, when it is the first occurrence, or when you cannot name the second implementation.* *Strategy vs State = same structure, but State transitions *itself*; Strategy vs Template Method = composition vs inheritance.

Day 072 · system-design — Observer

- **One subject holds a list of observers behind one narrow interface, and `notify` loops the list.** The subject knows the interface and **nothing about the concrete listeners** — that missing arrow in the class diagram is the pattern. Adding the sixth listener is a new file plus one line in the composition root; checkout is not edited or re-tested.
- **Four rules for the implementation:** save first and **notify after** (or you email about an order that does not exist) · publish an **immutable event object**, so listener 3 cannot change what listener 4 sees · iterate over a **copy** of the list, or a self-detaching observer skips the next one · wrap **each** call in its own try/except, and then log and alert, or you have built a machine for losing work silently.
- **The honest concession: in-process Observer buys decoupling, not speed.** The five calls still run one after another and the cost is the **sum** — 25 ms of save plus 40 + 120 + 90 + 60 + 30 = **365 ms** the customer waits. Publishing to a broker instead is **~27 ms and stops growing**. Async is a *separate* decision from the pattern.
- **Observer → pub/sub is where the list moves out of the subject and into a broker.** `addEventListener` · Django signals · Spring `@EventListener` · Redis pub/sub (**fire-and-forget, lost if offline**) · Kafka (**durable, replayable, redelivered**) · Postgres `LISTEN/NOTIFY` (8 KB payload cap) · webhooks. `java.util.Observable` was **deprecated in Java 9** for being too weak an interface.
- **Do not use it if the order of steps matters** (that is a workflow), **if it must be all-or-nothing** (that is a transaction or a saga), **if the listener must return an answer**, or **if there is only one listener**. Watch for the **lapsed listener leak** — detach in a `finally`, or hold observers in a `weakref.WeakSet`.

Day 073 · system-design — State

- **State = one class per stage, all behind one interface, and the context delegates every call to the one it currently holds.** `Order.cancel()` is `self._state.cancel(self)` and the order holds **no rules at all** — which also gives you exactly one place for the audit row, the metric and the published event.

- **The base class refuses everything; each state overrides only what it allows.** So a new state fails **closed**, and 22 of the 30 (state, event) pairs are handled by writing nothing. Compare **36 branches across 6 methods with 7 methods across 6 files** — adding a state is **one new file, zero edits**.
- **State versus Strategy: identical diagram, opposite direction.** *The client chooses a strategy; the object replaces its own state.* **States know their successors** — the tell is `order.state = ShippedState()` inside an implementation. Strategies never reference each other.
- **Three legitimate forms, and the choice is about content:** classes when states carry real work (the refund inside `PaidState.cancel()` · a `(state, event) → state` **dictionary** when only the arrows matter, because the whole machine then fits on one screen · an enum plus `match` for tiny machines. Persist the state as a **string with a CHECK constraint**, never a pickled object.
- **The pattern gives you no concurrency guarantee, and this is the follow-up that separates people.** Two workers can both legally ship the same paid order. Enforce it in the database: `UPDATE ... WHERE id = ? AND status = 'paid'`, and **0 rows updated means you lost the race**. Real machines: TCP, Stripe `PaymentIntent`, Kubernetes pod phases, `django-fsm`, Step Functions, XState.

Day 074 · system-design — Command and chain of responsibility

- **Command = stop calling the operation, start creating it.** Two methods, `execute` and `undo`. Once the action is an object you get four things you did not have: **queue it** · **undo it** · **log it** · **replay it**. Celery/Sidekiq jobs, Postgres WAL, Redis AOF, `git revert`, event sourcing are all this.
- **The number that wins the undo question: snapshots store state, commands store the difference.** 1 MB document × 500 levels = **500 MB**; the same history as commands ≈ **0.5 MB** — about **1,000×**. Cap the depth and drop the oldest; snapshot every ~200 commands so a deep undo is a restore plus a short replay.
- **Two stacks, and the rule people forget: executing a new command clears the redo stack** — history has forked. **Delete is the interesting command**, because it must capture the removed text during `execute` or undo loses it. A **macro command** is a command holding commands, and its `undo` runs the children **in reverse**.
- **Chain of responsibility = each handler knows two things: what it can handle, and who is next.** Handling **stops** the walk — if every handler must run it is not a chain. **Always terminate the chain deliberately**, or a request is approved by nobody. Real chains: Django/Express middleware, Spring Security filters, Python `logging` propagation, `iptables` with a default policy, DOM event bubbling.
- **Order is a hidden contract and it costs money: cheapest rejection first.** An 8-stage pipeline is ~**4.1 ms** before your handler runs; rate-limit before auth or a 50,000 req/s attack burns 40 core-seconds a second on JWT verification. **Boundaries to name:** actions

with no inverse (compensate, do not undo) · `execute` must be repeatable for redo · two users need **OT or CRDTs**, not a bigger undo stack · **Memento** beats Command when state is small.

Day 075 · system-design — Template method and iterator

- **Template method: the base class owns the ORDER in one method that must not be overridden, and leaves blanks.** Three kinds of step — **fixed** (base implements, do not override) · **abstract** (`@abstractmethod`, subclass must implement, forgetting is a `TypeError` at construction) · **hook** (base implements as a no-op, subclass may override). “Don’t call us, we’ll call you.”
- **The argument is not lines, it is edit count.** 373 → 179 lines is weak; *a bug in the shared step is 1 edit instead of 3*, and the real failure is **the fix applied to 2 of 3 files, unnoticed for a quarter**. Adding a fourth report = one file, two methods. The six extra method calls cost **under half a microsecond** — never a performance argument.
- **Template method vs Strategy: inheritance vs composition, fixed at subclass-writing time vs swappable at run time.** Use Strategy the moment **two things vary independently** (three formats × three delivery channels) or the choice must change while running. Python has no `final`, so the “do not override” is convention — a real weakness worth admitting.
- **Iterator separates walking from the collection, so `for row in source:` never changes when the source does.** `__iter__` must return a **fresh** position each time. A **generator is one-shot** — iterating twice silently gives zero the second time — so type-hint it `Iterator[Row]`, not `list[Row]`, and remember it can hold a cursor open.
- **The memory number: 10M rows × 200 B = 2 GB loaded, vs 1,000 × 200 B = 200 KB streamed — 10,000×, and the failure looks like exit code 137 with nothing in the log.** Page with a **keyset** (`WHERE id > last_seen`), never `OFFSET`: paging 10M rows costs $\sim 5 \times 10^{10}$ row reads with `OFFSET` against 10^7 with a keyset. Already in your hands: `unittest`, Django CBVs, `JdbcTemplate`, servlet `doGet` · every `for` loop, `itertools`, S3 continuation tokens, Kafka consumers.

Day 076 · system-design — Design patterns revision and interview questions

- **Name the axis of change before naming a pattern, and say how often it changes.** Five moves for any prompt: *restate the axis · say the naive version and what breaks · name the pattern and what it makes cheap · name the cost · name the condition for not doing it*. The reasoning scores higher than the choice.
- **The wrapping four, told apart by two questions: does the caller’s interface change? —** Adapter and Facade **yes**, Decorator and Proxy **no**; *adding behaviour or controlling access? —* Decorator **adds and stacks**, Proxy **controls** (lazy, remote, permission). Facade = a **new smaller** interface over several subsystems; Adapter = **one** class behind the wrong interface.

- **The behavioural look-alikes, one line each.** Strategy vs State: *the client picks a strategy; the object replaces its own state.* Strategy vs Template Method: *composition and run-time vs inheritance and fixed.* Observer vs Chain: *everyone gets it vs the first able handler stops the walk.* Command vs Strategy: *if you need `undo`, it is Command.* Factory vs Builder: *which class vs fifteen optional fields.*
- **The economics: ~65 extra lines and 4 extra files for a three-case rule, and the payoff is flatness** — the twelfth case costs what the fourth cost, while the `if` chain gets worse for ever. **Rule of three.** Getting it wrong in month 2 costs a day; in month 14 it costs a quarter or never gets undone — so **under-apply rather than over-apply.** Runtime cost is ~60 ns per hop: **never a performance argument.**
- **“No pattern” is a real answer and often the right one:** one variation today · only a **value** differs, not a behaviour · a closed set · you cannot name the second implementation. In Python, Strategy is often a **dict of functions** (12 lines vs 90), Command without undo is `functools.partial`, Singleton is a **module**, Iterator is `yield`. The strongest sentence in the whole phase: *“I would write the `if` today, and the third provider is the signal that makes me extract it.”*

Hashing: maps and sets

Day 060 · dsa — Hash tables: how a dictionary finds anything instantly

- **It does not search, it computes:** `bucket = hash(key) % capacity`, then a direct index. That one line is the whole structure. A hash function must be **deterministic** (find it again), **uniform** (spread the keys), and **fast** (it runs on every operation) — and Mr Shenoy’s first-letter rule failed only on uniformity.
- **Collisions are guaranteed** (more keys than buckets). **Chaining** = each bucket holds a short list, so a lookup checks 1-2 entries — which is why the **key is stored alongside the value**. Python uses **open addressing** instead ([day 061](#)).
- **O(1) average, O(n) worst case.** Average = 4 operations independent of n, resting on a uniform hash *and* the table growing. Worst = everything in one bucket: n comparisons per operation, O(n²) to build — the real hash-DoS attack, defended by Python’s per-process **hash randomisation** (so never persist a hash).
- **Resize when the load factor passes ~0.66:** double the buckets and **rehash everything**, because `% 8` and `% 16` differ. That insert is O(n), but $8 + 16 + 32 + \dots + n/2 \approx n$, so it is **amortised O(1)** — with a real tail-latency spike on the unlucky insert.
- **Keys must be hashable = effectively immutable.** Hash on fields that **never change**, or the entry becomes unfindable while sitting in the table (`obj in d` → `False`, no error). And the absolute contract: `a == b` **implies** `hash(a) == hash(b)`. `@dataclass` without `frozen=True` sets `__hash__` to `None`; `d.values()` search is **O(n)**; deleting while iterating raises `RuntimeError: dictionary changed size during iteration`.

Day 061 · dsa — Collisions, and why a hash map can turn slow

- **Collisions are guaranteed and normal** — more keys than buckets, and they arrive early (birthday problem: 10^6 buckets, $\sim 1,178$ keys $\rightarrow$ 50%). They cost **time**, **never correctness**, as long as `__eq__` distinguishes the keys.
- **Chaining**: a list per bucket. Deletion trivial, load factor may exceed 1, degrades *gently* (~ 1.5 probes even at load 0.99), but scattered memory. **Java 8+ turns a chain of 8 into a tree** $\rightarrow O(\log n)$. **Open addressing**: probe for another slot. Denser, cache-friendly — what **Python and Go** use — and it falls off a **cliff** (1.5 probes at load 0.5 $\rightarrow$ 2.5 at 0.75 $\rightarrow$ 10.5 at 0.95 $\rightarrow$ ~ 50 at 0.99), which is why everyone resizes at ~ 0.66 .
- **You cannot delete by blanking a slot**. A lookup stops at the first EMPTY slot, so anything that probed past becomes **unreachable with no error** while `len()` still counts it. Use a **tombstone** — empty / occupied / deleted — which lookups skip and inserts may reuse.
- **Tombstones then accumulate**: each costs a probe and holds nothing, so **resize on `entries + tombstones`, not on `len`** — a *shrinking* map can need a rehash.
- **Three real causes of $O(n)$** : a hash using too little of the key (`hash(order_id[:4])`) · integer keys stepping by exactly the capacity · **an adversary** — 20,000 colliding form fields turned 20,000 operations into 200,000,000 in the 2011 hash-flooding attacks. Hence Python's per-process **hash randomisation**: never persist, transmit, or assert on a `hash()` value — use `hashlib`.

Day 062 · dsa — Sets: membership, deduplication, and the $O(1)$ habit

- **A set is a hash table with the values deleted**. `in` is $O(1)$ average, duplicates are impossible, order does not exist, elements must be hashable — so tuples and `frozenset`, never lists or dicts.
- **The seen-set pattern is the reflex**: walk once, `if x in seen`, then `seen.add(x)`. The invariant is `seen` holds exactly the elements before the current one. It turns 50,000,000 comparisons at $n = 10,000$ into 20,000 operations.
- `seen = []` **is the trap** — `in` on a list is $O(n)$, so identical-looking code runs $700\times$ slower at $n = 20,000$. And `{}` is an empty dict; the empty set is `set()`.
- **Longest consecutive: put it in a set, and only walk from a number whose `number - 1` is absent**. Without that guard `range(100000)` is 5×10^9 steps. The `while` is bounded in total, not per iteration — n outer checks plus n total inner steps.
- **You are trading memory for time** — $O(n)$ extra space, roughly 32 MB per million integers. Asked for $O(1)$ space, sort instead ($O(n \log n)$) or mark in place when the range is known. Real errors: `TypeError: unhashable type: 'list'`, `RuntimeError: Set changed size during iteration`.

Day 063 · dsa — Counting with dictionaries

- **Three lines, and know all four spellings:** `counts[x] = counts.get(x, 0) + 1` · the explicit `if` · `defaultdict(int)` · `Counter(items)` . Building it is $O(n)$ time, $O(m)$ extra space, where m is the number of *distinct* values — not n .
- `counts[x] += 1` on a missing key raises `KeyError` . And `defaultdict` creates the key when you merely read it, so `len()` silently grows; `Counter` returns `0` for anything, so typos never raise. Check membership with `in` , never by indexing.
- **Top k has three answers:** sort $O(m \log m)$ · heap $O(m \log k)$ · bucket by count $O(n + m)$. At $n = 10^6$, $m = 10^5$, $k = 10$ that is 1.7M vs 330K vs 1.1M. **Buckets beat the $n \log n$ bound because they do not compare** — a count is an integer in $1..n$, so you index by it.
- **Say the trade, not just the winner:** buckets allocate $n + 1$ slots regardless, so with n huge and m tiny the heap wins. And the `len(result) == k` check goes *inside* the inner loop.
- **Two $O(n^2)$ traps that look clean:** `words.count(word)` inside a loop (400M ops at $n = 20,000$), and walking the counter instead of the original string for a “first such” question. Real errors: `KeyError` , `RuntimeError: dictionary changed size during iteration` , and `TypeError: unsupported operand type(s) for +: 'int' and 'Counter'` from `sum()` .

Day 064 · dsa — Grouping: the key-design skill

- **Grouping is four lines; the key is the whole problem.** `groups = defaultdict(list)` then `groups[key_of(item)].append(item)` . Without the `import`, `groups.setdefault(k, []).append(x)` — correct, but it allocates a throwaway `[]` on every call.
- **The recipe:** say in words what makes two items belong together, then compute exactly that — no more, no less. Then run both tests out loud: *same key but shouldn't be together?* (too coarse) · *should be together but different keys?* (too fine).
- **The keys worth memorising:** anagrams → `"".join(sorted(word))` (destroys order, keeps counts) · shifted strings → `tuple((ord(b)-ord(a)) % 26 ...)` · Sudoku box → `(r // 3, c // 3)` · diagonals → `r - c` , anti-diagonals → `r + c` · duplicate files → size first, then SHA-256, then compare bytes.
- **The count key is not automatically better.** $k \log k$ vs $k + 52$ crosses over at about $k = 25$, so for interview-length words the sorted key wins — and it survives Unicode, which the 26-slot array does not. Cost: $O(n \cdot k \log k)$ time, $O(n \cdot k)$ space, and the space is not optional because the output is that big. Brute force is $5,000 \times$ worse at $n = 10,000$.
- **Three things that bite:** the key must be hashable (`tuple(counts)` , never `counts`) · `itertools.groupby` only groups **adjacent** runs, so it splits `aet` three ways unless you `sorted(words, key=key)` first · reading a missing key from a `defaultdict(list)` **inserts an empty list** and it turns up in your output.

Day 065 · dsa — Hashing your own objects

- **The default is identity, not value.** No `__eq__` / `__hash__` means Python compares memory addresses, so `len({Order("A-1"), Order("A-1")}) = 2`. That one sentence is the whole interview question.
- **Two rules.** `a == b ⇒ hash(a) == hash(b)`, and the hash must not change while the object is a key. The converse of rule 1 is *not* required — unequal objects sharing a hash is a collision, which costs time, never correctness.
- **Breaking rule 2 is silent.** Mutate a key in place and `x in d` is `False` while `len(d)` is 1 and iteration still yields it. No exception, ever. `@dataclass(frozen=True)` makes it impossible — `FrozenInstanceError` at the moment of the mistake instead of a wrong answer later.
- **Defining `__eq__` sets `__hash__ = None`**, so a plain `@dataclass` raises `TypeError: unhashable type`. Fixes: `frozen=True` (the default choice) · `NamedTuple` (free, and faster — tuple hashing is C) · `eq=False` to keep identity · `unsafe_hash=True` **never**, it just re-enables the bug.
- **Hash the identity fields, cheaply.** `__hash__` runs on *every* lookup: a 100 KB field is 8 μs per operation; `tuple(self.items)` on a 10,000-list is 200 μs. And a too-coarse hash (customer instead of order id) gives 200-long chains — 200× slower with no error. Combine fields with `hash((a, b))`, and always guard `__eq__` with `isinstance` returning `NotImplemented`.

Day 066 · dsa — When a hash map is the wrong answer

- **A hash map buys time with memory and throws away structure.** Three cases where it is the wrong tool: **space is taken away** · **keys are small dense integers** · **you need order**.
- **No extra space → three moves.** Sort and walk ($O(n \log n)$ time, $O(1)$ space) · two pointers on sorted input (Two Sum sorted is $O(n)/O(1)$, no map) · **mark in the input itself** when values are bounded by n . Always ask first: *may I modify the input?* — every $O(1)$ -space answer destroys it.
- **First Missing Positive is the flagship.** The answer is bounded by $n + 1$ (n numbers: either they are exactly $1..n$, or one of $1..n$ is missing). Three passes: clean out-of-range to `n+1` · mark slot `v-1` with `-abs(...)`, **never** `-x` (on `[1, 1]` plain negation returns 1 instead of 2) · return the first positive index + 1.
- **Direct addressing needs dense keys, not just integer ones.** `[0]*26` for letters is right; `[0]*(max+1)` on `[1, 2, 10**9]` is 8 GB and a `MemoryError`. Memory per million ints: **list 7 MB · set 32 MB · dict 40 MB**. And below ~8-16 elements, just scan — at $n = 8$ a set wins by only $1.8\times$ before you count building it.
- **A map answers one question: is this key present?** Smallest, k -th, next, previous and range are all $O(n)$ scans against it — use `bisect` on a sorted list, `heapq`, or `SortedList` (Python has no `TreeMap`). And its $O(1)$ is an *average*: where you must promise a bound, a guaranteed $O(\log n)$ can beat a faster average.

Day 067 · dsa — Hashing revision and mock round

- **Run the five questions before touching the keyboard.** What do I ask repeatedly? · What exactly is the key? · Does the input remove the need for a map (sorted / bounded by n / dense ints)? · What am I trading? · What is the invariant? The two that could **remove** the map come first, or you will commit before you can see them.
- **Structure by question:** *seen it?* → set · *how many?* → Counter · *where?* → dict value→index · *which group?* → defaultdict(list) + a designed key · *smallest / k-th / next / between?* → **not a map** — heap, or sorted + bisect.
- **Five sentences carry the phase:** a map computes, it does not search · **$O(1)$ is an average**, $O(n)$ worst · in on a list is $O(n)$ and looks identical ($700\times$ at $n = 20,000$) · the key is the problem, the loop is four lines · a map buys time with memory and **throws away structure**.
- **Numbers to have ready:** nested loop vs map at $n = 10,000 = 50M$ vs **20K ($2,500\times$)** · memory per million ints = **list 7 MB, set 32 MB, dict 40 MB** · top-k at $n = 10^6$, $m = 10^5$, $k = 10 = 1.7M$ (sort) vs 330K (heap) vs 1.1M (buckets).
- **The round is scored on five lines, not one.** Communication · approach *before* code · correctness · **edge cases you found yourself** (empty, one, all-same, negatives) · follow-up handling. Minutes 0-12 spent writing a brute force in silence is a quarter of the interview thrown away.

Low-level design case studies

Day 077 · system-design — How to run a low-level design interview: the forty-minute script

- **Six moves, always in this order: clarify · requirements and scope · nouns · class diagram · the interesting part · concurrency and extension.** Budget them **8 · 5 · 9 · 11 · 7** minutes. A third of the round goes before you draw a box, and that is correct. Announce the shape in the first thirty seconds and the interviewer will agree to it.
- **Every LLD prompt has exactly ONE interesting part — find it and spend a third of the time there.** The test: *if the interviewer changed one requirement, what would have to change?* Parking lot → allocation. Lift → scheduling. Vending machine → the state machine. BookMyShow → seat locking. Splitwise → minimising settlements. Cache → eviction.
- **Put an interface at the interesting part and show TWO implementations.** One is a class; two is proof the design survives change. The gate for every interface: **can you name a second implementation somebody would actually want?** If not, do not build the first. **6–10 classes and 1–2 interfaces** is right; 4 classes means merged responsibilities, 18 means you are building a framework.
- **Never present the design as if it appeared fully formed.** Version one, what breaks, then the fix. That is the only way an interviewer can tell an understood design from a memorised one.

- **Raise concurrency yourself at minute 33.** Find-then-claim is two steps with a real window — $\sim 51 \mu\text{s}$, about one collision every few hours at peak — so: a lock **per resource, not per system**, or a **conditional update** where zero rows updated means you lost and must retry. Then answer the requirement they are about to add, before they add it.

Object-oriented design

Day 051 · system-design — Modelling a real domain

- **Six steps, in order:** nouns and verbs *in the business's words* → entities vs value objects → **the noun nobody wrote down** → invariants as sentences → aggregate boundaries → behaviour on the class that owns the data.
- **Use their vocabulary — the ubiquitous language.** `Consignment` if they say consignment, `Spoil` if the floor says spoil. A model whose names differ from the conversation costs a translation forever, and renaming later touches everything.
- **The missing noun is where the value is:** `Loan`, `Ticket`, `Movement`, `Booking`, `Quote`. The test — *is there a rule, a date or an amount of money with no home?* If yes, that is a class, not a field.
- **An aggregate is a consistency boundary with one door.** Inside: things a rule spans; changes go through the root. Outside: **referenced by id**. Too big → loading one order loads the database; too small → an atomic rule becomes eventually consistent, and you must say so.
- **Entity = identity persists (compare by id, mutable). Value object = defined by its values (frozen, compare by value).** Test: *would I care which one I got?* A **domain service** is the last resort — reach for it only after checking you have not missed a class. 6-10 classes, 2-4 roots.

Day 052 · system-design — Common object-oriented interview questions

- **Every standard OOP question is a handbrake question: booklet answer, then the one that matters.** Answer in three beats and stop — **definition** (one sentence) · **situation** where it decided something · **cost** with a number. Budget 40-50 seconds, about 120 words.
- **Abstract class = “is a kind of”; interface = “is able to”.** Abstract class carries state and working methods and spends the one inheritance slot; interface carries signatures and you can have unlimited ones. Python: `abc.ABC` + `@abstractmethod` (fails at *construction*) versus `typing.Protocol` (structural, implementer inherits nothing).
- **Encapsulation hides data so a rule cannot be broken; abstraction hides mechanism so it can be replaced.** The pillars split the same way: encapsulation and abstraction are about *hiding*; inheritance and polymorphism are about *varying*.
- **Python has no overloading** — a second `def` silently replaces the first. Use defaults, `*args`, or `functools singledispatch`. It has full **overriding**, resolved at runtime through `__mro__`, and `super()` means “next in the MRO”, not “my parent” — which is the dia-

mond problem, answered.

- **Say numbers, not principles.** Composition against inheritance: $3 + 2 + 2 + 3 = 10$ classes against $3 \times 2 \times 2 \times 3 = 36$. Polymorphism against a switch: 1 new file + 1 line against 4 edits to working code. And define `__hash__` whenever you define `__eq__`, on fields that never change.

Day 053 · system-design — Writing clean, testable classes

- **The whole technique in one sentence: a class asks for what it needs and never goes looking for it.** A constructor that names a driver, host or URL has welded itself shut; take the collaborator as an argument instead. That is **dependency injection**, and it needs no framework — it is argument passing.
- **Every injected dependency is a seam.** Three things destroy testability: constructing collaborators inside `__init__` · calling `datetime.now()`, `random`, `uuid` or `os.environ` directly · global mutable state. Inject the first, inject a **clock** for the second, delete the third.
- **Four doubles, used precisely:** dummy (never used) · stub (canned answers) · **fake** (a real simplified implementation — the in-memory dict) · mock (records calls). **Prefer fakes;** a mock asserts *how* and breaks on honest refactors. Use a mock when the outcome is an effect leaving the system.
- **The argument isn't speed, it's reachability.** ~18 of 40 branches in a service are error paths and none are reachable with a live database; a repository that raises `ConnectionError` on demand makes them all testable. Speed follows: 800 tests, 48 s → 0.3 s.
- **Concrete classes are named once, in the composition root** (`main()`, or FastAPI's `dependency_overrides`); `grep -rn "psycopg" src/ --exclude=main.py` should be empty. And concede the limit: **fake the collaborator to test your logic, use the real engine (test-containers) to test the integration** — plus one contract test run against both, so the fake cannot drift.

Day 054 · system-design — Object-oriented design revision and interview questions

- **Run five moves on a clock, and say the plan at minute one:** 0-5 clarify and scope (name what you leave out) · 5-10 nouns, 6-10 classes, one sentence each · 10-20 the class diagram with a multiplicity on every line · 20-35 code the one flow that matters · 35-42 their extension, answered in **edits** · 42-45 gaps.
- **Find the noun nobody wrote down.** “A guest books a room” has no home for the dates, the price, the status or the cancellation deadline — that is `Booking`. Second missing noun: `Inventory`. The test: *is there a rule, a date or an amount of money with no home?*
- **Behaviour on the class that owns the data.** `booking.cancel(on)`, `room.can_host(n)`, `DateRange` refusing to exist with `end ≤ start`. Fields plus a `BookingService` is the anaemic model and it is how this round is lost.

- **An interface only where you can name the second implementation.** Here: `PricingStrategy` (flat/seasonal — and **wrap**, don't subclass) and `CancellationPolicy` (free-until-48h / non-refundable). Not `IHotel`. A repository counts, because the second implementation is the test fake.
- **Every version of this prompt has the same race: check, then write.** ~2.5 ms window ≈ 5% per contended room-night ≈ a dozen double bookings a day. Fix it by making check-and-write **one** operation — `UPDATE ... WHERE remaining > 0` — plus a 10-minute **hold with a TTL**, because payment is 12 s median and 40 s at p95. An in-process lock is a placeholder; say so.

SOLID and design principles

Day 055 · system-design — Single responsibility

- **“One reason to change”, not “does one thing”** — and a *reason* is a **person or a role**, not a topic. `Employee` with `calculate_pay` (finance) + `report_hours` (HR) + `save` (platform) is three stakeholders in one file. The harm is not length; it is that two unrelated rules get coupled by accident and one team's change silently breaks another's.
- **Six ways to spot it:** you need an “and” to describe the class · the imports disagree (`decimal` and `smtplib`) · the name is a noise word (`Manager` , `Processor` , `Utils`) · **two teams keep editing the same file** (the only measurable one — check the version history) · long, irrelevant test setup · two clusters of fields used by two clusters of methods.
- **The refactor:** list the methods and name the *stakeholder* for each → group by stakeholder → check which fields each group touches → extract the most independent group first → inject rather than construct → re-read the imports.
- **Keep the business rule where it is.** `Invoice.total()` stays on `Invoice`; only the PDF, the persistence and the email move out. Pulling *everything* into a `Service` is the **anaemic model**, which is the commonest bad split. A coordinator whose one responsibility is the *sequence* is legitimate.
- **It is over-appliable.** Fifteen one-method classes is the same mistake inverted — 15 files and 38 lines of wiring to follow one flow. The other half of the rule is *gather what changes together*. In a new codebase, leave two concerns together and let the **second unrelated change** show you the seam. Numbers to quote: GST change = 22 lines and 9 tests instead of a 642-line file and 63 tests; the discount test goes from 180 ms with a database to 0.1 ms without one.

Day 056 · system-design — Open for extension, closed for modification

- **Open for extension, closed for modification** — add behaviour by writing *new* code, not by editing code that already works. The mechanism is always: **find the axis of change, put an interface there, let new behaviour arrive as a new implementation**. The caller never learns how many implementations exist.

- **Argue it with the edit count, not the principle.** New offer with the `if` chain: 1 shared 140-line file, 40 tests re-run, 12 live rules at risk. With the interface: **1 new file + 1 wiring line**, 0 existing tests re-run. And the cost stays **flat** — the twelfth costs what the second cost.
- **Concede that the change moved rather than vanished.** One line is still added to a list — but a line in a list of constructors cannot break Diwali, and a new branch inside a shared function can. A decorator registry removes even that line, at the cost of the active list being invisible.
- **Four times to leave the `if` chain alone:** the set is closed (weekdays, suits, sizes) · the variation is **data**, so a dict beats a class hierarchy (GST rates) · it is the *first* occurrence — build it when the **second** arrives · the code is genuinely finished. The test is always **name the second implementation**; speculative generality costs ~22 lines and 3 files, forever.
- **You can only close against a *chosen* axis.** Closed to new discount types is still open to “discount” changing meaning. Every platform you use is this principle having won — `sorted(key=)`, Django middleware, pytest plugins, VS Code extensions, Kubernetes CRDs, payment webhooks.

Day 057 · system-design — Liskov substitution

- **Anywhere the code expects a type, any subtype must work — and the caller must not need to know which one it got.** The three rules: a subtype must **not demand more** (stronger precondition), **not deliver less** (weaker postcondition), and **not break an invariant** the parent guaranteed — including the ones nobody wrote down.
- **Square/Rectangle:** `set_width(5); set_height(4); area()` gives **20 and 16**. The caller is correct, both classes are correct, the **is-a claim** is wrong. `Rectangle` silently promises that the sides are independent. **Most Liskov violations are really about mutation — make it immutable and the problem dissolves.**
- **Four tells:** `isinstance` appearing in *callers* · an override that **raises** (`ReadOnlyList.append`, `Collections.unmodifiableList`, `java.sql.Date`) · an override throwing an exception the parent never documented (`CacheMissError` instead of `None`) · docs saying “do not call X on Y”.
- **Fixes, in order:** model the **capability, not the taxonomy** (`Sequence` vs `MutableSequence` in `collections.abc`) · make it immutable · compose instead of inherit · move the method down onto a narrower interface.
- **Detect it with contract tests** — one suite written against the *interface*, three lines to wire each implementation. And say the second-order cost: **a Liskov violation destroys open/closed**, because once 40 call sites check the type, the seventh implementation is no longer one new file.

Day 058 · system-design — Interface segregation

- **No client should be forced to depend on methods it does not use** — and *depend on* is stronger than *call*: taking a 15-method type and calling one means all 15 changes reach you. Prefer **role interfaces** named after a capability (`UserReader` , `RowScanner`) over **header interfaces** named after a thing (`Repository` , `Machine`). `-er` names keep them small.
- **Two costs, and give both.** Implementations are **forced to lie** — `CsvRepository.save` raising `NotImplementedError` is a Liskov violation *created by the interface*. And **churn**: one new method = 6 implementations + 4 fakes + a redeploy, for callers that will never invoke it. Mr Kulkarni changed the store-room lock and stopped the milk.
- **The diagnosis is a grep.** List every caller and count methods called against methods depended on — 5 callers of a 6-method interface gave **22 forced dependencies**, and 19 of 23 `NotImplementedError`s were ISP failures. Test fakes are where you feel it: 18 lines of stubs against 4, times 40 tests.
- **In Python the narrow interface is free.** `typing.Protocol` is **structural**, so declare it in the *consuming* module — the provider is never edited. ~12 lines, 0 implementation changes. Compose with protocol inheritance (`class ReadWriteStore(Reader, RowWriter, Protocol)`) rather than widening.
- **ISP splits the contract; SRP splits the code.** One `PostgresStore` class (one reason to change) satisfying four role interfaces (four kinds of caller) is correct, not contradictory. Stop at three or four roles — going to one method each retrofits Go's design onto an app and nobody can answer “what can this thing do?”

Day 059 · system-design — Dependency inversion

- **High-level must not depend on low-level; both depend on an abstraction — and the abstraction must not depend on details.** The problem is never “there is no interface”, it is **the direction of the arrow**. Before: `orders/` imports `psycpg`. After: `orders/` and `adapters/` both import `orders/ports.py`.
- **The half everyone skips: the interface lives in the *policy's* package, in the policy's vocabulary.** Your `Order`, your `Money`, your `OrderNotFound`. A port in a `persistence/` package, or a **vendor type in a signature**, is a layer that inverted nothing — and it looks identical on a diagram.
- **Three terms, kept apart.** DIP = which way source dependencies point. DI = passing collaborators in (a technique, 3 lines). IoC = who calls whom at runtime (a framework calling your code). `__init__(self, conn: psycpg.Connection)` is injection **without** inversion.
- **The proof is a command:** `grep -rn "psycpg" orders/` returns nothing, and `import orders` works in a bare interpreter. The adapter **translates at its edge** — rows → objects, their errors → your exceptions, their units → your types.
- **Numbers and limits.** Razorpay → Stripe: 38 references in 14 files becomes **1 new file + 1 line**; the declined-payment test goes from impossible to 0.3 ms. Cost: ~108 lines of structure and one hop of indirection — so **skip it when you cannot name the second imple-**

mentation (the test fake counts), and remember it buys *source-level* independence, not a free swap.

Day 060 · system-design — DRY, KISS, and YAGNI

- **DRY is about *knowledge*, not *characters*:** every **fact** gets one authoritative home. “The GST rate is 18%” in nine files is real duplication — extract it. Two validators that look identical but are owned by growth and by security are **coincidental** — leave them.
- **The test:** *would a change to one always require the same change to the other?* and *who would ask?* Different owners means different futures, however identical the code. And usually there is a real fact hiding inside a coincidence — **extract the fact, not the function** (the email predicate, not the whole validator).
- **The wrong abstraction grows flags.** `is_admin` → `min_password` → `require_mfa` → five parameters and an if-chain rebuilding the distinction you deleted; 24 lines → 71, 6 tests → 14, ~3 days to untangle. **Duplication is far cheaper than the wrong abstraction.**
- **The asymmetry is the argument.** Duplicate → shared: ~20 minutes, low risk. Shared → duplicate: 20-50× **more expensive** and behaviour-changing. So use the **rule of three** — two is a coincidence, three shows the shape.
- **KISS = fewest moving parts that solve the actual problem** (a 4-line dict beats a 40-line plugin registry unless plugins ship separately). **YAGNI = no mechanism for a requirement you cannot name** — but build it early when adding it later is *superlinear* (audit trails, tenant ids, migrations). And duplicate deliberately across **service boundaries, bounded contexts, and tests**, where DRY’s coupling is the thing you are avoiding.

Day 061 · system-design — Coupling, cohesion, and code smells

- **Cohesion is what is *inside* a module; coupling is what *crosses the boundary*.** Aim for high cohesion, low coupling. Cohesion ladder (worst→best): **coincidental** (`utils.py`) · **logical** (grouped by kind — the kitchen by size) · **temporal** (grouped by when) · ... · **functional**. Coupling ladder (worst→best): **content** · **common** (globals) · **control** (a flag picking a branch) · **stamp** (a whole object for one field) · **data** · message.
- **Two smells are the diagnosis, and they are opposites.** **Shotgun surgery** = one change, many files → knowledge is *scattered*, so **gather** it (one `Address` type: 10 files and 60 lines becomes 2 files and 8). **Divergent change** = one file, many unrelated reasons → knowledge is *piled up*, so **split** it.
- **The named smells, with fixes:** feature envy → move the method to the data · data clump → a missing class · primitive obsession → a value object · long parameter list / boolean flags → control coupling, use an enum or split · message chain (`a.b.c.d`) → Law of Demeter, ask for what you need · middle man · god object · temporal coupling (`configure()` before `run()`).

- **Review in six checks:** read the imports · say it in one sentence with no “and” · count the fan-out · read the signatures for clumps and flags · read one test’s setup · **run `git log` on the file** — 11 authors from 4 teams is divergent change *measured*, not asserted.
- **Coupling cannot be removed, only shaped**, and the smells are hints not rules. A `Matrix` with 40 methods is cohesive; a fluent `df.groupby().agg()` chain is not a Demeter violation; an adapter that only forwards is a legitimate middle man. And **a file with two commits in three years is finished** — refactor what changes.

Day 062 · system-design — Design principles revision and interview questions

- **All eight principles answer one question: when requirements change, how much must I touch?** SRP keeps co-changing things together · OCP makes the common change an addition · LSP and ISP keep the abstraction honest · DIP points the import arrows · DRY gives each *fact* one home · KISS and YAGNI refuse flexibility you cannot name a second use for.
- **Review in a fixed order, cheapest evidence first:** imports · one-sentence description (count the “and”s) · signatures (flags, vendor types, clumps) · branching (`if` chains, `isinstance` in callers) · state and globals · `git log`. The history is the only *measurable* check — 4 teams on one file is divergent change proved, not asserted.
- **Name findings with evidence, never with labels.** Not “violates SRP” but “`decimal` and `smtpLib` in one import block”. Not “too many parameters” but “4 booleans = 16 combinations, ~5 meaningful, and `email=False, dry_run=True` silently does nothing”.
- **Always end on a ranking and a thing you would leave alone.** One first change, its reason (usually testability: 180 ms → 0.1 ms), and one honest “I would not touch the three-way `if` until somebody names the fourth kind.”
- **The principles contradict each other and you should say so.** DRY loses to SRP when the owners differ · OCP loses to KISS until you can name implementation two · DIP loses to YAGNI unless the test double counts · and a file with two commits in three years is finished.

Sorting

Day 051 · dsa — Why sorting matters more than any single sorting algorithm

- **Ask it on every array problem: “would this be easier in order?”** Sorting buys four things — equal values adjacent · **near values adjacent** · binary search and early exit · a correct greedy order.
- **The sentence that earns the $n \log n$:** *if two values are closest, nothing lies between them — so after sorting they are neighbours.* All-pairs 500 billion → 21 million at $n = 10^6$.
- **Sort only when it lowers the class.** Four times not to: a hash set already gives $O(n)$ (duplicates, Two Sum) · you want the top k (heap $O(n \log k)$, quickselect $O(n)$) · you need the original indices · the data is a stream.

- **Carry the tag when order would destroy something:** `sorted((value, i) for i, value in enumerate(nums))`. And `.sort()` returns `None` and mutates; `sorted()` returns a new list.
- `key=` is called once per element and tuple keys break ties (`(city, -age)`); multi-pass sorting needs **stability** and the *least* significant key first. Python runs **Timsort** — $O(n \log n)$, stable, adaptive, in C. In an interview you *call* a sort; the next eight days are about how they work because interviewers ask *about* them.

Day 052 · dsa — Bubble, selection and insertion sort, and what each one teaches

- **All three are $O(n^2)$, in place, $O(1)$ space — and they differ where it counts.** Bubble: swap out-of-order neighbours, largest sinks right, needs the `swapped` flag to be $O(n)$ on sorted input. Selection: find the smallest, swap it forward — **exactly $n - 1$ swaps**, never adaptive, **not stable**. Insertion: walk each value back into the sorted prefix, shifting as you go.
- **Insertion sort is the only one with a real answer to “when would you use it”:** small inputs (under ~ 50 ; Timsort’s threshold is 64), nearly-sorted input (**adaptive** — $O(n)$), and values arriving one at a time.
- **The invariant to say out loud:** *before step j , `nums[0:j]` holds the first j values in sorted order.* And it **shifts rather than swaps** — one write per displaced element instead of three.
- **Two one-character traps.** `while i ≥ 0 and nums[i] > value` — the guard must come first or `nums[-1]` wraps round and corrupts the list, silently. And `>` not `≥`, or you lose **stability** with no visible failure.
- **The arithmetic:** $n(n-1)/2 = 499,500$ comparisons at $n = 1,000$. Writes on reversed input: bubble 1,498,500 · insertion 500,499 · **selection 2,997** — selection sort’s only virtue, and it matters for flash memory and large records.

Day 053 · dsa — Merge sort

- **The sentence:** *a list of 0 or 1 is sorted; otherwise sort both halves and merge them.* The merge is the engine — compare only the two fronts, take the smaller, and when one side empties copy the rest across with no comparisons. Merging is **$O(n)$** because nothing is ever searched for.
- **Why $O(n \log n)$, in two facts multiplied:** every level of merging moves all n elements ($1 \times n$, then $2 \times n/2$, then $4 \times n/4 \dots$), and there are **$\log_2 n$** levels of halving. Not “because it halves” — binary search halves too and is $O(\log n)$, because it discards one side.
- **No bad input.** Best = average = worst = $n \log n$, and it is **stable** because the merge uses `≤` and takes from the left on a tie. Quicksort is faster in practice but can hit $O(n^2)$ and is not stable.
- **The cost is $O(n)$ extra space** — the merge cannot write over its own inputs. Copy only the *left* half into the buffer; the right half is always at or beyond the write position. Two exceptions: linked lists (pure relinking, $O(1)$) and LeetCode 88 (merge **backwards** into the free tail).

- **Three traps:** `//` not `/` for the middle (`TypeError: slice indices must be integers`) · base case `≤ 1` not `= 1` or an empty half recurses forever (`RecursionError`) · the two `extend` lines, or the tail is silently dropped. And the free follow-up: **count inversions** in $n \log n$ by adding `len(left) - i` whenever you take from the right.

Day 054 · dsa — Quicksort and partitioning

- **Quicksort is one operation: partition.** Pick a pivot, rearrange so smaller is left and larger is right — the pivot is then in its **final** position, and the two sides are independent. No merge step; the work happens on the way *down*, which is why it sorts **in place**.
- **The invariant to say out loud:** `nums[lo:i]` is all `< pivot`, `nums[i:j]` is all `≥ pivot`; at the end swap the pivot into `i`. Recurse on `p-1` and `p+1` — **excluding the pivot**, or it recurses forever.
- **Worst case $O(n^2)$, and the input is an already sorted list** with a first- or last-element pivot: one side empty, depth n , $n(n-1)/2$ comparisons. 500 sorted values: 124,750 comparisons against $\sim 4,000$ with a random pivot.
- **Fix it with two lines of randomisation** — the time now depends on your dice, not on the input, so no *particular* input is bad. Then median-of-three (deterministic, beatable) and **introsort** (bail to heapsort past depth $2 \log n$ — what `std::sort` does). Separately, **three-way partition** for duplicates: a million equal values goes from $O(n^2)$ to one pass.
- **Against merge sort:** quicksort is in place ($O(1)$ data, $O(\log n)$ stack if you recurse into the smaller side) and $2\text{--}3\times$ faster despite $\sim 39\%$ more comparisons. It gives up the **guarantee** and **stability**. Choose merge sort for adversarial input, multi-key sorting, linked lists, and data too big for memory.

Day 055 · dsa — Quickselect: finding the Kth largest without sorting

- **Same partition as quicksort, one change: recurse into ONE side.** The pivot lands at its final index `p`; compare `p` with the target index. `p == target` → answer · `target < p` → search left, **discard the right** · else search right. Two recursive calls means you wrote quicksort.
- **The k-th largest is at index $n - k$.** Check both ends out loud: `k=1` → `n-1` (largest), `k=n` → `0` (smallest). Getting this wrong returns a plausible number with no error — the commonest bug on LeetCode 215.
- **Expected $O(n)$, and the arithmetic is the answer:** `n + n/2 + n/4 + ... = 2n`, because a halving series sums to **twice the first term**. Quicksort is $n \log n$ because it does full work at *every* level; here only one branch survives.
- **Worst case $O(n^2)$ if the pivot is always extreme** — fix with a **random pivot**. Randomising the *position* does not help when every *value* is equal, so use a **three-way partition** and return the pivot when `lt ≤ target ≤ gt`: a million identical values in one pass.

- **Three answers, chosen by k and by shape:** `sorted()[-k]` $O(n \log n)$ · heap $O(n \log k)$, $O(k)$ space, **the only one that works on a stream** · quickselect $O(n)$ expected, **$O(1)$ space but it mutates**. At $k = 10$ of 10^6 heap and quickselect are level; at $k = n/2$ quickselect wins six times over.

Day 056 · dsa — Counting sort, radix sort, and bucket sort

- **These three don't compare — that is how they beat $n \log n$.** No comparison sort can do better than $O(n \log n)$; counting, radix and bucket sort use the **value as a position** instead. The price is a restriction on the keys, and *if you cannot state the restriction, you cannot use the sort*.
- **Counting sort is $O(n + k)$ where k is the RANGE, not the count.** Count $\rightarrow$ running totals (how many are $\leq v$) $\rightarrow$ place walking the input **backwards** (that is what keeps it stable). 10^6 marks in 0..100: $\sim 2,000,101$ operations against 20,000,000. Three values up to 2×10^9 : **MemoryError**.
- **Radix sort has no correctness of its own — it borrows all of it from stability.** d passes of counting sort, least significant digit first; a later pass must not disturb order among equal digits, or the earlier passes are destroyed silently. 32-bit ints in base 256 = **4 passes, 256 counters**.
- **Bucket sort is $O(n)$ only if the values are uniformly spread** — otherwise $O(n^2)$. 10,000 values in one bucket costs 50,000,000 operations instead of 20,000. State the distribution assumption in the same breath as the complexity.
- **None of them sorts in place** ($O(n + k)$, $O(n + b)$, $O(n)$), none handles negatives without a `- min` shift, and none works on arbitrary objects or custom comparisons. And in Python, `sorted()` is C: ten times fewer operations in bytecode can still lose on the clock — give the asymptotic answer, then say that.

Day 057 · dsa — Stability, and what Python's sort actually does

- **Stable = elements that compare *equal* keep their original relative order.** It says nothing about unequal keys. Python's `list.sort` and `sorted` are **guaranteed** stable; most other languages are not (`std::sort` no / `std::stable_sort` yes; Java stable for objects, **not** for primitives; Go and Rust each have both).
- **Why you care: two-pass multi-key sorting.** Sort by the **least significant key first**, then the most significant — the second pass sees ties and a stable sort must not move them. Prefer a tuple key `((dept, name))` when you control both; you *need* two passes when a string field must go descending, because `-p.name` raises `TypeError`.
- **The rule for which sorts are stable: neighbour swaps are stable, long-distance swaps are not.** Stable: insertion, bubble, merge, counting (placed backwards), radix (it *depends* on it). Unstable: **selection, quicksort, heapsort**. Selection's counter-example is `[2a, 2b, 1]`.

- **The bugs are one character and silent.** `<` instead of `≤` in a merge, `≥` instead of `>` in insertion sort, counting sort placed forwards — all still sort correctly and quietly lose stability. Keep the four-line `is_stable` test with `(key, tag)` pairs. And `reverse=True` **pre-serves** ties; `reversed(sorted(...))` flips them.
- **Python runs Timsort:** finds natural runs (a *strictly* descending one is reversed in place), extends short runs with **binary insertion sort** to a minrun of 32-64, merges with **galloping** (1,000 comparisons → ~17 on concatenated sorted lists). Adaptive — 2M sorted integers in ~0.02 s against ~1.1 s random, $55\times$. Stable, $O(n)$ space, **not** $O(n)$ in general.

Day 058 · dsa — Custom comparators and sorting by keys

- **key=** is a function called once per element that returns what to order by. Sorting does not change; what it *looks at* does. `key=len`, `key=attrgetter("dept", "name")` (~2× faster than a lambda — it is C), `key=lambda t: STATUS_ORDER.get(t["status"], 99)` for an order given as data.
- **Two fields: a tuple key.** It compares position by position and stops at the first difference, so the tuple's order is the order of the rules. Opposite directions: **negate the number** — `(-s.score, s.name)`. Never `reverse=True` on a tuple key: it reverses the tie-break too and gives a **plausible wrong answer with no error**.
- **A descending string field needs two passes**, because `-p.name` raises `TypeError: bad operand type for unary -: 'str'`. Sort by the **least significant key first**, then the most significant with `reverse=True` — correct only because Python's sort is stable.
- **key runs n times; a comparator runs ~n log n times.** At $n = 10^6$ that is 1M calls against ~20M, and on the clock 30-50× because of the wrapper. Use `functools.cmp_to_key` **only when no per-element key exists** — the test case is LeetCode 179, where `a` before `b` iff `a+b > b+a`, and **negative means the first argument comes first**.
- **Prefer key= to __lt__** unless the type has one order everybody would agree on (`Version`, `Money`). And always end the key with something **unique** if a person or a test will see the output, or two runs on the same data can differ.

Day 059 · dsa — Sorting revision and mock round

- **Run the procedure out loud, in order:** what is the data (size · key type · range · nearly sorted · fits in memory) → *would this be easier in order?* → does a **set** answer it in one pass → do I need the **whole order, one value, or the top k** → which sort, and why → the cost with arithmetic.
- **Five leaves of the decision tree are not “sort the array”.** Set for membership · quickselect for the k-th ($O(n)$ expected) · heap of size k for the top k and the only one that works on a **stream** · counting sort for bounded integer keys · Timsort for everything else.

- **Pick two of: guaranteed · in place · stable.** Merge = guaranteed + stable ($O(n)$ space) · heapsort = guaranteed + in place (not stable) · quicksort = in place + fastest, giving up the guarantee. Quicksort does **39% more comparisons** than merge sort and is still 2-3× faster.
- **In Python the default answer is `sorted()`, and that is defensible:** Timsort is stable, $O(n \log n)$ worst case, **linear on nearly-ordered data**, and in C — $\sim 40\times$ faster than a hand-written merge sort (0.15 s against 8 s at $n = 10^6$). Switch only for bounded integer keys, a needed guarantee, tight memory, a linked list, or a stream.
- **Three numbers and five test inputs.** 20,000,000 against 500,000,000,000 ($25,000\times$) · $n + n/2 + n/4 = 2n$ · sort-once break-even at ~ 17 queries. Always test: empty · one element · all equal · already sorted · reverse sorted. And **keep talking** — a mock round scores clarifying, stating the cost before coding, narrating while stuck, and testing out loud.

Stacks and queues

Day 068 · dsa — Stacks: last in, first out

- **A stack is defined by its restriction: you can only touch one end.** That is *why* push, pop and peek are all $O(1)$ — nothing searches, nothing shifts. LIFO, so the order out is the reverse of the order in.
- **In Python, a list is a stack.** `append` / `pop` / `[-1]`. **The end is the $O(1)$ end** — `insert(0, x)` and `pop(0)` shift everything and are $O(n)$, giving an identical-looking $O(n^2)$ solution (2.31 s vs 0.04 s in the measurement). `deque` if you also need a queue; never `queue.LifoQueue` outside threads ($\sim 100\times$ slower).
- **The recognition sentence: reach for a stack when you need *the most recent thing not yet resolved*.** Unclosed bracket · last action to undo · last page visited · operator not yet applied · directory entered but not left · node whose neighbours are unexplored.
- **The counting argument for every stack problem: each element is pushed at most once and popped at most once, so it is $O(n)$ — even with a `pop` inside the loop.** Push is amortised $O(1)$ because the list grows geometrically ($8+16+\dots+n/2 \approx n$).
- **The call stack is a stack**, which is why `RecursionError: maximum recursion depth exceeded` fires at **1000** frames and why any recursion can become an explicit stack (limited only by heap). Traps: `IndexError: pop from empty list` — always write `if stack and stack[-1] == x` · `part = ".."` not `startswith("..")`, because `"..."` is a real name · and remember the stack gives you things **backwards**.

Day 069 · dsa — Balanced brackets and the parsing family

- **Three ways to fail, and name all three before coding:** the closer does not match the top · a closer arrives with an **empty stack** · the stack is **non-empty at the end** (the one people forget — every step was fine and the job is not done).

- **Invariant: the stack holds exactly the openers seen and not yet closed, in order.** So a closer can only legally close the top. Map **closer** $\rightarrow$ **opener**, not the other way, because you look up the thing in your hand. Write `if not stack or stack.pop()  $\neq$  PAIRS[c]` — or short-circuits, so `pop` never hits an empty stack.
- **Counting is not enough:** `"([)]"` has matching counts and is invalid, because counting throws away order. For **one** bracket type a counter *does* work — increment/decrement, fail if it goes negative, end at zero — giving **$O(1)$ space**. Two counters answer “minimum additions”.
- **The family is one sentence: push the context you are leaving, pop and combine on the closer.** Decode String pushes `(current, repeat)` · RPN pushes operands · Basic Calculator pushes the running result and sign · a compiler pushes the scope *and its line number*, which is why it can say “unmatched { at line 41”.
- **Costs and traps.** `is_balanced` : $O(n)$ time, $O(n)$ space worst case (all openers), $O(1)$ on `()()` . `decode_string` : linear in the **output**, not the input — `10[10[10[a]]]` is 13 in and 1000 out. Traps: `elif character in PAIRS` or a letter raises `KeyError` · `number*10 + digit` for multi-digit counts · `previous + current * repeat` (outside in front, inside repeats) · in RPN the **second** pop is the left operand, which only `-` and `/` expose.

Day 070 · dsa — Min stack, and stacks that carry extra state

- **The observation, not the code, is the answer:** a stack only changes at the top, so the **minimum before a push is exactly the minimum after the matching pop** — record it on the way in and never recompute it. `pop` does no work at all.
- **Write the pairs version:** each entry is `(value, min from the bottom to here)`; push does one comparison against `stack[-1][1]`; `get_min` is one index. Four operations, all **$O(1)$** ; space **$2n$** . Remember the `[0]` in `pop` and `top`, or you hand the caller a tuple.
- **The two-stack variant’s optimisation has the famous bug:** `<` **must be** `<=` . `push(3)`, `push(3)`, `pop()`, `get_min()` loses the record while a 3 is still in the stack. And the optimisation saves **nothing** on a decreasing sequence — best case n , worst case $2n$.
- **$O(1)$ extra space exists:** push `2*value - min` on a new minimum, recover with `2*min - stored` . Mention it, do not write it — it **overflows** in fixed-width languages and forces a special case in `top()` .
- **The transferable rule:** you can carry any aggregate computable from `(new element, previous aggregate)` in **$O(1)$** — min, max, sum, count, product. **Median cannot** (needs two heaps, $O(\log n)$), and naming that boundary is what shows understanding rather than memorisation. At $n = 10,000$ with 10,000 queries this is 10^8 comparisons versus 10^4 reads.

Day 071 · dsa — Monotonic stack: the next greater element

- **The stack holds exactly the elements whose answer is not yet known, and it stays ordered by itself** — anything that would break the order is popped before the new element goes on. **The moment you pop something, you have found its answer.**

- **Eight lines:** `while stack and numbers[stack[-1]] < value: answer[stack.pop()] = value, then stack.append(index)`. Resolve first, then push. **Store indices, never values** — the usual follow-up wants a *distance* (`index - stack.pop()`), and values cannot give you one.
- **Four variants from two knobs:** direction of the walk $\times$ direction of the comparison $\rightarrow$ next greater $\cdot$ next smaller $\cdot$ previous greater $\cdot$ previous smaller. Next-greater leaves the stack **decreasing** bottom-to-top; next-smaller **increasing**. And ask about duplicates: `<` gives strictly greater, `<=` gives greater-or-equal — on `[2, 2, 3]` that is 3 versus 2.
- **The complexity defence is the interview.** n pushes total, **each element popped at most once**, so $\leq 2n$ operations — **the inner loop is bounded in total, not per iteration**. One iteration popping n elements has consumed all the pops the rest of the run was going to do. 5×10^9 vs 2×10^5 at $n = 100,000$, about **25,000** $\times$.
- **Space $O(n)$ worst case** (a strictly decreasing input, nothing ever pops), **$O(1)$ best case** (a strictly increasing one). **Pre-fill the answer array and write by index — never append**, or the leftovers vanish. Circular: walk `2n` with `step % n` and **push only on the first pass**. The family: daily temperatures $\cdot$ stock span $\cdot$ largest rectangle $\cdot$ trapping rain water.

Day 072 $\cdot$ dsa — Largest rectangle in a histogram

- **Kill the infinity first: the best rectangle's top edge is level with some bar** — otherwise you could raise it. So there are exactly `n` candidates, one per bar: **the widest stretch in which that bar is the shortest**.
- Per bar you need two boundaries — nearest strictly shorter on the left and on the right — and both are monotonic-stack passes. `width = right - left - 1`, because both boundaries are the shorter bars themselves and neither is inside the span. That `- 1` is where the question is lost.
- **One pass, because the pop knows both boundaries at once:** the current index is the right boundary, `stack[-1]` if stack else `-1` is the left. Stack holds **indices**, heights **increasing** bottom to top. Five lines: pop, `left`, `width`, `max`, push.
- **A sentinel height of 0 at the end drains the leftovers** with the right boundary at `n`, and removes the whole cleanup loop. Without it, `[1, 2, 3, 4, 5]` returns 0. Safe only because heights are non-negative.
- **$O(n)$ time** — `n + 1` pushes, each index popped at most once, $\leq 2n$ operations, bounded in total not per iteration (10^{10} vs 2×10^5 at $n = 100,000$). **$O(n)$ space** (strictly increasing input). Equal bars are safe: the lower one collects the full span. Escalation: **maximal rectangle in a binary grid** = one histogram per row, **$O(\text{rows} \times \text{columns})$** .

Day 073 $\cdot$ dsa — Queues: first in, first out

- **A queue is FIFO: add at the back, remove from the front, never touch the middle** — and that restriction is *why* enqueue, dequeue, peek and is-empty are all **$O(1)$** . It is the mirror of the stack. Every **BFS** and every **level-order traversal** is a queue.

- `list.pop(0)` is $O(n)$ because a list is a contiguous block and element 0 must stay at the start, so everything shifts left. Draining is $O(n^2)$: measured 10.2 s vs 0.0034 s at $n = 100,000$ ($\sim 3,000\times$), and $n \rightarrow 2n$ gave $3.9\times$ the time, which is the quadratic signature. The back of a list is the cheap end — same list, perfectly good stack.
- The ring buffer is the answer they want: fixed slots, `head` for the front, `tail` for the next free slot, both advanced with `(marker + 1) % capacity`. Nothing ever moves. Clear the dequeued slot to `None` or the objects are never collected.
- `head = tail` means empty and full — keep a `_size` counter (or waste one slot). Never loop on `head < tail`; it breaks the moment tail wraps. Read the ring in order with `(head + i) % capacity`, never by slot number. Growing = double and unroll while copying, amortised $O(1)$.
- In production: `collections.deque` — a linked list of blocks, both ends $O(1)$. Not `queue.Queue`, which locks on every operation and measured $48\times$ slower single-threaded; name it only when the question is producer–consumer threads.

Day 074 · dsa — Deques and the sliding-window maximum

- The whole problem is one sentence: when a bigger element arrives, every earlier smaller element is worthless for ever — every future window containing the old one also contains the new one, and the new one is bigger. So delete, do not deprioritise. What survives is exactly the live candidates, automatically falling front to back.
- Two removals at two ends, for two different reasons — that is why it is a deque: `popleft` when the front index has slid out (`window[0] ≤ index - k`), `pop` while the back is too small (`numbers>window[-1] ≤ value`). Store indices, because a value cannot tell you whether it is still in the window.
- Four steps per element, in this order: expire front · crush back · append · record. One `if` suffices for the expiry because the window moves by exactly one. Record only when `index ≥ k - 1`, or you return `n` answers instead of `n - k + 1`.
- $O(n)$ time — n appends, each index removed at most once, $\leq 2n$ operations, bounded in total not per iteration. $O(k)$ space, not $O(n)$: worst case a strictly decreasing array, best case one index on a rising one. Measured at $n = 200,000$, $k = 1,000$: brute 1.98 s, heap 0.092 s, deque 0.050 s — $40\times$ the brute force, $1.8\times$ the heap.
- Name the heap and reject it properly: push `(-value, index)` and discard expired entries lazily — correct, $O(n \log n)$, $O(n)$ space. The heap keeps everything and filters at read time; the deque discards at write time. The family: sliding-window minimum (flip to `≥`) · two deques for $\max - \min \leq \text{limit}$ · DP transitions that need “the best of the last k ”.

Day 075 · dsa — A queue from two stacks, and a stack from queues

- Pouring one stack into another reverses the order — that single side effect turns two LIFO stacks into a FIFO queue. `inbox` takes every arrival (enqueue is $O(1)$, always); `outbox` serves every departure.

- **The correctness rule: pour ONLY when outbox is empty.** Pouring onto a non-empty outbox puts newer elements above older ones and **fails silently** — `enqueue 1, 2 · dequeue · enqueue 3 · dequeue` returns **3 instead of 2**. Put the guard inside one `_pour()` that both `dequeue` and `peek` call. `is_empty` must check **both** stacks.
- **Say both numbers: dequeue is $O(n)$ for one call, $O(1)$ amortised.** The proof is a count, not a claim: each element is touched exactly **4 times** in its life — inbox push, inbox pop, outbox push, outbox pop — and **can never move backwards**, so n enqueues + n dequeues $\leq 4n$ operations.
- **Amortised is not average-case.** Average-case assumes something about the inputs; amortised bounds the **total cost of any sequence**, including an adversarial one. But it says nothing about a single call, so it is the wrong guarantee for a **p99 latency target** — there you want `deque`.
- **The asymmetry is the real lesson: two stacks make a good queue; queues do not make a good stack.** A one-queue stack rotates `len - 1` on **every** push, so n pushes cost $n(n-1)/2$ — no amortised saving, because the same elements move again and again. Rotate `len - 1`, not `len`, or your stack becomes a queue.

Day 076 · dsa — LRU cache: the structure interviewers love

- **One structure cannot do it, and saying why is the first ninety seconds.** A hash map has **no order** (finding the LRU is an $O(\text{capacity})$ scan); a list has **no lookup** (finding a key is a scan, moving it shifts everything). So: **map from key $\rightarrow$ node**, plus a **doubly linked list of nodes, most recent at the head**.
- **Doubly linked, because unlinking is two assignments** — `node.prev.next = node.next` and `node.next.prev = node.prev` — and that needs both neighbours reachable *from the node*. A singly linked list must walk to find the predecessor: $O(\text{capacity})$.
- **Two things everyone forgets.** `get` is a **write** — it moves the node to the head, or you evict by insertion order and the hot entry gets thrown away while returning correct values. **The node stores its own key**, because eviction starts from the list and must then `del map[node.key]` — **every change touches both structures**.
- **Write the sentinels first.** A fake head and fake tail that always exist delete every empty-list and first-node branch, and with them `AttributeError: 'NoneType' object has no attribute 'next'`. Keep `put`'s two cases apart with an early `return` on the existing key. Test `capacity = 1` and `capacity = 0`.
- **All $O(1)$ worst case, ~ 6 writes per get and ~ 10 per put; space $O(\text{capacity})$** — about **190 B per entry** in Python, so 100,000 entries $\approx$ **19 MB** of overhead. In production: `OrderedDict (move_to_end, popitem(last=False))`, `functools.lru_cache`, or Redis `allkeys-lru`, which **approximates** LRU by sampling. **Not thread-safe** — one lock, or shard by `hash(key) % 16`. LFU needs a **third** structure plus a `min_frequency` counter.

Day 077 · dsa — Stacks and queues revision and mock round

- **Four trigger sentences cover the whole phase.** “The most recent thing not yet resolved” → **stack**. “In the order they arrived / the last k seconds / one level at a time” → **queue**. “For every element, the nearest one on a side satisfying a comparison” → **monotonic stack**. “Lookup fast AND oldest fast” → **map** + **doubly linked list**.
- **Say what the structure holds before you write the loop.** “The stack holds the elements whose answer is not yet known.” That one sentence is what the round is scoring — the driving test cannot see what you know, only what you say.
- **The complexity defence, said before it is challenged: n pushes, each element popped at most once, $\leq 2n$ operations — bounded in total, not per iteration.** And its cousin for the two-stack queue: **four touches per element, never backwards**, so amortised $O(1)$ — which is not average-case and is not a promise about one call.
- **The four measurements to quote.** `pop(0)` 10.2 s vs `deque.popleft` 0.0034 s at 100,000 $(\sim 3,000 \times) \cdot n \rightarrow 2n$ gave $3.9 \times$, the quadratic signature · window max at $n = 200,000$, $k = 1,000$: brute **1.98 s**, heap **0.092 s**, deque **0.050 s** · monotonic stack vs brute force $\sim 25,000 \times$.
- **The traps that end rounds:** `pop(0)` · a missing `while stack and` · values instead of indices · appending the answer instead of writing by index · forgetting the leftovers · `<` vs `≤` · `width = right - left - 1` · `head = tail` needing a size counter · pouring onto a non-empty outbox · `get` that does not reorder · removing from one structure and not the other · recording a *rejected* request in a rate limiter.

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
051	Why sorting matters more than any single sorting algorithm	Modelling a real domain
052	Bubble, selection and insertion sort, and what each one teaches	Common object-oriented interview questions
053	Merge sort	Writing clean, testable classes
054	Quicksort and partitioning	Object-oriented design revision and interview questions
055	Quickselect: finding the Kth largest without sorting	Single responsibility
056	Counting sort, radix sort, and bucket sort	Open for extension, closed for modification
057	Stability, and what Python's sort actually does	Liskov substitution
058	Custom comparators and sorting by keys	Interface segregation
059	Sorting revision and mock round	Dependency inversion
060	Hash tables: how a dictionary finds anything instantly	DRY, KISS, and YAGNI
061	Collisions, and why a hash map can turn slow	Coupling, cohesion, and code smells
062	Sets: membership, deduplication, and the O(1) habit	Design principles revision and interview questions
063	Counting with dictionaries	What a design pattern actually is
064	Grouping: the key-design skill	Singleton
065	Hashing your own objects	Factory and abstract factory
066	When a hash map is the wrong answer	Builder
067	Hashing revision and mock round	Prototype, and cloning objects
068	Stacks: last in, first out	Adapter
069	Balanced brackets and the parsing family	Decorator
070	Min stack, and stacks that carry extra state	Facade and proxy
071	Monotonic stack: the next greater element	Strategy
072	Largest rectangle in a histogram	Observer
073	Queues: first in, first out	State
074	Dequeues and the sliding-window maximum	Command and chain of responsibility
075	A queue from two stacks, and a stack from queues	Template method and iterator
076	LRU cache: the structure interviewers love	Design patterns revision and interview questions
077	Stacks and queues revision and mock round	How to run a low-level design interview: the forty-minute script

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [\[day 033\]\(../day-033-window-with-a-map/README.md\)](#).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology ([PostgresOrderRepository](#)).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: `list.append` is $O(1)$ amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [\[day 009\]\(../day-009-what-an-array-is/README.md\)](#).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: **two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.**

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [tomorrow](../day-101-bfs-level-order/README.md).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's histogram_quantile does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to google.com, the machine sends back a certificate — a small document containing the name it claims (google.com), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: /bags.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually main() or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([day 006](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([day 008](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `__init__` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [day 008](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [\[day 074\]\(../day-074-deques-and-window-max/README.md\)](#) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: *"exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key."* Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like OrderPlaced(order_id, user_id, total, items).

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so Order ♦—— OrderLine because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and \$7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort Day 113 — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

heartbeat Day 124 — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof Day 111 — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism Day 052 — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node Day 104 — Every path has exactly one highest node — the point where it turns around.

historical snapshot Day 029 — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle Day 050 — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook Day 075 — award is a hook: the base class implements it as *doing nothing*, so a subclass may override it and most do not.

horizontal scaling Day 098 — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor Day 013 — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image Day 013 — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph Day 130 — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [day 125](../day-125-what-a-graph-is/README.md).

in parallel Day 111 — Components in parallel — any one will do — multiply their *failure* probabilities, which is a completely different shape.

in series Day 111 — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree Day 125 — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index Day 025 — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap Day 114 — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance Day 071 — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder Day 110 — The standing order is inorder: left subtree, then root, then right subtree.

input Day 008 — In a problem: the input — its type, its size, and what is in it.

integrity Day 006 — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity Day 070 — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface Day 052 — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant Day 028 — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — > *Find the smallest X such that some task is possible.* > > *Find the largest X such that some condition still holds.*

item [Day 016](#) — An item is one of them: /bags/213.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [\[day 007\]\(../day-007-space-complexity/README.md\)](#); it looks like {"id": 42, "name": "Zoya"}.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position k - 1; - the 1st largest is at position n - 1, so the k-th largest is at position n - k.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2 + 1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by *causality* rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS .m3u8 or DASH .mpd — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — D._mro_ — is computed by a rule called C3 linearisation, and super() follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [\[day 037\]\(../day-037-prefix-sums/README.md\)](#).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a min_frequency counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a *different table*, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely *is* a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, "I do not know", and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows *sorted inside* that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for *queries per second* — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [\[day 73\]\(../day-073-queues/README.md\)](#).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [\[day 117\]\(../day-117-merge-k-sorted/README.md\)](#).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the loft itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running $\% k$ and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are **not** building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `,` `.join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a **reference** to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the **next** one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `> *Find the smallest X such that some task is possible.*` `> *Find the largest X such that some condition still holds.*`

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in *which end* the region grows from and *how* the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | |---|---|---| | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, `typing.Protocol` is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing (`[day 052](../day-052-quadratic-sorts/README.md)`).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

table Day 026 — A table holds one kind of thing.

tail call Day 088 — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive Day 090 — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp Day 004 — TCP and UDP are the two ways one machine sends data to another.

template Day 092 — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double Day 053 — A test double is a stand-in you pass in during a test.

the return address Day 088 — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median Day 113 — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

third table Day 026 — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

thread Day 008 — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings Day 092 — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition Day 054 — The fix is a three-way partition: split into *less than*, *equal to*, and *greater than*, and recurse only into the outer two.

tight coupling Day 061 — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live Day 037 — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint Day 097 — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast Day 134 — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token Day 019 — The server verifies them and issues a token — a session id or a signed token.

tombstone Day 011 — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems Day 113 — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [day 116](../day-116-top-k/README.md) - merging k sorted lists — [day 117](../day-117-merge-k-sorted/README.md) - the running median — [day 118](../day-118-two-heaps/README.md), with two heaps -...

total order Day 123 — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself Day 071 — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency Day 029 — This is a transitive dependency: $id \rightarrow department_id \rightarrow department_name$.

tree Day 040 — A domain of independent records — a tree — tolerates anything.

ttl Day 003 — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple Day 005 — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [day 015](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the N + 1 problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where `nums[i] > 4`, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.



yes Day 093 — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---| | Combinations (LC 77) | $i + 1$ |
no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | |
Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library*. <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations*. <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations*. <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms*. <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set*. <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART VII · SORTING (DAYS 51–59)

Tim Peters. *listsort.txt - the design of Timsort*. <https://github.com/python/cpython/blob/main/Objects/listsort.txt> [PSF Licence]

The design note for the algorithm behind sorted(). Long, and worth it.

Python Software Foundation. *Sorting Techniques HOWTO*. <https://docs.python.org/3/howto/sorting.html> [PSF Licence]

key functions, reverse, and why stability matters.

Wikipedia. *Sorting algorithm*. https://en.wikipedia.org/wiki/Sorting_algorithm [CC BY-SA]

The comparison table of every sort named in Part VII.

PART VIII · HASHING: MAPS AND SETS (DAYS 60–67)

CPython. *Objects/dictobject.c - the dict implementation*. <https://github.com/python/cpython/blob/main/Objects/dictobject.c> [PSF Licence]

The header comment explains open addressing and the compact layout.

Python Software Foundation. *collections.Counter and defaultdict*. <https://docs.python.org/3/library/collections.html> [PSF Licence]

The two containers that shorten half the problems in this part.

Wikipedia. *Hash table.* https://en.wikipedia.org/wiki/Hash_table [CC BY-SA]
Collision strategies, load factor, and why worst case is still O(n).

PART IX · STACKS AND QUEUES (DAYS 68–77)

Python Software Foundation. *queue and collections.deque.* <https://docs.python.org/3/library/queue.html> [PSF Licence]

What you would reach for outside an interview, and why.

Wikipedia. *Stack (abstract data type).* [https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type)) [CC BY-SA]
Includes the call-stack connection that Part XI builds on.

SYSTEM DESIGN, BY PHASE

OBJECT-ORIENTED DESIGN (DAYS 43–54)

Python Software Foundation. *Classes tutorial.* <https://docs.python.org/3/tutorial/classes.html> [PSF Licence]
Inheritance and the method resolution order, from the source.

Python Software Foundation. *abc - abstract base classes.* <https://docs.python.org/3/library/abc.html> [PSF Licence]
How to express an interface in Python, which comes up in every LLD round.

Wikipedia. *Object-oriented programming.* https://en.wikipedia.org/wiki/Object-oriented_programming [CC BY-SA]
Encapsulation, inheritance, polymorphism, with the criticisms included.

SOLID AND DESIGN PRINCIPLES (DAYS 55–62)

Wikipedia. *SOLID.* <https://en.wikipedia.org/wiki/SOLID> [CC BY-SA]
The five principles with their original sources cited.

Wikipedia. *Liskov substitution principle.* https://en.wikipedia.org/wiki/Liskov_substitution_principle [CC BY-SA]
The one candidates state correctly least often.

Martin Fowler. *Articles on design and refactoring.* <https://martinfowler.com/tags/design.html> [Free to read]
Short essays. Start with the ones on coupling and cohesion.

DESIGN PATTERNS (DAYS 63–76)

Alexander Shvets. *Refactoring Guru - design patterns catalogue.* <https://refactoring.guru/design-patterns> [Free to read online]
Each pattern with a diagram and a worked example.

Wikipedia. *Software design pattern.* https://en.wikipedia.org/wiki/Software_design_pattern [CC BY-SA]
The catalogue index, and the criticism section worth knowing.

Python Software Foundation. *functools and contextlib.* <https://docs.python.org/3/library/functools.html> [PSF Licence]
Several classic patterns are one decorator in Python. Say so in interviews.

LOW-LEVEL DESIGN CASE STUDIES (DAYS 77–96)

Wikipedia. *Unified Modeling Language - class diagrams.* https://en.wikipedia.org/wiki/Class_diagram [CC BY-SA]
The notation interviewers expect on the whiteboard.

Python Software Foundation. *dataclasses.* <https://docs.python.org/3/library/dataclasses.html> [PSF Licence]
The fastest honest way to show entities in a timed design round.

Python Software Foundation. *enum.* <https://docs.python.org/3/library/enum.html> [PSF Licence]
State machines in the parking-lot and vending-machine problems.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
051	Minimum Absolute Difference	LeetCode 1200 (Easy)	The purest form: closest-in-value becomes adjacent-after-sorting.
051	Merge Intervals	LeetCode 56 (Medium)	That the sort <i>*is*</i> the algorithm, and which field you sort on.
051	3Sum	LeetCode 15 (Medium)	One sort buying two things — the two-pointer sweep and easy duplicate skipping.
051	Two Sum	LeetCode 1 (Easy)	The trap: sorting is slower here <i>*and*</i> destroys the answer.
052	Sort an Array	LeetCode 912 (Medium)	Whether your hand-written sort survives 50,000 elements — insertion sort will time out, and noticing that is the point.
052	Insertion Sort List	LeetCode 147 (Medium)	The insertion idea without the ability to shift — you have to relink instead.
052	Sort Colors	LeetCode 75 (Medium)	That a selection-sort instinct is wrong here, and one pass with three regions is right.
052	Height Checker	LeetCode 1051 (Easy)	Counting how far the input is from sorted — the exact quantity insertion sort is sensitive to.
053	Merge Sorted Array	LeetCode 88 (Easy)	The merge step alone — and whether you spot that it must run backwards.
053	Merge Two Sorted Lists	LeetCode 21 (Easy)	The same merge on linked data, where relinking replaces copying.
053	Sort an Array	LeetCode 912 (Medium)	Whether you can produce the whole sort, base case included, under a time limit.
053	Sort List	LeetCode 148 (Medium)	Why merge sort is <i>*the*</i> answer for linked lists — no random access needed, $O(1)$ extra space.
054	Sort Colors	LeetCode 75 (Medium)	A three-way partition with the pivot value fixed at 1 — the Dutch flag, in its purest form.
054	Partition Array According to Given Pivot	LeetCode 2161 (Medium)	Three-way partitioning where <i>*stability*</i> is required, so the in-place swap version is wrong.
054	Sort an Array	LeetCode 912 (Medium)	Whether your quicksort survives sorted input and 50,000 duplicates.
054	Wiggle Sort II	LeetCode 324 (Medium)	Partitioning as a tool rather than a sort — find the median, then place around it.
055	Kth Largest Element in an Array	LeetCode 215 (Medium)	The whole lesson. And whether your version survives an input of identical values.
055	Top K Frequent Elements	LeetCode 347 (Medium)	Quickselect on a derived key — you select on counts, not on the values themselves.
055	K Closest Points to Origin	LeetCode 973 (Medium)	That you never need the actual distance, only something that orders the same way.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
055	Kth Largest Element in a Stream	LeetCode 703 (Easy)	The boundary: quickselect cannot be used at all, and knowing why is the answer.
056	Sort Colors	LeetCode 75 (Medium)	Counting sort with $k = 3$ — and whether you notice it is the same problem as the Dutch flag.
056	Maximum Gap	LeetCode 164 (Hard)	The classic bucket-sort problem: $O(n)$ is required, and the pigeonhole argument is the whole trick.
056	Sort an Array	LeetCode 912 (Medium)	The range is $-50,000$ to $50,000$. Counting sort is genuinely the best answer, and spotting that is the point.
056	H-Index	LeetCode 274 (Medium)	Counting sort in disguise — citations above n can all be bucketed together.
057	Relative Sort Array	LeetCode 1122 (Easy)	A custom order with a stable tie-break — the cleanest use of a tuple key.
057	Sort the People	LeetCode 2418 (Easy)	Sorting one list by another list's values, and keeping the pairing intact.
057	Custom Sort String	LeetCode 791 (Medium)	An ordering that is data rather than comparison, and where ties come from.
057	Sort Characters By Frequency	LeetCode 451 (Medium)	Two keys in opposite directions, which is where tuple keys start to run out.
058	Sort Array By Parity II	LeetCode 922 (Easy)	That not every ordering problem is a sort — sometimes the key is a placement rule.
058	Sort Characters By Frequency	LeetCode 451 (Medium)	Two keys in opposite directions, with a count you have to build first.
058	Largest Number	LeetCode 179 (Medium)	The one problem where no per-element key exists. <code>cmp_to_key</code> , and the sign convention.
058	Reorder Data in Log Files	LeetCode 937 (Medium)	A tuple key mixing types and a group that must keep its original order — stability in production.
059	Meeting Rooms II	LeetCode 253 (Medium)	That "sort by what?" is the whole question — and this one wants two separate sorted lists.
059	Top K Frequent Words	LeetCode 692 (Medium)	Choosing between sort, heap and quickselect, and defending the choice with arithmetic.
059	Maximum Gap	LeetCode 164 (Hard)	A linear-time requirement, so the comparison sorts are all excluded by the constraints.
059	Merge Intervals	LeetCode 56 (Medium)	The phase's cleanest "the sort is the algorithm" — and which field you sort on.
060	Design HashMap	LeetCode 706 (Easy)	Whether you can actually build one — buckets, collisions, and storing the key.
060	Two Sum	LeetCode 1 (Easy)	The canonical $O(n^2) \rightarrow O(n)$ trade, and that the dict keeps the indices.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
060	Group Anagrams	LeetCode 49 (Medium)	That the key can be *computed*, and that it must be hashable — a tuple, not a list.
060	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and Python's insertion-order guarantee doing real work.
061	Design HashSet	LeetCode 705 (Easy)	The same machinery as yesterday, minus the values — so the collision handling is all that is left.
061	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Deletion done right: a dict plus a list, with the swap-with-last trick.
061	LRU Cache	LeetCode 146 (Medium)	A hash map that must also maintain order — where the dict alone is not enough.
061	Design HashMap	LeetCode 706 (Easy)	Write it a second time, with open addressing and tombstones instead of chaining.
062	Contains Duplicate	LeetCode 217 (Easy)	The seen set in its simplest form, and whether you know why <code>len(set(x)) < len(x)</code> behaves differently on early duplicates.
062	Intersection of Two Arrays	LeetCode 349 (Easy)	Set operators instead of a nested loop, and whether you noticed the question is about distinct values.
062	Happy Number	LeetCode 202 (Easy)	A set used to detect a cycle, not a duplicate — the same tool answering a different question.
062	Longest Consecutive Sequence	LeetCode 128 (Medium)	Whether you can defend an inner while loop that is bounded in total rather than per iteration.
063	Valid Anagram	LeetCode 242 (Easy)	Counting versus sorting, and whether you can say which wins on time and which on space.
063	First Unique Character in a String	LeetCode 387 (Easy)	Two passes, and whether you walk the original or the counter on the second one.
063	Sort Characters By Frequency	LeetCode 451 (Medium)	Counts as sort keys, and the tie-break question nobody asks.
063	Top K Frequent Elements	LeetCode 347 (Medium)	Whether you know all three selection strategies and can pick one with a reason.
064	Group Anagrams	LeetCode 49 (Medium)	The standard vehicle. Whether you can defend the key, not whether you can write a defaultdict.
064	Find Duplicate File in System	LeetCode 609 (Medium)	Grouping by content rather than by name, and the cheap-key-first idea.
064	Group Shifted Strings	LeetCode 249 (Medium)	A key you have to invent, with a wrap-around case that most people miss.
064	Valid Sudoku	LeetCode 36 (Medium)	Three groupings at once, and the <code>(r // 3, c // 3)</code> key.
065	Number of Boomerangs	LeetCode 447 (Medium)	A tuple as a key, and noticing that the natural key is a distance rather than a point.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
065	Max Points on a Line	LeetCode 149 (Hard)	Designing a key for "same line" — the reduced-fraction slope, and why floats are the wrong key.
065	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Objects as keys with a positional value, and deletion done right.
066	Two Sum II — Input Array Is Sorted	LeetCode 167 (Medium)	Whether you notice the word "sorted" and drop the map for two pointers.
066	Find All Numbers Disappeared in an Array	LeetCode 448 (Easy)	The marking trick, without the cleaning pass to complicate it.
066	Find the Duplicate Number	LeetCode 287 (Medium)	O(1) space with the input read-only — so marking is banned too.
066	First Missing Positive	LeetCode 41 (Hard)	The bound nobody gives you, and the -abs line.
067	Top K Frequent Words	LeetCode 692 (Medium)	Counting plus a two-direction tie-break — the reverse = True trap in a hashing costume.
067	Subarray Sum Equals K	LeetCode 560 (Medium)	Prefix sums plus a frequency map: the composition that carries two phases at once.
067	Longest Substring Without Repeating Characters	LeetCode 3 (Medium)	Sliding window plus a map from value to index — and the index-jump most people get wrong.
067	Insert Delete GetRandom O(1)	LeetCode 380 (Medium)	Designing with two structures, and deletion done in O(1) with the swap-with-last trick.
068	Remove All Adjacent Duplicates In String	LeetCode 1047 (Easy)	The cancel-with-the-top pattern, and the if stack and ... guard.
068	Baseball Game	LeetCode 682 (Easy)	Reading a specification carefully while the stack part stays trivial.
068	Simplify Path	LeetCode 71 (Medium)	Real edge cases: ../ at the root, //, and "..." being a legal name.
068	Backspace String Compare	LeetCode 844 (Easy)	Write it with a stack, then do it again in O(1) space with two pointers from the right.
069	Valid Parentheses	LeetCode 20 (Easy)	All three failures, and whether you map closer→opener or the wrong way round.
069	Minimum Add to Make Parentheses Valid	LeetCode 921 (Medium)	Two counters instead of a stack — whether you noticed the stack was not needed.
069	Evaluate Reverse Polish Notation	LeetCode 150 (Medium)	Operand order, which only - and / expose.
069	Decode String	LeetCode 394 (Medium)	Pushing a *context*, multi-digit counts, and which side of the concatenation goes where.
070	Min Stack	LeetCode 155 (Medium)	The observation, and the < = duplicate trap in the two-stack variant.
070	Max Stack	LeetCode 716 (Hard)	One comparison flipped — until popMax is asked for, and then it is a different problem.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
070	Design a Stack With Increment Operation	LeetCode 1381 (Medium)	Lazy increments carried per element: the same idea with a different aggregate.
070	Maximum Frequency Stack	LeetCode 895 (Hard)	Extra state that is a map of stacks. The senior version of today.
071	Next Greater Element I	LeetCode 496 (Easy)	The base pattern, plus a map from value to answer for the lookup half.
071	Daily Temperatures	LeetCode 739 (Medium)	The distance variant — which is why the stack must hold indices.
071	Next Greater Element II	LeetCode 503 (Medium)	Circular: 2n steps, and pushing only on the first pass.
071	Online Stock Span	LeetCode 901 (Medium)	Previous-greater, arriving one element at a time rather than as an array.
072	Sum of Subarray Minimums	LeetCode 907 (Medium)	The same two boundaries, used to *count* spans instead of measuring one. Forces the width formula into your hands.
072	Largest Rectangle in Histogram	LeetCode 84 (Hard)	Today's problem. The reduction, the one-pass pop, and the sentinel.
072	Trapping Rain Water	LeetCode 42 (Hard)	The same stack accumulating as it pops — and the two-pointer solution that beats it on space.
072	Maximal Rectangle	LeetCode 85 (Hard)	Recognising that each row of a binary grid is a histogram. Almost no new code.
073	Number of Recent Calls	LeetCode 933 (Easy)	The simplest real queue: things expire from the front while new ones arrive at the back.
073	Design Circular Queue	LeetCode 622 (Medium)	The ring buffer, and specifically whether you can tell full from empty.
073	Binary Tree Level Order Traversal	LeetCode 102 (Medium)	The for _ in range(len(queue)) trick that separates one level from the next.
073	Rotting Oranges	LeetCode 994 (Medium)	Multi-source BFS on a grid, where the queue's FIFO order *is* the passage of time.
074	Moving Average from Data Stream	LeetCode 346 (Easy)	The plain deque: add at the back, expire at the front, keep a running total. No monotonicity yet.
074	Sliding Window Maximum	LeetCode 239 (Hard)	Today's problem. The delete-for-ever observation, both removals, and the O(n) defence.
074	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	LeetCode 1438 (Medium)	Two dequeues at once, and a window that shrinks rather than slides.
074	Shortest Subarray with Sum at Least K	LeetCode 862 (Hard)	A monotonic deque over prefix sums, with negatives — the case where a plain sliding window is simply wrong.
075	Implement Stack using Queues	LeetCode 225 (Easy)	Choosing which operation pays, and noticing there is no amortised saving here.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
075	Implement Queue using Stacks	LeetCode 232 (Easy)	The pour-only-when-empty rule, and the amortised argument on demand.
075	Design Front Middle Back Queue	LeetCode 1670 (Medium)	Two deques kept balanced — the same "two containers, one invariant" habit.
075	Max Stack	LeetCode 716 (Hard)	Two structures cooperating under one interface, and the honest cost of the expensive operation.
076	Design HashMap	LeetCode 706 (Easy)	Buckets and chaining, so that "the map is $O(1)$ " is something you have built rather than assumed.
076	LRU Cache	LeetCode 146 (Medium)	Today's problem. Two structures, sentinels, and the fact that get is a write.
076	Design Browser History	LeetCode 1472 (Medium)	The same doubly linked list, with a cursor instead of an eviction rule.
076	LFU Cache	LeetCode 460 (Hard)	A third structure and a min_frequency counter, and why it stays $O(1)$.
077	Simplify Path	LeetCode 71 (Medium)	Recognising a stack in a problem that never says "stack", and the .. versus ... trap.
077	Basic Calculator II	LeetCode 227 (Medium)	The stack holding "the context I am leaving", and precedence without recursion.
077	Remove K Digits	LeetCode 402 (Medium)	A monotonic stack used for construction rather than for lookup, plus three edge cases.
077	Design Hit Counter	LeetCode 362 (Medium)	"The last 300 seconds" as a queue, and the follow-up about a very large hit rate.

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 889, 1038, 1039, 1048

A

a replicated log, 1038
 abstract class, 37, 55, 57, 59, 60, 61, 64, 65, 66, ...
 acid, 1038, 1039, 1053
 adaptee, 669, 671, 677, 680, 1009, 1038
 adapter, 60, 101, 105, 215, 218, **335**, 337, 338, 339, ...
 add a tie-breaker, 1038
 algorithm, 1, 2, 3, 9, 11, 15, 33, 81, **951**, ...
 always comparable, 1038, 1047
 amortised, 362, 364, 369, 660, 661, 825, 892, 895, 899, ...
 anaemic model, 22, 29, 59, 134, 148, 183, 184, 185, 186, ...
 and a direction, 1038
 anti-diagonals, 1038
 anti-sniping, 1038
 approximate, 1038
 array, 4, 5, 7, 11, 13, 17, 34, 71, 89, ...
 artefact, 1038, 1041
 assignment rule, 1038
 at-least-once, 1039
 atomic, 30, 32, 135, 137, 141, 142, 144, 145, 152, ...
 atomicity, 1038, 1039
 availability, 137, 141, 142, 145, 147, 1039, 1041, 1045, 1051
 aws ecs, 1039, 1047

B

backend, 344, 737, 741, 743, 744, 800, 1039, 1053
 bearer token, 1039
 best case, 12, 38, 42, 47, 49, 51, 53, 72, 74, ...
 binary tree, 1039, 1063
 blast radius, 181, 216, 299, 346, 776, 960, 1039
 block, 44, 55, 66, 89, 90, 104, 112, 118, 145, ...
 blocked, 880, 948, 1039
 both, 4, 5, 15, 17, 29, 31, 33, 36, 42, ...
 boundary, 7, 25, 26, 119, 130, 265, 791, 794, 1039, ...
 bounded context, 1039
 bounded optimism, 1039
 breadth-first, 821, 824, 832, 1039
 break point, 709, 1039
 broken links, 1039, 1048
 bucket by time, 1039
 bucket counts, 1039
 bulkhead, 1039

burst size, 1039, 1049

C

cache, 91, 117, 129, 259, 264, 269, 271, 274, 314, ...
 cache it, 1040
 cache stampede, 1040
 candidates, 7, 156, 990, 1038, 1040, 1052, 1053, 1056
 capacity, 141, 364, 365, 366, 367, 368, 374, 375, 393, ...
 cdn, 737, 749, 954, 1040, 1045, 1051
 certificate, 1040
 chaining, 364, 366, 368, 393, 398, 401, 403, 405, 411, ...
 channels, 1040
 checks permission, 1040
 chunks, 16, 78, 91, 332, 492, 1040
 circular buffer, 1040, 1049
 class, 5, 14, 15, 16, 17, 19, 20, 21, 22, ...
 client, 111, 277, 338, 354, 499, 569, 579, **669**, 670, ...
 collapse key, 1040
 collapsing writes, 1040
 collection, 61, 259, 263, 264, 268, 269, 271, 303, 366, ...
 column, 42, 47, 176, 387, 388, 396, 444, 447, 516, ...
 compact, 825, 881, 1040, 1041, 1055
 comparator, 1, 277, 278, 285, 286, 290, 311, 1029, 1037, ...
 comparison, 7, 12, **42**, 45, 46, 47, 64, 86, 123, ...
 complete, 11, 22, 23, 45, 83, 122, 162, 203, 243, ...
 component, **702**, 916, 1040, 1044, 1046, 1051
 composite, 1040
 composite key, 1040
 composition, 58, 65, 97, 101, 103, 222, 344, 346, **1040**, ...
 composition root, 97, 101, 103, 222, 344, 346, 533, 534, 1040
 connection, 95, 97, 100, 102, 105, 107, 108, 111, 182, ...
 connection-oriented, 1041
 consequence, 386, 498, 1041, 1048, 1050
 consistency, 25, 1038, 1039, 1041, 1042, 1046, 1050
 constraints, 1041, 1042, 1060
 constructor, 182, 533, 535, 582, 601, 605, 1038, 1041
 container, 102, 104, 303, 468, 535, 586, 594, 629, 1038, ...
 container image, 1038, 1041
 containerd, 1041
 context, 299, 569, 689, 693, 707, 715, 773, **842**, 916, ...
 coordinator, 134, 179, 180, 187, 1021, 1041, 1048
 correlation id, 1041
 cost, 6, 9, 12, 13, 15, 16, 17, 20, 26, ...
 count map, **482**, 1041
 counter, 17, 36, 52, 72, 142, 150, 152, 172, **482**, ...

cri-o, 345, 1041
critical path, 469, 586, 594, 1041
cycle, 24, 25, 52, 105, 475, 611, 619, 633, 635, ...

D

dangerous, 740, 1041
dashed, 60, 219, 738, 841, 987, 1041
dau, 256, 378, 513, 597, 801, 802, 932, 1041
deep-copied, 634, 648, 1041
deleted, 374, 380, 382, 389, 390, 391, 392, 402, 406, ...
delta of delta, 1042
dependency, 101, 105, 108, 178, 300, 304, 316, 335, 354, ...
depth, 82, 86, 87, 116, 118, 120, 121, 122, 124, ...
depth-first, 655, 1042
deque, 623, 624, 651, 656, 663, 665, 666, 682, 820, ...
diminishing returns, 1042
direct addressing, 587, 589, 590, 592, 627, 629, 1042
domain name, 1042
draw two diagrams, 1042
duck typing, 60, 1042
dummy node, 1042

E

earlier, 198, 204, 210, 213, 238, 293, 359, 400, 418, ...
edge cases, 457, 493, 521, 526, 619, 622, 632, 647, 649, ...
edge compute, 1042
effect, 13, 96, 99, 104, 105, 107, 108, 210, 291, ...
elastic scaling, 1042
election timeout, 1042
empty, 68, 77, 79, 81, 82, 84, 87, 88, 399, ...
etl, 50, 58, 255, 258, 260, 268, 270, 453, 483, ...
event object, 802, 814, 1011, 1042
eventual consistency, 1042
exactly, 3, 16, 20, 21, 25, 28, 29, 31, 35, ...
exactly correct, 1042
exactly one, 21, 25, 28, 31, 41, 52, 69, 79, 80, ...
exclusive lock, 1043
expiry, 104, 142, 609, 859, 865, 866, 867, 871, 887, ...

F

facts, 90, 92, 93, 110, 112, 115, 252, 275, 276, ...
false negative, 1043
false positive, 1043
fanout, 1043
fee, 20, 22, 27, 58, 102, 297, 298, 299, 302, ...
fifo, 771, 821, 827, 832, 835, 851, 854, 894, 896, ...
file, 11, 13, 14, 27, 45, 48, 50, 51, 57, ...
filled diamond, 1043
finite state machine, 839, 843, 1043
float, 59, 81, 86, 87, 136, 138, 139, 163, 179, ...
four-direction move, 1043
fragile, 701, 702, 704, 709, 1043
frontend, 1043

G

galloping mode, 1043
gap, 43, 47, 53, 64, 71, 114, 133, 134, 135, ...
generator expression, 1043
greedy simulation, 1043
guest operating system, 1043, 1044

H

hash map, 8, 357, 358, 359, 361, 366, 367, 373, 582, ...
hash ring, 1043
hateoas, 1043
header interface, 298, 301, 1043
heap, 5, 8, 12, 13, 16, 17, 34, 38, 40, ...
heapsort, 89, 90, 116, 125, 129, 131, 241, 253, 255, ...
heartbeat, 1044
hidden spof, 1044
hiding mechanism, 1044
highest node, 1044
historical snapshot, 1044
hollow triangle, 1044
hook, 58, 68, 104, 218, 223, 231, 235, 314, 910, ...
horizontal scaling, 1044
hypervisor, 1043, 1044

I

image, 639, 643, 644, 645, 826, 1008, 1038, 1041, 1044
implicit graph, 1044
in parallel, 535, 740, 1044
in series, 1044
in-degree, 1044, 1048
index, 5, 9, 10, 14, 15, 17, 22, 48, 50, ...
indexed heap, 1044
inheritance, 58, 60, 65, 66, 559, 703, 708, 716, 1044, ...
inorder, 1044, 1049
input, 5, 6, 8, 12, 14, 15, 17, 25, 29, ...
integrity, 1044
intent plus multiplicity, 1044
interface, 37, 57, 59, 60, 61, 64, 65, 67, 69, ...
invariant, 26, 45, 60, 122, 258, 261, 264, 325, 622, ...
is possible, 1045, 1050
item, 16, 52, 58, 61, 63, 67, 95, 98, 99, ...
iterator, 296, 297, 300, 301, 302, 313, 371, 497, 892, ...

J

json, 27, 103, 234, 275, 347, 385, 426, 687, 697, ...
jump, 41, 59, 402, 416, 470, 502, 588, 646, 749, ...
jwt, 880, 884, 886, 1012, 1045

K

k closest points, 188, 1045, 1059
k most frequent, 480, 487, 1045
k-th largest specifically, 1045
key, 1, 6, 9, 10, 11, 14, 15, 18, 22, ...
key property, 1045

L

lag, 3, 31, 42, 47, 49, 51, 53, 71, 72, ...
largest, 1, 3, 12, 16, 34, 39, 41, 53, 154, ...
latency, 91, 369, 373, 374, 376, 586, 593, 594, 665, ...
lazy, 105, 306, 500, 532, 604, 708, 711, 717, 739, ...
lazy deletion, 865, 1045
lazy loading, 1045
lazy refill, 1045
leader, 251, 273, 1042, 1045, 1051
leaf, 103, 1045, 1051
lease, 135, 137, 145, 256, 323, 345, 388, 390, 482, ...
ledger entries, 1045
left, 4, 7, 11, 14, 38, 39, 40, 41, 42, ...
lfu, 771, 944, 945, 947, 962, 964, 971, 1034, 1045, ...
linear scan, 368, 1046
linearizability, 1046
list, 6, 9, 10, 11, 12, 13, 14, 15, 16, ...
list of lists, 364, 393, 648, 1046
load balancer, 1046
load factor, 361, 364, 365, 368, 373, 374, 375, 376, 393, ...
load shedding, 1046
log-structured merge tree, 1046
logical cohesion, 424, 1046
logical time, 1046
loop, 3, 9, 15, 33, 42, 43, 46, 48, 49, ...
loop body, 851, 855, 1046
low cohesion, 463, 1046
lru, 437, 499, 504, 506, 651, 684, 701, 704, 707, ...

M

manifest, 1046
many times, 27, 35, 38, 55, 90, 92, 96, 195, 291, ...
many-to-many, 1046, 1052
matrix, 183, 185, 191, 432, 436, 441, 470, 520, 790, ...
mau, 1046
max-heap, 332, 859, 870, 1046
method resolution order, 62, 1046, 1056
milliseconds, 98, 165, 793, 1041, 1046
min-heap, 16, 169, 332, 352, 623, 731, 1046
minimum spanning forest, 1046
mixin, 65, 66, 454, 1047, 1060
monotone, 1047
monotonic deque, 856, 865, 971, 975, 976, 1047, 1063

N

name mangling, 1047
namespace, 1047
negative, 200, 201, 203, 204, 207, 285, 286, 326, 387, ...
negative infinity, 1047
negatives, 200, 203, 622, 1042, 1047, 1063
nested loop, 443, 447, 452, 1047, 1061
network partition, 1047
never equal, 1038, 1047
never forgets, 1047
never persisted, 1047
no children, 1045, 1047

node, 71, 109, 353, 412, 416, 544, 550, 655, 662, ...
not a map, 618, 622, 631, 647, 1018, 1047

O

$O(1)$, 6, 7, 12, 13, 48, 51, 53, 64, 80, ...
object, 1, 2, 13, 14, 20, 21, 23, 24, 25, ...
observers, 803, 805, 1047
occupied, 409, 447, 1041, 1042, 1047
offset, 200, 203, 912, 917, 919, 920, 922, 923, 924, ...
on first touch, 1047
one-shot, 912, 917, 920, 922, 924, 929, 1013, 1047
opens, 3, 38, 96, 186, 218, 225, 296, 441, 495, ...
orchestrator, 1039, 1047
origin, 6, 8, 10, 13, 14, 63, 122, 165, 246, ...
origin pull, 1048
orphan rows, 1048
orphans, 1039, 1048
out-degree, 1044, 1048
overloading, 57, 1048
overriding, 57, 60, 261, 1048

P

page, 2, 290, 319, 323, 324, 325, 496, 500, 501, ...
parallel edges, 1048
parameters, 412, 582, 1048
parent, 261, 264, 1044, 1046, 1048, 1051, 1062
partial failure, 1048
participants, 1041, 1048, 1050
partition key, 1039, 1048
partitions, 124, 1048
path, 25, 68, 97, 100, 101, 102, 108, 143, 159, ...
port, 3, 11, 14, 22, 25, 29, 31, 43, 44, ...
prefix sums, 1047, 1048, 1062, 1063
preorder, 1048
priority queue, 1048
procedure, 20, 318, 618, 969, 1048
process, 2, 14, 141, 175, 177, 178, 181, 182, 186, ...
program, 1040, 1048, 1049, 1051, 1056
protecting state, 1048

Q

qps, 1049
queue, 172, 173, 175, 238, 243, 380, 396, 496, 651, ...
quorum, 1049

R

race condition, 1049
ram, 22, 34, 55, 57, 58, 67, 68, 74, 86, ...
rank, 126, 287, 322, 323, 324, 327, 331, 465, 470, ...
rebalancing, 1049
rebuilding the tree, 1049
recursion, 81, 82, 83, 86, 87, 93, 109, 112, 120, ...
references, 24, 25, 142, 571, 640, 674, 1039, 1048, 1049, ...
refill rate, 1039, 1049
register, 67, 217, 222, 223, 228, 384, 569, 575, 806, ...

relative epsilon, 1049
removal, 446, 821, 822, 859, 860, 863, 865, 866, 869, ...
replica, 808, 1038, 1040, 1049, 1053
replica divergence, 1049
request-aware routing, 1049
resolver, 1049
resources, 223, 468, 642, 645, 920, 1008, 1049, 1053
returned, 22, 24, 25, 29, 50, 119, 149, 150, 264, ...
ring buffer, 826, 971, 976, 977, 1040, 1049, 1063
role interface, 298, 300, 301, 304, 1049
root, 21, 24, 25, 26, 28, 31, 32, 35, 36, ...
rotation, 898, 902, 1049
routine, 1049
row, 3, 4, 5, 21, 22, 27, 36, 38, 39, ...
run, 3, 5, 6, 7, 12, 16, 18, 19, 20, ...

S

same remainder, 361, 400, 1050
schema, 19, 95, 385, 388, 469, 471, 988, 995, 996, ...
scoping, 601, 604, 1050
script, 13, 29, 31, 50, 97, 100, 144, 282, 967, ...
seam, 96, 97, 98, 102, 103, 104, 107, 108, 111, ...
search space, 1050
selectivity, 1050
sentinel, 786, 789, 791, 794, 933, 936, 938, 939, 943, ...
separator, 1050
sequence diagram, 1050, 1057
serialisable, 1050
service credits, 1050
session, 569, 579, 880, 1045, 1050, 1051, 1052
session guarantees, 1050
set, 5, 8, 9, 12, 13, 14, 15, 16, 17, ...
shallow-copied, 1050
shape, 13, 31, 51, 56, 59, 62, 64, 67, 68, ...
shard, 492, 518, 624, 630, 631, 943, 945, 947, 980, ...
shared lock, 1050
shared store, 1050
singleton, 97, 105, 106, 136, 495, 497, 498, 500, 512, ...
singly linked list, 932, 933, 943, 944, 947, 963, 964, 1034, 1050
slice, 87, 93, 250, 274, 488, 585, 639, 1027, 1050
smallest, 10, 11, 41, 43, 45, 50, 72, 116, 118, ...
snapshot, 880, 881, 1044, 1050
snipe, 1050
sorted region, 41, 43, 1051
sorting everything, 1051
splits, 51, 86, 114, 127, 156, 169, 307, 310, 468, ...
spof, 1044, 1051
squares, 1051
stable, 6, 12, 15, 17, 18, 38, 41, 44, 45, ...
stack, 78, 86, 92, 101, 105, 121, 125, 128, 892, ...
stack frame, 92, 656, 662, 713, 1010, 1051
stack overflow, 1051
state, 17, 21, 29, 33, 51, 57, 59, 60, 61, ...
statements, 751, 1048, 1051, 1058
status, 29, 64, 134, 135, 136, 137, 140, 144, 147, ...

still one machine, 1045, 1051
stonith, 1051
strategy, 135, 136, 139, 141, 143, 144, 145, 146, 751, ...
strictly shorter, 81, 785, 786, 788, 789, 791, 795, 799, 816, ...
structural, 224, 338, 498, 500, 1048, 1051
subarray, 864, 1051, 1062, 1063
subject, 178, 802, 803, 1051
subscription list, 1051
substring, 1051, 1062
subtree, 1044, 1048, 1051
surge multiplier especially, 1051
swap, 39, 40, 41, 42, 43, 45, 47, 50, 51, ...
system call, 1051

T

table, 3, 4, 6, 12, 13, 15, 16, 17, 18, ...
tail call, 1052
tail-recursive, 1052
tcp, 843, 850, 854, 954, 1012, 1041, 1052, 1053
template, 177, 181, 234, 386, 425, 430, 640, 892, 909, ...
test double, 96, 103, 112, 145, 267, 470, 472, 474, 501, ...
the return address, 1052
the running median, 1044, 1052
third table, 1046, 1052
thread, 138, 141, 143, 145, 146, 512, 528, 529, 530, ...
three distinct orderings, 1052
three-way partition, 120, 123, 125, 1052, 1059
tight coupling, 1052
time-to-live, 1052
tiny constraint, 1052
toast, 1052
token, 61, 135, 178, 179, 180, 338, 342, 343, 384, ...
tombstone, 402, 404, 405, 406, 407, 409, 410, 411, 412, ...
top-k problems, 1044, 1052
total order, 14, 1052
transitions itself, 1052
transitive dependency, 1052
tree, 3, 58, 76, 118, 159, 333, 399, 401, 403, ...
ttl, 39, 55, 76, 142, 143, 145, 147, 148, 214, ...
tuple, 6, 10, 18, 84, 99, 141, 203, 204, 240, ...
two, 3, 4, 5, 7, 8, 9, 10, 11, 12, ...
two implementations, 792, 1053

U

udp, 1052, 1053
under-fetching, 1053
union filesystem, 1053
upper bound, 1049, 1053
uri, 7, 24, 29, 31, 80, 194, 257, 371, 376, ...
url, 97, 101, 107, 111, 338, 344, 354, 426, 430, ...

V

value object, 20, 23, 24, 25, 26, 27, 29, 31, 32, ...
version, 33, 81, 84, 85, 86, 109, 160, 163, 173, ...
vertex, 1041, 1047, 1048, 1053

vertical scaling, 1053

victim, 1053

virtual address space, 1053

virtual machine, 1039, 1047, 1053

volatile, 531, 537, 538, 539, 540, 1006, 1053

W

weight, 24, 29, 31, 226, 377, 380, 424, 426, 427, ...

which class, 190, 951, 961, 1014, 1053

which is hours, 1053

within, 6, 50, 52, 238, 239, 240, 244, 252, 254, ...

worst case, 6, 12, 17, 38, 43, 46, 51, 53, 72, ...

write skew, 1053

write-around with invalidation, 1053

write-through, 1053

Y

yes, 5, 8, 9, 15, 16, 20, 21, 29, 32, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.